



QSpace tensor library

v4.0

Andreas Weichselbaum
(May 2025)

Open source under the Apache 2.0 license.

Generated by Doxygen 1.12.0

1 Hierarchical Index	1
1.1 Class Hierarchy	1
2 Class Index	5
2.1 Class List	5
3 File Index	9
3.1 File List	9
4 Class Documentation	11
4.1 __wb_msgbuf Struct Reference	11
4.2 Wb::ARGV Class Reference	11
4.3 Mx::Array< T > Class Template Reference	12
4.4 Wb::Bitset Class Reference	13
4.5 Wb::bitset< T > Class Template Reference	20
4.6 blockSpaceQS< TQ, TD > Class Template Reference	20
4.7 Wb::cc_quad Class Reference	20
4.8 CData< TQ, TD > Class Template Reference	21
4.9 cdata< TD > Class Template Reference	25
4.10 cgc_contract_id< TM > Class Template Reference	31
4.11 cgdStatus Class Reference	38
4.12 cgrType Class Reference	39
4.13 Wb::citerator< T, T2 > Class Template Reference	40
4.14 Wb::citerator< T, T2 * > Class Template Reference	40
4.15 Wb::CleanUp Class Reference	41
4.16 Wb::Clock Class Reference	41
4.17 Wb::ClockSet Class Reference	42
4.18 Wb::counter Class Reference	42
4.19 CPAT< TQ, TD > Class Template Reference	43
4.20 CPAT_Set< TQ, TD > Class Template Reference	43
4.21 cpu_time Class Reference	44
4.22 CRef< TQ > Class Template Reference	45
4.23 CStore< TQ > Class Template Reference	47
4.24 ctrlIdx Class Reference	49
4.25 CG::FileLock Class Reference	56
4.26 genRG_base< TQ, TD > Class Template Reference	56
4.27 genRG_struct< TQ, TD > Class Template Reference	57
4.28 Wb::gpara__ Class Reference	58
4.29 groupIndex< T > Class Template Reference	58
4.30 icFlags Class Reference	59
4.31 indexSparseRef< TD > Class Template Reference	59
4.32 Wb::IOstat Class Reference	60
4.33 is_pointer_< T > Class Template Reference	60

4.34 is_pointer_< T * > Class Template Reference	60
4.35 is_pointer_< T ** > Class Template Reference	61
4.36 is_pointer_< T *** > Class Template Reference	61
4.37 itag_ Class Reference	61
4.38 iTags Class Reference	62
4.39 Wb::iterator< T, T2 > Class Template Reference	70
4.40 Wb::iterator< T, T2 * > Class Template Reference	70
4.41 IterOM_DATA< TD > Class Template Reference	71
4.42 iterSparseCMref< TD > Class Template Reference	71
4.43 Wb::line_stat_ Class Reference	72
4.44 Wb::LogException Struct Reference	72
4.45 Wb::matFile Class Reference	73
4.46 MEM_SET< T > Class Template Reference	74
4.47 MEM_SET< T * > Class Template Reference	74
4.48 MHash< TM > Class Template Reference	74
4.49 Wb::mpfr_< P > Class Template Reference	75
4.50 Wb::MREC Class Reference	77
4.51 mxArray_buf Class Reference	78
4.52 MXPut Class Reference	78
4.53 mxStruct Class Reference	79
4.54 Wb::myname_ Class Reference	80
4.55 NRGData< TQ, TD > Class Template Reference	80
4.56 NRGIndex< TQ, TD > Class Template Reference	82
4.57 Wb::num2Fmt< T > Class Template Reference	83
4.58 opFlags< T > Class Template Reference	83
4.59 OPTS Class Reference	84
4.60 pairPatternCG< T > Class Template Reference	84
4.61 Wb::Path Class Reference	85
4.62 Wb::Profile Class Reference	85
4.63 QBlock< TQ, TD, TE > Class Template Reference	86
4.64 QDir Class Reference	87
4.65 QHash< TQ > Class Template Reference	94
4.66 QHash_ Class Reference	94
4.67 QMap< TQ > Class Template Reference	94
4.68 QSet< TQ > Class Template Reference	95
4.69 qset< TQ > Class Template Reference	97
4.70 QSpace< TQ, TD > Class Template Reference	104
4.71 QType Class Reference	111
4.72 QVec Class Reference	112
4.73 Wb::rand Class Reference	120
4.74 RCStore Class Reference	120
4.75 RIXS_para Class Reference	121

4.76 RSpecData< TS > Class Template Reference	122
4.77 RStore< TQ, TD > Class Template Reference	122
4.78 Wb::SBUF Class Reference	123
4.79 Wb::SigHandler Class Reference	123
4.80 sparseCollector2D< TD > Class Template Reference	124
4.81 sparseIndex2D< T > Class Template Reference	124
4.82 Spectral< TS > Class Template Reference	125
4.83 Wb::sptr< T > Class Template Reference	126
4.84 STAT_DGEMM Class Reference	126
4.85 wblog::stdio_buf Class Reference	127
4.86 Wb::String Class Reference	127
4.87 SVD_Data< TQ, TD > Class Template Reference	134
4.88 DY::Symmetry Class Reference	134
4.89 TDSpectral< TS > Class Template Reference	135
4.90 tensor_ref_ Class Reference	135
4.91 tensorRef_ Class Reference	136
4.92 tensorRefs Class Reference	137
4.93 Wb::termcolor Class Reference	144
4.94 Wb::tmpFile Class Reference	144
4.95 Wb::VersionInfo Class Reference	145
4.96 wbarray< T > Class Template Reference	145
4.97 wbarrRef< T > Class Template Reference	153
4.98 wbcomplex Class Reference	154
4.99 wbdebug_dummy Class Reference	155
4.100 wbhist Class Reference	155
4.101 wbIndex Class Reference	156
4.102 wbindex Class Reference	163
4.103 Wb::WbListMTRACK Class Reference	170
4.104 wblogbuf_struct Class Reference	170
4.105 wblogbuffer Class Reference	171
4.106 wbMatrix< T > Class Template Reference	171
4.107 wbMsg Class Reference	179
4.108 wbperm Class Reference	179
4.109 wbperm_helper< T > Class Template Reference	187
4.110 wbRecs< T > Class Template Reference	188
4.111 wbREF< TX > Class Template Reference	188
4.112 wbsparray< TD > Class Template Reference	188
4.113 wbstring Class Reference	194
4.114 wbtopy Class Reference	202
4.115 WbUtil< T > Class Template Reference	203
4.116 wbvector< T > Class Template Reference	203
4.117 DY::weight_info< TQ > Class Template Reference	210

4.118 DY::Weights< TQ > Class Template Reference	211
4.119 X3Map< TQ, TD > Class Template Reference	211
4.120 x3map< TQ, TD > Class Template Reference	212
4.121 XMAP Class Reference	212
4.122 XMAP_CMat Class Reference	219
4.123 XMAP_Node Class Reference	220
5 File Documentation	221
5.1 clebsch.hh	221
5.2 cpat.hh	276
5.3 dmrho.cc	290
5.4 gathered.cc	297
5.5 gathered.hh	320
5.6 matlib.cc	327
5.7 memlib.cc	330
5.8 memlib.hh	333
5.9 memtrack.hh	336
5.10 mexlib.cc	338
5.11 mexlib.hh	349
5.12 mpsortho.cc	356
5.13 mpsortho.hh	369
5.14 mxlib.hh	372
5.15 nomex.hh	378
5.16 nrgdata.hh	380
5.17 QSpace.hh	409
5.18 QSpace_aux.hh	445
5.19 qspace_lib.cc	447
5.20 spectral.hh	455
5.21 util.cc	476
5.22 util.hh	489
5.23 wbararray.cc	497
5.24 wbararray.hh	547
5.25 wbararray_blas.cc	581
5.26 wbararray_blas.hh	594
5.27 wbbitsset.cc	598
5.28 wbbitsset.hh	599
5.29 wbbblas.hh	601
5.30 wbclock.cc	605
5.31 wbclock.hh	608
5.32 wbcomplex.cc	612
5.33 wbcomplex.hh	613
5.34 wbeigs.hh	617

5.35 wfft.hh	619
5.36 wbindex.cc	620
5.37 wbindex.hh	638
5.38 wbio.c	653
5.39 wbio.h	657
5.40 wblib.h	659
5.41 wlibx.h	668
5.42 wblog.c	669
5.43 wblog.h	684
5.44 wbMatIO.hh	689
5.45 wbMatrix.cc	690
5.46 wbMatrix.hh	715
5.47 wbMatrix_blas.cc	749
5.48 wbMatrix_blas.hh	756
5.49 wbmpfr.hh	757
5.50 wmsg.hh	763
5.51 wbomp.hh	766
5.52 wbopts.hh	769
5.53 wbperm.cc	774
5.54 wbperm.hh	779
5.55 wbsort.cc	783
5.56 wbsort.hh	787
5.57 wbsparray.cc	788
5.58 wbsparray.hh	837
5.59 wbstring.cc	860
5.60 wbstring.hh	865
5.61 wbutil.hh	871
5.62 wbvector.cc	874
5.63 wbvector.hh	891
5.64 xmap.hh	914
Index	917

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

__wb_msgbuf	11
Wb::ARGV	11
Mx::Array< T >	12
Wb::bitset< T >	20
blockSpaceQS< TQ, TD >	20
Wb::cc_quad	20
cgdStatus	38
cgrType	39
Wb::citerator< T, T2 >	40
Wb::citerator< T, T2 * >	40
Wb::CleanUp	41
Wb::Clock	41
Wb::ClockSet	42
Wb::counter	42
CPAT< TQ, TD >	43
CPAT_Set< TQ, TD >	43
cpu_time	44
CRef< TQ >	45
CStore< TQ >	47
exception	
Wb::LogException	72
CG::FileLock	56
genRG_base< TQ, TD >	56
genRG_struct< TQ, TD >	57
Wb::gpara__	58
groupIndex< T >	58
icFlags	59
indexSparseRef< TD >	59
Wb::IOstat	60
is_pointer_< T >	60
is_pointer_< T * >	60
is_pointer_< T ** >	61
is_pointer_< T *** >	61
itag_	61
Wb::iterator< T, T2 >	70

Wb::iterator< T, T2 * >	70
IterOM_DATA< TD >	71
iterSparseCMref< TD >	71
Wb::line_stat__	72
Wb::matFile	73
MEM_SET< T >	74
MEM_SET< T * >	74
MHash< TM >	74
Wb::mpfr__< P >	75
Wb::MREC	77
mxArray_buf	78
MXPut	78
mxStruct	79
Wb::myname__	80
NRGData< TQ, TD >	80
NRGIndex< TQ, TD >	82
Wb::num2Fmt< T >	83
opFlags< T >	83
OPTS	84
pairPatternCG< T >	84
pairPatternCG< double >	84
Wb::Path	85
Wb::Profile	85
QBlock< TQ, TD, TE >	86
QBlock< TQ, TD, double >	86
QHash< TQ >	94
QHash_	94
QMap< TQ >	94
QSet< TQ >	95
CData< TQ, RTD >	21
CData< TQ, TD >	21
QSpace< TQ, TD >	104
QType	111
Wb::rand	120
RCStore	120
RIXS_para	121
RSpecData< TS >	122
RStore< TQ, TD >	122
Wb::SBUF	123
Wb::SigHandler	123
sparseCollector2D< TD >	124
sparseIndex2D< T >	124
Spectral< TS >	125
Wb::sptr< T >	126
Wb::sptr< double >	126
Wb::sptr< int >	126
Wb::sptr< TD >	126
Wb::sptr< TS >	126
Wb::sptr< wbcomplex >	126
STAT_DGEMM	126
wblog::stdio_buf	127
SVD_Data< TQ, TD >	134
DY::Symmetry	134
TDSpectral< TS >	135
tensor_ref_	135
tensorRef_	136
Wb::termcolor	144
Wb::tmpFile	144

Wb::VersionInfo	145
wbarray< T >	145
wbarray< double >	145
wbarray< int >	145
wbarray< TD >	145
wbarray< TS >	145
wbarray< wbcomplex >	145
wbarrRef< T >	153
wbcomplex	154
wbdebug_dummy	155
wbhist	155
Wb::WbListMTRACK	170
wblogbuf_struct	170
wblogbuffer	171
wbMatrix< T >	171
wbMatrix< const CRef< TQ > * >	171
wbMatrix< CRef< TQ > >	171
wbMatrix< double >	171
wbMatrix< int >	171
wbMatrix< SPIDX_T >	171
wbMatrix< TQ >	171
wbMatrix< TS >	171
wbMatrix< unsigned >	171
wbMatrix< wbcomplex >	171
wbMatrix< widx_t >	171
wbMsg	179
wbperm_helper< T >	187
wbRecs< T >	188
wbREF< TX >	188
wbsparray< TD >	188
cdata< TD >	25
wbsparray< RTD >	188
cdata< RTD >	25
wbtop	202
WbUtil< T >	203
wbvector< T >	203
wbvector< char >	203
QDir	87
Wb::Bitset	13
Wb::String	127
wbstring	194
wbvector< double >	203
wbvector< int >	203
wbvector< itag_ >	203
iTags	62
wbvector< mxArray * >	203
wbvector< NRGIndex< TQ, TD > >	203
wbvector< pairPatternCG< double > >	203
wbvector< QBlock< TQ, TD, double > >	203
wbvector< QSpace< TQ, TD > >	203
wbvector< QType >	203
QVec	112
wbvector< RSpecData< TS > >	203
wbvector< RTD >	203
wbvector< size_t >	203
wbvector< SPIDX_T >	203
wbvector< sSPIDX_T >	203

wbvector< TD >	203
wbvector< tensorRef_ >	203
tensorRefs	137
wbvector< TM >	203
cgc_contract_id< TM >	31
wbvector< TQ >	203
qset< TQ >	97
wbvector< TS >	203
wbvector< unsigned >	203
ctrlIdx	49
wbvector< unsigned long >	203
wbvector< warray< TD > * >	203
wbvector< wbMatrix< unsigned > >	203
wbvector< wbSparrowTD >	203
wbvector< wbstring >	203
wbvector< wbvector< CPAT_Set< TQ, TD > > >	203
wbvector< wbvector< QSpace< TQ, TD > > >	203
wbvector< widx_t >	203
wbIndex	156
wbindex	163
wbvector< wperm_t >	203
wbperm	179
wbvector< XMAP_Node >	203
XMAP	212
DY::weight_info< TQ >	210
DY::Weights< TQ >	211
X3Map< TQ, TD >	211
x3map< TQ, TD >	212
XMAP_CMat	219
XMAP_Node	220

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

__wb_msgbuf	11
Wb::ARGV	11
Mx::Array< T >	12
Wb::Bitset	13
Wb::bitset< T >	20
blockSpaceQS< TQ, TD >	20
Wb::cc_quad	20
CData< TQ, TD >	21
cdata< TD >	25
cgc_contract_id< TM >	31
cgdStatus	38
cgrType	39
Wb::citerator< T, T2 >	40
Wb::citerator< T, T2 * >	40
Wb::CleanUp	41
Wb::Clock	41
Wb::ClockSet	42
Wb::counter	42
CPAT< TQ, TD >	43
CPAT_Set< TQ, TD >	43
cpu_time	44
CRef< TQ >	45
CStore< TQ >	47
ctrlIdx	49
CG::FileLock	56
genRG_base< TQ, TD >	56
genRG_struct< TQ, TD >	57
Wb::gpara__	58
groupIndex< T >	58
icFlags	59
indexSparseRef< TD >	59
Wb::IOstat	60
is_pointer_< T >	60
is_pointer_< T * >	60
is_pointer_< T ** >	61

is_pointer_< T *** >	61
itag_	61
iTags	62
Wb::iterator< T, T2 >	70
Wb::iterator< T, T2 * >	70
IterOM_DATA< TD >	71
iterSparseCMref< TD >	71
Wb::line_stat__	72
Wb::LogException	72
Wb::matFile	73
MEM_SET< T >	74
MEM_SET< T * >	74
MHash< TM >	74
Wb::mpfr__< P >	75
Wb::MREC	77
mxArray_buf	78
MXPut	78
mxStruct	79
Wb::myname__	80
NRGData< TQ, TD >	80
NRGIndex< TQ, TD >	82
Wb::num2Fmt< T >	83
opFlags< T >	83
OPTS	84
pairPatternCG< T >	84
Wb::Path	85
Wb::Profile	85
QBlock< TQ, TD, TE >	86
QDir	87
QHash< TQ >	94
QHash_	94
QMap< TQ >	94
QSet< TQ >	95
qset< TQ >	97
QSpace< TQ, TD >	104
QType	111
QVec	112
Wb::rand	120
RCStore	120
RIXS_para	121
RSpecData< TS >	122
RStore< TQ, TD >	122
Wb::SBUF	123
Wb::SigHandler	123
sparseCollector2D< TD >	124
sparseIndex2D< T >	124
Spectral< TS >	125
Wb::sptr< T >	126
STAT_DGEMM	126
wblog::stdio_buf	127
Wb::String	127
SVD_Data< TQ, TD >	134
DY::Symmetry	134
TDSpectral< TS >	135
tensor_ref_	135
tensorRef_	136
tensorRefs	137
Wb::termcolor	144

Wb::tmpFile	144
Wb::VersionInfo	145
wbarray< T >	145
wbarrRef< T >	153
wbcomplex	154
wbdebug_dummy	155
wbhist	155
wbIndex	156
wbindex	163
Wb::WbListMTRACK	170
wblogbuf_struct	170
wblogbuffer	171
wbMatrix< T >	171
wbMsg	179
wbperm	179
wbperm_helper< T >	187
wbRecs< T >	188
wbREF< TX >	188
wbsparray< TD >	188
wbstring	194
wbtop	202
WbUtil< T >	203
wbvector< T >	203
DY::weight_info< TQ >	210
DY::Weights< TQ >	211
X3Map< TQ, TD >	211
x3map< TQ, TD >	212
XMAP	212
XMAP_CMat	219
XMAP_Node	220

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

clebsch.hh	221
cpat.hh	276
dmrho.cc	290
gathered.cc	297
gathered.hh	320
matlib.cc	327
memlib.cc	330
memlib.hh	333
memtrack.hh	336
mexlib.cc	338
mexlib.hh	349
mpsortho.cc	356
mpsortho.hh	369
mxlib.hh	372
nomex.hh	378
nrgdata.hh	380
QSpace.hh	409
QSpace_aux.hh	445
qspace_lib.cc	447
spectral.hh	455
util.cc	476
util.hh	489
wbarray.cc	497
wbarray.hh	547
wbarray_blas.cc	581
wbarray_blas.hh	594
wbbitsset.cc	598
wbbitsset.hh	599
wbblas.hh	601
wbclock.cc	605
wbclock.hh	608
wbcomplex.cc	612
wbcomplex.hh	613
wbeigs.hh	617
wbfft.hh	619

wbindex.cc	620
wbindex.hh	638
wbio.c	653
wbio.h	657
wblib.h	659
wblibx.h	668
wblog.c	669
wblog.h	684
wbMatIO.hh	689
wbMatrix.cc	690
wbMatrix.hh	715
wbMatrix_blas.cc	749
wbMatrix_blas.hh	756
wbmpfr.hh	757
wbmsg.hh	763
wbomp.hh	766
wbopts.hh	769
wbperm.cc	774
wbperm.hh	779
wbsort.cc	783
wbsort.hh	787
wbsparray.cc	788
wbsparray.hh	837
wbstring.cc	860
wbstring.hh	865
wbutil.hh	871
wbvector.cc	874
wbvector.hh	891
xmap.hh	914

Chapter 4

Class Documentation

4.1 `__wb_msgbuf` Struct Reference

Public Attributes

- long **type**
- time_t **time**
- char **text** [128]
- double **val**

The documentation for this struct was generated from the following file:

- `wbmsg.hh`

4.2 `Wb::ARGV` Class Reference

Public Member Functions

- **ARGV** (`va_list *vl=NULL`)

The documentation for this class was generated from the following file:

- `wblib.h`

4.3 Mx::Array< T > Class Template Reference

Public Member Functions

- **Array** (unsigned ndim, size_t *dims, char t_=0, char c_=-1)
- **Array** (size_t d1, size_t d2, char t_=0, char c_=-1)
- **Array** (const [wbvector](#)< size_t > &S, char t_=0, char c_=-1)
- **Array** (const mxArray *a, char t_=0, char c_=-1)
- **Array** (const char *F, int L, const mxArray *a, char t_=0, char c_=-1)
- **wbstring toStr** (char vflag=0, unsigned l=64) const
- **wbstring toStrT** (char vflag=0) const
- void **init_xstr** ()
- const char * **xStr** () const
- mxArray * **Return** (T *&dd)
- mxArray * **Return** ()
- **Array & init** (unsigned ndim, size_t *dims, char t_=-1, char c_=-1)
- **Array & init** (const mxArray *a, char t_=-1, char c_=-1)
- template<class Tb >
mxArray * **copyFromTR** (const Tb *b, char tcheck=0)
- template<class Tb >
int **copy_from** (const Tb *b, char tcheck=0)
- template<class Tb >
size_t **copyTo** (Tb *b, char tcheck=0) const
- size_t **copy_to** (T *b, const [wbperm](#) *P=NULL, char tcheck=0) const
- size_t **rcopy_to** (T *b, const [wbperm](#) *P=NULL, char tcheck=0) const
- size_t **zcopy_to** (T *b, const [wbperm](#) *P=NULL, char tcheck=0) const
- template<class Tx >
size_t **cpy_to_** (T *b, const [wbperm](#) *P, char tcheck, const Tx *ad_) const
- size_t **ncopy_to** (T *b, size_t n, char tcheck=0)
- size_t **copy_to** (CGD_TYPE *b, const [wbperm](#) *P, char tcheck) const
- size_t **copy_to** (CGR_TYPE *b, const [wbperm](#) *P, char tcheck) const
- size_t **zcopy_to** ([wbcomplex](#) *b, const [wbperm](#) *P, char ck) const
- size_t **copy_to** ([wbcomplex](#) *b, const [wbperm](#) *P, char tcheck) const
- size_t **copy_to** (double *b, const [wbperm](#) *P, char tcheck) const
- size_t **copy_to** (int *b, const [wbperm](#) *P, char tcheck) const
- size_t **copy_to** (unsigned *b, const [wbperm](#) *P, char tcheck) const
- size_t **copy_to** (size_t *b, const [wbperm](#) *P, char tcheck) const
- size_t **copy_to** (char *b, const [wbperm](#) *P, char tcheck) const

Public Attributes

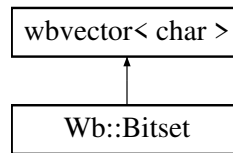
- mxArray * **ax**
- T * **data**
- size_t **len**
- unsigned **rank**
- bool **trans**
- char **cmplx**
- char **mxref**
- std::string **xstr**

The documentation for this class was generated from the following files:

- mxlib.hh
- mexlib.cc

4.4 Wb::Bitset Class Reference

Inheritance diagram for Wb::Bitset:



Public Member Functions

- **Bitset** (unsigned n=0)
- **Bitset** (const [Bitset](#) &b)
- [Bitset](#) & **init** (unsigned n=0)
- [Bitset](#) & **init** (unsigned n, const char *b)
- [Bitset](#) & **setall** (bool t)
- [Bitset](#) & **set** (unsigned i)
- [Bitset](#) & **unset** (unsigned i)
- bool **isset** (unsigned i)
- unsigned **nnz** () const
- bool **operator==** (const [Bitset](#) &b) const
- bool **operator!=** (const [Bitset](#) &b) const
- [wbstring toStr](#) () const
- mxArray * **toMx** () const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i) const
- void **put** (const char *vname, const char *ws="base") const
- void **put** (const char *file, int line, const char *vname, const char *ws="base") const

Public Member Functions inherited from [wbvector< char >](#)

- **wbvector** (size_t l, char x)
- **wbvector** (size_t l, const char *d0=NULL, char ref=0)
- **wbvector** (const [wbvector](#) &v)
- **wbvector** (const [wbvector](#)< T2 > &v)
- **wbvector** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- **wbvector** (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- **wbvector** (const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) & **init** (size_t l=0)
- [wbvector](#) & **init** (size_t l, const char *d0)
- [wbvector](#) & **init** (size_t l, const char *d0, char ref)
- [wbvector](#) & **init** (const [wbvector](#) &v)
- [wbvector](#) & **init** (size_t n, const [wbvector](#) &v)
- [wbvector](#)< char > & **init** (const [wbsparray](#)< char > &S)
- [wbvector](#) & **init** (size_t l1, const char *d1, size_t l2, const char *d2)
- int **init** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type QS_UNUSED_↔ VAR=1, char ref=0)
- int **init** (const mxArray *a, const char *istr=NULL, char check_type=1)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)

- `int init (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)`
- `int init (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)`
- `int init (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)`
- `wbvector & initp (size_t l, const char *d0, size_t n, char init=1)`
- `wbvector & init2val (size_t l, const char &x)`
- `wbvector & initStride (size_t l, const char *d0, size_t stride)`
- `wbvector & init_stride (const char *d0, size_t n, size_t m, size_t D0)`
- `wbvector & init_data (const char *d0)`
- `wbvector & init_data (const T0 *d0)`
- `wbvector & init_data (const char *F, int L, const T0 *d0)`
- `wbvector< char > & initT (const wbspparray< T2 > &S)`
- `wbvector< char > & initT (size_t l, const T2 *v)`
- `wbvector< char > & initT (const wbvector< T2 > &v)`
- `wbvector< char > & initT (const char *F, int L, const wbvector< T2 > &v)`
- `wbvector< char > & initT (const wbspparray< TD > &S)`
- `wbvector< char > & initT_ (const wbvector< T2 > &v)`
- `wbvector & init2ref (const wbvector &v)`
- `wbvector & init2ref (size_t l, const char *d0)`
- `wbvector & unRef ()`
- `int init_Struct (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)`
- `int init_base (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)`
- `int init_mpfr (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)`
- `void initDef (size_t n)`
- `void initDef (const wbvector &v)`
- `wbvector & operator= (const wbvector &v)`
- `wbvector< char > & operator= (const wbvector< TB > &v)`
- `wbvector & set (const char &x)`
- `wbvector & set (const wbvector< size_t > &l, const char &x)`
- `void set (size_t offset, const char *x, size_t n)`
- `void set (const wbvector< double > &R, const wbvector< double > &l)`
- `void set (const wbvector< double > &R, const wbvector< double > &l)`
- `wbvector & setMin (const char &x)`
- `wbvector & setMax (const char &x)`
- `wbvector & add (const wbvector< size_t > &l, const char &x)`
- `wbvector & add (const char *d)`
- `wbvector & Set (const wbvector< size_t > &l, const wbvector &v)`
- `void Sort (wbperm &p, char dir=+1)`
- `void Sort (char dir=+1)`
- `void sort (wbvector &v2, wbperm &p) const`
- `void sort (wbvector &v2) const`
- `wbperm & pSort (wbperm &p) const`
- `bool isSorted () const`
- `bool isSortedU () const`
- `bool isUnique () const`
- `int contains (const char &x) const`
- `bool isNormal () const`
- `bool isNormal () const`
- `bool isNormal () const`
- `bool isNormal () const`
- `bool isNormal () const`
- `bool isNormal () const`
- `bool isNormal () const`
- `bool isNormal () const`
- `bool isNormal () const`
- `bool isFinite () const`
- `size_t find1 (const char &x) const`

- `size_t find1 ()` const
- `size_t count (const char &x)` const
- `size_t icount (const char &x)` const
- `size_t nnz ()` const
- `size_t nnz (char eps)` const
- `size_t numZeros (char eps=char(1E-14))` const
- `size_t numEQ (const char &)` const
- `size_t numGT (const char &)` const
- `size_t numLT (const char &)` const
- `wbindex & find (const char &x, wbindex &l, char iflag=0)` const
- `wbindex & find (const char &x, wbindex &l, wbindex &lx)` const
- `wbindex & find (wbindex &l)` const
- `wbindex & find (wbindex &l, wbindex &lx)` const
- `wbvector< size_t > find ()` const
- `wbindex & findGT (const char &x, wbindex &l, char iflag=0)` const
- `wbindex & findGT (const char &x, wbindex &l, wbindex &lx)` const
- `wbindex & findzeros (wbindex &l)` const
- `wbindex & findRange (const char &x1, const char &x2, wbindex &l, const char *w=NULL)` const
- `int findClosestSorted (char r)` const
- `int findClosest (char r)` const
- `size_t findValues (const wbvector &v, wbindex &l)` const
- `wbvector & setp (const char *x)`
- `void reset ()`
- `void cpy (const char *v)`
- `void cpy (const char *x, size_t m)`
- `wbvector & save2 (wbvector &v)`
- `void swap (wbvector &v)`
- `wbvector & swap (size_t i, size_t j)`
- `wbvector & Collect (char &A1, char &A2)`
- `wbvector & Collect (char &A1, char &A2, char &A3)`
- `wbvector & Skip (size_t)`
- `wbvector & Skip (const WBINDEX &, char rflag=0)`
- `wbvector & Move2_FE (const WBINDEX &l, char FE_flag)`
- `wbvector & Move2front (const WBINDEX &l)`
- `wbvector & Move2front (size_t k)`
- `wbvector & Move2end (const WBINDEX &l)`
- `wbvector & Move2end (size_t k)`
- `wbvector & mvLastTo (size_t k)`
- `wbvector & mvFirstTo (size_t k)`
- `bool checkSameLength (const wbvector &a, const char *F=NULL, int L=0, const char *istr="")` const
- `bool checkSameLength (const wbvector &a, const wbvector &b, const char *F=NULL, int L=0, const char *istr="")` const
- `void operator+= (const T_ &a)`
- `void operator+= (const wbvector &b)`
- `void operator-= (const T_ &a)`
- `void operator-= (const wbvector &b)`
- `wbvector & operator*= (const T_ &a)`
- `wbvector & operator/= (const T_ &a)`
- `wbvector operator* (const T_ &a)` const
- `wbvector operator+ (const T_ &a)` const
- `wbvector operator+ (const wbvector &b)` const
- `wbvector operator- (const T_ &a)` const
- `wbvector operator- (const wbvector &b)` const
- `wbvector & Plus (const char *b)`
- `wbvector & Plus (const char *b, char bfac)`

- [wbvector](#) & **Plus** (const [wbvector](#) &b, char bfac=1)
- void **plus** (const [wbvector](#) &b, [wbvector](#) &c, char bfac=1, char cfac=0) const
- [wbvector](#) & **times** (char fac, [wbvector](#) &x) const
- [wbvector](#) & **RevertSigns** ()
- int **isum** () const
- char **sum** () const
- char **sum** (size_t i1, const char *estr) const
- char **sum** (size_t i1, size_t i2) const
- char **cumsum_** ([wbvector](#) &S, char xflag=0) const
- char **Cumsum** (char zflag=0) const
- char **cumsum0prod** ([wbvector](#)< T2 > &b, [wbvector](#)< char > &cs, char xflag) const
- [wbvector](#) & **Eldiv** (const [wbvector](#) &v) const
- [wbvector](#) & **tensorProd** (const [wbvector](#) &v2, [wbvector](#) &vv) const
- char **scalarProd** (const [wbvector](#) &v) const
- char **scalarProd** (const char *v) const
- char **norm** () const
- char **norm2** () const
- double **normDiff** (const [wbvector](#) &v, char fac=1) const
- char **normDiff2** (const [wbvector](#) &v, char fac=1) const
- double **normDiff2_** (const [wbvector](#) &v, double fac=1) const
- char **GSPProject** (const char *v, char isnorm=0, char tflag=0) const
- [wbvector](#) & **Normalize** (const char *F=NULL, int L=0)
- [wbvector](#) & **subtract** (const char *d)
- void **getReal** ([wbvector](#)< double > &R) const
- void **getReal** ([wbvector](#)< double > &R) const
- void **getImag** ([wbvector](#)< double > &I) const
- void **getImag** ([wbvector](#)< double > &I) const
- [wbvector](#) & **Conj** ()
- [wbvector](#)< [wbcomplex](#) > & **Conj** ()
- [wbvector](#) & **applyFlag** (char &flag, const char &fac, [wbvector](#) &a)
- bool **allEqual** () const
- bool **allEqual** (const char &a) const
- bool **allZero** () const
- bool **allOnes** () const
- bool **allEqual2** (const char &a, char eps=1E-15) const
- bool **allUnequal** (const char &a) const
- bool **any** () const
- bool **anyUnequal** (const char &a) const
- bool **anyUnequal** (const [wbvector](#)< size_t > &I, const char &x, size_t *k=NULL) const
- bool **anyEqual** (const char &a, size_t *i1=NULL) const
- bool **anyEqual** (const [wbvector](#)< size_t > &I, const char &x, size_t *k=NULL) const
- bool **anyGT** (const char &a) const
- bool **anyGE** (const char &a) const
- bool **anyLT** (const char &a) const
- bool **anyLE** (const char &a) const
- bool **allGT** (const char &a) const
- bool **allGE** (const char &a) const
- bool **allLE** (const char &a) const
- bool **allLT** (const char &a) const
- bool **allIn** (const char &a1, const char &a2) const
- bool **anySmallVals** (const char eps=1E-15) const
- bool **isEqual** (const char *v) const
- bool **isEqual** (const [wbvector](#) &v) const
- bool **isEmpty** () const
- **operator bool** () const

- bool **operator!** () const
- bool **operator==** (const [wbvector](#) &v) const
- bool **operator!=** (const [wbvector](#) &v) const
- bool **isEqualR** (const [wbvector](#) &v) const
- bool **operator^** (const [wbvector](#) &v) const
- bool **deepEqualP** (const [wbvector](#) &B) const
- char **cmp** (const [wbvector](#) &v) const
- bool **operator<** (const [wbvector](#) &v) const
- int **operator<** (const char &r) const
- bool **operator<=** (const [wbvector](#) &v) const
- bool **operator>** (const [wbvector](#) &v) const
- bool **operator>=** (const [wbvector](#) &v) const
- char & **el** (size_t i) const
- char **elx** (size_t i, const char &x) const
- const char & **operator[]** (size_t i) const
- char & **operator[]** (size_t i)
- [wbvector](#) **operator[]** (const [wbvector](#)< size_t > &l) const
- char & **last** () const
- char & **last** (size_t &i) const
- char & **last** (T1 &i_) const
- char & **p2last** () const
- char * **ref** (size_t i) const
- [wbvector](#) & **setRand** (double fac=1., double shift=0.)
- [wbvector](#)< [wbcomplex](#) > & **setRand** (double fac, double shift)
- void **setIdx** (double shift=0., double fac=1.)
- int **set2Group** (const [wbperm](#) &P, const WBINDEX &D, const char *S0=NULL, char iflag=0)
- [wbvector](#) & **Resize_** (size_t n)
- [wbvector](#) & **Shorten2** (size_t n)
- [wbvector](#) & **Resize** (size_t n, const char *d=NULL)
- [wbvector](#) & **resize** (size_t n, [wbvector](#) &X) const
- [wbvector](#) & **cat** (const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) > &vv)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) * > &vv)
- [wbvector](#) & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)
- [wbvector](#) & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3=0, const [wbvector](#)< T3 > *v4=0)
- [wbvector](#)< char > & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3, const [wbvector](#)< T3 > *v4)
- [wbvector](#)< char > & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)
- [wbvector](#) & **Append** (const [wbvector](#) &v2)
- [wbvector](#) & **Append** (const char &x)
- [wbvector](#) & **Append** (size_t d2, const char *x)
- [wbvector](#) **append** (const [wbvector](#) &v) const
- [wbvector](#) & **append** (const [wbvector](#) &v2, [wbvector](#) &vout) const
- void **append** (const char *x, size_t d2, [wbvector](#) &vout) const
- int **recLess** (const char *v) const
- int **recLessE** (const char *v) const
- int **recCompare** (const char *v) const
- char **max** () const
- char **max** (size_t *k) const
- char **max** (unsigned &k) const
- char **max_** (char x) const

- char **min** () const
- char **min** (size_t *k) const
- char **min** (unsigned &k) const
- char **min_** (char x) const
- char **aMax** (size_t *k=NULL) const
- char **aMax** (unsigned &k) const
- char **aMin** (char zflag=0, size_t *k=NULL) const
- char **maxneg** (size_t *k=NULL) const
- char **minpos** (size_t *k=NULL) const
- char **avg** () const
- char **std** () const
- void **range** (char &xmin, char &xmax) const
- char **range** () const
- **wbvector** & **Permute** (const **wbperm** &P, char iflag=0)
- **wbvector** & **permute** (**wbvector** &, const **wbperm** &P, char iflag=0) const
- **wbvector** **permute** (const **wbperm** &P, char iflag=0) const
- **wbvector** & **blockPermute** (const **wbperm** &P, **wbvector** &, char iflag=0) const
- **wbvector** & **BlockPermute** (const **wbperm** &P, char iflag=0)
- int **blockCompare** (const **wbperm** &pa, const **wbvector** &b, const **wbperm** &pb) const
- void **get** (const WBINDEX &I0, **wbvector** &v) const
- void **get** (const **wbvector**< size_t > &I, char *v) const
- void **get** (size_t i1, size_t i2, **wbvector** &v) const
- void **selectSU** (const WBINDEX &I)
- void **select** (const WBINDEX &I, char *r) const
- **wbvector** & **select** (const WBINDEX &I, **wbvector** &a) const
- **wbvector** **select** (const WBINDEX &I) const
- **wbvector** & **Select** (const WBINDEX &I)
- **wbvector** & **BlockSelect** (const WBINDEX &I, const WBINDEX &D)
- **wbvector** & **getI** (size_t k, **wbvector** &v) const
- **wbvector** & **getI** (const **wbindex** &k, **wbvector** &v, char uflag=1) const
- **wbvector** & **getI** (const WBINDEX &k, **wbvector** &v, char uflag=1) const
- **wbvector** & **initI** (const **wbvector** &a, const **wbindex** &ia)
- **wbvector** & **initI** (const **wbvector** &a, const **wbindex** &ia, const **wbvector** &b, const **wbindex** &ib, char uflag=1)
- **wbvector** & **Flip** ()
- void **flip** (**wbvector** &a) const
- **wbvector** **flip** () const
- char **prod** () const
- char **prod** (char x) const
- char **prod** (size_t i, size_t i2) const
- char **prod** (size_t i, size_t i2, char x) const
- char **prod** (const T1 *I, size_t n) const
- T2 **prod** (const T1 *I, size_t n, T2 &s2) const
- void **prod2** (char &p1, char &p2, const char &p0=0) const
- void **info** (const char *istr) const
- void **print** (const char *istr, char mflag=0) const
- void **printdata** (const char *istr=0) const
- void **put** (const char *F, int L, const char *vname="ans", char tflag=0, const char *ws="base") const
- void **put** (const char *vname="ans", char tflag=0, const char *ws="base") const
- void **dispWithMsg** (const char *file, int line, const char *msg, const char *vname="") const
- void **mat2mx** (mxArray *a, char dim=2) const
- void **mat2mxs** (mxArray *a, char field_nr, char dim=2) const
- mxArray * **toMx_Struct** (const char tflag=0) const
- mxArray * **toMxP_S** (const char tflag=0) const
- mxArray * **toMx** (const char tflag=0) const
- mxArray * **toMx** (const char tflag) const

- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMxP (const char tflag=0) const`
- `mxArray * toMx_base (const char tflag=0) const`
- `mxArray * toMx_base (const char tflag) const`
- `mxArray * toMx_base (const char tflag) const`
- `mxArray * toMx_base (const char tflag) const`
- `mxArray * toMx_base_d (const char tflag=0) const`
- `mxArray * toMx_offset (const char &offset, const char tflag=0) const`
- `mxArray * mxCreateStruct (unsigned m, unsigned n) const`
- `void add2MxStruct (mxArray *S, unsigned i, char tflag=0) const`
- `void add2MxStruct (mxArray *S, const char *vname, char tflag=0) const`
- `mxArray * mxCreateCell (unsigned m, unsigned n) const`
- `void add2MxCell (mxArray *S, unsigned i, char tflag=0) const`
- `void putg (const char *vname="ans", char tflag=0) const`
- `void putx (const char *vname="ans", char tflag=0) const`
- `wstring toStr (int n, const char *sep) const`
- `wstring toStr () const`
- `wstring toStr (int n, const char *sep) const`
- `wstring sizeStr () const`
- `wstring toStrf (const char *fmt0="", const char *sep=" ", unsigned stride=0, const char *sep2="") const`
- `wstring toStrf (const char *fmt_, const char *sep, unsigned stride, const char *sep2) const`

Public Attributes

- unsigned **nbits**

Public Attributes inherited from `wstring< char >`

- char * **data**
- `size_t len`
- char **isref**

Protected Member Functions

- `Bitset & NEW_BITSET (unsigned n, const char *d=NULL)`

Protected Member Functions inherited from `wstring< char >`

- `wstring & RENEW (size_t n, const char *d=NULL, size_t l=-1, char init=1)`
- `wstring & RENEW_VAL (size_t n, const char &x)`

The documentation for this class was generated from the following files:

- `wbbitset.hh`
- `wbbitset.cc`

4.5 Wb::bitset< T > Class Template Reference

Public Member Functions

- **bitset** (const [bitset](#)< T > &b)
- [bitset](#) & **setall** (bool t)
- [bitset](#) & **set** (unsigned i)
- [bitset](#) & **unset** (unsigned i)
- bool **isset** (unsigned i)
- unsigned **nnz** () const
- bool **operator==** (const [bitset](#) &b) const
- bool **operator!=** (const [bitset](#) &b) const
- [wstring toStr](#) () const
- mxArray * **toMx** () const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i) const
- void **put** (const char *vname, const char *ws="base") const
- void **put** (const char *file, int line, const char *vname, const char *ws="base") const

Public Attributes

- T **data**

The documentation for this class was generated from the following files:

- wbbiset.hh
- wbbiset.cc

4.6 blockSpaceQS< TQ, TD > Class Template Reference

The documentation for this class was generated from the following file:

- clebsch.hh

4.7 Wb::cc_quad Class Reference

Public Member Functions

- **cc_quad** (long double l=0)
- [cc_quad sqrt](#) () const
- [cc_quad fabs](#) () const
- [cc_quad abs](#) () const
- [cc_quad operator+](#) (const [cc_quad](#) &b) const
- [cc_quad operator-](#) (const [cc_quad](#) &b) const
- [cc_quad operator*](#) (const [cc_quad](#) &b) const
- [cc_quad operator/](#) (const [cc_quad](#) &b) const
- [cc_quad operator+=](#) (const [cc_quad](#) &b)
- [cc_quad operator-=](#) (const [cc_quad](#) &b)
- [cc_quad operator*=](#) (const [cc_quad](#) &b)
- [cc_quad operator/=](#) (const [cc_quad](#) &b)

Public Attributes

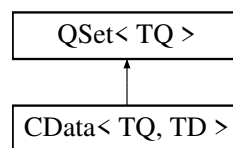
- long double **f**

The documentation for this class was generated from the following file:

- wbmprf.h

4.8 CData< TQ, TD > Class Template Reference

Inheritance diagram for CData< TQ, TD >:

**Public Member Functions**

- **CData** (const **CData** &C)
- **CData** (const **QSet**< TQ > &Q)
- **CData** (const **CRef**< TQ > &R, unsigned I)
- **CData** & **init** ()
- **CData** & **init** (const **QSet**< TQ > &Q, bool full=true)
- **CData** & **init** (const **CRef**< TQ > &R, unsigned I)
- **CData** & **init** (const **CData** &B)
- template<class T2 >
 CData & **init** (const **CData**< TQ, T2 > &B)
- **CData** & **operator=** (const **CData** &B)
- **CData** & **operator=** (const **QSet**< TQ > &Q)
- **CData** & **save2** (**CData** &B)
- **CData** & **initAbelian** (const **QSet**< TQ > &Q)
- **CData** & **initScalar** (const **QSet**< TQ > &Q)
- **CData** & **init3** (const **QType** &t_, SPIDX_T D1, SPIDX_T D2, SPIDX_T D, unsigned M=1)
- **CData** & **init3** (const **QType** &t_, const **qset**< TQ > &J1, const **qset**< TQ > &J2, const **qset**< TQ > &J, unsigned M=1)
- **CData** & **init3_abelian** (const **QType** &t_, const TQ &J1, const TQ &J2, const TQ &J)
- int **Reflnit** (const char *F, int L, const **CData**< TQ, TD > &B, char lflag=0, char bare=0)
- **CData** & **Reflnit_auxtr** (const char *F, int L, const **wbvector**< SPIDX_T > &S, const **wbvector**< RTD > &cgt)
- int **Reduce2Ref** (const char *F=0, int L=0, char force=0)
- int **LoadRef** (const char *F=0, int L=0, char force=1)
- int **Load_CRef** (const char *F, int L, **CData** &Cr, const char *cgd_file) const
- **CData** & **initX3** (const char *F, int L, const **QSet**< TQ > &Q, **wbvector**< char > &cflags, **wbperm** &P, unsigned loadRC=0, unsigned char flag=0)
- int **init_mxCRef** (const char *F, int L, const mxArray *S, unsigned k)
- TQ * **qptr** (unsigned k) const
- bool **olderThan** (const **CData** &B) const
- bool **operator!=** (const **CData** &B) const
- bool **operator==** (const **CData** &B) const
- int **gotDiff** (const **CData** &B, char lflag=0) const

- bool **sameAs** (const [CData](#) &B, char lflag=0) const
- bool **operator!=** (const [QSet](#)< TQ > &B) const
- bool **operator==** (const [QSet](#)< TQ > &B) const
- bool **operator!=** (const [cgdStatus](#) &b) const
- bool **operator==** (const [cgdStatus](#) &b) const
- bool **isEmpty** (char check=3) const
- bool **isAbelian** (const char *F=NULL, int L=0) const
- bool **sizeable** (unsigned Dmin=1024) const
- bool **isScalar** (char dflag=0) const
- TD **getScalar** (const char *F=NULL, int L=0) const
- unsigned **rank** (const char *F=NULL, int L=0) const
- unsigned **rankS** () const
- bool **isrank** (unsigned r, unsigned *r_=NULL) const
- template<class T >
 [wbvector](#)< T > & **getSize** ([wbvector](#)< T > &S, char bare=0, const [wbperm](#) *cgp=NULL) const
- SPIDX_T **dim** () const
- bool **isRefInit** (char check=1) const
- bool **checkValidStat** (const char *F=0, int L=0) const
- int **checkSameStat** (const char *F, int L, const [CData](#) &B, char lflag=1) const
- bool **sameType** (const [CData](#) &S, const char *F=NULL, int L=0) const
- int **cmpOM** (const [CData](#) &B) const
- template<class DB >
 bool **sameSizeR** (const [CData](#)< TQ, DB > &B, unsigned *r=NULL, const char *F=NULL, int L=0) const
- template<class DB >
 bool **sameSizeR** (const [cdata](#)< DB > &B, unsigned *r=NULL, const char *F=NULL, int L=0) const
- template<class T2 >
 int **sameSizeR** (const char *F, int L, const [wbvector](#)< T2 > &S) const
- bool **isSymmmetric** () const
- [CData](#) & **initOM** (const char *F, int L)
- unsigned **getOM** (const char *F=NULL, int L=0) const
- unsigned **gotOM** (const char *F=NULL, int L=0) const
- unsigned **checkOM** (const char *F=NULL, int L=0) const
- int **gotFullOM** (const char *F=NULL, int L=0) const
- int **completeOM_DegQ** (const char *F, int L, const [wbperm](#) &p, unsigned level=0)
- [CData](#) & **Project** (const char *F, int L, [cdata](#)< TD > X, [wbvector](#)< TD > &w)
- double **normDiff** (const char *F, int L, const [CData](#) &S, double eps=1E14) const
- double **norm2** (unsigned k=-1) const
- [wbvector](#)< RTD > & **trace** (const char *F, int L, [wbvector](#)< RTD > &cgt) const
- [wbvector](#)< RTD > **trace** (const char *F=NULL, int L=0) const
- [CData](#) & **trace** (const char *F, int L, [ctrlidx](#) &i1, [ctrlidx](#) &i2, [CData](#)< TQ, TD > &X) const
- [CData](#) & **AddMultiplicity** (const char *F, int L, const [cdata](#)< TD > &c)
- [CData](#) & **AddMultiplicity** (const char *F, int L, const [CData](#) &C)
- [cdata](#)< TD > & **getMultiplicity** (unsigned im, [cdata](#)< TD > &c) const
- bool **isBasicCG** (const char *F=NULL, int L=0) const
- SPIDX_T **getBasicCGSize** (unsigned *m=NULL) const
- [wbsparray](#)< TD > & **getBasicCG** (unsigned k, [wbsparray](#)< TD > &a) const
- int **getCG_set** (const char *F, int L, [wbIndex](#) &idx, [wbsparray](#)< TD > &a) const
- [CData](#) & **reduceto1JSymbol** (const char *F, int L, [CData](#)< TQ, TD > &C) const
- int **checkConsistency** (const char *F=NULL, int L=0) const
- int **checkNormSign** (const char *F=NULL, int L=0, char xflag=0) const
- int **checkQ** (const char *F, int L, const [QSet](#)< TQ > &Q, const char *istr=NULL) const
- bool **checkAdditivityZ** (const char *F, int L, const [wbMatrix](#)< double > &Z1, const [wbMatrix](#)< double > &Z2, const [wbMatrix](#)< double > &Z, TD eps=CG_EPS2) const
- double **SkipTiny** (const char *F=NULL, int L=0)
- [CData](#) & **Permute** (const [wbperm](#) &P, char iflag=0)

- [CData](#) & **permute** ([CData](#) &B, const [wbperm](#) &P, char iflag=0) const
- [CData](#) & **Conj** ()
- [CData](#) & **Conj** (unsigned i)
- size_t **memSize** () const
- void **info** (const char *istr=NULL, const char *F=NULL, int L=0) const
- [wbstring](#) **dStr** () const
- [wbstring](#) **qStr** () const
- [wbstring](#) **toStr** () const
- [wbstring](#) **toStr** (char vflag) const
- [wbstring](#) **sizeStr** () const
- [wbstring](#) **sizeStr** (char xflag) const
- mxArray * **toMx** () const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tst=0) const
- mxArray * **mxCreateCell** (unsigned m, unsigned n) const
- void **add2MxCell** (mxArray *S, unsigned i, char tst=0) const
- void **put** (const char *vname, const char *ws="caller") const
- void **put** (const char *F, int L, const char *vname, const char *ws="caller") const
- void **setuser_BUF** (unsigned char c=1)
- void **setuser_BUF** (unsigned char c, unsigned l)
- void **setuser_BUF_ref** (unsigned char c=1)
- bool **gotuser_BUF** () const
- bool **gotuser_BUF_ref** () const

Public Member Functions inherited from [QSet< TQ >](#)

- **QSet** (const [QSet](#) &B)
- **QSet** (const [QType](#) &t_, const [qset](#)< TQ > &qset_, unsigned iout=0, char isref=0)
- **QSet** (const [QType](#) &t_, const [iTags](#) &it, unsigned r=-1)
- **QSet** (const [CRef](#)< TQ > &A)
- **QSet** (const char *F, int L, const mxArray *a)
- **QSet** (const mxArray *a)
- **QSet** (const char *F, int L, const [CRef](#)< TQ > &A, const [ctrlIdx](#) &ica, const [CRef](#)< TQ > &B, const [ctrlIdx](#) &icb, [wbperm](#) *P=NULL)
- **QSet** (const char *F, int L, const char *s)
- **QSet** & **init** (const [CRef](#)< TQ > &A)
- **QSet** & **init** (const char *F, int L, const mxArray *a, unsigned k=0)
- **QSet** & **init** (const [QType](#) t_=QTYPE_UNKNOWN)
- **QSet** & **init** (const [QSet](#) &B)
- **QSet** & **init2ref** (const [QSet](#) &B)
- **QSet** & **init** (const [QType](#) &t_, const TQ *q, const char *qd)
- **QSet** & **init** (const [QType](#) &t_, const [qset](#)< gTQ > &qset_, unsigned iout=0, char isref=0)
- **QSet** & **init** (const [QType](#) t_, const TQ *qset_, unsigned r, unsigned N, unsigned iout=0)
- **QSet** & **init** (const [QType](#) t_, const TQ *qset_, unsigned r, unsigned N, const [iTags](#) &it)
- **QSet** & **init1J** (const [QType](#) &t_, const TQ *q, char iflag=0)
- **QSet** & **init2** (const [QType](#) &t_, const TQ *q=NULL)
- **QSet** & **init2** (const [QType](#) &t_, const [qset](#)< TQ > &q1, const [qset](#)< TQ > &q2, const char *qds="++")
- **QSet** & **init1** (const [QType](#) &t_, TQ *q=NULL)
- **QSet** & **init3** (const [QType](#) &t_)
- **QSet** & **init3** (const [QType](#) &t_, const [qset](#)< TQ > &J1, const [qset](#)< TQ > &J2, const [qset](#)< TQ > &J)
- **QSet** & **init3** (const [QType](#) &t_, const TQ &J1, const TQ &J2, const TQ &J)
- **QSet** & **init** (const char *F, int L, const [CRef](#)< TQ > &A, const [ctrlIdx](#) &ica, const [CRef](#)< TQ > &B, const [ctrlIdx](#) &icb, [wbperm](#) *Pcgd=NULL)
- **QSet** & **init_str** (const char *F, int L, const char *s)

- [QSet](#) & **Conj** ()
- [QSet](#) & **Conj** (unsigned i)
- unsigned **rank** (const char *F=NULL, int L=0) const
- [QSet](#) & **permute** ([QSet](#) &B, const [wbperm](#) &P, char iflag=0) const
- [QSet](#) & **Permute** (const [wbperm](#) &P, char iflag=0)
- void **swap** ([QSet](#) &X)
- [QSet](#) & **save2** ([QSet](#) &X)
- [QSet](#) & **operator=** (const [QSet](#) &B)
- bool **operator==** (const [QSet](#) &B) const
- bool **operator!=** (const [QSet](#) &B) const
- char **cmp** (const [QSet](#)< TQ > &B) const
- bool **operator<** (const [QSet](#)< TQ > &B) const
- **operator bool** () const
- bool **operator!** () const
- bool **isEmpty** (char check=3) const
- bool **gotRCData** () const
- int **isZero** () const
- unsigned **isStd3** () const
- unsigned **checkStd3** () const
- int **init2opt3** (const [QSet](#)< TQ > &Q, [wbperm](#) &P, [wbvector](#)< char > &cflags)
- [QSet](#) & **Convert** (const char *s, [wbvector](#)< char > &cflags)
- [wbstring](#) **sizeStr** () const
- bool **isScalar** () const
- bool **is1JSymbol** () const
- bool **isAbelian** () const
- int **permitsOM** () const
- int **checkQ_abelian** (const char *F, int L) const
- int **checkQ_SU2** (const char *F, int L) const
- [QSet](#) & **Sort** ([wbperm](#) *cgp=NULL, char *conj=NULL, char iflag=0)
- bool **isSorted** () const
- size_t **memSize** () const
- [wbstring](#) **QStrS** (const [wbperm](#) *cgp=NULL, char sep='|') const
- [wbstring](#) **QStr** () const
- [wbstring](#) **toStr** () const
- [wbstring](#) **toStr** (const char *istr) const
- [wbstring](#) **toTag** () const
- mxArray * **toMx** () const

Public Attributes

- [cdata](#)< TD > **cgd**
- [cgdStatus](#) **cstat**

Public Attributes inherited from [QSet](#)< TQ >

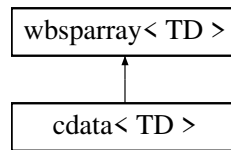
- [QType](#) **t**
- [qset](#)< TQ > **qs**
- [QDir](#) **qdir**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc
- clebsch_aux.cc
- clebsch_io.cc

4.9 cdata< TD > Class Template Reference

Inheritance diagram for cdata< TD >:



Public Member Functions

- **cdata** & **init** (SPIDX_T l=0)
- **cdata** & **init** (SPIDX_T D1, SPIDX_T D2, SPIDX_T D, unsigned M=1)
- **cdata** & **init** (const **wbvector**< SPIDX_T > &S)
- **cdata** & **init** (const **cdata** &C)
- template<class TQ >
cdata & **init** (const **CRef**< TQ > &R, unsigned l, char full=1)
- template<class T2 >
cdata & **init** (const **wbsparray**< T2 > &C)
- **cdata** & **init** (const char *F, int L, const mxArray *a, unsigned k, const **QType** &q, unsigned r=-1)
- **cdata** & **initScalar** (double x=1, unsigned r=-1)
- **cdata** & **initIdentity** (const **QType** &q, SPIDX_T d, TD dval=1)
- **cdata** & **initIdentity** (const **wbvector**< SPIDX_T > &S)
- bool **isScalar** () const
- unsigned **len** () const
- unsigned **getOM** (const char *F=NULL, int L=0, const char *istr=NULL) const
- **cdata** & **Conj** ()
- **cdata** & **permute** (**cdata** &B, const **wbperm** &P0, char iflag=0) const
- **cdata** & **Permute** (const **wbperm** &P0, char iflag=0)
- **cdata** & **Kron** (const **cdata** &B)
- TD **contract** (const char *F, int L, const **ctrlIdx** &ica, const **cdata** &B, const **ctrlIdx** &icb, **cdata** &C, **cdata** *Cx=NULL, **wbperm** *P=NULL, char normalize=1) const
- double **SkipTiny** (const char *F=NULL, int L=0)
- TD **NormSignC** (const char *F=NULL, int L=0, unsigned m=0, TD eps=CG_EPS1, TD eps2=CG_EPS2)
- void **info** () const
- template<class TQ >
cdata< TD > & **init** (const **CRef**< TQ > &R, unsigned k, char full)

Public Member Functions inherited from **wbsparray**< TD >

- **wbsparray** (SPIDX_T d1)
- **wbsparray** (SPIDX_T d1, SPIDX_T d2)
- **wbsparray** (SPIDX_T d1, SPIDX_T d2, SPIDX_T d3)
- **wbsparray** (SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T d4)
- **wbsparray** (const **wbvector**< SPIDX_T > &d, SPIDX_T l=0)
- **wbsparray** (char *sidx, SPIDX_T l=0)
- **wbsparray** (const **wbsparray** &a)
- **wbsparray** (const char *F, int L, const mxArray *a)
- **wbsparray** (const mxArray *a)
- **wbsparray** & **init** (const char *F, int L, const **wbarray**< TD > &A, TD eps=0)

- `template<class TA >`
`wbarray< TA > & toFull (wbarray< TA > &A) const`
- `template<class TA >`
`wbvector< TA > & toFull (wbvector< TA > &A) const`
- `template<class DB >`
`wbsparray & init (const wbsparray< DB > &a)`
- `SPIDX_T catRecs (SPIDX_T i0, const wbsparray &a, SPIDX_T id)`
- `wbsparray & init (const wbvector< SPIDX_T > &S0, SPIDX_T l=0)`
- `template<class I2 >`
`wbsparray & init (const wbvector< I2 > &S_, SPIDX_T l=0)`
- `wbsparray & init ()`
- `wbsparray & init (SPIDX_T d1)`
- `wbsparray & init (SPIDX_T d1, SPIDX_T d2)`
- `wbsparray & init (SPIDX_T d1, SPIDX_T d2, SPIDX_T d3)`
- `wbsparray & init (SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T d4)`
- `wbsparray & initz ()`
- `wbsparray & initz (SPIDX_T d1, SPIDX_T n)`
- `wbsparray & initz (SPIDX_T d1, SPIDX_T d2, SPIDX_T n)`
- `wbsparray & initz (SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T n)`
- `wbsparray & init_rzd (SPIDX_T r, SPIDX_T n)`
- `wbsparray & init_nnz (SPIDX_T n=0, char kflag=0)`
- `wbsparray & setRand (double x=1, char sflag=0)`
- `wbsparray & setRand_data (double x=1, char sflag=0)`
- `template<class DB >`
`wbsparray & init (const char *F, int L, SPIDX_T d1, SPIDX_T d2, const wbsparray< DB > &B)`
- `template<class DB >`
`wbsparray & init (const char *F, int L, SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, const wbsparray< DB > &B)`
- `template<class DB >`
`wbsparray & init (const char *F, int L, SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T d4, const wbsparray< DB > &B)`
- `wbsparray & initEmpty (const wbsparray &a)`
- `wbsparray & init (const char *F, int L, const mxArray *a, unsigned k=0)`
- `wbsparray & init2ref (const wbsparray &a)`
- `wbsparray & unRef ()`
- `wbsparray & initCAT (const char *F, int L, wbvector< wbsparray * > &X, wbvector< SPIDX_T > *I0=NULL, char del=0)`
- `wbsparray & initCAT (const char *F, int L, wbvector< wbsparray > &X, const wbvector< SPIDX_T > *I0=NULL)`
- `wbsparray & initCAT (const char *F, int L, const wbvector< const wbsparray * > &X, unsigned dim)`
- `wbsparray & initCAT (const char *F, int L, const wbvector< wbsparray > &X, unsigned dim)`
- `wbsparray & Cat (const char *F, int L, const wbsparray &B, unsigned dim)`
- `wbsparray & init_kron (const wbvector< SPIDX_T > &sa, const wbvector< SPIDX_T > &sb, SPIDX_T nab, char kflag)`
- `void setSize_kron (const char *F, int L, const wbvector< SPIDX_T > &sa, const wbvector< SPIDX_T > &sb, char kflag=0)`
- `void setSize_kron (const char *F, int L, char kflag=0)`
- `wbsparray & setRec (SPIDX_T i, SPIDX_T j)`
- `template<class T2 >`
`wbsparray & setRec (SPIDX_T i, SPIDX_T i1, SPIDX_T i2, const T2 &x)`
- `wbsparray & setRecP (SPIDX_T i, const SPIDX_T *I, const TD &x)`
- `wbsparray & setRec_kron (SPIDX_T I, TD x, unsigned r, SPIDX_T *i1, SPIDX_T *i2, unsigned kflag=0)`
- `wbsparray & Resize (SPIDX_T l)`
- `wbsparray & REDSIZE_NNZ (SPIDX_T l)`
- `wbsparray & Resize (const wbvector< SPIDX_T > &S)`
- `wbsparray & operator= (const wbsparray &a)`

- `template<class DB >`
`wbsparray` & `operator=` (const `wbsparray`< DB > &a)
- `wbsparray` & `save2` (`wbsparray` &a, char ref=0)
- `SPIDX_T sub2ind` (`SPIDX_T` k) const
- void `ind2sub` (`SPIDX_T` k, `SPIDX_T` *l) const
- void `ind2sub` (`SPIDX_T` k, `wbindex` &l) const
- `wbsparray`< TD > & `make2D` (const `wbindex` &ic, `wbsparray`< TD > &B, char pos=1) const
- `sparseIndex2D`< `SPIDX_T` > & `toIndex2D` (const `wbindex` &ic, `sparseIndex2D`< `SPIDX_T` > &a, char pos, char use_rmaj=0) const
- `wbsparray` & `initDiag` (`SPIDX_T` d)
- `wbsparray` & `initDiag` (const `wbvector`< TD > &dd)
- `wbsparray` & `initDiag2` (`SPIDX_T` d, TD dval=1)
- bool `isDiagMatrix` (TD eps=1E-14) const
- bool `isDiag` (const char *F=0, int L=0) const
- `wbvector`< TD > & `getDiag` (const char *F, int L, `wbvector`< TD > &dd) const
- TD `getDiag` (const char *F, int L, `SPIDX_T` k) const
- `wbsparray` & `getDiag` (const char *F, int L, `wbsparray` &A) const
- `wbsparray` & `diag2reg` (const char *F=0, int L=0, unsigned r=-1)
- `wbsparray` & `diag2reg` (const char *F, int L, `wbsparray` &A, unsigned r=-1) const
- `wbsparray` & `toDiag` (const char *F=0, int L=0)
- int `toDiagSafe` (const char *F, int L, TD eps=1E-14)
- `wbsparray` & `initIdentity` (`SPIDX_T` d, TD dval=1)
- `wbsparray` & `initIdentity` (`SPIDX_T` d1, `SPIDX_T` d2, `sSPIDX_T` m=0, TD dval=1)
- `wbsparray` & `initIdentity` (const `wbvector`< `SPIDX_T` > &S0, int m=-1, `sSPIDX_T` k=0, char fflag='f')
- `wbsparray` & `initIdentityB3` (`SPIDX_T` d1, `SPIDX_T` d2, `SPIDX_T` D, `sSPIDX_T` i0=0, char fflag='f')
- unsigned `rank` (const char *F=NULL, int L=0) const
- `SPIDX_T dim` () const
- `SPIDX_T dim` (unsigned k) const
- `SPIDX_T dim0` (unsigned k) const
- `SPIDX_T matDim` (unsigned k=1)
- `SPIDX_T maxDim` (unsigned r=-1) const
- TD `norm2` (char tnorm=0) const
- `wbstring sizeStr` () const
- TD `SkipTinyCols` (TD eps=1E-14)
- TD `SkipTiny` (TD eps=1E-14)
- bool `isVector` (const char *F=0, int L=0) const
- bool `isMatrix` (const char *F=0, int L=0) const
- bool `isRank` (unsigned r) const
- bool `isSMatrix` (const char *F=0, int L=0, `SPIDX_T` *d=NULL) const
- bool `isSMatrix` (const char *F, int L, `SPIDX_T` d) const
- bool `isEmpty` () const
- bool `isEmpty_` () const
- `operator bool` () const
- bool `operator!` () const
- bool `isZero` (TD eps=0, char flag=0) const
- `SPIDX_T nnz` () const
- `SPIDX_T nnz` (const TD eps1, const TD eps2=1E9) const
- `SPIDX_T nnz_diag` () const
- `SPIDX_T numel` () const
- TD `norm` (char tnorm=0) const
- `template<class T2 >`
`wbvector`< T2 > & `norm2vec` (unsigned k, `wbvector`< T2 > &A, char tnorm=0) const
- void `swap` (`wbsparray` &B)
- `wbsparray` & `TimesEI` (const `wbsparray` &B, char tnorm=0)
- TD `dotProd` (const char *F, int L, const `wbsparray` &B, char tnorm=0) const

- TD **froNorm2** (const [wbsparray](#) &B) const
- double **normDiff2** (const [wbsparray](#) &B) const
- double **normDiff** (const [wbsparray](#) &B) const
- TD **Normalize** (char tnorm=0, char qflag=0)
- TD **NormalizeCol** (SPIDX_T k, char tnorm=0, char qflag=0)
- [wbsparray](#) & **QRdecomp** (const char *F, int L, [wbsparray](#) &Q, char posR=0, TD eps=1E-14)
- [wbsparray](#) & **OrthoNormalizeColsQR** (const char *F=NULL, int L=0, TD eps=1E-14)
- [wbsparray](#) & **OrthoNormalizeColsGS** (const char *F=NULL, int L=0, char tnorm=0, char qxflag=0, TD eps=1E-14, unsigned np=1)
- [wbsparray](#) & **setCol** (const char *F, int L, SPIDX_T k, const [wbsparray](#) &v)
- [wbsparray](#) & **setCol** (SPIDX_T k, const [wbsparray](#) &v)
- double **sparsity** () const
- int **checkTrailingSingletons** (const char *F=0, int L=0, unsigned *r=NULL) const
- int **SkipTrailingSingletons** (const char *F=0, int L=0, unsigned r=-1)
- int **skipTrailingSingletons** (const char *F, int L, [wbsparray](#) &c, unsigned r=-1) const
- bool **hasSingletons** (const [wbindex](#) &l) const
- [wbsparray](#) & **skipSingletons** (const char *F, int L, const [wbindex](#) &l, [wbsparray](#) &A) const
- [wbsparray](#) & **AddTrailingSingletons** (const char *F, int L, unsigned r)
- [wbsparray](#) & **initScalar** (const TD x=0, unsigned r=-1)
- char **isScalar** (const char *F=0, int L=0, char zflag=0) const
- [wbsparray](#) & **toScalar** (const char *F=0, int L=0, unsigned r=-1)
- TD **getScalar** (const char *F=0, int L=0) const
- char **checkSize** (const char *F=0, int L=0, const char *fct=0, const char *istr=0) const
- char **checkSize2** (const char *F, int L, const [wbsparray](#) &B) const
- void **check_IDX_range** (const char *F=0, int L=0) const
- bool **isProptold** () const
- bool **isProptold** (TD &q, TD eps=1E-14) const
- bool **isIdentityMatrix** (TD eps=1E-14) const
- char **isIdentity** (TD eps=1E-14) const
- char **isIdentity** (const [wbperm](#) &P, TD *dval=NULL, TD eps=1E-14) const
- bool **hasSize** (SPIDX_T d1) const
- bool **hasSize** (SPIDX_T d1, SPIDX_T d2) const
- bool **hasSize** (SPIDX_T d1, SPIDX_T d2, SPIDX_T d3) const
- template<class T >
[wbvector](#)< T > & **getSize** ([wbvector](#)< T > &S) const
- char **fitsSize** (const [wbsparray](#) &, char strict=0) const
- char **sameSize** (const [wbsparray](#) &, char strict=0) const
- char **sameSizeUp2Singletons** (const [wbvector](#)< SPIDX_T > &S, char strict=0) const
- char **matchWithSingletons** (const char *F, int L, const [wbvector](#)< SPIDX_T > &S, [wbvector](#)< int > &M) const
- char **sameUptoFac** (const [wbsparray](#) &B, TD *fac=NULL, char lex=-1, TD eps=1E-12) const
- bool **operator==** (const [wbsparray](#) &B) const
- bool **operator!=** (const [wbsparray](#) &B) const
- [wbsparray](#) & **Plus** (const char *F, int L, const [wbsparray](#) &B, TD bfac=1, TD afac=1, TD eps=0)
- TD **sum** () const
- TD **aMin** () const
- TD **aMax** () const
- SPIDX_T **findRecSortedP** (SPIDX_T *idx, SPIDX_T n, char lex=-1)
- [wbsparray](#) & **Eldiv** (const char *F, int L, const [wbvector](#)< TD > &x_)
- [wbsparray](#) & **operator+=** (const [wbsparray](#) &B)
- [wbsparray](#) & **operator-=** (const [wbsparray](#) &B)
- [wbsparray](#) & **Conj** ()
- [wbsparray](#) & **operator*=** (TD x)
- [wbsparray](#) & **operator/=** (TD x)
- const TD & **operator[]** (SPIDX_T i) const

- TD & **operator[]** (SPIDX_T i)
- TD **value** (const SPIDX_T *idx, SPIDX_T n, char lex=-1) const
- **wbstring** **info2Str** (const char *istr=0) const
- **size_t** **memSize** () const
- void **info** (const char *istr="") const
- void **info** (const char *F, int L, const char *="") const
- void **print** (const char *istr="", const char *fmt="") const
- void **print** (const char *F, int L, const char *istr="", const char *fmt="") const
- void **printdata** (const char *istr="ans", const char *fmt="") const
- void **sperror_this** (const char *F, int L, const char *fct=0, const char *istr=0) const
- TD **trace** () const
- template<class T2 >
wbvector< T2 > & **trace** (unsigned k, **wbvector**< T2 > &A) const
- bool **isSym_aux** (const char *file, int line, const char *fct, const **wbsparray** &B, TD eps, TD *xref, const char symflag='s', const char lflag=1) const
- bool **isHConj** (const **wbsparray** &B, TD eps=1E-12, TD *xref=NULL) const
- bool **isHConj** (TD eps=1E-12, TD *xref=NULL) const
- bool **isAHerm** (const **wbsparray** &B, TD eps=1E-12, TD *xref=NULL) const
- bool **isAHerm** (TD eps=1E-12, TD *xref=NULL) const
- **wbsparray** & **Sort** (char lex=-1)
- double **Compress** (const char *F=0, int L=0, TD eps=0, char lex=-1, const **wbperm** *P=NULL)
- bool **isCompressed** (char lex=-1, char dir=0) const
- int **splitSparseCM** (const char *F, int L, **wbvector**< **wbsparray** > &X, TD eps=0, char ref=0, char tnorm=0)
- **wbsparray** & **reshape** (const **wbvector**< SPIDX_T > &S, **wbsparray** &A) const
- **wbsparray** & **Reshape** (const **wbvector**< SPIDX_T > &S)
- **wbsparray** & **Reshape** (SPIDX_T s1, SPIDX_T s2)
- **wbsparray** & **Reshape** (SPIDX_T s1, SPIDX_T s2, SPIDX_T s3)
- **wbvector**< TD > & **getCol** (SPIDX_T k, **wbvector**< TD > &v) const
- **wbvector**< TD > & **getRow** (SPIDX_T k, **wbvector**< TD > &v) const
- **wbsparray** & **getBlock** (const char *F, int L, SPIDX_T i1, SPIDX_T i2, SPIDX_T j1, SPIDX_T j2, **wbsparray**< TD > &B) const
- **wbsparray** & **getCol** (SPIDX_T k, **wbsparray**< TD > &B)
- **wbsparray** & **splitBlock** (const char *F, int L, sSPIDX_T i1, sSPIDX_T i2, sSPIDX_T j1, sSPIDX_T j2, **wbsparray**< TD > &B)
- **wbsparray** & **SplitQR** (sSPIDX_T k, sSPIDX_T k_, **wbsparray**< TD > &X)
- TD **getHouseholderVec** (const char *F, int L, SPIDX_T k, SPIDX_T j, **wbsparray**< TD > &b, TD eps2=0) const
- **wbsparray** & **IDX_Shift** (sSPIDX_T i1, sSPIDX_T i2)
- **wbsparray**< TD > & **LProject1** (const **wbsparray**< TD > &b, **wbsparray**< TD > &X, TD xfac=1, char tnorm=0) const
- SPIDX_T **LContractVec_omp** (const **wbsparray**< TD > &b, **wbvector**< TD > &x, char tnorm) const
- **wbsparray** & **splitLast** (SPIDX_T k, **wbsparray** &a) const
- **wbsparray** & **select0** (const WBINDEX &, unsigned dim, **wbsparray** &) const
- **wbsparray** & **Select0** (const WBINDEX &l, unsigned dim)
- **wbsparray** & **Permute** (const char *s, char iflag=0)
- **wbsparray** & **Permute** (const **wbperm** &, char iflag=0)
- **wbsparray** & **permute** (const char *s, **wbsparray** &, char iflag=0) const
- **wbsparray** & **permute** (const **wbperm** &, **wbsparray** &, char iflag=0) const
- **wbsparray** & **transpose** (const char *F, int L, **wbsparray** &A) const
- **wbsparray** & **Transpose** (const char *F=0, int L=0)
- **wbsparray** & **htranspose** (const char *F, int L, **wbsparray** &A) const
- **wbsparray** & **hTranspose** (const char *F=0, int L=0)
- **wbsparray** & **ColPermute** (const **wbperm** &P, char iflag=0)
- **wbsparray** & **MatPermute** (const **wbperm** &P, char iflag=0)
- **wbsparray** & **mtimes** (const char *F, int L, const **wbsparray** &B, **wbsparray** &C, char sflag=0, const TD eps=0)

- char **contract_scalar** (const char *F, int L, const [ctrldx](#) &ica, const [wbsparray](#) &B, const [ctrldx](#) &icb, [wbsparray](#) &C, const [wbperm](#) &pfinal=[wbperm](#)(), const TD &afac=1., const TD &cfac=0.) const
- [wbsparray](#) & **contract_diag_diag** (const char *F, int L, const [ctrldx](#) &ica, const [wbsparray](#) &B, const [ctrldx](#) &icb, [wbsparray](#) &C, const [wbperm](#) &pfinal, const TD &afac, const TD &cfac) const
- char **contract_check_2full** (const char *F, int L, const [ctrldx](#) &ica, const [wbsparray](#) &B, const [ctrldx](#) &icb, [wbsparray](#) &C, const [wbperm](#) &pfinal, const TD &afac, const TD &cfac, char flag, TD eps, SPIDX_T *nnz_↔ C=NULL) const
- [wbsparray](#) & **contract** (const char *F, int L, const [ctrldx](#) &ica, const [wbsparray](#) &B, const [ctrldx](#) &icb, [wbsparray](#) &C, const [wbperm](#) &pfinal=[wbperm](#)(), const TD &afac=1., const TD &cfac=0., char sflag=0, const TD eps=0) const
- [wbsparray](#) & **contract** (const char *F, int L, const char *ica, const [wbsparray](#) &B, const char *icb, [wbsparray](#) &C, const [wbperm](#) &pfinal=[wbperm](#)(), const TD &afac=1., const TD &cfac=0., char sflag=0, const TD eps=0) const
- [wbarray](#)< TD > & **contract** (const char *F, int L, const [ctrldx](#) &ica, const [wbsparray](#) &B, const [ctrldx](#) &icb, [wbarray](#)< TD > &C, const [wbperm](#) &pfinal, const TD &afac, const TD &cfac) const
- [wbsparray](#) & **contract** (const char *F, int L, unsigned i1, const [wbvector](#)< TD > &B, [wbsparray](#) &C) const
- [wbsparray](#) & **comm** (const char *F, int L, const [wbsparray](#) &B, [wbsparray](#) &C, char aflag='N', char bflag='N') const
- [wbsparray](#) & **tensorProdX** (const char *F, int L, const [wbsparray](#) &B, [wbsparray](#) &C, char conja=0, const TD &afac=1, char conjb=0, const TD &cfac=0, const [wbperm](#) *Pfinal=NULL) const
- [wbsparray](#) & **tensorProd** (const char *F, int L, const [wbsparray](#) &B, [wbsparray](#) &C, char aflag='N', char bflag='N', char kflag=0) const
- [wbsparray](#) & **Kron** (const [wbsparray](#) &B, char aflag='N', char bflag='N', char kflag='k')
- [wbsparray](#) & **kron** (const char *F, int L, const [wbsparray](#) &B, [wbsparray](#) &C, char aflag='N', char bflag='N', char kflag='k') const
- mxArray * **toMxSp** () const
- mxArray * **toMx** (int m=-99) const
- mxArray * **IDtoMx** () const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *a, unsigned i, int ma=-99) const
- mxArray * **mxCreateCell** (unsigned m, unsigned n) const
- void **add2MxCell** (mxArray *a, unsigned i) const
- mxArray * **toMx0** (int m=-99) const
- mxArray * **mxCreateStructX** (unsigned m, unsigned n, int ma=-99) const
- void **add2MxStructX** (mxArray *a, unsigned i, int ma=-99) const
- mxArray * **toMX** () const
- mxArray * **mxCreateSTRUCT** (unsigned m, unsigned n) const
- mxArray * **add2MxSTRUCT** (mxArray *a, unsigned i) const
- void **put** (const char *vname, unsigned ma=-99, const char *ws="caller") const
- void **put0** (const char *F, int L, const char *vname, const char *ws="caller") const
- void **put** (const char *F, int L, const char *vname, unsigned ma=-99, const char *ws="caller", char flag=0) const
- template<class T2 >
[wbvector](#)< T2 > & **toFull** ([wbvector](#)< T2 > &A) const
- template<class DB >
[wbsparray](#)< TD > & **init** (const char *F, int L, SPIDX_T d1, SPIDX_T d2, const [wbsparray](#)< DB > &B)
- template<class DB >
[wbsparray](#)< TD > & **init** (const char *F, int L, SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, const [wbsparray](#)< DB > &B)
- template<class DB >
[wbsparray](#)< TD > & **init** (const char *F, int L, SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T d4, const [wbsparray](#)< DB > &B)

Additional Inherited Members**Public Attributes inherited from `wbsparray< TD >`**

- `wbvector< SPIDX_T > SIZE`
- `wbMatrix< SPIDX_T > IDX`
- `wbvector< TD > D`
- char `isref`

Protected Member Functions inherited from `wbsparray< TD >`

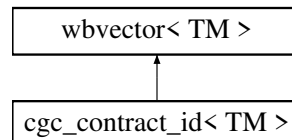
- unsigned `get2DSize` (const char *F, int L, SPIDX_T &d1, SPIDX_T &d2, int m=-1, char fflag=0) const
- unsigned `get2DIndex` (`wbMatrix< SPIDX_T >` &IJ, SPIDX_T &d1, SPIDX_T &d2, int m=-1) const

The documentation for this class was generated from the following files:

- `clebsch.hh`
- `clebsch.cc`
- `clebsch_aux.cc`
- `clebsch_io.cc`

4.10 `cgc_contract_id< TM >` Class Template Reference

Inheritance diagram for `cgc_contract_id< TM >`:

**Public Member Functions**

- template<class TQ, class TD >
`cgc_contract_id` (const `CData< TQ, TD >` &A, const `ctrlIdx` &ica, const `CData< TQ, TD >` &B, const `ctrlIdx` &icb)
- template<class TQ, class TD >
void `extract` (`CData< TQ, TD >` &A, `ctrlIdx` &ica, `CData< TQ, TD >` &B, `ctrlIdx` &icb, char xflag=1) const
- size_t `memSize` () const
- void `wblog_` (const char *F=0, int L=0, const char *istr=0) const
- `wbstring toStr` (char vflag=7) const

Public Member Functions inherited from `wbvector< TM >`

- `wbvector` (size_t l, TM x)
- `wbvector` (size_t l, const TM *d0=NULL, char ref=0)
- `wbvector` (const `wbvector` &v)
- `wbvector` (const `wbvector`< T2 > &v)
- `wbvector` (const `wbvector` &v1, const `wbvector` &v2)
- `wbvector` (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- `wbvector` (const mxArray *a, char check_type=1, char ref=0)
- `wbvector` & `init` (size_t l=0)
- `wbvector` & `init` (size_t l, const TM *d0)
- `wbvector` & `init` (size_t l, const TM *d0, char ref)
- `wbvector` & `init` (const `wbvector` &v)
- `wbvector` & `init` (size_t n, const `wbvector` &v)
- `wbvector`< TM > & `init` (const `wbsparray`< TM > &S)
- `wbvector` & `init` (size_t l1, const TM *d1, size_t l2, const TM *d2)
- int `init` (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type QS_UNUSED_↵
VAR=1, char ref=0)
- int `init` (const mxArray *a, const char *istr=NULL, char check_type=1)
- int `init` (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int `init` (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int `init` (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int `init` (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int `init` (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int `init` (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- `wbvector` & `initp` (size_t l, const TM *d0, size_t n, char init=1)
- `wbvector` & `init2val` (size_t l, const TM &x)
- `wbvector` & `initStride` (size_t l, const TM *d0, size_t stride)
- `wbvector` & `init_stride` (const TM *d0, size_t n, size_t m, size_t D0)
- `wbvector` & `init_data` (const TM *d0)
- `wbvector` & `init_data` (const T0 *d0)
- `wbvector` & `init_data` (const char *F, int L, const T0 *d0)
- `wbvector`< TM > & `initT` (const `wbsparray`< T2 > &S)
- `wbvector`< TM > & `initT` (size_t l, const T2 *v)
- `wbvector`< TM > & `initT` (const `wbvector`< T2 > &v)
- `wbvector`< TM > & `initT` (const char *F, int L, const `wbvector`< T2 > &v)
- `wbvector`< TM > & `initT` (const `wbsparray`< TD > &S)
- `wbvector`< TM > & `initT_` (const `wbvector`< T2 > &v)
- `wbvector` & `init2ref` (const `wbvector` &v)
- `wbvector` & `init2ref` (size_t l, const TM *d0)
- `wbvector` & `unRef` ()
- int `init_Struct` (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)
- int `init_base` (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)
- int `init_mpfr` (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)
- void `initDef` (size_t n)
- void `initDef` (const `wbvector` &v)
- `wbvector` & `operator=` (const `wbvector` &v)
- `wbvector`< TM > & `operator=` (const `wbvector`< TB > &v)
- `wbvector` & `set` (const TM &x)
- `wbvector` & `set` (const `wbvector`< size_t > &l, const TM &x)
- void `set` (size_t offset, const TM *x, size_t n)
- void `set` (const `wbvector`< double > &R, const `wbvector`< double > &l)
- void `set` (const `wbvector`< double > &R, const `wbvector`< double > &l)
- `wbvector` & `setMin` (const TM &x)
- `wbvector` & `setMax` (const TM &x)

- [wbvector](#) & **add** (const [wbvector](#)< size_t > &l, const TM &x)
- [wbvector](#) & **add** (const TM *d)
- [wbvector](#) & **Set** (const [wbvector](#)< size_t > &l, const [wbvector](#) &v)
- void **Sort** ([wbperm](#) &p, char dir=+1)
- void **Sort** (char dir=+1)
- void **sort** ([wbvector](#) &v2, [wbperm](#) &p) const
- void **sort** ([wbvector](#) &v2) const
- [wbperm](#) & **pSort** ([wbperm](#) &p) const
- bool **isSorted** () const
- bool **isSortedU** () const
- bool **isUnique** () const
- int **contains** (const TM &x) const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isFinite** () const
- size_t **find1** (const TM &x) const
- size_t **find1** () const
- size_t **count** (const TM &x) const
- size_t **icount** (const TM &x) const
- size_t **nnz** () const
- size_t **nnz** (TM eps) const
- size_t **numZeros** (TM eps=TM(1E-14)) const
- size_t **numEQ** (const TM &) const
- size_t **numGT** (const TM &) const
- size_t **numLT** (const TM &) const
- [wbindex](#) & **find** (const TM &x, [wbindex](#) &l, char iflag=0) const
- [wbindex](#) & **find** (const TM &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **find** ([wbindex](#) &l) const
- [wbindex](#) & **find** ([wbindex](#) &l, [wbindex](#) &lx) const
- [wbvector](#)< size_t > **find** () const
- [wbindex](#) & **findGT** (const TM &x, [wbindex](#) &l, char iflag=0) const
- [wbindex](#) & **findGT** (const TM &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **findzeros** ([wbindex](#) &l) const
- [wbindex](#) & **findRange** (const TM &x1, const TM &x2, [wbindex](#) &l, const char *w=NULL) const
- int **findClosestSorted** (TM r) const
- int **findClosest** (TM r) const
- size_t **findValues** (const [wbvector](#) &v, [wbindex](#) &l) const
- [wbvector](#) & **setp** (const TM *x)
- void **reset** ()
- void **cpy** (const TM *v)
- void **cpy** (const TM *x, size_t m)
- [wbvector](#) & **save2** ([wbvector](#) &v)
- void **swap** ([wbvector](#) &v)
- [wbvector](#) & **swap** (size_t i, size_t j)
- [wbvector](#) & **Collect** (TM &A1, TM &A2)
- [wbvector](#) & **Collect** (TM &A1, TM &A2, TM &A3)
- [wbvector](#) & **Skip** (size_t)
- [wbvector](#) & **Skip** (const WBINDEX &, char rflag=0)
- [wbvector](#) & **Move2_FE** (const WBINDEX &l, char FE_flag)

- [wbvector](#) & **Move2front** (const WBINDEX &l)
- [wbvector](#) & **Move2front** (size_t k)
- [wbvector](#) & **Move2end** (const WBINDEX &l)
- [wbvector](#) & **Move2end** (size_t k)
- [wbvector](#) & **mvLastTo** (size_t k)
- [wbvector](#) & **mvFirstTo** (size_t k)
- bool **checkSameLength** (const [wbvector](#) &a, const char *F=NULL, int L=0, const char *istr="") const
- bool **checkSameLength** (const [wbvector](#) &a, const [wbvector](#) &b, const char *F=NULL, int L=0, const char *istr="") const
- void **operator+=** (const T_ &a)
- void **operator+=** (const [wbvector](#) &b)
- void **operator-=** (const T_ &a)
- void **operator-=** (const [wbvector](#) &b)
- [wbvector](#) & **operator*=** (const T_ &a)
- [wbvector](#) & **operator/=** (const T_ &a)
- [wbvector](#) **operator*** (const T_ &a) const
- [wbvector](#) **operator+** (const T_ &a) const
- [wbvector](#) **operator+** (const [wbvector](#) &b) const
- [wbvector](#) **operator-** (const T_ &a) const
- [wbvector](#) **operator-** (const [wbvector](#) &b) const
- [wbvector](#) & **Plus** (const TM *b)
- [wbvector](#) & **Plus** (const TM *b, TM bfac)
- [wbvector](#) & **Plus** (const [wbvector](#) &b, TM bfac=1)
- void **plus** (const [wbvector](#) &b, [wbvector](#) &c, TM bfac=1, TM cfac=0) const
- [wbvector](#) & **times** (TM fac, [wbvector](#) &x) const
- [wbvector](#) & **RevertSigns** ()
- int **isum** () const
- TM **sum** () const
- TM **sum** (size_t i1, const char *estr) const
- TM **sum** (size_t i1, size_t i2) const
- TM **cumsum_** ([wbvector](#) &S, char xflag=0) const
- TM **Cumsum** (char zflag=0) const
- TM **cumsum0prod** ([wbvector](#)< T2 > &b, [wbvector](#)< TM > &cs, char xflag) const
- [wbvector](#) & **Eldiv** (const [wbvector](#) &v) const
- [wbvector](#) & **tensorProd** (const [wbvector](#) &v2, [wbvector](#) &vv) const
- TM **scalarProd** (const [wbvector](#) &v) const
- TM **scalarProd** (const TM *v) const
- TM **norm** () const
- TM **norm2** () const
- double **normDiff** (const [wbvector](#) &v, TM fac=1) const
- TM **normDiff2** (const [wbvector](#) &v, TM fac=1) const
- double **normDiff2_** (const [wbvector](#) &v, double fac=1) const
- TM **GSPProject** (const TM *v, char isnorm=0, char tflag=0) const
- [wbvector](#) & **Normalize** (const char *F=NULL, int L=0)
- [wbvector](#) & **subtract** (const TM *d)
- void **getReal** ([wbvector](#)< double > &R) const
- void **getReal** ([wbvector](#)< double > &R) const
- void **getImag** ([wbvector](#)< double > &I) const
- void **getImag** ([wbvector](#)< double > &I) const
- [wbvector](#) & **Conj** ()
- [wbvector](#)< [wbcomplex](#) > & **Conj** ()
- [wbvector](#) & **applyFlag** (char &flag, const TM &fac, [wbvector](#) &a)
- bool **allEqual** () const
- bool **allEqual** (const TM &a) const
- bool **allZero** () const

- `bool allOnes () const`
- `bool allEqual2 (const TM &a, TM eps=1E-15) const`
- `bool allUnequal (const TM &a) const`
- `bool any () const`
- `bool anyUnequal (const TM &a) const`
- `bool anyUnequal (const wbvector< size_t > &l, const TM &x, size_t *k=NULL) const`
- `bool anyEqual (const TM &a, size_t *i1=NULL) const`
- `bool anyEqual (const wbvector< size_t > &l, const TM &x, size_t *k=NULL) const`
- `bool anyGT (const TM &a) const`
- `bool anyGE (const TM &a) const`
- `bool anyLT (const TM &a) const`
- `bool anyLE (const TM &a) const`
- `bool allGT (const TM &a) const`
- `bool allGE (const TM &a) const`
- `bool allLE (const TM &a) const`
- `bool allLT (const TM &a) const`
- `bool allIn (const TM &a1, const TM &a2) const`
- `bool anySmallVals (const TM eps=1E-15) const`
- `bool isEqual (const TM *v) const`
- `bool isEqual (const wbvector &v) const`
- `bool isEmpty () const`
- `operator bool () const`
- `bool operator! () const`
- `bool operator== (const wbvector &v) const`
- `bool operator!= (const wbvector &v) const`
- `bool isEqualR (const wbvector &v) const`
- `bool operator^ (const wbvector &v) const`
- `bool deepEqualP (const wbvector &B) const`
- `char cmp (const wbvector &v) const`
- `bool operator< (const wbvector &v) const`
- `int operator< (const TM &r) const`
- `bool operator<= (const wbvector &v) const`
- `bool operator> (const wbvector &v) const`
- `bool operator>= (const wbvector &v) const`
- `TM & el (size_t i) const`
- `TM elx (size_t i, const TM &x) const`
- `const TM & operator[] (size_t i) const`
- `TM & operator[] (size_t i)`
- `wbvector operator[] (const wbvector< size_t > &l) const`
- `TM & last () const`
- `TM & last (size_t &i) const`
- `TM & last (TI &i_) const`
- `TM & p2last () const`
- `TM * ref (size_t i) const`
- `wbvector & setRand (double fac=1., double shift=0.)`
- `wbvector< wbcomplex > & setRand (double fac, double shift)`
- `void setIdx (double shift=0., double fac=1.)`
- `int set2Group (const wbperm &P, const WBINDEX &D, const TM *S0=NULL, char iflag=0)`
- `wbvector & Resize_ (size_t n)`
- `wbvector & Shorten2 (size_t n)`
- `wbvector & Resize (size_t n, const TM *d=NULL)`
- `wbvector & resize (size_t n, wbvector &X) const`
- `wbvector & cat (const wbvector &v2)`
- `wbvector & Cat (const wbvector &v1, const wbvector &v2)`
- `wbvector & Cat (const wbvector< wbvector > &vv)`

- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) * > &vv)
- [wbvector](#) & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)
- [wbvector](#) & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3=0, const [wbvector](#)< T3 > *v4=0)
- [wbvector](#)< TM > & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3, const [wbvector](#)< T3 > *v4)
- [wbvector](#)< TM > & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)
- [wbvector](#) & **Append** (const [wbvector](#) &v2)
- [wbvector](#) & **Append** (const TM &x)
- [wbvector](#) & **Append** (size_t d2, const TM *x)
- [wbvector](#) **append** (const [wbvector](#) &v) const
- [wbvector](#) & **append** (const [wbvector](#) &v2, [wbvector](#) &vout) const
- void **append** (const TM *x, size_t d2, [wbvector](#) &vout) const
- int **recLess** (const TM *v) const
- int **recLessE** (const TM *v) const
- int **recCompare** (const TM *v) const
- TM **max** () const
- TM **max** (size_t *k) const
- TM **max** (unsigned &k) const
- TM **max_** (TM x) const
- TM **min** () const
- TM **min** (size_t *k) const
- TM **min** (unsigned &k) const
- TM **min_** (TM x) const
- TM **aMax** (size_t *k=NULL) const
- TM **aMax** (unsigned &k) const
- TM **aMin** (char zflag=0, size_t *k=NULL) const
- TM **maxneg** (size_t *k=NULL) const
- TM **minpos** (size_t *k=NULL) const
- TM **avg** () const
- TM **std** () const
- void **range** (TM &xmin, TM &xmax) const
- TM **range** () const
- [wbvector](#) & **Permute** (const [wbperm](#) &P, char iflag=0)
- [wbvector](#) & **permute** ([wbvector](#) &, const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) **permute** (const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) & **blockPermute** (const [wbperm](#) &P, [wbvector](#) &, char iflag=0) const
- [wbvector](#) & **BlockPermute** (const [wbperm](#) &P, char iflag=0)
- int **blockCompare** (const [wbperm](#) &pa, const [wbvector](#) &b, const [wbperm](#) &pb) const
- void **get** (const WBINDEX &I0, [wbvector](#) &v) const
- void **get** (const [wbvector](#)< size_t > &I, TM *v) const
- void **get** (size_t i1, size_t i2, [wbvector](#) &v) const
- void **selectSU** (const WBINDEX &I)
- void **select** (const WBINDEX &I, TM *r) const
- [wbvector](#) & **select** (const WBINDEX &I, [wbvector](#) &a) const
- [wbvector](#) **select** (const WBINDEX &I) const
- [wbvector](#) & **Select** (const WBINDEX &I)
- [wbvector](#) & **BlockSelect** (const WBINDEX &I, const WBINDEX &D)
- [wbvector](#) & **getI** (size_t k, [wbvector](#) &v) const
- [wbvector](#) & **getI** (const [wbindex](#) &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **getI** (const WBINDEX &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **initI** (const [wbvector](#) &a, const [wbindex](#) &ia)
- [wbvector](#) & **initI** (const [wbvector](#) &a, const [wbindex](#) &ia, const [wbvector](#) &b, const [wbindex](#) &ib, char uflag=1)

Additional Inherited Members

Public Attributes inherited from [wbvector< TM >](#)

- TM * **data**
- size_t **len**
- char **isref**

Protected Member Functions inherited from [wbvector< TM >](#)

- [wbvector](#) & **RENEW** (size_t n, const TM *d=NULL, size_t l=-1, char init=1)
- [wbvector](#) & **RENEW_VAL** (size_t n, const TM &x)

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch_aux.cc

4.11 cgdStatus Class Reference

Public Member Functions

- **cgdStatus** (CGD_TYPE t_)
- **cgdStatus** (double t_)
- **cgdStatus** (const char *F, int L, mxArray *a)
- template<class TQ >
 cgdStatus (const CDATA_TQ &C)
- **cgdStatus** & **init** ()
- **cgdStatus** & **init** (const CGD_TYPE &t_)
- **cgdStatus** & **init0** (const CGD_TYPE &t_)
- **cgdStatus** & **update_m** ()
- **cgdStatus** & **update_m** (const CGD_TYPE &t_)
- **cgdStatus** & **operator=** (const [cgdStatus](#) &b)
- **cgdStatus** & **init** (const [cgdStatus](#) &b)
- CGR_TYPE **init** (const char *F, int L, const mxArray *a)
- int **sameID** (const [cgdStatus](#) &b) const
- int **cmp** (const char *F, int L, const [cgdStatus](#) &b) const
- bool **olderThan** (const [cgdStatus](#) &b) const
- bool **newerThan** (const [cgdStatus](#) &b) const
- bool **hasID** (char lflag=0) const
- bool **operator==** (const CGD_TYPE t_) const
- bool **operator!=** (const CGD_TYPE t_) const
- bool **operator==** (const [cgdStatus](#) &b) const
- bool **operator!=** (const [cgdStatus](#) &b) const
- bool **isSet** () const
- bool **isEmpty** () const
- int **invalid** () const
- bool **sameAs** (const [cgdStatus](#) &b, char lflag=0) const
- const char * **tstr** () const
- [wbstring](#) **toStr** (char vflag=0) const
- [wbstring](#) **u2Str** (const char *istr=NULL) const
- mxArray * **toMx** () const
- mxArray * **toMx** (CGR_TYPE rt) const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- mxArray * **add2MxStruct** (mxArray *S, unsigned k) const

Public Attributes

- CGD_TYPE **t**
- double **ctime**
- double **mtime**
- unsigned **ID**
- unsigned char **user** [4]

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch_aux.cc
- clebsch_io.cc

4.12 cgrType Class Reference

Public Member Functions

- **cgrType** (CGR_TYPE t_=CGR_DEFAULT)
- **cgrType** (double t_)
- template<class T >
 cgrType & **init** (const char *F, int L, T t_)
- **cgrType** & **operator=** (const **cgrType** &q)
- **cgrType** & **operator=** (const CGR_TYPE &t_)
- bool **operator<** (const CGR_TYPE t_) const
- bool **operator<=** (const CGR_TYPE t_) const
- bool **operator>** (const CGR_TYPE t_) const
- bool **operator>=** (const CGR_TYPE t_) const
- bool **operator==** (const CGR_TYPE t_) const
- bool **operator!=** (const CGR_TYPE t_) const
- bool **operator<** (const **cgrType** &q) const
- bool **operator<=** (const **cgrType** &q) const
- bool **operator>** (const **cgrType** &q) const
- bool **operator>=** (const **cgrType** &q) const
- bool **operator==** (const **cgrType** &q) const
- bool **operator!=** (const **cgrType** &q) const
- const char * **tostr** () const
- **wbstring** **toStr** () const

Public Attributes

- CGR_TYPE **t**

The documentation for this class was generated from the following file:

- clebsch.hh

4.13 Wb::citerator< T, T2 > Class Template Reference

Public Member Functions

- **citerator** (const T *p_)
- **citerator** & **operator++** ()
- bool **end** ()
- size_t **iter** () const
- size_t **numel** () const
- const T2 & **operator[]** (size_t k)
- const T2 & **at** ()

Public Attributes

- size_t **i**
- size_t **n**
- const T * **p**

The documentation for this class was generated from the following file:

- util.hh

4.14 Wb::citerator< T, T2 * > Class Template Reference

Public Member Functions

- **citerator** (const T *p_)
- **citerator** & **operator++** ()
- bool **end** ()
- size_t **iter** () const
- size_t **numel** () const
- const T2 * & **operator[]** (size_t k)
- const T2 * & **at** ()

Public Attributes

- size_t **i**
- size_t **n**
- const T * **p**

The documentation for this class was generated from the following file:

- util.hh

4.15 Wb::CleanUp Class Reference

Public Member Functions

- void **Check** ()

The documentation for this class was generated from the following file:

- wblib.h

4.16 Wb::Clock Class Reference

Public Member Functions

- **Clock** (const char *F, int L, char use_tag=1, unsigned u=0)
- **Clock** (const char *istr, char use_tag=0)
- **Clock** (const char *istr, char use_tag, unsigned u, [ClockSet](#) *gcs_=NULL, [Clock](#) **g=NULL, char rflag=1)
- void **done** ()
- void **init** ()
- void **reset** ()
- void **init** (const char *s, char use_tag=1)
- void **start** ()
- void **resume** ()
- int **stop** (const char *F=NULL, int L=0)
- double **gettime** (char cflag=0) const
- void **Add** (const [Clock](#) &t)
- int **Switch** (const char *istr, [Clock](#) **g=NULL, char rflag=1)
- mxArray * **toMx** (const char *F=NULL, int L=0) const
- mxArray * **toMxS** (const char *F=NULL, int L=0) const
- void **info** (const char *istr=NULL, char vflag=0) const

Public Attributes

- double **ttot**
- double **ctot**
- double **tneg**
- CHRONO_CLOCK **tref**
- clock_t **cref**
- size_t **nz**
- size_t **ncall**
- [wbstring](#) **name**
- unsigned **clk_flags**
- unsigned **user**
- [ClockSet](#) * **gcs**

The documentation for this class was generated from the following files:

- wbclock.hh
- wbclock.cc

4.17 Wb::ClockSet Class Reference

Public Member Functions

- void **init** (char vflag=1)
- int **info_u** (unsigned u, char rflag=1)
- int **reset_u** (unsigned u, char iflag=0)
- int **reset** (const char *istr, char uset_tag=1)
- unsigned **info** (char vflag=0) const
- [Clock](#) * **use** (const char *istr, char mode)
- [Clock](#) * **use** (const char *F, int L, char mode)
- [Clock](#) * **get** (const char *s, char use_tag=1)
- [Clock](#) * **insert** ([Wb::Clock](#) *clk, char lflag=0)
- int **erase** ([Wb::Clock](#) *clk)
- mxArray * **toMx** () const

Public Attributes

- map< std::string, [Wb::Clock](#) * > **buf**

The documentation for this class was generated from the following files:

- wbclock.hh
- wbclock.cc

4.18 Wb::counter Class Reference

Public Member Functions

- **counter** (const char *F, int L, const char *fmt_)
- [counter](#) & **operator++** ()

Public Attributes

- long **n**
- int **line**
- [wbstring](#) **file**
- [wbstring](#) **fmt**

The documentation for this class was generated from the following file:

- wbutil.hh

4.19 CPAT< TQ, TD > Class Template Reference

Public Member Functions

- **CPAT** (const [CPAT](#)< TQ, TD > &D)
- void **init** (TD oz, const mxArray **, unsigned n, unsigned ic1, unsigned r1, unsigned ic2, unsigned r2, const char twoSite, const char ldir)
- int **getCtrlIndex** (const [QSpace](#)< TQ, TD > &PSI)
- [QSpace](#)< TQ, TD > & **HZTimes** (const [QSpace](#)< TQ, TD > &PSI, [QSpace](#)< TQ, TD > &HZPsi, const char='N', char disp=1)
- [QSpace](#)< TQ, TD > & **HZApplyDiag** (const [QSpace](#)< TQ, TD > &PSI, [QSpace](#)< TQ, TD > &EPsi, TD a, TD b, char lflag=0, char disp=1)
- void **check_contraction** (const [QSpace](#)< TQ, TD > &PSI, char disp=1)
- void **initCounter** ()
- [QSpace](#)< TQ, TD > & **HZTimes_tst** (const [QSpace](#)< TQ, TD > &PSI, [QSpace](#)< TQ, TD > &HZPsi, const char='N', char disp=1)
- void **getQRange** (unsigned k, [wbMatrix](#)< TQ > &QQ, [wbvector](#)< unsigned > &SS, [QSpace](#)< TQ, TD > *B=NULL) const
- mxArray * **toMx** () const
- void **put** (const char *vname, const char *ws="base") const
- void **PermIC** (const [wbperm](#) &P)

Public Attributes

- [wbvector](#)< [wbvector](#)< [CPAT_Set](#)< TQ, TD > > > **CP**
- TD **z**
- unsigned **sameCtrlIdx**
- unsigned **nr_check_ctr**
- unsigned **HPsi_counter**

The documentation for this class was generated from the following file:

- cpat.hh

4.20 CPAT_Set< TQ, TD > Class Template Reference

Public Member Functions

- **CPAT_Set** (const [CPAT_Set](#)< TQ, TD > &D)
- int **init** (const mxArray *a, unsigned ic1, unsigned r1, unsigned ic2, unsigned r2, const char twoSite, const char ldir)
- int **getCtrlIndex** (const [QSpace](#)< TQ, TD > &PSI)
- int **getCtrlIndex** (const [wbvector](#)< [QSpace](#)< TQ, TD > > &, const char='N')
- int **getCtrlIndex** (const unsigned i, const [QSpace](#)< TQ, TD > &, char='N')
- void **contract** (const [QSpace](#)< TQ, TD > &PSI, [wbvector](#)< [QSpace](#)< TQ, TD > > &C, char='N') const
- void **contract** (const [wbvector](#)< [QSpace](#)< TQ, TD > > &PSI1, [wbvector](#)< [QSpace](#)< TQ, TD > > &C, char='N') const
- void **contract** (const unsigned k, const [QSpace](#)< TQ, TD > &PSI, [QSpace](#)< TQ, TD > &C, char='N') const
- void **contract_diag** (const [QSpace](#)< TQ, TD > &PSI, [wbvector](#)< [QSpace](#)< TQ, TD > > &C, TD a, TD b, char lflag) const
- void **contract_diag** (const [wbvector](#)< [QSpace](#)< TQ, TD > > &PSI1, [wbvector](#)< [QSpace](#)< TQ, TD > > &C, TD a, TD b, char lflag) const
- void **contract_diag** (const unsigned k, const [QSpace](#)< TQ, TD > &PSI, [QSpace](#)< TQ, TD > &C, TD a, TD b, char lflag) const
- mxArray * **toMx** () const

Public Attributes

- [wbvector](#)< [QSpace](#)< TQ, TD > > **A**
- char **opA**
- unsigned **ic**
- [wbperm](#) **P**
- [wbvector](#)< [QSpace](#)< TQ, TD > > **CQn**
- [wbvector](#)< [QSpace](#)< TQ, TD > > **CQt**
- [wbvector](#)< [wbMatrix](#)< unsigned > > **CIn**
- [wbvector](#)< [wbMatrix](#)< unsigned > > **CIt**

The documentation for this class was generated from the following file:

- cpat.hh

4.21 `cpu_time` Class Reference

Public Member Functions

- void **init** ()
- [cpu_time](#) **since** ()
- [wbstring](#) **toStr** (char flag='c')

Static Public Member Functions

- static double **time** (char flag)

Public Attributes

- double **tcpu**
- double **tsys**

The documentation for this class was generated from the following files:

- util.hh
- util.cc

4.22 CRef< TQ > Class Template Reference

Public Member Functions

- **CRef** (CGR_TYPE rt_=CGR_DEFAULT)
- **CRef** (const [CRef](#) &R)
- **CRef** (const CDATA_TQ &C, unsigned m=-1)
- **CRef & init** (CGR_TYPE rt_=CGR_DEFAULT)
- **CRef & init** (const [CRef](#) &R, char full=1)
- **CRef & operator=** (const [CRef](#) &R)
- void **swap** ([CRef](#) &R)
- **CRef & save2** ([CRef](#) &R)
- double **safeCpy** (const char *F, int L, const [CRef](#) &B, [CRef](#) &C) const
- **CRef & init_1** (const [CRef](#) &R, unsigned j)
- **CRef & init_wld** (const [CRef](#) &R, unsigned wid=1, unsigned l1=-1, const unsigned *d_=NULL)
- **CRef & init** (const char *F, int L, const mxArray *a, unsigned k, char refC=0, const [QSet](#)< TQ > *Q=NULL, char xflag=0)
- **CRef< TQ > & initIdentityR** (const char *F, int L, const [QType](#) &q, const TQ *qs, unsigned dim=-1, char xflag=0)
- **CRef< TQ > & initIdentity1J** (const char *F, int L, const [QType](#) &q, const TQ *qs, unsigned dim=-1, char xflag=0)
- **CRef< TQ > & Reduce2Identity** (char xflag=0)
- **wbvector< RTD > & trace** (const char *F, int L, **wbvector< RTD >** &cgt) const
- **wbvector< double > & trace** (const char *F, int L, **wbvector< double >** &cgt) const
- **wbvector< double > trace** (const char *F, int L) const
- double **trace** () const
- double **trace** (const char *F, int L, const [ctrldx](#) &i1, const [ctrldx](#) &i2, [CRef](#) *Rt=NULL) const
- **CRef & initAbelian** ()
- **CRef & initCtrScalar** (double x=1)
- **CRef & initBase** (const CDATA_TQ *cgr, unsigned m=-1, const CDATA_TQ *X=NULL)
- **CRef & initBase** (const CDATA_TQ &C, unsigned m=-1)
- int **LoadRef** (const char *F=0, int L=0, char force=1) const
- int **Reduce2Ref** (const char *F=0, int L=0, char force=0) const
- **operator bool** () const
- bool **operator!** () const
- bool **isEmpty** () const
- bool **isSymmetric** (const char *F=0, int L=0) const
- bool **isRefInit** (const char *F=0, int L=0, char check=1) const
- bool **isAbelian** (const char *F=NULL, int L=0) const
- bool **isScalar** (char dflag=0) const
- size_t **isw2** (const char *F=NULL, int L=0) const
- size_t **isw3** () const
- unsigned **Reduce_w3ld** (unsigned d=-1, char lflag=0)
- int **cgw_check** (const char *F, int L) const
- bool **cgw_exists** (unsigned i, unsigned j)
- int **cgw_check_std3** (const char *F=NULL, int L=-1, unsigned d3=0, unsigned im=0)
- int **checkQ** (const char *F, int L, const [QSet](#)< TQ > &Q) const
- unsigned **numel** () const
- unsigned **wnumel** () const
- bool **wscalar** (const char *F=NULL, int L=0) const
- bool **wscalar1** (const char *F=NULL, int L=0) const
- double **wel** (unsigned k) const
- bool **wOM** (const char *F=NULL, int L=0) const
- bool **wdim_is** (unsigned m) const

- unsigned **wdim** (const char *F=NULL, int L=1) const
- unsigned **wdim1** (const char *F=NULL, int L=0) const
- unsigned **wdim2** (const char *F=NULL, int L=0) const
- unsigned **wdim12_** (const char *F, int L, unsigned &d2) const
- unsigned **wdim12** (const char *F, int L, unsigned &d2, unsigned d0=1) const
- unsigned **wdim3** (const char *F=NULL, int L=0) const
- double **wget0** () const
- double **wget1** () const
- bool **wSame** (const [wbarray](#)< double > &cgw_, char lenient=0, double eps=1e-14) const
- template<class T >
 [wbvector](#)< T > & **getSize** ([wbvector](#)< T > &S, char bare=0) const
- unsigned **Size** (unsigned k, unsigned r=-1) const
- unsigned **qdim** (unsigned k, const [QType](#) *qt=NULL) const
- int **checkAbelian** (const char *F=NULL, int L=0) const
- bool **isDiagCSC** (RTD eps=1E-14) const
- bool **isIdentityCG** ([wbvector](#)< double > *nrm=NULL, double eps=1E-14) const
- int **wisld** () const
- double **norm2** (char checks=1) const
- double **NormSignW** (const char *F=NULL, int L=0, char useExt=1, double eps=CG_SKIP_DEPS1, double eps2=CG_SKIP_DEPS2)
- char **sameUptoFac** (const [CRef](#) &B, double *fac=NULL, double eps=1E-12) const
- char **sameAs** (const [CRef](#) &B, double eps=1E-12) const
- char **sameAs_fix** (const char *F, int L, [CRef](#) &B, double eps=1E-12)
- bool **sameQSet** (const [QSet](#)< TQ > &Q) const
- bool **sameQSet** (const [CRef](#) &B) const
- bool **sameQDir** (const char *F, int L, const [QDir](#) &qd) const
- bool **sameQDir** (const char *F, int L, const [CRef](#) &B) const
- bool **sameQDir** (const [iTags](#) &b) const
- int **gotSameCData** (const char *F, int L, const [CRef](#) &B, char xflag=0) const
- double **NormStd** (const char *F, int L, unsigned r=-1)
- double **normExt** (const char *F=NULL, int L=0) const
- int **SortDegQ** (const char *F=NULL, int L=0, [QSet](#)< TQ > *QS=NULL)
- bool **isSortedDegQ** ([QSet](#)< TQ > *QS, [wbperm](#) &pxt, char &cxt) const
- bool **isSortedDegQ** ([QSet](#)< TQ > *QS=NULL) const
- char **got3** (const char *F, int L, const [QType](#) &q, const [qset](#)< TQ > &J12, const [qset](#)< TQ > &J3) const
- bool **operator==** (const [CRef](#) &B)
- bool **operator!=** (const [CRef](#) &B)
- double **normDiff2** (const char *F, int L, const [CRef](#) &B) const
- double **normDiff** (const char *F, int L, const [CRef](#) &B) const
- [wbarray](#)< double > & **wProd** (const [CRef](#) &B, [wbarray](#)< double > &x) const
- int **check** (const char *F, int L) const
- int **check** (const char *F, int L, const [QType](#) &q, const [QDir](#) &qdir) const
- unsigned **getOM** (const char *F=NULL, int L=0, char wflag=0) const
- unsigned **checkOM** (const char *F=NULL, int L=0) const
- unsigned **rankS** (char lflag=0) const
- unsigned **rank** (const char *F=NULL, int L=0, char lflag=0) const
- [CRef](#) **wget** (unsigned j) const
- [CRef](#) & **Permute** (const [wbperm](#) &P, char iflag=0, char isnew=0)
- [CRef](#) & **permute** (const [wbperm](#) &P, [CRef](#) &B, char iflag=0, char isnew=0) const
- [CRef](#) & **Conj** ()
- int **HConjOpScalar** ()
- bool **gotConj** () const
- bool **gotPerm** (const char *F=NULL, int L=0) const
- bool **anyTrafo** (char pflag=1) const
- bool **anyTrafo** (char &ip, char &ic, char pflag=1) const

- bool **gotnoTrafos** () const
- unsigned **getP** (unsigned k) const
- **wbperm** & **getP** (**wbperm** &p, char iflag=0) const
- char **stat** () const
- bool **affectsQDir** () const
- **qset**< TQ > & **adapt** (**qset**< TQ > &qs, char iflag=0) const
- **QSet**< TQ > & **adapt** (**QSet**< TQ > &Q, char iflag=0) const
- **ctrlIdx** & **adapt** (**ctrlIdx** &l, char iflag=0) const
- **QDir** & **get_qdir** (**QDir** &qd) const
- **QDir** **get_qdir** () const
- **wbstring** **qdir2Str** (char vflag=0) const
- **wbstring** **toStr** (char lflag=0) const
- **wbstring** **sizeStr** () const
- **wbstring** **statStr** (char vflag=0) const
- **wbstring** **qStr** () const
- **wbstring** **QStr** () const
- mxArray * **toMx** (char flag=0) const
- mxArray * **toMX** () const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n, char flag=0) const
- void **add2MxStruct** (mxArray *S, unsigned i, char flag=0) const

Public Attributes

- const CDATA_TQ * **cgb**
- **wbarray**< double > **cgw**
- **wbperm** **cgp**
- char **conj**
- **cgrType** **rtype**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc
- clebsch_io.cc

4.23 CStore< TQ > Class Template Reference

Public Types

- typedef MAP32::iterator **iMAP32**
- typedef MAP31::iterator **iMAP31**

Public Member Functions

- void **clear** ()
- void **init** ()
- const CDATA_TQ & **getIdentityC** (const char *F, int L, const QType &t, const TQ *qs, unsigned dim=-1, unsigned loadRC=LB_GEN)
- const CDATA_TQ & **getIdentity1J** (const char *F, int L, CRef< TQ > &C, const QType &t, const TQ *qs, unsigned dim=-1, unsigned loadRC=LB_GEN)
- void **add_CData_abelian** (const QType &q, const TQ &J1, const TQ &J2)
- void **add_CData** (const QType &q, const qset< TQ > &J1, const qset< TQ > &J2)
- void **add2BUF** (const char *F, int L, const CDATA_TQ &S, char nflag=0)
- unsigned **add3** (const QType &q, const mxArray *S, char loadC=0)
- unsigned **add2BUF** (const QType &q, const mxArray *S, unsigned k=-1)
- CDATA_TQ & **getBUF** (const char *F, int L, const QSet< TQ > &Q, unsigned loadRC=0)
- size_t **reduceMemUsage** (const char *F=NULL, int L=0, double mfac=-1)
- void **getQfinal_1** (const char *F, int L, const QType &type, const qset< TQ > &J1, const qset< TQ > &J2, wbMatrix< TQ > &J, wbMatrix< const CRef< TQ > * > *ss=NULL, char loadC=0)
- void **getQfinal_v** (const char *F, int L, const QVec &type, const qset< TQ > &q1, const qset< TQ > &q2, wbMatrix< TQ > &QQ, wbMatrix< const CRef< TQ > * > *SS=NULL, char loadC=0)
- int **getQfinal_zdim** (const char *F, int L, const QVec &qvec, const wbMatrix< TQ > &Q1, const wbMatrix< TQ > &Q2, const wbMatrix< TQ > &Q, wbvector< widx_t > &s1, wbvector< widx_t > &s2, wbvector< widx_t > &s, wbvector< unsigned > *m=NULL, wbMatrix< const CRef< TQ > * > *S3=NULL)
- void **getQfinal** (const char *F, int L, const QVec &type, const wbMatrix< TQ > &Q1, const wbMatrix< TQ > &Q2, QMap< TQ > &M, unsigned char cgflag=0)
- void **Info** (const char *F=0, int L=0, char vflag=0) const
- int **checkInit** (const char *F, int L, const QType &t) const
- unsigned **LoadStore** (const char *F, int L, const char *file)
- mxArray * **toMx** (const QType &q=QTYPE_UNKNOWN) const
- void **put** (const char *F, int L, const char *vname, const char *ws="caller") const
- void **put** (const char *vname, const char *ws="caller") const
- int **find_map3** (const QType &q, const qset< TQ > &J1, const qset< TQ > &J2) const
- int **find_map3** (const QType &q, const qset< TQ > &J12) const
- int **valid_mp3_data** (const QType &t, const qset< TQ > &J1, const qset< TQ > &J2, const qset< TQ > &J3, char rflag=0) const
- CDATA_TQ * **BUF_find** (const QSet< TQ > &Q)
- int **Reduce2Ref** (const char *F, int L, const QSet< TQ > &Q, char force=0, unsigned Dmin=1024)

Public Attributes

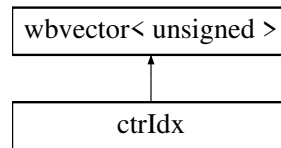
- map< QType, map< qset< TQ >, map< qset< TQ >, CRef< TQ > > > > **map3**
- unordered_map< QSet< TQ >, CDATA_TQ, QHash< TQ > > **BUF**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch_aux.cc
- clebsch_io.cc
- clebsch_old.cc

4.24 ctrIdx Class Reference

Inheritance diagram for ctrIdx:



Public Member Functions

- **ctrIdx** (unsigned l, unsigned *d, char c=0)
- **ctrIdx** (const char *F, int L, const char *s, char c=0)
- **ctrIdx** (const char *F, int L, const mxArray *a)
- **ctrIdx** (const [ctrIdx](#) &l)
- **ctrIdx** (const [wbvector](#)< unsigned > &l, char c=0)
- **ctrIdx & init** (unsigned l=0, unsigned *d=NULL, char c=0)
- template<class T >
 ctrIdx & initT (unsigned l=0, T *d=NULL, char c=0)
- **ctrIdx & init** (const [wbvector](#)< unsigned > &l, char c=0)
- **ctrIdx & init1** (unsigned k, char c=0)
- **ctrIdx & init2** (unsigned k1, unsigned k2, char c=0)
- template<class TQ, class TD >
 ctrIdx & initOp (const char *F, int L, [itag_](#) t1, const [QSpace](#)< TQ, TD > &B, const char *sopt)
- int **init** (const char *F, int L, const char *s, unsigned offset=1)
- **ctrIdx & init** (const char *F, int L, const mxArray *a)
- **ctrIdx & init** (const mxArray *a)
- **ctrIdx & Index** (unsigned l, char c=0)
- int **checkUniqueS** (const char *F=0, int L=0, const unsigned r=-1) const
- **ctrIdx & Sort** ([wbperm](#) &P, unsigned r=-1)
- **ctrIdx & sort** ([ctrIdx](#) &l, [wbperm](#) &P, unsigned r=-1) const
- **ctrIdx & Sort_** ([ctrIdx](#) &J, unsigned r=-1)
- **ctrIdx & invert** (unsigned r, [ctrIdx](#) &l) const
- **ctrIdx & Invert** (unsigned r)
- **ctrIdx & Conj** ()
- int **extend2Perm** (widx_t N, [wbperm](#) &P, char toend=0) const
- [wbperm](#) & **extend2Perm** (unsigned ra, unsigned ma, const [ctrIdx](#) &icb, unsigned rb, unsigned mb, const [wbperm](#) &pab, char conj, unsigned mc, [wbperm](#) &Pbc, [ctrIdx](#) &kb, [ctrIdx](#) &kcb, char Bflag=1) const
- **ctrIdx & save2** ([ctrIdx](#) &J)
- mxArray * **toMx** () const
- [wbstring](#) **toStr** (char xflag=0) const

Public Member Functions inherited from [wbvector< unsigned >](#)

- **wbvector** (size_t l, unsigned x)
- **wbvector** (size_t l, const unsigned *d0=NULL, char ref=0)
- **wbvector** (const [wbvector](#) &v)
- **wbvector** (const [wbvector](#)< T2 > &v)
- **wbvector** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- **wbvector** (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- **wbvector** (const mxArray *a, char check_type=1, char ref=0)

- [wbvector](#) & **init** (size_t l=0)
- [wbvector](#) & **init** (size_t l, const unsigned *d0)
- [wbvector](#) & **init** (size_t l, const unsigned *d0, char ref)
- [wbvector](#) & **init** (const [wbvector](#) &v)
- [wbvector](#) & **init** (size_t n, const [wbvector](#) &v)
- [wbvector](#)< unsigned > & **init** (const [wbsparray](#)< unsigned > &S)
- [wbvector](#) & **init** (size_t l1, const unsigned *d1, size_t l2, const unsigned *d2)
- int **init** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type QS_UNUSED_↵
VAR=1, char ref=0)
- int **init** (const mxArray *a, const char *istr=NULL, char check_type=1)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- [wbvector](#) & **initp** (size_t l, const unsigned *d0, size_t n, char init=1)
- [wbvector](#) & **init2val** (size_t l, const unsigned &x)
- [wbvector](#) & **initStride** (size_t l, const unsigned *d0, size_t stride)
- [wbvector](#) & **init_stride** (const unsigned *d0, size_t n, size_t m, size_t D0)
- [wbvector](#) & **init_data** (const unsigned *d0)
- [wbvector](#) & **init_data** (const T0 *d0)
- [wbvector](#) & **init_data** (const char *F, int L, const T0 *d0)
- [wbvector](#)< unsigned > & **initT** (const [wbsparray](#)< T2 > &S)
- [wbvector](#)< unsigned > & **initT** (size_t l, const T2 *v)
- [wbvector](#)< unsigned > & **initT** (const [wbvector](#)< T2 > &v)
- [wbvector](#)< unsigned > & **initT** (const char *F, int L, const [wbvector](#)< T2 > &v)
- [wbvector](#)< unsigned > & **initT** (const [wbsparray](#)< TD > &S)
- [wbvector](#)< unsigned > & **initT_** (const [wbvector](#)< T2 > &v)
- [wbvector](#) & **init2ref** (const [wbvector](#) &v)
- [wbvector](#) & **init2ref** (size_t l, const unsigned *d0)
- [wbvector](#) & **unRef** ()
- int **init_Struct** (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)
- int **init_base** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)
- int **init_mpf** (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)
- void **initDef** (size_t n)
- void **initDef** (const [wbvector](#) &v)
- [wbvector](#) & **operator=** (const [wbvector](#) &v)
- [wbvector](#)< unsigned > & **operator=** (const [wbvector](#)< TB > &v)
- [wbvector](#) & **set** (const unsigned &x)
- [wbvector](#) & **set** (const [wbvector](#)< size_t > &l, const unsigned &x)
- void **set** (size_t offset, const unsigned *x, size_t n)
- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- [wbvector](#) & **setMin** (const unsigned &x)
- [wbvector](#) & **setMax** (const unsigned &x)
- [wbvector](#) & **add** (const [wbvector](#)< size_t > &l, const unsigned &x)
- [wbvector](#) & **add** (const unsigned *d)
- [wbvector](#) & **Set** (const [wbvector](#)< size_t > &l, const [wbvector](#) &v)
- void **Sort** ([wbperm](#) &p, char dir=+1)
- void **Sort** (char dir=+1)
- void **sort** ([wbvector](#) &v2, [wbperm](#) &p) const
- void **sort** ([wbvector](#) &v2) const
- [wbperm](#) & **pSort** ([wbperm](#) &p) const
- bool **isSorted** () const

- bool **isSortedU** () const
- bool **isUnique** () const
- int **contains** (const unsigned &x) const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isFinite** () const
- size_t **find1** (const unsigned &x) const
- size_t **find1** () const
- size_t **count** (const unsigned &x) const
- size_t **icount** (const unsigned &x) const
- size_t **nnz** () const
- size_t **nnz** (unsigned eps) const
- size_t **numZeros** (unsigned eps=unsigned(1E-14)) const
- size_t **numEQ** (const unsigned &) const
- size_t **numGT** (const unsigned &) const
- size_t **numLT** (const unsigned &) const
- [wbindex](#) & **find** (const unsigned &x, [wbindex](#) &l, char iflag=0) const
- [wbindex](#) & **find** (const unsigned &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **find** ([wbindex](#) &l) const
- [wbindex](#) & **find** ([wbindex](#) &l, [wbindex](#) &lx) const
- [wbvector](#)< size_t > **find** () const
- [wbindex](#) & **findGT** (const unsigned &x, [wbindex](#) &l, char iflag=0) const
- [wbindex](#) & **findGT** (const unsigned &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **findzeros** ([wbindex](#) &l) const
- [wbindex](#) & **findRange** (const unsigned &x1, const unsigned &x2, [wbindex](#) &l, const char *w=NULL) const
- int **findClosestSorted** (unsigned r) const
- int **findClosest** (unsigned r) const
- size_t **findValues** (const [wbvector](#) &v, [wbindex](#) &l) const
- [wbvector](#) & **setp** (const unsigned *x)
- void **reset** ()
- void **cpy** (const unsigned *v)
- void **cpy** (const unsigned *x, size_t m)
- [wbvector](#) & **save2** ([wbvector](#) &v)
- void **swap** ([wbvector](#) &v)
- [wbvector](#) & **swap** (size_t i, size_t j)
- [wbvector](#) & **Collect** (unsigned &A1, unsigned &A2)
- [wbvector](#) & **Collect** (unsigned &A1, unsigned &A2, unsigned &A3)
- [wbvector](#) & **Skip** (size_t)
- [wbvector](#) & **Skip** (const WBINDEX &, char rflag=0)
- [wbvector](#) & **Move2_FE** (const WBINDEX &l, char FE_flag)
- [wbvector](#) & **Move2front** (const WBINDEX &l)
- [wbvector](#) & **Move2front** (size_t k)
- [wbvector](#) & **Move2end** (const WBINDEX &l)
- [wbvector](#) & **Move2end** (size_t k)
- [wbvector](#) & **mvLastTo** (size_t k)
- [wbvector](#) & **mvFirstTo** (size_t k)
- bool **checkSameLength** (const [wbvector](#) &a, const char *F=NULL, int L=0, const char *istr="") const
- bool **checkSameLength** (const [wbvector](#) &a, const [wbvector](#) &b, const char *F=NULL, int L=0, const char *istr="") const

- void **operator+=** (const T_ &a)
- void **operator+=** (const [wbvector](#) &b)
- void **operator-=** (const T_ &a)
- void **operator-=** (const [wbvector](#) &b)
- [wbvector](#) & **operator*=** (const T_ &a)
- [wbvector](#) & **operator/=** (const T_ &a)
- [wbvector](#) **operator*** (const T_ &a) const
- [wbvector](#) **operator+** (const T_ &a) const
- [wbvector](#) **operator+** (const [wbvector](#) &b) const
- [wbvector](#) **operator-** (const T_ &a) const
- [wbvector](#) **operator-** (const [wbvector](#) &b) const
- [wbvector](#) & **Plus** (const unsigned *b)
- [wbvector](#) & **Plus** (const unsigned *b, unsigned bfac)
- [wbvector](#) & **Plus** (const [wbvector](#) &b, unsigned bfac=1)
- void **plus** (const [wbvector](#) &b, [wbvector](#) &c, unsigned bfac=1, unsigned cfac=0) const
- [wbvector](#) & **times** (unsigned fac, [wbvector](#) &x) const
- [wbvector](#) & **RevertSigns** ()
- int **isum** () const
- unsigned **sum** () const
- unsigned **sum** (size_t i1, const char *estr) const
- unsigned **sum** (size_t i1, size_t i2) const
- unsigned **cumsum_** ([wbvector](#) &S, char xflag=0) const
- unsigned **Cumsum** (char zflag=0) const
- unsigned **cumsum0prod** ([wbvector](#)< T2 > &b, [wbvector](#)< unsigned > &cs, char xflag) const
- [wbvector](#) & **Eldiv** (const [wbvector](#) &v) const
- [wbvector](#) & **tensorProd** (const [wbvector](#) &v2, [wbvector](#) &vv) const
- unsigned **scalarProd** (const [wbvector](#) &v) const
- unsigned **scalarProd** (const unsigned *v) const
- unsigned **norm** () const
- unsigned **norm2** () const
- double **normDiff** (const [wbvector](#) &v, unsigned fac=1) const
- unsigned **normDiff2** (const [wbvector](#) &v, unsigned fac=1) const
- double **normDiff2_** (const [wbvector](#) &v, double fac=1) const
- unsigned **GSProject** (const unsigned *v, char isnorm=0, char tflag=0) const
- [wbvector](#) & **Normalize** (const char *F=NULL, int L=0)
- [wbvector](#) & **subtract** (const unsigned *d)
- void **getReal** ([wbvector](#)< double > &R) const
- void **getReal** ([wbvector](#)< double > &R) const
- void **getImag** ([wbvector](#)< double > &I) const
- void **getImag** ([wbvector](#)< double > &I) const
- [wbvector](#) & **Conj** ()
- [wbvector](#)< [wbcomplex](#) > & **Conj** ()
- [wbvector](#) & **applyFlag** (char &flag, const unsigned &fac, [wbvector](#) &a)
- bool **allEqual** () const
- bool **allEqual** (const unsigned &a) const
- bool **allZero** () const
- bool **allOnes** () const
- bool **allEqual2** (const unsigned &a, unsigned eps=1E-15) const
- bool **allUnequal** (const unsigned &a) const
- bool **any** () const
- bool **anyUnequal** (const unsigned &a) const
- bool **anyUnequal** (const [wbvector](#)< size_t > &I, const unsigned &x, size_t *k=NULL) const
- bool **anyEqual** (const unsigned &a, size_t *i1=NULL) const
- bool **anyEqual** (const [wbvector](#)< size_t > &I, const unsigned &x, size_t *k=NULL) const
- bool **anyGT** (const unsigned &a) const

- bool **anyGE** (const unsigned &a) const
- bool **anyLT** (const unsigned &a) const
- bool **anyLE** (const unsigned &a) const
- bool **allGT** (const unsigned &a) const
- bool **allGE** (const unsigned &a) const
- bool **allLE** (const unsigned &a) const
- bool **allLT** (const unsigned &a) const
- bool **allIn** (const unsigned &a1, const unsigned &a2) const
- bool **anySmallVals** (const unsigned eps=1E-15) const
- bool **isEqual** (const unsigned *v) const
- bool **isEqual** (const [wbvector](#) &v) const
- bool **isEmpty** () const
- **operator bool** () const
- bool **operator!** () const
- bool **operator==** (const [wbvector](#) &v) const
- bool **operator!=** (const [wbvector](#) &v) const
- bool **isEqualR** (const [wbvector](#) &v) const
- bool **operator^** (const [wbvector](#) &v) const
- bool **deepEqualP** (const [wbvector](#) &B) const
- char **cmp** (const [wbvector](#) &v) const
- bool **operator<** (const [wbvector](#) &v) const
- int **operator<** (const unsigned &r) const
- bool **operator<=** (const [wbvector](#) &v) const
- bool **operator>** (const [wbvector](#) &v) const
- bool **operator>=** (const [wbvector](#) &v) const
- unsigned & **el** (size_t i) const
- unsigned **elx** (size_t i, const unsigned &x) const
- const unsigned & **operator[]** (size_t i) const
- unsigned & **operator[]** (size_t i)
- [wbvector](#) **operator[]** (const [wbvector](#) < size_t > &l) const
- unsigned & **last** () const
- unsigned & **last** (size_t &i) const
- unsigned & **last** (T1 &i_) const
- unsigned & **p2last** () const
- unsigned * **ref** (size_t i) const
- [wbvector](#) & **setRand** (double fac=1., double shift=0.)
- [wbvector](#) < [wbcomplex](#) > & **setRand** (double fac, double shift)
- void **setIdx** (double shift=0., double fac=1.)
- int **set2Group** (const [wbperm](#) &P, const WBINDEX &D, const unsigned *S0=NULL, char iflag=0)
- [wbvector](#) & **Resize_** (size_t n)
- [wbvector](#) & **Shorten2** (size_t n)
- [wbvector](#) & **Resize** (size_t n, const unsigned *d=NULL)
- [wbvector](#) & **resize** (size_t n, [wbvector](#) &X) const
- [wbvector](#) & **cat** (const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#) < [wbvector](#) > &vv)
- [wbvector](#) & **Cat** (const [wbvector](#) < [wbvector](#) * > &vv)
- [wbvector](#) & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)
- [wbvector](#) & **Cat** (const [wbvector](#) < T1 > *v1, const [wbvector](#) < T2 > *v2, const [wbvector](#) < T3 > *v3=0, const [wbvector](#) < T3 > *v4=0)
- [wbvector](#) < unsigned > & **Cat** (const [wbvector](#) < T1 > *v1, const [wbvector](#) < T2 > *v2, const [wbvector](#) < T3 > *v3, const [wbvector](#) < T3 > *v4)
- [wbvector](#) < unsigned > & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)

- [wbvector](#) & **Append** (const [wbvector](#) &v2)
- [wbvector](#) & **Append** (const unsigned &x)
- [wbvector](#) & **Append** (size_t d2, const unsigned *x)
- [wbvector](#) **append** (const [wbvector](#) &v) const
- [wbvector](#) & **append** (const [wbvector](#) &v2, [wbvector](#) &vout) const
- void **append** (const unsigned *x, size_t d2, [wbvector](#) &vout) const
- int **recLess** (const unsigned *v) const
- int **recLessE** (const unsigned *v) const
- int **recCompare** (const unsigned *v) const
- unsigned **max** () const
- unsigned **max** (size_t *k) const
- unsigned **max** (unsigned &k) const
- unsigned **max_** (unsigned x) const
- unsigned **min** () const
- unsigned **min** (size_t *k) const
- unsigned **min** (unsigned &k) const
- unsigned **min_** (unsigned x) const
- unsigned **aMax** (size_t *k=NULL) const
- unsigned **aMax** (unsigned &k) const
- unsigned **aMin** (char zflag=0, size_t *k=NULL) const
- unsigned **maxneg** (size_t *k=NULL) const
- unsigned **minpos** (size_t *k=NULL) const
- unsigned **avg** () const
- unsigned **std** () const
- void **range** (unsigned &xmin, unsigned &xmax) const
- unsigned **range** () const
- [wbvector](#) & **Permute** (const [wbperm](#) &P, char iflag=0)
- [wbvector](#) & **permute** ([wbvector](#) &, const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) **permute** (const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) & **blockPermute** (const [wbperm](#) &P, [wbvector](#) &, char iflag=0) const
- [wbvector](#) & **BlockPermute** (const [wbperm](#) &P, char iflag=0)
- int **blockCompare** (const [wbperm](#) &pa, const [wbvector](#) &b, const [wbperm](#) &pb) const
- void **get** (const WBINDEX &I0, [wbvector](#) &v) const
- void **get** (const [wbvector](#)< size_t > &I, unsigned *v) const
- void **get** (size_t i1, size_t i2, [wbvector](#) &v) const
- void **selectSU** (const WBINDEX &I)
- void **select** (const WBINDEX &I, unsigned *r) const
- [wbvector](#) & **select** (const WBINDEX &I, [wbvector](#) &a) const
- [wbvector](#) **select** (const WBINDEX &I) const
- [wbvector](#) & **Select** (const WBINDEX &I)
- [wbvector](#) & **BlockSelect** (const WBINDEX &I, const WBINDEX &D)
- [wbvector](#) & **getI** (size_t k, [wbvector](#) &v) const
- [wbvector](#) & **getI** (const [wbindex](#) &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **getI** (const WBINDEX &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **initI** (const [wbvector](#) &a, const [wbindex](#) &ia)
- [wbvector](#) & **initI** (const [wbvector](#) &a, const [wbindex](#) &ia, const [wbvector](#) &b, const [wbindex](#) &ib, char uflag=1)
- [wbvector](#) & **Flip** ()
- void **flip** ([wbvector](#) &a) const
- [wbvector](#) **flip** () const
- unsigned **prod** () const
- unsigned **prod** (unsigned x) const
- unsigned **prod** (size_t i, size_t i2) const
- unsigned **prod** (size_t i, size_t i2, unsigned x) const
- unsigned **prod** (const T1 *I, size_t n) const
- T2 **prod** (const T1 *I, size_t n, T2 &s2) const

- void **prod2** (unsigned &p1, unsigned &p2, const unsigned &p0=0) const
- void **info** (const char *istr) const
- void **print** (const char *istr, char mflag=0) const
- void **printdata** (const char *istr=0) const
- void **put** (const char *F, int L, const char *vname="ans", char tflag=0, const char *ws="base") const
- void **put** (const char *vname="ans", char tflag=0, const char *ws="base") const
- void **dispWithMsg** (const char *file, int line, const char *msg, const char *vname="") const
- void **mat2mx** (mxArray *&a, char dim=2) const
- void **mat2mxs** (mxArray *a, char field_nr, char dim=2) const
- mxArray * **toMx_Struct** (const char tflag=0) const
- mxArray * **toMxP_S** (const char tflag=0) const
- mxArray * **toMx** (const char tflag=0) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMxP** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base_d** (const char tflag=0) const
- mxArray * **toMx_offset** (const unsigned &offset, const char tflag=0) const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tflag=0) const
- void **add2MxStruct** (mxArray *S, const char *vname, char tflag=0) const
- mxArray * **mxCreateCell** (unsigned m, unsigned n) const
- void **add2MxCell** (mxArray *S, unsigned i, char tflag=0) const
- void **putg** (const char *vname="ans", char tflag=0) const
- void **putx** (const char *vname="ans", char tflag=0) const
- **wbstring toStr** (int n, const char *sep) const
- **wbstring toStr** () const
- **wbstring toStr** (int n, const char *sep) const
- **wbstring sizeStr** () const
- **wbstring toStrf** (const char *fmt0="", const char *sep=" ", unsigned stride=0, const char *sep2="") const
- **wbstring toStrf** (const char *fmt , const char *sep, unsigned stride, const char *sep2) const

Public Attributes

- char **conj**
- **iTags** newtags

Public Attributes inherited from `wbvector< unsigned >`

- unsigned * **data**
- size_t **len**
- char **isref**

Additional Inherited Members

Protected Member Functions inherited from [wbvector< unsigned >](#)

- [wbvector](#) & **RENEW** (size_t n, const unsigned *d=NULL, size_t l=-1, char init=1)
- [wbvector](#) & **RENEW_VAL** (size_t n, const unsigned &x)

The documentation for this class was generated from the following files:

- wbindex.hh
- wbindex.cc

4.25 CG::FileLock Class Reference

Public Member Functions

- template<class TQ >
FileLock (const [QType](#) &q_, const [qset](#)< TQ > &J1, const [qset](#)< TQ > &J2, const char *ext="mp3")
- template<class TQ >
int **init** (const [QType](#) &q_, const [qset](#)< TQ > &J1, const [qset](#)< TQ > &J2, const char *ext="mp3")
- int **GetLock** (const char *F=NULL, int L=0, char flag='W')
- int **ReleaseLock** (const char *F=NULL, int L=0)
- int **ensureFOpen** (const char *F=NULL, int L=0) const

Public Attributes

- int **fid**
- [QType](#) **q**
- [wbstring](#) **fname**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch_aux.cc
- clebsch_io.cc

4.26 genRG_base< TQ, TD > Class Template Reference

Public Member Functions

- **genRG_base** (const [genRG_base](#) &S)
- bool **isEmpty** () const
- double **normDiff** (const char *F, int L, const [genRG_base](#) &B, TD eps=1E12) const
- void **checkOrthoCommRel** (const char *F, int L, const [genRG_base](#) &B) const
- [genRG_base](#) & **Sort** ()
- double **SkipTiny** (const char *F=0, int L=0)
- [genRG_base](#) & **ApplyQFac** (const [wbvector](#)< double > &qfac, const [wbarray](#)< double > *JM=NULL)
- [genRG_base](#) & **init** ()
- [genRG_base](#) & **init** (const [genRG_base](#) &S)
- [genRG_base](#) & **operator=** (const [genRG_base](#) &S)
- SPIDX_T **dim** () const
- void **compareStdSU2** (const char *F=NULL, int L=0) const
- [genRG_base](#) & **save2** ([genRG_base](#) &S)
- [wbstring](#) **info** () const
- mxArray * **toMx** () const
- size_t **memSize** () const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i) const

Public Attributes

- [QType](#) **q**
- [wbvector](#)< [wbSpararrayTD](#) > **Sp**
- [wbvector](#)< [wbSpararrayTD](#) > **Sz**
- [qset](#)< TQ > **J**
- [wbMatrix](#)< double > **Z**
- double **err**
- [wbstring](#) **istr**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc
- clebsch_aux.cc
- clebsch_io.cc

4.27 genRG_struct< TQ, TD > Class Template Reference**Public Member Functions**

- [genRG_struct](#) & **checkInit** (const char *F, int L, const [QType](#) &q0)
- [genRG_struct](#) & **SetupSym** (const char *F, int L, const [QType](#) &q0, [qset](#)< TQ > *qs=NULL)
- [genRG_struct](#) & **Setup_SUN** (const char *F, int L, [qset](#)< TQ > *qs=NULL)
- [genRG_struct](#) & **Setup_SpN** (const char *F, int L, [qset](#)< TQ > *qs=NULL)
- [genRG_struct](#) & **Setup_SON** (const char *F, int L, [qset](#)< TQ > *qs=NULL)
- [genRG_struct](#) & **Setup_SEN** (const char *F, int L, [qset](#)< TQ > *qs=NULL)
- void **initSU2** (const [qset](#)< TQ > &J)
- void **initCommRel** (const char *F, int L, const [genRG_base](#)< TQ, TD > &R, char vflag=0)
- double **checkCommRel** (const char *F, int L, const [genRG_base](#)< TQ, TD > &R) const
- void **printStatus** (const char *F, int L, const char *istr=NULL)
- int **get1J_gen** (const char *F, int L, const [qset](#)< TQ > &q, char flag=0)
- int **getTensorProdReps_gen** (const [qset](#)< TQ > &q1, const [qset](#)< TQ > &q2, unsigned flag=TP3_LDM)
- void **getTensorProdReps** (const [qset](#)< TQ > &q1, const [qset](#)< TQ > &q2)
- unsigned **genTensorProds** (const [qset](#)< TQ > &q, unsigned flag=0)
- unsigned **genTensorProds** (unsigned dmax=-1, char sdig=0, char vflag=0)
- void **put** (const char *vname, const char *ws="caller") const
- [mxArray](#) * **toMx** (char bflag=1) const
- void **put** (const char *F, int L, const char *vname, const char *ws="caller") const
- void **wblog_RC_1st_build** (const char *F, int L, const char *fct, int nrep) const

Public Attributes

- [QType](#) **q**
- map< [qset](#)< TQ >, [genRG_base](#)< TQ, TD > > **RSet**
- [qset](#)< TQ > **qdef**
- [wbvector](#)< [pairPatternCG](#)< double > > **CR**
- [wbMatrix](#)< double > **DZ**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc
- clebsch_io.cc

4.28 Wb::gpara__ Class Reference

Public Member Functions

- void **init** (char force=0)

The documentation for this class was generated from the following file:

- wblib.h

4.29 groupIndex< T > Class Template Reference

Public Member Functions

- **groupIndex** (const [wbvector](#)< T > &P_, const [wbvector](#)< T > &D_, const wperm_t **p=NULL)
- **groupIndex** (const [wbvector](#)< T > &DI, T ng=-1)
- void **init** ()
- **groupIndex** & **init** (const [wbvector](#)< T > &P_, const [wbvector](#)< T > &D_)
- **groupIndex** & **initX** ([wbvector](#)< wperm_t > &P_, [wbvector](#)< T > &D_, const wperm_t **p=NULL)
- **groupIndex** & **init** (const [wbvector](#)< T > &DI, T ng=-1)
- **groupIndex** & **init** (T ng)
- **groupIndex** & **init_g** (T ig, T nz)
- **groupIndex** & **setD** (T i, T nz)
- T **init_ic** ()
- **groupIndex** & **setup** (char full=0)
- T **dim** (T ig)
- T **numGroups** () const
- void **getGroup** (T ig, T *p0, T &d) const
- T **getGroup** (T ig, T &j, T &j2) const
- T **getGroup** (T ig) const
- template<class T2 >
void **getRecs0** (const [wbMatrix](#)< T2 > &, [wbMatrix](#)< T2 > &, T m) const
- T **groupIter** (T &ig, T &j, T &j2) const
- mxArray * **toMx** () const

Public Attributes

- [wbvector](#)< T > **D**
- [wbperm](#) **P**
- [wbvector](#)< T > **I0**
- [wbvector](#)< T > **Ig**

The documentation for this class was generated from the following files:

- wbindex.hh
- wbindex.cc

4.30 icFlags Class Reference

Public Member Functions

- **icFlags** (const mxArray *ain=NULL)
- **icFlags** & **checkQSpace** (const mxArray *a)
- char **check_arg** (const mxArray *ain)
- **icFlags** & **init** (const mxArray *ain)
- char **set** (const mxArray *a, const char *istr="")
- template<class TQ, class TD >
void **apply** (**QSpace**< TQ, TD > &X, char vflag=1)

Public Attributes

- const mxArray * **a**
- char **conj**
- char **xflag**
- **wbstring** **otags**
- **wbstring** **regex_r**
- **wbstring** **sidx**

The documentation for this class was generated from the following file:

- contractQS.cc

4.31 indexSparseRef< TD > Class Template Reference

Public Member Functions

- **indexSparseRef** (const **wbsparray**< TD > &A_, unsigned M2=1)
- **indexSparseRef** (const char *F, int L, const **wbsparray**< TD > &A_, unsigned M2=1)
- **indexSparseRef** & **init** (const char *F, int L, const **wbsparray**< TD > &A_, unsigned M2=1)
- SPIDX_T **numCols** () const
- SPIDX_T **numRows** () const
- SPIDX_T **nnz** (SPIDX_T k) const
- TD **LContractVec** (const **wbsparray**< TD > &b, SPIDX_T k, char tnorm=0)
- mxArray * **toMx** () const

Public Attributes

- unsigned **M**
- **wbvector**< SPIDX_T > **JA**
- **wbvector**< sSPIDX_T > **JJ**
- const **wbsparray**< TD > * **A**

The documentation for this class was generated from the following file:

- wbsparray.hh

4.32 Wb::IOstat Class Reference

Public Member Functions

- [IOstat](#) & **init** ()
- void **gotread** (size_t s=0)
- void **gotwrite** (size_t s=0)
- bool **isEmpty** ()
- void **print** (const char *istr, const char *hstr=0, const char *xstr1=0, const char *xstr2=0)

Public Attributes

- unsigned **nread**
- unsigned **nwrite**
- size_t **sread**
- size_t **swrite**
- size_t **aux1**
- size_t **aux2**

The documentation for this class was generated from the following files:

- util.hh
- util.cc

4.33 is_pointer_< T > Class Template Reference

Public Member Functions

- char **q** ()

The documentation for this class was generated from the following file:

- wblibx.h

4.34 is_pointer_< T * > Class Template Reference

Public Member Functions

- char **q** ()

The documentation for this class was generated from the following file:

- wblibx.h

4.35 `is_pointer_< T ** >` Class Template Reference

Public Member Functions

- `char q ()`

The documentation for this class was generated from the following file:

- `wblibx.h`

4.36 `is_pointer_< T *** >` Class Template Reference

Public Member Functions

- `char q ()`

The documentation for this class was generated from the following file:

- `wblibx.h`

4.37 `itag_` Class Reference

Public Member Functions

- `itag_ (const char *F, int L, const char *s)`
- `itag_ (const char *s)`
- `itag_ (const itag_ &x)`
- `itag_ & init (const char *F, int L, const char *s, unsigned n=-99)`
- `itag_ & init (const char *s=NULL)`
- `itag_ & init_tag ()`
- `itag_ & init_qdir (char c)`
- `template<class TQ, class TD > itag_ & init (const char *F, int L, const wvector< QSpace< TQ, TD > > &A, windex ia)`
- `itag_ & init (const char *F, int L, const mxArray *a)`
- `itag_ & operator= (unsigned n)`
- `itag_ & operator= (const char *s)`
- `itag_ & operator= (const itag_ &x)`
- `bool operator== (const itag_ &x) const`
- `bool operator!= (const itag_ &x) const`
- `char sameAs (const itag_ &x, char lflag=0) const`
- `bool isConj (const itag_ &x, char full=1) const`
- `bool isConj_x (const itag_ &x, char lflag) const`
- `int isValid () const`
- `bool isEmpty () const`
- `bool isEmpty (char lenient) const`
- `bool isEmpty_m () const`
- `char getMark () const`
- `operator bool () const`

- bool **operator!** () const
- **operator const char *** () const
- [itag_](#) & **Set** (const char *s)
- [itag_](#) & **tSet** (const [itag_](#) &x)
- [itag_](#) & **Set_** (const [itag_](#) &x, char keep=0)
- [itag_](#) & **Conj** ()
- [itag_](#) & **setConj** ()
- bool **isConj** () const
- [itag_](#) & **deConj** ()
- [itag_](#) **conj** () const
- [itag_](#) **deconj** () const
- [itag_](#) & **SetFlag** (unsigned l)
- [itag_](#) & **SetFlags** (unsigned l=-1, char check=1)
- [itag_](#) & **UnsetFlag** (unsigned l)
- [itag_](#) & **UnsetFlags** ()
- bool **GotFlag** (unsigned l) const
- unsigned **GotFlags** (unsigned l=1) const
- [itag_](#) & **SetSI** (const char *F, int L, const char *tag, unsigned k, char mflag=6, char nfmt=1)
- [itag_](#) & **AppendChar** (char q, const char *qs=NULL)
- [itag_](#) & **PrependChar** (char q, const char *qs=NULL)
- int **CheckFirstChar** (char q, const char *qs=NULL)
- [itag_](#) & **MarkDual** (char m=QS_MARK_DUAL, bool always=1)
- [itag_](#) & **AddTilde** ()
- [itag_](#) & **SetM** ()
- [itag_](#) & **SetK** ()
- [itag_](#) & **SetD** ()
- unsigned **to_str** (char *s, unsigned len, char cflag=1) const
- [wbstring](#) **toStr** (char cflag=1) const
- mxArray * **toMx** () const

Static Public Member Functions

- static void **Reset** (unsigned char q=-1)
- static void **iter_flag_id** ()

Public Attributes

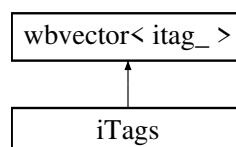
- IDT **t**

The documentation for this class was generated from the following files:

- [wbindex.hh](#)
- [wbindex.cc](#)

4.38 iTags Class Reference

Inheritance diagram for iTags:



Public Member Functions

- **iTags** (unsigned n=0)
- **iTags** (const char *F, int L, const char *s)
- **iTags** (const char *s)
- **iTags** (const [iTags](#) &b)
- unsigned **init** (const char *F, int L, const char *s)
- unsigned **init** (const char *s)
- [iTags](#) & **init** (unsigned n=0)
- [iTags](#) & **init** (const [iTags](#) &b)
- [iTags](#) & **operator=** (const [iTags](#) &b)
- [iTags](#) & **Update** (const [iTags](#) &b)
- [iTags](#) & **tSet** (const [iTags](#) &b)
- unsigned **init** (const char *F, int L, const mxArray *a)
- [iTags](#) & **init3** ()
- [iTags](#) & **init** (size_t l1, const [itag_](#) *d1, size_t l2, const [itag_](#) *d2)
- unsigned **Init** (const char *F, int L, const char *s, unsigned k=-1, char lflag=-1)
- [iTags](#) & **Init** (unsigned r, int k, const char **st)
- [iTags](#) & **initOp** ([itag_](#) t, const [iTags](#) &b, const char *ot="op")
- [iTags](#) & **init_alpha** (unsigned r, unsigned s, const char *tag, char t, unsigned k)
- unsigned **Set** (const char *F, int L, const char *tag, unsigned r=-1, const char *tom=NULL, unsigned m=-1, bool conjOM=1)
- [iTags](#) & **SetConj** (const char *F, int L, unsigned i)
- [iTags](#) & **Conj** ()
- [iTags](#) & **SetFlag** (const char *F, int L, unsigned k, unsigned l)
- [iTags](#) & **SetFlags** (const char *F, int L, unsigned k, unsigned l=-1, char check=1)
- [iTags](#) & **UnsetFlag** (const char *F, int L, unsigned k, unsigned l)
- [iTags](#) & **UnsetFlags** ()
- [iTags](#) & **init_tags** ()
- bool **operator==** (const [iTags](#) &b) const
- bool **operator!=** (const [iTags](#) &b) const
- int **findRegEx** (const char *r) const
- int **RegEx_replace** (const char *F, int L, const char *rstr) const
- int **getCtr** (const char *F, int L, const [iTags](#) &b, [ctrldx](#) &ia, [ctrldx](#) &ib, [iTags](#) *c=NULL, [wbvector](#)< char > *ma=NULL, [wbvector](#)< char > *mb=NULL) const
- int **UpdateItagsCtr** (const char *F, int L, const [ctrldx](#) &ia, [iTags](#) &b, const [ctrldx](#) &ib, const char *tag)
- int **FlagItagsCtr** (const char *F, int L, const [ctrldx](#) &ia, [iTags](#) &B, const [ctrldx](#) &ib, char all=1, char sgn=0)
- bool **sameConj** (const [iTags](#) &b) const
- bool **isOp** (int r=-1, char lflag=0) const
- bool **isOpX** (unsigned r=-1) const
- bool **isAtensor** (unsigned r=-1) const
- bool **got_qdir** (const char *qdir, char fflag=0) const
- unsigned **SkipLast** (unsigned n=1)
- [wbstring](#) **toStr** (char vflag=1) const
- const char * **toStrk** (char *sout, unsigned l, unsigned k, const char *st="(itag?)", char cflag=0) const
- mxArray * **toMx** () const
- int **got_op_labels** (const char *F=NULL, int L=0) const
- unsigned **gotLabels** () const

Public Member Functions inherited from [wbvector< itag_ >](#)

- [wbvector](#) (size_t l, [itag_](#) x)
- [wbvector](#) (size_t l, const [itag_](#) *d0=NULL, char ref=0)
- [wbvector](#) (const [wbvector](#) &v)
- [wbvector](#) (const [wbvector](#)< T2 > &v)
- [wbvector](#) (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) (const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) & [init](#) (size_t l=0)
- [wbvector](#) & [init](#) (size_t l, const [itag_](#) *d0)
- [wbvector](#) & [init](#) (size_t l, const [itag_](#) *d0, char ref)
- [wbvector](#) & [init](#) (const [wbvector](#) &v)
- [wbvector](#) & [init](#) (size_t n, const [wbvector](#) &v)
- [wbvector](#)< [itag_](#) > & [init](#) (const [wbsparray](#)< [itag_](#) > &S)
- [wbvector](#) & [init](#) (size_t l1, const [itag_](#) *d1, size_t l2, const [itag_](#) *d2)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type QS_UNUSED_↵VAR=1, char ref=0)
- int [init](#) (const mxArray *a, const char *istr=NULL, char check_type=1)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- [wbvector](#) & [initp](#) (size_t l, const [itag_](#) *d0, size_t n, char init=1)
- [wbvector](#) & [init2val](#) (size_t l, const [itag_](#) &x)
- [wbvector](#) & [initStride](#) (size_t l, const [itag_](#) *d0, size_t stride)
- [wbvector](#) & [init_stride](#) (const [itag_](#) *d0, size_t n, size_t m, size_t D0)
- [wbvector](#) & [init_data](#) (const [itag_](#) *d0)
- [wbvector](#) & [init_data](#) (const T0 *d0)
- [wbvector](#) & [init_data](#) (const char *F, int L, const T0 *d0)
- [wbvector](#)< [itag_](#) > & [initT](#) (const [wbsparray](#)< T2 > &S)
- [wbvector](#)< [itag_](#) > & [initT](#) (size_t l, const T2 *v)
- [wbvector](#)< [itag_](#) > & [initT](#) (const [wbvector](#)< T2 > &v)
- [wbvector](#)< [itag_](#) > & [initT](#) (const char *F, int L, const [wbvector](#)< T2 > &v)
- [wbvector](#)< [itag_](#) > & [initT](#) (const [wbsparray](#)< TD > &S)
- [wbvector](#)< [itag_](#) > & [initT_](#) (const [wbvector](#)< T2 > &v)
- [wbvector](#) & [init2ref](#) (const [wbvector](#) &v)
- [wbvector](#) & [init2ref](#) (size_t l, const [itag_](#) *d0)
- [wbvector](#) & [unRef](#) ()
- int [init_Struct](#) (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)
- int [init_base](#) (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)
- int [init_mpfr](#) (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)
- void [initDef](#) (size_t n)
- void [initDef](#) (const [wbvector](#) &v)
- [wbvector](#) & [operator=](#) (const [wbvector](#) &v)
- [wbvector](#)< [itag_](#) > & [operator=](#) (const [wbvector](#)< TB > &v)
- [wbvector](#) & [set](#) (const [itag_](#) &x)
- [wbvector](#) & [set](#) (const [wbvector](#)< size_t > &l, const [itag_](#) &x)
- void [set](#) (size_t offset, const [itag_](#) *x, size_t n)
- void [set](#) (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- void [set](#) (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- [wbvector](#) & [setMin](#) (const [itag_](#) &x)
- [wbvector](#) & [setMax](#) (const [itag_](#) &x)

- [wbvector](#) & **add** (const [wbvector](#)< size_t > &l, const [itag_](#) &x)
- [wbvector](#) & **add** (const [itag_](#) *d)
- [wbvector](#) & **Set** (const [wbvector](#)< size_t > &l, const [wbvector](#) &v)
- void **Sort** ([wbperm](#) &p, char dir=+1)
- void **Sort** (char dir=+1)
- void **sort** ([wbvector](#) &v2, [wbperm](#) &p) const
- void **sort** ([wbvector](#) &v2) const
- [wbperm](#) & **pSort** ([wbperm](#) &p) const
- bool **isSorted** () const
- bool **isSortedU** () const
- bool **isUnique** () const
- int **contains** (const [itag_](#) &x) const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isFinite** () const
- size_t **find1** (const [itag_](#) &x) const
- size_t **find1** () const
- size_t **count** (const [itag_](#) &x) const
- size_t **icount** (const [itag_](#) &x) const
- size_t **nnz** () const
- size_t **nnz** ([itag_](#) eps) const
- size_t **numZeros** ([itag_](#) eps=[itag_](#) (1E-14)) const
- size_t **numEQ** (const [itag_](#) &) const
- size_t **numGT** (const [itag_](#) &) const
- size_t **numLT** (const [itag_](#) &) const
- [wbindex](#) & **find** (const [itag_](#) &x, [wbindex](#) &l, char iflag=0) const
- [wbindex](#) & **find** (const [itag_](#) &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **find** ([wbindex](#) &l) const
- [wbindex](#) & **find** ([wbindex](#) &l, [wbindex](#) &lx) const
- [wbvector](#)< size_t > **find** () const
- [wbindex](#) & **findGT** (const [itag_](#) &x, [wbindex](#) &l, char iflag=0) const
- [wbindex](#) & **findGT** (const [itag_](#) &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **findzeros** ([wbindex](#) &l) const
- [wbindex](#) & **findRange** (const [itag_](#) &x1, const [itag_](#) &x2, [wbindex](#) &l, const char *w=NULL) const
- int **findClosestSorted** ([itag_](#) r) const
- int **findClosest** ([itag_](#) r) const
- size_t **findValues** (const [wbvector](#) &v, [wbindex](#) &l) const
- [wbvector](#) & **setp** (const [itag_](#) *x)
- void **reset** ()
- void **cpy** (const [itag_](#) *v)
- void **cpy** (const [itag_](#) *x, size_t m)
- [wbvector](#) & **save2** ([wbvector](#) &v)
- void **swap** ([wbvector](#) &v)
- [wbvector](#) & **swap** (size_t i, size_t j)
- [wbvector](#) & **Collect** ([itag_](#) &A1, [itag_](#) &A2)
- [wbvector](#) & **Collect** ([itag_](#) &A1, [itag_](#) &A2, [itag_](#) &A3)
- [wbvector](#) & **Skip** (size_t)
- [wbvector](#) & **Skip** (const WBINDEX &, char rflag=0)
- [wbvector](#) & **Move2_FE** (const WBINDEX &l, char FE_flag)

- [wbvector](#) & **Move2front** (const WBINDEX &l)
- [wbvector](#) & **Move2front** (size_t k)
- [wbvector](#) & **Move2end** (const WBINDEX &l)
- [wbvector](#) & **Move2end** (size_t k)
- [wbvector](#) & **mvLastTo** (size_t k)
- [wbvector](#) & **mvFirstTo** (size_t k)
- bool **checkSameLength** (const [wbvector](#) &a, const char *F=NULL, int L=0, const char *istr="") const
- bool **checkSameLength** (const [wbvector](#) &a, const [wbvector](#) &b, const char *F=NULL, int L=0, const char *istr="") const
- void **operator+=** (const T_ &a)
- void **operator+=** (const [wbvector](#) &b)
- void **operator-=** (const T_ &a)
- void **operator-=** (const [wbvector](#) &b)
- [wbvector](#) & **operator*=** (const T_ &a)
- [wbvector](#) & **operator/=** (const T_ &a)
- [wbvector](#) **operator*** (const T_ &a) const
- [wbvector](#) **operator+** (const T_ &a) const
- [wbvector](#) **operator+** (const [wbvector](#) &b) const
- [wbvector](#) **operator-** (const T_ &a) const
- [wbvector](#) **operator-** (const [wbvector](#) &b) const
- [wbvector](#) & **Plus** (const [itag_](#) *b)
- [wbvector](#) & **Plus** (const [itag_](#) *b, [itag_](#) bfac)
- [wbvector](#) & **Plus** (const [wbvector](#) &b, [itag_](#) bfac=1)
- void **plus** (const [wbvector](#) &b, [wbvector](#) &c, [itag_](#) bfac=1, [itag_](#) cfac=0) const
- [wbvector](#) & **times** ([itag_](#) fac, [wbvector](#) &x) const
- [wbvector](#) & **RevertSigns** ()
- int **isum** () const
- [itag_](#) **sum** () const
- [itag_](#) **sum** (size_t i1, const char *estr) const
- [itag_](#) **sum** (size_t i1, size_t i2) const
- [itag_](#) **cumsum_** ([wbvector](#) &S, char xflag=0) const
- [itag_](#) **Cumsum** (char zflag=0) const
- [itag_](#) **cumsum0prod** ([wbvector](#)< T2 > &b, [wbvector](#)< [itag_](#) > &cs, char xflag) const
- [wbvector](#) & **Eldiv** (const [wbvector](#) &v) const
- [wbvector](#) & **tensorProd** (const [wbvector](#) &v2, [wbvector](#) &vv) const
- [itag_](#) **scalarProd** (const [wbvector](#) &v) const
- [itag_](#) **scalarProd** (const [itag_](#) *v) const
- [itag_](#) **norm** () const
- [itag_](#) **norm2** () const
- double **normDiff** (const [wbvector](#) &v, [itag_](#) fac=1) const
- [itag_](#) **normDiff2** (const [wbvector](#) &v, [itag_](#) fac=1) const
- double **normDiff2_** (const [wbvector](#) &v, double fac=1) const
- [itag_](#) **GSPProject** (const [itag_](#) *v, char isnorm=0, char tflag=0) const
- [wbvector](#) & **Normalize** (const char *F=NULL, int L=0)
- [wbvector](#) & **subtract** (const [itag_](#) *d)
- void **getReal** ([wbvector](#)< double > &R) const
- void **getReal** ([wbvector](#)< double > &R) const
- void **getImag** ([wbvector](#)< double > &I) const
- void **getImag** ([wbvector](#)< double > &I) const
- [wbvector](#) & **Conj** ()
- [wbvector](#)< [wbcomplex](#) > & **Conj** ()
- [wbvector](#) & **applyFlag** (char &flag, const [itag_](#) &fac, [wbvector](#) &a)
- bool **allEqual** () const
- bool **allEqual** (const [itag_](#) &a) const
- bool **allZero** () const

- bool **allOnes** () const
- bool **allEqual2** (const [itag_](#) &a, [itag_](#) eps=1E-15) const
- bool **allUnequal** (const [itag_](#) &a) const
- bool **any** () const
- bool **anyUnequal** (const [itag_](#) &a) const
- bool **anyUnequal** (const [wbvector](#)< size_t > &l, const [itag_](#) &x, size_t *k=NULL) const
- bool **anyEqual** (const [itag_](#) &a, size_t *i1=NULL) const
- bool **anyEqual** (const [wbvector](#)< size_t > &l, const [itag_](#) &x, size_t *k=NULL) const
- bool **anyGT** (const [itag_](#) &a) const
- bool **anyGE** (const [itag_](#) &a) const
- bool **anyLT** (const [itag_](#) &a) const
- bool **anyLE** (const [itag_](#) &a) const
- bool **allGT** (const [itag_](#) &a) const
- bool **allGE** (const [itag_](#) &a) const
- bool **allLE** (const [itag_](#) &a) const
- bool **allLT** (const [itag_](#) &a) const
- bool **allIn** (const [itag_](#) &a1, const [itag_](#) &a2) const
- bool **anySmallVals** (const [itag_](#) eps=1E-15) const
- bool **isEqual** (const [itag_](#) *v) const
- bool **isEqual** (const [wbvector](#) &v) const
- bool **isEmpty** () const
- **operator bool** () const
- bool **operator!** () const
- bool **operator==** (const [wbvector](#) &v) const
- bool **operator!=** (const [wbvector](#) &v) const
- bool **isEqualR** (const [wbvector](#) &v) const
- bool **operator^** (const [wbvector](#) &v) const
- bool **deepEqualP** (const [wbvector](#) &B) const
- char **cmp** (const [wbvector](#) &v) const
- bool **operator<** (const [wbvector](#) &v) const
- int **operator<** (const [itag_](#) &r) const
- bool **operator<=** (const [wbvector](#) &v) const
- bool **operator>** (const [wbvector](#) &v) const
- bool **operator>=** (const [wbvector](#) &v) const
- [itag_](#) & **el** (size_t i) const
- [itag_](#) **elx** (size_t i, const [itag_](#) &x) const
- const [itag_](#) & **operator[]** (size_t i) const
- [itag_](#) & **operator[]** (size_t i)
- [wbvector](#) **operator[]** (const [wbvector](#)< size_t > &l) const
- [itag_](#) & **last** () const
- [itag_](#) & **last** (size_t &i) const
- [itag_](#) & **last** (TI &i_) const
- [itag_](#) & **p2last** () const
- [itag_](#) * **ref** (size_t i) const
- [wbvector](#) & **setRand** (double fac=1., double shift=0.)
- [wbvector](#)< [wbcomplex](#) > & **setRand** (double fac, double shift)
- void **setIdx** (double shift=0., double fac=1.)
- int **set2Group** (const [wbperm](#) &P, const WBINDEX &D, const [itag_](#) *S0=NULL, char iflag=0)
- [wbvector](#) & **Resize_** (size_t n)
- [wbvector](#) & **Shorten2** (size_t n)
- [wbvector](#) & **Resize** (size_t n, const [itag_](#) *d=NULL)
- [wbvector](#) & **resize** (size_t n, [wbvector](#) &X) const
- [wbvector](#) & **cat** (const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) > &vv)

- **wbvector** & **Cat** (const **wbvector**< **wbvector** * > &vv)
- **wbvector** & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)
- **wbvector** & **Cat** (const **wbvector**< T1 > *v1, const **wbvector**< T2 > *v2, const **wbvector**< T3 > *v3=0, const **wbvector**< T3 > *v4=0)
- **wbvector**< **itag_** > & **Cat** (const **wbvector**< T1 > *v1, const **wbvector**< T2 > *v2, const **wbvector**< T3 > *v3, const **wbvector**< T3 > *v4)
- **wbvector**< **itag_** > & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)
- **wbvector** & **Append** (const **wbvector** &v2)
- **wbvector** & **Append** (const **itag_** &x)
- **wbvector** & **Append** (size_t d2, const **itag_** *x)
- **wbvector** **append** (const **wbvector** &v) const
- **wbvector** & **append** (const **wbvector** &v2, **wbvector** &vout) const
- void **append** (const **itag_** *x, size_t d2, **wbvector** &vout) const
- int **recLess** (const **itag_** *v) const
- int **recLessE** (const **itag_** *v) const
- int **recCompare** (const **itag_** *v) const
- **itag_** **max** () const
- **itag_** **max** (size_t *k) const
- **itag_** **max** (unsigned &k) const
- **itag_** **max_** (**itag_** x) const
- **itag_** **min** () const
- **itag_** **min** (size_t *k) const
- **itag_** **min** (unsigned &k) const
- **itag_** **min_** (**itag_** x) const
- **itag_** **aMax** (size_t *k=NULL) const
- **itag_** **aMax** (unsigned &k) const
- **itag_** **aMin** (char zflag=0, size_t *k=NULL) const
- **itag_** **maxneg** (size_t *k=NULL) const
- **itag_** **minpos** (size_t *k=NULL) const
- **itag_** **avg** () const
- **itag_** **std** () const
- void **range** (**itag_** &xmin, **itag_** &xmax) const
- **itag_** **range** () const
- **wbvector** & **Permute** (const **wbperm** &P, char iflag=0)
- **wbvector** & **permute** (**wbvector** &, const **wbperm** &P, char iflag=0) const
- **wbvector** **permute** (const **wbperm** &P, char iflag=0) const
- **wbvector** & **blockPermute** (const **wbperm** &P, **wbvector** &, char iflag=0) const
- **wbvector** & **BlockPermute** (const **wbperm** &P, char iflag=0)
- int **blockCompare** (const **wbperm** &pa, const **wbvector** &b, const **wbperm** &pb) const
- void **get** (const WBINDEX &I0, **wbvector** &v) const
- void **get** (const **wbvector**< size_t > &I, **itag_** *v) const
- void **get** (size_t i1, size_t i2, **wbvector** &v) const
- void **selectSU** (const WBINDEX &I)
- void **select** (const WBINDEX &I, **itag_** *r) const
- **wbvector** & **select** (const WBINDEX &I, **wbvector** &a) const
- **wbvector** **select** (const WBINDEX &I) const
- **wbvector** & **Select** (const WBINDEX &I)
- **wbvector** & **BlockSelect** (const WBINDEX &I, const WBINDEX &D)
- **wbvector** & **getI** (size_t k, **wbvector** &v) const
- **wbvector** & **getI** (const **wbindex** &k, **wbvector** &v, char uflag=1) const
- **wbvector** & **getI** (const WBINDEX &k, **wbvector** &v, char uflag=1) const
- **wbvector** & **initI** (const **wbvector** &a, const **wbindex** &ia)
- **wbvector** & **initI** (const **wbvector** &a, const **wbindex** &ia, const **wbvector** &b, const **wbindex** &ib, char uflag=1)

- **wbvector** & **Flip** ()
- void **flip** (**wbvector** &a) const
- **wbvector flip** () const
- **itag_prod** () const
- **itag_prod** (**itag_** x) const
- **itag_prod** (size_t i, size_t i2) const
- **itag_prod** (size_t i, size_t i2, **itag_** x) const
- **itag_prod** (const T1 *l, size_t n) const
- T2 **prod** (const T1 *l, size_t n, T2 &s2) const
- void **prod2** (**itag_** &p1, **itag_** &p2, const **itag_** &p0=0) const
- void **info** (const char *istr) const
- void **print** (const char *istr, char mflag=0) const
- void **printdata** (const char *istr=0) const
- void **put** (const char *F, int L, const char *vname="ans", char tflag=0, const char *ws="base") const
- void **put** (const char *vname="ans", char tflag=0, const char *ws="base") const
- void **dispWithMsg** (const char *file, int line, const char *msg, const char *vname="") const
- void **mat2mx** (mxArray *&a, char dim=2) const
- void **mat2mxs** (mxArray *a, char field_nr, char dim=2) const
- mxArray * **toMx_Struct** (const char tflag=0) const
- mxArray * **toMxP_S** (const char tflag=0) const
- mxArray * **toMx** (const char tflag=0) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMxP** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base_d** (const char tflag=0) const
- mxArray * **toMx_offset** (const **itag_** &offset, const char tflag=0) const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tflag=0) const
- void **add2MxStruct** (mxArray *S, const char *vname, char tflag=0) const
- mxArray * **mxCreateCell** (unsigned m, unsigned n) const
- void **add2MxCell** (mxArray *S, unsigned i, char tflag=0) const
- void **putg** (const char *vname="ans", char tflag=0) const
- void **putx** (const char *vname="ans", char tflag=0) const
- **wbstring toStr** (int n, const char *sep) const
- **wbstring toStr** () const
- **wbstring toStr** (int n, const char *sep) const
- **wbstring sizeStr** () const
- **wbstring toStrf** (const char *fmt0="", const char *sep=" ", unsigned stride=0, const char *sep2="") const
- **wbstring toStrf** (const char *fmt , const char *sep, unsigned stride, const char *sep2) const

Additional Inherited Members

Public Attributes inherited from [wbvector< itag_ >](#)

- [itag_](#) * **data**
- [size_t](#) **len**
- char **isref**

Protected Member Functions inherited from [wbvector< itag_ >](#)

- [wbvector](#) & **RENEW** ([size_t](#) n, const [itag_](#) *d=NULL, [size_t](#) l=-1, char init=1)
- [wbvector](#) & **RENEW_VAL** ([size_t](#) n, const [itag_](#) &x)

The documentation for this class was generated from the following files:

- [wbindex.hh](#)
- [wbindex.cc](#)

4.39 Wb::iterator< T, T2 > Class Template Reference

Public Member Functions

- **iterator** (T *p_)
- [iterator](#) & **operator++** ()
- bool **end** ()
- [size_t](#) **iter** () const
- [size_t](#) **numel** () const
- T2 & **operator[]** ([size_t](#) k)
- T2 & **at** ()

Public Attributes

- [size_t](#) **i**
- [size_t](#) **n**
- T * **p**

The documentation for this class was generated from the following file:

- [util.hh](#)

4.40 Wb::iterator< T, T2 * > Class Template Reference

Public Member Functions

- **iterator** (T *p_)
- [iterator](#) & **operator++** ()
- bool **end** ()
- [size_t](#) **iter** () const
- [size_t](#) **numel** () const
- T2 * **operator[]** ([size_t](#) k)
- T2 * **at** ()

Public Attributes

- `size_t i`
- `size_t n`
- `T * p`

The documentation for this class was generated from the following file:

- `util.hh`

4.41 IterOM_DATA< TD > Class Template Reference**Public Member Functions**

- `template<class TQ >`
`IterOM_DATA` (`const QSpace< TQ, TD > &A`, `unsigned i`)
- `void begin ()`
- `bool end ()`
- `IterOM_DATA & operator++ ()`
- `wbarray< TD > DATA ()`

Public Attributes

- `unsigned iter`
- `unsigned niter`
- `size_t nS`
- `TD * data0`
- `wbarray< TD > X`

The documentation for this class was generated from the following file:

- `QSpace.hh`

4.42 iterSparseCMref< TD > Class Template Reference**Public Member Functions**

- `iterSparseCMref` (`const char *F`, `int L`, `wbsparray< TD > &A`)
- `SPIDX_T next` (`wbsparray< TD > &a`)
- `wbsparray< TD > & getIter` (`SPIDX_T i`, `wbsparray< TD > &a`, `char ref=0`) `const`
- `TD norm2` (`SPIDX_T i`, `char tnorm=0`) `const`
- `SPIDX_T numIter` () `const`
- `bool gotLast` () `const`
- `SPIDX_T index` () `const`
- `mxArray * toMx` () `const`

Public Attributes

- SPIDX_T **it**
- [wbvector](#)< SPIDX_T > **IO**
- [wbvector](#)< SPIDX_T > **IX**
- [wbMatrix](#)< SPIDX_T > **IDX**

The documentation for this class was generated from the following file:

- wbsparray.hh

4.43 Wb::line_stat__ Class Reference**Public Member Functions**

- void **add** (double dt_, double dx)

Public Attributes

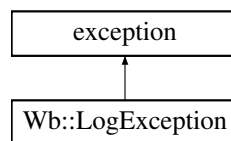
- double **dt**
- double **aux**
- size_t **ncall**

The documentation for this class was generated from the following file:

- wblock.hh

4.44 Wb::LogException Struct Reference

Inheritance diagram for Wb::LogException:

**Public Member Functions**

- **LogException** (WBLOG_TYPE I=OK, const char *s=NULL)
- **LogException** (const [LogException](#) &e)
- [LogException](#) & **operator=** (const [LogException](#) &e)
- [LogException](#) & **operator++** ()
- [LogException](#) & **operator+=** (const [LogException](#) &e)
- bool **operator!** () const
- **operator bool** () const
- [LogException](#) & **init** ()
- const char * **what** () const throw ()
- [wbstring](#) **toStr** () const
- void **report** (const char *F=0, int L=0, const char *func=0)

Public Attributes

- `WBLOG_TYPE` **type**
- `int * nrefs`
- `char istr [64]`
- unsigned **count**
- `int ith`
- `int nth`

The documentation for this struct was generated from the following files:

- `wblog.h`
- `wblog.c`

4.45 Wb::matFile Class Reference

Public Member Functions

- **matFile** (`const char *F`, `int L`, `const char *fname`, `const char *rw`)
- **matFile** (`const matFile &X`)
- void **close** ()
- void **init** ()
- `int Open` (`const char *F`, `int L`, `const char *FName`, `const char *RW`)
- `int open` (`const char *file`, `int line`, `const char *FName`, `const char *RW`, `char xflag=0`)
- [matFile](#) & **put** (`const char *file`, `int line`, `const char *vname`, `const mxArray *a`)
- [matFile](#) & **put** (`const char *vname`, `const mxArray *a`)
- `int existsVar` (`const char *vn`) `const`
- `int deleteVar` (`const char *vn`) `const`
- `int deleteVar` (`const char *f`, `int L`, `const char *vn`) `const`

Public Attributes

- `MATFile * mfp`
- [wbstring](#) **F**
- [wbstring](#) **fname**
- [wbstring](#) **rw**
- `int L`

The documentation for this class was generated from the following file:

- `wbMatIO.hh`

4.46 MEM_SET< T > Class Template Reference

Public Member Functions

- **MEM_SET** (T *data QS_UNUSED_VAR, size_t len QS_UNUSED_VAR, const T *a QS_UNUSED_VAR=NULL)
- **MEM_SET** (int *data, size_t len, const int *a)
- **MEM_SET** (unsigned *data, size_t len, const unsigned *a)
- **MEM_SET** (unsigned long *data, size_t len, const unsigned long *a)
- **MEM_SET** (long *data, size_t len, const long *a)
- **MEM_SET** (char *data, size_t len, const char *a)
- **MEM_SET** (double *data, size_t len, const double *a)
- **MEM_SET** (float *data, size_t len, const float *a)
- **MEM_SET** ([wbcomplex](#) *data, size_t len, const [wbcomplex](#) *a)

The documentation for this class was generated from the following file:

- memlib.hh

4.47 MEM_SET< T * > Class Template Reference

Public Member Functions

- **MEM_SET** (T **data, size_t len, T *const *a=NULL)

The documentation for this class was generated from the following file:

- memlib.hh

4.48 MHash< TM > Class Template Reference

Public Member Functions

- **size_t operator()** (const [cgc_contract_id](#)< TM > &S) const

The documentation for this class was generated from the following file:

- clebsch.hh

4.49 Wb::mpfr__ < P > Class Template Reference

Public Member Functions

- **mpfr__** (double val=0)
- **mpfr__** (const char *F, int L, double val)
- **mpfr__** (const [mpfr__](#) &b)
- void **Cleanup** (const char *F=NULL, int L=0) const
- **mpfr__** & **init** (const char *F, int L, double val, char lflag=0)
- **mpfr__** & **init_s** (const char *F, int L, const char *s, unsigned base=10)
- **mpfr__** & **init_d** (const char *F, int L, double val, char dcheck=1)
- **mpfr__** & **set_pi** ()
- **mpfr__** & **set_log2** ()
- **mpfr__** & **set_e** ()
- **mpfr__** & **set_nan** ()
- **mpfr__** & **set_inf** (int s=+1)
- **mpfr__** & **set_zero** (int s=+1)
- int **isnan** () const
- int **isinf** () const
- int **iszero** () const
- int **isnumber** () const
- int **isnonzero** () const
- int **sign** ()
- void **swap** ([mpfr__](#) < P > &b)
- int **cmp** (const [mpfr__](#) &b) const
- int **cmp** (const double &val) const
- int **cmp** (const int &val) const
- int **cmp** (const long &val) const
- int **cmp** (const unsigned &val) const
- int **cmp** (const unsigned long &val) const
- bool **operator>** (const [mpfr__](#) &x) const
- bool **operator>** (const double &x) const
- bool **operator>** (const int &x) const
- bool **operator>** (const long &x) const
- bool **operator>** (const unsigned &x) const
- bool **operator>** (const unsigned long &x) const
- bool **operator<** (const [mpfr__](#) &x) const
- bool **operator<** (const double &x) const
- bool **operator<** (const int &x) const
- bool **operator<** (const long &x) const
- bool **operator<** (const unsigned &x) const
- bool **operator<** (const unsigned long &x) const
- bool **operator>=** (const [mpfr__](#) &x) const
- bool **operator>=** (const double &x) const
- bool **operator>=** (const int &x) const
- bool **operator>=** (const long &x) const
- bool **operator>=** (const unsigned &x) const
- bool **operator>=** (const unsigned long &x) const
- bool **operator<=** (const [mpfr__](#) &x) const
- bool **operator<=** (const double &x) const
- bool **operator<=** (const int &x) const
- bool **operator<=** (const long &x) const
- bool **operator<=** (const unsigned &x) const
- bool **operator<=** (const unsigned long &x) const

- `bool operator!= (const mpfr__ &x) const`
- `bool operator!= (const double &x) const`
- `bool operator!= (const int &x) const`
- `bool operator!= (const long &x) const`
- `bool operator!= (const unsigned &x) const`
- `bool operator!= (const unsigned long &x) const`
- `bool operator== (const mpfr__ &x) const`
- `bool operator== (const double &x) const`
- `bool operator== (const int &x) const`
- `bool operator== (const long &x) const`
- `bool operator== (const unsigned &x) const`
- `bool operator== (const unsigned long &x) const`
- `mpfr__ & operator= (double val)`
- `mpfr__ & operator= (const mpfr__ &b)`
- `mpfr__ & times (const mpfr__ &b, mpfr__ &c) const`
- `mpfr__ & divby (const mpfr__ &b, mpfr__ &c) const`
- `mpfr__ & operator*= (const mpfr__ &b)`
- `mpfr__ & operator/= (const mpfr__ &b)`
- `mpfr__ & operator+= (const mpfr__ &b)`
- `mpfr__ & operator-= (const mpfr__ &b)`
- `mpfr__ & operator*= (double val)`
- `mpfr__ & operator/= (double val)`
- `mpfr__ & operator+= (double val)`
- `mpfr__ & operator-= (double val)`
- `mpfr__ operator* (const mpfr__ &b) const`
- `mpfr__ operator/ (const mpfr__ &b) const`
- `mpfr__ operator+ (const mpfr__ &b) const`
- `mpfr__ operator- (const mpfr__ &b) const`
- `mpfr__ operator* (int n) const`
- `mpfr__ operator/ (int n) const`
- `mpfr__ operator+ (int n) const`
- `mpfr__ operator- (int n) const`
- `mpfr__ operator* (unsigned n) const`
- `mpfr__ operator/ (unsigned n) const`
- `mpfr__ operator+ (unsigned n) const`
- `mpfr__ operator- (unsigned n) const`
- `mpfr__ operator* (long n) const`
- `mpfr__ operator/ (long n) const`
- `mpfr__ operator+ (long n) const`
- `mpfr__ operator- (long n) const`
- `mpfr__ operator* (unsigned long n) const`
- `mpfr__ operator/ (unsigned long n) const`
- `mpfr__ operator+ (unsigned long n) const`
- `mpfr__ operator- (unsigned long n) const`
- `mpfr__ & Sqrt ()`
- `mpfr__ & Exp ()`
- `mpfr__ & Log ()`
- `mpfr__ & Log2 ()`
- `mpfr__ & Atan ()`
- `mpfr__ & Tanh ()`
- `mpfr__ & Atanh ()`
- `mpfr__ & Rand ()`
- `mpfr__ & Log10 ()`
- `mpfr__ & Round ()`
- `mpfr__ sqrt () const`

- [mpfr__exp](#) () const
- [mpfr__log](#) () const
- [mpfr__log2](#) () const
- [mpfr__atan](#) () const
- [mpfr__log10](#) () const
- [mpfr__round](#) () const
- **operator double** () const
- **operator wbcomplex** () const
- **operator bool** () const
- double **getval_d** () const
- bool **operator<** (double val) const
- bool **operator<=** (double val) const
- bool **operator>** (double val) const
- bool **operator>=** (double val) const
- int **prec** (unsigned base=2) const
- [mpfr__eps](#) ()
- [mpfr__ & FlipSign](#) ()
- [mpfr__ & Abs](#) ()
- [mpfr__abs](#) () const
- [mpfr__fabs](#) () const
- [mpfr__abs2](#) () const
- void **print** (const char *vname="", const char *istr=0, char sep=0) const
- [size_t to_str](#) (char *s, [size_t](#) n_, int base=10) const
- [wbstring to_str](#) (int base=10) const
- void **toStr** (char *s, unsigned n, int base=10, int nd=-1, char align=0) const
- [wbstring & toStr](#) ([wbstring](#) &sout, int base=10, int nd=-1, char align=0) const

Public Attributes

- [mpfr_t](#) f

Static Public Attributes

- static unsigned long **num_vars** =0

The documentation for this class was generated from the following files:

- [wbmpfr.hh](#)
- [wbmpfr.cc](#)

4.50 Wb::MREC Class Reference

Public Member Functions

- template<class T >
const char * **init** (const char *F QS_UNUSED_VAR, int L QS_UNUSED_VAR, [size_t](#) i, const T *p QS_UNUSED_VAR, [size_t](#) l)
- void **println** ([size_t](#) i, void *p) const

Public Attributes

- `size_t id`
- `size_t len`
- unsigned `unit`

The documentation for this class was generated from the following file:

- `memtrack.cc`

4.51 `mxArray_buf` Class Reference

Public Member Functions

- `mxArray_buf` (`const char *n=NULL`)
- void `add` (`void *p, mxArray *a`)
- `mxArray *` `find` (`const char *F, int L, void *p`)
- `mxArray *` `Return` (`const char *F, int L, void *p`)
- int `erase` (`void *p`)

Public Attributes

- `unordered_map< void *, mxArray * >` `BUF`
- `char *` `name`

The documentation for this class was generated from the following file:

- `mexlib.hh`

4.52 `MXPut` Class Reference

Public Member Functions

- `MXPut` (`const char *vn=0, const char *ws=0`)
- `MXPut` (`const char *file, int line, const char *vn="ans", const char *ws=0`)
- void `init` ()
- void `init` (`const char *file, int line, const char *vn=NULL`)
- void `put` (`const char *ws=0, const char *vn=0`)
- void `save2` (`mxArray *&S0`)
- `mxArray *` `toMx` ()
- void `add2Struct` (`mxArray *S0`)
- `MXPut` & `addFL` ()
- `template<class T >`
`MXPut` & `add` (`const T &x, const char *fn`)
- `template<class T >`
`MXPut` & `addP` (`const T *x QS_UNUSED_VAR, const char *fn QS_UNUSED_VAR`)
- `template<>` `MXPut` & `addP` (`const mxArray *a, const char *fn`)
- `template<>` `MXPut` & `addP` (`const char *s, const char *fn`)

- `template<> MXPut & add (const double &x, const char *fn)`
- `template<> MXPut & add (const float &x, const char *fn)`
- `template<> MXPut & add (const long double &x, const char *fn)`
- `template<> MXPut & add (const wcomplex &x, const char *fn)`
- `template<> MXPut & add (const char &x, const char *fn)`
- `template<> MXPut & add (char *const &s, const char *fn)`
- `template<> MXPut & add (const char *const &s, const char *fn)`
- `template<> MXPut & add (const unsigned &x, const char *fn)`
- `template<> MXPut & add (const int &x, const char *fn)`
- `template<> MXPut & add (const bool &x, const char *fn)`
- `template<> MXPut & add (const long &x, const char *fn)`
- `template<> MXPut & add (const unsigned long &x, const char *fn)`

Public Attributes

- `mxArray * S`
- `char wsp [16]`
- `char * vname`
- `char * F`
- `unsigned L`

Static Public Attributes

- `static const unsigned wlen =16`

The documentation for this class was generated from the following file:

- `mexlib.hh`

4.53 mxStruct Class Reference

Public Member Functions

- `mxStruct (const char *file, int line)`
- `void init (const char *file, int line)`
- `void put (const char *name)`
- `void save2Struct (mxArray *S0, const char *name)`
- `void checkNewField (const char *f=NULL, int l=0)`
- `mxStruct & operator() (const char *fn)`
- `template<class T >`
`mxStruct & operator= (const T &x)`
- `mxStruct & operator= (mxArray *a)`
- `template<class T >`
`mxStruct & operator= (const wvector< T > &x)`
- `template<class T >`
`mxStruct & operator= (const wbMatrix< T > &x)`
- `template<> mxStruct & operator= (const wbperm &P)`
- `template<> mxStruct & operator= (const windex &I)`

Public Attributes

- mxArray * **S**
- [wbstring](#) **F**
- [wbstring](#) **fld**
- unsigned **L**

The documentation for this class was generated from the following file:

- mexlib.hh

4.54 Wb::myname__ Class Reference**Public Attributes**

- char **data** [64]

The documentation for this class was generated from the following file:

- wblib.h

4.55 NRGData< TQ, TD > Class Template Reference**Public Member Functions**

- **NRGData** (const char *s=NULL)
- **NRGData** (const [NRGData](#) &A)
- void **initName** (const char *n)
- void **init** (const char *F, int L, const mxArray *a, [wbvector](#)< [QSpace](#)< TQ, TD > > &C0, int k=0, char force=0, const char *vname0=0)
- void **init** (const char *F, int L, const char *XX, int iter=-1)
- void **initXX** (const char *F, int L, int m=4, int iter=-1)
- void **initX** (const char *F, int L, int m=2, int iter=-1)
- void **init** (char flag=0)
- void **setupMX** (const char *F, int L, int N, char store_=1, char locMX_=1)
- void **PUT** (const char *F, int L, const char *vn, const char *ws="caller")
- void **check_dloc** (const char *F, int L, const char *fun, unsigned dloc, unsigned iter) const
- template<class TD_>
int **checkOpConsistency** (const char *F, int L, const [NRGData](#)< TQ, TD_ > &B) const
- bool **isQSpaceVec** (const char *nm, int iter=-1)
- bool **checkVec** (const char *F, int L, const char *XX, unsigned R, const char *istr="")
- bool **checkVecVec** (const char *XX, unsigned r)
- void **forceCalc** ()
- void **getRelevantOps** ([wbindex](#) &I) const
- [wbvector](#)< [QSpace](#)< TQ, TD > > & **getOpsXX** (const char *t)
- const [QSpace](#)< TQ, TD > & **getQSpace** (const char *t, unsigned i=0) const
- [QSpace](#)< TQ, TD > & **getQSpace** (const char *t, unsigned i=0)
- const [QSpace](#)< TQ, TD > & **getQSpace** (char t1, char t2, unsigned i=0) const
- [QSpace](#)< TQ, TD > & **getQSpace** (char t1, char t2, unsigned i=0)

- const [QSpace](#)< TQ, TD > & **getQSpace** (char t) const
- [QSpace](#)< TQ, TD > & **getQSpace** (char t)
- void **info** (const char *vstr=NULL) const
- void **displdx** (const char *F, int L) const
- void **info** (const char *F, int L, const char *istr=" * ") const
- void **disp_otype** (const char *F=0, int L=0) const
- const mxArray * **getMxInfo** (const char *n)
- template<class TX >
void **getMxInfo** (const char *F, int L, const char *v, TX &X)
- mxArray * **getMxData** (const char *nm, int iter=-1, char ionly=0)
- const char * **wnrg** ()
- mxArray * **toMx** () const
- void **put** (const char *vname, const char *ws="base")
- void **put** (const char *F, int L, const char *vname, const char *ws="base")
- template<class T >
int **getPara** (const char *name, T &x, int iter=-1, char i=0)
- bool **opExists** (char tflag=1)
- void **initOp** (unsigned n=1, char calc_=1, char store_=0)
- bool **initOp** (const char *F, int L, const [wbvector](#)< [QSpace](#)< TQ, TD > > &C0, char tflag=1, [NRGData](#)< TQ, TD > *CR=NULL, const char *istr="")
- bool **initOp** (const char *F, int L, const [wbvector](#)< [wbvector](#)< [QSpace](#)< TQ, TD > > &CC, [NRGData](#)< TQ, TD > *CR=NULL, const char *istr="", char tflag=1)
- void **initSym** (const char *tag)
- void **updatePara** (const char *F, int L, const char *tag, mxArray *, int iter=-1) const
- void **updatePara** (const char *F, int L, const char *tag, int iter=-1, const char *tag2=NULL, char all=-1) const
- void **updateOp** (const char *F, int L, const [NRGData](#)< TQ, TD > &A, const [NRGData](#)< TQ, TD > &B, unsigned nop, int iter=-1)
- template<class TC >
void **updateOpProd** (const char *F, int L, unsigned iter, const [NRGData](#)< TQ, TD > &B, const [NRGData](#)< TQ, TC > &C)
- void **updateOp** (const char *F, int L, const [NRGData](#)< TQ, TD > &A, unsigned nop, int iter=-1)
- template<class TA >
void **backPropagateOp** (const char *F, int L, const [NRGData](#)< TQ, TA > &A0, const [NRGData](#)< TQ, TA > &A2)
- template<class TD_ >
void **initBG** (const char *F, int L, const [NRGData](#)< TQ, TD_ > &B, const [NRGData](#)< TQ, TD_ > &H1, const [NRGData](#)< TQ, TD_ > &H2, int m=3)
- void **unsetAllRefs** ()
- unsigned **skipOpZeros** (double eps=DEPS, char bflag='b')
- void **saveOp** (const char *, int, int iter=-1, unsigned m=4) const
- void **times** (TD fac, const char *flag)
- void **setupIO** (const char *, int, const char *, const mxArray *, const char *istr=0)
- void **updateInfo** (const char *F, int L, const char *, mxArray *, char keep=-1) const
- void **updateInfof** (const char *F, int L, const char *vtag, mxArray *, char keep=-1) const
- const char * **getFileName** (unsigned k, char lenient=0) const
- template<class TD >
void **updateOpProd** (const char *F, int L, unsigned iter, const [NRGData](#)< TQ, TS > &B, const [NRGData](#)< TQ, TD > &C)
- template<class TD >
void **initBG** (const char *F, int L, const [NRGData](#)< TQ, TD > &B, const [NRGData](#)< TQ, TD > &H0, const [NRGData](#)< TQ, TD > &H2, int m)
- template<class TD >
void **backPropagateOp** (const char *F, int L, const [NRGData](#)< TQ, TD > &A0, const [NRGData](#)< TQ, TD > &A2)

Public Attributes

- [wbstring](#) **name**
- [wbvector](#)< [NRGIndex](#)< TQ, TD > > **nrgIdx**
- char **calc**
- char **store**
- int **kloc**
- char **locMX**
- [wbvector](#)< [wbvector](#)< [QSpace](#)< TQ, TD > > > **CI**
- [wbvector](#)< [QSpace](#)< TQ, TD > > **KK**
- [wbvector](#)< [QSpace](#)< TQ, TD > > **KD**
- [wbvector](#)< [QSpace](#)< TQ, TD > > **DK**
- [wbvector](#)< [QSpace](#)< TQ, TD > > **DD**
- [QSpace](#)< TQ, TD > **K**
- [QSpace](#)< TQ, TD > **D**
- [wbvector](#)< char > **issym**
- mxArray * **MX**
- [wbstring](#) **NAME**
- mxArray * **aux**

The documentation for this class was generated from the following file:

- nrgdata.hh

4.56 [NRGIndex](#)< TQ, TD > Class Template Reference

Public Member Functions

- **NRGIndex** (const [NRGIndex](#) &l)
- void **init** ()
- void **init** (int i, [NRGData](#)< TQ, TD > *d)
- void **init** (int i, const char *t)
- mxArray * **toMx** () const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tst=0) const
- void **put** (const char *vname, const char *ws="base") const

Public Attributes

- int **idx**
- [NRGData](#)< TQ, TD > * **nrg**
- [wbstring](#) **tag**

The documentation for this class was generated from the following file:

- nrgdata.hh

4.57 Wb::num2Fmt< T > Class Template Reference

Public Member Functions

- **num2Fmt** (const char *F, int L, int m_=-2, int p_=-2, const char *t_="")
- int **check_init** ()
- **operator wstring** () const
- int **check_init** ()
- int **check_init** ()
- int **check_init** ()
- int **check_init** ()
- int **check_init** ()
- int **check_init** ()
- int **check_init** ()
- int **check_init** ()
- int **check_init** ()

The documentation for this class was generated from the following files:

- gathered.hh
- gathered.cc

4.58 opFlags< T > Class Template Reference

Public Member Functions

- **opFlags** (const [opFlags](#) &f)
- **opFlags** (const char *F, int L, char f)
- **opFlags** (const [opFlags](#) &f1, const [opFlags](#) &f2)
- [opFlags](#)< T > & **operator=** (char f)
- [opFlags](#)< T > & **operator=** (const [opFlags](#)< T > &x)
- **opFlags** (char f)
- **operator char** ()
- [opFlags](#)< T > & **init** (const char *F, int L, char f)
- [opFlags](#)< T > & **init** (char f)
- [opFlags](#)< T > & **init** (const [opFlags](#)< T > &f1, const [opFlags](#)< T > &f2)
- char **trans** () const
- char **conj** () const
- char **idle** () const
- void **applyOrRef** (const char *F, int L, const [wbararray](#)< T > &A0, [wbararray](#)< T > &A, const T fac=1) const
- template<class T2 >
T2 & **applyOrRef** (const char *F, int L, const T2 &A0, const T fac=1) const
- template<class T2 >
const T2 & **applyConjOrRef** (const char *F, int L, const T2 &A0, const T fac=1) const
- template<class T2 >
char **apply** (const char *F, int L, T2 &A, const T fac=1) const
- char **toChar** () const
- mxArray * **toMx** () const
- void **info** (const char *F, int L, const char *istr="")

Public Attributes

- char **p**

The documentation for this class was generated from the following file:

- wbararray_blas.hh

4.59 OPTS Class Reference

Public Member Functions

- **OPTS** (const mxArray **ain, const unsigned n)
- **OPTS** (const char *fname)
- void **init** (const mxArray **ain, const unsigned n)
- void **init** (const char *fname)
- void **init** ()
- int **len** ()
- template<class T >
bool **getOpt** (const char *, int, const char *vn, T &x, char force=0, const char *istr=0)
- template<class T >
bool **getOpt** (const char *, int, const char *vn, [wbvector](#)< T > &x, char force=0, const char *istr=0)
- template<class T >
bool **getOpt** (const char *, int, const char *vn, [wbMatrix](#)< T > &x, char force=0, const char *istr=0)
- bool **getOpt** (const char *, int, const char *vn, const char *istr=0)
- template<class T >
bool **getOpt** (const char *vn, T &x, char force=0, const char *istr=0)
- template<class T >
bool **getOpt** (const char *vn, [wbvector](#)< T > &x, char force=0, const char *istr=0)
- template<class T >
bool **getOpt** (const char *vn, [wbMatrix](#)< T > &x, char force=0, const char *istr=0)
- bool **getOpt** (const char *vn, const char *istr=0)
- void **checkAnyLeft** (const char *MAT=NULL, int L=0)
- template<> bool **getOpt** (const char *F, int L, const char *vn, [wbstring](#) &x, char force, const char *istr)
- template<> bool **getOpt** (const char *F, int L, const char *vn, mxArray *&a, char force, const char *istr)
- template<> bool **getOpt** (const char *F QS_UNUSED_VAR, int L QS_UNUSED_VAR, const char *vn, [itag](#)_
&t, char force, const char *istr)

The documentation for this class was generated from the following file:

- wbopts.hh

4.60 pairPatternCG< T > Class Template Reference

Public Member Functions

- [pairPatternCG](#) & **init** (unsigned i0=0, unsigned j0=0, unsigned n=0)
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i) const

Public Attributes

- unsigned **i**
- unsigned **j**
- [wbvector](#)< unsigned > **k**
- [wbvector](#)< T > **fac**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch_io.cc

4.61 Wb::Path Class Reference

Public Member Functions

- **Path** (const char *s)
- **Path** (const char *F, int L, const char *s)
- int **init** ()
- int **init** (const char *F, int L, const char *s, const char *delim=":", unsigned n_=-1)
- int **append** (const char *F, int L, const char *s, const char *delim=":")
- char * **operator[]** (unsigned i)
- const char * **operator[]** (unsigned i) const
- const char * **last** () const
- unsigned **numel** () const
- [Wb::iterator](#)< [Path](#), char * > **begin** ()
- [Wb::citerator](#)< [Path](#), char * > **cbegin** () const
- void **print** (const char *istr="") const

Public Attributes

- [wbstring](#) **buf**
- [wbvector](#)< unsigned > **idx**

The documentation for this class was generated from the following file:

- wbutil.hh

4.62 Wb::Profile Class Reference

Public Member Functions

- **Profile** (const char *F, int L, const char *istr=0)
- void **tic** ()
- void **toc** (int L, double x=0)

Public Attributes

- `CHRONO_CLOCK tref`

The documentation for this class was generated from the following file:

- `wblock.hh`

4.63 QBlock< TQ, TD, TE > Class Template Reference

Public Member Functions

- `QBlock & init ()`
- `QBlock & operator= (const QBlock &B)`
- `QBlock & init_bare_refA (const QBlock &B, unsigned w=0)`
- `void toBlockMatrix (wbarray< TD > &MM, unsigned K, const wbperm &P=wbperm())`
- `void initFromBlockMatrix (const char *F, int L, QSpace< TQ, TD > &B, const wbarray< TD > &MM, const iTags &it, const wbperm *cgp=NULL) const`
- `void updateBlockDim (unsigned dim2)`
- `double get_s2t (const wbvector< char > &mark)`
- `void print (const char *F=NULL, int L=0, const iTags *t=NULL) const`
- `mxArray * toMx () const`
- `void put (const char *vname, const char *ws="caller")`

Public Attributes

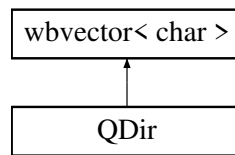
- unsigned `qdim_tot`
- `wbvector< unsigned > qdim`
- `QSpace< TQ, TD > A`
- `wbindex I0`
- `wbindex Ik`
- `wbindex It`
- `wbMatrix< TQ > Q1`
- `wbMatrix< TQ > Q2`
- `wbMatrix< widx_t > S1`
- `wbMatrix< widx_t > S2`
- `wbMatrix< widx_t > D1`
- `wbMatrix< widx_t > D2`
- `wbMatrix< int > idx`
- `wbarray< TD > U`
- `wbarray< TD > Vc`
- `wbvector< TE > S`

The documentation for this class was generated from the following files:

- `mpsortho.hh`
- `mpsortho.cc`

4.64 QDir Class Reference

Inheritance diagram for QDir:



Public Member Functions

- **QDir** (unsigned l, const char *d=NULL, char ref=0)
- **QDir** (const [iTags](#) &it)
- **QDir** (const char *F, int L, const char *s)
- **QDir** (const char *s)
- **QDir** (const char *F, int L, const mxArray *a)
- **QDir & init** (unsigned l=0, const char *d=NULL, char ref=0)
- **QDir & operator=** (const char *s)
- **QDir & init** (const char *F, int L, const char *s)
- **QDir & init** (const char *s)
- **QDir & init** (const [iTags](#) &it)
- **QDir & init** (const char *F, int L, const mxArray *a)
- **QDir & init_iout** (const char *F, int L, unsigned r, unsigned iout)
- unsigned **isSorted** () const
- bool **operator==** (const char *s) const
- bool **operator!=** (const char *s) const
- bool **operator==** (const [QDir](#) &q) const
- bool **operator!=** (const [QDir](#) &q) const
- bool **operator<** (const [QDir](#) &q) const
- [QDir](#) & **Conj** ()
- char & **Conj** (const unsigned &i)
- unsigned **nconj** ()
- unsigned **nconj** (const [ctrlidx](#) &lx)
- int **find1** () const
- [wstring](#) **toStr** () const
- [wstring](#) **toTag** () const
- mxArray * **toMx** () const

Public Member Functions inherited from [wvector< char >](#)

- **wvector** (size_t l, char x)
- **wvector** (size_t l, const char *d0=NULL, char ref=0)
- **wvector** (const [wvector](#) &v)
- **wvector** (const [wvector](#)< T2 > &v)
- **wvector** (const [wvector](#) &v1, const [wvector](#) &v2)
- **wvector** (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- **wvector** (const mxArray *a, char check_type=1, char ref=0)
- [wvector](#) & **init** (size_t l=0)
- [wvector](#) & **init** (size_t l, const char *d0)
- [wvector](#) & **init** (size_t l, const char *d0, char ref)
- [wvector](#) & **init** (const [wvector](#) &v)

- [wbvector](#) & **init** (size_t n, const [wbvector](#) &v)
- [wbvector](#)< char > & **init** (const [wbsparray](#)< char > &S)
- [wbvector](#) & **init** (size_t l1, const char *d1, size_t l2, const char *d2)
- int **init** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type QS_UNUSED_↵
VAR=1, char ref=0)
- int **init** (const mxArray *a, const char *istr=NULL, char check_type=1)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- [wbvector](#) & **initp** (size_t l, const char *d0, size_t n, char init=1)
- [wbvector](#) & **init2val** (size_t l, const char &x)
- [wbvector](#) & **initStride** (size_t l, const char *d0, size_t stride)
- [wbvector](#) & **init_stride** (const char *d0, size_t n, size_t m, size_t D0)
- [wbvector](#) & **init_data** (const char *d0)
- [wbvector](#) & **init_data** (const T0 *d0)
- [wbvector](#) & **init_data** (const char *F, int L, const T0 *d0)
- [wbvector](#)< char > & **initT** (const [wbsparray](#)< T2 > &S)
- [wbvector](#)< char > & **initT** (size_t l, const T2 *v)
- [wbvector](#)< char > & **initT** (const [wbvector](#)< T2 > &v)
- [wbvector](#)< char > & **initT** (const char *F, int L, const [wbvector](#)< T2 > &v)
- [wbvector](#)< char > & **initT** (const [wbsparray](#)< TD > &S)
- [wbvector](#)< char > & **initT_** (const [wbvector](#)< T2 > &v)
- [wbvector](#) & **init2ref** (const [wbvector](#) &v)
- [wbvector](#) & **init2ref** (size_t l, const char *d0)
- [wbvector](#) & **unRef** ()
- int **init_Struct** (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)
- int **init_base** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)
- int **init_mpf** (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)
- void **initDef** (size_t n)
- void **initDef** (const [wbvector](#) &v)
- [wbvector](#) & **operator=** (const [wbvector](#) &v)
- [wbvector](#)< char > & **operator=** (const [wbvector](#)< TB > &v)
- [wbvector](#) & **set** (const char &x)
- [wbvector](#) & **set** (const [wbvector](#)< size_t > &l, const char &x)
- void **set** (size_t offset, const char *x, size_t n)
- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- [wbvector](#) & **setMin** (const char &x)
- [wbvector](#) & **setMax** (const char &x)
- [wbvector](#) & **add** (const [wbvector](#)< size_t > &l, const char &x)
- [wbvector](#) & **add** (const char *d)
- [wbvector](#) & **Set** (const [wbvector](#)< size_t > &l, const [wbvector](#) &v)
- void **Sort** ([wbperm](#) &p, char dir=+1)
- void **Sort** (char dir=+1)
- void **sort** ([wbvector](#) &v2, [wbperm](#) &p) const
- void **sort** ([wbvector](#) &v2) const
- [wbperm](#) & **pSort** ([wbperm](#) &p) const
- bool **isSorted** () const
- bool **isSortedU** () const
- bool **isUnique** () const
- int **contains** (const char &x) const
- bool **isNormal** () const

- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isFinite** () const
- size_t **find1** (const char &x) const
- size_t **find1** () const
- size_t **count** (const char &x) const
- size_t **icount** (const char &x) const
- size_t **nnz** () const
- size_t **nnz** (char eps) const
- size_t **numZeros** (char eps=char(1E-14)) const
- size_t **numEQ** (const char &) const
- size_t **numGT** (const char &) const
- size_t **numLT** (const char &) const
- [wbindex](#) & **find** (const char &x, [wbindex](#) &l, char iflag=0) const
- [wbindex](#) & **find** (const char &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **find** ([wbindex](#) &l) const
- [wbindex](#) & **find** ([wbindex](#) &l, [wbindex](#) &lx) const
- [wbvector](#)< size_t > **find** () const
- [wbindex](#) & **findGT** (const char &x, [wbindex](#) &l, char iflag=0) const
- [wbindex](#) & **findGT** (const char &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **findzeros** ([wbindex](#) &l) const
- [wbindex](#) & **findRange** (const char &x1, const char &x2, [wbindex](#) &l, const char *w=NULL) const
- int **findClosestSorted** (char r) const
- int **findClosest** (char r) const
- size_t **findValues** (const [wbvector](#) &v, [wbindex](#) &l) const
- [wbvector](#) & **setp** (const char *x)
- void **reset** ()
- void **cpy** (const char *v)
- void **cpy** (const char *x, size_t m)
- [wbvector](#) & **save2** ([wbvector](#) &v)
- void **swap** ([wbvector](#) &v)
- [wbvector](#) & **swap** (size_t i, size_t j)
- [wbvector](#) & **Collect** (char &A1, char &A2)
- [wbvector](#) & **Collect** (char &A1, char &A2, char &A3)
- [wbvector](#) & **Skip** (size_t)
- [wbvector](#) & **Skip** (const WBINDEX &, char rflag=0)
- [wbvector](#) & **Move2_FE** (const WBINDEX &l, char FE_flag)
- [wbvector](#) & **Move2front** (const WBINDEX &l)
- [wbvector](#) & **Move2front** (size_t k)
- [wbvector](#) & **Move2end** (const WBINDEX &l)
- [wbvector](#) & **Move2end** (size_t k)
- [wbvector](#) & **mvLastTo** (size_t k)
- [wbvector](#) & **mvFirstTo** (size_t k)
- bool **checkSameLength** (const [wbvector](#) &a, const char *F=NULL, int L=0, const char *istr="") const
- bool **checkSameLength** (const [wbvector](#) &a, const [wbvector](#) &b, const char *F=NULL, int L=0, const char *istr="") const
- void **operator+=** (const T_ &a)
- void **operator+=** (const [wbvector](#) &b)
- void **operator-=** (const T_ &a)
- void **operator-=** (const [wbvector](#) &b)

- [wbvector](#) & **operator*=** (const T_ &a)
- [wbvector](#) & **operator/=** (const T_ &a)
- [wbvector](#) **operator*** (const T_ &a) const
- [wbvector](#) **operator+** (const T_ &a) const
- [wbvector](#) **operator+** (const [wbvector](#) &b) const
- [wbvector](#) **operator-** (const T_ &a) const
- [wbvector](#) **operator-** (const [wbvector](#) &b) const
- [wbvector](#) & **Plus** (const char *b)
- [wbvector](#) & **Plus** (const char *b, char bfac)
- [wbvector](#) & **Plus** (const [wbvector](#) &b, char bfac=1)
- void **plus** (const [wbvector](#) &b, [wbvector](#) &c, char bfac=1, char cfac=0) const
- [wbvector](#) & **times** (char fac, [wbvector](#) &x) const
- [wbvector](#) & **RevertSigns** ()
- int **isum** () const
- char **sum** () const
- char **sum** (size_t i1, const char *estr) const
- char **sum** (size_t i1, size_t i2) const
- char **cumsum_** ([wbvector](#) &S, char xflag=0) const
- char **Cumsum** (char zflag=0) const
- char **cumsum0prod** ([wbvector](#)< T2 > &b, [wbvector](#)< char > &cs, char xflag) const
- [wbvector](#) & **Eldiv** (const [wbvector](#) &v) const
- [wbvector](#) & **tensorProd** (const [wbvector](#) &v2, [wbvector](#) &vv) const
- char **scalarProd** (const [wbvector](#) &v) const
- char **scalarProd** (const char *v) const
- char **norm** () const
- char **norm2** () const
- double **normDiff** (const [wbvector](#) &v, char fac=1) const
- char **normDiff2** (const [wbvector](#) &v, char fac=1) const
- double **normDiff2_** (const [wbvector](#) &v, double fac=1) const
- char **GSProject** (const char *v, char isnorm=0, char tflag=0) const
- [wbvector](#) & **Normalize** (const char *F=NULL, int L=0)
- [wbvector](#) & **subtract** (const char *d)
- void **getReal** ([wbvector](#)< double > &R) const
- void **getReal** ([wbvector](#)< double > &R) const
- void **getImag** ([wbvector](#)< double > &I) const
- void **getImag** ([wbvector](#)< double > &I) const
- [wbvector](#) & **Conj** ()
- [wbvector](#)< [wbcomplex](#) > & **Conj** ()
- [wbvector](#) & **applyFlag** (char &flag, const char &fac, [wbvector](#) &a)
- bool **allEqual** () const
- bool **allEqual** (const char &a) const
- bool **allZero** () const
- bool **allOnes** () const
- bool **allEqual2** (const char &a, char eps=1E-15) const
- bool **allUnequal** (const char &a) const
- bool **any** () const
- bool **anyUnequal** (const char &a) const
- bool **anyUnequal** (const [wbvector](#)< size_t > &l, const char &x, size_t *k=NULL) const
- bool **anyEqual** (const char &a, size_t *i1=NULL) const
- bool **anyEqual** (const [wbvector](#)< size_t > &l, const char &x, size_t *k=NULL) const
- bool **anyGT** (const char &a) const
- bool **anyGE** (const char &a) const
- bool **anyLT** (const char &a) const
- bool **anyLE** (const char &a) const
- bool **allGT** (const char &a) const

- bool **allGE** (const char &a) const
- bool **allLE** (const char &a) const
- bool **allLT** (const char &a) const
- bool **allIn** (const char &a1, const char &a2) const
- bool **anySmallVals** (const char eps=1E-15) const
- bool **isEqual** (const char *v) const
- bool **isEqual** (const [wbvector](#) &v) const
- bool **isEmpty** () const
- **operator bool** () const
- bool **operator!** () const
- bool **operator==** (const [wbvector](#) &v) const
- bool **operator!=** (const [wbvector](#) &v) const
- bool **isEqualR** (const [wbvector](#) &v) const
- bool **operator^** (const [wbvector](#) &v) const
- bool **deepEqualP** (const [wbvector](#) &B) const
- char **cmp** (const [wbvector](#) &v) const
- bool **operator<** (const [wbvector](#) &v) const
- int **operator<** (const char &r) const
- bool **operator<=** (const [wbvector](#) &v) const
- bool **operator>** (const [wbvector](#) &v) const
- bool **operator>=** (const [wbvector](#) &v) const
- char &**el** (size_t i) const
- char **elx** (size_t i, const char &x) const
- const char & **operator[]** (size_t i) const
- char & **operator[]** (size_t i)
- [wbvector](#) **operator[]** (const [wbvector](#) < size_t > &l) const
- char & **last** () const
- char & **last** (size_t &i) const
- char & **last** (T1 &i_) const
- char & **p2last** () const
- char * **ref** (size_t i) const
- [wbvector](#) & **setRand** (double fac=1., double shift=0.)
- [wbvector](#) < [wbcomplex](#) > & **setRand** (double fac, double shift)
- void **setIdx** (double shift=0., double fac=1.)
- int **set2Group** (const [wbperm](#) &P, const WBINDEX &D, const char *S0=NULL, char iflag=0)
- [wbvector](#) & **Resize_** (size_t n)
- [wbvector](#) & **Shorten2** (size_t n)
- [wbvector](#) & **Resize** (size_t n, const char *d=NULL)
- [wbvector](#) & **resize** (size_t n, [wbvector](#) &X) const
- [wbvector](#) & **cat** (const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#) < [wbvector](#) > &vv)
- [wbvector](#) & **Cat** (const [wbvector](#) < [wbvector](#) * > &vv)
- [wbvector](#) & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)
- [wbvector](#) & **Cat** (const [wbvector](#) < T1 > *v1, const [wbvector](#) < T2 > *v2, const [wbvector](#) < T3 > *v3=0, const [wbvector](#) < T3 > *v4=0)
- [wbvector](#) < char > & **Cat** (const [wbvector](#) < T1 > *v1, const [wbvector](#) < T2 > *v2, const [wbvector](#) < T3 > *v3, const [wbvector](#) < T3 > *v4)
- [wbvector](#) < char > & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)
- [wbvector](#) & **Append** (const [wbvector](#) &v2)
- [wbvector](#) & **Append** (const char &x)
- [wbvector](#) & **Append** (size_t d2, const char *x)
- [wbvector](#) **append** (const [wbvector](#) &v) const

- **wbvector** & **append** (const **wbvector** &v2, **wbvector** &vout) const
- void **append** (const char *x, size_t d2, **wbvector** &vout) const
- int **recLess** (const char *v) const
- int **recLessE** (const char *v) const
- int **recCompare** (const char *v) const
- char **max** () const
- char **max** (size_t *k) const
- char **max** (unsigned &k) const
- char **max_** (char x) const
- char **min** () const
- char **min** (size_t *k) const
- char **min** (unsigned &k) const
- char **min_** (char x) const
- char **aMax** (size_t *k=NULL) const
- char **aMax** (unsigned &k) const
- char **aMin** (char zflag=0, size_t *k=NULL) const
- char **maxneg** (size_t *k=NULL) const
- char **minpos** (size_t *k=NULL) const
- char **avg** () const
- char **std** () const
- void **range** (char &xmin, char &xmax) const
- char **range** () const
- **wbvector** & **Permute** (const **wbperm** &P, char iflag=0)
- **wbvector** & **permute** (**wbvector** &, const **wbperm** &P, char iflag=0) const
- **wbvector** **permute** (const **wbperm** &P, char iflag=0) const
- **wbvector** & **blockPermute** (const **wbperm** &P, **wbvector** &, char iflag=0) const
- **wbvector** & **BlockPermute** (const **wbperm** &P, char iflag=0)
- int **blockCompare** (const **wbperm** &pa, const **wbvector** &b, const **wbperm** &pb) const
- void **get** (const WBINDEX &I0, **wbvector** &v) const
- void **get** (const **wbvector**< size_t > &I, char *v) const
- void **get** (size_t i1, size_t i2, **wbvector** &v) const
- void **selectSU** (const WBINDEX &I)
- void **select** (const WBINDEX &I, char *r) const
- **wbvector** & **select** (const WBINDEX &I, **wbvector** &a) const
- **wbvector** **select** (const WBINDEX &I) const
- **wbvector** & **Select** (const WBINDEX &I)
- **wbvector** & **BlockSelect** (const WBINDEX &I, const WBINDEX &D)
- **wbvector** & **getI** (size_t k, **wbvector** &v) const
- **wbvector** & **getI** (const **wbindex** &k, **wbvector** &v, char uflag=1) const
- **wbvector** & **getI** (const WBINDEX &k, **wbvector** &v, char uflag=1) const
- **wbvector** & **initI** (const **wbvector** &a, const **wbindex** &ia)
- **wbvector** & **initI** (const **wbvector** &a, const **wbindex** &ia, const **wbvector** &b, const **wbindex** &ib, char uflag=1)
- **wbvector** & **Flip** ()
- void **flip** (**wbvector** &a) const
- **wbvector** **flip** () const
- char **prod** () const
- char **prod** (char x) const
- char **prod** (size_t i, size_t i2) const
- char **prod** (size_t i, size_t i2, char x) const
- char **prod** (const T1 *I, size_t n) const
- T2 **prod** (const T1 *I, size_t n, T2 &s2) const
- void **prod2** (char &p1, char &p2, const char &p0=0) const
- void **info** (const char *istr) const
- void **print** (const char *istr, char mflag=0) const
- void **printdata** (const char *istr=0) const

- void **put** (const char *F, int L, const char *vname="ans", char tflag=0, const char *ws="base") const
- void **put** (const char *vname="ans", char tflag=0, const char *ws="base") const
- void **dispWithMsg** (const char *file, int line, const char *msg, const char *vname="") const
- void **mat2mx** (mxArray *&a, char dim=2) const
- void **mat2mxs** (mxArray *a, char field_nr, char dim=2) const
- mxArray * **toMx_Struct** (const char tflag=0) const
- mxArray * **toMxP_S** (const char tflag=0) const
- mxArray * **toMx** (const char tflag=0) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMxP** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base_d** (const char tflag=0) const
- mxArray * **toMx_offset** (const char &offset, const char tflag=0) const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tflag=0) const
- void **add2MxStruct** (mxArray *S, const char *vname, char tflag=0) const
- mxArray * **mxCreateCell** (unsigned m, unsigned n) const
- void **add2MxCell** (mxArray *S, unsigned i, char tflag=0) const
- void **putg** (const char *vname="ans", char tflag=0) const
- void **putx** (const char *vname="ans", char tflag=0) const
- [wbstring](#) **toStr** (int n, const char *sep) const
- [wbstring](#) **toStr** () const
- [wbstring](#) **toStr** (int n, const char *sep) const
- [wbstring](#) **sizeStr** () const
- [wbstring](#) **toStrf** (const char *fmt0="", const char *sep=" ", unsigned stride=0, const char *sep2="") const
- [wbstring](#) **toStrf** (const char *fmt_, const char *sep, unsigned stride, const char *sep2) const

Additional Inherited Members

Public Attributes inherited from [wbvector< char >](#)

- char * **data**
- size_t **len**
- char **isref**

Protected Member Functions inherited from [wbvector< char >](#)

- [wbvector](#) & **RENEW** (size_t n, const char *d=NULL, size_t l=-1, char init=1)
- [wbvector](#) & **RENEW_VAL** (size_t n, const char &x)

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc

4.65 QHash< TQ > Class Template Reference

Public Member Functions

- `size_t operator()` (const [QSet](#)< TQ > &S, const char *istr=0) const

The documentation for this class was generated from the following file:

- `clebsch.hh`

4.66 QHash_ Class Reference

Public Member Functions

- `size_t operator()` (const [qset](#)< double > &x) const

The documentation for this class was generated from the following file:

- `wbhash.cc`

4.67 QMap< TQ > Class Template Reference

Public Member Functions

- `void init ()`
- `QMap< TQ > & init` (const char *F, int L, const [QVec](#) &qv, const [wbMatrix](#)< TQ > &q1, const [wbMatrix](#)< TQ > &q2, const [wbMatrix](#)< [wbMatrix](#)< TQ > > &QQ, const [wbMatrix](#)< [wbMatrix](#)< const [CRef](#)< TQ > * > > *SS=NULL)
- `QMap< TQ > & init` (const char *F, int L, const mxArray *a)
- `template<class TD >`
`int getCGZlist` (const char *F, int L, [wbMatrix](#)< TQ > &qv, [wbvector](#)< TD > &dc, [wbvector](#)< widx_t > *IC=NULL, const [wbvector](#)< widx_t > *lx=NULL, const [qset](#)< TQ > *q3=NULL, [wbIndex](#) *m3=NULL, const [wbvector](#)< TD > *cc=NULL, double eps1=1E-10, double eps2=1E-14) const
- `void Skip` (const [wbindex](#) &l)
- `template<class TD >`
`void getIdentityQ` (const char *F, int L, const [wbvector](#)< widx_t > &Sa, const [wbvector](#)< widx_t > &Sb, [QSpace](#)< TQ, TD > &A, char vflag=1) const
- `bool isConsistent` (const char *F=NULL, int L=0, const [QVec](#) *qv=NULL) const
- `mxArray * mxCreateStruct` (unsigned m, unsigned n) const
- `void add2MxStruct` (mxArray *S, unsigned i, char tst=0) const
- `mxArray * toMx ()` const
- `void put` (const char *vname, const char *ws="caller") const
- `void put` (const char *F, int L, const char *vname, const char *ws="caller") const

Public Attributes

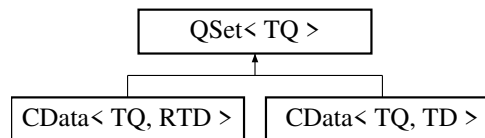
- [QVec](#) `qvec`
- [wbMatrix](#)< TQ > `Q1`
- [wbMatrix](#)< TQ > `Q2`
- [wbMatrix](#)< TQ > `Q`
- [wbvector](#)< widx_t > `D`
- [wbvector](#)< widx_t > `I1`
- [wbvector](#)< widx_t > `I2`
- [wbMatrix](#)< widx_t > `I1`
- [wbMatrix](#)< const [CRef](#)< TQ > * > `cg3`

The documentation for this class was generated from the following files:

- `clebsch.hh`
- `clebsch_aux.cc`
- `clebsch_io.cc`

4.68 QSet< TQ > Class Template Reference

Inheritance diagram for QSet< TQ >:



Public Member Functions

- **QSet** (const [QSet](#) &B)
- **QSet** (const [QType](#) &t_, const [qset](#)< TQ > &qset_, unsigned iout=0, char isref=0)
- **QSet** (const [QType](#) &t_, const [iTags](#) &it, unsigned r=-1)
- **QSet** (const [CRef](#)< TQ > &A)
- **QSet** (const char *F, int L, const mxArray *a)
- **QSet** (const mxArray *a)
- **QSet** (const char *F, int L, const [CRef](#)< TQ > &A, const [ctrlIdx](#) &ica, const [CRef](#)< TQ > &B, const [ctrlIdx](#) &icb, [wbperm](#) *P=NULL)
- **QSet** (const char *F, int L, const char *s)
- **QSet & init** (const [CRef](#)< TQ > &A)
- **QSet & init** (const char *F, int L, const mxArray *a, unsigned k=0)
- **QSet & init** (const [QType](#) t_=QTYPE_UNKNOWN)
- **QSet & init** (const [QSet](#) &B)
- **QSet & init2ref** (const [QSet](#) &B)
- **QSet & init** (const [QType](#) &t_, const TQ *q, const char *qd)
- **QSet & init** (const [QType](#) &t_, const [qset](#)< gTQ > &qset_, unsigned iout=0, char isref=0)
- **QSet & init** (const [QType](#) t_, const TQ *qs_, unsigned r, unsigned N, unsigned iout=0)
- **QSet & init** (const [QType](#) t_, const TQ *qs_, unsigned r, unsigned N, const [iTags](#) &it)
- **QSet & init1J** (const [QType](#) &t_, const TQ *q, char iflag=0)
- **QSet & init2** (const [QType](#) &t_, const TQ *q=NULL)

- **QSet** & **init2** (const **QType** &t_, const **qset**< TQ > &q1, const **qset**< TQ > &q2, const char *qds="++")
- **QSet** & **init1** (const **QType** &t_, TQ *q_=NULL)
- **QSet** & **init3** (const **QType** &t_)
- **QSet** & **init3** (const **QType** &t_, const **qset**< TQ > &J1, const **qset**< TQ > &J2, const **qset**< TQ > &J)
- **QSet** & **init3** (const **QType** &t_, const TQ &J1, const TQ &J2, const TQ &J)
- **QSet** & **init** (const char *F, int L, const **CRef**< TQ > &A, const **ctrlIdx** &ica, const **CRef**< TQ > &B, const **ctrlIdx** &icb, **wbperm** *Pcgp=NULL)
- **QSet** & **init_str** (const char *F, int L, const char *s)
- **QSet** & **Conj** ()
- **QSet** & **Conj** (unsigned i)
- unsigned **rank** (const char *F=NULL, int L=0) const
- **QSet** & **permute** (**QSet** &B, const **wbperm** &P, char iflag=0) const
- **QSet** & **Permute** (const **wbperm** &P, char iflag=0)
- void **swap** (**QSet** &X)
- **QSet** & **save2** (**QSet** &X)
- **QSet** & **operator=** (const **QSet** &B)
- bool **operator==** (const **QSet** &B) const
- bool **operator!=** (const **QSet** &B) const
- char **cmp** (const **QSet**< TQ > &B) const
- bool **operator<** (const **QSet**< TQ > &B) const
- **operator bool** () const
- bool **operator!** () const
- bool **isEmpty** (char check=3) const
- bool **gotRCData** () const
- int **isZero** () const
- unsigned **isStd3** () const
- unsigned **checkStd3** () const
- int **init2opt3** (const **QSet**< TQ > &Q, **wbperm** &P, **wbvector**< char > &cflags)
- **QSet** & **Convert** (const char *s, **wbvector**< char > &cflags)
- **wbstring** **sizeStr** () const
- bool **isScalar** () const
- bool **is1JSymbol** () const
- bool **isAbelian** () const
- int **permitsOM** () const
- int **checkQ_abelian** (const char *F, int L) const
- int **checkQ_SU2** (const char *F, int L) const
- **QSet** & **Sort** (**wbperm** *cgp=NULL, char *conj=NULL, char iflag=0)
- bool **isSorted** () const
- size_t **memSize** () const
- **wbstring** **QStrS** (const **wbperm** *cgp=NULL, char sep='|') const
- **wbstring** **QStr** () const
- **wbstring** **toStr** () const
- **wbstring** **toStr** (const char *istr) const
- **wbstring** **toTag** () const
- mxArray * **toMx** () const

Public Attributes

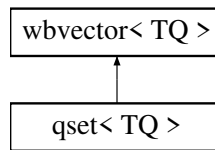
- **QType** t
- **qset**< TQ > qs
- **QDir** qdir

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc
- clebsch_io.cc

4.69 qset< TQ > Class Template Reference

Inheritance diagram for qset< TQ >:



Public Member Functions

- **qset** (unsigned I, const TQ *d0=NULL, char ref=0)
- **qset** (const qset< TQ > &q)
- **qset** (const QType &Q, const TQ *qq)
- **qset** (const QVec &Q, const TQ *qq)
- **qset** (const char *F, int L, const mxArray *a)
- **qset** (const char *F, int L, const mxArray *a, const QVec *Q, char check_type=1)
- **qset** (const qset< TQ > &q1, const qset< TQ > &q2)
- **qset** (const qset< TQ > &q1, const qset< TQ > &q2, const qset< TQ > &q3)
- **qset** (const qset< TQ > &q1, char op, const qset< TQ > &q2)
- **qset & init** (const qset< TQ > &q1)
- **qset & operator=** (const qset< TQ > &q1)
- **qset & init** (const char *F, int L, const char *s, QType *t=NULL)
- **qset & init** (const char *F, int L, const mxArray *a, const QVec *Q=NULL, char check_type=1)
- **qset & init** (const QType &Q, const TQ *qq)
- **qset & init** (const QVec &Q, const TQ *qq)
- **qset & init** (const qset< TQ > &q1, const qset< TQ > &q2)
- **qset & init** (const qset< TQ > &q1, const qset< TQ > &q2, const qset< TQ > &q3)
- **qset & init** ()
- **qset & init** (unsigned I, const TQ *d0=NULL)
- **qset & init** (unsigned I1, const TQ *d1, unsigned I2, const TQ *d2)
- **qset & init** (unsigned I1, const TQ *d1, unsigned I2, const TQ *d2, unsigned I3, const TQ *d3)
- **qset< TQ > & init** (const TQ q1)
- **qset< TQ > & init** (const TQ q1, const TQ q2)
- **qset< TQ > & init** (const TQ q1, const TQ q2, const TQ q3)
- **qset< TQ > & initStride** (const TQ *d_, unsigned n, unsigned r, unsigned QDIM)
- **bool operator<** (const qset< TQ > &b) const
- **int checkNQs** (const QVec &qq, const char *F=NULL, int L=0)
- **bool isConsistent** (const QVec &qq, int rank=-1, const char *F=NULL, int L=0) const
- **unsigned QDim** (const QVec &q, int rank=-1) const
- **qset< TQ > & times** (TQ fac, qset< TQ > &x) const
- **wbstring toStr** () const
- **wbstring toStr** (const QType &t QS_UNUSED_VAR) const
- **bool operator<** (const qset< double > &b) const
- **bool operator<** (const qset< float > &b) const

Public Member Functions inherited from [wbvector< TQ >](#)

- [wbvector](#) (size_t l, TQ x)
- [wbvector](#) (size_t l, const TQ *d0=NULL, char ref=0)
- [wbvector](#) (const [wbvector](#) &v)
- [wbvector](#) (const [wbvector](#)< T2 > &v)
- [wbvector](#) (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) (const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) & [init](#) (size_t l=0)
- [wbvector](#) & [init](#) (size_t l, const TQ *d0)
- [wbvector](#) & [init](#) (size_t l, const TQ *d0, char ref)
- [wbvector](#) & [init](#) (const [wbvector](#) &v)
- [wbvector](#) & [init](#) (size_t n, const [wbvector](#) &v)
- [wbvector](#)< TQ > & [init](#) (const [wbsparray](#)< TQ > &S)
- [wbvector](#) & [init](#) (size_t l1, const TQ *d1, size_t l2, const TQ *d2)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type QS_UNUSED_↵VAR=1, char ref=0)
- int [init](#) (const mxArray *a, const char *istr=NULL, char check_type=1)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- [wbvector](#) & [initp](#) (size_t l, const TQ *d0, size_t n, char init=1)
- [wbvector](#) & [init2val](#) (size_t l, const TQ &x)
- [wbvector](#) & [initStride](#) (size_t l, const TQ *d0, size_t stride)
- [wbvector](#) & [init_stride](#) (const TQ *d0, size_t n, size_t m, size_t D0)
- [wbvector](#) & [init_data](#) (const TQ *d0)
- [wbvector](#) & [init_data](#) (const T0 *d0)
- [wbvector](#) & [init_data](#) (const char *F, int L, const T0 *d0)
- [wbvector](#)< TQ > & [initT](#) (const [wbsparray](#)< T2 > &S)
- [wbvector](#)< TQ > & [initT](#) (size_t l, const T2 *v)
- [wbvector](#)< TQ > & [initT](#) (const [wbvector](#)< T2 > &v)
- [wbvector](#)< TQ > & [initT](#) (const char *F, int L, const [wbvector](#)< T2 > &v)
- [wbvector](#)< TQ > & [initT](#) (const [wbsparray](#)< TD > &S)
- [wbvector](#)< TQ > & [initT_](#) (const [wbvector](#)< T2 > &v)
- [wbvector](#) & [init2ref](#) (const [wbvector](#) &v)
- [wbvector](#) & [init2ref](#) (size_t l, const TQ *d0)
- [wbvector](#) & [unRef](#) ()
- int [init_Struct](#) (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)
- int [init_base](#) (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)
- int [init_mpfr](#) (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)
- void [initDef](#) (size_t n)
- void [initDef](#) (const [wbvector](#) &v)
- [wbvector](#) & [operator=](#) (const [wbvector](#) &v)
- [wbvector](#)< TQ > & [operator=](#) (const [wbvector](#)< TB > &v)
- [wbvector](#) & [set](#) (const TQ &x)
- [wbvector](#) & [set](#) (const [wbvector](#)< size_t > &l, const TQ &x)
- void [set](#) (size_t offset, const TQ *x, size_t n)
- void [set](#) (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- void [set](#) (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- [wbvector](#) & [setMin](#) (const TQ &x)
- [wbvector](#) & [setMax](#) (const TQ &x)

- `wbvector` & `add` (const `wbvector< size_t >` &l, const `TQ` &x)
- `wbvector` & `add` (const `TQ` *d)
- `wbvector` & `Set` (const `wbvector< size_t >` &l, const `wbvector` &v)
- void `Sort` (`wbperm` &p, char dir=+1)
- void `Sort` (char dir=+1)
- void `sort` (`wbvector` &v2, `wbperm` &p) const
- void `sort` (`wbvector` &v2) const
- `wbperm` & `pSort` (`wbperm` &p) const
- bool `isSorted` () const
- bool `isSortedU` () const
- bool `isUnique` () const
- int `contains` (const `TQ` &x) const
- bool `isNormal` () const
- bool `isNormal` () const
- bool `isNormal` () const
- bool `isNormal` () const
- bool `isNormal` () const
- bool `isNormal` () const
- bool `isNormal` () const
- bool `isNormal` () const
- bool `isNormal` () const
- bool `isNormal` () const
- bool `isFinite` () const
- `size_t` `find1` (const `TQ` &x) const
- `size_t` `find1` () const
- `size_t` `count` (const `TQ` &x) const
- `size_t` `icount` (const `TQ` &x) const
- `size_t` `nnz` () const
- `size_t` `nnz` (`TQ` eps) const
- `size_t` `numZeros` (`TQ` eps=`TQ`(1E-14)) const
- `size_t` `numEQ` (const `TQ` &) const
- `size_t` `numGT` (const `TQ` &) const
- `size_t` `numLT` (const `TQ` &) const
- `wbindex` & `find` (const `TQ` &x, `wbindex` &l, char iflag=0) const
- `wbindex` & `find` (const `TQ` &x, `wbindex` &l, `wbindex` &lx) const
- `wbindex` & `find` (`wbindex` &l) const
- `wbindex` & `find` (`wbindex` &l, `wbindex` &lx) const
- `wbvector< size_t >` `find` () const
- `wbindex` & `findGT` (const `TQ` &x, `wbindex` &l, char iflag=0) const
- `wbindex` & `findGT` (const `TQ` &x, `wbindex` &l, `wbindex` &lx) const
- `wbindex` & `findzeros` (`wbindex` &l) const
- `wbindex` & `findRange` (const `TQ` &x1, const `TQ` &x2, `wbindex` &l, const char *w=NULL) const
- int `findClosestSorted` (`TQ` r) const
- int `findClosest` (`TQ` r) const
- `size_t` `findValues` (const `wbvector` &v, `wbindex` &l) const
- `wbvector` & `setp` (const `TQ` *x)
- void `reset` ()
- void `cpy` (const `TQ` *v)
- void `cpy` (const `TQ` *x, `size_t` m)
- `wbvector` & `save2` (`wbvector` &v)
- void `swap` (`wbvector` &v)
- `wbvector` & `swap` (`size_t` i, `size_t` j)
- `wbvector` & `Collect` (`TQ` &A1, `TQ` &A2)
- `wbvector` & `Collect` (`TQ` &A1, `TQ` &A2, `TQ` &A3)
- `wbvector` & `Skip` (`size_t`)
- `wbvector` & `Skip` (const `WBINDEX` &, char rflag=0)
- `wbvector` & `Move2_FE` (const `WBINDEX` &l, char FE_flag)

- [wbvector](#) & **Move2front** (const WBINDEX &l)
- [wbvector](#) & **Move2front** (size_t k)
- [wbvector](#) & **Move2end** (const WBINDEX &l)
- [wbvector](#) & **Move2end** (size_t k)
- [wbvector](#) & **mvLastTo** (size_t k)
- [wbvector](#) & **mvFirstTo** (size_t k)
- bool **checkSameLength** (const [wbvector](#) &a, const char *F=NULL, int L=0, const char *istr="") const
- bool **checkSameLength** (const [wbvector](#) &a, const [wbvector](#) &b, const char *F=NULL, int L=0, const char *istr="") const
- void **operator+=** (const T_ &a)
- void **operator+=** (const [wbvector](#) &b)
- void **operator-=** (const T_ &a)
- void **operator-=** (const [wbvector](#) &b)
- [wbvector](#) & **operator*=** (const T_ &a)
- [wbvector](#) & **operator/=** (const T_ &a)
- [wbvector](#) **operator*** (const T_ &a) const
- [wbvector](#) **operator+** (const T_ &a) const
- [wbvector](#) **operator+** (const [wbvector](#) &b) const
- [wbvector](#) **operator-** (const T_ &a) const
- [wbvector](#) **operator-** (const [wbvector](#) &b) const
- [wbvector](#) & **Plus** (const TQ *b)
- [wbvector](#) & **Plus** (const TQ *b, TQ bfac)
- [wbvector](#) & **Plus** (const [wbvector](#) &b, TQ bfac=1)
- void **plus** (const [wbvector](#) &b, [wbvector](#) &c, TQ bfac=1, TQ cfac=0) const
- [wbvector](#) & **times** (TQ fac, [wbvector](#) &x) const
- [wbvector](#) & **RevertSigns** ()
- int **isum** () const
- TQ **sum** () const
- TQ **sum** (size_t i1, const char *estr) const
- TQ **sum** (size_t i1, size_t i2) const
- TQ **cumsum_** ([wbvector](#) &S, char xflag=0) const
- TQ **Cumsum** (char zflag=0) const
- TQ **cumsum0prod** ([wbvector](#)< T2 > &b, [wbvector](#)< TQ > &cs, char xflag) const
- [wbvector](#) & **Eldiv** (const [wbvector](#) &v) const
- [wbvector](#) & **tensorProd** (const [wbvector](#) &v2, [wbvector](#) &vv) const
- TQ **scalarProd** (const [wbvector](#) &v) const
- TQ **scalarProd** (const TQ *v) const
- TQ **norm** () const
- TQ **norm2** () const
- double **normDiff** (const [wbvector](#) &v, TQ fac=1) const
- TQ **normDiff2** (const [wbvector](#) &v, TQ fac=1) const
- double **normDiff2_** (const [wbvector](#) &v, double fac=1) const
- TQ **GSPProject** (const TQ *v, char isnorm=0, char tflag=0) const
- [wbvector](#) & **Normalize** (const char *F=NULL, int L=0)
- [wbvector](#) & **subtract** (const TQ *d)
- void **getReal** ([wbvector](#)< double > &R) const
- void **getReal** ([wbvector](#)< double > &R) const
- void **getImag** ([wbvector](#)< double > &I) const
- void **getImag** ([wbvector](#)< double > &I) const
- [wbvector](#) & **Conj** ()
- [wbvector](#)< [wbcomplex](#) > & **Conj** ()
- [wbvector](#) & **applyFlag** (char &flag, const TQ &fac, [wbvector](#) &a)
- bool **allEqual** () const
- bool **allEqual** (const TQ &a) const
- bool **allZero** () const

- bool **allOnes** () const
- bool **allEqual2** (const TQ &a, TQ eps=1E-15) const
- bool **allUnequal** (const TQ &a) const
- bool **any** () const
- bool **anyUnequal** (const TQ &a) const
- bool **anyUnequal** (const [wbvector](#)< size_t > &l, const TQ &x, size_t *k=NULL) const
- bool **anyEqual** (const TQ &a, size_t *i1=NULL) const
- bool **anyEqual** (const [wbvector](#)< size_t > &l, const TQ &x, size_t *k=NULL) const
- bool **anyGT** (const TQ &a) const
- bool **anyGE** (const TQ &a) const
- bool **anyLT** (const TQ &a) const
- bool **anyLE** (const TQ &a) const
- bool **allGT** (const TQ &a) const
- bool **allGE** (const TQ &a) const
- bool **allLE** (const TQ &a) const
- bool **allLT** (const TQ &a) const
- bool **allIn** (const TQ &a1, const TQ &a2) const
- bool **anySmallVals** (const TQ eps=1E-15) const
- bool **isEqual** (const TQ *v) const
- bool **isEqual** (const [wbvector](#) &v) const
- bool **isEmpty** () const
- **operator bool** () const
- bool **operator!** () const
- bool **operator==** (const [wbvector](#) &v) const
- bool **operator!=** (const [wbvector](#) &v) const
- bool **isEqualR** (const [wbvector](#) &v) const
- bool **operator^** (const [wbvector](#) &v) const
- bool **deepEqualP** (const [wbvector](#) &B) const
- char **cmp** (const [wbvector](#) &v) const
- bool **operator<** (const [wbvector](#) &v) const
- int **operator<** (const TQ &r) const
- bool **operator<=** (const [wbvector](#) &v) const
- bool **operator>** (const [wbvector](#) &v) const
- bool **operator>=** (const [wbvector](#) &v) const
- TQ & **el** (size_t i) const
- TQ **elx** (size_t i, const TQ &x) const
- const TQ & **operator[]** (size_t i) const
- TQ & **operator[]** (size_t i)
- [wbvector](#) **operator[]** (const [wbvector](#)< size_t > &l) const
- TQ & **last** () const
- TQ & **last** (size_t &i) const
- TQ & **last** (Tl &i_) const
- TQ & **p2last** () const
- TQ * **ref** (size_t i) const
- [wbvector](#) & **setRand** (double fac=1., double shift=0.)
- [wbvector](#)< [wbcomplex](#) > & **setRand** (double fac, double shift)
- void **setIdx** (double shift=0., double fac=1.)
- int **set2Group** (const [wbperm](#) &P, const WBINDEX &D, const TQ *S0=NULL, char iflag=0)
- [wbvector](#) & **Resize_** (size_t n)
- [wbvector](#) & **Shorten2** (size_t n)
- [wbvector](#) & **Resize** (size_t n, const TQ *d=NULL)
- [wbvector](#) & **resize** (size_t n, [wbvector](#) &X) const
- [wbvector](#) & **cat** (const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) > &vv)

- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) * > &vv)
- [wbvector](#) & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)
- [wbvector](#) & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3=0, const [wbvector](#)< T3 > *v4=0)
- [wbvector](#)< TQ > & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3, const [wbvector](#)< T3 > *v4)
- [wbvector](#)< TQ > & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)
- [wbvector](#) & **Append** (const [wbvector](#) &v2)
- [wbvector](#) & **Append** (const TQ &x)
- [wbvector](#) & **Append** (size_t d2, const TQ *x)
- [wbvector](#) **append** (const [wbvector](#) &v) const
- [wbvector](#) & **append** (const [wbvector](#) &v2, [wbvector](#) &vout) const
- void **append** (const TQ *x, size_t d2, [wbvector](#) &vout) const
- int **recLess** (const TQ *v) const
- int **recLessE** (const TQ *v) const
- int **recCompare** (const TQ *v) const
- TQ **max** () const
- TQ **max** (size_t *k) const
- TQ **max** (unsigned &k) const
- TQ **max_** (TQ x) const
- TQ **min** () const
- TQ **min** (size_t *k) const
- TQ **min** (unsigned &k) const
- TQ **min_** (TQ x) const
- TQ **aMax** (size_t *k=NULL) const
- TQ **aMax** (unsigned &k) const
- TQ **aMin** (char zflag=0, size_t *k=NULL) const
- TQ **maxneg** (size_t *k=NULL) const
- TQ **minpos** (size_t *k=NULL) const
- TQ **avg** () const
- TQ **std** () const
- void **range** (TQ &xmin, TQ &xmax) const
- TQ **range** () const
- [wbvector](#) & **Permute** (const [wbperm](#) &P, char iflag=0)
- [wbvector](#) & **permute** ([wbvector](#) &, const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) **permute** (const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) & **blockPermute** (const [wbperm](#) &P, [wbvector](#) &, char iflag=0) const
- [wbvector](#) & **BlockPermute** (const [wbperm](#) &P, char iflag=0)
- int **blockCompare** (const [wbperm](#) &pa, const [wbvector](#) &b, const [wbperm](#) &pb) const
- void **get** (const WBINDEX &I0, [wbvector](#) &v) const
- void **get** (const [wbvector](#)< size_t > &I, TQ *v) const
- void **get** (size_t i1, size_t i2, [wbvector](#) &v) const
- void **selectSU** (const WBINDEX &I)
- void **select** (const WBINDEX &I, TQ *r) const
- [wbvector](#) & **select** (const WBINDEX &I, [wbvector](#) &a) const
- [wbvector](#) **select** (const WBINDEX &I) const
- [wbvector](#) & **Select** (const WBINDEX &I)
- [wbvector](#) & **BlockSelect** (const WBINDEX &I, const WBINDEX &D)
- [wbvector](#) & **getI** (size_t k, [wbvector](#) &v) const
- [wbvector](#) & **getI** (const [wbindex](#) &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **getI** (const WBINDEX &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **initI** (const [wbvector](#) &a, const [wbindex](#) &ia)
- [wbvector](#) & **initI** (const [wbvector](#) &a, const [wbindex](#) &ia, const [wbvector](#) &b, const [wbindex](#) &ib, char uflag=1)

- **wbvector** & **Flip** ()
- void **flip** (**wbvector** &a) const
- **wbvector flip** () const
- TQ **prod** () const
- TQ **prod** (TQ x) const
- TQ **prod** (size_t i, size_t i2) const
- TQ **prod** (size_t i, size_t i2, TQ x) const
- TQ **prod** (const T1 *l, size_t n) const
- T2 **prod** (const T1 *l, size_t n, T2 &s2) const
- void **prod2** (TQ &p1, TQ &p2, const TQ &p0=0) const
- void **info** (const char *istr) const
- void **print** (const char *istr, char mflag=0) const
- void **printdata** (const char *istr=0) const
- void **put** (const char *F, int L, const char *vname="ans", char tflag=0, const char *ws="base") const
- void **put** (const char *vname="ans", char tflag=0, const char *ws="base") const
- void **dispWithMsg** (const char *file, int line, const char *msg, const char *vname="") const
- void **mat2mx** (mxArray *&a, char dim=2) const
- void **mat2mxs** (mxArray *a, char field_nr, char dim=2) const
- mxArray * **toMx_Struct** (const char tflag=0) const
- mxArray * **toMxP_S** (const char tflag=0) const
- mxArray * **toMx** (const char tflag=0) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMxP** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base_d** (const char tflag=0) const
- mxArray * **toMx_offset** (const TQ &offset, const char tflag=0) const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tflag=0) const
- void **add2MxStruct** (mxArray *S, const char *vname, char tflag=0) const
- mxArray * **mxCreateCell** (unsigned m, unsigned n) const
- void **add2MxCell** (mxArray *S, unsigned i, char tflag=0) const
- void **putg** (const char *vname="ans", char tflag=0) const
- void **putx** (const char *vname="ans", char tflag=0) const
- **wbstring toStr** (int n, const char *sep) const
- **wbstring toStr** () const
- **wbstring toStr** (int n, const char *sep) const
- **wbstring sizeStr** () const
- **wbstring toStrf** (const char *fmt0="", const char *sep=" ", unsigned stride=0, const char *sep2="") const
- **wbstring toStrf** (const char *fmt , const char *sep, unsigned stride, const char *sep2) const

Additional Inherited Members

Public Attributes inherited from [wbvector< TQ >](#)

- TQ * **data**
- size_t **len**
- char **isref**

Protected Member Functions inherited from [wbvector< TQ >](#)

- [wbvector](#) & **RENEW** (size_t n, const TQ *d=NULL, size_t l=-1, char init=1)
- [wbvector](#) & **RENEW_VAL** (size_t n, const TQ &x)

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc

4.70 QSpace< TQ, TD > Class Template Reference

Public Member Functions

- **QSpace** (unsigned n=0)
- **QSpace** (unsigned m, unsigned n, unsigned nq)
- **QSpace** (const [QSpace](#) &B)
- template<class TQ_, class TD_ >
 QSpace (const [QSpace](#)< TQ_, TD_ > &B, char xflag=[WbUtil](#)< TD_ >::isComplex() ? 1 : 0)
- **QSpace** (const char *F, int L, const mxArray *a, char ref=0, char skip_empty=1)
- **QSpace** (const mxArray *a, char ref=0, char skip_empty=1)
- [QSpace](#) & **clearQSpace** (char mflag=0)
- void **clearDATA** (char mflag=0)
- unsigned **rank** (const char *F=NULL, int L=0) const
- unsigned **len** () const
- unsigned **nsym** () const
- TQ * **Qref** (unsigned r, unsigned k)
- const TQ * **Qref** (unsigned r, unsigned k) const
- void **init** ()
- void **init** (unsigned m, unsigned n, unsigned nq)
- void **init** (const [wbMatrix](#)< TQ > &Q, unsigned qdim, const [QVec](#) *qt=NULL)
- [QSpace](#) & **init** (const [QSpace](#) &B)
- template<class TB >
 [QSpace](#) & **initT** (const [QSpace](#)< TQ, TB > &B)
- template<class TR >
 [QSpace](#) & **getReal** ([QSpace](#)< TQ, TR > &R) const
- [QSpace](#)< TQ, double > **getReal** () const
- template<class TI >
 [QSpace](#) & **getImag** ([QSpace](#)< TQ, TI > &I) const
- [QSpace](#)< TQ, double > **getImag** () const
- void **init** (const char *F, int L, const mxArray *S, char ref, unsigned k, char skip_empty=1)
- void **init** (const char *F, int L, const mxArray *S, char ref=0, char skip_empty=1)

- [QSpace](#) & **setupDATA** (const char *F=NULL, int L=0)
- int **gotCGS** (const char *F=NULL, int L=0, char xflag=0) const
- int **gotCGX** (const char *F=NULL, int L=0) const
- [QSpace](#) & **Enlarge** (unsigned m)
- [QSpace](#) & **Trim** (unsigned n)
- [QSpace](#) & **initScalar** (const TD &x)
- void **init2ref** (const mxArray *S)
- void **init2ref** (const [QSpace](#) &)
- void **initOpZ_WET** (const char *F, int L, const [QVec](#) &qvec, const [wbMatrix](#)< TQ > &Q1, const [wbMatrix](#)< TQ > &Q2, const [wbMatrix](#)< TQ > &Q, const [wbarray](#)< TD > &D3, const double eps1=1E-10, const double eps2=1E-13)
- void **reduceMatEI** (const char *F, int L, const [wbMatrix](#)< TQ > &Q1, const [wbvector](#)< TD > &dd, const [wbMatrix](#)< TQ > &Q2, const [wbvector](#)< TD > &dc, [wbvector](#)< unsigned > &l1, [wbvector](#)< unsigned > &l2, [wbvector](#)< TD > &dr) const
- void **unRef** ()
- void **initIdentity** (const [QSpace](#) &A, char dflag=0)
- void **initIdentity** (const [wbMatrix](#)< TQ > &Q)
- [QSpace](#) & **setupCGR** (const char *F=NULL, int L=0)
- [QSpace](#) & **initIdentityCGS** (const char *F=NULL, int L=0)
- [QSpace](#) & **initIdentityCGS** (const char *F, int L, unsigned k, unsigned r=0)
- template<class TD2 >
 [QSpace](#) & **initIdentityCGS** (const char *F, int L, const [QSpace](#)< TQ, TD2 > &H)
- template<class TA >
 [QSpace](#)< TQ, TD > & **initIdentityCG** (const [wbvector](#)< [QSpace](#)< TQ, TA > > &A, const [wbindex](#) &ia, char zflag=0)
- template<class TA , class TB >
 [QSpace](#)< TQ, TD > & **initIdentityCG** (const [wbvector](#)< [QSpace](#)< TQ, TA > > &A, const [wbindex](#) &ia, const [wbvector](#)< [QSpace](#)< TQ, TB > > &B, const [wbindex](#) &ib, char vflag='v')
- template<class TA , class TB >
 [QSpace](#)< TQ, TD > & **initIdentityCG** (const [wbvector](#)< [QSpace](#)< TQ, TA > > &A, const [wbvector](#)< [QSpace](#)< TQ, TB > > &B, const char *pstr=NULL)
- [QSpace](#) & **initDiagonal** (const [wbMatrix](#)< TQ > &Q, const [wbvector](#)< unsigned > &S, const [wbvector](#)< TD > &D, char rcflag)
- [QSpace](#) & **init2DiffOp** (const [QSpace](#) &A, unsigned ia, const [QSpace](#) &B, unsigned ib)
- template<class TB >
 [QSpace](#) & **initQT_safe** (const [QSpace](#)< TQ, TB > &B)
- template<class TA >
 [QSpace](#) & **initQ** (const [QSpace](#)< TQ, TA > &A, char full=1)
- template<class TA , class TB >
 [QSpace](#) & **initQ** (const char *F, int L, const [QSpace](#)< TQ, TA > &A, char full=1, const [QSpace](#)< TQ, TB > *B=NULL)
- void **initQ** (unsigned n, unsigned r, const [QSpace](#) &A)
- void **initQ** (const [wbMatrix](#)< TQ > &Qk, const [QSpace](#) &A)
- void **init_tags** (const char *F, int L, const char *s, unsigned r=-1, char lflag=-1)
- [QSpace](#) & **SetTag** (unsigned k, const [itag_](#) &b)
- [QSpace](#) & **SetTags** (const [itag_](#) &b)
- [QSpace](#) & **SetConjTags** ()
- [QSpace](#) & **SetConjTags** (unsigned k1, unsigned k2)
- [QSpace](#) & **SetConjTag** (unsigned k)
- [QSpace](#) & **SetConjTag** (const char *F, int L, unsigned k)
- [QSpace](#) & **SetFlag** (const char *F, int L, unsigned k, unsigned l)
- [QSpace](#) & **UnsetFlag** (const char *F, int L, unsigned k, unsigned l)
- [QSpace](#) & **UnsetFlags** ()
- [QSpace](#) & **SetFlags** (const char *F, int L, unsigned k, unsigned l=-1)
- [QSpace](#) & **init_tags** ()

- `template<class TB >`
`unsigned matchTags (const char *F, int L, const QSpace< TQ, TB > &B, ctrlidx &ia, ctrlidx &ib) const`
- `unsigned RemoveZLabels (const char *F, int L, wbMatrix< TQ > &QQ, wbMatrix< TQ > *Z=NULL) const`
- `void RemoveZLabels (const char *F, int L, wbMatrix< TQ > *Z)`
- `void ExpandDiagonal (unsigned i1=1, unsigned i2=2)`
- `size_t getDataSize (wbMatrix< size_t > &S) const`
- `size_t getDataSize (wbvector< size_t > &S) const`
- `size_t getDataSize () const`
- `size_t getCGSSize () const`
- `wbstring totSize2Str () const`
- `wbstring itags2Str () const`
- `unsigned map2Vec (const char *F, int L, wbvector< TD > &V, const wbvector< unsigned > &S=wbvector< unsigned >())`
- `unsigned map2Vec (TD *v, char lflag=0)`
- `void ExpandQ (CPAT< TQ, TD > &CP, double nrm, wbvector< unsigned > &xflag, char disp=0)`
- `void ResetRec (unsigned k)`
- `void setData (const TD c)`
- `void setRand (double nrm=0)`
- `void markUnequalZero (const TD m=1, const TD eps=0)`
- `void PrependSingletons (unsigned R)`
- `void swap (QSpace &B)`
- `QSpace & save2 (QSpace &B)`
- `unsigned Reduce2AbelianOp (const char *F=NULL, int L=0)`
- `char allAbelian () const`
- `char isNonAbelian () const`
- `bool permitsOM () const`
- `bool hasOM () const`
- `bool allAdditive () const`
- `bool isEmpty () const`
- `operator bool () const`
- `bool operator! () const`
- `bool isConsistent (const char *F, int L, unsigned &r, char level=3) const`
- `bool isConsistent (const char *F=NULL, int L=0) const`
- `bool isConsistent (unsigned &r, char level=3) const`
- `bool isConsistent_r (unsigned r, char level=3) const`
- `bool isScalar () const`
- `bool isComplex () const`
- `bool isDiagBlock (unsigned k) const`
- `bool isBlockDiagMatrix (const char *F, int L, char dflag=0) const`
- `bool isBlockDiagMatrix (char dflag=0) const`
- `bool isDiagMatrix (const double eps=1e-14) const`
- `bool isIdentityMatrix (const TD eps=1e-14) const`
- `char hasIdentityCGS (const TD eps=1e-14) const`
- `bool isR3Op () const`
- `bool isOp (int r=-1, char lflag=0) const`
- `int isOperator (unsigned *r_, char xflag=0) const`
- `bool isOperator () const`
- `bool isAtensor (unsigned r=-1) const`
- `int checkAbelianOp () const`
- `bool isAbelianOp () const`
- `bool isQSym (const char *F=0, int L=0, char dflag=0) const`
- `bool isHConj (const char *F=0, int L=0, double eps=1e-14, char vflag=0) const`
- `bool isAHerm (const char *F=0, int L=0, double eps=1e-14, char vflag=0) const`
- `int isSingleton (unsigned k) const`

- `template<class T2 >`
`bool hasSameQ (const QSpace< TQ, T2 > &B) const`
- `bool hasSameQ (unsigned r1, unsigned k1, unsigned r2, unsigned k2) const`
- `template<class T2 >`
`bool hasSameRank (const QSpace< TQ, T2 > &B) const`
- `char checkQ (const char *F=NULL, int L=0) const`
- `template<class TB >`
`void checkQ (const char *F, int L, const QSpace< TQ, TB > &B, char aflag=0) const`
- `char checkQ_CGR (const char *F, int L, C_UVEC *l=NULL) const`
- `int check_CGR_cgw (const char *F=0, int L=0) const`
- `void checkScalarCGS (const char *F=0, int L=0) const`
- `template<class TB >`
`bool sameType (const QSpace< TQ, TB > &B, int r=-1, const char *istr=NULL) const`
- `QDir & getQDir (QDir &qdir) const`
- `bool sameQDir (const char *F, int L, const QSpace &B) const`
- `bool hasQOverlap (const QSpace &A) const`
- `template<class TB >`
`int hasQOverlap (unsigned ica, const QSpace< TQ, TB > &B, unsigned icb, const char type=0) const`
- `int getQOverlapU (const char *F, int L, const QSpace &B, wbMatrix< TQ > &QQ, wbMatrix< int > &IQ, char olonly=0) const`
- `int getQOverlap (const char *F, int L, const QSpace &B, wbMatrix< TQ > &QQ, wbMatrix< int > &IQ, wbvector< widx_t > &DQ) const`
- `bool findDimQ (const TQ *q, unsigned &d) const`
- `bool findDimQ (const TQ *q, unsigned k, unsigned &d) const`
- `bool operator== (const QSpace &B) const`
- `bool operator!= (const QSpace &B) const`
- `template<class T >`
`void operator*= (const T fac)`
- `template<class T >`
`QSpace & times (const T fac, QSpace &B) const`
- `QSpace & operator= (const QSpace &A)`
- `void operator+= (const QSpace &Q)`
- `void operator-= (const QSpace &Q)`
- `QSpace & Cat (const char *F, int L, const QSpace< TQ, TD > &A, const QSpace< TQ, TD > &B, TD afac=1, TD bfac=1, char uflag=1)`
- `QSpace & plus (const QSpace &B, QSpace &C, TD bfac=1, char vflag=0) const`
- `QSpace & minus (const QSpace &B, QSpace &C, TD bfac=1, char vflag=0) const`
- `QSpace & Plus (const QSpace &A, TD bfac=1)`
- `QSpace & Minus (const QSpace &A, TD bfac=1)`
- `QSpace & plus_plain (const QSpace &B, QSpace &C, TD bfac=1) const`
- `int NormCGW (char full=0, char skipzeros=1, char rcpy=0)`
- `TD norm2 (char checks=1) const`
- `TD norm (char checks=1) const`
- `TD normDiff2 (const QSpace &B) const`
- `TD scalarProd (const QSpace &B) const`
- `void checkNorm (const char *F, int L, double nrm, char vflag=1, double eps=1e-12) const`
- `QSpace & TimesEI (const QSpace &B, char conj=0, unsigned r2=-1)`
- `TD sumData () const`
- `template<class TM >`
`wbMatrix< TM > & diag (wbMatrix< TM > &xd) const`
- `QSpace & TensorProd (const QSpace &B, QSpace &C, const char aflag='N', const char bflag='N') const`
- `template<class TC, ENABLE_IF_COMPLEX(TC) >`
`QSpace< TQ, TC > & TensorProd (const QSpace &B, QSpace< TQ, TC > &C, const char aflag='N', const char bflag='N') const`

- `template<class TC, ENABLE_IF_COMPLEX(TC) >`
`QSpace< TQ, TC > & TensorProd` (const `QSpace< TQ, TC > &B`, `QSpace< TQ, TC > &C`, const char `aflag='N'`, const char `bflag='N'`) const
- void `TensorProdUnity` (const `wbMatrix< TQ > &Q`, `QSpace &C`) const
- void `EigenSymmetric` (`wbMatrix< double > &Etot`) const
- void `EigenSymmetric` (`QSpace &Ak`, `QSpace< TQ, double > &Ek`, `wbMatrix< double > &Etot`) const
- void `EigenSymmetric` (`QSpace &Ak`, `QSpace< TQ, double > &Ek`) const
- void `EigenSymmetric` (`QSpace &Ak`, `QSpace &Ad`, `QSpace< TQ, double > &Ek`, `QSpace< TQ, double > &Et`, `wbMatrix< double > &Etot`, `wbMatrix< unsigned > &DD`, int `&Nkeep`, double `Etrunc=0`, double `*E0=NULL`, char `mKD=0`, const `wbperm &P=wbperm()`, double `eps=0.`, double `b=0.`, int `dmax=-1`, const char `*sort=NULL`) const
- void `GroupIndices` (unsigned `K`)
- int `ExpandOM` (const char `*F`, int `L`, `wbvector< unsigned > &iOM`, char `fflag=0`)
- int `ExpandOM_` (const char `*F`, int `L`, unsigned `ia`, `QSpace< TQ, TD > &B`, unsigned `ib`)
- int `ExpandOM_` (const char `*F`, int `L`, unsigned `ia`, unsigned `ib`)
- void `Append` (unsigned `k`)
- void `Append` (const `wbvector< TQ > &`, const `wbarray< TD > &D`)
- `QSpace & Append` (const char `*F`, int `L`, const `QSpace &A`, char `uflag=1`)
- int `Append2AndDestroy` (const char `*F`, int `L`, `QSpace &A`, char `unique=0`)
- `template<class TA, ENABLE_IF_COMPLEX(TA) >`
int `Append2AndDestroy` (const char `*F`, int `L`, `QSpace< TQ, TA > &A`, char `unique=0`)
- void `TransferSpace` (const char `*F`, int `L`, `QSpace &B`, unsigned `k`, const `QSpace< TQ, double > &W`, double `eps`)
- void `TransferSpace` (const char `*F`, int `L`, `QSpace &B`, unsigned `k`)
- unsigned `MakeUnique` ()
- bool `isZero` (const double `eps=0.`, char `flag=0`) const
- unsigned `SkipZeroData` (double `eps=0.`, char `bflag=0`, char `cgflag=1`, char `all=0`)
- unsigned `skipZeroOffDiag` (double `eps=0.`, char `bflag=0`)
- unsigned `SkipEmptyData` (const char `*F=0`, int `L=0`)
- bool `gotZeroData` (unsigned `i`, const double `eps=0.`) const
- void `Set` (unsigned `k`, const `wbvector< TQ > &Q`, const `wbarray< TD > &D`)
- void `SetQ` (unsigned `i`, unsigned `k`, const `TQ *d`)
- `QSpace & Select` (const `wbindex &l`, char `data_only=0`)
- `template<class TM >`
`QSpace & SelectMarked` (const `wbvector< TM > &mark`)
- void `Sort` (const `QSpace &A`)
- void `Sort` ()
- unsigned `getOM` (`wbMatrix< unsigned > &OM`, `wbvector< unsigned > *om1=NULL`, char `wflag=0`) const
- unsigned `cgsDimScalar` (unsigned `k`) const
- unsigned `cgsDim` (unsigned `k`, unsigned `r`) const
- unsigned `getDim` (unsigned `k`, `widx_t *D=NULL`) const
- unsigned `getDIM` (unsigned `k`) const
- void `getDim` (`wbvector< widx_t > &D`, `wbvector< widx_t > *DD=NULL`) const
- `wbvector< widx_t > getDim` () const
- void `getQDim` (unsigned `k`, `wbMatrix< TQ > &Q`, `wbvector< widx_t > &S`, `wbMatrix< widx_t > *SC=NULL`) const
- void `getQDim` (`wbMatrix< TQ > &Q`, `wbvector< widx_t > &S`, `wbMatrix< widx_t > *SC=NULL`) const
- void `getDRange` (`TD &dmin`, `TD &dmax`) const
- void `getDistQtot` (`wbMatrix< TQ > &Qtot`, `wbvector< double > &w`) const
- `widx_t getDQtot` ()
- `wbMatrix< TQ > & getQsub` (const `wbindex &l`, `wbMatrix< TQ > &Ql`) const
- `wbMatrix< TQ > & getQsub` (const unsigned `k`, `wbMatrix< TQ > &Qk`, char `iflag=0`) const
- void `getQsub` (const `wbindex &l`, `wbMatrix< TQ > &Ql`, `wbMatrix< TQ > &Q2`) const
- `wbMatrix< TQ > getQsub` (const unsigned `k`) const
- `wbMatrix< TQ > & getQtot` (`wbMatrix< TQ > &Qtot`, char `uflag=0`) const

- **wbMatrix**< TQ > & **getQsum** (const **wbindex** &J, **wbMatrix**< TQ > &Q) const
- **wbMatrix**< TQ > & **getQsum** (const **wbMatrix**< TQ > &Q0, **wbMatrix**< TQ > &Q) const
- **wbMatrix**< TQ > & **getQfinal** (const **wbindex** &J, **wbMatrix**< TQ > &Q) const
- **QSpace** & **getSub** (const **wbindex** &I, **QSpace** &A, char ref=0) const
- **QSpace** & **getSubInit** (const **wbindex** &I, **QSpace** &A) const
- void **saveSub2** (**QSpace** &A, const **wbindex** &I)
- void **toFull** (**wbarray**< TD > &A, **wbvector**< **wbMatrix**< TQ > > &Q, **wbvector**< **wbvector**< unsigned > > &S) const
- void **toFull2** (**wbarray**< TD > &A, **wbMatrix**< TQ > &Q, **wbMatrix**< widx_t > &S) const
- void **ExtendQ** (const **QSpace** &B)
- void **takeMainQS** (char disp=1)
- void **setQ** (unsigned k, const **QSpace** &B, unsigned k0)
- void **setQRec** (unsigned r, unsigned j, const TQ *q0, const TQ *q2=NULL)
- bool **hasEqualQ** (unsigned i1, unsigned i2) const
- double **maxDiff** (const **QSpace** &B) const
- void **PermuteQ** (const **wbperm** &P)
- void **permuteQ** (const **wbperm** &P, **wbMatrix**< TQ > &) const
- **QSpace** & **Permute** (const **wbperm** &P, char iflag=0, char rcpy=0)
- **QSpace** & **permute** (const **wbperm** &P, **QSpace** &B, char iflag=0) const
- void **permute** (const char *pstr, **QSpace** &B, unsigned offset=1) const
- void **Permute** (const char *pstr, unsigned offset=1)
- void **PermuteFirstTo** (unsigned k)
- void **permuteFirstTo** (unsigned k, **QSpace** &C)
- void **PermuteLastTo** (unsigned k)
- void **permuteLastTo** (unsigned k, **QSpace** &C)
- void **FlipQ** ()
- **QSpace** & **trace** (const char *F, int L, **ctrlidx** &i1, **ctrlidx** &i2, **QSpace** &C) const
- TD **trace** () const
- void **contractMat** (unsigned ia, const **QSpace** &B, unsigned ib, **QSpace** &C) const
- template<class TB, class TC >
int **contract_getIdxSet** (const char *F, int L, const **ctrlidx** &ica, const **QSpace**< TQ, TB > &B, const **ctrlidx** &icb, **wbindex** &Ia, **wbindex** &Ib, **wbvector**< widx_t > &Dc, **QSpace**< TQ, TC > &C) const
- template<class TB, class TC >
double **contract** (const char *F, int L, const **ctrlidx** &ica, const **QSpace**< TQ, TB > &B, const **ctrlidx** &icb, **QSpace**< TQ, TC > &C, const **wbperm** &P=**wbperm**(), char preview=0) const
- template<class TB, class TC >
double **contract** (const char *F, int L, unsigned ia, const **QSpace**< TQ, TB > &B, unsigned ib, **QSpace**< TQ, TC > &C, const **wbperm** &P=**wbperm**(), char preview=0) const
- template<class TB, class TC >
int **contract** (const char *F, int L, unsigned ia, const **QSpace**< TQ, TB > &B, const char *sb, **QSpace**< TQ, TC > &C, const **wbperm** &P=**wbperm**(), char preview=0) const
- template<class TB, class TC >
int **contract** (const char *F, int L, const char *sa, const **QSpace**< TQ, TB > &B, unsigned ib, **QSpace**< TQ, TC > &C, const **wbperm** &P=**wbperm**(), char preview=0) const
- template<class TB, class TC >
int **contract** (const char *F, int L, const char *sa, const **QSpace**< TQ, TB > &B, const char *sb, **QSpace**< TQ, TC > &C, const **wbperm** &P=**wbperm**(), char preview=0) const
- template<class TB, class TC >
int **contract** (const char *sa, const **QSpace**< TQ, TB > &B, const char *sb, **QSpace**< TQ, TC > &C, const **wbperm** &P=**wbperm**(), char preview=0) const
- template<class TB, class TC >
int **contract** (const **ctrlidx** &ica, const **QSpace**< TQ, TB > &B, const **ctrlidx** &icb, **QSpace**< TQ, TC > &C, const **wbperm** &P=**wbperm**(), char preview=0) const
- template<class TB, class TC >
int **contract** (unsigned ia, const **QSpace**< TQ, TB > &B, unsigned ib, **QSpace**< TQ, TC > &C, const **wbperm** &P=**wbperm**(), char preview=0) const

- `mxArray * toMx (bool setcls=0) const`
- `mxArray * save2Mx (char vflag=0)`
- `mxArray * QIDX_toMx () const`
- `mxArray * INFO_toMx () const`
- `mxArray * DATA_toMx () const`
- `mxArray * DATA_save2Mx (char vflag=0)`
- `mxArray * mxCreateStruct (unsigned m, unsigned n) const`
- `void add2MxStruct (mxArray *S, unsigned i, bool setcls=0, char chk=0) const`
- `void save2MxStruct (mxArray *S, unsigned i, char tst=0, char vflag=0)`
- `mxArray * mxCreateCell (unsigned m QS_UNUSED_VAR, unsigned n QS_UNUSED_VAR) const`
- `void add2MxCell (mxArray *S QS_UNUSED_VAR, unsigned i QS_UNUSED_VAR, char tst QS_UNUSED_VAR=0) const`
- `void put (const char *F, int L, const char *vname, const char *ws="caller") const`
- `void put (const char *vname, const char *ws="caller") const`
- `wbstring sizeStrQ (const char *F=NULL, int L=0) const`
- `wbstring sizeStr (char vflag=0) const`
- `void print_SIZE (const char *F=NULL, int L=0) const`
- `void print_rankDATA (const char *F=NULL, int L=0) const`
- `wbstring qStr () const`
- `wbstring qrec2Str (unsigned k) const`
- `void disp_cgs (const char *F, int L, const char *istr) const`
- `void info (const char *vname=NULL, unsigned nlt=0, unsigned nlb=0, unsigned nind=2) const`
- `void print () const`
- `void print (const char *vname, char vflag=3) const`
- `void print_tst_ (const char *F, int L, const char *istr, unsigned k=0) const`
- `void initOType (const char *F, int L, const mxArray *a)`
- `wbstring otype2Str (const char *fmt="%s") const`
- `int ConjOpScalar ()`
- `QSpace & SortDegQ (const char *F=0, int L=0)`
- `QSpace & Conj ()`
- `QSpace & DConj ()`
- `QSpace & RConj ()`
- `QSpace & HConj ()`
- `QSpace & hconj (QSpace &B) const`
- `QSpace & transp (QSpace &B) const`
- `const QSpace & opA (char tflag, QSpace &X) const`
- `template<class TD2 >
QSpace< TQ, TD > & initIdentityCGS (const char *F, int L, const QSpace< TQ, TD2 > &H)`

Public Attributes

- `wbMatrix< TQ > QIDX`
- `wbvector< wbarray< TD > * > DATA`
- `wbMatrix< CRef< TQ > > CGR`
- `QVec qtype`
- `unsigned QDIM`
- `QS_TYPES otype`
- `iTags itags`
- `Wb::MTYPE mt`
- `char isref`
- `double ctime`

The documentation for this class was generated from the following files:

- `QSpace.hh`
- `cpat.hh`
- `QSpace.cc`

4.71 QType Class Reference

Public Member Functions

- **QType** (QTYPE_SET t=QTYPE_UNKNOWN, unsigned s=0)
- **QType** (const char *s)
- **QType** (const [QType](#) &q)
- **QType** (const char *F, int L, const mxArray *a)
- **QType** & **init** ()
- **QType** & **initx** (QTYPE_SET t, unsigned m=0)
- **QType** & **init** (QTYPE_SET t, unsigned m=0)
- int **init_s** (const char *s)
- **QType** & **init** (const char *s)
- **QType** & **init** (const char *F, int L, const char *s)
- **QType** & **init** (const char *F, int L, const mxArray *a)
- **QType** & **init** (const [QType](#) &q)
- bool **isUnknown** () const
- bool **isKnown** () const
- bool **validType** (const char *F=0, int L=0) const
- bool **isAbelian** (const char *F=NULL, int L=0) const
- bool **isNonAbelian** (const char *F=NULL, int L=0, char lflag=0) const
- bool **isSU2** () const
- bool **isSUN** () const
- bool **isSU** (unsigned N) const
- bool **isSpN** () const
- bool **isSON** () const
- bool **isSEN** () const
- bool **isU1** () const
- bool **isParity** () const
- bool **isAdditive** () const
- int **permitsOM** () const
- int **permitsOM** (unsigned r) const
- unsigned **qlen** () const
- unsigned **qrnk** () const
- bool **isLargeD** (unsigned d) const
- bool **isLargeD** (const [wbvector](#)< unsigned > &dd) const
- unsigned **getDimDef** () const
- template<class TQ >
size_t **wdim** (const TQ *qs) const
- template<class TQ >
size_t **wdim** (const [qset](#)< TQ > &qs) const
- unsigned **maxDimLocal** () const
- unsigned **maxDimLocal0** () const
- void **printDimInfo** (const char *F=NULL, int L=0) const
- template<class T >
unsigned **qdim** (const T *q) const
- template<class T >
[wbvector](#)< unsigned > & **QDim** (const T *q, unsigned r, unsigned stride, [wbvector](#)< unsigned > &S) const
- bool **isSelfDual** () const
- template<class TQ >
char **setDual** (TQ *q) const
- template<class TQ >
unsigned **getDual** (const TQ *q0, TQ *q) const

- `template<class TQ >`
`bool isDual (const TQ *q, const TQ *x) const`
- `QType & operator= (const char *s)`
- `QType & operator= (const QType &q)`
- `QType & operator= (QTYPE_SET t)`
- `bool operator< (const QType &q) const`
- `char cmp (const QType &q) const`
- `bool operator> (const QType &q) const`
- `bool operator<= (const QType &q) const`
- `bool operator== (const QType &q) const`
- `bool operator!= (const QType &q) const`
- `wbstring toStr (const char tflag=0) const`
- `mxArray * mxCreateStruct (unsigned m, unsigned n) const`
- `void add2MxStruct (mxArray *S, unsigned i) const`
- `mxArray * toMx (const char tflag=0) const`
- `template<class TQ >`
`unsigned qdim (const TQ *q) const`
- `template<class TQ >`
`wbvector< unsigned > & QDim (const TQ *q, unsigned r, unsigned stride, wbvector< unsigned > &S) const`

Public Attributes

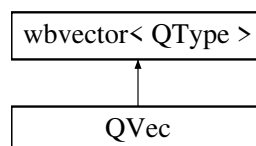
- `QTYPE_SET type`
- `unsigned sub`

The documentation for this class was generated from the following files:

- `clebsch.hh`
- `clebsch.cc`
- `clebsch_io.cc`

4.72 QVec Class Reference

Inheritance diagram for QVec:



Public Member Functions

- **QVec** (unsigned l, const [QType](#) *d0, char ref=0)
- **QVec** (const char *F, int L, const mxArray *a, unsigned d=0)
- **QVec & init** (const char *F, int L, const mxArray *a, unsigned d=0)
- **QVec & init** (unsigned l=0)
- **QVec & init** (const char *F, int L, const char *s, unsigned d=0)
- unsigned **Qlen** (unsigned n=-1) const
- unsigned **Qrank** () const
- unsigned **Qlen** ([wbvector](#)< unsigned > &dd) const
- unsigned **Qlen** ([wbvector](#)< unsigned > &dd, [wbvector](#)< unsigned > &dz) const
- unsigned **Qpos** ([wbvector](#)< unsigned > &dc) const
- unsigned **Qlenz** () const
- [wbvector](#)< gTQ > & **getScalar** ([wbvector](#)< gTQ > &q, char zflag=0)
- unsigned **maxRank** () const
- template<class TQ >
unsigned **QDim** (const TQ *q) const
- template<class TQ >
unsigned **QDim** (const TQ *qq, unsigned k) const
- template<class TQ >
[wbvector](#)< unsigned > & **QDim** (const TQ *qq, [wbvector](#)< unsigned > &S) const
- template<class TQ >
[wbvector](#)< unsigned > & **QDim** (const TQ *qq, unsigned k, unsigned r, [wbvector](#)< unsigned > &S) const
- template<class TQ >
[wbMatrix](#)< TQ > & **getQsub** (const [wbMatrix](#)< TQ > &Q, const [wbindex](#) &l, [wbMatrix](#)< TQ > &Ql, [wbMatrix](#)< TQ > *Qx=NULL) const
- bool **allU1** () const
- void **ReduceU1** ()
- char **allAbelian** () const
- bool **anyAbelian** () const
- bool **anyParity** () const
- bool **allNonAbelian** () const
- char **isNonAbelian** () const
- bool **allAdditive** () const
- int **permitsOM** () const
- int **permitsOM** (unsigned r) const
- bool **sameAs** (const [QVec](#) &b, char lflag=1) const
- bool **sameType** (const [QVec](#) &b) const
- [wbstring](#) **toStr** (const char vflag=0) const
- int **print_qset** (const char *F, int L, const gTQ *qs, char *s, unsigned n) const

Public Member Functions inherited from [wbvector](#)< [QType](#) >

- **wbvector** (size_t l, [QType](#) x)
- **wbvector** (size_t l, const [QType](#) *d0=NULL, char ref=0)
- **wbvector** (const [wbvector](#) &v)
- **wbvector** (const [wbvector](#)< T2 > &v)
- **wbvector** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- **wbvector** (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- **wbvector** (const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) & **init** (size_t l=0)
- [wbvector](#) & **init** (size_t l, const [QType](#) *d0)
- [wbvector](#) & **init** (size_t l, const [QType](#) *d0, char ref)
- [wbvector](#) & **init** (const [wbvector](#) &v)

- [wbvector](#) & **init** (size_t n, const [wbvector](#) &v)
- [wbvector](#)< QType > & **init** (const [wbsparray](#)< QType > &S)
- [wbvector](#) & **init** (size_t l1, const [QType](#) *d1, size_t l2, const [QType](#) *d2)
- int **init** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type QS_UNUSED_↵
VAR=1, char ref=0)
- int **init** (const mxArray *a, const char *istr=NULL, char check_type=1)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- [wbvector](#) & **initp** (size_t l, const [QType](#) *d0, size_t n, char init=1)
- [wbvector](#) & **init2val** (size_t l, const [QType](#) &x)
- [wbvector](#) & **initStride** (size_t l, const [QType](#) *d0, size_t stride)
- [wbvector](#) & **init_stride** (const [QType](#) *d0, size_t n, size_t m, size_t D0)
- [wbvector](#) & **init_data** (const [QType](#) *d0)
- [wbvector](#) & **init_data** (const T0 *d0)
- [wbvector](#) & **init_data** (const char *F, int L, const T0 *d0)
- [wbvector](#)< QType > & **initT** (const [wbsparray](#)< T2 > &S)
- [wbvector](#)< QType > & **initT** (size_t l, const T2 *v)
- [wbvector](#)< QType > & **initT** (const [wbvector](#)< T2 > &v)
- [wbvector](#)< QType > & **initT** (const char *F, int L, const [wbvector](#)< T2 > &v)
- [wbvector](#)< QType > & **initT** (const [wbsparray](#)< TD > &S)
- [wbvector](#)< QType > & **initT_** (const [wbvector](#)< T2 > &v)
- [wbvector](#) & **init2ref** (const [wbvector](#) &v)
- [wbvector](#) & **init2ref** (size_t l, const [QType](#) *d0)
- [wbvector](#) & **unRef** ()
- int **init_Struct** (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)
- int **init_base** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)
- int **init_mpf** (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)
- void **initDef** (size_t n)
- void **initDef** (const [wbvector](#) &v)
- [wbvector](#) & **operator=** (const [wbvector](#) &v)
- [wbvector](#)< QType > & **operator=** (const [wbvector](#)< TB > &v)
- [wbvector](#) & **set** (const [QType](#) &x)
- [wbvector](#) & **set** (const [wbvector](#)< size_t > &l, const [QType](#) &x)
- void **set** (size_t offset, const [QType](#) *x, size_t n)
- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- [wbvector](#) & **setMin** (const [QType](#) &x)
- [wbvector](#) & **setMax** (const [QType](#) &x)
- [wbvector](#) & **add** (const [wbvector](#)< size_t > &l, const [QType](#) &x)
- [wbvector](#) & **add** (const [QType](#) *d)
- [wbvector](#) & **Set** (const [wbvector](#)< size_t > &l, const [wbvector](#) &v)
- void **Sort** ([wbperm](#) &p, char dir=+1)
- void **Sort** (char dir=+1)
- void **sort** ([wbvector](#) &v2, [wbperm](#) &p) const
- void **sort** ([wbvector](#) &v2) const
- [wbperm](#) & **pSort** ([wbperm](#) &p) const
- bool **isSorted** () const
- bool **isSortedU** () const
- bool **isUnique** () const
- int **contains** (const [QType](#) &x) const
- bool **isNormal** () const

- `bool isNormal () const`
- `bool isNormal () const`
- `bool isNormal () const`
- `bool isNormal () const`
- `bool isNormal () const`
- `bool isNormal () const`
- `bool isNormal () const`
- `bool isFinite () const`
- `size_t find1 (const QType &x) const`
- `size_t find1 () const`
- `size_t count (const QType &x) const`
- `size_t icount (const QType &x) const`
- `size_t nnz () const`
- `size_t nnz (QType eps) const`
- `size_t numZeros (QType eps=QType(1E-14)) const`
- `size_t numEQ (const QType &) const`
- `size_t numGT (const QType &) const`
- `size_t numLT (const QType &) const`
- `wbindex & find (const QType &x, wbindex &l, char iflag=0) const`
- `wbindex & find (const QType &x, wbindex &l, wbindex &lx) const`
- `wbindex & find (wbindex &l) const`
- `wbindex & find (wbindex &l, wbindex &lx) const`
- `wbvector< size_t > find () const`
- `wbindex & findGT (const QType &x, wbindex &l, char iflag=0) const`
- `wbindex & findGT (const QType &x, wbindex &l, wbindex &lx) const`
- `wbindex & findzeros (wbindex &l) const`
- `wbindex & findRange (const QType &x1, const QType &x2, wbindex &l, const char *w=NULL) const`
- `int findClosestSorted (QType r) const`
- `int findClosest (QType r) const`
- `size_t findValues (const wbvector &v, wbindex &l) const`
- `wbvector & setp (const QType *x)`
- `void reset ()`
- `void cpy (const QType *v)`
- `void cpy (const QType *x, size_t m)`
- `wbvector & save2 (wbvector &v)`
- `void swap (wbvector &v)`
- `wbvector & swap (size_t i, size_t j)`
- `wbvector & Collect (QType &A1, QType &A2)`
- `wbvector & Collect (QType &A1, QType &A2, QType &A3)`
- `wbvector & Skip (size_t)`
- `wbvector & Skip (const WBINDEX &, char rflag=0)`
- `wbvector & Move2_FE (const WBINDEX &l, char FE_flag)`
- `wbvector & Move2front (const WBINDEX &l)`
- `wbvector & Move2front (size_t k)`
- `wbvector & Move2end (const WBINDEX &l)`
- `wbvector & Move2end (size_t k)`
- `wbvector & mvLastTo (size_t k)`
- `wbvector & mvFirstTo (size_t k)`
- `bool checkSameLength (const wbvector &a, const char *F=NULL, int L=0, const char *istr="") const`
- `bool checkSameLength (const wbvector &a, const wbvector &b, const char *F=NULL, int L=0, const char *istr="") const`
- `void operator+= (const T_ &a)`
- `void operator+= (const wbvector &b)`
- `void operator-= (const T_ &a)`
- `void operator-= (const wbvector &b)`

- [wbvector](#) & **operator*=** (const T_ &a)
- [wbvector](#) & **operator/=** (const T_ &a)
- [wbvector](#) **operator*** (const T_ &a) const
- [wbvector](#) **operator+** (const T_ &a) const
- [wbvector](#) **operator+** (const [wbvector](#) &b) const
- [wbvector](#) **operator-** (const T_ &a) const
- [wbvector](#) **operator-** (const [wbvector](#) &b) const
- [wbvector](#) & **Plus** (const [QType](#) *b)
- [wbvector](#) & **Plus** (const [QType](#) *b, [QType](#) bfac)
- [wbvector](#) & **Plus** (const [wbvector](#) &b, [QType](#) bfac=1)
- void **plus** (const [wbvector](#) &b, [wbvector](#) &c, [QType](#) bfac=1, [QType](#) cfac=0) const
- [wbvector](#) & **times** ([QType](#) fac, [wbvector](#) &x) const
- [wbvector](#) & **RevertSigns** ()
- int **isum** () const
- [QType](#) **sum** () const
- [QType](#) **sum** (size_t i1, const char *estr) const
- [QType](#) **sum** (size_t i1, size_t i2) const
- [QType](#) **cumsum_** ([wbvector](#) &S, char xflag=0) const
- [QType](#) **Cumsum** (char zflag=0) const
- [QType](#) **cumsum0prod** ([wbvector](#)< T2 > &b, [wbvector](#)< [QType](#) > &cs, char xflag) const
- [wbvector](#) & **Eldiv** (const [wbvector](#) &v) const
- [wbvector](#) & **tensorProd** (const [wbvector](#) &v2, [wbvector](#) &vv) const
- [QType](#) **scalarProd** (const [wbvector](#) &v) const
- [QType](#) **scalarProd** (const [QType](#) *v) const
- [QType](#) **norm** () const
- [QType](#) **norm2** () const
- double **normDiff** (const [wbvector](#) &v, [QType](#) fac=1) const
- [QType](#) **normDiff2** (const [wbvector](#) &v, [QType](#) fac=1) const
- double **normDiff2_** (const [wbvector](#) &v, double fac=1) const
- [QType](#) **GSProject** (const [QType](#) *v, char isnorm=0, char tflag=0) const
- [wbvector](#) & **Normalize** (const char *F=NULL, int L=0)
- [wbvector](#) & **subtract** (const [QType](#) *d)
- void **getReal** ([wbvector](#)< double > &R) const
- void **getReal** ([wbvector](#)< double > &R) const
- void **getImag** ([wbvector](#)< double > &I) const
- void **getImag** ([wbvector](#)< double > &I) const
- [wbvector](#) & **Conj** ()
- [wbvector](#)< [wbcomplex](#) > & **Conj** ()
- [wbvector](#) & **applyFlag** (char &flag, const [QType](#) &fac, [wbvector](#) &a)
- bool **allEqual** () const
- bool **allEqual** (const [QType](#) &a) const
- bool **allZero** () const
- bool **allOnes** () const
- bool **allEqual2** (const [QType](#) &a, [QType](#) eps=1E-15) const
- bool **allUnequal** (const [QType](#) &a) const
- bool **any** () const
- bool **anyUnequal** (const [QType](#) &a) const
- bool **anyUnequal** (const [wbvector](#)< size_t > &l, const [QType](#) &x, size_t *k=NULL) const
- bool **anyEqual** (const [QType](#) &a, size_t *i1=NULL) const
- bool **anyEqual** (const [wbvector](#)< size_t > &l, const [QType](#) &x, size_t *k=NULL) const
- bool **anyGT** (const [QType](#) &a) const
- bool **anyGE** (const [QType](#) &a) const
- bool **anyLT** (const [QType](#) &a) const
- bool **anyLE** (const [QType](#) &a) const
- bool **allGT** (const [QType](#) &a) const

- bool **allGE** (const [QType](#) &a) const
- bool **allLE** (const [QType](#) &a) const
- bool **allLT** (const [QType](#) &a) const
- bool **allIn** (const [QType](#) &a1, const [QType](#) &a2) const
- bool **anySmallVals** (const [QType](#) eps=1E-15) const
- bool **isEqual** (const [QType](#) *v) const
- bool **isEqual** (const [wbvector](#) &v) const
- bool **isEmpty** () const
- **operator bool** () const
- bool **operator!** () const
- bool **operator==** (const [wbvector](#) &v) const
- bool **operator!=** (const [wbvector](#) &v) const
- bool **isEqualR** (const [wbvector](#) &v) const
- bool **operator^** (const [wbvector](#) &v) const
- bool **deepEqualP** (const [wbvector](#) &B) const
- char **cmp** (const [wbvector](#) &v) const
- bool **operator<** (const [wbvector](#) &v) const
- int **operator<** (const [QType](#) &r) const
- bool **operator<=** (const [wbvector](#) &v) const
- bool **operator>** (const [wbvector](#) &v) const
- bool **operator>=** (const [wbvector](#) &v) const
- [QType](#) & **el** (size_t i) const
- [QType](#) **elx** (size_t i, const [QType](#) &x) const
- const [QType](#) & **operator[]** (size_t i) const
- [QType](#) & **operator[]** (size_t i)
- [wbvector](#) **operator[]** (const [wbvector](#) < size_t > &l) const
- [QType](#) & **last** () const
- [QType](#) & **last** (size_t &i) const
- [QType](#) & **last** (T1 &i_) const
- [QType](#) & **p2last** () const
- [QType](#) * **ref** (size_t i) const
- [wbvector](#) & **setRand** (double fac=1., double shift=0.)
- [wbvector](#) < [wbcomplex](#) > & **setRand** (double fac, double shift)
- void **setIdx** (double shift=0., double fac=1.)
- int **set2Group** (const [wbperm](#) &P, const WBINDEX &D, const [QType](#) *S0=NULL, char iflag=0)
- [wbvector](#) & **Resize_** (size_t n)
- [wbvector](#) & **Shorten2** (size_t n)
- [wbvector](#) & **Resize** (size_t n, const [QType](#) *d=NULL)
- [wbvector](#) & **resize** (size_t n, [wbvector](#) &X) const
- [wbvector](#) & **cat** (const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#) < [wbvector](#) > &vv)
- [wbvector](#) & **Cat** (const [wbvector](#) < [wbvector](#) * > &vv)
- [wbvector](#) & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)
- [wbvector](#) & **Cat** (const [wbvector](#) < T1 > *v1, const [wbvector](#) < T2 > *v2, const [wbvector](#) < T3 > *v3=0, const [wbvector](#) < T3 > *v4=0)
- [wbvector](#) < [QType](#) > & **Cat** (const [wbvector](#) < T1 > *v1, const [wbvector](#) < T2 > *v2, const [wbvector](#) < T3 > *v3, const [wbvector](#) < T3 > *v4)
- [wbvector](#) < [QType](#) > & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)
- [wbvector](#) & **Append** (const [wbvector](#) &v2)
- [wbvector](#) & **Append** (const [QType](#) &x)
- [wbvector](#) & **Append** (size_t d2, const [QType](#) *x)
- [wbvector](#) **append** (const [wbvector](#) &v) const

- **wbvector** & **append** (const **wbvector** &v2, **wbvector** &vout) const
- void **append** (const **QType** *x, size_t d2, **wbvector** &vout) const
- int **recLess** (const **QType** *v) const
- int **recLessE** (const **QType** *v) const
- int **recCompare** (const **QType** *v) const
- **QType** **max** () const
- **QType** **max** (size_t *k) const
- **QType** **max** (unsigned &k) const
- **QType** **max_** (**QType** x) const
- **QType** **min** () const
- **QType** **min** (size_t *k) const
- **QType** **min** (unsigned &k) const
- **QType** **min_** (**QType** x) const
- **QType** **aMax** (size_t *k=NULL) const
- **QType** **aMax** (unsigned &k) const
- **QType** **aMin** (char zflag=0, size_t *k=NULL) const
- **QType** **maxneg** (size_t *k=NULL) const
- **QType** **minpos** (size_t *k=NULL) const
- **QType** **avg** () const
- **QType** **std** () const
- void **range** (**QType** &xmin, **QType** &xmax) const
- **QType** **range** () const
- **wbvector** & **Permute** (const **wbperm** &P, char iflag=0)
- **wbvector** & **permute** (**wbvector** &, const **wbperm** &P, char iflag=0) const
- **wbvector** **permute** (const **wbperm** &P, char iflag=0) const
- **wbvector** & **blockPermute** (const **wbperm** &P, **wbvector** &, char iflag=0) const
- **wbvector** & **BlockPermute** (const **wbperm** &P, char iflag=0)
- int **blockCompare** (const **wbperm** &pa, const **wbvector** &b, const **wbperm** &pb) const
- void **get** (const WBINDEX &I0, **wbvector** &v) const
- void **get** (const **wbvector**< size_t > &I, **QType** *v) const
- void **get** (size_t i1, size_t i2, **wbvector** &v) const
- void **selectSU** (const WBINDEX &I)
- void **select** (const WBINDEX &I, **QType** *r) const
- **wbvector** & **select** (const WBINDEX &I, **wbvector** &a) const
- **wbvector** **select** (const WBINDEX &I) const
- **wbvector** & **Select** (const WBINDEX &I)
- **wbvector** & **BlockSelect** (const WBINDEX &I, const WBINDEX &D)
- **wbvector** & **getI** (size_t k, **wbvector** &v) const
- **wbvector** & **getI** (const **wbindex** &k, **wbvector** &v, char uflag=1) const
- **wbvector** & **getI** (const WBINDEX &k, **wbvector** &v, char uflag=1) const
- **wbvector** & **initI** (const **wbvector** &a, const **wbindex** &ia)
- **wbvector** & **initI** (const **wbvector** &a, const **wbindex** &ia, const **wbvector** &b, const **wbindex** &ib, char uflag=1)
- **wbvector** & **Flip** ()
- void **flip** (**wbvector** &a) const
- **wbvector** **flip** () const
- **QType** **prod** () const
- **QType** **prod** (**QType** x) const
- **QType** **prod** (size_t i, size_t i2) const
- **QType** **prod** (size_t i, size_t i2, **QType** x) const
- **QType** **prod** (const T1 *I, size_t n) const
- T2 **prod** (const T1 *I, size_t n, T2 &s2) const
- void **prod2** (**QType** &p1, **QType** &p2, const **QType** &p0=0) const
- void **info** (const char *istr) const
- void **print** (const char *istr, char mflag=0) const
- void **printdata** (const char *istr=0) const

- void **put** (const char *F, int L, const char *vname="ans", char tflag=0, const char *ws="base") const
- void **put** (const char *vname="ans", char tflag=0, const char *ws="base") const
- void **dispWithMsg** (const char *file, int line, const char *msg, const char *vname="") const
- void **mat2mx** (mxArray *&a, char dim=2) const
- void **mat2mxs** (mxArray *a, char field_nr, char dim=2) const
- mxArray * **toMx_Struct** (const char tflag=0) const
- mxArray * **toMxP_S** (const char tflag=0) const
- mxArray * **toMx** (const char tflag=0) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMxP** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base_d** (const char tflag=0) const
- mxArray * **toMx_offset** (const [QType](#) &offset, const char tflag=0) const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tflag=0) const
- void **add2MxStruct** (mxArray *S, const char *vname, char tflag=0) const
- mxArray * **mxCreateCell** (unsigned m, unsigned n) const
- void **add2MxCell** (mxArray *S, unsigned i, char tflag=0) const
- void **putg** (const char *vname="ans", char tflag=0) const
- void **putx** (const char *vname="ans", char tflag=0) const
- [wbstring](#) **toStr** (int n, const char *sep) const
- [wbstring](#) **toStr** () const
- [wbstring](#) **toStr** (int n, const char *sep) const
- [wbstring](#) **sizeStr** () const
- [wbstring](#) **toStrf** (const char *fmt0="", const char *sep=" ", unsigned stride=0, const char *sep2="") const
- [wbstring](#) **toStrf** (const char *fmt_, const char *sep, unsigned stride, const char *sep2) const

Additional Inherited Members

Public Attributes inherited from [wbvector< QType >](#)

- [QType](#) * **data**
- [size_t](#) **len**
- char **isref**

Protected Member Functions inherited from [wbvector< QType >](#)

- [wbvector](#) & **RENEW** ([size_t](#) n, const [QType](#) *d=NULL, [size_t](#) l=-1, char init=1)
- [wbvector](#) & **RENEW_VAL** ([size_t](#) n, const [QType](#) &x)

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc

4.73 Wb::rand Class Reference

Public Member Functions

- double **get_dfac** ()
- template<class T >
T & **rand_** (T &x)
- template<class T >
T & **rand_** (T &x, double fac)
- template<class T >
T & **rands** (T &x)
- template<class T >
T & **randb** (T &x)
- template<class T >
void **scale_rand** (T *x, size_t n, double fac)
- template<> double & **rand_** (double &x)
- template<> int & **rand_** (int &x)
- template<> unsigned & **rand_** (unsigned &x)
- template<> long & **rand_** (long &x)
- template<> size_t & **rand_** (size_t &x)
- template<> int & **rand_** (int &x, double fac)
- template<> unsigned & **rand_** (unsigned &x, double fac)
- template<> long & **rand_** (long &x, double fac)
- template<> size_t & **rand_** (size_t &x, double fac)
- template<> double & **randb** (double &x)
- template<> int & **randb** (int &x)
- template<> double & **rands** (double &x)
- template<> int & **rands** (int &x)
- template<> void **scale_rand** (int *x, size_t n, double fac)

The documentation for this class was generated from the following file:

- util.hh

4.74 RCStore Class Reference

Public Member Functions

- int **setupDirs** (const char *F=NULL, int L=0)
- template<class TQ, class TD >
int **save_CData** (const char *F, int L, const CData< TQ, TD > &A)
- template<class TQ, class TD >
int **load_CData** (const char *F, int L, const QSet< TQ > &Q, CData< TQ, TD > &A, char bflag=0)
- template<class TQ >
int **save_Std3** (const char *F, int L, const QType &t, const qset< TQ > &J12, const wvector< double > *c2eps=NULL) const
- template<class TQ >
int **load_Std3** (const char *F, int L, const QType &t, const qset< TQ > &J1, const qset< TQ > &J2, unsigned loadRC=0) const
- template<class TQ, class TD >
int **save_RSet** (const char *F, int L, const genRG_base< TQ, TD > &R, char xflag=0)

- `template<class TQ >`
`int load_RSet (const char *F, int L, const QType &t, const qset< TQ > &J, char vflag=0)`
- `int save_XMap (const char *F, int L, const cgc_contract_id< MTI > &idc) const`
- `int load_XMap (const char *F, int L, const cgc_contract_id< MTI > &idc) const`
- `template<class TQ >`
`int get_file_name (const char *F, int L, wstring &file, const QSet< TQ > &Q, const char *ext=NULL, char mflag=0) const`
- `template<class TQ, class TD >`
`int get_file_name (const char *F, int L, wstring &file, const genRG_base< TQ, TD > &R, const char *ext=NULL, char mflag=0) const`
- `int get_file_name (const char *F, int L, wstring &file, const cgc_contract_id< MTI > &idc, const char *ext=NULL, char mflag=0, QType *t=NULL) const`
- `size_t rlog (const QType &t, const char *F, int L, const char *fmt,...)`
- `size_t rlog (const QType &t, const char *F, int L, char vflag, const char *fmt,...)`
- `size_t vrlog (const QType &t, const char *F, int L, const char *fmt, va_list args)`
- `void Info (const char *F=0, int L=0)`
- `int is_new (const QType &t)`

Public Attributes

- [Wb::Path](#) `root`

The documentation for this class was generated from the following files:

- `clebsch.hh`
- `clebsch_aux.cc`
- `clebsch_io.cc`

4.75 RIXS_para Class Reference

Public Member Functions

- `void init ()`
- `void setE0 (int it, double e0, double e2)`
- `void check (const char *F=0, int L=0, char init=0)`
- `Wb::String toStr () const`
- `mxArray * toMx () const`

Public Attributes

- [wbcomplex](#) `Einc`
- `double Eoffset`
- `double sigma`
- `double afac`
- `char wrxs`
- `double E0`
- `double E2`
- `unsigned dloc`
- `int iter`

The documentation for this class was generated from the following file:

- `nrgdata.hh`

4.76 RSpecData< TS > Class Template Reference

Public Member Functions

- [RSpecData](#) & **Append** (const double *op, const TS *cp, unsigned n)
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tflag=0) const

Public Attributes

- [wbvector](#)< double > **om**
- [wbvector](#)< TS > **cc**

The documentation for this class was generated from the following file:

- spectral.hh

4.77 RStore< TQ, TD > Class Template Reference

Public Member Functions

- **RStore** (const char *s=NULL)
- size_t **bufsize** (const [QType](#) &q)
- bool **isEmpty** (const [QType](#) &q)
- [genRG_struct](#)< TQ, TD > & **Buf** (const [QType](#) &q)
- [genRG_base](#)< TQ, TD > * **find_RSet** (const [QType](#) &q, const TQ *qptr)
- [genRG_struct](#)< TQ, TD > & **getRSet** (const [QType](#) &q, [qset](#)< TQ > *qvec=NULL)
- [genRG_base](#)< TQ, TD > & **getR** (const char *F, int L, const [QType](#) &qtype, const TQ *q) const
- [genRG_base](#)< TQ, TD > & **getR** (const char *F, int L, const CDATA_TQ &S, unsigned k) const
- [wbMatrix](#)< TQ > **getZ** (const char *F, int L, const [QType](#) &qtype, const TQ *q) const
- unsigned **qdim** (const char *F, int L, const [QType](#) &qtype, const TQ *q) const
- mxArray * **toMx** (const [QType](#) &q=QTYPE_UNKNOWN, char bflag=1) const
- unsigned **LoadStore** (const char *F, int L, const char *file)
- unsigned **add** (const [QType](#) &q, const mxArray *S, unsigned k)
- void **Info** (const char *F=0, int L=0) const

Public Attributes

- map< [QType](#), [genRG_struct](#)< TQ, TD > > **buf**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch_aux.cc
- clebsch_io.cc
- clebsch_old.cc

4.78 Wb::SBUF Class Reference

Public Member Functions

- **SBUF** (unsigned l0=wblog::SLEN, char *s=NULL)
- **SBUF** & **init** (unsigned l0=0, char *s=NULL)
- int **cat** (const char *s)
- int **catf** (const char *fmt,...)
- void **error_bounds** (const char *F, int L, const char *fct, const char *fmt)
- void **increase_size** (const char *F, int L, unsigned n)
- void **flush** (FILE *fid=stdout, char fflag=0)
- void **print** (const char *F=0, int L=0, const char *istr=0)
- int **skipEscCols** ()

Public Attributes

- unsigned **l**
- unsigned **slen**
- unsigned **ref**
- char * **sbuf**

The documentation for this class was generated from the following files:

- wblog.h
- wblog.c

4.79 Wb::SigHandler Class Reference

Public Member Functions

- **SigHandler** (const char *F, int L)
- int **check911** (char tflag=0)

Static Public Member Functions

- static int **check911_** (const char *F=NULL, int L=0)

Static Public Attributes

- static int **NC0** = 3
- static int **icount** = SigHandler::NC0
- static int **acount** = SigHandler::NC0

The documentation for this class was generated from the following file:

- gathered.hh

4.80 `sparseCollector2D< TD >` Class Template Reference

Public Member Functions

- `sparseCollector2D` (SPIDX_T m=0, SPIDX_T n=0)
- void `clear` ()
- void `init` (SPIDX_T m=0, SPIDX_T n=0)
- `sparseCollector2D` & `add` (const char *F, int L, SPIDX_T i, SPIDX_T j, const TD &a)
- void `getData` (`wbMatrix`< SPIDX_T > &IJ, `wbvector`< TD > &X)
- `mxAarray` * `toMx` () const

The documentation for this class was generated from the following file:

- `wbsparray.hh`

4.81 `sparseIndex2D< T >` Class Template Reference

Public Member Functions

- `sparseIndex2D` & `Setup` (char use_rmaj=0)
- T `cidx` (T j) const
- T `ridx` (T ic) const
- `wbstring` `sizeStr` () const

Public Attributes

- char `rmaj`
- `wbMatrix`< T > `IJ`
- `wbvector`< T > `S`
- `wbperm` `P`
- `wbvector`< T > `cidx`

The documentation for this class was generated from the following files:

- `wbindex.hh`
- `wbindex.cc`

4.82 Spectral< TS > Class Template Reference

Public Member Functions

- void **init** ()
- void **init** (const char *istr, unsigned Nops, double delta=0., double T_=0, unsigned Niter=0)
- void **initBuf** ()
- void **calcSpectralMoments** (const char *F, int L, unsigned n)
- void **Add** (const [wbararray](#)< double > &Om, const [wbararray](#)< TS > &CC, unsigned s, char dblock=0, char mflag=0)
- void **Add2buf** (const [wbararray](#)< double > &Om, const [wbararray](#)< TS > &CC, unsigned s)
- void **crossAddBuf** (const char minor=1, char vflag=1)
- void **savelter** (const char *F, int L, unsigned iter)
- void **getOmega** ([wbvector](#)< double > &om, unsigned &linfit) const
- void **getOmega** ([wbvector](#)< double > &om) const
- void **applyIROPfac** (const char *F, int L, const [wbvector](#)< double > &fac)
- void **getRawData** ([wbvector](#)< double > &om, [wbMatrix](#)< TS > &A, unsigned linfit=0, const char *isbuf="") const
- void **getPARTIAL** ([wbararray](#)< TS > &A1, [wbararray](#)< TS > &A2) const
- void **pairRawData** ()
- void **detailedBalance** (const double T, [wbvector](#)< double > &om, [wbMatrix](#)< TS > &aa) const
- void **dispSumRule** ()
- void **dispSumRule** (const [wbvector](#)< double > &om, const [wbMatrix](#)< TS > &A)
- void **getSmoothSpec** ([wbvector](#)< double > &om, [wbMatrix](#)< TS > &A, char vflag=1)
- void **getSmoothSpec** ([wbMatrix](#)< TS > &A, const [wbvector](#)< double > &om, char vflag=1)
- void **KKreal** ([wbMatrix](#)< [wbcomplex](#) > &GZ, [wbvector](#)< double > &OM, [wbvector](#)< double > &DOM) const

Public Attributes

- [wbMatrix](#)< TS > **Ap**
- [wbMatrix](#)< TS > **An**
- [wbMatrix](#)< TS > **Ap_buf**
- [wbMatrix](#)< TS > **An_buf**
- [wbvector](#)< TS > **Ar**
- [wbMatrix](#)< TS > **A0**
- [wbMatrix](#)< TS > **mspec**
- [wbararray](#)< TS > **AP**
- [wbararray](#)< TS > **AN**
- [wbstring](#) **istr**
- double **Temp**
- double **sigma**
- double **sigma2**
- double **alpha**
- double **Emin**
- double **Emax**
- double **eps**
- unsigned **nlog**

The documentation for this class was generated from the following file:

- spectral.hh

4.83 Wb::sptr< T > Class Template Reference

Public Member Functions

- void **init_def** ()
- T * **malloc_base** (size_t n, char init=1, MTYPE mt=MEM_DEF)
- T * **malloc_data** (size_t n, const T *d0=NULL, MTYPE mt=MEM_DEF)
- int **rm_dref** (char mflag=0)
- unsigned **add_dref** ()
- T * **save_dref** (const char *F=0, int L=0)
- T * **init2ref** (size_t n, const T *d0)
- int **isRef** () const
- **operator bool** () const
- bool **operator!** () const
- int **check_consistency** (char force=1) const
- [wbstring](#) **toStr** () const

Public Attributes

- MTYPE **mtype**
- size_t **len**
- unsigned **nref**
- T * **data**

The documentation for this class was generated from the following files:

- memlib.hh
- memlib.cc

4.84 STAT_DGEMM Class Reference

Public Member Functions

- void **account** (unsigned n0)

Public Attributes

- [wbvector](#)< unsigned > **nn**
- [wbvector](#)< unsigned long > **ss**

The documentation for this class was generated from the following file:

- wbbblas.hh

4.85 wblog::stdio_buf Class Reference

Public Member Functions

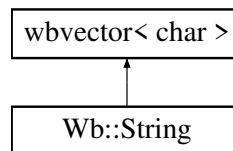
- **stdio_buf** ([Wb::SBUF](#) &S, FILE *f_, int t_)
- **stdio_buf** (const [stdio_buf](#) &b)
- void **print_stdout** (FILE *f=NULL) const

The documentation for this class was generated from the following file:

- wblog.h

4.86 Wb::String Class Reference

Inheritance diagram for Wb::String:



Public Member Functions

- **String** (unsigned n)
- **String** (const char *F, int L, unsigned n)
- int **pushf** (const char *fmt,...)

Public Member Functions inherited from [wbvector< char >](#)

- **wbvector** (size_t l, char x)
- **wbvector** (size_t l, const char *d0=NULL, char ref=0)
- **wbvector** (const [wbvector](#) &v)
- **wbvector** (const [wbvector](#)< T2 > &v)
- **wbvector** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- **wbvector** (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- **wbvector** (const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) & **init** (size_t l=0)
- [wbvector](#) & **init** (size_t l, const char *d0)
- [wbvector](#) & **init** (size_t l, const char *d0, char ref)
- [wbvector](#) & **init** (const [wbvector](#) &v)
- [wbvector](#) & **init** (size_t n, const [wbvector](#) &v)
- [wbvector](#)< char > & **init** (const [wbsparray](#)< char > &S)
- [wbvector](#) & **init** (size_t l1, const char *d1, size_t l2, const char *d2)
- int **init** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type QS_UNUSED_↵ VAR=1, char ref=0)
- int **init** (const mxArray *a, const char *istr=NULL, char check_type=1)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)

- `int init (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)`
- `int init (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)`
- `int init (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)`
- `int init (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)`
- `wbvector & initp (size_t l, const char *d0, size_t n, char init=1)`
- `wbvector & init2val (size_t l, const char &x)`
- `wbvector & initStride (size_t l, const char *d0, size_t stride)`
- `wbvector & init_stride (const char *d0, size_t n, size_t m, size_t D0)`
- `wbvector & init_data (const char *d0)`
- `wbvector & init_data (const T0 *d0)`
- `wbvector & init_data (const char *F, int L, const T0 *d0)`
- `wbvector< char > & initT (const wbspparray< T2 > &S)`
- `wbvector< char > & initT (size_t l, const T2 *v)`
- `wbvector< char > & initT (const wbvector< T2 > &v)`
- `wbvector< char > & initT (const char *F, int L, const wbvector< T2 > &v)`
- `wbvector< char > & initT (const wbspparray< TD > &S)`
- `wbvector< char > & initT_ (const wbvector< T2 > &v)`
- `wbvector & init2ref (const wbvector &v)`
- `wbvector & init2ref (size_t l, const char *d0)`
- `wbvector & unRef ()`
- `int init_Struct (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)`
- `int init_base (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)`
- `int init_mpfr (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)`
- `void initDef (size_t n)`
- `void initDef (const wbvector &v)`
- `wbvector & operator= (const wbvector &v)`
- `wbvector< char > & operator= (const wbvector< TB > &v)`
- `wbvector & set (const char &x)`
- `wbvector & set (const wbvector< size_t > &l, const char &x)`
- `void set (size_t offset, const char *x, size_t n)`
- `void set (const wbvector< double > &R, const wbvector< double > &l)`
- `void set (const wbvector< double > &R, const wbvector< double > &l)`
- `wbvector & setMin (const char &x)`
- `wbvector & setMax (const char &x)`
- `wbvector & add (const wbvector< size_t > &l, const char &x)`
- `wbvector & add (const char *d)`
- `wbvector & Set (const wbvector< size_t > &l, const wbvector &v)`
- `void Sort (wbperm &p, char dir=+1)`
- `void Sort (char dir=+1)`
- `void sort (wbvector &v2, wbperm &p) const`
- `void sort (wbvector &v2) const`
- `wbperm & pSort (wbperm &p) const`
- `bool isSorted () const`
- `bool isSortedU () const`
- `bool isUnique () const`
- `int contains (const char &x) const`
- `bool isNormal () const`
- `bool isNormal () const`
- `bool isNormal () const`
- `bool isNormal () const`
- `bool isNormal () const`
- `bool isNormal () const`
- `bool isNormal () const`
- `bool isNormal () const`
- `bool isNormal () const`
- `bool isFinite () const`

- `size_t find1 (const char &x) const`
- `size_t find1 () const`
- `size_t count (const char &x) const`
- `size_t icount (const char &x) const`
- `size_t nnz () const`
- `size_t nnz (char eps) const`
- `size_t numZeros (char eps=char(1E-14)) const`
- `size_t numEQ (const char &) const`
- `size_t numGT (const char &) const`
- `size_t numLT (const char &) const`
- `wbindex & find (const char &x, wbindex &l, char iflag=0) const`
- `wbindex & find (const char &x, wbindex &l, wbindex &lx) const`
- `wbindex & find (wbindex &l) const`
- `wbindex & find (wbindex &l, wbindex &lx) const`
- `wbvector< size_t > find () const`
- `wbindex & findGT (const char &x, wbindex &l, char iflag=0) const`
- `wbindex & findGT (const char &x, wbindex &l, wbindex &lx) const`
- `wbindex & findzeros (wbindex &l) const`
- `wbindex & findRange (const char &x1, const char &x2, wbindex &l, const char *w=NULL) const`
- `int findClosestSorted (char r) const`
- `int findClosest (char r) const`
- `size_t findValues (const wbvector &v, wbindex &l) const`
- `wbvector & setp (const char *x)`
- `void reset ()`
- `void cpy (const char *v)`
- `void cpy (const char *x, size_t m)`
- `wbvector & save2 (wbvector &v)`
- `void swap (wbvector &v)`
- `wbvector & swap (size_t i, size_t j)`
- `wbvector & Collect (char &A1, char &A2)`
- `wbvector & Collect (char &A1, char &A2, char &A3)`
- `wbvector & Skip (size_t)`
- `wbvector & Skip (const WBINDEX &, char rflag=0)`
- `wbvector & Move2_FE (const WBINDEX &l, char FE_flag)`
- `wbvector & Move2front (const WBINDEX &l)`
- `wbvector & Move2front (size_t k)`
- `wbvector & Move2end (const WBINDEX &l)`
- `wbvector & Move2end (size_t k)`
- `wbvector & mvLastTo (size_t k)`
- `wbvector & mvFirstTo (size_t k)`
- `bool checkSameLength (const wbvector &a, const char *F=NULL, int L=0, const char *istr="") const`
- `bool checkSameLength (const wbvector &a, const wbvector &b, const char *F=NULL, int L=0, const char *istr="") const`
- `void operator+= (const T_ &a)`
- `void operator+= (const wbvector &b)`
- `void operator-= (const T_ &a)`
- `void operator-= (const wbvector &b)`
- `wbvector & operator*= (const T_ &a)`
- `wbvector & operator/= (const T_ &a)`
- `wbvector operator* (const T_ &a) const`
- `wbvector operator+ (const T_ &a) const`
- `wbvector operator+ (const wbvector &b) const`
- `wbvector operator- (const T_ &a) const`
- `wbvector operator- (const wbvector &b) const`
- `wbvector & Plus (const char *b)`

- [wbvector](#) & **Plus** (const char *b, char bfac)
- [wbvector](#) & **Plus** (const [wbvector](#) &b, char bfac=1)
- void **plus** (const [wbvector](#) &b, [wbvector](#) &c, char bfac=1, char cfac=0) const
- [wbvector](#) & **times** (char fac, [wbvector](#) &x) const
- [wbvector](#) & **RevertSigns** ()
- int **isum** () const
- char **sum** () const
- char **sum** (size_t i1, const char *estr) const
- char **sum** (size_t i1, size_t i2) const
- char **cumsum_** ([wbvector](#) &S, char xflag=0) const
- char **Cumsum** (char zflag=0) const
- char **cumsum0prod** ([wbvector](#)< T2 > &b, [wbvector](#)< char > &cs, char xflag) const
- [wbvector](#) & **Eldiv** (const [wbvector](#) &v) const
- [wbvector](#) & **tensorProd** (const [wbvector](#) &v2, [wbvector](#) &vv) const
- char **scalarProd** (const [wbvector](#) &v) const
- char **scalarProd** (const char *v) const
- char **norm** () const
- char **norm2** () const
- double **normDiff** (const [wbvector](#) &v, char fac=1) const
- char **normDiff2** (const [wbvector](#) &v, char fac=1) const
- double **normDiff2_** (const [wbvector](#) &v, double fac=1) const
- char **GSProject** (const char *v, char isnorm=0, char tflag=0) const
- [wbvector](#) & **Normalize** (const char *F=NULL, int L=0)
- [wbvector](#) & **subtract** (const char *d)
- void **getReal** ([wbvector](#)< double > &R) const
- void **getReal** ([wbvector](#)< double > &R) const
- void **getImag** ([wbvector](#)< double > &I) const
- void **getImag** ([wbvector](#)< double > &I) const
- [wbvector](#) & **Conj** ()
- [wbvector](#)< [wbcomplex](#) > & **Conj** ()
- [wbvector](#) & **applyFlag** (char &flag, const char &fac, [wbvector](#) &a)
- bool **allEqual** () const
- bool **allEqual** (const char &a) const
- bool **allZero** () const
- bool **allOnes** () const
- bool **allEqual2** (const char &a, char eps=1E-15) const
- bool **allUnequal** (const char &a) const
- bool **any** () const
- bool **anyUnequal** (const char &a) const
- bool **anyUnequal** (const [wbvector](#)< size_t > &l, const char &x, size_t *k=NULL) const
- bool **anyEqual** (const char &a, size_t *i1=NULL) const
- bool **anyEqual** (const [wbvector](#)< size_t > &l, const char &x, size_t *k=NULL) const
- bool **anyGT** (const char &a) const
- bool **anyGE** (const char &a) const
- bool **anyLT** (const char &a) const
- bool **anyLE** (const char &a) const
- bool **allGT** (const char &a) const
- bool **allGE** (const char &a) const
- bool **allLE** (const char &a) const
- bool **allLT** (const char &a) const
- bool **allIn** (const char &a1, const char &a2) const
- bool **anySmallVals** (const char eps=1E-15) const
- bool **isEqual** (const char *v) const
- bool **isEqual** (const [wbvector](#) &v) const
- bool **isEmpty** () const

- **operator bool** () const
- bool **operator!** () const
- bool **operator==** (const [wbvector](#) &v) const
- bool **operator!=** (const [wbvector](#) &v) const
- bool **isEqualR** (const [wbvector](#) &v) const
- bool **operator^** (const [wbvector](#) &v) const
- bool **deepEqualP** (const [wbvector](#) &B) const
- char **cmp** (const [wbvector](#) &v) const
- bool **operator<** (const [wbvector](#) &v) const
- int **operator<** (const char &r) const
- bool **operator<=** (const [wbvector](#) &v) const
- bool **operator>** (const [wbvector](#) &v) const
- bool **operator>=** (const [wbvector](#) &v) const
- char & **el** (size_t i) const
- char **elx** (size_t i, const char &x) const
- const char & **operator[]** (size_t i) const
- char & **operator[]** (size_t i)
- [wbvector](#) **operator[]** (const [wbvector](#)< size_t > &l) const
- char & **last** () const
- char & **last** (size_t &i) const
- char & **last** (T1 &i_) const
- char & **p2last** () const
- char * **ref** (size_t i) const
- [wbvector](#) & **setRand** (double fac=1., double shift=0.)
- [wbvector](#)< [wbcomplex](#) > & **setRand** (double fac, double shift)
- void **setIdx** (double shift=0., double fac=1.)
- int **set2Group** (const [wbperm](#) &P, const WBINDEX &D, const char *S0=NULL, char iflag=0)
- [wbvector](#) & **Resize_** (size_t n)
- [wbvector](#) & **Shorten2** (size_t n)
- [wbvector](#) & **Resize** (size_t n, const char *d=NULL)
- [wbvector](#) & **resize** (size_t n, [wbvector](#) &X) const
- [wbvector](#) & **cat** (const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) > &vv)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) * > &vv)
- [wbvector](#) & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)
- [wbvector](#) & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3=0, const [wbvector](#)< T3 > *v4=0)
- [wbvector](#)< char > & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3, const [wbvector](#)< T3 > *v4)
- [wbvector](#)< char > & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)
- [wbvector](#) & **Append** (const [wbvector](#) &v2)
- [wbvector](#) & **Append** (const char &x)
- [wbvector](#) & **Append** (size_t d2, const char *x)
- [wbvector](#) **append** (const [wbvector](#) &v) const
- [wbvector](#) & **append** (const [wbvector](#) &v2, [wbvector](#) &vout) const
- void **append** (const char *x, size_t d2, [wbvector](#) &vout) const
- int **recLess** (const char *v) const
- int **recLessE** (const char *v) const
- int **recCompare** (const char *v) const
- char **max** () const
- char **max** (size_t *k) const
- char **max** (unsigned &k) const

- char **max_** (char x) const
- char **min** () const
- char **min** (size_t *k) const
- char **min** (unsigned &k) const
- char **min_** (char x) const
- char **aMax** (size_t *k=NULL) const
- char **aMax** (unsigned &k) const
- char **aMin** (char zflag=0, size_t *k=NULL) const
- char **maxneg** (size_t *k=NULL) const
- char **minpos** (size_t *k=NULL) const
- char **avg** () const
- char **std** () const
- void **range** (char &xmin, char &xmax) const
- char **range** () const
- **wbvector** & **Permute** (const **wbperm** &P, char iflag=0)
- **wbvector** & **permute** (**wbvector** &, const **wbperm** &P, char iflag=0) const
- **wbvector** **permute** (const **wbperm** &P, char iflag=0) const
- **wbvector** & **blockPermute** (const **wbperm** &P, **wbvector** &, char iflag=0) const
- **wbvector** & **BlockPermute** (const **wbperm** &P, char iflag=0)
- int **blockCompare** (const **wbperm** &pa, const **wbvector** &b, const **wbperm** &pb) const
- void **get** (const WBINDEX &I0, **wbvector** &v) const
- void **get** (const **wbvector**< size_t > &I, char *v) const
- void **get** (size_t i1, size_t i2, **wbvector** &v) const
- void **selectSU** (const WBINDEX &I)
- void **select** (const WBINDEX &I, char *r) const
- **wbvector** & **select** (const WBINDEX &I, **wbvector** &a) const
- **wbvector** **select** (const WBINDEX &I) const
- **wbvector** & **Select** (const WBINDEX &I)
- **wbvector** & **BlockSelect** (const WBINDEX &I, const WBINDEX &D)
- **wbvector** & **getI** (size_t k, **wbvector** &v) const
- **wbvector** & **getI** (const **wbindex** &k, **wbvector** &v, char uflag=1) const
- **wbvector** & **getI** (const WBINDEX &k, **wbvector** &v, char uflag=1) const
- **wbvector** & **initI** (const **wbvector** &a, const **wbindex** &ia)
- **wbvector** & **initI** (const **wbvector** &a, const **wbindex** &ia, const **wbvector** &b, const **wbindex** &ib, char uflag=1)
- **wbvector** & **Flip** ()
- void **flip** (**wbvector** &a) const
- **wbvector** **flip** () const
- char **prod** () const
- char **prod** (char x) const
- char **prod** (size_t i, size_t i2) const
- char **prod** (size_t i, size_t i2, char x) const
- char **prod** (const T1 *I, size_t n) const
- T2 **prod** (const T1 *I, size_t n, T2 &s2) const
- void **prod2** (char &p1, char &p2, const char &p0=0) const
- void **info** (const char *istr) const
- void **print** (const char *istr, char mflag=0) const
- void **printdata** (const char *istr) const
- void **put** (const char *F, int L, const char *vname="ans", char tflag=0, const char *ws="base") const
- void **put** (const char *vname="ans", char tflag=0, const char *ws="base") const
- void **dispWithMsg** (const char *file, int line, const char *msg, const char *vname="") const
- void **mat2mx** (mxArray *&a, char dim=2) const
- void **mat2mxs** (mxArray *a, char field_nr, char dim=2) const
- mxArray * **toMx_Struct** (const char tflag=0) const
- mxArray * **toMxP_S** (const char tflag=0) const
- mxArray * **toMx** (const char tflag=0) const

- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMxP (const char tflag=0) const`
- `mxArray * toMx_base (const char tflag=0) const`
- `mxArray * toMx_base (const char tflag) const`
- `mxArray * toMx_base (const char tflag) const`
- `mxArray * toMx_base (const char tflag) const`
- `mxArray * toMx_base_d (const char tflag=0) const`
- `mxArray * toMx_offset (const char &offset, const char tflag=0) const`
- `mxArray * mxCreateStruct (unsigned m, unsigned n) const`
- `void add2MxStruct (mxArray *S, unsigned i, char tflag=0) const`
- `void add2MxStruct (mxArray *S, const char *vname, char tflag=0) const`
- `mxArray * mxCreateCell (unsigned m, unsigned n) const`
- `void add2MxCell (mxArray *S, unsigned i, char tflag=0) const`
- `void putg (const char *vname="ans", char tflag=0) const`
- `void putx (const char *vname="ans", char tflag=0) const`
- `wbstring toStr (int n, const char *sep) const`
- `wbstring toStr () const`
- `wbstring toStr (int n, const char *sep) const`
- `wbstring sizeStr () const`
- `wbstring toStrf (const char *fmt0="", const char *sep=" ", unsigned stride=0, const char *sep2="") const`
- `wbstring toStrf (const char *fmt_, const char *sep, unsigned stride, const char *sep2) const`

Additional Inherited Members

Public Attributes inherited from `wbvector< char >`

- `char * data`
- `size_t len`
- `char isref`

Protected Member Functions inherited from `wbvector< char >`

- `wbvector & RENEW (size_t n, const char *d=NULL, size_t l=-1, char init=1)`
- `wbvector & RENEW_VAL (size_t n, const char &x)`

The documentation for this class was generated from the following files:

- `wbstring.hh`
- `wbstring.cc`

4.87 SVD_Data< TQ, TD > Class Template Reference

Public Member Functions

- void **blockSVD** (const [QSpace](#)< TQ, TD > &PSI, const unsigned K)
- unsigned **dmrgTruncate** (unsigned Nkmin, unsigned Nkeep, double stol, [itag_](#) tx, [wbMatrix](#)< double > &SV, [QSpace](#)< TQ, TD > &UQ, [QSpace](#)< TQ, TD > *SQ, [QSpace](#)< TQ, TD > &VC, mxArray *Sout=NULL, char vflag=0)
- mxArray * **toMx** () const
- void **put** (const char *vname, const char *ws="base") const
- void **put** (const char *F, int L, const char *vname, const char *ws="base") const

Public Attributes

- [wbvector](#)< [QBlock](#)< TQ, TD, double > > **QB**

The documentation for this class was generated from the following files:

- mpsortho.hh
- mpsortho.cc

4.88 DY::Symmetry Class Reference

Public Member Functions

- **Symmetry** (const [QType](#) &q_)
- **Symmetry** (const mxArray *a)
- [Symmetry](#) & **init** (const [QType](#) &q)
- template<class TQ >
size_t **getWeightsFT** (const char *F, int L, const [qset](#)< TQ > &qs, [Weights](#)< TQ > &R)
- template<class TQ >
int **getInnerMultFT** (const [qset](#)< TQ > &qs, const [weight_info](#)< TQ > &w, [Weights](#)< TQ > &R, char vflag=0)
- template<class TQ >
int **Get** (const [qset](#)< TQ > &qk, map< [qset](#)< TQ >, [weight_info](#)< TQ > > &W, char vflag=0)
- template<class TQ >
int **wExpand** (const char *F, int L, [Weights](#)< TQ > &R) const
- mxArray * **toMx** () const

Public Attributes

- [QType](#) **q**
- unsigned **n**
- [wbarray](#)< double > **R**
- [wbMatrix](#)< int > **A**
- [wbvector](#)< int > **n2**
- [wbarray](#)< int > **M**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc
- clebsch_io.cc

4.89 TDSpectral< TS > Class Template Reference

Public Member Functions

- void **init** (const char *F, int L, const char *istr0, const [wbvector](#)< double > &t0, unsigned Nnrg, unsigned Nop, double Emin=1E-8, double Emax=10., unsigned nl=512, char vflag=1)
- void **checkAvgOm** ()
- [wbarray](#)< TS > **getAvgOm** ()
- void **Add** (const [wbarray](#)< double > &Om, const [wbarray](#)< TS > &CC, unsigned iter, unsigned s)
- void **addPartialFourier** ()
- void **getOmega** ([wbvector](#)< double > &om, unsigned &linfit) const
- void **getOmega** ([wbvector](#)< double > &om) const
- void **applyIROPfac** (const char *F, int L, const [wbvector](#)< double > &fac, char rixs=0)
- void **getSpecData** ([wbvector](#)< double > &om, [wbarray](#)< TS > &A, unsigned linfit=0) const
- void **getSpecData** ([wbvector](#)< double > &om, [wbMatrix](#)< TS > &A, unsigned linfit=0)
- void **mergeNRGData** ()
- void **getSmoothSpec_t** (double sigma, double eps, char bswitch=LOG_GAUSS_BRD, char vflag=1)
- void **Fourier** ([wbvector](#)< double > &om, [wbMatrix](#)< [wbcomplex](#) > &ST, double alpha, double Lambda, char vflag=1)
- void **Fourier** ([wbvector](#)< double > &om, [wbMatrix](#)< TS > &A, const [wbvector](#)< double > &tt, [wbMatrix](#)< [wbcomplex](#) > &at, double alpha=0., char vflag=1)
- void **dispSumRule** ()
- mxArray * **toMx** () const
- int **init** (const mxArray *S, double &emin, double &emax, unsigned &nlog)

Public Attributes

- [wbarray](#)< TS > **AP**
- [wbarray](#)< TS > **AN**
- [wbMatrix](#)< TS > **Ap**
- [wbMatrix](#)< TS > **An**
- [wbMatrix](#)< [wbcomplex](#) > **At**
- [wbvector](#)< double > **tt**
- [wbstring](#) **istr**
- unsigned **raw**
- [wbvector](#)< [RSpecData](#)< TS > > **RAW**

The documentation for this class was generated from the following file:

- spectral.hh

4.90 tensor_ref_ Class Reference

Public Member Functions

- **tensor_ref_** (const [tensor_ref_](#) &b)
- [tensor_ref_](#) & **init** (const [tensor_ref_](#) &b)
- [tensor_ref_](#) & **operator=** (const [tensor_ref_](#) &b)
- bool **operator==** (const [wbarray](#)< double > *A) const
- bool **operator==** (const [wbarray](#)< [wbcomplex](#) > *A) const

- `tensor_ref_ & init` (const `wbarray`< double > &A)
- `tensor_ref_ & init` (const `wbarray`< `wbcomplex` > &A)
- `operator bool` () const
- `bool operator!` () const
- `int check` (const char *F=NULL, int L=0) const
- `int rank` (unsigned r) const
- `bool sameSize` (const `wbvector`< size_t > &S) const
- `double xflops` () const
- `wbstring sizeStr` () const
- `const wbvector`< size_t > * `getSIZE` () const
- `template<class TC >`
`void contract` (const char *F, int L, const `ctrlidx` &ia, const `tensor_ref_` &b, const `ctrlidx` &ib, `wbarray`< TC > &C) const
- `template<>` `void contract` (const char *F, int L, const `ctrlidx` &ia, const `tensor_ref_` &b, const `ctrlidx` &ib, `wbarray`< double > &C) const
- `template<>` `void contract` (const char *F, int L, const `ctrlidx` &ia, const `tensor_ref_` &b, const `ctrlidx` &ib, `wbarray`< `wbcomplex` > &C) const

Public Attributes

- const `wbarray`< double > * **Ar**
- const `wbarray`< `wbcomplex` > * **Az**

The documentation for this class was generated from the following files:

- `wbarray.hh`
- `wbarray.cc`

4.91 tensorRef_ Class Reference

Public Member Functions

- `tensorRef_` (const `tensorRef_` &B)
- `template<class T >`
`tensorRef_` (const `wbarray`< T > &A, const `iTags` *t_=NULL)
- `tensorRef_ & operator=` (const `tensorRef_` &B)
- `tensorRef_ & init` (const `tensorRef_` &B)
- `tensorRef_ & save2` (`tensorRef_` &B)
- `template<class T >`
`tensorRef_ & init` (const `wbarray`< T > &A, const `iTags` *t_=NULL, char c=0)
- `size_t numel` () const
- `char check` (const char *F=NULL, int L=0) const
- `char Check` (const char *F, int L, unsigned &id_)
- `int UpdateItagsCtr` (const char *F, int L, const `ctrlidx` &ia, `tensorRef_` &b, const `ctrlidx` &ib, const char *tag)
- `template<class T >`
`int contract` (const char *F, int L, const `tensorRef_` &b, `tensorRef_` &c, double *flops, `wbarray`< T > *C) const
- `int contract` (const char *F, int L, const `tensorRef_` &b, `tensorRef_` &c, double &flops) const
- `template<class T >`
`int contract` (const char *F, int L, const `tensorRef_` &b, `tensorRef_` &c, `wbarray`< T > &C) const
- `wbstring toStr` () const
- `const tensorRef_ & print` (const char *istr=NULL, char vflag=1) const

Public Attributes

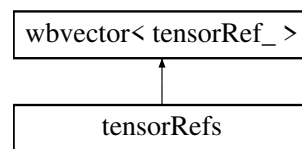
- unsigned **id**
- char **conj**
- [tensor_ref_ R](#)
- [iTags it](#)
- [wbvector](#)< size_t > **S**
- [wbvector](#)< unsigned > **ID**

The documentation for this class was generated from the following files:

- wbararray.hh
- wbararray.cc

4.92 tensorRefs Class Reference

Inheritance diagram for tensorRefs:

**Public Member Functions**

- **tensorRefs** (unsigned l=0)
- char **check** (const char *F, int L) const
- char **Check** (const char *F, int L)
- void **print** ()
- void **print** (const char *F, int L) const
- double **getOptimalCtrOrder** (const char *F, int L, [wbperm](#) &P, [wbperm](#) *pfin=NULL, char useP=0, char vflag=0)

Public Member Functions inherited from [wbvector](#)< [tensorRef_](#) >

- **wbvector** (size_t l, [tensorRef_](#) x)
- **wbvector** (size_t l, const [tensorRef_](#) *d0=NULL, char ref=0)
- **wbvector** (const [wbvector](#) &v)
- **wbvector** (const [wbvector](#)< T2 > &v)
- **wbvector** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- **wbvector** (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- **wbvector** (const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) & **init** (size_t l=0)
- [wbvector](#) & **init** (size_t l, const [tensorRef_](#) *d0)
- [wbvector](#) & **init** (size_t l, const [tensorRef_](#) *d0, char ref)
- [wbvector](#) & **init** (const [wbvector](#) &v)
- [wbvector](#) & **init** (size_t n, const [wbvector](#) &v)
- [wbvector](#)< [tensorRef_](#) > & **init** (const [wbsparray](#)< [tensorRef_](#) > &S)

- **wbvector** & **init** (size_t l1, const **tensorRef_** *d1, size_t l2, const **tensorRef_** *d2)
- int **init** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type QS_UNUSED_↵
VAR=1, char ref=0)
- int **init** (const mxArray *a, const char *istr=NULL, char check_type=1)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- **wbvector** & **initp** (size_t l, const **tensorRef_** *d0, size_t n, char init=1)
- **wbvector** & **init2val** (size_t l, const **tensorRef_** &x)
- **wbvector** & **initStride** (size_t l, const **tensorRef_** *d0, size_t stride)
- **wbvector** & **init_stride** (const **tensorRef_** *d0, size_t n, size_t m, size_t D0)
- **wbvector** & **init_data** (const **tensorRef_** *d0)
- **wbvector** & **init_data** (const T0 *d0)
- **wbvector** & **init_data** (const char *F, int L, const T0 *d0)
- **wbvector**< **tensorRef_** > & **initT** (const **wbsparray**< T2 > &S)
- **wbvector**< **tensorRef_** > & **initT** (size_t l, const T2 *v)
- **wbvector**< **tensorRef_** > & **initT** (const **wbvector**< T2 > &v)
- **wbvector**< **tensorRef_** > & **initT** (const char *F, int L, const **wbvector**< T2 > &v)
- **wbvector**< **tensorRef_** > & **initT** (const **wbsparray**< TD > &S)
- **wbvector**< **tensorRef_** > & **initT_** (const **wbvector**< T2 > &v)
- **wbvector** & **init2ref** (const **wbvector** &v)
- **wbvector** & **init2ref** (size_t l, const **tensorRef_** *d0)
- **wbvector** & **unRef** ()
- int **init_Struct** (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)
- int **init_base** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)
- int **init_mpf** (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)
- void **initDef** (size_t n)
- void **initDef** (const **wbvector** &v)
- **wbvector** & **operator=** (const **wbvector** &v)
- **wbvector**< **tensorRef_** > & **operator=** (const **wbvector**< TB > &v)
- **wbvector** & **set** (const **tensorRef_** &x)
- **wbvector** & **set** (const **wbvector**< size_t > &l, const **tensorRef_** &x)
- void **set** (size_t offset, const **tensorRef_** *x, size_t n)
- void **set** (const **wbvector**< double > &R, const **wbvector**< double > &l)
- void **set** (const **wbvector**< double > &R, const **wbvector**< double > &l)
- **wbvector** & **setMin** (const **tensorRef_** &x)
- **wbvector** & **setMax** (const **tensorRef_** &x)
- **wbvector** & **add** (const **wbvector**< size_t > &l, const **tensorRef_** &x)
- **wbvector** & **add** (const **tensorRef_** *d)
- **wbvector** & **Set** (const **wbvector**< size_t > &l, const **wbvector** &v)
- void **Sort** (**wbperm** &p, char dir=+1)
- void **Sort** (char dir=+1)
- void **sort** (**wbvector** &v2, **wbperm** &p) const
- void **sort** (**wbvector** &v2) const
- **wbperm** & **pSort** (**wbperm** &p) const
- bool **isSorted** () const
- bool **isSortedU** () const
- bool **isUnique** () const
- int **contains** (const **tensorRef_** &x) const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const

- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isFinite** () const
- size_t **find1** (const [tensorRef_](#) &x) const
- size_t **find1** () const
- size_t **count** (const [tensorRef_](#) &x) const
- size_t **icount** (const [tensorRef_](#) &x) const
- size_t **nnz** () const
- size_t **nnz** ([tensorRef_](#) eps) const
- size_t **numZeros** ([tensorRef_](#) eps=[tensorRef_](#)(1E-14)) const
- size_t **numEQ** (const [tensorRef_](#) &) const
- size_t **numGT** (const [tensorRef_](#) &) const
- size_t **numLT** (const [tensorRef_](#) &) const
- [wbindex](#) & **find** (const [tensorRef_](#) &x, [wbindex](#) &l, char iflag=0) const
- [wbindex](#) & **find** (const [tensorRef_](#) &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **find** ([wbindex](#) &l) const
- [wbindex](#) & **find** ([wbindex](#) &l, [wbindex](#) &lx) const
- [wbvector](#)< size_t > **find** () const
- [wbindex](#) & **findGT** (const [tensorRef_](#) &x, [wbindex](#) &l, char iflag=0) const
- [wbindex](#) & **findGT** (const [tensorRef_](#) &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **findzeros** ([wbindex](#) &l) const
- [wbindex](#) & **findRange** (const [tensorRef_](#) &x1, const [tensorRef_](#) &x2, [wbindex](#) &l, const char *w=NULL) const
- int **findClosestSorted** ([tensorRef_](#) r) const
- int **findClosest** ([tensorRef_](#) r) const
- size_t **findValues** (const [wbvector](#) &v, [wbindex](#) &l) const
- [wbvector](#) & **setp** (const [tensorRef_](#) *x)
- void **reset** ()
- void **cpy** (const [tensorRef_](#) *v)
- void **cpy** (const [tensorRef_](#) *x, size_t m)
- [wbvector](#) & **save2** ([wbvector](#) &v)
- void **swap** ([wbvector](#) &v)
- [wbvector](#) & **swap** (size_t i, size_t j)
- [wbvector](#) & **Collect** ([tensorRef_](#) &A1, [tensorRef_](#) &A2)
- [wbvector](#) & **Collect** ([tensorRef_](#) &A1, [tensorRef_](#) &A2, [tensorRef_](#) &A3)
- [wbvector](#) & **Skip** (size_t)
- [wbvector](#) & **Skip** (const WBINDEX &, char rflag=0)
- [wbvector](#) & **Move2_FE** (const WBINDEX &l, char FE_flag)
- [wbvector](#) & **Move2front** (const WBINDEX &l)
- [wbvector](#) & **Move2front** (size_t k)
- [wbvector](#) & **Move2end** (const WBINDEX &l)
- [wbvector](#) & **Move2end** (size_t k)
- [wbvector](#) & **mvLastTo** (size_t k)
- [wbvector](#) & **mvFirstTo** (size_t k)
- bool **checkSameLength** (const [wbvector](#) &a, const char *F=NULL, int L=0, const char *istr="") const
- bool **checkSameLength** (const [wbvector](#) &a, const [wbvector](#) &b, const char *F=NULL, int L=0, const char *istr="") const
- void **operator+=** (const T_ &a)
- void **operator+=** (const [wbvector](#) &b)
- void **operator-=** (const T_ &a)
- void **operator-=** (const [wbvector](#) &b)
- [wbvector](#) & **operator*=** (const T_ &a)
- [wbvector](#) & **operator/=** (const T_ &a)

- **wbvector operator*** (const T_ &a) const
- **wbvector operator+** (const T_ &a) const
- **wbvector operator+** (const **wbvector** &b) const
- **wbvector operator-** (const T_ &a) const
- **wbvector operator-** (const **wbvector** &b) const
- **wbvector & Plus** (const **tensorRef_** *b)
- **wbvector & Plus** (const **tensorRef_** *b, **tensorRef_** bfac)
- **wbvector & Plus** (const **wbvector** &b, **tensorRef_** bfac=1)
- void **plus** (const **wbvector** &b, **wbvector** &c, **tensorRef_** bfac=1, **tensorRef_** cfac=0) const
- **wbvector & times** (**tensorRef_** fac, **wbvector** &x) const
- **wbvector & RevertSigns** ()
- int **isum** () const
- **tensorRef_ sum** () const
- **tensorRef_ sum** (size_t i1, const char *estr) const
- **tensorRef_ sum** (size_t i1, size_t i2) const
- **tensorRef_ cumsum_** (**wbvector** &S, char xflag=0) const
- **tensorRef_ Cumsum** (char zflag=0) const
- **tensorRef_ cumsum0prod** (**wbvector**< T2 > &b, **wbvector**< **tensorRef_** > &cs, char xflag) const
- **wbvector & Eldiv** (const **wbvector** &v) const
- **wbvector & tensorProd** (const **wbvector** &v2, **wbvector** &vv) const
- **tensorRef_ scalarProd** (const **wbvector** &v) const
- **tensorRef_ scalarProd** (const **tensorRef_** *v) const
- **tensorRef_ norm** () const
- **tensorRef_ norm2** () const
- double **normDiff** (const **wbvector** &v, **tensorRef_** fac=1) const
- **tensorRef_ normDiff2** (const **wbvector** &v, **tensorRef_** fac=1) const
- double **normDiff2_** (const **wbvector** &v, double fac=1) const
- **tensorRef_ GSProject** (const **tensorRef_** *v, char isnorm=0, char tflag=0) const
- **wbvector & Normalize** (const char *F=NULL, int L=0)
- **wbvector & subtract** (const **tensorRef_** *d)
- void **getReal** (**wbvector**< double > &R) const
- void **getReal** (**wbvector**< double > &R) const
- void **getImag** (**wbvector**< double > &I) const
- void **getImag** (**wbvector**< double > &I) const
- **wbvector & Conj** ()
- **wbvector**< **wbcomplex** > & **Conj** ()
- **wbvector & applyFlag** (char &flag, const **tensorRef_** &fac, **wbvector** &a)
- bool **allEqual** () const
- bool **allEqual** (const **tensorRef_** &a) const
- bool **allZero** () const
- bool **allOnes** () const
- bool **allEqual2** (const **tensorRef_** &a, **tensorRef_** eps=1E-15) const
- bool **allUnequal** (const **tensorRef_** &a) const
- bool **any** () const
- bool **anyUnequal** (const **tensorRef_** &a) const
- bool **anyUnequal** (const **wbvector**< size_t > &l, const **tensorRef_** &x, size_t *k=NULL) const
- bool **anyEqual** (const **tensorRef_** &a, size_t *i1=NULL) const
- bool **anyEqual** (const **wbvector**< size_t > &l, const **tensorRef_** &x, size_t *k=NULL) const
- bool **anyGT** (const **tensorRef_** &a) const
- bool **anyGE** (const **tensorRef_** &a) const
- bool **anyLT** (const **tensorRef_** &a) const
- bool **anyLE** (const **tensorRef_** &a) const
- bool **allGT** (const **tensorRef_** &a) const
- bool **allGE** (const **tensorRef_** &a) const
- bool **allLE** (const **tensorRef_** &a) const

- bool **allLT** (const [tensorRef_](#) &a) const
- bool **allIn** (const [tensorRef_](#) &a1, const [tensorRef_](#) &a2) const
- bool **anySmallVals** (const [tensorRef_](#) eps=1E-15) const
- bool **isEqual** (const [tensorRef_](#) *v) const
- bool **isEqual** (const [wbvector](#) &v) const
- bool **isEmpty** () const
- **operator bool** () const
- bool **operator!** () const
- bool **operator==** (const [wbvector](#) &v) const
- bool **operator!=** (const [wbvector](#) &v) const
- bool **isEqualR** (const [wbvector](#) &v) const
- bool **operator^** (const [wbvector](#) &v) const
- bool **deepEqualP** (const [wbvector](#) &B) const
- char **cmp** (const [wbvector](#) &v) const
- bool **operator<** (const [wbvector](#) &v) const
- int **operator<** (const [tensorRef_](#) &r) const
- bool **operator<=** (const [wbvector](#) &v) const
- bool **operator>** (const [wbvector](#) &v) const
- bool **operator>=** (const [wbvector](#) &v) const
- [tensorRef_](#) & **el** (size_t i) const
- [tensorRef_](#) **elx** (size_t i, const [tensorRef_](#) &x) const
- const [tensorRef_](#) & **operator[]** (size_t i) const
- [tensorRef_](#) & **operator[]** (size_t i)
- [wbvector](#) **operator[]** (const [wbvector](#) < size_t > &l) const
- [tensorRef_](#) & **last** () const
- [tensorRef_](#) & **last** (size_t &i) const
- [tensorRef_](#) & **last** (T1 &i_) const
- [tensorRef_](#) & **p2last** () const
- [tensorRef_](#) * **ref** (size_t i) const
- [wbvector](#) & **setRand** (double fac=1., double shift=0.)
- [wbvector](#) < [wbcomplex](#) > & **setRand** (double fac, double shift)
- void **setIdx** (double shift=0., double fac=1.)
- int **set2Group** (const [wbperm](#) &P, const WBINDEX &D, const [tensorRef_](#) *S0=NULL, char iflag=0)
- [wbvector](#) & **Resize_** (size_t n)
- [wbvector](#) & **Shorten2** (size_t n)
- [wbvector](#) & **Resize** (size_t n, const [tensorRef_](#) *d=NULL)
- [wbvector](#) & **resize** (size_t n, [wbvector](#) &X) const
- [wbvector](#) & **cat** (const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#) < [wbvector](#) > &vv)
- [wbvector](#) & **Cat** (const [wbvector](#) < [wbvector](#) * > &vv)
- [wbvector](#) & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)
- [wbvector](#) & **Cat** (const [wbvector](#) < T1 > *v1, const [wbvector](#) < T2 > *v2, const [wbvector](#) < T3 > *v3=0, const [wbvector](#) < T3 > *v4=0)
- [wbvector](#) < [tensorRef_](#) > & **Cat** (const [wbvector](#) < T1 > *v1, const [wbvector](#) < T2 > *v2, const [wbvector](#) < T3 > *v3, const [wbvector](#) < T3 > *v4)
- [wbvector](#) < [tensorRef_](#) > & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)
- [wbvector](#) & **Append** (const [wbvector](#) &v2)
- [wbvector](#) & **Append** (const [tensorRef_](#) &x)
- [wbvector](#) & **Append** (size_t d2, const [tensorRef_](#) *x)
- [wbvector](#) **append** (const [wbvector](#) &v) const
- [wbvector](#) & **append** (const [wbvector](#) &v2, [wbvector](#) &vout) const
- void **append** (const [tensorRef_](#) *x, size_t d2, [wbvector](#) &vout) const

- int **recLess** (const [tensorRef_](#) *v) const
- int **recLessE** (const [tensorRef_](#) *v) const
- int **recCompare** (const [tensorRef_](#) *v) const
- [tensorRef_](#) **max** () const
- [tensorRef_](#) **max** (size_t *k) const
- [tensorRef_](#) **max** (unsigned &k) const
- [tensorRef_](#) **max_** ([tensorRef_](#) x) const
- [tensorRef_](#) **min** () const
- [tensorRef_](#) **min** (size_t *k) const
- [tensorRef_](#) **min** (unsigned &k) const
- [tensorRef_](#) **min_** ([tensorRef_](#) x) const
- [tensorRef_](#) **aMax** (size_t *k=NULL) const
- [tensorRef_](#) **aMax** (unsigned &k) const
- [tensorRef_](#) **aMin** (char zflag=0, size_t *k=NULL) const
- [tensorRef_](#) **maxneg** (size_t *k=NULL) const
- [tensorRef_](#) **minpos** (size_t *k=NULL) const
- [tensorRef_](#) **avg** () const
- [tensorRef_](#) **std** () const
- void **range** ([tensorRef_](#) &xmin, [tensorRef_](#) &xmax) const
- [tensorRef_](#) **range** () const
- [wbvector](#) & **Permute** (const [wbperm](#) &P, char iflag=0)
- [wbvector](#) & **permute** ([wbvector](#) &, const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) **permute** (const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) & **blockPermute** (const [wbperm](#) &P, [wbvector](#) &, char iflag=0) const
- [wbvector](#) & **BlockPermute** (const [wbperm](#) &P, char iflag=0)
- int **blockCompare** (const [wbperm](#) &pa, const [wbvector](#) &b, const [wbperm](#) &pb) const
- void **get** (const WBINDEX &I0, [wbvector](#) &v) const
- void **get** (const [wbvector](#) < size_t > &I, [tensorRef_](#) *v) const
- void **get** (size_t i1, size_t i2, [wbvector](#) &v) const
- void **selectSU** (const WBINDEX &I)
- void **select** (const WBINDEX &I, [tensorRef_](#) *r) const
- [wbvector](#) & **select** (const WBINDEX &I, [wbvector](#) &a) const
- [wbvector](#) **select** (const WBINDEX &I) const
- [wbvector](#) & **Select** (const WBINDEX &I)
- [wbvector](#) & **BlockSelect** (const WBINDEX &I, const WBINDEX &D)
- [wbvector](#) & **getI** (size_t k, [wbvector](#) &v) const
- [wbvector](#) & **getI** (const [wbindex](#) &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **getI** (const WBINDEX &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **initI** (const [wbvector](#) &a, const [wbindex](#) &ia)
- [wbvector](#) & **initI** (const [wbvector](#) &a, const [wbindex](#) &ia, const [wbvector](#) &b, const [wbindex](#) &ib, char uflag=1)
- [wbvector](#) & **Flip** ()
- void **flip** ([wbvector](#) &a) const
- [wbvector](#) **flip** () const
- [tensorRef_](#) **prod** () const
- [tensorRef_](#) **prod** ([tensorRef_](#) x) const
- [tensorRef_](#) **prod** (size_t i, size_t i2) const
- [tensorRef_](#) **prod** (size_t i, size_t i2, [tensorRef_](#) x) const
- [tensorRef_](#) **prod** (const T1 *l, size_t n) const
- T2 **prod** (const T1 *l, size_t n, T2 &s2) const
- void **prod2** ([tensorRef_](#) &p1, [tensorRef_](#) &p2, const [tensorRef_](#) &p0=0) const
- void **info** (const char *istr) const
- void **print** (const char *istr, char mflag=0) const
- void **printdata** (const char *istr=0) const
- void **put** (const char *F, int L, const char *vname="ans", char tflag=0, const char *ws="base") const
- void **put** (const char *vname="ans", char tflag=0, const char *ws="base") const

- void **dispWithMsg** (const char *file, int line, const char *msg, const char *vname="") const
- void **mat2mx** (mxArray *&a, char dim=2) const
- void **mat2mxs** (mxArray *a, char field_nr, char dim=2) const
- mxArray * **toMx_Struct** (const char tflag=0) const
- mxArray * **toMxP_S** (const char tflag=0) const
- mxArray * **toMx** (const char tflag=0) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMxP** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base_d** (const char tflag=0) const
- mxArray * **toMx_offset** (const [tensorRef_](#) &offset, const char tflag=0) const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tflag=0) const
- void **add2MxStruct** (mxArray *S, const char *vname, char tflag=0) const
- mxArray * **mxCreateCell** (unsigned m, unsigned n) const
- void **add2MxCell** (mxArray *S, unsigned i, char tflag=0) const
- void **putg** (const char *vname="ans", char tflag=0) const
- void **putx** (const char *vname="ans", char tflag=0) const
- [wbstring](#) **toStr** (int n, const char *sep) const
- [wbstring](#) **toStr** () const
- [wbstring](#) **toStr** (int n, const char *sep) const
- [wbstring](#) **sizeStr** () const
- [wbstring](#) **toStrf** (const char *fmt0="", const char *sep=" ", unsigned stride=0, const char *sep2="") const
- [wbstring](#) **toStrf** (const char *fmt_, const char *sep, unsigned stride, const char *sep2) const

Additional Inherited Members

Public Attributes inherited from [wbvector< tensorRef_ >](#)

- [tensorRef_](#) * data
- size_t len
- char isref

Protected Member Functions inherited from [wbvector< tensorRef_ >](#)

- [wbvector](#) & **RENEW** (size_t n, const [tensorRef_](#) *d=NULL, size_t l=-1, char init=1)
- [wbvector](#) & **RENEW_VAL** (size_t n, const [tensorRef_](#) &x)

The documentation for this class was generated from the following files:

- wbararray.hh
- wbararray.cc

4.93 Wb::termcolor Class Reference

Public Member Functions

- **termcolor** (const char *tag)
- **termcolor** (Wb::TCOLS k)
- **termcolor** (unsigned k)
- **termcolor** & **init** (unsigned k)

Static Public Member Functions

- static int **ID** (const char *tag)

Public Attributes

- char * **em**
- char * **e1**

The documentation for this class was generated from the following files:

- wblog.h
- wblog.c

4.94 Wb::tmpFile Class Reference

Public Member Functions

- **tmpFile** (const char *F, int L, const char *f=NULL, const char *x=NULL, const char *mode=NULL)
- void **init** (const char *F, int L, const char *f=NULL, const char *x=NULL, const char *mode=NULL)
- int **blogf** (const char *F, int L, const char *fmt,...)
- int **sepline** (const char *w="-", unsigned len=-1)
- void **flush** ()

Public Attributes

- FILE * **fid**
- int **ncall**
- **wbstring** **fout**

The documentation for this class was generated from the following files:

- wbio.h
- wbio.c

4.95 Wb::VersionInfo Class Reference

Public Member Functions

- [VersionInfo](#) & **init** ()
- [wbstring](#) **toStr** () const
- void **print** () const
- mxArray * **toMx** () const

Public Attributes

- char **fctn** [32]
- char **tag** [8]
- char **matlab** [16]
- char **mpfr** [16]
- char **compiler** [16]
- char **os** [24]
- char **qspace** [16]
- char **git** [16]
- char **flags** [128]
- char **compiled** [64]

The documentation for this class was generated from the following files:

- wblibx.h
- wblog.c

4.96 wbarrray< T > Class Template Reference

Public Member Functions

- **wbarrray** (size_t d1)
- **wbarrray** (size_t d1, size_t d2)
- **wbarrray** (size_t d1, size_t d2, size_t d3)
- **wbarrray** (size_t d1, size_t d2, size_t d3, size_t d4)
- **wbarrray** (char *sidx)
- **wbarrray** (const [wbvector](#)< unsigned > &S)
- **wbarrray** (const [wbvector](#)< size_t > &S, T *d0, char ref=0)
- **wbarrray** (const [wbarrray](#) &a, char ref=0)
- template<class T2 >
 wbarrray (const [wbarrray](#)< T2 > &a)
- **wbarrray** (const mxArray *a, char ref=0, char vec=0)
- **wbarrray** (const char *F, int L, const mxArray *a, char ref=0, char vec=0)
- **wbarrray** (const [wbMatrix](#)< T > &a, char ref=0)
- [wbarrray](#) & **initX** ()
- [wbarrray](#) & **init** ()
- [wbarrray](#) & **init** (size_t d1, const T *d=NULL)
- [wbarrray](#) & **init** (size_t d1, size_t d2, const T *d=NULL)
- [wbarrray](#) & **init** (size_t d1, size_t d2, size_t d3, const T *d=NULL)
- [wbarrray](#) & **init** (size_t d1, size_t d2, size_t d3, size_t d4, const T *d=NULL)

- [wbarray](#) & [init2ref](#) (const [wbarray](#) &a)
- [wbarray](#) & [initScalar](#) (T x=0, unsigned r=2)
- [wbarray](#) & [init](#) (const [wbarray](#) &a, char ref=0)
- [wbarray](#) & [set](#) (const [wbarray](#) &A)
- template<class T_ >
 [wbarray](#) & [set](#) (const [wbarray](#) &A, T_ fac)
- [wbarray](#) & [init](#) (const [wbMatrix](#)< T > &a, char ref=0)
- [wbindex](#) & [ind2sub](#) (size_t k, [wbindex](#) &l) const
- void [adjustMMat](#) (const char *F, int L, char flag, const T fac, [wbarray](#) &A) const
- void [adjustDMat](#) (const char *F, int L, char flag, const T fac, [wbarray](#) &A) const
- void [adjustVec](#) (const char *F, int L, char &flag, [wbarray](#) &A, char pos) const
- void [adjustVec](#) (const char *F, int L, char &flag, char pos)
- void [adjustCMat](#) (const char *F, int L, T cfac, size_t s1, size_t s2, char cforce)
- void [setBlock](#) (const [wbvector](#)< WBINDEX > &D, const [wbindex](#) &IB, const [wbarray](#) &A)
- void [addBlock](#) (const WBINDEX &l, const [wbarray](#) &A, const char dflag=0)
- [wbarray](#) & [addBlock](#) (size_t i, size_t j, size_t n, size_t m, [wbarray](#) &a) const
- T [norm2block](#) (size_t i, size_t j, size_t n, size_t m) const
- [wbarray](#) & [getBlock](#) (size_t i, size_t j, size_t n, size_t m, [wbarray](#) &a) const
- [wbarray](#) & [BlockDiag](#) (const [wbvector](#)< [wbarray](#) > &D)
- [wbarray](#) & [blockTrace](#) (size_t D, [wbarray](#) &a, size_t D2=-1) const
- void [copyStride](#) (T *dd, size_t stride, T afac=1) const
- void [addStride](#) (T *dd, size_t stride, T afac=1) const
- template<class T0 >
 [wbarray](#) & [init_T](#) (const T0 *dd)
- template<class T0 >
 [wbarray](#) & [init_T](#) (const char *F, int L, const T0 *dd)
- [wbarray](#) & [initT](#) (const [wbarray](#)< T > &a)
- template<class T0 >
 [wbarray](#) & [initT](#) (const [wbarray](#)< T0 > &a)
- template<class T0 >
 [wbarray](#) & [initT](#) (const char *F, int L, const [wbarray](#)< T0 > &a)
- [wbarray](#) & [init](#) (char *s, T *d=NULL)
- [wbarray](#) & [init](#) (const WBINDEX &S, T *d=NULL)
- template<class IT >
 [wbarray](#) & [init](#) (const [wbvector](#)< IT > &S_, T *d=NULL)
- [wbarray](#) & [init_bare](#) (const WBINDEX &S)
- [wbarray](#) & [init_bare](#) (size_t d1, size_t d2)
- [wbarray](#) & [init](#) (const char *F, int L, const mxArray *a, char ref=0, char vec=0)
- [wbarray](#) & [init](#) (const char *F, int L, const mxArray *a, [wbperm](#) &P0)
- [wbarray](#) & [init](#) (const mxArray *a, char ref=0, char vec=0)
- [wbarray](#) & [init](#) (const mxArray *a, [wbperm](#) &P0)
- [wbarray](#) & [init2ref](#) (const T *x, const WBINDEX &S)
- [wbarray](#) & [init2ref](#) (const T *x)
- [wbarray](#) & [init2ref](#) (size_t d1, const T *x)
- [wbarray](#) & [init2ref](#) (size_t d1, size_t d2, const T *x)
- [wbarray](#) & [init2ref](#) (size_t d1, size_t d2, size_t d3, const T *x)
- int [isRef](#) () const
- [wbarray](#) & [Instantiate](#) (const char *F=NULL, int L=0)
- [wbarray](#) & [unRef](#) ()
- void [init2Vec](#) (const T *d0, size_t len, char ref=0)
- void [init2ref](#) (const [wbarray](#) &A, size_t s1, size_t s2)
- void [initTst](#) ()
- void [init2Vec](#) (const [wbvector](#)< T > &a)
- [wbarray](#) & [initIdentity](#) (size_t d, char dflag=0, T dval=1)
- [wbarray](#) & [initIdentity](#) (const WBINDEX &S, char dflag=0)

- [wbarry](#) & **Reduce2Id** ()
- [wbarry](#) & **initIdentityB** (size_t d1, size_t d2, size_t k=0)
- [wbarry](#) & **initIdentityB3** (size_t d1, size_t d2, size_t D, size_t i0, T x=1)
- [wbarry](#) & **Expand2Projector** (const T eps=0)
- [wbarry](#) & **ExpandDiagonal** (unsigned i1, unsigned i2)
- [wbarry](#) & **ExpandDiagonal** ()
- void **Diag2Vec** ()
- [wbarry](#) & **Reduce2Diag** (T eps=0)
- [wbarry](#) & **initDiag** (unsigned n, const T *d)
- [wbarry](#) & **initDiag** (const [wbvector](#)< T > &d)
- void **initVec** (size_t d, const T val=0)
- [wbarry](#) & **resize** (const [wbvector](#)< size_t > &S, [wbarry](#) &B) const
- [wbarry](#) & **resizeNumCols** (size_t n, [wbarry](#) &B) const
- void **Resize** (const [wbvector](#)< size_t > &S, const T *d0=NULL)
- void **Resize** (size_t s1, size_t s2)
- [wbarry](#) & **Resize1** (unsigned i, size_t Si)
- [wbarry](#) & **ResizeM** (unsigned r, const [wbvector](#)< size_t > &Sm)
- [wbarry](#) & **Enlarge** (size_t dim, size_t D, size_t istart=0)
- void **swap** ([wbarry](#) &A)
- [wbarry](#) & **save2** ([wbarry](#) &A)
- [wbarry](#) & **save2** ([wbMatrix](#)< T > &)
- [wbarry](#) & **Append2** (const char *F, int L, [wbarry](#) &B, unsigned dim)
- [wbarry](#) & **Append2** ([wbarry](#) &B, unsigned dim)
- void **Append2** (const char *F, int L, [wbarry](#) &B, unsigned dim, [wbarry](#)< double > &W, double eps)
- void **Append2** ([wbarry](#) &B, unsigned dim, [wbarry](#) &W, double eps)
- void **rand** (const UVEC &d)
- void **setRand** (const char pnflag=0)
- [wbarry](#) & **Reset** (const T *dref)
- [wbarry](#) & **set** (const T &c)
- [wbarry](#) & **operator=** (const [wbarry](#) &a)
- template<class T0 >
 [wbarry](#)< T > & **operator=** (const [wbarry](#)< T0 > &a)
- void **SetRef** (const widx_t *l, size_t len, const T *d0)
- void **SetRef** (size_t k, const T *d0)
- const T * **ref** (const widx_t *l, size_t n) const
- T * **ref** (const widx_t *l, size_t n)
- T * **col** (size_t k)
- const T * **col** (size_t k) const
- T * **row** (size_t k)
- const T * **row** (size_t k) const
- [wbvector](#)< T > & **getCol** (size_t k, [wbvector](#)< T > &v) const
- [wbarry](#)< T > & **getCol** (size_t k, [wbarry](#)< T > &a) const
- [wbvector](#)< T > & **getRow** (size_t k, [wbvector](#)< T > &v) const
- [wbarry](#)< T > & **getRow** (size_t k, [wbarry](#)< T > &a) const
- [wbvector](#)< T > **getDiag** () const
- T * **ref** (const WBINDEX &l)
- const T * **ref** (const WBINDEX &l) const
- const T * **ref** (widx_t i) const
- T * **ref** (widx_t i)
- const T * **ref** (widx_t i1, widx_t i2) const
- T * **ref** (widx_t i1, widx_t i2)
- const T * **ref** (widx_t i1, widx_t i2, widx_t i3) const
- T * **ref** (widx_t i1, widx_t i2, widx_t i3)
- const T & **operator[]** (size_t i) const
- T & **operator[]** (size_t i)

- `const T & operator() (size_t i) const`
- `T & operator() (size_t i)`
- `const T & operator() (size_t i, size_t j) const`
- `T & operator() (size_t i, size_t j)`
- `const T & operator() (size_t i, size_t j, size_t k) const`
- `T & operator() (size_t i, size_t j, size_t k)`
- `const T & operator() (const WBINDEX &l) const`
- `T & operator() (const WBINDEX &l)`
- `const T & element (const WBINDEX &l) const`
- `T & element (const WBINDEX &l)`
- `size_t numel () const`
- `size_t numOM (unsigned r0) const`
- `wbstring sizeStr (unsigned stride=0, const char *sep2="") const`
- `wbarray & Squeeze ()`
- `int skipSingletons (const char *F, int L, unsigned r=-1)`
- `int skipSingletons (unsigned r=-1)`
- `wbarray & appendSingletons (const char *F, int L, unsigned r, unsigned m=0)`
- `wbarray & appendSingletons (unsigned r, unsigned m=0)`
- `void prependSingletons (unsigned r)`
- `wbarray & ExpandOM (const char *F, int L, unsigned r0, const wvector< unsigned > &S, const unsigned *sx=NULL)`
- `wbarray & ExpandOM (const char *F, int L, unsigned r0, unsigned m, const unsigned *s, const unsigned *sx=NULL)`
- `int ExpandOM (const char *F, int L, wbarray &B, unsigned r0)`
- `wbarray & FuseOM (const char *F, int L, unsigned r, unsigned mc=-1)`
- `size_t length () const`
- `size_t size (unsigned i) const`
- `size_t dim (unsigned i) const`
- `size_t dim0 (unsigned i) const`
- `size_t dim1 () const`
- `size_t dim2 () const`
- `void getMatSize (const char *F, int L, size_t &dim1, size_t &dim2) const`
- `void getMatSize (size_t &dim1, size_t &dim2, unsigned m) const`
- `void getMatSize (const ctrlidx &ic, size_t &dimc, size_t &dimk, unsigned r=-1, size_t *dom=NULL) const`
- `unsigned rank () const`
- `bool isRank (unsigned r) const`
- `bool isRankM (unsigned r, unsigned m) const`
- `bool isOpS (size_t *n=NULL) const`
- `bool isOpS (unsigned &n) const`
- `wbarray & SkipTiny_float (const T eps QS_UNUSED_VAR=1e-14)`
- `double SkipTiny_imag (double eps QS_UNUSED_VAR=1e-14)`
- `double SkipTiny (double eps_=1E-14)`
- `wbarray< T > & SkipTrailingZeroSpace (unsigned k, size_t n=0, double eps=1e-14)`
- `bool isMatrix () const`
- `bool isMatrix (size_t d1, size_t d2)`
- `bool isSMatrix (unsigned d=-1) const`
- `bool isSquare () const`
- `bool isEmpty () const`
- `bool isComplex () const`
- `bool isZero (double eps=0., char flag=0) const`
- `bool isFinite () const`
- `operator bool () const`
- `bool operator! () const`
- `wbarray & Symmetrize (const char *F, int L, double *delta=NULL, char cflag=1, char tflag=0, char fflag=0)`
- `wbarray & swapRows (size_t i1, size_t i2)`

- [wbarry](#) & **swapCols** (size_t j1, size_t j2)
- [wbarry](#) & **setCol** (size_t k, size_t k0)
- [wbarry](#) & **setCol** (size_t k, const T *d, T fac=1, size_t stride=1)
- [wbarry](#) & **setRow** (size_t k, const T *d)
- [wbarry](#) & **setCol** (size_t k, const [wbvector](#)< T > &v)
- template<class T2 >
 [wbarry](#)< T > & **setCol** (size_t k, const [wbvector](#)< T2 > &v)
- [wbvector](#)< T > & **colNorm2** ([wbvector](#)< T > &a) const
- T **colNorm2** (size_t k) const
- [wbvector](#)< T > & **normDim2** (unsigned k, [wbvector](#)< T > &xk) const
- [wbvector](#)< T > & **normDim** (unsigned k, [wbvector](#)< T > &xk) const
- unsigned **QRdecomp** (const char *F, int L, [wbarry](#)< T > &Q, [wbarry](#)< T > &R, T eps=1E-15)
- int **Householder** (const char *F, int L, size_t k, T *u, WBPERM *P=NULL, T eps=1E-15)
- unsigned **eigTriDiag** (const char *F, int L, [wbvector](#)< T > &E, [wbarry](#)< T > *U=NULL, T eps=1E-15) const
- [wbarry](#) & **ColProject** (size_t k1, size_t k2, char nflag=0, char tnorm=0)
- [wbarry](#) & **ColPermute** (const [wbperm](#) &P, char iflag=0)
- bool **isOrthogonalCol** (size_t k, char tnorm=0) const
- bool **isOrthoCols** (T *x2=NULL, char tnorm=0, T eps=1E-14) const
- [wbarry](#) & **SignCol** (size_t k, const T *d0, char tnorm=0)
- [wbarry](#) & **FlipSignCol** (size_t k)
- [wbarry](#) & **SignConventionCol** (size_t k=-1, double eps=1E-12)
- T **NormalizeCol** (size_t k, char tnorm=0, char qflag=0)
- [wbarry](#) & **NormalizeCols** (const char *F, int L, double *amin=NULL, double *amax=NULL, char tnorm=0)
- [wbarry](#) & **OrthoNormalizeCols** (const char *F=NULL, int L=0, char tnorm=0, char qxflag=0, double eps=1E-14, unsigned np=1)
- [wbarry](#) & **balanceOp** (const char *F, int L, T &xref, double &xscale)
- bool **isProptold** (T &x, T eps=1E-14) const
- bool **isProptold** (T *x_=NULL, T eps=1E-14) const
- bool **isDiagMatrix** (double eps=1E-14) const
- bool **isIdentityMatrix** (double eps=1E-14) const
- bool **isDiagMatrix** (double *eps) const
- bool **isIdentityMatrix** (double *eps) const
- size_t **nnz** (const T eps=0, WBINDEX *I=NULL) const
- double **sparsity** (const T eps=0) const
- bool **isHConj** (const [wbarry](#) &B, double eps=1E-12, double *xref=NULL) const
- bool **isHConj** (double eps=1E-12, double *xref=NULL) const
- bool **isAHerm** (const [wbarry](#) &B, double eps=1E-12, double *xref=NULL) const
- bool **isAHerm** (double eps=1E-12, double *xref=NULL) const
- char **hasGroupSize** (size_t s1, size_t s2) const
- template<class T1 >
 int **aMax** (T &x, T1 &k) const
- T **aMax** () const
- template<class T1 >
 T **aMax** (T1 &i, T1 &j, const [wbarry](#) *B=NULL) const
- double **maxDiff** (const [wbarry](#) &B) const
- T **froNorm2** (const [wbarry](#) &B) const
- T **norm2** () const
- T **norm** () const
- T **hasNorm2** (const T &x2) const
- T **normCol2** (size_t k) const
- T **norm2Recs** (long i1, long i2) const
- T **norm2Cols** (long j1, long j2) const
- [wbvector](#)< T > & **norm2Cols** ([wbvector](#)< T > &x2) const
- [wbvector](#)< T > **normCols** () const
- T **scalarProd** (const [wbarry](#) &B) const

- **T sum** () const
- **wbarray & sum** (const WBINDEX &l, **wbarray** &) const
- **wbarray & sum** (const char *sidx, **wbarray** &A) const
- char **fitsSize** (const **wbarray** &, char strict=0) const
- size_t **getSize** (unsigned i, char lflag=1) const
- **wbvector**< size_t > & **getSize** (const **wbvector**< unsigned > &l, **wbvector**< size_t > &S) const
- size_t **getSizeM** (unsigned r)
- bool **sameSize** (const size_t *s, unsigned n) const
- bool **sameSize** (size_t d1, size_t d2) const
- template<class Ts >
bool **sameSizeM** (unsigned r, unsigned m, const Ts *s) const
- template<class TB >
bool **sameSize** (const **wbarray**< TB > &, char lflag=2) const
- bool **sameSize1** (const size_t *s, unsigned n) const
- template<class TB >
bool **hasSameSize** (unsigned ia, const **wbarray**< TB > &B, unsigned ib, char lflag=1) const
- char **sameUptoFac** (const **wbarray** &B, T *fac=NULL, double eps=1e-12) const
- bool **sameAs** (const **wbarray** &B, double eps=1e-12) const
- bool **operator==** (const **wbarray** &B) const
- bool **operator!=** (const **wbarray** &B) const
- **T normDiff2** (const **wbarray** &B, char sflag=0) const
- **T normDiff** (const **wbarray** &B, char sflag=0) const
- **T normDiff2** (const **wbvector**< T > &B) const
- **T normDiff** (const **wbvector**< T > &B) const
- void **TimesEI** (const **wbarray** &, char conj=0)
- **wbarray & timesEI** (const **wbarray** &B, **wbarray** &C, T bfac=1, T cfac=0, char conj=0) const
- **wbarray & timesEI_OM** (unsigned r, const **wbarray** &B, **wbarray** &C, char conj=0) const
- **T dotProd** (const **wbarray** &, char conj=0) const
- **T weightedAvg** (const **wbarray** &) const
- void **TensorProd** (const **wbarray** &B, const char aflag='N', const char bflag='N', const char kflag=0)
- **wbarray & tensorProd** (const **wbarray** &, **wbarray** &, const char aflag='N', const char bflag='N', const char kflag=0) const
- **wbarray & kron** (const **wbarray** &B, **wbarray** &X, char aflag='N', char bflag='N') const
- **wbarray & Kron** (const **wbarray** &B, char aflag='N', char bflag='N')
- **wbarray & Cat** (unsigned dim, const **wbvector**< **wbarray** * > &ap)
- **wbarray & Cat** (unsigned dim, const **wbvector**< **wbarray** > &ap)
- **wbarray & BlockCat** (const **wbarray**< **wbarray** > &aa, WBINDEX *D1=NULL, WBINDEX *D2=NULL)
- **wbarray & BlockCat** (const **wbarray**< const **wbarray** * > &ap, WBINDEX *D1=NULL, WBINDEX *D2=NULL)
- void **operator+=** (T x)
- void **operator-=** (T x)
- template<class T_ >
wbarray & Times (const T_ a, char rcpy=0)
- template<class T_ >
wbarray & times (const T_ a, **wbarray**< T > &X) const
- template<class T_ >
wbarray & operator*= (const T_ a)
- template<class T_ >
wbarray & operator/= (const T_ &a)
- void **operator+=** (const **wbarray** &B)
- void **operator-=** (const **wbarray** &B)
- **wbarray & plus** (const **wbarray** &B, **wbarray** &C, T bfac=1, char iflag=0) const
- template<class TB >
wbarray & Plus (const **wbarray**< TB > &B, TB bfac=TB(1), char iflag=0, T afac=T(1))
- bool **equal** (const **wbarray** &, T eps) const
- bool **unequal** (const **wbarray** &, T eps) const

- bool **equal** (const wbarry &, double eps, double &maxdiff) const
- bool **unequal** (const wbarry &, double eps, double &maxdiff) const
- bool **allEqual** (const T &x)
- bool **allLE** (const T &x)
- char **isVector** () const
- unsigned **isScalar** () const
- bool **requiresDataPerm** (const wbarry &P, char lflag=3) const
- void **markUnequalZero** (const T m=1, const T eps=0)
- void **toMatrixRef** (wbarry &A, const UVEC &I, int pos, wbarry &P) const
- bool **toMatrixRef** (const char *F, int L, wbarry &A, const UVEC &I, int pos, char &tflag) const
- void **toMatrixRef** (wbarry &A, unsigned K, const wbarry &P0) const
- void **toMatrixRefM** (wbarry &A, unsigned K, const wbarry &P0, unsigned R, wbarry pfin=wbarry()) const
- void **toMatrixRef** (wbarry &A, unsigned i, int pos, wbarry &P) const
- void **toMatrixRef** (wbarry &A, const char *sidx, int pos, wbarry &P) const
- wbarry & **Reshape** (const wbarry < size_t > &, bool lflag=0)
- wbarry & **Reshape** (size_t s1, size_t s2, bool lflag=0)
- wbarry & **Reshape** (size_t s1, size_t s2, size_t s3, bool lflag=0)
- wbarry < T > & **Reshape1** (const size_t *s, unsigned n)
- wbarry & **Reshape1** (size_t s1, size_t s2)
- wbarry & **Reshape1** (size_t s1, size_t s2, size_t s3)
- void **GroupIndices** (size_t K)
- wbarry & **groupIndices_DREF** (wbarry &A, unsigned k1, unsigned k2) const
- void **GroupIndices** (unsigned k1, unsigned k2)
- void **groupIndices_P** (const WBINDEX &, int, wbarry &, size_t *=NULL, size_t *=NULL) const
- wbarry & **transpose** (const char *F, int L, wbarry &A) const
- wbarry & **Transpose** (const char *F=0, int L=0)
- wbarry & **flipLR** (const char *F, int L, wbarry &A) const
- wbarry & **FlipLR** (const char *F=0, int L=0)
- wbarry & **flipUD** (const char *F, int L, wbarry &A) const
- wbarry & **FlipUD** (const char *F=0, int L=0)
- wbarry & **MatPermute** (const wbarry &P, char iflag=0)
- wbarry & **Permute** (const char *s, char iflag=0, char rcpy=0)
- wbarry & **Permute** (const wbarry &, char iflag=0, char rcpy=0)
- wbarry & **permute** (wbarry &, const char *s, char iflag=0) const
- wbarry & **permute** (wbarry &, const wbarry &, char iflag=0) const
- wbarry & **select0** (const WPERM &, unsigned dim, wbarry &) const
- wbarry & **Select0** (const UVEC &I, unsigned dim)
- wbarry & **select0** (size_t i1, size_t i2, unsigned dim, wbarry &Q) const
- wbarry & **selectSqueeze** (wbarry &A, unsigned p, unsigned dim) const
- T **min** () const
- T **max** () const
- T **aMin** (char zflag=0, size_t *k=NULL) const
- void **info** (const char *="") const
- void **info** (const char *F, int L, const char *="", unsigned k=-1, unsigned nlt=0, unsigned nlb=0) const
- void **print** (const char *F, int L, const char *istr="", const char *fmt="") const
- void **print** (const char *istr="", const char *fmt="") const
- int **printdata** (const char *istr="ans", const char *fmt="") const
- void **print_ref** (const char *bstr=" ***", const char *nl="\n") const
- wbarry & **toStr** () const
- void **contractDiag** (unsigned ic, const wbarry &B, T a, T b, wbarry &C, char lflag=0, char bflag='N') const
- template<class TB, class TC>
wbarry < TC > & **contractMat** (const char *F, int L, unsigned i1, const wbarry < TB > &M, unsigned i2, wbarry < TC > &) const

- `template<class TB, class TC>`
`wbarray< TC> & contractMat` (const char *F, int L, unsigned i1, const `wbarray< TB> &M`, `wbarray< TC> &C`) const
- `wbarray & ContractMat` (const char *F, int L, unsigned i1, const `wbarray &M`, unsigned i2=1)
- `template<class TB, class TC>`
`wbarray< TC> & contract` (const char *F, int L, const `ctrlIdx &`, const `wbarray< TB> &B`, const `ctrlIdx &`, `wbarray< TC> &C`, const `wbperm &pfinal=wbperm()`, T afac=1, TC cfac=1) const
- `template<class TB, class TC>`
`wbarray< TC> & contract` (const char *F, int L, const `iTags &ta`, const `wbarray< TB> &B`, const `iTags &tb`, `wbarray< TC> &C`, `iTags &tc`, const `wbperm &pfinal=wbperm()`, T afac=1, TC cfac=1) const
- `template<class TB>`
`wbarray< T> & Contract` (const `wbvector< unsigned> &`, const `wbarray< TB> &`, const `wbvector< unsigned> &`, const `wbperm &pfinal=wbperm()`)
- `template<class TB>`
`wbarray< T> & Contract` (char *, const `wbarray< TB> &`, char *, const `wbperm &pfinal=wbperm()`)
- `template<class TB, class TC>`
`wbarray< TC> & contract` (const char *F, const int L, const char *, const `wbarray< TB> &`, const char *, `wbarray< TC> &`, const `wbperm &pfinal=wbperm()`) const
- `template<class TB, class TC>`
`wbarray< TC> & contract` (char *i1, const `wbarray< TB> &B`, char *i2, `wbarray< TC> &Q`, const `wbperm &pfinal=wbperm()`) const
- `template<class TB, class TC>`
`wbarray< TC> & contract` (const `wbvector< unsigned> &i1`, const `wbarray< TB> &B0`, const `wbvector< unsigned> &i2`, `wbarray< TC> &Q0`, const `wbperm &P`) const
- `template<class TB, class TC>`
`wbarray< TC> & contract` (const char *F, int L, unsigned i1, const `wbarray< TB> &B`, unsigned i2, `wbarray< TC> &C`, const `wbperm &pfinal=wbperm()`) const
- `template<class TB, class TC>`
`wbarray< TC> & contract` (unsigned i1, const `wbarray< TB> &B`, unsigned i2, `wbarray< TC> &C`, const `wbperm &pfinal=wbperm()`) const
- `template<class TB, class TC>`
`wbarray< TC> & comm` (const `wbarray< TB> &`, `wbarray< TC> &`, char aflag='N', char bflag='N') const
- `template<class TB, class TC>`
`wbarray< TC> & acomm` (const `wbarray< TB> &`, `wbarray< TC> &`, char aflag='N', char bflag='N') const
- `wbarray operator+` (const `wbarray &B`) const
- `wbarray operator-` (const `wbarray &B`) const
- `wbarray operator*` (const `wbarray &B`) const
- `T trace` () const
- `wbarray & trace` (unsigned i1, unsigned i2, `wbarray &C`) const
- `wbarray & trace` (const `ctrlIdx &l1`, const `ctrlIdx &l2`, `wbarray< T> &C`) const
- `mxArray * toMx` () const
- `mxArray * toMx_Struct` () const
- `mxArray * toMx_base` () const
- `mxArray * mxCreateStruct` (unsigned m, unsigned n) const
- void `add2MxStruct` (mxArray *S, unsigned i, char tflag=0) const
- void `put` (const char *vname="ans", const char *ws="base") const
- void `put` (const char *F, int L, const char *vname="ans", const char *ws="base") const
- void `getReal` (`wbarray< double> &R`) const
- void `getImag` (`wbarray< double> &I`) const
- void `Conj` ()
- void `set` (const `wbarray< double> &R`, const `wbarray< double> &I`)
- const `wbarray & opA` (char tflag, `wbarray &aux`)
- bool `isSym_aux` (const char *file, int line, const char *fct, const `wbarray &B`, double eps, double *xref, const char symflag='s', const char lflag=1) const
- double `SkipTiny` (double eps)
- `wbarray< double> & SkipTiny_float` (double ref)

- `wbarray< wbcomplex > & SkipTiny_float (wbcomplex ref)`
- `double SkipTiny_imag (double eps)`
- `template<class TB >`
`wbarray< T > & Plus (const wbarray< TB > &B, TB bfac, char iflag, T afac)`
- `bool isComplex () const`
- `int printdata (const char *istr, const char *fmt0) const`
- `template<class TB, class TC >`
`wbarray< TC > & contract (const char *F, const int L, const ctrldx &ica, const wbarray< TB > &B, const ctrldx &icb, wbarray< TC > &C0, const wbperm &P, TA afac, TC cfac) const`
- `void getReal (wbarray< double > &R) const`
- `void getImag (wbarray< double > &I) const`
- `void getImag (wbarray< double > &I) const`
- `void Conj ()`
- `template<class T >`
`void set (const wbarray< double > &R QS_UNUSED_VAR, const wbarray< double > &I QS_UNUSED_↵VAR)`
- `void set (const wbarray< double > &R_, const wbarray< double > &I_)`
- `mxAarray * toMx () const`
- `mxAarray * toMx () const`
- `mxAarray * toMx () const`
- `mxAarray * toMx () const`
- `mxAarray * toMx () const`
- `mxAarray * toMx () const`
- `wbcomplex norm2 () const`
- `wbcomplex scalarProd (const wbarray< wbcomplex > &B) const`

Public Attributes

- `wbvector< size_t > SIZE`
- `Wb::MTYPE mtype`
- `Wb::sptr< T > * sptr`
- `T * data`

Protected Member Functions

- `bool isDiag_aux (const T eps, const char *task) const`
- `bool isDiag_aux (double *eps, const char *task) const`
- `size_t serial_index (const widx_t *l, size_t n) const`

The documentation for this class was generated from the following files:

- `wbarray.hh`
- `wbarray.cc`

4.97 `wbarrRef< T >` Class Template Reference

Public Member Functions

- `wbarrRef (const wbarrRef< T > &r)`
- `wbarrRef< T > & operator= (const wbarrRef< T > &r)`
- `void print (const char *vname="ans")`

Public Attributes

- double **dc**
- double **fac**
- bool **tflag**
- [wbarray](#)< T > * **D**

The documentation for this class was generated from the following file:

- [wbarray.hh](#)

4.98 wbcomplex Class Reference

Public Member Functions

- **wbcomplex** (const [wbcomplex](#) &z)=default
- **wbcomplex** (double r0, double i0=0.)
- [wbcomplex](#) & **operator=** (const [wbcomplex](#) &z)=default
- [wbcomplex](#) & **operator=** (const double &dbl)
- **operator bool** () const
- bool **operator!** () const
- **operator double** () const
- [wbcomplex](#) & **init** (double r0, double i0)
- [wbcomplex](#) & **set** (double r0, double i0)
- [wbcomplex](#) & **setRand** (double fac=2., double shift=-1.)
- [wbcomplex](#) & **init_expi** (double ph)
- mxComplexDouble & **explicit_copy2** (mxComplexDouble &zm) const
- bool **isinf** () const
- bool **isnan** () const
- bool **isfinite** () const
- bool **isnormal** () const
- bool **operator==** (const [wbcomplex](#) &z) const
- bool **operator!=** (const [wbcomplex](#) &z) const
- bool **operator==** (const double &dbl) const
- bool **operator!=** (const double &dbl) const
- bool **operator>** (const [wbcomplex](#) &z) const
- bool **operator>=** (const [wbcomplex](#) &z) const
- bool **operator<** (const [wbcomplex](#) &z) const
- bool **operator<=** (const [wbcomplex](#) &z) const
- [wbcomplex](#) & **operator+=** (const [wbcomplex](#) &z)
- [wbcomplex](#) & **operator-=** (const [wbcomplex](#) &z)
- [wbcomplex](#) & **operator*=** (const [wbcomplex](#) &z)
- [wbcomplex](#) & **operator/=** (const [wbcomplex](#) &z)
- [wbcomplex](#) & **operator*=** (double dbl)
- [wbcomplex](#) & **operator+=** (double dbl)
- [wbcomplex](#) & **operator-=** (double dbl)
- [wbcomplex](#) **operator+** (const [wbcomplex](#) &z) const
- [wbcomplex](#) **operator-** (const [wbcomplex](#) &z) const
- [wbcomplex](#) **operator*** (const [wbcomplex](#) &z) const
- [wbcomplex](#) **operator/** (const [wbcomplex](#) &z) const
- [wbcomplex](#) **operator+** (const double &dbl) const
- [wbcomplex](#) **operator-** (const double &dbl) const

- `wbcomplex operator*` (const double &dbl) const
- `wbcomplex operator/` (const double &dbl) const
- `wbcomplex operator/=` (double dbl)
- `wbcomplex times` (const `wbcomplex` &) const
- `wbcomplex ConjProd` (const `wbcomplex` &z) const
- void `Conj` ()
- `wbcomplex conj` () const
- double `abs2` () const
- double `abs` () const
- `wbcomplex sqrt` () const
- `wbcomplex sign` () const
- double `arg` () const
- double `arg_degree` ()
- `wbcomplex round` () const
- `wbstring toStr` (const char *fmt="%.4g") const
- void `put` (const char *vname="ans", const char *ws="base") const

Public Attributes

- double `r`
- double `i`

The documentation for this class was generated from the following files:

- `wbcomplex.hh`
- `wbcomplex.cc`

4.99 `wbdebug_dummy` Class Reference

The documentation for this class was generated from the following file:

- `memtrack.cc`

4.100 `wbhst` Class Reference

Public Member Functions

- `wbhst` (const mxArray *a)
- `wbhst & init` (const mxArray *a)
- `wbhst & add` (const char *F, int L, const mxArray *ax, const mxArray *ay)

Public Attributes

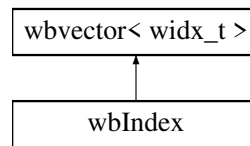
- long unsigned `ndat`
- long unsigned `ncalls`
- `wbvector`< double > `xbdr`
- `wbvector`< double > `xbin`
- `wbvector`< double > `ybin`

The documentation for this class was generated from the following file:

- `wbhst.cc`

4.101 wbIndex Class Reference

Inheritance diagram for wbIndex:



Public Member Functions

- **wbIndex** (const [wbvector](#)< widx_t > &S)
- template<class T1 >
 wbIndex (widx_t n, const T1 *S)
- **wbIndex** (const [wbIndex](#) &l)
- [wbIndex](#) & **init** ()
- [wbIndex](#) & **init** (const [wbvector](#)< widx_t > &S, bool pp=1)
- template<class T1 >
 [wbIndex](#) & **init** (unsigned n, const T1 *S, bool pp=1)
- [wbIndex](#) & **reset** (bool pp=1)
- [wbIndex](#) & **set** ([wbvector](#)< widx_t > &l)
- bool **isValid** () const
- void **checkValid** (const char *F, int L) const
- bool **operator++** ()
- bool **operator--** ()
- bool **isEmpty** () const
- bool **gotmore** () const
- bool **isdiag** () const
- widx_t **numel** () const
- widx_t **serial** () const
- widx_t **serial** (const widx_t *stride) const
- [wbstring](#) **sizeStr** () const
- [wbstring](#) **toStr** () const
- mxArray * **toMx** () const

Public Member Functions inherited from [wbvector](#)< [widx_t](#) >

- **wbvector** (size_t l, widx_t x)
- **wbvector** (size_t l, const widx_t *d0=NULL, char ref=0)
- **wbvector** (const [wbvector](#) &v)
- **wbvector** (const [wbvector](#)< T2 > &v)
- **wbvector** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- **wbvector** (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- **wbvector** (const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) & **init** (size_t l=0)
- [wbvector](#) & **init** (size_t l, const widx_t *d0)
- [wbvector](#) & **init** (size_t l, const widx_t *d0, char ref)
- [wbvector](#) & **init** (const [wbvector](#) &v)
- [wbvector](#) & **init** (size_t n, const [wbvector](#) &v)
- [wbvector](#)< widx_t > & **init** (const [wbsparray](#)< widx_t > &S)
- [wbvector](#) & **init** (size_t l1, const widx_t *d1, size_t l2, const widx_t *d2)

- `int init (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type QS_UNUSED_↵, VAR=1, char ref=0)`
- `int init (const mxArray *a, const char *istr=NULL, char check_type=1)`
- `int init (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)`
- `int init (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)`
- `int init (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)`
- `int init (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)`
- `int init (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)`
- `int init (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)`
- `wbvector & initp (size_t l, const widx_t *d0, size_t n, char init=1)`
- `wbvector & init2val (size_t l, const widx_t &x)`
- `wbvector & initStride (size_t l, const widx_t *d0, size_t stride)`
- `wbvector & init_stride (const widx_t *d0, size_t n, size_t m, size_t D0)`
- `wbvector & init_data (const widx_t *d0)`
- `wbvector & init_data (const T0 *d0)`
- `wbvector & init_data (const char *F, int L, const T0 *d0)`
- `wbvector< widx_t > & initT (const wbsparray< T2 > &S)`
- `wbvector< widx_t > & initT (size_t l, const T2 *v)`
- `wbvector< widx_t > & initT (const wbvector< T2 > &v)`
- `wbvector< widx_t > & initT (const char *F, int L, const wbvector< T2 > &v)`
- `wbvector< widx_t > & initT (const wbsparray< TD > &S)`
- `wbvector< widx_t > & initT_ (const wbvector< T2 > &v)`
- `wbvector & init2ref (const wbvector &v)`
- `wbvector & init2ref (size_t l, const widx_t *d0)`
- `wbvector & unRef ()`
- `int init_Struct (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)`
- `int init_base (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)`
- `int init_mpf (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)`
- `void initDef (size_t n)`
- `void initDef (const wbvector &v)`
- `wbvector & operator= (const wbvector &v)`
- `wbvector< widx_t > & operator= (const wbvector< TB > &v)`
- `wbvector & set (const widx_t &x)`
- `wbvector & set (const wbvector< size_t > &l, const widx_t &x)`
- `void set (size_t offset, const widx_t *x, size_t n)`
- `void set (const wbvector< double > &R, const wbvector< double > &l)`
- `void set (const wbvector< double > &R, const wbvector< double > &l)`
- `wbvector & setMin (const widx_t &x)`
- `wbvector & setMax (const widx_t &x)`
- `wbvector & add (const wbvector< size_t > &l, const widx_t &x)`
- `wbvector & add (const widx_t *d)`
- `wbvector & Set (const wbvector< size_t > &l, const wbvector &v)`
- `void Sort (wbperm &p, char dir=+1)`
- `void Sort (char dir=+1)`
- `void sort (wbvector &v2, wbperm &p) const`
- `void sort (wbvector &v2) const`
- `wbperm & pSort (wbperm &p) const`
- `bool isSorted () const`
- `bool isSortedU () const`
- `bool isUnique () const`
- `int contains (const widx_t &x) const`
- `bool isNormal () const`
- `bool isNormal () const`
- `bool isNormal () const`
- `bool isNormal () const`

- `bool isNormal () const`
- `bool isNormal () const`
- `bool isNormal () const`
- `bool isNormal () const`
- `bool isFinite () const`
- `size_t find1 (const widx_t &x) const`
- `size_t find1 () const`
- `size_t count (const widx_t &x) const`
- `size_t icount (const widx_t &x) const`
- `size_t nnz () const`
- `size_t nnz (widx_t eps) const`
- `size_t numZeros (widx_t eps=widx_t(1E-14)) const`
- `size_t numEQ (const widx_t &) const`
- `size_t numGT (const widx_t &) const`
- `size_t numLT (const widx_t &) const`
- `wbindex & find (const widx_t &x, wbindex &l, char iflag=0) const`
- `wbindex & find (const widx_t &x, wbindex &l, wbindex &lx) const`
- `wbindex & find (wbindex &l) const`
- `wbindex & find (wbindex &l, wbindex &lx) const`
- `wbvector< size_t > find () const`
- `wbindex & findGT (const widx_t &x, wbindex &l, char iflag=0) const`
- `wbindex & findGT (const widx_t &x, wbindex &l, wbindex &lx) const`
- `wbindex & findzeros (wbindex &l) const`
- `wbindex & findRange (const widx_t &x1, const widx_t &x2, wbindex &l, const char *w=NULL) const`
- `int findClosestSorted (widx_t r) const`
- `int findClosest (widx_t r) const`
- `size_t findValues (const wbvector &v, wbindex &l) const`
- `wbvector & setp (const widx_t *x)`
- `void reset ()`
- `void cpy (const widx_t *v)`
- `void cpy (const widx_t *x, size_t m)`
- `wbvector & save2 (wbvector &v)`
- `void swap (wbvector &v)`
- `wbvector & swap (size_t i, size_t j)`
- `wbvector & Collect (widx_t &A1, widx_t &A2)`
- `wbvector & Collect (widx_t &A1, widx_t &A2, widx_t &A3)`
- `wbvector & Skip (size_t)`
- `wbvector & Skip (const WBINDEX &, char rflag=0)`
- `wbvector & Move2_FE (const WBINDEX &l, char FE_flag)`
- `wbvector & Move2front (const WBINDEX &l)`
- `wbvector & Move2front (size_t k)`
- `wbvector & Move2end (const WBINDEX &l)`
- `wbvector & Move2end (size_t k)`
- `wbvector & mvLastTo (size_t k)`
- `wbvector & mvFirstTo (size_t k)`
- `bool checkSameLength (const wbvector &a, const char *F=NULL, int L=0, const char *istr="") const`
- `bool checkSameLength (const wbvector &a, const wbvector &b, const char *F=NULL, int L=0, const char *istr="") const`
- `void operator+= (const T_ &a)`
- `void operator+= (const wbvector &b)`
- `void operator-= (const T_ &a)`
- `void operator-= (const wbvector &b)`
- `wbvector & operator*= (const T_ &a)`
- `wbvector & operator/= (const T_ &a)`
- `wbvector operator* (const T_ &a) const`

- [wbvector](#) **operator+** (const T_ &a) const
- [wbvector](#) **operator+** (const [wbvector](#) &b) const
- [wbvector](#) **operator-** (const T_ &a) const
- [wbvector](#) **operator-** (const [wbvector](#) &b) const
- [wbvector](#) & **Plus** (const widx_t *b)
- [wbvector](#) & **Plus** (const widx_t *b, widx_t bfac)
- [wbvector](#) & **Plus** (const [wbvector](#) &b, widx_t bfac=1)
- void **plus** (const [wbvector](#) &b, [wbvector](#) &c, widx_t bfac=1, widx_t cfac=0) const
- [wbvector](#) & **times** (widx_t fac, [wbvector](#) &x) const
- [wbvector](#) & **RevertSigns** ()
- int **isum** () const
- widx_t **sum** () const
- widx_t **sum** (size_t i1, const char *estr) const
- widx_t **sum** (size_t i1, size_t i2) const
- widx_t **cumsum_** ([wbvector](#) &S, char xflag=0) const
- widx_t **Cumsum** (char zflag=0) const
- widx_t **cumsum0prod** ([wbvector](#)< T2 > &b, [wbvector](#)< widx_t > &cs, char xflag) const
- [wbvector](#) & **Eldiv** (const [wbvector](#) &v) const
- [wbvector](#) & **tensorProd** (const [wbvector](#) &v2, [wbvector](#) &vv) const
- widx_t **scalarProd** (const [wbvector](#) &v) const
- widx_t **scalarProd** (const widx_t *v) const
- widx_t **norm** () const
- widx_t **norm2** () const
- double **normDiff** (const [wbvector](#) &v, widx_t fac=1) const
- widx_t **normDiff2** (const [wbvector](#) &v, widx_t fac=1) const
- double **normDiff2_** (const [wbvector](#) &v, double fac=1) const
- widx_t **GSProject** (const widx_t *v, char isnorm=0, char tflag=0) const
- [wbvector](#) & **Normalize** (const char *F=NULL, int L=0)
- [wbvector](#) & **subtract** (const widx_t *d)
- void **getReal** ([wbvector](#)< double > &R) const
- void **getReal** ([wbvector](#)< double > &R) const
- void **getImag** ([wbvector](#)< double > &I) const
- void **getImag** ([wbvector](#)< double > &I) const
- [wbvector](#) & **Conj** ()
- [wbvector](#)< [wbcomplex](#) > & **Conj** ()
- [wbvector](#) & **applyFlag** (char &flag, const widx_t &fac, [wbvector](#) &a)
- bool **allEqual** () const
- bool **allEqual** (const widx_t &a) const
- bool **allZero** () const
- bool **allOnes** () const
- bool **allEqual2** (const widx_t &a, widx_t eps=1E-15) const
- bool **allUnequal** (const widx_t &a) const
- bool **any** () const
- bool **anyUnequal** (const widx_t &a) const
- bool **anyUnequal** (const [wbvector](#)< size_t > &l, const widx_t &x, size_t *k=NULL) const
- bool **anyEqual** (const widx_t &a, size_t *i1=NULL) const
- bool **anyEqual** (const [wbvector](#)< size_t > &l, const widx_t &x, size_t *k=NULL) const
- bool **anyGT** (const widx_t &a) const
- bool **anyGE** (const widx_t &a) const
- bool **anyLT** (const widx_t &a) const
- bool **anyLE** (const widx_t &a) const
- bool **allGT** (const widx_t &a) const
- bool **allGE** (const widx_t &a) const
- bool **allLE** (const widx_t &a) const
- bool **allLT** (const widx_t &a) const

- bool **allIn** (const widx_t &a1, const widx_t &a2) const
- bool **anySmallVals** (const widx_t eps=1E-15) const
- bool **isEqual** (const widx_t *v) const
- bool **isEqual** (const [wbvector](#) &v) const
- bool **isEmpty** () const
- **operator bool** () const
- bool **operator!** () const
- bool **operator==** (const [wbvector](#) &v) const
- bool **operator!=** (const [wbvector](#) &v) const
- bool **isEqualR** (const [wbvector](#) &v) const
- bool **operator^** (const [wbvector](#) &v) const
- bool **deepEqualP** (const [wbvector](#) &B) const
- char **cmp** (const [wbvector](#) &v) const
- bool **operator<** (const [wbvector](#) &v) const
- int **operator<** (const widx_t &r) const
- bool **operator<=** (const [wbvector](#) &v) const
- bool **operator>** (const [wbvector](#) &v) const
- bool **operator>=** (const [wbvector](#) &v) const
- widx_t & **el** (size_t i) const
- widx_t **elx** (size_t i, const widx_t &x) const
- const widx_t & **operator[]** (size_t i) const
- widx_t & **operator[]** (size_t i)
- [wbvector](#) **operator[]** (const [wbvector](#)< size_t > &l) const
- widx_t & **last** () const
- widx_t & **last** (size_t &i) const
- widx_t & **last** (T1 &i_) const
- widx_t & **p2last** () const
- widx_t * **ref** (size_t i) const
- [wbvector](#) & **setRand** (double fac=1., double shift=0.)
- [wbvector](#)< [wbcomplex](#) > & **setRand** (double fac, double shift)
- void **setIdx** (double shift=0., double fac=1.)
- int **set2Group** (const [wbperm](#) &P, const WBINDEX &D, const widx_t *S0=NULL, char iflag=0)
- [wbvector](#) & **Resize_** (size_t n)
- [wbvector](#) & **Shorten2** (size_t n)
- [wbvector](#) & **Resize** (size_t n, const widx_t *d=NULL)
- [wbvector](#) & **resize** (size_t n, [wbvector](#) &X) const
- [wbvector](#) & **cat** (const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) > &vv)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) * > &vv)
- [wbvector](#) & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)
- [wbvector](#) & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3=0, const [wbvector](#)< T3 > *v4=0)
- [wbvector](#)< widx_t > & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3, const [wbvector](#)< T3 > *v4)
- [wbvector](#)< widx_t > & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)
- [wbvector](#) & **Append** (const [wbvector](#) &v2)
- [wbvector](#) & **Append** (const widx_t &x)
- [wbvector](#) & **Append** (size_t d2, const widx_t *x)
- [wbvector](#) **append** (const [wbvector](#) &v) const
- [wbvector](#) & **append** (const [wbvector](#) &v2, [wbvector](#) &vout) const
- void **append** (const widx_t *x, size_t d2, [wbvector](#) &vout) const
- int **recLess** (const widx_t *v) const

- int **recLessE** (const widx_t *v) const
- int **recCompare** (const widx_t *v) const
- widx_t **max** () const
- widx_t **max** (size_t *k) const
- widx_t **max** (unsigned &k) const
- widx_t **max_** (widx_t x) const
- widx_t **min** () const
- widx_t **min** (size_t *k) const
- widx_t **min** (unsigned &k) const
- widx_t **min_** (widx_t x) const
- widx_t **aMax** (size_t *k=NULL) const
- widx_t **aMax** (unsigned &k) const
- widx_t **aMin** (char zflag=0, size_t *k=NULL) const
- widx_t **maxneg** (size_t *k=NULL) const
- widx_t **minpos** (size_t *k=NULL) const
- widx_t **avg** () const
- widx_t **std** () const
- void **range** (widx_t &xmin, widx_t &xmax) const
- widx_t **range** () const
- **wbvector** & **Permute** (const **wbperm** &P, char iflag=0)
- **wbvector** & **permute** (**wbvector** &, const **wbperm** &P, char iflag=0) const
- **wbvector** **permute** (const **wbperm** &P, char iflag=0) const
- **wbvector** & **blockPermute** (const **wbperm** &P, **wbvector** &, char iflag=0) const
- **wbvector** & **BlockPermute** (const **wbperm** &P, char iflag=0)
- int **blockCompare** (const **wbperm** &pa, const **wbvector** &b, const **wbperm** &pb) const
- void **get** (const WBINDEX &I0, **wbvector** &v) const
- void **get** (const **wbvector** < size_t > &I, widx_t *v) const
- void **get** (size_t i1, size_t i2, **wbvector** &v) const
- void **selectSU** (const WBINDEX &I)
- void **select** (const WBINDEX &I, widx_t *r) const
- **wbvector** & **select** (const WBINDEX &I, **wbvector** &a) const
- **wbvector** **select** (const WBINDEX &I) const
- **wbvector** & **Select** (const WBINDEX &I)
- **wbvector** & **BlockSelect** (const WBINDEX &I, const WBINDEX &D)
- **wbvector** & **getI** (size_t k, **wbvector** &v) const
- **wbvector** & **getI** (const **wbindex** &k, **wbvector** &v, char uflag=1) const
- **wbvector** & **getI** (const WBINDEX &k, **wbvector** &v, char uflag=1) const
- **wbvector** & **initI** (const **wbvector** &a, const **wbindex** &ia)
- **wbvector** & **initI** (const **wbvector** &a, const **wbindex** &ia, const **wbvector** &b, const **wbindex** &ib, char uflag=1)
- **wbvector** & **Flip** ()
- void **flip** (**wbvector** &a) const
- **wbvector** **flip** () const
- widx_t **prod** () const
- widx_t **prod** (widx_t x) const
- widx_t **prod** (size_t i, size_t i2) const
- widx_t **prod** (size_t i, size_t i2, widx_t x) const
- widx_t **prod** (const T1 *I, size_t n) const
- T2 **prod** (const T1 *I, size_t n, T2 &s2) const
- void **prod2** (widx_t &p1, widx_t &p2, const widx_t &p0=0) const
- void **info** (const char *istr) const
- void **print** (const char *istr, char mflag=0) const
- void **printdata** (const char *istr=0) const
- void **put** (const char *F, int L, const char *vname="ans", char tflag=0, const char *ws="base") const
- void **put** (const char *vname="ans", char tflag=0, const char *ws="base") const
- void **dispWithMsg** (const char *file, int line, const char *msg, const char *vname="") const

- void **mat2mx** (mxArray *&a, char dim=2) const
- void **mat2mxs** (mxArray *a, char field_nr, char dim=2) const
- mxArray * **toMx_Struct** (const char tflag=0) const
- mxArray * **toMxP_S** (const char tflag=0) const
- mxArray * **toMx** (const char tflag=0) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMxP** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base_d** (const char tflag=0) const
- mxArray * **toMx_offset** (const widx_t &offset, const char tflag=0) const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tflag=0) const
- void **add2MxStruct** (mxArray *S, const char *vname, char tflag=0) const
- mxArray * **mxCreateCell** (unsigned m, unsigned n) const
- void **add2MxCell** (mxArray *S, unsigned i, char tflag=0) const
- void **putg** (const char *vname="ans", char tflag=0) const
- void **putx** (const char *vname="ans", char tflag=0) const
- [wbstring](#) **toStr** (int n, const char *sep) const
- [wbstring](#) **toStr** () const
- [wbstring](#) **toStr** (int n, const char *sep) const
- [wbstring](#) **sizeStr** () const
- [wbstring](#) **toStrf** (const char *fmt0="", const char *sep=" ", unsigned stride=0, const char *sep2="") const
- [wbstring](#) **toStrf** (const char *fmt_, const char *sep, unsigned stride, const char *sep2) const

Public Attributes

- [wbvector](#)< widx_t > **SIZE**

Public Attributes inherited from [wbvector](#)< widx_t >

- widx_t * **data**
- size_t **len**
- char **isref**

Additional Inherited Members

Protected Member Functions inherited from [wbvector](#)< widx_t >

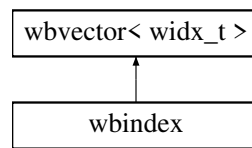
- [wbvector](#) & **RENEW** (size_t n, const widx_t *d=NULL, size_t l=-1, char init=1)
- [wbvector](#) & **RENEW_VAL** (size_t n, const widx_t &x)

The documentation for this class was generated from the following file:

- [wbindex.hh](#)

4.102 wbindex Class Reference

Inheritance diagram for wbindex:



Public Member Functions

- **wbindex** (widx_t n=0, char iflag=0)
- **wbindex** (const [ctrlidx](#) &ic)
- **wbindex** (widx_t n, const widx_t *d)
- **wbindex** (const char *s, unsigned offset=0)
- [wbindex](#) & **init** (widx_t n=0, const widx_t *d=NULL)
- [wbindex](#) & **init** (const [ctrlidx](#) &ic)
- [wbindex](#) & **init** (const char *F, int L, const mxArray *a, widx_t offset=0)
- widx_t **init** (const char *F, int L, const mxArray *a, const char *istr, char check_type=1)
- [wbindex](#) & **init** (const mxArray *a, widx_t offset=0)
- [wbindex](#) & **initGroup** (const [wbvector](#)< widx_t > &P, [wbvector](#)< widx_t > &D)
- [wbindex](#) & **initGroup0** (const [wbvector](#)< widx_t > &P, [wbvector](#)< widx_t > &D)
- void **initStr** (const char *s, widx_t offset=0)
- bool **isIndex** (widx_t n=0)
- [wbindex](#) & **Index** (widx_t n)
- [wbindex](#) & **Index_ex** (widx_t n, widx_t ix)
- [wbindex](#) & **Index** (widx_t i1, widx_t i2)
- [wbindex](#) & **Index** (widx_t i1, widx_t i2, int p)
- [wbindex](#) & **BlockIndex** (const [wbvector](#)< widx_t > &D, char sub=0)
- [wbindex](#) & **BlockIndex** (widx_t n, widx_t D)
- [wbindex](#) & **BlockIndexI** (widx_t n, widx_t D)
- [wbindex](#) & **IndexTranspose** (widx_t m, widx_t n)
- [wbindex](#) & **operator=** (const char *s)
- int **extend2Perm** (widx_t N, [wbperm](#) &P, char toend=0) const
- void **toPerm** ([wbperm](#) &P) const
- [wbindex](#) & **invert** (widx_t N, [wbindex](#) &I2, char uflag=1) const
- [wbindex](#) & **Invert** (widx_t N, char uflag=1)
- [wbindex](#) **flipIdx** (widx_t ndim) const
- char **isUnique** (widx_t imax, widx_t offset=0) const
- mxArray * **toMx** (const char tflag=0) const

Public Member Functions inherited from [wbvector](#)< widx_t >

- **wbvector** (size_t l, widx_t x)
- **wbvector** (size_t l, const widx_t *d0=NULL, char ref=0)
- **wbvector** (const [wbvector](#) &v)
- **wbvector** (const [wbvector](#)< T2 > &v)
- **wbvector** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- **wbvector** (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- **wbvector** (const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) & **init** (size_t l=0)

- [wbvector](#) & **init** (size_t l, const widx_t *d0)
- [wbvector](#) & **init** (size_t l, const widx_t *d0, char ref)
- [wbvector](#) & **init** (const [wbvector](#) &v)
- [wbvector](#) & **init** (size_t n, const [wbvector](#) &v)
- [wbvector](#)< widx_t > & **init** (const [wbsparray](#)< widx_t > &S)
- [wbvector](#) & **init** (size_t l1, const widx_t *d1, size_t l2, const widx_t *d2)
- int **init** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type QS_UNUSED_↵
VAR=1, char ref=0)
- int **init** (const mxArray *a, const char *istr=NULL, char check_type=1)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- [wbvector](#) & **initp** (size_t l, const widx_t *d0, size_t n, char init=1)
- [wbvector](#) & **init2val** (size_t l, const widx_t &x)
- [wbvector](#) & **initStride** (size_t l, const widx_t *d0, size_t stride)
- [wbvector](#) & **init_stride** (const widx_t *d0, size_t n, size_t m, size_t D0)
- [wbvector](#) & **init_data** (const widx_t *d0)
- [wbvector](#) & **init_data** (const T0 *d0)
- [wbvector](#) & **init_data** (const char *F, int L, const T0 *d0)
- [wbvector](#)< widx_t > & **initT** (const [wbsparray](#)< T2 > &S)
- [wbvector](#)< widx_t > & **initT** (size_t l, const T2 *v)
- [wbvector](#)< widx_t > & **initT** (const [wbvector](#)< T2 > &v)
- [wbvector](#)< widx_t > & **initT** (const char *F, int L, const [wbvector](#)< T2 > &v)
- [wbvector](#)< widx_t > & **initT** (const [wbsparray](#)< TD > &S)
- [wbvector](#)< widx_t > & **initT_** (const [wbvector](#)< T2 > &v)
- [wbvector](#) & **init2ref** (const [wbvector](#) &v)
- [wbvector](#) & **init2ref** (size_t l, const widx_t *d0)
- [wbvector](#) & **unRef** ()
- int **init_Struct** (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)
- int **init_base** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)
- int **init_mpfr** (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)
- void **initDef** (size_t n)
- void **initDef** (const [wbvector](#) &v)
- [wbvector](#) & **operator=** (const [wbvector](#) &v)
- [wbvector](#)< widx_t > & **operator=** (const [wbvector](#)< TB > &v)
- [wbvector](#) & **set** (const widx_t &x)
- [wbvector](#) & **set** (const [wbvector](#)< size_t > &l, const widx_t &x)
- void **set** (size_t offset, const widx_t *x, size_t n)
- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- [wbvector](#) & **setMin** (const widx_t &x)
- [wbvector](#) & **setMax** (const widx_t &x)
- [wbvector](#) & **add** (const [wbvector](#)< size_t > &l, const widx_t &x)
- [wbvector](#) & **add** (const widx_t *d)
- [wbvector](#) & **Set** (const [wbvector](#)< size_t > &l, const [wbvector](#) &v)
- void **Sort** ([wbperm](#) &p, char dir=+1)
- void **Sort** (char dir=+1)
- void **sort** ([wbvector](#) &v2, [wbperm](#) &p) const
- void **sort** ([wbvector](#) &v2) const
- [wbperm](#) & **pSort** ([wbperm](#) &p) const
- bool **isSorted** () const
- bool **isSortedU** () const

- bool **isUnique** () const
- int **contains** (const widx_t &x) const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isFinite** () const
- size_t **find1** (const widx_t &x) const
- size_t **find1** () const
- size_t **count** (const widx_t &x) const
- size_t **icount** (const widx_t &x) const
- size_t **nnz** () const
- size_t **nnz** (widx_t eps) const
- size_t **numZeros** (widx_t eps=widx_t(1E-14)) const
- size_t **numEQ** (const widx_t &) const
- size_t **numGT** (const widx_t &) const
- size_t **numLT** (const widx_t &) const
- **wbindex** & **find** (const widx_t &x, **wbindex** &l, char iflag=0) const
- **wbindex** & **find** (const widx_t &x, **wbindex** &l, **wbindex** &lx) const
- **wbindex** & **find** (**wbindex** &l) const
- **wbindex** & **find** (**wbindex** &l, **wbindex** &lx) const
- **wbvector**< size_t > **find** () const
- **wbindex** & **findGT** (const widx_t &x, **wbindex** &l, char iflag=0) const
- **wbindex** & **findGT** (const widx_t &x, **wbindex** &l, **wbindex** &lx) const
- **wbindex** & **findzeros** (**wbindex** &l) const
- **wbindex** & **findRange** (const widx_t &x1, const widx_t &x2, **wbindex** &l, const char *w=NULL) const
- int **findClosestSorted** (widx_t r) const
- int **findClosest** (widx_t r) const
- size_t **findValues** (const **wbvector** &v, **wbindex** &l) const
- **wbvector** & **setp** (const widx_t *x)
- void **reset** ()
- void **cpy** (const widx_t *v)
- void **cpy** (const widx_t *x, size_t m)
- **wbvector** & **save2** (**wbvector** &v)
- void **swap** (**wbvector** &v)
- **wbvector** & **swap** (size_t i, size_t j)
- **wbvector** & **Collect** (widx_t &A1, widx_t &A2)
- **wbvector** & **Collect** (widx_t &A1, widx_t &A2, widx_t &A3)
- **wbvector** & **Skip** (size_t)
- **wbvector** & **Skip** (const WBINDEX &, char rflag=0)
- **wbvector** & **Move2_FE** (const WBINDEX &l, char FE_flag)
- **wbvector** & **Move2front** (const WBINDEX &l)
- **wbvector** & **Move2front** (size_t k)
- **wbvector** & **Move2end** (const WBINDEX &l)
- **wbvector** & **Move2end** (size_t k)
- **wbvector** & **mvLastTo** (size_t k)
- **wbvector** & **mvFirstTo** (size_t k)
- bool **checkSameLength** (const **wbvector** &a, const char *F=NULL, int L=0, const char *istr="") const
- bool **checkSameLength** (const **wbvector** &a, const **wbvector** &b, const char *F=NULL, int L=0, const char *istr="") const
- void **operator+=** (const T_ &a)

- void **operator+=** (const [wbvector](#) &b)
- void **operator-=** (const T_ &a)
- void **operator-=** (const [wbvector](#) &b)
- [wbvector](#) & **operator*=** (const T_ &a)
- [wbvector](#) & **operator/=** (const T_ &a)
- [wbvector](#) **operator*** (const T_ &a) const
- [wbvector](#) **operator+** (const T_ &a) const
- [wbvector](#) **operator+** (const [wbvector](#) &b) const
- [wbvector](#) **operator-** (const T_ &a) const
- [wbvector](#) **operator-** (const [wbvector](#) &b) const
- [wbvector](#) & **Plus** (const widx_t *b)
- [wbvector](#) & **Plus** (const widx_t *b, widx_t bfac)
- [wbvector](#) & **Plus** (const [wbvector](#) &b, widx_t bfac=1)
- void **plus** (const [wbvector](#) &b, [wbvector](#) &c, widx_t bfac=1, widx_t cfac=0) const
- [wbvector](#) & **times** (widx_t fac, [wbvector](#) &x) const
- [wbvector](#) & **RevertSigns** ()
- int **isum** () const
- widx_t **sum** () const
- widx_t **sum** (size_t i1, const char *estr) const
- widx_t **sum** (size_t i1, size_t i2) const
- widx_t **cumsum_** ([wbvector](#) &S, char xflag=0) const
- widx_t **Cumsum** (char zflag=0) const
- widx_t **cumsum0prod** ([wbvector](#)< T2 > &b, [wbvector](#)< widx_t > &cs, char xflag) const
- [wbvector](#) & **Eldiv** (const [wbvector](#) &v) const
- [wbvector](#) & **tensorProd** (const [wbvector](#) &v2, [wbvector](#) &vv) const
- widx_t **scalarProd** (const [wbvector](#) &v) const
- widx_t **scalarProd** (const widx_t *v) const
- widx_t **norm** () const
- widx_t **norm2** () const
- double **normDiff** (const [wbvector](#) &v, widx_t fac=1) const
- widx_t **normDiff2** (const [wbvector](#) &v, widx_t fac=1) const
- double **normDiff2_** (const [wbvector](#) &v, double fac=1) const
- widx_t **GSProject** (const widx_t *v, char isnorm=0, char tflag=0) const
- [wbvector](#) & **Normalize** (const char *F=NULL, int L=0)
- [wbvector](#) & **subtract** (const widx_t *d)
- void **getReal** ([wbvector](#)< double > &R) const
- void **getReal** ([wbvector](#)< double > &R) const
- void **getImag** ([wbvector](#)< double > &I) const
- void **getImag** ([wbvector](#)< double > &I) const
- [wbvector](#) & **Conj** ()
- [wbvector](#)< [wbcomplex](#) > & **Conj** ()
- [wbvector](#) & **applyFlag** (char &flag, const widx_t &fac, [wbvector](#) &a)
- bool **allEqual** () const
- bool **allEqual** (const widx_t &a) const
- bool **allZero** () const
- bool **allOnes** () const
- bool **allEqual2** (const widx_t &a, widx_t eps=1E-15) const
- bool **allUnequal** (const widx_t &a) const
- bool **any** () const
- bool **anyUnequal** (const widx_t &a) const
- bool **anyUnequal** (const [wbvector](#)< size_t > &I, const widx_t &x, size_t *k=NULL) const
- bool **anyEqual** (const widx_t &a, size_t *i1=NULL) const
- bool **anyEqual** (const [wbvector](#)< size_t > &I, const widx_t &x, size_t *k=NULL) const
- bool **anyGT** (const widx_t &a) const
- bool **anyGE** (const widx_t &a) const

- bool **anyLT** (const widx_t &a) const
- bool **anyLE** (const widx_t &a) const
- bool **allGT** (const widx_t &a) const
- bool **allGE** (const widx_t &a) const
- bool **allLE** (const widx_t &a) const
- bool **allLT** (const widx_t &a) const
- bool **allIn** (const widx_t &a1, const widx_t &a2) const
- bool **anySmallVals** (const widx_t eps=1E-15) const
- bool **isEqual** (const widx_t *v) const
- bool **isEqual** (const [wbvector](#) &v) const
- bool **isEmpty** () const
- **operator bool** () const
- bool **operator!** () const
- bool **operator==** (const [wbvector](#) &v) const
- bool **operator!=** (const [wbvector](#) &v) const
- bool **isEqualR** (const [wbvector](#) &v) const
- bool **operator^** (const [wbvector](#) &v) const
- bool **deepEqualP** (const [wbvector](#) &B) const
- char **cmp** (const [wbvector](#) &v) const
- bool **operator<** (const [wbvector](#) &v) const
- int **operator<** (const widx_t &r) const
- bool **operator<=** (const [wbvector](#) &v) const
- bool **operator>** (const [wbvector](#) &v) const
- bool **operator>=** (const [wbvector](#) &v) const
- widx_t & **el** (size_t i) const
- widx_t **elx** (size_t i, const widx_t &x) const
- const widx_t & **operator[]** (size_t i) const
- widx_t & **operator[]** (size_t i)
- [wbvector](#) **operator[]** (const [wbvector](#)< size_t > &l) const
- widx_t & **last** () const
- widx_t & **last** (size_t &i) const
- widx_t & **last** (T1 &i_) const
- widx_t & **p2last** () const
- widx_t * **ref** (size_t i) const
- [wbvector](#) & **setRand** (double fac=1., double shift=0.)
- [wbvector](#)< [wbcomplex](#) > & **setRand** (double fac, double shift)
- void **setIdx** (double shift=0., double fac=1.)
- int **set2Group** (const [wbperm](#) &P, const WBINDEX &D, const widx_t *S0=NULL, char iflag=0)
- [wbvector](#) & **Resize_** (size_t n)
- [wbvector](#) & **Shorten2** (size_t n)
- [wbvector](#) & **Resize** (size_t n, const widx_t *d=NULL)
- [wbvector](#) & **resize** (size_t n, [wbvector](#) &X) const
- [wbvector](#) & **cat** (const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) > &vv)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) * > &vv)
- [wbvector](#) & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)
- [wbvector](#) & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3=0, const [wbvector](#)< T3 > *v4=0)
- [wbvector](#)< widx_t > & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3, const [wbvector](#)< T3 > *v4)
- [wbvector](#)< widx_t > & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)
- [wbvector](#) & **Append** (const [wbvector](#) &v2)

- [wbvector](#) & **Append** (const widx_t &x)
- [wbvector](#) & **Append** (size_t d2, const widx_t *x)
- [wbvector](#) **append** (const [wbvector](#) &v) const
- [wbvector](#) & **append** (const [wbvector](#) &v2, [wbvector](#) &vout) const
- void **append** (const widx_t *x, size_t d2, [wbvector](#) &vout) const
- int **recLess** (const widx_t *v) const
- int **recLessE** (const widx_t *v) const
- int **recCompare** (const widx_t *v) const
- widx_t **max** () const
- widx_t **max** (size_t *k) const
- widx_t **max** (unsigned &k) const
- widx_t **max_** (widx_t x) const
- widx_t **min** () const
- widx_t **min** (size_t *k) const
- widx_t **min** (unsigned &k) const
- widx_t **min_** (widx_t x) const
- widx_t **aMax** (size_t *k=NULL) const
- widx_t **aMax** (unsigned &k) const
- widx_t **aMin** (char zflag=0, size_t *k=NULL) const
- widx_t **maxneg** (size_t *k=NULL) const
- widx_t **minpos** (size_t *k=NULL) const
- widx_t **avg** () const
- widx_t **std** () const
- void **range** (widx_t &xmin, widx_t &xmax) const
- widx_t **range** () const
- [wbvector](#) & **Permute** (const [wbperm](#) &P, char iflag=0)
- [wbvector](#) & **permute** ([wbvector](#) &, const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) **permute** (const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) & **blockPermute** (const [wbperm](#) &P, [wbvector](#) &, char iflag=0) const
- [wbvector](#) & **BlockPermute** (const [wbperm](#) &P, char iflag=0)
- int **blockCompare** (const [wbperm](#) &pa, const [wbvector](#) &b, const [wbperm](#) &pb) const
- void **get** (const WBINDEX &I0, [wbvector](#) &v) const
- void **get** (const [wbvector](#)< size_t > &I, widx_t *v) const
- void **get** (size_t i1, size_t i2, [wbvector](#) &v) const
- void **selectSU** (const WBINDEX &I)
- void **select** (const WBINDEX &I, widx_t *r) const
- [wbvector](#) & **select** (const WBINDEX &I, [wbvector](#) &a) const
- [wbvector](#) **select** (const WBINDEX &I) const
- [wbvector](#) & **Select** (const WBINDEX &I)
- [wbvector](#) & **BlockSelect** (const WBINDEX &I, const WBINDEX &D)
- [wbvector](#) & **getI** (size_t k, [wbvector](#) &v) const
- [wbvector](#) & **getI** (const [wbindex](#) &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **getI** (const WBINDEX &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **initI** (const [wbvector](#) &a, const [wbindex](#) &ia)
- [wbvector](#) & **initI** (const [wbvector](#) &a, const [wbindex](#) &ia, const [wbvector](#) &b, const [wbindex](#) &ib, char uflag=1)
- [wbvector](#) & **Flip** ()
- void **flip** ([wbvector](#) &a) const
- [wbvector](#) **flip** () const
- widx_t **prod** () const
- widx_t **prod** (widx_t x) const
- widx_t **prod** (size_t i, size_t i2) const
- widx_t **prod** (size_t i, size_t i2, widx_t x) const
- widx_t **prod** (const T1 *I, size_t n) const
- T2 **prod** (const T1 *I, size_t n, T2 &s2) const
- void **prod2** (widx_t &p1, widx_t &p2, const widx_t &p0=0) const

- void **info** (const char *istr) const
- void **print** (const char *istr, char mflag=0) const
- void **printdata** (const char *istr=0) const
- void **put** (const char *F, int L, const char *vname="ans", char tflag=0, const char *ws="base") const
- void **put** (const char *vname="ans", char tflag=0, const char *ws="base") const
- void **dispWithMsg** (const char *file, int line, const char *msg, const char *vname="") const
- void **mat2mx** (mxArray *a, char dim=2) const
- void **mat2mxs** (mxArray *a, char field_nr, char dim=2) const
- mxArray * **toMx_Struct** (const char tflag=0) const
- mxArray * **toMxP_S** (const char tflag=0) const
- mxArray * **toMx** (const char tflag=0) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMxP** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base_d** (const char tflag=0) const
- mxArray * **toMx_offset** (const widx_t &offset, const char tflag=0) const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tflag=0) const
- void **add2MxStruct** (mxArray *S, const char *vname, char tflag=0) const
- mxArray * **mxCreateCell** (unsigned m, unsigned n) const
- void **add2MxCell** (mxArray *S, unsigned i, char tflag=0) const
- void **putg** (const char *vname="ans", char tflag=0) const
- void **putx** (const char *vname="ans", char tflag=0) const
- [wbstring](#) **toStr** (int n, const char *sep) const
- [wbstring](#) **toStr** () const
- [wbstring](#) **toStr** (int n, const char *sep) const
- [wbstring](#) **sizeStr** () const
- [wbstring](#) **toStrf** (const char *fmt0="", const char *sep=" ", unsigned stride=0, const char *sep2="") const
- [wbstring](#) **toStrf** (const char *fmt_, const char *sep, unsigned stride, const char *sep2) const

Additional Inherited Members

Public Attributes inherited from [wbvector< widx_t >](#)

- widx_t * **data**
- size_t **len**
- char **isref**

Protected Member Functions inherited from [wbvector< widx_t >](#)

- [wbvector](#) & **RENEW** (size_t n, const widx_t *d=NULL, size_t l=-1, char init=1)
- [wbvector](#) & **RENEW_VAL** (size_t n, const widx_t &x)

The documentation for this class was generated from the following files:

- wbindex.hh
- wbindex.cc

4.103 Wb::WbListMTRACK Class Reference

Public Member Functions

- template<class T >
void **rm_ptr** (const char *F, const int &L, T *&p)
- template<class T >
void **add_ptr** (const char *F, int L, T *&p, size_t l)
- template<class T >
bool **exists** (T *&p)
- template<class T >
void **tst_exists** (const char *F, int L, T *&p)
- void **checkTotSize** ()
- char * **totStr** (char mflag=0)
- void **printSize** (const char *F, int L, char mflag=0)

Public Attributes

- map< void *, [MREC](#) > **M**
- unsigned long long **idx**
- unsigned long long **totsize**
- unsigned long long **maxsize**

The documentation for this class was generated from the following file:

- memtrack.cc

4.104 wblogbuf_struct Class Reference

Public Member Functions

- **wblogbuf_struct** (const char *s)
- **wblogbuf_struct** (const char *F, int L, const char *s)
- **wblogbuf_struct** (const [wblogbuf_struct](#) &S)
- void **unset** ()

Public Attributes

- char * **file**
- int **line**
- char * **text**

The documentation for this class was generated from the following file:

- wbststring.hh

4.105 wblogbuffer Class Reference

Public Member Functions

- void **push** (const char *F, int L, const char *s)
- void **flush** ()

The documentation for this class was generated from the following file:

- wbststring.hh

4.106 wbMatrix< T > Class Template Reference

Public Member Functions

- **wbMatrix** (size_t r=0, size_t c=0)
- **wbMatrix** (size_t r, size_t c, T *d, const char ref=0)
- **wbMatrix** (const [wbMatrix](#) &M)
- template<class T2 >
 wbMatrix (const [wbMatrix](#)< T2 > &M)
- **wbMatrix** (const mxArray *a, char tflag=0, char ref=0)
- **wbMatrix** (const char *F, int L, const mxArray *a)
- [wbMatrix](#) & **init** (size_t r=0, size_t c=0, const T *d=NULL)
- [wbMatrix](#) & **init** (size_t r, size_t c, const T *d0, size_t stride)
- [wbMatrix](#) & **initIdentity** (size_t n)
- [wbMatrix](#) & **init2val** (size_t r, size_t c, const T &d)
- [wbMatrix](#) & **initDef** (size_t r=0, size_t c=0, T *d=NULL)
- template<class T2 >
 [wbMatrix](#)< T > & **initT** (size_t r, size_t c, T2 *d)
- template<class T2 >
 [wbMatrix](#)< T > & **initT** (const [wbMatrix](#)< T2 > &B)
- template<class T2 >
 [wbMatrix](#)< T > & **initT** (const char *F, int L, const [wbMatrix](#)< T2 > &B)
- template<class TA, class TB >
 [wbMatrix](#)< T > & **init** (const char *F, int L, const [wbMatrix](#)< TA > &B, const [wbvector](#)< TB > &V, char dim=2)
- template<class T2 >
 [wbMatrix](#)< T > & **initDim** (const [wbMatrix](#)< T2 > &B)
- [wbMatrix](#) & **init** (const [wbMatrix](#) &B)

- [wbMatrix](#) & **init** (const char *F, int L, const mxArray *a, char tflag=0, char ref=0, char tcheck=1)
- void **init** (const mxArray *a)
- void **init0** (const mxArray *a)
- void **initTST** ()
- [wbMatrix](#) & **init2ref** (size_t r, size_t c, const T *d)
- [wbMatrix](#) & **init2ref** (const [wbvector](#)< T > &v, const char tflag=0)
- [wbMatrix](#) & **init2ref** (const [wbMatrix](#) &M)
- [wbMatrix](#) & **unRef** ()
- void **no_ref_** (const char *F, int L, const char *fct)
- void **swap** ([wbMatrix](#) &B, char ref=0)
- [wbMatrix](#) & **save2** ([wbMatrix](#) &A, char ref=0)
- void **copyStride** (T *dd, size_t stride) const
- [wbMatrix](#) & **operator=** (const [wbMatrix](#) &B)
- template<class TB >
 [wbMatrix](#) & **operator=** (const [wbMatrix](#)< TB > &B)
- **operator bool** () const
- bool **operator!** () const
- bool **operator==** (const T &x) const
- bool **operator==** (const [wbMatrix](#) &B) const
- bool **operator!=** (const [wbMatrix](#) &B) const
- bool **deepEqualP** (const [wbMatrix](#) &B) const
- bool **anyEqual** (const T &x) const
- bool **allEqual** (const T &x) const
- bool **anyUnequal** (const T &x) const
- bool **anyLT** (const T &x) const
- bool **anyGT** (const T &x) const
- bool **col_anyGT** (size_t j, const T &x) const
- [wbMatrix](#) & **operator+=** (const [wbMatrix](#) &B)
- [wbMatrix](#) **operator+** (const [wbMatrix](#) &B) const
- [wbMatrix](#) **operator+** (const T &x) const
- void **operator-=** (const [wbMatrix](#) &B)
- [wbMatrix](#) **operator-** (const [wbMatrix](#) &B) const
- void **operator*=** (const T c)
- [wbMatrix](#) & **operator+=** (const T c)
- void **operator-=** (const T c)
- [wbMatrix](#) **operator*** (const T x) const
- [wbMatrix](#) & **add** (const [wbMatrix](#) &B, const T &c=1, const char zflag=0)
- [wbMatrix](#) & **minus** (const [wbMatrix](#) &B, const T &c=1, const char zflag=0)
- [wbMatrix](#) & **times** (T fac, [wbMatrix](#) &X) const
- [wbMatrix](#) & **rowTimes** (size_t i, T fac)
- [wbMatrix](#) & **colTimes** (size_t j, T fac)
- T **prod** () const
- T **min** () const
- T **max** () const
- T **aMax** () const
- T **aMax** (unsigned &i_, unsigned &j_) const
- T **sum** () const
- T **cumsum0prodRec** ([wbvector](#)< T > &cs, char xflag=0) const
- T **cumsum0prodRec** ([wbvector](#)< T > &cs, unsigned i1, unsigned i2, char xflag=0) const
- T **norm2** () const
- T **norm** () const
- double **normReal** () const
- double **normImag** () const
- T **norm2Cols** (long j1, long j2) const
- T **norm2Recs** (long i1, long i2) const

- [wbvector](#)< T > & **norm2Cols** ([wbvector](#)< T > &x2_) const
- [wbvector](#)< T > **norm2Cols** () const
- [wbvector](#)< T > **normCols** () const
- **size_t maxRec** (char lex=1, **size_t** *m=NULL) const
- **size_t maxRec_float** (char lex=1, **size_t** *m=NULL, T xref QS_UNUSED_VAR=-1) const
- T **recMax** (**size_t** r, **size_t** *k=NULL) const
- T **colMax** (**size_t** c, **size_t** *k=NULL) const
- T **colMax** (**size_t** c, unsigned &k) const
- [wbMatrix](#) & **NormalizeCol** (**size_t** k)
- bool **isOrthoRows** (T *x=NULL, char tnorm=0, T eps=1E-14) const
- bool **isOrthoRows** (const [wbMatrix](#) &B, T *x=NULL, char tnorm=0, T eps=1E-14) const
- double **recMaxA** (**size_t** r, **size_t** *k=NULL) const
- **size_t skipNanRecs** ([wbindex](#) &l)
- [wbMatrix](#) & **ResizeXRecs** (**size_t** r)
- [wbMatrix](#) & **Resize** (**size_t** r, **size_t** c, const T *dx=NULL)
- [wbMatrix](#) & **resize** (**size_t** m, **size_t** n, [wbMatrix](#) &M) const
- [wbMatrix](#) & **Resize2Mult** (const WBINDEX &M)
- [wbMatrix](#) & **transpose** ([wbMatrix](#) &M) const
- [wbMatrix](#) & **Transpose** ()
- [wbMatrix](#) & **Reshape** (**size_t** r, **size_t** c)
- template<class T2 >
 [wbMatrix](#)< T > & **AddCols** (**size_t** n, const T2 *d0=NULL)
- template<class T2 >
 [wbMatrix](#)< T > & **AddRows** (**size_t** n, const T2 *d0=NULL)
- bool **isEmpty** () const
- bool **isNormal** () const
- bool **isFinite** () const
- bool **isSquare** () const
- bool **isSquare** (**size_t** d) const
- bool **isScalar** () const
- bool **isVector** () const
- bool **isDiag** () const
- bool **isDiagMatrix** (T eps=1E-14) const
- bool **isIdentity** (double eps, double &maxdiff) const
- bool **isProptold** (T &x, T eps=1E-14) const
- bool **isHConj** (const [wbMatrix](#) &B, double eps=1E-12, double *xref=NULL) const
- bool **isHConj** (double eps=1E-12, double *xref=NULL) const
- bool **isAHerm** (const [wbMatrix](#) &B, double eps=1E-12, double *xref=NULL) const
- bool **isAHerm** (double eps=1E-12, double *xref=NULL) const
- bool **isComplex** () const
- bool **isUnique** () const
- bool **isUniqueSorted** (char dir=0, char lex=1) const
- int **isSorted** (char dir=0, char lex=1, **size_t** m=-1) const
- bool **quickCheckSorted** (char dir, char lex=1, const char *F=NULL, int L=0, **size_t** n=2) const
- bool **thisIsInt** () const
- int **getDiff** (const [wbMatrix](#) &B, [wbindex](#) &la, [wbindex](#) *lb=NULL) const
- int **getDiffSorted** (const [wbMatrix](#) &B, [wbindex](#) &la, [wbindex](#) *lb=NULL) const
- **size_t length** () const
- **size_t numel** () const
- void **flipSign** ()
- [wbMatrix](#) & **Symmetrize** ()
- void **set** (const T &x)
- [wbMatrix](#) & **setRand** (double fac=1., double shift=0.)
- void **setDiagRand** (double fac=1., double shift=0.)
- double **maxRelDiff** (const [wbMatrix](#) &M) const

- `template<class T2 >`
`double normDiff (const wbMatrix< T2 > &M) const`
- `T normDiff2 (const wbMatrix &M, size_t *k=NULL) const`
- `T normDiff (const wbMatrix &M, size_t *k=NULL) const`
- `bool sameSize (const wbMatrix B) const`
- `WBINDEX & toIndex (WBINDEX &I, const WBINDEX *S QS_UNUSED_VAR=NULL) const`
- `WBIDXMAT & toIndex2D (const WBINDEX &S QS_UNUSED_VAR, const wbindex &ic QS_UNUSED_VAR, WBIDXMAT &IJ, char pos QS_UNUSED_VAR=1) const`
- `wbMatrix & SkipTiny_float (T ref QS_UNUSED_VAR=-1)`
- `void reset ()`
- `size_t nnz () const`
- `size_t nnz (T eps) const`
- `wbMatrix & CAT (C_UINT d12, const wbMatrix **M, C_UINT len)`
- `wbMatrix & CAT (C_UINT d12, const wbvector< wbMatrix > &M0)`
- `wbMatrix & CAT (C_UINT d12, wbvector< wbMatrix const * > M)`
- `wbMatrix & CAT (C_UINT d12, const wbvector< wbvector< T > > &M)`
- `wbMatrix & CAT (C_UINT d12, const wbvector< wbvector< T > const * > &M)`
- `wbMatrix & cat (C_UINT d12, const wbMatrix &)`
- `wbMatrix & cat (C_UINT d12, const wbMatrix &, const wbMatrix &)`
- `wbMatrix & cat (C_UINT d12, const wbMatrix &, const wbMatrix &, const wbMatrix &)`
- `wbMatrix & Cat (C_UINT d12, const wbMatrix &, const wbMatrix &)`
- `wbMatrix & Cat (C_UINT d12, const wbMatrix &, const wbMatrix &, const wbMatrix &)`
- `wbMatrix< T > & CAT (const unsigned dim, wbvector< wbMatrix< T > const * > M, wbvector< WBINDEX const * > I)`
- `wbMatrix & Cat (const unsigned dim, const wbMatrix &M1, const wbindex &I1, const wbMatrix &M2, const wbindex &I2)`
- `void split (C_UINT d12, wbvector< wbMatrix * > &M) const`
- `void split (C_UINT d12, wbMatrix **M, C_UINT len) const`
- `void split (C_UINT d12, wbMatrix &M1, wbMatrix &M2, wbMatrix &M3) const`
- `const T & operator() (size_t i, size_t j, const T &x) const`
- `T & operator() (size_t i, size_t j, const T &x)`
- `const T & operator() (size_t i, size_t j) const`
- `T & operator() (size_t i, size_t j)`
- `const T & el (size_t i, size_t j) const`
- `T & el (size_t i, size_t j)`
- `const T & els (long i, long j) const`
- `T & els (long i, long j)`
- `const T * operator() (size_t i) const`
- `T * operator() (size_t i)`
- `const T & operator[] (size_t i) const`
- `T & operator[] (size_t i)`
- `const T * ref (size_t i, size_t j=0) const`
- `T * ref (size_t i, size_t j=0)`
- `const T * rec (size_t i) const`
- `T * rec (size_t i)`
- `void getDiag (wbvector< T > &D) const`
- `wbvector< T > getDiag () const`
- `void appendRows (size_t n, const T *=NULL)`
- `void appendRow (const wbvector< T > &v)`
- `mxArray * toMx (char raw=0) const`
- `mxArray * toMxP (char flag=0) const`
- `mxArray * toMx_base (char raw=0) const`
- `mxArray * toMx_base_d (char raw=0) const`
- `mxArray * toMx_Struct () const`

- `mxArray * toMxP_S () const`
- `mxArray * toMxP_C () const`
- `mxArray * toMxT (char raw QS_UNUSED_VAR=0) const`
- `mxArray * mxCreateStruct (unsigned m, unsigned n) const`
- `void add2MxStruct (mxArray *S, unsigned i, char tst=0) const`
- `void toMxStruct (mxArray *S, const char *vname) const`
- `void mat2mx (mxArray *&a) const`
- `void mat2mxs (mxArray *a, char field_nr) const`
- `void info (const char *istr="ans") const`
- `void print (const char *istr="", char mflag=0) const`
- `void Print (const char *istr="", char mflag=0) const`
- `void recPrint (size_t i, const char *istr="", char mflag=0) const`
- `void printdata (const char *istr, const char *dbl_fmt=" %8.4g", const char *rsep="\n") const`
- `wbstring toStr () const`
- `wbstring toStr (const char *fmt, const char *sep=" ", const char *rsep="; ", size_t stride=0, const char *sep2="") const`
- `wbstring rec2Str (size_t k, const char *fmt="", const char *sep=" ", size_t stride=0, const char *sep2="") const`
- `wbstring sizeStr () const`
- `void put (const char *F, int L, const char *vname="ans", const char *ws="caller", const char raw=0) const`
- `void put (const char *vname="ans", const char *ws="caller", const char raw=0) const`
- `void putg (const char *vname="ans", const char raw=0) const`
- `void putx (const char *vname="ans", const char raw=0) const`
- `void put0 (const char *vname="ans", const char *ws="caller") const`
- `void getReal (wbMatrix< double > &R) const`
- `void getImag (wbMatrix< double > &I) const`
- `void set (const wbMatrix< double > &R, const wbMatrix< double > &I)`
- `wbMatrix & Conj ()`
- `void setRec (size_t r, const T x)`
- `void setRec (size_t r, const T x1, const T x2)`
- `void recSet (size_t idest, size_t isrc)`
- `void recSet (size_t, const wbvector< T > &)`
- `template<class T2 >`
`void recSetT (size_t, const wbvector< T2 > &)`
- `void recSet (size_t, const wbMatrix &, size_t)`
- `void recSet (size_t i, const wbvector< T > &, const wbvector< T > &)`
- `void recSet (size_t, const T *, T bfac=1)`
- `void recSetP (size_t, const T *, size_t n=-1)`
- `void recSetP (size_t, const T *, size_t, const T *, size_t)`
- `void recSetP (size_t, const T *, const wbindex &i1, const T *, const wbindex &i2)`
- `void recSetB (size_t, size_t, size_t D, const T *, const T *=NULL, const T bfac=1)`
- `void recSetB (size_t, const wbindex &, size_t D, const T *)`
- `void recSet (size_t k, const wbvector< T > &a, const wbindex &ia, const wbvector< T > &b, const wbindex &ib)`
- `void recSet (size_t k, const T *a, const wbindex &ia, const T *b, const wbindex &ib)`
- `template<class T2 >`
`void setCol (size_t k, const wbvector< T2 > &v)`
- `template<class T2 >`
`void setCol (size_t k, const T2 *v, T fac=1, size_t stride=1)`
- `void setCol (size_t k, const T &x)`
- `template<class T2 >`
`void setColT (size_t k, const wbvector< T2 > &v)`
- `template<class T2 >`
`void setColT (size_t k, const T2 *v, T fac=1, size_t stride=1)`
- `void setLastCol (const T &x)`

- void **recAddP** (size_t, const T *)
- int **recLess** (size_t, size_t, char lex=1) const
- int **recLesse** (size_t, size_t, char lex=1) const
- int **recEqual** (size_t, size_t) const
- int **recEqual** (size_t j1, const T *d) const
- T **recDiff2** (size_t i1, size_t i2, size_t n=-1) const
- T **recDiff2P** (size_t i1, const T *b, size_t n=-1) const
- bool **recAllEqual** (const T *v)
- bool **recAllEqual** ()
- bool **recIsZero** (size_t r)
- char **recCompare** (size_t, size_t, size_t m=-1, char lex=1, T eps=0) const
- char **recCompare** (size_t, const [wbvector](#)< T > &r2, char lex=1) const
- char **recCompareP** (size_t, const T *, size_t m=-1, char lex=1, T eps=0) const
- void **set2Rec** (size_t i)
- [wbMatrix](#) & **Set2Recs** (const WBINDEX &l)
- [wbvector](#)< T > & **getRec** (size_t j0, [wbvector](#)< T > &v, char ref=0) const
- [wbvector](#)< T > **getRec** (size_t j0) const
- size_t **UnionRecs** (const [wbMatrix](#) &B, [wbindex](#) &la)
- bool **gotRecOverlap** (const [wbMatrix](#) &B) const
- bool **gotRecOverlapSA** (const [wbMatrix](#) &B) const
- size_t **findValsCol** (const size_t k, const [wbvector](#)< T > &v, [wbindex](#) &la) const
- [wbMatrix](#) & **getRecs** (const WBINDEX &, [wbMatrix](#) &M) const
- [wbMatrix](#) **getRecs** (const WBINDEX &l) const
- [wbMatrix](#) & **getRecs** (size_t j1, size_t j2, [wbMatrix](#) &M, size_t m=0) const
- [wbMatrix](#) **getRecs** (size_t i1, size_t i2, size_t m=0) const
- [wbvector](#)< T > & **getCol** (const size_t c, [wbvector](#)< T > &v) const
- [wbvector](#)< T > **getCol** (const size_t c) const
- [wbMatrix](#) **getCols** (const size_t j1, const size_t j2) const
- [wbMatrix](#) & **getCols** (size_t j1, size_t j2, [wbMatrix](#) &M) const
- template<class TI >
[wbMatrix](#) & **getCols** (size_t n, const TI *l, [wbMatrix](#)< T > &M) const
- [wbMatrix](#) & **getCols** (const [wbperm](#) &p, [wbMatrix](#) &M) const
- [wbMatrix](#) & **getCols** (const [wbindex](#) &l, [wbMatrix](#) &M, char iflag=0) const
- [wbMatrix](#) & **set2Cols** (const size_t j1, const size_t j2)
- [wbMatrix](#) & **set2Cols** (const [wbindex](#) &J, char iflag=0)
- [wbMatrix](#) & **recPermute** (const [wbperm](#) &P, [wbMatrix](#) &M, char iflag=0) const
- [wbMatrix](#) & **recPermute** (const [wbperm](#) &P, char iflag=0)
- [wbMatrix](#) & **colPermute** (const [wbperm](#) &P, [wbMatrix](#) &M, char iflag=0) const
- [wbMatrix](#) & **colPermute** (const [wbperm](#) &P, char iflag=0)
- [wbMatrix](#) & **FlipRecs** ()
- [wbMatrix](#) & **FlipCols** ()
- [wbMatrix](#) & **cols2Front** (const WBINDEX &l1, [wbMatrix](#) &M) const
- [wbMatrix](#) & **cols2End** (const WBINDEX &l1, [wbMatrix](#) &M) const
- void **blockPermute** (const [wbperm](#) &P, [wbMatrix](#) &M) const
- void **BlockPermute** (const [wbperm](#) &P)
- [wbvector](#)< T > & **recProd** ([wbvector](#)< T > &p) const
- T **recProd** (size_t) const
- T **recSum** (size_t) const
- [wbvector](#)< T > & **recSum** ([wbvector](#)< T > &) const
- [wbvector](#)< T > & **recSumA** ([wbvector](#)< T > &) const
- [wbvector](#)< T > **recSum** () const
- T **colSum** (size_t) const
- [wbMatrix](#) & **repmat** (size_t m, size_t n, [wbMatrix](#) &Q, size_t pad1=0, size_t pad2=0) const
- [wbMatrix](#) & **Repmat** (size_t m, size_t n, size_t pad1=0, size_t pad2=0)
- void **ColKron** (const [wbMatrix](#) &Q, int d=-1)

- **wbMatrix** & **getBlocks** (size_t j1, size_t j2, const size_t D, **wbMatrix** &M) const
- **wbMatrix** & **getBlock** (size_t j1, const size_t D, **wbMatrix** &M) const
- **wbMatrix** **getBlock** (size_t j1, const size_t D) const
- template<class T2 >
wbMatrix< T > & **getBlock** (size_t k, size_t D, const **wbMatrix**< T2 > &R, size_t k2, size_t D2, **wbMatrix** &M) const
- **wbMatrix** & **getBlock** (size_t i, size_t j, size_t n, size_t m, **wbMatrix** &M) const
- **wbMatrix** & **blockSum** (size_t D, **wbMatrix** &) const
- void **blockSum** (size_t, size_t, const size_t, **wbMatrix**< double > &) const
- void **blockSum** (const **wbindex** &J, const size_t, **wbMatrix**< double > &) const
- void **setBlock** (size_t k, const size_t D, const T *)
- void **SetBlock** (size_t i0, size_t j0, const **wbMatrix** &M)
- **WBIDXMAT** & **toBlockIndex** (widx_t D, **WBIDXMAT** &I1, **wbvector**< **wbMatrix** > *QI=NULL) const
- template<class T2 >
void **toBlockIndex** (widx_t D, **wbMatrix**< T2 > &R, widx_t DR, **WBIDXMAT** *I1=NULL, **WBIDXMAT** *I2=NULL, **WBIDXMAT** *SS=NULL, **wbvector**< **wbMatrix** > *QI=NULL) const
- void **groupRecs** (**wbperm** &P, **WBINDEX** &d, size_t m=-1, char lex=1, **wbindex** *I1=NULL, **wbMatrix**< T > *X=NULL)
- void **groupRecs** (**wbindex** &I1, widx_t m=-1, char lex=1)
- void **groupRecs** (**groupIndex**< widx_t > &IG, size_t m=-1, char lex=1, **wbindex** *I1=NULL)
- void **groupRecs** (**groupIndex**< widx_t > &IG, size_t m, char lex, const wperm_t **p)
- void **groupRecs** (**wbperm** &P, **WBINDEX** &d, **wbMatrix** &B, size_t m=-1, char lex=1, **wbindex** *I1=NULL) const
- void **groupRecs** (**wbperm** &P, **WBINDEX** &d, const **WBINDEX** *I1, char lex=1)
- template<class T2 >
void **groupRecs_rdeg** (**wbperm** &P, **WBINDEX** &D, const **wbMatrix**< T2 > &R, **WBINDEX** *I1=NULL, **WBINDEX** *I2=NULL, **WBINDEX** *Sb=NULL, char iflag=0)
- void **groupRecs** (**wbperm** &P, **WBINDEX** &D, size_t nc, **WBINDEX** &I1, **WBINDEX** &I2, **WBINDEX** &Sb, char iflag=0)
- void **groupSortedRecs** (**WBINDEX** &d, char keepall=0, size_t m=-1, char lex=1)
- void **groupSortedRecs** (**WBINDEX** &d, const **WBINDEX** *I1)
- void **groupSortedRecs** (**WBINDEX** &d, size_t mc, char lex, **WBINDEX** &I1, **WBINDEX** &I2, **WBINDEX** &Sb)
- void **makeUnique** ()
- void **makeUnique** (**wbperm** &P, **WBINDEX** &D)
- void **makeUnique** (**wbindex** &I1)
- void **findUnique** (**wbindex** &I1) const
- size_t **findUniqueRecSorted1** (size_t n=-1, T eps=T(1E-12)) const
- void **makeUnique_ig** (**wbindex** &I1)
- void **makeUnique** (**groupIndex**< widx_t > &IG)
- **wbMatrix** & **SortRecs** (**wbperm** &P, char dir=+1, char lex=1)
- **wbMatrix** & **sortRecs** (**wbMatrix** &B, **wbperm** &P, char dir=+1, char lex=1)
- **wbMatrix** & **SortRecs** (char dir=+1, char lex=1)
- **wbMatrix** & **sortRecs_float** (**wbperm** &P, char dir QS_UNUSED_VAR=+1)
- **wbMatrix** & **sortRecs_float** ()
- char **findRecsInSet** (const **wbMatrix** &S, **wbvector**< int > &I1) const
- int **findRecSorted** (const T *r, size_t n=-1, char lex=1) const
- int **findRec** (const T *r, size_t n=-1, char lex=1) const
- bool **thisIsInt** () const
- bool **thisIsInt** () const
- bool **thisIsInt** () const
- bool **thisIsInt** () const
- bool **thisIsInt** () const
- bool **thisIsInt** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const

- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- WBINDEX & **toIndex** (WBINDEX &l, const WBINDEX *S) const
- WBIDXMAT & **toIndex2D** (const WBINDEX &S, const [wbindex](#) &lc, WBIDXMAT &IJ, char pos) const
- [wbMatrix](#)< double > & **SkipTiny_float** (double x)
- [wbMatrix](#)< [wbcomplex](#) > & **SkipTiny_float** ([wbcomplex](#) x)
- size_t **maxRec_float** (char lex, size_t *m_, double xref) const
- template<class T1 >
 [wbMatrix](#)< T > & **getCols** (size_t n, const T1 *l, [wbMatrix](#)< T > &M) const
- size_t **skipNanRecs** ([wbindex](#) &l)
- [wbstring](#) **toStr** (const char *fmt0, const char *sep, const char *rsep, size_t stride, const char *sep2) const
- template<class T2 >
 void **groupRecs_rdeg** ([wbperm](#) &P, WBINDEX &D, const [wbMatrix](#)< T2 > &R, [wbvector](#)< widx_t > *lb,
 [wbvector](#)< widx_t > *l2, [wbvector](#)< widx_t > *Sb, char iflag)
- [wbMatrix](#)< double > & **sortRecs_float** ([wbperm](#) &P, char dir)
- bool **isComplex** () const
- [wbMatrix](#)< [wbcomplex](#) > & **setRand** (double fac, double shift)
- [wbMatrix](#)< [wbcomplex](#) > & **Symmetrize** ()
- mxArray * **toMx_base** (char raw) const
- mxArray * **toMx_base** (char raw) const
- mxArray * **toMx_base** (char raw) const
- mxArray * **toMx_base** (char raw) const
- mxArray * **toMx** (char raw) const
- mxArray * **toMxT** (char raw) const
- mxArray * **toMx** (char raw) const
- mxArray * **toMxT** (char raw) const
- mxArray * **toMx** (char raw) const
- mxArray * **toMxT** (char raw) const
- mxArray * **toMx** (char raw) const
- mxArray * **toMxT** (char raw) const
- mxArray * **toMx** (char raw) const
- mxArray * **toMxT** (char raw) const
- mxArray * **toMx** (char raw) const
- mxArray * **toMxT** (char raw) const
- mxArray * **toMx** (char raw) const
- mxArray * **toMxT** (char raw) const
- mxArray * **toMx** (char raw) const
- mxArray * **toMxT** (char raw) const
- mxArray * **toMx** (char raw) const
- mxArray * **toMxT** (char raw) const
- double **normReal** () const
- double **normImag** () const
- [wbMatrix](#)< [wbcomplex](#) > & **Conj** ()
- void **getReal** ([wbMatrix](#)< double > &R) const
- void **getImag** ([wbMatrix](#)< double > &I) const
- void **set** (const [wbMatrix](#)< double > &R, const [wbMatrix](#)< double > &I)

Public Attributes

- T * **data**
- size_t **dim1**
- size_t **dim2**
- char **isdiag**
- char **isref**

Protected Member Functions

- void **NEW_DATA** (T *d=NULL)
- void **RENEW** (const size_t &d1, const size_t &d2, const T *dd=NULL, char iflag=1)

The documentation for this class was generated from the following files:

- wblib.h
- wbMatrix.hh
- wbMatrix.cc

4.107 wbMsg Class Reference

Public Member Functions

- **wbMsg** (const char *file, int line, const char *istr="Message queue", int key_=0)
- int **Rec** (const char *, int, [wbMsgBuf](#) &rbuf, unsigned flag=IPC_NOWAIT)
- int **Snd** (const char *, int, [wbMsgBuf](#) &sbuf, const char *istr, double val=0., unsigned type=1)

Public Attributes

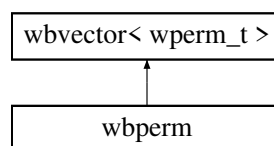
- [wbstring](#) **info**
- int **msqid**
- int **msgflg**
- key_t **key**

The documentation for this class was generated from the following file:

- wbmsg.hh

4.108 wbperm Class Reference

Inheritance diagram for wbperm:



Public Member Functions

- **wbperm** (wperm_t n, char reverse=0)
- **wbperm** (const [wbperm](#) &P, char iflag=0, unsigned r=-1)
- **wbperm** (const [wbperm](#) &P, size_t r)
- template<class T >
 wbperm (const [wbvector](#)< T > &b, wperm_t offset=0)
- **wbperm** (const [wbindex](#) &P)
- **wbperm** (const char *s, wperm_t offset=1)
- wperm_t **el1** (wperm_t i) const
- [wbperm](#) & **operator=** (const [wbperm](#) &P)
- [wbperm](#) & **operator=** (const char *s)
- int **initStr** (const char *F, int L, const char *s, wperm_t offset=1)
- [wbperm](#) & **init** (const [wbperm](#) &P, char iflag=0, unsigned r=-1)
- [wbperm](#) & **Extend** (wperm_t r)
- [wbperm](#) & **init_trafo** (const [wbperm](#) &p1_, const [wbperm](#) &p2_)
- [wbperm](#) & **init_pad** (const [wbperm](#) &P, wperm_t r)
- [wbperm](#) & **init** (unsigned n=0, char reverse=0)
- [wbperm](#) & **init** (unsigned n, const wperm_t *d, char ref=0)
- [wbperm](#) & **init** (const char *F, int L, const mxArray *a)
- [wbperm](#) & **init** (const char *F, int L, const mxArray *a, wperm_t offset)
- template<class T >
 [wbperm](#) & **init** (const char *F, int L, const [wbvector](#)< T > &b, wperm_t offset=0)
- [wbperm](#) & **initTranspose** (wperm_t r)
- [wbperm](#) & **initPerm2** (wperm_t r)
- [wbperm](#) & **initFirstTo** (wperm_t k, wperm_t N)
- [wbperm](#) & **initLastTo** (wperm_t k, wperm_t N)
- [wbperm](#) & **init2Front** (const WBPPerm &l, wperm_t N)
- [wbperm](#) & **init2End** (const WBPPerm &l, wperm_t N)
- [wbperm](#) & **init2FrontB** (wperm_t m, wperm_t N)
- [wbperm](#) & **init2EndB** (wperm_t m, wperm_t N)
- [wbperm](#) & **init2Front** (wperm_t k, wperm_t N)
- [wbperm](#) & **init2End** (wperm_t k, wperm_t N)
- [wbperm](#) & **Cycle** (wperm_t k1, wperm_t k2)
- [wbperm](#) & **initCycle** (wperm_t n, wperm_t k1, wperm_t k2)
- [wbperm](#) & **Index** (wperm_t n, char reverse=0)
- [wbperm](#) & **Index** (wperm_t i1, wperm_t i2)
- char **isValidPerm** (const char *F, int L, wperm_t r=-1) const
- char **isValidPerm** (wperm_t r=-1) const
- [wbperm](#) & **Complete** (wperm_t l)
- [wbperm](#) & **Invert** ()
- [wbperm](#) & **invert** ([wbperm](#) &iP) const
- [wbperm](#) & **Rotate** (swperm_t l)
- [wbperm](#) & **initRotate** (wperm_t n, swperm_t k)
- [wbperm](#) & **Permute** (const [wbperm](#) &P, char iflag, wperm_t r)
- [wbperm](#) & **Permute** (const [wbperm](#) &P, char iflag=0)
- bool **sameAs** (const [wbperm](#) &b) const
- bool **isIdentityPerm** () const
- bool **isIdentityPerm** (const char *F, int L, wperm_t r=-1) const
- bool **isReversePerm** () const
- bool **isCyclic2F** (wperm_t m, wperm_t n=-1, char iflag=0) const
- bool **isOpTranspose** () const
- **operator bool** () const
- bool **operator!** () const
- [wbperm](#) & **times** (const [wbperm](#) &p2, [wbperm](#) &Pout) const

- [wperm](#) & [flipldx](#) ([wperm](#) &P) const
- [wperm](#) & [Flipldx](#) ()
- [wperm](#) [flipldx](#) () const
- void [ToRaw](#) ()
- void [ToRaw](#) (wperm_t n)
- [wbstring](#) [toStr](#) () const
- mxArray * [toMx](#) (const char tflag=0) const

Public Member Functions inherited from [wbvector](#)< [wperm_t](#) >

- [wbvector](#) (size_t l, wperm_t x)
- [wbvector](#) (size_t l, const wperm_t *d0=NULL, char ref=0)
- [wbvector](#) (const [wbvector](#) &v)
- [wbvector](#) (const [wbvector](#)< T2 > &v)
- [wbvector](#) (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) (const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) & [init](#) (size_t l=0)
- [wbvector](#) & [init](#) (size_t l, const wperm_t *d0)
- [wbvector](#) & [init](#) (size_t l, const wperm_t *d0, char ref)
- [wbvector](#) & [init](#) (const [wbvector](#) &v)
- [wbvector](#) & [init](#) (size_t n, const [wbvector](#) &v)
- [wbvector](#)< wperm_t > & [init](#) (const [wbsparray](#)< wperm_t > &S)
- [wbvector](#) & [init](#) (size_t l1, const wperm_t *d1, size_t l2, const wperm_t *d2)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type QS_UNUSED_↵, VAR=1, char ref=0)
- int [init](#) (const mxArray *a, const char *istr=NULL, char check_type=1)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- [wbvector](#) & [initp](#) (size_t l, const wperm_t *d0, size_t n, char init=1)
- [wbvector](#) & [init2val](#) (size_t l, const wperm_t x)
- [wbvector](#) & [initStride](#) (size_t l, const wperm_t *d0, size_t stride)
- [wbvector](#) & [init_stride](#) (const wperm_t *d0, size_t n, size_t m, size_t D0)
- [wbvector](#) & [init_data](#) (const wperm_t *d0)
- [wbvector](#) & [init_data](#) (const T0 *d0)
- [wbvector](#) & [init_data](#) (const char *F, int L, const T0 *d0)
- [wbvector](#)< wperm_t > & [initT](#) (const [wbsparray](#)< T2 > &S)
- [wbvector](#)< wperm_t > & [initT](#) (size_t l, const T2 *v)
- [wbvector](#)< wperm_t > & [initT](#) (const [wbvector](#)< T2 > &v)
- [wbvector](#)< wperm_t > & [initT](#) (const char *F, int L, const [wbvector](#)< T2 > &v)
- [wbvector](#)< wperm_t > & [initT](#) (const [wbsparray](#)< TD > &S)
- [wbvector](#)< wperm_t > & [initT_](#) (const [wbvector](#)< T2 > &v)
- [wbvector](#) & [init2ref](#) (const [wbvector](#) &v)
- [wbvector](#) & [init2ref](#) (size_t l, const wperm_t *d0)
- [wbvector](#) & [unRef](#) ()
- int [init_Struct](#) (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)
- int [init_base](#) (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)
- int [init_mpfr](#) (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)
- void [initDef](#) (size_t n)
- void [initDef](#) (const [wbvector](#) &v)

- [wbvector](#) & **operator=** (const [wbvector](#) &v)
- [wbvector](#)< wperm_t > & **operator=** (const [wbvector](#)< TB > &v)
- [wbvector](#) & **set** (const wperm_t &x)
- [wbvector](#) & **set** (const [wbvector](#)< size_t > &l, const wperm_t &x)
- void **set** (size_t offset, const wperm_t *x, size_t n)
- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- [wbvector](#) & **setMin** (const wperm_t &x)
- [wbvector](#) & **setMax** (const wperm_t &x)
- [wbvector](#) & **add** (const [wbvector](#)< size_t > &l, const wperm_t &x)
- [wbvector](#) & **add** (const wperm_t *d)
- [wbvector](#) & **Set** (const [wbvector](#)< size_t > &l, const [wbvector](#) &v)
- void **Sort** ([wbperm](#) &p, char dir=+1)
- void **Sort** (char dir=+1)
- void **sort** ([wbvector](#) &v2, [wbperm](#) &p) const
- void **sort** ([wbvector](#) &v2) const
- [wbperm](#) & **pSort** ([wbperm](#) &p) const
- bool **isSorted** () const
- bool **isSortedU** () const
- bool **isUnique** () const
- int **contains** (const wperm_t &x) const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isFinite** () const
- size_t **find1** (const wperm_t &x) const
- size_t **find1** () const
- size_t **count** (const wperm_t &x) const
- size_t **icount** (const wperm_t &x) const
- size_t **nnz** () const
- size_t **nnz** (wperm_t eps) const
- size_t **numZeros** (wperm_t eps=wperm_t(1E-14)) const
- size_t **numEQ** (const wperm_t &) const
- size_t **numGT** (const wperm_t &) const
- size_t **numLT** (const wperm_t &) const
- [wbindex](#) & **find** (const wperm_t &x, [wbindex](#) &l, char iflag=0) const
- [wbindex](#) & **find** (const wperm_t &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **find** ([wbindex](#) &l) const
- [wbindex](#) & **find** ([wbindex](#) &l, [wbindex](#) &lx) const
- [wbvector](#)< size_t > **find** () const
- [wbindex](#) & **findGT** (const wperm_t &x, [wbindex](#) &l, char iflag=0) const
- [wbindex](#) & **findGT** (const wperm_t &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **findzeros** ([wbindex](#) &l) const
- [wbindex](#) & **findRange** (const wperm_t &x1, const wperm_t &x2, [wbindex](#) &l, const char *w=NULL) const
- int **findClosestSorted** (wperm_t r) const
- int **findClosest** (wperm_t r) const
- size_t **findValues** (const [wbvector](#) &v, [wbindex](#) &l) const
- [wbvector](#) & **setp** (const wperm_t *x)
- void **reset** ()
- void **cpy** (const wperm_t *v)

- void **cpy** (const wperm_t *x, size_t m)
- [wbvector](#) & **save2** ([wbvector](#) &v)
- void **swap** ([wbvector](#) &v)
- [wbvector](#) & **swap** (size_t i, size_t j)
- [wbvector](#) & **Collect** (wperm_t &A1, wperm_t &A2)
- [wbvector](#) & **Collect** (wperm_t &A1, wperm_t &A2, wperm_t &A3)
- [wbvector](#) & **Skip** (size_t)
- [wbvector](#) & **Skip** (const WBINDEX &, char rflag=0)
- [wbvector](#) & **Move2_FE** (const WBINDEX &l, char FE_flag)
- [wbvector](#) & **Move2front** (const WBINDEX &l)
- [wbvector](#) & **Move2front** (size_t k)
- [wbvector](#) & **Move2end** (const WBINDEX &l)
- [wbvector](#) & **Move2end** (size_t k)
- [wbvector](#) & **mvLastTo** (size_t k)
- [wbvector](#) & **mvFirstTo** (size_t k)
- bool **checkSameLength** (const [wbvector](#) &a, const char *F=NULL, int L=0, const char *istr="") const
- bool **checkSameLength** (const [wbvector](#) &a, const [wbvector](#) &b, const char *F=NULL, int L=0, const char *istr="") const
- void **operator+=** (const T_ &a)
- void **operator+=** (const [wbvector](#) &b)
- void **operator-=** (const T_ &a)
- void **operator-=** (const [wbvector](#) &b)
- [wbvector](#) & **operator*=** (const T_ &a)
- [wbvector](#) & **operator/=** (const T_ &a)
- [wbvector](#) **operator*** (const T_ &a) const
- [wbvector](#) **operator+** (const T_ &a) const
- [wbvector](#) **operator+** (const [wbvector](#) &b) const
- [wbvector](#) **operator-** (const T_ &a) const
- [wbvector](#) **operator-** (const [wbvector](#) &b) const
- [wbvector](#) & **Plus** (const wperm_t *b)
- [wbvector](#) & **Plus** (const wperm_t *b, wperm_t bfac)
- [wbvector](#) & **Plus** (const [wbvector](#) &b, wperm_t bfac=1)
- void **plus** (const [wbvector](#) &b, [wbvector](#) &c, wperm_t bfac=1, wperm_t cfac=0) const
- [wbvector](#) & **times** (wperm_t fac, [wbvector](#) &x) const
- [wbvector](#) & **RevertSigns** ()
- int **isum** () const
- wperm_t **sum** () const
- wperm_t **sum** (size_t i1, const char *estr) const
- wperm_t **sum** (size_t i1, size_t i2) const
- wperm_t **cumsum_** ([wbvector](#) &S, char xflag=0) const
- wperm_t **Cumsum** (char zflag=0) const
- wperm_t **cumsum0prod** ([wbvector](#) < T2 > &b, [wbvector](#) < wperm_t > &cs, char xflag) const
- [wbvector](#) & **Eldiv** (const [wbvector](#) &v) const
- [wbvector](#) & **tensorProd** (const [wbvector](#) &v2, [wbvector](#) &vv) const
- wperm_t **scalarProd** (const [wbvector](#) &v) const
- wperm_t **scalarProd** (const wperm_t *v) const
- wperm_t **norm** () const
- wperm_t **norm2** () const
- double **normDiff** (const [wbvector](#) &v, wperm_t fac=1) const
- wperm_t **normDiff2** (const [wbvector](#) &v, wperm_t fac=1) const
- double **normDiff2_** (const [wbvector](#) &v, double fac=1) const
- wperm_t **GSPProject** (const wperm_t *v, char isnorm=0, char tflag=0) const
- [wbvector](#) & **Normalize** (const char *F=NULL, int L=0)
- [wbvector](#) & **subtract** (const wperm_t *d)
- void **getReal** ([wbvector](#) < double > &R) const

- void **getReal** ([wbvector](#)< double > &R) const
- void **getImag** ([wbvector](#)< double > &I) const
- void **getImag** ([wbvector](#)< double > &I) const
- [wbvector](#) & **Conj** ()
- [wbvector](#)< [wbcomplex](#) > & **Conj** ()
- [wbvector](#) & **applyFlag** (char &flag, const wperm_t &fac, [wbvector](#) &a)
- bool **allEqual** () const
- bool **allEqual** (const wperm_t &a) const
- bool **allZero** () const
- bool **allOnes** () const
- bool **allEqual2** (const wperm_t &a, wperm_t eps=1E-15) const
- bool **allUnequal** (const wperm_t &a) const
- bool **any** () const
- bool **anyUnequal** (const wperm_t &a) const
- bool **anyUnequal** (const [wbvector](#)< size_t > &l, const wperm_t &x, size_t *k=NULL) const
- bool **anyEqual** (const wperm_t &a, size_t *i1=NULL) const
- bool **anyEqual** (const [wbvector](#)< size_t > &l, const wperm_t &x, size_t *k=NULL) const
- bool **anyGT** (const wperm_t &a) const
- bool **anyGE** (const wperm_t &a) const
- bool **anyLT** (const wperm_t &a) const
- bool **anyLE** (const wperm_t &a) const
- bool **allGT** (const wperm_t &a) const
- bool **allGE** (const wperm_t &a) const
- bool **allLE** (const wperm_t &a) const
- bool **allLT** (const wperm_t &a) const
- bool **allIn** (const wperm_t &a1, const wperm_t &a2) const
- bool **anySmallVals** (const wperm_t eps=1E-15) const
- bool **isEqual** (const wperm_t *v) const
- bool **isEqual** (const [wbvector](#) &v) const
- bool **isEmpty** () const
- **operator bool** () const
- bool **operator!** () const
- bool **operator==** (const [wbvector](#) &v) const
- bool **operator!=** (const [wbvector](#) &v) const
- bool **isEqualR** (const [wbvector](#) &v) const
- bool **operator^** (const [wbvector](#) &v) const
- bool **deepEqualP** (const [wbvector](#) &B) const
- char **cmp** (const [wbvector](#) &v) const
- bool **operator<** (const [wbvector](#) &v) const
- int **operator<** (const wperm_t &r) const
- bool **operator<=** (const [wbvector](#) &v) const
- bool **operator>** (const [wbvector](#) &v) const
- bool **operator>=** (const [wbvector](#) &v) const
- wperm_t & **el** (size_t i) const
- wperm_t **elx** (size_t i, const wperm_t &x) const
- const wperm_t & **operator[]** (size_t i) const
- wperm_t & **operator[]** (size_t i)
- [wbvector](#) **operator[]** (const [wbvector](#)< size_t > &l) const
- wperm_t & **last** () const
- wperm_t & **last** (size_t &i) const
- wperm_t & **last** (TI &i_) const
- wperm_t & **p2last** () const
- wperm_t * **ref** (size_t i) const
- [wbvector](#) & **setRand** (double fac=1., double shift=0.)
- [wbvector](#)< [wbcomplex](#) > & **setRand** (double fac, double shift)

- void **setidx** (double shift=0., double fac=1.)
- int **set2Group** (const [wbperm](#) &P, const WBINDEX &D, const wperm_t *S0=NULL, char iflag=0)
- [wbvector](#) & **Resize_** (size_t n)
- [wbvector](#) & **Shorten2** (size_t n)
- [wbvector](#) & **Resize** (size_t n, const wperm_t *d=NULL)
- [wbvector](#) & **resize** (size_t n, [wbvector](#) &X) const
- [wbvector](#) & **cat** (const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) > &vv)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) * > &vv)
- [wbvector](#) & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)
- [wbvector](#) & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3=0, const [wbvector](#)< T3 > *v4=0)
- [wbvector](#)< wperm_t > & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3, const [wbvector](#)< T3 > *v4)
- [wbvector](#)< wperm_t > & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)
- [wbvector](#) & **Append** (const [wbvector](#) &v2)
- [wbvector](#) & **Append** (const wperm_t &x)
- [wbvector](#) & **Append** (size_t d2, const wperm_t *x)
- [wbvector](#) **append** (const [wbvector](#) &v) const
- [wbvector](#) & **append** (const [wbvector](#) &v2, [wbvector](#) &vout) const
- void **append** (const wperm_t *x, size_t d2, [wbvector](#) &vout) const
- int **recLess** (const wperm_t *v) const
- int **recLessE** (const wperm_t *v) const
- int **recCompare** (const wperm_t *v) const
- wperm_t **max** () const
- wperm_t **max** (size_t *k) const
- wperm_t **max** (unsigned &k) const
- wperm_t **max_** (wperm_t x) const
- wperm_t **min** () const
- wperm_t **min** (size_t *k) const
- wperm_t **min** (unsigned &k) const
- wperm_t **min_** (wperm_t x) const
- wperm_t **aMax** (size_t *k=NULL) const
- wperm_t **aMax** (unsigned &k) const
- wperm_t **aMin** (char zflag=0, size_t *k=NULL) const
- wperm_t **maxneg** (size_t *k=NULL) const
- wperm_t **minpos** (size_t *k=NULL) const
- wperm_t **avg** () const
- wperm_t **std** () const
- void **range** (wperm_t &xmin, wperm_t &xmax) const
- wperm_t **range** () const
- [wbvector](#) & **Permute** (const [wbperm](#) &P, char iflag=0)
- [wbvector](#) & **permute** ([wbvector](#) &, const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) **permute** (const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) & **blockPermute** (const [wbperm](#) &P, [wbvector](#) &, char iflag=0) const
- [wbvector](#) & **BlockPermute** (const [wbperm](#) &P, char iflag=0)
- int **blockCompare** (const [wbperm](#) &pa, const [wbvector](#) &b, const [wbperm](#) &pb) const
- void **get** (const WBINDEX &I0, [wbvector](#) &v) const
- void **get** (const [wbvector](#)< size_t > &I, wperm_t *v) const
- void **get** (size_t i1, size_t i2, [wbvector](#) &v) const
- void **selectSU** (const WBINDEX &I)
- void **select** (const WBINDEX &I, wperm_t *r) const

- [wbvector](#) & **select** (const WBINDEX &l, [wbvector](#) &a) const
- [wbvector](#) **select** (const WBINDEX &l) const
- [wbvector](#) & **Select** (const WBINDEX &l)
- [wbvector](#) & **BlockSelect** (const WBINDEX &l, const WBINDEX &d)
- [wbvector](#) & **getl** (size_t k, [wbvector](#) &v) const
- [wbvector](#) & **getl** (const [wbindex](#) &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **getl** (const WBINDEX &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **initl** (const [wbvector](#) &a, const [wbindex](#) &ia)
- [wbvector](#) & **initl** (const [wbvector](#) &a, const [wbindex](#) &ia, const [wbvector](#) &b, const [wbindex](#) &ib, char uflag=1)
- [wbvector](#) & **Flip** ()
- void **flip** ([wbvector](#) &a) const
- [wbvector](#) **flip** () const
- wperm_t **prod** () const
- wperm_t **prod** (wperm_t x) const
- wperm_t **prod** (size_t i, size_t i2) const
- wperm_t **prod** (size_t i, size_t i2, wperm_t x) const
- wperm_t **prod** (const T1 *l, size_t n) const
- T2 **prod** (const T1 *l, size_t n, T2 &s2) const
- void **prod2** (wperm_t &p1, wperm_t &p2, const wperm_t &p0=0) const
- void **info** (const char *istr) const
- void **print** (const char *istr, char mflag=0) const
- void **printdata** (const char *istr=0) const
- void **put** (const char *F, int L, const char *vname="ans", char tflag=0, const char *ws="base") const
- void **put** (const char *vname="ans", char tflag=0, const char *ws="base") const
- void **dispWithMsg** (const char *file, int line, const char *msg, const char *vname="") const
- void **mat2mx** (mxArray *&a, char dim=2) const
- void **mat2mxs** (mxArray *a, char field_nr, char dim=2) const
- mxArray * **toMx_Struct** (const char tflag=0) const
- mxArray * **toMxP_S** (const char tflag=0) const
- mxArray * **toMx** (const char tflag=0) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMxP** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base_d** (const char tflag=0) const
- mxArray * **toMx_offset** (const wperm_t &offset, const char tflag=0) const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tflag=0) const
- void **add2MxStruct** (mxArray *S, const char *vname, char tflag=0) const
- mxArray * **mxCreateCell** (unsigned m, unsigned n) const
- void **add2MxCell** (mxArray *S, unsigned i, char tflag=0) const
- void **putg** (const char *vname="ans", char tflag=0) const
- void **putx** (const char *vname="ans", char tflag=0) const
- [wbstring](#) **toStr** (int n, const char *sep) const

- `wbstring toStr ()` const
- `wbstring toStr (int n, const char *sep)` const
- `wbstring sizeStr ()` const
- `wbstring toStrf (const char *fmt0="", const char *sep=" ", unsigned stride=0, const char *sep2="")` const
- `wbstring toStrf (const char *fmt_, const char *sep, unsigned stride, const char *sep2)` const

Additional Inherited Members

Public Attributes inherited from `wbvector< wperm_t >`

- `wperm_t * data`
- `size_t len`
- `char isref`

Protected Member Functions inherited from `wbvector< wperm_t >`

- `wbvector & RENEW (size_t n, const wperm_t *d=NULL, size_t l=-1, char init=1)`
- `wbvector & RENEW_VAL (size_t n, const wperm_t &x)`

The documentation for this class was generated from the following files:

- `wbperm.hh`
- `wbindex.hh`
- `wbperm.cc`

4.109 `wbperm_helper< T >` Class Template Reference

Public Member Functions

- `wbperm_helper (const wbvector< widx_t > &sz_, const wbperm &perm)`
- `void permute (const T *src, T *dest, int np=1)` const
- `wbperm_helper & save2 (wbperm_helper &X)`
- `mxArray * toMx ()` const

Public Attributes

- unsigned `rk_`
- unsigned `rk`
- `widx_t numel`
- `widx_t * sz`
- `widx_t * stride`
- unsigned `l1`
- unsigned `l2`
- unsigned `m_blk`

The documentation for this class was generated from the following files:

- `wbarray.hh`
- `wbarray.cc`

4.110 `wbRecs< T >` Class Template Reference

Public Member Functions

- **wbRecs** (const `wbMatrix< T >` &A, `wbperm` &P, char dir_=1, char lex_=1)
- bool **operator()** (const size_t &pa, const size_t &pb) const

The documentation for this class was generated from the following file:

- `wbMatrix.hh`

4.111 `wbREF< TX >` Class Template Reference

Public Member Functions

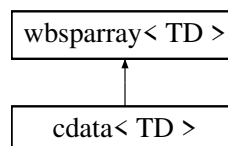
- **wbREF** (const TX &A, char flag)
- TX & **Ref** ()

The documentation for this class was generated from the following file:

- `gathered.hh`

4.112 `wbsparray< TD >` Class Template Reference

Inheritance diagram for `wbsparray< TD >`:



Public Member Functions

- **wbsparray** (SPIDX_T d1)
- **wbsparray** (SPIDX_T d1, SPIDX_T d2)
- **wbsparray** (SPIDX_T d1, SPIDX_T d2, SPIDX_T d3)
- **wbsparray** (SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T d4)
- **wbsparray** (const `wbvector< SPIDX_T >` &d, SPIDX_T l=0)
- **wbsparray** (char *sidx, SPIDX_T l=0)
- **wbsparray** (const `wbsparray` &a)
- **wbsparray** (const char *F, int L, const mxArray *a)
- **wbsparray** (const mxArray *a)
- `wbsparray` & **init** (const char *F, int L, const `wbarray< TD >` &A, TD eps=0)
- template<class TA >
`wbarray< TA >` & **toFull** (`wbarray< TA >` &A) const

- `template<class TA >`
`wbvector`< TA > & **toFull** (`wbvector`< TA > &a) const
- `template<class DB >`
`wbsparray` & **init** (const `wbsparray`< DB > &a)
- `SPIDX_T catRecs` (`SPIDX_T` i0, const `wbsparray` &a, `SPIDX_T` id)
- `wbsparray` & **init** (const `wbvector`< `SPIDX_T` > &S0, `SPIDX_T` l=0)
- `template<class I2 >`
`wbsparray` & **init** (const `wbvector`< I2 > &S_, `SPIDX_T` l=0)
- `wbsparray` & **init** ()
- `wbsparray` & **init** (`SPIDX_T` d1)
- `wbsparray` & **init** (`SPIDX_T` d1, `SPIDX_T` d2)
- `wbsparray` & **init** (`SPIDX_T` d1, `SPIDX_T` d2, `SPIDX_T` d3)
- `wbsparray` & **init** (`SPIDX_T` d1, `SPIDX_T` d2, `SPIDX_T` d3, `SPIDX_T` d4)
- `wbsparray` & **initz** ()
- `wbsparray` & **initz** (`SPIDX_T` d1, `SPIDX_T` n)
- `wbsparray` & **initz** (`SPIDX_T` d1, `SPIDX_T` d2, `SPIDX_T` n)
- `wbsparray` & **initz** (`SPIDX_T` d1, `SPIDX_T` d2, `SPIDX_T` d3, `SPIDX_T` n)
- `wbsparray` & **init_rzd** (`SPIDX_T` r, `SPIDX_T` n)
- `wbsparray` & **init_nnz** (`SPIDX_T` n=0, char kflag=0)
- `wbsparray` & **setRand** (double x=1, char sflag=0)
- `wbsparray` & **setRand_data** (double x=1, char sflag=0)
- `template<class DB >`
`wbsparray` & **init** (const char *F, int L, `SPIDX_T` d1, `SPIDX_T` d2, const `wbsparray`< DB > &B)
- `template<class DB >`
`wbsparray` & **init** (const char *F, int L, `SPIDX_T` d1, `SPIDX_T` d2, `SPIDX_T` d3, const `wbsparray`< DB > &B)
- `template<class DB >`
`wbsparray` & **init** (const char *F, int L, `SPIDX_T` d1, `SPIDX_T` d2, `SPIDX_T` d3, `SPIDX_T` d4, const `wbsparray`< DB > &B)
- `wbsparray` & **initEmpty** (const `wbsparray` &a)
- `wbsparray` & **init** (const char *F, int L, const mxArray *a, unsigned k=0)
- `wbsparray` & **init2ref** (const `wbsparray` &a)
- `wbsparray` & **unRef** ()
- `wbsparray` & **initCAT** (const char *F, int L, `wbvector`< `wbsparray` * > &X, `wbvector`< `SPIDX_T` > *i0=NULL, char del=0)
- `wbsparray` & **initCAT** (const char *F, int L, `wbvector`< `wbsparray` > &X, const `wbvector`< `SPIDX_T` > *i0=NULL)
- `wbsparray` & **initCAT** (const char *F, int L, const `wbvector`< const `wbsparray` * > &X, unsigned dim)
- `wbsparray` & **initCAT** (const char *F, int L, const `wbvector`< `wbsparray` > &X, unsigned dim)
- `wbsparray` & **Cat** (const char *F, int L, const `wbsparray` &B, unsigned dim)
- `wbsparray` & **init_kron** (const `wbvector`< `SPIDX_T` > &sa, const `wbvector`< `SPIDX_T` > &sb, `SPIDX_T` nab, char kflag)
- void **setSize_kron** (const char *F, int L, const `wbvector`< `SPIDX_T` > &sa, const `wbvector`< `SPIDX_T` > &sb, char kflag=0)
- void **setSize_kron** (const char *F, int L, char kflag=0)
- `wbsparray` & **setRec** (`SPIDX_T` i, `SPIDX_T` j)
- `template<class T2 >`
`wbsparray` & **setRec** (`SPIDX_T` i, `SPIDX_T` i1, `SPIDX_T` i2, const T2 &x)
- `wbsparray` & **setRecP** (`SPIDX_T` i, const `SPIDX_T` *l, const TD &x)
- `wbsparray` & **setRec_kron** (`SPIDX_T` l, TD x, unsigned r, `SPIDX_T` *i1, `SPIDX_T` *i2, unsigned kflag=0)
- `wbsparray` & **Resize** (`SPIDX_T` l)
- `wbsparray` & **REDSIZE_NNZ** (`SPIDX_T` l)
- `wbsparray` & **Resize** (const `wbvector`< `SPIDX_T` > &S)
- `wbsparray` & **operator=** (const `wbsparray` &a)
- `template<class DB >`
`wbsparray` & **operator=** (const `wbsparray`< DB > &a)

- [wbsparray](#) & **save2** ([wbsparray](#) &a, char ref=0)
- SPIDX_T **sub2ind** (SPIDX_T k) const
- void **ind2sub** (SPIDX_T k, SPIDX_T *l) const
- void **ind2sub** (SPIDX_T k, [wbindex](#) &l) const
- [wbsparray](#)< TD > & **make2D** (const [wbindex](#) &ic, [wbsparray](#)< TD > &B, char pos=1) const
- **sparseIndex2D**< SPIDX_T > & **toIndex2D** (const [wbindex](#) &ic, **sparseIndex2D**< SPIDX_T > &a, char pos, char use_rmaj=0) const
- [wbsparray](#) & **initDiag** (SPIDX_T d)
- [wbsparray](#) & **initDiag** (const [wbvector](#)< TD > &dd)
- [wbsparray](#) & **initDiag2** (SPIDX_T d, TD dval=1)
- bool **isDiagMatrix** (TD eps=1E-14) const
- bool **isDiag** (const char *F=0, int L=0) const
- [wbvector](#)< TD > & **getDiag** (const char *F, int L, [wbvector](#)< TD > &dd) const
- TD **getDiag** (const char *F, int L, SPIDX_T k) const
- [wbsparray](#) & **getDiag** (const char *F, int L, [wbsparray](#) &A) const
- [wbsparray](#) & **diag2reg** (const char *F=0, int L=0, unsigned r=-1)
- [wbsparray](#) & **diag2reg** (const char *F, int L, [wbsparray](#) &A, unsigned r=-1) const
- [wbsparray](#) & **toDiag** (const char *F=0, int L=0)
- int **toDiagSafe** (const char *F, int L, TD eps=1E-14)
- [wbsparray](#) & **initIdentity** (SPIDX_T d, TD dval=1)
- [wbsparray](#) & **initIdentity** (SPIDX_T d1, SPIDX_T d2, sSPIDX_T m=0, TD dval=1)
- [wbsparray](#) & **initIdentity** (const [wbvector](#)< SPIDX_T > &S0, int m=-1, sSPIDX_T k=0, char fflag='f')
- [wbsparray](#) & **initIdentityB3** (SPIDX_T d1, SPIDX_T d2, SPIDX_T D, sSPIDX_T i0=0, char fflag='f')
- unsigned **rank** (const char *F=NULL, int L=0) const
- SPIDX_T **dim** () const
- SPIDX_T **dim** (unsigned k) const
- SPIDX_T **dim0** (unsigned k) const
- SPIDX_T **matDim** (unsigned k=1)
- SPIDX_T **maxDim** (unsigned r=-1) const
- TD **norm2** (char tnorm=0) const
- [wbstring](#) **sizeStr** () const
- TD **SkipTinyCols** (TD eps=1E-14)
- TD **SkipTiny** (TD eps=1E-14)
- bool **isVector** (const char *F=0, int L=0) const
- bool **isMatrix** (const char *F=0, int L=0) const
- bool **isRank** (unsigned r) const
- bool **isSMatrix** (const char *F=0, int L=0, SPIDX_T *d=NULL) const
- bool **isSMatrix** (const char *F, int L, SPIDX_T d) const
- bool **isEmpty** () const
- bool **isEmpty_** () const
- **operator bool** () const
- bool **operator!** () const
- bool **isZero** (TD eps=0, char flag=0) const
- SPIDX_T **nnz** () const
- SPIDX_T **nnz** (const TD eps1, const TD eps2=1E9) const
- SPIDX_T **nnz_diag** () const
- SPIDX_T **numel** () const
- TD **norm** (char tnorm=0) const
- template<class T2 >
 [wbvector](#)< T2 > & **norm2vec** (unsigned k, [wbvector](#)< T2 > &A, char tnorm=0) const
- void **swap** ([wbsparray](#) &B)
- [wbsparray](#) & **TimesEI** (const [wbsparray](#) &B, char tnorm=0)
- TD **dotProd** (const char *F, int L, const [wbsparray](#) &B, char tnorm=0) const
- TD **froNorm2** (const [wbsparray](#) &B) const
- double **normDiff2** (const [wbsparray](#) &B) const

- double **normDiff** (const wbsparray &B) const
- TD **Normalize** (char tnorm=0, char qflag=0)
- TD **NormalizeCol** (SPIDX_T k, char tnorm=0, char qflag=0)
- wbsparray & **QRdecomp** (const char *F, int L, wbsparray &Q, char posR=0, TD eps=1E-14)
- wbsparray & **OrthoNormalizeColsQR** (const char *F=NULL, int L=0, TD eps=1E-14)
- wbsparray & **OrthoNormalizeColsGS** (const char *F=NULL, int L=0, char tnorm=0, char qxflag=0, TD eps=1E-14, unsigned np=1)
- wbsparray & **setCol** (const char *F, int L, SPIDX_T k, const wbsparray &v)
- wbsparray & **setCol** (SPIDX_T k, const wbsparray &v)
- double **sparsity** () const
- int **checkTrailingSingletons** (const char *F=0, int L=0, unsigned *r=NULL) const
- int **SkipTrailingSingletons** (const char *F=0, int L=0, unsigned r=-1)
- int **skipTrailingSingletons** (const char *F, int L, wbsparray &c, unsigned r=-1) const
- bool **hasSingletons** (const wbindex &l) const
- wbsparray & **skipSingletons** (const char *F, int L, const wbindex &l, wbsparray &A) const
- wbsparray & **AddTrailingSingletons** (const char *F, int L, unsigned r)
- wbsparray & **initScalar** (const TD x=0, unsigned r=-1)
- char **isScalar** (const char *F=0, int L=0, char zflag=0) const
- wbsparray & **toScalar** (const char *F=0, int L=0, unsigned r=-1)
- TD **getScalar** (const char *F=0, int L=0) const
- char **checkSize** (const char *F=0, int L=0, const char *fct=0, const char *istr=0) const
- char **checkSize2** (const char *F, int L, const wbsparray &B) const
- void **check_IDX_range** (const char *F=0, int L=0) const
- bool **isProptold** () const
- bool **isProptold** (TD &q, TD eps=1E-14) const
- bool **isIdentityMatrix** (TD eps=1E-14) const
- char **isIdentity** (TD eps=1E-14) const
- char **isIdentity** (const wbpem &P, TD *dval=NULL, TD eps=1E-14) const
- bool **hasSize** (SPIDX_T d1) const
- bool **hasSize** (SPIDX_T d1, SPIDX_T d2) const
- bool **hasSize** (SPIDX_T d1, SPIDX_T d2, SPIDX_T d3) const
- template<class T >
wvector< T > & **getSize** (wvector< T > &S) const
- char **fitsSize** (const wbsparray &, char strict=0) const
- char **sameSize** (const wbsparray &, char strict=0) const
- char **sameSizeUp2Singletons** (const wvector< SPIDX_T > &S, char strict=0) const
- char **matchWithSingletons** (const char *F, int L, const wvector< SPIDX_T > &S, wvector< int > &M) const
- char **sameUptoFac** (const wbsparray &B, TD *fac=NULL, char lex=-1, TD eps=1E-12) const
- bool **operator==** (const wbsparray &B) const
- bool **operator!=** (const wbsparray &B) const
- wbsparray & **Plus** (const char *F, int L, const wbsparray &B, TD bfac=1, TD afac=1, TD eps=0)
- TD **sum** () const
- TD **aMin** () const
- TD **aMax** () const
- SPIDX_T **findRecSortedP** (SPIDX_T *idx, SPIDX_T n, char lex=-1)
- wbsparray & **Eldiv** (const char *F, int L, const wvector< TD > &x_)
- wbsparray & **operator+=** (const wbsparray &B)
- wbsparray & **operator-=** (const wbsparray &B)
- wbsparray & **Conj** ()
- wbsparray & **operator*=** (TD x)
- wbsparray & **operator/=** (TD x)
- const TD & **operator[]** (SPIDX_T i) const
- TD & **operator[]** (SPIDX_T i)
- TD **value** (const SPIDX_T *idx, SPIDX_T n, char lex=-1) const

- [wbstring](#) **info2Str** (const char *istr=0) const
- [size_t](#) **memSize** () const
- void **info** (const char *istr="") const
- void **info** (const char *F, int L, const char *s="") const
- void **print** (const char *istr="", const char *fmt="") const
- void **print** (const char *F, int L, const char *istr="", const char *fmt="") const
- void **printdata** (const char *istr="ans", const char *fmt="") const
- void **sperror_this** (const char *F, int L, const char *fct=0, const char *istr=0) const
- TD **trace** () const
- template<class T2 >
[wbvector](#)< T2 > & **trace** (unsigned k, [wbvector](#)< T2 > &A) const
- bool **isSym_aux** (const char *file, int line, const char *fct, const [wbsparray](#) &B, TD eps, TD *xref, const char symflag='s', const char lflag=1) const
- bool **isHConj** (const [wbsparray](#) &B, TD eps=1E-12, TD *xref=NULL) const
- bool **isHConj** (TD eps=1E-12, TD *xref=NULL) const
- bool **isAHerm** (const [wbsparray](#) &B, TD eps=1E-12, TD *xref=NULL) const
- bool **isAHerm** (TD eps=1E-12, TD *xref=NULL) const
- [wbsparray](#) & **Sort** (char lex=-1)
- double **Compress** (const char *F=0, int L=0, TD eps=0, char lex=-1, const [wbperm](#) *P=NULL)
- bool **isCompressed** (char lex=-1, char dir=0) const
- int **splitSparseCM** (const char *F, int L, [wbvector](#)< [wbsparray](#) > &X, TD eps=0, char ref=0, char tnorm=0)
- [wbsparray](#) & **reshape** (const [wbvector](#)< SPIDX_T > &S, [wbsparray](#) &A) const
- [wbsparray](#) & **Reshape** (const [wbvector](#)< SPIDX_T > &S)
- [wbsparray](#) & **Reshape** (SPIDX_T s1, SPIDX_T s2)
- [wbsparray](#) & **Reshape** (SPIDX_T s1, SPIDX_T s2, SPIDX_T s3)
- [wbvector](#)< TD > & **getCol** (SPIDX_T k, [wbvector](#)< TD > &v) const
- [wbvector](#)< TD > & **getRow** (SPIDX_T k, [wbvector](#)< TD > &v) const
- [wbsparray](#) & **getBlock** (const char *F, int L, SPIDX_T i1, SPIDX_T i2, SPIDX_T j1, SPIDX_T j2, [wbsparray](#)< TD > &B) const
- [wbsparray](#) & **getCol** (SPIDX_T k, [wbsparray](#)< TD > &B)
- [wbsparray](#) & **splitBlock** (const char *F, int L, sSPIDX_T i1, sSPIDX_T i2, sSPIDX_T j1, sSPIDX_T j2, [wbsparray](#)< TD > &B)
- [wbsparray](#) & **SplitQR** (sSPIDX_T k, sSPIDX_T k_, [wbsparray](#)< TD > &X)
- TD **getHouseholderVec** (const char *F, int L, SPIDX_T k, SPIDX_T j, [wbsparray](#)< TD > &b, TD eps2=0) const
- [wbsparray](#) & **IDX_Shift** (sSPIDX_T i1, sSPIDX_T i2)
- [wbsparray](#)< TD > & **LProject1** (const [wbsparray](#)< TD > &b, [wbsparray](#)< TD > &X, TD xfac=1, char tnorm=0) const
- SPIDX_T **LContractVec_omp** (const [wbsparray](#)< TD > &b, [wbvector](#)< TD > &x, char tnorm) const
- [wbsparray](#) & **splitLast** (SPIDX_T k, [wbsparray](#) &a) const
- [wbsparray](#) & **select0** (const WBINDEX &, unsigned dim, [wbsparray](#) &) const
- [wbsparray](#) & **Select0** (const WBINDEX &I, unsigned dim)
- [wbsparray](#) & **Permute** (const char *s, char iflag=0)
- [wbsparray](#) & **Permute** (const [wbperm](#) &, char iflag=0)
- [wbsparray](#) & **permute** (const char *s, [wbsparray](#) &, char iflag=0) const
- [wbsparray](#) & **permute** (const [wbperm](#) &, [wbsparray](#) &, char iflag=0) const
- [wbsparray](#) & **transpose** (const char *F, int L, [wbsparray](#) &A) const
- [wbsparray](#) & **Transpose** (const char *F=0, int L=0)
- [wbsparray](#) & **htranspose** (const char *F, int L, [wbsparray](#) &A) const
- [wbsparray](#) & **hTranspose** (const char *F=0, int L=0)
- [wbsparray](#) & **ColPermute** (const [wbperm](#) &P, char iflag=0)
- [wbsparray](#) & **MatPermute** (const [wbperm](#) &P, char iflag=0)
- [wbsparray](#) & **mtimes** (const char *F, int L, const [wbsparray](#) &B, [wbsparray](#) &C, char sflag=0, const TD eps=0)
- char **contract_scalar** (const char *F, int L, const [ctrlidx](#) &ica, const [wbsparray](#) &B, const [ctrlidx](#) &icb, [wbsparray](#) &C, const [wbperm](#) &pfinal=[wbperm](#)(), const TD &afac=1., const TD &cfac=0.) const

- `wbsparray` & `contract_diag_diag` (const char *F, int L, const `ctrlidx` &ica, const `wbsparray` &B, const `ctrlidx` &icb, `wbsparray` &C, const `wbperm` &pfinal, const TD &afac, const TD &cfac) const
- char `contract_check_2full` (const char *F, int L, const `ctrlidx` &ica, const `wbsparray` &B, const `ctrlidx` &icb, `wbsparray` &C, const `wbperm` &pfinal, const TD &afac, const TD &cfac, char flag, TD eps, SPIDX_T *nnz_↵ C=NULL) const
- `wbsparray` & `contract` (const char *F, int L, const `ctrlidx` &ica, const `wbsparray` &B, const `ctrlidx` &icb, `wbsparray` &C, const `wbperm` &pfinal=`wbperm`(), const TD &afac=1., const TD &cfac=0., char sflag=0, const TD eps=0) const
- `wbsparray` & `contract` (const char *F, int L, const char *ica, const `wbsparray` &B, const char *icb, `wbsparray` &C, const `wbperm` &pfinal=`wbperm`(), const TD &afac=1., const TD &cfac=0., char sflag=0, const TD eps=0) const
- `wbarray< TD >` & `contract` (const char *F, int L, const `ctrlidx` &ica, const `wbsparray` &B, const `ctrlidx` &icb, `wbarray< TD >` &C, const `wbperm` &pfinal, const TD &afac, const TD &cfac) const
- `wbsparray` & `contract` (const char *F, int L, unsigned i1, const `wbvector< TD >` &B, `wbsparray` &C) const
- `wbsparray` & `comm` (const char *F, int L, const `wbsparray` &B, `wbsparray` &C, char aflag='N', char bflag='N') const
- `wbsparray` & `tensorProdX` (const char *F, int L, const `wbsparray` &B, `wbsparray` &C, char conja=0, const TD &afac=1, char conjb=0, const TD &cfac=0, const `wbperm` *Pfinal=NULL) const
- `wbsparray` & `tensorProd` (const char *F, int L, const `wbsparray` &B, `wbsparray` &C, char aflag='N', char bflag='N', char kflag=0) const
- `wbsparray` & `Kron` (const `wbsparray` &B, char aflag='N', char bflag='N', char kflag='k')
- `wbsparray` & `kron` (const char *F, int L, const `wbsparray` &B, `wbsparray` &C, char aflag='N', char bflag='N', char kflag='k') const
- `mxArray` * `toMxSp` () const
- `mxArray` * `toMx` (int m=-99) const
- `mxArray` * `IDtoMx` () const
- `mxArray` * `mxCreateStruct` (unsigned m, unsigned n) const
- void `add2MxStruct` (`mxArray` *a, unsigned i, int ma=-99) const
- `mxArray` * `mxCreateCell` (unsigned m, unsigned n) const
- void `add2MxCell` (`mxArray` *a, unsigned i) const
- `mxArray` * `toMx0` (int m=-99) const
- `mxArray` * `mxCreateStructX` (unsigned m, unsigned n, int ma=-99) const
- void `add2MxStructX` (`mxArray` *a, unsigned i, int ma=-99) const
- `mxArray` * `toMX` () const
- `mxArray` * `mxCreateSTRUCT` (unsigned m, unsigned n) const
- `mxArray` * `add2MxSTRUCT` (`mxArray` *a, unsigned i) const
- void `put` (const char *vname, unsigned ma=-99, const char *ws="caller") const
- void `put0` (const char *F, int L, const char *vname, const char *ws="caller") const
- void `put` (const char *F, int L, const char *vname, unsigned ma=-99, const char *ws="caller", char flag=0) const
- template<class T2 >
 `wbvector< T2 >` & `toFull` (`wbvector< T2 >` &A) const
- template<class DB >
 `wbsparray< TD >` & `init` (const char *F, int L, SPIDX_T d1, SPIDX_T d2, const `wbsparray< DB >` &B)
- template<class DB >
 `wbsparray< TD >` & `init` (const char *F, int L, SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, const `wbsparray< DB >` &B)
- template<class DB >
 `wbsparray< TD >` & `init` (const char *F, int L, SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T d4, const `wbsparray< DB >` &B)

Public Attributes

- `wbvector< SPIDX_T >` **SIZE**
- `wbMatrix< SPIDX_T >` **IDX**
- `wbvector< TD >` **D**
- char **isref**

Protected Member Functions

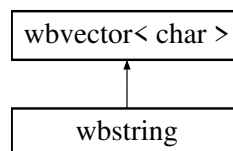
- unsigned **get2DSize** (const char *F, int L, SPIDX_T &d1, SPIDX_T &d2, int m=-1, char fflag=0) const
- unsigned **get2DIndex** ([wbMatrix](#)< SPIDX_T > &IJ, SPIDX_T &d1, SPIDX_T &d2, int m=-1) const

The documentation for this class was generated from the following files:

- wblib.h
- wbsparray.hh
- wbsparray.cc

4.113 wbsting Class Reference

Inheritance diagram for wbsting:



Public Member Functions

- **wbsting** (unsigned n=0)
- **wbsting** (unsigned n, const char c)
- **wbsting** (const char *s, unsigned n)
- **wbsting** (const [wbstring](#) &s)
- **wbsting** (const char *s1)
- **wbsting** (const char *s1, const char *s2, const char *s3=NULL, const char *s4=NULL)
- **wbsting** (const mxArray *a)
- [wbstring](#) & **init** (const mxArray *a)
- **wbsting** (const char *F, int L, const mxArray *a)
- [wbstring](#) & **init** (unsigned n=0)
- [wbstring](#) & **init** (const char *s)
- [wbstring](#) & **init** (const char *s, unsigned n)
- [wbstring](#) & **init** (const char *F, int L, const mxArray *a)
- [wbstring](#) & **initrep** (const char *s, unsigned n)
- [wbstring](#) & **initrep** (char c, unsigned n)
- void **set** (char c)
- [wbstring](#) & **operator=** (const char *s)
- [wbstring](#) & **operator=** (const [wbstring](#) &s)
- bool **gotTerm0** ()
- bool **strchr** (const char *s)
- [wbstring](#) & **push** (const char *F, int L, const char *s, char extend=1)
- [wbstring](#) & **push** (const char *s, char extend=1)
- [wbstring](#) & **pushf** (const char *F, int L, const char *fmt,...)
- [wbstring](#) & **cpy** (const char *F, int L, const char *s)
- int **RegEx_replace** (const char *F, int L, const char *pat, const char *rep, char icode=0, char gflag=1)
- int **printf** (const char *F, int L, const char *fmt,...)
- [wbstring](#) & **time** ()

- [wstring](#) & **time** (const time_t &t)
- [wstring](#) & **time_sys** ()
- [wstring](#) & **time_sys** (const time_t &t)
- [wstring](#) & **Upper** ()
- [wstring](#) & **Lower** ()
- [wstring](#) **toupper** () const
- [wstring](#) **tolower** () const
- [wstring](#) & **Chomp** ()
- char **getopt** (char c)
- char **getopt** ()
- const char * **basename** (const char c="/") const
- mxArray * **toMx** () const
- size_t **toHash** (unsigned offset=0) const
- void **add2MxStruct** (mxArray *S, unsigned i) const
- template<class T >
char * **init2Fmt** (const T &x QS_UNUSED_VAR, int n=-1, int p=-1)
- template<class T >
[wstring](#) & **operator**<< (const T *x)
- template<class T >
[wstring](#) & **operator**<< (const T &x)
- [wstring](#) **operator**+ (const [wstring](#) &s) const
- [wstring](#) **operator**+ (const char *s) const
- [wstring](#) & **operator**+= (const char *s)
- [wstring](#) & **operator**+= (const [wstring](#) &s)
- unsigned **cat** (unsigned &k, const char *s)
- [wstring](#) **toStr** (char level=0) const
- char & **operator**[] (unsigned i) const
- unsigned **strlen_** () const
- bool **isEmpty** () const
- char **isPrint** () const
- bool **operator!** () const
- **operator** bool () const
- unsigned **isName** (int L=-1) const
- int **operator**== (const char *s) const
- int **operator**!= (const char *s) const
- int **operator**== (const [wstring](#) &s) const
- int **operator**!= (const [wstring](#) &s) const
- int **operator**< (const [wstring](#) &s) const
- int **operator**<= (const [wstring](#) &s) const
- int **operator**> (const [wstring](#) &s) const
- int **operator**>= (const [wstring](#) &s) const
- int **firstLocation** (const [wstring](#) &, unsigned k=0)
- int **firstLocation** (const char *, unsigned offset=0)
- std::istream & **getline** (std::istream &istr, char c='\n')
- template<> [wstring](#) & **operator**<< (const char *s)
- template<> [wstring](#) & **operator**<< (const char &x)
- template<> [wstring](#) & **operator**<< (const int &x)
- template<> [wstring](#) & **operator**<< (const unsigned &x)
- template<> [wstring](#) & **operator**<< (const long &x)
- template<> [wstring](#) & **operator**<< (const unsigned long &x)
- template<> [wstring](#) & **operator**<< (const double &x)
- template<> [wstring](#) & **operator**<< (const float &x)
- template<> [wstring](#) & **operator**<< (const [wbcomplex](#) &x)

Public Member Functions inherited from [wbvector< char >](#)

- [wbvector](#) (size_t l, char x)
- [wbvector](#) (size_t l, const char *d0=NULL, char ref=0)
- [wbvector](#) (const [wbvector](#) &v)
- [wbvector](#) (const [wbvector](#)< T2 > &v)
- [wbvector](#) (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) (const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) & [init](#) (size_t l=0)
- [wbvector](#) & [init](#) (size_t l, const char *d0)
- [wbvector](#) & [init](#) (size_t l, const char *d0, char ref)
- [wbvector](#) & [init](#) (const [wbvector](#) &v)
- [wbvector](#) & [init](#) (size_t n, const [wbvector](#) &v)
- [wbvector](#)< char > & [init](#) (const [wbsparray](#)< char > &S)
- [wbvector](#) & [init](#) (size_t l1, const char *d1, size_t l2, const char *d2)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type QS_UNUSED_↵
VAR=1, char ref=0)
- int [init](#) (const mxArray *a, const char *istr=NULL, char check_type=1)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int [init](#) (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- [wbvector](#) & [initp](#) (size_t l, const char *d0, size_t n, char init=1)
- [wbvector](#) & [init2val](#) (size_t l, const char &x)
- [wbvector](#) & [initStride](#) (size_t l, const char *d0, size_t stride)
- [wbvector](#) & [init_stride](#) (const char *d0, size_t n, size_t m, size_t D0)
- [wbvector](#) & [init_data](#) (const char *d0)
- [wbvector](#) & [init_data](#) (const T0 *d0)
- [wbvector](#) & [init_data](#) (const char *F, int L, const T0 *d0)
- [wbvector](#)< char > & [initT](#) (const [wbsparray](#)< T2 > &S)
- [wbvector](#)< char > & [initT](#) (size_t l, const T2 *v)
- [wbvector](#)< char > & [initT](#) (const [wbvector](#)< T2 > &v)
- [wbvector](#)< char > & [initT](#) (const char *F, int L, const [wbvector](#)< T2 > &v)
- [wbvector](#)< char > & [initT](#) (const [wbsparray](#)< TD > &S)
- [wbvector](#)< char > & [initT_](#) (const [wbvector](#)< T2 > &v)
- [wbvector](#) & [init2ref](#) (const [wbvector](#) &v)
- [wbvector](#) & [init2ref](#) (size_t l, const char *d0)
- [wbvector](#) & [unRef](#) ()
- int [init_Struct](#) (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)
- int [init_base](#) (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)
- int [init_mpfr](#) (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)
- void [initDef](#) (size_t n)
- void [initDef](#) (const [wbvector](#) &v)
- [wbvector](#) & [operator=](#) (const [wbvector](#) &v)
- [wbvector](#)< char > & [operator=](#) (const [wbvector](#)< TB > &v)
- [wbvector](#) & [set](#) (const char &x)
- [wbvector](#) & [set](#) (const [wbvector](#)< size_t > &l, const char &x)
- void [set](#) (size_t offset, const char *x, size_t n)
- void [set](#) (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- void [set](#) (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- [wbvector](#) & [setMin](#) (const char &x)
- [wbvector](#) & [setMax](#) (const char &x)

- [wvector](#) & **add** (const [wvector](#)< size_t > &l, const char &x)
- [wvector](#) & **add** (const char *d)
- [wvector](#) & **Set** (const [wvector](#)< size_t > &l, const [wvector](#) &v)
- void **Sort** ([wbperm](#) &p, char dir=+1)
- void **Sort** (char dir=+1)
- void **sort** ([wvector](#) &v2, [wbperm](#) &p) const
- void **sort** ([wvector](#) &v2) const
- [wbperm](#) & **pSort** ([wbperm](#) &p) const
- bool **isSorted** () const
- bool **isSortedU** () const
- bool **isUnique** () const
- int **contains** (const char &x) const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isFinite** () const
- size_t **find1** (const char &x) const
- size_t **find1** () const
- size_t **count** (const char &x) const
- size_t **icount** (const char &x) const
- size_t **nnz** () const
- size_t **nnz** (char eps) const
- size_t **numZeros** (char eps=char(1E-14)) const
- size_t **numEQ** (const char &) const
- size_t **numGT** (const char &) const
- size_t **numLT** (const char &) const
- [windex](#) & **find** (const char &x, [windex](#) &l, char iflag=0) const
- [windex](#) & **find** (const char &x, [windex](#) &l, [windex](#) &lx) const
- [windex](#) & **find** ([windex](#) &l) const
- [windex](#) & **find** ([windex](#) &l, [windex](#) &lx) const
- [wvector](#)< size_t > **find** () const
- [windex](#) & **findGT** (const char &x, [windex](#) &l, char iflag=0) const
- [windex](#) & **findGT** (const char &x, [windex](#) &l, [windex](#) &lx) const
- [windex](#) & **findzeros** ([windex](#) &l) const
- [windex](#) & **findRange** (const char &x1, const char &x2, [windex](#) &l, const char *w=NULL) const
- int **findClosestSorted** (char r) const
- int **findClosest** (char r) const
- size_t **findValues** (const [wvector](#) &v, [windex](#) &l) const
- [wvector](#) & **setp** (const char *x)
- void **reset** ()
- void **cpy** (const char *v)
- void **cpy** (const char *x, size_t m)
- [wvector](#) & **save2** ([wvector](#) &v)
- void **swap** ([wvector](#) &v)
- [wvector](#) & **swap** (size_t i, size_t j)
- [wvector](#) & **Collect** (char &A1, char &A2)
- [wvector](#) & **Collect** (char &A1, char &A2, char &A3)
- [wvector](#) & **Skip** (size_t)
- [wvector](#) & **Skip** (const WBINDEX &, char rflag=0)
- [wvector](#) & **Move2_FE** (const WBINDEX &l, char FE_flag)

- [wbvector](#) & **Move2front** (const WBINDEX &l)
- [wbvector](#) & **Move2front** (size_t k)
- [wbvector](#) & **Move2end** (const WBINDEX &l)
- [wbvector](#) & **Move2end** (size_t k)
- [wbvector](#) & **mvLastTo** (size_t k)
- [wbvector](#) & **mvFirstTo** (size_t k)
- bool **checkSameLength** (const [wbvector](#) &a, const char *F=NULL, int L=0, const char *istr="") const
- bool **checkSameLength** (const [wbvector](#) &a, const [wbvector](#) &b, const char *F=NULL, int L=0, const char *istr="") const
- void **operator+=** (const T_ &a)
- void **operator+=** (const [wbvector](#) &b)
- void **operator-=** (const T_ &a)
- void **operator-=** (const [wbvector](#) &b)
- [wbvector](#) & **operator*=** (const T_ &a)
- [wbvector](#) & **operator/=** (const T_ &a)
- [wbvector](#) **operator*** (const T_ &a) const
- [wbvector](#) **operator+** (const T_ &a) const
- [wbvector](#) **operator+** (const [wbvector](#) &b) const
- [wbvector](#) **operator-** (const T_ &a) const
- [wbvector](#) **operator-** (const [wbvector](#) &b) const
- [wbvector](#) & **Plus** (const char *b)
- [wbvector](#) & **Plus** (const char *b, char bfac)
- [wbvector](#) & **Plus** (const [wbvector](#) &b, char bfac=1)
- void **plus** (const [wbvector](#) &b, [wbvector](#) &c, char bfac=1, char cfac=0) const
- [wbvector](#) & **times** (char fac, [wbvector](#) &x) const
- [wbvector](#) & **RevertSigns** ()
- int **isum** () const
- char **sum** () const
- char **sum** (size_t i1, const char *estr) const
- char **sum** (size_t i1, size_t i2) const
- char **cumsum_** ([wbvector](#) &S, char xflag=0) const
- char **Cumsum** (char zflag=0) const
- char **cumsum0prod** ([wbvector](#)< T2 > &b, [wbvector](#)< char > &cs, char xflag) const
- [wbvector](#) & **Eldiv** (const [wbvector](#) &v) const
- [wbvector](#) & **tensorProd** (const [wbvector](#) &v2, [wbvector](#) &vv) const
- char **scalarProd** (const [wbvector](#) &v) const
- char **scalarProd** (const char *v) const
- char **norm** () const
- char **norm2** () const
- double **normDiff** (const [wbvector](#) &v, char fac=1) const
- char **normDiff2** (const [wbvector](#) &v, char fac=1) const
- double **normDiff2_** (const [wbvector](#) &v, double fac=1) const
- char **GSPProject** (const char *v, char isnorm=0, char tflag=0) const
- [wbvector](#) & **Normalize** (const char *F=NULL, int L=0)
- [wbvector](#) & **subtract** (const char *d)
- void **getReal** ([wbvector](#)< double > &R) const
- void **getReal** ([wbvector](#)< double > &R) const
- void **getImag** ([wbvector](#)< double > &I) const
- void **getImag** ([wbvector](#)< double > &I) const
- [wbvector](#) & **Conj** ()
- [wbvector](#)< [wbcomplex](#) > & **Conj** ()
- [wbvector](#) & **applyFlag** (char &flag, const char &fac, [wbvector](#) &a)
- bool **allEqual** () const
- bool **allEqual** (const char &a) const
- bool **allZero** () const

- bool **allOnes** () const
- bool **allEqual2** (const char &a, char eps=1E-15) const
- bool **allUnequal** (const char &a) const
- bool **any** () const
- bool **anyUnequal** (const char &a) const
- bool **anyUnequal** (const [wbvector](#)< size_t > &l, const char &x, size_t *k=NULL) const
- bool **anyEqual** (const char &a, size_t *i1=NULL) const
- bool **anyEqual** (const [wbvector](#)< size_t > &l, const char &x, size_t *k=NULL) const
- bool **anyGT** (const char &a) const
- bool **anyGE** (const char &a) const
- bool **anyLT** (const char &a) const
- bool **anyLE** (const char &a) const
- bool **allGT** (const char &a) const
- bool **allGE** (const char &a) const
- bool **allLE** (const char &a) const
- bool **allLT** (const char &a) const
- bool **allIn** (const char &a1, const char &a2) const
- bool **anySmallVals** (const char eps=1E-15) const
- bool **isEqual** (const char *v) const
- bool **isEqual** (const [wbvector](#) &v) const
- bool **isEmpty** () const
- **operator bool** () const
- bool **operator!** () const
- bool **operator==** (const [wbvector](#) &v) const
- bool **operator!=** (const [wbvector](#) &v) const
- bool **isEqualR** (const [wbvector](#) &v) const
- bool **operator^** (const [wbvector](#) &v) const
- bool **deepEqualP** (const [wbvector](#) &B) const
- char **cmp** (const [wbvector](#) &v) const
- bool **operator<** (const [wbvector](#) &v) const
- int **operator<** (const char &r) const
- bool **operator<=** (const [wbvector](#) &v) const
- bool **operator>** (const [wbvector](#) &v) const
- bool **operator>=** (const [wbvector](#) &v) const
- char & **el** (size_t i) const
- char **elx** (size_t i, const char &x) const
- const char & **operator[]** (size_t i) const
- char & **operator[]** (size_t i)
- [wbvector](#) **operator[]** (const [wbvector](#)< size_t > &l) const
- char & **last** () const
- char & **last** (size_t &i) const
- char & **last** (TI &i_) const
- char & **p2last** () const
- char * **ref** (size_t i) const
- [wbvector](#) & **setRand** (double fac=1., double shift=0.)
- [wbvector](#)< [wbcomplex](#) > & **setRand** (double fac, double shift)
- void **setIdx** (double shift=0., double fac=1.)
- int **set2Group** (const [wbperm](#) &P, const WBINDEX &D, const char *S0=NULL, char iflag=0)
- [wbvector](#) & **Resize_** (size_t n)
- [wbvector](#) & **Shorten2** (size_t n)
- [wbvector](#) & **Resize** (size_t n, const char *d=NULL)
- [wbvector](#) & **resize** (size_t n, [wbvector](#) &X) const
- [wbvector](#) & **cat** (const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) > &vv)

- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) * > &vv)
- [wbvector](#) & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)
- [wbvector](#) & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3=0, const [wbvector](#)< T3 > *v4=0)
- [wbvector](#)< char > & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3, const [wbvector](#)< T3 > *v4)
- [wbvector](#)< char > & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)
- [wbvector](#) & **Append** (const [wbvector](#) &v2)
- [wbvector](#) & **Append** (const char &x)
- [wbvector](#) & **Append** (size_t d2, const char *x)
- [wbvector](#) **append** (const [wbvector](#) &v) const
- [wbvector](#) & **append** (const [wbvector](#) &v2, [wbvector](#) &vout) const
- void **append** (const char *x, size_t d2, [wbvector](#) &vout) const
- int **recLess** (const char *v) const
- int **recLessE** (const char *v) const
- int **recCompare** (const char *v) const
- char **max** () const
- char **max** (size_t *k) const
- char **max** (unsigned &k) const
- char **max_** (char x) const
- char **min** () const
- char **min** (size_t *k) const
- char **min** (unsigned &k) const
- char **min_** (char x) const
- char **aMax** (size_t *k=NULL) const
- char **aMax** (unsigned &k) const
- char **aMin** (char zflag=0, size_t *k=NULL) const
- char **maxneg** (size_t *k=NULL) const
- char **minpos** (size_t *k=NULL) const
- char **avg** () const
- char **std** () const
- void **range** (char &xmin, char &xmax) const
- char **range** () const
- [wbvector](#) & **Permute** (const [wbperm](#) &P, char iflag=0)
- [wbvector](#) & **permute** ([wbvector](#) &, const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) **permute** (const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) & **blockPermute** (const [wbperm](#) &P, [wbvector](#) &, char iflag=0) const
- [wbvector](#) & **BlockPermute** (const [wbperm](#) &P, char iflag=0)
- int **blockCompare** (const [wbperm](#) &pa, const [wbvector](#) &b, const [wbperm](#) &pb) const
- void **get** (const WBINDEX &l0, [wbvector](#) &v) const
- void **get** (const [wbvector](#)< size_t > &l, char *v) const
- void **get** (size_t i1, size_t i2, [wbvector](#) &v) const
- void **selectSU** (const WBINDEX &l)
- void **select** (const WBINDEX &l, char *r) const
- [wbvector](#) & **select** (const WBINDEX &l, [wbvector](#) &a) const
- [wbvector](#) **select** (const WBINDEX &l) const
- [wbvector](#) & **Select** (const WBINDEX &l)
- [wbvector](#) & **BlockSelect** (const WBINDEX &l, const WBINDEX &D)
- [wbvector](#) & **getl** (size_t k, [wbvector](#) &v) const
- [wbvector](#) & **getl** (const [wbindex](#) &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **getl** (const WBINDEX &k, [wbvector](#) &v, char uflag=1) const
- [wbvector](#) & **initl** (const [wbvector](#) &a, const [wbindex](#) &ia)
- [wbvector](#) & **initl** (const [wbvector](#) &a, const [wbindex](#) &ia, const [wbvector](#) &b, const [wbindex](#) &ib, char uflag=1)

- **wbvector** & **Flip** ()
- void **flip** (**wbvector** &a) const
- **wbvector flip** () const
- char **prod** () const
- char **prod** (char x) const
- char **prod** (size_t i, size_t i2) const
- char **prod** (size_t i, size_t i2, char x) const
- char **prod** (const T1 *l, size_t n) const
- T2 **prod** (const T1 *l, size_t n, T2 &s2) const
- void **prod2** (char &p1, char &p2, const char &p0=0) const
- void **info** (const char *istr) const
- void **print** (const char *istr, char mflag=0) const
- void **printdata** (const char *istr=0) const
- void **put** (const char *F, int L, const char *vname="ans", char tflag=0, const char *ws="base") const
- void **put** (const char *vname="ans", char tflag=0, const char *ws="base") const
- void **dispWithMsg** (const char *file, int line, const char *msg, const char *vname="") const
- void **mat2mx** (mxArray *&a, char dim=2) const
- void **mat2mxs** (mxArray *a, char field_nr, char dim=2) const
- mxArray * **toMx_Struct** (const char tflag=0) const
- mxArray * **toMxP_S** (const char tflag=0) const
- mxArray * **toMx** (const char tflag=0) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMx** (const char tflag) const
- mxArray * **toMxP** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base_d** (const char tflag=0) const
- mxArray * **toMx_offset** (const char &offset, const char tflag=0) const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tflag=0) const
- void **add2MxStruct** (mxArray *S, const char *vname, char tflag=0) const
- mxArray * **mxCreateCell** (unsigned m, unsigned n) const
- void **add2MxCell** (mxArray *S, unsigned i, char tflag=0) const
- void **putg** (const char *vname="ans", char tflag=0) const
- void **putx** (const char *vname="ans", char tflag=0) const
- **wbstring toStr** (int n, const char *sep) const
- **wbstring toStr** () const
- **wbstring toStr** (int n, const char *sep) const
- **wbstring sizeStr** () const
- **wbstring toStrf** (const char *fmt0="", const char *sep=" ", unsigned stride=0, const char *sep2="") const
- **wbstring toStrf** (const char *fmt , const char *sep, unsigned stride, const char *sep2) const

Additional Inherited Members

Public Attributes inherited from [wbvector< char >](#)

- char * **data**
- size_t **len**
- char **isref**

Protected Member Functions inherited from [wbvector< char >](#)

- [wbvector](#) & **RENEW** (size_t n, const char *d=NULL, size_t l=-1, char init=1)
- [wbvector](#) & **RENEW_VAL** (size_t n, const char &x)

The documentation for this class was generated from the following files:

- [wbstring.hh](#)
- [wbstring.cc](#)

4.114 wbttop Class Reference

Public Member Functions

- [wbstring](#) **VmSize2Str** () const
- size_t **getVmSize** () const
- [wbstring](#) **VmPeak2Str** () const
- size_t **getVmPeak** () const
- int **runningLarge** (const char *F=NULL, int L=0, double th0=0.80, double fac=1.2)
- void **MemStat** (const char *F=0, int L=0, char lflag=0) const

Public Attributes

- pid_t **pid**
- pid_t **ppid**
- size_t **systemem**

The documentation for this class was generated from the following files:

- [util.hh](#)
- [util.cc](#)

4.115 WbUtil< T > Class Template Reference

Public Member Functions

- **T eps** ()
- [wbcomplex eps](#) ()
- void **adjust_tnorm** (char &tnorm QS_UNUSED_VAR)
- constexpr char **isComplex** ()
- constexpr char **isComplex** ()
- bool **isFloat** ()
- bool **isFloat** ()
- bool **isFloat** ()
- bool **isFloat** ()
- bool **isInt** ()
- bool **isInt** ()
- bool **isInt** ()
- bool **isInt** ()
- bool **isInt** ()
- bool **isInt** ()
- double **eps** ()
- float **eps** ()
- long double **eps** ()

Static Public Member Functions

- static void **adjust_tnorm** (char &tnorm)
- static constexpr bool **isPOD** ()
- static bool **isFloat** ()
- static bool **isInt** ()
- static constexpr char **isComplex** ()
- static char **hasConj** ()
- static char **isReal** ()

The documentation for this class was generated from the following file:

- util.hh

4.116 wbvector< T > Class Template Reference

Public Member Functions

- **wbvector** (size_t l, T x)
- **wbvector** (size_t l, const T *d0=NULL, char ref=0)
- **wbvector** (const [wbvector](#) &v)
- template<class T2 >
 wbvector (const [wbvector](#)< T2 > &v)
- **wbvector** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- **wbvector** (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- **wbvector** (const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) & **init** (size_t l=0)

- [wbvector](#) & **init** (size_t l, const T *d0)
- [wbvector](#) & **init** (size_t l, const T *d0, char ref)
- [wbvector](#) & **initp** (size_t l, const T *d0, size_t n, char init=1)
- [wbvector](#) & **init2val** (size_t l, const T &x)
- [wbvector](#) & **initStride** (size_t l, const T *d0, size_t stride)
- [wbvector](#) & **init_stride** (const T *d0, size_t n, size_t m, size_t D0)
- [wbvector](#) & **init** (const [wbvector](#) &v)
- [wbvector](#) & **init** (size_t n, const [wbvector](#) &v)
- [wbvector](#) & **init_data** (const T *d0)
- template<class T0 >
 [wbvector](#) & **init_data** (const T0 *d0)
- template<class T0 >
 [wbvector](#) & **init_data** (const char *F, int L, const T0 *d0)
- [wbvector](#)< T > & **init** (const [wbsparray](#)< T > &S)
- template<class T2 >
 [wbvector](#)< T > & **initT** (const [wbsparray](#)< T2 > &S)
- template<class T2 >
 [wbvector](#)< T > & **initT** (size_t l, const T2 *v)
- template<class T2 >
 [wbvector](#)< T > & **initT** (const [wbvector](#)< T2 > &v)
- template<class T2 >
 [wbvector](#)< T > & **initT_** (const [wbvector](#)< T2 > &v)
- template<class T2 >
 [wbvector](#)< T > & **initT** (const char *F, int L, const [wbvector](#)< T2 > &v)
- [wbvector](#) & **init2ref** (const [wbvector](#) &v)
- [wbvector](#) & **init2ref** (size_t l, const T *d0)
- [wbvector](#) & **unRef** ()
- [wbvector](#) & **init** (size_t l1, const T *d1, size_t l2, const T *d2)
- int **init** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type QS_UNUSED_↵
 VAR=1, char ref=0)
- int **init_Struct** (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)
- int **init_base** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)
- int **init** (const mxArray *a, const char *istr=NULL, char check_type=1)
- int **init_mpf** (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)
- void **initDef** (size_t n)
- void **initDef** (const [wbvector](#) &v)
- [wbvector](#) & **operator=** (const [wbvector](#) &v)
- template<class TB >
 [wbvector](#)< T > & **operator=** (const [wbvector](#)< TB > &v)
- [wbvector](#) & **set** (const T &x)
- [wbvector](#) & **setMin** (const T &x)
- [wbvector](#) & **setMax** (const T &x)
- [wbvector](#) & **set** (const [wbvector](#)< size_t > &l, const T &x)
- [wbvector](#) & **add** (const [wbvector](#)< size_t > &l, const T &x)
- [wbvector](#) & **Set** (const [wbvector](#)< size_t > &l, const [wbvector](#) &v)
- void **Sort** ([wbperm](#) &p, char dir=+1)
- void **sort** ([wbvector](#) &v2, [wbperm](#) &p) const
- void **Sort** (char dir=+1)
- void **sort** ([wbvector](#) &v2) const
- [wbperm](#) & **pSort** ([wbperm](#) &p) const
- bool **isSorted** () const
- bool **isSortedU** () const
- bool **isUnique** () const
- int **contains** (const T &x) const
- bool **isNormal** () const

- `bool isFinite () const`
- `size_t find1 (const T &x) const`
- `size_t find1 () const`
- `size_t count (const T &x) const`
- `size_t icount (const T &x) const`
- `size_t nnz () const`
- `size_t nnz (T eps) const`
- `size_t numZeros (T eps=T(1E-14)) const`
- `size_t numEQ (const T &) const`
- `size_t numGT (const T &) const`
- `size_t numLT (const T &) const`
- `wbindex & find (const T &x, wbindex &l, char iflag=0) const`
- `wbindex & find (const T &x, wbindex &l, wbindex &lx) const`
- `wbindex & findGT (const T &x, wbindex &l, char iflag=0) const`
- `wbindex & findGT (const T &x, wbindex &l, wbindex &lx) const`
- `wbindex & findzeros (wbindex &l) const`
- `wbindex & find (wbindex &l) const`
- `wbindex & find (wbindex &l, wbindex &lx) const`
- `wbvector< size_t > find () const`
- `wbindex & findRange (const T &x1, const T &x2, wbindex &l, const char *w=NULL) const`
- `int findClosestSorted (T r) const`
- `int findClosest (T r) const`
- `size_t findValues (const wbvector &v, wbindex &l) const`
- `wbvector & setp (const T *x)`
- `void set (size_t offset, const T *x, size_t n)`
- `void reset ()`
- `void cpy (const T *v)`
- `void cpy (const T *x, size_t m)`
- `wbvector & save2 (wbvector &v)`
- `void swap (wbvector &v)`
- `wbvector & swap (size_t i, size_t j)`
- `wbvector & Collect (T &A1, T &A2)`
- `wbvector & Collect (T &A1, T &A2, T &A3)`
- `wbvector & Skip (size_t)`
- `wbvector & Skip (const WBINDEX &, char rflag=0)`
- `wbvector & Move2_FE (const WBINDEX &l, char FE_flag)`
- `wbvector & Move2front (const WBINDEX &l)`
- `wbvector & Move2end (const WBINDEX &l)`
- `wbvector & Move2front (size_t k)`
- `wbvector & Move2end (size_t k)`
- `wbvector & mvLastTo (size_t k)`
- `wbvector & mvFirstTo (size_t k)`
- `bool checkSameLength (const wbvector &a, const char *F=NULL, int L=0, const char *istr="") const`
- `bool checkSameLength (const wbvector &a, const wbvector &b, const char *F=NULL, int L=0, const char *istr="") const`
- `template<class T_ >`
`void operator+= (const T_ &a)`
- `template<class T_ >`
`void operator-= (const T_ &a)`
- `template<class T_ >`
`wbvector & operator*= (const T_ &a)`
- `template<class T_ >`
`wbvector & operator/= (const T_ &a)`
- `template<class T_ >`
`wbvector operator* (const T_ &a) const`

- `template<class T_ >`
`wbvector operator+ (const T_ &a) const`
- `template<class T_ >`
`wbvector operator- (const T_ &a) const`
- `wbvector operator+ (const wbvector &b) const`
- `wbvector operator- (const wbvector &b) const`
- `void operator+= (const wbvector &b)`
- `void operator-= (const wbvector &b)`
- `wbvector & Plus (const T *b)`
- `wbvector & Plus (const T *b, T bfac)`
- `wbvector & Plus (const wbvector &b, T bfac=1)`
- `void plus (const wbvector &b, wbvector &c, T bfac=1, T cfac=0) const`
- `wbvector & times (T fac, wbvector &x) const`
- `wbvector & RevertSigns ()`
- `int isum () const`
- `T sum () const`
- `T sum (size_t i1, const char *estr) const`
- `T sum (size_t i1, size_t i2) const`
- `T cumsum_ (wbvector &S, char xflag=0) const`
- `T Cumsum (char zflag=0) const`
- `template<class T2 >`
`T cumsum0prod (wbvector< T2 > &b, wbvector< T > &cs, char xflag) const`
- `wbvector & Eldiv (const wbvector &v) const`
- `wbvector & tensorProd (const wbvector &v2, wbvector &vv) const`
- `T scalarProd (const wbvector &v) const`
- `T scalarProd (const T *v) const`
- `T norm () const`
- `T norm2 () const`
- `double normDiff (const wbvector &v, T fac=1) const`
- `T normDiff2 (const wbvector &v, T fac=1) const`
- `double normDiff2_ (const wbvector &v, double fac=1) const`
- `T GSProject (const T *v, char isnorm=0, char tflag=0) const`
- `wbvector & Normalize (const char *F=NULL, int L=0)`
- `wbvector & add (const T *d)`
- `wbvector & subtract (const T *d)`
- `void getReal (wbvector< double > &R) const`
- `void getImag (wbvector< double > &I) const`
- `void set (const wbvector< double > &R, const wbvector< double > &I)`
- `wbvector & Conj ()`
- `wbvector & applyFlag (char &flag, const T &fac, wbvector &a)`
- `bool allEqual () const`
- `bool allZero () const`
- `bool allOnes () const`
- `bool allEqual (const T &a) const`
- `bool allEqual2 (const T &a, T eps=1E-15) const`
- `bool allUnequal (const T &a) const`
- `bool any () const`
- `bool anyUnequal (const T &a) const`
- `bool anyEqual (const T &a, size_t *i1=NULL) const`
- `bool anyGT (const T &a) const`
- `bool anyGE (const T &a) const`
- `bool anyLT (const T &a) const`
- `bool anyLE (const T &a) const`
- `bool allGT (const T &a) const`
- `bool allGE (const T &a) const`

- `bool allLE (const T &a) const`
- `bool allLT (const T &a) const`
- `bool allIn (const T &a1, const T &a2) const`
- `bool anySmallVals (const T eps=1E-15) const`
- `bool isEqual (const T *v) const`
- `bool anyEqual (const wbvector< size_t > &l, const T &x, size_t *k=NULL) const`
- `bool anyUnequal (const wbvector< size_t > &l, const T &x, size_t *k=NULL) const`
- `bool isEmpty () const`
- `operator bool () const`
- `bool operator! () const`
- `bool operator== (const wbvector &v) const`
- `bool operator!= (const wbvector &v) const`
- `bool isEqual (const wbvector &v) const`
- `bool isEqualR (const wbvector &v) const`
- `bool operator^ (const wbvector &v) const`
- `bool deepEqualP (const wbvector &B) const`
- `char cmp (const wbvector &v) const`
- `bool operator< (const wbvector &v) const`
- `bool operator<= (const wbvector &v) const`
- `bool operator> (const wbvector &v) const`
- `bool operator>= (const wbvector &v) const`
- `int operator< (const T &r) const`
- `T &el (size_t i) const`
- `T elx (size_t i, const T &x) const`
- `const T &operator[] (size_t i) const`
- `T &operator[] (size_t i)`
- `T &last () const`
- `T &last (size_t &i) const`
- `template<class T1 >`
`T &last (T1 &i_) const`
- `T &p2last () const`
- `T * ref (size_t i) const`
- `wbvector &setRand (double fac=1., double shift=0.)`
- `void setIdx (double shift=0., double fac=1.)`
- `int set2Group (const wbperm &P, const WBINDEX &D, const T *S0=NULL, char iflag=0)`
- `wbvector &Resize_ (size_t n)`
- `wbvector &Shorten2 (size_t n)`
- `wbvector &Resize (size_t n, const T *d=NULL)`
- `wbvector &resize (size_t n, wbvector &X) const`
- `wbvector &cat (const wbvector &v2)`
- `wbvector &Cat (const wbvector &v1, const wbvector &v2)`
- `wbvector &Cat (const wbvector< wbvector > &vv)`
- `wbvector &Cat (const wbvector< wbvector * > &vv)`
- `template<class T1, class T2, class T3, class T4 >`
`wbvector &Cat (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)`
- `template<class T1, class T2, class T3, class T4 >`
`wbvector &Cat (const wbvector< T1 > *v1, const wbvector< T2 > *v2, const wbvector< T3 > *v3=0, const wbvector< T3 > *v4=0)`
- `wbvector &Append (const wbvector &v2)`
- `wbvector &Append (const T &x)`
- `wbvector &Append (size_t d2, const T *x)`
- `wbvector append (const wbvector &v) const`
- `wbvector &append (const wbvector &v2, wbvector &vout) const`
- `void append (const T *x, size_t d2, wbvector &vout) const`

- int **recLess** (const T *v) const
- int **recLessE** (const T *v) const
- int **recCompare** (const T *v) const
- T **max** () const
- T **max** (size_t *k) const
- T **max** (unsigned &k) const
- T **max_** (T x) const
- T **min** () const
- T **min** (size_t *k) const
- T **min** (unsigned &k) const
- T **min_** (T x) const
- T **aMax** (size_t *k=NULL) const
- T **aMax** (unsigned &k) const
- T **aMin** (char zflag=0, size_t *k=NULL) const
- T **maxneg** (size_t *k=NULL) const
- T **minpos** (size_t *k=NULL) const
- T **avg** () const
- T **std** () const
- void **range** (T &xmin, T &xmax) const
- T **range** () const
- **wbvector** & **Permute** (const **wbperm** &P, char iflag=0)
- **wbvector** & **permute** (**wbvector** &, const **wbperm** &P, char iflag=0) const
- **wbvector** **permute** (const **wbperm** &P, char iflag=0) const
- **wbvector** & **blockPermute** (const **wbperm** &P, **wbvector** &, char iflag=0) const
- **wbvector** & **BlockPermute** (const **wbperm** &P, char iflag=0)
- int **blockCompare** (const **wbperm** &pa, const **wbvector** &b, const **wbperm** &pb) const
- void **get** (const WBINDEX &I0, **wbvector** &v) const
- void **get** (const **wbvector**< size_t > &I, T *v) const
- void **get** (size_t i1, size_t i2, **wbvector** &v) const
- void **selectSU** (const WBINDEX &I)
- void **select** (const WBINDEX &I, T *r) const
- **wbvector** & **select** (const WBINDEX &I, **wbvector** &a) const
- **wbvector** & **Select** (const WBINDEX &I)
- **wbvector** & **BlockSelect** (const WBINDEX &I, const WBINDEX &D)
- **wbvector** **select** (const WBINDEX &I) const
- **wbvector** & **getI** (size_t k, **wbvector** &v) const
- **wbvector** & **getI** (const **wbindex** &k, **wbvector** &v, char uflag=1) const
- **wbvector** & **getI** (const WBINDEX &k, **wbvector** &v, char uflag=1) const
- **wbvector** & **initI** (const **wbvector** &a, const **wbindex** &ia)
- **wbvector** & **initI** (const **wbvector** &a, const **wbindex** &ia, const **wbvector** &b, const **wbindex** &ib, char uflag=1)
- **wbvector** & **Flip** ()
- void **flip** (**wbvector** &a) const
- **wbvector** **flip** () const
- **wbvector** **operator[]** (const **wbvector**< size_t > &I) const
- T **prod** () const
- T **prod** (T x) const
- T **prod** (size_t i, size_t i2) const
- T **prod** (size_t i, size_t i2, T x) const
- template<class T1 >
 T **prod** (const T1 *I, size_t n) const
- template<class T1 , class T2 >
 T2 **prod** (const T1 *I, size_t n, T2 &s2) const
- void **prod2** (T &p1, T &p2, const T &p0=0) const
- void **info** (const char *istr) const
- void **print** (const char *istr, char mflag=0) const

- void **printdata** (const char *istr=0) const
- void **put** (const char *F, int L, const char *vname="ans", char tflag=0, const char *ws="base") const
- void **put** (const char *vname="ans", char tflag=0, const char *ws="base") const
- void **dispWithMsg** (const char *file, int line, const char *msg, const char *vname="") const
- void **mat2mx** (mxArray *&a, char dim=2) const
- void **mat2mxs** (mxArray *a, char field_nr, char dim=2) const
- mxArray * **toMx_Struct** (const char tflag=0) const
- mxArray * **toMxP_S** (const char tflag=0) const
- mxArray * **toMx** (const char tflag=0) const
- mxArray * **toMxP** (const char tflag=0) const
- mxArray * **toMx_base** (const char tflag=0) const
- mxArray * **toMx_base_d** (const char tflag=0) const
- mxArray * **toMx_offset** (const T &offset, const char tflag=0) const
- mxArray * **mxCreateStruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned i, char tflag=0) const
- void **add2MxStruct** (mxArray *S, const char *vname, char tflag=0) const
- mxArray * **mxCreateCell** (unsigned m, unsigned n) const
- void **add2MxCell** (mxArray *S, unsigned i, char tflag=0) const
- void **putg** (const char *vname="ans", char tflag=0) const
- void **putx** (const char *vname="ans", char tflag=0) const
- [wbstring](#) **toStr** (int n, const char *sep) const
- [wbstring](#) **toStr** () const
- [wbstring](#) **sizeStr** () const
- [wbstring](#) **toStrf** (const char *fmt0="", const char *sep=" ", unsigned stride=0, const char *sep2="") const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- template<class T1 , class T2 , class T3 , class T4 >
[wbvector](#)< T > & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3, const [wbvector](#)< T3 > *v4)
- template<class T1 , class T2 , class T3 , class T4 >
[wbvector](#)< T > & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)
- template<class TD >
[wbvector](#)< T > & **initT** (const [wbsparray](#)< TD > &S)
- [wbstring](#) **toStr** (int n, const char *sep) const
- [wbstring](#) **toStrf** (const char *fmt_, const char *sep, unsigned stride, const char *sep2) const
- void **getReal** ([wbvector](#)< double > &R) const
- void **getImag** ([wbvector](#)< double > &I) const
- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &I)
- [wbvector](#)< [wbcomplex](#) > & **Conj** ()
- [wbvector](#)< [wbcomplex](#) > & **setRand** (double fac, double shift)
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- mxArray * **toMx_base** (const char tflag) const
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)

- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`
- `mxArray * toMx (const char tflag) const`

Public Attributes

- `T * data`
- `size_t len`
- `char isref`

Protected Member Functions

- `wbvector & RENEW (size_t n, const T *d=NULL, size_t l=-1, char init=1)`
- `wbvector & RENEW_VAL (size_t n, const T &x)`

The documentation for this class was generated from the following files:

- `wbllib.h`
- `wbvector.hh`
- `wbvector.cc`

4.117 DY::weight_info< TQ > Class Template Reference

Public Member Functions

- `weight_info (const weight_info &w)`
- `weight_info (unsigned m_, unsigned n)`
- `weight_info & init (unsigned m_, unsigned n)`
- `weight_info & init (const weight_info &w)`
- `weight_info & operator= (const weight_info &w)`
- `weight_info & save2 (weight_info &w)`

Public Attributes

- `int m`
- `qset< TQ > p`

The documentation for this class was generated from the following file:

- `clebsch.hh`

4.118 DY::Weights< TQ > Class Template Reference

Public Member Functions

- **Weights** (const [qset](#)< TQ > &q_s)
- **Weights & init** (const [qset](#)< TQ > &q_s)
- **size_t dim** () const
- **void print** (const char *F, int L, const [QType](#) &q, char vflag=0) const
- **mxArray * toMx** () const

Public Attributes

- unsigned **n**
- [qset](#)< TQ > **qm**
- **map**< [qset](#)< TQ >, [weight_info](#)< TQ > > **W**
- **map**< [qset](#)< TQ >, const [weight_info](#)< TQ > * > **X**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch.cc
- clebsch_io.cc

4.119 X3Map< TQ, TD > Class Template Reference

Public Member Functions

- **void clear** ()
- **int init** (const char *F, int L, const mxArray *S, unsigned k=0)
- **x3map**< TQ, TD > & **getXBUF** (const char *F, int L, const [cgc_contract_id](#)< MTI > &idc)
- **int contains** (const char *F, int L, const [x3map](#)< TQ, TD > &x, const [cgc_contract_id](#)< MTI > &idc)
- **int contractCGR** (const char *F, int L, const [CRef](#)< TQ > &A, const [ctrlIdx](#) &ica, const [CRef](#)< TQ > &B, const [ctrlIdx](#) &icb, [CRef](#)< TQ > &C, char cg_preview=0)
- **CRef**< TQ > & **contractDegQ** (const char *F, int L, const [CRef](#)< TQ > &A, [CRef](#)< TQ > &B)
- **unsigned add** (const char *F, int L, const [cgc_contract_id](#)< MTI > &idc, const mxArray *S, unsigned k)
- **mxArray * toMx** () const
- **void Info** (const char *F=0, int L=0) const

Public Attributes

- **unordered_map**< [cgc_contract_id](#)< MTI >, [x3map](#)< TQ, TD >, [MHash](#)< MTI > > **XBUF**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch_aux.cc
- clebsch_io.cc

4.120 x3map< TQ, TD > Class Template Reference

Public Member Functions

- **x3map** (const char *F, int L, const CRef< TQ > &A_, const ctrldx &ica, const CRef< TQ > &B_, const ctrldx &icb, wbperm &cgp, char xflag=0)
- **x3map** & **init** ()
- int **initCtr** (const char *F, int L, const CRef< TQ > &A_, const ctrldx &ica_, const CRef< TQ > &B_, const ctrldx &icb_, wbperm &cgp, char xflag=1)
- **x3map** & **init** (const char *F, int L, const mxArray *S, unsigned k=0, ctrldx *ica=NULL, ctrldx *icb=NULL, char recalc=1)
- bool **isEmpty** () const
- void **print** (const char *F=NULL, int L=0, const char *istr="", char vflag=0) const
- **wbstring toStr** (char vflag=0) const
- **wbstring x3Str** (char vflag=0) const
- size_t **memSize** () const
- mxArray * **mxCreatestruct** (unsigned m, unsigned n) const
- void **add2MxStruct** (mxArray *S, unsigned l, const cgc_contract_id< MTI > *idc=NULL) const
- mxArray * **toMx** (const cgc_contract_id< MTI > *idc=NULL) const

Public Attributes

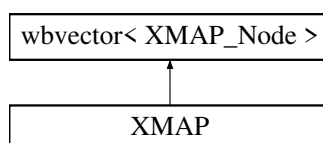
- CData< TQ, RTD > **a**
- CData< TQ, RTD > **b**
- CData< TQ, RTD > **c**
- const CDATA_TQ * **cgb**
- wbperm **pab**
- char **conj**
- int **zflag**
- cgrType **rtype**
- wbarray< TD > **x3**
- wbarray< TD > & **X3**

The documentation for this class was generated from the following files:

- clebsch.hh
- clebsch_aux.cc
- clebsch_io.cc

4.121 XMAP Class Reference

Inheritance diagram for XMAP:



Public Member Functions

- **XMAP** (unsigned n)
- **free** ()
- void **init** (unsigned n)
- template<class T >
void **set** (unsigned k, const wbArray< T > &A, unsigned ic, unsigned i1, unsigned i2, [wbcomplex](#) z)
- void **set** (unsigned k, unsigned ic, unsigned i1, unsigned i2, [wbcomplex](#) z)

Public Member Functions inherited from [wbvector< XMAP_Node >](#)

- **wbvector** (size_t l, [XMAP_Node](#) x)
- **wbvector** (size_t l, const [XMAP_Node](#) *d0=NULL, char ref=0)
- **wbvector** (const [wbvector](#) &v)
- **wbvector** (const [wbvector](#)< T2 > &v)
- **wbvector** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- **wbvector** (const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
- **wbvector** (const mxArray *a, char check_type=1, char ref=0)
- [wbvector](#) & **init** (size_t l=0)
- [wbvector](#) & **init** (size_t l, const [XMAP_Node](#) *d0)
- [wbvector](#) & **init** (size_t l, const [XMAP_Node](#) *d0, char ref)
- [wbvector](#) & **init** (const [wbvector](#) &v)
- [wbvector](#) & **init** (size_t n, const [wbvector](#) &v)
- [wbvector](#)< [XMAP_Node](#) > & **init** (const [wbsparray](#)< [XMAP_Node](#) > &S)
- [wbvector](#) & **init** (size_t l1, const [XMAP_Node](#) *d1, size_t l2, const [XMAP_Node](#) *d2)
- int **init** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type QS_UNUSED_↵
VAR=1, char ref=0)
- int **init** (const mxArray *a, const char *istr=NULL, char check_type=1)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- int **init** (const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
- [wbvector](#) & **initp** (size_t l, const [XMAP_Node](#) *d0, size_t n, char init=1)
- [wbvector](#) & **init2val** (size_t l, const [XMAP_Node](#) &x)
- [wbvector](#) & **initStride** (size_t l, const [XMAP_Node](#) *d0, size_t stride)
- [wbvector](#) & **init_stride** (const [XMAP_Node](#) *d0, size_t n, size_t m, size_t D0)
- [wbvector](#) & **init_data** (const [XMAP_Node](#) *d0)
- [wbvector](#) & **init_data** (const T0 *d0)
- [wbvector](#) & **init_data** (const char *F, int L, const T0 *d0)
- [wbvector](#)< [XMAP_Node](#) > & **initT** (const [wbsparray](#)< T2 > &S)
- [wbvector](#)< [XMAP_Node](#) > & **initT** (size_t l, const T2 *v)
- [wbvector](#)< [XMAP_Node](#) > & **initT** (const [wbvector](#)< T2 > &v)
- [wbvector](#)< [XMAP_Node](#) > & **initT** (const char *F, int L, const [wbvector](#)< T2 > &v)
- [wbvector](#)< [XMAP_Node](#) > & **initT** (const [wbsparray](#)< TD > &S)
- [wbvector](#)< [XMAP_Node](#) > & **initT_** (const [wbvector](#)< T2 > &v)
- [wbvector](#) & **init2ref** (const [wbvector](#) &v)
- [wbvector](#) & **init2ref** (size_t l, const [XMAP_Node](#) *d0)
- [wbvector](#) & **unRef** ()
- int **init_Struct** (const char *F, int L, const mxArray *a, const char *istr=NULL, char ref=0)
- int **init_base** (const char *F, int L, const mxArray *a, const char *istr=NULL, char check_type=1, char ref=0)
- int **init_mpfr** (const char *F, int L, const mxArray *a, char base=-1, const char *istr="", char tcheck=1)
- void **initDef** (size_t n)

- void **initDef** (const [wbvector](#) &v)
- [wbvector](#) & **operator=** (const [wbvector](#) &v)
- [wbvector](#)< [XMAP_Node](#) > & **operator=** (const [wbvector](#)< TB > &v)
- [wbvector](#) & **set** (const [XMAP_Node](#) &x)
- [wbvector](#) & **set** (const [wbvector](#)< size_t > &l, const [XMAP_Node](#) &x)
- void **set** (size_t offset, const [XMAP_Node](#) *x, size_t n)
- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- void **set** (const [wbvector](#)< double > &R, const [wbvector](#)< double > &l)
- [wbvector](#) & **setMin** (const [XMAP_Node](#) &x)
- [wbvector](#) & **setMax** (const [XMAP_Node](#) &x)
- [wbvector](#) & **add** (const [wbvector](#)< size_t > &l, const [XMAP_Node](#) &x)
- [wbvector](#) & **add** (const [XMAP_Node](#) *d)
- [wbvector](#) & **Set** (const [wbvector](#)< size_t > &l, const [wbvector](#) &v)
- void **Sort** ([wbperm](#) &p, char dir=+1)
- void **Sort** (char dir=+1)
- void **sort** ([wbvector](#) &v2, [wbperm](#) &p) const
- void **sort** ([wbvector](#) &v2) const
- [wbperm](#) & **pSort** ([wbperm](#) &p) const
- bool **isSorted** () const
- bool **isSortedU** () const
- bool **isUnique** () const
- int **contains** (const [XMAP_Node](#) &x) const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isNormal** () const
- bool **isFinite** () const
- size_t **find1** (const [XMAP_Node](#) &x) const
- size_t **find1** () const
- size_t **count** (const [XMAP_Node](#) &x) const
- size_t **icount** (const [XMAP_Node](#) &x) const
- size_t **nnz** () const
- size_t **nnz** ([XMAP_Node](#) eps) const
- size_t **numZeros** ([XMAP_Node](#) eps=[XMAP_Node](#)(1E-14)) const
- size_t **numEQ** (const [XMAP_Node](#) &) const
- size_t **numGT** (const [XMAP_Node](#) &) const
- size_t **numLT** (const [XMAP_Node](#) &) const
- [wbindex](#) & **find** (const [XMAP_Node](#) &x, [wbindex](#) &l, char iflag=0) const
- [wbindex](#) & **find** (const [XMAP_Node](#) &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **find** ([wbindex](#) &l) const
- [wbindex](#) & **find** ([wbindex](#) &l, [wbindex](#) &lx) const
- [wbvector](#)< size_t > **find** () const
- [wbindex](#) & **findGT** (const [XMAP_Node](#) &x, [wbindex](#) &l, char iflag=0) const
- [wbindex](#) & **findGT** (const [XMAP_Node](#) &x, [wbindex](#) &l, [wbindex](#) &lx) const
- [wbindex](#) & **findzeros** ([wbindex](#) &l) const
- [wbindex](#) & **findRange** (const [XMAP_Node](#) &x1, const [XMAP_Node](#) &x2, [wbindex](#) &l, const char *w=NULL) const
- int **findClosestSorted** ([XMAP_Node](#) r) const
- int **findClosest** ([XMAP_Node](#) r) const
- size_t **findValues** (const [wbvector](#) &v, [wbindex](#) &l) const
- [wbvector](#) & **setp** (const [XMAP_Node](#) *x)

- void **reset** ()
- void **cpy** (const [XMAP_Node](#) *v)
- void **cpy** (const [XMAP_Node](#) *x, size_t m)
- [wbvector](#) & **save2** ([wbvector](#) &v)
- void **swap** ([wbvector](#) &v)
- [wbvector](#) & **swap** (size_t i, size_t j)
- [wbvector](#) & **Collect** ([XMAP_Node](#) &A1, [XMAP_Node](#) &A2)
- [wbvector](#) & **Collect** ([XMAP_Node](#) &A1, [XMAP_Node](#) &A2, [XMAP_Node](#) &A3)
- [wbvector](#) & **Skip** (size_t)
- [wbvector](#) & **Skip** (const WBINDEX &, char rflag=0)
- [wbvector](#) & **Move2_FE** (const WBINDEX &l, char FE_flag)
- [wbvector](#) & **Move2front** (const WBINDEX &l)
- [wbvector](#) & **Move2front** (size_t k)
- [wbvector](#) & **Move2end** (const WBINDEX &l)
- [wbvector](#) & **Move2end** (size_t k)
- [wbvector](#) & **mvLastTo** (size_t k)
- [wbvector](#) & **mvFirstTo** (size_t k)
- bool **checkSameLength** (const [wbvector](#) &a, const char *F=NULL, int L=0, const char *istr="") const
- bool **checkSameLength** (const [wbvector](#) &a, const [wbvector](#) &b, const char *F=NULL, int L=0, const char *istr="") const
- void **operator+=** (const T_ &a)
- void **operator+=** (const [wbvector](#) &b)
- void **operator-=** (const T_ &a)
- void **operator-=** (const [wbvector](#) &b)
- [wbvector](#) & **operator*=** (const T_ &a)
- [wbvector](#) & **operator/=** (const T_ &a)
- [wbvector](#) **operator*** (const T_ &a) const
- [wbvector](#) **operator+** (const T_ &a) const
- [wbvector](#) **operator+** (const [wbvector](#) &b) const
- [wbvector](#) **operator-** (const T_ &a) const
- [wbvector](#) **operator-** (const [wbvector](#) &b) const
- [wbvector](#) & **Plus** (const [XMAP_Node](#) *b)
- [wbvector](#) & **Plus** (const [XMAP_Node](#) *b, [XMAP_Node](#) bfac)
- [wbvector](#) & **Plus** (const [wbvector](#) &b, [XMAP_Node](#) bfac=1)
- void **plus** (const [wbvector](#) &b, [wbvector](#) &c, [XMAP_Node](#) bfac=1, [XMAP_Node](#) cfac=0) const
- [wbvector](#) & **times** ([XMAP_Node](#) fac, [wbvector](#) &x) const
- [wbvector](#) & **RevertSigns** ()
- int **isum** () const
- [XMAP_Node](#) **sum** () const
- [XMAP_Node](#) **sum** (size_t i1, const char *estr) const
- [XMAP_Node](#) **sum** (size_t i1, size_t i2) const
- [XMAP_Node](#) **cumsum_** ([wbvector](#) &S, char xflag=0) const
- [XMAP_Node](#) **Cumsum** (char zflag=0) const
- [XMAP_Node](#) **cumsum0prod** ([wbvector](#) < T2 > &b, [wbvector](#) < [XMAP_Node](#) > &cs, char xflag) const
- [wbvector](#) & **Eldiv** (const [wbvector](#) &v) const
- [wbvector](#) & **tensorProd** (const [wbvector](#) &v2, [wbvector](#) &vv) const
- [XMAP_Node](#) **scalarProd** (const [wbvector](#) &v) const
- [XMAP_Node](#) **scalarProd** (const [XMAP_Node](#) *v) const
- [XMAP_Node](#) **norm** () const
- [XMAP_Node](#) **norm2** () const
- double **normDiff** (const [wbvector](#) &v, [XMAP_Node](#) fac=1) const
- [XMAP_Node](#) **normDiff2** (const [wbvector](#) &v, [XMAP_Node](#) fac=1) const
- double **normDiff2_** (const [wbvector](#) &v, double fac=1) const
- [XMAP_Node](#) **GSProject** (const [XMAP_Node](#) *v, char isnorm=0, char tflag=0) const
- [wbvector](#) & **Normalize** (const char *F=NULL, int L=0)

- **wbvector** & **subtract** (const [XMAP_Node](#) *d)
- void **getReal** ([wbvector](#)< double > &R) const
- void **getReal** ([wbvector](#)< double > &R) const
- void **getImag** ([wbvector](#)< double > &I) const
- void **getImag** ([wbvector](#)< double > &I) const
- **wbvector** & **Conj** ()
- **wbvector**< [wbcomplex](#) > & **Conj** ()
- **wbvector** & **applyFlag** (char &flag, const [XMAP_Node](#) &fac, [wbvector](#) &a)
- bool **allEqual** () const
- bool **allEqual** (const [XMAP_Node](#) &a) const
- bool **allZero** () const
- bool **allOnes** () const
- bool **allEqual2** (const [XMAP_Node](#) &a, [XMAP_Node](#) eps=1E-15) const
- bool **allUnequal** (const [XMAP_Node](#) &a) const
- bool **any** () const
- bool **anyUnequal** (const [XMAP_Node](#) &a) const
- bool **anyUnequal** (const [wbvector](#)< size_t > &l, const [XMAP_Node](#) &x, size_t *k=NULL) const
- bool **anyEqual** (const [XMAP_Node](#) &a, size_t *i1=NULL) const
- bool **anyEqual** (const [wbvector](#)< size_t > &l, const [XMAP_Node](#) &x, size_t *k=NULL) const
- bool **anyGT** (const [XMAP_Node](#) &a) const
- bool **anyGE** (const [XMAP_Node](#) &a) const
- bool **anyLT** (const [XMAP_Node](#) &a) const
- bool **anyLE** (const [XMAP_Node](#) &a) const
- bool **allGT** (const [XMAP_Node](#) &a) const
- bool **allGE** (const [XMAP_Node](#) &a) const
- bool **allLE** (const [XMAP_Node](#) &a) const
- bool **allLT** (const [XMAP_Node](#) &a) const
- bool **allIn** (const [XMAP_Node](#) &a1, const [XMAP_Node](#) &a2) const
- bool **anySmallVals** (const [XMAP_Node](#) eps=1E-15) const
- bool **isEqual** (const [XMAP_Node](#) *v) const
- bool **isEqual** (const [wbvector](#) &v) const
- bool **isEmpty** () const
- **operator bool** () const
- bool **operator!** () const
- bool **operator==** (const [wbvector](#) &v) const
- bool **operator!=** (const [wbvector](#) &v) const
- bool **isEqualR** (const [wbvector](#) &v) const
- bool **operator^** (const [wbvector](#) &v) const
- bool **deepEqualP** (const [wbvector](#) &B) const
- char **cmp** (const [wbvector](#) &v) const
- bool **operator<** (const [wbvector](#) &v) const
- int **operator<** (const [XMAP_Node](#) &r) const
- bool **operator<=** (const [wbvector](#) &v) const
- bool **operator>** (const [wbvector](#) &v) const
- bool **operator>=** (const [wbvector](#) &v) const
- [XMAP_Node](#) & **el** (size_t i) const
- [XMAP_Node](#) **elx** (size_t i, const [XMAP_Node](#) &x) const
- const [XMAP_Node](#) & **operator[]** (size_t i) const
- [XMAP_Node](#) & **operator[]** (size_t i)
- [wbvector](#) **operator[]** (const [wbvector](#)< size_t > &l) const
- [XMAP_Node](#) & **last** () const
- [XMAP_Node](#) & **last** (size_t &i) const
- [XMAP_Node](#) & **last** (TI &i_) const
- [XMAP_Node](#) & **p2last** () const
- [XMAP_Node](#) * **ref** (size_t i) const

- [wbvector](#) & **setRand** (double fac=1., double shift=0.)
- [wbvector](#)< [wbcomplex](#) > & **setRand** (double fac, double shift)
- void **setIdx** (double shift=0., double fac=1.)
- int **set2Group** (const [wbperm](#) &P, const WBINDEX &D, const [XMAP_Node](#) *S0=NULL, char iflag=0)
- [wbvector](#) & **Resize_** (size_t n)
- [wbvector](#) & **Shorten2** (size_t n)
- [wbvector](#) & **Resize** (size_t n, const [XMAP_Node](#) *d=NULL)
- [wbvector](#) & **resize** (size_t n, [wbvector](#) &X) const
- [wbvector](#) & **cat** (const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#) &v1, const [wbvector](#) &v2)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) > &vv)
- [wbvector](#) & **Cat** (const [wbvector](#)< [wbvector](#) * > &vv)
- [wbvector](#) & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3=0, size_t l3=0, const T4 *v4=0, size_t l4=0)
- [wbvector](#) & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3=0, const [wbvector](#)< T3 > *v4=0)
- [wbvector](#)< [XMAP_Node](#) > & **Cat** (const [wbvector](#)< T1 > *v1, const [wbvector](#)< T2 > *v2, const [wbvector](#)< T3 > *v3, const [wbvector](#)< T3 > *v4)
- [wbvector](#)< [XMAP_Node](#) > & **Cat** (const T1 *v1, size_t l1, const T2 *v2, size_t l2, const T3 *v3, size_t l3, const T4 *v4, size_t l4)
- [wbvector](#) & **Append** (const [wbvector](#) &v2)
- [wbvector](#) & **Append** (const [XMAP_Node](#) &x)
- [wbvector](#) & **Append** (size_t d2, const [XMAP_Node](#) *x)
- [wbvector](#) **append** (const [wbvector](#) &v) const
- [wbvector](#) & **append** (const [wbvector](#) &v2, [wbvector](#) &vout) const
- void **append** (const [XMAP_Node](#) *x, size_t d2, [wbvector](#) &vout) const
- int **recLess** (const [XMAP_Node](#) *v) const
- int **recLessE** (const [XMAP_Node](#) *v) const
- int **recCompare** (const [XMAP_Node](#) *v) const
- [XMAP_Node](#) **max** () const
- [XMAP_Node](#) **max** (size_t *k) const
- [XMAP_Node](#) **max** (unsigned &k) const
- [XMAP_Node](#) **max_** ([XMAP_Node](#) x) const
- [XMAP_Node](#) **min** () const
- [XMAP_Node](#) **min** (size_t *k) const
- [XMAP_Node](#) **min** (unsigned &k) const
- [XMAP_Node](#) **min_** ([XMAP_Node](#) x) const
- [XMAP_Node](#) **aMax** (size_t *k=NULL) const
- [XMAP_Node](#) **aMax** (unsigned &k) const
- [XMAP_Node](#) **aMin** (char zflag=0, size_t *k=NULL) const
- [XMAP_Node](#) **maxneg** (size_t *k=NULL) const
- [XMAP_Node](#) **minpos** (size_t *k=NULL) const
- [XMAP_Node](#) **avg** () const
- [XMAP_Node](#) **std** () const
- void **range** ([XMAP_Node](#) &xmin, [XMAP_Node](#) &xmax) const
- [XMAP_Node](#) **range** () const
- [wbvector](#) & **Permute** (const [wbperm](#) &P, char iflag=0)
- [wbvector](#) & **permute** ([wbvector](#) &, const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) **permute** (const [wbperm](#) &P, char iflag=0) const
- [wbvector](#) & **blockPermute** (const [wbperm](#) &P, [wbvector](#) &, char iflag=0) const
- [wbvector](#) & **BlockPermute** (const [wbperm](#) &P, char iflag=0)
- int **blockCompare** (const [wbperm](#) &pa, const [wbvector](#) &b, const [wbperm](#) &pb) const
- void **get** (const WBINDEX &I0, [wbvector](#) &v) const
- void **get** (const [wbvector](#)< size_t > &I, [XMAP_Node](#) *v) const
- void **get** (size_t i1, size_t i2, [wbvector](#) &v) const

- void **putx** (const char *vname="ans", char tflag=0) const
- [wbstring toStr](#) (int n, const char *sep) const
- [wbstring toStr](#) () const
- [wbstring toStr](#) (int n, const char *sep) const
- [wbstring sizeStr](#) () const
- [wbstring toStrf](#) (const char *fmt0="", const char *sep=" ", unsigned stride=0, const char *sep2="") const
- [wbstring toStrf](#) (const char *fmt_, const char *sep, unsigned stride, const char *sep2) const

Public Attributes

- char **isref**

Public Attributes inherited from [wbvector< XMAP_Node >](#)

- [XMAP_Node](#) * **data**
- size_t **len**
- char **isref**

Additional Inherited Members

Protected Member Functions inherited from [wbvector< XMAP_Node >](#)

- [wbvector](#) & **RENEW** (size_t n, const [XMAP_Node](#) *d=NULL, size_t l=-1, char init=1)
- [wbvector](#) & **RENEW_VAL** (size_t n, const [XMAP_Node](#) &x)

The documentation for this class was generated from the following file:

- xmap.hh

4.122 XMAP_CMat Class Reference

Public Attributes

- wbCMat * **M**
- unsigned **ic**
- unsigned **i1**
- unsigned **i2**
- [wbcomplex](#) **z**

The documentation for this class was generated from the following file:

- bicgMatC.cc

4.123 XMAP_Node Class Reference

Public Member Functions

- **free** ()
- void **init** (const mxArray *a)
- [wbvector](#)< unsigned > **size** () const
- unsigned **rank** () const
- bool **sameSize** (const [XMAP_Node](#) &B) const
- template<class T >
void **set** (const wbArray< T > &A, unsigned ic, unsigned i1, unsigned i2, [wbcomplex](#) z, char ref=1)
- void **set** (unsigned ic, unsigned i1, unsigned i2, [wbcomplex](#) z, char ref=1)
- template<class T >
void **set** (unsigned k, const wbArray< T > &A, unsigned ic_, unsigned i1_, unsigned i2_, [wbcomplex](#) z_, char ref)

Public Attributes

- wbArray< [wbcomplex](#) > * **Z**
- wbArray< double > * **R**
- char **tflag**
- [wbperm](#) **P**
- unsigned **ic**
- unsigned **i1**
- unsigned **i2**
- unsigned **isref**
- [wbcomplex](#) **z**

The documentation for this class was generated from the following file:

- [xmap.hh](#)

Chapter 5

File Documentation

5.1 clebsch.hh

```
00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : clebsch (for abelian and non-abelian symmetries)
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the license for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_CLEBSCH_GORDAN_HH__
00021 #define __WB_CLEBSCH_GORDAN_HH__
00022
00023 //=====//
00024 // CStore => global CG container: CStore<gTQ> gCS; // CG_GLOBAL
00025 //
00026 //     global container for Clebsch Gordan coefficients
00027 //     - based on CData for single (Q1,Q2->Q) mapping.
00028 //     - based on QVec, qset, cdata below.
00029 //
00030 // cdata : wbararray<double>    // full data arrays!
00031 // cdata : wbSpparray    // switching to sparse // Wb,Dec22,11
00032 //
00033 //     actual storage of ClebschGordan coefficients
00034 //     - of rank 3(+1) for standard CG (3+1=4 in case of multiplicity)
00035 //     - cdata also includes (contracted) derivatives of ANY rank
00036 //
00037 // QVec : wbvector<QType>
00038 //
00039 //     single set of types of quantum numbers with QType
00040 //     - the most basic symmetry unit
00041 //
00042 // qset : wbvector<TQ>
00043 //
00044 //     actual set of quantum numbers interpreted through QVec
00045 //
00046 // Wb,Sep20,09 ; Wb,May01,10
00047 //
00048 // ----- //
00049 // NB! sign convention on CGCs #CGD_NORM_QS #CGD_NORM_EXTERN
00050 //
00051 // using CRef, ensure that CData is simply normalized to 1
00052 // internally within gCS. This ensures that for each OM set
00053 // * overlaps can simply be computed based on cgw data!
00054 // * OM within gCS can be also determined through cdata.norm2().
00055 // return value: number of trailing dimensions to be interpreted
00056 // as OM space.
00057 //
00058 // Changing cgw from vector to // CGW_ARRAY
```

```

00059 //      -> wbMatrix // Wb,Jun08,20
00060 //      -> wbararray // Wb,Apr23,21
00061 //
00062 // * 1st (fast) index contracts onto OM index of CData // wdim1()
00063 // * 2nd (slow) index onto OM index of A.data // wdim2()
00064 //      #CGW_INDEX_ORDER
00065 //
00066 // => normalization convention is  $cgw^T * cgw = 1$  // #CGD_NORM_QS
00067 //      such that cgw represents an isometry
00068 //      where cgw may have have fewer (but not more) columns than rows
00069 //      so it may hold  $cgw * cgw^T \neq 1$ 
00070 //
00071 // Benefits of this normalization
00072 // - OM normalization convention of CData // #CGD_NORM_QS
00073 //      required for simple orthonormal OM basis decomposition
00074 //      remains intact also for  $CData_{actual} = CData * cgw$ 
00075 //      irrespective of direction of legs
00076 // - therefore A.data already carries correct 3j weight factors for simple
00077 //      contraction, such that e.g.,  $A.norm2()$  of some QSpace A
00078 //      is simply carried by  $|A.data|^2$ 
00079 //
00080 // NB! for 'external' purposes only (readability) #CGD_NORM_EXTERN
00081 // and only for rank-2 tensors (i.e. scalar operators,
00082 // which never have OM!), use  $|cgw| = \sqrt{qdim}$ 
00083 // to reflect actual matrix elements in A.data, e.g., see
00084 // QSpace::NormCGW() -> CGR->NormSignW -> normExt()!
00085 // @QSpace/display.m also assumes this convention throughout.
00086 //
00087 // ----- //
00088 // Other (not so useful) attempts to normalize cgw
00089 // (with corresponding inverse factor multiplied onto data{:}):
00090 //
00091 //      |cgw| := sqrt(max(size(cdata))) // former version of CGD_NORM_EXTERN
00092 //
00093 // (+) rank-2 tensor are exact identities (or untaries for 1J symbols)
00094 // (+) rank-3 tensors are mostly normalized to max(|CGC|)
00095 //      e.g. (SS|0) is normalized the same way as (S0|S)
00096 //      whereas their standard normalization [see NormStd()]
00097 //      would be sqrt(d_out), here 1 or (2S+1), respectively!
00098 //
00099 // (-) standard rank-3 identity A tensors do not have cgd = Id
00100 // (-) does not work specifically well for rank>3 tensors.
00101 //      e.g. consider
00102 //      contract( (1,1|2), 3, (1,1|2), 3 ) (i)
00103 //      vs. contract( (1,1|0), 3, (1,1|0), 3 ) (ii)
00104 //      while the output CData space is the same (1,1,1,1)
00105 //      in either case, already the normalization of the
00106 //      CData requires *different* normalization factors  $1/\sqrt{d3}$ 
00107 //      depending on the *contracted* 3rd index containing a multiplet
00108 //      of dimension d3. This information is no longer available
00109 //      once the CGCs have been contracted!
00110 //
00111 // Wb,Oct16,14 ; Wb,Nov04,14
00112 //
00113 // ===== //
00114 // RStore => global constructor class for non-abelian Lie groups
00115 //
00116 // RStore <TQ,TD> gRS; // global
00117 // * map[QType] := genRG_struct buf[]
00118 //
00119 //      genRG_struct<TQ,TD>
00120 //      * map[qset] := genRG_base<TQ,RTD> RSet[]
00121 //
00122 //      genRG_base<TQ,RTD>
00123 //      * basic symmetry settings such as
00124 //      generators, commutator relations (CR), ...
00125 //
00126 // Wb,Jun07,10
00127 //
00128 // ===== //
00129 /* CHANGE LOG
00130
00131 // Wb,Aug28,20 // PERMIT_RANK1_Q0 ----- //
00132
00133 Permit rank-1 QSpace if scalar, i.e. for q=0 only;
00134 e.g. for SU(2), can contract (S,0|S) at '13'
00135 for any spin-S multiplet, resulting in a non-zero weight
00136 since by DEF_CGD_NORM with d=2S+1 the multiplet dimension,
00137  $trace('S,0;S,'13') = 1/\sqrt{d} * d = \sqrt{d}$ 
00138 [e.g. see @QSpace/getIdentity3.m with --rho flag]
00139 however,  $contract('S,S;S,'13') = 0$ , i.e., rank-1 QSpace is
00140 not permitted for non-zero (non-scalar) symmetry label q!=0.
00141
00142 NB! a rank-1 QSpace may be generated, e.g., by contraction
00143 to obtain expectation values [HAM/calc_SdotS.m] after using
00144 @QSpace/getIdentity3.m
00145

```

```

00146     NB! contract('0,S,S*,'13','S,S*', '12')
00147     = sqrt(d) * trace('S,0;S,'13')
00148     = 1.
00149     since 2nd line acquires additional sqrt(d) factor from 'S,S*'
00150     => X-symbol has x3=1 - ok!
00151
00152     => permit rank-1 QSpace as output, but not necessarily as input
00153     yet with exceptions, such as getDimQS so that at least QSpace
00154     can be displayed using @QSpace/display.m
00155
00156 // Wb, June 2014 ----- //
00157
00158     * removed CG_SU2 (merged with treatment of QTYPE_SUN)
00159     * removed zflag from qset [e.g. nq = n(q) + n(qz)]
00160     * removed zflags from QType and qset
00161     * removed qset::RemoveZLabels (still got QSpace::RemoveZLabels!)
00162     * removed following variables from CData as these are redundant
00163       with data in RStore => genRG_struct => genRG_base:
00164       - qset<TQ> Q1,Q2,Q; // (Q1,Q2) -> Q
00165       - wbMatrix<TQ> Z1,Z2,Z; // (Z1,Z2) -> Z
00166       use gRS container to retrieve these where required
00167     * renamed CData => cdata; CGSpace => CData
00168
00169     => see Archive/clebsch_140627.[ch]* for version including zflag
00170
00171 //=====*/
00172
00173 #ifndef RTD
00174     #ifdef __WB_MPFR_HH__
00175
00176         #define MTI unsigned long
00177         #define RTD Wb::quad
00178
00179     #else
00180
00181         #define MTI unsigned long
00182         #define RTD double
00183
00184     #endif
00185 #endif
00186
00187 #ifndef gTQ
00188     #define gTQ int
00189 #endif
00190
00191 #ifndef gTD
00192     #define gTD double
00193 #endif
00194
00195 #define CData_TQ    CData<TQ,RTD>
00196 #define cdata__    cdata<RTD>
00197 #define cgsparray  wbsparray<RTD>
00198
00199
00200     template <class TQ, class TD> class CData;
00201     template <class TQ> class CRef;
00202
00203     class QVec;
00204
00205     int MAX_OM=9;
00206
00207     int get_CG_VERBOSE(const char *F=0, int L=0);
00208     int CG_VERBOSE=get_CG_VERBOSE();
00209
00210     unsigned get_CG_FIXIT(const char *F=0, int L=0);
00211     unsigned CG_FIXIT=get_CG_FIXIT();
00212
00213     #define FIX_CID3 1
00214     #define FIX_CTR 2
00215
00216     unsigned get_CG_PREVIEW(const char *F=0, int L=0);
00217     unsigned CG_PREVIEW=get_CG_PREVIEW();
00218
00219     #define RC_LOAD_ERR_AB_MSG "RC_LOAD_ERROR (A or B)"
00220
00221     wstring RC_LOG;
00222
00223     #define CG_SKIP_DEPS1 1E-12
00224     #define CG_SKIP_DEPS2 1E-14
00225
00226     #define CG_SKIP_REPS 1E-17
00227
00228 #ifdef __WB_MPFR_HH__
00229     #define CG_SKIP_EPS1 1E-16
00230     #define CG_SKIP_EPS2 1E-18
00231
00232     #define CG_EPS 1E-38

```

```

00233     #define CG_EPS0 1E-36
00234     #define CG_EPS1 1E-16
00235     #define CG_EPS2 1E-18
00236
00237 #else
00238
00239     #define CG_EPS 2.22E-16
00240     #define CG_EPS0 1E-15
00241
00242     #define CG_SKIP_EPS1 1E-14
00243     #define CG_SKIP_EPS2 1E-16
00244
00245     #define CG_EPS1 1E-10
00246     #define CG_EPS2 1E-12
00247
00248 #endif
00249
00250     double cg_eps1=CG_EPS1;
00251     double cg_eps2=CG_EPS2;
00252
00253     enum QTYPE_SET {
00254         QTYPE_UNKNOWN,
00255         QTYPE_P,
00256         QTYPE_ZN,
00257         QTYPE_U1,
00258         QTYPE_ASEP,
00259         QTYPE_SUN,
00260         QTYPE_SpN,
00261         QTYPE_SON,
00262         QTYPE_SEN,
00263         QTYPE_NUM_TYPES
00264     };
00265
00266     const char* QTYPE_STR[QTYPE_NUM_TYPES]= {
00267         "?QTYPE?",
00268         "P",
00269         "ZN",
00270         "A",
00271         "!!?",
00272         "SUN",
00273         "Sp2N",
00274         "SO2N",
00275         "SO2N_1"
00276     };
00277
00278     #define CGC_ALL_ABELIAN "A*"
00279
00280     enum CGD_TYPE {
00281         CGD_UNKNOWN,
00282
00283         CGD_ABELIAN,
00284         CGD_IDENTITY,
00285
00286         CGD_REF_INIT,
00287
00288         CGD_BSZ_INIT,
00289
00290         CGD_FROM_CGR,
00291
00292         CGD_FROM_CTR,
00293         CGD_FROM_DEC,
00294
00295         CGD_STD3,
00296
00297         CGD_1JSY_ST3,
00298         CGD_1JSY_GEN,
00299
00300         CGD_STD3_X,
00301         CGD_GEN_X,
00302         CGD_EXPLICIT,
00303
00304         CGD_NUM_TYPES };
00305
00306     #define CID_RANK1_Q0 60
00307
00308     const char* CGD_TYPE_STR[CGD_NUM_TYPES+1] = { "C:??",
00309         "C:iA",
00310         "C:iE",
00311         "C:iR",
00312         "C:iS",
00313         "C:cR",
00314         "C:cX",
00315         "C:cD",
00316         "C:c3",
00317         "C:r1J",
00318         "C:c1J",
00319         "C:c3x",

```



```

00320     "C:cgx",
00321     "C:iX",
00322     "C:##";
00323
00324     enum CGR_TYPE { CGR_DEFAULT,
00325                     CGR_ABELIAN,
00326                     CGR_CTR_SCALAR,
00327                     CGR_CTR_ZERO,
00328                     CGR_CTR_ZERO,
00329                     CGR_CTR_ZERO,
00330                     CGR_CTR_ZERO,
00331     CGR_NUM_TYPES };
00332
00333     const char* CGR_TYPE_STR[CGR_NUM_TYPES]= { "?CGR_TYPE?",
00334         "R:iA",
00335         "R:cS",
00336         "R:c0",
00337     };
00338
00339     namespace Mx {
00340
00341     template <>
00342     size_t Mx::Array<CGD_TYPE>::copy_to(CGD_TYPE* b, const wbperm *P, char tcheck)
00343     const { return rcopy_to(b,P,tcheck); };
00344
00345     template <>
00346     size_t Mx::Array<CGR_TYPE>::copy_to(CGR_TYPE* b, const wbperm *P, char tcheck)
00347     const { return rcopy_to(b,P,tcheck); };
00348
00349     };
00350
00351     template <class T> class qset;
00352     template <class T> class CStore;
00353
00354     template <class T> class QMap;
00355     template <class TQ, class TD> class blockSpaceQS;
00356
00357     #include <map>
00358
00359     namespace CG {
00360
00361     template <class TD> inline
00362     int signFirstVal(const char *F, int L,
00363                     const TD *d, SPIDX_T n,
00364                     double eps1=CG_EPS1, double eps2=CG_EPS2
00365     );
00366
00367     template <class TD> inline
00368     int rangeSignConvention(const char *F, int L,
00369                             TD *d, SPIDX_T n,
00370                             double eps1=CG_EPS1, double eps2=CG_EPS2
00371     );
00372
00373     template<class T>
00374     double FixRational(const char *F, int L,
00375                       T* d, SPIDX_T n, unsigned niter=0,
00376                       double eps1=-1, double eps2=-1
00377     );
00378     };
00379
00380     namespace DY {
00381
00382     template<class TQ>
00383     size_t wdim_A(unsigned n, const TQ *qs);
00384
00385     template<class TQ>
00386     size_t wdim_B(unsigned n, const TQ *qs);
00387
00388     template<class TQ>
00389     size_t wdim_C(unsigned n, const TQ *qs);
00390
00391     template<class TQ>
00392     size_t wdim_D(unsigned n, const TQ *qs);
00393
00394     };
00395
00396     namespace CG {
00397
00398     template <class TQ> inline
00399     unsigned gotQAlpha(const TQ* q, unsigned n) {
00400
00401         if (!WbUtil<TQ>().isInt()) return 0;
00402
00403         if (int(n)<0) wblog(FL,"ERR %s() got n=%d",FCT,n);
00404
00405         TQ qmax=61;
00406         #if __APPLE__

```

```

00407     qmax=35;
00408 #endif
00409
00410     for (unsigned i=0; i<n; ++i) { if (q[i]<0 || q[i]>qmax) return 0; }
00411
00412     return 1;
00413 };
00414
00415 template <> inline
00416 unsigned gotQAlpha(const char* s, unsigned n) {
00417
00418     unsigned i=0;
00419
00420     #if __APPLE__
00421         for (; i<n; ++i) { if (!isalnum(s[i]) || s[i]>90) break; }
00422     #else
00423         for (; i<n; ++i) { if (!isalnum(s[i])) break; }
00424     #endif
00425
00426     if (int(n)<0) { return i; }
00427     else { return (i<n ? (i+1) : 0); }
00428
00429     return 1;
00430 };
00431
00432 template <class TQ> inline
00433 unsigned cstr2qset(const char *s, TQ *q, unsigned n=-1) {
00434
00435     unsigned i=0;
00436     for (; i<n; ++i) {
00437         if (s[i]>='0' && s[i]<='9') { q[i]=s[i]-'0'; } else
00438         if (s[i]>='A' && s[i]<='Z') { q[i]=s[i]-'A'+10; } else
00439         #if !(defined __APPLE__)
00440             if (s[i]>='a' && s[i]<='z') { q[i]=s[i]-'a'+36; } else
00441         #endif
00442             break;
00443     }
00444
00445     if (int(n)<0) { return i; }
00446     else { return (i<n ? (i+1) : 0); }
00447 };
00448
00449 template <class TQ> inline
00450 int qset2cstr(const TQ *q, char *s, unsigned n) {
00451
00452     for (unsigned i=0; i<n; ++i) {
00453         if (q[i]<0) { return (i+1); }
00454         if (q[i]<10) { s[i]='0'+q[i]; } else
00455         if (q[i]<36) { s[i]='A'-10+q[i]; } else
00456         #if !(defined __APPLE__)
00457             if (q[i]<62) { s[i]='a'-36+q[i]; } else
00458         #endif
00459             { return (i+1); }
00460     }
00461     return 0;
00462 };
00463
00464 };
00465
00466 class QType {
00467
00468     public:
00469
00470     QType(QTYPE_SET t=QTYPE_UNKNOWN, unsigned s=0)
00471         : type(t), sub(s) { if (t || s) validType(FL); };
00472
00473     QType(const char *s) : type(QTYPE_UNKNOWN), sub(0) { init(0,0,s); };
00474     QType(const QType &q) : type(q.type), sub(q.sub) { };
00475
00476     QType(const char *F, int L, const mxArray *a)
00477         : type(QTYPE_UNKNOWN), sub(0) { init(F,L,a); };
00478
00479     ~QType() { init(); };
00480
00481     QType& init() { type=QTYPE_UNKNOWN; sub=0; return *this; };
00482
00483     QType& initx(QTYPE_SET t, unsigned m=0) {
00484         if (!t && !m) { init(); return *this; }
00485         type=t; sub=m; validType(FL);
00486         return *this;
00487     };
00488
00489     QType& init(QTYPE_SET t, unsigned m=0) {
00490
00491         if (!t && !m) { init(); return *this; }
00492
00493         type=t;

```

```

00494         if (type==QTYPE_SpN) { sub=m/2;
00495             if (m%2) wblog(FL,"ERR %s() invalid sub=%d with SpN",FCT,m);
00496         }
00497         else { sub=m; }
00498
00499         validType(FL); return *this;
00500     };
00501
00502     int init_s(const char *s);
00503
00504     QType& init(const char *s) { return init(FL,s); }
00505
00506     QType& init(const char *F, int L, const char *s) {
00507         if (!s || !s[0]) { type=QTYPE_UNKNOWN; sub=0; }
00508         else {
00509             if (init_s(s)) wblog(FL,"ERR invalid QType '%s'",s);
00510             else validType(F_L);
00511         }
00512         return *this;
00513     };
00514
00515     QType& init(const char *F, int L, const mxArray *a) {
00516         wbsttring s(F_L,a);
00517         return init(F_L,s.data);
00518     };
00519
00520     QType& init(const QType &q) {
00521         type=q.type; sub=q.sub; return *this;
00522     };
00523
00524     bool isUnknown() const {
00525         return (type==QTYPE_UNKNOWN && sub==0);
00526     };
00527
00528     bool isKnown() const {
00529         validType();
00530         return (type!=QTYPE_UNKNOWN);
00531     };
00532
00533     bool validType(const char *F=0, int L=0) const;
00534
00535     bool isAbelian(const char *F=NULL, int L=0) const {
00536         if (type<=QTYPE_UNKNOWN || type>=QTYPE_NUM_TYPES) wblog(F_L,
00537             "ERR invalid type %d, '%s'",type,STR_(this));
00538         return ((type && type<QTYPE_ASEP)? 1 : 0);
00539     };
00540
00541     bool isNonAbelian(const char *F=NULL, int L=0, char lflag=0) const {
00542         if (!lflag && !isUnknown()) {
00543             if (type<=QTYPE_UNKNOWN || type>=QTYPE_NUM_TYPES) wblog(F_L,
00544                 "ERR invalid type %d, '%s'",type,STR_(this)); }
00545         return ((type && type>QTYPE_ASEP)? 1 : 0);
00546     };
00547
00548     bool isSU2() const { return (type==QTYPE_SUN && sub==1); };
00549     bool isSUN() const { return (type==QTYPE_SUN); };
00550     bool isSU(unsigned N) const { return (type==QTYPE_SUN && sub+1==N); };
00551
00552     bool isSpN() const { return (type==QTYPE_SpN); };
00553     bool isSON() const { return (type==QTYPE_SON); };
00554     bool isSEN() const { return (type==QTYPE_SEN); };
00555
00556     bool isU1() const { return (type==QTYPE_U1 && sub==0); };
00557     bool isParity() const { return (type==QTYPE_P && sub==0); };
00558
00559     bool isAdditive() const {
00560         if (type==QTYPE_U1) {
00561             if (sub) wblog(FL,
00562                 "WRN invalid type %s (%d,%d)",STR_(this),type,sub);
00563             return 1;
00564         }
00565         return 0;
00566     };
00567
00568     int permitsOM() const {
00569         return (isAbelian() ? 0 : (sub<=1 ? 1 : 2));
00570     };
00571
00572     int permitsOM(unsigned r) const {
00573         return (isAbelian() || r<3 || (r==3 && sub<=1) ? 0 : 1);
00574     };
00575
00576     unsigned qlen() const {
00577         if (type && type<QTYPE_ASEP) { return 1; }
00578         if (type>=QTYPE_NUM_TYPES) wblog(FL,
00579             "ERR invalid type '%s' (%d)",STR_(this),type);
00580         return sub;

```

```

00581     };
00582
00583     unsigned qrank() const {
00584         if (type && type < QTYPE_ASEP) { return 0; }
00585         if (!type || type >= QTYPE_NUM_TYPES) wblog(FL,
00586             "ERR invalid type `%s' (%d)", STR_(this), type);
00587         if (!sub) wblog(FL,
00588             "ERR %s() got %s @ sub=%d", FCT, STR_(this), sub);
00589         return sub;
00590     };
00591
00592     bool isLargeD(unsigned d) const {
00593         if (type && type < QTYPE_ASEP) { return (d > 1); }
00594         switch (type) {
00595             case QTYPE_SUN: return (d > 12000 || d > (sub != 1 ? exp10(qlen()) : 20));
00596             case QTYPE_SON:
00597             case QTYPE_SEN:
00598             case QTYPE_SpN: return (d > 12000 || d > exp10(qlen()));
00599
00600             case QTYPE_UNKNOWN : return (d > 100);
00601             default:
00602                 wblog(FL, "ERR invalid type `%s' (%d)", STR_(this), type);
00603         }
00604         return 1;
00605     };
00606
00607     bool isLargeD(const wbvector<unsigned> &dd) const {
00608         for (unsigned i=0; i < dd.len; ++i) {
00609             if (isLargeD(dd[i])) return 1;
00610         }
00611         return 0;
00612     };
00613
00614     unsigned getDimDef() const {
00615         unsigned d=1;
00616         if (type && type > QTYPE_ASEP) {
00617             switch (type) {
00618                 case QTYPE_SUN: d=sub+1; return d;
00619                 case QTYPE_SpN: d=2*sub; return d;
00620                 case QTYPE_SON: d=2*sub+1; return d;
00621                 case QTYPE_SEN: d=2*sub; return d;
00622                 default:
00623                     wblog(FL, "ERR invalid type `%s' (%d)", STR_(this), type);
00624             }
00625         }
00626         return d;
00627     };
00628
00629     template<class TQ>
00630     size_t wdim(const TQ *qs) const {
00631         unsigned d=1;
00632         if (type && type > QTYPE_ASEP) {
00633             switch (type) {
00634                 case QTYPE_SUN: d=DY::wdim_A(sub,qs); break;
00635                 case QTYPE_SpN: d=DY::wdim_C(sub,qs); break;
00636                 case QTYPE_SON: d=DY::wdim_B(sub,qs); break;
00637                 case QTYPE_SEN: d=DY::wdim_D(sub,qs); break;
00638                 default:
00639                     wblog(FL, "ERR invalid type `%s' (%d)", STR_(this), type);
00640             }
00641         }
00642         return d;
00643     };
00644
00645     template<class TQ>
00646     size_t wdim(const qset<TQ> &qs) const { return wdim(qs.data); }
00647
00648     unsigned maxDimLocal() const {
00649         unsigned d=1;
00650         if (type) { d=maxDimLocal0(); if (d < 20) d=20; }
00651         return d;
00652     };
00653
00654     unsigned maxDimLocal0() const {
00655         unsigned d=2;
00656         if (type > QTYPE_ASEP) {
00657
00658             d=pow(double(getDimDef()), 2.25+double(sub)/4.);
00659         }
00660         return d;
00661     };
00662
00663     void printDimInfo(const char *F=NULL, int L=0) const {
00664         wblog(F_L, " * %s: sub=%d, dim_def=%d, dloc<=%d",
00665             STR_(this), sub, getDimDef(), maxDimLocal());
00666     };
00667

```

```

00668     template <class T>
00669     unsigned qdim(const T* q) const;
00670
00671     template <class T>
00672     wbvector<unsigned>& QDim(
00673         const T* q, unsigned r, unsigned stride,
00674         wbvector<unsigned> &S) const;
00675
00676     bool isSelfDual() const {
00677         switch (type) {
00678             case QTYPE_P :
00679             case QTYPE_SpN :
00680             case QTYPE_SON : return 1;
00681
00682             case QTYPE_SEN : return (qlen()%2 ? 0 : 1);
00683
00684             case QTYPE_U1 :
00685             case QTYPE_ZN :
00686             case QTYPE_SUN : return 0;
00687
00688             default:
00689                 wblog(FL, "ERR dual not yet defined for '%s'", STR_(this));
00690         }
00691         return 0;
00692     };
00693
00694     template<class TQ>
00695     char setDual(TQ* q) const {
00696         switch (type) {
00697             case QTYPE_P : { return 0; }
00698             case QTYPE_U1 : { q[0]=-q[0]; return 1; }
00699             case QTYPE_ZN : { q[0]=(sub-q[0])%sub; return 1; }
00700
00701             case QTYPE_SUN: {
00702                 unsigned n=qlen(); if (n<2) return 0;
00703                 else {
00704                     unsigned i=0, l=n-1; TQ x;
00705                     for (n/=2; i<n; ++i) { x=q[i]; q[i]=q[l-i]; q[l-i]=x; }
00706                     return 1;
00707                 }
00708             }
00709             case QTYPE_SpN:
00710             case QTYPE_SON: return 0;
00711
00712             case QTYPE_SEN: {
00713                 unsigned n=qlen();
00714                 if (n<3) wblog(FL, "ERR %s() got %s", FCT, STR_(this));
00715                 if ((n%2)==0) { return 0; }
00716                 else {
00717                     unsigned l=n-2; n-=1;
00718                     if (q[l]==q[n]) { return 0; }
00719                     else {
00720                         TQ x=q[l]; q[l]=q[n]; q[n]=x;
00721                         return 1;
00722                     }
00723                 }
00724             }
00725
00726             default:
00727                 wblog(FL, "ERR dual not yet defined for '%s'", STR_(this));
00728         }
00729         return 0;
00730     };
00731
00732     template<class TQ>
00733     unsigned getDual(const TQ* q0, TQ* q) const {
00734         unsigned n=qlen(); memcpy(q, q0, n*sizeof(TQ));
00735         setDual(q); return n;
00736     };
00737
00738     template<class TQ>
00739     bool isDual(const TQ *q, const TQ *x) const {
00740         switch (type) {
00741             case QTYPE_P : { return x[0]== q[0]; }
00742             case QTYPE_U1 : { return x[0]==-q[0]; }
00743             case QTYPE_ZN : { return x[0]==TQ((sub-q[0])%sub); }
00744
00745             case QTYPE_SUN: {
00746                 unsigned i=0, l=qlen();
00747                 if (!l) wblog(FL, "ERR %s() got n=%d", FCT, l);
00748                 for (--l; i<=l; ++i) { if (x[i]!=q[l-i]) return 0; }
00749                 return 1;
00750             }
00751             case QTYPE_SpN:
00752             case QTYPE_SON: {
00753                 if (x!=q) {
00754                     unsigned i=0, n=qlen();

```

```

00755         for (; i<n; ++i) { if (x[i]!=q[i]) return 0; }
00756     }; return 1;
00757 }
00758 case QTYPE_SEN: {
00759     unsigned n=qlen();
00760     if (n<3) wblog(FL,"ERR %s() got %s",FCT,STR_(this));
00761
00762     if (x==q) {
00763         return (n%2 ? x[n-2]==x[n-1] : 1);
00764     }
00765     else {
00766         unsigned i=0, l=n-(n%2 ? 2 : 0);
00767         for (; i<l; ++i) { if (x[i]!=q[i]) { return 0; } }
00768         if (i<n) {
00769             if (x[i]!=q[i+1] || x[i+1]!=q[i]) { return 0; }
00770         }; return 1;
00771     }
00772 }
00773
00774 default:
00775     wblog(FL,"ERR dual not yet defined for '%s'",STR_(this));
00776 }
00777 return 0;
00778 };
00779
00780 QType& operator=(const char *s) { return init(FL,s); };
00781 QType& operator=(const QType &q) { return init(q); };
00782
00783 QType& operator=(QTYPE_SET t) {
00784     type=t; sub=0; if (t) validType(FL);
00785     return *this;
00786 };
00787
00788 bool operator<(const QType &q) const {
00789     if (type!=q.type)
00790         return (type<q.type);
00791     else return (sub<q.sub);
00792 };
00793
00794 char cmp(const QType &q) const {
00795     if (type<q.type) return -1;
00796     if (type>q.type) return +1;
00797     if (sub <q.sub ) return -1;
00798     if (sub >q.sub ) return +1; else return 0;
00799 };
00800
00801 bool operator> (const QType &q) const { return !((*this)<=q); };
00802 bool operator<=(const QType &q) const {
00803     if (type!=q.type) return (type<q.type);
00804     else return (sub<=q.sub);
00805 };
00806
00807 bool operator== (const QType &q) const {
00808     return (type==q.type && sub==q.sub);
00809 };
00810 bool operator!= (const QType &q) const {
00811     return (type!=q.type || sub!=q.sub);
00812 };
00813
00814 wstring toStr(const char tflag=0) const {
00815     wstring s(8);
00816     char e=0; s[0]=0;
00817
00818     if (type==QTYPE_U1 ) { s=(tflag ? QTYPE_STR[type] : "U(1)" ); } else
00819     if (type==QTYPE_P ) { s=(tflag ? QTYPE_STR[type] : "Parity"); } else
00820     if (!type && !sub) { s=(tflag ? "???" : "unknown"); }
00821
00822     if (s[0]) {
00823         if (sub) wblog(FL,
00824             "ERR invalid QType %s with sub=%d",s.data,sub);
00825         return s;
00826     }
00827
00828     if (sub<1 || sub>99) { e|=1; }
00829     else if (type==QTYPE_ZN) { if (sub<2) e|=2;
00830         if (tflag)
00831             { snprintf(s.data,s.len,"Z%d", sub); }
00832         else { snprintf(s.data,s.len,"Z(%d)",sub); }
00833     }
00834     else if (type==QTYPE_SUN) {
00835         if (tflag)
00836             { snprintf(s.data,s.len,"SU%d", sub+1); }
00837         else { snprintf(s.data,s.len,"SU(%d)",sub+1); }
00838     }
00839     else if (type==QTYPE_SpN) { if (sub<2) e|=4;
00840         if (!tflag)
00841             { snprintf(s.data,s.len,"Sp(%d)",2*sub); }

```

```

00842         else { snprintf(s.data,s.len,"Sp%d", 2*sub);
00843             if (tflag=='u') { s[l]='P'; }
00844         }
00845     }
00846     else if (type==QTYPE_SON) {
00847         if (!tflag)
00848             { snprintf(s.data,s.len,"SO(%d)",2*sub+1); }
00849         else { snprintf(s.data,s.len,"SO%d", 2*sub+1); }
00850     }
00851     else if (type==QTYPE_SEN) { if (sub<2) e|=8;
00852         if (!tflag)
00853             { snprintf(s.data,s.len,"SO(%d)",2*sub); }
00854         else { snprintf(s.data,s.len,"SO%d", 2*sub); }
00855     }
00856     else {
00857         if (type) wblog(FL,"ERR %s() invalid type=%d",FCT,type);
00858         s=(tflag ? "???" : "unknown");
00859     }
00860
00861     if (e) wblog(FL,
00862         "ERR invalid QType %s with sub=%d", type < QTYPE_NUM_TYPES ?
00863         QTYPE_STR[type] : "(type out of bounds)", sub
00864     );
00865
00866     return s;
00867 };
00868
00869 mxArray* mxCreateStruct(unsigned m, unsigned n) const;
00870 void add2MxStruct(mxArray *S, unsigned i) const;
00871
00872 mxArray* toMx(const char tflag=0) const {
00873     const size_t n=32; wbststring s(n);
00874     size_t l=snprintf(s.data,n,"%s",toStr(tflag).data);
00875     if (l>n) wblog(FL,
00876         "WRN %s() QType too long (%s; %d)",FCT,s.data,l);
00877     return s.toMx();
00878 };
00879
00880 QTYPE_SET type;
00881 unsigned sub;
00882
00883 private:
00884
00885     int atoi(const char *s, int &k, unsigned n) {
00886         if (s[0]!='(') { return Wb::atoi(s,k,n); }
00887         else {
00888             unsigned l=strlen(++s);
00889             if (l && s[l-1]==')') {
00890                 char x[l]; memcpy(x,s,l); x[l-1]=0;
00891                 return Wb::atoi(x,k,n);
00892             }
00893         }
00894         return -6;
00895     }
00896 };
00897
00898 QType SU2("SU2");
00899
00900 bool operator!(const QType &q) {
00901     if (q.type == QTYPE_UNKNOWN) {
00902         if (q.sub) wblog(FL,
00903             "ERR %s() invalid QType %s",FCT,STR(q));
00904         return 1;
00905     }
00906     return 0;
00907 };
00908
00909 map<QType, time_t> load_cstore;
00910
00911 QType load_store_qtype(const char *F, int L, const Wb::matFile &M);
00912 void load_RCStore(const char *F, int L, const QVec &qvec);
00913
00914 class QVec : public wbvector<QType> {
00915 public:
00916     QVec() : wbvector<QType>() {};
00917
00918     QVec(unsigned l, const QType* d0, char ref=0)
00919         : wbvector<QType>(l,d0,ref) {};
00920
00921 #ifndef NOMEX
00922     QVec(const char *F, int L, const mxArray *a, unsigned d=0)
00923         : wbvector<QType>() { init(F,L,a,d); };
00924
00925     QVec& init(const char *F, int L, const mxArray *a, unsigned d=0){
00926         if (!a) { init(); return *this; }
00927         if (mxIsChar(a)) { wbststring s(a); return init(F,L,s.data,d); }

```

```

00929         else wblog(FL,"ERR %s() invalid QVec (%s)",FCT,mxGetClassName(a));
00930         return *this;
00931     };
00932 #endif
00933
00934     QVec& init(unsigned l=0) {
00935         wbvector<QType>::init(l); return *this;
00936     };
00937
00938     QVec& init(const char *F, int L, const char *s, unsigned d=0);
00939
00940     unsigned Qlen(unsigned n=-1) const;
00941
00942     unsigned Qrank() const;
00943
00944     unsigned Qlen(wbvector<unsigned> &dd) const;
00945     unsigned Qlen(
00946         wbvector<unsigned> &dd,
00947         wbvector<unsigned> &dz
00948     ) const;
00949
00950     unsigned Qpos(wbvector<unsigned> &dc) const;
00951
00952     unsigned Qlenz() const {
00953         unsigned i=0, n=0;
00954         for (; i<len; ++i) { n+=(data[i].qlen() + data[i].qrank()); }
00955         return n;
00956     }
00957
00958     wbvector<gTQ>& getScalar(wbvector<gTQ> &qs, char zflag=0) {
00959         unsigned l=0, i=0;
00960         for (; i<len; ++i) {
00961             if (data[i].isParity()) {
00962                 if (data[i].qlen()!=1 || data[i].qrank()!=0) wblog(FL,
00963                     "ERR %s() %s @ %d/%d",
00964                     FCT,STR(data[i]),data[i].qlen(),data[i].qrank());
00965                 break;
00966             }
00967         }
00968         if (i==len) {
00969             qs.init(zflag ? Qlenz() : Qlen());
00970         }
00971         else if (!zflag) {
00972             wbvector<unsigned> dd; qs.init(Qlen(dd));
00973             for (i=0; i<len; ++i) {
00974                 if (data[i].isParity()) { qs[l]=1; }; l+=dd[i];
00975             }
00976         }
00977         else {
00978             wbvector<unsigned> dd,dz;
00979             unsigned l1=Qlen(dd,dz), l2=dz.sum(); qs.init(l1+l2);
00980
00981             for (i=0; i<dd.len; ++i) {
00982                 if (data[i].isParity()) { qs[l]=1; }
00983                 l+=dd[i]; if (zflag!='z') l+=dz[i];
00984             }
00985         }
00986         return qs;
00987     };
00988
00989     unsigned maxRank() const {
00990         unsigned r=0, i=0, q=0;
00991         for (; i<len; ++i) { q=data[i].qrank(); if (r<q) { r=q; } }
00992         return r;
00993     };
00994
00995     template <class TQ>
00996     unsigned QDim(const TQ *q) const;
00997
00998     template <class TQ>
00999     unsigned QDim(const TQ *qq, unsigned k) const;
01000
01001     template <class TQ>
01002     wbvector<unsigned>& QDim(
01003         const TQ *qq, wbvector<unsigned> &S
01004     ) const;
01005
01006     template <class TQ>
01007     wbvector<unsigned>& QDim(
01008         const TQ *qq, unsigned k, unsigned r, wbvector<unsigned> &S
01009     ) const;
01010
01011     template <class TQ>
01012     wbMatrix<TQ>& getQsub(
01013         const wbMatrix<TQ>& Q, const wbindex &I,
01014         wbMatrix<TQ>& QI,
01015         wbMatrix<TQ> *Qx=NULL

```



```

01016     ) const;
01017
01018     bool allU1() const {
01019         for (unsigned i=0; i<len; ++i) { if (!data[i].isU1()) return 0; }
01020         return 1;
01021     };
01022
01023     void ReduceU1() {
01024         if (len) {
01025             for (unsigned i=0; i<len; ++i) { if (!data[i].isU1())
01026                 return;
01027             }; init();
01028         }
01029     };
01030
01031     char allAbelian() const {
01032         char q=1;
01033         for (unsigned i=0; i<len; ++i) {
01034             if (!data[i].isAbelian()) { q=0; break; }
01035             if (data[i]!=QTYPE_U1) q|=2;
01036         }
01037         return q;
01038     };
01039
01040     bool anyAbelian() const {
01041         if (!len) return 1;
01042         for (unsigned i=0; i<len; ++i) {
01043             if (data[i].isAbelian()) return 1; }
01044         return 0;
01045     };
01046
01047     bool anyParity() const {
01048         if (len) {
01049             for (unsigned i=0; i<len; ++i) {
01050                 if (data[i].isParity()) return 1; }
01051             }
01052         return 0;
01053     };
01054
01055     bool allNonAbelian() const { return !anyAbelian(); };
01056
01057     char isNonAbelian() const {
01058         if (!len) return 0;
01059         for (unsigned i=0; i<len; ++i) {
01060             if (data[i].isNonAbelian()) { return data[i].permitsOM(); }
01061         }
01062         return 0;
01063     };
01064
01065     bool allAdditive() const {
01066         for (unsigned i=0; i<len; ++i) {
01067             if (!data[i].isAdditive()) return 0;
01068         }
01069         return 1;
01070     };
01071
01072     int permitsOM() const {
01073         int q=0; if (len) { unsigned i=0; int qi;
01074             for (; i<len; ++i) { if (q<(qi=data[i].permitsOM())) { q=qi; } }
01075         }
01076         return q;
01077     };
01078
01079     int permitsOM(unsigned r) const {
01080         for (unsigned i=0; i<len; ++i) { if (data[i].permitsOM(r)) return 1; }
01081         return 0;
01082     };
01083
01084     bool sameAs(const QVec &b, char lflag=1) const {
01085         if (len==b.len || !lflag) { return (*this)==b; }
01086         if (len && !b.len) return allU1();
01087         if (b.len && !len) return b.allU1();
01088         return 0;
01089     };
01090
01091     bool sameType (const QVec &b) const {
01092         if (len!=b.len) return 0;
01093         for (unsigned i=0; i<len; i++)
01094             if (data[i].type!=b.data[i].type) return 0;
01095         return 1;
01096     };
01097
01098     wstring toStr(const char vflag=0) const;
01099     int print_qset(
01100         const char *F, int L, const gTQ *qs, char *s, unsigned n) const;
01101 };
01102

```

```

01103 namespace DY {
01104
01105 template <class TQ>
01106 class weight_info {
01107
01108 public:
01109
01110     weight_info() : m(0) {};
01111
01112     weight_info(const weight_info& w) { init(w); };
01113
01114     weight_info(unsigned m_, unsigned n) : m(m_) {
01115         p.init(n); };
01116
01117     weight_info& init(unsigned m_, unsigned n) {
01118         m=m_; p.init(n); return *this;
01119     };
01120
01121     weight_info& init(const weight_info& w) {
01122         m=w.m; p.init(w.p);
01123         return *this;
01124     };
01125
01126     weight_info& operator=(const weight_info& w) { return init(w); }
01127
01128     weight_info& save2(weight_info& w) {
01129         m=w.m; w.m=0; w.p.save2(p);
01130         return *this;
01131     };
01132
01133     int m;
01134
01135     qset<TQ> p;
01136
01137 protected:
01138 private:
01139 };
01140
01141 template <class TQ>
01142 class Weights {
01143
01144 public:
01145     Weights() : n(0) {};
01146
01147     Weights(const qset<TQ> &qs) : n(qs.len), qm(qs) {
01148         if (!n) wblog(FL, "ERR %s() got empty qset", FCT);
01149         weight_info<TQ> &w=W[qs];
01150         w.init(1, n);
01151         X[qs]=&w;
01152     };
01153
01154     Weights& init(const qset<TQ> &qs) {
01155         if (!n=qs.len) wblog(FL, "ERR %s() got empty qset", FCT);
01156         qm.init(qs); W.clear(); X.clear();
01157         weight_info<TQ> &w=W[qs];
01158         w.init(1, n);
01159         X[qs]=&w;
01160         return *this;
01161     };
01162
01163     size_t dim() const {
01164         size_t mtot=0, merr=0;
01165         for (auto I=W.begin(); I!=W.end(); ++I) {
01166             if (I->second.m>0) { mtot+=I->second.m; }
01167             else { ++merr; }
01168         }
01169         if (merr) wblog(FL,
01170             "WRN %s() got %d undetermined weight spaces", FCT, merr);
01171         return mtot;
01172     };
01173
01174     void print(
01175         const char *F, int L, const QType &q,
01176         char vflag=0) const;
01177
01178     mxArray* toMx() const;
01179
01180     unsigned n;
01181     qset<TQ> qm;
01182
01183     map <
01184         qset<TQ>,
01185         weight_info<TQ>
01186     > W;
01187
01188     map <
01189         qset<TQ>,

```

```

01190         const weight_info<TQ>*
01191         > X;
01192
01193     protected:
01194     private:
01195 };
01196
01197 class Symmetry {
01198
01199     public:
01200
01201         Symmetry(const QType &q_) : q(q_) { init(q); };
01202         Symmetry(const mxArray *a) { q.init(FL,a); init(q); };
01203
01204         Symmetry& init(const QType &q);
01205
01206         template <class TQ>
01207         size_t getWeightsFT(
01208             const char *F, int L, const qset<TQ> &qs,
01209             Weights<TQ> &R
01210         );
01211
01212         template <class TQ>
01213         int getInnerMultFT(
01214             const qset<TQ> &qs, const weight_info<TQ> &w,
01215             Weights<TQ> &R, char vflag=0
01216         );
01217
01218         template <class TQ>
01219         int Get(
01220             const qset<TQ> &qk, map < qset<TQ>, weight_info<TQ> > &W,
01221             char vflag=0
01222         );
01223
01224         template <class TQ>
01225         int wExpand(const char *F, int L, Weights<TQ> &R) const;
01226
01227         mxArray* toMx() const;
01228
01229         QType q;
01230         unsigned n;
01231
01232         wbararray<double> R;
01233
01234         wbMatrix<int> A;
01235
01236         wbvector<int> n2;
01237
01238         wbararray<int> M;
01239
01240     protected:
01241     private:
01242 };
01243 };
01244
01245 };
01246
01247 class QDir: public wbvector<char> {
01248
01249     public:
01250
01251         QDir() : wbvector<char>() {};
01252
01253         QDir(unsigned l, const char* d=NULL, char ref=0)
01254             : wbvector<char>(l,d,ref) {};
01255
01256         QDir(const iTags& it) { init(it); };
01257         QDir(const char *F, int L, const char *s) { init(F,L,s); };
01258         QDir(const char *s) { init(FL,s); };
01259
01260 #ifndef NOMEX
01261         QDir(const char *F, int L, const mxArray *a) { init(F,L,a); };
01262 #endif
01263
01264         QDir& init(unsigned l=0, const char* d=NULL, char ref=0) {
01265             wbvector<char>::init(l,d,ref);
01266             return *this;
01267         };
01268
01269         QDir& operator=(const char *s) { return init(FL,s); };
01270
01271         QDir& init(const char *F, int L, const char *s);
01272         QDir& init(const char *s) { return init(FL,s); };
01273         QDir& init(const iTags& it);
01274
01275         QDir& init(const char *F, int L, const mxArray *a) {
01276             wbstring s(F,L,a); init(F,L,s.data);

```

```

01277     return *this;
01278 };
01279
01280 QDir& init_iout(const char *F, int L,
01281 unsigned r, unsigned iout
01282 );
01283
01284 unsigned isSorted() const {
01285     unsigned i=0, j=0;
01286
01287     #ifndef WB_SKIP_ASSERT
01288     for (; i<len; ++i) if (!data[i]) {
01289         wblog(FL,"ERR %s() invalid qdir '%s'",FCT,STR_(this));
01290     }; i=0;
01291     #endif
01292
01293     for (; i<len; ++i) { if (data[i]>0) break; };
01294     if (i==len) return 1;
01295     if (i) return 0;
01296
01297     for (; i<len; ++i) { if (data[i]<0) break; };
01298     if (i==len) return (len+1);
01299
01300     for (j=i+1; i<len; ++i) { if (data[i]>0) break; };
01301     if (i==len) return j;
01302
01303     return 0;
01304 };
01305
01306 bool operator==(const char *s) const;
01307 bool operator!=(const char *s) const { return !((*this)==s); };
01308
01309 bool operator==(const QDir &q) const {
01310     return this->wbvector<char>::operator==(q); };
01311 bool operator!=(const QDir &q) const { return !((*this)==q); };
01312
01313 bool operator<(const QDir &q) const {
01314     return wbvector<char>::operator<(q); };
01315
01316 QDir& Conj() {
01317     for (unsigned i=0; i<len; ++i) { data[i]=-data[i]; }
01318     return *this;
01319 };
01320
01321 char& Conj(const unsigned &i) {
01322     if (i>=len) wblog(FL,
01323 "ERR %s() index out of bounds (%d/%d)",FCT,i,len);
01324     return (data[i]=-data[i]);
01325 };
01326
01327 unsigned nconj() {
01328     unsigned i=0, n=0;
01329     for (; i<len; ++i) { if (data[i]<0) ++n; }
01330     return n;
01331 };
01332
01333 unsigned nconj(const ctrIdx &Ix) {
01334     unsigned n=0, i=0;
01335     wbvector<char> q(*this); const unsigned* ix=Ix.data;
01336
01337     for (; i<Ix.len; ++i) {
01338         if (ix[i]>=len) wblog(FL,"ERR %s() index out of bounds"
01339 "%d/%d: %d/%d",FCT,i,Ix.len, ix[i],len);
01340         q[ix[i]]=0;
01341     }
01342     if (Ix.conj)
01343         { for (i=0; i<len; ++i) { if (q[i]>0) ++n; }}
01344     else { for (i=0; i<len; ++i) { if (q[i]<0) ++n; }}
01345
01346     return n;
01347 };
01348
01349 int findl() const {
01350     int rval=-1024; unsigned k=0, i=0, o=0, ni=0, no=0;
01351
01352     for (; k<len; ++k) { if (data[k]>0) { ++ni; i=k; } else { ++no; o=k; }}
01353     if (no==1) { rval=-o; } else
01354     if (ni==1) { rval= i; }
01355
01356     return rval;
01357 };
01358
01359 wbstring toStr() const;
01360 wbstring toTag() const;
01361
01362 mxArray* toMx() const { return toStr().toMx(); };
01363

```

```

01364     protected:
01365     private:
01366
01367 };
01368
01369 class cgdStatus {
01370
01371     public:
01372
01373     cgdStatus() : t(CGD_UNKNOWN), ctime(0), mtime(0), ID(0) {
01374         memset(user,0,4); };
01375
01376     cgdStatus(CGD_TYPE t_) : cgdStatus() { t=t_; init_time(); };
01377
01378     cgdStatus(double t_) : cgdStatus() {
01379         if (double(t)!=t_ || t>=CGD_NUM_TYPES) wblog(FL,
01380             "ERR %s() invalid CData type %g",FCT,t_);
01381         t=CGD_TYPE(t_);
01382     };
01383
01384     cgdStatus(const char *F, int L, mxArray *a)
01385         : cgdStatus() { t=CGD_UNKNOWN; init(F,L,a); };
01386
01387     template <class TQ>
01388     cgdStatus(const CDATA_TQ& C) { init(C.cstat); }
01389
01390     cgdStatus& init() {
01391         t=CGD_UNKNOWN; ctime=0; mtime=0; ID=0;
01392         memset(user,0,2);
01393         return *this;
01394     };
01395
01396     cgdStatus& init(const CGD_TYPE &t_) {
01397         t=t_; init_time(); return *this;
01398     };
01399
01400     cgdStatus& init0(const CGD_TYPE &t_) {
01401         t=t_; ctime=0; mtime=0; ID=0;
01402         memset(user,0,2);
01403         return *this;
01404     };
01405
01406     cgdStatus& update_m() { init_time('m'); return *this; };
01407     cgdStatus& update_m(const CGD_TYPE &t_) {
01408         t=t_; init_time('m'); return *this;
01409     };
01410
01411     cgdStatus& operator=(const cgdStatus &b) { return init(b); };
01412
01413     cgdStatus& init(const cgdStatus &b) {
01414         if (ID && b.ID && (ID!=b.ID || ctime!=b.ctime)) wblog(FL,
01415             "ERR got initialized cstat with ID mismatch!\n%s\n%s",
01416             STR2_(this,'V'),STR2(b,'V'));
01417         t=b.t; ctime=b.ctime; mtime=b.mtime; ID=b.ID;
01418
01419         user[0]=b.user[0];
01420         user[1]=b.user[1];
01421
01422         return *this;
01423     };
01424
01425     CGR_TYPE init(const char *F, int L, const mxArray *a);
01426
01427     int sameID(const cgdStatus &b) const {
01428         return (ID==b.ID && ctime==b.ctime);
01429     };
01430
01431     int cmp(const char *F, int L, const cgdStatus &b) const {
01432         if (ID!=b.ID || ctime!=b.ctime) { if (F) wblog(F,L,
01433             "ERR %s() got CData status mismatch!\nN    %s%N<> %s",
01434             FCT, toStr('V').data, b.toStr('V').data);
01435             else return -11;
01436         }
01437         if (mtime<ctime || b.mtime<b.ctime) { if (F) wblog(F,L,
01438             "ERR %s() invalid mtime",FCT);
01439             else return -12;
01440         }
01441         return (mtime<b.mtime ? -1 : (mtime==b.mtime ? 0 : +1));
01442     };
01443
01444     bool olderThan(const cgdStatus &b) const { return (cmp(FL,b)<0); };
01445     bool newerThan(const cgdStatus &b) const { return (cmp(FL,b)>0); };
01446
01447     bool hasID(char lflag=0) const {
01448         if (!lflag)
01449             return (ctime && mtime && ID);
01450         else return (ctime || mtime || ID);

```

```

01451     };
01452
01453     bool operator==(const CGD_TYPE t_) const { return (t==t_); };
01454     bool operator!=(const CGD_TYPE t_) const { return (t!=t_); };
01455
01456     bool operator==(const cgdStatus &b) const {
01457         return (ctime==b.ctime && mtime==b.mtime && ID==b.ID);
01458     };
01459
01460     bool operator!=(const cgdStatus &b) const { return !((*this)==b); };
01461
01462     bool isSet() const { return (ctime || mtime || ID); };
01463     bool isEmpty() const {
01464         return (!ctime && !mtime && !ID && t==CGD_UNKNOWN);
01465     };
01466
01467     int invalid() const {
01468         int e=0;
01469         if ((ctime!=0) ^ (mtime!=0)) { e|=1; }
01470         if (t==CGD_EXPLICIT) {
01471             if (ctime || ID>99) { e|=8; }
01472         }
01473         else {
01474             if ((ctime!=0) ^ (ID!=0)) { e|=2; }
01475             if ((ctime!=0) ^ (t!=CGD_UNKNOWN)) { e|=4; }
01476         }
01477         return e;
01478     };
01479
01480     bool sameAs(const cgdStatus &b, char lflag=0) const {
01481         if (ctime!=b.ctime || ID!=b.ID) { return 0; }
01482
01483         if (lflag>2) {
01484             if (lflag=='l') lflag=1; else
01485             if (lflag=='L') lflag=2; else
01486             wblog(FL,"WRN %s() invalid lflag=%d (using 2='L')",FCT,lflag);
01487         }
01488         if (lflag<2 && mtime!=b.mtime) { return 0; }
01489         if (lflag<1 && t!=b.t) { return 0; }
01490         return 1;
01491     };
01492
01493     const char* tstr() const {
01494         if (t>=CGD_NUM_TYPES) wblog(FL,"ERR %s() "
01495             "cgd type out of bounds (%d/%d)",FCT,t,CGD_NUM_TYPES);
01496         return CGD_TYPE_STR[t];
01497     };
01498
01499     wstring toStr(char vflag=0) const;
01500     wstring u2Str(const char *istr=NULL) const;
01501
01502     mxArray* toMx() const;
01503     mxArray* toMx(CGR_TYPE rt) const;
01504
01505     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
01506     mxArray* add2MxStruct(mxArray *S, unsigned k) const;
01507
01508     CGD_TYPE t;
01509
01510     double ctime;
01511     double mtime;
01512
01513     unsigned ID;
01514
01515     unsigned char user[4];
01516
01517 protected:
01518 private:
01519
01520     cgdStatus& init_time(char flag=0);
01521     wstring to_vstr(double t) const;
01522
01523     void setID() {
01524         unsigned i=0, n=3;
01525         for (; i<n; ++i) {
01526             ID= (::rand() & ((1<20)-1)); if (ID) break;
01527         }
01528         if (!ID) wblog(FL,"ERR %s() got ID=%d (%d/%d)",FCT,ID,i,n);
01529     };
01530 };
01531
01532 bool operator!(const cgdStatus &q) {
01533     return (q.t==CGD_UNKNOWN && !q.ctime && !q.mtime && !q.ID);
01534 };
01535
01536 class cgrType {
01537

```

```

01538 public:
01539
01540     cgrType(CGR_TYPE t_=CGR_DEFAULT) : t(t_) {};
01541
01542     cgrType(double t_) { init(FL,t_); };
01543
01544     template<class T>
01545     cgrType& init(const char *F, int L, T t_) {
01546         t=CGR_TYPE(t_);
01547         if (T(t)!=t_ || t>=CGR_NUM_TYPES) wblog(F,L,
01548             "ERR %s() invalid CRef type %g",FCT,double(t_))
01549         );
01550         return *this;
01551     };
01552
01553     cgrType& operator=(const cgrType &q) { t=q.t; return *this; };
01554     cgrType& operator=(const CGR_TYPE &t_) { t=t_; return *this; };
01555
01556     bool operator< (const CGR_TYPE t_) const { return (t< t_); };
01557     bool operator<= (const CGR_TYPE t_) const { return (t<=t_); };
01558     bool operator> (const CGR_TYPE t_) const { return (t> t_); };
01559     bool operator>= (const CGR_TYPE t_) const { return (t>=t_); };
01560     bool operator==(const CGR_TYPE t_) const { return (t==t_); };
01561     bool operator!=(const CGR_TYPE t_) const { return (t!=t_); };
01562
01563     bool operator< (const cgrType &q) const { return (q.t< t); };
01564     bool operator<= (const cgrType &q) const { return (q.t<=t); };
01565     bool operator> (const cgrType &q) const { return (q.t> t); };
01566     bool operator>= (const cgrType &q) const { return (q.t>=t); };
01567     bool operator==(const cgrType &q) const { return (q.t==t); };
01568     bool operator!=(const cgrType &q) const { return (q.t!=t); };
01569
01570     const char* tostr() const {
01571         if (t>=CGR_NUM_TYPES) wblog(FL,"ERR %s() "
01572             "cgrType out of bounds (%d/%d)",FCT,t,CGR_NUM_TYPES);
01573         return CGR_TYPE_STR[t];
01574     };
01575
01576     wstring toStr() const { return tostr(); }
01577
01578     CGR_TYPE t;
01579
01580 protected:
01581 private:
01582 };
01583
01584 template <class TQ>
01585 class qset : public wbvector<TQ> {
01586
01587 public:
01588     qset() : wbvector<TQ>() {};
01589
01590     qset(unsigned l, const TQ* d0=NULL, char ref=0)
01591         : wbvector<TQ>(l,d0,ref) {};
01592
01593     qset(const qset<TQ> &q)
01594         : wbvector<TQ>(q.len,q.data) {};
01595
01596     qset(const QType &Q, const TQ *qq)
01597         : wbvector<TQ>() { init(Q,qq); };
01598
01599     qset(const QVec &Q, const TQ *qq)
01600         : wbvector<TQ>() { init(Q,qq); };
01601
01602     qset(const char *F, int L, const mxArray *a) : wbvector<TQ>() {
01603         if (mxIsChar(a)) { wstring s(a); init(F_L,s.data); }
01604         else init(F,L,a,NULL,l);
01605     };
01606
01607     qset(const char *F, int L, const mxArray *a,
01608         const QVec *Q, char check_type=1) : wbvector<TQ>() {
01609         init(F,L,a,Q,check_type);
01610     };
01611
01612     qset(const qset<TQ> &q1, const qset<TQ> &q2)
01613         : wbvector<TQ>() { init(q1,q2); };
01614
01615     qset(const qset<TQ> &q1, const qset<TQ> &q2, const qset<TQ> &q3)
01616         : wbvector<TQ>() { init(q1,q2,q3); };
01617
01618     qset(const qset<TQ> &q1, char op, const qset<TQ> &q2);
01619
01620     qset& init(const qset<TQ> &q1) {
01621         if (this!=&q1) { wbvector<TQ>::init(q1); }
01622         return *this;
01623     };
01624

```

```

01625     qset& operator=(const qset<TQ> &q1) {
01626         if (this!=&q1) { wbvector<TQ>::init(q1); }
01627         return *this;
01628     };
01629
01630     qset& init(const char *F, int L, const char *s, QType *t=NULL);
01631     qset& init(
01632         const char *F, int L, const mxArray *a,
01633         const QVec *Q=NULL, char check_type=1
01634     ){
01635         if (mxIsChar(a)) { wbsttring s(a); init(F_L,s.data); }
01636         else wbvector<TQ>::init(F,L,a,NULL,check_type);
01637
01638         if (Q) {
01639             unsigned n=Q->Qlen();
01640             if ((this->len)%n) wblog(FL,
01641                 "ERR %s() got invalid qset.len=%d @ %d",FCT,this->len,n
01642             );
01643         }
01644         return *this;
01645     };
01646
01647     qset& init(const QType &Q, const TQ *qq) {
01648         wbvector<TQ>::init(Q.Qlen(),qq);
01649         return *this;
01650     };
01651
01652     qset& init(const QVec &Q, const TQ *qq) {
01653         init(Q.Qlen(),qq);
01654         return *this;
01655     };
01656
01657     qset& init(const qset<TQ> &q1, const qset<TQ> &q2) {
01658         init(q1.len, q1.data, q2.len, q2.data);
01659         return *this;
01660     };
01661
01662     qset& init(const qset<TQ> &q1, const qset<TQ> &q2, const qset<TQ> &q3) {
01663         init(q1.len, q1.data, q2.len, q2.data, q3.len, q3.data);
01664         return *this;
01665     };
01666
01667     qset& init() { wbvector<TQ>::init(); return *this; };
01668
01669     qset& init(unsigned l, const TQ *d0=NULL) {
01670         wbvector<TQ>::RENEW(l,d0);
01671         return *this;
01672     };
01673
01674     qset& init(
01675         unsigned l1, const TQ *d1,
01676         unsigned l2, const TQ *d2
01677     );
01678
01679     qset& init(
01680         unsigned l1, const TQ* d1,
01681         unsigned l2, const TQ *d2,
01682         unsigned l3, const TQ *d3
01683     );
01684
01685     qset<TQ>& init(const TQ q1) { return init(1U,&q1); }
01686
01687     qset<TQ>& init(const TQ q1, const TQ q2) {
01688         return init(1,&q1,1,&q2);
01689     };
01690     qset<TQ>& init(const TQ q1, const TQ q2, const TQ q3) {
01691         return init(1,&q1,1,&q2,1,&q3);
01692     };
01693
01694     qset<TQ>& initStride(
01695         const TQ* d_, unsigned n, unsigned r, unsigned QDIM
01696     ){
01697         init(n*r); if (d_ && this->len) {
01698             Wb::cpyStride(this->data,d_,n,NULL,r,-1,QDIM);
01699         }
01700         return *this;
01701     };
01702
01703     bool operator<(const qset<TQ> &b) const;
01704
01705     int checkNQs(const QVec &qq, const char *F=NULL, int L=0);
01706
01707     bool isConsistent(
01708         const QVec &qq, int rank=-1, const char *F=NULL, int L=0) const;
01709
01710     unsigned QDim(const QVec &q, int rank=-1) const;
01711

```



```

01712     qset<TQ>& times(TQ fac, qset<TQ> &x) const {
01713         x=*this; x*=fac; return x;
01714     };
01715
01716     wstring toStr() const { return wvector<TQ>::toStr(); };
01717
01718     wstring toStr(const QType &t QS_UNUSED_VAR) const {
01719         if (CG::gotQAlpha(this->data, this->len)) {
01720             wstring sout(this->len+1);
01721             if (CG::qset2cstr(this->data, sout.data, this->len)) wblog(FL,
01722                 "ERR %s() failed to obtain compact qset string", FCT);
01723             sout.data[this->len]=0;
01724             return sout;
01725         }
01726         return wvector<TQ>::toStr();
01727     };
01728
01729 };
01730
01731 template <class TQ>
01732 bool qset<TQ>::operator<(const qset<TQ> &b) const {
01733
01734     if (!this->len || !b.len) wblog(FL,
01735         "ERR %s() got empty object (%d/%d)", FCT, this->len, b.len);
01736
01737     return wvector<TQ>::operator<(b);
01738 };
01739
01740 template <>
01741 bool qset<double>::operator<(const qset<double> &b) const {
01742
01743     if (!len || !b.len) wblog(FL,
01744         "ERR %s() got empty object (%d/%d)", FCT, len, b.len);
01745     if (len!=b.len) wblog(FL,
01746         "ERR %s() got length mismatch (%d/%d)", FCT, len, b.len);
01747
01748     double *d=b.data;
01749     for (unsigned i=0; i<len; i++) {
01750         if (float(data[i])<float(d[i])) return 1;
01751         if (float(data[i])>float(d[i])) return 0;
01752     }
01753     return 0;
01754 };
01755
01756 template <>
01757 bool qset<float>::operator<(const qset<float> &b) const {
01758     wblog(FL, "ERR %s() double precision preferred over '%s' (0x%lX)",
01759         TSTR(float), &b);
01760     return 0;
01761 };
01762
01763 template <class TQ>
01764 class QSet {
01765
01766 public:
01767
01768     QSet() : t(QTYPE_UNKNOWN) {};
01769
01770     QSet(const QSet &B) { t=B.t; qs=B.qs; qdir=B.qdir; };
01771
01772     QSet(const QType &t_, const qset<TQ> &qs_,
01773         unsigned iout=0,
01774         char isref=0
01775     ){
01776         init(t_, qs_, iout, isref);
01777     };
01778
01779     QSet(const QType &t_, const iTags &it, unsigned r=-1){
01780         if (int(r)<0) { r=it.len; } else
01781             if (r!=it.len) wblog(FL, "ERR %s() rank mismatch (%d/%d)", FCT, r, it.len);
01782         t=t_; qdir.init(it); qs.init(r*t.qlen());
01783     };
01784
01785     QSet(const CRef<TQ> &A) { init(A); };
01786
01787     QSet(const char *F, int L, const mxArray *a) { init(F, L, a); };
01788     QSet(const mxArray *a) { init(0, 0, a); };
01789
01790     QSet(const char *F, int L,
01791         const CRef<TQ> &A, const ctrIdx &ica,
01792         const CRef<TQ> &B, const ctrIdx &icb, wbpemr *P=NULL
01793     ){ init(F, L, A, ica, B, icb, P); };
01794
01795     QSet(const char *F, int L, const char *s) { init_str(F, L, s); };
01796
01797     QSet& init(const CRef<TQ> &A) {
01798         if (!A.cgb) init();

```

```

01799         else { init(*A.cgb);
01800             if (A.gotPerm()) Permute(A.cgp);
01801             if (A.gotConj()) Conj();
01802         }
01803         return *this;
01804     };
01805
01806     QSet& init(const char *F, int L, const mxArray *a, unsigned k=0);
01807
01808     QSet& init(const QType t_ = QTYPE_UNKNOWN) { t=QTYPE_UNKNOWN;
01809         t=t_; if (qs.len ) qs.init();
01810             if (qdir.len) qdir.init();
01811         return *this;
01812     };
01813
01814     QSet& init(const QSet &B) {
01815         t=B.t; qs=B.qs; qdir=B.qdir;
01816         return *this;
01817     };
01818
01819     QSet& init2ref(const QSet &B) {
01820         t=B.t; qs.init2ref(B.qs); qdir.init2ref(B.qdir);
01821         return *this;
01822     };
01823
01824     QSet& init(const QType &t_, const TQ* q, const char *qd);
01825
01826     QSet& init(
01827         const QType &t_, const qset<TQ>&qs_,
01828         unsigned iout=0,
01829         char isref=0
01830     );
01831
01832     QSet& init(
01833         const QType t_, const TQ* qs_, unsigned r, unsigned N,
01834         unsigned iout=0
01835     );
01836
01837     QSet& init(
01838         const QType t_, const TQ* qs_, unsigned r, unsigned N,
01839         const iTags &it
01840     );
01841
01842     QSet& init1J(const QType &t_, const TQ *q, char iflag=0) {
01843         unsigned m=t_.qlen();
01844         t=t_; qdir.init2val(2, iflag ? -1 : +1);
01845
01846         qs.init(2*m); memcpy(qs.data, q, m*sizeof(TQ));
01847         t.getDual(q, qs.data+m);
01848
01849         return *this;
01850     };
01851
01852     QSet& init2(const QType &t_, const TQ *q=NULL) {
01853         unsigned m=t_.qlen();
01854         t=t_; qdir.init_iout(FL,2,2); qs.init(2*m);
01855         if (q) { unsigned l=m*sizeof(TQ);
01856             memcpy(qs.data, q, l);
01857             memcpy(qs.data+m, q, l);
01858         }
01859         return *this;
01860     };
01861
01862     QSet& init2(const QType &t_,
01863         const qset<TQ> &q1, const qset<TQ> &q2, const char *qds="++") {
01864
01865         unsigned m=t_.qlen(), l=m*sizeof(TQ);
01866         if (q1.len<m || q2.len<m) wblog(FL,
01867             "ERR %s() invalid qset length (%s: %d,%d/%d)",
01868             FCT,STR(t_),q1.len,q2.len,m
01869         );
01870         t=t_; qdir=qds; qs.init(2*m);
01871         memcpy(qs.data, q1.data, l);
01872         memcpy(qs.data+m, q2.data, l);
01873         return *this;
01874     };
01875
01876     QSet& init1(const QType &t_, TQ *q_=NULL) {
01877         t=t_; qs.init(t_.qlen(),q_);
01878         qdir.init2val(1,+1);
01879         return *this;
01880     };
01881
01882     QSet& init3(const QType &t_) {
01883         t=t_; qs.init(3*t_.qlen());
01884         qdir.init_iout(FL,3,3); return *this;
01885     };

```

```

01886
01887 QSet& init3(const QType &t_,
01888     const qset<TQ> &J1, const qset<TQ> &J2, const qset<TQ> &J
01889 ) {
01890     unsigned n=t_.qlen();
01891     if (J1.len!=n || J1.len!=J2.len || J1.len!=J.len) {
01892         if (J1.len!=n || J2.len!=n || J.len!=n) wblog(FL,
01893             "ERR %s() severe length inconsistency [%d %d %d; %d]", FCT,
01894             J1.len,J2.len,J.len,n
01895         );
01896     }
01897     qs.init(J1,J2,J); t=t_;
01898     qdir.init_iout(FL,3,3);
01899
01900     return *this;
01901 };
01902
01903 QSet& init3(const QType &t_,
01904     const TQ &J1, const TQ &J2, const TQ &J
01905 ) {
01906     unsigned n=t_.qlen();
01907     if (n!=1) wblog(FL,
01908         "ERR %s() got QType %s (%d/1)",FCT,STR(t_),n);
01909     qs.init(3U); qs[0]=J1; qs[1]=J2; qs[2]=J;
01910     qdir.init_iout(FL,3,3); t=t_;
01911     return *this;
01912 };
01913
01914 QSet& init(const char *F, int L,
01915     const CRef<TQ> &A, const ctrIdx &ica,
01916     const CRef<TQ> &B, const ctrIdx &icb, wbperm *Pcgd=NULL
01917 );
01918
01919 QSet& init_str(const char *F, int L, const char *s);
01920
01921 QSet& Conj() { qdir.Conj(); return *this; };
01922 QSet& Conj(unsigned i);
01923
01924 unsigned rank(const char *F=NULL, int L=0) const {
01925     unsigned n=t_.qlen();
01926     if (n*qdir.len!=qs.len) wblog(F_L,
01927         "ERR %s() invalid qs (%d*d == %d?)",FCT,n,qdir.len,qs.len);
01928     return qdir.len;
01929 };
01930
01931 QSet& permute(QSet &B, const wbperm &P, char iflag=0) const;
01932
01933 QSet& Permute(const wbperm &P, char iflag=0) {
01934     QSet<TQ> X; permute(X,P,iflag);
01935     return X.save2(*this);
01936 };
01937
01938 void swap(QSet &X) {
01939     if (this!=&X) {
01940         SWAP(t,X.t); qdir.swap(X.qdir); qs.swap(X.qs);
01941     }
01942 };
01943
01944 QSet& save2(QSet &X) {
01945     if (this!=&X) { this->swap(X); init(); }
01946     return X;
01947 };
01948
01949 QSet& operator=(const QSet &B) { return init(B); };
01950
01951 bool operator==(const QSet &B) const {
01952     return (t==B.t && qs==B.qs && qdir==B.qdir);
01953 };
01954
01955 bool operator!=(const QSet &B) const {
01956     return (t!=B.t || qs!=B.qs || qdir!=B.qdir);
01957 };
01958
01959 char cmp(const QSet<TQ> &B) const {
01960     char i=0;
01961     if ((i=t.cmp(B.t))) { return i; }
01962     if ((i=qdir.cmp(B.qdir))) { return i; }
01963     return (i=qs.cmp(B.qs));
01964 };
01965
01966 bool operator<(const QSet<TQ> &B) const { return cmp(B)<0; }
01967
01968 explicit operator bool() const { return !isEmpty(); }
01969 bool operator! () const { return isEmpty(); }
01970
01971 bool isEmpty(char check=3) const {
01972     if (check>3) {

```

```

01973         if (check=='c' ) { check=2; } else
01974         if (check=='C' || check=='!') { check=3; } else
01975         wblog(FL,"ERR %s() invalid flag=%s",FCT,cSTR(check));
01976     }
01977
01978     if (qs.len || qdir.len) {
01979         if ( ((check&1) && t==QTYPE_UNKNOWN) ||
01980             ((check&2) && qs.len!=qdir.len*t.qlen()) ) {
01981             wblog(FL,"ERR %s() mismatch itag / QSet %s (%d*d / %d)",
01982                 FCT,STR_(this), qdir.len, t.qlen(), qs.len);
01983         }
01984         return 0;
01985     }
01986     return (t==QTYPE_UNKNOWN ? 1 : 0);
01987 };
01988
01989 bool gotRCDData() const;
01990
01991 int isZero() const;
01992
01993 unsigned isStd3() const;
01994
01995 unsigned checkStd3() const;
01996
01997 int init2opt3(
01998     const QSet<TQ> &Q, wbperm &P, wbvector<char> &cflags);
01999
02000 QSet& Convert(const char *s, wbvector<char> &cflags);
02001
02002 wbstring sizeStr() const;
02003
02004 bool isScalar() const;
02005 bool isJSymbol() const;
02006
02007 bool isAbelian() const { return t.isAbelian(); };
02008 int permitsOM() const { return t.permitsOM(rank(FL)); };
02009
02010 int checkQ_abelian(const char *F, int L) const;
02011
02012 int checkQ_SU2(const char *F, int L) const;
02013
02014 QSet& Sort(wbperm *cgp=NULL, char *conj=NULL, char iflag=0);
02015 bool isSorted() const;
02016
02017 size_t memSize() const {
02018     return ( sizeof(t)
02019         + sizeof(qs ) + qs .len*sizeof(TQ)
02020         + sizeof(qdir) + qdir.len*sizeof(char)
02021     );
02022 };
02023
02024 wbstring QStrS(const wbperm *cgp=NULL, char sep='|') const;
02025 wbstring QStr() const;
02026
02027 wbstring toStr() const { return toStr(NULL); };
02028 wbstring toStr(const char *istr) const;
02029
02030 wbstring toTag() const;
02031
02032 mxArray* toMx() const;
02033
02034 QType t;
02035 qset<TQ> qs;
02036
02037 QDir qdir;
02038
02039 protected:
02040 private:
02041
02042     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
02043     void add2MxStruct(mxAarray *S, unsigned i) const;
02044 };
02045
02046 template <class TQ>
02047 unsigned QSet<TQ>::isStd3() const {
02048
02049     unsigned q=0; { if (qdir!="++-") return q; }
02050
02051     unsigned m=t.qlen(), l=t.maxDimLocal(),
02052         d1=t.qdim(qs.data ),
02053         d2=t.qdim(qs.data+ m),
02054         d3=t.qdim(qs.data+2*m), dmax=MAX(1U<<6,l); dmax*=dmax;
02055
02056     if (qdir.len+m!=qs.len) wblog(FL,
02057         "ERR %s %d*d =? %d",FCT,STR_(this),qdir.len,m,qs.len);
02058
02059     q=(d1*d2<=dmax || d1<=1 || d2<=1) ? 1 : 2;

```

```

02060
02061     if (d1>2*d3 && (d2*d3<=dmax || d2<=1 || d3<=1)) q|=4;
02062     if (d2>2*d3 && (d1*d3<=dmax || d1<=1 || d3<=1)) q|=8;
02063
02064     return q;
02065 };
02066
02067 template <class TQ>
02068 unsigned QSet<TQ>::checkStd3() const {
02069
02070     unsigned q=0; { if (qdir.len!=3) return q; }
02071     unsigned m=t.qlen(), l=t.maxDimLocal(),
02072         d1=t.qdim(qs.data),
02073         d2=t.qdim(qs.data+m),
02074         d3=t.qdim(qs.data+2*m), dmax=MAX(1U<<6,1); dmax*=dmax;
02075
02076     if (qdir.len*m!=qs.len) wblog(FL,
02077         "ERR %s having %d*%d =? %d",FCT,STR_(this),qdir.len,m,qs.len);
02078
02079     if (d1*d2<=dmax || d1*d3<=dmax || d2*d3<=dmax) q|=(1<<0);
02080
02081     if (!q) { dmax=1U<<16;
02082         if (d1*d2<=dmax || d1*d3<=dmax || d2*d3<=dmax) q|=(1<<1);
02083     }
02084     if (!q) { dmax=1U<<19;
02085         if (d1*d2<=dmax || d1*d3<=dmax || d2*d3<=dmax) q|=(1<<2);
02086     }
02087
02088     if (d1<=1) q|=(1<<3);
02089     if (d2<=1) q|=(1<<4);
02090     if (d3<=1) q|=(1<<5);
02091
02092     if (d1<d2) {
02093         if (d2<=d3) q|=(3<<6);
02094         else q|=(2<<6);
02095     } else {
02096         if (d1<=d3) q|=(3<<6);
02097         else q|=(1<<6);
02098     }
02099
02100     return q;
02101 };
02102
02103 template <class TQ>
02104 int QSet<TQ>::init2opt3(
02105     const QSet<TQ> &Q, wbperm &iP, wbvector<char> &cflags) {
02106
02107     unsigned i3=Q.checkStd3(), w=(i3>6);
02108     unsigned m=Q.t.qlen();
02109     wbperm P;
02110
02111     if (!(i3&63)) { return -1; }
02112
02113     if (!w || w>3 || m*Q.qdir.len!=Q.qs.len) wblog(FL,
02114         "ERR %s() got %s\n(%d -> %d)",FCT,STR(Q),BITS(i3),BITS(w));
02115     if (w==1) P.init("231"); else
02116     if (w==2) P.init("132"); else
02117     if (w==3) P.init();
02118
02119     Q.permute(*this,P);
02120     Convert("++-",cflags);
02121
02122     if (Wb::cmpRange(qs.data,qs.data+m,m)>0) {
02123         wbperm p2="213"; Permute(p2);
02124         P.Permute(p2); cflags.Permute(p2);
02125     }
02126
02127     P.invert(iP);
02128
02129     return 0;
02130 };
02131
02132 template <class TQ>
02133 QSet<TQ>& QSet<TQ>::Convert(const char *s, wbvector<char> &cflags) {
02134
02135     unsigned i=0, r=(!s || !s[0] ? 0 : strlen(s));
02136
02137     if (r!=qdir.len) wblog(FL,
02138         "ERR %s() size mismatch %s [0/%d]",FCT,STR_(this),qdir.len);
02139     if (!s || !s[0]) { cflags.init(); return *this; }
02140
02141     cflags.init(r);
02142
02143     for (; i<r; ++i) {
02144         if ((s[i]=='+' && qdir[i]<0) || (s[i]=='-' && qdir[i]>0)) {
02145             Conj(i); cflags[i]=1;
02146         }

```

```

02147         else if (s[i]!='+' && s[i]!='-')
02148             wblog(FL,"ERR %s() invalid qdir='%s'",FCT,s);
02149     }
02150
02151     return *this;
02152 };
02153
02154 template <class TQ>
02155 QSet<TQ>& QSet<TQ>::Conj(unsigned i) {
02156     unsigned m=t.qlen();
02157     if (i>=qdir.len || !qdir[i]) wblog(FL,"ERR %s() "
02158         "index out of bounds %s (%d/%d)",FCT,STR_(this),i+1,qdir.len);
02159     if (m*qdir.len!=qs.len) wblog(FL,"ERR %s() "
02160         "invalid %s (%d*%d=? %d)",FCT,STR_(this),m,qdir.len,qs.len);
02161
02162     qdir[i]=--qdir[i];
02163     t.setDual(qs.data+i*m);
02164
02165     return *this;
02166 };
02167
02168 template <class TQ>
02169 wstring QSet<TQ>::sizeStr() const {
02170     unsigned m=t.qlen(), r=qdir.len;
02171
02172     if (r*m!=qs.len) wblog(FL,
02173         "ERR %s: %d != %d*%d",STR_(this),qs.len,r,m);
02174     try {
02175         if (qs.len) { wvector<size_t> dd(r);
02176             for (unsigned i=0; i<r; ++i) { dd[i]=t.qdim(qs.data+i*m); }
02177             return SSTR(dd);
02178         }
02179     }
02180     catch (...) {
02181         wblog(FL,"ERR %s() failed to obtain size(Q)\n%s",FCT,STR_(this));
02182     }
02183     return "";
02184 };
02185
02186 template <class TQ>
02187 class QHash {
02188 public:
02189     size_t operator()(const QSet<TQ> &S, const char *istr=0) const {
02190         unsigned i, j=1, r=S.qdir.len, nq=S.qs.len,
02191             l=1+(1+nq+r)/sizeof(long),
02192             lb=l*sizeof(long);
02193         unsigned long h=5381U, x[l];
02194         char *s=(char*)x;
02195
02196         const TQ *qs=S.qs.data;
02197
02198         if (!(r && nq) || (r && nq&r)) wblog(FL,
02199             "ERR %s() got QSet inconsistency (%d/%d)",FCT,nq,r);
02200         x[l-1]=0;
02201
02202         s[0]=S.t.type;
02203         s[1]=S.t.sub;
02204
02205         for (i=0; i<nq; ++i) {
02206             s[++j]=char(qs[i]);
02207         }; ++j;
02208
02209         if ((j+r)>lb) {
02210             wblog(FL,"ERR %s() string/id out of bounds (%d/(%d*%d))",
02211                 FCT,j,l,sizeof(long));
02212         }
02213
02214         memcpy(s+j,S.qdir.data,r);
02215
02216         if (istr && istr[0]) {
02217             unsigned n=strlen(istr); if (n>lb) {
02218                 wblog(FL,"ERR %s() istr `%s' out of bounds (%d/%d*%d)",
02219                     FCT,istr,n,l,sizeof(long));
02220             }
02221             for (j=lb-n, i=0; i<n; ++i) { s[j+i] ^= istr[i]; }
02222         }
02223
02224         for (i=0; i<l; ++i) { h ^= ((h<<6) + (h>>2)) + x[i]; }
02225
02226         return h;
02227     };
02228 };
02229
02230
02231
02232
02233

```

```

02234 template <class TD>
02235 class cdata : public wbsparray<TD> {
02236
02237 public:
02238
02239     cdata& init(SPIDX_T l=0) {
02240         wbsparray<TD>::init(l);
02241         return *this;
02242     };
02243
02244     cdata& init(
02245         SPIDX_T D1, SPIDX_T D2, SPIDX_T D,
02246         unsigned M=1
02247     ){
02248         if (M==1)
02249             wbsparray<TD>::init(D1,D2,D);
02250         else wbsparray<TD>::init(D1,D2,D,M);
02251         return *this;
02252     };
02253
02254     cdata& init(const wbvector<SPIDX_T> &S) {
02255         wbsparray<TD>::init(S); return *this;
02256     };
02257
02258     cdata& init(const cdata &C) {
02259         wbsparray<TD>::init(C);
02260         return *this;
02261     };
02262
02263     template <class TQ>
02264     cdata& init(const CRef<TQ> &R, unsigned l, char full=1);
02265
02266     template <class T2>
02267     cdata& init(const wbsparray<T2> &C) {
02268         wbsparray<TD>::init(C);
02269         return *this;
02270     };
02271
02272     cdata& init(
02273         const char *F, int L, const mxArray *a, unsigned k,
02274         const QType &q, unsigned r=-1
02275     );
02276
02277     cdata& initScalar(double x=1, unsigned r=-1) {
02278         wbsparray<TD>::initScalar(x,r);
02279         return *this;
02280     };
02281
02282     cdata& initIdentity(const QType &q, SPIDX_T d, TD dval=1);
02283     cdata& initIdentity(const wbvector<SPIDX_T> &S);
02284
02285     bool isScalar() const {
02286         return wbsparray<TD>::isScalar(FL,'z');
02287     };
02288
02289     unsigned len() const { return this->SIZE.len; };
02290
02291     unsigned getOM(
02292         const char *F=NULL, int L=0, const char *istr=NULL) const;
02293
02294     cdata& Conj() { return *this; };
02295
02296     cdata& permute(cdata &B, const wbperm &P0, char iflag=0) const;
02297     cdata& Permute(const wbperm &P0, char iflag=0) {
02298         cdata X; this->save2(X);
02299         return X.permute(*this,P0,iflag);
02300     };
02301
02302     cdata& Kron(const cdata &B){
02303         wbsparray<TD>::Kron(B,'N','N','k');
02304         return *this;
02305     };
02306
02307     TD contract(
02308         const char *F, int L, const ctrIdx &ica,
02309         const cdata &B, const ctrIdx &icb, cdata &C,
02310         cdata *Cx=NULL, wbperm *P=NULL, char normalize=1
02311     ) const;
02312
02313     double SkipTiny(const char *F=NULL, int L=0);
02314
02315     TD NormSignC(
02316         const char *F=NULL, int L=0, unsigned m=0,
02317         TD eps=CG_EPS1,
02318         TD eps2=CG_EPS2
02319     );
02320

```

```

02321     void info() const {
02322         printf("\n data: %s\n",this->sizeStr().data);
02323     };
02324
02325     protected:
02326     private:
02327 };
02328
02329 template <class TQ, class TD>
02330 class CData : public QSet<TQ> {
02331
02332     public:
02333
02334         CData() : cstat(CGD_UNKNOWN) { init(); };
02335
02336         CData(const CData &C)
02337             : QSet<TQ>((QSet<TQ>&)C), cgd(C.cgd), cstat(C.cstat) {};
02338
02339         explicit CData(const QSet<TQ> &Q) : QSet<TQ>(Q) {};
02340
02341         explicit CData(const CRef<TQ> &R, unsigned l) { init(R,l); }
02342
02343         CData& init() {
02344             this->t.init(); this->qs.init(); this->qdir.init();
02345             cgd.init(); cstat.init();
02346             return *this;
02347         };
02348
02349         CData& init(const QSet<TQ> &Q, bool full=true) {
02350             this->t=Q.t; this->qs=Q.qs; this->qdir=Q.qdir;
02351             if (full) {
02352                 cgd.init(); cstat.init();
02353             }
02354             return *this;
02355         };
02356
02357         CData& init(const CRef<TQ> &R, unsigned l);
02358
02359         CData& init(const CData &B) {
02360             this->t=B.t; this->qs=B.qs; this->qdir=B.qdir;
02361             cgd=B.cgd; cstat=B.cstat;
02362             return *this;
02363         };
02364
02365         template <class T2>
02366         CData& init(const CData<TQ,T2> &B) {
02367             this->t=B.t; this->qs.init(B.qs); this->qdir=B.qdir;
02368             cgd.wbsspararray<TD>::init(B.cgd); cstat=B.cstat;
02369             return *this;
02370         };
02371
02372         CData& operator=(const CData &B) { return init(B); }
02373
02374         CData& operator=(const QSet<TQ> &Q) { return init(Q,0); };
02375
02376         CData& save2(CData &B);
02377
02378         CData& initAbelian(const QSet<TQ> &Q) {
02379             this->t=Q.t; this->qs=Q.qs; this->qdir=Q.qdir;
02380             cstat.init(CGD_ABELIAN); cgd.init();
02381             return *this;
02382         };
02383
02384         CData& initScalar(const QSet<TQ> &Q) {
02385             if (!Q.t.isAbelian() && Q.qs.norm2()) {
02386                 wblog(FL,"ERR %s() got QSet %s",FCT,STR(Q)); }
02387             this->t=Q.t; this->qs=Q.qs; this->qdir=Q.qdir;
02388             cstat.init(CGD_EXPLICIT); cgd.initScalar(1,Q.qdir.len);
02389             return *this;
02390         };
02391
02392         CData& init3(const QType &t_,
02393             SPIDX_T D1, SPIDX_T D2, SPIDX_T D,
02394             unsigned M=1
02395         ){
02396             QSet<TQ>::init3(t_);
02397             cgd.init(D1,D2,D, M); cstat=CGD_UNKNOWN;
02398             return *this;
02399         };
02400
02401         CData& init3(const QType &t_,
02402             const qset<TQ> &J1, const qset<TQ> &J2, const qset<TQ> &J,
02403             unsigned M=1
02404         ){
02405             SPIDX_T D1,D2,D;
02406             QSet<TQ>::init3(t_,J1,J2,J);
02407             cstat.init(CGD_UNKNOWN);

```



```

02408
02409     D1=t_.qdim(J1.data);
02410     D2=t_.qdim(J2.data);
02411     D =t_.qdim(J .data); cgd.init(D1,D2,D,M);
02412
02413     return *this;
02414 };
02415
02416 CData& init3_abelian(const QType &t_,
02417     const TQ &J1, const TQ &J2, const TQ &J
02418 ) {
02419     QSet<TQ>::init3(t_,J1,J2,J);
02420     cgd.init();
02421     cstat.init(CGD_ABELIAN);
02422     return *this;
02423 };
02424
02425 int RefInit(
02426     const char *F, int L, const CData<TQ,TD> &B,
02427     char lflag=0, char bare=0);
02428
02429 CData& RefInit_auxtr(
02430     const char *F, int L,
02431     const wbvector<SPIDX_T> &S, const wbvector<RTD> &cgt
02432 );
02433
02434 int Reduce2Ref(const char *F=0, int L=0, char force=0);
02435 int LoadRef (const char *F=0, int L=0, char force=1);
02436
02437 int Load_CRef(
02438     const char *F, int L, CData &Cr, const char *cgd_file) const;
02439
02440 CData& initX3(
02441     const char *F, int L, const QSet<TQ> &Q,
02442     wbvector<char> &cflags, wbperm &P,
02443     unsigned loadRC=0, unsigned char flag=0);
02444
02445 int init_mxCRef(
02446     const char *F, int L, const mxArray *S, unsigned k
02447 );
02448
02449 TQ* qptra(unsigned k) const {
02450     unsigned n=this->t.qlen();
02451     if (this->qs.len%n || k>=this->qs.len/n) wblog(FL,
02452         "ERR %s() index out of bounds (%d/%d <> %d)",
02453         FCT,this->qs.len,n,k);
02454     return (this->qs.data+k*n);
02455 };
02456
02457 bool olderThan(const CData &B) const {
02458     if ((QSet<TQ>&)*this!=(QSet<TQ>&)B) wblog(FL,
02459         "ERR %s() got incompatible CData\n %s\n<> %s",
02460         FCT, STR_(this), STR(B)
02461     );
02462     checkSameStat(FL,B);
02463     return cstat.olderThan(B.cstat);
02464 };
02465
02466 bool operator!=(const CData &B) const { return !((*this)==B); };
02467 bool operator==(const CData &B) const {
02468     return (
02469         this->t==B.t && this->qs==B.qs && this->qdir==B.qdir &&
02470         cgd==B.cgd && cstat.sameAs(B.cstat,0)
02471     );
02472 };
02473
02474 int gotDiff(const CData &B, char lflag=0) const;
02475 bool sameAs(const CData &B, char lflag=0) const {
02476     return !gotDiff(B,lflag);
02477 };
02478
02479 bool operator!=(const QSet<TQ> &B) const {
02480     return (this->QSet<TQ>::operator!=(B));
02481 };
02482 bool operator==(const QSet<TQ> &B) const {
02483     return (this->QSet<TQ>::operator==(B));
02484 };
02485
02486 bool operator!=(const cgdStatus &b) const { return (cstat!=b); };
02487 bool operator==(const cgdStatus &b) const { return (cstat==b); };
02488
02489 bool isEmpty(char check=3) const {
02490     return (QSet<TQ>::isEmpty(check) && cgd.isEmpty());
02491 };
02492
02493 bool isAbelian(const char *F=NULL, int L=0) const {
02494     if (this->t.isAbelian()) {

```

```

02495         if (cstat.t==CGD_ABELIAN) {
02496             if (cgd.D.len || cgd.SIZE.len) wblog(F_L,
02497                 "ERR %s() invalid abelian CData\n%s",FCT,STR_(this));
02498             return 1;
02499         }
02500         if (!isScalar() || (cgd.D.len && (cgd.D.len>1 || cgd.D[0]!=1)))
02501             wblog(F_L,"ERR %s() invalid abelian CData\n%s",STR_(this));
02502         return 1;
02503     }
02504     return 0;
02505 };
02506
02507 bool sizeable(unsigned Dmin=1024) const {
02508     return (cgd.D.len>Dmin);
02509 };
02510
02511 bool isScalar(char dflag=0) const;
02512
02513 TD getScalar(const char *F=NULL, int L=0) const {
02514     if (!isScalar()) wblog(F_L,
02515         "ERR %s() got non-scalar CData\n%s",FCT,STR_(this));
02516     return (cgd.D.len ? cgd.D.data[0] : TD(1));
02517 };
02518
02519 unsigned rank(const char *F=NULL, int L=0) const;
02520
02521 unsigned rankS() const { return cgd.rank(); };
02522
02523 bool isrank(unsigned r, unsigned *r_=NULL) const {
02524     unsigned l=rank(F_L); if (r_) { (*r_)=l; }
02525     return (l!=r);
02526 };
02527
02528 template <class T>
02529 wbvector<T>& getSize(
02530     wbvector<T> &S, char bare=0, const wbperm *cgp=NULL) const;
02531
02532 SPIDX_T dim() const;
02533
02534 bool isRefInit(char check=1) const {
02535     if (cstat!=CGD_REF_INIT) { return 0; }
02536     if (check) {
02537         const unsigned r=cgd.SIZE.len, m=cgd.D.len;
02538         if (!r || (m>1 && m!=cgd.SIZE[r-1])) wblog(F_L,
02539             "ERR %s() invalid CData size ref (%s @ D.len=%d)",
02540             FCT, SSTR(cgd), cgd.D.len
02541         );
02542     }
02543     return 1;
02544 };
02545
02546 bool checkValidStat(const char *F=0, int L=0) const {
02547     if (this->t.isAbelian()) {
02548         if (cstat.hasID('l') || cstat.t!=CGD_ABELIAN) {
02549             if (F) wblog(F,L,"ERR %s() "
02550                 "abelian\n%s\n%s",FCT,STR_(this),STR2(cstat,'V'));
02551             return 0;
02552         }
02553     }
02554     else if (cstat.inValid() || cstat.t<=CGD_ABELIAN) {
02555         if (F) wblog(F,L,"ERR %s() "
02556             "non-abelian\n%s\n%s",FCT,STR_(this),STR2(cstat,'V'));
02557         return 0;
02558     }
02559     return 1;
02560 };
02561
02562 int checkSameStat(
02563     const char *F, int L, const CData &B, char lflag=1) const {
02564
02565     int e=0;
02566     if (!lflag) { if (cstat.t!=B.cstat.t) e=1; }
02567     if (cstat.ID!=B.cstat.ID || cstat.ctime!=B.cstat.ctime) e|=2;
02568
02569     if (e && F) {
02570         wblog(F_L,"WRN %s() cstat mismatch (e=%d)",FCT,e);
02571         wblog(F_L," > %s\n > %s\n... having\n > %s\n > %s",
02572             STR_(this), STR(B), STR2(cstat,'V'), STR2(B.cstat,'V'));
02573         if (e>1) wblog(F_L,"ERR %s() e=%d",FCT,e);
02574     }
02575     return e;
02576 };
02577
02578 bool sameType(const CData &S, const char *F=NULL, int L=0) const;
02579
02580 int cmpOM(const CData &B) const;
02581

```

```

02582     template <class DB>
02583     bool sameSizeR(
02584         const CData<TQ,DB> &B,
02585         unsigned *r=NULL, const char *F=NULL, int L=0) const;
02586
02587     template <class DB>
02588     bool sameSizeR(
02589         const cdata<DB> &B,
02590         unsigned *r=NULL, const char *F=NULL, int L=0) const;
02591
02592     template <class T2>
02593     int sameSizeR(const char *F, int L, const wvector<T2> &S) const;
02594
02595     bool isSymmetric() const;
02596
02597     CData& initOM(const char *F, int L);
02598
02599     unsigned getOM(const char *F=NULL, int L=0) const {
02600         unsigned m=0;
02601
02602         if (cgd.SIZE) { if (!(m=getOM(F,L))) { m=1; } } else
02603         if (this->qdir.len==2 && cgd.isDiag()) { m=1; } else
02604         if (cgd.isEmpty() && cstat==CGD_ABELIAN) { m=1; } else
02605         if (!isEmpty()) { wblog(FL,
02606             "WRN %s() got %s -> OM=%d",FCT,STR_(this),m);
02607             this->rank(F_L);
02608             cgd.wbspararray<TD>::checkSize(F_LF);
02609         }
02610         return m;
02611     };
02612
02613     unsigned gotOM(const char *F=NULL, int L=0) const {
02614         if (this->qdir.len) {
02615             unsigned l=cgd.SIZE.len, r=this->rank(F_L);
02616             if (l==r+1) {
02617                 SPIDX_T i=cgd.SIZE[r];
02618                 if (!i || (i>1 && !this->t.permitsOM(r))) wblog(FL,
02619                     "ERR %s() invalid OM setting (i=%d,r=%d)\n%s",
02620                     FCT,i,r,STR_(this));
02621                 return i;
02622             }
02623             else if (l && l!=r) wblog(FL,
02624                 "ERR %s() invalid OM setting\n%s",FCT,STR_(this)
02625             );
02626         }
02627         return 0;
02628     };
02629
02630     unsigned checkOM(const char *F=NULL, int L=0) const {
02631         if (this->qdir.len) {
02632             unsigned r=this->rank(F_L);
02633             if (this->t.permitsOM(r)) {
02634                 if (this->cgd.SIZE.len>r) {
02635                     SPIDX_T i=this->cgd.SIZE[r]; if (!i)
02636                     wblog(FL,"ERR %s() got OM=%d",FCT,i);
02637                     return i;
02638                 }
02639                 return 1;
02640             }
02641             else if (this->cgd.SIZE.len>r) wblog(FL,
02642                 "ERR %s() CData with invalid OM\n%s",FCT,STR_(this));
02643         }
02644         return 0;
02645     };
02646
02647     int gotFullOM(const char *F=NULL, int L=0) const {
02648
02649         unsigned r=this->qdir.len, m=getOM(F_L);
02650
02651         if (isEmpty()) { return -1; }
02652
02653         if (isAbelian(F_L)) { return 1; }
02654         if (r<2 && m!=1) wblog(FL,"ERR %s() %s with m=%d",FCT,STR_(this),m);
02655
02656         if (r<2) { return -2; }
02657         if (r>3) { return -4; }
02658
02659         if (r==2) {
02660             if (cstat.t!=CGD_IDENTITY && cstat.t!=CGD_1JSY_ST3 &&
02661                 cstat.t!=CGD_1JSY_GEN)
02662                 wblog(FL,"WRN %s() got %s",FCT,STR_(this));
02663             return 2;
02664         }
02665         else {
02666             if (cstat.t==CGD_STD3 || cstat.t==CGD_STD3_X
02667                 || this->t.sub<2) { return +3; }
02668             if (cstat.t!=CGD_FROM_CTR && cstat.t!=CGD_FROM_DEC &&

```

```

02669         cstat.t!=CGD_GEN_X) wblog(FL,
02670         "WRN %s() got %s",FCT,STR_(this));
02671         return -3;
02672     }
02673 };
02674
02675 int completeOM_DegQ(
02676     const char *F, int L, const wbperm &p,
02677     unsigned level=0
02678 );
02679
02680 CData& Project(
02681     const char *F, int L, cdata<TD> X,
02682     wbvector<TD> &w
02683 );
02684
02685 double normDiff(
02686     const char *F, int L, const CData &S,
02687     double eps=1E14
02688 ) const;
02689
02690 double norm2(unsigned k=-1) const;
02691
02692 wbvector<RTD>& trace(
02693     const char *F, int L, wbvector<RTD> &cgt) const;
02694
02695 wbvector<RTD> trace(const char *F=NULL, int L=0) const {
02696     wbvector<RTD> cgt; return trace(F,L,cgt);
02697 };
02698
02699 CData& trace(const char *F, int L,
02700     ctrIdx &i1, ctrIdx &i2, CData<TQ,TD> &X) const;
02701
02702 CData& AddMultiplicity(const char *F, int L, const cdata<TD> &c);
02703
02704 CData& AddMultiplicity(const char *F, int L, const CData &C) {
02705     if (this->t!=C.t || this->qs!=C.qs) wblog(F_L,
02706         "ERR %s() incompatible CData\n[%s] <> [%s] (%s, %s)",
02707         FCT, STR_(this), STR(C), sizeStr().data, C.sizeStr().data
02708     );
02709     return AddMultiplicity(F_L,C.cgd);
02710 };
02711
02712 cdata<TD>& getMultiplicity(unsigned im, cdata<TD> &c) const;
02713
02714 bool isBasicCG(const char *F=NULL, int L=0) const;
02715
02716 SPIDX_T getBasicCGSize(unsigned *m=NULL) const;
02717
02718 wbsparray<TD>&
02719 getBasicCG(unsigned k, wbsparray<TD> &a) const;
02720
02721 int getCG_set(
02722     const char *F, int L, wbIndex &Idx, wbsparray<TD> &a) const;
02723
02724 CData& reducetolJSymbol(const char *F, int L, CData<TQ,TD> &C) const;
02725
02726 int checkConsistency(const char *F=NULL, int L=0) const;
02727 int checkNormSign(const char *F=NULL, int L=0, char xflag=0) const;
02728
02729 int checkQ(const char *F, int L,
02730     const QSet<TQ> &Q, const char* istr=NULL) const;
02731
02732 bool checkAdditivityZ(const char *F, int L,
02733     const wbMatrix<double> &Z1,
02734     const wbMatrix<double> &Z2, const wbMatrix<double> &Z,
02735     TD eps=CG_EPS2
02736 ) const;
02737
02738 double SkipTiny(const char *F=NULL, int L=0);
02739
02740 CData& Permute(const wbperm &P, char iflag=0) {
02741     CData<TQ,TD> X; this->save2(X);
02742     return X.permute(*this,P,iflag);
02743 };
02744
02745 CData& permute(CData &B, const wbperm &P, char iflag=0) const {
02746     QSet<TQ>::permute((QSet<TQ>&)B,P,iflag);
02747
02748     cgd.permute(B.cgd,P,iflag);
02749     B.cstat=cstat;
02750
02751     return B;
02752 };
02753
02754 CData& Conj() { QSet<TQ>::Conj(); return *this; }
02755 CData& Conj(unsigned i);

```

```

02756
02757     size_t memSize() const {
02758         return (
02759             QSet<TQ>::memSize() + cgd.memSize() + sizeof(cstat)
02760         );
02761     };
02762
02763     void info(const char *istr=NULL, const char *F=NULL, int L=0) const;
02764
02765     wstring dStr() const { return this->qdir.toStr(); };
02766     wstring qStr() const { return this->t.toStr(); };
02767
02768     wstring toStr() const { return toStr(0); };
02769     wstring toStr(char vflag) const;
02770
02771     wstring sizeStr() const { return sizeStr(0); };
02772     wstring sizeStr(char xflag) const;
02773
02774     mxArray* toMx() const;
02775
02776     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
02777     void add2MxStruct(mxAarray *S, unsigned i, char tst=0) const;
02778
02779     mxArray* mxCreateCell(unsigned m, unsigned n) const {
02780         wblog(FL, "ERR %s(%d,%d)", FCT, m, n); return 0; };
02781     void add2MxCell(mxAarray *S, unsigned i, char tst=0) const {
02782         wblog(FL, "ERR %s(%d,%d)", FCT, S, i, tst); };
02783
02784     void put(const char *vname, const char *ws="caller"
02785             ) const { put(0,0,vname,ws); };
02786
02787     void put(const char *F, int L,
02788             const char *vname, const char *ws="caller"
02789             ) const {
02790         mxArray *a=toMx();
02791         int i=mexPutVariable(ws,vname,a);
02792
02793         if (i) wblog(FL,
02794             "ERR failed to write variable '%s' (%d)",vname,i);
02795         if (F) wblog(FL, "I/O putting '%s' to %s",vname,ws);
02796         mxDestroyArray(a);
02797     };
02798
02799     #define UBUF_ID 2
02800     #define UBUF_REF 2
02801
02802     void setUser_BUF(unsigned char c=1) { cstat.user[UBUF_ID] = c; };
02803
02804     void setUser_BUF(unsigned char c, unsigned l) { unsigned k=UBUF_ID;
02805         if (l>7 || (!l && c==1)) wblog(FL,
02806             "WRN %s() got u[%d]=%d (%d/%d)",FCT,k,cstat.user[k],l,c);
02807
02808         if (c) { cstat.user[k] |= (c<1); }
02809         else { c=(1<l); cstat.user[k] &= ~c; };
02810     };
02811
02812     void setUser_BUF_ref(unsigned char c=1) {
02813         if (c) { c=(1<UBUF_REF); cstat.user[UBUF_ID] |= c; }
02814         else {
02815             c=(1<UBUF_REF)-1; cstat.user[UBUF_ID] &= c;
02816         };
02817     };
02818
02819     bool gotuser_BUF() const { return cstat.user[UBUF_ID]; };
02820
02821     bool gotuser_BUF_ref() const { unsigned char c=(1<UBUF_REF);
02822         return (cstat.user[UBUF_ID] & c);
02823     };
02824
02825     cdata<TD> cgd;
02826
02827     cgdStatus cstat;
02828
02829 private:
02830 };
02831
02832 template <class TQ>
02833 class CRef {
02834 public:
02835     CRef( CGR_TYPE rt_=CGR_DEFAULT ) : cgb(NULL), conj(0), rtype(rt_) {};
02836
02837     CRef(const CRef &R) { init(R); };
02838     CRef(const CDATA_TQ &C, unsigned m=-1) : CRef() { initBase(C,m); };
02839
02840     CRef& init( CGR_TYPE rt_=CGR_DEFAULT ) {

```

```

02843         cgb=NULL; conj=0; rtype=rt_; cgp.init(); cgw.init();
02844         return *this;
02845     };
02846
02847     CRef& init(const CRef &R, char full=1) {
02848         if (this!=&R) {
02849             cgb=R.cgb; rtype=R.rtype; conj=R.conj; cgp=R.cgp;
02850             if (full) { cgw=R.cgw; }
02851         }
02852         return *this;
02853     };
02854
02855     CRef& operator=(const CRef &R) { return init(R); }
02856
02857     void swap(CRef &R) {
02858         if (this!=&R) { SWAP(cgb, R.cgb );
02859             cgp.swap(R.cgp); SWAP(conj, R.conj );
02860             cgw.swap(R.cgw); SWAP(rtype,R.rtype);
02861         }
02862     };
02863
02864     CRef& save2(CRef &R) {
02865         if (this!=&R) { R.cgb=cgb; cgb=0;
02866             cgp.swap(R.cgp); R.conj=conj; conj=0;
02867             cgw.swap(R.cgw); R.rtype=rtype; rtype=0;
02868             cgp.init(); cgw.init();
02869         }; return R;
02870     };
02871
02872     double safeCpy(const char *F, int L, const CRef &B, CRef &C) const;
02873
02874     CRef& init_l(const CRef &R, unsigned j) {
02875         unsigned m=R.wdim2(); if (j>=m) wblog(FL,
02876             "ERR %s() index out of bounds (%d/%d)\nhaving %s",FCT,j,m,STR(R));
02877         init(R,0); R.cgw.getCol(j,cgw);
02878         return *this;
02879     };
02880
02881     CRef& init_wId(const CRef &R,
02882         unsigned wid=1, unsigned ll=-1, const unsigned *d=NULL
02883     ){
02884         double w=1;
02885         unsigned m = (wid==1 ? R.wdim1() : R.wdim2());
02886         if (!wid || wid>2) { wblog(FL,"ERR %s() invalid wid=%d",FCT,wid); }
02887
02888         init(R,0);
02889
02890         if (int(ll)>=0) { unsigned d=qdim(ll);
02891             if (d_ && d!=(d_)) wblog(FL,
02892                 "ERR %s() qdim inconsistency (%d/%d)",FCT,d,d_);
02893             w=::sqrt(double(d));
02894         }
02895         else if (d_) {
02896             if (*d_<1) wblog(FL,"ERR %s() invalid qdim=%d",FCT,*d_);
02897             w=::sqrt(double(*d_));
02898         }
02899
02900         cgw.initIdentity(m,0,w);
02901         return *this;
02902     };
02903
02904     CRef& init(
02905         const char *F, int L, const mxArray *a, unsigned k,
02906         char refC=0, const QSet<TQ> *Q=NULL,
02907         char xflag=0
02908     );
02909
02910     CRef<TQ>& initIdentityR(const char *F, int L,
02911         const QType &q, const TQ *qs, unsigned dim=-1, char xflag=0);
02912
02913     CRef<TQ>& initIdentitylJ(const char *F, int L,
02914         const QType &q, const TQ *qs, unsigned dim=-1, char xflag=0);
02915
02916     CRef<TQ>& Reduce2Identity(char xflag=0);
02917
02918     wbvector<RTD>& trace(const char *F, int L, wbvector<RTD> &cgt) const;
02919
02920 #ifndef QS_SKIP_MPFR
02921     wbvector<double>& trace(const char *F, int L, wbvector<double> &cgt) const {
02922         wbvector<RTD> cgt_; trace(F,L,cgt_);
02923         return cgt.initT(cgt_);
02924     };
02925 #endif
02926
02927     wbvector<double> trace(const char *F, int L) const {
02928         wbvector<double> cgt;
02929         return trace(F,L,cgt);

```

```

02930     };
02931
02932     double trace() const;
02933
02934     double trace(const char *F, int L,
02935                 const CtrIdx &i1, const CtrIdx &i2, CRef *Rt=NULL) const;
02936
02937     CRef& initAbelian() {
02938         cgb=NULL; rtype=CGR_ABELIAN;
02939         conj=0; cgp.init(); cgw.init();
02940         return *this;
02941     };
02942
02943     CRef& initCtrScalar(double x=1) {
02944         cgb=NULL; rtype=CGR_CTR_SCALAR;
02945         conj=0; cgp.init(); cgw.init(1,1,&x);
02946         return *this;
02947     };
02948
02949     CRef& initBase(const CDATA_TQ *cgr, unsigned m=-1, const CDATA_TQ* X=NULL);
02950     CRef& initBase(const CDATA_TQ &C, unsigned m=-1);
02951
02952     int LoadRef(const char *F=0, int L=0, char force=1) const {
02953         if (!cgb) {
02954             wblog(FL, "WRN %s() got cgb=null\n%s", FCT, STR_(this));
02955             return 0;
02956         }
02957         return ((CDATA_TQ*)cgb)->LoadRef(F,L,force);
02958     };
02959
02960     int Reduce2Ref(const char *F=0, int L=0, char force=0) const {
02961         if (isRefInit(F,L,force)) { return 0; }
02962         if (!cgb) wblog(FL, "ERR %s() got cgb=null\n%s", FCT, STR_(this));
02963         return ((CDATA_TQ*)cgb)->Reduce2Ref(F,L,force);
02964     };
02965
02966     explicit operator bool() const { return !isEmpty(); }
02967     bool operator! () const { return isEmpty(); }
02968
02969     bool isEmpty() const {
02970         if (cgb && !cgb->isEmpty()) { return 0; }
02971
02972         size_t n=cgw.numel();
02973         if (n>1 || cgp.len) wblog(FL,
02974             "ERR %s() invalid abelian cref (%d)", FCT, STR_(this));
02975         return (rtype==CGR_DEFAULT && !n);
02976     };
02977
02978     bool isSymmetric(const char *F=0, int L=0) const;
02979
02980     bool isRefInit(const char *F=0, int L=0, char check=1) const {
02981         if (cgb && cgb->isRefInit(check)) {
02982             if (rtype!=CGR_DEFAULT) wblog(F_L,
02983                 "ERR %s() REF_INIT mismatch\n%s", FCT, STR_(this));
02984             return 1;
02985         }
02986         return 0;
02987     };
02988
02989     bool isAbelian(const char *F=NULL, int L=0) const;
02990
02991     bool isScalar(char dflag=0) const;
02992
02993     size_t isw2(const char *F=NULL, int L=0) const;
02994
02995     size_t isw3() const {
02996         return (cgw.SIZE.len==3 ? cgw.numel() : 0);
02997     };
02998
02999     unsigned Reduce_w3Id(unsigned d=-1, char lflag=0);
03000
03001     int cgw_check(const char *F, int L) const;
03002
03003     bool cgw_exists(unsigned i, unsigned j);
03004     int cgw_check_std3(
03005         const char *F=NULL, int L=-1, unsigned d3=0, unsigned im=0);
03006
03007     int checkQ(const char *F, int L, const QSet<TQ> &Q) const;
03008
03009     unsigned numel() const;
03010     unsigned wnumel() const { return cgw.numel(); }
03011
03012     bool wscalar(const char *F=NULL, int L=0) const;
03013     bool wscalar1(const char *F=NULL, int L=0) const {
03014         return (wscalar(F,L) && cgw[0]==1);
03015     };
03016

```

```

03017     double wel(unsigned k) const;
03018
03019     bool wom(const char *F=NULL, int L=0) const {
03020         return (isw2(F_L)>1); };
03021
03022     bool wdim_is(unsigned m) const {
03023         return (isw2() ? cgw.isSMatrix(m) : 0); };
03024
03025     unsigned wdim(const char *F=NULL, int L=1) const;
03026
03027     unsigned wdim1(const char *F=NULL, int L=0) const {
03028         return isw2(F_L); }
03029     unsigned wdim2(const char *F=NULL, int L=0) const { isw2(F_L);
03030         return cgw.SIZE[1]; }
03031
03032     unsigned wdim12_(const char *F, int L, unsigned &d2) const;
03033
03034     unsigned wdim12(
03035         const char *F, int L, unsigned &d2, unsigned d0=1) const;
03036
03037     unsigned wdim3(const char *F=NULL, int L=0) const;
03038
03039     double wget0() const;
03040     double wget1() const;
03041
03042     bool wSame(
03043         const wbarray<double> &cgw_, char lenient=0, double eps=1e-14) const;
03044
03045     template <class T>
03046     wvector<T>& getSize(wbvector<T> &S, char bare=0) const;
03047
03048     unsigned Size(
03049         unsigned k,
03050         unsigned r=-1) const;
03051
03052     unsigned qdim(
03053         unsigned k,
03054         const QType *qt=NULL) const;
03055
03056     int checkAbelian(const char *F=NULL, int L=0) const;
03057
03058     bool isDiagCSC(RTD eps=1E-14) const;
03059     bool isIdentityCG(wbvector<double> *nrm=NULL, double eps=1E-14) const;
03060
03061     int wisId() const;
03062
03063     double norm2(char checks=1) const;
03064
03065     double NormSignW(
03066         const char *F=NULL, int L=0,
03067         char useExt=1,
03068         double eps =CG_SKIP_DEPS1,
03069         double eps2=CG_SKIP_DEPS2
03070     );
03071
03072     char sameUptoFac(const CRef &B, double *fac=NULL, double eps=1E-12) const;
03073     char sameAs(const CRef &B, double eps=1E-12) const;
03074     char sameAs_fix(const char *F, int L, CRef &B, double eps=1E-12);
03075
03076     bool sameQSet(const QSet<TQ> &Q) const;
03077     bool sameQSet(const CRef &B) const;
03078     bool sameQDir(const char *F, int L, const QDir &qd) const;
03079
03080     bool sameQDir(const char *F, int L, const CRef& B) const {
03081         QDir qdb; B.get_qdir(qdb);
03082         return sameQDir(F,L,qdb);
03083     };
03084
03085     bool sameQDir(const iTags& b) const;
03086
03087     int gotSameCData(
03088         const char *F, int L, const CRef& B, char xflag=0
03089     ) const;
03090
03091     double NormStd(const char *F, int L, unsigned r=-1);
03092
03093     double normExt(const char *F=NULL, int L=0) const;
03094
03095     int SortDegQ(const char *F=NULL, int L=0, QSet<TQ> *QS=NULL);
03096
03097     bool isSortedDegQ(QSet<TQ> *QS, wbperm &pxt, char &cxt) const;
03098     bool isSortedDegQ(QSet<TQ> *QS=NULL) const {
03099         wbperm pxt; char cxt=0;
03100         return isSortedDegQ(QS,pxt,cxt);
03101     };
03102
03103     char got3(const char *F, int L,

```



```

03104     const QType &q, const qset<TQ> &J12, const qset<TQ> &J3
03105 ) const;
03106
03107 bool operator==(const CRef &B) { return (sameAs(B)==0 ? 1 : 0); };
03108 bool operator!=(const CRef &B) { return (sameAs(B)==0 ? 0 : 1); };
03109
03110 double normDiff2(const char *F, int L, const CRef &B) const;
03111 double normDiff (const char *F, int L, const CRef &B) const {
03112     return Wb::sqrt(normDiff2(F,L,B)); };
03113
03114 wbararray<double> & wProd(const CRef &B, wbararray<double> &x) const;
03115
03116 int check(const char *F, int L) const;
03117 int check(const char *F, int L,
03118     const QType &q, const QDir &qdir) const;
03119
03120 unsigned getOM(const char *F=NULL, int L=0, char wflag=0) const;
03121
03122 unsigned checkOM(const char *F=NULL, int L=0) const {
03123     return (cgb ? cgb->checkOM(F,L) : 0);
03124 };
03125
03126 unsigned rankS(char lflag=0) const;
03127 unsigned rank(const char *F=NULL, int L=0, char lflag=0) const;
03128
03129 CRef wget(unsigned j) const {
03130     CRef<TQ> R;
03131     return R.init(*this,j);
03132 };
03133
03134 CRef& Permute(const wperm &P, char iflag=0, char isnew=0);
03135 CRef& permute(
03136     const wperm &P, CRef &B, char iflag=0, char isnew=0
03137 ) const { B=(*this); return B.Permute(P,iflag,isnew); };
03138
03139 CRef& Conj() {
03140     if (cgb) { conj=(conj ? 0 : 1); }
03141     return *this;
03142 };
03143
03144 int HConjOpScalar();
03145
03146 bool gotConj() const { return (conj!=0); };
03147
03148 bool gotPerm(const char *F=NULL, int L=0) const {
03149     if (cgp.len) {
03150         if (!cgb) wblog(F_L,"ERR %s() "
03151             "invalid perm (len=%d; cgb=NULL)",FCT, cgp.len);
03152         if (cgp.len!=cgb->qdir.len) wblog(F_L,"ERR %s() "
03153             "invalid perm (len=%d/%d)",FCT, cgp.len, cgb->qdir.len);
03154         return !cgp.isIdentityPerm(F_L,cgb->qdir.len);
03155     }
03156     return 0;
03157 };
03158
03159 bool anyTrafo(char pflag=1) const {
03160     if (conj || cgp.len) return 1;
03161     if (pflag && !cgp.isIdentityPerm()) return 1;
03162     return 0;
03163 };
03164
03165 bool anyTrafo(char &ip, char &ic, char pflag=1) const {
03166     if (pflag)
03167         { ip=(cgp.len && !cgp.isIdentityPerm()); }
03168     else { ip=(cgp.len ? 1 : 0); }
03169     ic=(conj!=0);
03170     return (ip || ic);
03171 };
03172
03173 bool gotnoTrafos() const {
03174     return !anyTrafo(); };
03175
03176 unsigned getP(unsigned k) const {
03177     if (cgp.len) {
03178         if (k>=cgp.len) wblog(F_L,
03179             "ERR %s() index out of bounds (%d/%d)",FCT,k,cgp.len);
03180         return cgp.data[k];
03181     }
03182     return k;
03183 };
03184
03185 wperm& getP(wperm &p, char iflag=0) const {
03186     if (!cgp.len)
03187         return p.init(rank(F_L));
03188     else return p.init(cgp,iflag);
03189 };
03190

```

```

03191     char stat() const { char i=0;
03192         if (rtype!=CGR_DEFAULT) i ^= 1;
03193         if (conj) i ^= 2;
03194         if (cgp.len && !cgp.isIdentityPerm()) i ^= 4;
03195         return i;
03196     };
03197
03198     bool affectsQDir() const;
03199
03200     qset<TQ>& adapt(qset<TQ> &qs, char iflag=0) const {
03201         if (cgp.len) qs.BlockPermute(cgp,iflag ? 0 : 'i');
03202         return qs;
03203     };
03204
03205     QSet<TQ>& adapt(QSet<TQ> &Q, char iflag=0) const;
03206
03207     ctrIdx& adapt(ctrIdx &I, char iflag=0) const;
03208
03209     QDir& get_qdir(QDir &qd) const {
03210         if (!cgb) { qd.init(); return qd; }
03211
03212         if (cgp.len)
03213             cgb->qdir.permute(qd,cgp);
03214         else qd=cgb->qdir;
03215
03216         if (conj) qd.Conj();
03217
03218         return qd;
03219     };
03220
03221     QDir get_qdir() const { QDir qd;
03222         return get_qdir(qd);
03223     };
03224
03225     wstring qdir2Str(char vflag=0) const {
03226
03227         if (!cgb || !cgb->qdir.len) return "";
03228
03229         if (vflag<=0) return get_qdir().toStr();
03230         if (vflag==1 || vflag=='v') {
03231             unsigned l, n=2*(cgb->qdir.len)+5; char s[n];
03232             l=snprintf(s,n,"%s => %s",
03233                 cgb->qdir.toStr().data, STR(get_qdir()));
03234             if (l>=n) wblog(FL,
03235                 "ERR %s() string out of bounds (%d/%d)",FCT,l,n);
03236             return s;
03237         }
03238         else {
03239             unsigned l, n=2*(cgb->qdir.len)+5; char s[n];
03240             l=snprintf(s,n,"%s[%s]%s",
03241                 cgb->qdir.toStr().data, (cgp+1).toStr(1,"").data,
03242                 gotConj()?"*":"" );
03243             if (l>=n) wblog(FL,
03244                 "ERR %s() string out of bounds (%d/%d)",FCT,l,n);
03245             return s;
03246         }
03247     };
03248
03249     wstring toStr(char lflag=0) const;
03250
03251     wstring sizeStr() const;
03252
03253     wstring statStr(char vflag=0) const;
03254
03255     wstring qStr() const { return cgb ? cgb->qStr() : ""; };
03256     wstring QStr() const {
03257         return cgb ? cgb->QStrS(&cgp) : "";
03258     };
03259
03260     mxArray* toMx(char flag=0) const;
03261
03262     mxArray* toMX() const { return toMx('f'); };
03263
03264     mxArray* mxCreateStruct(unsigned m, unsigned n, char flag=0) const;
03265     void add2MxStruct(mxArray *S, unsigned i, char flag=0) const;
03266
03267     const CDATA_TQ *cgb;
03268     warray<double> cgw;
03269
03270     wperm cgp;
03271
03272     char conj;
03273
03274     cgrType rtype;
03275 };
03276
03277

```

```

03278     #define MAP32    std::map < qset<TQ>, CRef<TQ> >
03279     #define MAP31    std::map < qset<TQ>, MAP32 >
03280
03281     #define MP3_LDC    1
03282     #define MP3_LDR    2
03283
03284     #define TP3_ITER    3
03285     #define TP3_LDM    4
03286     #define TP3_LDR    8
03287     #define TP3_TST    16
03288     #define TP3_LOAD    (TP3_LDM|TP3_LDR)
03289
03290     #define LB_LOAD    4
03291     #define LB_UPD    8
03292     #define LB_CALC    16
03293     #define LB_REF    32
03294
03295     #define LB_GEN    (LB_LOAD | LB_CALC )
03296     #define LB_GET    (LB_LOAD | LB_CALC | LB_UPD )
03297     #define LB_ANY    (31<2)
03298
03299     #define QF_LOADC    128
03300
03301     #define CX3_SAVE    1
03302     #define CX3_FORCE    2
03303
03304     template <class TQ>
03305     class CStore {
03306     public:
03307
03308         CStore() {
03309             BUF.max_load_factor(0.2);
03310         };
03311
03312         void clear() {
03313             map3.clear();
03314             BUF.clear();
03315         };
03316
03317         typedef typename MAP32::iterator iMAP32;
03318         typedef typename MAP31::iterator iMAP31;
03319
03320         void init() {};
03321
03322         const CDATA_TQ& getIdentityC(const char *F, int L,
03323             const QType &t, const TQ *qs, unsigned dim=-1,
03324             unsigned loadRC=LB_GEN
03325         );
03326         const CDATA_TQ& getIdentity1J(
03327             const char *F, int L, CRef<TQ> &C,
03328             const QType &t, const TQ *qs, unsigned dim=-1,
03329             unsigned loadRC=LB_GEN
03330         );
03331
03332         void add_CData_abelian(
03333             const QType &q, const TQ &J1, const TQ &J2);
03334
03335         void add_CData(
03336             const QType &q, const qset<TQ> &J1, const qset<TQ> &J2);
03337
03338         void add2BUF(const char *F, int L, const CDATA_TQ &S, char nflag=0);
03339
03340         unsigned add3(const QType &q, const mxArray* S, char loadC=0);
03341         unsigned add2BUF(const QType &q, const mxArray* S, unsigned k=-1);
03342
03343         CDATA_TQ& getBUF(
03344             const char *F, int L, const QSet<TQ> &Q, unsigned loadRC=0
03345         );
03346
03347         size_t reduceMemUsage(const char *F=NULL, int L=0, double mfac=-1);
03348
03349         void getQfinal_l(const char *F, int L, const QType &type,
03350             const qset<TQ> &J1, const qset<TQ> &J2, wbMatrix<TQ> &J,
03351             wbMatrix< const CRef<TQ>* > *ss=NULL,
03352             char loadC=0
03353         );
03354
03355         void getQfinal_v(const char *F, int L, const QVec &type,
03356             const qset<TQ> &q1, const qset<TQ> &q2, wbMatrix<TQ> &QQ,
03357             wbMatrix< const CRef<TQ>* > *SS=NULL,
03358             char loadC=0
03359         );
03360
03361         int getQfinal_zdim(
03362             const char *F, int L, const QVec &qvec,
03363             const wbMatrix<TQ> &Q1, const wbMatrix<TQ> &Q2, const wbMatrix<TQ> &Q,

```

```

03365         wbvector<widx_t> &s1, wbvector<widx_t> &s2, wbvector<widx_t> &s,
03366         wbvector<unsigned> *m=NULL, wbMatrix< const CRef<TQ> * > *S3=NULL
03367     );
03368
03369     void getQfinal(const char *F, int L, const QVec &type,
03370         const wbMatrix<TQ> &Q1, const wbMatrix<TQ> &Q2, QMap<TQ> &M,
03371         unsigned char cgflag=0
03372     );
03373
03374     void Info(const char *F=0, int L=0, char vflag=0) const;
03375
03376     int checkInit(const char *F, int L, const QType &t) const;
03377
03378     unsigned LoadStore(const char *F, int L, const char *file);
03379
03380     mxArray* toMx(const QType &q=QTYPE_UNKNOWN) const;
03381
03382     void put(const char *F, int L,
03383         const char *vname, const char *ws="caller"
03384     ) const {
03385         mxArray *a=toMx(); int i=mexPutVariable(ws,vname,a);
03386
03387         if (i) wblog(FL,
03388             "ERR failed to write variable '%s' to %s (%d)",vname,i,ws);
03389         if (F) wblog(FL,"I/O putting '%s' to %s",vname,ws);
03390         mxDestroyArray(a);
03391     };
03392
03393     void put(const char *vname, const char *ws="caller") const {
03394         put(0,0,vname,ws);
03395     };
03396
03397     int find_map3(const QType &q, const qset<TQ> &J1, const qset<TQ> &J2
03398     ) const {
03399         qset<TQ> J12(J1,J2);
03400         if (J1.len!=J2.len || J1.len!=q.qlen()) wblog(FL,
03401             "ERR %s() qset length inconsistency (%d/%d/%d)",
03402             FCT,J1.len,J2.len,q.qlen());
03403         return find_map3(q,J12);
03404     };
03405
03406     int find_map3(const QType &q, const qset<TQ> &J12) const {
03407         if (J12.len!=2*q.qlen()) {
03408             if (!q.isKnown()) wblog(FL,
03409                 "ERR %s() got uninitialized symmetry",FCT); else
03410                 wblog(FL,"ERR %s() "
03411                     "qset length inconsistency (%d/2*%d)",FCT,J12.len,q.qlen()
03412                 );
03413         }
03414         auto it1 = map3.find(q);
03415         if (it1!=map3.end()) {
03416             auto it2 = it1->second.find(J12);
03417             if (it2!=it1->second.end()) { return it2->second.size(); }
03418         }
03419         return -1;
03420     };
03421
03422     int valid_mp3_data(const QType &t,
03423         const qset<TQ> &J1, const qset<TQ> &J2, const qset<TQ> &J3,
03424         char rflag=0
03425     ) const;
03426
03427     CDATA_TQ* BUF_find(const QSet<TQ> &Q);
03428
03429     int Reduce2Ref(
03430         const char *F, int L, const QSet<TQ> &Q,
03431         char force=0, unsigned Dmin=1024);
03432
03433     map<QType,
03434         map<qset<TQ>,
03435             map<qset<TQ>,
03436                 CRef<TQ>
03437             >
03438         >
03439     > map3;
03440
03441     unordered_map<
03442         QSet<TQ>,
03443         CDATA_TQ,
03444         QHash<TQ>
03445     > BUF;
03446
03447 protected:
03448 private:
03449
03450     iMAP31 get_mp3_data(const char *F, int L,
03451         const QType &q, const qset<TQ> &J1, const qset<TQ> &J2,

```

```

03452     char force=0
03453 ) {
03454     if (J1.len!=J2.len) wblog(FL,
03455         "ERR qset length inconsistency (%d/%d)", J1.len, J2.len);
03456
03457     iMAP3l iJ12;
03458     qset<TQ> J12(J1, J2);
03459
03460     auto itype = map3.find(q); if (itype==map3.end()) { checkInit(FL, q);
03461         itype = map3.find(q); }
03462
03463     if (itype!=map3.end()) {
03464         iJ12 = itype->second.find(J12);
03465         if (iJ12!=itype->second.end()) {
03466             if (!iJ12->second.size()) wblog(FL,
03467                 "ERR %s() got empty map3 data (%s)", FCT, STR(q));
03468             return iJ12;
03469         }
03470     }
03471
03472     if (!force) {
03473         add_CData(q, J1, J2);
03474         return get_mp3_data(F, L, q, J1, J2, 'f');
03475     }
03476
03477     wblog(F, L, "ERR failed to find or generate map3 target\n"
03478         "%s [%s] x [%s] => [%s]", STR(q), STR(J1), STR(J2), STR(J12));
03479     return iJ12;
03480 };
03481
03482 void print_Coeff(const char *F, int L, const QType &type,
03483     const qset<TQ> &J1, const qset<TQ> &J2, const qset<TQ> &J,
03484     double c
03485 ) {
03486     wblog(F, L, " * CG coeff. %-8s [%s, %s; %s] = %.6g",
03487         type.toStr(), STR(J1), STR(J2), STR(J), c);
03488 };
03489 };
03490
03491 CStore<gTQ> gCS;
03492
03493 CData<gTQ, double> EMPTY_CGSTORE;
03494
03495 mxArray* map3_CreateStructMatrix(unsigned m, unsigned n);
03496
03497 template <class TQ>
03498 void map3_add2MxStruct(mxArray *S, unsigned k,
03499     const QType &q, const qset<TQ> &J12);
03500
03501 gTQ get_qtot_abelian(
03502     const QType &t, const gTQ *q0, const widx_t n,
03503     const widx_t *idx=NULL, const widx_t stride=1
03504 );
03505
03506 template <class TM>
03507 class cgc_contract_id : public wbvector<TM> {
03508
03509 public:
03510
03511     cgc_contract_id() { wbvector<TM>::init(); };
03512
03513     template <class TQ, class TD>
03514     cgc_contract_id(
03515         const CData<TQ, TD> &A, const ctrIdx &ica,
03516         const CData<TQ, TD> &B, const ctrIdx &icb
03517     );
03518
03519     template <class TQ, class TD>
03520     void extract(
03521         CData<TQ, TD> &A, ctrIdx &ica,
03522         CData<TQ, TD> &B, ctrIdx &icb,
03523         char xflag=1
03524     ) const;
03525
03526     size_t memSize() const {
03527         return ( sizeof(*this) + this->len*sizeof(TM) );
03528     };
03529
03530     void wblog_(const char* F=0, int L=0, const char *istr=0) const {
03531         #ifdef QS_USING_OMP
03532             Wb::ompGuard myLK(wblog_lock, 1);
03533         #endif
03534
03535         CData<gTQ, RTD> a, b; ctrIdx ica, icb;
03536         this->extract(a, ica, b, icb, 'l');
03537
03538         wblog(F, L, "TST cgc_contract_id() %s", istr ? istr : "");

```

```

03539         wblog(F_L, "\b a: %-32s @ %s\n b: %-32s @ %s\n\n    %s\n    %s\n",
03540             STR(a), STR(ica), STR(b), STR(icb),
03541             STR2(a.cstat, 'V'), STR2(b.cstat, 'V')
03542         );
03543     };
03544
03545     wbsttring toStr(char vflag=7) const {
03546         wbsttring sout;
03547
03548         CData<gTQ,RTD> a,b; ctrIdx ica,icb;
03549         this->extract(a,ica,b,icb, vflag & 6 ? '1' : 0);
03550
03551         if (vflag&4) {
03552             sout.init(256); snprintf(sout.data,sout.len,
03553                 "%-20s @%s #05x | %-20s @%s #05x",
03554                 STR2(a,1), STR(ica), a.cstat.ID,
03555                 STR2(b,1), STR(icb), b.cstat.ID
03556             );
03557         }
03558         else if (vflag&2) {
03559             sout.init(128); snprintf(sout.data,sout.len,
03560                 "%-20s @%s | %-20s @%s",
03561                 STR2(a,1), STR(ica), STR2(b,1), STR(icb)
03562             );
03563         }
03564         else {
03565             sout.init(64); snprintf(sout.data,sout.len,
03566                 "%s_%s * %s_%s",STR2(a,0), STR(ica), STR2(b,0), STR(icb));
03567         }
03568         return sout;
03569     };
03570
03571 protected:
03572 private:
03573
03574     template <class T1, class T2>
03575     unsigned set_val(const char *F, int L, T1* &s, const T2 &x) {
03576         s[0]=x;
03577         if (T2(s[0])!=x) wblog(F_L,"ERR got non-%s data entry (%g)",
03578             TSTR(T1), double(x)
03579         );
03580         s+=1; return 1;
03581     };
03582
03583     template <class T1, class T2>
03584     unsigned set_vec(const char *F, int L, T1* &s, const wbvector<T2> &x) {
03585         s[0]=x.len;
03586         if (unsigned(s[0])!=x.len) wblog(F_L,"ERR unexpected size "
03587             " (out of bounds %s @ %d)",TSTR(T1),x.len
03588         );
03589         set_range(F_L,++s,x.len,x.data);
03590         return (1+x.len);
03591     };
03592
03593     template <class T1, class T2>
03594     unsigned set_range(const char *F, int L, T1* &s, unsigned n, const T2 *x) {
03595         for (unsigned i=0; i<n; ++i) {
03596             s[i]=x[i];
03597             if (T2(s[i])!=x[i]) wblog(F_L,"ERR got non-%s data entry (%g)",
03598                 TSTR(T1), double(x[i])
03599             );
03600         }
03601         s+=n; return n;
03602     };
03603
03604     template <class TQ, class TD>
03605     unsigned set_CData(const char *F, int L,
03606         char* &s, unsigned n, const CData<TQ,TD> &A, char tflag=1
03607     ){
03608         char *s0=s;
03609         unsigned r=A.rank(), l=4+A.qs.len+r+(4+r)*sizeof(unsigned);
03610
03611         wbvector<size_t> S;
03612
03613         if (l>=n) wblog(F_L,"ERR %s() string out of bounds (%d/%d)",FCT,l,n);
03614
03615         if (tflag) {
03616             set_val(F_L,s, A.t.type);
03617             set_val(F_L,s, A.t.sub );
03618         }
03619
03620         set_vec(F_L,s, A.qs);
03621         set_vec(F_L,s, A.qdir);
03622
03623         unsigned *u=(unsigned*)s;
03624
03625         set_vec(F_L,u, A.getSize(S, 'b'));

```

```

03626     s+=(u-(unsigned*)s)*sizeof(unsigned); l=s-s0;
03627     if (l>=n) wblog(F_L,
03628         "ERR %s() string out of bounds (%d/%d)",FCT,l,n
03629     );
03630     };
03631     return l;
03632 };
03633
03634 template <class T1, class T2>
03635 void get_range(
03636     const char *F, int L, const T1* &s, unsigned n, T2 *x) const {
03637     for (unsigned i=0; i<n; ++i) {
03638         x[i]=s[i];
03639         if (T1(x[i])!=s[i]) wblog(F_L,"ERR got non-%s data entry (%g)",
03640             TSTR(T1), double(x[i])
03641         );
03642     }
03643     s+=n;
03644 };
03645
03646 template <class T1, class T2>
03647 void get_vec(
03648     const char *F, int L, const T1* &s, wbvector<T2> &x) const {
03649     if (!s || int(s[0])<0) wblog(F_L,
03650         "ERR %s() got invalid vec.len=%d",FCT,s?s[0]:-1);
03651     x.init(s[0]); ++s;
03652     get_range(F,L,s,x.len,x.data);
03653 };
03654
03655 template <class T1, class T2>
03656 void get_val(
03657     const char *F, int L, const T1* &s, T2 &x) const {
03658     get_range(F,L,s,1,&x);
03659 };
03660
03661 template <class TQ, class TD>
03662 unsigned get_CData(const char *F, int L,
03663     const char* &s, CData<TQ,TD> &A, char tflag=1) const {
03664     A.init();
03665     if (tflag) {
03666         A.t.initx(QTYPE_SET(s[0]),unsigned(s[1]));
03667         s+=2;
03668     }
03669     get_vec(F_L,s, A.qs);
03670     get_vec(F_L,s, A.qdir);
03671
03672     const unsigned *u = (const unsigned*)s;
03673     get_vec(F_L,u, A.cgd.SIZE);
03674     A.cstat.init0(CGD_BSZ_INIT);
03675
03676     s+=(u-(unsigned*)s)*sizeof(unsigned);
03677     return 0;
03678 };
03679
03680 template <class TM>
03681 class MHash {
03682 public:
03683     size_t operator()(const cgc_contract_id<TM> &S) const {
03684         MTI h=5381U, *const x=S.data;
03685         for (unsigned i=0; i<S.len; ++i) {
03686             h ^= ((h<<6) + (h>>2)) + x[i];
03687             h ^= ((h<<6) + (h>>2)) + x[i];
03688         }
03689         return h;
03690     };
03691 };
03692
03693 template <class TQ, class TD>
03694 class x3map {
03695 public:
03696     x3map() : cgb(NULL), conj(0), zflag(0), rtype(CGR_DEFAULT)
03697     #ifndef QS_USING_MPFR
03698     , X3(x3)
03699     #endif
03700     {};
03701 };

```

```

03713
03714     x3map(const char *F, int L,
03715           const CRef<TQ> &A_, const ctrIdx &ica,
03716           const CRef<TQ> &B_, const ctrIdx &icb, wbperm &cgp, char xflag=0
03717           ) : x3map() { initCtr(F_L,A_,ica,B_,icb, cgpp, xflag); };
03718
03719     x3map& init() {
03720         a.init(); b.init(); c.init(); pab.init(); x3.init();
03721         #ifdef QS_USING_MPFR
03722             X3.init();
03723         #endif
03724         cgb=NULL; conj=0; zflag=0; rtype=CGR_DEFAULT;
03725         return *this;
03726     };
03727
03728     int initCtr(const char *F, int L,
03729                const CRef<TQ> &A_, const ctrIdx &ica_,
03730                const CRef<TQ> &B_, const ctrIdx &icb_, wbperm &cgp,
03731                char xflag=1
03732                );
03733
03734     x3map& init(
03735         const char *F, int L, const mxArray* S, unsigned k=0,
03736         ctrIdx *ica=NULL, ctrIdx *icb=NULL, char recalc=1
03737     );
03738
03739     bool isEmpty() const {
03740         return (
03741             a.isEmpty() && b.isEmpty() && c.isEmpty() &&
03742             !pab.len && x3.isEmpty()
03743         );
03744     };
03745
03746     void print(
03747         const char *F=NULL, int L=0, const char *istr="", char vflag=0
03748         ) const;
03749
03750     wbstring toStr(char vflag=0) const;
03751     wbstring x3Str(char vflag=0) const;
03752
03753     size_t memSize() const {
03754         return ( sizeof(cgb) + sizeof(char)
03755             + a.memSize() + b.memSize() + c.memSize()
03756             + sizeof(pab) + pab.len*sizeof(widx_t)
03757             + sizeof(x3) + x3.numel()*sizeof(TD)
03758             + sizeof(rtype)
03759         );
03760     };
03761
03762     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
03763
03764     void add2MxStruct(
03765         mxArray *S, unsigned l,
03766         const cgc_contract_id<MTI> *idc=NULL) const;
03767
03768     mxArray* toMx(const cgc_contract_id<MTI> *idc=NULL) const {
03769         mxArray *S=mxCreateStruct(1,1);
03770         add2MxStruct(S,0,idc);
03771         return S;
03772     };
03773
03774     CData<TQ,RTD> a, b, c;
03775
03776     const CData_TQ* cgb;
03777
03778     wbperm pab;
03779
03780     char conj;
03781
03782     int zflag;
03783
03784     cgrType rtype;
03785
03786     wbarray<TD> x3;
03787
03788     #ifdef QS_USING_MPFR
03789         wbarray<RTD> X3;
03790     #else
03791         wbarray<TD> &X3;
03792     #endif
03793
03794     protected:
03795     private:
03796 };
03797
03798 template <class TQ, class TD>
03799 class X3Map {

```



```

03800
03801 public:
03802
03803     X3Map() {
03804         XBUF.max_load_factor(0.2);
03805     };
03806
03807     void clear() {
03808         XBUF.clear();
03809     };
03810
03811     int init(const char *F, int L, const mxArray *S, unsigned k=0);
03812
03813     x3map<TQ,TD>& getXBUF(
03814         const char *F, int L,
03815         const cgc_contract_id<MTI> &idc
03816     );
03817
03818     int contains(const char *F, int L,
03819         const x3map<TQ,TD> &x, const cgc_contract_id<MTI> &idc);
03820
03821     int contractCGR(const char *F, int L,
03822         const CRef<TQ> &A, const ctrIdx &ica,
03823         const CRef<TQ> &B, const ctrIdx &icb, CRef<TQ> &C,
03824         char cg_preview=0
03825     );
03826
03827     CRef<TQ>& contractDegQ(const char *F, int L,
03828         const CRef<TQ> &A, CRef<TQ> &B);
03829
03830     unsigned add(
03831         const char *F, int L, const cgc_contract_id<MTI> &idc,
03832         const mxArray* S, unsigned k
03833     );
03834
03835     mxArray* toMx() const;
03836
03837     void Info(const char *F=0, int L=0) const;
03838
03839     unordered_map <
03840         cgc_contract_id<MTI>,
03841         x3map<TQ,TD>,
03842         MHash<MTI>
03843     > XBUF;
03844
03845 protected:
03846 private:
03847 };
03848
03849     X3Map<gTQ,double> gXS;
03850
03851 template <class TQ>
03852 class QMap {
03853
03854 public:
03855
03856     void init() { qvec.init();
03857         Q1.init(); Q2.init(); Q.init();
03858         I1.init(); I2.init(); II.init(); D.init();
03859     };
03860
03861     QMap<TQ>& init(const char *F, int L,
03862         const QVec &qv, const wbMatrix<TQ> &q1, const wbMatrix<TQ> &q2,
03863         const wbMatrix< wbMatrix<TQ> > &QQ,
03864         const wbMatrix< wbMatrix< const CRef<TQ>* > > *SS=NULL
03865     );
03866
03867     QMap<TQ>& init(const char *F, int L, const mxArray *a);
03868
03869     template<class TD>
03870     int getCGZlist(const char *F, int L,
03871         wbMatrix<TQ> &qc,
03872         wbvector<TD> &dc,
03873         wbvector<widx_t> *IC=NULL,
03874         const wbvector<widx_t>* Ix=NULL,
03875         const qset<TQ> *q3=NULL,
03876         wbIndex *m3=NULL,
03877         const wbvector<TD>* cc=NULL,
03878         double eps1=1E-10,
03879         double eps2=1E-14
03880     ) const;
03881
03882     void Skip(const wbindex &I);
03883
03884     template<class TD>
03885     void getIdentityQ(const char *F, int L,
03886         const wbvector<widx_t> &Sa, const wbvector<widx_t> &Sb,

```

```

03887     QSpace<TQ,TD> &A, char vflag=1
03888 ) const;
03889
03890 bool isConsistent(
03891     const char *F=NULL, int L=0, const QVec *qv=NULL
03892 ) const;
03893
03894 mxArray* mxCreateStruct(unsigned m, unsigned n) const;
03895 void add2MxStruct(mxArray *S, unsigned i, char tst=0) const;
03896
03897 mxArray* toMx() const {
03898     mxArray *S=mxCreateStruct(1,1);
03899     add2MxStruct(S,0); return S;
03900 };
03901
03902 void put(const char *vname, const char *ws="caller") const
03903 { put(0,0,vname,ws); }
03904
03905 void put(const char *F, int L,
03906     const char *vname, const char *ws="caller"
03907 ) const {
03908     mxArray *a=toMx(); int i=mexPutVariable(ws,vname,a);
03909
03910     if (i) wblog(F,L,
03911         "ERR failed to write variable '%s' (%d)",vname,i);
03912     if (F) wblog(F,L,"I/O putting '%s' to %s",vname,ws);
03913     mxDestroyArray(a);
03914 };
03915
03916 QVec qvec;
03917
03918 wbMatrix<TQ> Q1,Q2,Q;
03919
03920 wbvector<wdx_t> D;
03921
03922 wbvector<wdx_t> I1, I2;
03923 wbMatrix<wdx_t> II;
03924
03925 wbMatrix< const CRef<TQ>*> cg3;
03926
03927 };
03928
03929 template <class TQ>
03930 bool QMap<TQ>::isConsistent(const char *F, int L, const QVec *qv) const {
03931
03932     unsigned d=qvec.Qlen();
03933
03934     if (Q1.dim2!=d || Q2.dim2!=d) {
03935         if (!F) return 0; else wblog(F,L,
03936             "ERR QMap: invalid qset record length (%d,%d/%d)",
03937             Q1.dim2, Q1.dim2, d);
03938     }
03939     if (qv && (!qvec.sameType(*qv) || d!=(qv->Qlen()))) {
03940         if (!F) return 0; else wblog(F,L,
03941             "ERR QMap: Q type mismatch: %s; %s (%d/%d)",
03942             qv->toStr().data, STR(qvec), qv->Qlen(), d);
03943     }
03944     return 1;
03945 };
03946
03947 template <class T>
03948 class pairPatternCG {
03949 public:
03950
03951     pairPatternCG& init(unsigned i0=0, unsigned j0=0, unsigned n=0) {
03952         i=i0; j=j0; k.init(n); fac.init(n);
03953         return *this;
03954     };
03955
03956     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
03957     void add2MxStruct(mxArray *S, unsigned i) const;
03958
03959     unsigned i,j;
03960     wbvector<unsigned> k;
03961     wbvector<T> fac;
03962
03963 protected:
03964 private:
03965 };
03966
03967 template <class TQ, class TD>
03968 class genRG_base{
03969
03970 public:
03971
03972     genRG_base() : err(0) {};
03973     genRG_base(const genRG_base& S) { init(S); }

```

```

03974
03975     bool isEmpty() const {
03976         return (!q && !Sp.len && !Sz.len && !J.len && Z.isEmpty());
03977     };
03978
03979     double normDiff(const char *F, int L,
03980         const genRG_base& B,
03981         TD eps=1E12
03982     ) const;
03983
03984     void checkOrthoCommRel(
03985         const char *F, int L, const genRG_base &B) const;
03986
03987     genRG_base& Sort();
03988     double SkipTiny(const char *F=0, int L=0);
03989
03990     genRG_base& ApplyQFac(
03991         const wvector<double> &qfac, const wbararray<double> *JM=NULL);
03992
03993     genRG_base& init() {
03994         q=0; J.init(); Z.init(); Sp.init(); Sz.init();
03995         err=0; istr.init();
03996         return *this;
03997     };
03998
03999     genRG_base& init(const genRG_base& S) {
04000         q=S.q; J=S.J; Z=S.Z; Sp=S.Sp; Sz=S.Sz;
04001         err=S.err; istr=S.istr;
04002         return *this;
04003     };
04004
04005     genRG_base& operator=(const genRG_base& S) { return init(S); }
04006
04007     SPIDX_T dim() const {
04008         if (!Sz.len || Sz.len!=Sp.len) wblog(FL,
04009             "ERR %s() got empty generators for %s [%s]",
04010             FCT,STR(q),STR(J),Sz.len,Sp.len);
04011         SPIDX_T d=Sz[0].dim();
04012         if (d!=Sp[0].dim()) wblog(FL,"ERR %s() got "
04013             "inconsistent Sp/z dimensions (%d/%d)",FCT,Sp[0].dim(),d);
04014         return d;
04015     };
04016
04017     void compareStdSU2(const char *F=NULL, int L=0) const;
04018
04019     genRG_base& save2(genRG_base& S) {
04020         S.q=q; J.save2(S.J); Z.save2(S.Z); q=0;
04021         Sp.save2(S.Sp); Sz.save2(S.Sz);
04022 #ifdef CG_CHECK_MW_PERM
04023         P0.save2(S.P0);
04024 #endif
04025         S.err=err; istr.save2(S.istr);
04026         return S;
04027     };
04028
04029     wstring info() const {
04030         unsigned n=128; char s[n]; char sep[2]=" ";
04031         if (!q.isAbelian() && J.wbvector<TQ>::allIn(0,9)) sep[0]=0;
04032         unsigned l=snprintf(s,n,"%s [%s]",
04033             q.toStr().data, J.toStrf(" ",sep,q.qlen(),";").data);
04034         if (l>=n) wblog(FL,
04035             "ERR %s() string out of bounds (%d/%d)",FCT,l,n);
04036         return s;
04037     };
04038
04039     mxArray* toMx() const {
04040         mxArray *a=mxCreateStruct(1,1);
04041         add2MxStruct(a,0);
04042         return a;
04043     };
04044
04045     size_t memSize() const {
04046         size_t mtot=sizeof(QType) + J.len*sizeof(TQ);
04047         unsigned i=0;
04048         for ( ; i<Sp.len; ++i) mtot+=Sp[i].memSize();
04049         for (i=0; i<Sz.len; ++i) mtot+=Sz[i].memSize();
04050         return mtot;
04051     };
04052
04053     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
04054     void add2MxStruct(mxArray *S, unsigned i) const;
04055
04056     QType q;
04057
04058     wvector< wSparrayTD > Sp;
04059     wvector< wSparrayTD > Sz;
04060

```

```

04061     qset<TQ> J;
04062
04063     wbMatrix<double> Z;
04064
04065     #ifdef CG_CHECK_MW_PERM
04066         wbpem P0;
04067     #endif
04068
04069     double err;
04070     wbstring istr;
04071
04072     protected:
04073     private:
04074 };
04075
04076 template <class TQ, class TD>
04077 class genRG_struct {
04078     public:
04079
04080     genRG_struct() {}
04081
04082     genRG_struct& checkInit(const char *F, int L, const QType &q0) {
04083         if (q0.isNonAbelian()) {
04084             unsigned n=RSet.size();
04085             if (n>0) {
04086                 if (!q0.type) wblog(FL,"ERR %s() got %d elements "
04087                     "in buf for '%s'",FCT,n,STR(q));
04088                 return *this;
04089             }
04090             SetupSym(F,L,q0);
04091         }
04092         return *this;
04093     };
04094
04095     genRG_struct& SetupSym(
04096         const char *F, int L, const QType &q0, qset<TQ> *qs=NULL);
04097
04098     genRG_struct& Setup_SUN(const char *F, int L, qset<TQ> *qs=NULL);
04099     genRG_struct& Setup_SpN(const char *F, int L, qset<TQ> *qs=NULL);
04100     genRG_struct& Setup_SON(const char *F, int L, qset<TQ> *qs=NULL);
04101     genRG_struct& Setup_SEN(const char *F, int L, qset<TQ> *qs=NULL);
04102
04103     void initSU2(const qset<TQ> &J);
04104
04105     void initCommRel(const char *F, int L,
04106         const genRG_base<TQ,TD> &R, char vflag=0);
04107
04108     double checkCommRel(
04109         const char *F, int L, const genRG_base<TQ,TD> &R
04110     ) const;
04111
04112     void printStatus(const char *F, int L, const char *istr=NULL);
04113
04114     int get1J_gen(const char *F, int L, const qset<TQ> &q, char flag=0);
04115
04116     int getTensorProdReps_gen(
04117         const qset<TQ> &q1, const qset<TQ> &q2,
04118         unsigned flag=TP3_LDM);
04119
04120     void getTensorProdReps(
04121         const qset<TQ> &q1, const qset<TQ> &q2
04122     ){
04123         q.validType(FL);
04124         int n=getTensorProdReps_gen(q1,q2,TP3_TST);
04125         if (n>=0) return;
04126
04127         wblog(FL,"ERR %s() "
04128             "failed to access RSets of input spaces (%d)",FCT,n);
04129     };
04130
04131     unsigned genTensorProds(const qset<TQ> &q, unsigned flag=0);
04132     unsigned genTensorProds(unsigned dmax=-1, char sdig=0, char vflag=0);
04133
04134     void put(const char *vname, const char *ws="caller") const
04135     { put(0,0,vname,ws); }
04136
04137     mxArray* toMx(char bflag=1) const;
04138     void put(const char *F, int L,
04139         const char *vname, const char *ws="caller"
04140     ) const;
04141
04142     void wblog_RC_1st_build(
04143         const char *F, int L, const char *fct, int nrep) const {
04144
04145         if (CG_VERBOSE && !q.isSU2()) {
04146             if (q.sub>1) { PRINTF("\n"

```

```

04148         "    RC_STORE %s() first-time build for %s\n"
04149         "    %d passes, please wait ...\n\n",fct,STR(q),nrep); }
04150     else wblog(FL,
04151         "NB! %s() RC_STORE first-time build for %s",fct,STR(q));
04152     }
04153 };
04154
04155     QType q;
04156
04157     map <qset<TQ>,
04158         genRG_base<TQ,TD>
04159     > RSet;
04160
04161     qset<TQ> qdef;
04162
04163     wbvector< pairPatternCG<double> > CR;
04164
04165     wbMatrix<double> DZ;
04166
04167     protected:
04168     private:
04169 };
04170
04171 template <class TQ>
04172 TQ getTensorProdReps_abelian(const QType &q, const TQ &J1, const TQ &J2) {
04173     TQ J=0;
04174
04175     switch (q.type) {
04176     case QTYPE_U1: J=J1+J2; break;
04177
04178     case QTYPE_ZN: J=int(J1+J2)%q.sub;
04179
04180         if (TQ(int(J1)%q.sub)!=J1 || TQ(int(J2)%q.sub)!=J2)
04181             wblog(FL,"ERR %s() invalid %s labels (%g,%g)",
04182                 FCT, STR(q), double(J1), double(J2));
04183         break;
04184
04185     case QTYPE_P: J=J1*J2;
04186
04187         if (fabs(double(J1))!=1 || fabs(double(J2))!=1)
04188             wblog(FL,"ERR %s() invalid %s labels (%g,%g)",
04189                 FCT,STR(q),double(J1),double(J2));
04190         break;
04191
04192     default: wblog(FL,"ERR %s() got invalid symmetry %s",FCT,STR(q));
04193     }
04194 }
04195
04196     return J;
04197 };
04198
04199 template <class TQ, class TD>
04200 class RStore {
04201
04202     public:
04203
04204     RStore(const char *s=NULL) {
04205         if (s) {
04206             if (!strcmp(s,"info")) {
04207                 #ifdef __WB_MPFR_HH__
04208                 wblog(FL,"<i> %s() uses type %s (=mpfr@%d) ",
04209                     FCT, TSTR(TD), TD().prec());
04210                 #else
04211                 wblog(FL,"<i> %s() uses type %s",FCT,TSTR(TD));
04212                 #endif
04213             }
04214             else wblog(FL,"ERR %s() invalid usage (%s)",FCT,TSTR(TD));
04215         }
04216     };
04217
04218     size_t bufsize(const QType &q) {
04219         auto it=buf.find(q);
04220         return (it==buf.end() ? 0 : it->second.RSet.size());
04221     };
04222
04223     bool isEmpty(const QType &q) {
04224         return (bufsize(q)>0 ? 1 : 0);
04225     };
04226
04227     genRG_struct<TQ,TD>& Buf(const QType &q) {
04228         auto it=buf.find(q);
04229         if (it!=buf.end()) {
04230             genRG_struct<TQ,TD> &B=(it->second);
04231             if (!B.q.isKnown()) {
04232                 if (B.RSet.size()) wblog(FL,
04233                     "ERR %s() got RSet with unknown qtype (len=%d; %s)",
04234                     FCT,B.RSet.size(),STR(B.q));

```

```

04235         B.checkInit (FL,q);
04236     }
04237     return B;
04238 }
04239 else {
04240     genRG_struct<TQ,TD> &B=buf[q];
04241     B.checkInit (FL,q);
04242     return B;
04243 }
04244 };
04245
04246 genRG_base<TQ,TD>* find_RSet(const QType &q, const TQ *qpPtr){
04247     genRG_struct<TQ,TD> &B=Buf(q);
04248     qset<TQ> qs(q.qLen(),qpPtr,'r');
04249
04250     auto it = B.RSet.find(qs);
04251     return (it!=B.RSet.end() ? &(it->second) : NULL);
04252 };
04253
04254 genRG_struct<TQ,TD>& getRSet(
04255     const QType &q,
04256     qset<TQ> *qvec=NULL
04257 ){
04258     genRG_struct<TQ,TD> &B=Buf(q);
04259     if (qvec) { (*qvec)=B.qdef; }
04260     return B;
04261 };
04262
04263 genRG_base<TQ,TD>& getR(
04264     const char *F, int L, const QType &qtype, const TQ *q) const;
04265
04266 genRG_base<TQ,TD>& getR(
04267     const char *F, int L, const CData_TQ &S,
04268     unsigned k
04269 ) const { return getR(F_L, S.t, S.qptr(k)); };
04270
04271 wbMatrix<TQ> getZ(
04272     const char *F, int L, const QType &qtype, const TQ *q
04273 ) const {
04274     const genRG_base<TQ,TD> &R = getR(F_L,qtype,q);
04275     return R.Z;
04276 };
04277
04278 unsigned qdim(
04279     const char *F, int L, const QType &qtype, const TQ *q
04280 ) const {
04281
04282     if (qtype.isAbelian()) return 1;
04283     const genRG_base<TQ,TD> &R = getR(F_L,qtype,q);
04284
04285     if (qtype.isSU2() && TQ(R.Z.dim1)!=q[0]+1) wblog(FL,"ERR %s() "
04286         "got multiplet dim mismatch (%d/%d)",FCT,R.Z.dim1,int(q[0]+1));
04287
04288     return R.Z.dim1;
04289 };
04290
04291 mxArray* toMx(const QType &q=QTYPE_UNKNOWN, char bflag=1) const;
04292
04293 unsigned LoadStore(const char *F, int L, const char *file);
04294 unsigned add(const QType &q, const mxArray* S, unsigned k);
04295
04296 void Info(const char *F=0, int L=0) const;
04297
04298 map<QType,
04299     genRG_struct<TQ,TD>
04300 > buf;
04301
04302 protected:
04303 private:
04304 };
04305
04306 RStore<gTQ,RTD> gRS;
04307
04308 #ifndef QS_USING_OMP
04309     Wb::ompNLock RCS_lock("RCStore");
04310 #endif
04311
04312 map<QType,int> g_sym;
04313
04314 class RCStore {
04315 public:
04316
04317     RCStore() { };
04318
04319     int setupDirs(const char *F=NULL, int L=0) {
04320
04321

```

```

04322     if (root.idx.len) { return -root.idx.len; }
04323
04324 #ifdef QS_USING_OMP
04325     Wb::ompGuard dLK(RCS_lock);
04326     if (root.idx.len) { return -root.idx.len; }
04327 #endif
04328
04329     int i1,i2=0,e=0;
04330
04331     const char rcs[]="RC_STORE", rsy[]="RC_SYNC";
04332
04333     i1=setup_root_directory(F_L,rcs,root);
04334     if (!i1) wblog(FL,"ERR missing env variable '%s'",rcs);
04335
04336     i2=setup_root_directory(F_L,rsy,root,'+');
04337
04338     if (i1<0 || i2<0) { root.init(); wblog(FL,
04339         "ERR invalid environmental variables RC_STORE%s",
04340         i2<0 ? " or RC_SYNC":"" ); }
04341     if (i2>1) { root.init(); wblog(FL,
04342         "ERR %s() only one RC_SYNC directory allowed",FCT); }
04343
04344     for (auto it=root.begin(); it.end(); ++it) {
04345         if (!strcmp(it.at(),"/home/",7)) { e=1;
04346             wblog(FL,"WRN %s() RC_%s points to HOME (%d/%d)\n"
04347                 "(recommended to use /data or similar, instead)",
04348                 FCT, !it.i || it.i+1<it.n?"STORE":"SYNC", it.i+1,it.n);
04349             root.print("RC_STORE+SYNC"); break;
04350         }
04351     }
04352     if (!e && CG_VERBOSE>6) { root.print("RC_STORE+SYNC"); }
04353
04354     if (!i2 && i1) {
04355         root.append(FL,root.last(),"");
04356     }
04357     return root.idx.len;
04358 };
04359
04360 template <class TQ, class TD>
04361 int save_CData(const char *F, int L, const CData<TQ,TD> &A);
04362
04363 template <class TQ, class TD>
04364 int load_CData(
04365     const char *F, int L, const QSet<TQ> &Q, CData<TQ,TD> &A,
04366     char bflag=0);
04367
04368 template <class TQ>
04369 int save_Std3(
04370     const char *F, int L, const QType &t, const qset<TQ> &J12,
04371     const wbvector<double> *c2eps=NULL) const;
04372
04373 template <class TQ>
04374 int load_Std3(const char *F, int L,
04375     const QType &t, const qset<TQ> &J1, const qset<TQ> &J2,
04376     unsigned loadRC=0) const;
04377
04378 template <class TQ, class TD>
04379 int save_RSet(
04380     const char *F, int L, const genRG_base<TQ,TD> &R, char xflag=0);
04381
04382 template <class TQ>
04383 int load_RSet(
04384     const char *F, int L, const QType &t, const qset<TQ> &J,
04385     char vflag=0);
04386
04387 int save_XMap(
04388     const char *F, int L, const cgc_contract_id<MTI> &idc) const;
04389
04390 int load_XMap(
04391     const char *F, int L, const cgc_contract_id<MTI> &idc) const;
04392
04393 template <class TQ>
04394 int get_file_name(
04395     const char *F, int L, wbstring &file,
04396     const QSet<TQ> &Q, const char *ext=NULL, char mflag=0) const;
04397
04398 template <class TQ, class TD>
04399 int get_file_name(
04400     const char *F, int L, wbstring &file,
04401     const genRG_base<TQ,TD> &R, const char *ext=NULL, char mflag=0
04402 ) const;
04403
04404 int get_file_name(
04405     const char *F, int L, wbstring &file,
04406     const cgc_contract_id<MTI> &idc, const char *ext=NULL, char mflag=0,
04407     QType *t=NULL
04408 ) const;

```

```

04409
04410     size_t rclog(const QType &t,
04411                 const char *F, int L, const char *fmt, ...);
04412
04413     size_t rclog(const QType &t,
04414                 const char *F, int L, char vflag, const char *fmt, ...);
04415
04416     size_t vrclog(const QType &t,
04417                  const char *F, int L, const char *fmt, va_list args);
04418
04419     void Info(const char *F=0, int L=0) {
04420         if (F) wblog(F,L," * %s()\n",FCT);
04421         root.print("RCStore::root");
04422         fprintf(stdout,"\n\n");
04423     }
04424
04425     int is_new(const QType &t) {
04426         return (g_sym[t] ? 0 : 1);
04427     };
04428
04429     Wb::Path root;
04430
04431 protected:
04432 private:
04433
04434     int get_rcs_path(
04435         const char *F, int L, char *path, unsigned len, const QType &t,
04436         const char *sub1=NULL, const char *sub2=NULL, const char *file=NULL,
04437         char mflag=0
04438     ) const;
04439
04440     int setup_root_directory(
04441         const char *F, int L, const char *env, Wb::Path &root,
04442         char append=0
04443     );
04444 };
04445
04446     RCStore gStore;
04447
04448     Wb::IOstat gstatR,gstatM,gstatC,gstatX;
04449
04450     #define RC_MDIR 1
04451     #define RC_SYNC 2
04452     #define RC_CURR 4
04453
04454     #define RC_SAVE (RC_MDIR|RC_CURR)
04455     #define RC_SAVE_S (RC_MDIR|RC_SYNC)
04456
04457     #define RCL_V1 1
04458     #define RCL_V2 2
04459
04460     #define RCL_ALL (RCL_V1|RCL_V2)
04461
04462 int RCStore::setup_root_directory(
04463     const char *F, int L, const char *env, Wb::Path &root, char append
04464 ){
04465     int m=0, k=0, e=0, kmax;
04466     const char *sval;
04467
04468     if (!env || !env[0]) wblog(F,L,
04469         "ERR %s() no variable specified",FCT);
04470
04471     sval=getenv(env); if (!sval || !sval[0]) {
04472         if (!append) { root.init(); }
04473         return m;
04474     }
04475
04476     if (!append)
04477         { m=root.init(F,L,sval,":",1024); }
04478     else { k=root.numel(); m=root.append(F,L,sval,":"); }
04479
04480     for (kmax=k+m; k<kmax; ++k) { char *dir=root[k];
04481
04482         if (!dir || !dir[0]) {
04483             if (!e) { PRINTF("\n"); }; e|=1;
04484             PRINTF(" ERR invalid %s entry (empty)\n",env);
04485         }
04486         if (Wb::fexist(dir,'d')) { continue; }
04487
04488         if (k+1<kmax) {
04489             if (!e) { PRINTF("\n"); }; e|=2;
04490             PRINTF(" ERR invalid %s entry (%s) ^1\n",env,dir);
04491         }
04492
04493         unsigned n=strlen(dir), i=n-1, i0=0; int q;
04494
04495         for (; i<n; --i) { if (dir[i]=='/') { i0=i; break; }}

```



```

04496     if (i0 && (
04497         !strncasecmp(dir+i0+1,"RCStore",7) ||
04498         !strncasecmp(dir+i0+1,"RCSync", 6))
04499     ){
04500         dir[i0]=0; q=Wb::fexist(dir,'d');
04501         dir[i0]='/' ;
04502         if (q) {
04503             mode_t p=(S_IRWXU | S_IRWXG | S_IROTH | S_IXOTH);
04504             if ((q=mkdir(dir,p))==0) {
04505                 PRINTF("    creating %-8s root directory (%s)\n",
04506                     env, Wb::repHome(dir).data);
04507             }
04508             else {
04509                 if (!e) { PRINTF("\n"); }; e|=4;
04510                 PRINTF("    ERR invalid %s entry (%s) ^2\n",env,dir);
04511             }
04512         }
04513         else {
04514             if (!e) { PRINTF("\n"); }; e|=8;
04515             PRINTF("    ERR invalid %s entry (%s) ^3\n",env,dir);
04516         }
04517     }
04518     else if (!Wb::fexist(dir,'d')) {
04519         if (!e) { PRINTF("\n"); }; e|=16;
04520         PRINTF("    ERR invalid %s entry (%s) ^4\n",env,dir);
04521     }
04522 }
04523 if (e) { PRINTF("\n");
04524 if (e& 2) { PRINTF("    ^1 all but last entry must exist\n"); }
04525 if (e& 4) { PRINTF("    ^2 failed to create directory\n"); }
04526 if (e& 8) { PRINTF("    ^3 invalid %sdirectory\n",kmax>1?"last ":""); }
04527 if (e&16) { PRINTF("    ^4 %sdirectory must also exist if not named %s*\n",
04528     strncmp(env,"RC_SYNC",8) ? "RCStore":"RCSync", kmax>1?"last ":""); }
04529 if (e> 1) { PRINTF("\n"); }
04530 }
04531
04532 if (e) {
04533     root.init();
04534     wblog(FL,"ERR invalid env %s directory (e=%d)",env,e);
04535 }
04536
04537 return (e ? -m : m);
04538 };
04539
04540 namespace CG {
04541
04542 template <class TQ, class TD>
04543 double getSymmetryStates(const char *F, int L, const QType &q,
04544     const wbvector< wbSpararrayTD > &Sp,
04545     const wbvector< wbSpararrayTD > &Sz,
04546     wbvector< wbSpararrayTD > &U,
04547     wbvector<unsigned> &dd,
04548     wbvector<genRG_base<TQ,TD> > &RR,
04549     wbvector<double> &c2eps,
04550     wbMatrix<unsigned> *iOM=NULL,
04551     char vflag=1
04552 );
04553
04554 template <class TQ>
04555 SPIDX_T findMaxWeight(
04556     const QType &q, const wbMatrix<double> &Z, qset<TQ> *J=NULL, wbperm *P=NULL
04557 );
04558
04559 template <class TQ, class TD>
04560 void get_SU2mat(TQ s2,
04561     wbvector<double> &sz,
04562     wbsparray<TD> &Sp, wbsparray<TD> &Sz,
04563     wbsparray<TD> &S2, wbsparray<TD> &E
04564 );
04565
04566 int isQSet (const mxArray *S, unsigned k=-1);
04567 int isCData(const mxArray *S, unsigned k=-1);
04568 int isCRef (const mxArray *S, unsigned k=-1);
04569
04570 int MemStat(const char *F=0, int L=0) {
04571     int n=0; const char *hstr="RCStore"; const char *xstr="/full";
04572
04573     wblog(F_L,"%N%s() %33R",FCT,"-");
04574     gRS.Info();
04575     gCS.Info();
04576     gXS.Info();
04577     Wb::MemStat(FL);
04578
04579     if (!gstatR.isEmpty()) gstatR.print("RData", (++n==1)? hstr:0, xstr);
04580     if (!gstatM.isEmpty()) gstatM.print("map3",   (++n==1)? hstr:0, xstr);
04581     if (!gstatC.isEmpty()) gstatC.print("CData",  (++n==1)? hstr:0, xstr);
04582     if (!gstatX.isEmpty()) gstatX.print("XData",  (++n==1)? hstr:0, xstr);

```

```

04583
04584     if (n) { PRINTF("\n"); }
04585     else wblog(FL,"I/O no data read or written yet to RCStore");
04586
04587     return 0;
04588 };
04589
04590 #ifdef QS_USING_OMP
04591
04592     map <size_t, Wb::ompNLock*> lock_map;
04593
04594     class Guard {
04595     public:
04596
04597         Guard() : active(false) {};
04598
04599         template<class TQ>
04600         Guard(const char *F, int L, const QSet<TQ> &Q, const char *tag=NULL,
04601             const char *cond=NULL)
04602             : active(false) { acquire(F_L,Q,tag,cond); };
04603
04604         template<class TM>
04605         Guard(const char *F, int L, const cgc_contract_id<TM> &x,
04606             const char *cond=NULL)
04607             : active(false) { acquire(F_L,x,cond); }
04608
04609         template<class TQ>
04610         Guard(const char *F, int L, const QSet<TQ> &QA, const QSet<TQ> &QB)
04611             : active(false) { acquire(F_L,QA,QB); };
04612
04613         template<class TQ>
04614         Guard(const char *F, int L,
04615             const QSet<TQ> &QA, const QSet<TQ> &QB, const QSet<TQ> &QC)
04616             : active(false) { acquire(F_L,QA,QB,QC); };
04617
04618     ~Guard() {
04619         if (hid.len) { release(); hid.init(); }
04620         active=false;
04621     };
04622
04623     explicit operator bool() const { return (hid.len || active); }
04624     bool operator! () const { return (!hid.len && !active); }
04625
04626     int acquire(const char *F, int L,
04627         const FileLock &flk, const char *tag=NULL, const char *cond=NULL);
04628
04629     template<class TQ>
04630     int acquire(const char *F, int L,
04631         const QSet<TQ> &Q, const char *tag=NULL, const char *cond=NULL);
04632
04633     template<class TM>
04634     int acquire(const char *F, int L,
04635         const cgc_contract_id<TM> &x, const char *cond=NULL);
04636
04637     template<class TQ>
04638     void acquire(const char *F, int L, const QSet<TQ>* Q[], unsigned n);
04639
04640     template<class TQ>
04641     void acquire(const char *F, int L,
04642         const QSet<TQ> &QA, const QSet<TQ> &QB)
04643         { const QSet<TQ>* QQ[2]={ &QA, &QB }; acquire(F_L,QQ,2); };
04644
04645     template<class TQ>
04646     void acquire(const char *F, int L,
04647         const QSet<TQ> &QA, const QSet<TQ> &QB, const QSet<TQ> &QC)
04648         { const QSet<TQ>* QQ[3]={ &QA, &QB, &QC }; acquire(F_L,QQ,3); };
04649
04650     inline int acquire_set(const char *ss[], int ntry=-1) {
04651         if (!hid.len) { return -1; }
04652
04653         int got[hid.len];
04654         Wb::ompNLock *lk[hid.len];
04655
04656         int nlks=acquire_set_iter(got,lk,ss,0);
04657         if (nlks<=0) { if (ntry==1 || !ntry) return nlks;
04658             nlks=acquire_set_wait(got,lk,ss,ntry);
04659         }
04660
04661         if (nlks<=0) {
04662             if (ntry<=0 || ntry>99) wblog(FL,
04663                 "ERR %s() failed to qdquire locks (%d/%d)",FCT,nlks,hid.len);
04664             return nlks;
04665         }
04666
04667         for (unsigned i=0; i<hid.len; ++i) {
04668             lk[i]->set_istr(ss[i]);
04669         }
04670     }

```

```

04670         if (*got < got[i]) { *got=got[i]; }
04671     }
04672     active=true;
04673
04674     return *got;
04675 };
04676
04677 int acquire_set_iter(
04678     int *got, Wb::ompNLock *lk[], const char *ss[], unsigned iter=0);
04679 int acquire_set_wait(
04680     int *got, Wb::ompNLock *lk[], const char *ss[], int ntry=-1);
04681
04682 void release();
04683
04684 int level() { int l=-1;
04685     for (unsigned i=0; i<hid.len; ++i) {
04686         const Wb::ompNLock *lp = Guard::find(hid[i]);
04687         if (lp && l<(lp->level)) { l=lp->level; }
04688     }
04689     return l;
04690 };
04691
04692 int used() { int l=0;
04693     for (unsigned i=0; i<hid.len; ++i) {
04694         const Wb::ompNLock *lp = Guard::find(hid[i]);
04695         if (lp && l<(lp->level)) {
04696             if (l< lp->level ) { l=lp->level; }
04697             if (l< lp->nactive) { l=lp->nactive; }
04698             if (l< lp->ntot ) { l=lp->ntot; }
04699         }
04700     }
04701     if (!l && active) { l=-1; }
04702     return l;
04703 };
04704
04705 void print(const char *F, int L);
04706
04707 wstring toStr() const;
04708
04709 static wstring toStr(size_t h, const char *istr=NULL);
04710 static int toStr(wstring &s, size_t h, const char *istr=NULL);
04711
04712 template<class TQ>
04713 static wstring Status(const QSet<TQ> &Q, const char *tag=NULL);
04714
04715 template<class TM>
04716 static wstring Status(const cgc_contract_id<TM> &x);
04717
04718 static wstring Status(size_t h1, const char *istr=NULL);
04719
04720 static Wb::ompNLock* find(size_t h);
04721
04722 static int print_CG_locks();
04723
04724 protected:
04725 private:
04726
04727     wvector<size_t> hid;
04728
04729     bool active;
04730
04731     const char* sprintf_locks(
04732         wstring &sout, const int* got, const char *istr[],
04733         const char *indent=NULL);
04734
04735     int deal_with_cond(const char *F, int L, size_t h0, const char *cond);
04736
04737 };
04738
04739 #endif
04740
04741 class FileLock {
04742
04743 public:
04744     FileLock() : fid(0) {};
04745
04746     ~FileLock() {
04747         if (fid) { ReleaseLock(FL); }
04748     };
04749
04750     template<class TQ>
04751     FileLock(
04752         const QType &q_, const qset<TQ> &J1, const qset<TQ> &J2,
04753         const char *ext="mp3"
04754     ){ init(q_,J1,J2,ext); }
04755
04756

```

```

04757     template<class TQ>
04758     int init(
04759         const QType &q_, const qset<TQ> &J1, const qset<TQ> &J2,
04760         const char *ext="mp3"
04761     );
04762
04763     int GetLock(const char *F=NULL, int L=0, char flag='W') {
04764         return tryGetLock(F,L,flag);
04765     };
04766
04767     int ReleaseLock(const char *F=NULL, int L=0) {
04768         return tryGetLock(F,L,'u' );
04769     };
04770
04771     int ensureFOpen(const char *F=NULL, int L=0) const {
04772         int q=0;
04773         if (!fname) { q|=1; }
04774         if (fid<3 ) { q|=2; }
04775         if (q && F) {
04776             if (q&1) wblog(F_L,"ERR %s() got empty fname (e=%d)",FCT,q);
04777             if (q&2) wblog(F_L,"ERR %s() file not yet open "
04778                 "{fid=%d; e=%d}\n%s",FCT,fid, q,fname.data
04779             );
04780         }
04781         return q;
04782     };
04783
04784     int fid;
04785
04786     QType q;
04787     wbsting fname;
04788
04789 #ifdef QS_USING_OMP
04790     CG::Guard fLK;
04791 #endif
04792
04793     protected:
04794     private:
04795
04796     int tryGetLock(const char *F, int L, char flag);
04797 };
04798
04799 };
04800
04801 template<class TD>
04802 int BuildKrylovH(
04803     const QType &q,
04804     wbarray<TD> &HK, unsigned nk,
04805     wbSparrayTD &X,
04806     wbvector<TD> &EK,
04807     const wbSparrayTD &HL,
04808     const wbSparrayTD &HR,
04809     const wbvector< wbSparrayTD > SL,
04810     const wbvector< wbSparrayTD > SR,
04811     double &xmin
04812 );
04813
04814 template<class TD>
04815 void GetHPsi(
04816     wbSparrayTD &HX, const wbSparrayTD &X,
04817     const wbSparrayTD &HL,
04818     const wbSparrayTD &HR,
04819     const wbvector< wbSparrayTD > SL,
04820     const wbvector< wbSparrayTD > SR
04821 );
04822
04823 #endif
04824

```

5.2 cpat.hh

```

00001 #ifndef __WB_CPAT_HCC__
00002 #define __WB_CPAT_HCC__
00003
00004 /* ----- *
00005  * class CPAT: class to store contraction pattern in QSpace structure
00006  * ----- *
00007  * contraction pattern (default: twoSite mode)
00008  *
00009  * deals with MatLab cell array of the form { A, i1, i2 [,opA] }
00010  * where A can be a single QSpace or a cell vector of these
00011  * i[12] specifies index onto which to contract onto Psi[12]
00012  * NB! either i1 or i2 is set, the other one must be empty, e.g. []

```

```

00013  *
00014  * single site mode: expects { A, ic, [,opA] } instead as
00015  * contraction pattern where A is always contracted onto "Psi1"
00016  *
00017  * ----- *
00018  * AW (C) Jun 2006
00019  * ----- */
00020
00021 #define CP1 "{ A, ic, [,opA] }"
00022 #define CP2 "{ A, i1, i2 [,opA] }"
00023 #define _CP (twoSite ? "{ A, i1, i2 [,opA] }" : "{ A, ic, [,opA] }")
00024
00025 template <class TQ, class TD>
00026 class CPAT_Set;
00027
00028 int isCPAT(const mxArray *C, char twoSite=-1);
00029 int isCPattern(const mxArray *C);
00030
00031 unsigned NR_ICTR_CALL=0;
00032
00033 template <class TQ, class TD>
00034 class CPAT {
00035
00036 public:
00037
00038     CPAT() : z(0), sameCtrIdx(0), nr_check_ctr(0), HPsi_counter(0) {};
00039
00040     CPAT(const CPAT<TQ,TD> &D) {
00041         wblog(FL,"ERR do not copy CPAT object!");
00042     };
00043
00044     void init(
00045         TD oz, const mxArray**, unsigned n,
00046         unsigned ic1, unsigned r1, unsigned ic2, unsigned r2,
00047         const char twoSite, const char ldir
00048     );
00049
00050     int getCtrIndex(const QSpace<TQ,TD> &PSI);
00051
00052     QSpace<TQ,TD>& HZTimes(
00053         const QSpace<TQ,TD> &PSI, QSpace<TQ,TD> &HZPsi,
00054         const char ='N', char disp=1
00055     );
00056
00057     QSpace<TQ,TD>& HZApplyDiag(
00058         const QSpace<TQ,TD> &PSI, QSpace<TQ,TD> &EPsi,
00059         TD a, TD b, char iflag=0, char disp=1
00060     );
00061
00062     void check_contraction(const QSpace<TQ,TD> &PSI, char disp=1);
00063     void initCounter() {
00064         HPsi_counter=0;
00065     };
00066
00067     QSpace<TQ,TD>& HZTimes_tst(
00068         const QSpace<TQ,TD> &PSI, QSpace<TQ,TD> &HZPsi,
00069         const char ='N', char disp=1);
00070
00071     void getQRange(
00072         unsigned k, wbMatrix<TQ> &QQ, wbvector<unsigned> &SS,
00073         QSpace<TQ,TD> *B=NULL
00074     ) const;
00075
00076     mxArray* toMx() const;
00077
00078     void put(const char *vname, const char* ws="base") const {
00079         mxPutAndDestroy(FL,toMx(),vname, ws);
00080     };
00081
00082     void PermIC(const wbperm &P);
00083
00084     wbvector< wbvector< CPAT_Set<TQ,TD> > > CP;
00085     TD z;
00086
00087     unsigned sameCtrIdx;
00088     unsigned nr_check_ctr;
00089     unsigned HPsi_counter;
00090
00091 protected:
00092 private:
00093 };
00094
00095 template <class TQ, class TD>
00096 class CPAT_Set {
00097
00098 public:
00099

```

```

00100     CPAT_Set () : opA(0), ic(0) {};
00101
00102     CPAT_Set (const CPAT_Set<TQ,TD> &D) : opA(0), ic(0) {
00103         wblog(FL,"WRN Do not copy CPAT_Set!");
00104     };
00105
00106     int init(
00107         const mxArray *a,
00108         unsigned ic1, unsigned r1, unsigned ic2, unsigned r2,
00109         const char twoSite, const char ldir
00110     );
00111
00112     int getCtrIndex(const QSpace<TQ,TD> &PSI);
00113     int getCtrIndex(const wbvector< QSpace<TQ,TD> >&, const char ='N');
00114
00115     int getCtrIndex(const unsigned i, const QSpace<TQ,TD>&, char ='N');
00116
00117     void contract (
00118         const QSpace<TQ,TD> &PSI, wbvector< QSpace<TQ,TD> > &C,
00119         char='N') const;
00120
00121     void contract (
00122         const wbvector< QSpace<TQ,TD> > &PSI1, wbvector< QSpace<TQ,TD> > &C,
00123         char='N') const;
00124
00125     void contract (
00126         const unsigned k, const QSpace<TQ,TD> &PSI, QSpace<TQ,TD> &C,
00127         char='N') const;
00128
00129     void contract_diag(
00130         const QSpace<TQ,TD> &PSI, wbvector< QSpace<TQ,TD> > &C,
00131         TD a, TD b, char Iflag) const;
00132
00133     void contract_diag(
00134         const wbvector< QSpace<TQ,TD> > &PSI1, wbvector< QSpace<TQ,TD> > &C,
00135         TD a, TD b, char Iflag) const;
00136
00137     void contract_diag(
00138         const unsigned k, const QSpace<TQ,TD> &PSI, QSpace<TQ,TD> &C,
00139         TD a, TD b, char Iflag) const;
00140
00141     mxArray* toMx() const;
00142
00143     wbvector< QSpace<TQ,TD> > A;
00144     char opA;
00145     unsigned ic;
00146
00147     wbperm P;
00148
00149     wbvector< QSpace<TQ,TD> > CQn, CQt;
00150     wbvector< wbMatrix<unsigned> > CIn, CIt;
00151
00152 protected:
00153 private:
00154 };
00155
00156 template <class TQ, class TD>
00157 void CPAT<TQ,TD>::init(
00158     TD oz, const mxArray** aa, unsigned N,
00159     unsigned ic1, unsigned r1, unsigned ic2, unsigned r2,
00160     const char twoSite, const char ldir
00161 ){
00162     unsigned i,j,e,k,p,m=0,n;
00163     wbvector<unsigned> dd;
00164     mxArray *a;
00165
00166     if (ic1>=r1 || ic2>=r2) wblog(FL,
00167         "ERR Contraction index out of bounds (%d/%d; %d/%d (%d))",
00168         ic1+1, r1, ic2+1, r2, twoSite
00169     );
00170
00171     z=oz;
00172
00173     CP.initDef(N);
00174
00175     for (p=k=0; k<N; k++) {
00176         if (!mxIsCell(aa[k]))
00177             wblog(FL, "ERR Invalid (pair) pattern %s", _CP);
00178
00179         n=isCPAT(aa[k], twoSite);
00180
00181         if (!n) wbdie(FL,str);
00182
00183         CP[p].initDef(n);
00184
00185         if (n==1) {
00186             e=CP[p][0].init(aa[k],ic1,r1,ic2,r2,twoSite,ldir);

```

```

00187         if (e) wbdie(FL, str);
00188         m=CP[p][0].A.len;
00189     }
00190     else {
00191         for (i=0; i<n; i++) {
00192             a=mxGetCell(aa[k], i);
00193             e=CP[p][i].init(a, ic1, r1, ic2, r2, twoSite, ldir);
00194
00195             if (e) {
00196                 printf("\n%s:%d contraction pattern %d:%d\n", FL, k+1, i+1);
00197                 printf("%s\n", str); ExitMsg("");
00198             }
00199
00200             if (i>0) { if (m!=CP[p][i].A.len) wblog(FL,
00201 "ERR Pattern pair dimension mismatch in A (%d.%d: %d,%d)",
00202 k+1, i, CP[p][0].A.len, CP[p][i].A.len); }
00203             else m=CP[p][0].A.len;
00204         }
00205     }
00206
00207     dd.init(m);
00208     for (i=0; i<n; i++) {
00209         const wvector< QSpace<TQ, TD> > &Aki=CP[p][i].A;
00210         for (j=0; j<m; j++) if (!Aki[j].isEmpty()) dd[j]++;
00211     }
00212
00213     for (j=0; j<m; j++) if (dd[j]==n) { p++; break; }
00214 }
00215
00216 if (p<N) {
00217     if (p==0) wblog(FL, "WRN eigs: no relevant contraction pattern!");
00218
00219     CP.len=p;
00220 }
00221
00222 NR_ICTR_CALL=0;
00223 }
00224
00225 template <class TQ, class TD>
00226 int CPAT_Set<TQ, TD>::init(
00227     const mxArray *C,
00228     unsigned ic1, unsigned r1, unsigned ic2, unsigned r2,
00229     const char twoSite, const char ldir
00230 ){
00231
00232     unsigned k,m,w, ra, ica, ic0, N=mxGetNumberOfElements(C);
00233     const char isC=(typeid(TD)==typeid(wbcomplex) ? 'C' : 0);
00234     int r=-1;
00235
00236     mxArray *a; str[0]=0;
00237
00238     if (ic1>=r1 || ic2>=r2) {
00239         sprintf_str( "%s:%d ERR Invalid input (%d/%d; %d/%d (%d))",
00240 FL, ic1+1, r1, ic2+1, r2, twoSite);
00241         return 1;
00242     }
00243
00244     m=isCPattern(C);
00245
00246     if ((char)m!=twoSite+1) wblog(FL,
00247 "ERR invalid input (not a %d-site CPattern; %d,%d)", twoSite+1, m);
00248
00249     a=mxGetCell(C, 0);
00250
00251     if (mxIsQSpace(0, 0, a, r, isC)>0) {
00252         A.initDef(1);
00253         A[0].init(FL, a, 'r');
00254
00255         if (!A[0].isConsistent_r(2)) {
00256             sprintf_str("%s:%d rank-2 tensor required (arg #1; %s)", FL, _CP);
00257             return 1;
00258         }
00259     }
00260     else {
00261         if (mxIsQSpaceVec(a, 2, isC)) { sprintf_str(
00262 "%s:%d rank-2 QSpace (vector) required (arg #1; %s)", FL, _CP);
00263             return 1;
00264         }
00265         mxInitQSpaceVec(FL, a, A, 'r');
00266     }
00267
00268     if (twoSite) {
00269         w = (mxIsEmpty(mxGetCell(C, 1)) ? 0 : 1) +
00270             (mxIsEmpty(mxGetCell(C, 2)) ? 0 : 2);
00271         k=w;
00272
00273         if (w<1 || w>2) {

```

```

00274         sprintf_str("%s:%d Either arg #2 or #3 in CPAT %s",FL,_CP);
00275         return 1;
00276     }
00277 }
00278 else {
00279     k=1; w=(ldir>0 ? 1 : 2);
00280
00281     if (mxIsEmpty(mxGetCell(C,k))) {
00282         sprintf_str("%s:%d Invalid arg #d in CPAT %s",FL,k,CP1);
00283         return 1;
00284     }
00285 }
00286
00287 ra = (w==1 ? r1 : r2);
00288 ica = (w==1 ? ic1 : ic2);
00289
00290 a=mxGetCell(C,k);
00291
00292 if (mxGetNumber(a,ic)) { sprintf(str+strlen(str),
00293     "\n%s:%d Invalid arg #d in CPAT %s", FL, k+1, _CP);
00294     return 1;
00295 }
00296
00297 if (!ic || ic>ra) { sprintf(str+strlen(str),
00298     "\n%s contraction index out of bounds (%d/%d; %d,%d) in CPAT %s",
00299     SHORT_FL, ic, ra,r1,r2,_CP); return 1;
00300 }
00301
00302 ic0=ic;
00303 ic--;
00304
00305 if (twoSite && ic==ica) { sprintf(str+strlen(str),
00306     "\n%s Index %d overlaps with ctr index in CPAT %s",
00307     SHORT_FL, ic0, _CP); return 1;
00308 }
00309
00310 if (twoSite) {
00311     if (ic>ica) ic--;
00312     if (ldir>0 && w==2) ic+=(r1-1); else
00313     if (ldir<0 && w==1) ic+=(r2-1);
00314 }
00315 else {
00316     if (ic>ica) ic--; else
00317     if (ic==ica) ic=ra-1;
00318 }
00319
00320 opA=0; k=(twoSite ? 3 : 2);
00321
00322 if (k<N) {
00323     a=mxGetCell(C,k);
00324
00325     if (mxGetNumber(a,opA)) { sprintf_str(
00326         "%s:%d Invalid arg #d in CPAT %s", FL, 3+twoSite, _CP);
00327         return 1;
00328     }
00329     if (N>k+1) { sprintf_str(
00330         "%s:%d Too many arguments for CPAT %s", FL, _CP);
00331         return 1;
00332     }
00333 }
00334
00335 opA=toupper(opA); if (!opA || opA==' ' || opA=='N')
00336
00337 if (!Wb::strchr("NITC*",opA)) { sprintf_str(
00338     "%s:%d ERR invalid opA=%c<%d> in CPAT %s", FL, opA, opA, _CP);
00339     return 1;
00340 }
00341
00342 if (opA!='N') {
00343     char tflag= (opA=='T' || opA=='C'),
00344             cflag= (opA=='C' || opA=='*');
00345     wbpPerm P("2 1");
00346
00347     for (unsigned i=0; i<A.len; i++) {
00348         if (A[i].isEmpty()) continue;
00349         if (tflag) A[i].Permute(P);
00350         if (cflag) A[i].Conj();
00351     }
00352 }
00353
00354 return 0;
00355 }
00356
00357 template <class TQ, class TD>
00358 void CPAT<TQ,TD>::PermIC(const wbpPerm &P) {
00359     unsigned i,j, m=CP.len, n;
00360     wbpPerm iP;

```



```

00361
00362     P.invert(iP);
00363
00364     for (i=0; i<m; i++) {
00365         for (n=CP[i].len, j=0; j<n; j++)
00366             CP[i][j].ic = iP[CP[i][j].ic];
00367     }
00368 }
00369
00370 template <class TQ, class TD>
00371 int CPAT<TQ,TD>::getCtrIndex(const QSpace<TQ,TD> &PSI) {
00372
00373     unsigned i,j,n; int m=0;
00374     const QSpace<TQ,TD> PSI2;
00375
00376     if ((++NR_ICTR_CALL)%20==0) wblog(FL,
00377     "%NWRN calling %s already %d times (?)", FCT, NR_ICTR_CALL);
00378
00379     for (i=0; i<CP.len; i++) {
00380         m += CP[i][0].getCtrIndex(PSI);
00381
00382         for (n=CP[i].len, j=1; j<n; j++) {
00383             if (CP[i][0].A.len!=CP[i][j].A.len) wblog(FL,
00384             "ERR Severe dimension mismatch (%d: %d,%d)", j,
00385             CP[i][0].A.len, CP[i][j].A.len);
00386
00387             m += CP[i][j].getCtrIndex(CP[i][j-1].CQn, 'N');
00388             m += CP[i][j].getCtrIndex(CP[i][j-1].CQt, 'T');
00389         }
00390     }
00391
00392     return m;
00393 }
00394
00395 template <class TQ, class TD>
00396 int CPAT_Set<TQ,TD>::getCtrIndex(const QSpace<TQ,TD> &PSI) {
00397
00398     unsigned i,m=0,n=A.len;
00399
00400     if (CIn.len!=n) { CIn.initDef(n); m++; }
00401     if (CIt.len!=n) { CIt.initDef(n); m++; }
00402     if (CQn.len!=n) { CQn.initDef(n); m++; }
00403     if (CQt.len!=n) { CQt.initDef(n); m++; }
00404
00405     if (A.len==0) return (m!=0);
00406
00407     P.Index(PSI.rank());
00408     P.mvFirstTo(ic);
00409
00410     for (i=0; i<A.len; i++) {
00411         m += getCtrIndex(i,PSI, 'N');
00412         m += getCtrIndex(i,PSI, 'T');
00413     }
00414
00415     return (m!=0);
00416 }
00417
00418 template <class TQ, class TD>
00419 int CPAT_Set<TQ,TD>::getCtrIndex(
00420     const wvector< QSpace<TQ,TD> > &PSI1,
00421     const char tflag
00422 ) {
00423     unsigned i, m=0, n=A.len;
00424
00425     if (!Wb::strchr("NT",tflag)) wblog(FL,
00426     "ERR Invalid flag %c<%d>",tflag,tflag);
00427
00428     if (tflag=='N') {
00429         if (CIn.len!=n) { CIn.initDef(n); m++; }
00430         if (CQn.len!=n) { CQn.initDef(n); m++; }
00431     }
00432     else {
00433         if (CIt.len!=n) { CIt.initDef(n); m++; }
00434         if (CQt.len!=n) { CQt.initDef(n); m++; }
00435     }
00436
00437     if (n==0) return (m!=0);
00438
00439     if (n!=PSI1.len) wblog(FL,
00440     "ERR Severe dimension mismatch (%d,%d)", n, PSI1.len);
00441
00442     for (i=1; i<n; i++) if (PSI1[i].QDIM!=PSI1[0].QDIM)
00443         wblog(FL, "ERR Severe QDIM inconsistency (%d,%d).",
00444         PSI1[i].QDIM, PSI1[0].QDIM);
00445
00446     P.Index(PSI1[0].rank());
00447     P.mvFirstTo(ic);

```

```

00448
00449     for (i=0; i<n; i++)
00450         m += getCtrIndex(i, PSI[i], tflag);
00451
00452     return (m!=0);
00453 }
00454
00455 template <class TQ, class TD>
00456 int CPAT_Set<TQ,TD>::getCtrIndex(
00457     const unsigned i,
00458     const QSpace<TQ,TD> &PSI,
00459     char tflag
00460 ){
00461     int e;
00462     unsigned ica = tflag=='N' ? 2 : (tflag=='T' ? 1 : 0);
00463
00464     QSpace<TQ,TD> &CQi = (tflag=='N' ? CQn[i] : CQt[i]);
00465     wbMatrix<unsigned> &CIi = (tflag=='N' ? CIn[i] : CIt[i]);
00466
00467     QSpace<TQ,TD> CQ0;
00468     const wbMatrix<unsigned> CIO(CIi);
00469
00470     if (!ica) wblog(FL, "ERR Invalid flag %c<%d>",tflag,tflag);
00471
00472     CQ0.QDIM=CQi.QDIM; CQi.QDIM=PSI.QDIM;
00473     CQ0.QIDX=CQi.QIDX;
00474
00475     e=A[i].contract_getIdxSet(ica, PSI, ic+1, CQi.QIDX, CIi);
00476     if (e<0) wbdie(FL,str);
00477
00478     CQi.PermuteQ(P);
00479
00480     return (CQi.QDIM!=CQ0.QDIM || CQi.QIDX!=CQ0.QIDX || CIi!=CIO);
00481 }
00482
00483 template <class TQ, class TD>
00484 QSpace<TQ,TD>& CPAT<TQ,TD>::HZTimes(
00485     const QSpace<TQ,TD> &PSI,
00486     QSpace<TQ,TD> &HZPsi,
00487     const char tflag,
00488     char disp
00489 ){
00490     unsigned i,j,n;
00491
00492     #ifdef WB_CLOCK
00493         Wb::Clock clk("cpat:HZTimes",0);
00494     #endif
00495
00496     wbvector< QSpace<TQ,TD> > X1,X2;
00497
00498     if (!Wb::strchr("NTC",tflag))
00499         wblog(FL,"ERR Invalid flag %c<%d>", tflag, tflag);
00500
00501     check_contraction(PSI,disp);
00502     HPsi_counter++;
00503
00504     HZPsi=PSI;
00505     HZPsi *= (tflag=='C' ? -CONJ(z) : -z);
00506
00507     for (i=0; i<CP.len; i++) {
00508
00509         CP[i][0].contract(PSI, X1, tflag);
00510
00511         for (n=CP[i].len, j=1; j<n; j++) {
00512             X1.save2(X2);
00513             CP[i][j].contract(X2,X1,tflag);
00514         }
00515
00516         for (j=0; j<X1.len; j++)
00517             X1[j].Append2AndDestroy(FL,HZPsi);
00518     }
00519
00520     HZPsi.makeUnique();
00521
00522     return HZPsi;
00523 };
00524
00525 template <class TQ, class TD>
00526 QSpace<TQ,TD>& CPAT<TQ,TD>::HZApplyDiag(
00527     const QSpace<TQ,TD> &PSI, QSpace<TQ,TD> &EPsi,
00528     TD a, TD b, char Iflag, char disp
00529 ){
00530     unsigned i,j,n;
00531
00532     wbvector< QSpace<TQ,TD> > X1,X2;
00533
00534     check_contraction(PSI,disp);

```

```

00535
00536     EPsi.init();
00537
00538     for (i=0; i<CP.len; i++) {
00539
00540         CP[i][0].contract_diag(Psi, X1, a,b,Iflag);
00541
00542         for (n=CP[i].len, j=1; j<n; j++) {
00543             X1.save2(X2);
00544             CP[i][j].contract_diag(X2,X1, a,b,Iflag);
00545         }
00546
00547         for (j=0; j<X1.len; j++) {
00548             X1[j].SkipEmptyData();
00549             X1[j].Append2AndDestroy(FL,EPsi);
00550         }
00551     }
00552
00553     EPsi.makeUnique();
00554
00555     if (Psi.QIDX!=EPsi.QIDX) {
00556         wblog(FL,
00557             "WRN QIDX different for Psi and EPsi !? (%d/%d)",
00558             EPsi.QIDX.dim1,Psi.QIDX.dim1);
00559         wblog(FL,"TST a=%g, b=%g, '%c'",a,b,Iflag);
00560 #ifdef MATLAB_MEX_FILE
00561         Psi.put("Psi"); EPsi.put("EPsi");
00562 #endif
00563     }
00564
00565     return EPsi;
00566 };
00567
00568 template <class TQ, class TD>
00569 void CPAT<TQ,TD>::check_contraction(
00570     const QSpace<TQ,TD> &Psi, char disp
00571 ) {
00572     if (!nr_check_ctr) getCtrIndex(Psi); else
00573     if (sameCtrIdx<nr_check_ctr) {
00574         unsigned i=getCtrIndex(Psi);
00575         if (i==0) {
00576             sameCtrIdx++;
00577
00578             if (sameCtrIdx==nr_check_ctr && disp>10)
00579                 wblog(FL,"<i> using sameCtrIdx=%d", sameCtrIdx);
00580         }
00581         else {
00582             if (sameCtrIdx && disp)
00583                 wblog(FL,"WRN sameCtrIdx=%d reset to 0.", sameCtrIdx);
00584
00585             sameCtrIdx=0;
00586         }
00587     }
00588     else ARG_CHECK=0;
00589 };
00590
00591 template <class TQ, class TD>
00592 inline void CPAT<TQ,TD>::getQRange(
00593     unsigned k,
00594     wbMatrix<TQ> &QQ,
00595     wbvector<unsigned> &SS,
00596     QSpace<TQ,TD> *B
00597 ) const {
00598
00599     unsigned i,j,l,n,nA,m=CP.len; int e=0;
00600     wbvector<unsigned> S,D;
00601     wbMatrix<TQ> Q;
00602     wperm P;
00603
00604     for (i=0; i<m; i++) { n=CP[i].len;
00605         for (j=0; j<n; j++) { nA=CP[i][j].A.len; ;
00606             if (CP[i][j].ic!=k) continue;
00607
00608             for (l=0; l<nA; l++) {
00609                 CP[i][j].A[l].getQDim(Q,S);
00610                 QQ.cat(1,Q);
00611                 SS.Append(S);
00612             }
00613         }
00614     }
00615     if (B) {
00616         B->getQDim(k,Q,S);
00617         QQ.cat(1,Q);
00618         SS.Append(S);
00619     }
00620
00621     QQ.groupRecs(P,D);

```

```

00622     e=SS.set2Group(P,D,0,'i');
00623
00624     if (e) wblog(FL,
00625         "ERR QSpace size inconsistency (dim=%d): %s (%d)",k+1,str,e);
00626 }
00627
00628 template <class TQ, class TD>
00629 inline void QSpace<TQ,TD>::ExpandQ(
00630     CPAT<TQ,TD> &CP,
00631     double rnm,
00632     wbvector<unsigned> &xflag,
00633     char disp
00634 ) {
00635     unsigned i,j,k,l,n,nr,s,r=rank(FL), dmin=0;
00636     const char EXISTS=1, OLDSYM=1<<1, XPAND=1<<2, XPAND2=1<<3, rank2=(r<3);
00637     int e=0;
00638
00639     wbvector< wbMatrix<TQ> > QQ;
00640     wbvector< wbvector<unsigned> > SS;
00641     wbvector<unsigned> D(r),S(r),dc;
00642     wbMatrix<unsigned> SA;
00643     wbvector<char> mark;
00644     wbMatrix<TQ> Q1,Q2,QK1,QK2;
00645     QSpace<TQ,TD> AA;
00646     wbindex I1,I2,d1,d2,I(r);
00647     wperm p1,p2;
00648     TD n1,n2;
00649
00650     QQ.initDef(r);
00651     SS.initDef(r);
00652
00653     for (i=0; i<r; i++) {
00654         CP.getQRange(i,QQ[i],SS[i], this);
00655
00656         S[i]=SS[i].len;
00657
00658         D[i]=SS[i].sum();
00659         if (D[i]>1) {
00660             if (dmin==0) dmin=D[i]; else dmin=MIN(dmin,D[i]); }
00661     }
00662
00663     if (xflag.isEmpty()) {
00664         if (!rank2) {
00665             xflag.init(r);
00666             for (i=0; i<r; i++) if (D[i]<=dmin) xflag[i]=1;
00667         }
00668     }
00669     else if (xflag.len!=r) wblog(FL,
00670         "ERR %s - invalid xflag (dim. mismatch %d/%d)",FCT,xflag.len,r);
00671
00672     if (disp>9)
00673         wblog(FL,"TST D=[%s], expand=[%s] (%d)",
00674             D.toStr().data, xflag.toStr().data, dmin);
00675
00676     s=S.prod(); I.set(0);
00677     AA.init(s,r,QDIM); SA.init(s,r); mark.init(s);
00678
00679     for (l=S.len-1, i=0; i<s; i++) {
00680         for (k=0; k<I.len; k++) {
00681             AA.setQRec(i, k, QQ[k].rec(I[k]));
00682             SA(i,k)=SS[k][I[k]];
00683         }
00684
00685         I[l]++; k=1;
00686         while(I[k]>=S[k] && k>0) { I[k]=0; ++I[--k]; }
00687     }
00688
00689     QIDX.blockSum(QDIM,Q1); Q1.makeUnique();
00690     AA.QIDX.blockSum(QDIM,Q2); Q2.groupRecs(p2,d2);
00691
00692     e=matchIndex(Q1,Q2,I1,I2);
00693     if (e || I1.len!=Q1.dim1) wblog(FL,
00694         "ERR Failed to locate all existing symmetry sectors\n"
00695         "(e=%d; len=%d/%d)", e, I1.len, Q1.dim1);
00696
00697     d2.cumsum_(dc);
00698     for (i=0; i<I2.len; i++) {
00699         l=dc[I2[i]]; n=d2[I2[i]];
00700         for (j=0; j<n; j++) mark[p2[l+j]] |= OLDSYM;
00701     }
00702
00703     if (disp>9) sprintf_str(
00704         "Psi having %d/%d symmetry sector(s)", Q1.dim1, Q2.dim1);
00705
00706     xflag.find(0,I); if (!I.isEmpty()) {
00707         getQsub(I,QK1); QK1.groupRecs(p1,d1);
00708         AA.getQsub(I,QK2); QK2.groupRecs(p2,d2);

```

```

00709
00710     e=matchIndex(QK1,QK2,I1,I2);
00711     if (e || I1.len!=QK1.dim1) wblog(FL,
00712         "ERR Failed to locate all existing symmetry sectors (%d/%d)",
00713         I1.len, QK1.dim1);
00714
00715     d2.cumsum_(dc);
00716     for (i=0; i<I2.len; i++) {
00717         l=dc[I2[i]]; n=d2[I2[i]];
00718         for (j=0; j<n; j++) mark[p2[l+j]] |= XPAND;
00719     }
00720 }
00721 else {
00722     for (i=0; i<mark.len; i++) mark[i] |= XPAND;
00723 }
00724
00725 e=matchIndex(QIDX, AA.QIDX, I1, I2);
00726 if (e || I1.len!=QIDX.dim1) wblog(FL,
00727     "ERR Failed to locate all existing data blocks (%d/%d)",
00728     I1.len, QIDX.dim1);
00729
00730 for (i=0; i<I2.len; i++) {
00731     mark[I2[i]] |= EXISTS;
00732     DATA[I1[i]]->save2(*(AA.DATA[I2[i]]));
00733
00734     if (!SA.recEqual(I2[i], AA.DATA[I2[i]]->SIZE.data)) {
00735         SA.recPrint(i,"SA");
00736         AA.DATA[I2[i]]->SIZE.print("SIZE");
00737         wblog(FL, "ERR Severe size inconsistency (%d)", i+1);
00738     }
00739 }
00740
00741 if (rnrnrm<=0) {
00742     for (i=0; i<mark.len; i++)
00743         if (mark[i]>0 && !(mark[i] & OLDSYM)) mark[i]=-mark[i];
00744 }
00745
00746 for (n=i=0; i<mark.len; i++) if (mark[i]>0) n++;
00747
00748 init(n,r,QDIM);
00749 I1.init(n);
00750
00751 for (nr=l=i=0; i<mark.len; i++) {
00752     if (mark[i]<=0) continue;
00753
00754     QIDX.recSetP(l, AA.QIDX.rec(i));
00755
00756     if (AA.DATA[i]->isEmpty()) {
00757         S.init2ref(SA.dim2, SA.rec(i));
00758         DATA[l]->init(S);
00759         if (!(mark[i] & OLDSYM)) {
00760             DATA[l]->setRand('s');
00761             I1[l]=1; nr++;
00762         }
00763     }
00764     else AA.DATA[i]->save2(*DATA[l]);
00765
00766     l++;
00767 }
00768
00769 if (rnrnrm>0 && nr) {
00770     double dbl,fac=0,r2=rnrnrm*rnrnrm*1E-3;
00771     if (fabs(rnrnrm)>1) wblog(FL,"WRN rnrnrm = %g !?",rnrnrm);
00772
00773     for (n1=n2=0,i=0; i<I1.len; i++) {
00774         if (I1[i]) n2+=(DATA[i]->norm2());
00775         else {
00776             dbl=(DATA[i]->norm2()); fac+=dbl;
00777             if (dbl>r2) n1+=dbl;
00778             else {
00779                 DATA[i]->setRand('s');
00780                 I1[i]=2; n2+=(DATA[i]->norm2());
00781             }
00782         }
00783     }
00784
00785     if (n1!=0 && n2!=0) {
00786         r2=rnrnrm*rnrnrm; fac/=(n1+r2);
00787
00788         if (disp>9) wblog(FL,
00789             "TST n1=%g, n2=%g, r2=%g -> fac=%g",n1,n2,r2,fac);
00790
00791         n1=(TD)sqrt(fac*n1);
00792         n2=(TD)sqrt(fac*(r2/n2));
00793
00794         for (i=0; i<I1.len; i++)
00795             if (I1[i]) (*DATA[i])*=n2; else (*DATA[i])*=n1;

```

```

00796     }
00797     else wblog(FL,"WRN rand. norm = %.3g (%.3g) !?",n2,n1);
00798 }
00799
00800 if (xflag.count(0)) {
00801
00802     unsigned itry=0,iTRY=0, ntry=16, nQ=1, m=CP.CP.len, mx=0, m0=QIDX.dim1;
00803     QSpace<TQ,TD> AX;
00804
00805     Q1.init(); Q2.init();
00806
00807     for (iTRY=0; iTRY<ntry && nQ; iTRY++) {
00808     for (itry=0; itry<ntry; itry++) {
00809         CP.getCtrIndex(*this);
00810
00811         for (i=0; i<m; i++) {
00812             j=CP.CP[i].len-1;
00813             nQ=CP.CP[i][j].CQn.len;
00814
00815             for (l=0; l<nQ; l++)
00816                 Q1.cat(l,
00817                     CP.CP[i][j].CQn[l].QIDX
00818                 );
00819         }
00820
00821         Q1.makeUnique();
00822         if (Q1==Q2) break; else Q1.save2(Q2);
00823     }
00824
00825     if (itry==ntry) wblog(FL,
00826 "ERR CP|psi> keeps generating new terms (%d).\n"
00827 "Symmetries not preserved !?", ntry);
00828     if (Q1.dim2!=QIDX.dim2) wblog(FL,
00829 "ERR Severe QIDX dimension mismatch (%d/%d)",Q1.dim2,QIDX.dim2);
00830
00831     e=matchIndex(Q1,AA.QIDX,I1,I2);
00832
00833     if (e || I1.len!=Q1.dim1) wblog(FL,
00834 "ERR Failed to locate all existing data blocks (%d/%d)",
00835     I1.len, Q1.dim1);
00836
00837     for (nQ=i=0; i<I2.len; i++) if (mark[I2[i]]<=0) nQ++;
00838     if (nQ) {
00839         l=QIDX.dim1; if (l!=DATA.len) wblog(FL,"%d/%d ???",l,DATA.len);
00840         QIDX.Resize(l+nQ, QIDX.dim2);
00841         DATA.Resize(l+nQ);
00842         for (i=0; i<nQ; i++) DATA[l+i] = new wbararray<TD>;
00843
00844         for (i=0; i<I2.len; i++) {
00845             j=I2[i]; if (mark[j]>0) continue;
00846             if (mark[j]<0) mark[j]=-mark[j]; else
00847                 mark[j] |= XPAND2; mx++;
00848
00849             QIDX.recSetP(l, AA.QIDX.rec(j));
00850
00851             if (AA.DATA[j]->isEmpty()) {
00852                 S.init2ref(SA.dim2, SA.rec(j));
00853                 DATA[l]->init(S);
00854             }
00855             else {
00856                 wblog(FL,"WRN excuse moi !?");
00857                 AA.DATA[j]->save2(*DATA[l]);
00858             }
00859
00860             l++;
00861         }
00862     }
00863
00864     if (disp>9) wblog(FL,"TST itry=%d, iTRY=%d (%d)",itry,iTRY,ntry);
00865
00866     if (iTRY==ntry) wblog(FL,
00867 "ERR CP|psi> keeps generating new terms (%d).\n"
00868 "Symmetries not preserved !?", ntry);
00869
00870     if (disp>9)
00871         wblog(FL,"TST D=[%s], %d/%d completing to %d+%d blocks",
00872         getDim().toStr().data, iTRY+1,itry+1,m0,mx);
00873 }
00874
00875 if (disp>9)
00876     wblog(FL,"TST %s -> %d", str, getDQtot());
00877 }
00878
00879 template <class TQ, class TD>
00880 QSpace<TQ,TD>& CPAT<TQ,TD>::HZTimes_tst(
00881     const QSpace<TQ,TD> &PSI,
00882     QSpace<TQ,TD> &HZPsi,

```

```

00883     const char tflag,
00884     char disp
00885 ){
00886     unsigned i,j,it;
00887     static unsigned call=0;
00888     wbpem P;
00889
00890     if (!(call++)) wblog(FL,"TST %N*** using HZTimes_tst ***%N");
00891
00892     QSpace<TQ,TD> X1,X2,XX;
00893
00894     if (tflag=='C' && ISREAL(TD)) tflag='T';
00895
00896     if (!Wb::strchr("NTC",tflag)) wblog(FL,
00897     "ERR Invalid flag %c<%d>", tflag, tflag);
00898
00899     it=( tflag=='N' ? 2 : ( tflag=='T' ? 1 : -1 ));
00900
00901     HZPsi=PSI;
00902     HZPsi *= (tflag=='C' ? -CONJ(z) : -z);
00903
00904     for (i=0; i<CP.len; i++) {
00905
00906         if (CP[i].len<=1) {
00907             CPAT_Set<TQ,TD> &C1=CP[i][0];
00908             for (j=0; j<C1.A.len; j++) {
00909                 P.initFirstTo(C1.ic, PSI.rank());
00910                 if (tflag!='C')
00911                     C1.A[j].contract(it,PSI, C1.ic+1, X1, P);
00912                 else C1.A[j].opA(tflag,XX).contract(2, PSI, C1.ic+1, X1, P);
00913
00914                 HZPsi+=X1;
00915             }
00916         }
00917         else {
00918             CPAT_Set<TQ,TD> &C1=CP[i][0], &C2=CP[i][1];
00919             if (C1.A.len!=C2.A.len || CP[i].len>2) wblog(FL,"ERR ???");
00920
00921             for (j=0; j<C1.A.len; j++) {
00922                 P.initFirstTo(C1.ic, PSI.rank());
00923                 if (tflag!='C')
00924                     C1.A[j].contract(it,PSI, C1.ic+1, X1, P);
00925                 else C1.A[j].opA(tflag,XX).contract(2, PSI, C1.ic+1, X1, P);
00926
00927                 P.initFirstTo(C2.ic, PSI.rank());
00928                 if (tflag!='C')
00929                     C2.A[j].contract(it,X1 , C2.ic+1, X2, P);
00930                 else C2.A[j].opA(tflag,XX).contract(2, X1 , C2.ic+1, X2, P);
00931
00932                 HZPsi+=X2;
00933             }
00934         }
00935     }
00936
00937     return HZPsi;
00938 };
00939
00940 template <class TQ, class TD>
00941 inline void CPAT_Set<TQ,TD>::contract (
00942     const QSpace<TQ,TD> &PSI,
00943     wvector< QSpace<TQ,TD> > &C,
00944     char tflag
00945 ) const {
00946
00947     unsigned i,n=A.len;
00948     if (C.len!=n) C.initDef(n);
00949
00950     for (i=0; i<n; i++)
00951         contract(i,PSI,C[i],tflag);
00952 }
00953
00954 template <class TQ, class TD>
00955 inline void CPAT_Set<TQ,TD>::contract_diag(
00956     const QSpace<TQ,TD> &PSI,
00957     wvector< QSpace<TQ,TD> > &C,
00958     TD a, TD b, char Iflag
00959 ) const {
00960
00961     unsigned i,n=A.len;
00962     if (C.len!=n) C.initDef(n);
00963
00964     for (i=0; i<n; i++)
00965         contract_diag(i,PSI,C[i],a,b,Iflag);
00966 }
00967
00968 template <class TQ, class TD>
00969 inline void CPAT_Set<TQ,TD>::contract (

```

```

00970     const wbvector< QSpace<TQ,TD> > &PSI1,
00971     wbvector< QSpace<TQ,TD> > &C,
00972     char tflag
00973 ) const {
00974
00975     unsigned i,n=A.len;
00976     if (C.len!=n) C.initDef(n);
00977
00978     for (i=0; i<n; i++)
00979         contract(i,PSI1[i],C[i],tflag);
00980 }
00981
00982 template <class TQ, class TD>
00983 inline void CPAT_Set<TQ,TD>::contract_diag(
00984     const wbvector< QSpace<TQ,TD> > &PSI1,
00985     wbvector< QSpace<TQ,TD> > &C,
00986     TD a, TD b, char Iflag
00987 ) const {
00988
00989     unsigned i,n=A.len;
00990     if (C.len!=n) C.initDef(n);
00991
00992     for (i=0; i<n; i++)
00993         contract_diag(i,PSI1[i],C[i],a,b,Iflag);
00994 }
00995
00996 template <class TQ, class TD>
00997 void CPAT_Set<TQ,TD>::contract(
00998     const unsigned k,
00999     const QSpace<TQ,TD> &PSI,
01000     QSpace<TQ,TD> &C,
01001     char tflag
01002 ) const {
01003
01004     unsigned i,n,ia,ib,iu,it;
01005     wbMatrix<unsigned> &Iabu = (tflag=='N' ? CIN[k] : CIt[k]);
01006     wbarray<TD> XX;
01007
01008 #ifdef WB_CLOCK
01009     Wb::Clock clk("cpat:contract",0);
01010 #endif
01011
01012     if (tflag=='C' && ISREAL(TD)) tflag='T';
01013
01014     if (!Wb::strchr("NTC",tflag)) wblog(FL,
01015         "ERR Invalid flag %c<%d>",tflag,tflag);
01016
01017     C.QDIM=PSI.QDIM;
01018     C.QIDX=(tflag=='N' ? CQn[k].QIDX : CQt[k].QIDX);
01019     C.setupDATA();
01020
01021     it=( tflag=='N' ? 2 : ( tflag=='T' ? 1 : -1 ));
01022
01023     for (n=Iabu.dim1, i=0; i<n; i++) {
01024         ia=Iabu(i,0); ib=Iabu(i,1); iu=Iabu(i,2);
01025
01026         if (tflag!='C') {
01027             A[k].DATA[ia]->
01028                 contract(it,*(PSI.DATA[ib]), ic+1, *(C.DATA[iu]), P); }
01029         else {
01030             A[k].DATA[ia]->opA(tflag,XX).
01031                 contract(2, *(PSI.DATA[ib]), ic+1, *(C.DATA[iu]), P);
01032         }
01033     }
01034 }
01035
01036 template <class TQ, class TD>
01037 void CPAT_Set<TQ,TD>::contract_diag(
01038     const unsigned k,
01039     const QSpace<TQ,TD> &PSI,
01040     QSpace<TQ,TD> &C,
01041     TD a, TD b, char Iflag
01042 ) const {
01043
01044     unsigned i,n,ia,ib,iu;
01045     wbMatrix<unsigned> &Iabu = CIN[k];
01046     wbarray<TD> XX;
01047
01048 #ifdef WB_CLOCK
01049     Wb::Clock clk("cpat:contract",0);
01050 #endif
01051
01052     C.QDIM=PSI.QDIM; C.QIDX=CQn[k].QIDX;
01053     C.setupDATA();
01054
01055     for (n=Iabu.dim1, i=0; i<n; i++) {
01056         ia=Iabu(i,0); ib=Iabu(i,1); iu=Iabu(i,2);

```



```

01057
01058     if (A[k].isDiagBlock(ia))
01059         PSI.DATA[ib]->contractDiag(
01060             ic+1,*A[k].DATA[ia], a,b, *C.DATA[iu], Iflag
01061         );
01062     }
01063 }
01064
01065 template <class TQ, class TD>
01066 mxArray* CPAT<TQ,TD>::toMx() const {
01067
01068     unsigned i,j,l,k, m=CP.len, n;
01069     mxArray *S, *C, *a;
01070
01071     C=mxCreateCellMatrix(m,1);
01072
01073     for (i=0; i<m; i++) {
01074         n=CP[i].len;
01075         S=mxCreateStructMatrix(1,n,0,NULL);
01076
01077         for (j=0; j<n; j++) {
01078             a=CP[i][j].toMx();
01079             k=mxGetNumberOfFields(a);
01080
01081             if (j==0)
01082                 for (l=0; l<k; l++) {
01083                     if (mxAddField(S,mxGetFieldNameByNumber(a,l))<0) wblog(FL,
01084                         "ERR Could not add field `%s'", mxGetFieldNameByNumber(a,l));
01085                 }
01086
01087                 for (l=0; l<k; l++) {
01088                     mxSetFieldByNumber(S, j, l,
01089                         mxDuplicateArray(mxGetFieldByNumber(a,0,l)));
01090                 }
01091
01092                 mxDestroyArray(a);
01093             }
01094             mxSetCell(C,i,S);
01095         }
01096
01097         S=mxCreateStructMatrix(1,1,0,NULL);
01098         mxAddField2Scalar(FL,S,"CP",C);
01099         mxAddField2Scalar(FL,S,"z",numtoMx(z));
01100
01101         return S;
01102     }
01103
01104 template <class TQ, class TD>
01105 mxArray* CPAT_Set<TQ,TD>::toMx() const {
01106
01107     mxArray *S, *a;
01108     unsigned i;
01109
01110     S=mxCreateStructMatrix(1,1,0,NULL);
01111
01112     mxAddField2Scalar(FL,S,"A", A.toMx() );
01113     mxAddField2Scalar(FL,S,"ic", numtoMx(ic+1));
01114     mxAddField2Scalar(FL,S,"opA", numtoMx(opA) );
01115     mxAddField2Scalar(FL,S,"P", (P+1).toMx() );
01116
01117     for (i=0; i<CQn.len; ++i) CQn[i].setupDATA();
01118     for (i=0; i<CQt.len; ++i) CQt[i].setupDATA();
01119
01120     mxAddField2Scalar(FL,S,"CQn", CQn.toMx());
01121     mxAddField2Scalar(FL,S,"CQt", CQt.toMx());
01122
01123     a=mxCreateCellMatrix(CIn.len,1);
01124     for (unsigned i=0; i<CIn.len; i++)
01125         mxSetCell(a, i, CIn[i].toMx());
01126     mxAddField2Scalar(FL,S,"CIn",a);
01127
01128     a=mxCreateCellMatrix(CIt.len,1);
01129     for (unsigned i=0; i<CIt.len; i++)
01130         mxSetCell(a, i, CIt[i].toMx());
01131     mxAddField2Scalar(FL,S,"CIt",a);
01132
01133     return S;
01134 }
01135
01136 int isCPAT(const mxArray *CC, char twoSite) {
01137     unsigned i,i0,k,n;
01138
01139     if (!CC || !mxIsCell(CC)) return 0;
01140     if (isCPattern(CC)) return 1;
01141
01142     n=mxGetNumberOfElements(CC);
01143     if (!n) return 0;

```

```

01144
01145     i0=isCPattern(mxGetCell(CC,0));
01146     if (!i0) return 0;
01147
01148     for (k=1; k<n; k++) {
01149         i=isCPattern(mxGetCell(CC,k));
01150         if (!i) return 0;
01151
01152         if (i!=i0) wblog(FL,
01153             "ERR CPAT mismatch twoSite=%d,%d ???", i0, i);
01154     }
01155
01156     if (twoSite>=0) if (1+twoSite!=(char)i0) wblog(FL,
01157         "ERR CPAT is not of type %d-site (%d)", 1+twoSite, i0);
01158
01159     return n;
01160 }
01161
01162 int isCPattern(const mxArray *C) {
01163     unsigned k,n,w;
01164     char twoSite, hasOp;
01165     mxArray *a;
01166
01167     if (!C || !mxIsCell(C)) {
01168         sprintf_str("%s:%d invalid CP",FL);
01169         return 0;
01170     }
01171
01172     n=mxGetNumberOfElements(C);
01173     if (n<2 || n>4) {
01174         sprintf_str("%s:%d invalid CP (%d entries)",FL,n);
01175         return 0;
01176     }
01177
01178     a=mxGetCell(C,0);
01179     if (!mxIsStruct(a) || mxIsCell(a)) {
01180         sprintf_str("%s:%d invalid arg #1",FL);
01181         return 0;
01182     }
01183
01184     a=mxGetCell(C,n-1);
01185     hasOp=mxIsChar(a);
01186
01187     if (hasOp) if (mxGetNumberOfElements(a)!=1) {
01188         sprintf_str("%s:%d arg #%d not a flag (char)",FL,n);
01189         return 0;
01190     }
01191
01192     k=n-hasOp;
01193     if (k==2) twoSite=0; else
01194     if (k==3) twoSite=1; else {
01195         sprintf_str("%s:%d invalid CP (k=%d)",FL,k);
01196         return 0;
01197     }
01198
01199     if (twoSite) {
01200         w = (mxIsEmpty(mxGetCell(C,1)) ? 0 : 1) +
01201             (mxIsEmpty(mxGetCell(C,2)) ? 0 : 2);
01202         if (w<1 || w>2) {
01203             sprintf_str(" %s:%d either arg #2 or #3",FL);
01204             return 0;
01205         }
01206     }
01207     else w=1;
01208
01209     a=mxGetCell(C,w);
01210     if (!Mx::IsNumber(a)) {
01211         sprintf_str("%s:%d arg #%d not a number",FL,w+1);
01212         return 0;
01213     }
01214
01215     return (1+twoSite);
01216 }
01217
01218 #endif
01219

```

5.3 dmrho.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : QSpace NRG routines
00004  *

```

```

00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_DMRHO_CC__
00021 #define __WB_DMRHO_CC__
00022
00023 /* ----- */
00024 /* ----- */
00025
00026 template <class TQ, class TD>
00027 double initRHO(
00028     wbMatrix<double> &rhoNorm,
00029     const double T,
00030     NRGData<TQ,TD> &H,
00031     wbvector<double> E0,
00032     const unsigned dloc,
00033     const unsigned NRho=0,
00034     char vflag=1
00035 );
00036
00037 template <class TQ, class TD>
00038 void getEData(
00039     const QSpace<TQ,TD> &HK,
00040     wbvector<double> &E, wbvector<unsigned> &D,
00041     wbvector<unsigned> *dz=NULL
00042 );
00043
00044 template <class TQ, class TD>
00045 TD getERange(const QSpace<TQ,TD> &HK, double *emin=NULL, double *emax=NULL);
00046
00047 template <class TQ, class TD>
00048 unsigned getGSDegeneracy(const QSpace<TQ,TD> &H);
00049
00050 template <class TQ, class TD>
00051 bool getBoltzman(
00052     const QSpace<TQ,TD> &H,
00053     wbvector<double> &R, wbvector<unsigned> &D, unsigned iter,
00054     wbvector<unsigned> *DZ=NULL, char useE0=1
00055 );
00056
00057 template <class TD>
00058 bool getBoltzman_base(
00059     const wbvector<TD> &E, const wbvector<unsigned> &dz,
00060     wbvector<TD> &R, unsigned iter, double T=0., char useE0=1
00061 );
00062
00063 template <class TQ, class TD>
00064 double initRho(
00065     QSpace<TQ,TD> &Rho,
00066     const QSpace<TQ,TD> &HK,
00067     const int iter,
00068     double efac=1.
00069 );
00070
00071 template <class TQ, class TD>
00072 void nrgUpdaterRHO(
00073     const wbMatrix<double> &R,
00074     NRGData<TQ,TD> &A,
00075     NRGData<TQ,TD> &H,
00076     NRGData<TQ,TD> &RHO,
00077     unsigned NRho,
00078     unsigned dloc,
00079     Wb::SigHandler *sig=NULL
00080 );
00081
00082 template <class TQ, class TD>
00083 void updateRho(
00084     QSpace<TQ,TD> &Rho,
00085     const QSpace<TQ,TD> &AK
00086 );
00087
00088 template <class TQ, class TD, class TX>
00089 void updateOp_SR(QSpace<TQ,TX> &X, const QSpace<TQ,TD> &AK);
00090
00091 template <class TQ, class TD, class TZ>

```

```

00092 void initG_SR(
00093     QSpace<TQ,TZ> &G2, const QSpace<TQ,TD> &H2,
00094     wbcomplex z, unsigned iter
00095 );
00096
00097 template <class TQ, class TD>
00098 void checkRho (const char *F, int L,
00099     const QSpace<TQ,TD> &Rho,
00100     const double traceRho=1.
00101 ){
00102     unsigned i,n=Rho.QIDX.dim1;
00103     double rsum=0, rmin=0;
00104
00105     wbvector< wbvector<double> > E(n);
00106     wbvector<double> rr;
00107     wbarray<TD> U;
00108
00109     char cgflag=Rho.getCGS(FL);
00110
00111     if (cgflag>0 && Rho.qtype.permitsOM()>1) {
00112         cgflag=2;
00113     }
00114
00115     if (!Rho.isConsistent_r(2) || !Rho.isBlockDiagMatrix() || !Rho.isHConj())
00116     wblog(F,L,"ERR %s() inconsistency in Rho QSpace (%d;%d;%d)",
00117     FCT, Rho.isBlockDiagMatrix(), Rho.isHConj());
00118
00119     for (i=0; i<n; i++) {
00120         wbEigenS(* (Rho.DATA[i]),U,E[i]);
00121         if (cgflag>0) {
00122             unsigned d=Rho.cgsDimScalar(i);
00123             if (cgflag<=1) {
00124                 unsigned d0=Rho.qtype.QDim(Rho.QIDX.rec(i)); if (d0!=d)
00125                 wblog(FL,"ERR %s() cgsDim inconsistency! (%d/%d)",FCT,d,d0);
00126             }
00127             E[i]*=d;
00128         }
00129     }
00130     rr.Cat(E);
00131
00132     rmin=rr.min();
00133     rsum=rr.sum();
00134
00135     if (rmin<-EPS) { wblog(F,L,
00136         "ERR eig(rho) is in (%.4g .. %.4g; %.4g)",rmin,rr.max(),rsum);
00137     }
00138     else if (fabs(rsum-traceRho)>EPS) {
00139         char e=(fabs(rsum-traceRho)>1E-8 && rsum>1E-8 && traceRho>1E-8);
00140         if (e) {
00141             printf("\n\n");
00142             MXPut(FL).add(Rho,"Rho").add(rr,"rr").add(traceRho,"t");
00143         }
00144         sprintf_str("%s tr(rho) inconsistency !?\n"
00145             "%.4g / %.4g @ %.3g (%.2g; %.2g; %.2g)", e ? "ERR":"WRN",
00146             rsum,traceRho,rsum-traceRho,DEPS,DEPS2,EPS); printf("\n");
00147         wblog(F,L,str);
00148     }
00149 };
00150
00151 template <class TQ, class TD>
00152 double initRHO(
00153     wbMatrix<double> &rhoNorm,
00154     const double T,
00155     NRGData<TQ,TD> &H,
00156     wbvector<double> E0,
00157     const unsigned dloc,
00158     const unsigned NRho,
00159     char vflag
00160 ){
00161     unsigned i=0,i1,i2, iter, isneg=0, N=NRG_N; char i0flag=0;
00162     wbvector<double> R;
00163     wbvector<unsigned> D;
00164     double dbl, dfac=1;
00165     double DFAC=DBL_MAX*1E-2;
00166
00167     if (NRho>N) wblog(FL,"ERR NRho out of bounds (%d/%d)");
00168
00169     i1 = (dloc && NRho ? NRho-1 : 0);
00170     i2 = (NRho ? NRho-1 : N-1);
00171
00172     if (int(i1)>int(i2) || int(i1)<0) wblog(FL,
00173         "ERR rho_i = [%d..%d] !?",i1,i2);
00174
00175     rhoNorm.init(N,2);
00176
00177     dbl=E0.min(i);
00178     if (i==0) { i0flag=1;

```

```

00179     E0[0]+=E0[1]; dbl=E0.min(i);
00180     E0[0]-=E0[1];
00181 }
00182
00183 if (i<i2) {
00184     wblog(FL,"WRN Emin set by site %d/%d !?",i+1,N);
00185     MXPut(FL).add(E0,"E0");
00186 }
00187
00188 getBoltzman_base(E0,D,E0,-1,T);
00189
00190 for (iter=i1; iter<=i2; ++iter) {
00191     H.init(FL,"D",iter);
00192     if (getBoltzman(H.D,R,D,iter)) ++isneg;
00193     rhoNorm(iter,1)=R.sum();
00194
00195     if (dloc && iter<i2) continue;
00196
00197     H.init(FL,"K",iter);
00198     if (getBoltzman(H.K,R,D,iter)) ++isneg;
00199     rhoNorm(iter,0)=R.sum();
00200 }
00201
00202 if (isneg) wblog(FL,
00203     "WRN %s() negative (eigen)energies (%d) !?",FCT,isneg);
00204 if (i0flag && (rhoNorm(0,0)>1 || rhoNorm(0,1)>1)) wblog(FL,
00205     "WRN %s() density matrix from first site may be unstable (%.3g,%.3g)",
00206     FCT, rhoNorm(0,0), rhoNorm(0,1));
00207
00208 if (!H.K.isEmpty())
00209     i=getGSDegeneracy(H.K);
00210 else i=getGSDegeneracy(H.D);
00211
00212 if (vflag) {
00213     if (i==1)
00214         wblog(FL," * non-degenerate ground state");
00215     else wblog(FL,"<i> ground state degeneracy k=%d (%d)",iter-1,i);
00216 }
00217
00218 if (!dloc) {
00219     if (!NRho) wblog(FL,"ERR NRho=%d !?", NRho);
00220     wblog(FL,"<i> generate weights for plain NRG (T=%.3g)", T);
00221
00222     for (iter=0; iter<NRho; iter++) {
00223         dbl=rhoNorm.recSum(iter);
00224         if (dbl<1) wblog(FL,"WRN sum_rhoNorm(%d,.)=%.3g",iter+1,dbl);
00225         dbl=1/dbl;
00226         rhoNorm(iter,0)*=dbl;
00227         rhoNorm(iter,1)*=dbl;
00228     }
00229     return 0;
00230 }
00231
00232 if (!NRho)
00233 for (dfac=dloc, iter=i2-1; (int)iter>=0; iter--, dfac*=dloc) {
00234     rhoNorm(iter,1)*=dfac;
00235
00236     if (dfac>DFAC) wblog(FL,"WRN dfac reaches DBL_MAX (%.3g)", dfac);
00237 }
00238
00239 dbl=rhoNorm.sum();
00240 if (dbl<=0.) wblog(FL,"ERR %s() normRH0=%g ??? DIV/0",FCT,dbl);
00241 rhoNorm *= (1/dbl);
00242
00243 if (vflag) {
00244     double rmax;
00245     rhoNorm.recSum(R); rmax=R.max(i);
00246     wblog(FL," * maxRho=%.4g at iter=%d/%d (T=%.4g)",rmax,i+1,N,T);
00247 }
00248
00249 return (-T*log(dbl));
00250 };
00251
00252 template <class TQ, class TD>
00253 void getEData(
00254     const QSpace<TQ,TD> &HK,
00255     wvector<double> &E,
00256     wvector<unsigned> &D,
00257     wvector<unsigned> *DZ
00258 ){
00259     unsigned i,l,m=0,d, e=0, k,K=HK.QIDX.dim1, *dz=NULL;
00260     char cgflag=HK.getCGS(FL);
00261
00262     if (HK.DATA.len!=K) wblog(FL,
00263         "ERR %s() severe QSpace inconsistency (%d,%d)",
00264         FCT, HK.DATA.len, HK.QIDX.dim1);
00265     if (!HK.isBlockDiagMatrix('d')) wblog(FL,

```

```

00266     "ERR %s() Hamiltonian must be (block)diagonal!",FCT);
00267
00268     D.init(K);
00269     for (l=k=0; k<K; k++) D[k]=HK.DATA[k]->SIZE.max();
00270
00271     E.init(D.sum());
00272     if (DZ && cgflag>0) { DZ->init(E.len); dz=DZ->data; }
00273
00274     if (cgflag>0 && HK.qtype.permitsOM(>1) {
00275         cgflag=2;
00276     }
00277
00278     for (l=k=0; k<K; ++k) {
00279         const wbararray<TD> &Hk = *HK.DATA[k];
00280         const wbvector<unsigned> &S = Hk.SIZE;
00281         const TD* const E0=Hk.data;
00282
00283         d=D[k]; if (dz) {
00284             m=HK.cgsDimScalar(k);
00285             if (cgflag<=1) {
00286                 unsigned m0=HK.qtype.QDim(HK.QIDX.rec(k)); if (m0!=m)
00287                     wblog(FL,"ERR %s() cgsDim inconsistency! (%d/%d)",FCT,m,m0);
00288             }
00289         }
00290
00291         if (S[0]!=S[1]) { if (S[0]!=1 && S[1]!=1) e++; else {
00292             Wb::cpyRange(E.data+l, E0, d);
00293             if (dz)
00294                 { for (i=0; i<d; ++i,++l) dz[l]=m; }
00295             else { l+=d; }
00296         }
00297         else { if (S.len!=2) e++; else {
00298             if (d>1 && (fabs(E0[1])>1E-12 || fabs(E0[d])>1E-12)) wblog(FL,
00299                 "WRN HK[%d] not E diagonal (%g,%g)", k+1,E0[1], E0[d]);
00300
00301             for (i=0; i<d; ++i,++l) {
00302                 E[l]=E0[i*d+i]; if (dz) dz[l]=m;
00303             }
00304         }
00305         if (e) wblog(FL,
00306             "ERR HK[%d] must be rank-2 tensor! (%s)",
00307             k+1, HK.DATA[k]->sizeStr().data
00308         );
00309     }
00310 };
00311
00312 template <class TQ, class TD>
00313 unsigned getGSDegeneracy(const QSpace<TQ,TD> &H) {
00314
00315     wbvector<double> E; wbvector<unsigned> D,dz;
00316     unsigned i=0, n=0;
00317     double e0, *ee;
00318
00319     getEData(H,E,D,&dz); e0=E.min(); ee=E.data;
00320     for (; i<E.len; ++i) if (fabs(ee[i]-e0)<EPS) n++;
00321
00322     return n;
00323 };
00324
00325 template <class TQ, class TD>
00326 TD getErange(const QSpace<TQ,TD> &H, double *emin, double *emax){
00327
00328     if (!H.isBlockDiagMatrix('d')) {
00329         QSpace<TQ,TD> AK;
00330         QSpace<TQ,double> HK; wbMatrix<double> EE;
00331         H.EigenSymmetric(AK,HK,EE);
00332         return getErange(HK,emin,emax);
00333     }
00334
00335     wbvector<double> E; wbvector<unsigned> D;
00336     TD e1,e2;
00337
00338     getEData(H,E,D);
00339     e1=E.min(); if (emin) (*emin)=e1;
00340     e2=E.max(); if (emax) (*emax)=e2;
00341     return (e2-e1);
00342 };
00343
00344 template <class TQ, class TD>
00345 bool getBoltzman(
00346     const QSpace<TQ,TD> &H,
00347     wbvector<double> &R, wbvector<unsigned> &D, unsigned iter,
00348     wbvector<unsigned> *DZ, char useE0
00349 ){
00350     wbvector<unsigned> dz;
00351     wbvector<double> E;
00352     bool e=0;

```

```

00353
00354     getEData(H,E,D,&dz); e=(E<0);
00355
00356     if (getBoltzman_base(E,dz,R,iter,0.,useE0)) { e=1; }
00357     if (DZ) dz.save2(*DZ);
00358
00359     return e;
00360 };
00361
00362 template <class TD>
00363 bool getBoltzman_base(
00364     const wbvector<TD>& E,
00365     const wbvector<unsigned>& dz,
00366     wbvector<TD>& R,
00367     unsigned iter,
00368     double T0,
00369     char useE0
00370 ){
00371     static double T=0.;
00372     static wbvector<double> E0;
00373
00374     unsigned i,e=0;
00375
00376     if (int(iter)<0) { T=T0; E0=R; return 0; }
00377
00378     if (iter>=E0.len) wblog(FL,
00379         "ERR %s() index out of bounds (%d/%d)",FCT,iter+1,E0.len);
00380     if (dz.len && dz.len!=E.len) wblog(FL,
00381         "ERR %s() size mismatch (%d/%d)",FCT,dz.len,E.len);
00382
00383     if (E.isEmpty()) { R.init(); return 0; }
00384
00385     double beta,r, Escal=gEScale(iter);
00386     double Eref=(useE0 ? E0[iter]/Escal : 0.);
00387
00388     if (E<-Eref) { ++e; wblog(FL,
00389         "WRN Eref should be lower bound (%.4g < %.4g) @ %.4g !?",
00390         E.min(), Eref, Escal);
00391     }
00392
00393     if (&R!=&E) R.init(E.len);
00394
00395     if (T>0.) {
00396         beta=Escal/T;
00397         for (i=0; i<E.len; ++i) {
00398             r=exp(-beta*(E[i]+Eref)); if (dz.len) r*=dz[i];
00399             R[i]=r;
00400         }
00401     }
00402     else {
00403         for (i=0; i<E.len; i++)
00404             R[i]=( Wb::abs(E[i]+Eref)<EPS ? (dz.len ? dz[i] : 1) : 0. );
00405
00406         i=unsigned(R.sum());
00407         if (i) wblog(FL," * T=0 ground state degeneracy %d", i);
00408         else wblog(FL," * T=0 got unique ground state");
00409     }
00410
00411     return e;
00412 };
00413
00414 template <class TQ, class TD>
00415 double initRho(
00416     QSpace<TQ,TD> &Rho,
00417     const QSpace<TQ,TD> &H,
00418     const int iter,
00419     double rhofac
00420 ){
00421     unsigned i,k,l,d,n, K=H.QIDX.dim1, has0=0;
00422     wbvector<unsigned> D,dz;
00423     wbvector<double> R;
00424     double Z, *r;
00425
00426     getBoltzman(H,R,D,iter,&dz,0);
00427
00428     Z=R.sum();
00429
00430     if (Z==0.) {
00431         MXPput(FL,"Ix").add(H,"H").add(R,"R").add(iter,"iter").add(dz,"dz");
00432         wblog(FL,"ERR rhoNorm[%d]=%g -> DIV/0 !?", iter+1,Z);
00433     }
00434     if (dz.len && R.len!=dz.len) wblog(FL,
00435         "ERR %s() length mismatch %d/%d",FCT,R.len,dz.len);
00436
00437     R *= (rhofac/Z); r=R.data;
00438
00439     if (dz.len) {

```

```

00440         for (i=0; i<R.len; ++i) { if (dz.data[i]>1) r[i]/=dz.data[i]; }
00441     }
00442
00443     Rho=H;
00444
00445     for (l=k=0; k<K; k++, l+=d) {
00446         wbararray<TD> &Rk = (*Rho.DATA[k]);
00447
00448         d=D[k]; Rk.init(d,d);
00449
00450         for (n=i=0; i<d; i++) {
00451             Rk.data[i*d+i] = r[l+i];
00452             if (!n && r[l+i]>DEPS) n++;
00453         }
00454         if (!n) has0++;
00455     }
00456
00457     if (has0) Rho.SkipZeroData(DEPS2,'b');
00458     return Z;
00459 };
00460
00461 template <class TQ, class TD>
00462 void updateRho(QSpace<TQ,TD> &Rho, const QSpace<TQ,TD> &AK) {
00463
00464     QSpace<TQ,TD> Xk;
00465
00466     if (!Rho.isBlockDiagMatrix()) wblog(FL,
00467         "WRN %s() Rho not blockdiag !?",FCT);
00468
00469     AK.contract(2,Rho,1,Xk);
00470     Xk.contract("3,2",AK,"2,3;*",Rho);
00471 };
00472
00473 template <class TQ, class TD>
00474 void nrgUpdateRHO(
00475     const wbMatrix<double> &R,
00476     NRGData<TQ,TD> &A,
00477     NRGData<TQ,TD> &H,
00478     NRGData<TQ,TD> &RHO,
00479     unsigned NRho,
00480     unsigned dloc,
00481     Wb::SigHandler *sig
00482 ){
00483     unsigned iter, ilast=0, rzero, N=NRG_N; char ie;
00484     const size_t n=16; size_t l=0; char tag[n];
00485     wbvector<double> w; double w2, wtot=0;
00486
00487     QSpace<TQ,TD> Rho;
00488
00489     RHO.K.clearQSpace();
00490
00491     if (R.dim1!=N || R.dim2!=2)
00492         wblog(FL, "ERR %s() size mismatch of rhoNorm (%dx%d; %d %d)",
00493             FCT, R.dim1, R.dim2, N, NRho);
00494
00495     for (iter=N-1; (int)iter>=0; iter--) {
00496         if (sig) { sig->check911(); }
00497
00498         R.getRec(iter,w); rzero=(w<DEPS);
00499         w2=w.sum();
00500
00501         l=snprintf(tag,n,
00502             "%s %2d/%d", !NRho ? "FDM":"DMK", iter,N);
00503         if (l>=n) wblog(FL,
00504             "ERR %s() string out of bounds (%s; %d/%d)",FCT,tag,l,n);
00505
00506         RHO.updatePara(FL,"K",iter);
00507
00508         if (rzero && RHO.K.isEmpty()) {
00509             wblog(FL,"%s skip ...%16s\r\\", tag,"");
00510             continue;
00511         }
00512
00513         A.init(FL,"K",iter);
00514         A.init(FL,"D",iter);
00515         H.init(FL,"D",iter);
00516
00517         A.check_dloc(FLF,dloc,iter);
00518
00519         if (!(ie=RHO.K.isEmpty())) {
00520             updateRho(RHO.K, A.K); }
00521
00522         if (!NRho) {
00523             if (H.D.isEmpty()) wblog(FL,
00524                 "%s no discarded space (%.3g) %16s\r\\",tag,wtot,"");
00525             else if (w2!=0) wblog(FL,
00526                 "%s dRho=%.3g (%.3g) %16s\r\\",tag,w2,wtot,"");

```



```

00527         else wblog(FL,
00528             "%s dRho=%.3g %16s\r\\",tag,wtot,"");
00529     }
00530     else wblog(FL,"%-30s\r\\",tag);
00531
00532     if (!rzero) {
00533         if (w[1]) {
00534             initRho(Rho, H.D, iter, w[1]);
00535             updateRho(Rho, A.D);
00536             RHO.K+=Rho; if (!ilast) { ilast=iter; }
00537         }
00538
00539         if (w[0] && (!ilast || ilast==iter)) { H.init(FL,"K",iter);
00540             initRho(Rho, H.K, iter, w[0]);
00541             updateRho(Rho, A.K);
00542             RHO.K+=Rho; if (!ilast) { ilast=iter; }
00543         }
00544
00545         RHO.K.SkipZeroData(DEPS2,'b');
00546     }
00547
00548     wtot+=w2;
00549
00550     if (!RHO.K.isEmpty()) { try {
00551         checkRho(FL, RHO.K, Wb::addRange(R.ref(iter,-1),N-iter,R.dim2));
00552     }
00553     catch (...) {
00554         wblog(FL,"ERR %s() iter=%d/%d%s",FCT,iter+1,N,
00555             iter+1==N ? "\nhint: got kept space at last iteration?":""");
00556     }}
00557     else {
00558         if (ie && rzero) wblog(FL,
00559             "%s empty RHO (will not contribute) %16s\r\\",tag,"");
00560         else wblog(FL,"WRN %2d/%d got empty RHO !? %16s",iter,N,"");
00561     }
00562     doflush();
00563 }
00564 };
00565
00566 template <class TQ, class TD, class TZ>
00567 void initG_SR(
00568     QSpace<TQ,TZ> &G2, const QSpace<TQ,TD> &H2,
00569     wcomplex z, unsigned iter
00570 ){
00571     unsigned k=0, K=H2.QIDX.dim1;
00572     size_t i,dk; TZ *gk;
00573
00574     double Escale=gEScale(iter);
00575
00576     G2.initT(H2);
00577
00578     for (; k<K; ++k) {
00579         gk=G2.DATA[k]->data;
00580         dk=G2.DATA[k]->dim1();
00581         for (i=0; i<dk; ++i) { gk[i]=1./(z-Escale*gk[i]); }
00582         G2.DATA[k]->ExpandDiagonal();
00583     }
00584 };
00585
00586 template <class TQ, class TD, class TX>
00587 void updateOp_SR(QSpace<TQ,TX> &X, const QSpace<TQ,TD> &AK) {
00588
00589     QSpace<TQ,TX> Xk;
00590
00591     if (!X.isBlockDiagMatrix()) wblog(FL,
00592         "ERR %s() input X not blockdiag !?",FCT);
00593     if (X.itags.len!=2) wblog(FL,
00594         "ERR %s() input X not a scalar op (%s) !?",FCT,IT2STR(X));
00595
00596     AK.contract(2,X,1,Xk);
00597     Xk.contract("32",AK,"23*",X);
00598 };
00599
00600 #endif
00601

```

5.4 gathered.cc

```

00001 /* -----
00002 * Project : QSpace tensor library (v4.0)
00003 * Class   : QSpace additional routines
00004 *
00005 * Copyright 2024 Andreas Weichselbaum

```

```

00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_GATHERED_CC__
00021 #define __WB_GATHERED_CC__
00022
00023 // auxilliary routine for mexlib.* -> Mx::Array
00024 // to deal with type conversions in a controlled way // Wb,May11,19
00025
00026 void setSigHandler(char w) {
00027
00028     static sighandler_t prev_SIGINT;
00029     static sighandler_t prev_SIGABRT;
00030
00031     static int iflag=0;
00032
00033     if (w=='i') {
00034         if (++iflag==1) {
00035             prev_SIGINT = signal(SIGINT, wbSigHandler);
00036             prev_SIGABRT = signal(SIGABRT, wbSigHandler);
00037         }
00038     }
00039     else if (w=='r') {
00040         if (--iflag==0) {
00041             signal(SIGINT, prev_SIGINT);
00042             signal(SIGABRT, prev_SIGABRT);
00043         }
00044         else if (iflag<0) { wblog(gsh_F, gsh_L,
00045             "ERR SigHandler not yet initialized (iflag=%d)",iflag);
00046         }
00047     }
00048     else wblog(gsh_F, gsh_L,"ERR invalid flag=%s",cSTR(w));
00049 };
00050
00051 void wbSigHandler(int sig) {
00052
00053     unsigned i=Wb::SigHandler::icount,
00054             a=Wb::SigHandler::acount, n=Wb::SigHandler::NC0;
00055     fflush(0);
00056
00057     if (sig==911) {
00058         if (i!=n || a!=n) {
00059             Wb::SigHandler::icount=Wb::SigHandler::acount=n;
00060             wblog(gsh_F, gsh_L,
00061                 "ERR terminating due to %s (%d/%d) -----",
00062                 i<a ? "SIGINT":"SIGABRT", n-MIN(i,a), n
00063             );
00064         }
00065         return;
00066     }
00067
00068     unsigned l=128; char istr[l], s[16];
00069     istr[0]=0; strcpy(s,"(signal)");
00070
00071     if (sig==SIGINT) {
00072         strcpy(s,"SIGINT"); --Wb::SigHandler::icount;
00073     }
00074     else if (sig==SIGABRT) {
00075         strcpy(s,"SIGABRT"); --Wb::SigHandler::acount;
00076         i=a;
00077     }
00078     else { printf("\n");
00079         snprintf(istr,l,"%s ERR terminating due to uncaught signal %d",
00080             shortFL(gsh_F,gsh_L), sig);
00081         ExitMsg(istr); fflush(0); exit(1);
00082     }
00083
00084     if (i<=0) {
00085         snprintf(istr,l,"%s ERR terminating due to %s (%d/%d) %s",
00086             shortFL(gsh_F,gsh_L),s,n,n,"-----");
00087         ExitMsg(istr);
00088         fflush(0); exit(-1);
00089     }
00090
00091     if (i>=3) {
00092         snprintf(istr,l,"--- received %s (%d) %s",

```

```

00093     s,i,"-----"); }
00094 else if (i==2) {
00095     snprintf(istr,l,"--- received %s (%s)",
00096     s,"waiting for natural termination pt"); }
00097 else {
00098     snprintf(istr,l,"WRN next %s will result in immediate stop %s",
00099     s,"-----");
00100 }
00101
00102 time_t curr_time=time(NULL);
00103 struct tm *tblock=localtime(&curr_time);
00104 char t[16]; strftime(t,16,"%T",tblock);
00105
00106 fprintf(stdout,"%-20s %s %s\n",shortFL(gsh_F,gsh_L), t, istr);
00107
00108 };
00109
00110 void dbstop(const char* F, int L) {
00111
00112     unsigned ith=omp_get_thread_num(), nth=omp_get_num_threads();
00113     char sx[8]; sx[0]=0;
00114     if (nth>1) { snprintf(sx,8," (%d/%d)",ith,nth); }
00115
00116     fprintf(stdout,"%s %s %s() raising SIGTRAP%s ...\n\n",
00117     shortFL(F_L),TIME,FCT,sx);
00118     fflush(0);
00119
00120     raise(SIGTRAP);
00121 };
00122
00123 template<class T> inline
00124 int num2int(const char *F, int L, const T &x){
00125     double d=x; int i=round(d);
00126     if (F) {
00127         double e=fabs(d-i); if (e>1E-12) {
00128             wblog(F,L,"ERR %s() got non-int value %g !?",FCT,d); }
00129     }
00130     return i;
00131 };
00132
00133 template<> inline
00134 int num2int(const char *F QS_UNUSED_VAR,
00135     int L QS_UNUSED_VAR, const char &x){ return x; }
00136
00137 template<> inline
00138 int num2int(const char *F QS_UNUSED_VAR,
00139     int L QS_UNUSED_VAR, const unsigned char &x){ return x; }
00140
00141 template<> inline
00142 int num2int(const char *F QS_UNUSED_VAR,
00143     int L QS_UNUSED_VAR, const int &x){ return x; }
00144
00145 template<> inline
00146 int num2int(const char *F QS_UNUSED_VAR,
00147     int L QS_UNUSED_VAR, const long &x){ return x; }
00148
00149 template<> inline
00150 int num2int(const char *F QS_UNUSED_VAR,
00151     int L QS_UNUSED_VAR, const unsigned &x){ return x; }
00152
00153 template<> inline
00154 int num2int(const char *F QS_UNUSED_VAR,
00155     int L QS_UNUSED_VAR, const unsigned long &x){ return x; }
00156
00157 template<class T> inline
00158 unsigned checkInt(const char *F, int L,
00159     const T* x, size_t n, double eps
00160 ){
00161     for (size_t i=0; i<n; ++i) {
00162         if (fabs(x[i]-round(x[i]))>eps) {
00163             if (F) wblog(F_L,
00164                 "ERR %s() got data[%d]=%g (having %s) !?",FCT,
00165                 i+1, double(x[i]), TSTR(T));
00166             return (i+1);
00167         }
00168     }
00169     return 0;
00170 };
00171
00172 template<> inline
00173 unsigned checkInt(
00174     const char *F QS_UNUSED_VAR,
00175     int L QS_UNUSED_VAR,
00176     const int* x QS_UNUSED_VAR,
00177     size_t n QS_UNUSED_VAR,
00178     double eps QS_UNUSED_VAR
00179 ){ return 0; }

```

```

00180
00181 template<> inline
00182 unsigned checkInt(
00183     const char *F      QS_UNUSED_VAR,
00184     int L              QS_UNUSED_VAR,
00185     const unsigned* x  QS_UNUSED_VAR,
00186     size_t n          QS_UNUSED_VAR,
00187     double eps         QS_UNUSED_VAR
00188 ){ return 0; }
00189
00190 template<> inline
00191 unsigned checkInt(
00192     const char *F      QS_UNUSED_VAR,
00193     int L              QS_UNUSED_VAR,
00194     const char* x      QS_UNUSED_VAR,
00195     size_t n          QS_UNUSED_VAR,
00196     double eps         QS_UNUSED_VAR
00197 ){ return 0; }
00198
00199 const char* Wb::basename(const char *data, char c) {
00200
00201     int i, n=strlen(data);
00202     for (i=n-1; i>=0; i--) if (data[i]==c) { i++; break; }
00203
00204     return (data + (i>0 ? i : 0));
00205 };
00206
00207 inline const char* Wb::strchri(const char *s, char c) {
00208
00209     if (!s) wblog(FL,"ERR %s() got invalid null string",FCT);
00210
00211     for (c=tolower(c); *s; ++s) {
00212         if (tolower(*s)==c) return s;
00213     }
00214
00215     return NULL;
00216 };
00217
00218 inline const char* Wb::rstrstr(const char *s1, const char *s2) {
00219     if (!s1 || !s2) wblog(FL,"ERR %s() got null strings (%p/%p)",FCT,s1,s2);
00220     if (!*s2) { return s1; }
00221     if ( ! *s1) { unsigned n=strlen(s1), m=strlen(s2);
00222         if (n>=m) { unsigned i,j,il=n-1;
00223             for (; il<n; --il) {
00224                 for (i=il, j=m-1; i<n && j<m && s1[i]==s2[j]; --i, --j) {};
00225                 if (j>m) { ++i; return s1+i; }
00226             }
00227         }
00228     }
00229     return NULL;
00230 };
00231
00232 inline const char* Wb::strstri(const char *s1, const char *s2) {
00233
00234     if (!s1 || !s2) wblog(FL,
00235         "ERR %s() got invalid null strings (%p, %p)",FCT,s1,s2);
00236
00237     const char *a, *b;
00238
00239     for(; *s1; ++s1) {
00240         for(a=s1, b=s2; *a && tolower(*a)==tolower(*b); ++a, ++b) {};
00241         if(!*b) return s1;
00242     }
00243
00244     return NULL;
00245 };
00246
00247 inline const char* Wb::strstrw(const char *s1, const char *s2) {
00248
00249     if (!s1 || !s2) wblog(FL,
00250         "ERR %s() got invalid null strings (%p, %p)",FCT,s1,s2);
00251
00252     unsigned i=0, j=0, k=0;
00253
00254     for(; s1[i]; ++i) {
00255         for (j=i, k=0; s1[j] && s1[j]==s2[k]; ++j, ++k) {};
00256         if (!s2[k]) {
00257             if ((!i || !Wb::isword(s1[i-1])) && !Wb::isword(s1[j]))
00258                 return s1+i;
00259         }
00260     }
00261
00262     return NULL;
00263 };
00264
00265 inline const char* Wb::strstrwi(const char *s1, const char *s2) {
00266

```

```

00267     if (!s1 || !s2) wblog(FL,
00268         "ERR %s() got invalid null strings (%p, %p)", FCT, s1, s2);
00269
00270     unsigned i=0, j=0, k=0;
00271
00272     for(; s1[i]; ++i) {
00273         for (j=i, k=0; s1[j] && tolower(s1[j])==tolower(s2[k]); ++j, ++k);
00274         if (!s2[k]) {
00275             if ((!i || !Wb::isword(s1[i-1])) && !Wb::isword(s1[j]))
00276                 return s1+i;
00277         }
00278     }
00279
00280     return NULL;
00281 };
00282
00283 void Wb::shift(char *s, unsigned n, int p, char erase) {
00284
00285     if (unsigned(abs(p))>=n) {
00286         if (erase && n) { unsigned i=0;
00287             if (p>0) { for (; i<n; ++i) { s[i]=' '; } }
00288             else {
00289                 for (; i<n && s[i]; ++i) { s[i]=' '; }
00290                 if (i<n) { s[0]=0; }
00291             }
00292         }
00293     }
00294     else if (p>0) {
00295         unsigned i=n-p-1; for (; i<n; --i) { s[i+p]=s[i]; }
00296         if (erase) { for (i=0; int(i)<p; ++i) { s[i]=' '; } }
00297     }
00298     else if (p<0) {
00299         unsigned i=-p; for (; i<n && s[i]; ++i) { s[i+p]=s[i]; }
00300         if (erase) {
00301             if (n>i) { n=i; }
00302             for (i=i+p+1; i<n && s[i]; ++i) { s[i]=' '; }
00303         }
00304     }
00305 };
00306
00307 void Wb::shift(char *s, unsigned n, int p, const char *s_) {
00308
00309     if (!p) return;
00310     if (!s || !s_) wblog(FL, "ERR %s() invalid input %p / %p", FCT, s, s_);
00311
00312     if (unsigned(abs(p))>=n) { if (n) {
00313         if (p>0) { strncpy(s, s_, n); }
00314         else {
00315             unsigned i=0; for (; i<n && s[i]; ++i) { }
00316             if (i<n) { s[0]=0; }
00317             else { strncpy(s, s_, n); }
00318         }
00319     } }
00320     else if (p>0) {
00321         unsigned i=n-p-1; for (; i<n; --i) { s[i+p]=s[i]; }
00322         for (i=0; int(i)<p; ++i) { s[i]=s_[i]; if (!s[i]) break; }
00323     }
00324     else if (p<0) { unsigned i=-p;
00325         for (; i<n && s[i]; ++i) { s[i+p]=s[i]; }
00326         for (i=n+p; i<n; ++i) { s[i]=s_[i]; if (!s[i]) break; }
00327     }
00328 };
00329
00330 template <class T>
00331 int Wb::charGetNumber(const char *F, int L, const char *s, T &x) {
00332
00333     float fl=0;
00334     int i;
00335
00336     if (!s) {
00337         sprintf_str("%s got null string", shortFL(F_L));
00338         return 1;
00339     }
00340
00341     i=sscanf (s, "%f", &fl);
00342     x=(T)fl;
00343
00344     if (!i || fl!=(float)x) {
00345         sprintf_str("invalid input for type '%s' (%g, %d)", TSTR(T), fl, i);
00346         return 1;
00347     }
00348
00349     return 0;
00350 }
00351
00352 template<>
00353 int Wb::GetEnv(const char *F, int L, const char *name, double &val) {

```

```

00354
00355     char *s, *s2;
00356     double dbl;
00357
00358     s=getenv(name);    if (!s || !s[0]) return -1;
00359     dbl=strtod(s,&s2); if (!s2) return -2;
00360     val=dbl;
00361
00362     if (F) wblog1(F,L," *   getenv %-6s = %g", name, val);
00363
00364     if (*s2) {
00365         wblog1(FL,"WRN %s() got invalid %s='%s' !?",FCT,name,s);
00366         if (isspace(*s2)) {
00367             ++s2; while (*s2 && isspace(*s2)) ++s2;
00368         }
00369     }
00370
00371     return (*s2 ? -3 : 0);
00372 };
00373
00374 template<>
00375 int Wb::GetEnv(const char *F, int L, const char *name, int &val) {
00376
00377     double dbl=0;
00378
00379     int i=GetEnv(F,L,name,dbl); if (i) return i;
00380
00381     i=int(dbl); if (double(i)!=dbl) return -2;
00382     val=i;
00383
00384     return 0;
00385 }
00386
00387 template<>
00388 int Wb::GetEnv(const char *F, int L, const char *name, unsigned &val) {
00389
00390     double dbl=0;
00391     int i=GetEnv(F,L,name,dbl); if (i) return i;
00392
00393     i=unsigned(dbl); if (double(i)!=dbl) return -2;
00394     val=i;
00395
00396     return 0;
00397 };
00398
00399 template<>
00400 int Wb::GetEnv(const char *F, int L, const char *name, char &val) {
00401
00402     char *s=getenv(name); if (!s || !s[0]) return -1;
00403     if (s[1]) { return -2; }
00404     val=s[0];
00405
00406     if (F) wblog1(F,L," *   getenv %-6s = '%c'(%d)", name, val,val);
00407     return 0;
00408 };
00409
00410 template<>
00411 int Wb::GetEnv(const char *F, int L, const char *name, wstring &val) {
00412
00413     if (!name || !name[0]) wblog1(F_L,
00414         "ERR %s() invalid env '%s' !?",FCT,name);
00415
00416     char *s=getenv(name);
00417
00418     if (!s) {
00419         val.init(); return -1;
00420     }
00421     else {
00422         val=s;
00423         if (F) wblog1(F,L," *   getenv %-6s = '%s'",name,val.data());
00424         return 0;
00425     }
00426 };
00427
00428 int Wb::EnvIsSet(const char *F, int L, const char *name) {
00429     int x=0, e=GetEnv(F,L,name,x);
00430     return (!e && x>0 ? 1 : 0);
00431 };
00432
00433 int Wb::get_WB_VERBOSE(const char *F, int L) {
00434
00435     const char *vname="WB_VERBOSE";
00436     int i, k=0xF1;
00437
00438     if ((i=GetEnv(0,0,vname,k))==0) {
00439         if (k<0) wblog1(F_L,
00440             "WRN %s() invalid %s=%d/255",FCT,vname,k);

```

```

00441     else if (k!=Wb::envVRB) {
00442         if ((F && k) || k&8) { unsigned n=32; char s[n];
00443             i=sprintf(s,n,"%s()",myname);
00444             if (i<0) wblog(F_L,"WRN %s() i=%d",FCT,i);
00445             wblog1(PF_L,"ENV %-14s %s = %X->%X",s,vname,Wb::envVRB,k);
00446         }
00447         Wb::envVRB=k;
00448     }
00449 }
00450 else if (int(i)!=-1) {
00451     wblog1(F_L,"WRN %s() invalid %s (e=%d)",FCT,vname,i);
00452 }
00453
00454 { int q=0; i=GetEnv(0,0,"QS_LOG_COLOR",q);
00455     if (i) { q=(Wb::envDKT>1 ? WLC_DARK : WLC_OFF); } else
00456         if (q>NUM_WBL_COLOR_SCHEME) { q=WLC_DARK; } else
00457         if (q<0) { q=WLC_OFF; }
00458     Wb::useCol=(WBL_COLOR_SCHEME)q;
00459 }
00460
00461 return k;
00462 };
00463
00464 int Wb::got_DBSTOP(const char *F, int L) {
00465     int i=0,x=0,e,q;
00466     const char *ss[]={ "DBSTOP", "ML_DEBUG", "DEBUG" };
00467
00468     for (; i<3; ++i) { q=0;
00469         e=GetEnv(0,0,ss[i],q);
00470         if (!e) { if (q) { x|=(1<<i); } } else
00471             if (e!=-1 && F) wblog1(F_L,
00472                 "WRN %s() invalid value for ENV %s (e=%d)",FCT,ss[i],e);
00473     }
00474
00475     if (x!=Wb::envDBG) {
00476         if ((F && x && Wb::envVRB&15) ||
00477             (Wb::envDBG && Wb::envVRB&8)) wblog1(PF_L," * "
00478                 "using %-20s = %d -> %d", "Wb::envDBG", Wb::envDBG, x);
00479         Wb::envDBG=x;
00480     }
00481 }
00482
00483 return x;
00484 };
00485
00486 #ifdef MATLAB_MEX_FILE
00487 int Wb::got_DESKTOP() {
00488     int q=0;
00489     if (Wb::CallMatlab(0,0,"isdeployed")) { q=0; }
00490     else if (Wb::CallMatlab(0,0,"usejava","desktop")) { q=1; }
00491     else if (Wb::CallMatlab(0,0,"usejava","awt")) { q=2; }
00492
00493     if ((Wb::envDKT>=0 && Wb::envDKT!=q) || (Wb::envVRB&8)) { wblog1(F_L,
00494         "ENV %s() Wb::envDKT = %d (%s)",myname, q,
00495         q>1? "terminal":(q? "desktop":"batch"));
00496     }
00497     Wb::envDKT=q;
00498
00499     return q;
00500 }
00501 };
00502
00503 int Wb::is_DEPLOYED(const char *F, int L) {
00504     return (int)Wb::CallMatlab(F,L,"isdeployed");
00505 };
00506
00507 #else
00508 int Wb::got_DESKTOP() { return 0; }
00509 int Wb::is_DEPLOYED(
00510     const char *F QS_UNUSED_VAR,
00511     int L QS_UNUSED_VAR) { return 1; }
00512 #endif
00513
00514 int Wb::GetNumThreads(const char *F, int L, int &n, const char *name) {
00515     #ifndef QS_USING_OMP
00516         return 0;
00517     #endif
00518
00519     double q=0;
00520     int i=GetEnv(0,0,name,q), ncpu=wbsys::getNumCores();
00521
00522     if (i<0) {
00523         if (i<-1 && F) wblog1(F_L,
00524             "ERR %s() invalid value for env %s (i=%d)",FCT,name,i);
00525         return q;
00526     }
00527 }

```

```

00528     if (q<0 || double(int(q))!=q) wblog1(PF_L,
00529     "ERR %s() invalid value %g\nfor env %s",FCT,q,name);
00530     if (q>ncpu) wblog1(PF_L,"ERR %s() value exceeds number of cores\n"
00531     "env %s = %g / %d",FCT,name,q,ncpu);
00532
00533     if (n!=q) { i=0;
00534
00535         if (!GetEnv(0,0,"ML_DEBUG",i) && i<1) { q=0; } else
00536         if (!GetEnv(0,0,"DEBUG", i) && i<1) { q=0; } else
00537         if (F && (Wb::envVRB &12)
00538             #ifdef LD_CLEBSCH_QS
00539                 || CG_VERBOSE>6
00540             #endif
00541         )) wblog1(PF_L," * using %s-20s = %d -> %g",name,n,q);
00542         n=q;
00543     }
00544
00545     return q;
00546 };
00547
00548 int Wb::strrep(
00549     const char *S0,
00550     const char *t0,
00551     const char *t2,
00552     char *Sout,
00553     size_t N,
00554     char gflag
00555 ){
00556     if (!S0 || !S0[0] || !t0 || !t0[0]) return 0;
00557
00558     int nrep=0;
00559     size_t k, l=0, m0=strlen(t0), m=(t2 ? strlen(t2):0);
00560     const char *s, *s0=S0; char *s2=Sout;
00561
00562     while ((s=strstr(s0,t0))) { k=s-s0; ++nrep;
00563         if (k) {
00564             if (l+k>N) wblog(F,
00565             "ERR string out of bounds (%d+%d / %d)",l,k,N);
00566             memcpy(s2,s0,k); l+=k; s2+=k;
00567         }
00568         if (m) {
00569             if (l+m>N) wblog(F,
00570             "ERR string out of bounds (%d+%d / %d)",l,m,N);
00571             memcpy(s2,t2,m); l+=m; s2+=m;
00572         }
00573         s0+=(k+m0); if (!gflag) break;
00574     }
00575
00576     if ((k=strlen(s0))) {
00577         if (l+k>N) wblog(F,
00578         "ERR string out of bounds (%d+%d / %d)",l,k,N);
00579         memcpy(s2,s0,k); l+=k; s2+=k;
00580     }
00581     s2[0]=0;
00582
00583     return nrep;
00584 };
00585
00586 wstring Wb::repHome(const char *file) {
00587
00588     size_t n=2*strlen(file);
00589     const char *s; char F0[n+1], F[n+1];
00590     char *f0=F0, *f=F; strcpy(f,file);
00591
00592     if ((s=getenv("MEX" ))) {
00593         SWAP(f,f0); if (Wb::strrep(f0,s,"$MEX", f,n)) return f; }
00594     if ((s=getenv("HOME" ))) {
00595         SWAP(f,f0); if (Wb::strrep(f0,s,"$HOME", f,n)) return f; }
00596     if ((s=getenv("LMA" ))) {
00597         SWAP(f,f0); if (Wb::strrep(f0,s,"$LMA", f,n)) return f; }
00598     if ((s=getenv("PROJ" ))) {
00599         SWAP(f,f0); if (Wb::strrep(f0,s,"$PROJ", f,n)) return f; }
00600     if ((s=getenv("PROJ_LEO" ))) {
00601         SWAP(f,f0); if (Wb::strrep(f0,s,"$PROJ_LEO", f,n)) return f; }
00602     if ((s=getenv("USER" ))) {
00603         SWAP(f,f0); if (Wb::strrep(f0,s,"$USER", f,n)) return f; }
00604
00605     return f;
00606 };
00607
00608 void Wb::print_backtrace(const char *F, int L, const char *istr) {
00609
00610     unsigned i=0, n=4;
00611     GetEnv(0,0,"QS_NUM_BTRACE",i); if (n<i) { n=i; }
00612
00613     void *array[n];
00614

```



```

00615     n=backtrace(array,n);
00616
00617     char **ss=backtrace_symbols(array,n);
00618
00619     printf("\n# %s: stack backtrace",shortFL(F_L));
00620     if (istr && istr[0]) { printf(" %s",istr); }
00621     printf(" (%d frame%s)\n",n,n!=1? "s":"" );
00622     printf("# hint: may use addr2line to decode this\n\n");
00623
00624     if (ss) {
00625         for (i=0; i<n; ++i) {
00626             printf(" [%*d] %s\n", n>10? 2:1, i,ss[i]);
00627         }; printf("\n");
00628         free(ss);
00629     }
00630 };
00631
00632 std::string getName(
00633     const std::type_info &t,
00634     char v
00635 ){
00636
00637     std::string s;
00638
00639     if (t==typeid(float)) s=(v? "float" : "f" ); else
00640     if (t==typeid(double)) s=(v? "double" : "d" ); else
00641     if (t==typeid(bool)) s=(v? "bool" : "b" ); else
00642     if (t==typeid(char)) s=(v? "char" : "c" ); else
00643     if (t==typeid(unsigned char)) s=(v? "uchar" : "uc" ); else
00644     if (t==typeid(int)) s=(v? "int" : "i" ); else
00645     if (t==typeid(unsigned)) s=(v? "unsigned" : "u" ); else
00646     if (t==typeid(long)) s=(v? "long" : "l" ); else
00647     if (t==typeid(size_t)) s=(v? "size_t" : "st" ); else
00648     if (t==typeid(unsigned long long)) s=(v? "uLong" : "uL" ); else
00649
00650     if (t==typeid(double*)) s=(v? "double*" : "d*" ); else
00651     if (t==typeid(float*)) s=(v? "float*" : "f*" ); else
00652     if (t==typeid(bool*)) s=(v? "bool*" : "b*" ); else
00653     if (t==typeid(char*)) s=(v? "char*" : "c*" ); else
00654     if (t==typeid(int*)) s=(v? "int*" : "i*" ); else
00655     if (t==typeid(unsigned*)) s=(v? "unsigned*" : "u*" ); else
00656     if (t==typeid(long*)) s=(v? "long*" : "l*" ); else
00657     if (t==typeid(size_t*)) s=(v? "size_t*" : "st*" ); else
00658     if (t==typeid(unsigned long long*)) s=(v? "uLong*" : "uL*" ); else
00659
00660     if (t==typeid(const double*)) s=(v? "c-double*" : "Kd*" ); else
00661     if (t==typeid(const float*)) s=(v? "c-float*" : "Kf*" ); else
00662     if (t==typeid(const bool*)) s=(v? "c-bool*" : "Kb*" ); else
00663     if (t==typeid(const char*)) s=(v? "c-char*" : "Kc*" ); else
00664     if (t==typeid(const int*)) s=(v? "c-int*" : "Ki*" ); else
00665     if (t==typeid(const unsigned*)) s=(v? "c-unsigned*" : "Ku*" ); else
00666     if (t==typeid(const long*)) s=(v? "c-long*" : "Kl*" ); else
00667     if (t==typeid(const size_t*)) s=(v? "c-size_t*" : "Kst*" ); else
00668
00669     if (t==typeid(wbcomplex)) s=(v? "wbcomplex" : "z" ); else
00670     if (t==typeid(wbcomplex*)) s=(v? "wbcomplex*" : "z*" ); else
00671
00672     if (t==typeid(long double)) s=(v? "long double" : "dl" ); else
00673     if (t==typeid(long double*)) s=(v? "long double*" : "dl*" ); else
00674
00675     #ifdef __WB_MPFH_HH__
00676     if (t==typeid(Wb::quad)) s=(v? "Wb::quad" : "q" ); else
00677     if (t==typeid(Wb::qqquad)) s=(v? "Wb::qqquad" : "qq" ); else
00678     if (t==typeid(Wb::quad2)) s=(v? "Wb::quad2" : "q2" ); else
00679     if (t==typeid(Wb::quad3)) s=(v? "Wb::quad3" : "q2" ); else
00680
00681     if (t==typeid(Wb::quad*)) s=(v? "Wb::quad" : "q*" ); else
00682     if (t==typeid(Wb::qqquad*)) s=(v? "Wb::qqquad" : "qq*" ); else
00683     if (t==typeid(Wb::quad2*)) s=(v? "Wb::quad2" : "q2*" ); else
00684     if (t==typeid(Wb::quad3*)) s=(v? "Wb::quad3" : "q2*" ); else
00685     #endif
00686
00687     s=(char*)(t.name());
00688
00689     return s;
00690 };
00691
00692 bool Wb::isBaseType(const std::type_info &t) {
00693
00694     if ( t==typeid(unsigned)
00695         || t==typeid(double)
00696         || t==typeid(float)
00697         || t==typeid(int)
00698         || t==typeid(char)
00699         || t==typeid(unsigned char)
00700         || t==typeid(long long)
00701         || t==typeid(size_t)

```

```

00702     || t==typeid(wbcomplex)
00703 ) return 1;
00704
00705 return 0;
00706 };
00707
00708 bool isComplexType(const std::type_info &t) {
00709
00710     if ( t==typeid(wbcomplex) ) return 1;
00711
00712     return 0;
00713 };
00714
00715 template<class T>
00716 Wb::num2Fmt<T>::operator wbstring() const {
00717     wbstring s;
00718     get_fmt(s); return s;
00719 };
00720
00721 template<class T>
00722 void Wb::num2Fmt<T>::get_fmt(wbstring &s) const {
00723
00724     unsigned l=0; s.init(12);
00725
00726     if (m<-1 || p<-1 || t[0]<=0) wblog(FL,
00727         "ERR %s<%s> invalid type %d.%d%s",FCT,TSTR(T),m,p,t);
00728
00729     if (m>0) {
00730         if (p>=0)
00731             { l=snprintf(s.data,s.len,"%d.%d%s",m,p,t); }
00732         else { l=snprintf(s.data,s.len,"%d.%d%s",m,t); }
00733     }
00734     else {
00735         if (p>=0)
00736             { l=snprintf(s.data,s.len,"%d.%d%s",p,t); }
00737         else { l=snprintf(s.data,s.len,"%d.%d%s",t); }
00738     }
00739
00740     if (l>=s.len) wblog(FL,
00741         "ERR %s<%s> string out of bounds (%d.%d%s; %d/%d)",
00742         FCT,TSTR(T),m,p,t,l,s.len
00743     );
00744 };
00745
00746 template<>
00747 void Wb::num2Fmt<wbcomplex>::get_fmt(wbstring &s) const {
00748
00749     unsigned l=0; s.init(16);
00750
00751     if (m<-1 || p<-1 || t[0]<=0) wblog(FL,
00752         "ERR %s<%s> invalid type %d.%d%s",FCT,TSTR(wbcomplex),m,p,t);
00753
00754     for (int i=0; i<2; ++i) {
00755         l+=snprintf(s.data+l,s.len-l,"%d.%d%s",i?"+"+"");
00756         if (m>0) {
00757             if (p>=0)
00758                 { l+=snprintf(s.data+l,s.len-l,"%d.%d%s",m,p,t); }
00759             else { l+=snprintf(s.data+l,s.len-l,"%d.%d%s",m,t); }
00760         }
00761         else {
00762             if (p>=0)
00763                 { l+=snprintf(s.data+l,s.len-l,"%d.%d%s",p,t); }
00764             else { l+=snprintf(s.data+l,s.len-l,"%d.%d%s",t); }
00765         }
00766     }
00767
00768     if (l+2>=s.len) wblog(FL,
00769         "ERR %s<%s> string out of bounds (%d.%d%s; %d/%d)",
00770         FCT,TSTR(wbcomplex),m,p,t,l,s.len
00771     );
00772
00773     snprintf(s.data+l,s.len-l,"i");
00774 };
00775
00776 template<class T>
00777 int Wb::num2Fmt<T>::check_init() {
00778     wblog(FL,"ERR %s<> invalid data type '%s'",FCT,TSTR(T));
00779     return 1;
00780 };
00781
00782 template<> inline
00783 int Wb::num2Fmt<double>::check_init() {
00784     if (m<-1) m=8;
00785     if (p<-1) p=5;
00786     if (t[0])
00787         { if (!Wb::strchr("ge",t[0])) return 't'; }
00788     else { strcpy(t,"g"); }

```

```

00789     return 0;
00790 };
00791 template<> inline
00792 int Wb::num2Fmt<float>::check_init() {
00793     return ((num2Fmt<double>*)this)->check_init();
00794 };
00795 template<> inline
00796 int Wb::num2Fmt<wbcomplex>::check_init() {
00797     return ((num2Fmt<double>*)this)->check_init();
00798 };
00799
00800 template<> inline
00801 int Wb::num2Fmt<int>::check_init() {
00802     if (m<-1) m=4;
00803     if (p>=0) return 'p';
00804     if (t[0])
00805         { if (!strchr("ducxX",t[0])) return 't'; }
00806     else { strcpy(t,"d"); }
00807     return 0;
00808 };
00809 template<> inline
00810 int Wb::num2Fmt<unsigned>::check_init() {
00811     return ((num2Fmt<int>*)this)->check_init();
00812 };
00813
00814 template<> inline
00815 int Wb::num2Fmt<long>::check_init() {
00816     if (m<-1) m=4;
00817     if (p>=0) return 'p';
00818     if (t[0])
00819         { if (!strchr("lducxX",t[0])) return 't'; }
00820     else { strcpy(t,"ld"); }
00821     return 0;
00822 };
00823
00824 template<> inline
00825 int Wb::num2Fmt<unsigned long>::check_init() {
00826     return ((num2Fmt<long>*)this)->check_init();
00827 };
00828
00829 template<> inline
00830 int Wb::num2Fmt<char>::check_init() {
00831     if (m<-1) m=4;
00832     if (p>=0) return 'p';
00833     if (t[0])
00834         { if (!strchr("ducxX",t[0])) return 't'; }
00835     else { strcpy(t,"d"); }
00836     return 0;
00837 };
00838
00839 template<> inline
00840 int Wb::num2Fmt<char*>::check_init() {
00841     if (m<-1) m=-1;
00842     if (p>=0) return 'p';
00843     if (t[0])
00844         { if (t[0]!='s') return 't'; }
00845     else { strcpy(t,"s"); }
00846     return 0;
00847 };
00848
00849 template <class T> inline
00850 char* defaultFmt(char *fmt,
00851 const T &x QS_UNUSED_VAR, int m, int p, char t) {
00852
00853     const unsigned n=16;
00854     unsigned l=0, flen=n;
00855
00856     char dd[n]; dd[0]=0; fmt[0]=0;
00857
00858     if (m>0) {
00859         if (p>=0)
00860             l=snprintf(dd,n,"%d.%d",m,p);
00861         else l=snprintf(dd,n,"%d",m);
00862     }
00863     else if (p>=0) l=snprintf(dd,n,"%d",p);
00864
00865     if (l>=n) wblog(FL,
00866 "ERR %s() string out of bounds (%d/%d)",FCT,l,n);
00867
00868     if (typeid(T)==typeid(double) || typeid(T)==typeid(float)) {
00869         if (!t) { t='g'; }
00870         l=snprintf(fmt,flen,"%s%c",dd,t);
00871     }
00872     else if (typeid(T)==typeid(wbcomplex)) {
00873         if (!t) { t='g'; }
00874         l=snprintf(fmt,flen+n,"%s%c%+%s%ci",dd,t,dd,t);
00875     }

```

```

00876     else if (
00877         typeid(T)==typeid(int) || typeid(T)==typeid(char) ||
00878         typeid(T)==typeid(unsigned)
00879     ){
00880         if (p>=0) wblog(FL,
00881             "WRN %s() got format *.%d* for type '%s'",FCT,p,TSTR(T));
00882         if (!t) { t='d'; }
00883         l=snprintf(fmt,flen,"%%%sc",dd,t);
00884     }
00885     else if (typeid(T)==typeid(char*)) {
00886         if (t && t!='s') wblog(FL,"ERR %s() "
00887             "got format type '%c'<%d> for type '%s'",FCT,t,t,TSTR(T));
00888         l=snprintf(fmt,flen,"%%%ss",dd);
00889     }
00890     else wblog(FL,
00891         "ERR %s() unsupported type '%s'",FCT,TSTR(T));
00892
00893     if (l>=flen) wblog(FL,
00894         "WRN %s() string out of bounds ('%s', l=%d/%d)",FCT,fmt,l,flen);
00895
00896     return fmt;
00897 };
00898
00899 wbststring Wb::TimeStamp(char type) {
00900
00901     wbststring s;
00902     unsigned l,n;
00903
00904     #pragma omp critical (got_CPTR_TIME)
00905     { time_t t=time(NULL);
00906       struct tm *tb=localtime(&t);
00907       if (type=='t' || type=='T') {
00908           s.init(32); l=strftime(s.data,s.len,"%T",tb);
00909
00910           if (type=='T') {
00911               using namespace std::chrono;
00912               time_point<system_clock> t=system_clock::now();
00913
00914               l+=snprintf(s.data+l,s.len-l,"%03ld",
00915                   size_t(duration_cast<milliseconds>(t.time_since_epoch()).count())%1000);
00916           }
00917       }
00918       else if (type=='D') {
00919           s.init(32); l=strftime(s.data,s.len,"%d-%b-%Y %Z",tb);
00920       }
00921       else { s=asctime(tb); }
00922     }
00923
00924     n=strlen(s.data);
00925     if (n && s.data[n-1]<32) s.data[n-1]=0;
00926
00927     return s;
00928 }
00929
00930 template <class T1, class T2> inline
00931 void Wb::safeConvert(const char *F, int L, const T1 &x1, T2 &x2) {
00932     x2=T2(x1); if (T1(x2)!=x1) wblog(F_L,
00933         "ERR converting %s -> %s changes value\n%g, %g",
00934         TSTR(T1), TSTR(T2),
00935         double(x1), double(x2)
00936     );
00937 }
00938
00939 template <> inline
00940 void Wb::safeConvert(const char *F, int L, const wbcplx &x1, double &x2) {
00941     if (x1.i!=0.) wblog(F_L,
00942         "ERR cannot cast complex number to real (%.4g%.4gi)",x1.r,x1.i);
00943     x2=x1.r;
00944 }
00945
00946 template <> inline
00947 void Wb::safeConvert(
00948     const char *F QS_UNUSED_VAR, int L QS_UNUSED_VAR,
00949     const double &x1, wbcplx &x2
00950 ){ x2=wbcplx(x1,0); }
00951
00952 template <class T> inline
00953 bool Wb::isLower(const T* a, const T* b, const size_t n) {
00954     for (size_t i=0; i<n; i++) {
00955         if (a[i]<b[i]) return 1; else
00956         if (a[i]>b[i]) return 0;
00957     }
00958     return 0;
00959 }
00960
00961 template <class T> inline
00962 bool Wb::isEqual(const T* a, const T* b, const size_t n) {

```

```

00963
00964     for (size_t i=0; i<n; i++) { if (a[i]!=b[i]) return 0; }
00965     return 1;
00966 }
00967
00968 template <class T> inline
00969 bool Wb::allEqual(const T* a, const size_t n, const T &x) {
00970
00971     for (size_t i=0; i<n; i++) { if (a[i]!=x) return 0; }
00972     return 1;
00973 }
00974
00975 template <class T> inline
00976 bool Wb::anyUnequal(const T* a, const size_t n, const T &x) {
00977     for (size_t i=0; i<n; i++) { if (a[i]!=x) return 1; }
00978     return 0;
00979 };
00980
00981 template <class T> inline
00982 bool Wb::anyEqual(const T* a, const size_t n, const T &x) {
00983
00984     for (size_t i=0; i<n; i++) { if (a[i]==x) return 1; }
00985     return 0;
00986 };
00987
00988 template <class T> inline
00989 void Wb::scale_eps(T &eps, const T* d, size_t n) {
00990     if (double(eps)>0) {
00991         if (double(eps)>1E-8) wblog(FL,
00992             "WRN %s() got eps=%g (ignore)",FCT,double(eps));
00993         else {
00994             T x=getdscale(d,n);
00995             if (x>T(1E-8)) eps*=x;
00996         }
00997     }
00998 };
00999
01000 template <class T>
01001 T Wb::getdscale(const T* data, size_t n) {
01002
01003     if (n) { T d,x=0;
01004         for (size_t i=0; i<n; ++i) { d=Wb::abs(data[i]); if (x<d) x=d; }
01005         return x;
01006     }
01007     return 0;
01008 };
01009
01010 inline
01011 void Wb::update_range_stride(size_t &n, const size_t stride) {
01012     if (stride!=1) {
01013         if (int(stride)<=0 && n) wblog(FL,
01014             "ERR %s() got stride=%ld (n=%ld) !?",FCT,stride,n);
01015         n*=stride;
01016     }
01017 };
01018
01019 template <class T> inline
01020 T Wb::addRange2(const T* d, size_t n, const size_t stride) {
01021     T s=0; Wb::update_range_stride(n,stride);
01022     for (size_t i=0; i<n; i+=stride) { s+=Wb::abs2(d[i]); }
01023     return s;
01024 };
01025
01026 template <class T> inline
01027 T Wb::addRange(const T* d, size_t n, const size_t stride) {
01028     T s=0; Wb::update_range_stride(n,stride);
01029     for (size_t i=0; i<n; i+=stride) { s+=d[i]; }
01030     return s;
01031 };
01032
01033 template <class T> inline
01034 void Wb::addRange(const T* a, T* c, size_t n, const size_t stride) {
01035     Wb::update_range_stride(n,stride);
01036     for (size_t i=0; i<n; i+=stride) { c[i]+=a[i]; }
01037 };
01038
01039 template <class T> inline
01040 void Wb::addRange(
01041     const T* a, const T* b, T* c, size_t n, size_t stride
01042 ){
01043     Wb::update_range_stride(n,stride);
01044     for (size_t i=0; i<n; i+=stride) { c[i]=a[i]+b[i]; }
01045 };
01046
01047 template <class T> inline
01048 void Wb::minusRange(
01049     const T* a, const T* b, T* c, size_t n, size_t stride

```

```

01050 ){
01051     Wb::update_range_stride(n, stride);
01052     for (size_t i=0; i<n; i+=stride) { c[i]=a[i]-b[i]; }
01053 };
01054
01055 template <class T> inline
01056 void Wb::diffRange(
01057     const T* a, const T* b, T* c, size_t n, size_t stride
01058 ){
01059     Wb::update_range_stride(n, stride);
01060     for (size_t i=0; i<n; i+=stride) { c[i]=a[i]-b[i]; }
01061 };
01062
01063 template <class T> inline
01064 void Wb::setRange2avg(T* a, size_t n) {
01065     T x=0; size_t i;
01066     for (i=0; i<n; ++i) { x+=a[i]; }; x*=(T(1)/n);
01067     for (i=0; i<n; ++i) { a[i]=x; }
01068 };
01069
01070 template <class T> inline
01071 size_t Wb::nnzRange(const T* a, size_t n) {
01072     size_t m=0, i=0;
01073     for (; i<n; ++i) { if (a[i]!=0) m++; }
01074     return m;
01075 };
01076
01077 template <class T> inline
01078 T Wb::rangeNormDiff2(
01079     const T* d1, const T* d2, size_t n
01080 ){
01081     if (!n || d1==d2) return 0;
01082
01083     T x2, x2sum=0;
01084
01085     for (size_t i=0; i<n; ++i) {
01086         x2=d1[i]-d2[i]; x2*=Wb::CONJ(x2);
01087         x2sum+=x2;
01088     }
01089
01090     return x2sum;
01091 };
01092
01093 template <class T> inline
01094 T Wb::rangeNormDiff2(
01095     const T* d1, const T* d2, size_t n, const T &fac, size_t *k
01096 ){
01097     if (fac==1 && !k) { return rangeNormDiff2(d1, d2, n); }
01098     if (!n || (d1==d2 && fac==1)) { if (k) { (*k)=0; }; return 0; }
01099
01100     T x2, x2sum=0;
01101
01102     if (k) { T x2max=0;
01103         for (size_t i=0; i<n; ++i) {
01104             x2=d1[i]-fac*d2[i]; x2*=Wb::CONJ(x2);
01105             x2sum+=x2; if (x2>x2max) { x2max=x2; (*k)=i; }
01106         }
01107     }
01108     else {
01109         for (size_t i=0; i<n; ++i) {
01110             x2=d1[i]-fac*d2[i]; x2*=Wb::CONJ(x2);
01111             x2sum+=x2;
01112         }
01113     }
01114
01115     return x2sum;
01116 };
01117
01118 template <class T> inline
01119 T Wb::rangeNorm2(const T* d, size_t n, size_t *k
01120 ){
01121     T x2sum=0;
01122     if (!n) { if (k) { (*k)=0; }; return x2sum; }
01123
01124     if (k) {
01125         T x2, x2max=0; (*k)=0;
01126
01127         for (size_t i=0; i<n; ++i) {
01128             x2=Wb::CONJ(d[i])*d[i]; if (x2>x2max) { x2max=x2; (*k)=i; }
01129             x2sum+=x2;
01130         }
01131     }
01132     else {
01133         for (size_t i=0; i<n; ++i) { x2sum+=Wb::CONJ(d[i])*d[i]; }
01134     }
01135
01136     return x2sum;

```

```

01137 };
01138
01139 template <class T> inline
01140 T Wb::rangeMaxDiff(const T* d1, const T* d2, size_t n, size_t *k) {
01141     size_t imax=0, i=0; T x,xmax=0;
01142
01143     if (!n || d1==d2) { if (k) (*k)=0;
01144         if (!n) wblog(FL,"WRN %s() got empty input",FCT);
01145         return 0;
01146     }
01147
01148     for (; i<n; i++) {
01149         x=Wb::abs(d1[i]-d2[i]); if (x>xmax) { xmax=x; imax=i; }
01150     }
01151
01152     if (k) (*k)=imax;
01153     return xmax;
01154 };
01155
01156 template <class T, class T2>
01157 void Wb::cpyStride(
01158     T2* d2,
01159     const T* d0,
01160     unsigned n,
01161     const unsigned *idx, unsigned m,
01162     unsigned D2,
01163     unsigned D0,
01164     char add
01165 ){
01166     if (!n || !m) { return; }
01167     if (!idx) { return Wb::cpyStride(d2,d0,n,m,D2,D0, T2(add? 1:0), T(1)); }
01168
01169     unsigned i=0, j; const T *d0_=d0;
01170
01171     if (int(D0)<0) { D0=n; } else if (D0<n) { ++i; }
01172     if (int(D2)<0) { D2=n; } else if (D2<n) { ++i; }
01173
01174     if (i) wblog(FL,
01175         "ERR %s() stride too small (%d,%d/%d)",FCT,D0,D2,n);
01176     if (gotMemOverlap(d0, (m-1)*D0+n, d2, (m-1)*D2+n)) wblog(FL,
01177         "ERR %s() must not copy onto itself",FCT);
01178
01179     for (j=0; j<m; ++j, d2+=D2) { d0 = d0_+ idx[j] * D0;
01180         if (!add) { for (i=0; i<n; ++i) { d2[i] = d0[i]; } }
01181         else      { for (i=0; i<n; ++i) { d2[i] += d0[i]; } }
01182     }
01183 };
01184
01185 template <class T, class T2>
01186 void Wb::cpyStride(
01187     T2* d2,
01188     const T* d0,
01189     size_t n,
01190     size_t m,
01191     size_t D2,
01192     size_t D0,
01193     T2 fac2, T fac0
01194 ){
01195     if (!n || !m) { return; }
01196
01197     unsigned j=0;
01198     if (int(D0)<0) { D0=n; } else if (D0<n) { ++j; }
01199     if (int(D2)<0) { D2=n; } else if (D2<n) { ++j; }
01200
01201     if (j) wblog(FL,
01202         "ERR %s() stride too small (%d,%d/%d)",FCT,D0,D2,n);
01203     if (gotMemOverlap(d0, (m-1)*D0+n, d2, (m-1)*D2+n)) wblog(FL,
01204         "ERR %s() must not copy onto itself",FCT);
01205
01206     for (j=0; j<m; ++j, d2+=D2, d0+=D0) {
01207         Wb::cpyRange(d2,d0,n,fac2,fac0);
01208     }
01209 };
01210
01211 template <class T, class TB>
01212 void Wb::cpyRange(T* a, const TB* b, size_t n, T afac, TB bfac) {
01213
01214     size_t i=0;
01215
01216     if (bfac) {
01217         if (!afac) {
01218             if (bfac==TB( 1)) { for (; i<n; ++i) { a[i] = b[i]; } } else
01219             if (bfac==TB(-1)) { for (; i<n; ++i) { a[i] =-b[i]; } }
01220             else                { for (; i<n; ++i) { a[i] = b[i]*bfac; } }
01221         }
01222         else if (afac==T(1)) {
01223             if (bfac==TB( 1)) { for (; i<n; ++i) { a[i] += b[i]; } } else

```

```

01224         if (bfac==TB(-1)) { for (; i<n; ++i) { a[i]-= b[i]; }}
01225     else { for (; i<n; ++i) { a[i]+= b[i]*bfac; }}
01226 }
01227 else {
01228     if (bfac==TB( 1)) { for (; i<n; ++i) { (a[i]*=afac) += b[i]; }} else
01229     if (bfac==TB(-1)) { for (; i<n; ++i) { (a[i]*=afac) -= b[i]; }}
01230     else { for (; i<n; ++i) { (a[i]*=afac) += b[i]*bfac; }}
01231 }
01232 }
01233 else if (afac!=T(1)) {
01234     if (afac) { for (; i<n; ++i) { a[i]*=afac; }}
01235     else { T z=0; for (; i<n; ++i) { a[i]=z; }}
01236 }
01237 };
01238
01239 template <class T>
01240 void Wb::cpyRange(T* a, const T* b, size_t n) {
01241     for (size_t i=0; i<n; ++i) { a[i]=b[i]; }
01242 };
01243
01244 template <class T, class Tb>
01245 void Wb::cpyRange(T* a, const Tb* b, size_t n, char tcheck) {
01246
01247     if (!tcheck) {
01248         for (size_t i=0; i<n; ++i) { a[i]=T(b[i]); }}
01249     else {
01250         for (size_t i=0; i<n; ++i) { a[i]=T(b[i]);
01251             if (Tb(a[i])!=b[i]) wblog(FL,
01252                 "ERR %s() rounding error (%g/%g)",FCT,double(a[i]),double(b[i])
01253             );
01254         }
01255     }
01256 };
01257
01258 template <>
01259 void Wb::cpyRange(double* a, const double* b, size_t n) {
01260     memcpy(a,b,n*sizeof(double));
01261 };
01262
01263 template <>
01264 void Wb::cpyRange(wbcomplex* a, const wbcomplex* b, size_t n) {
01265     memcpy(a,b,n*sizeof(wbcomplex));
01266 };
01267
01268 template <>
01269 void Wb::cpyRange(int* a, const int* b, size_t n) {
01270     memcpy(a,b,n*sizeof(int));
01271 };
01272
01273 template <>
01274 void Wb::cpyRange(double* a, const wbcomplex* b, size_t n, char tcheck) {
01275
01276     double i2=0;
01277     for (size_t i=0; i<n; ++i) {
01278         a[i]=b[i].r; if (b[i].i) { i2+=(b[i].i*b[i].i); }
01279     }
01280
01281     if (i2) {
01282         sprintf_str("%s() got |Im(b)|^2 = %.3g",FCT,i2);
01283         wblog(FL,tcheck? "ERR %s":"WRN %s",str);
01284     }
01285 };
01286
01287 template <>
01288 void Wb::cpyRange(
01289     wbcomplex* a, const double* b, size_t n, char tcheck QS_UNUSED_VAR
01290 ) { for (size_t i=0; i<n; ++i) { a[i]=b[i]; }};
01291
01292 template <class T>
01293 void Wb::cpyRangeR(T* a, const wbcomplex* z, size_t n) {
01294     for (size_t i=0; i<n; ++i) { a[i]=z[i].r; }
01295 };
01296
01297 template <class T>
01298 void Wb::cpyRangeI(T* a, const wbcomplex* z, size_t n) {
01299     for (size_t i=0; i<n; ++i) { a[i]=z[i].i; }
01300 };
01301
01302 template <class T>
01303 void Wb::cpyRangeA(T* a, const wbcomplex* z, size_t n) {
01304     for (size_t i=0; i<n; ++i) { a[i]=z[i].abs(); }
01305 };
01306
01307 template <class T> inline
01308 bool Wb::uniformRange(const T* d, size_t n) {
01309     for (size_t i=1; i<n; ++i) if (d[i]!=d[0]) return 0;
01310     return 1;

```



```

01311 };
01312
01313 template <class T> inline
01314 T Wb::maxRange(const T* d, size_t n) {
01315     if (!n) wblog(FL,"ERR %s() got empty range",FCT);
01316     T x=d[0];
01317     for (size_t i=1; i<n; ++i) { if (x<d[i]) { x=d[i]; } }
01318     return x;
01319 };
01320
01321 template <class T> inline
01322 T Wb::minRange(const T* d, size_t n) {
01323     if (!n) wblog(FL,"ERR %s() got empty range",FCT);
01324     T x=d[0];
01325     for (size_t i=1; i<n; ++i) { if (x>d[i]) { x=d[i]; } }
01326     return x;
01327 };
01328
01329 template <class T> inline
01330 T Wb::prodRange(const T* d, size_t n, T x) {
01331     if (!n) wblog(FL,"WRN %s() got empty range (returning 0)",FCT);
01332     return prodRange(d,n,T(0));
01333 };
01334
01335 template <class TD, class TX> inline
01336 void Wb::timesRange(TD* d, TX x, size_t n, size_t stride) {
01337     if (!n) return;
01338     if (!d) wblog(FL,"ERR %s() got null space (n=%d)",FCT,n);
01339     Wb::update_range_stride(n,stride);
01340     for (size_t i=0; i<n; i+=stride) { d[i]*=x; }
01341 };
01342
01343 template <class T> inline
01344 char Wb::check_conj_flag(
01345     const T* a QS_UNUSED_VAR,
01346     char &conj
01347 ) {
01348     if (conj) {
01349         if (!ISCOMPLX(T)) { conj=0; } else {
01350             if (conj>2) {
01351                 if (strchr("2bBcC*",conj)) { conj=2; } else
01352                 if (strchr("1aA", conj)) { conj=1; }
01353             }
01354             if (conj<0 || conj>2) wblog(FL,
01355                 "WRN %s() invalid conj=%s",FCT,cSTR(conj));
01356         }
01357     }
01358     return conj;
01359 };
01360
01361 template <class T> inline
01362 T Wb::dotProd(const T* a, const T* b, size_t n, T x, char conj) {
01363     if (n) {
01364         size_t i=0; x=0;
01365         if (!a || !b) wblog(FL,"ERR %s() got null (%p, %p)",FCT,a,b);
01366         if (check_conj_flag(b,conj)) {
01367             if (conj==1)
01368                 { for (; i<n; ++i) x+=Wb::CONJ(a[i])*b[i]; }
01369             else { for (; i<n; ++i) x+=a[i]*Wb::CONJ(b[i]); }
01370         }
01371         else { for (; i<n; ++i) x+=(a[i]*b[i]); }
01372     }
01373     return x;
01374 };
01375
01376 template <class T> inline
01377 T Wb::dotProd(const T* a, const T* b, size_t n, char conj) {
01378     T x=T(0);
01379     if (n) { x=dotProd(a,b,n,x,conj); }
01380     else {
01381         wblog(FL,"WRN %s() got empty range",FCT);
01382     }
01383 }

```

```

01398     }
01399
01400     return x;
01401 };
01402
01403 template <class T> inline
01404 void Wb::TimesElRange(T* a, const T* b, size_t n, char conj) {
01405
01406     if (!n) { return; }
01407
01408     if (!a || !b) wblog(FL,
01409         "ERR %s() got null space (%p, %p)", FCT, a, b);
01410
01411     if (check_conj_flag(b, conj)) {
01412         if (conj==1)
01413             { for (size_t i=0; i<n; ++i) a[i]=Wb::CONJ(a[i])*b[i]; }
01414         else { for (size_t i=0; i<n; ++i) a[i]*=Wb::CONJ(b[i]); }
01415     }
01416     else { for (size_t i=0; i<n; ++i) a[i]*=b[i]; }
01417 };
01418
01419 template <class T> inline
01420 void Wb::timesElRange(T* c, const T* a, const T* b, size_t n, char conj) {
01421
01422     if (!n) { return; }
01423     if (!a || !b || !c) wblog(FL,
01424         "ERR %s() got null space (%p, %p, %p)", FCT, a, b, c);
01425     size_t i=0;
01426
01427     if (check_conj_flag(b, conj)) {
01428         if (conj==1)
01429             { for (; i<n; ++i) { c[i] = Wb::CONJ(a[i]) * b[i]; } }
01430         else { for (; i<n; ++i) { c[i] = a[i] * Wb::CONJ(b[i]); } }
01431     }
01432     else { for (; i<n; ++i) { c[i] = a[i]*b[i]; } }
01433 };
01434
01435 template <class T> inline
01436 void Wb::timesElRange_add(
01437     T* c, const T* a, const T* b, size_t n, T bfac, char conj
01438 ){
01439     size_t i=0;
01440
01441     if (!n || !bfac) { return; }
01442     if (!a || !b || !c) wblog(FL,
01443         "ERR %s() got null space (%p, %p, %p)", FCT, a, b, c);
01444
01445     if (check_conj_flag(b, conj)) {
01446         if (conj==1) {
01447             if (bfac==T(+1)) { for (; i<n; ++i) c[i]+=Wb::CONJ(a[i])*b[i]; } else
01448             if (bfac==T(-1)) { for (; i<n; ++i) c[i]-=Wb::CONJ(a[i])*b[i]; }
01449             else { for (; i<n; ++i) c[i]+=Wb::CONJ(a[i])*b[i]*bfac; }
01450         }
01451         else {
01452             if (bfac==T(+1)) { for (; i<n; ++i) c[i]+=a[i]*Wb::CONJ(b[i]); } else
01453             if (bfac==T(-1)) { for (; i<n; ++i) c[i]-=a[i]*Wb::CONJ(b[i]); }
01454             else { for (; i<n; ++i) c[i]+=a[i]*Wb::CONJ(b[i])*bfac; }
01455         }
01456     }
01457     else { if (bfac==T(+1)) { for (; i<n; ++i) c[i]+=a[i]*b[i]; } else
01458           if (bfac==T(-1)) { for (; i<n; ++i) c[i]-=a[i]*b[i]; }
01459           else { for (; i<n; ++i) c[i]+=a[i]*b[i]*bfac; } }
01460 }
01461 };
01462
01463 template <class T> inline
01464 void Wb::timesElRange_OM(T* c,
01465     const T* a, const T* b,
01466     size_t N, unsigned M, char conj
01467 ){
01468     if (!N || !M) { wblog(FL,
01469         "WRN %s() got empty NxM=%dx%d - return", FCT, N, M); return; }
01470     if (!a || !b || !c) wblog(FL,
01471         "ERR %s() got null (%p, %p, %p)", FCT, a, b, c);
01472
01473     unsigned m=0; size_t i=0;
01474     check_conj_flag(b, conj);
01475
01476     for (; m<M; ++m) {
01477         if (m) { i=0; a+=N; b+=N;
01478             if (!conj)
01479                 { for (; i<N; ++i) { c[i] += a[i]*b[i]; } } else
01480             if (conj==1)
01481                 { for (; i<N; ++i) { c[i] += Wb::CONJ(a[i]) * b[i]; } }
01482             else { for (; i<N; ++i) { c[i] += a[i] * Wb::CONJ(b[i]); } }
01483         }
01484         else {

```

```

01485         if (!conj)
01486             { for (; i<N; ++i) { c[i] = a[i]*b[i]; }} else
01487             if (conj==1)
01488                 { for (; i<N; ++i) { c[i] = Wb::CONJ(a[i]) *b[i]; }}
01489             else { for (; i<N; ++i) { c[i] = a[i] * Wb::CONJ(b[i]); }}
01490         }
01491     }
01492 };
01493
01494 template<class T> inline
01495 T Wb::overlap(
01496     const T* a, const T* b, size_t n,
01497     size_t stride,
01498     char tnorm QS_UNUSED_VAR
01499 ){
01500     T x2=0;
01501     if (n) {
01502         Wb::update_range_stride(n,stride);
01503         if (!a || !b) wblog(FL,"ERR %s() got null (%p, %p)",FCT,a,b);
01504         for (size_t i=0; i<n; i+=stride) { x2+=Wb::CONJ(a[i])*b[i]; }
01505     }
01506     return x2;
01507 };
01508
01509 template<> inline
01510 wbcomplex Wb::overlap(
01511     const wbcomplex* a, const wbcomplex* b, size_t n,
01512     size_t stride, char tnorm
01513 ){
01514     wbcomplex z2=0;
01515     if (n) { size_t i=0;
01516         Wb::update_range_stride(n,stride);
01517         if (!a || !b) wblog(FL,"ERR %s() got null (%p, %p)",FCT,a,b);
01518         if (tnorm) { for (; i<n; i+=stride) { z2+=a[i]*b[i]; }} else
01519             if (a!=b) { for (; i<n; i+=stride) { z2+=a[i].conj()*b[i]; }}
01520             else { double nrm2=0;
01521                 for (; i<n; i+=stride) { nrm2+=a[i].abs2(); }
01522                 z2=nrm2;
01523             }
01524     }
01525     return z2;
01526 };
01527
01528 template<class T>
01529 T Wb::gs_project_range(
01530     T *a, const T *b,
01531     size_t n, size_t stride, char isnorm, char tnorm
01532 ){
01533     T x=overlap(b,a,n,stride,tnorm);
01534
01535     if (!isnorm) {
01536         T nrm2=overlap(b,b,n,stride,tnorm);
01537         if (std::fabs(double(nrm2))<1E-12) wblog(FL,
01538             "WRN %s() got norm=%.3g !?",FCT,double(nrm2));
01539         x/=nrm2;
01540     }
01541
01542     Wb::update_range_stride(n,stride);
01543     for (size_t i=0; i<n; i+=stride) { a[i]-=x*b[i]; }
01544
01545     return x;
01546 };
01547
01548 template <class T> inline
01549 size_t Wb::replRange(T* a, size_t n, T x, T v) {
01550
01551     size_t i,m=0;
01552
01553     if (!Wb::isnan(x))
01554         { for (i=0; i<n; ++i) if (a[i]==x) { a[i]=v; ++m; }}
01555     else { for (i=0; i<n; ++i) if (Wb::isnan(a[i])) { a[i]=v; ++m; }}
01556
01557     return m;
01558 }
01559
01560 inline size_t Wb::countNaN(double* a, size_t m) {
01561     size_t i,n=0;
01562     for (i=0; i<m; i++) { if (std::isnan(a[i])) ++n; }
01563     return n;
01564 };
01565
01566 template <class T>
01567 void Wb::cpyZRange(
01568     const double *R, const double *I, T *Z, size_t n
01569 ){
01570     wblog(FL,"ERR %s not applicable for type '%s'\n(%p,%p,%p,%d)",
01571         FCT, TSTR(T),R,I,Z,n);

```

```

01572     };
01573
01574     template <>
01575     void Wb::cpyZRange(
01576         const double *R, const double *I, wbcomplex *Z, size_t n
01577     ){
01578         for (size_t i=0; i<n; i++)
01579             Z[i].set(R[i], I ? I[i] : 0);
01580     }
01581
01582     template <class T>
01583     void Wb::splitZRange(
01584         const T *Z QS_UNUSED_VAR,
01585         double *R QS_UNUSED_VAR,
01586         double *I QS_UNUSED_VAR,
01587         size_t n QS_UNUSED_VAR
01588     ){ wblog(FL,"ERR %s not applicable for type '%s'",FCT,TSTR(T)); }
01589
01590     template <>
01591     void Wb::splitZRange(
01592         const wbcomplex *Z, double *R, double *I, size_t n
01593     ){
01594         for (size_t i=0; i<n; i++) {
01595             R[i]=Z[i].r; if (I) I[i]=Z[i].i; }
01596     }
01597
01598     template<>
01599     void Wb::chopTiny_float(double *d, size_t n, double ref) {
01600
01601         size_t i; if (!n) return;
01602
01603         if (ref<0) {
01604             for (ref=0, i=0; i<n; ++i) ref=MAX(ref,::fabs(d[i]));
01605             if (ref==0) return;
01606         }
01607
01608         if (ref!=0) {
01609             int k=int(std::log(ref)/std::log(2));
01610             if (k>=0) ref=(1<<k); else ref=1/(1<<-k);
01611
01612             for (i=0; i<n; ++i) {
01613                 if (d[i]>0.) d[i]=float(d[i]+ref)-ref; else
01614                     if (d[i]<0.) d[i]=float(d[i]-ref)+ref;
01615             }
01616         }
01617         else {
01618             for (i=0; i<n; ++i)
01619                 if (d[i]!=0.) d[i]=double(float(d[i]));
01620         }
01621     };
01622
01623     template<>
01624     double Wb::chopTiny_imag(wbcomplex *z, size_t n, double eps) {
01625
01626         double i2=0, r2=0; size_t k=0;
01627
01628         for (; k<n; ++k) { r2+=(z[k].r*z[k].r); i2+=(z[k].i*z[k].i); }
01629         if (i2) {
01630             if (r2>1) { eps*=std::sqrt(r2); }
01631             if ((i2=std::sqrt(i2))>eps) { i2=-i2; }
01632             else {
01633                 for (k=0; k<n; ++k) { z[k].i=0; }
01634             }
01635         }
01636         return i2;
01637     };
01638
01639     template<>
01640     void Wb::chopTiny_z(wbcomplex *z, size_t n, double eps) {
01641
01642         size_t k=0; double a2, r2=0, i2=0;
01643
01644         for (; k<n; ++k) {
01645             r2+=(z[k].r*z[k].r);
01646             i2+=(z[k].i*z[k].i);
01647         }
01648         a2=r2+i2; if (a2>1) { eps*=std::sqrt(a2); }
01649
01650         if (std::sqrt(i2)<=eps) { for (k=0; k<n; ++k) { z[k].i=0; }} else
01651         if (std::sqrt(r2)<=eps) { for (k=0; k<n; ++k) { z[k].r=0; }}
01652     };
01653
01654     template<class T>
01655     void Wb::invertIndex(
01656         const wbvector<T> &i1, unsigned N,
01657         wbvector<T> &i2, char lflag
01658     ){

```

```

01659     wbvector<char> flag(N);
01660     unsigned i,k,n;
01661
01662     for (i=0; i<i1.len; i++) {
01663         k=i1[i]; if (k<N) flag[k]=1; else
01664             wblog(FL, "ERR index out of bounds (%d/%d)", k, N);
01665     }
01666     n=flag.isum();
01667
01668     if (n!=i1.len && lflag)
01669         wblog(FL, "WRN index set not unique!? (%d/%d)", n, i1.len);
01670
01671     i2.init(N-n);
01672     for (k=i=0; i<N; i++) if (flag[i]==0) i2[k++]=i;
01673 }
01674
01675 void Wb::setRand(wbvector<wbcomplex> &zz, double fac, double shift) {
01676     size_t i=0; fac/=(double)RAND_MAX;
01677
01678     if (shift==0.)
01679         for (; i<zz.len; ++i)
01680             zz.data[i].set(fac*std::rand(), fac*std::rand());
01681     else
01682         for (; i<zz.len; ++i)
01683             zz.data[i].set(fac*std::rand()+shift,fac*std::rand()+shift);
01684 }
01685
01686 template<class T>
01687 void Wb::set2avg(T *dd, const size_t n) {
01688     size_t i; T dbl=0; if (!n) return;
01689     for (i=0; i<n; ++i) { dbl+=dd[i]; }; dbl/= n;
01690     for (i=0; i<n; ++i) { dd[i]=dbl; }
01691 };
01692
01693 template<class T>
01694 void Wb::getDiff(const wbvector<T> &xx, wbvector<T> &dx) {
01695     size_t i, n=xx.len; dx.init(n);
01696     for (i=1; i<n; i++) {
01697         dx[i-1] += (dx[i] = xx[i]-xx[i-1]);
01698         if (i>1) dx[i-1] *= 0.5;
01699     }
01700 }
01701
01702 template<class TX, class TY>
01703 TY Wb::IntTrapez(const TX *xx, const TY *yy, const size_t n) {
01704     TY s=0;
01705
01706     for (size_t i=1; i<n; ++i) {
01707         s += 0.5 * (xx[i] - xx[i-1]) * (yy[i] + yy[i-1]);
01708     }
01709
01710     return s;
01711 };
01712
01713 template<class T>
01714 void markSet(
01715     wbvector< wbvector<T>* > &E0,
01716     wbvector< wbvector<char> > &mark,
01717     unsigned Nkmin,
01718     int &Nkeep,
01719     double Etrunc,
01720     wbvector<T> &E,
01721     double eps,
01722     double b,
01723     int dmax,
01724     const char *dir
01725 ){
01726     unsigned i,d,l,N,n=E0.len;
01727     wbvector<char> mm;
01728     wbperm P,iP;
01729
01730     for (N=i=0; i<n; i++) N+=(E0[i]->len);
01731
01732     E.init(N); mark.initDef(n);
01733     if (!N) return;
01734
01735     for (l=i=0; i<n; i++, l+=d) {
01736         d=(E0[i]->len); mark[i].init(d);
01737         memcpy(E.data+l, E0[i]->data, d*sizeof(T));
01738     }
01739
01740     E.Sort(P); P.invert(iP);
01741
01742     if (eps>0) {
01743         unsigned i0=0;
01744
01745         eps *= Wb::abs(E.last()-E[0]);

```

```

01746         for (i=1; i<=N; i++) {
01747             if (i==N || Wb::abs(E[i]-E[i-1])>eps) {
01748                 d=i-i0; if (d>1)
01749                     Wb::setRange2avg(E.data+i0,d);
01750                 i0=i;
01751             }
01752         }
01753
01754         wbvector<T> x; E.permute(x,iP);
01755         for (l=i=0; i<n; ++i, l+=d) {
01756             d=E0[i]->len;
01757             memcpy(E0[i]->data, x.data+l, d*sizeof(T));
01758         }
01759     }
01760
01761     if (Nkeep<0) Nkeep=N;
01762     if (Nkeep==0) return;
01763     if (Nkeep>=(int)N && Etrunc<=0) {
01764         for (i=0; i<n; i++) mark[i].set(1);
01765         Nkeep=N; return;
01766     }
01767
01768     markSet (FL,E,mm,Nkmin,Nkeep,Etrunc,b,dmax,dir);
01769
01770     mm.Permute (iP);
01771
01772     for (l=i=0; i<n; i++, l+=d) {
01773         d=mark[i].len;
01774         memcpy(mark[i].data, mm.data+l, d*sizeof(char));
01775     }
01776
01777 };
01778
01779 template<>
01780 void markSet(
01781     wbvector< wbvector<wbcomplex>>* > &E0,
01782     wbvector< wbvector<char> > &mark,
01783     unsigned Nkmin,
01784     int &Nkeep,
01785     double Etrunc,
01786     wbvector<wbcomplex> &E,
01787     double eps,
01788     double b,
01789     int dmax,
01790     const char *dir_
01791 ){
01792     unsigned i,d,l,N, n=E0.len;
01793     const char *s=NULL;
01794
01795     wbvector<char> mm;
01796     wbvector<double> ee;
01797     wbperm P,iP;
01798
01799     if (dir_ && dir_[0]) {
01800         if (!strcmp(dir_,"asc", 3)) { s=dir_+3; } else
01801         if (!strcmp(dir_,"desc",4)) { s=dir_+4; } else
01802             wblog(FL,"ERR %s() invalid direction >%s<",FCT,dir_);
01803     }
01804
01805     for (N=i=0; i<n; i++) N+=(E0[i]->len);
01806     if (!N) return;
01807
01808     E.init(N); ee.init(N); mark.initDef(n);
01809
01810     for (l=i=0; i<n; i++, l+=d) {
01811         d=(E0[i]->len); mark[i].init(d);
01812         memcpy(E.data+l, E0[i]->data, d*sizeof(wbcomplex));
01813     }
01814
01815     if (s==NULL || !strcmp(s,"R")) {
01816         Wb::cpyRangeR(ee.data,E.data,N);
01817     }
01818     else if (!strcmp(s,"I")) {
01819         Wb::cpyRangeI(ee.data,E.data,N);
01820     }
01821     else if (!strcmp(s,"A")) {
01822         wbcomplex z=E.min();
01823         wbvector<wbcomplex> x(E); z.i=0; x-=z;
01824         Wb::cpyRangeA(ee.data,x.data,N);
01825     }
01826     else wblog(FL,"ERR markSet() invalid direction >%s<", dir_);
01827
01828     ee.Sort(P); E.Select(P); P.invert(iP);
01829
01830     if (Nkeep<0) Nkeep=N;
01831     if (Nkeep==0) return;
01832

```

```

01833     if (Nkeep>=(int)N && Etrunc<=0) {
01834         for (i=0; i<n; ++i) mark[i].set(1);
01835         Nkeep=N; return;
01836     }
01837
01838     markSet (FL,ee,mm,Nkmin,Nkeep,Etrunc,MAX(b,eps),dmax,dir_);
01839
01840     mm.Permute(iP);
01841
01842     for (l=i=0; i<n; i++, l+=d) {
01843         d=mark[i].len;
01844         memcpy(mark[i].data, mm.data+l, d*sizeof(char));
01845     }
01846 }
01847
01848 template<class T>
01849 void markSet(const char *F, int L,
01850             const wbvector<T> &E,
01851             wbvector<char> &mark,
01852             unsigned Nkmin,
01853             int &Nkeep,
01854             double Etrunc,
01855             double b,
01856             int dmax,
01857             const char *dir_)
01858 ){
01859
01860     unsigned i,imax,l;
01861     unsigned N=E.len, nx;
01862     double x, xmax=0;
01863     char dir=+1;
01864
01865     if (dir_ && dir_[0]) {
01866         if (!strcmp(dir_,"asc", 3)) dir=+1; else
01867         if (!strcmp(dir_,"desc",4)) dir=-1; else
01868         wblog(F,L,"ERR %s() invalid direction >%s<",FCT,dir_);
01869     }
01870
01871     mark.init(N);
01872
01873     if (Nkeep<0 || Nkeep>int(N)) Nkeep=N;
01874     if (Nkeep==0) return;
01875
01876     if (Nkeep>=(int)N && Etrunc<=0) { mark.set(1); Nkeep=N; return; }
01877     if (N<1) wblog(FL,"ERR %s() N=%d !?",FCT,N);
01878
01879     if (int(Nkmin)<0) Nkmin=0;
01880
01881     if (dir>0) {
01882         if (Etrunc>0) {
01883             x=Etrunc+E[0];
01884             for (l=1; l<N; ++l) {
01885                 if (E[l-1]>E[l]) wblog(FL,"ERR %s() data not sorted!",FCT);
01886                 if (E[l]>=x && l>Nkmin) break;
01887             }
01888             if (int(l)>Nkeep) l=Nkeep;
01889         }
01890         else { l=unsigned(Nkeep); }
01891
01892         if (l==N) { mark.set(1); Nkeep=N; return; }
01893         if (l==0) { Nkeep=0; return; }
01894
01895         nx=(dmax>=0 ? (unsigned)dmax : MAX(16U,1/5));
01896
01897         if (b>0) {
01898             for (imax=i=l; i<N && i<l+nx; ++i) {
01899                 x=fabs(E[i]-E[i-1]);
01900                 if (x<=b) { if (xmax<x) { xmax=x; imax=i; } }
01901                 else break;
01902             }
01903             if (i>=l+nx) wblog(F,L,
01904                 "WRN %s() reached Nkeep+dmax=%d+%d states\n"
01905                 "(db=%.3g too small !? => %d @ %.3g)", FCT,l,nx,b,imax,xmax);
01906
01907             Nkeep=l=imax;
01908         }
01909         else if (b<0 && nx && l<N) {
01910
01911             double q=10/(-b*nx);
01912             unsigned NX=5/q, i2=MIN(N,l+NX);
01913             i=l-NX; if (int(i)<1) i=1;
01914
01915             for (imax=l; i<i2; ++i) { x=q*int(i-1);
01916                 x=exp(-x*x)*fabs(E[i]-E[i-1]);
01917                 if (x>xmax) { xmax=x; imax=i; }
01918             }
01919             if (i==N) { x=q*int(i-1);

```

```

01920         x=0.1*exp(-x*x)*fabs(E[N-1]-E[0]);
01921         if (x>xmax) { xmax=x; imax=i; }
01922     }
01923
01924     Nkeep=l=imax;
01925 }
01926
01927     for (i=0; i<l; ++i) mark[i]=1;
01928 }
01929 else {
01930     if (Etrunc>0) {
01931         unsigned lx=(Nkmin<N ? N-Nkmin : 0);
01932
01933         x=Etrunc;
01934
01935         for (l=N-1; l<N; --l) {
01936             if (l && E[l-1]>E[l]) wblog(FL,"ERR %s() data not sorted!",FCT);
01937             if (E[l]<=x && l<lx) break;
01938         }
01939         if (int(N-1-l)>Nkeep) { l = N-1-(unsigned)Nkeep; }
01940     }
01941     else { l=N-1-(unsigned)Nkeep; }
01942
01943     if (l>N ) { mark.set(l); Nkeep=N; return; }
01944     if (l==N-1) { Nkeep=0; return; }
01945
01946     nx=(dmax>=0 ? (unsigned)dmax : MAX(16U, (N-1)/5));
01947
01948     if (b>0) {
01949         for (imax=i=1; i<N && (l-i)<nx; --i) {
01950             x=fabs(E[i+1]-E[i]);
01951             if (x<=b) { if (xmax<x) { xmax=x; imax=i; }}
01952             else break;
01953         }
01954         if ((l-i)>=nx) wblog(F,L,
01955             "WRN %s() reached Nkeep+dmax=%d+%d states (%d/%d)\n"
01956             "(db=%3g too small !? => %d @ %3g)",FCT,
01957             Nkeep,nx, Nkeep+nx,N, b,N-imax,xmax);
01958         l=imax;
01959     }
01960     else if (b<0 && nx && l<N ) {
01961
01962         double q=10/(-b*nx);
01963         unsigned NX=5/q, i1=l-NX; if (int(i1)<0) i1=0;
01964         i=MIN(N-1,l+NX);
01965
01966         for (imax=l; i>i1; --i) { x=q*int(i-l);
01967             x=exp(-x*x)*fabs(E[i+1]-E[i]);
01968             if (xmax<x) { xmax=x; imax=i; }
01969         }
01970         if (i==0) { x=q*int(i-l);
01971             x=0.1*exp(-x*x)*fabs(E[N-1]-E[0]);
01972             if (x>xmax) { xmax=x; imax=i-1; }
01973         }
01974         l=imax;
01975     }
01976
01977     if (l>N) i=0;
01978     else { i=l+1; }; Nkeep=(l<N ? N-1-l : 0);
01979
01980     for (; i<N; ++i) mark[i]=1;
01981 }
01982 };
01983
01984 #endif
01985

```

5.5 gathered.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : QSpace additional routines
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,

```



```

00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the license for the specific language governing permissions and
00017  * limitations under the license.
00018  * ----- */
00019
00020 #ifndef __WB_GATHERED_HH__
00021 #define __WB_GATHERED_HH__
00022
00023 /* ----- */
00024 /* ----- */
00025
00026 namespace Wb {
00027     const char* basename(const char *data, char c='/');
00028
00029     void print_backtrace(
00030         const char *F=NULL, int L=0, const char *istr=NULL);
00031
00032     int GetNumThreads(const char *F, int L, int &n, const char *name);
00033
00034     template<class T>
00035     int GetEnv(const char *F, int L, const char *name, T& val);
00036
00037     int EnvIsSet(const char *F, int L, const char *name);
00038
00039     wstring repHome(const char *file);
00040
00041     inline int INT(const double a) {
00042         int i=:round(a);
00043         if (std::fabs(i-a)>1E-10) wblog(FL,"ERR %s(%g) !?",FCT,a);
00044         return i;
00045     }
00046
00047     const char* strstri (const char *s1, const char *s2);
00048     const char* strstrw (const char *s1, const char *s2);
00049     const char* strstrwi(const char *s1, const char *s2);
00050     const char* strchrri (const char *s, char c);
00051
00052     const char* rstrstr (const char *s1, const char *s2);
00053
00054     int strrep(
00055         const char *S0,
00056         const char *t0,
00057         const char *t2,
00058         char *Sout,
00059         size_t N,
00060         char gflag='g'
00061     );
00062
00063     void shift(char *s, unsigned n, int p, char erase=0);
00064
00065     void shift(char *s, unsigned n, int p, const char *srep);
00066
00067     bool isword(const char &c) { return (
00068         (c>='0' && c<='9') ||
00069         (c>='a' && c<='z') ||
00070         (c>='A' && c<='Z') || c=='_'
00071     ); };
00072
00073     template <class T>
00074     double mod(double a, T b) {
00075         if (b<=0) wblog(FL,"ERR %s() got b=%g",FCT,b);
00076         a=:fmod(a,double(b)); if (a<0) { a+=b; }
00077         return a;
00078     };
00079
00080     template <class T, ENABLE_IF_isINT(T)>
00081     int mod(int a, T b) {
00082         if (b<=0) wblog(FL,"ERR %s() got b=%d",FCT,b);
00083         a %= int(b); if (a<0) { a+=b; }
00084         return a;
00085     };
00086
00087     unsigned strlen(const char *s, unsigned n) {
00088         for (unsigned i=0; i<n; ++i) if (!s[i]) return i;
00089         return n;
00090     };
00091
00092     template <class T>
00093     bool isLower(const T* a, const T* b, const size_t n);
00094
00095     template <class T>
00096     bool isEqual(const T* a, const T* b, const size_t n);
00097
00098     template <class T>
00099     bool allEqual(const T* a, const size_t n, const T &x);
00100
00101     template <class T>

```

```

00102     bool anyEqual(const T* a, const size_t n, const T &x);
00103
00104     template <class T>
00105     bool anyUnqual(const T* a, const size_t n, const T &x);
00106
00107     template <class T> inline
00108     bool anyUnequal(const T* a, const size_t n, const T &x);
00109
00110     template <class T>
00111     T getdscale(const T* d, size_t n);
00112
00113     template <class T> inline
00114     void scale_eps(T &eps, const T* d, size_t n);
00115
00116 }
00117
00118 template <class T, class T2> inline
00119 char GOT_EPS(const T &a, const T2 &eps) {
00120     if (a==0 || (eps!=0 && ABS(a)<eps)) return 1;
00121     return 0;
00122 }
00123
00124 void setSigHandler(char i);
00125 void wbSigHandler(int sig);
00126
00127 char *gsh_F=NULL; int gsh_L=0;
00128
00129 namespace Wb {
00130 class SigHandler {
00131
00132 public:
00133     SigHandler(const char *F, int L) {
00134         if (F && F[0]) {
00135             const char *s=strrchr(F,'/'); if (!s) { s=F; }
00136             gsh_F = new char[strlen(s)+1]; strcpy(gsh_F,s);
00137             gsh_L=L;
00138         }
00139         else {
00140             const char s0[]=__FILE__, *s=strrchr(s0,'/'); if (!s) { s=s0; }
00141             gsh_F = new char[strlen(s)+1]; strcpy(gsh_F,s);
00142             gsh_L=__LINE__;
00143         }
00144         #ifdef DBSTOP
00145         #else
00146             setSigHandler('i');
00147         #endif
00148     };
00149
00150     ~SigHandler() {
00151         #ifdef DBSTOP
00152         #else
00153             setSigHandler('r');
00154         #endif
00155         gsh_L=0; if (gsh_F) { delete [] gsh_F; gsh_F=NULL; }
00156     };
00157
00158     int check911(char tflag=0) { doflush();
00159         if (tflag<=0) {
00160             #ifndef DBSTOP
00161                 wbSigHandler(911);
00162             #endif
00163             return 0;
00164         }
00165         else { return check911_(); }
00166     };
00167
00168     static int check911_(const char *F=NULL, int L=0) {
00169         int i=SigHandler::icount,
00170             a=SigHandler::account, n=SigHandler::NC0;
00171
00172         if (i>=n) { i=0; } else { i=n-i; if (i>9) { i=9; }}
00173         if (a>=n) { a=0; } else { a=n-a; if (a>9) { a=9; }}
00174         if (F) { wblog(F_L,"ERR %s() ",FCT); }
00175         return (i + 10*a);
00176     };
00177
00178     static int NC0, icount, account;
00179
00180 protected:
00181 private:
00182
00183 };
00184
00185 int SigHandler::NC0 = 3;
00186
00187 int SigHandler::icount = SigHandler::NC0;
00188 int SigHandler::account = SigHandler::NC0;

```

```

00189
00190 };
00191
00192 template <class TX>
00193 class wbREF {
00194
00195     public:
00196
00197         wbREF (const TX &A, char flag) : orig(&A), ref(0) {
00198             if (flag) ref=new TX(A); else ref=&A;
00199         };
00200
00201         TX& Ref() { return *ref; }
00202
00203         ~wbREF() { if (ref!=orig) delete ref; }
00204
00205     protected:
00206     private:
00207         const TX *orig, *ref;
00208 };
00209
00210 namespace Wb {
00211
00212     template <class T>
00213     class num2Fmt {
00214
00215     public:
00216         num2Fmt(const char *F, int L,
00217             int m_=-2, int p_=-2, const char *t_="" ) : m(m_), p(p_) {
00218
00219             if (t_ && t_[0]) {
00220                 if (strlen(t_)>2) wblog(F_L,"ERR %s() invalid t='%s'",FCT,t_);
00221                 strcpy(t,t_);
00222             }
00223             else { memset(t,0,3); }
00224
00225             check_init();
00226         };
00227
00228         int check_init();
00229
00230         explicit operator wstring() const;
00231
00232     private:
00233         char m,p;
00234         char t[3];
00235
00236         void get_fmt(wstring &s) const;
00237 };
00238
00239     template<class T> inline
00240     int num2int(const char *F, int L, const T &x);
00241
00242     template<class T> inline
00243     unsigned checkInt(
00244         const char *F, int L, const T* x, size_t n,
00245         double eps=1E-14
00246     );
00247
00248     char* strpad(char *s, char c, unsigned n, unsigned w=1) {
00249         unsigned i,l=strlen(s);
00250         for (i=0; i<w; ++i) s[l++]=c;
00251         for (i=l; i<n; ++i) s[l++]=c;
00252         return s;
00253     };
00254
00255     unsigned strpad_(char *s, unsigned n, char c=' ', int z=1) {
00256         if (int(n)>0) {
00257             unsigned i=0; for (; i<n; ++i) { s[i]=c; }
00258             if (z) s[i]=0;
00259             return i;
00260         }
00261         return 0;
00262     };
00263
00264     template <class T>
00265     int charGetNumber(const char *F, int L, const char *s, T &x);
00266
00267     bool isBaseType(const std::type_info &type_id);
00268     bool isComplexType(const std::type_info &type_id);
00269
00270     wstring TimeStamp(char type=0);
00271
00272     void inl(unsigned n) { for (unsigned i=0; i<n; ++i) printf("\n"); }
00273
00274     void quietErrLog(const char *F, int L, const char *msg, char qflag) {
00275         if (qflag) snprintf(str,256,"%s [%s:%d]",msg,Wb::basename(F),L);

```

```

00276         else wblog(F,L,"ERR %s",msg);
00277     }
00278
00279     template <class T1, class T2> inline
00280     void safeConvert(const char *F, int L, const T1 &x1, T2 &x2);
00281
00282     template <class T1, class T2> inline
00283     int checkUpdate(T1& a, const T2 &r) {
00284         if (a!=T1(r)) { a=T1(r); return 1; } else return 0;
00285     };
00286
00287     template <class T>
00288     bool uniformRange(const T* d, const size_t n);
00289
00290     template <class T>
00291     T maxRange(const T* d, const size_t n);
00292
00293     template <class T>
00294     T minRange(const T* d, const size_t n);
00295
00296     template <class T>
00297     T addRange2(const T* d, size_t n, const size_t stride=1);
00298
00299     template <class T>
00300     T addRange(const T* d, size_t n, const size_t stride=1);
00301
00302     template <class T>
00303     void addRange(
00304         const T* a, const T* b, T* c, size_t n,
00305         const size_t stride=1
00306     );
00307
00308     template <class T>
00309     void addRange(const T* a, T* c, size_t n, const size_t stride=1);
00310
00311     template <class T>
00312     void minusRange(
00313         const T* a, const T* b, T* c, const size_t n,
00314         size_t stride=1
00315     );
00316
00317     template <class T>
00318     void diffRange(
00319         const T* a, const T* b, T* c, size_t n,
00320         size_t stride=1
00321     );
00322
00323     template <class T>
00324     void setRange2avg(T* a, size_t m);
00325
00326     template <class T>
00327     size_t nnzRange(const T* a, size_t m);
00328
00329     void update_range_stride(size_t &n, const size_t stride);
00330
00331     template <class T, class T2> inline
00332     void cpyStride(
00333         T2* d,
00334         const T* d0,
00335         unsigned n,
00336         const unsigned *idx, unsigned m,
00337         unsigned D2=-1,
00338         unsigned D0=-1,
00339         char add=0
00340     );
00341
00342     template <class T, class T2>
00343     void cpyStride(
00344         T2* d2,
00345         const T* d0,
00346         size_t n,
00347         size_t m,
00348         size_t D2=-1,
00349         size_t D0=-1,
00350         T2 fac2=0, T fac0=1
00351     );
00352
00353     template <class T, class TB>
00354     void cpyRange(T* a, const TB* b, size_t n, T afac, TB bfac);
00355
00356     template <class T>
00357     void cpyRange(T* a, const T* b, size_t n);
00358
00359     template <class T, class Tb>
00360     void cpyRange(T* a, const Tb* b, size_t n, char check_type=1);
00361
00362     template <class T>

```

```

00363 void cpyRangeR(T* a, const wbcomplex* z, size_t n);
00364 template <class T>
00365 void cpyRangeI(T* a, const wbcomplex* z, size_t n);
00366 template <class T>
00367 void cpyRangeA(T* a, const wbcomplex* z, size_t n);
00368
00369 template <class T>
00370 T prodRange(const T* d, const size_t n);
00371 template <class T>
00372 T prodRange(const T* d, const size_t n, T x0);
00373
00374 template <class T>
00375 T dotProd(const T* a, const T* b, size_t n, char conj=0);
00376 template <class T>
00377 T dotProd(const T* a, const T* b, size_t n, T x, char conj=0);
00378
00379 template <class T> inline
00380 void TimesElRange(T* a, const T* b, size_t n, char conj=0);
00381
00382 template <class T> inline
00383 void timesElRange(T* c, const T* a, const T* b, size_t n, char conj=0);
00384
00385 template <class T> inline
00386 char check_conj_flag(const T* a, char &conj);
00387
00388 template <class T> inline
00389 void timesElRange_OM(
00390     T* c,
00391     const T* a, const T* b,
00392     size_t n, unsigned m,
00393     char conj=0);
00394
00395 template <class T>
00396 void timesElRange_add(
00397     T* c, const T* a, const T* b, size_t n, T bfac=T(1), char conj=0);
00398
00399 template <class TD, class TX> inline
00400 void timesRange(TD* d, TX x, const size_t n, size_t stride=1);
00401
00402 template <class T>
00403 T rangeNorm2(const T* d, size_t n, size_t *k=NULL);
00404
00405 template <class T>
00406 T rangeNormDiff2(const T* d1, const T* d2, size_t n,
00407     const T &fac, size_t *k=NULL
00408 );
00409
00410 template <class T>
00411 T rangeNormDiff2(const T* d1, const T* d2, size_t n);
00412
00413 template <class T>
00414 T rangeMaxDiff(const T* d1, const T* d2, size_t n, size_t *k=NULL);
00415
00416 template<class T>
00417 T overlap(
00418     const T *v1, const T *v2, size_t n,
00419     size_t stride=1, char tnorm=0
00420 );
00421
00422 template<class T>
00423 T gs_project_range(
00424     T *v1, const T *v2,
00425     size_t n, size_t stride=1, char isnorm=0, char tnorm=0
00426 );
00427
00428 template <class T>
00429 size_t replRange(T* a, size_t n, T x, T v);
00430
00431 template <class T> inline
00432 size_t replRange(const char *F, int L, T *a, size_t n, T x, T v) {
00433     size_t m=replRange(a,n,x,v);
00434     if (m) wblog(F,L,"Skipped %d NaN's",m);
00435     return m;
00436 };
00437
00438 size_t countNaN(double* a, size_t m);
00439
00440 template<class T>
00441 void chopTiny_float(
00442     T *d      QS_UNUSED_VAR,
00443     size_t n  QS_UNUSED_VAR,
00444     T dref    QS_UNUSED_VAR ==-1
00445 ) { return; };
00446
00447 template<class T>
00448 void chopTiny_z(
00449     T *d      QS_UNUSED_VAR,

```

```

00450     size_t n    QS_UNUSED_VAR,
00451     double eps  QS_UNUSED_VAR =1e-14
00452 ) { return; };
00453
00454 template<class T>
00455 double chopTiny_imag(
00456     T *d        QS_UNUSED_VAR,
00457     size_t n    QS_UNUSED_VAR,
00458     double eps  QS_UNUSED_VAR =1e-14
00459 ) { return 0; };
00460
00461 template <class T> inline
00462 int recCompare(
00463     const T* a, const T* b, const size_t n, char lex=1
00464 ){
00465     if (lex>0) {
00466         for (size_t i=0; i<n; ++i) {
00467             if (a[i]<b[i]) return -1; else
00468             if (a[i]>b[i]) return +1;
00469         }
00470     }
00471     else {
00472         for (size_t i=n-1; i<n; --i) {
00473             if (a[i]<b[i]) return -1; else
00474             if (a[i]>b[i]) return +1;
00475         }
00476     }
00477     return 0;
00478 };
00479
00480 template <class T> inline
00481 int recCompare(
00482     const T* a, const T* b, const size_t n, char lex,
00483     T eps, int *nwrn=NULL
00484 ){
00485     if (eps<=T(0)) return recCompare(a,b,n,lex);
00486
00487     int nwrn0=0, *nwrn=(nwrn ? nwrn : &nwrn0);
00488     T x=0.2*eps;
00489
00490     if (lex>0) {
00491         for (size_t i=0; i<n; ++i) {
00492             if (a[i]<b[i]-eps) return -1; else
00493             if (a[i]>b[i]+eps) return +1; else
00494             if (!nwrn[0] && std::fabs(a[i]-b[i])>x) {
00495                 wblog(FL,"WRN %s() got |a-b|/eps=%.3g (%d/%d)",
00496                     FCT,(a[i]-b[i])/eps,i+1,n); ++nwrn[0];
00497             }
00498         }
00499     }
00500     else {
00501         for (size_t i=n-1; i<n; --i) {
00502             if (a[i]<b[i]-eps) return -1; else
00503             if (a[i]>b[i]+eps) return +1; else
00504             if (!nwrn[0] && std::fabs(a[i]-b[i])>x) {
00505                 wblog(FL,"WRN %s() got |a-b|/eps=%.3g (%d/%d)",
00506                     FCT,(a[i]-b[i])/eps,i+1,n); ++nwrn[0];
00507             }
00508         }
00509     }
00510     return 0;
00511 };
00512
00513 template <class T>
00514 int cmpRange(
00515     const T *a, const T *b, const size_t n, char lex=+1
00516 ){
00517     if (lex>0) {
00518         for (size_t i=0; i<n; ++i) { if (a[i]!=b[i]) {
00519             return (a[i]<b[i] ? -1 : +1);
00520         }}
00521         return 0;
00522     }
00523     else {
00524         if (!lex) wblog(FL,"WRN %s() got lex=%d",FCT,lex);
00525         for (size_t i=n-1; i<n; --i) { if (a[i]!=b[i]) {
00526             return (a[i]<b[i] ? -1 : +1);
00527         }}
00528         return 0;
00529     }
00530 };
00531
00532 template <class T> void cpyZRange(
00533     const double *R, const double *I, T *Z, const size_t n);
00534
00535 template <class T> void splitZRange(
00536     const T *Z, double *R, double *I, const size_t n);

```

```

00537
00538     template<class T>
00539     void invertIndex(
00540         const wbvector<T> &i1, unsigned N,
00541         wbvector<T> &i2, char lflag=1
00542     );
00543
00544     void setRand(wbvector<wbcomplex> &zz, double fac=1., double shift=0.);
00545
00546     template<class T>
00547     void set2avg(T *dd, const size_t n);
00548
00549     template<class T>
00550     void getDiff(const wbvector<T> &xx, wbvector<T> &dx);
00551
00552     template<class TX, class TY>
00553     TY IntTrapez(const TX *xx, const TY *yy, const size_t n);
00554
00555     inline bool contains(const char* s, const char &x, const unsigned &m=16) {
00556         if (s)
00557             for (unsigned i=0; s[i] && i<m; i++) { if (s[i]==x) return 1; };
00558         return 0;
00559     };
00560 };
00561
00562 };
00563
00564     template<class T>
00565     void markSet(
00566         wbvector< wbvector<T>* > &E0,
00567         wbvector< wbvector<char> > &mark,
00568         unsigned Nkmin,
00569         int &Nkeep,
00570         double Etrunc,
00571         wbvector<T> &E,
00572         double eps=0.,
00573         double b=0.,
00574         int dmax=-1,
00575         const char *dir=0
00576     );
00577
00578     template<class T>
00579     void markSet(const char *F, int L,
00580         const wbvector<T> &E,
00581         wbvector<char> &mark,
00582         unsigned Nkmin,
00583         int &Nkeep,
00584         double Etrunc=0,
00585         double b=0.,
00586         int dmax=-1,
00587         const char *dir=0
00588     );
00589
00590 #endif
00591

```

5.6 matlib.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : QSpace MEX routines
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_MATLIB_CC__
00021 #define __WB_MATLIB_CC__
00022
00023 /* ----- //
00024  * NB! intended for mex routines that communicate with matlab desktop
00025  * i.e. mex-files, but not programs e.g. with MAIN defined // Wb,Mar31,20

```

```

00026     NB! mexCallMATLAB(), mexDisp, mexPutVariable, mexGetVariablePtr, ...
00027 // ----- */
00028 #ifdef MATLAB_MEX_FILE
00029 /* ----- */
00030     Safeguard: matlab 'engine' may only be called from the same thread
00031     that also called the mex routine! see printf -> mexPrintf() error
00032     in tstmex_assert_error.cc // Wb,May06,19
00033 // ----- */
00034
00035 int Wb::CallMatlab(
00036     int nargsout, mxArray *argout[], int nargsin, mxArray *argin[],
00037     const char *fctname
00038 ){
00039     if (Wb::my_caller_tid==omp_get_thread_num()) {
00040         return mexCallMATLAB(nargsout,argout,nargin,argin,fctname);
00041     }
00042     else { fprintf(stdout,
00043         "mexCallMATLAB() to '%s' only allowed main thread (%d/%d)",
00044         fctname ? fctname : "(function?)", Wb::my_caller_tid,
00045         omp_get_thread_num());
00046
00047         #ifdef DBSTOP
00048             dbstop(FL);
00049         #endif
00050
00051         return -1;
00052     }
00053 };
00054
00055 double Wb::CallMatlab(
00056     const char *F, int L, const char* cmd, const char *arg1) {
00057
00058     double q=0; int e=0;
00059     mxArray *argout[1], *argin[1]={NULL};
00060
00061     if (!cmd || !cmd[0]) wblog(F,L,"ERR %s() got empty cmd",FCT);
00062
00063     if (Wb::my_caller_tid!=omp_get_thread_num()) {
00064         char s[16]=""; if (arg1) snprintf(s,16,"('%s')",arg1);
00065
00066         fprintf(stdout,
00067             "mexCallMATLAB() to %s%s only allowed main thread (%d/%d)",
00068             cmd, s, Wb::my_caller_tid, omp_get_thread_num()
00069         );
00070         #ifdef DBSTOP
00071             dbstop(FL);
00072         #endif
00073
00074         return (q=-1);
00075     }
00076
00077     if (arg1) {
00078         argin[0]=mxCreateString(arg1);
00079         try { e=mexCallMATLAB(1,argout,1,argin,cmd); }
00080         catch (...) { e=-1; }
00081
00082         mxDestroyArray(argin[0]);
00083     }
00084     else {
00085         try { e=mexCallMATLAB(1,argout,0,argin,cmd); }
00086         catch (...) { e=-2; }
00087     }
00088
00089     if (e) { int l=0, n=64; char s[n]; s[0]=0;
00090         if (e<0)
00091             l=snprintf(s,n,"invalid command » %s",cmd);
00092         else l=snprintf(s,n,"» %s",cmd);
00093
00094         if (l<n && arg1) l+=snprintf(s+l,n-l,"('%s')",arg1);
00095         if (l<n) l+=snprintf(s+l,n-l,
00096             e>0? " %% returned error e=%d":" %% (e=%d)",e);
00097
00098         if (F)
00099             wblog(F,L,"ERR %s %s",FCT,s);
00100         else wblog(FL, "WRN %s %s",FCT,s);
00101
00102         return q=0;
00103     }
00104
00105     mxGetNumber(argout[0],q);
00106     mxDestroyArray(argout[0]);
00107
00108     if (F) { unsigned l, n=32; char s[n];
00109         l=snprintf(s,n,"%s",cmd);
00110         if (arg1 && l<n) l+=snprintf(s+l,n-l,"('%s')",arg1);
00111         wblog(F,L, " * %s » %-24s %% %g",FCT,s,q);
00112     }

```



```

00113
00114     return q;
00115 };
00116
00117 void mxPutArray(const char *F, int L,
00118     mxArray *a, const char *vn, const char* ws, const char* dstr
00119 ){
00120     int i;
00121
00122     #pragma omp critical (using_MEX_API)
00123     i=mexPutVariable(ws, vn, a);
00124
00125     if (i) wblog(F_L,
00126         "ERR failed to put '%s' to '%s' (%d)",vn,ws,i);
00127
00128     if (dstr!=NULL) {
00129         if (!strcmp(dstr,"destroy")) mxDestroyArray(a);
00130         else if (dstr[0]!=0)
00131             wblog(F_L,"ERR invalid dstr '%s'", dstr);
00132     }
00133 }
00134
00135 void mexDisp(const mxArray *a, const char *vn) {
00136     if (vn && vn[0]) printf("\n%s = \n\n", vn);
00137     Wb::CallMatlab(0, NULL, 1, (mxArray**) &a,"display");
00138 };
00139
00140 void mexWRN(const char *s) {
00141
00142     unsigned n=32; char tag[n];
00143     snprintf(tag,n,"Wb:MEX:%.24s",myname);
00144
00145     mexWarnMsgIdAndTxt(tag, s && s[0] ? s : "");
00146 };
00147
00148 wbststring sprint_info(const mxArray *a) {
00149     wbststring s(64);
00150     unsigned l=0;
00151
00152     if (!a) { strcpy(s.data,"(null)"); }
00153     else {
00154         unsigned ndim=mxGetNumberOfDimensions(a);
00155         const size_t *dims=mxGetDimensions(a);
00156
00157         l+=snprintf(s.data,s.len,"%s array of type '%s'",
00158             SSTR(wbvector<size_t>(ndim,dims,'r')), mxGetClassName(a)
00159         );
00160     }
00161
00162     if (l>=s.len) wblog(FL,
00163         "WRN %s() string out of bounds (%d/%d)",FCT,l,s.len);
00164     return s;
00165 };
00166
00167 double wbtoc(
00168     const char *F, int L, const char *istr, char restart
00169 ){
00170     double t=-1.; mxArray *a;
00171
00172     if ((F==NULL) ? 1 : (F[0]==0 ? 1 : 0)) {
00173         Wb::CallMatlab(0, NULL, 0, NULL, "tic");
00174         return t;
00175     }
00176
00177     Wb::CallMatlab(1,&a,0,NULL,"toc");
00178     Mx::Array<double>(a).ncopy_to(&t,1);
00179     mxDestroyArray(a);
00180
00181     if (istr) {
00182         if (!istr[0])
00183             wblog(F_L,"elapsed time: %.6f sec", t);
00184         else wblog(F_L,"elapsed time: %.6f sec (%s)", t, istr);
00185     }
00186
00187     if (restart)
00188         Wb::CallMatlab(0, NULL, 0, NULL, "tic");
00189
00190     return t;
00191 };
00192
00193 template<class T>
00194 int getNumGlobal(const char *vn, T &x, const char* ws){
00195
00196     const mxArray *a=mexGetVariablePtr(ws, vn);
00197     if (!a || !Mx::IsNumber(0,0,a)) { return 1; }
00198     else {
00199         double dbl=0;

```

```

00200         Mx::Array<double>(a).ncopy_to(&dbl,1);
00201
00202         x=T(dbl);
00203         if ((double)x!=dbl) wblog(FL,
00204             "WRN () %s invalid value for type '%s' (%g)",FCT,TSTR(T),dbl);
00205
00206         return 0;
00207     }
00208 };
00209
00210 template<>
00211 int getNumGlobal(const char *vn, wbststring &s, const char *ws) {
00212     const mxArray *a=mexGetVariablePtr(ws, vn);
00213
00214     char istr[64];
00215
00216     if (!a || !mxIsChar(a)) return 1;
00217     if (mxGetString(a,istr,63)) return 1;
00218     s=istr;
00219
00220     return 0;
00221 }
00222
00223 #else
00224
00225 int Wb::CallMatlab(
00226     int nargout, mxArray *argout[], int nargin, mxArray *argin[],
00227     const char *fctname
00228 ){
00229     wblog(FL,"ERR %s(%d,%p,%d,%p,'%s')\nnot available outside MatLab",
00230         FCT,nargout,argout,nargin,argin,fctname?fctname:""); return 0;
00231 };
00232
00233 double Wb::CallMatlab(
00234     const char *F, int L, const char* cmd, const char *argl) {
00235
00236     char s[16]=""; if (argl) snprintf(s,16,('%s')",argl);
00237     wblog(F_L,
00238         "ERR %s -> %s%s\nnot available outside MatLab",
00239         FCT, cmd? cmd:"(cmd!?)",s);
00240     return 0;
00241 };
00242
00243 double wbtoc(
00244     const char *F, int L, const char *istr,
00245     char restart QS_UNUSED_VAR
00246 ){
00247     wblog(FL,"WRN %s(%s,%s) not available outside MatLab",
00248         FCT,shortFL(F_L),istr ? istr:""); return 0;
00249 }
00250
00251 void mxPutArray(const char *F, int L,
00252     mxArray *a, const char *vn, const char* ws,
00253     const char* dstr
00254 ){
00255     wblog(F_L,"ERR %s(%s,%s,%lX,%s) not available outside MatLab",
00256         FCT,vn?vn:"",ws?ws:"",&a,dstr?dstr:"");
00257 };
00258
00259 template<class T>
00260 int getNumGlobal(
00261     const char *vn    QS_UNUSED_VAR,
00262     T &x              QS_UNUSED_VAR,
00263     const char* ws    QS_UNUSED_VAR
00264 ){
00265     wblog(FL,"ERR %s(%s,%s) not available outside MatLab",
00266         FCT,vn?vn:"",ws?ws:""); return 0;
00267 };
00268
00269 void mexDisp(const mxArray *a, const char *vn) {
00270     wblog(FL,"WRN %s(%s,%lX) not available outside MatLab",FCT,vn?vn:"",a);
00271 };
00272
00273 void mexWRN(const char *s) {
00274     wblog(FL,"WRN %s() %s",myname,s?s:"");
00275 };
00276
00277 #endif
00278 #endif

```

5.7 memlib.cc

```

00001 /* -----

```

```

00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : QSpace memory routines
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_MEM_LIB_CC__
00021 #define __WB_MEM_LIB_CC__
00022
00023 // ----- //
00024 // ----- //
00025
00026 template <class T>
00027 wbstring Wb::sptr<T>::toStr() const {
00028
00029     wbstring s(128);
00030     unsigned l=0;
00031
00032     if (mtype!=Wb::MEM_DEF) {
00033         l+=snprintf(s.data+l,s.len-l,"%s ",Wb::MTYPE_STR[mtype]); }
00034
00035     if (data || len)
00036         { l+=snprintf(s.data+l,s.len-l,"%p %s[%.3g]",data,sTSTR(T),double(len)); }
00037     else { l+=snprintf(s.data+l,s.len-l,"%p",data); }
00038
00039     if (nref) {
00040         l+=snprintf(s.data+l,s.len-l," w/ %d refs",nref); }
00041
00042     return s;
00043 };
00044
00045 template <class T> inline
00046 T* Wb::sptr<T>::malloc_base(size_t n, char init, Wb::MTYPE mt) {
00047
00048     char do_alloc=(n>0);
00049     if (data) {
00050         if (check_consistency()) wblog(FL,"ERR Wb::sptr() %s",STR_(this));
00051         if (n!=len || nref) rm_dref();
00052         else {
00053             do_alloc=0;
00054         }
00055     }
00056
00057     if (do_alloc) {
00058         if (data || len) wblog(FL,"ERR %s() %s !?",FCT,STR_(this));
00059         len=n; mtype=mt;
00060
00061         if (mtype==Wb::MEM_DEF) { WB_NEW(data,n); } else
00062         if (mtype==Wb::MEX_RETURN) {
00063             if (typeid(T)==typeid(double)) {
00064                 mxArray *a=Mx::Array<double>(1,&n).Return();
00065                 data=(T*)mxGetDoubles(a);
00066                 P2X.add(data,a);
00067
00068                 nref=0;
00069             }
00070             else if (typeid(T)==typeid(wbcomplex)) {
00071                 mxArray *a=Mx::Array<wbcomplex>(1,&n).Return();
00072                 data=(T*)mxGetComplexDoubles(a);
00073                 P2X.add(data,a);
00074             }
00075             else wblog(FL,"ERR %s() got %s",FCT,TSTR(T));
00076
00077             if (WBLOG_MMX) wblog(FL,"new %s",STR_(this));
00078         }
00079     }
00080     if (n && init) { T z=T();
00081         for (size_t i=0; i<n; ++i) data[i]=z;
00082     }
00083
00084     if ((!len ^ !data) || (nref && !len)) wblog(FL,
00085         "ERR Wb::sptr<%s>=%p -> %s",sTSTR(T),this,STR_(this));
00086
00087     return data;
00088 };

```

```

00089
00090 template <class T> inline
00091 T* Wb::sptr<T>::malloc_data(size_t n, const T *d0, Wb::MTYPE mt) {
00092
00093     malloc_base(n,0,mt);
00094     if (n) {
00095         if (d0)
00096             MEM_COPY<T>(data,n,d0);
00097         else MEM_SET<T>(data,n);
00098     }
00099
00100     return data;
00101 };
00102
00103 template <class T> inline
00104 T* Wb::sptr<T>::init2ref(size_t n, const T* d0) {
00105
00106     if (data==d0 && len==n && nref) { return data; }
00107     if (data || len || nref) wblog(FL,"ERR %s() got %s",FCT,STR_(this));
00108     if (!d0 ^ !n) wblog(FL,"ERR %s() got n=%ld with %p",FCT,n,d0);
00109
00110     #pragma omp critical (manage_SPTR)
00111     { data=(T*)d0;
00112       len=n; mtype=Wb::MEM_REF;
00113       nref=1;
00114     }
00115     return data;
00116 };
00117
00118 template <class T> inline
00119 unsigned Wb::sptr<T>::add_dref() {
00120
00121     if (!data) { check_consistency(); return 0; }
00122
00123     #pragma omp critical (manage_SPTR)
00124     { ++nref; }
00125
00126     if (nref>128) wblog(FL,"WRN %s() %p @ nref=%d",FCT,data,nref);
00127
00128     return nref;
00129 };
00130
00131 template <class T> inline
00132 int Wb::sptr<T>::rm_dref(char mflag) {
00133
00134     int rval=0, e=0;
00135
00136     #pragma omp critical (manage_SPTR)
00137     { try {
00138
00139         if (!len ^ !data) { e|=1; }
00140         if (nref && !len) { e|=2; }
00141
00142         if (!e && data) {
00143             if (mtype==Wb::MEM_DEF) {
00144                 if (nref) { --nref; }
00145                 else {
00146                     WB_DELETE(data);
00147                     rval=1;
00148                 }
00149             }
00150             else if (mtype==Wb::MEM_REF) {
00151                 if (nref==1) { rval=2; }
00152                 else if (nref>1) { --nref; }
00153                 else { e|=16; }
00154             }
00155             else if (mtype==Wb::MEX_RETURN) {
00156                 if (!nref) { rval=3; }
00157                 else { e|=32; }
00158
00159                 if (mflag) {
00160                     mxArray *a=P2X.Return(0,0,data);
00161                     if (a) { mxDestroyArray(a); data=NULL; }
00162                 }
00163                 else if (P2X.erase(data)) { e|=64; }
00164             }
00165             else e|=128;
00166         }
00167     } catch (...) { e|=256; }}
00168
00169     if (e) {
00170         if (e==64)
00171             { wblog(FL,"WRN %s() got %s",FCT,STR_(this)); }
00172         else { wblog(FL,"ERR %s() got e=%d\n%s",FCT,e,STR_(this)); }
00173     }
00174
00175     if (rval) { init_def(); }

```

```

00176
00177     if (!data) { mtype=Wb::MEM_DEF; }
00178
00179     return rval;
00180 };
00181
00182 template <class T> inline
00183 T* Wb::sptr<T>::save_dref(const char *F, int L) {
00184
00185     T* dref=data; int e=0;
00186
00187     #pragma omp critical (manage_SPTR)
00188     { if (!len ^ !data) { e|=1; }
00189       if (nref || mtype) { e|=2; }
00190       else if (len) { len=0; data=NULL; }
00191     }
00192
00193     if (e) wblog(F_L,"ERR %s() %s got e=%d !?",FCT,STR_(this),e);
00194     return dref;
00195 };
00196
00197 template <class T1, class T2>
00198 bool gotMemOverlap(
00199     const T1* src, size_t len0,
00200     const T2* dest, size_t len2
00201 ){
00202     if (len0 && len2) {
00203         const char
00204             *L=(char*)src, *R=(char*)(src+len0)-1,
00205             *a=(char*)dest, *b=(char*)(dest+(long(len2)<0 ? len0 : len2))-1;
00206
00207         if (!src || !dest) wblog(F_L,
00208             "ERR %s() received null pointer (%p, %p)",FCT,src,dest);
00209         return ((a<=L && b>=L) || (a<=R && b>=R) || (a>=L && b<=R) );
00210     }
00211     return 0;
00212 };
00213
00214 #endif
00215

```

5.8 memlib.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : QSpace memory routines and smart pointers
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_MEM_LIB_HH__
00021 #define __WB_MEM_LIB_HH__
00022
00023 // ----- //
00024 // ----- //
00025
00026 template <class T1, class T2>
00027 bool gotMemOverlap(
00028     const T1* src, size_t len0,
00029     const T2* dest, size_t len2=-1
00030 );
00031
00032 template <class T>
00033 inline void MEM_CPY_BASE(T* data, size_t n, T const* const d=NULL) {
00034
00035     #ifdef MLB_VFLAG
00036         wblog(F_L,"TST base init");
00037     #endif
00038     if (n) {
00039         if (!d)
00040             { memset((void*)data,0,n*sizeof(T)); }

```

```

00041         else { memcpy((void*)data,d,n*sizeof(T)); }
00042     }
00043 };
00044
00045 template <class T>
00046 inline void MEM_COPY_BASE(
00047     T* data, size_t n, size_t n1, T const* const d1, T const* const d2
00048 ){
00049
00050 #ifdef MLB_VFLAG
00051     wblog(FL,"TST base init %d->%d",n1,n);
00052 #endif
00053     if (n1) {
00054         if (d1==NULL) wblog(FL,"ERR got (null) pointer"); else
00055             if (n1>n) { wblog(FL,"WRN length out of bounds (%d/%d)",n1,n); n1=n; }
00056         memcpy(data,d1,n1*sizeof(T));
00057     }
00058     if (n1<n) {
00059         if (d2) memcpy(data+n1, d2, (n-n1)*sizeof(T));
00060         else     memset((void*)(data+n1), 0, (n-n1)*sizeof(T));
00061     }
00062 };
00063
00064 namespace Wb {
00065
00066 enum MTYPE {
00067     MEM_DEF,
00068     MEM_REF,
00069     MEX_RETURN,
00070     MEM_NUM_TYPES
00071 };
00072
00073 const char* MTYPE_STR[MEM_NUM_TYPES] {
00074     "MEM:default",
00075     "MEM:ref",
00076     "MEX:return"
00077 };
00078
00079 template <class T>
00080 class sptr {
00081 public:
00082
00083     sptr() : mtype(MEM_DEF), len(0), nref(0), data(NULL) {};
00084
00085     ~sptr() {
00086         #pragma omp critical (manage_SPTR)
00087         { if (mtype || len || data || nref) wblog(FL,
00088             "WRN %s() got %s",FCT,STR_(this));
00089             init_def();
00090         }
00091     }
00092
00093     void init_def() {
00094         mtype=MEM_DEF; len=nref=0; data=NULL;
00095     }
00096
00097     T* malloc_base(size_t n, char init=1, MTYPE mt=MEM_DEF);
00098     T* malloc_data(size_t n, const T *d0=NULL, MTYPE mt=MEM_DEF);
00099
00100     int rm_dref(char mflag=0);
00101
00102     unsigned add_dref();
00103
00104     T* save_dref(const char *F=0, int L=0);
00105
00106     T* init2ref(size_t n, const T* d0);
00107
00108     int isRef() const {
00109         return (nref || mtype==Wb::MEM_REF);
00110     };
00111
00112     explicit operator bool() const {
00113         check_consistency(); return (len!=0); };
00114     bool operator! () const {
00115         check_consistency(); return (len==0); };
00116
00117     int check_consistency(char force=1) const {
00118         int e=0;
00119
00120         #pragma omp critical (manage_SPTR)
00121         { if (!len ^ !data) { e|=1; }
00122             if (nref && !len) { e|=2; }
00123         }
00124
00125         if (e && force) wblog(FL,
00126             "ERR Wb::sptr() inconsistency (%p @ n=%d, r=%d)",data,len,nref);

```

```

00128         return e;
00129     };
00130
00131     wstring toStr() const;
00132
00133     MTPE mtype;
00134
00135     size_t len;
00136     unsigned nref;
00137
00138     T* data;
00139
00140     protected:
00141     private:
00142 };
00143
00144 };
00145
00146 template <class T, ENABLE_IF_isPOD(T) >
00147 void MEM_COPY(T* data, size_t n, const T* d=NULL) {
00148     MEM_COPY_BASE(data,n,d);
00149 };
00150
00151 template <class T, ENABLE_IF_noPOD(T) >
00152 void MEM_COPY(T* data, size_t n, const T* d=NULL) {
00153
00154     #ifdef MLB_VFLAG
00155         wlog(FL,"TST class init");
00156     #endif
00157
00158     if (d) { for (size_t i=0; i<n; ++i) data[i]=d[i]; }
00159     else {
00160         const T z=T();
00161         for (size_t i=0; i<n; ++i) data[i]=z;
00162     }
00163 };
00164
00165 template <class T, ENABLE_IF_isPOD(T) >
00166 void MEM_COPY(T* data, size_t n, size_t n1, const T* d1, const T* d2) {
00167     MEM_COPY_BASE(data,n,n1,d1,d2);
00168 };
00169
00170 template <class T, ENABLE_IF_noPOD(T) >
00171 void MEM_COPY(T* data, size_t n, size_t n1, const T* d1, const T* d2) {
00172     #ifdef MLB_VFLAG
00173         wlog(FL,"TST class init %d->%d",n1,n);
00174     #endif
00175
00176     size_t i=0;
00177     if (n1) {
00178         if (d1==NULL) wlog(FL,"ERR got (null) pointer"); else
00179         if (n1>n) { wlog(FL,
00180             "WRN length out of bounds (%d->%d)",n1,n); n1=n; }
00181         for (; i<n1; ++i) data[i]=d1[i];
00182     }
00183     if (d2) { for (; i<n; ++i) { data[i]=d2[i-n1]; }}
00184     else {
00185         const T z=T();
00186         for (; i<n; ++i) data[i]=z;
00187     }
00188 };
00189
00190 template <class T>
00191 void MEM_COPY(T** data, size_t n, T* const* const d=NULL) {
00192     #ifdef MLB_VFLAG
00193         wlog(FL,"TST got pointer space");
00194     #endif
00195     MEM_COPY_BASE(data,n,d);
00196 };
00197
00198 template <class T>
00199 void MEM_COPY(T** data, size_t n, size_t n1,
00200     T* const* const d1, T* const* const d2
00201 ){
00202     #ifdef MLB_VFLAG
00203         wlog(FL,"TST got pointer space");
00204     #endif
00205     MEM_COPY_BASE(data,n,n1,d1,d2);
00206 };
00207
00208 template <class T> inline
00209 void MEM_SET_BASE(T* data, size_t len, const T *a=NULL) {
00210     #ifdef MLB_VFLAG
00211         wlog(FL,"TST base memset");
00212     #endif
00213     if (!a)
00214         { memset((void*)data,0,len*sizeof(T)); }

```

```

00215     else { for (size_t i=0; i<len; ++i) data[i]=(*a); }
00216 };
00217
00218 template <class T> inline
00219 void MEM_SET_BASE(T** data, size_t len, T* const *a=NULL) {
00220 #ifdef MLB_VFLAG
00221     wblog(FI,"TST base memset (pointer space)");
00222 #endif
00223     if (!a)
00224     { memset((void*)data,0,len*sizeof(T*)); }
00225     else { for (size_t i=0; i<len; ++i) data[i]=(*a); }
00226 };
00227
00228 template <class T>
00229 class MEM_SET {
00230
00231 public:
00232
00233     MEM_SET(
00234         T* data      QS_UNUSED_VAR,
00235         size_t len    QS_UNUSED_VAR,
00236         const T *a    QS_UNUSED_VAR = NULL
00237     ){
00238     };
00239
00240 private:
00241
00242 };
00243
00244 template <class T>
00245 class MEM_SET<T*> {
00246
00247 public:
00248
00249     MEM_SET(T** data, size_t len, T* const *a=NULL) {
00250 #ifdef MLB_VFLAG
00251         wblog(FI,"TST pointer memset");
00252 #endif
00253         MEM_SET_BASE(data,len,a);
00254     }
00255
00256 private:
00257
00258 };
00259
00260 template <> inline MEM_SET<int>::MEM_SET(
00261     int* data, size_t len, const int *a
00262 ){ MEM_SET_BASE(data,len,a); }
00263
00264 template <> inline MEM_SET<unsigned>::MEM_SET(
00265     unsigned* data, size_t len, const unsigned *a
00266 ){ MEM_SET_BASE(data,len,a); }
00267
00268 template <> inline MEM_SET<unsigned long>::MEM_SET(
00269     unsigned long* data, size_t len, const unsigned long *a
00270 ){ MEM_SET_BASE(data,len,a); }
00271
00272 template <> inline MEM_SET<long>::MEM_SET(
00273     long* data, size_t len, const long *a
00274 ){ MEM_SET_BASE(data,len,a); }
00275
00276 template <> inline MEM_SET<char>::MEM_SET(
00277     char* data, size_t len, const char *a
00278 ){ MEM_SET_BASE(data,len,a); }
00279
00280 template <> inline MEM_SET<double>::MEM_SET(
00281     double* data, size_t len, const double *a
00282 ){ MEM_SET_BASE(data,len,a); }
00283
00284 template <> inline MEM_SET<float>::MEM_SET(
00285     float* data, size_t len, const float *a
00286 ){ MEM_SET_BASE(data,len,a); }
00287
00288 template <> inline MEM_SET<wbcomplex>::MEM_SET(
00289     wbcomplex* data, size_t len, const wbcomplex *a
00290 ){ MEM_SET_BASE(data,len,a); }
00291
00292 #endif
00293

```

5.9 memtrack.hh

```

00001 /* -----

```



```

00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : QSpace memory routines
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_MEM_TRACK_HH__
00021 #define __WB_MEM_TRACK_HH__
00022
00023 // Wb,Jul27,12
00024
00025 namespace Wb {
00026     void MemStat(const char *F=0, int L=0, char lflag=0);
00027 };
00028
00029 #if !(defined __WBDEBUG__ || defined __WB_MEM_CHECK__ )
00030
00031 #define WB_NEW(p,n) Wb::NEW(p,n)
00032 #define WB_NEW_1(p) Wb::NEW_1(p);
00033
00034 #define WB_NEW_2(p,x) { p = new x; \
00035     if (!p) wblog(__FILE__,__LINE__,"ERR failed to allocate instance"); \
00036 }
00037
00038 #define WB_DELETE(p) if (p) { delete [] p; p=NULL; }
00039 #define WB_DELETE_1(p) if (p) { delete p; p=NULL; }
00040
00041 namespace Wb {
00042
00043     template<class T>
00044     inline void NEW(T* &p, const size_t &n) {
00045
00046         if (n) {
00047             { try { p = new T[n]; } catch (...) { p=NULL; } }
00048
00049             if (!p) { printf("\n");
00050                 MemStat(FL);
00051                 wblog(FL,"ERR out of memory (%ld * %d = %.1fG) !?",
00052                     n, sizeof(T), n*sizeof(T)/double(1<<30)
00053                 );
00054             }
00055         }
00056         else { p=NULL; }
00057     };
00058
00059     template<class T>
00060     inline void NEW_1(T* &p) {
00061         try { p = new T; } catch (...) { p=NULL; }
00062
00063         if (!p) { printf("\n");
00064             MemStat(FL);
00065             wblog(FL,"ERR failed to allocate instance !?");
00066         }
00067     };
00068
00069 };
00070
00071 #else
00072
00073 #define WB_NEW(p,n) Wb::NEW(__FILE__,__LINE__,p,n);
00074 #define WB_NEW_1(p) Wb::NEW_1(__FILE__,__LINE__,p);
00075
00076 #define WB_NEW_2(p,x) { p = new x; \
00077     if (!p) wblog(__FILE__,__LINE__,"ERR failed to allocate instance"); \
00078 }
00079
00080 #define WB_DELETE(p) \
00081     if (p) { Wb::gML.rm_ptr(__FILE__,__LINE__,p); delete [] p; p=NULL; }
00082
00083 #define WB_DELETE_1(p) \
00084     if (p) { Wb::gML.rm_ptr(__FILE__,__LINE__,p); delete p; p=NULL; }
00085
00086 #include "memtrack.cc"
00087
00088 #endif

```

```

00089
00090 #endif
00091

```

5.10 mexlib.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : QSpace MEX routines
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_MEXLIB_CC__
00021 #define __WB_MEXLIB_CC__
00022
00023 // also using mxPutAndDestroy() outside MEX file
00024 // by writing data to file to capture debug data // Wb,Oct25,16
00025
00026 void mxPutAndDestroy(
00027     const char *F, int L, mxArray *a, const char *vn,
00028     const char* ws
00029 ){
00030     int i=0;
00031     if (!ws || !ws[0]) wblog(FL,"ERR %s() invalid workspace `%s'",FCT,ws);
00032     if (!strcmp(ws,"caller") || !strcmp(ws,"base")) { i=1; }
00033
00034     #ifndef MATLAB_MEX_FILE
00035         if (i>0) { i=-1; }
00036     #endif
00037     if (i>0 && Wb::my_caller_tid!=omp_get_thread_num()) { i=-2; }
00038
00039     if (i<0) {
00040         char tmp[64];
00041         tmpmatFL(F_L,vn && vn[0] && !strcmp(vn,"ans")? vn:ws, tmp,64);
00042         wblog(F_L,"WRN thread %d/%d cannot put `%s' to %s (i=%d)",
00043             omp_get_thread_num(), omp_get_num_threads(), vn?vn:"",ws,i);
00044
00045         if (!strcmp(tmp,"caller") || !strcmp(tmp,"base")) wblog(FL,
00046             "ERR %s() invalid tmp=`%s' !?",FCT,tmp);
00047         mxPutAndDestroy(F,L,a,vn,tmp);
00048         return;
00049     }
00050
00051     #ifdef QS_USING_OMP
00052         Wb::ompGuard myLK(mxapi_lock);
00053     #endif
00054
00055     if (i) {
00056         i=mxPutVariable(ws,vn,a);
00057         if (i) wblog(F_L,
00058             "ERR mexPutVariable('%s','%s',..) returned e=%d !?",ws,vn,i);
00059         if (F) wblog(F_L,"I/O putting `%s' to %s",vn,ws);
00060     }
00061     else {
00062         wbsting tmp;
00063         if (!ws || !ws[0] || !strcmp(ws,"tmpfile")) {
00064             tmp.init(64); tmpmatFL(F_L,
00065                 vn && vn[0] && strcmp(vn,"ans")? vn:"", tmp.data,tmp.len);
00066             ws=tmp.data; i=1;
00067         }
00068         else {
00069             unsigned j=0, k=0;
00070             for (; ws[j]; ++j) { if (ws[j]=='.') { k=j+1; } }
00071             if (!k || strcasecmp(ws+k,"mat")) {
00072                 tmp.init(k+6); snprintf(tmp.data,tmp.len,"%s.mat",ws);
00073                 ws=tmp.data;
00074             }
00075         }
00076     }
00077     if (F) wblog(F_L,"%s saving `%s' to `%s'",i?"TMP":"I/O",vn,ws);

```

```

00078
00079     char x[2]; strcpy(x, Wb::isFile(ws)? "u":"w");
00080     Wb::matFile f(F_L,ws,x);
00081
00082     matPutVariable(f.mfp,vn,a);
00083 }
00084
00085     mxDestroyArray(a);
00086 };
00087
00088 int Mx::IsNumArray(
00089     const char *F, int L, const mxArray* a,
00090     unsigned r,
00091     char cflag,
00092     char type
00093 ){
00094     int rval=0;
00095
00096     if (!a) {
00097         if (L) {
00098             strcpy(str,"got a=null");
00099             if (F) wblog(F,L,"ERR %s() %s",FCT,str);
00100         }
00101         return (rval=-128);
00102     }
00103
00104     if (mxIsDouble(a) ) { rval|= 2; } else
00105     if (mxIsNumeric(a)) { rval|= 8; } else
00106     if (mxIsChar(a) ) { rval|=16; } else
00107     if (mxIsLogical(a) ) { rval|=32; }
00108
00109     if (rval && mxIsComplex(a)) { rval|=4; }
00110
00111     if (type=='d') {
00112         if (!(rval & 2)) {
00113             if (L) {
00114                 sprintf_str("got data of type %s",mxGetClassName(a));
00115                 if (F) wblog(F,L,"ERR %s() %s",FCT,str);
00116             }
00117             return (rval=-129);
00118         }
00119     }
00120     else if (type=='*') {
00121         if (!(rval & 58)) {
00122             if (L) {
00123                 sprintf_str("got data of type %s",mxGetClassName(a));
00124                 if (F) wblog(F,L,"ERR %s() %s",FCT,str);
00125             }
00126             return (rval=-130);
00127         }
00128     }
00129     else wblog(F_L,"ERR %s() invalid type=%d<c>",FCT,type,type);
00130
00131     if (mxIsSparse(a)) {
00132         if (L) {
00133             strcpy(str,"got sparse input");
00134             if (F) wblog(F,L,"ERR %s() %s",FCT,str);
00135         }
00136         return (rval=-131);
00137     }
00138
00139     if (!cflag && (rval & 4)) {
00140         if (L) {
00141             strcpy(str,"got complex input");
00142             if (F) wblog(F,L,"ERR %s() %s",FCT,str);
00143         }
00144         return (rval=-132);
00145     }
00146
00147     if (int(r)>0) {
00148         unsigned ra=mxGetNumberOfDimensions(a);
00149         if (ra>r) {
00150             if (ra==2) {
00151                 const size_t *D=mxGetDimensions(a);
00152                 if (D[0]==1 || D[1]==1) { ra=1; }
00153             }
00154             if (ra>r) {
00155                 if (L) {
00156                     sprintf_str("got invalid rank (r=%d/%d)",ra,r);
00157                     if (F) wblog(F,L,"ERR %s() %s",FCT,str);
00158                 }
00159                 return (rval=-ra);
00160             }
00161         }
00162     }
00163     rval|=1;
00164

```

```

00165     return rval;
00166 };
00167
00168 template <class T>
00169 Mx::Array<T>::Array(const wvector<size_t> &S, char t_, char c_)
00170 : ax(0), data(0), len(0), trans(t_), cmplx(c_), mxref(0) {
00171
00172     if (cmplx<0) { cmplx=ISCOMPLX_(T); }
00173     init(S.len,S.data);
00174 };
00175
00176 template <class T>
00177 int Mx::Array<T>::init_dpnr() {
00178
00179     data=0; if (!ax) { return -1; }
00180
00181     if (mxref<=0) wblog(FL,"ERR %s() got ref=%d !?",FCT,mxref);
00182     if (mxIsSparse(ax)) wblog(FL,
00183         "ERR Mx::Array got invalid sparse `%s'",mxGetClassName(ax));
00184
00185     mxClassID t=mxGetClassID(ax); int e=0;
00186
00187     if (!mxIsComplex(ax)) {
00188         switch (t) {
00189             case mxDOUBLE_CLASS : safe_set_dpnr( mxGetDoubles (ax)); break;
00190             case mxCHAR_CLASS   : safe_set_dpnr( mxGetChars   (ax)); break;
00191             case mxINT8_CLASS    : safe_set_dpnr( mxGetInt8s   (ax)); break;
00192             case mxUINT8_CLASS   : safe_set_dpnr( mxGetUint8s  (ax)); break;
00193             case mxINT16_CLASS   : safe_set_dpnr( mxGetInt16s  (ax)); break;
00194             case mxUINT16_CLASS  : safe_set_dpnr( mxGetUint16s (ax)); break;
00195             case mxINT32_CLASS   : safe_set_dpnr( mxGetInt32s  (ax)); break;
00196             case mxUINT32_CLASS  : safe_set_dpnr( mxGetUint32s (ax)); break;
00197             case mxINT64_CLASS   : safe_set_dpnr( mxGetInt64s  (ax)); break;
00198             case mxUINT64_CLASS  : safe_set_dpnr( mxGetUint64s (ax)); break;
00199             case mxLOGICAL_CLASS : safe_set_dpnr( mxGetLogicals(ax)); break;
00200             default: e=1; break;
00201         }
00202     }
00203     else {
00204         if (t==mxDOUBLE_CLASS) safe_set_dpnr( mxGetComplexDoubles(ax) );
00205         else { e=2; }
00206     }
00207
00208     if (e && !mxIsNumChar(ax))
00209         wblog(FL,"ERR %s got invalid %s `%s' (e=%d)",xStr(),
00210             mxIsSparse(ax) ? "sparse":"type", mxGetClassName(ax),e);
00211     if (e==2 && !data)
00212         wblog(FL,"ERR %s got complex `%s'",xStr(),mxGetClassName(ax));
00213     if (e) wblog(FL,"WRN %s not match found for %s",xStr(),TSTR(T));
00214
00215     if (!e) { mxref+=8; }
00216
00217     return (e<0 ? e : (data ? 0 : 1));
00218 };
00219
00220 template<class T>
00221 Mx::Array<T>& Mx::Array<T>::init(
00222     unsigned ndim, size_t *dims, char t_, char c_) {
00223
00224     if (ax) { mxDestroyArray(ax); ax=NULL; data=NULL; }; mxref=0;
00225
00226     if (t_>=0) { trans=t_; }
00227     if (c_>=0) { cmplx=c_; }
00228     if ((rank=ndim)) {
00229         len=dims[0]; for (unsigned i=1; i<ndim; ++i) { len*=dims[i]; }
00230     } else len=0;
00231
00232     MX_INIT_DATA(ndim,dims);
00233     init_xstr();
00234
00235     if (ax && (cmplx ^ mxIsComplex(ax))) {
00236         if (cmplx) { cmplx=0; wblog(FL,
00237             "WRN %s ignoring cmplx=%d for %s",xStr(),mxGetClassName(ax)); }
00238         else wblog(FL,
00239             "ERR %s complex mismatch (%d/%d)",xStr(),cmplx,mxIsComplex(ax));
00240     }
00241
00242     return *this;
00243 };
00244
00245 template<class T>
00246 Mx::Array<T>& Mx::Array<T>::init(const mxArray *a, char t_, char c_) {
00247
00248     int e=0;
00249     if (ax) { if (mxref<=0) mxDestroyArray(ax); data=NULL; }
00250     mxref=1; len=rank=0;
00251

```

```

00252     if (t_>=0) { trans=t_; }
00253     if (c_>=0) { cmplx=c_; }
00254
00255     ax=(mxArray*)a; if (!a) { return *this; }
00256
00257     e=init_dptra();
00258     if (e<0) wblog(FL,
00259         "ERR %s() %s <> %s => e=%d !?",FCT,mxGetClassName(ax),TSTR(T),e);
00260
00261     len=mxGetNumberOfElements(a);
00262     rank=mxGetNumberOfDimensions(a); init_xstr();
00263
00264     if (ax) { bool x=mxIsComplex(ax);
00265         if (cmplx<0) { cmplx=x; } else
00266             if (cmplx ^ x) { const char *s=xStr();
00267                 if (cmplx) {
00268                     if (WBLOG_TCAST) wblog(FL,
00269                         "WRN %s unsetting cmplx->0 for %s%s",s,
00270                         mxIsComplex(ax)? "complex ":"", mxGetClassName(ax));
00271                     cmplx=0;
00272                 }
00273                 else wblog(FL,"ERR %s complex type mismatch (%d/%d)",s,cmplx,x);
00274             }
00275     }
00276
00277     return *this;
00278 };
00279
00280 template <class Ta, class Tb>
00281 size_t copy_row2col_major(
00282     size_t r, const size_t *S, Ta *a, const Tb *b,
00283     char from
00284 ) {
00285     if (!r || !S) {
00286         wblog(FL,"WRN %s() r=%d / sz=%p !?",FCT,r,S);
00287         return 0;
00288     }
00289     if (!a || !b) wblog(FL,
00290         "ERR %s() got null pointers (%p/%p) !?",FCT,a,b);
00291
00292     unsigned i=0, l=r-1;
00293     size_t j, k=0, len=S[0], I[r]; for (; i<r; ++i) { I[i]=0; }
00294
00295     for (i=1; i<r; ++i) len*=S[i];
00296
00297     for (k=0; k<len; ++k) {
00298         for (j=I[0], i=1; i<r; ++i) j = j*S[i] + I[i];
00299
00300         if (from)
00301             { a[k]=Ta(b[j]); }
00302         else { a[j]=Ta(b[k]); }
00303
00304         i=0; ++I[0];
00305         while (I[i]>=S[i] && i<l) { I[i]=0; ++I[++i]; }
00306     }
00307
00308     return len;
00309 };
00310
00311 template<class T>
00312 template <class Tb>
00313 int Mx::Array<T>::copy_from(const Tb* b, char tcheck) {
00314
00315     if (!data || !ax) { if (data || b) wblog(FL,
00316         "ERR %s() got null data (%p, %p, %p)",FCT,ax,data,b);
00317         return 0;
00318     }
00319
00320     if (!trans) {
00321         Wb::cpyRange(data,b,len,tcheck);
00322     }
00323     else {
00324         size_t l=
00325             copy_row2col_major(
00326                 rank, mxGetDimensions(ax), data, b,
00327                 'b'
00328             );
00329         if (l!=len) wblog(FL,"ERR %s() %d/%d",FCT,l,len);
00330     }
00331
00332     return (len ? 1 : 0);
00333 };
00334
00335 template<class T>
00336 size_t Mx::Array<T>::rcopy_to(
00337     T* b, const wbpem *P, char ck) const {
00338

```

```

00339     size_t l=0;
00340
00341     if (!ax) { if (data || b) wblog(FL,
00342         "ERR %s() got ax=%p (%p, %p)", FCT, ax, data, b);
00343         return l;
00344     }
00345     if (mxIsComplex(ax)) wblog(FL,
00346         "ERR %s got complex `%s'", xStr(), mxGetClassName(ax));
00347
00348     if (data) {
00349         return l=cpy_to_(b,P,ck,data);
00350     }
00351
00352     mxClassID t=mxGetClassID(ax);
00353     switch (t) {
00354         case mxDOUBLE_CLASS : l=cpy_to_(b,P,ck,mxGetDoubles (ax)); break;
00355         case mxCHAR_CLASS   : l=cpy_to_(b,P,ck,mxGetChars   (ax)); break;
00356         case mxINT8_CLASS   : l=cpy_to_(b,P,ck,mxGetInt8s   (ax)); break;
00357         case mxUINT8_CLASS  : l=cpy_to_(b,P,ck,mxGetUint8s  (ax)); break;
00358         case mxINT16_CLASS  : l=cpy_to_(b,P,ck,mxGetInt16s  (ax)); break;
00359         case mxUINT16_CLASS : l=cpy_to_(b,P,ck,mxGetUint16s (ax)); break;
00360         case mxINT32_CLASS  : l=cpy_to_(b,P,ck,mxGetInt32s  (ax)); break;
00361         case mxUINT32_CLASS : l=cpy_to_(b,P,ck,mxGetUint32s (ax)); break;
00362         case mxINT64_CLASS  : l=cpy_to_(b,P,ck,mxGetInt64s  (ax)); break;
00363         case mxUINT64_CLASS : l=cpy_to_(b,P,ck,mxGetUint64s (ax)); break;
00364         case mxLOGICAL_CLASS: l=cpy_to_(b,P,ck,mxGetLogicals(ax)); break;
00365         default: wblog(FL,"ERR %s unexpected type `%s'", xStr(), mxGetClassName(ax));
00366     }
00367
00368     if (l!=len) wblog(FL,"ERR %s len=%d/%d !?", xStr(), l, len);
00369     return l;
00370 };
00371
00372 namespace Mx {
00373
00374 template<>
00375 size_t Array<wbcomplex>::zcopy_to(
00376     wbcomplex *b, const wbpem *P, char ck) const {
00377
00378     size_t l=0;
00379
00380     if (!ax) { if (data || b) wblog(FL,
00381         "ERR %s() got null data (%p -> %p, %p)", FCT, ax, data, b);
00382         return l;
00383     }
00384
00385     if (data) { return l=cpy_to_(b,P,ck,data); }
00386
00387     mxClassID t=mxGetClassID(ax);
00388     if (t==mxDOUBLE_CLASS) {
00389         if (mxIsComplex(ax))
00390             l=cpy_to_(b,P,ck,(wbcomplex*)mxGetComplexDoubles(ax));
00391         else l=cpy_to_(b,P,ck,mxGetDoubles(ax));
00392     }
00393     else wblog(FL,"ERR %s() %s %s",FCT,
00394         mxIsComplex(ax) ? "complex": "type", mxGetClassName(ax));
00395
00396     if (l!=len) wblog(FL,"ERR %s len=%d/%d !?", xStr(), l, len);
00397     return l;
00398 }
00399
00400 template <>
00401 size_t Mx::Array<wbcomplex>::copy_to(wbcomplex *b, const wbpem *P, char tcheck)
00402 const { return zcopy_to(b,P,tcheck); };
00403
00404 };
00405
00406 template<class T>
00407 template<class Tx>
00408 size_t Mx::Array<T>::cpy_to_(
00409     T* b, const wbpem *P, char ck, const Tx *xd) const {
00410
00411     if (P && trans) wblog(FL,
00412         "ERR %s() got trans=%d together with permutation", FCT, trans);
00413     if (data && (void*)data!=(void*)xd) wblog(FL,
00414         "ERR %s got pointer mismatch %p / %p\n%s", xStr(), data, xd, STR_(this));
00415
00416     if (!trans && !P) {
00417         Wb::cpyRange(b,xd,len,ck);
00418         return len;
00419     }
00420
00421     if (!ax) wblog(FL,"ERR %s() %s",FCT,STR_(this));
00422
00423     if (trans) {
00424         size_t l=
00425         copy_row2col_major(

```

```

00426         rank, mxGetDimensions(ax), b, xd,
00427         0
00428     );
00429     if (l!=len) wblog(FL,"ERR %s() %ld/%ld",FCT,l,len);
00430 }
00431 else {
00432     size_t i=-1,j;
00433     unsigned k, l=rank-1;
00434     wbIndex I(rank);
00435
00436     const size_t *s0=mxGetDimensions(ax), *p=P->data;
00437
00438     if (!rank || P->len!=rank || len!=(i=mxGetNumberOfElements(ax)))
00439         wblog(FL,"ERR %s() size mismatch (r=%ld/%ld, len=%ld/%ld)",
00440             FCT, rank, P->len, len, i);
00441
00442     for (i=0; i<len; ++i) {
00443         for (j=I[p[l]], k=l-1; k<l; --k) j = j*s0[p[k]] + I[p[k]];
00444
00445         b[j]=T(xd[i]);
00446
00447         if (ck && Tx(b[j])!=xd[i]) wblog(FL,
00448             "ERR %s() rounding error (%g/%g)",FCT,double(b[j]),double(xd[i]));
00449
00450         k=0; ++I[0];
00451         while (I[k]>=s0[k] && k<l) { I[k]=0; ++I[++k]; }
00452     }
00453 }
00454
00455 return len;
00456 };
00457
00458 bool Mx::IsIndex(const mxArray *a, int base) {
00459     if (!a || !mxIsNumeric(a) || mxIsComplex(a)) { return 0; }
00460
00461     const size_t *s=mxGetDimensions(a);
00462     int r=mxGetNumberOfDimensions(a);
00463
00464     if (r!=2 || (s[0]!=1 && s[1]!=1)) { return 0; }
00465
00466     const double *d=mxGetDoubles(a);
00467     if (d) { return IsIndex(d,s[0]*s[1],base); }
00468     else {
00469         wvector<int> x(s[0]*s[1]);
00470         Mx::Array<int> (a).rcopy_to(x.data,0,'!');
00471         return IsIndex(x.data,x.len,base);
00472     }
00473 };
00474
00475 template<class T>
00476 wstring Mx::Array<T>::toStr(char vflag, unsigned l) const {
00477
00478     wstring s(l);
00479
00480     if (ax) {
00481         unsigned i=0;
00482         const size_t *S=mxGetDimensions(ax);
00483
00484         l=snprintf(s.data,s.len,"%s%s ",
00485             mxIsComplex(ax)? "complex ":"",mxGetClassName(ax));
00486         for (; i<rank && l<s.len; ++i) {
00487             l+=snprintf(s.data+l,s.len-l,"%ldx",S[i]); }
00488
00489         if (l<s.len) { if (i) { --l; }
00490             l+=snprintf(s.data+l,s.len-l," [%s",TSTR(T)); }
00491         if (l<s.len) {
00492             l+=snprintf(s.data+l,s.len-l," @ ");
00493             if (!data) { l+=snprintf(s.data+l,s.len-l,"d=null; "); } else
00494             if (vflag) { l+=snprintf(s.data+l,s.len-l,"d=%p; ",data); }
00495         }
00496         if (l<s.len) {
00497             l+=snprintf(s.data+l,s.len-l,"%d,%d,%d; l=%ld]",
00498                 cmplx,trans,mxref,len);
00499         }
00500
00501         if (l>=s.len) wblog(FL,"ERR %s() string out of bounds "
00502             "(%d/%d) %N' %s'",FCT,l,s.len,s.data);
00503     }
00504     else {
00505         if (vflag) snprintf(s.data,s.len,
00506             "ax=%p -> %p (%s; %d,%d,%d; len=%ld)",ax,data,TSTR(T),
00507             cmplx,trans,mxref,len);
00508         else snprintf(s.data,s.len,
00509             "null%s (%s; %d,%d,%d; n=%ld)", data ? " -> d!=0:":"",
00510             TSTR(T), cmplx,trans,mxref,len);
00511     }
00512 }

```

```

00513
00514     return s;
00515 };
00516
00517 template<class T>
00518 wbsttring Mx::Array<T>::toStrT(char vflag) const {
00519
00520     wbsttring s(24);
00521     unsigned l=0;
00522
00523     if (!data || vflag) l=snprintf(s.data,s.len,"%s%s -> %s",
00524         mxIsComplex(ax)? "complex ":"",mxGetClassName(ax),TSTR(T));
00525     else l=snprintf(s.data,s.len,"%s",TSTR(T));
00526
00527     if (l>=s.len) wblog(FL,
00528         "ERR %s() string out of bounds (%d/%d)%N'%s'",FCT,l,s.len,s.data);
00529     return s;
00530 };
00531
00532 template<class T>
00533 void Mx::Array<T>::issue_type_error(
00534     const char *F, int L, unsigned ndim, const size_t *dims) {
00535
00536     if (!ax || !mxGetNumberOfElements(ax)) {
00537         if (ndim) {
00538             size_t l=dims[0]; for (unsigned i=0; i<ndim; ++i) { l*=dims[i]; }
00539             if (l!=len) wblog(F_L,
00540                 "ERR %s length mismatch (%ld/%ld)",xStr(),l,len);
00541             if (l) wblog(FL,
00542                 "ERR %s failed to allocate memory\n%s (%ld)",xStr(),
00543                 SSTR(wbvector<size_t>(ndim,dims,'r')), len
00544             );
00545         }
00546         if (len || data) wblog(F_L,"ERR %s got null (%d;%p)",xStr(),len,data);
00547     }
00548     else {
00549         wblog(F_L, "ERR %s type mismatch (%s%s/%s)",xStr(),
00550             mxIsComplex(ax) ? "complex ":"",mxGetClassName(ax),TSTR(T));
00551     }
00552 };
00553
00554 template<class T> inline
00555 int mxGetNumber(const mxArray *a, T &d, const char qflag) {
00556
00557     int n=0; d=T(0);
00558     if (!a) { Wb::quietErrLog(FL,"mxArray is NULL",qflag); return 1; }
00559
00560     try {
00561         Mx::Array<T> A(a); n=A.len;
00562         A.ncopy_to(&d,1,1);
00563     }
00564     catch (...) { n=-1; }
00565
00566     if (n==1) { return 0; }
00567     else {
00568         unsigned l,m=64; char msg[m];
00569         l=snprintf(msg,m,"%s %s() invalid input",SHORT_FL,FCT);
00570         if (n>1) { l+=snprintf(msg+l,m-l," (got array, n=%d)",n); } else
00571         if (!n) { l+=snprintf(msg+l,m-l," (empty)"); }
00572         Wb::quietErrLog(FL,msg,qflag);
00573         return 1;
00574     }
00575 };
00576
00577 inline int mxGetString(const mxArray *a, char *s) {
00578     if (mxGetString(a,s,127)) {
00579         sprintf_str("%s:%d ERR could not read string (%s) !?", FL, s);
00580         return 1;
00581     }
00582     return 0;
00583 }
00584
00585 inline int mxGetString(const mxArray *a, wbsttring &s) {
00586     char istr[128]; istr[0]=0;
00587     if (mxGetString(a,istr,127)) {
00588         sprintf_str("%s:%d ERR could not read string (%s) !?", FL, istr);
00589         return 1;
00590     }
00591     s=istr; return 0;
00592 }
00593
00594 wbsttring mxSize2Str(const mxArray *a) {
00595     if (!a) { return "(null)"; }
00596
00597     size_t l=0, n=32; char s[n];
00598     int i=0, r=mxGetNumberOfDimensions(a);
00599     const size_t *S=mxGetDimensions(a);

```



```

00600
00601     for (; i<r; ++i) {
00602         l+=snprintf(s+l,n-l,"%s%d", i ? "x":"", unsigned(S[i]));
00603         if (l>=n) wblog(FL,"ERR %s() string out of bounds (%d/%d)",FCT,l,n);
00604     }
00605     return s;
00606 };
00607
00608 wbststring mxTypeSize2Str(const mxArray *a) {
00609     size_t n=64; char s[n];
00610     if (!a) { strcpy(s,"(null)"); return s; }
00611
00612     size_t l=snprintf(s,n,"%s: ",mxGetClassName(a));
00613     int r=mxGetNumberOfDimensions(a);
00614     const size_t *S=mxGetDimensions(a);
00615
00616     for (int i=0; i<r; ++i) {
00617         l+=snprintf(s+l,n-l,"%s%d",i?"x":"", (unsigned)S[i]);
00618         if (l>=n) wblog(FL,
00619             "ERR %s() string out of bounds (%d/%d)\n`%s'",FCT,l,n,s);
00620     }
00621     return s;
00622 };
00623
00624 template<class T>
00625 void mxGetVector(const char* F, int L,
00626     const mxArray *S, const char *vn,
00627     wbvector<T> &v
00628 ){
00629     if (!mxIsStruct(S) || !Mx::IsScalar(S)) wblog(F_L,
00630         "ERR input must be scalar structure");
00631
00632     mxArray *a=mxGetField(S,0,vn);
00633     if (!a) wblog(F_L,"ERR field %s does not exist",vn);
00634
00635     mxGetVector(F_L,a,v);
00636 };
00637
00638 double mxGetNumber(
00639     const char *F, int L,
00640     const mxArray *S, const char *vn, const char qflag
00641 ){
00642     mxArray *a;
00643     wbcomplex z;
00644
00645     if (!mxIsStruct(S)) wbdie(F_L,
00646         "need scalar structure on input");
00647     if (mxGetNumberOfElements(S)>1) wbdie(F_L,
00648         "need SINGLE structure on input");
00649
00650     a=mxGetField(S,0,vn);
00651
00652     if (!a) wblog(F_L,
00653         "ERR field %s does not exist in structure", vn);
00654
00655     if (mxGetNumber(a,z,qflag)) wbdie(FL,str);
00656     if (z.i!=0.) wblog(FL,"WRN Input number is complex! (%g)", z.i);
00657
00658     return z.r;
00659 }
00660
00661 template<class T>
00662 mxArray* cpyRange2Mx(const T* d, const wbvector<size_t> &S0){
00663
00664     mxArray *a;
00665     unsigned i=0, n=S0.prod(0), r=S0.len, len=MAX(2U,r);
00666     size_t s[len];
00667
00668     for (; i<r; ++i) s[i]=S0[i];
00669     if (i<len) {
00670         if (n) for (; i<len; ++i) s[i]=1;
00671         else for (; i<len; ++i) s[i]=0;
00672     }
00673
00674     a=Mx::Array<double>(len,s).copyFromTR(d);
00675     if (!a && d) wblog(FL,
00676         "ERR %s() got %p -> %p @ len=%d !?",FCT,d,a,len);
00677
00678     return a;
00679 };
00680
00681 template<>
00682 mxArray* cpyRange2Mx(const char* d0, const wbvector<size_t> &S0){
00683
00684     unsigned i, r=S0.len, n=S0.prod(0), len=MAX(2U,r);
00685     mxArray *a;
00686

```

```

00687     size_t s[len];
00688     for (i=0; i<r; i++) s[i]=S0[i];
00689     if (i<2) { if (i) s[1]=1; else s[0]=s[1]=0; }
00690
00691     a=Mx::Array<mxChar>(len,s).Return();
00692
00693     if (a==NULL) {
00694         if (!d0) wblog(FL,
00695             "ERR %s() invalid usage (%lX, %lX)",__FUNCTION__,d0,a);
00696         else return a;
00697     }
00698     if (!n) return a;
00699
00700     mxChar *d = (mxChar*)mxGetData(a);
00701     if (!d) wblog(FL,"ERR invalid mxCharArray");
00702
00703     for (i=0; i<n; ++i) { d[i]=mxChar(d0[i]);
00704         if (char(d[i])!=d0[i]) wblog(FL,
00705             "ERR %s() got data conversion error (%d/%d)",
00706             FCT,int(d[i]),int(d0[i])
00707         );
00708     }
00709
00710     return a;
00711 };
00712
00713 template<>
00714 mxArray* cpyRange2Mx(const wbcomplex* z, const wbvector<size_t> &S){
00715
00716     size_t i=0, n=S.prod(0); int isz=0;
00717     mxArray *a;
00718
00719     for (; i<n; ++i) { if (z[i].i!=0) { isz=1; break; }}
00720
00721     if (isz) {
00722         a=Mx::Array<wbcomplex>(S).copyFromTR(z);
00723     }
00724     else {
00725         Mx::Array<double> A(S); if (A.data) {
00726             for (i=0; i<n; ++i) { A.data[i]=z[i].r; }}
00727         a=A.Return();
00728     }
00729
00730     if (!a && z) wblog(FL,"ERR %s() failed to allocate data !?",FCT);
00731     return a;
00732 };
00733
00734 mxArray* matGetVariable(
00735     const char *F, int L,
00736     const char *fname, const char *vn, char force
00737 ){
00738     Wb::matFile f;
00739     int i=f.open(F,L,fname,"r", force); if (!i) return NULL;
00740     mxArray *a=matGetVariable(f.mfp,vn);
00741     return a;
00742 };
00743
00744 mxArray* matGetVariableInfo(
00745     const char *F, int L,
00746     const char *fname, const char *vn, char force
00747 ){
00748     Wb::matFile f;
00749     int i=f.open(F,L,fname,"r", force); if (!i) return NULL;
00750     mxArray *a=matGetVariableInfo(f.mfp,vn);
00751     return a;
00752 };
00753
00754 double* mexSetCell2Matrix(mxArray *C, int fid, unsigned d1, unsigned d2) {
00755
00756     double *data;
00757     Mx::Array<double> A(d1,d2); data=A.data;
00758
00759     if (A.data) { mxSetCell(C,fid,A.Return()); } else
00760     if (A.ax || (d1 && d2)) { wblog(FL,"ERR %s() %s",FCT,STR(A)); }
00761
00762     return data;
00763 };
00764
00765 namespace Wb {
00766
00767 template <class TQ, class TD>
00768 mxArray* mxCreateSparse(
00769     const char *F, int L, unsigned d1, unsigned d2,
00770     const wbMatrix<TQ> &IJ,
00771     const wbvector<TD> &D,
00772     const wperm_t *p,
00773     int *info

```

```

00774 ){
00775     const char lex=-1;
00776     widx_t n=D.len;
00777     bool gotp=(p!=NULL);
00778
00779     if (D.data && ISCOMPLX_(TD)) wblog(F,L,
00780 "ERR %s() got complex sparse data set (%dx%d; %d)",FCT,d1,d2,D.len);
00781     if (IJ.dim1!=n || (n && IJ.dim2!=2)) wblog(F,L,
00782 "ERR %s() size mismatch (%dx%d : ij=%dx%d @ %d)",
00783 FCT,d1,d2,IJ.dim1,IJ.dim2,D.len);
00784
00785     if (!gotp && !IJ.isSorted(+1,lex)) {
00786         wbMatrix<TQ> IX(IJ);
00787         wbpperm P;
00788
00789         IX.SortRecs(P,+1,lex);
00790
00791         return mxCreateSparse(F_L,d1,d2,IX,D,P.data);
00792     }
00793
00794     mxArray *a=mxCreateSparse(d1,d2,D.len,mxREAL);
00795     if (!a) wblog(F,L,"ERR %s() "
00796 "failed to allocate sparse matrix (%dx%d; %d)",FCT,d1,d2,D.len);
00797     if (!n) { return a; }
00798
00799     double *sr=mxGetDoubles(a);
00800     mwIndex *is=mxGetIr(a), *js=mxGetJc(a),k=0;
00801     unsigned i,j=0, nz=0, m=0; TQ *ij=IJ.data;
00802
00803     if (!WbUtil<TQ>::isInt()) {
00804         wblog(FL,"ERR %s() invalid non-integer data type '%s'",FCT,
00805 TSTR(TQ));
00806     }
00807
00808     for (i=0; i<n; ++i, ij+=2) {
00809         const TD& x=(gotp ? D.data[p[i]] : D.data[i]);
00810         if (int(ij[0])<0 || ij[0]>=d1 || int(ij[1])<0 || ij[1]>=d2) wblog(FL,
00811 "ERR %s() index out of bounds (%d,%d) [%dx%d]",
00812 FCT,ij[0],ij[1],d1,d2);
00813         if (x!=0) {
00814             if (i) {
00815                 if (ij[-2]==ij[0] && ij[-1]==ij[1]) {
00816                     sr[k]+=double(x); ++m; continue;
00817                 }
00818                 else if (ij[-1]>ij[1] || (ij[-1]==ij[1] && ij[-2]>ij[0])) {
00819                     wblog(FL,"ERR %s() data not sorted (%d/%d) !?",FCT,i+1,n);
00820                 }
00821                 else ++k;
00822             }
00823             sr[k]=double(x);
00824             is[k]=ij[0];
00825
00826             for (; j<=ij[1]; ++j) js[j]=k;
00827         }
00828         else if (ij[0]!=ij[1]) ++nz;
00829     }
00830
00831     for (++k; j<=d2; ++j) js[j]=k;
00832
00833     if (F) {
00834         if (nz) { wblog(F,L,
00835 "WRN %s() got off-diag zero data (%d/%d) !?",FCT,nz,D.len);
00836         }
00837         if (m) { wblog(F,L,
00838 "WRN %s() got non-unique sparse data (%d/%d)",FCT,m,D.len);
00839         }
00840     }
00841     if (info) (*info)=nz+m;
00842
00843     return a;
00844 };
00845
00846 }
00847
00848 int mxAddField2Scalar(
00849     const char *F, int L,
00850     mxArray* S, const char *vn, mxArray *a
00851 ){
00852     int fid;
00853
00854     if (!mxIsStruct(S)) wblog(F_L,
00855 "ERR scalar input structure required");
00856     if (mxGetNumberOfElements(S)>1) wblog(F_L,
00857 "ERR single input structure required");
00858
00859     fid=mxAddField(S,vn);
00860

```

```

00861     if (fid<0) wblog(F_L,
00862         "ERR failed to add field '%s' !? (%d)", vn,fid);
00863
00864     if (a) mxSetFieldByNumber(S,0,fid,a);
00865     return fid;
00866 };
00867
00868 int mxAddField(
00869     const char *F, int L,
00870     mxArray* S, const char *name
00871 ){
00872     int fid = mxAddField(S, name);
00873
00874     if (fid<0) wblog(F,L,
00875         "ERR Failed to add field %s to structure (%d) ???", name, fid);
00876
00877     return fid;
00878 }
00879
00880 void mxAppendStructToStruct(
00881     const char *F, int L,
00882     mxArray *S0, mxArray *S
00883 ){
00884     unsigned i,n;
00885     const char* name;
00886
00887     if (!mxIsStruct(S0) || mxGetNumberOfElements(S0)!=1 ||
00888         !mxIsStruct(S) || mxGetNumberOfElements(S)!=1) wblog(F,L,
00889         "ERR %s requires two scalar input structures",__FUNCTION__);
00890
00891     n=mxGetNumberOfFields(S0);
00892
00893     for (i=0; i<n; i++) {
00894         name=mxGetFieldNameByNumber(S0,i);
00895
00896         if (mxGetFieldNumber(S,name)>=0) wblog(F,L,
00897             "WRN overwriting field '%s' in structure", name);
00898
00899         mxAddField2Scalar(F,L,
00900             S, name, mxGetFieldByNumber(S0,0,i)
00901         );
00902
00903         mxSetFieldByNumber(S0,0,i,NULL);
00904     }
00905
00906     mxDestroyArray(S0); S0=NULL;
00907 }
00908
00909 int mxUpdateField(
00910     const char *F, int L,
00911     mxArray* S, const char *name, unsigned k, mxArray *a
00912 ){
00913     int fid=mxGetFieldNumber(S,name);
00914     unsigned n;
00915
00916     if (fid>=0) {
00917         mxArray *x=mxGetFieldByNumber(S,k,fid);
00918         if (x) mxDestroyArray(x);
00919     }
00920     else fid=mxAddField(F,L, S, name);
00921
00922     n=(unsigned)mxGetNumberOfElements(S);
00923
00924     if (k>=n) wblog(F,L,
00925         "ERR %s: index out of bounds (%d/%d)",FCT,k,n);
00926
00927     mxSetFieldByNumber(S,k,fid,a);
00928
00929     return fid;
00930 }
00931
00932 void matPutVariable(const char *F, int L,
00933     const char *file, const char *name, mxArray *a, unsigned keep
00934 ){
00935     Wb::matFile f;
00936
00937     f.Open(F,L,file,"u");
00938
00939     if (matPutVariable(f.mfp, name, a)) wblog(F,L,
00940         "ERR Failed to write parameter '%s' to file '%s'", name, file);
00941
00942     f.close();
00943     if (!keep) mxDestroyArray(a);
00944 }
00945
00946 void mxReplaceField(
00947     const char *F, int L,

```

```

00948     mxArray* S, unsigned k, int fid, mxArray *a
00949 ) {
00950     if (!S || !mxIsStruct(S)) wblog(F_L,
00951         "ERR %s() got %s", FCT, S ? mxGetClassName(S) : "(empty)");
00952
00953     mxArray *x=mxGetFieldByNumber(S,k,fid);
00954     if (x) mxDestroyArray(x);
00955
00956     mxSetFieldByNumber(S,k,fid,a);
00957 };
00958
00959 void mxSetFieldToNumber(
00960     mxArray *S, unsigned i, unsigned fid, wbcomplex z, char check) {
00961
00962     if (check) { unsigned n;
00963         if (!mxIsStruct(S)) wblog(F_L, "ERR invalid input structure");
00964         if (i >= (n=mxGetNumberOfElements(S))) wblog(F_L,
00965             "ERR index out of bounds (i=%d/%d)", i, n);
00966         if (fid >= (n=mxGetNumberOfFields(S))) wblog(F_L,
00967             "ERR index out of bounds (fid=%d/%d)", fid, n);
00968     }
00969
00970     mxArray *a; size_t sz[]={1,1};
00971     if (z.i)
00972         { a=Mx::Array<wbcomplex>(2,sz).copyFromTR(&z ); }
00973     else { a=Mx::Array<double> (2,sz).copyFromTR(&z.r); }
00974
00975     mxSetFieldByNumber(S,i,fid,a);
00976 };
00977
00978 int isFlagGlobal(const char *vn) {
00979     double flag=0;
00980     if (getNumGlobal(vn,flag)) return 0;
00981     return (flag!=0);
00982 }
00983
00984 template<class T>
00985 mxArray* vecVec2Mx(
00986     const wbvector<wbvector<T> > &V,
00987     const char tflag
00988 ) {
00989     mxArray *C=mxCreateCellMatrix(1,V.len);
00990
00991     for (unsigned i=0; i<V.len; i++)
00992         mxSetCell(C, i, V[i].toMx(tflag));
00993
00994     return C;
00995 }
00996
00997 template<class T>
00998 mxArray* vecMat2Mx(
00999     const wbvector<wbMatrix<T> > &M,
01000     const char rawflag
01001 ) {
01002     mxArray *C=mxCreateCellMatrix(1,M.len);
01003
01004     for (unsigned i=0; i<M.len; i++)
01005         mxSetCell(C, i, M[i].toMx(rawflag));
01006
01007     return C;
01008 }
01009
01010 template<class T>
01011 mxArray* vecVec2Mx(
01012     const wbvector<wbvector<T>*> &V,
01013     const char tflag
01014 ) {
01015     mxArray *C=mxCreateCellMatrix(1,V.len);
01016     wbvector<T> emptyVec;
01017
01018     for (unsigned i=0; i<V.len; i++)
01019         if (V[i]) mxSetCell(C,i,V[i]->toMx(tflag));
01020     else mxSetCell(C, i, emptyVec.toMx(tflag));
01021
01022     return C;
01023 }
01024
01025 #endif
01026

```

5.11 mexlib.hh

```
00001 /* -----
```

```

00002 * Project : QSpace tensor library (v4.0)
00003 * Class   : QSpace MEX routines
00004 *
00005 * Copyright 2024 Andreas Weichselbaum
00006 *
00007 * Licensed under the Apache License, Version 2.0 (the "License");
00008 * you may not use this file except in compliance with the License.
00009 * You may obtain a copy of the License at
00010 *
00011 *     http://www.apache.org/licenses/LICENSE-2.0
00012 *
00013 * Unless required by applicable law or agreed to in writing, software
00014 * distributed under the License is distributed on an "AS IS" BASIS,
00015 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016 * See the License for the specific language governing permissions and
00017 * limitations under the License.
00018 * ----- */
00019
00020 #ifndef __WB_MEXLIB_HH__
00021 #define __WB_MEXLIB_HH__
00022
00023 /* ----- */
00024 /* ----- */
00025
00026 void mexDisp(const mxArray *a, const char *vname=0);
00027
00028 void mexWRN(const char* s);
00029
00030 wbststring sprint_info(const mxArray *a);
00031
00032 double wbtoc(
00033     const char *F=NULL, int L=0,
00034     const char *istr="", char restart=0
00035 );
00036
00037 void wbtic() { wbtoc(0,0); }
00038
00039 template<class T>
00040 int mxGetNumber(const mxArray *a, T &d, const char qflag=0);
00041
00042 double mxGetNumber(
00043     const char *F, int L,
00044     const mxArray *S, const char *vname, const char qflag=0
00045 );
00046
00047 int getFlag(const mxArray *a) {
00048     double q=0;
00049     Mx::Array<double> A(a);
00050     if (A.len>1) wblog(FL,
00051         "ERR %s() invalid flag (got array, n=%d)",FCT,A.len);
00052     A.ncopy_to(&q,1);
00053     return int(q);
00054 };
00055
00056 int mxGetString(const mxArray *a, char *s);
00057 int mxGetString(const mxArray *a, wbststring &s);
00058
00059 wbststring mxTypeSize2Str(const mxArray *a);
00060 wbststring mxSize2Str(const mxArray *a);
00061
00062 template<class T>
00063 void mxGetVector(const char* F, int L, const mxArray *a, wbstvector<T> &v) {
00064     size_t n=mxGetNumberOfElements(a);
00065     v.init(n);
00066     try {
00067         Mx::Array<T>(a).copyTo(v.data);
00068     }
00069     catch (...) { wblog(F_L,"ERR %s()",FCT); }
00070 };
00071
00072 template<class T>
00073 void mxGetVector( const char* F, int L,
00074     const mxArray *S, const char *vname,
00075     wbstvector<T> &v);
00076
00077 mxArray* matGetVariable(const char *F, int L,
00078     const char *fname, const char *vname, char force=0
00079 );
00080 mxArray* matGetVariableInfo(const char *F, int L,
00081     const char *fname, const char *vname, char force=0
00082 );
00083
00084 double* mexSetCell2Matrix(mxArray *a, int fid, unsigned d1, unsigned d2);
00085 void mxSetFieldToNumber(mxArray *S, unsigned i, unsigned fid,
00086     wbstcomplex z, char dblcheck=1);
00087
00088 template<class T>

```

```

00089     int getNumGlobal(const char *vname, T &x, const char *ws="global");
00090     int isFlagGlobal(const char *vname);
00091
00092     template<class T>
00093     mxArray* vecVec2Mx(
00094         const wbvector<wbvector<T> > &V, const char tflag=0);
00095
00096     template<class T>
00097     mxArray* vecVec2Mx(
00098         const wbvector<wbvector<T>* > &V, const char tflag=0);
00099
00100     template<class T>
00101     mxArray* vecMat2Mx(
00102         const wbvector<wbMatrix<T> > &M, const char rawflag=0);
00103
00104     template<class T>
00105     mxArray* cpyRange2Mx(const T* d, const wbvector<size_t> &S0);
00106
00107     int mxAddField2Scalar(
00108         const char *F, int L,
00109         mxArray* S, const char *vname, mxArray *a=NULL
00110     );
00111
00112     void mxAppendStructToStruct(
00113         const char *F, int L,
00114         mxArray *&S0, mxArray *S
00115     );
00116
00117     int mxAddField(
00118         const char *F, int L,
00119         mxArray* S, const char *vname
00120     );
00121
00122     void mxReplaceField(
00123         const char *F, int L,
00124         mxArray* S, unsigned k, int fid, mxArray *a
00125     );
00126
00127     int mxUpdateField(
00128         const char *F, int L,
00129         mxArray* S, const char *name, unsigned k, mxArray *a
00130     );
00131
00132     void matPutVariable(const char *F, int L,
00133         const char *file, const char *name, mxArray *a, unsigned keep=0
00134     );
00135
00136     namespace Wb {
00137
00138         template <class TQ, class TD>
00139         mxArray* mxCreateSparse(
00140             const char *F, int L, unsigned d1, unsigned d2,
00141             const wbMatrix<TQ> &IJ, const wbvector<TD> &D,
00142             const wperm_t *p=NULL, int *info=NULL
00143         );
00144     }
00145
00146     template<class T>
00147     mxArray* numtoMx(const T &x) {
00148         mxArray *a=mxCreateDoubleMatrix(1,1,mxREAL);
00149         mxGetDoubles(a)[0]=double(x); return a;
00150     };
00151
00152     template<>
00153     mxArray* numtoMx(const wbcomplex &z) {
00154         mxArray *a;
00155         if (z.i!=0.) {
00156             a=mxCreateDoubleMatrix(1,1,mxCOMPLEX);
00157             z.explicit_copy2(mxGetComplexDoubles(a)[0]);
00158         }
00159         else a=numtoMx(z.r);
00160         return a;
00161     };
00162
00163     template<>
00164     mxArray* numtoMx(const char &c) {
00165         if (isalpha(c)) {
00166             char s[2]={c,0};
00167             return mxCreateString(s);
00168         }
00169         else return numtoMx(double(c));
00170     };
00171
00172     class mxStruct {
00173     public:
00174
00175

```

```

00176     mxStruct(const char *file, int line) : S(NULL) { init(file,line); };
00177 ~mxStruct() {
00178     if (S) {
00179         wblog(FL,"WRN mxStruct() destroyed without further ado");
00180         mxDestroyArray(S);
00181     }
00182 };
00183
00184 void init(const char *file, int line) {
00185     F=file; L=line;
00186     if (S) mxDestroyArray(S);
00187     S=mxCreateStructMatrix(1,1,0,NULL);
00188 };
00189
00190 void put(const char *name) {
00191     if (S) { mxPutAndDestroy(F.data,L,S,name); S=NULL; } else
00192     wblog(FL,"WRN mxStruct::put() not initialized yet - return");
00193     F.init(); L=0;
00194 };
00195
00196 void save2Struct(mxArray *S0, const char *name) {
00197     if (S) {
00198         mxAddField2Scalar(F.data, L, S0, name, S);
00199         S=NULL;
00200     } else
00201     wblog(FL,"WRN mxStruct::save2Struct() not initialized yet");
00202     F.init(); L=0;
00203 };
00204
00205 void checkNewField(const char *f=NULL, int l=0);
00206
00207 mxStruct& operator()(const char *fn) { fld=fn; return *this; };
00208
00209 template<class T>
00210 mxStruct& operator=(const T&x) {
00211     checkNewField(FL);
00212     mxAddField2Scalar(F.data, L, S, fld.data, numtoMx(x));
00213     return *this;
00214 };
00215
00216 mxStruct& operator=(mxArray *a) {
00217     checkNewField(FL);
00218     mxAddField2Scalar(F.data, L, S, fld.data, a);
00219     return *this;
00220 };
00221
00222 template<class T>
00223 mxStruct& operator=(const wbvector<T>& x) {
00224     checkNewField(FL);
00225     mxAddField2Scalar(F.data, L, S, fld.data, x.toMx());
00226     return *this;
00227 };
00228
00229 template<class T>
00230 mxStruct& operator=(const wbMatrix<T>& x) {
00231     checkNewField(FL);
00232     mxAddField2Scalar(F.data, L, S, fld.data, x.toMx());
00233     return *this;
00234 };
00235
00236 mxArray *S;
00237 wbstring F, fld;
00238 unsigned L;
00239
00240 protected:
00241 private:
00242
00243 };
00244
00245 template<>
00246 mxStruct& mxStruct::operator=(const wbperm &P) {
00247     checkNewField(FL);
00248     mxAddField2Scalar(F.data, L, S, fld.data, P.toMx());
00249     return *this;
00250 }
00251
00252 template<>
00253 mxStruct& mxStruct::operator=(const wbindex &I) {
00254     checkNewField(FL);
00255     mxAddField2Scalar(F.data, L, S, fld.data, I.toMx());
00256     return *this;
00257 }
00258
00259 inline void mxStruct::checkNewField(const char *f, int l) {
00260     int i; if (!f) { f=F.data; l=L; }
00261
00262     if (!S) wblog(f,l,"ERR mxStruct() S not set yet");

```



```

00263     if (fld.isEmpty()) wblog(f,l,"ERR mxStruct() field not specified");
00264
00265     i=mxGetFieldNumber(S,fld.data);
00266     if (i>=0) wblog(f,l,"ERR Field '%s' already exists!");
00267 };
00268
00269 class MXPput {
00270 public:
00271
00272     MXPput(const char *vn=0, const char *ws=0)
00273     : S(0), vname(0), F(0), L(0) {
00274         strncpy(wsp, ws && ws[0]? ws:"caller",wlen-1); wsp[wlen-1]=0;
00275         init(0,0,vn);
00276     };
00277
00278     MXPput(const char *file, int line, const char *vn="ans", const char *ws=0)
00279     : S(0), vname(0), F(0), L(0) {
00280         strncpy(wsp, ws && ws[0]? ws:"caller",wlen-1); wsp[wlen-1]=0;
00281         init(file,line,vn);
00282     };
00283
00284     ~MXPput() { put(); }
00285
00286     void init() {
00287         if (F) { WB_DELETE(F); }; L=0;
00288         if (vname) { WB_DELETE(vname); }
00289         if (S) {
00290             if (mxGetNumberOfFields(S)) { const char *v="I_check_this";
00291                 wblog(F_L,"WRN %s() clearing non-empty S (see %s)",FCT,v);
00292                 mxPutAndDestroy(F,L,S,v,"caller");
00293             }
00294             else { mxDestroyArray(S); }
00295             S=NULL;
00296         }
00297     };
00298
00299     void init(const char *file, int line, const char *vn=NULL) {
00300         if (S || F || vname) init();
00301         if (file && file[0]) { L=line;
00302             WB_NEW(F,strlen(file)+1); strcpy(F,file);
00303         }
00304
00305         if (vn && vn[0])
00306             { WB_NEW(vname,strlen(vn)+1); strcpy(vname,vn); }
00307         else { WB_NEW(vname,4); strcpy(vname,"ans"); }
00308
00309         #pragma omp critical (using_MEX_API)
00310         S=mxCreateStructMatrix(1,1,0,NULL);
00311
00312         if (L) addFL();
00313     };
00314
00315     void put(const char *ws=0, const char *vn=0) {
00316         if (S) {
00317             if (!vname) wblog(F_L,
00318                 "ERR %s() got empty variable name",FCT,vname);
00319             const char s0[]="ans", *s=((vn && vn[0]) ? vn : vname);
00320             if (!s || !s[0]) s=s0;
00321
00322             if (!ws || !ws[0]) {
00323                 if (!wsp[0]) wblog(F_L,"ERR %s() workspace not set",FCT);
00324                 ws=wsp;
00325             }
00326
00327             mxPutAndDestroy(F,L,S,s,ws);
00328             S=NULL;
00329         }
00330         init();
00331     };
00332
00333     void save2(mxArray* &S0) { S0=S; S=NULL; init(); };
00334
00335     mxArray* toMx() { mxArray* S0=S; S=NULL; init(); return S0; };
00336
00337     void add2Struct(mxArray *S0) {
00338         if (S) {
00339             if (!vname) wblog(F_L,
00340                 "ERR %s() got empty variable name !?",FCT,vname);
00341             mxAddField2Scalar(F_L,S0,vname,S); S=NULL;
00342         }
00343         init();
00344     };
00345
00346     MXPput& addFL() {
00347         if (!S) wblog(F_L,"ERR MXPput::%s() S not initialized yet",FCT);
00348         else {
00349             if (!vname) wblog(F_L,

```

```

00350         "ERR %s() got empty variable name !?",FCT,vname);
00351     if (mxGetFieldNumber(S,"FL")<0)
00352         mxAddField2Scalar(F_L,S,"FL",wbstring(shortFL(F_L)).toMx());
00353     else wblog(F_L,"WRN %s() field FL already set",FCT);
00354 }
00355 return *this;
00356 };
00357
00358 template<class T>
00359 MXPut& add(const T&x, const char *fn) {
00360     if (!S) wblog(F_L,"ERR MXPut::%s() S not initialized yet",FCT);
00361     mxAddField2Scalar(F_L,S,fn,x.toMx());
00362     return *this;
00363 };
00364
00365 template<class T>
00366 MXPut& addP(
00367     const T* x      QS_UNUSED_VAR,
00368     const char *fn QS_UNUSED_VAR
00369 ) {
00370     if (!S) wblog(F_L,"ERR MXPut::%s() S not initialized yet",FCT);
00371     else wblog(F_L,
00372         "ERR %s() not defined yet for pointer to %s",FCT,TSTR(T));
00373     return *this;
00374 };
00375
00376 mxArray *S;
00377 char wsp[16];
00378 char *vname, *F;
00379 unsigned L;
00380
00381 static const unsigned wlen;
00382
00383 protected:
00384 private:
00385 };
00386
00387 const unsigned MXPut::wlen=16;
00388
00389 template<>
00390 MXPut& MXPut::addP(const mxArray* a, const char *fn) {
00391     if (!S) wblog(F_L,"ERR MXPut::%s() S not set yet",FCT);
00392     if (!vname) wblog(F_L,"ERR %s() got empty variable name",FCT,vname);
00393     mxAddField2Scalar(F_L,S, fn, (mxArray*)a);
00394     return *this;
00395 };
00396
00397 #ifndef NOMEX
00398 template<>
00399 MXPut& MXPut::addP(const char* s, const char *fn) {
00400     return addP(mxCreateString(s),fn);
00401 };
00402 #endif
00403
00404 template<>
00405 MXPut& MXPut::add(const double &x, const char *fn) {
00406     return addP(numtoMx(x),fn);
00407 };
00408 template<>
00409 MXPut& MXPut::add(const float &x, const char *fn) {
00410     return addP(numtoMx(x),fn);
00411 };
00412 template<>
00413 MXPut& MXPut::add(const long double &x, const char *fn) {
00414     return addP(numtoMx(x),fn);
00415 };
00416 template<>
00417 MXPut& MXPut::add(const wbcomplex &x, const char *fn) {
00418     return addP(numtoMx(x),fn);
00419 };
00420 template<>
00421 MXPut& MXPut::add(const char &x, const char *fn) {
00422     return addP(numtoMx(x),fn);
00423 };
00424
00425 template<>
00426 MXPut& MXPut::add(char* const &s, const char *fn) {
00427     return add(wbstring(s),fn);
00428 };
00429 template<>
00430 MXPut& MXPut::add(const char* const &s, const char *fn) {
00431     return add(wbstring(s),fn);
00432 };
00433
00434 template<>
00435 MXPut& MXPut::add(const unsigned &x, const char *fn) {
00436     return addP(numtoMx(x),fn);

```

```

00437 };
00438 template<>
00439 MXPut& MXPut::add(const int &x, const char *fn) {
00440     return addP(numtoMx(x),fn);
00441 };
00442 template<>
00443 MXPut& MXPut::add(const bool &x, const char *fn) {
00444     return addP(numtoMx(x),fn);
00445 };
00446 template<>
00447 MXPut& MXPut::add(const long &x, const char *fn) {
00448     return addP(numtoMx(x),fn);
00449 };
00450 template<>
00451 MXPut& MXPut::add(const unsigned long &x, const char *fn) {
00452     return addP(numtoMx(x),fn);
00453 };
00454
00455 #ifdef __WB_MPFR_HH__
00456 template<>
00457 MXPut& MXPut::add(const Wb::quad &x, const char *fn) {
00458     return addP(numtoMx(x),fn);
00459 };
00460 #endif
00461
00462 class mxArray_buf {
00463
00464 public:
00465     mxArray_buf(const char *n=NULL) {
00466         BUF.max_load_factor(0.2);
00467         if (n && n[0])
00468             name = new char[strlen(n)+1]; strcpy(name,n); }
00469         else { name = new char[4]; strcpy(name,"MXB"); }
00470     };
00471
00472     ~mxArray_buf() {
00473         size_t l=BUF.size();
00474         if (l) wblog(FL,"WRN %s() %s got %d entries still",PROG,name,l);
00475         BUF.clear();
00476         if (name) { delete [] name; name=NULL; }
00477     };
00478
00479     void add(void *p, mxArray *a) {
00480         if (p) {
00481             #pragma omp critical (using_mxArray_P2X)
00482             { auto im=BUF.insert({p,a});
00483                 if (!im.second) wblog(FL,
00484                     "ERR %s() %s already got entry for %p",PROG,name,p);
00485             }
00486         }
00487         else wblog(FL,"WRN %s() got null pointer",FCT,p);
00488     };
00489
00490     mxArray* find(const char *F, int L, void *p) {
00491         mxArray *a=NULL;
00492
00493         #pragma omp critical (using_mxArray_P2X)
00494         { auto im=BUF.find(p);
00495             if (im!=BUF.end()) { a=im->second; }
00496         }
00497         if (!a && F) wblog(F,L,
00498             "WRN %s() %s missing entry for %p",PROG,name,p);
00499         return a;
00500     };
00501
00502     mxArray* Return(const char *F, int L, void *p) {
00503         mxArray *a=NULL;
00504
00505         #pragma omp critical (using_mxArray_P2X)
00506         { auto im=BUF.find(p);
00507             if (im!=BUF.end()) {
00508                 a=im->second; BUF.erase(im->first);
00509             }
00510         }
00511         if (!a && F) wblog(F,L,
00512             "WRN %s() %s missing entry for %p",PROG,name,p);
00513         return a;
00514     };
00515
00516     int erase(void *p) {
00517         int e=0;
00518         #pragma omp critical (using_mxArray_P2X)
00519         { e=BUF.erase(p); }
00520         return e;
00521     };
00522
00523     unordered_map<void*, mxArray*> BUF;

```

```

00524
00525     char* name;
00526
00527 protected:
00528 private:
00529
00530 };
00531
00532     mxArray_buf P2X("P2X");
00533
00534 #endif
00535

```

5.12 mpsortho.cc

```

00001 #ifndef __WB_ORTHO_CC__
00002 #define __WB_ORTHO_CC__
00003
00004 // ===== //
00005 // QBlock :: class for block transformations of QSpace data{:}
00006 // e.g. combining/blocking data into fused symmetry sectors
00007 // ----- //
00008 // toBlockMatrix() operates on this->A
00009 // which is input and needs to be set prior to a call to *this
00010 //
00011 // => for all-abelian symmetries, *this can group multiple symmetry
00012 //      sectors into single full matrix MM; typically, however, also
00013 //      in the abelian case, either Q1 or Q2 represents a single
00014 //      total (fused) symmetry sector, which needs to be taken care of
00015 //      in the caller for all-abelian [e.g. using getQsum() on either
00016 //      Q1 or Q2 then yields the same result].
00017 //
00018 // => NB! for non-abelian symmetries, this->A must represent single
00019 //      well-defined symmetry sector to avoid conflicts with fusion
00020 //      rules hence Q1 or Q2 must represent a single leg / index,
00021 //      i.e., a single set of symmetry labels, and thus below either
00022 //      n1 or n2 must equal 1! Then fusion rules are respected
00023 //      automatically (since by construction, all symmetry labels
00024 //      from Q1 must be fuseable into Q2 and vice versa, as already
00025 //      described by the isometries (up to normalization) via
00026 //      CRef -> CData.
00027 //
00028 // Wb,Aug23,08 // Wb,Aug04,15
00029
00030 template <class TQ, class TD, class TE>
00031 void QBlock<TQ,TD,TE>::toBlockMatrix( // ::initBlockMatrix
00032     wbararray<TD> &MM, // output matrix
00033     unsigned K, // group first K dimension and also the rest
00034     const wpperm &P // permutation on data *before* using K
00035 ){
00036     char cgflag=A.getCGS(FL), wom=0;
00037     unsigned R=-1, nsym=A.CGR.dim2;
00038     widx_t i,j,k,d,l,l1=0, r,s, n1,n2, i1,i2, d1,d2, dim1,dim2;
00039
00040     wbarMatrix<unsigned> OM, OMD, mark;
00041     wbarvector<unsigned> M1,M2, omk;
00042     wbararray<TD> Ak,Mk;
00043     wbararray<double> CGW;
00044     wbarindex I1,I2;
00045
00046     groupIndex<widx_t> g1, g2;
00047     wbarvector<widx_t> s1,s2, cD1, cD2;
00048
00049     if (!A.isConsistent(R)) { wblog(FL,"%s",str); }
00050     if (A.isEmpty()) {
00051         if (K || P) wblog(FL,
00052             "ERR %s() got empty QSpace with K=%d / P.len=%d",FCT,K,P.len);
00053         MM.init(); this->init();
00054         return;
00055     }
00056
00057     if (int(R)<2) wblog(FL,"ERR %s() got invalid QSpace (rank r=%d)",FCT,R);
00058     if (int(K)<0) { K=R/2; } else
00059     if (K>R) wblog(FL,"ERR index out of bounds (k=%d/%d)",K,R);
00060     if (!K || K==R) wblog(FL,"ERR got empty group (k=%d/%d)",K,R);
00061
00062     qdim_tot=1; qdim.init2val(nsym,1);
00063
00064     if (cgflag>0) {
00065         if (K== 1) { wom=2; l1=0; } else
00066         if (K==(R-1)) { wom=1; l1=K; }
00067         else {
00068             wblog(FL,"ERR %s() got non-abelian CGR with K=%d/%d (cg=%d)\n"

```

```

00069         "(hint: fuse tensor to rank-2 since non-abelian)",FCT,cgflag,K,R);
00070     }
00071
00072     if (A.CGR) { qdim.init(A.CGR.dim2);
00073         for (i=0; i<A.CGR.dim1; ++i) {
00074             for (j=0; j<A.CGR.dim2; ++j) {
00075                 A.CGR(i,j).norm2();
00076                 if (i) {
00077                     d = (A.CGR(i,j).cgb ? A.CGR(i,j).qdim(l1) : 1);
00078                     if (d!=qdim[j]) wblog(FL,"ERR %s() "
00079                         "CGR(%d,%d) inconsisteny qdim=%d/%d",FCT,i+1,j+1,d,qdim[j]
00080                         );
00081                 }
00082                 else if (A.CGR(i,j).cgb) {
00083                     qdim_tot *= (qdim[j] = A.CGR(i,j).qdim(l1));
00084                 }
00085             }}
00086     }
00087     else { qdim.init(); }
00088 }
00089
00090 if (K!=1 && K!=(R-1) && (A.itags || A.CGR)) {
00091     int e=0; QDir qdir; A.getQDir(qdir);
00092
00093     if (qdir.len!=R) { e=1; }
00094     else {
00095         if (P) { qdir.Permute(P); }
00096         for (i=1; i<K; ++i) { if (qdir[i]!=qdir[0]) { e=2; break; } }
00097         for (++i; i<R; ++i) { if (qdir[i]!=qdir[K]) { e=3; break; } }
00098     }
00099     if (e) wblog(FL,
00100         "ERR %s() got mixed qdir=%s (k=%d/%d; e=%d)",FCT,STR(qdir), K,R,e);
00101 }
00102
00103 if (P) {
00104     if (P.len!=R) wblog(FL,
00105         "ERR invalid permutation (len=%d/%d)",P.len,R);
00106     I1.init( K, P.data );
00107     I2.init(R-K, P.data+K);
00108 }
00109 else {
00110     I1.Index(0,K-1);
00111     I2.Index(K,R-1);
00112 }
00113
00114 if (!A.QIDX.isUnique()) wblog(FL,
00115     "ERR %s() got non-unique Q-labels",FCT);
00116
00117 A.getQsub(I1,Q1).groupRecs(g1.P,g1.D); g1.setup('f');
00118 A.getQsub(I2,Q2).groupRecs(g2.P,g2.D); g2.setup('f');
00119
00120 n1=g1.D.len; S1.init(n1,I1.len);
00121 n2=g2.D.len; S2.init(n2,I2.len);
00122
00123 A.getOM(OM,&M2,2);
00124
00125 if (!A.getOM(OM,&M1,1)) {
00126     D1.init2val(n1,2, 1);
00127     D2.init2val(n2,2, 1);
00128 }
00129 else if (n1==1 && !(n2==1 && wom==1)) {
00130     if (wom!=2) wblog(FL,
00131         "ERR %s() wom=%d/2 (n1=%d, n2=%d, K=%d/%d)",FCT,wom,n1,n2,K,R);
00132     D1.init2val(n1,2, 1);
00133     D2.init2val(n2,2+OM.dim2, 1);
00134     for (l=i=0; i<n2; ++i, l+=d) { d=g2.D[i];
00135         if ((D2(i,1)=M1[g2.P[l]])>1) {
00136             Wb::cpyRange(D2.ref(i,2), OM.rec(g2.P[l]), OM.dim2); }
00137     }
00138 }
00139 else if (n2==1) {
00140     if (wom!=1) wblog(FL,
00141         "ERR %s() wom=%d/1 (n1=%d, n2=%d, K=%d/%d)",FCT,wom,n1,n2,K,R);
00142     D1.init2val(n1,2+OM.dim2, 1);
00143     D2.init2val(n2,2, 1);
00144     for (l=i=0; i<n1; ++i, l+=d) { d=g1.D[i];
00145         if ((D1(i,1)=M1[g1.P[l]])>1) {
00146             Wb::cpyRange(D1.ref(i,2), OM.rec(g1.P[l]), OM.dim2); }
00147     }
00148 }
00149 else { wblog(FL,
00150     "ERR %s() got composite indices for all groups\n"
00151     "in the presence of outer-multiplicity (%s)\n"
00152     "(hint: fuse indices at least for one group since non-abelian)",
00153     FCT, A.qStr().data);
00154 }
00155

```

```

00156 mark.init(n1+1,n2+1);
00157 idx.init2val(n1,n2,-1);
00158
00159 for (i=0; i<A.DATA.len; ++i) {
00160     A.DATA[i]->getSize(I1,s1); r=g1.Ig[i];
00161     A.DATA[i]->getSize(I2,s2); s=g2.Ig[i];
00162
00163     idx(r,s)=i;
00164
00165     if (++mark(r,s)>1 && !wom) { wblog(FL,
00166         "ERR %s() block %d not unique\n"
00167         "got degenerate records in QSpace w/out OM",FCT,i+1);
00168     }
00169     if (++mark(r,n2)==1) { S1.recSet(r,s1); D1(r,0)=s1.prod(0); }
00170     else if (S1.recCompare(r,s1)) { wblog(FL,
00171         "ERR %s() block size mismatch (%d, r: %d/%d)\n[%s, %s]",
00172         FCT, i, r+1, n1, S1.rec2Str(r,"","x").data, SSTR(s1));
00173     }
00174     if (++mark(n1,s)==1) { S2.recSet(s,s2); D2(s,0)=s2.prod(0); }
00175     else if (S1.recCompare(r,s1)) { wblog(FL,
00176         "ERR %s() block size mismatch (%d, c: %d/%d)\n[%s, %s]",
00177         FCT, i, s+1, n2, S1.rec2Str(r,"","x").data, SSTR(s1));
00178     }
00179 }
00180
00181 dim1=D1.cumsum0prodRec(cD1,0,1);
00182 dim2=D2.cumsum0prodRec(cD2,0,1);
00183 MM.init(dim1,dim2);
00184
00185 for (k=0; k<A.DATA.len; ++k) {
00186     const CRef<TQ> *Rk=(nsym ? A.CGR.rec(k) : NULL);
00187
00188     r = g1.Ig[k]; s = g2.Ig[k];
00189     i1 = cD1[r]; i2 = cD2[s];
00190     d1 = D1(r,0); d2 = D2(s,0);
00191
00192     if (M1[k]==1) { double cfac=1;
00193         A.DATA[k]->toMatrixRef(Mk,K,P);
00194
00195         for (i=0; i<nsym; ++i) {
00196             if ((l=Rk[i].wnumel())>1) wblog(FL,"ERR %s() "
00197                 "unexpected CGR(%d,%d) %s",FCT,k+1,i+1,SSTR(Rk[i].cgw));
00198             if (l==1) { cfac*=Rk[i].wget0(); }
00199         }
00200
00201         if (Mk.SIZE.len!=2 || Mk.SIZE[0]!=d1 || Mk.SIZE[1]!=d2) wblog(FL,
00202             "ERR %s() size inconsistency %s / %dx%d",FCT,SSTR(Mk),d1,d2);
00203         if (mark(r,s)!=1 || M2[k]!=1) { wblog(FL,"ERR %s() "
00204             "expecting unique block [mark(%d/%d,%d/%d)=%d] / M2=%d",
00205             FCT,r+1,n1,s+1,n2,mark(r,s),M2[k]); }
00206         if (i2+Mk.SIZE[1]>dim2) { wblog(FL,
00207             "ERR %s() index out of bounds:\n"
00208             "setting M(%d:%d,%d:%d) having size %s",FCT,
00209             i1+1, i1+Mk.SIZE[0], i2+1, i2+Mk.SIZE[1], SSTR(MM)); }
00210         cfac/=sqrt(double(qdim_tot));
00211
00212         Mk.copyStride( MM.ref(i1,i2), dim1, cfac);
00213     }
00214     else {
00215         if (mark(r,s)!=1) { wblog(FL,"ERR %s() "
00216             "unexpected unique block [mark(%d/%d,%d/%d)=%d] @ M1=%d",
00217             FCT,r+1,n1,s+1,n2,mark(r,s),M1[k]);
00218         }
00219         if (M1[k]<2) { wblog(FL,"ERR %s() got M1[%d]=%d",FCT,k,M1[k]); }
00220         if (!wom) { wblog(FL,"ERR %s() got wom=%d",FCT,wom); }
00221
00222         for (i=0; i<nsym; ++i) {
00223             if (i) { CGW.kron(Rk[i].cgw, CGW); }
00224             else { CGW=Rk[i].cgw; }
00225         }
00226
00227         if (!CGW.isMatrix(M1[k],M2[k])) wblog(FL,"ERR %s() "
00228             "size mismatch CGW %s / %dx%d",FCT,SSTR(CGW),M1[k],M2[k]);
00229
00230         if (qdim_tot>1) {
00231             CGW *= sqrt(1./double(qdim_tot));
00232         }
00233         else if (qdim_tot<1) wblog(FL,
00234             "ERR %s() invalid qdim=%d",FCT,qdim_tot);
00235
00236         A.DATA[k]->contractMat(FL,R+1,CGW,2,Ak);
00237
00238         Ak.toMatrixRefM(Mk,K,P,R, wom==1? "132":"" );
00239         { wvector<size_t> S(Mk.SIZE);
00240             if (wom==1)
00241                 { S[0]*=S[1]; S[1]=S[2]; }
00242             else { S[1]*=S[2]; }

```

```

00243         S.len=2;
00244         Mk.Reshape(S);
00245     }
00246
00247     if (i1+Mk.SIZE[0]>dim1 || i2+Mk.SIZE[1]>dim2) { wblog(FL,
00248         "ERR %s() index out of bounds:\n"
00249         "setting M(%d:%d,%d:%d) having %s",FCT,
00250         i1+1,i1+Mk.SIZE[0], i2+1,i2+Mk.SIZE[1], SSTR(MM));
00251     }
00252
00253     Mk.copyStride( MM.ref(i1,i2), dim1);
00254 }
00255 }
00256 };
00257
00258 template <class TQ, class TD, class TE>
00259 QBlock<TQ,TD,TE>& QBlock<TQ,TD,TE>::init_bare_refA(
00260     const QBlock &b, unsigned flag) {
00261
00262     if (this==&b) {
00263         if (flag) wblog(FL,"ERR %s() got same object (flag=%d)",FCT,flag);
00264         return *this;
00265     }
00266
00267     if (!strchr("USVX",flag)) wblog(FL,
00268         "ERR %s() invalid flag %c<%d>",FCT,flag,flag);
00269
00270     IO.init(); Ik.init(); It.init(); U.init(); S.init(); Vc.init();
00271
00272     A.init2ref(b.A);
00273
00274     if (!flag) {
00275         qdim_tot=b.qdim_tot; qdim=b.qdim;
00276         Q1=b.Q1; S1=b.S1; D1=b.D1;
00277         Q2=b.Q2; S2=b.S2; D2=b.D2; idx=b.idx;
00278         return *this;
00279     }
00280
00281     int k=0, e=0;
00282
00283     if (b.S2.dim2==1) { k=2;
00284         b.Q2.blockSum(b.A.QDIM,Q2);
00285         if (b.S2.dim2>1 && b.A.isNonAbelian()) { e=1; }
00286     }
00287     else { k=1;
00288         b.A.getQsum(b.Q1,Q2);
00289     }
00290
00291     if (e || !Q2.recAllEqual() || !Q2.dim1) {
00292         MXPut(FL,"q").add(b,"b").add(Q2,"Q2").add(k,"k");
00293         wblog(FL,
00294             "ERR %s() got inconsistency in block symmetry\n(%s; [%d+%d])",
00295             FCT,SSTR(b.idx),b.S1.dim2,b.S2.dim2
00296         );
00297     }
00298     Q2.dim1=1;
00299
00300     S2.init(1,1);
00301     D2.init2val(1,2, 1);
00302
00303     qdim_tot=b.qdim_tot;
00304     qdim=b.qdim;
00305
00306     if (flag=='U') {
00307         Q1=b.Q1; S1=b.S1; D1=b.D1; D2[0]=S2[0]=b.U.dim(2);
00308
00309         if (b.idx.dim2>1) {
00310             idx.init2val(b.idx.dim1,1, b.A.DATA.len);
00311         }
00312         else idx=b.idx;
00313     }
00314     else if (flag=='V' || flag=='X') {
00315         Q1=b.Q2; S1=b.S2; D1=b.D2; D2[0]=S2[0]=b.Vc.dim(2);
00316
00317         if (b.idx.dim1>1) {
00318             idx.init2val(b.idx.dim2,1, b.A.DATA.len);
00319         }
00320         else b.idx.transpose(idx);
00321     }
00322     else if (flag=='S') { D2[0]=S2[0]=b.S.len;
00323         Q1=Q2; D1=D2; S1=S2;
00324         idx.init(1,1);
00325     }
00326     else wblog(FL,"ERR %s() invalid flag=%d",FCT,flag);
00327
00328 #ifndef WB_SKIP_ASSERT
00329     { wvector<widxt_t> cD_; widxt_t dim2_=S2[0],

```

```

00330         d1=b.D1.cumsum0prodRec(cD_,0,1),
00331         d2=b.D2.cumsum0prodRec(cD_,0,1);
00332     if (dim2_>d1 && dim2_>d2) wblog(FL,
00333     "ERR %s() unexpected dS=%d (%dx%d)",FCT,dim2_,d1,d2
00334     ); }
00335 #endif
00336
00337     return *this;
00338 };
00339
00340 template <class TQ, class TD, class TE>
00341 void QBlock<TQ,TD,TE>::updateBlockDim(unsigned dim2_) {
00342
00343     if (S2.numel()!=1 || D2.dim1!=1 || D2.dim2<2 || D2[1]!=1)
00344         wblog(FL,"ERR %s() QBlock not yet initialized\n"
00345         "S2: %s, D2: %s",FCT,STR(S2),STR(D2)
00346         );
00347
00348     D2[0]=S2[0]=dim2_;
00349 };
00350
00351 template <class TQ, class TD, class TE>
00352 void QBlock<TQ,TD,TE>::initFromBlockMatrix(const char *F, int L,
00353     QSpace<TQ,TD> &B,
00354     const wbararray<TD> &MM,
00355     const iTags &it,
00356     const wbpemr *cgp
00357 ) const {
00358
00359     widxt i,j,k, i0,i1,i2, d1,d2, r,s, dim1, dim2;
00360     wvector<widxt> cD1, cD2, S;
00361     wvector<TQ> Q;
00362
00363     #ifndef WB_SKIP_ASSERT
00364     wvector<unsigned> Ik(1);
00365     #endif
00366
00367     char cgflag=A.getCGS(F_L);
00368     char cpR=0;
00369     double cfac=1;
00370
00371     if (cgflag>0) {
00372         r=it.len;
00373         s=S1.dim2+S2.dim2;
00374         if (s<2 || s!=r) wblog(FL,
00375         "ERR %s() got rank %d/%d",FCT,s,r);
00376
00377         if (!cgp) {
00378             if (A.itags.sameConj(it)) { cpR=1; }
00379         }
00380         else if (A.itags.len==cgp->len) {
00381             iTags tp(A.itags); tp.Permute(*cgp);
00382             if (tp.sameConj(it)) { cpR=2; }
00383         }
00384         if (!cpR) {
00385             if (it.len!=2 || !it[0].isConj(it[1],0)) wblog(FL,
00386             "ERR %s() unexpected non-abelian setting\n"
00387             "r=%d: %s (A: %s); cgp=[%s]", FCT, it.len, STR(it),
00388             STR(A.itags), STR(cgp ? (*cgp) : wbpemr())
00389             );
00390         }
00391     }
00392
00393     if (Q1.dim1!=S1.dim1 || D1.dim1!=S1.dim1 || D1.dim2<2 ||
00394     Q2.dim1!=S2.dim1 || D2.dim1!=S2.dim1 || D2.dim2<2) { wblog(FL,
00395     "ERR block index mismatch (%d/%d/%dx%d; %d/%d/%dx%d)",
00396     Q1.dim1, S1.dim1, D1.dim1, D1.dim2,
00397     Q2.dim1, S2.dim1, D2.dim1, D2.dim2);
00398 }
00399
00400     dim1=D1.cumsum0prodRec(cD1,0,1);
00401     dim2=D2.cumsum0prodRec(cD2,0,1);
00402
00403     if (MM.rank()!=2) wblog(FL,
00404     "ERR %s() got rank-%d object matrix",FCT,MM.rank());
00405     if (dim1!=MM.SIZE[0] || dim2!=MM.SIZE[1]) wblog(FL,
00406     "ERR block size mismatch (%dx%d; %s)",
00407     dim1, dim2, MM.sizeStr().data);
00408     if (!A.QDIM || Q1.dim2%A.QDIM || Q2.dim2%A.QDIM) wblog(FL,
00409     "ERR invalid QDIM (%d,%d / %d)",Q1.dim2,Q2.dim2,A.QDIM);
00410
00411     B.init(D1.colSum(1)*D2.colSum(1), S1.dim2+S2.dim2, A.QDIM);
00412     B.qtype=A.qtype;
00413     B.itags=it;
00414
00415     if (A.CGR.dim2) {
00416         B.setupCGR();

```



```

00417     }
00418
00419     for (k=r=0; r<D1.dim1; ++r) {
00420     for ( s=0; s<D2.dim1; ++s, ++k) {
00421         i1 = cD1[r]; i2 = cD2[s];
00422         d1 = D1(r,0); d2 = D2(s,0); if (!d1 || !d2) {
00423             wblog(FL,"WRN %s() got empty %dx%d sub-block",FCT,d1,d2);
00424             continue;
00425         }
00426
00427         if ((i0=idx(r,s))<0) {
00428             double x2=MM.norm2block(i1,i2,d1,d2);
00429             if (x2>CG_SKIP_DEPS1 || !isfinite(x2)) { char s[64];
00430                 snprintf(s,64,"block with norm x2=%.3g\nM(%ld:%ld,%ld:%ld) "
00431                     "having %s",x2,i1+1,i1+d1,i2+1,i2+d2,SSTR(MM));
00432                 if (x2>1e-8)
00433                     { wblog(FL,"ERR %s() got %s",FCT,s); }
00434                 else { wblog(FL,"WRN %s() skipping %s",FCT,s); }
00435             }
00436             continue;
00437         }
00438
00439         if (cpR && i0>=A.CGR.dim1) {
00440             MXPut(FL,"q").add(A,"A").add(B,"B")
00441                 .add(D1,"D1").add(cD1,"cD1").add(D2,"D2").add(cD2,"cD2")
00442                 .add(idx,"idx").add(r+1,"r").add(s+1,"s").add(k+1,"k");
00443             wblog(FL,"ERR %s() index out of bounds:\nid(%ld,%ld) = %ld/%ld",
00444                 FCT,r+1,s+1,i0+1,A.CGR.dim1);
00445         }
00446
00447         S.init(S1.dim2, S1.rec(r), S2.dim2, S2.rec(s));
00448         Q.init(Q1.dim2, Q1.rec(r), Q2.dim2, Q2.rec(s));
00449
00450         B.QIDX.recSet(k,Q); cfac=1;
00451         if (cpR) {
00452             for (j=0; j<A.CGR.dim2; ++j) {
00453                 B.CGR(k,j).init_wId( A.CGR(i0,j),1,-1,&qdim[j]);
00454
00455                 if (qdim[j]>1) { cfac*=B.CGR(k,j).NormSignW(FL); }
00456
00457                 if (cgp) { B.CGR(k,j).Permute(*cgp); }
00458             }
00459
00460             if (fabs(cfac)<1e-8) wblog(FL,
00461                 "WRN %s() got small cfac=%g",FCT,cfac);
00462
00463             #ifndef WB_SKIP_ASSERT
00464                 Ik[0]=k;
00465                 B.checkQ_CGR(FL,&Ik);
00466             #endif
00467         }
00468         else if (cgflag) {
00469             B.initIdentityCGS(FL);
00470         }
00471
00472         if (D1(r,1)==1 && D2(s,1)==1) {
00473             if (i1+d1>dim1 || i2+d2>dim2) { wblog(FL,"ERR %s() "
00474                 "index out of bounds (%d:%d; %d:%d) having %s",
00475                 FCT, i1+1,i1+d1, i2+1,i2+d2, SSTR(MM));
00476             }
00477
00478             wbararray<TD> Bk; Bk.init_bare(d1,d2);
00479             Wb::cpyStride(Bk.data, MM.ref(i1,i2), d1,d2,-1,dim1,TD(0),TD(cfac));
00480
00481             Bk.Reshape(S).save2(*B.DATA[k]);
00482         }
00483         else {
00484             unsigned m=0, wom=0; int e=0;
00485             widx_t d1_=d1, d2_=d2;
00486
00487             if (!cpR) wblog(FL,"ERR %s() "
00488                 "expecting cpR!=0 with OM=%dx%D",FCT,D1(r,1),D2(s,1));
00489
00490             if (D1(r,1)==1) { wom=2; m=D2(s,1); d2_*=m;
00491                 if (D2.dim2!=2+A.CGR.dim2) { e=1; }
00492             } else
00493             if (D2(s,1)==1) { wom=1; m=D1(r,1); d1_*=m;
00494                 if (D1.dim2!=2+A.CGR.dim2) { e=2; }
00495             }
00496             else { e=3; }
00497
00498             if (e) { char sx[80]; snprintf(sx,80,"block (%ld/%ld,%ld/%ld) "
00499                 "at OM=%ldx%ld)\nhaving %s | %s | cpR=%d, '%s' (k=%ld)",
00500                 r+1,D1.dim1, s+1,D2.dim1, D1(r,1), D2(s,1),
00501                 D1.rec2Str(r).data, D2.rec2Str(s).data, cpR,STR(it),k);
00502             if (e<3)
00503                 wblog(FL,"ERR %s() invalid D1/D2 data (e=%d)\n%s",FCT,e,sx);

```

```

00504         else wblog(FL,"ERR %s() got multiple OM for\n%s",FCT,sx);
00505     }
00506
00507     if (i1+d1_>dim1 || i2+d2_>dim2) { char sx[32];
00508         if (wom==1)
00509             { sprintf(sx,32,"(%d*%ld=%ld) x %ld", m,d1,d1_, d2); }
00510         else { sprintf(sx,32,"%ld x (%d*%ld=%ld)", d1, m,d2,d2_); }
00511
00512         wblog(FL,"ERR %s() index out of bounds "
00513             "(:d:%d/%d; %d:%d/%d)\nhaving block size %s",
00514             FCT, i1+1,i1+d1_,dim1, i2+1,i2+d2_,dim2, sx
00515         );
00516     }
00517
00518     wbararray<TD> Bk; Bk.init_bare(d1_,d2_);
00519     Wb::cpyStride(Bk.data, MM.ref(i1,i2), d1_,d2_,-1,dim1,TD(0),TD(cfac));
00520
00521     if (wom==1)
00522         { Bk.Reshape(d1,m,d2).Permute("132"); }
00523
00524     S.Append(m);
00525     Bk.Reshape(S).save2(*B.DATA[k]);
00526 }
00527 }}
00528
00529 for (k=i=0; i<B.DATA.len; ++i) {
00530     if (B.DATA[i]->isEmpty()) { continue; }
00531     if (k<i) {
00532         B.QIDX.recSet(k,i);
00533         for (j=0; j<B.CGR.dim2; ++j) { B.CGR(i,j).save2(B.CGR(k,j)); }
00534         B.DATA[k]=B.DATA[i];
00535     }; ++k;
00536 }
00537
00538 if (k<B.DATA.len) {
00539     B.QIDX.Resize(k,B.QIDX.dim2);
00540     B.CGR .Resize(k,B.CGR .dim2);
00541     B.DATA.Resize(k);
00542 }
00543 };
00544
00545 template <class TQ, class TD>
00546 void SVD_Data<TQ,TD>::blockSVD(
00547     const QSpace<TQ,TD> &PSI,
00548     const unsigned K
00549 ){
00550     unsigned i,n, l=0, r=PSI.rank(FL);
00551     wvector<widx_t> D,Dc;
00552     wbMatrix<TQ> Qb;
00553     windex Ib;
00554     wperm P;
00555
00556     if (PSI.isEmpty()) wblog(FL,"ERR %s() got empty PSI",FCT);
00557     if (!K || K>=r) wblog(FL,
00558         "ERR %s() index specs out of bounds (%d/%d)",FCT,K,r);
00559
00560     if (K!=1) { QDir qdir; PSI.getQDir(qdir);
00561         if (qdir.len!=r) l=1;
00562         else {
00563             for (i=0; i<K; ++i) { if (qdir[i]<=0) { l=2; break; } }
00564             for (++i; i<r; ++i) { if (qdir[i]!=qdir[K]) { l=3; break; } }
00565         }
00566         if (l) wblog(FL,
00567             "ERR %s() got invalid orthonormalization\n"
00568             "['%s' => %s grouped into %d+%d, »]\nhint: %s",
00569             FCT, STR(PSI.itags), STR(qdir), K,r-K, (l==2 ?
00570             "only ingoing indices can be fused into outgoing" : (l==3 ?
00571             "2nd/right group is mixed in+out" : "rank mismatch"))
00572         );
00573     }
00574
00575     if (K== 1) { Ib.Index(0,K-1); } else
00576     if (K==r-1) { Ib.Index(K,r-1); }
00577     else {
00578         if (!PSI.allAbelian()) wblog(FL,
00579             "ERR %s() got non-abelian symmetries %s\nwith K=%d/%d",
00580             FCT, PSI.qStr().data,K,r);
00581         Ib.Index(0,K-1);
00582     }
00583
00584     PSI.getQsum(Ib,Qb).groupRecs(P,D);
00585     n=D.len; QB.init(n);
00586
00587     Wb::LogException ex;
00588
00589     int np=MIN(int(n),QSP_NUM_THREADS);
00590     if (np<1) np=1;

```

```

00591
00592     D.cumsum_(Dc);
00593
00594     #pragma omp parallel for num_threads(np)
00595     for (i=0; i<n; ++i) { if (!ex) { try {
00596         QBlock<TQ,TD> &b=QB[i];
00597         wbarry<TD> MM;
00598
00599         b.I0.init(D[i],P.data+Dc[i]);
00600         PSI.getSub(b.I0, b.A);
00601
00602         b.toBlockMatrix(MM, K);
00603
00604         wbSVD(MM, b.U, b.S, b.Vc);
00605     }
00606         catch (Wb::LogException &e_) { ex+=e_; }
00607         catch (...) { ++ex; }
00608     }}
00609     ex.report(FLF);
00610 };
00611
00612 template <class TQ, class TD>
00613 unsigned SVD_Data<TQ,TD>::dmrgTruncate(
00614     unsigned Nkmin, unsigned Nkeep, double stol,
00615     itag_ tx,
00616     wbMatrix<double> &SM,
00617     QSpace<TQ,TD> &UQ,
00618     QSpace<TQ,TD> *SQ,
00619     QSpace<TQ,TD> &VC,
00620     mxArray *Sout,
00621     char vflag
00622 ){
00623
00624     unsigned i,j,i0,d, nk=0, nt=0, nq=QB.len, flag=0;
00625     unsigned QDIM, N;
00626     int NK=Nkeep;
00627
00628     int revert_Uidx=0;
00629
00630     iTags tU,tS,tV;
00631     wbperm P, cgp2;
00632
00633     QBlock<TQ,TD,double> x;
00634     QSpace<TQ,TD> X;
00635
00636     wbvector< wbvector<double>* > SV;
00637     wbvector< wbvector<char> > mark;
00638     wbvector<double> Stot;
00639
00640     double snorm, snorm2=0, s2t=0, sfac=1,
00641         xtol2=(stol>0 ? stol*1e-14 : 0);
00642     char cgflag=0;
00643
00644     unsigned l=0, n=128; char istr[n]; istr[0]=0;
00645
00646     if (!nq) wblog(FL,"ERR %s() got empty QB data",FCT);
00647     else {
00648         QDIM=QB[0].A.QDIM;
00649         if (!QDIM) wblog(FL,"ERR QDIM=%d",QDIM);
00650         cgflag=QB[0].A.gotCGS(FL);
00651     }
00652
00653     UQ.init(); if (SQ) { SQ->init(); }
00654     VC.init();
00655
00656     for (N=i=0; i<nq; i++) { N+=QB[i].S.len; }
00657
00658     SV.init(nq);
00659     SM.init(N, cgflag? 2:1);
00660
00661     if (!N) wblog(FL,"ERR %s() got empty SVD data",FCT);
00662     else --N;
00663
00664     for (i0=i=0; i<nq; ++i, i0+=d) {
00665         const wbvector<double> &S=QB[i].S; d=S.len;
00666
00667         SV[i]=&(QB[i].S);
00668         snorm2+=S.norm2()*QB[i].qdim_tot;
00669
00670         if (!cgflag) {
00671             for (j=0; j<d; ++j) { SM[i0+j]=S[j]; }
00672         }
00673         else {
00674             double *sm=SM.rec(i0);
00675             for (j=0; j<d; ++j, sm+=2) {
00676                 sm[0]=S[j];

```

```

00678         sm[l]=QB[i].qdim_tot;
00679     }
00680 }
00681 }
00682
00683 SM.SortRecs(P,-1);
00684
00685 if (int(Nkmin)<0) { Nkmin=0; }
00686
00687 snorm=Wb::sqrt(snorm2);
00688 markSet(SV,mark,Nkmin,NK,stol,Stot,0.,0.,-1,"desc");
00689
00690 for (i=0; i<nq; ++i) {
00691     QBlock<TQ,TD> &b=QB[i];
00692     s2t+=b.get_s2t(mark[i]);
00693     nk+=b.Ik.len; nt+=(mark[i].len-b.Ik.len);
00694 }
00695
00696 if (nk!=unsigned(NK) || nk+nt!=SM.dim1) wblog(FL,
00697     "WRN size inconsistency (nk=%d/%d, N=%d+%d / %d)",
00698     nk,NK, nk,nt,SM.dim1);
00699
00700 if (SM(N,0)>stol) { if (SM.dim1>Nkeep) {
00701     if (stol) {
00702         l+=snprintf(istr,n,"Smin=%.3g > stol=%.3g (NK=%d/%d/%li)",
00703             SM(N,0), stol, Nkmin,NK,SM.dim1); flag++;
00704     }
00705     else {
00706         l+=snprintf(istr,n,"using Nkeep=%d (%d/%d/%li; "
00707             "stol=%g; Smin=%.3g)",NK,Nkmin,Nkeep,SM.dim1, stol, SM(N,0));
00708     }
00709     else {
00710         l+=snprintf(istr,n,"keeping all (stol=%g; Smin=%.3g, "
00711             "NK=%d (%d/%d/%li)",stol, SM(N,0), NK,Nkmin,Nkeep,SM.dim1);
00712         xtol2=0;
00713     }
00714 }
00715 else {
00716     if (unsigned(NK)<Nkeep) {
00717         if (Nkmin) {
00718             l+=snprintf(istr,n,"truncation by stol=%.3g or Nkmin=%d "
00719                 "(Nkeep %d->%d; Smin=%.3g < %.3g)", stol,
00720                 Nkmin,Nkeep,NK, SM(N,0), SM(nk? nk-1:0,0));
00721         }
00722         else {
00723             l+=snprintf(istr,n,"truncation by stol=%.3g "
00724                 "(Nkeep %d->%d; Smin=%.3g < %.3g)", stol,
00725                 Nkeep,NK, SM(N,0), SM(nk? nk-1:0,0));
00726         }
00727     }
00728     else if (unsigned(NK)<SM.dim1) {
00729         l+=snprintf(istr,n,"truncation by Nkeep ([%d,%d] -> %d/%li; "
00730             "Smin=%.3g < %.3g <= stol=%.3g)", Nkmin,Nkeep,NK,SM.dim1,
00731             SM(N,0), SM(MAX(0,int(NK)-1),0), stol);
00732         flag++;
00733     }
00734 }
00735
00736 if (s2t) {
00737     if (l>=n) wblog(FL,"ERR %s() "
00738         "string out of bounds (%d/%d)%N%Ns%N",FCT,l,n,istr);
00739     else if (snorm2>s2t) {
00740         sfac=snorm/sqrt(snorm2-s2t);
00741         if (fabs(sfac-1)>0.01) { ++flag;
00742             snprintf(istr+l,n-1,
00743                 "%sWRN readjusting norm by factor %.4g",l?"\n":""",sfac);
00744             if (fabs(sfac-1)>0.1 && (vflag || Wb::envVRB&8)) {
00745                 wblog(FL,istr+(l?l+1:0));
00746             }
00747         }
00748         for (i=0; i<nq; ++i) QB[i].S*=sfac;
00749     }
00750     else if (snorm2==s2t) { snprintf(istr+l,n-1,
00751         "all states discarded (|SVD|=%.3g / %.3g; Nkeep=%d/%d/%li)",
00752         snorm,stol,NK,Nkeep,SM.dim1);
00753     }
00754     else wblog(FL,"ERR %s() %g/%g",FCT,snorm2,s2t);
00755 }
00756
00757 if (Sout) {
00758     mxAddField2Scalar(FL, Sout, "svd", SM.toMx());
00759     mxAddField2Scalar(FL, Sout, "stol", numtoMx(stol));
00760     mxAddField2Scalar(FL, Sout, "svd2tr", numtoMx(s2t/snorm2));
00761     mxAddField2Scalar(FL, Sout, "sv2tot", numtoMx(snorm2));
00762     mxAddField2Scalar(FL, Sout, "sfac", numtoMx(sfac));
00763     mxAddField2Scalar(FL, Sout, "Nkmin", numtoMx(Nkmin));
00764     mxAddField2Scalar(FL, Sout, "Nkeep", numtoMx(nk));

```

```

00765     mxAddField2Scalar(FL, Sout, "Ntot",    numtoMx(SM.dim1));
00766     mxAddField2Scalar(FL, Sout, "flag",    numtoMx(flag));
00767     mxAddField2Scalar(FL, Sout, "info",    mxCreateString(istr));
00768 }
00769 else if (vflag) {
00770     wblog(FL, "<i> sum(SVD) = %g = %g%.4g",
00771         snorm2, round(snorm2), snorm2-round(snorm2));
00772     wblog(FL, "TST keep %d/%d states (%d)", nk, SM.dim1, Nkeep); if (nt) {
00773         wblog(FL, ":x: truncate %d/%d states", nt, SM.dim1); }
00774 }
00775
00776 tx.deConj();
00777
00778 {   QBlock<TQ,TD> &b=QB[0];
00779
00780     itag_ tu(tx), tv;
00781
00782     b.A.checkQ(FL);
00783
00784     cgp2.init2End(0, b.S2.dim2+1);
00785
00786     if (b.A.itags.len!=b.S1.dim2+b.S2.dim2) wblog(FL,
00787         "ERR %s() got invalid/empty itags (%s; s=%d+%d)",
00788         FCT, b.A.itags.toStr().data, b.S1.dim2+b.S2.dim2);
00789
00790     if (b.S2.dim2==1) {
00791         tu.Set_(b.A.itags.last(i));
00792         if (!tu.isConj() && NK) {
00793             if (!b.A.isOp(2, 'L')) { revert_Uidx=1; }
00794         }
00795     }
00796     else if (b.S1.dim2==1) {
00797         tu.Set_(b.A.itags[i=0]); tu.Conj();
00798     }
00799     else { i=-1;
00800         if (!b.A.itags[0].isConj()) { tu.Conj(); }
00801     }
00802
00803     tv=tu;
00804     if (SQ && i==1 && b.A.rank()==2) {
00805         tu.tSet(b.A.itags[0]);
00806         if (!b.A.itags[1].isConj() && revert_Uidx) { tu.MarkDual(); }
00807     }
00808
00809     tU.init(b.S1.dim2, b.A.itags.data, 1, &tu);
00810
00811     tS.init(1, &tu, 1, &tv);
00812     tS[0].Conj();
00813
00814     tV.init(b.S2.dim2, b.A.itags.data+b.S1.dim2, 1, &tv);
00815     tV.last().Conj();
00816 }
00817
00818 for (i=0; i<nq; ++i) {
00819     QBlock<TQ,TD> &b=QB[i]; if (b.Ik.isEmpty()) continue;
00820     b.U .Select0(b.Ik, 1);
00821     b.S .Select(b.Ik);
00822     b.Vc.Select0(b.Ik, 1);
00823
00824     if (b.U .rank()!=2) wblog(FL, "ERR U: invalid rank %s", SSTR(b.U));
00825     if (!b.S.len ) wblog(FL, "ERR got empty S");
00826     if (b.Vc.rank()!=2) wblog(FL, "ERR V: invalid rank %s", SSTR(b.Vc));
00827
00828     x.init_bare_refA(b, 'U');
00829     x.initFromBlockMatrix(FL, X, b.U, tU);
00830
00831     if (X.Append2AndDestroy(FL, UQ, 'u')) wblog(FL,
00832         "ERR %s() U: QIDX must not overlap\n%s", FCT, istr);
00833
00834     if (SQ) {
00835         wbarry<double> Sf;
00836         Sf.initDiag(b.S.len, b.S.data);
00837
00838         x.init_bare_refA(b, 'S');
00839         x.initFromBlockMatrix(FL, X, Sf, tS);
00840
00841         if (X.DATA.len!=1) wblog(FL,
00842             "ERR %s() got S.DATA.len=%d", FCT, X.DATA.len);
00843         X.DATA[0]->Reduce2Diag();
00844
00845         if (X.Append2AndDestroy(FL, *SQ, 'u')) wblog(FL,
00846             "ERR %s() S: QIDX must not overlap\n%s", FCT, istr);
00847
00848         x.init_bare_refA(b, 'V');
00849         x.initFromBlockMatrix(FL, X, b.Vc, tV, &cgp2);
00850     }
00851 }

```

```

00852         if (X.Append2AndDestroy(FL,VC,'u')) wblog(FL,
00853             "ERR %s() V: QIDX must not overlap\n%s",FCT,istr);
00854     }
00855     else {
00856         wbararray<TD> VS; Wb::DMatProd(b.Vc,b.S,VS);
00857
00858         if (VS.rank()!=2) wblog(FL,
00859             "ERR VS: invalid rank-%d",VS.rank());
00860
00861         x.init_bare_refA(b,'X');
00862         x.initFromBlockMatrix(FL,X,VS,tV,&cgp2);
00863
00864         X.SkipZeroData(xtol2,'b',0);
00865
00866         if (X.Append2AndDestroy(FL,VC,'u')) wblog(FL,
00867             "ERR %s() VS: QIDX must not overlap\n%s",FCT,istr
00868         );
00869     }
00870 }
00871 }
00872
00873 UQ.checkQ(FL); if (SQ) SQ->checkQ(FL);
00874 VC.checkQ(FL);
00875
00876 if (revert_Uidx) {
00877     unsigned r1=UQ.rank(FL); wbindx ia(1);
00878     wvector< QSpace<TQ,TD> >X(1);
00879     QSpace<TQ,TD> U1J;
00880
00881     if (UQ.qtype!=VC.qtype || (!UQ.qtype.allAbelian() &&
00882         (!UQ.CGR.data || !VC.CGR.data))) wblog(FL,"ERR %s() "
00883         "got uninitialized CGR data\n(0x%lx, 0x%lx having %s)",
00884         FCT, UQ.CGR.data, VC.CGR.data, STR(VC.qtype));
00885     if (VC.itags.len!=2) wblog(FL,"ERR %s() "
00886         "unexpected rank-%d for %s",FCT,VC.itags.len, SQ ? "V":"VS");
00887
00888     if (SQ) {
00889         SQ->save2(X[0]); U1J.initIdentityCG(X,ia,'z');
00890         U1J.contract(FL,1,X[0],ia[0]+1,*SQ);
00891     }
00892     else {
00893         ia[0]=VC.rank(FL)-1;
00894         VC.save2(X[0]); U1J.initIdentityCG(X,ia,'z');
00895         X[0].contract(FL,ia[0]+1,U1J,1,VC);
00896     }
00897 }
00898
00899 UQ.save2(X[0]); {
00900     X[0].contract(FL,r1,U1J,"1*",UQ);
00901 }
00902 }
00903
00904 return flag;
00905 };
00906
00907 char mxGetLRDir(
00908     const char *F, int L,
00909     const mxArray *a, unsigned k
00910 ){
00911     char w=0, e=0; char s[8];
00912
00913     if (mxIsChar(a)) {
00914         if (mxGetString(a,s,8)) {
00915             if (F) wblog(F,L,
00916                 "ERR failed to read lrdir string (arg #%d)",k);
00917             return w;
00918         }
00919
00920         if (!strcmp(s,"»")) w+=3; else
00921         if (!strcmp(s,"«")) w-=3; else
00922         if (!strcmp(s,"LR")) w+=2; else
00923         if (!strcmp(s,"RL")) w=-2; else
00924         if (!strcmp(s,"+" )) w+=1; else
00925         if (!strcmp(s,"-" )) w=-1; else {
00926             if (F) wblog(F,L,
00927                 "ERR invalid lrdir (arg #%d: '%s')",k,s);
00928             return w;
00929         }
00930     }
00931     else if (Mx::IsNumber(0,0,a)) {
00932         if (mxGetNumber(a,w)) e=1;
00933         else {
00934             if (!w || w<-3 || w>+3) { e=2; }
00935         }
00936         if (e) {
00937             if (F) wblog(F,L,
00938                 "ERR invalid lrdir (arg #%d: %d; e=%d)",k,w,e);

```

```

00939         w=0; return w;
00940     }
00941 }
00942 else e=3;
00943
00944 if (!w) e=4;
00945 if (e) {
00946     if (F) wblog(F,L,
00947         "ERR invalid ldir (arg #%d; e=%d)",k,e);
00948     return w;
00949 }
00950
00951 return w;
00952 };
00953
00954 template <class TQ, class T1, class T2, class T3>
00955 void twoSiteInit(
00956     const QSpace<TQ,T1> &A1,
00957     const QSpace<TQ,T2> &A2,
00958     QSpace<TQ,T3> &PSI, unsigned &K,
00959     widx_t ic1, widx_t ic2,
00960     char twoSite, char ldir
00961 ){
00962     const unsigned r1=A1.rank(), r2=A2.rank();
00963     wbindx I1, I2;
00964     wperm P;
00965
00966     if (!ldir) wblog(FL,"ERR ldir=%d",ldir);
00967     if ((ldir>0 && ic1>=r1) || (ldir<0 && ic2>=r2)) wblog(FL,
00968         "ERR contraction index out of bounds (%d/%d, %d/%d)",
00969         ic1,r1,ic2,r2);
00970
00971     if (twoSite) {
00972         if (ldir>0) {
00973             A1.contract(ic1+1, A2, ic2+1, PSI);
00974             I1.Index(r1-1); I2.Index(r2-1)+=(r1-1);
00975         }
00976         else {
00977             A2.contract(ic2+1, A1, ic1+1, PSI);
00978             I1.Index(r2-1); I2.Index(r1-1)+=(r2-1);
00979         }
00980     }
00981     else {
00982         if (ldir>0)
00983             { PSI=A1; I1.Index(r1).Skip(ic1); I2.init(1,&ic1); }
00984         else { PSI=A2; I1.Index(r2).Skip(ic2); I2.init(1,&ic2); }
00985     }
00986
00987     if (PSI.isEmpty()) wblog(FL,"WRN %s yields empty QSpace",FCT);
00988
00989     P.Cat(I1,I2);
00990     PSI.Permute(P); K=I1.len;
00991 };
00992
00993 template <class TQ, class T1, class T2, class T3>
00994 void twoSiteFinal(
00995     const QSpace<TQ,T1> &Psi1,
00996     const QSpace<TQ,T2> &Psi2,
00997     QSpace<TQ,T3> &A1,
00998     QSpace<TQ,T3> &A2,
00999     unsigned ic1, unsigned ic2, char twoSite, char ldir
01000 ){
01001     QSpace<TQ,T3> AX;
01002
01003     if (!ldir) wblog(FL,"ERR ldir=%d",ldir);
01004
01005     if (twoSite) {
01006         if (ldir>0) {
01007             A1.PermuteLastTo (ic1);
01008             A2.PermuteFirstTo(ic2);
01009         }
01010         else {
01011             A2.permuteFirstTo(ic1, AX);
01012             A1.permuteLastTo (ic2, A2); AX.save2(A1);
01013         }
01014     }
01015     else {
01016         if (ldir>0) {
01017             A1.PermuteLastTo(ic1);
01018             A2.contract(2, Psi2, ic2+1, AX);
01019             AX.permuteFirstTo(ic2, A2);
01020         }
01021         else {
01022             A2.contract(1, Psi1, ic1+1, AX);
01023             A1.permuteLastTo(ic2, A2);
01024             AX.permuteFirstTo(ic1,A1);
01025         }
01026     }
01027 }

```

```

01026     }
01027 }
01028
01029 template <class TQ, class TD>
01030 mxArray* orthoQS(
01031     const QSpace<TQ,TD> &PSI,
01032     QSpace<TQ,TD> &A1,
01033     QSpace<TQ,TD> &A2,
01034     unsigned K,
01035     unsigned Nkmin,
01036     unsigned Nkeep,
01037     double stol,
01038     itag_ tx,
01039     const char *info
01040 ){
01041     mxArray *Sout=NULL;
01042     unsigned i=0;
01043     char vflag=0, sflag=0;
01044
01045     SVD_Data<TQ,TD> SVD;
01046     wbMatrix<double> SV;
01047
01048     if (info) {
01049         for (;;) ++i) {
01050             if (info[i]=='d') { vflag|=1; } else
01051             if (info[i]=='D') { vflag|=2; } else
01052             if (info[i]=='s') { sflag|=1; } else
01053             if (info[i]=='S') { sflag|=2; } else break;
01054         }
01055         if (i>3 || info[i])
01056             wblog(FL,"ERR %s() invalid flags '%s' (i=%d)",FCT,info,i);
01057     }
01058
01059     if (sflag)
01060         Sout=mxCreateStructMatrix(1,1,0,NULL);
01061
01062     SVD.blockSVD(PSI,K);
01063
01064     i=SVD.dmrgrTruncate(Nkmin,Nkeep,stol,tx,SV, A1,NULL,A2, Sout,vflag);
01065
01066     if (Sout) {
01067         mxAddField2Scalar(FL,Sout,"K", numtoMx(K));
01068         mxAddField2Scalar(FL,Sout,"D1", A1 .getDim().toMx());
01069         mxAddField2Scalar(FL,Sout,"DD", PSI.getDim().toMx());
01070     }
01071     else if (i) wblog(FL,str);
01072
01073     if (sflag>1)
01074         mxAddField2Scalar(FL,Sout,"SVD", SVD.toMx());
01075     return Sout;
01076 };
01077
01078 template <class TQ, class TD>
01079 mxArray* getSVD(
01080     const QSpace<TQ,TD> &PSI,
01081     QSpace<TQ,TD> &U,
01082     QSpace<TQ,TD> &S,
01083     QSpace<TQ,TD> &V,
01084     unsigned K,
01085     unsigned Nkmin,
01086     unsigned Nkeep,
01087     double stol,
01088     itag_ tx,
01089     const char *info
01090 ){
01091     SVD_Data<TQ,TD> SVD;
01092     wbMatrix<double> SV;
01093
01094     mxArray *Sout=NULL;
01095     unsigned i=0; char sflag=0, vflag=0;
01096
01097     if (info) {
01098         for (;;) ++i) {
01099             if (info[i]=='d') { vflag|=1; } else
01100             if (info[i]=='D') { vflag|=2; } else
01101             if (info[i]=='s') { sflag|=1; } else
01102             if (info[i]=='S') { sflag|=2; } else break;
01103         }
01104         if (i>3 || info[i])
01105             wblog(FL,"ERR %s() invalid flags '%s' (i=%d)",FCT,info,i);
01106     }
01107
01108     if (sflag)
01109         Sout=mxCreateStructMatrix(1,1,0,NULL);
01110
01111     SVD.blockSVD(PSI,K);
01112

```



```

01113     i=SVD.dmrgrTruncate(Nkmin,Nkeep,stol,tx,SV, U,&S,V, Sout,vflag);
01114
01115     if (Sout) {
01116         mxAddField2Scalar(FL,Sout,"K", numtoMx(K));
01117         mxAddField2Scalar(FL,Sout,"D1", U.getDim().toMx());
01118         mxAddField2Scalar(FL,Sout,"DD",PSI.getDim().toMx());
01119     }
01120     else if (i) wblog(FL,str);
01121
01122     if (sflag>1)
01123         mxAddField2Scalar(FL,Sout,"SVD", SVD.toMx());
01124     return Sout;
01125 };
01126
01127 template <class TQ, class TD>
01128 mxArray* SVD_Data<TQ,TD>::toMx() const {
01129
01130     const char *fn[]{"Q1","Q2","S1","S2","U","S","Vc","A","IO","Ik"};
01131
01132     mxArray *S=mxCreateStructMatrix(QB.len,1,10,fn);
01133
01134     for (unsigned i=0; i<QB.len; ++i) {
01135         mxSetFieldByNumber(S,i,0, QB[i].Q1.toMx() );
01136         mxSetFieldByNumber(S,i,1, QB[i].Q2.toMx() );
01137         mxSetFieldByNumber(S,i,2, QB[i].S1.toMx() );
01138         mxSetFieldByNumber(S,i,3, QB[i].S2.toMx() );
01139         mxSetFieldByNumber(S,i,4, QB[i].U .toMx() );
01140         mxSetFieldByNumber(S,i,5, QB[i].S .toMx() );
01141         mxSetFieldByNumber(S,i,6, QB[i].Vc.toMx() );
01142         mxSetFieldByNumber(S,i,7, QB[i].A .toMx() );
01143         mxSetFieldByNumber(S,i,8, QB[i].IO.toMx() );
01144         mxSetFieldByNumber(S,i,9, QB[i].Ik.toMx() );
01145     }
01146
01147     return S;
01148 }
01149
01150 #endif

```

5.13 mpsortho.hh

```

00001 #ifndef __WB_ORTHO_HH__
00002 #define __WB_ORTHO_HH__
00003
00004 #define WB_STOL_SVD 1E-8
00005
00006 /* ----- */
00007 /* ----- */
00008 // change log
00009 // * split off mpsortho.hh from mpsortho.cc
00010 // * transferred former QSpace member functions
00011 // toBlockMatrix() and initFromBlockMatrix()
00012 // to class QBlock(), as it fits more naturally there.
00013 // Wb,Aug05,15
00014
00015 template <class TQ, class TD>
00016 mxArray* orthoQS(
00017     const QSpace<TQ,TD> &PSI,
00018     QSpace<TQ,TD> &A1,
00019     QSpace<TQ,TD> &A2, unsigned K,
00020     unsigned Nkmin, unsigned Nkeep,
00021     double stol,
00022     itag_ tx,
00023     const char *info
00024 );
00025
00026 template <class TQ, class TD>
00027 mxArray* getSVD(
00028     const QSpace<TQ,TD> &PSI,
00029     QSpace<TQ,TD> &U,
00030     QSpace<TQ,TD> &S,
00031     QSpace<TQ,TD> &VT, unsigned K,
00032     unsigned Nkmin, unsigned Nkeep, double stol,
00033     itag_ tx,
00034     const char *info
00035 );
00036
00037 char mxGetLRDir(
00038     const char *F, int L, const mxArray *argin, unsigned k
00039 );
00040
00041 template <class TQ, class T1, class T2, class T3>
00042 void twoSiteInit(

```

```

00043     const QSpace<TQ,T1> &Psi1,
00044     const QSpace<TQ,T2> &Psi2,
00045     QSpace<TQ,T3> &PSI, unsigned &K,
00046     widx_t ic1, widx_t ic2,
00047     char twoSite, char ldir
00048 );
00049
00050 template <class TQ, class T1, class T2, class T3>
00051 void twoSiteFinal(
00052     const QSpace<TQ,T1> &Psi1,
00053     const QSpace<TQ,T2> &Psi2,
00054     QSpace<TQ,T3> &A1,
00055     QSpace<TQ,T3> &A2,
00056     unsigned ic1, unsigned ic2, char twoSite, char ldir
00057 );
00058
00059 template <class TQ, class TD, class TE=double>
00060 class QBlock {
00061 public:
00062
00063     QBlock() : qdim_tot(1) {};
00064
00065     QBlock& init() { qdim_tot=1; qdim.init();
00066         IO.init(); Ik.init(); It.init(); idx.init();
00067         Q1.init(); S1.init(); D1.init(); A.init(); S.init();
00068         Q2.init(); S2.init(); D2.init(); U.init(); Vc.init();
00069         return *this;
00070     };
00071
00072     QBlock& operator=(const QBlock &B) {
00073         if (this!=&B) { qdim_tot=B.qdim_tot; qdim=B.qdim;
00074             IO=B.IO; Ik=B.Ik; idx=B.idx;
00075             Q1=B.Q1; S1=B.S1; D1=B.D1; A=B.A; S=B.S;
00076             Q2=B.Q2; S2=B.S2; D2=B.D2; U=B.U; Vc=B.Vc;
00077         }
00078         return *this;
00079     };
00080
00081     QBlock& init_bare_refA(const QBlock &B, unsigned w=0);
00082
00083     void toBlockMatrix(
00084         wbararray<TD> &MM,
00085         unsigned K,
00086         const wbarperm &P=wbarperm()
00087     );
00088
00089     void initFromBlockMatrix(const char *F, int L,
00090         QSpace<TQ,TD> &B,
00091         const wbararray<TD> &MM,
00092         const iTags &it,
00093         const wbarperm *cgp=NULL
00094     ) const;
00095
00096     void updateBlockDim(unsigned dim2);
00097
00098     double get_s2t(const wvector<char> &mark) {
00099         double s2t=0; unsigned i=0;
00100         if (mark.len!=S.len) wblog(FL,"ERR %s() "
00101             "length mismatch (mark.len=%d/%d)",FCT,mark.len,S.len);
00102         mark.find(Ik);
00103         for (; i<mark.len; ++i) { if (!mark[i]) s2t+=S[i]*S[i]; }
00104         if (qdim_tot!=1) { s2t*=qdim_tot; }
00105         return s2t;
00106     };
00107
00108     void print(
00109         const char *F=NULL, int L=0, const iTags *t=NULL
00110     ) const {
00111
00112         wblog(F_L,"%s() QBlock content ===== %N",FCT);
00113         A.print("sub(PSI)"); if (t) {
00114             printf("=> itags: \"%s\\n\",STR_(t)); }
00115             printf("      qdim_tot=%d\\n\\n",qdim_tot);
00116             printf("      Q =[ %s; %s ]\\n",STR(Q1),STR(Q2));
00117             printf("      S =[ %s; %s ]\\n",STR(S1),STR(S2));
00118             printf("      D1=[ %s ]\\n",STR(D1));
00119             printf("      D2=[ %s ]\\n\\n",STR(D2));
00120         };
00121
00122     mxArray* toMx() const {
00123         const char *f[]={ "A","qdim_tot","qdim",
00124             "Q1","S1","D1", "Q2","S2","D2",
00125             "IO","Ik","idx","U", "S", "Vc"
00126         };
00127         unsigned i=0, n=1;
00128         mxArray *q=mxCreateStructMatrix(n,1,15,f);
00129

```

```

00130         mxSetFieldByNumber(q,i, 0, A .toMx());
00131         mxSetFieldByNumber(q,i, 1, numtoMx(double(qdim_tot)));
00132         mxSetFieldByNumber(q,i, 2, qdim.toMx());
00133         mxSetFieldByNumber(q,i, 3, Q1.toMx());
00134         mxSetFieldByNumber(q,i, 4, S1.toMx());
00135         mxSetFieldByNumber(q,i, 5, D1.toMx());
00136         mxSetFieldByNumber(q,i, 6, Q2.toMx());
00137         mxSetFieldByNumber(q,i, 7, S2.toMx());
00138         mxSetFieldByNumber(q,i, 8, D2.toMx());
00139         mxSetFieldByNumber(q,i, 9, I0.toMx());
00140         mxSetFieldByNumber(q,i,10, Ik.toMx());
00141         mxSetFieldByNumber(q,i,11, idx.toMx());
00142         mxSetFieldByNumber(q,i,12, U .toMx());
00143         mxSetFieldByNumber(q,i,13, S .toMx());
00144         mxSetFieldByNumber(q,i,14, Vc.toMx());
00145
00146         return q;
00147     };
00148
00149     void put(const char *vname, const char *ws="caller") {
00150         mxArray *a=toMx();
00151         mxPutAndDestroy(FL,a,vname,ws);
00152     };
00153
00154     unsigned qdim_tot;
00155     wbvector<unsigned> qdim;
00156
00157     QSpace<TQ,TD> A;
00158     wbindex I0;
00159     wbindex Ik;
00160     wbindex It;
00161
00162     wbMatrix<TQ> Q1, Q2;
00163     wbMatrix<widx_t> S1, S2;
00164     wbMatrix<widx_t> D1, D2;
00165
00166     wbMatrix<int> idx;
00167
00168     wbarray<TD> U,Vc;
00169     wbvector<TE> S;
00170
00171 protected:
00172 private:
00173 };
00174
00175 template <class TQ, class TD>
00176 class SVD_Data {
00177 public:
00178     SVD_Data() {};
00179
00180     void blockSVD(const QSpace<TQ,TD> &PSI, const unsigned K);
00181
00182     unsigned dmrgTruncate(
00183         unsigned Nkmin, unsigned Nkeep, double stol,
00184         itag_ tx,
00185         wbMatrix<double> &SV,
00186         QSpace<TQ,TD> &UQ,
00187         QSpace<TQ,TD> *SQ,
00188         QSpace<TQ,TD> &VC,
00189         mxArray *Sout=NULL, char vflag=0
00190     );
00191
00192     mxArray* toMx() const;
00193
00194     void put(const char *vname, const char *ws="base") const {
00195         put(0,0,vname,ws);
00196     };
00197
00198     void put(const char *F, int L,
00199             const char *vname, const char *ws="base"
00200     ) const {
00201         mxArray *a=toMx();
00202         int i=mexPutVariable(ws,vname,a);
00203         if (i) wblog(F,L,"ERR failed to write variable '%s' (%d)",vname,i);
00204         if (F) wblog(F,L,"I/O putting variable '%s' to '%s'",vname,ws);
00205         mxDestroyArray(a);
00206     };
00207
00208     wbvector< QBlock<TQ,TD,double> > QB;
00209
00210 protected:
00211 private:
00212 };
00213
00214 #endif
00215

```

5.14 mxlib.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : QSpace MEX routines
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_MXLIB_HH__
00021 #define __WB_MXLIB_HH__
00022
00023 /* ----- */
00024 // switching to matlab 2018a => got interleaved complex format
00025 // => phasing out mxGetPr_17(FL,) and mxGetPi_17(FL,)
00026
00027 #ifdef MATLAB_MEX_FILE
00028
00029     mxDouble* mxGetPr_17(const char *F, int L, const mxArray *pm) {
00030         wblog(F_L, "ERR mxGetPr() no longer supported in matlab>=2018");
00031         return 0;
00032     };
00033
00034     mxDouble* mxGetPi_17(const char *F, int L, const mxArray *pm) {
00035         wblog(F_L, "ERR mxGetPi() no longer supported in matlab>=2018");
00036         return 0;
00037     };
00038
00039 #endif
00040
00041 int isHelpIndicator(const char *s) {
00042     if (s && s[0]!='-') {
00043         if (!strcmp(s, "?") || !strcmp(s, "-h") || !strcmp(s, "--help"))
00044             return 1;
00045     }
00046     return 0;
00047 };
00048
00049 #ifdef MATLAB_MEX_FILE
00050
00051 int isHelpIndicator(const mxArray *a) {
00052     char s[8];
00053     if (!a || mxGetString(a, s, 8)) { return 0; }
00054     return isHelpIndicator(s);
00055 };
00056
00057 int checkHelpVersion(const mxArray *a0, mxArray **argout) {
00058
00059     char s[16];
00060     if (a0 && !mxGetString(a0, s, 16)) {
00061         if (isHelpIndicator(s)) { usage(); return 1; }
00062
00063         if (!strcmp(s, "--ping")) { return 2; }
00064
00065         if (!strcmp(s, "--version")) { Wb::VersionInfo I;
00066             if (argout)
00067                 { argout[0]=I.toMx(); }
00068             else { I.print(); }
00069             return 1;
00070         }
00071     }
00072     return 0;
00073 };
00074
00075 #else
00076
00077 int checkHelpVersion(const char *s) {
00078     if (s) {
00079         if (isHelpIndicator(s)) { usage(); return 1; }
00080         if (!strcmp(s, "--ping")) { return 2; }
00081
00082         if (!strcmp(s, "--version")) {
00083             Wb::VersionInfo().print();
00084             return 1;
00085         }
00086     }

```

```

00086     }
00087     return 0;
00088 };
00089
00090 #endif
00091
00092 inline mxArray* mx_get_field(mxArray *S, unsigned k, int i) {
00093     if (S && i>=0 && int(k)>=0) {
00094         mxArray *a=mxGetFieldByNumber(S,k,i);
00095         if (a && mxGetNumberOfElements(a)) return a;
00096     }
00097     return NULL;
00098 };
00099
00100 void mxPutArray(const char *file, int line,
00101     mxArray *a, const char *vname="ans", const char* ws="base",
00102     const char* dstr="");
00103
00104 void mxPutAndDestroy(const char *file, int line,
00105     mxArray *a, const char *vname="ans", const char* ws="base");
00106
00107 namespace Mx {
00108
00109 int IsEqual(const mxArray *a, const char *s) {
00110     int n=strlen(s)+1;
00111     char sx[n+1]; sx[0]=0;
00112
00113     if (mxGetString(a,sx,n)) { return 0; }
00114     else { return (strcmp(sx,s) ? 0 : 1); }
00115 };
00116
00117 template <class T>
00118 class Array {
00119 public:
00120     Array() : ax(0), data(0),
00121         len(0), rank(0), trans(0), cmplx(0), mxref(0) { };
00122
00123     Array(unsigned ndim, size_t *dims, char t_=0, char c_=-1)
00124         : ax(0), data(0), len(0), rank(ndim), trans(t_), cmplx(c_), mxref(0) {
00125         if (cmplx<0) { cmplx=ISCOMPLX_(T); }
00126         init(ndim,dims);
00127     };
00128
00129     Array(size_t d1, size_t d2, char t_=0, char c_=-1)
00130         : ax(0), data(0), len(0), rank(2), trans(t_), cmplx(c_), mxref(0) {
00131         size_t dims[2]={d1,d2}; if (cmplx<0) { cmplx=ISCOMPLX_(T); }
00132         init(2,dims);
00133     };
00134
00135     Array(const wvector<size_t> &S, char t_=0, char c_=-1);
00136
00137     Array(const mxArray* a, char t_=0, char c_=-1)
00138         : ax(0), data(0), len(0), rank(0), trans(t_), cmplx(c_), mxref(1) {
00139         if (cmplx<0) { cmplx=ISCOMPLX_(T); }
00140         init(a);
00141     };
00142
00143     Array(const char *F, int L, const mxArray* a, char t_=0, char c_=-1)
00144         : ax(0), data(0), len(0), rank(0), trans(t_), cmplx(c_), mxref(1)
00145         { if (cmplx<0) { cmplx=ISCOMPLX_(T); }
00146
00147         try { init(a); }
00148         catch (...) { wblog(F_L,"ERR %s() ",FCT); }
00149
00150         if (!ax || (!mxref && !data)) wblog(F_L,
00151             "ERR %s() invalid input\n%s %s",FCT,SHORT_FL,STR_(this));
00152     };
00153
00154     wstring toStr(char vflag=0, unsigned l=64) const;
00155     wstring toStrT(char vflag=0) const;
00156
00157     void init_xstr() {
00158         sprintf_str("Mx::Array<%s>",sTSTR(T));
00159         xstr=str;
00160     };
00161
00162     const char *xStr() const {
00163         return xstr.c_str();
00164     };
00165
00166     ~Array() {
00167         if ((ax || data) && mxref<=0) wblog(FL,
00168             "WRN %s() got ax=%p, data=%p !?",FCT,ax,data);
00169     }
00170
00171 }

```

```

00173
00174     mxArray* Return(T*& dd) { dd=data; return Return(); };
00175
00176     mxArray* Return() {
00177         mxArray *a=ax;
00178         ax=NULL; data=NULL; len=0;
00179         mxref=(mxref>0? 1:0);
00180         return a;
00181     };
00182
00183     Array& init(unsigned ndim, size_t *dims, char t_=-1, char c_=-1);
00184     Array& init(const mxArray *a, char t_=-1, char c_=-1);
00185
00186     template <class Tb>
00187     mxArray* copyFromTR(const Tb* b, char tcheck=0) {
00188
00189         if (!trans && typeid(T)==typeid(Tb) && data && b)
00190             { memcpy(data,b,len*sizeof(T)); }
00191         else { copy_from(b,tcheck); }
00192
00193         return Return();
00194     };
00195
00196     template <class Tb>
00197     int copy_from(const Tb* b, char tcheck=0);
00198
00199     template <class Tb>
00200     size_t copyTo(Tb* b, char tcheck=0) const {
00201         if (!trans && data && b) {
00202             if (mxref<=8) wblog(FL,"WRN %s() ref=%d !?",FCT,mxref);
00203             memcpy(b,data,len*sizeof(T));
00204         }
00205         else { copy_to(b,NULL,tcheck); }
00206         return len;
00207     };
00208
00209     size_t copy_to(T* b, const wbperm *P=NULL, char tcheck=0) const;
00210
00211     size_t rcopy_to(T* b, const wbperm *P=NULL, char tcheck=0) const;
00212     size_t zcopy_to(T* b, const wbperm *P=NULL, char tcheck=0) const;
00213
00214     template <class Tx>
00215     size_t cpy_to(T* b, const wbperm *P, char tcheck, const Tx *ad_) const;
00216
00217     size_t ncopy_to(T* b, size_t n, char tcheck=0) {
00218         size_t l=len; if (len>n) len=n;
00219         n=copy_to(b,NULL,tcheck); len=l;
00220         return n;
00221     };
00222
00223     mxArray *ax;
00224     T *data;
00225     size_t len;
00226     unsigned rank;
00227
00228     bool trans;
00229     char cmplx;
00230     char mxref;
00231
00232     std::string xstr;
00233
00234 protected:
00235 private:
00236
00237     void issue_type_error(
00238         const char *F, int L, unsigned ndim=0, const size_t *dims=NULL);
00239
00240     int init_dptra();
00241
00242     int safe_set_dptra(T *md) { data=md; return 0; }
00243
00244     template <class TM>
00245     int safe_set_dptra(TM *md QS_UNUSED_VAR) { return 1; }
00246
00247     void MX_INIT_DATA(unsigned ndim, const size_t *dims);
00248 };
00249
00250 template <> template <>
00251 int Mx::Array<wbcomplex>::safe_set_dptra(mxComplexDouble *md) {
00252     data=(wbcomplex*)md; return 0;
00253 };
00254
00255 template <> template <>
00256 int Mx::Array<char>::safe_set_dptra(int8_T *md) {
00257     unsigned m=sizeof(char), n=sizeof(int8_T);
00258     if (m!=n) wblog(FL,"ERR %s() size mismatch (char: %d/%d)",FCT,m,n);
00259     data=(char*)md; return 0;

```

```

00260 };
00261
00262 template <> template <>
00263 int Mx::Array<size_t>::safe_set_dptra(uint64_T *md) {
00264     unsigned m=sizeof(size_t), n=sizeof(uint64_T);
00265     if (m!=n) wblog(FL,"ERR %s() size mismatch (size_t: %d/%d)",FCT,m,n);
00266     data=(size_t*)md; return 0;
00267 };
00268
00269 template <> template <>
00270 int Mx::Array<unsigned long long>::safe_set_dptra(uint64_T *md) {
00271     unsigned m=sizeof(size_t), n=sizeof(uint64_T);
00272     if (m!=n) wblog(FL,"ERR %s() size mismatch (size_t: %d/%d)",FCT,m,n);
00273     data=(unsigned long long*)md; return 0;
00274 };
00275
00276 template <> template <>
00277 int Mx::Array<long long>::safe_set_dptra(int64_T *md) {
00278     unsigned m=sizeof(long long), n=sizeof(int64_T);
00279     if (m!=n) wblog(FL,"ERR %s() size mismatch (long: %d/%d)",FCT,m,n);
00280     data=(long long*)md; return 0;
00281 };
00282
00283 template <>
00284 size_t Mx::Array<double>::copy_to(double * b, const wbptra *P, char tcheck)
00285 const { return rcopy_to(b,P,tcheck); };
00286
00287 template <>
00288 size_t Mx::Array<int>::copy_to(int * b, const wbptra *P, char tcheck)
00289 const { return rcopy_to(b,P,tcheck); };
00290
00291 template <>
00292 size_t Mx::Array<unsigned>::copy_to(unsigned* b, const wbptra *P, char tcheck)
00293 const { return rcopy_to(b,P,tcheck); };
00294
00295 template <>
00296 size_t Mx::Array<size_t>::copy_to(size_t * b, const wbptra *P, char tcheck)
00297 const { return rcopy_to(b,P,tcheck); };
00298
00299 template <>
00300 size_t Mx::Array<char>::copy_to(char* b, const wbptra *P, char tcheck)
00301 const { return rcopy_to(b,P,tcheck); };
00302
00303 template <>
00304 void Mx::Array<double>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
00305     data=0;
00306     #pragma omp critical (using_MEX_API)
00307     { ax=mxCreateNumericArray(ndim,dims, mxDOUBLE_CLASS, mxREAL);
00308       if (ax) { safe_set_dptra(mxGetDoubles(ax)); }
00309     }; if (!data) issue_type_error(FL,ndim,dims);
00310 };
00311
00312 template <>
00313 void Mx::Array<wbcomplex>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
00314     data=0;
00315     #pragma omp critical (using_MEX_API)
00316     { ax=mxCreateNumericArray(ndim,dims, mxDOUBLE_CLASS, mxCOMPLEX);
00317       if (ax) { safe_set_dptra(mxGetComplexDoubles(ax)); }
00318     }; if (!data) issue_type_error(FL,ndim,dims);
00319 };
00320
00321 template <>
00322 void Mx::Array<char>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
00323     data=0;
00324     #pragma omp critical (using_MEX_API)
00325     { ax=mxCreateNumericArray(ndim,dims, mxINT8_CLASS, mxREAL);
00326       if (ax) { safe_set_dptra(mxGetInt8s(ax)); }
00327     }; if (!data) issue_type_error(FL,ndim,dims);
00328 };
00329
00330 template <>
00331 void Mx::Array<signed char>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
00332     data=0;
00333     #pragma omp critical (using_MEX_API)
00334     { ax=mxCreateNumericArray(ndim,dims, mxINT8_CLASS, mxREAL);
00335       if (ax) { safe_set_dptra(mxGetInt8s(ax)); }
00336     }; if (!data) issue_type_error(FL,ndim,dims);
00337 };
00338
00339 template <>
00340 void Mx::Array<unsigned char>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
00341     data=0;
00342     #pragma omp critical (using_MEX_API)
00343     { ax=mxCreateNumericArray(ndim,dims, mxUINT8_CLASS, mxREAL);
00344       if (ax) { safe_set_dptra(mxGetUInt8s(ax)); }
00345     }; if (!data) issue_type_error(FL,ndim,dims);
00346 };

```

```

00347
00348 template <>
00349 void Mx::Array<mxChar>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
00350     data=0;
00351     #pragma omp critical (using_MEX_API)
00352     { ax=mxCreateCharArray(ndim,dims);
00353       if (ax) { safe_set_dptra(mxGetChars(ax)); }
00354     }; if (!data) issue_type_error(FL,ndim,dims);
00355 };
00356
00357 template <>
00358 void Mx::Array<int>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
00359     data=0;
00360     #pragma omp critical (using_MEX_API)
00361     { ax=mxCreateNumericArray(ndim,dims, mxINT32_CLASS, mxREAL);
00362       if (ax) { safe_set_dptra(mxGetInt32s(ax)); }
00363     }; if (!data) issue_type_error(FL,ndim,dims);
00364 };
00365
00366 template <>
00367 void Mx::Array<unsigned>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
00368     data=0;
00369     #pragma omp critical (using_MEX_API)
00370     { ax=mxCreateNumericArray(ndim,dims, mxUINT32_CLASS, mxREAL);
00371       if (ax) { safe_set_dptra(mxGetUInt32s(ax)); }
00372     }; if (!data) issue_type_error(FL,ndim,dims);
00373 };
00374
00375 template <>
00376 void Mx::Array<long>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
00377     data=0;
00378     #pragma omp critical (using_MEX_API)
00379     { ax=mxCreateNumericArray(ndim,dims, mxINT64_CLASS, mxREAL);
00380       if (ax) { safe_set_dptra(mxGetInt64s(ax)); }
00381     }; if (!data) issue_type_error(FL,ndim,dims);
00382 };
00383
00384 template <>
00385 void Mx::Array<long long>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
00386     data=0;
00387     #pragma omp critical (using_MEX_API)
00388     { ax=mxCreateNumericArray(ndim,dims, mxINT64_CLASS, mxREAL);
00389       if (ax) { safe_set_dptra(mxGetInt64s(ax)); }
00390     }; if (!data) issue_type_error(FL,ndim,dims);
00391 };
00392
00393 template <>
00394 void Mx::Array<size_t>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
00395     data=0;
00396     #pragma omp critical (using_MEX_API)
00397     { ax=mxCreateNumericArray(ndim,dims, mxUINT64_CLASS, mxREAL);
00398       if (ax) { safe_set_dptra(mxGetUInt64s(ax)); }
00399     }; if (!data) issue_type_error(FL,ndim,dims);
00400 };
00401
00402 template <>
00403 void Mx::Array<unsigned long long>::MX_INIT_DATA(unsigned ndim, const size_t *dims) {
00404     data=0;
00405     #pragma omp critical (using_MEX_API)
00406     { ax=mxCreateNumericArray(ndim,dims, mxUINT64_CLASS, mxREAL);
00407       if (ax) { safe_set_dptra(mxGetUInt64s(ax)); }
00408     }; if (!data) issue_type_error(FL,ndim,dims);
00409 };
00410
00411 int IsNumArray(
00412     const char *F, int L,
00413     const mxArray* a, unsigned r=2, char cflag=0, char type='d'
00414 );
00415
00416 bool IsDblMat(const char *F, int L,
00417     const mxArray* a, char cflag=0, char dflag='d')
00418 { return Mx::IsNumArray(F,L,a,2,cflag,dflag)>0; }
00419
00420 bool IsDblArr(const char *F, int L,
00421     const mxArray* a, char cflag=0, char dflag='d')
00422 { return Mx::IsNumArray(F,L,a,-1,cflag,dflag)>0; }
00423
00424 bool IsDblVector(const char *F, int L,
00425     const mxArray *a, char cflag=0, char dflag='d')
00426 { return Mx::IsNumArray(F,L,a,1,cflag,dflag)>0; }
00427
00428 bool IsDblScalar(const char *F, int L,
00429     const mxArray *a, char cflag=0, char dflag='d') {
00430     if (Mx::IsNumArray(F,L,a,1,cflag,dflag)<=0) return 0;
00431     return mxGetNumberOfElements(a)==1;
00432 }
00433

```



```

00434 bool IsNumber(const char *F, int L,
00435               const mxArray *a, char cflag=0, char dflag='*') {
00436     if (Mx::IsNumArray(F,L,a,1,cflag,dflag)<=0) return 0;
00437     return (mxGetNumberOfElements(a)==1);
00438 }
00439
00440 bool IsScalar(const mxArray *a) {
00441     return (mxGetNumberOfDimensions(a)==2 && mxGetNumberOfElements(a)==1);
00442 };
00443
00444 int IsVector(const mxArray *a) {
00445     int q=0;
00446     if (mxGetNumberOfDimensions(a)==2) {
00447         const size_t *s=mxGetDimensions(a);
00448         if (s[0]==1) { q=1; }
00449         else if (s[1]==1) { q=2; }
00450     }
00451     return q;
00452 };
00453
00454 bool IsIndex(const mxArray *a, int base=0);
00455
00456 template <class T>
00457 bool IsIndex(const T *x, size_t n, int base=0) {
00458     if (!WbUtil<T>().isPOD()) wblog(FL,
00459     "ERR %s() invalid data type `%s'",FCT,TSTR(T));
00460
00461     if (WbUtil<T>().isInt()) {
00462         for (size_t i=0; i<n; ++i) { if (x[i]<base) return 0; }
00463     }
00464     else {
00465         for (size_t i=0; i<n; ++i) {
00466             if (x[i]<base || x[i]!=round(x[i])) return 0; }
00467     }
00468     return 1;
00469 };
00470
00471 };
00472
00473 namespace Wb {
00474     int CallMatlab(
00475         int nargout, mxArray *argout[], int nargin, mxArray *argin[],
00476         const char *fctname);
00477
00478     double CallMatlab(
00479         const char *F, int L, const char* cmd, const char *arg1=NULL);
00480
00481     double CallMatlab(const char* cmd, const char *arg1=NULL) {
00482         return CallMatlab(0,0,cmd,arg1);
00483     };
00484 };
00485
00486 #ifndef MATLAB_MEX_FILE
00487
00488 #undef mexPutVariable
00489 #define mexPutVariable mexPutVariable_wbx
00490 int mexPutVariable_wbx(const char *ws, const char *vname, const mxArray *a){
00491     wblog(FL,"WRN mexPutVariable(%s,%lX,%s) not available outside MatLab",
00492     vname?vname:"",a,ws?ws:""); return 1;
00493 };
00494
00495 #undef mexCallMATLAB
00496 #define mexCallMATLAB mexCallMATLAB_wbx
00497 int mexCallMATLAB_wbx(
00498     int nargout, mxArray **argin, int nargin, mxArray **argout,
00499     const char *cmd
00500 ){ wblog(FL,"ERR mexCallMATLAB(%d,%lX,%d,%lX,%s) not available outside MatLab",
00501     nargin,argin,nargout,argout,cmd?cmd:""); return 1;
00502 };
00503
00504 #undef mexGetVariable
00505 #define mexGetVariable mexGetVariable_wbx
00506 int mexGetVariable_wbx(const char *ws, const char *vname) {
00507     wblog(FL,"WRN mexGetVariable(%s,%s) not available outside MatLab",
00508     vname?vname:"",ws?ws:""); return 1;
00509 };
00510
00511 #undef mexGetVariablePtr
00512 #define mexGetVariablePtr mexGetVariablePtr_wbx
00513 mxArray* mexGetVariablePtr_wbx(const char *ws, const char *vname) {
00514     wblog(FL,"WRN mexGetVariablePtr(%s,%s) not available outside MatLab",
00515     vname?vname:"",ws?ws:""); return NULL;
00516 };
00517
00518 #undef mexPrintf
00519 #define mexPrintf mexPrintf_wbx
00520 int mexPrintf_wbx(const char *fmt, ...){

```

```

00521     wblog(FL,"WRN mexPrintf(%s,...) not available outside MatLab",
00522         fmt?fmt:""); return 1;
00523 };
00524
00525 #undef mexFunctionName
00526 #define mexFunctionName mexFunctionName_wbx
00527
00528 const char* mex_function_name="(program)";
00529
00530 const char* mexFunctionName_wbx(const char *fmt, ...){
00531     wblog(FL,"WRN mexFunctionNamef(%s,...) not available outside MatLab",
00532         fmt?fmt:""); return mex_function_name;
00533 };
00534
00535 const char* mexFunctionName_wbx(){
00536     wblog(FL,"WRN mexFunctionNamef() not available outside MatLab");
00537     return mex_function_name;
00538 };
00539
00540 #endif
00541
00542 #endif
00543

```

5.15 nomex.hh

```

00001 /* ===== */
00002 /* dummy version of bracketing out MEX library if
00003    (i) it is not available and
00004    (ii) also not required in the code per se!
00005    in case any of the mex-routines is required
00006    an error message issued and the program terminates.
00007
00008    NB! return types taken from Matlab R2013a documentation
00009    Wb, Apr02,13 */
00010 /* ===== */
00011
00012 typedef char mxArray;
00013
00014 // taken from Matlab R2013a documentation
00015 enum mxComplexity {mxREAL=0, mxCOMPLEX};
00016 // #define mxREAL 0
00017 // #define mxCOMPLEX 1
00018
00019 #define mxDOUBLE_CLASS 10
00020
00021 #define MATFile FILE
00022 #define mxChar short
00023 #define mwSize size_t
00024 #define mwIndex int
00025
00026 #define ERRMSG_NOMEX wblog(__FILE__, __LINE__, "ERR NOMEX (%s)", __FUNCTION__)
00027
00028 int wblog(const char* file, int line, const char *fmt, ...);
00029
00030 int mexCallMATLAB(
00031     unsigned nout, mxArray** aout,
00032     unsigned nin, mxArray** ain, const char* fstr)
00033 { ERRMSG_NOMEX; return 0; };
00034
00035 int mxIsChar (const mxArray* a) { ERRMSG_NOMEX; return 0; };
00036 int mxIsDouble (const mxArray* a) { ERRMSG_NOMEX; return 0; };
00037 int mxIsComplex (const mxArray* a) { ERRMSG_NOMEX; return 0; };
00038 int mxIsCell (const mxArray* a) { ERRMSG_NOMEX; return 0; };
00039 int mxIsStruct (const mxArray* a) { ERRMSG_NOMEX; return 0; };
00040 int mxIsSparse (const mxArray* a) { ERRMSG_NOMEX; return 0; };
00041
00042 int mxIsEmpty (const mxArray* a) { ERRMSG_NOMEX; return 0; };
00043 size_t mxGetM (const mxArray* a) { ERRMSG_NOMEX; return 0; };
00044 size_t mxGetN (const mxArray* a) { ERRMSG_NOMEX; return 0; };
00045
00046 int mxGetNzmax (const mxArray* a) { ERRMSG_NOMEX; return 0; };
00047 int* mxGetIr (const mxArray* a) { ERRMSG_NOMEX; return (int*)0; };
00048 int* mxGetJc (const mxArray* a) { ERRMSG_NOMEX; return (int*)0; };
00049
00050 int mxDestroyArray (mxArray* a) { ERRMSG_NOMEX; return 0; };
00051
00052 void mexErrMsgTxt(const char* estr) {
00053     printf("\nmexErrMsgTxt:%s\nExit.\n\n", estr);
00054     exit(66);
00055 };
00056
00057 char* mxGetClassName(const mxArray* a) {

```

```

00058     static char ostr[]="(none)";
00059     ERRMSG_NOMEX; return ostr;
00060 };
00061
00062 size_t mxGetNumberOfElements(const mxArray* a) {
00063     ERRMSG_NOMEX; return 0;
00064 };
00065
00066 int mxGetNumberOfFields(const mxArray* a) {
00067     ERRMSG_NOMEX; return 0;
00068 };
00069
00070 unsigned mxGetNumberOfDimensions(const mxArray* a) {
00071     ERRMSG_NOMEX; return 0;
00072 };
00073
00074 const size_t* mxGetDimensions(const mxArray* a) {
00075     ERRMSG_NOMEX; return (const size_t*)NULL;
00076 };
00077
00078 mxArray* mxGetFieldByNumber(const mxArray *a, unsigned i, unsigned j) {
00079     ERRMSG_NOMEX; return (mxArray*)NULL;
00080 };
00081
00082 const char* mxGetFieldNameByNumber(const mxArray *a, int fid) {
00083     ERRMSG_NOMEX; return (const char*)NULL;
00084 };
00085
00086 int mxGetFieldNumber(const mxArray *a, const char *fld) {
00087     ERRMSG_NOMEX; return 0;
00088 };
00089
00090 int mxSetFieldByNumber(mxArray* S, unsigned i, unsigned j, mxArray* a) {
00091     ERRMSG_NOMEX; return 0;
00092 };
00093
00094 mxArray* mxGetField(const mxArray *a, mwIndex idx, const char *fld) {
00095     ERRMSG_NOMEX; return (mxArray*)NULL;
00096 };
00097
00098 int mxAddField(const mxArray *a, const char *fld) {
00099     ERRMSG_NOMEX; return 0;
00100 };
00101
00102 int mxGetString(const mxArray* a, char *ostr, int len) {
00103     ERRMSG_NOMEX; return 0;
00104 };
00105
00106 mxArray* mxCreateString(const char *s) {
00107     ERRMSG_NOMEX; return (mxArray*)NULL;
00108 };
00109
00110 mxArray* mxCreateDoubleMatrix(size_t d1, size_t d2, char mxreal) {
00111     ERRMSG_NOMEX; return (mxArray*)NULL;
00112 };
00113
00114 mxArray* mxCreateNumericArray(int d, size_t *dims, int cflag, int rflag) {
00115     ERRMSG_NOMEX; return (mxArray*)NULL;
00116 };
00117
00118 mxArray* mxCreateSparse(size_t m, size_t n, size_t nzmax, mxComplexity cflag) {
00119     ERRMSG_NOMEX; return (mxArray*)NULL;
00120 };
00121
00122 mxArray* mxCreateCharArray(int d, size_t *dims) {
00123     ERRMSG_NOMEX; return (mxArray*)NULL;
00124 };
00125
00126 mxArray* mxCreateStructMatrix(size_t d1, size_t d2, int nf, const char** fnames) {
00127     ERRMSG_NOMEX; return (mxArray*)NULL;
00128 };
00129
00130 mxArray* mxCreateCellMatrix(size_t d1, size_t d2) {
00131     ERRMSG_NOMEX; return (mxArray*)NULL;
00132 };
00133
00134 mxArray* mxGetCell(const mxArray* S, unsigned idx) {
00135     ERRMSG_NOMEX; return (mxArray*)NULL;
00136 };
00137
00138 int mxSetCell(mxArray* S, unsigned idx, mxArray* b) {
00139     ERRMSG_NOMEX; return 0;
00140 };
00141
00142 int mexPutVariable(const char *ws, const char *vname, mxArray *a) {
00143     ERRMSG_NOMEX; return 0;
00144 };

```

```

00145
00146 int matPutVariable(MATFile *mfp, const char *vname, const mxArray *a) {
00147     ERRMSG_NOMEX; return 0;
00148 };
00149
00150 mxArray* matGetVariable(MATFile *mfp, const char *vname) {
00151     ERRMSG_NOMEX; return (mxArray*)NULL;
00152 };
00153
00154 mxArray* matGetVariableInfo(MATFile *mfp, const char *vname) {
00155     ERRMSG_NOMEX; return (mxArray*)NULL;
00156 };
00157
00158 int matDeleteVariable(MATFile *mfp, const char *name) {
00159     ERRMSG_NOMEX; return 0;
00160 };
00161
00162 MATFile* matOpen(const char *file, const char *mode) {
00163     ERRMSG_NOMEX; return 0;
00164 };
00165
00166 int matClose(MATFile *mfp) {
00167     ERRMSG_NOMEX; return 0;
00168 };
00169

```

5.16 nrgdata.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : QSpace NRG routines
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_NRGDATA_HCC__
00021 #define __WB_NRGDATA_HCC__
00022
00023 /* ----- //
00024 // change log
00025
00026 Wb, Feb22, 19: changed internally K,T labels to K,D labels,
00027 besides that T is also temperature, TK is Kondo temperature
00028 TD is a template data type name, etc.
00029
00030 // ----- */
00031
00032 const double DEPS2=1E-6*DBL_EPSILON;
00033
00034 template <class TQ, class TD>
00035 class NRGData;
00036
00037 #define NRG_FLEN 128
00038 const char NRG_EMPTY_STR[]="(NRG)";
00039
00040 unsigned NRG_N, NRG_ITER;
00041 char NRG_FILE[NRG_FLEN];
00042
00043 unsigned STRICT_ITER0=1;
00044
00045 Wb::ClockSet fdmClocks;
00046
00047 wbvector<double> gES;
00048
00049 inline double gEScale(int iter) {
00050     if (iter<0 || iter>=(int)gES.len) wblog(FL,
00051         "ERR %s() index out of bounds (%d/%d)",FCT,iter,gES.len);
00052     return gES[iter];
00053 };
00054
00055 void wblog_ERR_updateOp(

```

```

00056     const char *F, int L, const char *istr, int i, int nop, int iter) {
00057
00058     wblog(F_L,
00059         "ERR failed to update operator (%s-op %d/%d @ iter=%d)",
00060         istr && istr[0] ? istr : "(fdm)", i+1,nop,iter
00061     );
00062 }
00063
00064 template <class TQ, class TD, class TZ>
00065 void WeightE0_dim1(QSpace<TQ,TZ> &X, const QSpace<TQ,TD> &H, double w);
00066
00067 double nrgGetE0(
00068     const wvector<double> &dE,
00069     const wvector<double> &ES,
00070     wvector<double> &E0
00071 ){
00072     if (E0.len!=dE.len) { E0.init(dE.len); }
00073     if (!E0.len) { wblog(FL,"WRN %s() got empty E0 !?",FCT); return 0; }
00074     if (dE.len!=ES.len) wblog(FL,
00075         "ERR %s() length mismatch (%d/%d)",FCT,dE.len,ES.len);
00076
00077     unsigned k=dE.len-1;
00078     double x=0, xl=dE[k]*ES[k]; E0[k]=0;
00079
00080     for (; k; --k) {
00081         x=dE[k-1]*ES[k-1];
00082         E0[k-1]=E0[k]-xl;
00083         xl=x;
00084     }
00085
00086     return x-E0[0];
00087 };
00088
00089 template <class TQ, class TD>
00090 int getSpecDimOps(
00091     const wvector<QSpace<TQ,TD> > &FF, wvector<double> &fac
00092 ){
00093     unsigned i=0, r,m, n=FF.len, gotfac=0; widxt nq;
00094     fac.init2val(n,1);
00095
00096     for (; i<n; ++i) {
00097         r=FF[i].rank(FL); if (r<=2) continue;
00098         if (r>3) wblog(FL,"ERR %s() invalid rank-%g !?",FCT,r);
00099
00100         m=FF[i].getDim(2,&nq);
00101         if (int(nq)<1) wblog(FL,"ERR %s() %d/%d !?",FCT,m,nq);
00102         if (nq>1) { fac[i]=nq; ++gotfac; }
00103     }
00104     return gotfac;
00105 };
00106
00107 class RIXS_para {
00108
00109 public:
00110
00111     RIXS_para()
00112         : Eoffset(0.), sigma(0.), afac(-1.), wrxs(0),
00113         E0(0.), E2(0.), dloc(0), iter(0) {};
00114
00115     void init() {
00116         Einc=0; Eoffset=sigma=E0=E2=0; wrxs=0; dloc=iter=0;
00117         afac=-1;
00118     };
00119
00120     void setE0(int it, double e0, double e2) {
00121         iter=it; E0=e0+Eoffset; E2=e2;
00122     }
00123
00124     void check(const char *F=0, int L=0, char init=0);
00125
00126     Wb::String toStr() const;
00127     mxArray* toMx() const;
00128
00129     wcomplex Einc;
00130     double Eoffset;
00131     double sigma;
00132     double afac;
00133     char wrxs;
00134
00135     double E0, E2;
00136     unsigned dloc;
00137     int iter;
00138
00139 private:
00140 };
00141
00142 RIXS_para gRX;

```

```

00143
00144 void RIXS_para::check(const char *F, int L, char init) {
00145
00146     if (Einc.i<=0) {
00147         if (!Einc.i && init) {
00148             Einc.i=gES.last();
00149         }
00150         else wblog(F_L,
00151             "ERR %s() got imag(Einc)=%g < 0 (RIXS) !?",FCT,Einc.i
00152         );
00153     }
00154     else if (F && init) {
00155         if (Einc.i<1E-2*Einc.r || Einc.i>1E2*Einc.r) wblog(FL,
00156             "WRN %s() got Einc.i / Einc_r = %.3g / %.3g = %.3g",
00157             PROG, Einc.i, Einc.r, Einc.i/Einc.r
00158         );
00159     }
00160
00161     if (sigma<0) wblog(F_L,
00162         "ERR RIXS::%s() invalid sigma=%g<0",FCT,sigma);
00163     if (int(dloc)<=1 || dloc>(1<10)) wblog(F_L,
00164         "ERR RIXS::%s() invalid dloc=%d",FCT,dloc);
00165
00166     if (init) {
00167         if (!sigma) {
00168             if (Einc.r) { sigma = Einc.i/Einc.r; }
00169             else wblog(FL,"ERR %s() got bot sigma and real(Einc) zero !?",FCT);
00170         }
00171         else if (sigma<1E-3 && F) {
00172             wblog(FL,"WRN %s() got sigma = %g",PROG,sigma);
00173         }
00174     }
00175
00176 };
00177
00178 Wb::String RIXS_para::toStr() const {
00179
00180     Wb::String s(64);
00181     s.pushf("Einc = %.3g",Einc.r);
00182
00183     if (Einc.r) {
00184         double x=Einc.i/Einc.r;
00185         if (x && fabs((x-sigma)/x)<1E-4)
00186             s.pushf(" @ sigma=%.3g",sigma);
00187         else s.pushf(" @ %.3gi, sigma=%g",x,sigma);
00188     }
00189     else { s.pushf(" + %.3gi, sigma=%g",Einc.i,sigma); }
00190
00191     if (wrxs ) { s.pushf(", mode=%d",wrxs); }
00192     if (afac>=0) { s.pushf(", afac=%g",afac); }
00193
00194     return s;
00195 };
00196
00197 mxArray* RIXS_para::toMx() const {
00198     const char* fields[]={
00199         "Einc","Eoffset","sigma","wrxs","dloc";
00200
00201     mxArray* a=mxCreateStructMatrix(1,1,5,fields);
00202     mxSetFieldByNumber(a,0,0, numtoMx(Einc ));
00203     mxSetFieldByNumber(a,0,1, numtoMx(Eoffset));
00204     mxSetFieldByNumber(a,0,2, numtoMx(sigma));
00205     mxSetFieldByNumber(a,0,3, numtoMx(wrxs ));
00206     mxSetFieldByNumber(a,0,4, numtoMx(dloc ));
00207
00208     return a;
00209 };
00210
00211 template <class TQ, class TD>
00212 class NRGIndex {
00213
00214 public:
00215
00216     NRGIndex() : idx(-1), nrg(NULL) {};
00217     NRGIndex(const NRGIndex &I) : idx(I.idx), nrg(I.nrg), tag(I.tag) {};
00218
00219     void init() { idx=-1; nrg=NULL; };
00220     void init(int i, NRGData<TQ,TD> *d) { idx=i; nrg=d; }
00221     void init(int i, const char *t) { idx=i; tag=t; }
00222
00223     mxArray* toMx() const;
00224     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
00225     void add2MxStruct(mxArray *S, unsigned i, char tst=0) const;
00226
00227     void put (const char *vname, const char *ws="base") const {
00228         mxArray *S=toMx();
00229         mxPutAndDestroy(FL,S,vname,ws);

```

```

00230     };
00231
00232     int idx;
00233     NRGData<TQ,TD> *nrg;
00234     wstring tag;
00235
00236 private:
00237 };
00238
00239 template <class TQ, class TD>
00240 mxArray* NRGIndex<TQ,TD>::mxCreateStruct(unsigned m, unsigned n) const {
00241     const char *fields[] = {"idx","nrg","tag"};
00242     return mxCreateStructMatrix(m,n,3,fields);
00243 };
00244
00245 template <class TQ, class TD>
00246 void NRGIndex<TQ,TD>::add2MxStruct(
00247     mxArray *S, unsigned i, char tst QS_UNUSED_VAR
00248 ) const {
00249
00250     mxSetFieldByNumber(S,i,0, numtoMx(idx));
00251     mxSetFieldByNumber(S,i,1, numtoMx((unsigned long)(nrg)));
00252     mxSetFieldByNumber(S,i,2, mxCreateString(tag.isEmpty() ? tag.data:""));
00253 };
00254
00255 namespace Wb {
00256
00257     template <class TQ, class TD>
00258     int updateOp(const char *F, int L,
00259         const NRGData<TQ,TD> &NRG,
00260         wbvector< QSpace<TQ,TD> > &F12,
00261         const QSpace<TQ,TD> &A1,
00262         const QSpace<TQ,TD> &A2,
00263         const wbvector< QSpace<TQ,TD> > &FKK,
00264         unsigned iter
00265     );
00266
00267     template<class TT>
00268     void initLocalOp(const char *F, int L,
00269         const mxArray* a, TT &C,
00270         int k=0, char force=0, char *name=NULL
00271     );
00272
00273     template<class TQ, class TD>
00274     inline void initLocalOp_aux(const char *F, int L,
00275         const mxArray* a, wbvector< QSpace<TQ,TD> > &C
00276     ){ mxInitQSpaceVec(F,L,a,C); }
00277
00278     template<class TQ, class TD>
00279     inline void initLocalOp_aux(const char *F, int L,
00280         const mxArray* a, wbMatrix< QSpace<TQ,TD> > &C
00281     ){ mxInitQSpaceMat(F,L,a,C); }
00282
00283     template<class TQ, class TD>
00284     inline void initLocalOp_aux(const char *F, int L,
00285         const mxArray* a, wbvector< wbvector< QSpace<TQ,TD> > > &C
00286     ){ mxInitQSpaceVecVec(F,L,a,C); }
00287 };
00288
00289 template <class TQ, class TD>
00290 class NRGData {
00291 public:
00292
00293     NRGData(const char *s=NULL)
00294         : name(s), calc(0), store(0), kloc(0), locMX(0), MX(NULL), aux(NULL) {};
00295
00296     NRGData(const NRGData &A)
00297         : name(A.name), nrgIdx(A.nrgIdx), calc(0), store(0), kloc(A.kloc),
00298         locMX(0), MX(NULL), aux(NULL)
00299     {
00300         wblog(FL,"ERR generating copy of NRGData !?");
00301     };
00302
00303     ~NRGData() {
00304         if (MX && !NAME.isEmpty() && !locMX) wblog(FL,
00305             "WRN ** data return `s' not implemented yet **\n(%1X; %d)",
00306             NAME.data, MX, locMX);
00307         init();
00308     };
00309
00310     void initName(const char *n) { name=n; };
00311
00312     void init(const char *F, int L, const mxArray* a,
00313         wbvector< QSpace<TQ,TD> > &C0,
00314         int k=0, char force=0, const char *vname=0
00315     );
00316

```

```

00317 void init(const char *F, int L, const char *XX, int iter=-1);
00318
00319 void initXX(const char *F, int L, int m=4, int iter=-1) {
00320     if (m==1 || m==4) { init(F,L,"KK",iter); } else
00321     if (m==2 || m>4) wblog(FL,"ERR %s() invalid m=%d !?",FCT,m);
00322     if (m>2) {
00323         init(F,L,"KD",iter);
00324         init(F,L,"DK",iter);
00325         init(F,L,"DD",iter);
00326     }
00327 };
00328
00329 void initX(const char *F, int L, int m=2, int iter=-1) {
00330     if (m==2) { init(F,L,"D",iter); }
00331     else if (m!=1) wblog(FL,"ERR %s() invalid m=%d !?",FCT,m);
00332     init(F,L,"K",iter);
00333 };
00334
00335 void init(char flag=0) {
00336     if (!flag) { name.init(); NAME.init(); }
00337
00338     nrgIdx.init(); calc=0; store=0;
00339     if (aux) {
00340         mxDestroyArray(aux);
00341         aux=NULL;
00342     }
00343
00344     if (MX && locMX) { mxDestroyArray(MX); }
00345     MX=NULL; locMX=0;
00346
00347     KK.init(); DK.init(); K.clearQSpace();
00348     KD.init(); DD.init(); D.clearQSpace();
00349 };
00350
00351 void setupMX(
00352     const char *F, int L, int N, char store_=1, char locMX_=1
00353 ){
00354     if (MX || locMX) wblog(F,L,
00355         "ERR %s() already got MX=%p (%d,%d) !?",FCT,MX,locMX,store);
00356     MX=mxCreateStructMatrix(1,N,0,NULL);
00357     store=store_; locMX=locMX_;
00358 };
00359
00360 void PUT(
00361     const char *F, int L, const char *vn, const char *ws="caller"
00362 ){
00363     if (MX && locMX) {
00364         if (!vn || !vn[0]) wblog(FL,"ERR %s() invalid usage",FCT);
00365         int i=mexPutVariable(ws,vn,MX);
00366         if (!i) wblog(FL,"I/O putting %s to %s",vn,ws);
00367         else wblog(FL,"ERR %s() mexPutVariable() returned i=%d !?",FCT,i);
00368     }
00369     else wblog(FL,"WRN %s() failed having %p (%d)",FCT,MX,locMX);
00370 };
00371
00372 void check_dloc(
00373     const char *F, int L, const char *fun,
00374     unsigned dloc, unsigned iter) const;
00375
00376 template<class TD_>
00377 int checkOpConsistency(
00378     const char *F, int L, const NRGData<TQ,TD_> &B) const;
00379
00380 bool isQSpaceVec(const char *nm, int iter=-1);
00381
00382 bool checkVec(const char *F, int L,
00383     const char *XX, unsigned R, const char *istr="");
00384
00385 bool checkVecVec(const char *XX, unsigned r);
00386
00387 void forceCalc();
00388 void getRelevantOps(wbindex &I) const;
00389
00390 wbvector< QSpace<TQ,TD> >& getOpsXX(const char *t) {
00391     if (t && strlen(t)==2)
00392         switch (t[0]) {
00393             case 'K': return (t[1]=='K' ? KK : KD);
00394             case 'D': return (t[1]=='K' ? DK : DD);
00395         }
00396     wblog(FL,"ERR invalid tag '%s'", t ? t : "(null)");
00397     return KK;
00398 };
00399
00400 const QSpace<TQ,TD>& getQSpace(const char *t, unsigned i=0) const;
00401 QSpace<TQ,TD>& getQSpace(const char *t, unsigned i=0) {
00402     return const_cast<QSpace<TQ,TD>&>(
00403         static_cast<const NRGData<TQ,TD>*>(this)->getQSpace(t,i));

```



```

00404     }
00405
00406     const QSpace<TQ,TD>& getQSpace(char t1, char t2, unsigned i=0) const {
00407         char t[3]={t1,t2,0}; return getQSpace(t,i);
00408     };
00409     QSpace<TQ,TD>& getQSpace(char t1, char t2, unsigned i=0) {
00410         char t[3]={t1,t2,0}; return getQSpace(t,i);
00411     };
00412
00413     const QSpace<TQ,TD>& getQSpace(char t) const {
00414         switch (t) {
00415             case 'K': return K;
00416             case 'D': return D;
00417         }
00418         wblog(FL,"ERR invalid tag '%c' (%d)",t,t);
00419         return K;
00420     };
00421     QSpace<TQ,TD>& getQSpace(char t) {
00422         return const_cast<QSpace<TQ,TD>&>(&
00423             static_cast<const NRGData<TQ,TD>*>(this)->getQSpace(t));
00424     };
00425
00426     void info(const char *vstr=NULL) const;
00427     void dispIdx(const char *F, int L) const;
00428     void info(const char *F, int L, const char *istr=" * ") const;
00429     void disp_otype(const char *F=0, int L=0) const;
00430
00431     const mxArray* getMxInfo(const char *n);
00432
00433     template <class TX>
00434     void getMxInfo(const char *F, int L, const char *v, TX &X) {
00435         const mxArray *a=getMxInfo(v);
00436         if (a) {
00437             X.init(F_L,a);
00438         }
00439         else wblog(F_L,
00440             "ERR failed to read '%s' from %s_info.mat",v,wnrg());
00441     };
00442
00443     mxArray* getMxData(
00444         const char *nm, int iter=-1, char ionly=0);
00445
00446     const char* wnrg() {
00447         if (NAME.data) return NAME.data;
00448         return (name.data ? name.data : NRG_EMPTY_STR);
00449     };
00450
00451     mxArray* toMx() const;
00452
00453     void put(const char *vname, const char* ws="base") {
00454         mxPutAndDestroy(FL, toMx(), vname);
00455     };
00456
00457     void put(const char *F, int L, const char *vname, const char* ws="base"
00458     ){ wblog(F,L,"I/O putting '%s' to %s", vname, ws);
00459         mxPutAndDestroy(FL, toMx(), vname);
00460     };
00461
00462     template<class T>
00463     int getPara(const char *name, T &x, int iter=-1, char i=0) {
00464         wblog(FL,"ERR %s() check!",FCT);
00465         mxArray *a=getMxData(iter);
00466         return mxGetNumber(a,x);
00467     }
00468
00469     bool opExists(char tflag=1);
00470
00471     void initOp(unsigned n=1, char calc_=1, char store_=0) {
00472         KK.initDef(n); DK.initDef(n); nrgIdx.initDef(n);
00473         KD.initDef(n); DD.initDef(n); calc=calc_; store=store_;
00474     };
00475
00476     bool initOp(const char *F, int L,
00477         const wbvector< QSpace<TQ,TD> > &C0,
00478         char tflag=1,
00479         NRGData<TQ,TD>* CR=NULL,
00480         const char *istr="")
00481     );
00482
00483     bool initOp(const char *F, int L,
00484         const wbvector< wbvector< QSpace<TQ,TD> > > &CC,
00485         NRGData<TQ,TD>* CR=NULL,
00486         const char *istr="",
00487         char tflag=1
00488     );
00489
00490     void initSym(const char *tag);

```

```

00491
00492 void updatePara(
00493     const char *F, int L,
00494     const char *tag, mxArray *, int iter=-1
00495 ) const;
00496
00497 void updatePara(const char *F, int L,
00498     const char *tag, int iter=-1,
00499     const char *tag2=NULL, char all=-1
00500 ) const;
00501
00502 void updateOp(const char *F, int L,
00503     const NRGData<TQ,TD> &A,
00504     const NRGData<TQ,TD> &B, unsigned nop, int iter=-1
00505 );
00506
00507 template<class TC>
00508 void updateOpProd(
00509     const char *F, int L, unsigned iter,
00510     const NRGData<TQ,TD> &B,
00511     const NRGData<TQ,TC> &C
00512 );
00513
00514 void updateOp(const char *F, int L,
00515     const NRGData<TQ,TD> &A, unsigned nop, int iter=-1
00516 ) { updateOp(F,L,A,A,nop,iter); };
00517
00518 template <class TA>
00519 void backPropagateOp(const char *F, int L,
00520     const NRGData<TQ,TA> &A0,
00521     const NRGData<TQ,TA> &A2
00522 );
00523
00524 template<class TD_>
00525 void initBG(
00526     const char *F, int L,
00527     const NRGData<TQ,TD_> &B,
00528     const NRGData<TQ,TD_> &H1, const NRGData<TQ,TD_> &H2,
00529     int m=3
00530 );
00531
00532 void unsetAllRefs();
00533 unsigned skipOpZeros(double eps=DEPS, char bflag='b');
00534
00535 void saveOp(const char*, int, int iter=-1, unsigned m=4) const;
00536
00537 void times(TD fac, const char *flag);
00538
00539 void setupIO(const char*, int,
00540     const char*, const mxArray*, const char *istr=0);
00541
00542 void updateInfo(
00543     const char *F, int L,
00544     const char*, mxArray *, char keep=-1
00545 ) const;
00546
00547 void updateInfop(
00548     const char *F, int L,
00549     const char *vtag, mxArray *, char keep=-1
00550 ) const;
00551
00552 const char* getFileName(unsigned k, char lenient=0) const {
00553     if (!lenient && k>=NRG_N) wblog(FL,
00554         "ERR %s - index out of bounds (%d/%d)", __FUNCTION__, k, NRG_N);
00555     return getFileName_aux( k);
00556 };
00557
00558 wstring name;
00559
00560 wvector< NRGIndex<TQ,TD> > nrgIdx;
00561
00562 char calc, store;
00563 int kloc;
00564 char locMX;
00565
00566 wvector< wvector< QSpace<TQ,TD> > > CI;
00567
00568 wvector< QSpace<TQ,TD> > KK, KD, DK, DD;
00569
00570 QSpace<TQ,TD> K, D;
00571
00572 wvector<char> issym;
00573
00574 mxArray* MX;
00575 wstring NAME;
00576
00577 mxArray *aux;

```

```

00578
00579 private:
00580
00581     const char* getFileNameI() const {
00582         return getFileName_aux(-1);
00583     };
00584
00585     const char* getFileName_aux(int k) const {
00586         int i;
00587         if (k>=0)
00588             i=sprintf(NRG_FILE,NRG_FLEN,"%s_%02d.mat", NAME.data, k);
00589         else i=sprintf(NRG_FILE,NRG_FLEN,"%s_info.mat", NAME.data);
00590         if (i>=NRG_FLEN) wblog(FL,"ERR File name exceeds length (%d/127)",i);
00591         return NRG_FILE;
00592     }
00593 };
00594
00595 template<class TQ, class TD>
00596 void NRGData<TQ,TD>::setupIO(
00597     const char *F, int L, const char *s, const mxArray *a,
00598     const char *istr
00599 ){
00600     if (!s || !s[0]) {
00601         if (!a || (!mxIsCell(a) && !mxIsStruct(a))) wblog(F,L,
00602             "ERR invalid input NRG data (%s)", a ? mxGetClassName(a):"NULL");
00603         NAME.init(); NRG_N=0; NRG_ITER=0;
00604         MX=(mxArray*)a;
00605     }
00606     else {
00607         Wb::matFile f; unsigned n=strlen(s);
00608         Wb::Clock clio("data:I/O",1,0,&fdmClocks);
00609
00610         if (!n || n+16>NRG_FLEN) wblog(FL,
00611             "ERR invalid file/variable name `%s' (%d/%d)", s, n, NRG_FLEN);
00612
00613         NRG_N=0; NRG_ITER=0; MX=NULL;
00614         NAME=s;
00615
00616 #ifdef MATLAB_MEX_FILE
00617         mxArray *a=mexGetVariable("caller",s);
00618         if (a && (mxIsStruct(a) || mxIsCell(a))) {
00619             if (istr) wblog(F,L,"<i> I/O NRG%s structure variable: %s",istr,s);
00620
00621             MX=a;
00622             return;
00623         }
00624 #endif
00625
00626         if (!f.open(F,L, getFileName(0,'l'),"r"))
00627             wblog(F,L,"ERR cannot open files of type %s_##.mat", s);
00628         else f.close();
00629
00630         if (istr) {
00631             if (strlen(s)<40) {
00632                 wblog(F,L,"<i> NRG%s is %s_##.mat",istr,s);
00633             }
00634             else { char s[256];
00635                 if (getcwd(s,256)==NULL) wblog(FL,"ERR failed read cwd");
00636                 wblog(F,L,"<i> using NRG%s data ...",istr);
00637                 printf("\n   pwd: %s\n   I/O: %s_##.mat\n\n",
00638                     Wb::repHome(s).data, Wb::repHome(s).data);
00639             }
00640         }
00641     }
00642 };
00643
00644 template<class TQ, class TD>
00645 void NRGData<TQ,TD>::init(const char *F, int L, const char *XX, int iter) {
00646
00647     char s[64]; unsigned n=strlen(XX), e=0;
00648     mxArray *a;
00649
00650     if (!n || n>2) wblog(FL,"ERR invalid operator tag `%s'",XX);
00651     if (iter<0) iter=NRG_ITER;
00652
00653     strncpy(s,name.data,60); s[60]=0; strcat(s,XX);
00654     a=getMxData(s,iter);
00655     if (!a) wblog(FL,"ERR failed to read NRGData `%s' @ iter=%d",s,iter);
00656
00657     if (n==1) {
00658         if (!strcmp(XX,"K")) K.init(F,L,a); else
00659         if (!strcmp(XX,"D")) D.init(F,L,a); else ++e;
00660     }
00661     else {
00662         if (!strcmp(XX,"KK")) mxInitQSpaceVec(F,L,a,KK); else
00663         if (!strcmp(XX,"KD")) mxInitQSpaceVec(F,L,a,KD); else
00664         if (!strcmp(XX,"DK")) mxInitQSpaceVec(F,L,a,DK); else

```

```

00665         if (!strcmp(XX, "DD")) mxInitQSpaceVec(F, L, a, DD); else ++e;
00666     }
00667     if (e) wblog(FL, "ERR invalid operator tag '%s'", XX);
00668 };
00669
00670 template<class TQ, class TD>
00671 void NRGData<TQ, TD>::init(
00672     const char *F, int L, const mxArray* a,
00673     wbvector< QSpace<TQ, TD> > &C0,
00674     int k,
00675     char force,
00676     const char *vname0
00677 ){
00678     char nstr[32]; nstr[0]=0;
00679
00680     Wb::initLocalOp(F, L, a, C0, k, force, nstr);
00681
00682     if (vname0 && vname0[0]) { name=vname0; }
00683     else if (nstr[0]) { name=nstr; }
00684
00685 }
00686
00687 template<class TT>
00688 void Wb::initLocalOp(const char *F, int L,
00689     const mxArray* a,
00690     TT &C,
00691     int k,
00692     char force,
00693     char *name
00694 ){
00695     size_t n=12; char istr[n];
00696     if (k)
00697         snprintf(istr, n, "arg %d", k);
00698     else strcpy(istr, "operator");
00699
00700     if (!a || mxIsEmpty(a)) {
00701         if (force) wblog(F, L, "ERR invalid %s%s", istr, a ? " (empty)": "");
00702         return;
00703     }
00704
00705     if (mxIsChar(a)) {
00706         const unsigned n=32; wbstring aux;
00707
00708         if (name==NULL) { aux.init(n+1); name=aux.data; }
00709         name[0]=0;
00710
00711         if (mxGetString(a, name, n) || name[0]==0) wblog(F, L,
00712             "ERR invalid string of %s%s", istr, name[0] ? "":" (empty)");
00713
00714         a=mexGetVariablePtr("caller", name);
00715
00716         if (!a) wblog(F, L,
00717             "ERR Can not read QSpace '%s' (%s) from workspace", name, istr);
00718     }
00719
00720     Wb::initLocalOp_aux(F, L, a, C);
00721
00722     if (force && C.isEmpty()) wblog(F, L,
00723         "ERR invalid QSpace (%s is empty)", istr);
00724 };
00725
00726 template<class TQ, class TD>
00727 void NRGData<TQ, TD>::forceCalc() {
00728
00729     wbvector< NRGIndex<TQ, TD> > &I=nrgIdx;
00730     if (I.len==0) return;
00731
00732     for (unsigned i=0; i<I.len; ++i) {
00733         if (I[i].idx<0) continue;
00734         if (I[i].nrg==NULL) I[i].idx=-1;
00735         if (!I[i].tag.isEmpty()) {
00736             I[i].init(); I[i].idx=-1;
00737         }
00738     }
00739
00740     calc=store=1;
00741 }
00742
00743 template<class TQ, class TD>
00744 void NRGData<TQ, TD>::updateOp(
00745     const char *F, int L,
00746     const NRGData<TQ, TD> &A, const NRGData<TQ, TD> &B,
00747     unsigned nop,
00748     int iter
00749 ){
00750     unsigned i=0, j, nrefs=0, m=nrgIdx.len;
00751     const char* tags[4] = { "KK", "KD", "DK", "DD" };

```

```

00752     int e=0,k;
00753
00754     wbvector< QSpace<TQ,TD> > X, X2;
00755
00756     if (iter<0) iter=NRG_ITER;
00757     if (nrgIdx.len==0) return;
00758
00759     for (; i<m; ++i) { if (nrgIdx[i].idx>=0) ++nrefs; }
00760
00761     unsetAllRefs();
00762
00763     if (nrefs<nrgIdx.len) {
00764         if (nop==4) {
00765             Wb::updateOp(F,L,*this, DD, A.D, B.D, KK, iter);
00766             Wb::updateOp(F,L,*this, DK, A.D, B.K, KK, iter);
00767             Wb::updateOp(F,L,*this, KD, A.K, B.D, KK, iter);
00768             e+=Wb::updateOp(F,L,*this, KK, A.K, B.K, KK, iter);
00769         }
00770         else if (nop==3) {
00771             Wb::updateOp(F,L,*this, DD, A.D, B.D, KK, iter);
00772             Wb::updateOp(F,L,*this, DK, A.D, B.K, KK, iter);
00773             e+=Wb::updateOp(F,L,*this, KK, A.K, B.K, KK, iter);
00774
00775             if (KD.len!=DK.len) KD.initDef(DK.len);
00776             for (i=0; i<DK.len; ++i) DK[i].transp(KD[i]);
00777         }
00778         else if (nop==1) {
00779             e+=Wb::updateOp(F,L,*this, KK, A.K, B.K, KK, iter);
00780         }
00781         else wblog(FL,"ERR %s() invalid nop (%d)",FCT,nop);
00782     }
00783
00784     if (e) wblog(FL,
00785         "ERR %s() operator %s[%d] updates to empty QSpace!",
00786         FCT,name.data,iter
00787     );
00788
00789     if (nrefs) {
00790         for (i=0; i<nop; ++i) {
00791
00792             wbvector< QSpace<TQ,TD> > &C = getOpsXX(tags[i]);
00793             if (C.len!=m) C.initDef(m);
00794
00795             X.init(); X2.init();
00796
00797             for (j=0; j<m; ++j) {
00798                 k=nrgIdx[j].idx; if (k<0) continue;
00799
00800                 if (nrgIdx[j].nrg) {
00801                     wbvector< QSpace<TQ,TD> > &R=nrgIdx[j].nrg->getOpsXX(tags[i]);
00802
00803                     if (k>=(int)R.len) wblog(FL,
00804                         "ERR index out of bounds (%d/%d)",k,R.len);
00805
00806                     C[j].init2ref(R[k]);
00807                 }
00808                 else if (nrgIdx[j].tag.isEmpty()) {
00809                     if (!X.len) {
00810                         sprintf_str("%s%s", name.data, tags[i]);
00811                         mxInitQSpaceVecR23(F,L,getMxData(str,iter),X);
00812
00813                         if (k>=int(X.len)) { wblog(FL,
00814                             "ERR index out of bounds (%d/%d; %d)\n"
00815                             "(hint: is stored NRG operator data up to date?)",
00816                             k,X.len,nrgIdx.len);
00817                         }
00818                     }
00819                     if (k>=int(X.len)) wblog(FL,
00820                         "ERR index out of bounds (%d/%d; %d)",k,X.len,nrgIdx.len);
00821
00822                     X[k].save2(C[j]);
00823                 }
00824                 else {
00825                     if (!X2.len) {
00826                         sprintf_str("%s%s", nrgIdx[j].tag.data, tags[i]);
00827                         mxInitQSpaceVecR23(F,L, getMxData(str,iter),X2);
00828                     }
00829                     if (k>=(int)X2.len) wblog(FL,
00830                         "ERR index out of bounds (%d/%d)",k,X2.len);
00831
00832                     X2[k].save2(C[j]);
00833                 }
00834             }
00835         }
00836     };
00837
00838     template <class TQ, class TS>

```

```

00839 template<class TD>
00840 void NRGData<TQ,TS>::updateOpProd(
00841     const char *F, int L, unsigned iter,
00842     const NRGData<TQ,TS> &B,
00843     const NRGData<TQ,TD> &C
00844 ){
00845     unsigned i=0,r,n;
00846
00847     QSpace<TQ,TS> Xd;
00848
00849     char ics[13] = "2 2* 23 23*", *ic=0, *ic_=0;
00850     for (; ics[i]; ++i) { if (ics[i]!=' ') ics[i]=0; }
00851
00852     n=B.checkOpConsistency(0,1,C);
00853     if (int(n)<0) wblog(FL,
00854         "ERR got dim/itag inconsistency between B and C !?");
00855     if (!n || B.KK.len!=n || B.DD.len!=n) wblog(FL,
00856         "ERR %s() got empty operators (%d/%d/%d) !?",FCT,B.KK.len,B.DD.len,n);
00857     KK.init(n); KD.init(n); DK.init(n); DD.init(n);
00858
00859     for (i=0; i<n; ++i) {
00860         r=B.KK[i].rank(FL); if (!r) r=B.DD[i].rank(FL);
00861         if (r==2) { ic=ics+0; ic_=ics+3; } else
00862         if (r==3) { ic=ics+6; ic_=ics+9; } else
00863             wblog(FL,"ERR %s() invalid rank r=%d (i=%d)",FCT,r,i);
00864
00865         B.KK[i].contract(F,L,ic,C.KK[i],ic_,KK[i]);
00866         B.KD[i].contract(F,L,ic,C.KD[i],ic_,Xd); KK[i]+=Xd;
00867
00868         B.KK[i].contract(F,L,ic,C.DK[i],ic_,KD[i]);
00869         B.KD[i].contract(F,L,ic,C.DD[i],ic_,Xd); KD[i]+=Xd;
00870
00871         B.DK[i].contract(F,L,ic,C.KK[i],ic_,DK[i]);
00872         B.DD[i].contract(F,L,ic,C.KD[i],ic_,Xd); DK[i]+=Xd;
00873
00874         B.DK[i].contract(F,L,ic,C.DK[i],ic_,DD[i]);
00875         B.DD[i].contract(F,L,ic,C.DD[i],ic_,Xd); DD[i]+=Xd;
00876
00877         if (KK[i].itags.len>2) wblog(FL,
00878             "ERR %s() KK[%d] '%s' !?",FCT,i+1,IT2STR(KK[i]));
00879         if (DD[i].itags.len>2) wblog(FL,
00880             "ERR %s() DD[%d] '%s' !?",FCT,i+1,IT2STR(DD[i]));
00881     }
00882 };
00883
00884 template <class TQ, class TS, class TD>
00885 void applyBG(const char *F, int L,
00886     QSpace<TQ,TS> &B,
00887     const QSpace<TQ,TD> &H0,
00888     const QSpace<TQ,TD> &H2
00889 ){
00890     unsigned i,j,k,m,n;
00891
00892     wbvector<unsigned> I1,I2;
00893     wbvector<double> E1,E2;
00894     wbindex ilh,i2h;
00895
00896     wbararray<TD> Omega;
00897     TD *e1,*e2;
00898
00899     wbcomplex z, Einc = gRX.Einc + gRX.E0, dEinc = Einc - gRX.E2;
00900
00901     gRX.check();
00902
00903     double e12, sgm=gRX.sigma/2.;
00904
00905     if (B.isEmpty()) { return; }
00906     matchH12(FL,B,H0,H2,gRX.iter, ilh,E1,I1, i2h,E2,I2);
00907
00908     for (k=0; k<ilh.len; ++k) {
00909         wbararray<TS> &Bk = *B.DATA[k];
00910         TS *xk=Bk.data;
00911
00912         i=ilh[k]; m=I1[i+1]-I1[i];
00913         j=i2h[k]; n=I2[j+1]-I2[j];
00914
00915         if (!Bk.sameSize(m,n)) { Bk.info(FL,"B",k);
00916             MXPut(FL,"i").add(Bk,"B").add(m,"m").add(n,"n").add(k+1,"k");
00917             wblog(FL,"ERR %s() dimension mismatch (%d) !?",FCT,k+1);
00918         }
00919
00920         e1=E1.data+I1[i]; e2=E2.data+I2[j];
00921
00922         for (j=0; j<n; ++j) {
00923             for (i=0; i<m; ++i) { e12 = e2[j] - e1[i];
00924                 z.r = dEinc.r - e12;
00925                 z.i =-Einc.i;

```

```

00926
00927         if (!gRX.wrxs) {
00928             z/=sqrt(abs(z.i));
00929         }
00930     else {
00931         wblog(FL, "ERR %s() deprecated using BHeisenberg!!", FCT);
00932
00933         z.i -= sgm*(gRX.E0+gRX.E2);
00934         if (z.i>=0) wblog(FL, "ERR %s() got z=%g%+gi", FCT, z.r, z.i);
00935
00936         z/=sqrt(abs(z.i));
00937     }
00938
00939     xk[i+m*j]/=z;
00940 }
00941 }
00942 };
00943
00944 template <class TQ, class TS>
00945 template <class TD>
00946 void NRGData<TQ, TS>::initBG(
00947     const char *F, int L,
00948     const NRGData<TQ, TD> &B,
00949     const NRGData<TQ, TD> &H0,
00950     const NRGData<TQ, TD> &H2,
00951     int m
00952 ){
00953     unsigned i;
00954
00955     KD.initT_(B.KD); for (i=0; i<KD.len; ++i) applyBG(F_L, KD[i], H0.K, H2.D);
00956     DK.initT_(B.DK); for (i=0; i<DK.len; ++i) applyBG(F_L, DK[i], H0.D, H2.K);
00957     DD.initT_(B.DD); for (i=0; i<DD.len; ++i) applyBG(F_L, DD[i], H0.D, H2.D);
00958
00959     if (m==3) return; else
00960     if (m!=4) wblog(FL, "ERR %s() invalid m=%d !?", FCT, m);
00961
00962     KK.initT_(B.KK); for (i=0; i<KK.len; ++i) applyBG(F_L, KK[i], H0.K, H2.K);
00963 };
00964
00965 template <class TQ, class TD, class TZ>
00966 void WeightE0_dim1(QSpace<TQ, TZ> &X, const QSpace<TQ, TD> &H, double a) {
00967
00968     unsigned i, j, k; widxt_t m1, m2, d, D;
00969     double efac, w=1;
00970
00971     wbMatrix<TQ> Qh, Qx;
00972     wbindex ih, ix;
00973
00974     if (X.isEmpty()) { return; }
00975     if ((i=X.rank())<2 || i>3 || H.rank()!=2) wblog(FL,
00976         "ERR %s() got rank-%d data (%d)", FCT, i, H.rank());
00977
00978     if (a<0) { w=0; a=-a; } else
00979     if (!a) wblog(FL, "ERR %s() invalid w=%g", FCT, a);
00980
00981     X.getQsub(0, Qx);
00982     H.getQsub(0, Qh);
00983
00984     matchIndex(Qh, Qx, ih, ix, 1, &m1, &m2);
00985     if (m1 || ix.len!=Qx.dim1) wblog(FL, "ERR %s() "
00986         "need unique match (%d/%d; %d/%d)", FCT, m1, m2, ix.len, Qx.dim1);
00987     if (ix.len!=X.DATA.len) wblog(FL, "ERR %s() "
00988         "severe QSpace inconsistency (%d/%d)!", FCT, ix.len, X.DATA.len);
00989
00990     for (k=0; k<ix.len; ++k) {
00991         wbarray<TZ> &xk=*X.DATA[ix[k]];
00992         const wbarray<TD> &ek=*H.DATA[ih[k]];
00993         xk.getMatSize(d, D, 1);
00994
00995         if (!ek.isVector()) wblog(FL,
00996             "ERR %s() non-diagonal H (%d: %s)", FCT, k+1, SSTR(ek));
00997         if (d!=ek.dim1()) wblog(FL, "ERR %s() "
00998             "size mismatch (%d: d=%ld/%ld x %ld)", FCT, k+1, d, ek.dim1(), D);
00999
01000         for (i=0; i<d; ++i) {
01001             efac=exp(-a*ek[i]*ek[i]); if (!w) { efac=1-efac; }
01002             efac=sqrt(efac);
01003             for (j=0; j<D; ++j) { xk[i+d*j]*=efac; }
01004         }
01005     }
01006 };
01007
01008 template <class TQ, class TS, class TD>
01009 void matchH12(
01010     const char *F, int L,
01011     QSpace<TQ, TS> &X,
01012     const QSpace<TQ, TD> &H1, const QSpace<TQ, TD> &H2, unsigned iter,

```

```

01013     wbindex &i1h, wvector<double> &E1, wvector<unsigned> &I1,
01014     wbindex &i2h, wvector<double> &E2, wvector<unsigned> &I2,
01015     char redCGR=0
01016 ){
01017     unsigned i,j,e=0; widx_t m1,m2;
01018     wbindex i1,i2;
01019     wperm P;
01020
01021     wvector<unsigned> D1,D2;
01022     wbMatrix<TQ> Q1h,Q2h,Q1,Q2;
01023
01024     double Escale = gEScale(iter);
01025
01026     if (X.isEmpty()) {
01027         wblog(F_L,"ERR %N empty matrix elements !?");
01028     }
01029
01030     if (X.rank(F_L)==3 || !X.CGR.isEmpty()) { X.isConsistent(F_L);
01031         if (X.CGR.isEmpty()) {
01032             if (!X.allAbelian()) { wblog(F_L,
01033                 "ERR %s() expecting CG data for rank-3 data (%s)",
01034                 FCT, X.qtype.toStr().data);
01035             }
01036         }
01037         else {
01038             for (i=0; i<X.DATA.len; ++i) {
01039                 wbararray<TS> &D=(*X.DATA[i]);
01040
01041                 if (D.SIZE.len==3) {
01042                     if (D.SIZE[2]!=1) wblog(F_L,
01043                         "ERR expecting operator to have dim3=1 (%s)",SSTR(D));
01044                     if (redCGR) { D.SIZE.len=2; }
01045                 }
01046                 else if (D.SIZE.len!=2) wblog(F_L,
01047                     "ERR invalid rank operator (%s)",SSTR(D));
01048
01049                 if (redCGR) {
01050                     for (j=0; j<X.CGR.dim2; ++j) {
01051                         if (!X.CGR(i,j).isScalar()) wblog(F_L,
01052                             "ERR %s() got non-scalar cref (%d,%d): %s",
01053                             FCT,i+1,j+1,X.CGR(i,j).toStr().data
01054                         );
01055                     }
01056                 }
01057             }
01058         }
01059
01060         getEData(H1,E1,D1); E1*=Escale; D1.cumsum_(I1,'x');
01061         getEData(H2,E2,D2); E2*=Escale; D2.cumsum_(I2,'x');
01062
01063         H1.getQsub(0,Q1h); X.getQsub(0,Q1);
01064         H2.getQsub(0,Q2h); X.getQsub(1,Q2);
01065
01066         i=matchIndex(Q1h,Q1,i1h,i1,1,&m1,&m2); if (m1 || i1.len!=Q1.dim1) ++e;
01067         i=matchIndex(Q2h,Q2,i2h,i2,1,&m1,&m2); if (m1 || i2.len!=Q2.dim1) ++e;
01068
01069         if (e) wblog(F_L,
01070             "ERR %s() need unique and exact match (%d)!",FCT,e);
01071         if (i1.len!=X.DATA.len) wblog(F_L,
01072             "ERR %s() severe QSpace inconsistency (%d %d)!",
01073             FCT, i1.len, X.DATA.len
01074         );
01075
01076         i1.toPerm(P); P.Invert(); i1h.Select(P);
01077         i2.toPerm(P); P.Invert(); i2h.Select(P);
01078     };
01079
01080     template<class TQ, class TD>
01081     void NRGData<TQ,TD>::unsetAllRefs() {
01082
01083         unsigned i,j;
01084         const char* tags[4] = { "KK","KD","DK","DD" };
01085
01086         for (i=0; i<4; ++i) {
01087             wvector< QSpace<TQ,TD> > &C = getOpsXX(tags[i]);
01088
01089             for (j=0; j<C.len; ++j) {
01090                 if (nrgIdx[j].idx<0 || !C[j].isref) continue;
01091                 C[j].init();
01092             }
01093         }
01094     }
01095
01096     template<class TQ, class TD>
01097     unsigned NRGData<TQ,TD>::skipOpZeros(double eps, char bflag) {
01098
01099

```



```

01100     wbvector< QSpace<TQ,TD> > *XX[] = { &KK, &KD, &DK, &DD };
01101     unsigned i,j,n,r=0;
01102
01103     for (i=0; i<4; ++i) {
01104         n=XX[i]->len;
01105         for (j=0; j<n; ++j)
01106             r += (*XX[i])[j].SkipZeroData(eps,bflag);
01107     }
01108
01109     return r;
01110 }
01111
01112 template<class TQ, class TD>
01113 void NRGData<TQ,TD>::times(TD fac, const char *flag) {
01114
01115     unsigned i,j,n=(flag ? strlen(flag) : 0);
01116
01117     if (n==2) {
01118         if (!strcmp(flag,"XX")) {
01119             const char* tags[4] = { "KK","KD","DK","DD" };
01120             for (i=0; i<4; ++i) {
01121                 wbvector< QSpace<TQ,TD> > &X=getOpsXX(tags[i]);
01122                 for (j=0; j<X.len; ++j) X[j]*=fac;
01123             }
01124         }
01125         else {
01126             wbvector< QSpace<TQ,TD> > &X=getOpsXX(flag);
01127             for (j=0; j<X.len; ++j) X[j]*=fac;
01128         }
01129     }
01130     else if (n==1) {
01131         if (flag[0]=='X') {
01132             getQSpace('K')*=fac;
01133             getQSpace('D')*=fac;
01134         }
01135         else {
01136             getQSpace(flag[0])*=fac;
01137         }
01138     }
01139     else wblog(FL, "ERR invalid flag '%s'", flag ? flag : "(null)");
01140 };
01141
01142 template<class TQ, class TD>
01143 void NRGData<TQ,TD>::saveOp(
01144     const char *F, int L, int iter, unsigned m) const {
01145
01146     unsigned i=0, n=nrgIdx.len, nrefs=0;
01147     const char* tags[4]={ "KK","KD","DK","DD" };
01148
01149     if (!m || m==2 || m>4) wblog(FL,
01150         "ERR value out of bounds (m=%d/%d)",m,4);
01151     if (iter<0) iter=NRG_ITER;
01152
01153     for (; i<n; ++i) { if (nrgIdx[i].idx>=0) { ++nrefs; } }
01154
01155     if (store && (!n || nrefs<n)) { i=0;
01156         if (m==3) { i=1; m=4; }
01157         #if 0
01158         if (WBLOG_IO) { sprintf_str(
01159             "%s() iter=%2d: store=%d, nrefs=%d/%ld, i=%d/%d ",
01160             FCT, iter, store, nrefs,nrgIdx.len, i,m);
01161             wblog(FL, MX? "MX: %s":"MAT %s",str);
01162         }
01163         #endif
01164         for (; i<m; ++i) { updatePara(F,L,tags[i],iter); }
01165     }
01166 };
01167
01168 template<class TQ, class TD>
01169 void NRGData<TQ,TD>::updatePara(
01170     const char *F, int L, const char *tag, mxArray *a, int iter) const {
01171
01172     sprintf_str("%s%s", name.data, tag);
01173     if (!str[0]) wblog(F,L,"ERR invalid name (empty)");
01174
01175     if (iter<0) iter=NRG_ITER;
01176     if (MX) {
01177         if (!NAME.isEmpty())
01178             mxUpdateField(F,L, MX,str,iter,a);
01179     }
01180     else {
01181         Wb::Clock clio("data:I/O",1,0,&fdmClocks);
01182         matPutVariable(F,L,getFileName(iter),str,a);
01183     }
01184 };
01185
01186 template<class TQ, class TD>

```

```

01187 void NRGData<TQ,TD>::updatePara(const char *F, int L,
01188     const char *tag,
01189     int iter,
01190     const char *tag2,
01191     char all
01192 ) const {
01193
01194     unsigned n=strlen(tag), e=0;
01195     mxArray *a=0;
01196
01197     if (n==1) {
01198         if (tag[0]=='K') { a=K.toMx(); } else
01199         if (tag[0]=='D') { a=D.toMx(); } else e|=1;
01200     }
01201     else if (n==2) {
01202         const wbvector< QSpace<TQ,TD> > *XX=0;
01203         if (!strcmp(tag,"KK")) { XX=&KK; } else
01204         if (!strcmp(tag,"KD")) { XX=&KD; } else
01205         if (!strcmp(tag,"DK")) { XX=&DK; } else
01206         if (!strcmp(tag,"DD")) { XX=&DD; } else e|=2;
01207
01208         if (!e) {
01209             if (all>0 || (all<0 && store>1)) { a=XX->toMx(); }
01210             else {
01211                 wbindex I; getRelevantOps(I);
01212                 a=QSpaceVec2Mx(*XX,I);
01213             }
01214         }
01215     }
01216     else { e|=4; }
01217     if (e) wblog(FL,"ERR invalid tag '%s'",tag);
01218
01219     sprintf_str("%s%s", name.data, tag2 && tag2[0] ? tag2 : tag);
01220     if (!str[0] || !name.isName()) wblog(F,L,"ERR invalid name '%s'",str);
01221
01222     if (iter<0) iter=NRG_ITER;
01223     if (MX) {
01224         mxUpdateField(F,L, MX,str,iter,a);
01225     }
01226     else {
01227         Wb::Clock clio("data:I/O",1,0,&fdmClocks);
01228         matPutVariable(F,L,getFileName(iter),str,a);
01229     }
01230 };
01231
01232 template<class TQ, class TD>
01233 inline void NRGData<TQ,TD>::getRelevantOps(wbindex &I) const {
01234
01235     unsigned i,k;
01236     I.init(nrgIdx.len);
01237
01238     for (k=i=0; i<nrgIdx.len; ++i) {
01239         if (nrgIdx[i].idx<0) I[k++]=i;
01240     }
01241
01242     I.len=k;
01243 }
01244
01245 template<class TQ, class TD>
01246 const QSpace<TQ,TD>& NRGData<TQ,TD>::getQSpace(
01247     const char *t, unsigned i
01248 ) const {
01249
01250     int d=-1;
01251
01252     if (!t || !index("KD",t[0]) || !index("KD",t[1]) || t[2])
01253     wblog(FL,"ERR invalid tag '%s', t ? t : "(null)");
01254
01255     if (t[0]=='K') {
01256         if (t[1]=='K')
01257             { if (i<KK.len) return KK[i]; else d=KK.len; }
01258         else { if (i<KD.len) return KD[i]; else d=KD.len; }
01259     }
01260     else {
01261         if (t[1]=='K')
01262             { if (i<DK.len) return DK[i]; else d=DK.len; }
01263         else { if (i<DD.len) return DD[i]; else d=DD.len; }
01264     }
01265
01266     wblog(FL,"ERR index out of bounds (%s: %d/%d)",t,i,d);
01267     return K;
01268 };
01269
01270 template<class TQ, class TD>
01271 void NRGData<TQ,TD>::updateInfo(const char *F, int L,
01272     const char *vname, mxArray *a, char keep
01273 ) const {

```

```

01274
01275     if (!a) wblog(F,L,
01276         "WRN update info '%s' with NULL mxArray", vname);
01277
01278     if (MX) {
01279         mexPutVariable("caller", vname, a);
01280         if (!keep) mxDestroyArray(a);
01281     }
01282     else {
01283         Wb::Clock clio("data:I/O",1,0,&fdmClocks);
01284         matPutVariable(F,L,getFileNamel(),vname,a,keep);
01285     }
01286 }
01287
01288 template<class TQ, class TD>
01289 void NRGData<TQ,TD>::updateInfop(const char *F, int L,
01290     const char *vtag, mxArray *a, char keep
01291 ) const {
01292
01293     if (!name.data || !name.data[0]) wblog(F,L,
01294         "WRN update info '%s' of unnamed operator", vtag);
01295
01296     sprintf_str("%s%s",
01297         name.data ? name.data : "",
01298         vtag ? vtag : "");
01299 };
01300
01301     updateInfo(F,L,str,a,keep);
01302 }
01303
01304 template<class TQ, class TD>
01305 bool NRGData<TQ,TD>::opExists(char tflag) {
01306
01307     if (tflag) {
01308         unsigned i, k=(NRG_N>2 ? NRG_N-2 : 0);
01309         const char* tag[3] = { "KK","KD","DK" };
01310
01311         for (i=0; i<3; ++i) {
01312             sprintf_str("%s%s",name.data,tag[i]);
01313             if (!isQSpaceVec(str,k)) return 0;
01314         }
01315     }
01316     else {
01317         sprintf_str("%sKK",name.data);
01318         return isQSpaceVec(str,0);
01319     }
01320
01321     return 1;
01322 }
01323
01324 template<class TQ, class TD>
01325 bool NRGData<TQ,TD>::initOp(const char *F, int L,
01326     const wvector< QSpace<TQ,TD> > &C0,
01327     char tflag,
01328     NRGData<TQ,TD>* CR,
01329     const char *istr
01330 ) {
01331     unsigned i,j,m,iter,e,r, nrefs=0;
01332     const char* tag[3] = { "KK","KD","DK" };
01333     unsigned ntag=sizeof(tag)/sizeof(char*);
01334     wvector< QSpace<TQ,TD> > NRG;
01335     mxArray *a;
01336
01337     nrgIdx.initDef(C0.len); kloc=0;
01338
01339     if (C0.len==0) {
01340         init(1); return 0;
01341     }
01342
01343     if (istr && istr[0]) sprintf_str("%s ",istr); else str[0]=0;
01344
01345     if (!name.isName(99)) wblog(F,L,
01346         "ERR invalid name for %soperator (%d,'%s')",
01347         str, nrgIdx.len, name.data
01348     );
01349
01350     for (i=0; i<C0.len; ++i) {
01351         r=-1;
01352         if (!C0[i].isConsistent(r) || r<2 || r>4) wblog(F,L,
01353             "ERR invalid operator (got rank-%d QSpace) !?\n"
01354             "%s (%s)",r,SHORT_FL,str);
01355         if (C0[i].QDIM!=C0[0].QDIM) wblog(F,L,
01356             "ERR invalid operator (QDIM=%d, %d) !?\n%s:%d %s",
01357             C0[i].QDIM, C0[0].QDIM,FL,str);
01358     }
01359
01360     KK.initDef(C0);

```

```

01361
01362     for (i=0; i<C0.len; ++i) {
01363         if (nrgIdx[i].idx>=0) continue;
01364         for (j=i+1; j<C0.len; ++j)
01365             if (C0[j]==C0[i]) {
01366                 nrgIdx[j].init(i,this);
01367                 ++nrefs;
01368             }
01369     }
01370
01371     if (CR) {
01372         m=CR->KK.len;
01373         for (i=0; i<C0.len; ++i) {
01374             if (nrgIdx[i].idx>=0) continue;
01375             for (j=0; j<m; ++j) if (C0[i]==CR->KK[j]) {
01376                 nrgIdx[i].init(j,CR);
01377                 ++nrefs;
01378                 break;
01379             }
01380         }
01381         sprintf_str("%s%s",CR->name.data,"KK_");
01382         a=getMxData(str,0);
01383         if (a && mxIsQSpaceVecR23(FL,a)) {
01384             mxInitQSpaceVecR23(F,L,a,NRG); m=NRG.len;
01385             for (i=0; i<C0.len; ++i) {
01386                 if (nrgIdx[i].idx>=0) continue;
01387                 for (j=0; j<m; ++j) if (C0[i]==NRG[j]) {
01388                     nrgIdx[i].init(j,CR->name.data);
01389                     ++nrefs;
01390                     break;
01391                 }
01392             }
01393         }
01394
01395         if (nrefs==nrgIdx.len) {
01396             return nrefs;
01397         }
01398     }
01399
01400     e=0;
01401
01402     if (tflag) {
01403         iter=(NRG_N>2 ? NRG_N-2 : 0);
01404         for (i=0; i<ntag; ++i) {
01405             sprintf_str("%s%s",name.data,tag[i]);
01406             if (!isQSpaceVec(str,iter)) { ++e; break; }
01407         }
01408     }
01409
01410     iter=0;
01411
01412     sprintf_str("%s%s",name.data,"KK");
01413     if (!isQSpaceVec(str,iter)) ++e;
01414
01415     sprintf_str("%s%s",name.data,"KK_");
01416     a=getMxData(str,iter); if (!a || !mxIsQSpaceVecR23(FL,a)) ++e;
01417
01418     if (e) {
01419         calc=store=1;
01420         return nrefs;
01421     }
01422
01423     mxInitQSpaceVecR23(F,L,a,NRG);
01424
01425     for (i=0; i<C0.len; ++i) {
01426         if (nrgIdx[i].idx>=0 && nrgIdx[i].tag.isEmpty()) continue;
01427         for (j=0; j<NRG.len; ++j) if (C0[i]==NRG[j]) {
01428             nrgIdx[i].tag.init();
01429             nrgIdx[i].idx=j; ++nrefs;
01430             break;
01431         }
01432     }
01433
01434     calc= (nrefs==C0.len ? 0 : 1);
01435     store=(nrefs==C0.len ? 0 : 1);
01436
01437     return nrefs;
01438 };
01439
01440 template<class TQ, class TD>
01441 bool NRGData<TQ,TD>::initOp(const char *F, int L,
01442     const wvector< wvector< QSpace<TQ,TD> > > &CC,
01443     NRGData<TQ,TD>* CR,
01444     const char *istr,
01445     char tflag
01446 ){
01447     unsigned i,j,l,m,iter,q=0,e,r, nrefs=0;

```

```

01448     const char* tag[3]={ "KK","KD","DK" };
01449     unsigned ntag=sizeof(tag)/sizeof(char*);
01450     const mxArray *a;
01451
01452     wbvector< wbvector< QSpace<TQ,TD> > > NRG;
01453
01454     nrgIdx.initDef(CC.len);
01455
01456     if (CC.len==0) {
01457         init(1); return 0;
01458     }
01459
01460     if (istr && istr[0])
01461         sprintf_str("%s ",istr); else str[0]=0;
01462
01463     if (!name.isName(99)) wblog(F,L,
01464         "ERR invalid name for %soperator (%d,'%s')",
01465         str, nrgIdx.len, name.data
01466     );
01467
01468     kloc=-1;
01469
01470     for (i=0; i<CC.len; ++i) {
01471         for (l=j=0; j<CC[i].len; ++j) {
01472             const QSpace<TQ,TD> &x = CC[i][j]; if (x.isEmpty()) continue;
01473             r=-1; l=j; if (!q) q=x.QDIM;
01474
01475             if (!x.isConsistent(r) || r<2 || r>3) wblog(F,L,
01476                 "ERR invalid operator (rank-%d QSpace) !?\n"
01477                 "--> %s %s",r, SHORT_FL, str);
01478             if (x.QDIM!=q) wblog(FL,
01479                 "ERR invalid operator (QDIM=%d, %d) !?\n--> %s %s",
01480                 x.QDIM, q, SHORT_FL, str
01481             );
01482             kloc=MAX(kloc,(int)l); }
01483
01484     if (kloc<0) wblog(FL,"ERR all empty operators !?");
01485     kloc=MAX(0,kloc-1);
01486
01487     KK.initDef(CC.len);
01488     CI.initDef(CC.len);
01489     for (i=0; i<CC.len; ++i) {
01490         if (!CC[i].len) wblog(FL,"ERR empty operator set !?");
01491         CI[i].initDef(CC[i]);
01492         KK[i]=CC[i][0];
01493     }
01494
01495     for (i=0; i<CC.len; ++i) { if (nrgIdx[i].idx>=0) continue;
01496         for (j=i+1; j<CC.len; ++j)
01497             if (CC[i].isEqual(CC[j])) { ++nrefs;
01498                 nrgIdx[j].init(i,this);
01499             }
01500     }
01501
01502     if (CR && CR->CI.len) { m=CR->CI.len;
01503         for (i=0; i<CC.len; ++i) {
01504             if (nrgIdx[i].idx>=0) continue;
01505
01506             for (j=0; j<m; ++j)
01507                 if (CC[i].isEqual(CR->CI[j])) {
01508                     nrgIdx[i].init(j,CR);
01509                     ++nrefs; break;
01510                 }
01511         }
01512
01513         sprintf_str("%s%s",CR->name.data,"CI_");
01514         a=getMxInfo(str);
01515         if (a) { if (mxIsQSpaceVecVec(a,2)) {
01516             mxInitQSpaceVecVec(F,L,a,NRG); m=NRG.len;
01517             for (i=0; i<CC.len; ++i) {
01518                 if (nrgIdx[i].idx>=0) continue;
01519                 for (j=0; j<m; ++j) if (CC[i]==NRG[j]) {
01520                     nrgIdx[i].init(j,CR->name.data);
01521                     ++nrefs; break;
01522                 }
01523             } else wblog(FL,
01524                 "WRN %s::CI_ not a QSpace cell vector !?", CR->name.data);
01525         }
01526
01527         if (nrefs==nrgIdx.len) { return nrefs; }
01528     }
01529
01530     e=0;
01531
01532     if (tflag && NRG_N>2) {
01533         iter=NRG_N-2;
01534         for (i=0; i<ntag; ++i) {

```

```

01535         sprintf_str("%s%s",name.data,tag[i]);
01536         if (!isQSpaceVec(str,iter)) { ++e; break; }
01537     }
01538 }
01539
01540 iter=0;
01541 sprintf_str("%s%s",name.data,"KK");
01542 if (!isQSpaceVec(str,iter)) ++e;
01543
01544 sprintf_str("%s%s",name.data,"CI_");
01545 a=getMxData(str,iter); if (!a || !mxIsQSpaceVec(FL,a)) ++e;
01546
01547 if (e) { calc=store=1;
01548         wblog(FL," * calculate all operators %s (%d)",
01549             name.data, nrgIdx.len); return nrefs;
01550     }
01551
01552 mxInitQSpaceVecVec(F,L,a,NRG); m=NRG.len;
01553
01554 for (i=0; i<CC.len; ++i) {
01555     if (nrgIdx[i].idx>=0 && nrgIdx[i].tag.isEmpty()) continue;
01556     for (j=0; j<m; ++j) if (CC[i].isEqual(NRG[j])) {
01557         nrgIdx[i].tag.init();
01558         nrgIdx[i].idx=j; ++nrefs;
01559         break;
01560     }
01561 }
01562
01563 calc = (nrefs==CC.len) ? 0 : 1;
01564 store = (nrefs==CC.len) ? 0 : 1;
01565
01566 return nrefs;
01567 };
01568
01569 template<class TQ, class TD>
01570 void NRGData<TQ,TD>::initSym(const char *tag) {
01571
01572     if (strlen(tag)==2) {
01573         const wvector< QSpace<TQ,TD> > &K=getOpsXX(tag);
01574         unsigned i,n=K.len;
01575         issym.init(n);
01576
01577         for (i=0; i<n; ++i) {
01578             if (K[i].isHConj()) issym[i]=+1; else
01579             if (K[i].isAHerm()) issym[i]=-1;
01580         }
01581     }
01582     else if (strlen(tag)==1) {
01583         const QSpace<TQ,TD> &K=getQSpace(tag);
01584         issym.init(1);
01585         if (K.isHConj()) issym[0]=+1; else
01586         if (K.isAHerm()) issym[0]=-1;
01587     }
01588     else wblog(FL,"ERR invalid tag '%s'", tag ? tag : "(null)");
01589 }
01590
01591 template<class TQ, class TD>
01592 void NRGData<TQ,TD>::check_dloc(
01593     const char *F, int L, const char *fun, unsigned dloc, unsigned iter) const {
01594
01595     widx_t d;
01596     K.getDim(2,&d);
01597
01598     if (iter+1>=NRG_N) {
01599         if (d) wblog(F_L,
01600             "ERR %s() got dloc=%li/%d at k=%d/%d !?", fun,d,dloc,iter,NRG_N);
01601         return;
01602     }
01603
01604     if (d!=dloc && (iter || !D.isEmpty())) {
01605         char s[64]; snprintf(s,64,
01606             "got inconsistent dloc=%li/%d (k=%d/%d)",d,dloc,iter,NRG_N);
01607
01608         if (D.isEmpty())
01609             wblog(F_L,"WRN %s() %s",fun,s);
01610         else wblog(F_L,"ERR %s() %s",fun,s);
01611     }
01612 };
01613
01614 template<class TQ, class TD>
01615 template<class TD_>
01616 int NRGData<TQ,TD>::checkOpConsistency(
01617     const char *F, int L, const NRGData<TQ,TD_> &B) const {
01618
01619     unsigned e=0, n = (KK.len ? KK.len : B.KK.len);
01620     if (!n) n = (DD.len ? DD.len : B.DD.len);
01621     if (!n) { return n; }

```

```

01622
01623     if (KK.len && B.KK.len) {
01624         if (KK.len!=B.KK.len) { ++e;
01625             if (L) wblog(F_L,"WRN ... KK.len=%d/%d",KK.len,B.KK.len); }
01626         if (KD.len!=B.KD.len || KD.len!=n) { ++e;
01627             if (L) wblog(F_L,"WRN ... KD.len=%d/%d/%d",KD.len,B.KD.len,n); }
01628         if (DK.len!=B.DK.len || DK.len!=n) { ++e;
01629             if (L) wblog(F_L,"WRN ... DK.len=%d/%d/%d",DK.len,B.DK.len,n); }
01630         if (DD.len!=B.DD.len || DD.len!=n) { ++e;
01631             if (L) wblog(F_L,"WRN ... DD.len=%d/%d/%d",DD.len,B.DD.len,n); }
01632     }
01633     else if (KK.len) {
01634         if (KD.len!=n) { ++e;
01635             if (L) wblog(F_L,"WRN ... KD.len=%d/%d/%d",KD.len,B.KD.len,n); }
01636         if (DK.len!=n) { ++e;
01637             if (L) wblog(F_L,"WRN ... DK.len=%d/%d/%d",DK.len,B.DK.len,n); }
01638         if (DD.len!=n) { ++e;
01639             if (L) wblog(F_L,"WRN ... DD.len=%d/%d/%d",DD.len,B.DD.len,n); }
01640     }
01641     else {
01642         if (B.KD.len!=n) { ++e;
01643             if (L) wblog(F_L,"WRN ... KD.len=%d/%d/%d",KD.len,B.KD.len,n); }
01644         if (B.DK.len!=n) { ++e;
01645             if (L) wblog(F_L,"WRN ... DK.len=%d/%d/%d",DK.len,B.DK.len,n); }
01646         if (B.DD.len!=n) { ++e;
01647             if (L) wblog(F_L,"WRN ... DD.len=%d/%d/%d",DD.len,B.DD.len,n); }
01648     }
01649     if (e) {
01650         if (F) wblog(FL,"ERR %s() got size consistency",FCT);
01651         return -e;
01652     }
01653     return n;
01654 };
01655
01656 template<class TQ, class TD>
01657 bool NRGData<TQ,TD>::isQSpaceVec(const char *nm, int iter) {
01658     if (iter<0) iter=NRG_ITER;
01659     if (iter>=(int)NRG_N) wblog(FL,
01660         "ERR Index out of bounds (%d/%d)", iter, NRG_N);
01661     const mxArray *a=getMxData(nm,iter,'i');
01662     if (a && mxIsQSpaceVecR23(FL,a)) return 1;
01663     return 0;
01664 }
01665
01666 template<class TQ, class TD>
01667 bool NRGData<TQ,TD>::checkVec(
01668     const char *F, int L,
01669     const char *XX, unsigned R, const char *istr
01670 ) {
01671     unsigned l,n=128;
01672     char nm[n]; int fid;
01673     char cflag=WbUtil<TD>::isComplex();
01674     l=snprintf(nm,n,"%s%s", name.data, XX);
01675     if (l>=n) wblog(FL,"ERR strlen out of bounds (%d/%d)",l,n);
01676     if (MX) {
01677         const size_t *s=mxGetDimensions(MX);
01678         const int n=mxGetNumberOfDimensions(MX);
01679         fid=mxGetFieldNumber(MX, nm);
01680         if (fid<0) {
01681             if (istr) {
01682                 wblog(F,L,"WRN invalid %s", istr);
01683                 wblog(FL, "ERR field '%s' could not be found",nm);
01684             }
01685             return 1;
01686         }
01687         if (n!=2 || (s[0]!=1 && s[1]!=1)) {
01688             if (istr) {
01689                 wblog(F,L,"WRN invalid %s (%s)",istr,nm);
01690                 wblog(FL, "ERR NRG data must be row structure vector (%d,%d)",
01691                     (int)s[0],(int)s[1]);
01692             }
01693             return 1;
01694         }
01695         if (!NRG_N) NRG_N=unsigned(s[0]*s[1]); else
01696         if (NRG_N!=unsigned(s[0]*s[1])) {
01697             if (istr) {

```

```

01709         wblog(F,L,"WRN invalid %s (%s)",istr,nm);
01710         wblog(FL, "ERR size mismatch (%d/%d)", NRG_N, int(s[0]*s[1]));
01711     }
01712     return 1;
01713 }
01714
01715     return !mxIsQSpaceVec(MX,fid,R,cflag);
01716 }
01717 else {
01718     int i=mxIsQSpaceVec(F_L,NAME.data,nm,R,0,0,cflag,NRG_N);
01719     if (i<0) {
01720         wblog(F,L,"ERR invalid %s (%s)%N%s",istr? istr:"!?",nm,str);
01721         return 1;
01722     }
01723
01724     if (!NRG_N) NRG_N=i; else
01725     if (i!=(int)NRG_N) {
01726         wblog(F,L,"WRN invalid %s (%s)", istr?istr:"!?", nm);
01727         wblog(FL,"ERR size mismatch (%d,%d)", i, NRG_N);
01728         return 1;
01729     }
01730 }
01731
01732     return 0;
01733 };
01734
01735 template<class TQ, class TD>
01736 bool NRGData<TQ,TD>::checkVecVec(const char *XX, unsigned R) {
01737
01738     unsigned l,n=128;
01739     char nm[n]; int fid;
01740
01741     l=snprintf(nm,n,"%s",name.data,XX);
01742     if (l>=n) wblog(FL,"ERR strlen out of bounds (%d/%d)",l,n);
01743
01744     if (MX) {
01745         fid=mxGetFieldNumber(MX,nm);
01746         if (fid<0) {
01747             sprintf_str( "%s:%d field `%s' could not be found.",
01748                 FL,nm); return 1;
01749         }
01750
01751         if (!NRG_N) {
01752             const size_t *s=mxGetDimensions(MX);
01753             const int n=mxGetNumberOfDimensions(MX);
01754             if (n!=2 || (s[0]!=1 && s[1]!=1)) { sprintf_str(
01755                 "%s:%d NRG data must be row structure vector (%d,%d)",
01756                 FL, (int)s[0], (int)s[1]); return 1;
01757             }
01758             NRG_N=unsigned(s[0]*s[1]);
01759         }
01760         return !mxIsQSpaceVEC(MX, fid, R);
01761     }
01762     else {
01763
01764         int i=mxIsQSpaceVEC(FL,NAME.data, nm, R);
01765         if (i<0) return 1;
01766         if (!NRG_N) NRG_N=i;
01767         if (i!=(int)NRG_N) {
01768             sprintf_str("%s:%d size mismatch (%d,%d)",
01769                 FL, i, NRG_N); return 1;
01770         }
01771     }
01772
01773     return 0;
01774 }
01775
01776 template<class TQ, class TD>
01777 mxArray* NRGData<TQ,TD>::getMxData(const char *vs, int iter, char ionly) {
01778     if (iter<0) iter=NRG_ITER;
01779     if (iter>=(int)NRG_N) wblog(FL,
01780         "ERR %s() index out of bound (%d/%d)",FCT,iter,NRG_N);
01781
01782     if (MX) {
01783         int fid=mxGetFieldNumber(MX, vs);
01784         if (fid<0) return NULL;
01785         return mxGetFieldByNumber(MX,iter,fid);
01786     }
01787     else {
01788         Wb::matFile F(FL,getFileNames(iter),"r");
01789         Wb::Clock clcio("data:I/O",1,0,&fdmClocks);
01790
01791         if (aux) { mxDestroyArray(aux); aux=NULL; }
01792         if (ionly)
01793             { aux = matGetVariableInfo(F.mfp,vs); }
01794         else { aux = matGetVariable      (F.mfp,vs); }
01795     }

```



```

01796         F.close(); return aux;
01797     }
01798 };
01799
01800 template<class TQ, class TD>
01801 const mxArray* NRGData<TQ,TD>::getMxInfo(const char *v) {
01802
01803     if (!v || !v[0]) wblog(FL,
01804         "ERR %s() invalid usage (got %s)",FCT,v? "empty":"null");
01805
01806     if (aux) { mxDestroyArray(aux); aux=NULL; }
01807
01808     if (MX) {
01809         return mexGetVariablePtr("caller",v);
01810     }
01811     else {
01812         Wb::matFile F(FL,getFileNamesI(),"r");
01813         Wb::Clock clio("data:I/O",1,0,&fdmClocks);
01814
01815         aux=matGetVariable(F.mfp, v);
01816
01817         return aux;
01818     }
01819 };
01820
01821 template<class TQ, class TD>
01822 mxArray* NRGData<TQ,TD>::toMx() const {
01823
01824     MXPut S(0,0);
01825     S.add(name,"name")
01826       .add(calc,"calc").add(store,"store")
01827       .add(kloc,"kloc").add(issym,"issym")
01828       .addP(MXPut(0,0)
01829         .add(CI,"CI").add(nrgIdx,"nrgIdx").add(NAME,"NAME")
01830         .add(Wb::num2Str(MX,"0x%lX"),"MX").add(locMX,"locMX")
01831         .add(NRG_ITER,"ITER")
01832         .add(NRG_N,"N").toMx(),"Idx"
01833       );
01834
01835     if (KK.len || KD.len || DK.len || DD.len) {
01836         S.add(KK,"KK").add(KD,"KD").add(DK,"DK").add(DD,"DD"); }
01837
01838     if (!K.isEmpty() || !D.isEmpty()) {
01839         S.add(K,"K").add(D,"D"); }
01840
01841     return S.toMx();
01842 };
01843
01844 template<class TQ, class TD>
01845 void NRGData<TQ,TD>::info(const char *vstr) const {
01846
01847     unsigned i, nl=0, n_=128;
01848     char sx[nrgIdx.len+1], s_[n_];
01849
01850     for (i=0; i<nrgIdx.len; ++i) {
01851         if ( nrgIdx[i].idx<0) sx[i]='C';
01852         if (!nrgIdx[i].nrg) sx[i]='R';
01853         else if ( nrgIdx[i].nrg==this)
01854             sx[i]='1';
01855         else sx[i]='2';
01856     }; sx[i]=0;
01857
01858     snprintf(s_,n_,"%d/%d; calc=%d (%s; %d); store=%d",
01859     NRG_ITER+1, NRG_N, calc, sx, nrgIdx.len, store);
01860
01861     if (MX) {
01862         PRINTF("\n%s Structure `s' (0x%lX), %s\n",
01863         vstr && vstr[0] ? vstr : (name.data ? name.data : "ans"),
01864         NAME.data ? NAME.data : "",MX,s_);
01865     }
01866     else {
01867         PRINTF("\n%s File structure `s', %s\n",
01868         vstr && vstr[0] ? vstr : (name.data ? name.data : "ans"),
01869         NAME.data ? NAME.data : "",s_);
01870     }
01871
01872     if (!KK.isEmpty()) {
01873         if (!nl) { PRINTF("\n"); ++nl; }
01874         for (i=0; i<KK.len; ++i) {
01875             snprintf(s_,n_,"KK[%d]", i);
01876             KK[i].info(s_);
01877         }
01878     }
01879
01880     if (!KD.isEmpty()) {
01881         if (!nl) { PRINTF("\n"); ++nl; }
01882         for (i=0; i<KD.len; ++i) {

```

```

01883         snprintf(s_,n_,"KD[%d]", i);
01884         KD[i].info(s_);
01885     }
01886 }
01887 if (!DK.isEmpty()) {
01888     if (!nl) { PRINTF("\n"); ++nl; }
01889     for (i=0; i<DK.len; ++i) {
01890         snprintf(s_,n_,"DK[%d]", i);
01891         DK[i].info(s_);
01892     }
01893 }
01894 if (!DD.isEmpty()) {
01895     if (!nl) { PRINTF("\n"); ++nl; }
01896     for (i=0; i<DD.len; ++i) {
01897         snprintf(s_,n_,"DD[%d]", i);
01898         DD[i].info(s_);
01899     }
01900 }
01901
01902 nl=0;
01903
01904 if (!K.isEmpty()) { if (!nl) { PRINTF("\n"); ++nl; }; K.info("K"); }
01905 if (!D.isEmpty()) { if (!nl) { PRINTF("\n"); ++nl; }; D.info("D"); }
01906
01907 if (nl) PRINTF("\n");
01908 };
01909
01910 template <class TQ, class TD>
01911 void NRGData<TQ,TD>::dispIdx(const char *F, int L) const {
01912
01913     char s[32];
01914     wblog(F,L,"%NNRGIndex %s (calc=%d, store=%d)%N", name.data, calc, store);
01915
01916     for (unsigned i=0; i<nrgIdx.len; ++i) {
01917         if (!nrgIdx[i].tag.isEmpty())
01918             strncpy(s, nrgIdx[i].tag.data,32);
01919         else if (nrgIdx[i].nrg)
01920             strcpy(s, nrgIdx[i].nrg==this ? "(this)" : "(that)");
01921         else if (nrgIdx[i].idx>=0)
01922             strcpy(s, "(file)");
01923         else strcpy(s, "(calc)");
01924
01925         printf("%6d: %4d %s:%lX\n", i, nrgIdx[i].idx, s, nrgIdx[i].nrg);
01926     }
01927
01928     if (nrgIdx.len) printf("\n");
01929 }
01930
01931 template <class TQ, class TD>
01932 void NRGData<TQ,TD>::info(const char *F, int L, const char *istr) const {
01933
01934     unsigned i, nc=0;
01935     wbsting mark(nrgIdx.len+1);
01936     wbsting S(128);
01937
01938     for (i=0; i<nrgIdx.len; ++i) {
01939
01940         if (nrgIdx[i].nrg && nrgIdx[i].nrg!=this) { mark[i]='x'; }
01941         else if (!nrgIdx[i].tag.isEmpty()) { mark[i]='X'; }
01942         else if (nrgIdx[i].nrg==this) { mark[i]='r'; }
01943         else if (nrgIdx[i].idx>=0) { mark[i]='R'; }
01944         else { ++nc; mark[i]='*'; }
01945     }
01946
01947     if (nc) {
01948         S.pushf(FL," %s%d/%d op%s",
01949             calc ? "calc " : "", nc, KK.len, KK.len!=1 ? "s":"");
01950     }
01951     else {
01952         S.pushf(FL," %d op%s%s", KK.len, KK.len!=1 ? "s":"",
01953             calc ? ", calc!?" : "", all from store or referenced");
01954     }
01955
01956     if (store) S.push(FL," and store");
01957     else if (nc || calc) {
01958         S.push(FL," (not stored)");
01959     }
01960
01961     if (kloc          ) S.pushf(FL," nloc=%d", kloc+1);
01962
01963     if (strcmp(istr,name.data)) {
01964         wblog(F,L," * %s (%3s): [%s] %s",
01965             name.data, istr, mark.data, S.data[0] ? S.data+2 : "");
01966     }
01967     else {
01968         wblog(F,L," * %s [%s] %s",
01969             name.data, mark.data, S.data[0] ? S.data+2 : "");

```

```

01970     }
01971 };
01972
01973 template <class TQ, class TD>
01974 void NRGData<TQ,TD>::disp_otype(const char *F, int L) const {
01975
01976     wstring s(128); unsigned i=0;
01977
01978     if (name.data) s.cpy(FL,name.data);
01979
01980     if (KK.len) {
01981         s.pushf(FL,"%sKK: ",i?"; ":"");
01982         for (i=0; i<KK.len; ++i) s.pushf(FL,"%d",KK[i].otype);
01983     }
01984     if (DK.len) {
01985         s.pushf(FL,"%sTK: ",i?"; ":"");
01986         for (i=0; i<DK.len; ++i) s.pushf(FL,"%d",DK[i].otype);
01987     }
01988     if (KD.len) {
01989         s.pushf(FL,"%sKT: ",i?"; ":"");
01990         for (i=0; i<KD.len; ++i) s.pushf(FL,"%d",KD[i].otype);
01991     }
01992     if (DD.len) {
01993         s.pushf(FL,"%sTT: ",i?"; ":"");
01994         for (i=0; i<DD.len; ++i) s.pushf(FL,"%d",DD[i].otype);
01995     }
01996
01997     wblog(F_L," * NRGData of type %s() %s ",FCT,s);
01998 };
01999
02000 template <class TQ, class TD, class TZ>
02001 void backPropagateOps(const char *F, int L,
02002     wvector< QSpace<TQ,TZ> > &BK,
02003     const wvector< QSpace<TQ,TZ> > &BX,
02004     const QSpace<TQ,TD> &A0,
02005     const QSpace<TQ,TD> &A2
02006 ){
02007     if (!BX.len) return;
02008     if (!BK.len) BK.init(BX.len); else
02009     if (BK.len!=BX.len) wblog(F_L,
02010         "ERR %s() length mismatch (%d/%d)",FCT,BK.len,BX.len);
02011
02012     unsigned i=0,r;
02013     QSpace<TQ,TZ> Q1,Q2;
02014
02015     for (; i<BX.len; ++i) { r=BK[i].rank(F_L);
02016         if (!r && !i) {
02017             for (; i<BX.len; ++i) { if (BX[i].rank(F_L)) break; }
02018             if (i==BX.len) return;
02019         }
02020         if (r<2 || r>3) wblog(F_L,
02021             "ERR %s() invalid rank r=%d (%d/%d) !?",FCT,r,i+1,BX.len);
02022
02023         A2.contract(F_L,"2*",BX[i],2,Q1);
02024         A0.contract(F_L,"23",Q1,"32",Q2);
02025         BK[i]+=Q2;
02026     }
02027 };
02028
02029 template <class TQ, class TZ>
02030 template <class TD>
02031 void NRGData<TQ,TZ>::backPropagateOp(const char *F, int L,
02032     const NRGData<TQ,TD> &A0,
02033     const NRGData<TQ,TD> &A2
02034 ){
02035     unsigned i=0, e=0;
02036     wvector< QSpace<TQ,TZ> > BK;
02037
02038     A2.check_dloc(F_LF,gRX.dloc,gRX.iter);
02039
02040     backPropagateOps(F,L,BK, KK, A0.K, A2.K);
02041     backPropagateOps(F,L,BK, KD, A0.K, A2.D);
02042     backPropagateOps(F,L,BK, DK, A0.D, A2.K);
02043     backPropagateOps(F,L,BK, DD, A0.D, A2.D);
02044
02045     for (; i<BK.len; ++i) { if (BK[i].isEmpty()) { ++e; }
02046         BK[i]*=(1./gRX.dloc);
02047         BK[i].SkipZeroData(DEPS2,'b');
02048     }
02049     if (e && gRX.iter>kloc) {
02050         MXPut(FL,"Ie").add(*this,"B02").add(gRX.iter,"iter").add(BK,"BK")
02051         .add(A0,"A0").add(A2,"A2");
02052         wblog(FL,"ERR %s() got %d/%d empty BK at iter=%d/%d !?",
02053             FCT,e,BK.len,gRX.iter,kloc);
02054     }
02055
02056     BK.save2(KK);

```

```

02057 };
02058
02059 template <class TQ, class TD> inline
02060 void updateOp_L(const char *F, int L,
02061     const QSpace<TQ,TD> &Xin,
02062     const QSpace<TQ,TD> &A1,
02063     const QSpace<TQ,TD> &A2,
02064     QSpace<TQ,TD> &Xout
02065 ){
02066     unsigned rk, r_=-1;
02067     QSpace<TQ,TD> Xk;
02068
02069     if (Xin.isEmpty()) { wblog(F_L,
02070         "ERR %s() got empty L-operator",FCT); return; }
02071     if (int(rk=Xin.isOperator(&r_))<=0) { Xin.info("L-op");
02072         wblog(F_L,"ERR %s() got invalid L-operator (%d/%d)",FCT,rk,r_);
02073     }
02074
02075     Xin.contract(F,L,2,A2,1,Xk);
02076     A1.contract(F,L,"1,3;*",Xk, (rk!=3 ? "1,3":"1,4"), Xout);
02077
02078     if (rk==3) Xout.Permute("1,3,2");
02079
02080     Xout.otype=Xin.otype;
02081     Xout.SkipZeroData();
02082 };
02083
02084 template <class TQ, class TD> inline
02085 void updateOp_s(const char *F, int L,
02086     const QSpace<TQ,TD> &x,
02087     const QSpace<TQ,TD> &A1,
02088     const QSpace<TQ,TD> &A2,
02089     QSpace<TQ,TD> &Xout
02090 ){
02091     unsigned rk, r_=-1;
02092     QSpace<TQ,TD> Xk;
02093
02094     if (x.isEmpty()) { wblog(F_L,
02095         "WRN %s() got empty L-operator",FCT); return; }
02096     if (int(rk=x.isOperator(&r_))<=0) { x.info("local-op");
02097         wblog(F_L,"ERR %s() got invalid L-operator (%d/%d)",FCT,rk,r_);
02098     }
02099
02100     A2.contract(F,L,3,x,2, Xk);
02101     A1.contract(F,L,"1,3;*", Xk,"1,3", Xout);
02102
02103     Xout.otype=x.otype;
02104     Xout.SkipZeroData();
02105 };
02106
02107 template <class TQ, class TD>
02108 inline void updateOp_Ls(
02109     const QSpace<TQ,TD> &Xin,
02110     const QSpace<TQ,TD> &A1,
02111     const QSpace<TQ,TD> &A2,
02112     QSpace<TQ,TD> &Xout
02113 ){
02114     unsigned rk, r_=-1;
02115     QSpace<TQ,TD> Xk;
02116
02117     if (Xin.isEmpty()) { wblog(F_L,
02118         "WRN %s() got empty L-operator",FCT); return; }
02119     if ((rk=Xin.isOperator(&r_,'x'))!=4) { Xin.info("Ls-op");
02120         wblog(F_L,"ERR %s() got invalid Ls-operator (%d/%d)",FCT,rk,r_);
02121     }
02122
02123     Xin.contract("3,4",A2,"1,3", Xk);
02124     A1.contract("1,3;*",Xk,"1,2",Xout);
02125
02126     Xout.otype=Xin.otype;
02127     Xout.SkipZeroData();
02128 };
02129
02130 template <class TQ, class TD>
02131 inline void updateOp_sL(
02132     const QSpace<TQ,TD> &Xin,
02133     const QSpace<TQ,TD> &A1,
02134     const QSpace<TQ,TD> &A2,
02135     QSpace<TQ,TD> &Xout
02136 ){
02137     unsigned rk, r_=-1;
02138     QSpace<TQ,TD> Xk;
02139
02140     if (Xin.isEmpty()) { wblog(F_L,
02141         "WRN %s() got empty L-operator",FCT); return; }
02142     if ((rk=Xin.isOperator(&r_))!=4) { Xin.info("sL-op");
02143         wblog(F_L,"ERR %s() got invalid sL-operator (%d)",FCT,rk,r_);

```

```

02144     }
02145
02146     Xin.contract("4,3",A2,"1,3", Xk);
02147     A1.contract("1,3;*",Xk,"2,1",Xout);
02148
02149     Xout.otype=Xin.otype;
02150     Xout.SkipZeroData();
02151 };
02152
02153 template <class TQ, class TD> inline
02154 void updateOp_loc(const char *F, int L,
02155     const NRGData<TQ,TD> &NRG, unsigned i,
02156     const QSpace<TQ,TD> &XL,
02157     const QSpace<TQ,TD> &x,
02158     const QSpace<TQ,TD> &A1,
02159     const QSpace<TQ,TD> &A2,
02160     QSpace<TQ,TD> &Xout,
02161     unsigned iter
02162 ){
02163     bool o1=0, o2=0;
02164     unsigned rk, r_=-1;
02165     QSpace<TQ,TD> Ax,XA;
02166
02167     if (!x.isEmpty()) {
02168         if (int(rk=x.isOperator(&r_))<=0) { x.info("local_op"); wblog(FL,
02169             "ERR %s() got invalid local operator (%d/%d)",FCT,rk,r_); }
02170         if (XL.otype!=x.otype && !XL.isEmpty()) wblog(FL,
02171             "WRN %s() got different operator types (%s; %s)",
02172             FCT, QS_STR[XL.otype],QS_STR[x.otype]);
02173         o2=(rk==3);
02174
02175         wblog(FL," * contracting %s[%d] onto A[%d].%s",
02176             NRG.name.data,i, iter, A2.itags.toStrk(str,8,2,"s"));
02177
02178         A2.contract(F,L,3,x,"-op",Ax);
02179     }
02180     else Ax.init2ref(A2);
02181
02182     if (iter || !XL.isEmpty()) { r_=-1;
02183         if (int(rk=XL.isOperator(&r_))<=0) { x.info("L-op"); wblog(FL,
02184             "ERR %s() got invalid L-operator (%d/%d)",FCT,rk,r_); }
02185         o1=(rk==3);
02186
02187         if (iter<1) wblog(FL,"TST contracting onto A[%d].%s",
02188             iter, A2.itags.toStrk(str,8,0,"L"));
02189
02190         if (o1 && o2) { o1=o2=0;
02191             XL.contract(F,L,"23",Ax,"14",XA); }
02192         else {
02193             XL.contract(F,L,"-op",Ax,1,XA);
02194         }
02195     }
02196     else XA.init2ref(Ax);
02197
02198     if (o1)
02199         A1.contract(F,L,"13*",XA,"14", Xout,"132");
02200     else A1.contract(F,L,"13*",XA,"13", Xout);
02201
02202     if (o1) Xout.otype=XL.otype; else
02203     if (o2) Xout.otype=x.otype;
02204
02205     Xout.SkipZeroData();
02206 };
02207
02208 template <class TQ, class TD>
02209 int Wb::updateOp(const char *F, int L,
02210     const NRGData<TQ,TD> &NRG,
02211     wbvector< QSpace<TQ,TD> > &F12,
02212     const QSpace<TQ,TD> &A1,
02213     const QSpace<TQ,TD> &A2,
02214     const wbvector< QSpace<TQ,TD> > &FKK,
02215     unsigned iter
02216 ){
02217     unsigned i, j=0, m=0, r_=-1, rk=-1, nop=FKK.len;
02218     int e=0, il=0;
02219
02220     const wbvector< wbvector<QSpace<TQ,TD> > > &CI=NRG.CI;
02221     const wbvector< NRGIndex<TQ,TD> > &nrgIdx=NRG.nrgIdx;
02222
02223     if (A1.isEmpty() || A2.isEmpty()) { F12.initDef(nop); return 0; }
02224
02225     if (nrgIdx.len!=nop) wblog(FL,
02226         "ERR %s() severe size mismatch (nrgIdx: %d,%d)",FCT,nrgIdx.len,nop);
02227
02228     if (iter>0) {
02229         for (i=0; i<nop; ++i) {
02230             if (nrgIdx[i].idx>=0) continue;

```

```

02231         if (!i1=FKK[i].isConsistent(FL))
02232         || int(rk=FKK[i].isOperator(&r_)<=0) break;
02233     }
02234 }
02235 else if (CI.isEmpty()) {
02236     for (i=0; i<nop; ++i) {
02237         if (!i1=FKK[i].isConsistent(FL)) ||
02238         int(rk=FKK[i].isOperator(&r_,'x'))<=0
02239     ){ wblog(FL,
02240         "WRN FKK[%d] -> (%d; %d/%d/%d) ",i+1,i1,rk,r_,FKK[i].rank());
02241         break;
02242     }
02243 }
02244 }
02245 else {
02246     if (nop!=CI.len) wblog(FL,
02247         "ERR size inconsistency (%d/%d)",nop,CI.len);
02248
02249     for (i=0; i<nop; ++i) { m=CI[i].len;
02250     for (j=0; j<m; ++j) { if (!CI[i][j].isEmpty()) {
02251         if (!i1=CI[i][j].isConsistent(FL)) ||
02252         int(rk=CI[i][j].isOperator(&r_)<=0) break;
02253     }}}
02254 }
02255
02256 if (i<nop || j<m) wblog(F,L,
02257     "ERR invalid rank-%d operator (iter=%d, op=%d/%d)",rk,iter,i+1,nop);
02258 if (A1.rank(F,L)!=3 || A2.rank(F,L)!=3 || A1.QDIM!=A2.QDIM) wblog(F,L,
02259     "ERR %s() severe rank inconsistency (%s; %s)",
02260     FCT, A1.sizeStrQ().data, A2.sizeStrQ().data);
02261
02262 if (F12.len!=nop)
02263 F12.initDef(nop);
02264
02265 if ((int)iter>NRG.kloc) {
02266     for (i=0; i<nop; ++i) {
02267         if (nrgIdx[i].idx>=0) continue;
02268         updateOp_L(F,L, FKK[i], A1, A2, F12[i] );
02269         wblog().runningLarge(FL);
02270
02271         if (F12[i].isEmpty() && !FKK[i].isEmpty()) ++e;
02272     }
02273 }
02274 else if (!CI.isEmpty()) {
02275     QSpace<TQ,TD> E;
02276
02277     for (i=0; i<nop; ++i) { if (nrgIdx[i].idx>=0) continue;
02278     try { updateOp_loc(F,L,NRG,i,
02279         FKK[i], iter+2>CI[i].len ? E : CI[i][iter+1],
02280         A1, A2, F12[i],iter);
02281     }
02282     catch (...) { wblog_ERR_updateOp(FL,"loc",i,nop,iter); }
02283
02284     if (F12[i].isEmpty() && !FKK[i].isEmpty()) ++e;
02285 }
02286 }
02287 else {
02288     char Lflag = A1.getDim(1)>1 || A2.getDim(1)>1;
02289     unsigned l=0, tlen=16; char tag[tlen];
02290
02291     wblogstring mark(nop,'*');
02292
02293     static time_t last_call_iter0=0;
02294     time_t tnow=time(0);
02295
02296     for (i=0; i<nop; ++i) {
02297         if (nrgIdx[i].idx>=0) continue;
02298         rk=FKK[i].rank(FL);
02299
02300         if (rk==2 || (rk==3 && FKK[i].otype==QS_OPERATOR)) {
02301             if (Lflag &&
02302                 FKK[i].hasQOverlap(1,A1,1,'<')>0 &&
02303                 FKK[i].hasQOverlap(2,A1,1,'<')>0
02304             ){
02305                 try { updateOp_L(F,L,FKK[i],A1,A2, F12[i]); }
02306                 catch (...) { wblog_ERR_updateOp(FL,"L",i,nop,iter); }
02307                 mark[i]='L';
02308             }
02309             else
02310             if (FKK[i].hasQOverlap(1,A1,3,'<')>0 &&
02311                 FKK[i].hasQOverlap(2,A1,3,'<')>0
02312             ){
02313                 try { updateOp_s(F,L,FKK[i], A1, A2, F12[i]); }
02314                 catch (...) { wblog_ERR_updateOp(FL,"s",i,nop,iter); }
02315                 mark[i]='s';
02316             }
02317             else

```

```

02318         if (FKK[i].hasQOverlap(1,A1,2,'<')>0 &&
02319             FKK[i].hasQOverlap(2,A1,2,'<')>0
02320     ){
02321         mark[i]='R';
02322     }
02323     else wblog(FL,
02324         "ERR %s() failed to deal input operator with A%d",FCT,iter);
02325 }
02326 else if (rk==4) {
02327     wvector<int> m1,m2;
02328     int mm[] = {
02329         FKK[i].hasQOverlap(1,A1,1,'<')>0,
02330         FKK[i].hasQOverlap(2,A2,3,'<')>0,
02331         FKK[i].hasQOverlap(3,A2,1,'<')>0,
02332         FKK[i].hasQOverlap(4,A2,3,'<')>0,
02333
02334         FKK[i].hasQOverlap(1,A1,3,'<')>0,
02335         FKK[i].hasQOverlap(2,A2,1,'<')>0,
02336         FKK[i].hasQOverlap(3,A2,3,'<')>0,
02337         FKK[i].hasQOverlap(4,A2,1,'<')>0
02338     };
02339     m1.init(4,mm); m2.init(4,mm+4);
02340
02341     if (m1.allUnequal(0)) {
02342         try { updateOp_Ls(FKK[i], A1, A2, F12[i]); }
02343         catch (...) { wblog_ERR_updateOp(FL,"Ls",i,nop,iter); }
02344         mark[i]='D';
02345     }
02346     else
02347     if (m2.allUnequal(0)) {
02348         try { updateOp_sL(FKK[i], A1, A2, F12[i]); }
02349         catch (...) { wblog_ERR_updateOp(FL,"sL",i,nop,iter); }
02350         mark[i]='d';
02351     }
02352     else {
02353         if (STRICT_ITER0) { mark[i]='?';
02354             if (mark.count('?')==1 && last_call_iter0+5<tnow){
02355                 wblog(FL,
02356                     "WRN %NFailed to fully associate rank-4 tensor product\n"
02357                     "operator with first site. [ %s; %s ]\n"
02358                     "A1: [ %s ]; A2: [ %s ]%N"
02359                     "NB! Hint: A0 may not be complete due to truncation "
02360                     "in NRGWilsonQS already at iter=0 !?%N",
02361                     m1.toStr().data, m2.toStr().data,
02362                     SSTR(A1.getDim()), SSTR(A2.getDim()));
02363             }}
02364
02365         try { updateOp_Ls(FKK[i], A1, A2, F12[i]); }
02366         catch (...) { wblog_ERR_updateOp(FL,"Ls",i,nop,iter); }
02367     }
02368 }
02369 else wblog(FL,
02370     "ERR %s() invalid rank-%d operator (iter=%d)",FCT,rk,iter);
02371
02372     if (F12[i].isEmpty() && !FKK[i].isEmpty()) ++e;
02373 }
02374
02375 l=snprintf(tag,tlen,"FDM %2d/%d",iter,NRG_N); {
02376     if (NRG.name.data && NRG.name.data[0] && l<tlen)
02377         l+=snprintf(tag+l,tlen-l," %s:",NRG.name.data);
02378     if (l>=tlen) wblog(FL,
02379         "ERR %s() string out of bounds (%d/%d)",FCT,l,tlen);
02380 }
02381
02382 if (!nop)
02383     wblog(FL,"%s got empty spectral ops",tag);
02384 else {
02385     unsigned il=0, m=mark.count('*');
02386     for (i=0; i<nop; ++i) { if (mark[i]!='*') { il=i; break; }}
02387     for (; i<nop; ++i) { if (mark[i]!='*') {
02388         if (mark[i]!=mark[il]) break;
02389     }}
02390 }
02391
02392 if (m==nop) { wblog(FL,
02393     "%s no spectral ops to contract (all referenced)",
02394     tag,mark.data);
02395 }
02396 else if (i==nop && !m) {
02397     const unsigned slen=32;
02398     char ostr[slen];
02399     if (nop>1)
02400         { snprintf(ostr,slen,"all %d spectral ops",nop); }
02401     else { snprintf(ostr,slen,"spectral operator"); }
02402
02403     if (mark[0]!='R') wblog(FL,
02404         "%s contracting %s onto %c",tag,ostr,mark[0]);

```

```

02405         else wblog(FL,
02406             "%s %s already assumed in %c",tag,ostr,mark[0]
02407         );
02408     }
02409     else if (i==nop && m) {
02410         if (mark[i]!='R') wblog(FL,"%s contracting %d/%d "
02411             "spectral ops onto [%s]",tag,nop-m,nop,mark.data);
02412         else wblog(FL,"%s all %d/%d spectral "
02413             "ops already assumed in %s",tag,nop-m,nop,mark.data);
02414     }
02415     else {
02416         wblog(FL,"%s contracting "
02417             "%d/%d spectral ops onto [%s]",tag,nop-m,nop,mark.data);
02418     }
02419 }
02420
02421     last_call_iter0=tnow;
02422 }
02423
02424 if (iter==0) {
02425     for (i=0; i<F12.len; ++i) {
02426         if (F12[i].itags.len>2 && !F12[i].itags[2]) {
02427             F12[i].itags[2].init("op");
02428         }
02429     }
02430 }
02431
02432 wbtotop().runningLarge(FL);
02433 return e;
02434 };
02435
02436 template <class TQ, class TD>
02437 unsigned applyZ0(const QSpace<TQ,TD> &Z0, QSpace<TQ,TD> &CK) {
02438
02439     unsigned r=CK.rank(FL);
02440     const QS_TYPES otype=CK.otype;
02441
02442     if (r==2 || r==3) {
02443         if (r==3 && CK.otype!=QS_OPERATOR) wblog(FL,
02444             "ERR %s() expecting otype==%s for rank-%d op (%s)",
02445             FCT,QS_STR[QS_OPERATOR],r,QS_STR[CK.otype]);
02446         if (Z0.rank(FL)!=2) wblog(FL,
02447             "ERR %s() Z0 of rank-2 required for rank-%d op (%d)",
02448             FCT,r,Z0.rank(FL));
02449     };
02450
02451     if (Z0.hasQOverlap(2, CK,1,'>')<=0) {
02452         wblog(FL,"ERR %s() Z0 does not match operator space",FCT);
02453         return 1;
02454     }
02455     Z0.contract(2,CK,1,CK);
02456     CK.otype=otype;
02457 }
02458 else if (r==4) {
02459     if (Z0.rank(FL)!=4) wblog(FL,
02460         "ERR tensor Z0 required for tensor operator (rank=%d/%d)",
02461         Z0.rank(FL), r);
02462 };
02463
02464     if (Z0.hasQOverlap(3, CK,1,'>')<=0 ||
02465         Z0.hasQOverlap(4, CK,2,'>')<=0) {
02466         wblog(FL,"ERR %s() Z0 does not match operator space",FCT);
02467         return 2;
02468     }
02469     Z0.contract("3 4",CK,"1 2",CK);
02470     CK.otype=otype;
02471 }
02472 else wblog(FL,"ERR %s() invalid rank=%d.",FCT,r);
02473
02474 return 0;
02475 };
02476
02477 template <class TQ, class TD>
02478 unsigned getDLoc(
02479     NRGData<TQ,TD> &A, Wb::SigHandler *sig=NULL, unsigned *Dk=NULL,
02480     char fflag=0
02481 ){
02482     unsigned iter, N=NRG_N; widx_t d0=0, d=0;
02483     char cgflag=-1;
02484
02485     if (Dk) { (*Dk)=0; }; printf("\r");
02486
02487     for (iter=0; iter<N; ++iter) { if (sig) sig->check911();
02488         printf(" %s: %2d/%d loading AK ... \r",FCT,iter,N);
02489         fflush(0);
02490
02491         A.init(FL,"K",iter);

```



```

02492
02493     if (iter==0) cgflag=A.K.gotCGS(FL);
02494
02495     if (A.K.isEmpty()) {
02496         if (iter==N-1) continue; else wblog(FL,
02497         "ERR %s() empty AK at intermediate iteration %d/%d",FCT,iter,N);
02498     }
02499     else {
02500         unsigned r=-1;
02501         A.K.isConsistent(FL,r);
02502         if (cgflag<=1 && !A.K.QIDX.isUnique()) wblog(FL,
02503         "ERR invalid AK at iteration %d/%d",iter,N);
02504     }
02505
02506     #ifndef WB_SKIP_ASSERT
02507     if (fflag) { QSpace<TQ,TD> E;
02508         A.K.contract("1,3;*",A.K,"1,3",E);
02509         if (!E.isIdentityMatrix(1E-12)) {
02510             MXPut(FL).add(A,"A").add(E,"E");
02511             wblog(FL,"ERR NRG[%d].AK not in LRs index order",iter,N);
02512         }
02513     }
02514     #endif
02515
02516     A.K.getDim(2,&d);
02517
02518     if (iter>2 || d0) {
02519         if (d!=d0) {
02520             MXPut(FL,"a").add(A.K,"K").add(d,"d").add(d0,"d0")
02521             .add(iter,"iter").add(N,"N");
02522             wblog(FL,"ERR %s() inconsistent local dimension "
02523             "(d=%d/%d @ k=%d/%d)",FCT,d,d0,iter,N
02524             );
02525         }
02526     }
02527     else if (iter==2 || iter+2==N) { d0=d; }
02528     else {
02529         A.init(FL,"D",iter);
02530         if (!A.D.isEmpty()) { d0=d; }
02531     }
02532
02533     if (Dk) {
02534         d=A.K.getDim(1);
02535         if (*Dk<d) { *Dk=d; continue; }
02536     }
02537     if (!fflag && iter>2) break;
02538 }
02539
02540 if (!d0) wblog(FL,"ERR %s() "
02541 "failed to get local dimension (d=%d; %d/%d)",FCT,d0,iter,N);
02542 return d0;
02543 };
02544
02545 #endif
02546

```

5.17 QSpace.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : QSpace
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_QSPACE_COL_MAJOR_HH__
00021 #define __WB_QSPACE_COL_MAJOR_HH__
00022
00023 /* ----- //
00024  * QSpace - set of objects (possibly complex matrices)
00025  * organized by generalized index set storeed as row vectors (recs)

```

```

00026     in index matrix. // Wb,Aug08,05 Apr07,06
00027
00028     Introduction of non-abelian QSpace through variable 'qtype'.
00029     see also clebsch.hh // Wb,Sep25,09
00030
00031     NB! QSpace currently only implemented on linux system
00032     (matlab+gcc)
00033
00034     Trying to get it run on a macbook // Wb,Jan18,19
00035     Problems encountered:
00036     * OSX, by default, has case-*IN*sensitive files system (APFS)
00037     encountered trouble with wbcmat.hh + wbCMat.hh
00038     * default "gcc" compiler actually is with Xcode
00039     which is much more picky concerning header files!
00040
00041 // ----- */
00042
00043 #ifndef WB_CLOCK
00044 #define WBC_QSPACE_IO
00045 #endif
00046
00047 enum QS_TYPES {
00048     QS_NONE, QS_OPERATOR, QS_AMATRIX,
00049     QS_NUM_TYPES
00050 };
00051
00052 const char* QS_STR[QS_NUM_TYPES] = {
00053     "", "operator", "A-matrix",
00054 };
00055
00056 template <class TQ, class TD>
00057 class QSpace {
00058
00059 public:
00060
00061     QSpace(unsigned n=0)
00062         : QDIM(n), otype(QS_NONE), mt(Wb::MEM_DEF), isref(0), ctime(0.) {};
00063
00064     QSpace(unsigned m, unsigned n, unsigned nq)
00065         : QSpace(nq) { init(m,n,nq); };
00066
00067     QSpace(const QSpace &B)
00068         : QIDX(B.QIDX), CGR(B.CGR), qtype(B.qtype), QDIM(B.QDIM),
00069         otype(QS_NONE), itags(B.itags), mt(Wb::MEM_DEF),
00070         isref(0), ctime(B.ctime)
00071     {
00072         if (this!=&B) {
00073             unsigned i=0; setupDATA();
00074             for (; i<B.DATA.len; ++i) { DATA[i]->init(*(B.DATA[i])); }
00075         }
00076     };
00077
00078     template<class TQ_, class TD_>
00079     QSpace(const QSpace<TQ_,TD_> &B,
00080         char xflag = WbUtil<TD_>::isComplex() ? 1 : 0
00081     ): QIDX(B.QIDX), CGR(B.CGR), qtype(B.qtype), QDIM(B.QDIM),
00082     otype(QS_NONE), itags(B.itags), mt(Wb::MEM_DEF),
00083     isref(0), ctime(B.ctime)
00084     {
00085         if (!xflag) wblog(FL,"ERR %s() verify type cast %s -> %s",
00086             FCT,TSTR(TD_),TSTR(TD));
00087         setupDATA();
00088         for (unsigned i=0; i<B.DATA.len; ++i) {
00089             DATA[i]->initT(*(B.DATA[i]));
00090         }
00091     };
00092
00093     QSpace( const char *F, int L, const mxArray *a,
00094         char ref=0, char skip_empty=1)
00095         : otype(QS_NONE), mt(Wb::MEM_DEF), isref(0), ctime(0.)
00096         { init(F,L,a,ref,skip_empty); };
00097
00098     QSpace(const mxArray *a, char ref=0, char skip_empty=1)
00099         : otype(QS_NONE), mt(Wb::MEM_DEF), isref(0), ctime(0.)
00100         { init(FL,a,ref,skip_empty); };
00101
00102     ~QSpace() { clearQSpace(); };
00103
00104     QSpace& clearQSpace(char mflag=0) {
00105         if (QIDX.data || DATA.data) { mt=Wb::MEM_DEF; }
00106         QIDX.init(); qtype.init(); itags.init();
00107         clearDATA(mflag); CGR.init(); isref=0; ctime=0.;
00108         return *this;
00109     };
00110
00111     void clearDATA(char mflag=0) {
00112         if (DATA.len) {

```

```

00113         if (!isref && !DATA.isref) {
00114             for (unsigned i=0; i<DATA.len; ++i) { if (DATA[i]) {
00115                 if (mflag) { DATA[i]->initX(); }
00116                 WB_DELETE_1(DATA[i]);
00117             }}
00118         }
00119         DATA.init();
00120     }
00121 };
00122
00123
00124 unsigned rank(const char *F=NULL, int L=0) const {
00125     unsigned r=0;
00126     if (QDIM) { r=QIDX.dim2/QDIM;
00127         if (QIDX.dim2!=r*QDIM) wblog(F_L,
00128             "ERR %s() QSpace inconsistency (QIDX: %dx%d @ %d)",
00129             FCT,QIDX.dim1,QIDX.dim2,QDIM);
00130     }
00131     else if (QIDX.dim2) wblog(F_L,
00132         "ERR %s() QSpace inconsistency (QIDX: %dx%d @ %d)",
00133         FCT,QIDX.dim1,QIDX.dim2,QDIM);
00134     return r;
00135 };
00136
00137 unsigned len() const {
00138     if (QIDX.dim1!=DATA.len) wblog(F_L,
00139         "ERR qspace inconsistency (QIDX.dim1=%d, len=%d).",
00140         QIDX.dim1, DATA.len);
00141     return QIDX.dim1;
00142 };
00143
00144 unsigned nsym() const {
00145     if (CGR.dim2 && qtype.len!=CGR.dim2) wblog(F_L,
00146         "ERR qspace inconsistency (nsym=%d '%s'; %s !?)",
00147         qtype.len, STR(qtype), SSTR(CGR));
00148     return (qtype.len ? qtype.len : QDIM);
00149 };
00150
00151 TQ* Qref(unsigned r, unsigned k) { unsigned l=k*QDIM;
00152     if (!QDIM || r>=QIDX.dim1 || (l+QDIM)>QIDX.dim2) wblog(F_L,
00153         "ERR %s() index out of bounds (%d/%d; %d*%d/%d)",
00154         FCT,r,QIDX.dim1,k,QDIM,QIDX.dim2);
00155     };
00156     return (QIDX.data + r*QIDX.dim2 + l);
00157 };
00158
00159 const TQ* Qref(unsigned r, unsigned k) const {
00160     unsigned l=k*QDIM;
00161     if (!QDIM || r>=QIDX.dim1 || (l+QDIM)>QIDX.dim2) wblog(F_L,
00162         "ERR %s() index out of bounds (%d/%d; %d*%d/%d)",
00163         FCT,r,QIDX.dim1,k,QDIM,QIDX.dim2);
00164     };
00165     return (QIDX.data + r*QIDX.dim2 + l);
00166 };
00167
00168 void init() { clearQSpace(); };
00169
00170 void init(unsigned m, unsigned n, unsigned nq) {
00171     clearQSpace();
00172     QDIM=nq; QIDX.init(m,nq*n);
00173     setupDATA();
00174 };
00175
00176 void init(const wbMatrix<TQ> &Q, unsigned qdim, const QVec *qt=NULL) {
00177     clearQSpace();
00178     QIDX=Q; QDIM=qdim; if (qt) qtype=(*qt); else qtype.init();
00179     setupDATA(F_L);
00180 };
00181
00182 QSpace& init(const QSpace &B) {
00183     initQT_safe(B);
00184     for (unsigned i=0; i<B.DATA.len; ++i) {
00185         DATA[i]->init(*(B.DATA[i]));
00186     }
00187     return *this;
00188 };
00189
00190 template<class TB>
00191 QSpace& initT(const QSpace<TQ,TB> &B) {
00192     initQT_safe(B);
00193     for (unsigned i=0; i<B.DATA.len; ++i) {
00194         DATA[i]->initT(*(B.DATA[i]));
00195     }; return *this;
00196 };
00197
00198 template<class TR>
00199 QSpace& getReal(QSpace<TQ,TR> &R) const {

```

```

00200         unsigned i=0; R.initQT_safe(*this);
00201         for (; i<DATA.len; ++i) { DATA[i]->getReal(*(R.DATA[i])); }
00202         return R;
00203     };
00204
00205     QSpace<TQ,double> getReal() const {
00206         QSpace<TQ,double> R; unsigned i=0; R.initQT_safe(*this);
00207         for (; i<DATA.len; ++i) { DATA[i]->getReal(*(R.DATA[i])); }
00208         return R;
00209     };
00210
00211     template<class TI>
00212     QSpace& getImag(QSpace<TQ,TI> &I) const {
00213         unsigned i=0; I.initQT_safe(*this);
00214         for (; i<DATA.len; ++i) { DATA[i]->getImag(*(I.DATA[i])); }
00215         return I;
00216     };
00217
00218     QSpace<TQ,double> getImag() const {
00219         QSpace<TQ,double> I; unsigned i=0; I.initQT_safe(*this);
00220         for (; i<DATA.len; ++i) { DATA[i]->getImag(*(I.DATA[i])); }
00221         return I;
00222     };
00223
00224     void init(const char *F, int L,
00225              const mxArray *S, char ref, unsigned k, char skip_empty=1);
00226
00227     void init(const char *F, int L, const mxArray *S,
00228              char ref=0, char skip_empty=1
00229     ) {
00230         unsigned n = (S ? mxGetNumberOfElements(S) : 0);
00231         if (n!=1) {
00232             if (!n) wblog(FL,"ERR %s() got empty array",FCT);
00233             else wblog(FL,"ERR %s() got QSpace array (%d entries)",FCT,n);
00234         }
00235         init(F,L,S,ref,0,skip_empty);
00236     };
00237
00238     QSpace& setupDATA(const char *F=NULL, int L=0){
00239
00240         if (!QIDX.dim1) { clearDATA(); return *this; }
00241
00242         if (QIDX.dim2 && (!QDIM || QIDX.dim2%QDIM)) wblog(F,L,
00243         "ERR invalid QIDX (%dx%d; %d)",QIDX.dim1,QIDX.dim2,QDIM);
00244         clearDATA();
00245
00246         if (isref>1) wblog(FL,"ERR %s() got isref=%d",FCT,isref);
00247
00248         DATA.init(QIDX.dim1);
00249
00250         if (!isref) {
00251             if (mt) { if (mt==Wb::MEX_RETURN) {
00252                 if (WBLOG_MMEX) wblog(FL,
00253                 "TST QSpace::init() to %s",Wb::MTYPE_STR[mt]); }
00254             else wblog(FL,
00255             "ERR %s() unexpected mtype=%s",FCT,Wb::MTYPE_STR[mt]);
00256         }
00257
00258         for (unsigned i=0; i<DATA.len; ++i) {
00259             WB_NEW_1(DATA[i]);
00260             DATA[i]->mtype=mt;
00261         }
00262     }
00263
00264     return *this;
00265 };
00266
00267 int gotCGS(const char *F=NULL, int L=0, char xflag=0) const;
00268
00269 int gotCGX(const char *F=NULL, int L=0) const;
00270
00271 QSpace& Enlarge(unsigned m) {
00272     if (!isConsistent(FL) || !QIDX.dim1) wblog(FL,
00273     "ERR %s() can't enlarge empty QSpace (%d)",FCT,QIDX.dim1);
00274     if (!m) return *this;
00275
00276     unsigned i, n0=QIDX.dim1, n=n0+m;
00277     gotCGS(FL);
00278
00279     QIDX.Resize(n, QIDX.dim2);
00280     DATA.Resize(n);
00281     for (i=n0; i<n; ++i) { WB_NEW_1(DATA[i]); }
00282
00283     CGR.Resize(n,CGR.dim2);
00284     return *this;
00285 };
00286

```

```

00287     QSpace& Trim(unsigned n) {
00288         unsigned i, r=-1, n2=QIDX.dim1;
00289         if (!isConsistent(FL,r,0) || n>QIDX.dim1) wblog(FL,
00290             "ERR %s() can't trim to LARGER QSpace (%d/%d)",FCT,n,QIDX.dim1);
00291         if (n==QIDX.dim1) { return *this; }
00292
00293         gotCGS(FL);
00294
00295         QIDX.Resize(n, QIDX.dim2);
00296
00297         for (i=n; i<n2; ++i) { WB_DELETE_1(DATA[i]); }
00298         DATA.Resize(n);
00299
00300         CGR.Resize(n,CGR.dim2);
00301
00302         return *this;
00303     };
00304
00305     QSpace& initScalar(const TD &x) {
00306         clearQSpace(); DATA.init(1);
00307         QIDX.init(1,0);
00308
00309         WB_NEW_2(DATA[0],wbarray<TD>(1,1));
00310         DATA[0]->data[0]=x;
00311
00312         return *this;
00313     };
00314
00315     void init2ref(const mxArray *S){
00316         init(FL,S,'r',0);
00317     };
00318
00319     void init2ref(const QSpace &);
00320
00321     void initOpZ_WET(
00322         const char *F, int L, const QVec &qvec,
00323         const wbMatrix<TQ> &Q1, const wbMatrix<TQ> &Q2,
00324         const wbMatrix<TQ> &Q, const wbarray<TD> &D3,
00325         const double eps1=1E-10,
00326         const double eps2=1E-13
00327     );
00328
00329     void reduceMatEl(const char *F, int L,
00330         const wbMatrix<TQ> &Q1, const wbvector<TD> &dd,
00331         const wbMatrix<TQ> &Q2, const wbvector<TD> &dc,
00332         wbvector<unsigned> &I1, wbvector<unsigned> &I2,
00333         wbvector<TD> &dr
00334     ) const;
00335
00336     void unRef() {
00337
00338         if (!isref && !qtype.isref && !itags.isref &&
00339             !QIDX.isref && !DATA.isref && !CGR.isref
00340         ) return;
00341
00342         wblog(FL,"WRN %s() check this [%d%d%d%d]",FCT, isref,
00343             QIDX.isref, DATA.isref, CGR.isref, qtype.isref, itags.isref);
00344
00345         QIDX.unRef(); qtype.unRef(); itags.unRef();
00346
00347         if (DATA.isref) DATA.unRef(); else
00348             for (unsigned i=0; i<DATA.len; ++i) DATA[i]->unRef();
00349
00350         if (CGR.isref) CGR.unRef();
00351     };
00352
00353     void initIdentity(const QSpace &A, char dflag=0);
00354     void initIdentity(const wbMatrix<TQ> &Q);
00355
00356     QSpace& setupCGR(const char *F=NULL, int L=0);
00357
00358     QSpace& initIdentityCGS(const char *F=NULL, int L=0);
00359     QSpace& initIdentityCGS(const char *F, int L, unsigned k, unsigned r=0);
00360
00361     template<class TD2>
00362     QSpace& initIdentityCGS(const char *F, int L, const QSpace<TQ,TD2>& H);
00363
00364     template<class TA>
00365     QSpace<TQ,TD>& initIdentityCG(
00366         const wbvector< QSpace<TQ,TA> > &A, const wbindex &ia,
00367         char zflag=0
00368     );
00369
00370     template<class TA, class TB>
00371     QSpace<TQ,TD>& initIdentityCG(
00372         const wbvector< QSpace<TQ,TA> > &A, const wbindex &ia,
00373         const wbvector< QSpace<TQ,TB> > &B, const wbindex &ib,

```

```

00374     char vflag='v'
00375 );
00376
00377 template<class TA, class TB>
00378 QSpace<TQ,TD>& initIdentityCG(
00379     const wbvector< QSpace<TQ,TA> > &A,
00380     const wbvector< QSpace<TQ,TB> > &B,
00381     const char *pstr=NULL
00382 ){
00383     wbindex i;
00384     initIdentityCG(A,i,B,i);
00385     if (pstr && pstr[0]) Permute(wbperm(pstr));
00386     return *this;
00387 }
00388
00389 QSpace& initDiagonal(
00390     const wbMatrix<TQ> &Q,
00391     const wbvector<unsigned> &S,
00392     const wbvector<TD> &D,
00393     char rcflag
00394 );
00395
00396 QSpace& init2DiffOp(
00397     const QSpace &A, unsigned ia,
00398     const QSpace &B, unsigned ib
00399 );
00400
00401 template<class TB>
00402 QSpace& initQT_safe(const QSpace<TQ,TB> &B) {
00403     clearQSpace();
00404     QIDX=B.QIDX; QDIM=B.QDIM; qtype=B.qtype; otype=B.otype;
00405     setupDATA(); itags=B.itags; CGR=B.CGR;
00406
00407     if (CGR.dim2) {
00408         if (qtype.len!=CGR.dim2) wblog(FL,
00409             "ERR %s() got CGR size mismatch (%dx%d <> %dx%d)",
00410             FCT,CGR.dim1,CGR.dim2,QIDX.dim1,qtype.len
00411         );
00412     }
00413     else if (QIDX.data) {
00414         if (DATA.len && !qtype.allAbelian()) wblog(FL,
00415             "ERR %s() missing CGR data for %s (%dx%d <> %dx%d)",
00416             FCT,STR(qtype),CGR.dim1,CGR.dim2,QIDX.dim1,qtype.len
00417         );
00418     }
00419     return *this;
00420 };
00421
00422 template <class TA>
00423 QSpace& initQ(const QSpace<TQ,TA> &A, char full=1){
00424     if (isref) wblog(FL,"WRN %s() got QSpace ref!",FCT);
00425     qtype=A.qtype; QDIM=A.QDIM; if (full) {
00426         otype=A.otype; itags=A.itags; }
00427     return *this;
00428 };
00429
00430 template <class TA, class TB>
00431 QSpace& initQ(const char *F, int L,
00432     const QSpace<TQ,TA> &A, char full=1, const QSpace<TQ,TB> *B=NULL
00433 ){
00434     if (B) A.checkQ(F,L,*B);
00435     return initQ(A,full);
00436 };
00437
00438 void initQ(unsigned n, unsigned r, const QSpace &A){
00439     clearQSpace();
00440     QDIM=A.QDIM; qtype=A.qtype; QIDX.init(n,r*QDIM);
00441     setupDATA(FL);
00442     if (!qtype.allAbelian()) setupCGR(FL);
00443 };
00444
00445 void initQ(const wbMatrix<TQ> &Qk, const QSpace &A){
00446     clearQSpace();
00447     QDIM=A.QDIM; qtype=A.qtype; QIDX=Qk;
00448     setupDATA(FL);
00449     if (A.CGR.data) setupCGR(FL);
00450 };
00451
00452 void init_itags(const char *F, int L, const char *s,
00453     unsigned r=-1, char lflag=-1);
00454
00455 QSpace& SetTag(unsigned k, const itag_ &b) {
00456     if ((--k)>=itags.len) wblog(FL,
00457         "ERR %s() index out of bounds (%d/%d)",FCT,k+1,itags.len);
00458     itags.data[k].tSet(b);
00459     return *this;
00460 };

```

```

00461
00462 QSpace& SetTags(const itag_ &b) {
00463     unsigned k=rank(FL);
00464     if (b.t && k>2) wblog(FL,
00465         "ERR %s() got '%s' for rank-%d QSpace !?",FCT,STR(b),k);
00466     for (k=0; k<itags.len; ++k) { itags.data[k].tSet(b); }
00467     return *this;
00468 };
00469
00470 QSpace& SetConjTags() {
00471     for (unsigned k=0; k<itags.len; ++k) { itags.data[k].Conj(); }
00472     return *this;
00473 };
00474
00475 QSpace& SetConjTags(unsigned k1, unsigned k2) {
00476     if (!k1 || !k2) wblog(FL,
00477         "ERR %s(%d) expects 1-based index",FCT,k1,k2);
00478     itags.SetConj(0,0,k1-1);
00479     itags.SetConj(0,0,k2-1); return *this;
00480 };
00481
00482 QSpace& SetConjTag(unsigned k) {
00483     if (!k) wblog(FL,"ERR %s(%d) expects 1-based index",FCT,k);
00484     itags.SetConj(0,0,k-1); return *this;
00485 };
00486
00487 QSpace& SetConjTag(const char* F, int L, unsigned k) {
00488     if (!k) wblog(F_L,"ERR %s(%d) expects 1-based index",FCT,k);
00489     itags.SetConj(F,L,k-1); return *this;
00490 };
00491
00492 QSpace& SetFlag(const char *F, int L, unsigned k, unsigned l) {
00493     if (!k) wblog(F_L,"ERR %s(%d) expects 1-based index",FCT,k);
00494     itags.SetFlag(F_L,k-1,l);
00495     return *this;
00496 };
00497
00498 QSpace& UnsetFlag(const char *F, int L, unsigned k, unsigned l) {
00499     if (!k) wblog(F_L,"ERR %s(%d) expects 1-based index",FCT,k);
00500     itags.UnsetFlag(F_L,k-1,l);
00501     return *this;
00502 };
00503
00504 QSpace& UnsetFlags() {
00505     itags.UnsetFlags();
00506     return *this;
00507 };
00508
00509 QSpace& SetFlags(const char *F, int L, unsigned k, unsigned l=-1) {
00510     if (!k) wblog(F_L,"ERR %s(%d) expects 1-based index",FCT,k);
00511     itags.SetFlags(F_L,k-1,l);
00512     return *this;
00513 };
00514
00515 QSpace& init_tags() { itags.init_tags(); return *this; };
00516
00517 template<class TB>
00518 unsigned matchITags(
00519     const char *F, int L, const QSpace<TQ,TB> &B,
00520     ctrIdx &ia, ctrIdx &ib) const;
00521
00522 unsigned RemoveZLabels(
00523     const char *F, int L, wbMatrix<TQ> &QQ, wbMatrix<TQ> *Z=NULL) const;
00524
00525 void RemoveZLabels(const char *F, int L, wbMatrix<TQ> *Z){
00526     if (QDIM==0 && qtype.len) { QDIM=qtype.Qlenz(); }
00527     QDIM=RemoveZLabels(F_L,QIDX,Z);
00528 };
00529
00530 void ExpandDiagonal(unsigned i1=1, unsigned i2=2);
00531
00532 size_t getDataSize(wbMatrix<size_t> &S) const;
00533 size_t getDataSize(wbvector<size_t> &S) const;
00534 size_t getDataSize() const;
00535 size_t getCGSSize() const;
00536 wbstring totSize2Str() const;
00537
00538 wbstring itags2Str() const {
00539     return itags.toStr().data;
00540 };
00541
00542 unsigned map2Vec (const char *F, int L, wbvector<TD> &V,
00543     const wbvector<unsigned> &S=wbvector<unsigned>());
00544
00545 unsigned map2Vec(TD *v, char iflag=0);
00546
00547 void ExpandQ(
00548     CPAT<TQ,TD> &CP, double nrm, wbvector<unsigned> &xflag, char disp=0);
00549

```

```

00548 void ResetRec(unsigned k) {
00549     if (k>=QIDX.dim1) wblog(FL,
00550         "ERR %s() index out of bounds (%d/%d)",FCT,k,QIDX.dim1);
00551     isConsistent(FL);
00552
00553     QIDX.setRec(k,0); DATA[k]->init();
00554     for (unsigned j=0; j<CGR.dim2; ++j) { CGR(k,j).init(); }
00555 };
00556
00557 void setData(const TD c) {
00558     for (unsigned i=0; i<DATA.len; ++i) { DATA[i]->set(c); }
00559 };
00560
00561 void setRand(double nrm=0);
00562
00563 void markUnequalZero(const TD m=1, const TD eps=0) {
00564     for (unsigned i=0; i<DATA.len; ++i)
00565         DATA[i]->markUnequalZero(m,eps);
00566 };
00567
00568 void PrependSingletons(unsigned R);
00569
00570 void swap(QSpace &B) {
00571     if (isref || QIDX.isref || DATA.isref || CGR.isref) wblog(FL,
00572         "ERR must not save referenced data (isref=%d,%d,%d)",
00573         isref, QIDX.isref,DATA.isref);
00574     if (B.isref || B.QIDX.isref || B.DATA.isref || B.CGR.isref) wblog(FL,
00575         "ERR must not save referenced data (isref=%d,%d,%d)",
00576         B.isref, B.QIDX.isref, B.DATA.isref);
00577
00578     DATA .swap(B.DATA );
00579     QIDX .swap(B.QIDX ); SWAP(QDIM, B.QDIM );
00580     CGR .swap(B.CGR ); SWAP(otype,B.otype);
00581     qtype.swap(B.qtype);
00582     itags.swap(B.itags); SWAP(isref,B.isref); SWAP(mt,B.mt);
00583 };
00584
00585 QSpace& save2(QSpace &B) {
00586     if (isref || QIDX.isref || DATA.isref || CGR.isref) wblog(FL,
00587         "ERR must not save referenced data (isref=%d,%d,%d)",
00588         isref, QIDX.isref,DATA.isref);
00589     for (unsigned i=0; i<DATA.len; ++i) {
00590         if (DATA[i]->isRef()) wblog(FL,"%NERR using save2 with "
00591             "referenced data\n(data[%d] has isref=%d)",
00592             i,DATA[i]->isRef());
00593     };
00594     B.clearQSpace();
00595
00596     DATA .save2(B.DATA );
00597     QIDX .save2(B.QIDX ); B.QDIM=QDIM;
00598     CGR .save2(B.CGR );
00599
00600     qtype.save2(B.qtype);
00601     itags.save2(B.itags);
00602     B.otype=otype; otype=QS_NONE;
00603
00604     B.isref=isref; isref=0;
00605     B.ctime=ctime;
00606     B.mt=mt;
00607
00608     return B;
00609 };
00610
00611 unsigned Reduce2AbelianOp(const char *F=NULL, int L=0);
00612
00613 char allAbelian() const {
00614     return qtype.allAbelian();
00615 };
00616
00617 char isNonAbelian() const {
00618     return qtype.isNonAbelian();
00619 };
00620
00621 bool permitsOM() const {
00622     bool q=0;
00623     if (CGR || !qtype.allAbelian()) {
00624         q=qtype.permitsOM(rank(FL)); }
00625     return q;
00626 };
00627
00628 bool hasOM() const {
00629     bool om=0;
00630
00631     #ifndef WB_SKIP_ASSERT
00632     int q=QIDX.isSorted(), pm=permitsOM();
00633     if (! q ) wblog(FL,"WRN %s() got non-sorted QIDX",FCT);
00634

```



```

00635         if (!(q&2)) wblog(FL,"WRN %s() got non-unique QIDX "
00636             "%s @ r=%d; %d, %d)",FCT,STR(qtype),rank(),q,pm);
00637     #endif
00638
00639     unsigned i=0, n=CGR.numel();
00640     for (; i<n; ++i) { if (CGR[i].wnumel()>1) { om=1; break; } }
00641     return om;
00642 };
00643
00644 bool allAdditive() const {
00645     return qtype.allAdditive();
00646 };
00647
00648 bool isEmpty() const;
00649
00650 explicit operator bool() const { return !isEmpty(); }
00651 bool operator!() const { return isEmpty(); }
00652
00653 bool isConsistent(
00654     const char *F, int L, unsigned &r, char level=3) const;
00655
00656 bool isConsistent(
00657     const char *F=NULL, int L=0
00658 ) const { unsigned r=-1; return isConsistent(F,L,r,3); };
00659
00660 bool isConsistent(unsigned &r, char level=3) const {
00661     return isConsistent(NULL,0,r,level); };
00662
00663 bool isConsistent_r(unsigned r, char level=3) const {
00664     return isConsistent(NULL,0,r,level);
00665 };
00666
00667 bool isScalar() const {
00668     unsigned r=rank(FL);
00669     if (r || DATA.len!=1) { return 0; }
00670
00671     if (QIDX.dim1!=1 || DATA[0]->numel()!=1) {
00672         wblog(FL,"ERR %s() inconsistency (%d; S=%s)",
00673             FCT,QIDX.dim1,SSTR_(DATA[0]));
00674     }
00675     return 1;
00676 };
00677
00678 bool isComplex() const {
00679     unsigned i=0, n=DATA.len; isConsistent(FL);
00680     for (; i<n; ++i) { if (DATA[i]->isComplex()) return 1; }
00681     return 0;
00682 };
00683
00684 bool isDiagBlock(unsigned k) const;
00685
00686 bool isBlockDiagMatrix(const char *F, int L, char dflag=0) const;
00687 bool isBlockDiagMatrix(char dflag=0) const {
00688     return isBlockDiagMatrix(NULL,0,dflag); }
00689
00690 bool isDiagMatrix(const double eps=1e-14) const;
00691 bool isIdentityMatrix(const TD eps=1e-14) const;
00692 char hasIdentityCGS(const TD eps=1e-14) const;
00693
00694 bool isR3Op() const {
00695     return (rank(FL)==3 && otype==QS_OPERATOR);
00696 };
00697
00698 bool isOp(int r=-1, char lflag=0) const {
00699     return itags.isOp(r,lflag); };
00700
00701 int isOperator(unsigned *r_, char xflag=0) const;
00702
00703 bool isOperator() const { return (isOperator(NULL,0)>0 ? 1 : 0); };
00704
00705 bool isAtensor(unsigned r=-1) const { return itags.isAtensor(r); }
00706
00707 int checkAbelianOp() const;
00708 bool isAbelianOp() const { return (checkAbelianOp()>0); };
00709
00710 bool isQSym(const char *F=0, int L=0, char dflag=0) const;
00711
00712 bool isHConj(
00713     const char *F=0, int L=0, double eps=1e-14, char vflag=0
00714 ) const;
00715
00716 bool isAHerm(
00717     const char *F=0, int L=0, double eps=1e-14, char vflag=0
00718 ) const;
00719
00720 int isSingleton(unsigned k) const;
00721

```

```

00722     template <class T2>
00723     bool hasSameQ(const QSpace<TQ,T2> &B) const;
00724
00725     bool hasSameQ(
00726         unsigned r1, unsigned k1,
00727         unsigned r2, unsigned k2
00728     ) const {
00729         if (memcmp( Qref(r1,k1), Qref(r2,k2), QDIM*sizeof(TQ) )) return 0;
00730         if (DATA[r1]->dim0(k1) != DATA[r2]->dim0(k2)) {
00731             wblog(FI,
00732                 "WRN %s()=1, yet DATA size mismatch (%s <> %s; %d,%d)",
00733                 FCT, DATA[r1]->sizeStr().data,
00734                 DATA[r2]->sizeStr().data, k1+1, k2+1);
00735             return 0;
00736         }
00737         return 1;
00738     };
00739
00740     template <class T2>
00741     bool hasSameRank(const QSpace<TQ,T2> &B) const {
00742         return (QIDX.dim2==B.QIDX.dim2 && QDIM==B.QDIM);
00743     };
00744
00745     char checkQ(const char *F=NULL, int L=0) const;
00746
00747     template <class TB>
00748     void checkQ(
00749         const char *F, int L, const QSpace<TQ,TB> &B,
00750         char aflag=0
00751     ) const;
00752
00753     char checkQ_CGR(const char *F, int L, C_UVEC *I=NULL) const;
00754
00755     int check_CGR_cgw(const char *F=0, int L=0) const;
00756
00757     void checkScalarCGS(const char *F=0, int L=0) const;
00758
00759     template <class TB>
00760     bool sameType(
00761         const QSpace<TQ,TB> &B, int r=-1, const char *istr=NULL
00762     ) const;
00763
00764     QDir& getQDir(QDir &qdir) const;
00765     bool sameQDir(const char *F, int L, const QSpace &B) const;
00766
00767     bool hasQOverlap(const QSpace &A) const;
00768
00769     template <class TB>
00770     int hasQOverlap(
00771         unsigned ica, const QSpace<TQ,TB> &B, unsigned icb,
00772         const char type=0
00773     ) const;
00774
00775     int getQOverlapU(
00776         const char *F, int L, const QSpace &B,
00777         wbMatrix<TQ> &QQ, wbMatrix<int> &IQ,
00778         char olonly=0
00779     ) const;
00780
00781     int getQOverlap(
00782         const char *F, int L, const QSpace &B,
00783         wbMatrix<TQ> &QQ, wbMatrix<int> &IQ,
00784         wbvector<wdx_t> &DQ
00785     ) const;
00786
00787     bool findDimQ(const TQ* q, unsigned &d) const;
00788     bool findDimQ(const TQ* q, unsigned k, unsigned &d) const;
00789
00790     bool operator==(const QSpace &B) const {
00791         if (this!=&B) {
00792             if (! (QIDX==B.QIDX)) return 0;
00793             if (! (qtype==B.qtype)) return 0;
00794             if (! (CGR==B.CGR)) return 0;
00795             if (! DATA.deepEqualP(B.DATA)) return 0;
00796         }
00797         return 1;
00798     };
00799
00800     bool operator!=(const QSpace &B) const { return (!(this==B)); }
00801
00802     template<class T>
00803     void operator*= (const T fac);
00804
00805     template<class T>
00806     QSpace& times(const T fac, QSpace &B) const {
00807         B=*this; B*=fac; return B;
00808     };

```

```

00809
00810     QSpace& operator= (const QSpace &A) {
00811         if (this!=&A) { init(A); }
00812         return *this;
00813     };
00814
00815     void operator+= (const QSpace &Q);
00816     void operator-= (const QSpace &Q);
00817
00818     QSpace& Cat(const char *F, int L,
00819               const QSpace<TQ,TD> &A, const QSpace<TQ,TD> &B,
00820               TD afac=1, TD bfac=1, char uflag=1
00821     );
00822
00823     QSpace& plus(
00824         const QSpace &B, QSpace &C, TD bfac=1, char vflag=0) const;
00825
00826     QSpace& minus(
00827         const QSpace &B, QSpace &C, TD bfac=1, char vflag=0) const;
00828
00829     QSpace& Plus (const QSpace &A, TD bfac=1);
00830     QSpace& Minus(const QSpace &A, TD bfac=1);
00831
00832     QSpace& plus_plain(const QSpace &B, QSpace &C, TD bfac=1) const;
00833
00834     int NormCGW(char full=0, char skipzeros=1, char rcpy=0);
00835
00836     TD norm2(char checks=1) const;
00837     TD norm (char checks=1) const { return sqrt(norm2(checks)); }
00838
00839     TD normDiff2(const QSpace &B) const;
00840     TD scalarProd(const QSpace &B) const;
00841
00842     void checkNorm(
00843         const char *F, int L, double nrm, char vflag=1, double eps=1e-12
00844     ) const;
00845
00846     QSpace& TimesEl(const QSpace &B, char conj=0, unsigned r2=-1);
00847
00848     TD sumData() const;
00849
00850     template <class TM>
00851     wbMatrix<TM>& diag(wbMatrix<TM> &xd) const;
00852
00853     QSpace& TensorProd(const QSpace &B, QSpace &C,
00854         const char aflag='N', const char bflag='N') const;
00855
00856     template <class TC, ENABLE_IF_COMPLEX(TC) >
00857     QSpace<TQ,TC>& TensorProd(const QSpace &B, QSpace<TQ,TC> &C,
00858         const char aflag='N', const char bflag='N'
00859     ) const { QSpace<TQ,TD> C_;
00860         this->TensorProd(B,C_,aflag,bflag);
00861         return C.initT(C_);
00862     };
00863
00864     template <class TC, ENABLE_IF_COMPLEX(TC) >
00865     QSpace<TQ,TC>& TensorProd(const QSpace<TQ,TC> &B, QSpace<TQ,TC> &C,
00866         const char aflag='N', const char bflag='N'
00867     ) const { QSpace<TQ,TC> A; A.initT(*this);
00868         return A.TensorProd(B,C,aflag,bflag);
00869     };
00870
00871     void TensorProdUnity(const wbMatrix<TQ> &Q, QSpace &C) const;
00872
00873     void EigenSymmetric(wbMatrix<double> &Etot) const {
00874         QSpace Ak,Ad;
00875         QSpace<TQ,double> Ek,Et;
00876         wbMatrix<unsigned> DB; int Nkeep=-1;
00877
00878         EigenSymmetric(Ak,Ad,Ek,Et,Etot,DB,Nkeep);
00879     };
00880
00881     void EigenSymmetric(
00882         QSpace &Ak, QSpace<TQ,double> &Ek, wbMatrix<double> &Etot
00883     ) const {
00884         QSpace Ad;
00885         QSpace<TQ,double> Et;
00886         wbMatrix<unsigned> DB; int Nkeep=-1;
00887
00888         EigenSymmetric(Ak,Ad,Ek,Et,Etot,DB,Nkeep);
00889     };
00890
00891     void EigenSymmetric(QSpace &Ak, QSpace<TQ,double> &Ek) const {
00892         QSpace Ad;
00893         QSpace<TQ,double> Et;
00894         wbMatrix<unsigned> DB; int Nkeep=-1;
00895         wbMatrix<double> Etot;

```

```

00896
00897     EigenSymmetric(Ak,Ad,Ek,Et,Etot,DB,Nkeep);
00898 };
00899
00900 void EigenSymmetric(
00901     QSpace &Ak, QSpace &Ad,
00902     QSpace<TQ,double> &Ek, QSpace<TQ,double> &Et, wbMatrix<double> &Etot,
00903     wbMatrix<unsigned> &DD, int &Nkeep,
00904     double Etrunc=0, double *E0=NULL,
00905     char mKD=0,
00906     const wbperm &P=wbperm(), double eps=0., double b=0., int dmax=-1,
00907     const char *sort=NULL
00908 ) const;
00909
00910 void GroupIndices(unsigned K);
00911
00912 int ExpandOM(
00913     const char *F, int L, wbvector<unsigned> &iOM, char fflag=0);
00914
00915 int ExpandOM_(
00916     const char *F, int L, unsigned ia,
00917     QSpace<TQ,TD> &B, unsigned ib);
00918
00919 int ExpandOM_(const char *F, int L, unsigned ia, unsigned ib) {
00920     return ExpandOM_(F,L,ia,*this,ib);
00921 };
00922
00923 void Append(unsigned k);
00924 void Append(const wbvector<TQ> &, const wbarray<TD> &D);
00925
00926 QSpace& Append(const char *F, int L, const QSpace &A, char uflag=1);
00927
00928 int Append2AndDestroy(
00929     const char *F, int L, QSpace &A, char unique=0);
00930
00931 template <class TA, ENABLE_IF_COMPLEX(TA) >
00932 int Append2AndDestroy(
00933     const char *F, int L, QSpace<TQ,TA> &A, char unique=0
00934 ){
00935     QSpace<TQ,TA> X; X.initT(*this);
00936     return X.Append2AndDestroy(F,L,A,unique);
00937 };
00938
00939 void TransferSpace(const char *F, int L,
00940     QSpace &B, unsigned k,
00941     const QSpace<TQ,double> &W, double eps
00942 );
00943
00944 void TransferSpace(const char *F, int L,
00945     QSpace &B, unsigned k
00946 );
00947
00948 unsigned MakeUnique();
00949
00950 bool isZero(const double eps=0., char flag=0) const;
00951 unsigned SkipZeroData(
00952     double eps=0., char bflag=0, char cgflag=1,
00953     char all=0
00954 );
00955
00956 unsigned skipZeroOffDiag(double eps=0., char bflag=0);
00957
00958 unsigned SkipEmptyData(const char *F=0, int L=0);
00959
00960 bool gotZeroData(unsigned i, const double eps=0.) const;
00961
00962 void Set(unsigned k, const wbvector<TQ> &Q, const wbarray<TD> &D);
00963 void SetQ(unsigned i, unsigned k, const TQ *d);
00964
00965 QSpace& Select(const wbindex &I, char data_only=0);
00966
00967 template <class TM>
00968 QSpace& SelectMarked(const wbvector<TM> &mark){
00969     if (mark.len!=QIDX.dim1) wblog(FL,
00970         "ERR %s() size inconsistency (%d/%d)",FCT,mark.le,QIDX.dim1);
00971     wbindex I; mark.find(I);
00972     return Select(I);
00973 };
00974
00975 void Sort(const QSpace &A);
00976 void Sort() {
00977     wbperm P; QIDX.SortRecs(P); DATA.Select(P);
00978     if (qtype.allAbelian()) {
00979         if (!CGR.isEmpty()) wblog(FL,"ERR %s() "
00980             "invalid all-abelian CGR (%dx%d)",FCT,CGR.dim1,CGR.dim2);
00981     }
00982     else CGR.recPermute(P);

```

```

00983     };
00984
00985     unsigned getOM(
00986         wbMatrix<unsigned> &OM, wbvector<unsigned> *oml=NULL,
00987         char wflag=0) const;
00988
00989     unsigned cgsDimScalar(unsigned k) const;
00990     unsigned cgsDim(unsigned k, unsigned r) const;
00991     unsigned getDim(unsigned k, widx_t *D=NULL) const;
00992
00993     unsigned getDIM(unsigned k) const {
00994         widx_t D=0; getDim(k,&D);
00995         return D;
00996     };
00997
00998     void getDim(wbvector<widx_t> &D, wbvector<widx_t> *DD=NULL) const;
00999
01000     wbvector<widx_t> getDim() const {
01001         wbvector<widx_t> D; getDim(D); return D;
01002     }
01003
01004     void getQDim(unsigned k,
01005         wbMatrix<TQ> &Q,
01006         wbvector<widx_t>&S,
01007         wbMatrix<widx_t>*SC=NULL
01008     ) const;
01009
01010     void getQDim(wbMatrix<TQ> &Q,
01011         wbvector<widx_t> &S,
01012         wbMatrix<widx_t>*SC=NULL
01013     ) const;
01014
01015     void getDRange(TD &dmin, TD &dmax) const;
01016
01017     void getDistQtot(wbMatrix<TQ> &Qtot, wbvector<double> &w) const;
01018     widx_t getDQtot() {
01019         wbMatrix<TQ> Q; QIDX.blockSum(QDIM,Q); Q.makeUnique();
01020         return Q.dim1;
01021     };
01022
01023     wbMatrix<TQ>& getQsub(const wbindex &I, wbMatrix<TQ> &QI) const;
01024     wbMatrix<TQ>& getQsub(
01025         const unsigned k, wbMatrix<TQ> &Qk, char iflag=0) const;
01026
01027     void getQsub(const wbindex &I, wbMatrix<TQ> &QI, wbMatrix<TQ> &Q2) const;
01028
01029     wbMatrix<TQ> getQsub(const unsigned k) const {
01030         wbMatrix<TQ> Qk; getQsub(k,Qk); return Qk;
01031     };
01032
01033     wbMatrix<TQ>& getQtot(wbMatrix<TQ> &Qtot, char uflag=0) const;
01034     wbMatrix<TQ>& getQsum(const wbindex &J, wbMatrix<TQ> &Q) const;
01035     wbMatrix<TQ>& getQsum(const wbMatrix<TQ> &Q0, wbMatrix<TQ> &Q) const;
01036
01037     wbMatrix<TQ>& getQfinal(const wbindex &J, wbMatrix<TQ> &Q) const;
01038
01039     QSpace& getSub(const wbindex &I, QSpace &A, char ref=0) const;
01040
01041     QSpace& getSubInit(const wbindex &I, QSpace &A) const;
01042
01043     void saveSub2(QSpace &A, const wbindex &I);
01044
01045     void toFull(
01046         wbararray<TD> &A,
01047         wbvector< wbMatrix<TQ> > &Q,
01048         wbvector<wbvector<unsigned> > &S
01049     ) const;
01050
01051     void toFull2(
01052         wbararray<TD> &A,
01053         wbMatrix<TQ> &Q,
01054         wbMatrix<widx_t> &S
01055     ) const;
01056
01057     void ExtendQ(const QSpace &B);
01058
01059     void takeMainQS(char disp=1);
01060
01061     void setQ(unsigned k, const QSpace &B, unsigned k0);
01062     void setQRec(unsigned r, unsigned j, const TQ *q0, const TQ *q2=NULL);
01063
01064     bool hasEqualQ(unsigned i1, unsigned i2) const;
01065
01066     double maxDiff(const QSpace &B) const;
01067
01068     void PermuteQ(const wbperm &P);
01069     void permuteQ(const wbperm &P, wbMatrix<TQ>&) const;

```

```

01070
01071 QSpace& Permute(const wbperm &P, char iflag=0, char rcpy=0);
01072 QSpace& permute(const wbperm &P, QSpace &B, char iflag=0) const;
01073
01074 void permute (const char *pstr, QSpace &B, unsigned offset=1
01075 ) const {
01076     permute(wbperm(pstr,offset),B);
01077 };
01078
01079 void Permute(const char* pstr, unsigned offset=1) {
01080     Permute(wbperm(pstr,offset));
01081 };
01082
01083 void PermuteFirstTo (unsigned k) {
01084     unsigned r=rank(FL);
01085     if (k>=r) wblog(FL,"ERR %s() index out of range (%d,%d)",FCT,k,r);
01086     wbperm P2(r); P2.mvFirstTo(k); Permute(P2);
01087 };
01088 void permuteFirstTo (unsigned k, QSpace &C) {
01089     if (this==&C) { PermuteFirstTo(k); return; }
01090     unsigned r=rank(FL);
01091     if (k>=r) wblog(FL,"ERR %s() index out of range (%d,%d)",FCT,k,r);
01092     wbperm P2(r); P2.mvFirstTo(k); permute(P2,C);
01093 };
01094
01095 void PermuteLastTo (unsigned k) {
01096     unsigned r=rank(FL);
01097     if (k>=r) wblog(FL,"ERR %s() index out of range (%d,%d)",FCT,k,r);
01098     wbperm P2(r); P2.mvLastTo(k); Permute(P2);
01099 };
01100 void permuteLastTo (unsigned k, QSpace &C) {
01101     if (this==&C) { PermuteLastTo(k); return; }
01102     unsigned r=rank(FL);
01103     if (k>=r) wblog(FL,"ERR %s() index out of range (%d,%d)",FCT,k,r);
01104     wbperm P2(r); P2.mvLastTo(k); permute(P2,C);
01105 };
01106
01107 void FlipQ();
01108
01109 QSpace& trace(
01110     const char *F, int L, ctrIdx &i1, ctrIdx &i2, QSpace &C) const;
01111 TD trace() const;
01112
01113 void contractMat(
01114     unsigned ia, const QSpace &B, unsigned ib,
01115     QSpace &C
01116 ) const;
01117
01118 template <class TB, class TC>
01119 int contract_getIdxSet(const char *F, int L,
01120     const ctrIdx &ica, const QSpace<TQ,TB> &B, const ctrIdx &icb,
01121     wbindx &Ia, wbindx &Ib, wbvector<widx_t> &Dc,
01122     QSpace<TQ,TC> &C
01123 ) const;
01124
01125 template<class TB, class TC>
01126 double contract(
01127     const char *F, int L, const ctrIdx &ica,
01128     const QSpace<TQ,TB> &B, const ctrIdx &icb, QSpace<TQ,TC> &C,
01129     const wbperm &P=wbperm(), char preview=0
01130 ) const;
01131
01132 template<class TB, class TC>
01133 double contract(const char *F, int L,
01134     unsigned ia, const QSpace<TQ,TB> &B, unsigned ib,
01135     QSpace<TQ,TC> &C, const wbperm &P =wbperm(), char preview=0
01136 ) const {
01137
01138     unsigned ra=rank(FL), rb=B.rank(FL);
01139     if (!ia || (ia>ra && (QIDX.dim1 || QIDX.dim2))) wblog(FL,
01140         "ERR %s() index out of bounds (ia=%d/%d; %s)",
01141         FCT,ia,ra,SSTR(QIDX));
01142     if (!ib || (ib>rb && (B.QIDX.dim1 || B.QIDX.dim2))) wblog(FL,
01143         "ERR %s() index out of bounds (ib=%d/%d; %s)",
01144         FCT,ib,rb,SSTR(B.QIDX)
01145     );
01146
01147     ctrIdx ica(1,&(--ia)), icb(1,&(--ib));
01148     return contract(F,L,ica,B,icb,C,P,preview);
01149 };
01150
01151 template<class TB, class TC>
01152 int contract(const char *F, int L,
01153     unsigned ia,
01154     const QSpace<TQ,TB> &B, const char *sb,
01155     QSpace<TQ,TC> &C, const wbperm &P =wbperm(), char preview=0
01156 ) const {

```

```

01157
01158     ctrIdx ica(1,&(--ia)), icb;
01159
01160     if (isEmpty() || B.isEmpty()) { C.init(); return 1; }
01161     if (ia>=itags.len) {
01162         wblog(FL,"ERR %s() index out of bounds (ia=%d/%d; %s)",
01163             FCT,ia+1,itags.len,STR(itags));
01164     }
01165
01166     if (sb && isdigit(sb[0]))
01167         { icb.init(F_L,sb); }
01168     else { icb.initOp(FL,itags[ia],B,sb); }
01169
01170     return contract(F,L,ica,B,icb,C,P,preview);
01171 };
01172
01173 template<class TB, class TC>
01174 int contract(const char *F, int L,
01175     const char *sa,
01176     const QSpace<TQ,TB> &B, unsigned ib,
01177     QSpace<TQ,TC> &C, const wbperm &P =wbperm(), char preview=0
01178 ) const {
01179
01180     ctrIdx ica, icb(1,&(--ib));
01181
01182     if (isEmpty() || B.isEmpty()) { C.init(); return 1; }
01183
01184     if (ib>=B.itags.len) wblog(FL,
01185         "ERR %s() index out of bounds (ib=%d/%d; %s)",
01186         FCT,ib+1,B.itags.len,STR(B.itags));
01187
01188     if (sa && isdigit(sa[0]))
01189         { ica.init(F_L,sa); }
01190     else { ica.initOp(FL,B.itags[ib],*this,sa); }
01191
01192     return contract(F,L,ica,B,icb,C,P,preview);
01193 };
01194
01195 template<class TB, class TC> inline
01196 int contract(const char *F, int L,
01197     const char *sa, const QSpace<TQ,TB> &B, const char *sb,
01198     QSpace<TQ,TC> &C, const wbperm &P =wbperm(), char preview=0
01199 ) const {
01200
01201     ctrIdx ica(F_L,sa), icb(F_L,sb);
01202     return contract(F,L,ica,B,icb,C,P,preview);
01203 };
01204
01205 template<class TB, class TC>
01206 int contract(
01207     const char *sa, const QSpace<TQ,TB> &B, const char *sb,
01208     QSpace<TQ,TC> &C, const wbperm &P =wbperm(), char preview=0
01209 ) const { return contract(FL,sa,B,sb,C,P,preview); };
01210
01211 template<class TB, class TC>
01212 int contract(
01213     const ctrIdx &ica, const QSpace<TQ,TB> &B, const ctrIdx &icb,
01214     QSpace<TQ,TC> &C, const wbperm &P =wbperm(), char preview=0
01215 ) const { return contract(FL,ica,B,icb,C,P,preview); };
01216
01217 template<class TB, class TC>
01218 int contract(
01219     unsigned ia, const QSpace<TQ,TB> &B, unsigned ib,
01220     QSpace<TQ,TC> &C, const wbperm &P =wbperm(), char preview=0
01221 ) const { return contract(FL,ia,B,ib,C,P,preview); };
01222
01223 mxArray* toMx(bool setcls=0) const;
01224
01225 mxArray* save2Mx(char vflag=0);
01226
01227 mxArray* QIDX_toMx() const;
01228 mxArray* INFO_toMx() const;
01229 mxArray* DATA_toMx() const;
01230 mxArray* DATA_save2Mx(char vflag=0);
01231
01232 mxArray* mxCreateStruct(unsigned m, unsigned n) const;
01233 void add2MxStruct(mxArray *S, unsigned i, bool setcls=0, char chk=0) const;
01234
01235 void save2MxStruct(mxArray *S, unsigned i, char tst=0, char vflag=0);
01236
01237 mxArray* mxCreateCell(
01238     unsigned m QS_UNUSED_VAR,
01239     unsigned n QS_UNUSED_VAR) const { wblog(FL,"ERR %s()",FCT); return 0; };
01240 void add2MxCell(
01241     mxArray *S QS_UNUSED_VAR,
01242     unsigned i QS_UNUSED_VAR,
01243     char tst QS_UNUSED_VAR = 0) const { wblog(FL,"ERR %s()",FCT); };

```

```

01244
01245 void put (
01246     const char *F, int L,
01247     const char *vname, const char *ws="caller"
01248 ) const {
01249     wblog(F,L,"I/O putting '%s' to %s", vname, ws);
01250     put(vname,ws);
01251 };
01252
01253 void put (const char *vname, const char *ws="caller") const {
01254     mxArray *S=toMx();
01255     mxPutAndDestroy(0,0,S,vname,ws);
01256 };
01257
01258 wbststring sizeStrQ(const char *F=NULL, int L=0) const;
01259
01260 wbststring sizeStr(char vflag=0) const;
01261
01262 void print_SIZE(const char *F=NULL, int L=0) const;
01263 void print_rankDATA(const char *F=NULL, int L=0) const;
01264
01265 wbststring qStr() const { return qtype.toStr(); };
01266
01267 wbststring qrec2Str(unsigned k) const {
01268     return QIDX.rec2Str(k,"" " " QDIM,"");
01269 };
01270
01271 void disp_cgs(const char *F, int L, const char *istr) const;
01272
01273 void info(const char *vname=NULL,
01274     unsigned nlt=0, unsigned nlb=0,
01275     unsigned nind=2
01276 ) const;
01277
01278 void print() const { print("",3); };
01279 void print(const char *vname, char vflag=3) const;
01280
01281 void print_tst_(
01282     const char *F, int L, const char *istr, unsigned k=0) const {
01283
01284     if (k>=DATA.len) { wblog(FL,
01285         "WRN %s() index out of bounds (%d/%d)",FCT,k,DATA.len);
01286         return;
01287     }
01288     unsigned i=0, l=0, n=64; char s[n];
01289     const TD *dk=DATA[k]->data;
01290     l=snprintf(s,n,"%8s",NSTR(dk[0]));
01291     if (CGR.dim2) {
01292         for (; i<CGR.dim2; ++i) { if (l<n) {
01293             l+=snprintf(s+l,n-l," %c %4s",i?'*':'x',RATS(CGR(k,i).wel(0))); }
01294         }
01295         wblog(F_L,"TST %s @%2d/%d: %-32s %s",istr,k+1,DATA.len,s,STR(itags));
01296     };
01297
01298 void initOType(const char *F, int L, const mxArray* a);
01299
01300 wbststring otype2Str(const char *fmt="%s") const;
01301
01302 int ConjOpScalar() {
01303     int e=0;
01304     unsigned r=rank(FL);
01305     if (r!=3) {
01306         if (r!=2) { wblog(FL,"WRN %s() got rank-%d QSpace !?",FCT,r); }
01307         return e;
01308     }
01309     for (unsigned n=CGR.numel(), i=0; i<n; ++i) {
01310         if ((e=CGR[i].HConjOpScalar())) { return e; }
01311     }
01312
01313     if (itags.len!=3) wblog(FL,
01314         "ERR %s() got itags.len=%d/3 !?",FCT,itags.len);
01315     itags[2].Conj();
01316
01317     return e;
01318 };
01319
01320 QSpace& SortDegQ(const char *F=0, int L=0) {
01321     for (unsigned n=CGR.numel(), i=0; i<n; ++i) {
01322         CGR.data[i].SortDegQ(F_L); }
01323     return *this;
01324 };
01325
01326 QSpace& Conj() {
01327     SetConjTags();
01328     return DConj().RConj();
01329 };
01330

```



```

01331     QSpace& DConj() {
01332         if (WbUtil<TD>::isComplex()) {
01333             for (unsigned n=DATA.len, i=0; i<n; ++i) { DATA[i]->Conj(); }
01334         }; return *this;
01335     };
01336
01337     QSpace& RConj() {
01338         if (CGR) {
01339             unsigned i=0, n=CGR.numel();
01340             for (; i<n; ++i) { CGR[i].Conj(); }
01341             SortDegQ(FL);
01342         }
01343         return *this;
01344     };
01345
01346     QSpace& HConj() {
01347         QSpace B; this->save2(B);
01348         return B.hconj(*this);
01349     };
01350
01351     QSpace& hconj(QSpace &B) const {
01352         if (!isEmpty()) {
01353             if (!QDIM || QIDX.dim2>QDIM) wblog(FL,
01354                 "ERR %s() QSpace inconsistency (%dx%d/%d)",
01355                 FCT, QIDX.dim1, QIDX.dim2, QDIM);
01356             unsigned r=QIDX.dim2/QDIM;
01357             if (r%2 && (r!=3 || otype!=QS_OPERATOR)) wblog(FL, "ERR %s() "
01358                 "requires even-rank object or operator (%dx%d/%d; %s)",
01359                 FCT, QIDX.dim1, QIDX.dim2, QDIM, QS_STR[otype]);
01360             transp(B); B.Conj(); B.otype=otype;
01361             return B;
01362         }
01363         return B.clearQSpace();
01364     };
01365
01366     QSpace& transp(QSpace &B) const {
01367
01368         if (!isEmpty()) { unsigned r=rank(FL);
01369             if (r%2==0) { wbperm P2;
01370                 P2.initTranspose(r); permute(P2,B);
01371             }
01372             else if (r==3 && otype==QS_OPERATOR) {
01373                 permute("2,1,3", B);
01374             }
01375             else wblog(FL, "ERR %s got rank-%d object", FCT, r);
01376         }
01377         else B.clearQSpace();
01378
01379         return B;
01380     };
01381
01382     const QSpace& opA(char tflag, QSpace& X) const {
01383         if (!strchr("NTC", tflag))
01384             wblog(FL, "ERR %s() invalid flag %c<%d>", FCT, tflag, tflag);
01385
01386         if (tflag=='N' || isEmpty()) return (*this);
01387         else {
01388             wbperm P2("2 1"); unsigned r=rank(FL);
01389
01390             if (r!=2) wblog(FL,
01391                 "ERR %s() only applicable to rank-2 tensors (%d)", FCT, r);
01392
01393             permute(P2,X); if (tflag=='C') X.Conj();
01394             return X;
01395         }
01396     };
01397
01398     wbMatrix<TQ> QIDX;
01399
01400     wbvector<wbarray<TD>*> DATA;
01401     wbMatrix< CRef<TQ> > CGR;
01402
01403     QVec qtype;
01404
01405     unsigned QDIM;
01406
01407     QS_TYPES otype;
01408     iTags itags;
01409
01410     Wb::MTYPE mt;
01411
01412     char isref;
01413
01414     double ctime;
01415
01416 protected:
01417 private:

```

```

01418
01419     bool isSym_aux(
01420         const char *F, int L, const char *fct,
01421         RTD eps=1E-14, char symflag='s', char vflag=0
01422     ) const;
01423
01424     void recSave2(unsigned k, unsigned i);
01425
01426 };
01427
01428 template <class TD>
01429 class IterOM_DATA {
01430
01431     public:
01432
01433         template <class TQ>
01434         IterOM_DATA(const QSpace<TQ,TD> &A, unsigned i)
01435             : iter(0), niter(1), nS(0), data0(NULL) {
01436
01437             unsigned r, rA=A.rank(FL);
01438             if (i>=A.DATA.len) wblog(FL,
01439                 "ERR %s() index out of bounds (%d/%d)",FCT,i,A.DATA.len);
01440             data0=A.DATA[i]->data;
01441             X.SIZE.init2ref(A.DATA[i]->SIZE); r=X.SIZE.len;
01442             if (!rA || r<rA || r>rA+1) wblog(FL,
01443                 "ERR %s() rank inconsistency (r=%d/%d)",FCT,r,rA);
01444
01445             if (r>rA) { niter=X.SIZE[rA]; --X.SIZE.len; }
01446             nS=X.SIZE.prod();
01447         };
01448
01449         void begin() { iter=0; }
01450         bool end() { return iter>=niter; }
01451
01452         IterOM_DATA& operator++() { ++iter; return *this; };
01453
01454         wbararray<TD> DATA() {
01455             if (iter>=niter) wblog(FL,
01456                 "ERR %s() index out of bounds (%d/%d)",FCT,iter,niter);
01457             return X.init2ref(data0+iter*nS);
01458         };
01459
01460         unsigned iter, niter;
01461         size_t nS;
01462         TD *data0;
01463         wbararray<TD> X;
01464
01465     protected:
01466     private:
01467 };
01468
01469 template <class TQ, class TD>
01470 template <class T> inline
01471 void QSpace<TQ,TD>::operator*= (const T fac) {
01472     if (fac==T(1)) return;
01473     for (unsigned i=0; i<DATA.len; ++i)
01474         (*DATA[i]) *= fac;
01475 };
01476
01477 template <class TQ, class TD> inline
01478 void QSpace<TQ,TD>::operator+= (const QSpace &B) {
01479     QSpace<TQ,TD> X;
01480     this->plus(B,X); X.save2(*this);
01481 };
01482
01483 template <class TQ, class TD> inline
01484 void QSpace<TQ,TD>::operator-= (const QSpace &B) {
01485     QSpace<TQ,TD> X;
01486     this->plus(B,X,-1); X.save2(*this);
01487 };
01488
01489 template <class TQ, class TD> inline
01490 QSpace<TQ,TD>& QSpace<TQ,TD>::Plus(const QSpace &B, TD bfac) {
01491     QSpace<TQ,TD> X;
01492     this->plus(B,X,bfac); return X.save2(*this);
01493 };
01494
01495 template <class TQ, class TD> inline
01496 QSpace<TQ,TD>& QSpace<TQ,TD>::Minus(const QSpace &B, TD bfac) {
01497     QSpace<TQ,TD> X;
01498     this->plus(B,X,-bfac); return X.save2(*this);
01499 };
01500
01501 template <class TQ, class TD> inline
01502 QSpace<TQ,TD>& QSpace<TQ,TD>::minus(
01503     const QSpace &B, QSpace &C, TD bfac, char vflag
01504 ) const {

```

```

01505     plus(B, C, -bfac, vflag); return C;
01506 }
01507
01508 template <class TQ, class TD>
01509 QSpace<TQ,TD>& QSpace<TQ,TD>::plus(
01510     const QSpace &B, QSpace &C, TD bfac, char vflag
01511 ) const {
01512
01513     if (B.isEmpty() || !bfac) { C=(*this); } else
01514     if (this->isEmpty()) { C=B; if (bfac!=1.) { C*=bfac; }} else
01515     if (gotCGS(FL)<2) { plus_plain(B,C,bfac); }
01516     else {
01517         C=B; if (bfac!=1.) { C*=bfac; }
01518         C.Append(FL,*this, vflag<1 ? 'q':vflag);
01519     }
01520
01521     return C;
01522 };
01523
01524 template <class TQ, class TD>
01525 TD QSpace<TQ,TD>::scalarProd(const QSpace<TQ,TD> &B) const {
01526
01527     unsigned i; int i1,i2;
01528     wbMatrix<int> IQ;
01529     wbMatrix<TQ> QQ;
01530     TD x=0;
01531
01532     if (this==&B) return norm2();
01533
01534     #ifdef WB_CLOCK
01535     Wb::Clock clk("QS:scalarProd",1);
01536     #endif
01537
01538     if (isEmpty() || B.isEmpty()) {
01539         wblog(FL,"WRN Overlap with empty object (%d,%d) !?",
01540             isEmpty(), B.isEmpty()); return x;
01541     }
01542
01543     getQOverlapU(FL,B,QQ,IQ,1);
01544
01545     for (i=0; i<IQ.dim1; ++i) {
01546         i1=IQ(i,0); i2=IQ(i,1);
01547         if (i1<0 || i2<0) wblog(FL, "ERR i1=%d, i2=%d !?", i1, i2);
01548
01549         x+=(DATA[i1]->scalarProd(*B.DATA[i2]));
01550     }
01551
01552     return x;
01553 };
01554
01555 template <class TQ, class TD>
01556 QSpace<TQ,TD>& QSpace<TQ,TD>::TensorProd(
01557     const QSpace<TQ,TD> &B, QSpace<TQ,TD> &C,
01558     const char aflag, const char bflag
01559 ) const {
01560
01561     if (isEmpty() || B.isEmpty()) { C.clearQSpace(); return C; }
01562
01563     if (gotCGS(FL)>0 || B.gotCGS(FL)>0) wblog(FL,
01564         "ERR %s() not implemented for non-abelian symmetries (got %s)",
01565         FCT,STR(qtype));
01566
01567     if (aflag!='N' || bflag!='N') {
01568         QSpace<TQ,TD> AX, BX;
01569         const QSpace<TQ,TD> *Ap=this, *Bp=&B;
01570
01571         if (!strchr("NTC",aflag) || rank(FL)!=2) wblog(FL,
01572             "ERR invalid aflag=%c<%d> (rank=%d)",aflag,aflag,rank(FL));
01573         if (!strchr("NTC",bflag) || B.rank(FL)!=2) wblog(FL,
01574             "ERR invalid bflag=%c<%d> (rank=%d)",bflag,bflag,B.rank(FL));
01575
01576         if (aflag!='N') {
01577             permute("2,1",AX); if (aflag=='C') AX.Conj();
01578             Ap=&AX;
01579         }
01580         if (bflag!='N') {
01581             B.permute("2,1",BX); if (bflag=='C') BX.Conj();
01582             Bp=&BX;
01583         }
01584
01585         return Ap->TensorProd(*Bp,C,'N','N');
01586     }
01587
01588     unsigned i,ia,ib,l,r;
01589     const unsigned n1=QIDX.dim1, n2=B.QIDX.dim1, R=rank(FL), s=QDIM*sizeof(TQ);
01590     const TQ *qa, *qb; TQ *q;
01591

```

```

01592     if (R!=B.rank(FL) || QDIM!=B.QDIM) wblog(FL,
01593         "ERR %s() size mismatch (%d,%d)",FCT,R,B.rank(FL));
01594
01595     C.clearQSpace();
01596     C.QIDX.init(n1*n2, 2*QIDX.dim2); C.QDIM=QDIM; C.setupDATA();
01597
01598     C.initQ(FL,*this,'f',&B);
01599
01600     if (itags.len || B.itags.len) { C.itags.init(2*R);
01601         if (!B.itags.len) {
01602             if (itags.len!=R) wblog(FL,
01603                 "ERR %s() invalid itag %s",FCT, STR(itags));
01604             for (i=0; i<R; ++i) {
01605                 C.itags[2*i]=C.itags[2*i+1]=itags[i];
01606             }
01607         }
01608         else if (!itags.len) {
01609             if (B.itags.len!=R) wblog(FL,
01610                 "ERR %s() invalid itag %s",FCT, STR(B.itags));
01611             for (i=0; i<R; ++i) {
01612                 C.itags[2*i]=C.itags[2*i+1]=B.itags[i];
01613             }
01614         }
01615         else {
01616             if (itags.len!=B.itags.len || itags.len!=R || !sameQDir(FL,B))
01617                 wblog(FL,"ERR %s() itag/qdir mismatch (%s <> %s)",FCT,
01618                     STR(itags),STR(B.itags));
01619             for (i=0; i<R; ++i) {
01620                 C.itags[2*i] = itags[i];
01621                 C.itags[2*i+1]=B.itags[i];
01622             }
01623         }
01624     }
01625
01626     for (i=ib=0; ib<n2; ++ib) {
01627         qb=B.QIDX.rec(ib);
01628
01629         for (ia=0; ia<n1; ++ia, ++i) {
01630             qa=QIDX.rec(ia); q=C.QIDX.rec(i);
01631
01632             for (l=r=0; r<R; ++r, l+=QDIM) {
01633                 memcpy(q,qa+l,s); q+=QDIM;
01634                 memcpy(q,qb+l,s); q+=QDIM;
01635             }
01636             DATA[ia]->tensorProd(*B.DATA[ib], *C.DATA[i]);
01637         }
01638     }
01639
01640     return C;
01641 };
01642
01643 template <class TQ, class TD> inline
01644 void QSpace<TQ,TD>::GroupIndices(unsigned K) {
01645     unsigned i, r=-1;
01646
01647     if (!isConsistent(r)) wbdie(FL,str);
01648
01649     if (!K || unsigned(r)%K) wblog(FL,
01650         "ERR %s() cannot block QSpace (%d/%d)",FCT,r,K);
01651
01652     QDIM=K*QDIM;
01653
01654     for (i=0; i<DATA.len; ++i)
01655         DATA[i]->GroupIndices(K);
01656 }
01657
01658 template <class TQ, class TD>
01659 size_t QSpace<TQ,TD>::getDataSize(wbMatrix<size_t> &S) const {
01660     size_t s=0; unsigned i,m,n=DATA.len;
01661
01662     if (n!=QIDX.dim1) wblog(FL,
01663         "ERR size mismatch (%d/%d)",QIDX.dim1,DATA.len);
01664
01665     if (isEmpty()) { S.init(); return s; }
01666
01667     m=DATA[0]->SIZE.len; S.init(n,m);
01668
01669     for (i=0; i<n; ++i) {
01670         const wbvector<size_t> &si=DATA[i]->SIZE;
01671         if (si.len!=m) wblog(FL,"ERR size mismatch (%d/%d)",si.len,m);
01672         S.recSetP(i,si.data);
01673         s+=si.prod(0);
01674     }
01675     return s;
01676 };
01677 };
01678

```

```

01679 template <class TQ, class TD> inline
01680 size_t QSpace<TQ,TD>::getDataSize(wbvector<size_t> &S) const {
01681     size_t s=0; S.init(DATA.len);
01682     for (unsigned i=0; i<DATA.len; ++i) { s+=(S[i]=DATA[i]->SIZE.prod()); }
01683     return s;
01684 };
01685
01686 template <class TQ, class TD> inline
01687 size_t QSpace<TQ,TD>::getDataSize() const {
01688     size_t s=0;
01689     for (unsigned i=0; i<DATA.len; ++i) s+=DATA[i]->SIZE.prod();
01690     return s;
01691 };
01692
01693 template <class TQ, class TD> inline
01694 size_t QSpace<TQ,TD>::getCGSSize() const {
01695     size_t s=0;
01696     if (gotCGS(FL)>0) {
01697         unsigned i,j;
01698         for (i=0; i<CGR.dim1; ++i) {
01699             for (j=0; j<CGR.dim2; ++j) s+=CGR(i,j).numel(); }
01700     }
01701     return s;
01702 };
01703
01704 template <class TQ, class TD> inline
01705 wbsting QSpace<TQ,TD>::totSize2Str() const {
01706
01707     unsigned long ss[3]={
01708         sizeof(TD)*getDataSize(),
01709         sizeof(TQ)*(QIDX.dim1+QIDX.dim2),
01710         sizeof(double)*getCGSSize()
01711     };
01712     int n=32; char s_[n];
01713     Wb::memsize2Str(ss[0]+ss[1]+ss[2],s_,n);
01714     return s_;
01715 };
01716
01717 template <class TQ, class TD> inline
01718 bool QSpace<TQ,TD>::isZero(const double eps, char flag) const {
01719
01720     for (unsigned i=0; i<DATA.len; ++i)
01721         if (!(DATA[i]->isZero(eps,flag))) return 0;
01722
01723     return 1;
01724 }
01725
01726 template <class TQ, class TD> inline
01727 void QSpace<TQ,TD>::Set(
01728     unsigned k, const wbvector<TQ> &Q, const wbarray<TD> &D
01729 ){
01730
01731     QIDX.recSet(k,Q);
01732
01733     if (k>=DATA.len)
01734         wblog(FL,"ERR %s() index out of bounds (%d/%d)",FCT,k,DATA.len);
01735
01736     DATA[k]->init(D);
01737 }
01738
01739 template <class TQ, class TD> inline
01740 void QSpace<TQ,TD>::SetQ(unsigned i, unsigned k, const TQ *d){
01741
01742     if (i>=QIDX.dim1 || k>=rank(FL)) wblog(FL,
01743         "ERR index out of bounds (%d/%d; %d;%d)",i,QIDX.dim1,k,rank(FL));
01744
01745     memcpy(QIDX.data+QIDX.dim2*i+QDIM*k, d, QDIM*sizeof(TQ));
01746 }
01747
01748 template <class TQ, class TD> inline
01749 void QSpace<TQ,TD>::Sort(const QSpace<TQ,TD> &R) {
01750     int e;
01751     wbindex P,Ir; wbperm Pr;
01752
01753     if (QIDX.dim1==0) return;
01754
01755     e=matchIndex(QIDX,R.QIDX, P, Ir);
01756
01757     if (e || P.len!=QIDX.dim1) wblog(FL,
01758         "ERR QSpace R not complete to be used as sorting reference "
01759         "%d/%d",P.len, QIDX.dim1);
01760
01761     Ir.Sort(Pr); P.Select(Pr);
01762
01763     QIDX.recPermute(P); if (!CGR.isEmpty())
01764         CGR .recPermute(P);
01765     DATA.Select(P);

```

```

01766 };
01767
01768 template <class TQ, class TD> inline
01769 int QSpace<TQ,TD>::isSingleton(unsigned k) const {
01770     unsigned i;
01771
01772     if (k>=rank(FL)) wblog(FL,
01773         "ERR %s() index out of bounds (%d,%d)",FCT,k,rank(FL));
01774
01775     for (i=0; i<DATA.len; ++i)
01776         if (DATA[i]->SIZE[k]!=1) break;
01777
01778     return (i==DATA.len);
01779 }
01780
01781 template <class TQ, class TD> inline
01782 void QSpace<TQ,TD>::getDRange(TD &dmin, TD &dmax) const {
01783     unsigned i,n=DATA.len;
01784
01785     if (!isConsistent()) { info("this"); wbdie(FL,str); }
01786
01787     for (i=0; i<n; ++i) if (DATA[i] && !DATA[i]->isEmpty()) {
01788         dmin=dmax=DATA[i]->data[0];
01789         break;
01790     }
01791
01792     for (i=0; i<n; ++i) if (DATA[i] && !DATA[i]->isEmpty()) {
01793         dmin=MIN(dmin, DATA[i]->min());
01794         dmax=MAX(dmax, DATA[i]->max());
01795     }
01796 }
01797
01798 template <class TQ, class TD> inline
01799 QSpace<TQ,TD>& QSpace<TQ,TD>::getSub(
01800     const windex &I, QSpace<TQ,TD> &A, char ref) const {
01801
01802     if ((void*)this==(void*)&A) { QSpace<TQ,TD> X;
01803         if ( ref) wblog(FL,"ERR %s() requesting ref=%s on self",FCT,cSTR(ref));
01804         if (isref) wblog(FL,"ERR %s() got ref QSpace",FCT);
01805         return getSub(I,X).save2(A);
01806     }
01807
01808     unsigned i,j;
01809     char cgflag=gotCGS(FL);
01810
01811     A.init();
01812     QIDX.getRecs(I,A.QIDX); A.QDIM=QDIM; A.qtype=qtype; A.itags=itags;
01813
01814     A.isref=(ref ? 1 : 0);
01815
01816     A.setupDATA();
01817
01818     if (cgflag>0) {
01819         if (CGR.dim1!=DATA.len) wblog(FL,
01820             "ERR %s() size mismatch (CGR: %d/%dx%d)",
01821             FCT,DATA.len,CGR.dim1,CGR.dim2);
01822         A.setupCGR();
01823     }
01824
01825     for (i=0; i<I.len; ++i) {
01826         if (I[i]>=DATA.len) wblog(FL,
01827             "ERR %s() index out of bounds (%d,%d)",FCT,I[i],DATA.len);
01828
01829         if (A.isref)
01830             { A.DATA[i]=DATA[I[i]]; }
01831         else { A.DATA[i]->init(*DATA[I[i]]); }
01832
01833         if (cgflag>0) {
01834             for (j=0; j<CGR.dim2; ++j) {
01835                 A.CGR(i,j)=CGR(I[i],j);
01836             }
01837         }
01838     }
01839
01840     return A;
01841 };
01842
01843 template <class TQ, class TD> inline
01844 QSpace<TQ,TD>& QSpace<TQ,TD>::getSubInit(
01845     const windex &I,
01846     QSpace<TQ,TD> &A
01847 ) const {
01848
01849     if (this==&A) {
01850         QSpace<TQ,TD> X; A.save2(X);
01851         return X.getSubInit(I,A);
01852     }

```

```

01853
01854     unsigned i,j;
01855     char cgflag=gotCGS(FL);
01856
01857     A.init();
01858     QIDX.getRecs(I,A.QIDX); A.QDIM=QDIM; A.qtype=qtype;
01859     A.setupDATA();
01860
01861     if (cgflag) { A.setupCGR();
01862
01863         for (i=0; i<I.len; ++i) {
01864             if (I[i]>=QIDX.dim1) wblog(FL,
01865                 "ERR %s() index out of bounds (%d/%d)",FCT,I[i]+1,QIDX.dim1);
01866             for (j=0; j<CGR.dim2; ++j) A.CGR(i,j).init(CGR(I[i],j));
01867         }
01868     }
01869
01870     return A;
01871 };
01872
01873 template <class TQ, class TD> inline
01874 void QSpace<TQ,TD>::saveSub2(QSpace<TQ,TD> &A, const wbindex &I) {
01875
01876     if (this==&A) wblog(FL,
01877         "ERR %s() got overlapping input and output space!",FCT);
01878     if (isref) wblog(FL,"ERR %s() called for ISREF!",FCT);
01879
01880     if (!I.len) { A.init(); return; }
01881
01882     unsigned i,j;
01883     char cgflag=gotCGS(FL);
01884
01885     A.init();
01886     QIDX.getRecs(I,A.QIDX); A.QDIM=QDIM; A.qtype=qtype;
01887     A.setupDATA();
01888     A.setupCGR();
01889
01890     for (i=0; i<I.len; ++i) {
01891         if (I[i]>=DATA.len) wblog(FL,
01892             "ERR %s() index out of bounds (%d,%d)",FCT,I[i],DATA.len);
01893
01894         DATA[I[i]]->save2(*A.DATA[i]);
01895         WB_DELETE_1(DATA[I[i]]);
01896
01897         if (cgflag) {
01898             for (j=0; j<CGR.dim2; ++j) {
01899                 CGR(I[i],j).save2(A.CGR(i,j)); }
01900         }
01901     }
01902
01903     wbindex I2; I.invert(QIDX.dim1,I2);
01904
01905     QIDX.Set2Recs(I2);
01906     DATA.Select(I2); if (cgflag) { CGR.Set2Recs(I2); }
01907 };
01908
01909 template <class TQ, class TD>
01910 void QSpace<TQ,TD>::toFull(
01911     wbarray<TD> &A,
01912     wbvector< wbMatrix<TQ> > &QB,
01913     wbvector< wbvector<unsigned> > &SB
01914 ) const {
01915
01916     if (gotCGS(FL)>0 || gotCGX()) wblog(FL,
01917         "ERR %s() not implemented yet for %d %d",FCT,
01918         gotCGS(FL), gotCGX(FL)
01919     );
01920
01921     widx_t i,j,k,M=QIDX.dim1; unsigned r=-1;
01922     wbvector<widx_t> N,SA,D;
01923     wbarray<unsigned> mark;
01924     wbMatrix<widx_t> G;
01925     wbindex I,J;
01926     wbperm P,iP;
01927
01928     if (!isConsistent(r)) { wblog(FL,"%s",str); }
01929     if (isEmpty()) {
01930         A.init(); QB.init(); SB.init();
01931         return;
01932     }
01933
01934     QB.initDef(r); SB.initDef(r); D.init(r);
01935     I.init(r); P.init(r); iP.init(r); G.init(M,r); N.init(r);
01936
01937     for (k=0; k<r; ++k) {
01938         getQsub(k,QB[k]).groupRecs(P,D); P.invert(iP);
01939     }

```

```

01940     I.BlockIndex(D).Permute(iP);
01941     G.setCol(k, I.data);
01942
01943     N[k]=D.len; SB[k].init(N[k]);
01944 }
01945
01946 mark.init(N+1);
01947
01948 for (i=0; i<M; ++i) {
01949     G.getRec(i, I);
01950
01951     if (mark(I)++) wblog(FL,
01952         "ERR %s() block (%s) not unique (%d)",
01953         FCT, (I+1).toStrf(" ", ", ").data, mark(I)
01954     );
01955
01956     J.wbvector<widx_t>::operator=(N);
01957
01958     for (k=0; k<r; ++k) {
01959         wbvector<unsigned> &SBk=SB[k];
01960
01961         J[k]=j=I[k]; if (k) J[k-1]=N[k-1];
01962         if (!mark(J)++) {
01963             SBk[j]=DATA[i]->SIZE[k];
01964         }
01965         else if (SBk[j]!=DATA[i]->SIZE[k]) wblog(FL,
01966             "ERR %s - block size mismatch (%d: %d,%d)",
01967             FCT, i+1, SBk[j], DATA[i]->SIZE[k]
01968         );
01969     }
01970 }
01971
01972 SA.init(r); for (k=0; k<r; ++k) SA[k]=SB[k].sum();
01973 A.init(SA);
01974
01975 for (i=0; i<DATA.len; ++i) {
01976     G.getRec(i, I);
01977     A.setBlock(SB, I, *DATA[i]);
01978 }
01979 }
01980
01981 template <class TQ, class TD>
01982 void QSpace<TQ, TD>::toFull2(
01983     wbararray<TD> &A,
01984     wbMatrix<TQ> &QB,
01985     wbMatrix<widx_t> &SB
01986 ) const {
01987
01988     if (gotCGS(FL)>0 || gotCGX()) wblog(FL,
01989         "ERR %s() not implemented yet for %d %d", FCT,
01990         gotCGS(FL), gotCGX(FL)
01991     );
01992
01993     unsigned i, j, k, r, K, N, M=QIDX.dim1, r_=-1;
01994     wbMatrix<TQ> Q1, Q2;
01995     wbvector<wbvector<widx_t> > S2;
01996     wbvector<widx_t> s, D, SA;
01997     wbMatrix<unsigned> mark; wbMatrix<widx_t> IJ;
01998     wbindex I, I1, I2;
01999     wbararray<TD> a;
02000     wbperm P, iP;
02001
02002     if (!isConsistent(r_)) wblog(FL, "%s", str);
02003     if (isEmpty()) {
02004         A.init(); QB.init(); SB.init();
02005         return;
02006     }
02007
02008     if (r_==3 && getDim(2)==1) { --r_; }
02009
02010     if (r_%2) wblog(FL, "ERR %s even-rank object required (r=%d)", FCT, r_);
02011     r=unsigned(r_);
02012     K=r/2;
02013
02014     I.Index(0, K-1); getQsub(I, Q1);
02015     I.Index(K, r-1); getQsub(I, Q2);
02016
02017     QB.Cat(1, Q1, Q2);
02018
02019     QB.groupRecs(P, D); P.invert(iP);
02020
02021     I.BlockIndex(D); I.Select(iP);
02022
02023     I1.init(M, I.data);
02024     I2.init(M, I.data+M);
02025
02026     N=D.len;

```



```

02027     mark.init(N,N+1);
02028     SB.init(N,K);
02029     IJ.init(M,2);
02030
02031     for (k=0; k<M; ++k) {
02032         i=IJ(k,0)=I1[k]; j=IJ(k,1)=I2[k];
02033
02034         if (mark(i,j)++) wblog(FL,
02035             "WRN %s - block (%d,%d) not unique (%d)", FCT, i,j, mark(i,j));
02036
02037         s.init2ref(K, DATA[k]->SIZE.data);
02038         if (mark(i,N)++) {
02039             if (!SB.recEqual(i,s.data)) wblog(FL,
02040                 "ERR %s - block size mismatch (%d: %s != %s)",FCT,k+1,
02041                 SB.rec2Str(i,"","x").data, SSTR(s));
02042         }
02043         else SB.recSet(i,s);
02044
02045         s.init2ref(K, DATA[k]->SIZE.data+K);
02046         if (mark(j,N)++) {
02047             if (!SB.recEqual(j,s.data)) wblog(FL,
02048                 "ERR %s - block size mismatch (%d: %s != %s)",FCT,k+1,
02049                 SB.rec2Str(j,"","x").data, SSTR(s));
02050         }
02051         else SB.recSet(j,s);
02052     }
02053
02054     S2.init(2); SB.recProd(S2[0]); S2[1]=S2[0];
02055     SA.init(2); SA[0]=SA[1]=S2[0].sum();
02056
02057     A.init(SA);
02058
02059     for (i=0; i<DATA.len; ++i) {
02060         I.init2ref(2,IJ.rec(i));
02061
02062         a.init2ref(*DATA[i]);
02063         a.GroupIndizes(K);
02064
02065         A.setBlock(S2,I,a);
02066     }
02067 }
02068
02069 template <class TQ, class TD>
02070 void QSpace<TQ,TD>::ExtendQ(const QSpace<TQ,TD> &B) {
02071
02072     unsigned i,k,m,n=B.QIDX.dim1;
02073     wbvector<char> mark;
02074     windex Ia,Ib;
02075
02076     if (B.isEmpty()) return;
02077     if (isEmpty()) {
02078         (*this)=B;
02079         for (i=0; i<DATA.len; ++i) DATA[i]->set(TD(0));
02080         return;
02081     }
02082
02083     if (QIDX.dim2!=B.QIDX.dim2) wblog(FL,
02084         "ERR %s - size mismatch (%d/%d)", FCT, QIDX.dim2, B.QIDX.dim2);
02085
02086     matchIndex(QIDX,B.QIDX,Ia,Ib);
02087
02088     mark.init(n);
02089     for (i=0; i<Ib.len; ++i) mark[Ib[i]]++;
02090
02091     m=mark.count(0);
02092     if (m==0) return;
02093
02094     k=QIDX.dim1;
02095     Append(m);
02096
02097     for (i=0; i<n; ++i) {
02098         if (mark[i]) continue;
02099         QIDX.recSetP(k,B.QIDX.rec(i));
02100         DATA[k]->init(B.DATA[i]->SIZE);
02101         ++k;
02102     }
02103 };
02104
02105 template <class TQ, class TD>
02106 void QSpace<TQ,TD>::takeMainQS(char disp) {
02107
02108     unsigned i,j,n,i0,imax=0;
02109     wbvector<wid_x_t> d, dc, I;
02110     wperm P;
02111
02112     wbMatrix<TQ> QT;
02113     wbvector<TD> Z;

```

```

02114     TD z, zmax=0, ztot=0, eps=1E-3;
02115
02116     getQtot(QT);
02117     QT.groupRecs(P,d);
02118
02119     d.cumsum_(dc); Z.init(d.len);
02120
02121     for (i=0; i<d.len; ++i) {
02122         for (z=0, n=d[i], i0=dc[i], j=0; j<n; ++j)
02123             z+=DATA[P[i0+j]]->norm2();
02124
02125         Z[i]=z;
02126
02127         if (z>zmax) { zmax=z; imax=i; }
02128         ztot+=z;
02129     }
02130
02131     if (disp) { char s_[128];
02132         z=fabs((ztot-zmax)/ztot);
02133         snprintf(s_,128,"%s skipping weight %s of total %s (%s%s)\n"
02134             "    Q=[%s]", (ABS(z)>eps ? "WRN" : " * "),
02135             NSTR(ztot-zmax), NSTR(ztot), NSTR(z*100.),
02136             ABS(z)>0.1 ? "% !?" : "%", QT.rec2Str(imax).data
02137         );
02138         wblog(FL,s_);
02139     }
02140
02141     I.init(d[imax], P.data+dc[imax]);
02142
02143     QIDX.Set2Recs(I);
02144     DATA.Select(I);
02145 }
02146
02147 template <class TQ, class TD> inline
02148 wbMatrix<TQ>& QSpace<TQ,TD>::getQsub(
02149     const unsigned k, wbMatrix<TQ> &Qk, char iflag
02150 ) const {
02151     unsigned i, r=rank(FL), s=QDIM*sizeof(TQ), offset=k*QDIM;
02152
02153     if (iflag) {
02154         wbindex I; I.Index_ex(r,k); getQsub(I,Qk);
02155         return Qk;
02156     }
02157
02158     if (k>=r) wblog(FL,
02159         "ERR %s() index out of bounds (%dx%d/%d; %d)",
02160         FCT, QIDX.dim1, QIDX.dim2, QDIM, k);
02161
02162     Qk.init(QIDX.dim1, QDIM);
02163
02164     for (i=0; i<QIDX.dim1; ++i)
02165         memcpy(Qk.rec(i), QIDX.rec(i)+offset, s);
02166
02167     return Qk;
02168 };
02169
02170 template <class TQ, class TD> inline
02171 wbMatrix<TQ>& QSpace<TQ,TD>::getQsub(
02172     const wbindex &I,
02173     wbMatrix<TQ> &QI
02174 ) const {
02175     unsigned i,k, r=rank(FL);
02176     TQ *q;
02177
02178     if (&QI==&QIDX) wblog(FL,
02179         "ERR %s() QIN and QOUT space are the same!",FCT);
02180     for (i=0; i<I.len; ++i) if (I[i]>=r) wblog(FL,
02181         "ERR %s() index out of bounds (%d/%d)",FCT,I[i],r);
02182
02183     QI.init(QIDX.dim1, I.len*QDIM);
02184
02185     for (i=0; i<QIDX.dim1; ++i) {
02186         const TQ *q0=QIDX.rec(i); q=QI.rec(i);
02187
02188         for (k=0; k<I.len; ++k, q+=QDIM) {
02189             Wb::cpyRange(q, q0+I[k]*QDIM, QDIM);
02190         }
02191     }
02192
02193     return QI;
02194 }
02195
02196 template <class TQ, class TD> inline
02197 void QSpace<TQ,TD>::getQsub(
02198     const wbindex &I,
02199 
```

```

02201     wbMatrix<TQ> &QI,
02202     wbMatrix<TQ> &Q2
02203 ) const {
02204
02205     unsigned i, k, r=rank(FL), s=QDIM*sizeof(TQ);
02206     windex I2;
02207     TQ *q1,*q2; const TQ *q0;
02208
02209     I.invert(r,I2);
02210
02211     if (&QI==&QIDX || &Q2==&QIDX)
02212         wblog(FL,"ERR getQsub requires distinct QOUT space!");
02213
02214     QI.init(QIDX.dim1, I .len*QDIM);
02215     Q2.init(QIDX.dim1, I2.len*QDIM);
02216
02217     for (i=0; i<QIDX.dim1; ++i) {
02218         q0=QIDX.rec(i); q1=QI.rec(i); q2=Q2.rec(i);
02219
02220         for (k=0; k<I .len; ++k) memcpy(q1+k*QDIM, q0+I [k]*QDIM, s);
02221         for (k=0; k<I2.len; ++k) memcpy(q2+k*QDIM, q0+I2[k]*QDIM, s);
02222     }
02223 }
02224
02225 template <class TQ, class TD> inline
02226 wbMatrix<TQ>& QSpace<TQ,TD>::getQtot(
02227     wbMatrix<TQ> &Qtot,
02228     char uflag
02229 ) const {
02230
02231     unsigned i,k,l,r, r0=rank(FL);
02232     TQ *q; const TQ *q0;
02233
02234     if (&Qtot==&QIDX)
02235         wblog(FL,"ERR getQtot requires distinct QOUT space!");
02236
02237     Qtot.init(QIDX.dim1, QDIM);
02238
02239     for (i=0; i<QIDX.dim1; ++i) {
02240         q0=QIDX.rec(i); q=Qtot.rec(i);
02241         for (l=r=0; r<r0; ++r)
02242             for (k=0; k<QDIM; ++k, ++l) q[k]+=q0[l];
02243     }
02244
02245     if (uflag) Qtot.makeUnique();
02246
02247     return Qtot;
02248 }
02249
02250 template <class TQ, class TD> inline
02251 void QSpace<TQ,TD>::getDistQtot(
02252     wbMatrix<TQ> &Q, wbvector<double> &w
02253 ) const {
02254     unsigned i,j,l,d;
02255     wbperm P; wbvector<wid_x_t> D;
02256     double n2;
02257
02258     QIDX.blockSum(QDIM,Q); Q.groupRecs(P,D);
02259     w.init(D.len);
02260
02261     for (l=i=0; i<D.len; ++i, l+=d) { d=D[i];
02262         for (n2=0, j=0; j<d; ++j) n2 += (DATA[P[l+j]]->norm2());
02263         w[i]=sqrt((double)n2);
02264     }
02265 }
02266
02267 template <class TQ, class TD> inline
02268 wbMatrix<TQ>& QSpace<TQ,TD>::getQsum(
02269     const windex &J, wbMatrix<TQ> &Q
02270 ) const {
02271
02272     unsigned i,j, r=rank(FL);
02273     const TQ *q0; TQ *q;
02274
02275     if (J.isEmpty()) { Q.init(); return Q; }
02276
02277     for (i=0; i<J.len; ++i) if (J[i]>=r) wblog(FL,
02278         "ERR block index out of range (%d/%d)",J[i],r);
02279
02280     Q.init(QIDX.dim1,QDIM);
02281
02282     if (allAdditive() || J.len==1) {
02283         for (i=0; i<QIDX.dim1; ++i) {
02284             q=Q.rec(i); q0=QIDX.rec(i);
02285             for (j=0; j<J.len; ++j) Wb::addRange(q0+J[j]*QDIM, q, QDIM);
02286         }
02287     }

```

```

02288     else if (allAbelian()) {
02289         if (qtype.len!=QDIM) wblog(FL,
02290             "ERR %s() got inconsistent qtype (len=%d/%d; %s)",
02291             FCT,qtype.len,QDIM,qStr().data
02292         );
02293
02294         for (i=0; i<QIDX.dim1; ++i) { q=Q.rec(i); q0=QIDX.rec(i);
02295             for (j=0; j<QDIM; ++j) {
02296                 q[j]=get_qtot_abelian(qtype[j],q0+j,J.len,J.data,QDIM);
02297             }
02298         }
02299     }
02300     else wblog(FL,
02301         "ERR %s() got non-abelian symmetries (%s)",FCT,qStr().data);
02302
02303     return Q;
02304 };
02305
02306 template <class TQ, class TD> inline
02307 wbMatrix<TQ>& QSpace<TQ,TD>::getQsum(
02308     const wbMatrix<TQ> &Q0, wbMatrix<TQ> &Q
02309 ) const {
02310
02311     if (!Q0) { Q.init(); return Q; }
02312     if (Q0.dim2==QDIM) { Q=Q0; return Q; }
02313
02314     if (!QDIM || (Q0.dim2 % QDIM)) wblog(FL,
02315         "ERR size mismatch Q0 (%dx%d /%d)",Q0.dim1,Q0.dim2,QDIM);
02316
02317     unsigned i,j, m=Q0.dim2/QDIM;
02318     const TQ *q0; TQ *q;
02319
02320     Q.init(Q0.dim1,QDIM);
02321
02322     if (allAdditive()) {
02323         for (i=0; i<Q0.dim1; ++i) { q=Q.rec(i); q0=Q0.rec(i);
02324             for (j=0; j<m; ++j) Wb::addRange(q0+j*QDIM, q, QDIM);
02325         }
02326     }
02327     else if (allAbelian()) {
02328         if (qtype.len!=QDIM) wblog(FL,
02329             "ERR %s() got inconsistent qtype (len=%d/%d; %s)",
02330             FCT,qtype.len,QDIM,qStr().data
02331         );
02332
02333         for (i=0; i<Q0.dim1; ++i) { q=Q.rec(i); q0=Q0.rec(i);
02334             for (j=0; j<QDIM; ++j) {
02335                 q[j]=get_qtot_abelian(qtype[j],q0+j,m,NULL,QDIM);
02336             }
02337         }
02338     }
02339     else wblog(FL,"ERR %s() "
02340         "got non-abelian symmetries (%s, m=%d)",FCT,qStr().data,m);
02341
02342     return Q;
02343 };
02344
02345 template <class TQ, class TD> inline
02346 void QSpace<TQ,TD>::FlipQ() {
02347     wbperm P(rank(FL)); P.Flip();
02348     PermuteQ(P);
02349 }
02350
02351 template <class TQ, class TD> inline
02352 void QSpace<TQ,TD>::PermuteQ(const wbperm &P) {
02353
02354     unsigned i,k,r=rank(FL), s=QDIM*sizeof(TQ);
02355     wbMatrix<TQ> Q0(QIDX);
02356     TQ *q0, *qp;
02357
02358     if (!QDIM) wblog(FL,"WRN QDIM=0!");
02359     if (P.isEmpty()) return;
02360     if (isEmpty() || QIDX.dim2==0) return;
02361
02362     if (!P.isValidPerm(r)) wblog(FL,
02363         "ERR %s() invalid permutation [%s] (%d)",FCT,STR(P),r);
02364
02365     for (i=0; i<QIDX.dim1; ++i) {
02366         q0=Q0.rec(i); qp=QIDX.rec(i);
02367         for (k=0; k<P.len; ++k) memcpy(qp+k*QDIM, q0+P[k]*QDIM, s);
02368     }
02369 }
02370
02371 template <class TQ, class TD> inline
02372 void QSpace<TQ,TD>::permuteQ(const wbperm &P, wbMatrix<TQ> &Q) const {
02373
02374     if (&Q==&QIDX) {

```

```

02375     wblog(FL,"ERR You shall not permuteQ onto itself");
02376 }
02377 else {
02378     unsigned i, k, r=rank(FL), s=QDIM*sizeof(TQ);
02379     const TQ *q0; TQ *qp;
02380
02381     if (P.isEmpty()) { Q=QIDX; return; }
02382
02383     if (!P.isValidPerm(r)) wblog(FL,
02384         "ERR invalid permutation '%s'",STR(P));
02385
02386     Q.init(QIDX.dim1, QIDX.dim2);
02387
02388     for (i=0; i<QIDX.dim1; ++i) {
02389         q0=QIDX.rec(i); qp=Q.rec(i);
02390         for (k=0; k<P.len; ++k) memcpy(qp+k*QDIM, q0+P[k]*QDIM, s);
02391     }
02392 }
02393 }
02394
02395 template <class TQ, class TD>
02396 QSpace<TQ,TD>& QSpace<TQ,TD>::Permute(
02397     const wperm &P_, char iflag,
02398     char rcpy
02399 ) {
02400
02401     unsigned i,n,r=rank(FL);
02402     if ((!r && isEmpty()) || !P_.len) { return *this; }
02403
02404     if (P_.len>r) { wblog(FL,
02405         "ERR %s() invalid permutation '%s'\n(length out of bounds: %d/%d)",
02406         myname,STR(P_),P_.len,r);
02407     }
02408     if (P_.isIdentityPerm(FL)) { return *this; }
02409
02410     wperm P(P_,iflag,r);
02411     PermuteQ(P);
02412
02413     for (i=0; i<DATA.len; ++i) { DATA[i]->Permute(P,0,rcpy); }
02414     if (itags.len) { itags.Select(P); }
02415
02416     if (CGR) {
02417         for (n=CGR.numel(), i=0; i<n; ++i) {
02418             CGR[i].Permute(P);
02419         }
02420         NormCGW(0,0,rcpy);
02421     }
02422
02423     return *this;
02424 };
02425
02426 template <class TQ, class TD> inline
02427 QSpace<TQ,TD>& QSpace<TQ,TD>::permute(
02428     const wperm &P_, QSpace<TQ,TD> &B, char iflag
02429 ) const {
02430
02431     unsigned r=rank(FL);
02432     if (P_.isEmpty() || P_.isIdentityPerm(FL,r)) {
02433         B.init(*this); return B;
02434     }
02435
02436     unsigned i,n;
02437     wperm P(P_,iflag); B.init();
02438
02439     permuteQ(P,B.QIDX);
02440     B.QDIM=QDIM; B.qtype=qtype; B.itags=itags;
02441
02442     if (otype && P.isOpTranspose()) {
02443         B.otype=otype;
02444     }
02445
02446     if (B.itags.len) B.itags.Select(P);
02447
02448     B.setupDATA();
02449     for (n=DATA.len, i=0; i<n; ++i) {
02450         DATA[i]->permute(*B.DATA[i],P);
02451     }
02452
02453     if (CGR.isEmpty()) { B.CGR.init(); }
02454     else {
02455         B.setupCGR(); n=CGR.numel();
02456         if (n!=B.CGR.numel()) wblog(FL,"ERR %s() got CGR size mismatch "
02457             "%s <> %s",FCT,CGR.sizeStr().data,B.CGR.sizeStr().data);
02458         for (i=0; i<n; ++i) {
02459             CGR[i].permute(P,B.CGR[i]);
02460         }
02461     }

```

```

02462
02463     return B;
02464 };
02465
02466 template <class TQ, class TD> inline
02467 void QSpace<TQ,TD>::setQ(unsigned k,
02468     const QSpace<TQ,TD> &B, unsigned k0
02469 ) {
02470
02471     unsigned i, s=QDIM*sizeof(TQ), dd0=B.QIDX.dim2, dd2=QIDX.dim2;
02472     TQ *d0 = B.QIDX.data+k0*QDIM, *d2 = QIDX.data+k*QDIM;
02473
02474     if (QIDX.dim1!=B.QIDX.dim1 || QDIM!=B.QDIM) wblog(FL,
02475         "ERR %s() incompatible objects (%d,%d; %d,%d)",
02476         FCT, QIDX.dim1, B.QIDX.dim1, QDIM, B.QDIM);
02477
02478     if (rank(FL)<=k || B.rank(FL)<=k0) wblog(FL,
02479         "ERR %s() index out of bounds (%d/%d; %d/%d)",
02480         FCT, k, rank(FL), k0, B.rank(FL));
02481
02482     for (i=0; i<QIDX.dim1; ++i, d0+=dd0, d2+=dd2)
02483         memcpy(d2,d0,s);
02484 }
02485
02486 template <class TQ, class TD> inline
02487 void QSpace<TQ,TD>::setQRec(
02488     unsigned r, unsigned j, const TQ *q0, const TQ *q2
02489 ){
02490     if (r>=QIDX.dim1 || QDIM*(j+1)>QIDX.dim2) wblog(FL,
02491         "ERR %s() index out of bounds (%d,%d; %d,%d)",
02492         FCT, r, j, QIDX.dim1, QDIM);
02493
02494     TQ *q = QIDX.rec(r)+j*QDIM;
02495     if (q2==NULL)
02496         for (unsigned i=0; i<QDIM; ++i) q[i]=q0[i];
02497     else for (unsigned i=0; i<QDIM; ++i) q[i]=q0[i]+q2[i];
02498 }
02499
02500 template <class TQ, class TD> inline
02501 bool QSpace<TQ,TD>::hasEqualQ(unsigned i1, unsigned i2) const {
02502
02503     unsigned i, s=QDIM*sizeof(TQ), D=QIDX.dim2, r=rank(FL);
02504     TQ *d1 = QIDX.data+i1*QDIM, *d2 = QIDX.data+i2*QDIM;
02505
02506     if (i1>=r || i2>=r) wblog(FL,
02507         "ERR %s() index out of bounds (%d,%d; %d)",FCT,i1,i2,r);
02508
02509     for (i=0; i<QIDX.dim1; ++i, d1+=D, d2+=D)
02510         if (memcmp(d1,d2,s)) return 0;
02511
02512     return 1;
02513 };
02514
02515 template <class TQ, class TD>
02516 bool QSpace<TQ,TD>::hasQOverlap(const QSpace<TQ,TD> &A) const {
02517
02518     wbindx Ia, Ib;
02519     if (qtype!=A.qtype) {
02520         wblog(FL,"WRN got objects with different qtype");
02521         return 0;
02522     }
02523
02524     matchIndex(QIDX, A.QIDX, Ia, Ib);
02525     return (Ia.len!=0);
02526 };
02527
02528 template <class TQ, class TD>
02529 template <class TB>
02530 int QSpace<TQ,TD>::hasQOverlap(
02531     unsigned ica, const QSpace<TQ,TB> &B, unsigned icb,
02532     const char type
02533 ) const {
02534
02535     unsigned ra=rank(FL), rb=B.rank(FL); int q=0;
02536     wbMatrix<TQ> QA,QB;
02537     wbindx Ia,Ib,Ja,Jb;
02538
02539     if (!ica || !icb) wblog(FL,
02540         "ERR %s() index must be 1-based (%d,%d)",FCT,ica,icb);
02541     --ica; --icb;
02542
02543     if (QDIM!=B.QDIM) wblog(FL,
02544         "ERR %s() incompatible objects\nQDIM=%d,%d",
02545         FCT, QDIM, B.QDIM);
02546     if (ica>=ra || icb>=rb) wblog(FL,
02547         "ERR %s() index out of bounds\n%d/%d, %d/%d (QDIM=%d,%d)",
02548         FCT, ica,ra, icb,rb, QDIM, B.QDIM);

```

```

02549
02550     getQsub(ica,QA).makeUnique(Ja);
02551     B.getQsub(icb,QB).makeUnique(Jb); matchIndex(QA,QB,Ia,Ib);
02552
02553     if (Ia.len==QA.dim1) { q|=2; }
02554     if (Ib.len==QB.dim1) { q|=4; }
02555
02556     if (type=='<') { if (!(q&2)) { q=0; }} else
02557     if (type=='>') { if (!(q&4)) { q=0; }} else
02558     if (type=='-') { if ( q!=6) { q=0; }} else {
02559         if (type) wblog(FL,"WRN %s() invalid type=%c<%d>",FCT,type,type);
02560         if (!Ia.len) { q=0; }
02561     }
02562
02563     if (q && type) {
02564         for (unsigned i=0; i<Ia.len; ++i) {
02565             if (!DATA[Ja[Ia[i]]]->hasSameSize(ica, *B.DATA[Jb[Ib[i]]],icb)) {
02566                 q=-q; break;
02567             }
02568         }
02569     }
02570
02571     return q;
02572 };
02573
02574 template <class TQ, class TD>
02575 mxArray* QSpace<TQ,TD>::QIDX_toMx() const {
02576
02577     unsigned k, r;
02578     mxArray *a;
02579
02580     if (QIDX.isEmpty())
02581         return mxCreateCellMatrix(0,0);
02582
02583     if (!QDIM || QIDX.dim2%QDIM) wblog(FL,
02584         "ERR %s() invalid QSpace (QIDX: %dx%d; %d)",
02585         FCT,QIDX.dim1,QIDX.dim2,QDIM
02586     );
02587
02588     r=QIDX.dim2/QDIM; a=mxCreateCellMatrix(1,r);
02589     if (!a) wblog(FL,"ERR could not allocate cell array (%d)", r);
02590
02591     for (k=0; k<r; ++k)
02592         mxSetCell(a, k, getQsub(k).toMx());
02593
02594     return a;
02595 }
02596
02597 template <class TQ, class TD>
02598 mxArray* QSpace<TQ,TD>::INFO_toMx() const {
02599
02600     if ((qtype.isEmpty() && CGR.isEmpty()) &&
02601         itags.isEmpty() && otype==QS_NONE)
02602     ){ return mxCreateCellMatrix(0,0); }
02603
02604     const char *fields[] = { "qtype","otype","itags","ctime","cgr" };
02605
02606     mxArray *S=mxCreateStructMatrix(1,1,5,fields);
02607
02608     mxSetFieldByNumber(S,0,0,qStr().toMx());
02609     mxSetFieldByNumber(S,0,1,otype2Str().toMx());
02610     mxSetFieldByNumber(S,0,2,itags.toMx());
02611     mxSetFieldByNumber(S,0,3,numtoMx(ctime));
02612
02613     mxSetFieldByNumber(S,0,4,CGR.toMx());
02614
02615     return S;
02616 };
02617
02618 template <class TQ, class TD>
02619 mxArray* QSpace<TQ,TD>::DATA_toMx() const {
02620
02621     mxArray *a=mxCreateCellMatrix(DATA.len, DATA.len ? 1 : 0);
02622     if (!a) wblog(FL,"ERR failed to allocate cell array (%d)",DATA.len);
02623     if (DATA.len) {
02624         unsigned i, n=DATA.len; mxArray* x[n];
02625         int np=MAX( QSP_NUM_THREADS, MP_NUM_THREADS );
02626         if (np>int(n)) { np=(n ? n : 1); }
02627         for (i=0; i<n; ++i) { x[i]=0; }
02628
02629         Wb::LogException ex;
02630
02631         #pragma omp parallel for num_threads(np)
02632         for (i=0; i<n; ++i) { if (!ex) {
02633             try { x[i]=DATA[i]->toMx(); }
02634             catch (Wb::LogException &e_) { ex+=e_; }
02635             catch (...) { ++ex; }

```

```

02636     }}
02637     ex.report(FLF);
02638
02639     for (i=0; i<n; ++i) { if (x[i]) mxSetCell(a,i,x[i]); }
02640 }
02641
02642 return a;
02643 };
02644
02645 template <class TQ, class TD>
02646 mxArray* QSpace<TQ,TD>::DATA_save2Mx(char vflag) {
02647
02648     mxArray *a=mxCreateCellMatrix(DATA.len, DATA.len ? 1 : 0);
02649     if (!a) wblog(FL,"ERR failed to allocate cell array (%d)",DATA.len);
02650
02651     if (vflag) { size_t sd=getDataSize();
02652         if (sd<(1<30)) vflag=0; else wblog(FL,
02653             "TST %s() %d entries @ %.3gG ...",FCT,DATA.len,sd/double(1<30));
02654     }
02655
02656     for (unsigned k=0; k<DATA.len; ++k) {
02657         mxSetCell(a, k, DATA[k]->toMx());
02658         if (vflag && k%10==0) PRINTF(" %6d/%ld ... \r",k+1,DATA.len);
02659         DATA[k]->init();
02660     }
02661     if (vflag) PRINTF("\r%60s\r","");
02662
02663     return a;
02664 };
02665
02666 template <class TQ, class TD>
02667 mxArray* QSpace<TQ,TD>::toMx(bool setcls) const {
02668
02669     mxArray *S=mxCreateStruct(1,1);
02670     this->add2MxStruct(S,0,setcls);
02671     return S;
02672 };
02673
02674 template <class TQ, class TD>
02675 mxArray* QSpace<TQ,TD>::save2Mx(char vflag) {
02676
02677     mxArray *S=mxCreateStruct(1,1);
02678     this->save2MxStruct(S,0,0,vflag);
02679     return S;
02680 };
02681
02682 template <class TQ, class TD>
02683 mxArray* QSpace<TQ,TD>::mxCreatestruct(unsigned m, unsigned n) const {
02684
02685     const char *fields[] = { "Q","data","info" };
02686     return mxCreateStructMatrix(m,n,3,fields);
02687 }
02688
02689 template <class TQ, class TD>
02690 void QSpace<TQ,TD>::add2MxStruct(
02691     mxArray *S, unsigned i,
02692     bool setcls,
02693     char chk
02694 ) const {
02695
02696     mxArray *a;
02697
02698     #ifdef WBC_QSPACE_IO
02699     Wb::Clock clk("QS:toMx",1);
02700     #endif
02701
02702     if (chk) { unsigned s=0; int q=0;
02703         if (S==NULL || (s=mxGetNumberOfElements(S))<1 || i>=s ||
02704             (q=mxGetFieldNumber(S,"qtype")<0) wblog(FL,
02705                 "ERR %s() must follow mxCreateStruct()\n%lx, %d/%d, %d",
02706                 FCT,S,i+1,s,q);
02707     }
02708
02709     a=QIDX_toMx(); mxSetFieldByNumber(S,i,0,a);
02710     a=DATA_toMx(); mxSetFieldByNumber(S,i,1,a);
02711     a=INFO_toMx(); mxSetFieldByNumber(S,i,2,a);
02712
02713     if (setcls && Wb::my_caller_tid==omp_get_thread_num()) {
02714         mxSetClassName(S,"QSpace");
02715     }
02716 };
02717
02718 template <class TQ, class TD>
02719 void QSpace<TQ,TD>::save2MxStruct(
02720     mxArray *S, unsigned i, char tst, char vflag
02721 ){
02722     mxArray *a;

```



```

02723
02724 #ifndef WBC_QSPACE_IO
02725     Wb::Clock clk("QS:toMx",1);
02726 #endif
02727
02728     if (tst) {
02729         unsigned s=0; int q=0;
02730         if (S==NULL || (s=mxGetNumberOfElements(S))<1 || i>=s
02731             || (q=mxGetFieldNumber(S,"qtype")<0) wblog(FL,
02732                 "ERR %s() must follow mxCreateStruct()\n%lx, %d/%d, %d",
02733                 FCT,S,i+1,s,q);
02734     }
02735
02736     a=QIDX_toMx();           mxSetFieldByNumber(S,i,0,a);
02737     a=DATA_save2Mx(vflag);   mxSetFieldByNumber(S,i,1,a);
02738     a=INFO_toMx();           mxSetFieldByNumber(S,i,2,a);
02739
02740     init();
02741 };
02742
02743 template <class TQ, class TD>
02744 void QSpace<TQ,TD>::init(
02745     const char *F, int L, const mxArray *S,
02746     char ref,
02747     unsigned k,
02748     char skip_empty
02749 ){
02750     unsigned i,j,l,m,n,r,dim1=0,dim2=0; unsigned rk=-1;
02751     mxArray *aq,*ad,*ai;
02752     wbvector< wbMatrix<TQ> > MQ;
02753     wbvector< wbMatrix<TQ>*> m;
02754     const char cflag = (typeid(TD)==typeid(wbcomplex) ? 'C' : 0);
02755     char refD=0, refC=1, isa=0;
02756
02757     #ifndef WBC_QSPACE_IO
02758         Wb::Clock clk("QS:mXinit",1);
02759     #endif
02760
02761     clearQSpace();
02762
02763     if (!S || mxIsEmpty(S) || mxIsEmptyQSpace(S,k)) return;
02764
02765     aq=mxGetField(S,k,"Q");
02766     ad=mxGetField(S,k,"data");
02767     ai=mxGetField(S,k,"info");
02768
02769     if (mxIsQSpace(F,L,S,rk,cflag,k)<=0 || !aq || !ad) wblog(F,L,
02770         "ERR invalid QSpace (%d,%d)",aq==0,ad==0);
02771
02772     if (ref) {
02773         if (ref=='r') { refD = ref; } else
02774             wblog(FL,"ERR %s() got invalid ref=%c<%d>",FCT,ref,ref);
02775     }
02776
02777     m=mxGetM(aq); r=mxGetN(aq);
02778
02779     if ((m!=1 && r!=1 && m && r) || mxGetNumberOfDimensions(aq)>2) {
02780         wblog(F,L,"ERR %s() invalid QSpace\n"
02781             "Q must be blocked into row cell vector (%dx%d; %d)",
02782             FCT,m,r,mxGetNumberOfDimensions(aq));
02783     }
02784
02785     r*=m; MQ.init(r);
02786
02787     for (i=0; i<r; ++i) {
02788         MQ[i].init(mxGetCell(aq,i));
02789
02790         if (i==0) {
02791             dim1=MQ[i].dim1; dim2=MQ[i].dim2;
02792         }
02793         else if (MQ[i].dim1!=dim1 || MQ[i].dim2!=dim2) { wblog(F,L,
02794             "ERR %s() invalid QSpace\nsize mismatch in Q data (%dx%d; %dx%d)",
02795             FCT, MQ[i].dim1, MQ[i].dim2, dim1, dim2);
02796         }
02797     }
02798
02799     mq.init(MQ.len); for (i=0; i<MQ.len; ++i) mq[i] = &MQ[i];
02800
02801     QIDX.CAT(2, (const wbMatrix<TQ>**) mq.data, mq.len);
02802     QDIM=dim2;
02803
02804     if (!QIDX.isUnique()) wblog(FL,
02805         "ERR %s() invalid input QSpace\nrows in [Q{:}] are not unique",FCT);
02806     setupDATA();
02807
02808     if (ai && !mxIsEmpty(ai)) {

```

```

02810     if (mxIsStruct(ai)) {
02811         mxArray *a; QVec qv; unsigned dq=0;
02812
02813         a=mxGetField(ai,0,"qtype");
02814         if (a) qtype.init(F,L,a); else qtype.init();
02815
02816         isa=qtype.allAbelian();
02817         if (isa==1 && qtype.len) qtype.ReduceU1();
02818
02819         initOType(F_L,mxGetField(ai,0,"otype"));
02820
02821         itags.init(F_L,mxGetField(ai,0,"itags"));
02822
02823         if (itags.len && itags.len!=r) wblog(FL,
02824             "ERR %s() invalid number of itags (%s; %d/%d)",
02825             FCT,IT2STR__,itags.len,r);
02826
02827         a=mxGetField(ai,0,"ctime");
02828         if (a && !mxIsEmpty(a)) { mxGetNumber(a,ctime); }
02829
02830         a=mxGetField(ai,0,"cgr");
02831         if (a && !mxIsEmpty(a)) {
02832
02833             QSet<TQ> Q;
02834             char xflag=(isa ? 0 : 1);
02835             wvector<unsigned> qdc; qtype.Qpos(qdc);
02836
02837             setupCGR();
02838             m=mxGetM(a); n=mxGetN(a);
02839
02840             if (m==1 && !QIDX.dim1) {
02841                 if (!mxIsScalarQSpace(S) || m>1 || r) wblog(FL,
02842                     "ERR %s() invalid scalar QSpace",FCT);
02843                 QIDX.init(1,0); QDIM=qtype.Qlen(); r=2;
02844                 setupDATA(); setupCGR();
02845             }
02846             else
02847                 if (!QDIM || QIDX.dim2%QDIM || QIDX.dim2/QDIM!=r) wblog(FL,
02848                     "ERR %s() empty QDIM while info.cgr is specified (%d/%d)",
02849                     FCT,m,n);
02850             else
02851                 if (mxGetNumberOfDimensions(ad)>2 ||
02852                     m!=QIDX.dim1 || m!=CGR.dim1 ||
02853                     n!=qtype.len || n!=CGR.dim2 || qtype.Qlen()!=QDIM)
02854                 wblog(FL,"ERR %s() invalid dimensions for cell array "
02855                     "info.cgr\n%s cgr: %dx%d / %dx%d; QIDX: %dx%d/%d",
02856                     FCT, qStr().data, m,n, CGR.dim1, CGR.dim2,
02857                     QIDX.dim1, QIDX.dim2, QDIM
02858                 );
02859
02860             if (!mxIsCell(a)&& !mxIsStruct(a)) wblog(FL,
02861                 "ERR %s() cell array expected for info.cgr field (%s)",
02862                 FCT,mxGetClassName(a)
02863             );
02864
02865             for (l=j=0; j<n; ++j)
02866                 for ( i=0; i<m; ++i, ++l) {
02867                     Q.init(qtype[j], QIDX.rec(i)+qdc[j], r, QDIM, itags);
02868                     CGR(i,j).init(F_L,a,l,refC,&Q,xflag);
02869                 }
02870         }
02871
02872         if (qtype.len) { dq=qtype.Qlen();
02873             if (dq!=QDIM && (QIDX.dim2 || CGR.dim2)) wblog(FL,"ERR "
02874                 "incompatible number of symmetry labels (%d/%d; %s; %s)",
02875                 QDIM, dq, QIDX.sizeStr().data, CGR.sizeStr().data
02876             );
02877         }
02878     }
02879     else wblog(F,L,"ERR invalid qtype (structure required)");
02880
02881     if (isa) {
02882         if (!CGR.isEmpty()) {
02883             for (i=0; i<CGR.dim1; ++i)
02884                 for (j=0; j<CGR.dim2; ++j) { if (CGR(i,j).cgb) {
02885                     const cdata__ &c=(CGR(i,j).cgb->cgd);
02886                     if (!c.isScalar()) wblog(FL,
02887                         "ERR %s() invalid abelian CGC space (%s)",FCT,SSTR(c));
02888                     if (c.D.data && c.D.data[0]!=1) wblog(FL,
02889                         "ERR %s() invalid abelian CGC coefficient (%.4g)",
02890                         FCT,double(c.D.data[0])
02891                     );
02892                 }}
02893             checkQ_CGR(F,L); CGR.init();
02894         }
02895     }
02896 }

```

```

02897     }
02898     else qtype.init();
02899
02900     m=mxGetM(ad); n=mxGetN(ad);
02901
02902     if ((m>1 && n>1) || mxGetNumberOfDimensions(ad)>2) wblog(F,L,
02903         "ERR QSpace() cell VECTOR expected for data (%d,%d)",m,n);
02904
02905     m*=n;
02906
02907     if (DATA.len!=m) {
02908         if (m==1 && DATA.len==0 && QIDX.isEmpty() && CGR.isEmpty()) {
02909             QIDX.init(1,0); setupDATA();
02910             if (r<2) r=2;
02911         }
02912         else { wblog(F,L,
02913             "ERR QSpace() dimension mismatch (QIDX: %dx%d, data: %d)",
02914             QIDX.dim1, QIDX.dim2, m);
02915         }
02916     }
02917
02918     for (n=i=0; i<m; ++i) {
02919         DATA[i]->init(mxGetCell(ad,i), refD);
02920         DATA[i]->appendSingletons(FL,r,1);
02921         if (!*DATA[i]) { ++n; }
02922     }
02923
02924     if (n && skip_empty) {
02925         m=SkipEmptyData(FL);
02926     }
02927 };
02928
02929 template <class TQ, class TD> inline
02930 void QSpace<TQ,TD>::init2ref(const QSpace<TQ,TD> &A) {
02931
02932     clearQSpace();
02933
02934     QIDX.init2ref(A.QIDX); QDIM=A.QDIM;
02935     DATA.init2ref(A.DATA);
02936     qtype.init2ref(A.qtype); otype=A.otype;
02937     itags.init2ref(A.itags);
02938
02939     if (!A.CGR.isEmpty()) CGR.init2ref(A.CGR);
02940
02941     isref=1;
02942 };
02943
02944 template <class TQ, class TD> inline
02945 void QSpace<TQ,TD>::setRand(double nrm) {
02946
02947     unsigned i;
02948     for (i=0; i<DATA.len; ++i) DATA[i]->setRand();
02949
02950     if (nrm>0) {
02951         double fac=nrm/sqrt(double(norm2()));
02952         for (i=0; i<DATA.len; ++i) (*DATA[i]) *= fac;
02953     }
02954 };
02955
02956 template <class TQ, class TD>
02957 void QSpace<TQ,TD>::info(const char *vname,
02958     unsigned nlt, unsigned nlb,
02959     unsigned nind
02960 ) const {
02961
02962     wvector<widxt> D,DX;
02963     char cgflag=gotCGS();
02964     unsigned r=(QDIM!=0 ? QIDX.dim2/QDIM : 0);
02965     size_t l=0, n=64; char s[n], sp_[nind+1];
02966
02967     unsigned lsz=2*r+2; if (lsz<8) { lsz=8; }
02968
02969     if (QDIM && QIDX.dim2 && QIDX.dim2%QDIM) wblog(FL,
02970         "ERR %s() %d/%d = ?",FCT, QIDX.dim2, QDIM);
02971     if (nind) { memset(sp_,' ',nind); }; sp_[nind]=0;
02972
02973     l=snprintf(s,n,"QSpace<%s,%s> %s",
02974         sSTR(TQ), sSTR(TD), qtype.len ? qStr().data : "");
02975     if (l<n && itags.len) {
02976         l+=snprintf(s+l,n-l, " %s",STR(itags));
02977     }
02978     if (l>=n) wblog(FL,"WRN %s() string out of bounds (%d/%d)",FCT,l,n);
02979
02980     for (l=0; l<nlt; ++l) { PRINTF("\n"); }
02981     if (vname && vname[0]) {
02982         if (sp_[0] || strlen(vname)>3)

```

```

02984         { PRINTF("%s%-3s",sp_,vname); }
02985     else { PRINTF(" %-2s",vname); }
02986 }
02987 else if (sp_[0]) { PRINTF("%s",sp_); }
02988 PRINTF(" %-34s %3ld x (%d x%d)",s,QIDX.dim1,r,QDIM);
02989
02990 getDim(D,&DX);
02991
02992 PRINTF(" %-*s",lsz,SSTR(D )); if (cgflag>0) {
02993 PRINTF(" %-*s",lsz,SSTR(DX)); }
02994 PRINTF("%s\n", isref? " *REF*":"" );
02995
02996 for (l=0; l<n1b; ++l) { PRINTF("\n"); }
02997 };
02998
02999 template <class TQ, class TD>
03000 void QSpace<TQ,TD>::print(const char *vname, char vflag) const {
03001
03002     unsigned i=0,j,l,n, N=QIDX.dim1, r=rank(FL), m=(N<12? N:4), nstr=256;
03003     char cgflag=gotCGS(FL), qstr[nstr];
03004     const TQ *qs=QIDX.data;
03005
03006     unsigned lsz=2*r+2;
03007     if (lsz<8) { lsz=8; }
03008
03009     if (vflag & 'D') { vflag |=3; }
03010     if (vflag&1) { PRINTF("\n"); }
03011
03012     info(vname,0,0,0);
03013     PRINTF("\n");
03014
03015     QVec qt(qtype);
03016     if (!qt) { qt.init2val(QDIM,QType(QTYPE_U1)); }
03017
03018     for (; i<N; ++i) {
03019         if (i>=m) {
03020             if (N>2*m) { i=N-m; qs=QIDX.rec(i);
03021                 PRINTF("      : ... \n");
03022             }; m=N;
03023         }
03024
03025         if (i<N) {
03026             for (l=j=0; j<r; ++j, qs+=QDIM) {
03027                 if (j && l<nstr) { l+=snprintf(qstr+l,nstr-l," " ); }
03028                 if (l<nstr) { l+=qt.print_qset(FL,qs,qstr+l,nstr-l); }
03029                 else { break; }
03030             }
03031             else { strcpy(qstr,"!?"); }
03032
03033             PRINTF("%5d. [ %s ] %-*s ", i+1, qstr, lsz,
03034                 i<DATA.len ? DATA[i]->sizeStr(r," @").data : "!?");
03035
03036             if (i>=DATA.len) { PRINTF("      !?\n"); continue; }
03037             if (vflag & 64) {
03038                 if (!(DATA[i]->printdata(vname))) { PRINTF("\n"); }
03039             }
03040             else {
03041                 if (cgflag>0) {
03042                     for (j=0; j<CGR.dim2; ++j) { if (!CGR(i,j).isAbelian()) {
03043                         PRINTF(" %-*s",lsz,SSTR(CGR(i,j))); }
03044                     }
03045                 }
03046
03047                 n=DATA[i]->numel();
03048
03049                 if (n==0) { PRINTF("      []\n"); } else
03050                 if (n==1) { PRINTF(" %9s", Wb::num2Str(DATA[i]->data[0]).data); }
03051                 else {
03052                     n *= sizeof(TD);
03053                     if (n<(1<<10)) { PRINTF("      %5d B", n); } else
03054                     if (n<(1<<20)) { PRINTF("      %5.2f kB", n/double(1<<10)); }
03055                     else { PRINTF("      %5.2f MB", n/double(1<<20)); }
03056                 }
03057                 PRINTF("%s\n",DATA[i]->isRef() ? " *ref":"" );
03058             }
03059         }
03060
03061         if (vflag&2) { PRINTF("\n"); }
03062     };
03063
03064 #endif
03065

```

5.18 QSpace_aux.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : QSpace
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_QSpace_AUX_HH__
00021 #define __WB_QSpace_AUX_HH__
00022
00023 template <class TQ, class TD> class QSpace;
00024 template <class TQ, class TD> class CPAT;
00025
00026 template <class TQ, class TA, class TB, class TC>
00027 void contractDATA_plain(
00028     const char *F, int L,
00029     const QSpace<TQ,TA> &A, C_UVEC &Ia, const ctrIdx &ica,
00030     const QSpace<TQ,TB> &B, C_UVEC &Ib, const ctrIdx &icb,
00031     QSpace<TQ,TC> &C, unsigned ic
00032 );
00033
00034 template <class TQ, class TA, class TB, class TC>
00035 double contractDATA_group(
00036     const char *F, int L,
00037     const QSpace<TQ,TA> &A, C_UVEC &Ia, const ctrIdx &ica,
00038     const QSpace<TQ,TB> &B, C_UVEC &Ib, const ctrIdx &icb,
00039     QSpace<TQ,TC> &C, unsigned ic,
00040     char preview=0
00041 );
00042
00043 template<class TQ, class TD>
00044 void mxInitQSpaceVec(
00045     const char *F, int L, const mxArray* C,
00046     wbvector< QSpace<TQ,TD> > &Fk,
00047     const char ref=0,
00048     const unsigned *rmin=NULL,
00049     const unsigned *rmax=NULL
00050 );
00051
00052 template<class TQ, class TD>
00053 void mxInitQSpaceVecVec(
00054     const char *F, int L, const mxArray* C,
00055     wbvector< wbvector< QSpace<TQ,TD> > > &Fk, const char ref=0
00056 );
00057
00058 template<class TQ, class TD>
00059 void mxcInitQSpaceVec(
00060     const char *F, int L, const mxArray* C,
00061     wbvector< QSpace<TQ,TD> > &Fk, const char ref=0
00062 );
00063
00064 template<class TQ, class TD>
00065 mxArray* QSpaceVec2Mx(
00066     const wbvector< QSpace<TQ,TD> > &Fk
00067 );
00068
00069 template<class TQ, class TD>
00070 mxArray* QSpaceVec2Mx(
00071     const wbvector< QSpace<TQ,TD> > &F, wbindex &I
00072 );
00073
00074 template<class TQ, class TD>
00075 mxArray* QSpaceVecVec2Mx(
00076     const wbvector< wbvector< QSpace<TQ,TD> > > &F,
00077     const char ref=0
00078 );
00079
00080 template<class TQ, class TD>
00081 mxArray* QSpaceVec2Mxc(
00082     const wbvector< QSpace<TQ,TD> > &F,
00083     const char ref=0
00084 );
00085

```

```

00086 template<class TQ, class TD>
00087 void putQSpaceVec(
00088     const wbvector< QSpace<TQ,TD> > &Fk,
00089     const char *vname, const char *ws="caller"
00090 ){
00091     mxArray *a=Fk.toMx();
00092     mxPutAndDestroy(FL,a,vname,ws);
00093 };
00094
00095 template <class TQ, class TD>
00096 void getDiffOp(
00097     const QSpace<TQ,TD> &A, unsigned ia,
00098     const QSpace<TQ,TD> &B, unsigned ib, const QSpace<TQ,TD> &C
00099 );
00100
00101 template<class TQ, class TD>
00102 void getQall(const wbvector< QSpace<TQ,TD> > &F, wbMatrix<TQ> &QA);
00103
00104 int mxIsQSpace(
00105     const char *F, int L,
00106     const mxArray *a, unsigned &rank,
00107     char cflag=0,
00108     unsigned k=-1,
00109     const unsigned *rmin=NULL,
00110     const unsigned *rmax=NULL,
00111     const char *istr=NULL
00112 );
00113
00114 inline int mxIsQSpace(
00115     const char *F, int L, const mxArray *a, char cflag=0,
00116     const char *istr=NULL
00117 ){
00118     unsigned r=-1;
00119     return mxIsQSpace(F,L,a,r,cflag,-1,NULL,NULL,istr);
00120 };
00121
00122 inline int mxIsQSpace(
00123     const mxArray *a, char cflag=0, const char *istr=NULL
00124 ){
00125     unsigned r=-1;
00126     return mxIsQSpace(NULL,0,a,r,cflag,-1,NULL,NULL,istr);
00127 };
00128
00129 bool mxIsQSpaceArr(
00130     const char *F, int L,
00131     const mxArray *S,
00132     unsigned rank=-1,
00133     int ardim=2,
00134     const unsigned *rmin=NULL,
00135     const unsigned *rmax=NULL,
00136     char cflag=0
00137 );
00138
00139 bool mxIsQSpaceVecVec(const mxArray *C, unsigned r=-1, char cflag=0);
00140
00141 inline bool mxIsQSpaceVec(
00142     const char *F, int L,
00143     const mxArray *S,
00144     unsigned rank=-1,
00145     const unsigned *rmin=NULL,
00146     const unsigned *rmax=NULL,
00147     char cflag=0
00148 ){ return mxIsQSpaceArr(F,L,S,rank,1,rmin,rmax,cflag); };
00149
00150 inline bool mxIsQSpaceVecR23(
00151     const char *F, int L,
00152     const mxArray *S,
00153     char cflag=0
00154 ){
00155     unsigned rank=-1, rmin=2, rmax=3;
00156     return mxIsQSpaceArr(F,L,S,rank,1,&rmin,&rmax,cflag);
00157 };
00158
00159 inline bool mxIsQSpaceMat(
00160     const char *F, int L,
00161     const mxArray *S,
00162     unsigned rank=-1,
00163     const unsigned *rmin=NULL,
00164     const unsigned *rmax=NULL,
00165     char cflag=0
00166 ){ return mxIsQSpaceArr(F,L,S,rank,2,rmin,rmax,cflag); };
00167
00168 bool mxIsEmptyQSpace(const mxArray *a, unsigned k=0);
00169
00170 bool mxIsScalarQSpace(const mxArray *a, unsigned k=0);
00171
00172 inline bool mxIsQSpaceScalar(

```

```

00173     const char *F, int L,
00174     const mxArray *S,
00175     unsigned rank=-1,
00176     const unsigned *rmin=NULL,
00177     const unsigned *rmax=NULL,
00178     char cflag=0
00179 ){ return mxIsQSpaceArr (F,L,S,rank,0,rmin,rmax,cflag); };
00180
00181 bool mxIsQSpaceVec(const mxArray *S, int fid, unsigned r=-1, char cflag=0);
00182 bool mxIsQSpaceVEC(const mxArray *S, int fid, unsigned r=-1, char cflag=0);
00183
00184 int mxIsQSpaceVec(
00185     const char *F, int L,
00186     const char *fname, const char *vname,
00187     unsigned rank=-1,
00188     const unsigned *rmin=NULL,
00189     const unsigned *rmax=NULL,
00190     char cflag=0,
00191     unsigned N=-1
00192 );
00193
00194 int mxIsQSpaceVEC(
00195     const char *F, int L,
00196     const char *fname, const char *vname,
00197     unsigned rank=-1,
00198     const unsigned *rmin=NULL,
00199     const unsigned *rmax=NULL,
00200     char cflag=0
00201 );
00202
00203 bool mxIsQSpaceVecOrEmpty(const mxArray *S, int rank, char cflag=0) {
00204     if (mxIsEmpty(S)) return 1;
00205     return mxIsQSpaceVec (FL,S,rank,NULL,NULL,cflag);
00206 };
00207
00208 inline int mxIsQSpaceOrEmpty(
00209     const mxArray *a, unsigned rank=-1, char cflag=0, unsigned k=-1
00210 ){ if (mxIsEmpty(a)) return 1;
00211     else return mxIsQSpace (NULL,0,a,rank,cflag,k);
00212 };
00213
00214 inline int mxIsQSpaceOrEmpty(const char *F, int L,
00215     const mxArray *a, unsigned rank=-1, char cflag=0, unsigned k=-1
00216 ){ if (mxIsEmpty(a)) return 1;
00217     else return mxIsQSpace (F,L,a,rank,cflag,k);
00218 };
00219
00220 QVec qsGetSym(
00221     const char *F, int L, const mxArray *a, unsigned k=-1);
00222
00223 char qsGotCGS(
00224     const char *F, int L, const mxArray *a, unsigned k=-1
00225 ){
00226     QVec qtype=qsGetSym(F_L,a,k);
00227     return qtype.permitsOM();
00228 };
00229
00230 #endif
00231

```

5.19 qspace_lib.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : QSpace
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_QSPACE_LIB_HCC__
00021 #define __WB_QSPACE_LIB_HCC__

```

```

00022
00023 /* =====
00024 * #include "qspace_lib.cc"
00025 *
00026 * contractIndex.cc (library file)
00027 *
00028 * C++ program that gets index sets for generic contractQ()
00029 *
00030 * This is a MEX-file for MATLAB.
00031 * AW (C) Jan 2006
00032 * ===== */
00033
00034 void check_qdim      (C_MEX *C, unsigned &ma, wbvector<widx_t> &dq);
00035 void check_cidx      (C_MEX *a, const unsigned &adim, UVEC &ica);
00036 void getPerm         (C_MEX *a, const unsigned m, wbvector<unsigned> &P);
00037 char isUInt          (const wbvector<double> &dd);
00038
00039 void getDataSize     (const mxArray *AD, const unsigned nd, const unsigned ma,
00040                      wbMatrix<unsigned> &usa);
00041
00042 void getQsub         (C_MEX *C, unsigned m, C_UVEC &dQ, C_UVEC &I, DMAT &Qc);
00043 void getQall         (C_MEX *A, wbMatrix<double> &Q);
00044 void getQall         (C_MEX *C, unsigned ma, C_UVEC &dq, wbMatrix<double> &Q);
00045 void getQBlock       (C_MEX *C, C_UINT ic, DMAT &Qc);
00046 void cell2mat        (C_MEX *C, unsigned *idx, unsigned n, DMAT &Qc);
00047
00048 char isUnique        (wbMatrix<double> Q);
00049 char isUniqueSorted  (C_DMAT &Q);
00050
00051 void makeUnique      (DMAT &Q, UVEC &Iu);
00052 void makeUnique      (DMAT &Q);
00053
00054 int matchSortedIdx_old(
00055     C_DMAT &QA, C_DMAT &QB,
00056     wbvector<widx_t> &Ia, wbvector<widx_t> &Ib
00057 );
00058
00059 int matchIndex_old(C_DMAT &QA, C_DMAT &QB,
00060     wbvector<widx_t> &Ia, wbvector<widx_t> &Ib
00061 );
00062
00063 void QtoCell         (C_DMAT &Q, C_UVEC &dab, mxArray *&S, int fieldnr,
00064                      C_UVEC &P = wbvector<unsigned>());
00065 void Data2vec        (mxArray *AD, C_IMAT sda, wbvector<double> &Avec);
00066 void Sab2Cell        (C_IMAT &SA, C_IMAT &SB, C_UVEC &Ia, C_UVEC &Ib,
00067                      C_UVEC &Iu, C_UVEC &ika, C_UVEC &ikb,
00068                      mxArray* C, int fid1, int fid2);
00069
00070 void DtoCell         (C_IMAT &SA, C_IMAT &SB, C_UVEC &Ia, C_UVEC &Ib,
00071                      C_UMAT &IG, C_UVEC &ika, C_UVEC &ikb,
00072                      mxArray* C, int fid1, int fid2);
00073
00074 void check_qdim(
00075     const mxArray *C,
00076     unsigned &ma,
00077     wbvector<widx_t> &dq
00078 ){
00079     mxArray *a;
00080     unsigned k,da;
00081
00082     if (!C || !mxIsCell(C) || mxIsEmpty(C))
00083         usage(FLINE, "Input arguments 1 and 2 must be cell arrays.");
00084
00085     if (mxGetM(C)!=1)
00086         usage(FLINE, "Input arguments 1 and 2 must be cell arrays (row vectors).");
00087
00088     da=mxGetN(C); dq.init(da);
00089
00090     a=mxGetCell(C,0); dq[0]=mxGetN(a);
00091     ma=mxGetM(a);
00092
00093     for (k=1; k<da; k++) {
00094         a=mxGetCell(C,k); dq[k]=mxGetN(a);
00095
00096         if (ma!=(unsigned)mxGetM(a))
00097             usage(FLINE, "Q Space inconsistency in input arguments.");
00098
00099         if (mxGetClassID(a)!=mxDOUBLE_CLASS)
00100             usage(FLINE, "Cell array of type double as input required.");
00101     }
00102 }
00103
00104 void check_cidx(
00105     const mxArray *a, const unsigned &adim, wbvector<unsigned> &ica) {
00106
00107     int e=0;
00108     try {

```



```

00109         Mx::Array<unsigned> A(a); ica.init(A.len);
00110         if (A.len) {
00111             ica.init(A.len); A.copyTo(ica.data,'t');
00112         }
00113     }
00114     catch (...) { ++e; }
00115     if (e || !ica.len) usage(FLINE,"input arguments 3 and 4 must be vectors");
00116
00117     unsigned i=0, n=ica.max(); char mark[n];
00118
00119     for (; i<ica.len; ++i) {
00120         if (--ica[i]>n || ++mark[ica[i]]>1) wblog(FL,
00121             "ERR invalid contraction index (unique, 1-based)");
00122     }
00123 };
00124
00125 void getPerm(
00126     const mxArray *a, const unsigned m0, wbvector<unsigned> &P) {
00127
00128     Mx::Array<unsigned> A(a);
00129
00130     if (!A.len) { P.init(); return; }
00131     if (A.len!=m0) {
00132         sprintf_str("permutation dimension mismatch (%ld,%d)",A.len,m0);
00133         usage(FLINE, str);
00134     }
00135
00136     P.init(A.len); A.copyTo(P.data,'t');
00137     P-=1;
00138
00139     if (!validPerm(P)) {
00140         P.print("P"); wbdie(FLINE,"invalid permuation");
00141     }
00142 };
00143
00144 char isUInt(const wbvector<double> &dd) {
00145
00146     double di;
00147     for (unsigned i=0; i<dd.len; i++) {
00148         di=dd[i];
00149         if (floor(di)!=di) return 0;
00150         if (di<0) return 0;
00151     }
00152     return 1;
00153 }
00154
00155 void getDataSize(
00156     const mxArray *AD,
00157     const unsigned nd,
00158     const unsigned ma,
00159     wbMatrix<int> &sa
00160 ){
00161     unsigned i,n;
00162     mxArray *a;
00163
00164     if (!mxIsCell(AD))
00165         wblog(FLINE,"ERR Data must be cell structure.");
00166
00167     if (ma!=(n=(unsigned)mxGetNumberOfElements(AD)))
00168         wblog(FLINE,"ERR Dimension mismatch between Q and data (%d,%d)",ma,n);
00169
00170     if (mxGetM(AD)!=1 && mxGetN(AD)!=1)
00171         wblog(FLINE,"ERR Q must be vectorized cell structure");
00172
00173     sa.init(ma,nd); sa.set(1);
00174
00175     for (i=0; i<ma; i++) {
00176         a=mxGetCell(AD,i);
00177
00178         n=mxGetNumberOfDimensions(a);
00179         if (n>nd) wblog(FLINE,"ERR Dimension mismatch in data (%d,%d)",n,nd);
00180
00181         memcpy(sa.rec(i), mxGetDimensions(a), n*sizeof(int));
00182     }
00183
00184     return;
00185 }
00186
00187 void getQsub(
00188     const mxArray *C,
00189     unsigned ma,
00190     const wbvector<unsigned> &dQ,
00191     const wbvector<unsigned> &I,
00192     wbMatrix<double> &Qc
00193 ){
00194     unsigned i,j,k,m,ib,nq,nC=mxGetNumberOfElements(C);
00195     double *qik;

```

```

00196
00197     for (m=ib=0; ib<I.len; ib++) m += dQ[I[ib]];
00198
00199     Qc.init(ma,m);
00200
00201     for (k=ib=0; ib<I.len; ib++) {
00202         if (I[ib]>=nC) wbdie(FLINE,"index out of bounds");
00203
00204         Mx::Array<double> A(FL,mxGetCell(C,I[ib]));
00205         nq=dQ[I[ib]];
00206
00207         for (i=0; i<ma; ++i) { qik=Qc.rec(i)+k;
00208             for (j=0; j<nq; ++j) { qik[j]=A.data[i+ma*j]; }}
00209
00210         k+=nq;
00211     }
00212 };
00213
00214 void getQall(
00215     const mxArray *A,
00216     wbMatrix<double> &Q
00217 ){
00218     unsigned ma;
00219     int iQ;
00220
00221     mxArray *AQ;
00222     wbvector<widx_t> dqa;
00223
00224     if (!mxIsStruct(A)) usage(FLINE,
00225         "Input argument must be structure {Q [,data,...]}.");
00226
00227     if (mxGetNumberOfElements(A)!=1) usage(FLINE,
00228         "Input argument must be SINGLE structure {Q [,data,...]}.");
00229
00230     iQ=mxGetFieldNumber(A,"Q");
00231     if (iQ<0) usage(FLINE,
00232         "Input argument must be structure of the type {Q [,data,...]}.");
00233
00234     AQ=mxGetFieldByNumber(A,0,iQ);
00235
00236     check_qdim(AQ, ma, dqa);
00237
00238     getQall(AQ, ma, dqa, Q);
00239
00240     return;
00241 }
00242
00243 void getQall(
00244     const mxArray *C,
00245     unsigned ma,
00246     const wbvector<unsigned> &dq,
00247     wbMatrix<double> &Q
00248 ){
00249     unsigned i, j, k, D, ib, d2;
00250     double *q;
00251
00252     D=dq.sum(); Q.init(ma,D);
00253
00254     if (dq.len!=(unsigned)mxGetNumberOfElements(C))
00255         wbdie(FLINE,"Dimension mismatch.");
00256
00257     for (k=ib=0; ib<dq.len; ++ib) {
00258         Mx::Array<double> A(FL,mxGetCell(C,ib));
00259         d2=dq[ib];
00260
00261         for (i=0; i<ma; ++i) { q=Q.rec(i)+k;
00262             for (j=0; j<d2; ++j) { q[j]=A.data[i+ma*j]; }}
00263
00264         k+=d2;
00265     }
00266 };
00267
00268 void getQBlock(
00269     const mxArray *C,
00270     const unsigned ic,
00271     wbMatrix<double> &Qc
00272 ){
00273     unsigned i,j; double *qi;
00274
00275     if (ic>=(j=mxGetNumberOfElements(C))) wblog(FL,
00276         "ERR %s() index out of bounds (%d/%d)",FCT,ic,j);
00277
00278     Mx::Array<double> A(FL,mxGetCell(C,ic));
00279     if (A.rank!=2) wblog(FL,"ERR %s() invalid rank-%d object",FCT,A.rank);
00280
00281     const size_t *sz=mxGetDimensions(A.ax);
00282     Qc.init(sz[0],sz[1]);

```

```

00283
00284     for (i=0; i<sz[0]; ++i) { qi=Qc.rec(i);
00285     for (j=0; j<sz[1]; ++j) { qi[j]=A.data[i+sz[0]*j]; }}
00286
00287     return;
00288 };
00289
00290 void cell2mat(
00291     const mxArray *C,
00292     const wbMatrix<unsigned> &I,
00293     wbMatrix<double> &M
00294 ){
00295     unsigned i,j,k,d1,d2,N,r0,s0,r,s;
00296     double *d;
00297     mxArray *a;
00298
00299     wbvector<unsigned> D1(I.dim1), D2(I.dim2);
00300     wbMatrix<double*> dd(I.dim1, I.dim2);
00301
00302     N=(unsigned)mxGetNumberOfElements(C);
00303
00304     for (i=0; i<I.dim1; i++)
00305     for (j=0; j<I.dim2; j++) {
00306         k=I(i,j);
00307         if (k>=N) wbdie(FLINE, "Index out of bounds.");
00308         a=mxGetCell(C,k);
00309
00310         if (i==0) D2[j]=(unsigned)mxGetN(a);
00311         else if (D2[j]!=(unsigned)mxGetN(a))
00312             wbdie(FLINE, "Dimension mismatch.");
00313
00314         if (j==0) D1[i]=(unsigned)mxGetM(a);
00315         else if (D1[i]!=(unsigned)mxGetM(a))
00316             wbdie(FLINE, "Dimension mismatch.");
00317
00318         dd(i,j)=Mx::Array<double>(FL,a).data;
00319     }
00320
00321     M.init(D1.sum(), D2.sum());
00322
00323     for (r0=i=0; i<I.dim1; i++) {
00324         d1=D1[i];
00325         for (s0=j=0; j<I.dim2; j++) {
00326             d2=D2[j]; d=dd(i,j);
00327
00328             for (k=s=0; s<d2; s++)
00329                 for ( r=0; r<d1; r++) M(r0+r,s0+s)=d[k++];
00330
00331             s0+=d2;
00332         }
00333         r0+=d1;
00334     }
00335 };
00336
00337 char isUnique(wbMatrix<double> Q) {
00338
00339     wbperm iS;
00340     Q.SortRecs(iS);
00341     return isUniqueSorted(Q);
00342 }
00343
00344 char isUniqueSorted(const wbMatrix<double> &Q) {
00345
00346     unsigned j;
00347
00348     for (j=1; j<Q.dim1; j++)
00349         if (Q.recEqual(j-1,j)) return 0;
00350
00351     return 1;
00352 }
00353
00354 void makeUnique(wbMatrix<double> &Q, wbvector<unsigned> &Iu) {
00355
00356     unsigned i,k,m,ig,d, ng;
00357     wbvector<wid_x_t> dQ;
00358     wbperm iS;
00359
00360     Q.groupRecs(iS,dQ); ng=dQ.len;
00361
00362     Iu.init(iS.len+ng);
00363
00364     for (k=ig=0; ig<ng; ig++) {
00365         d=dQ[ig];
00366         Iu[k+ig]=d; m=ig+1;
00367         for (i=0; i<d; i++, k++) Iu[k+m]=iS[k];
00368     }
00369 }

```

```

00370
00371 void makeUnique(wbMatrix<double> &Q) {
00372     wbperm is; wbvector<widx_t> dQ;
00373     Q.groupRecs(is,dQ);
00374 }
00375
00376 int matchIndex_old(
00377     const wbMatrix<double> &QA,
00378     const wbMatrix<double> &QB,
00379     wbvector<widx_t> &Ia,
00380     wbvector<widx_t> &Ib
00381 ){
00382     wbMatrix<double> Q1(QA), Q2(QB);
00383     wbvector<widx_t> ix1, ix2;
00384     wbperm is1, is2;
00385
00386     Q1.SortRecs(is1);
00387     Q2.SortRecs(is2);
00388
00389     matchSortedIdx_old(Q1,Q2,ix1,ix2);
00390
00391     is1.get(ix1,Ia);
00392     is2.get(ix2,Ib);
00393
00394     return 0;
00395 }
00396
00397 int matchSortedIdx_old(
00398     const wbMatrix<double> &QA,
00399     const wbMatrix<double> &QB,
00400     wbvector<widx_t> &Ia,
00401     wbvector<widx_t> &Ib
00402 ){
00403
00404     unsigned i,j,k, ng, ia,ib, ig, iga, igb, m1,m2, mtot=0;
00405     unsigned *p1, *p2, na=QA.dim1, nb=QB.dim1;
00406     char c;
00407
00408     if (QA.dim2!=QB.dim2)
00409         wbdie(FLINE, "Dimension mismatch");
00410
00411     wbvector<unsigned> gra(2*na), grb(2*nb);
00412
00413     for (ng=iga=igb=ia=ib=0; ia<na && ib<nb;) {
00414         c=QA.recCompareP(ia, QB.rec(ib));
00415         if (c<0) ia++; else
00416             if (c>0) ib++;
00417         else {
00418             p1=gra.data+(iga++); gra[iga++]=(ia++);
00419             p2=grb.data+(igb++); grb[igb++]=(ib++);
00420
00421             m1=m2=1; ng++;
00422             while (ia<na) {
00423                 if (QA.recEqual(ia-1, ia)) { gra[iga++]=(ia++); m1++; }
00424                 else break;
00425             }
00426             while (ib<nb) {
00427                 if (QB.recEqual(ib-1, ib)) { grb[igb++]=(ib++); m2++; }
00428                 else break;
00429             }
00430             mtot+=m1*m2; (*p1)=m1; (*p2)=m2;
00431         }
00432     }
00433
00434     Ia.init(mtot); Ib.init(mtot);
00435
00436     for (k=ig=iga=igb=0; ig<ng; ig++) {
00437         m1=gra[iga]; p1=gra.ref(iga+1); iga+=(m1+1);
00438         m2=grb[igb]; p2=grb.ref(igb+1); igb+=(m2+1);
00439         for (i=0; i<m1; i++)
00440             for (j=0; j<m2; j++,k++) {
00441                 Ia[k]=p1[i];
00442                 Ib[k]=p2[j];
00443             }
00444     }
00445
00446     return 0;
00447 }
00448
00449 int matchUSortedIdx(
00450     const wbMatrix<double> &QA,
00451     const wbMatrix<double> &QB,
00452     wbvector<unsigned> &Ia,
00453     wbvector<unsigned> &Ib
00454 ){
00455
00456     unsigned n,ia,ib; char c;

```

```

00457
00458     if (QA.dim2!=QB.dim2)
00459         wbdie(FLINE, "Dimension mismatch");
00460
00461     #ifdef __WBDEBUG__
00462         if (!QA.isUniqueSorted('A') || !QB.isUniqueSorted('A'))
00463             wbdie(FLINE, "ERR Input must be unique increasing set of records.");
00464     #endif
00465
00466     n=MIN(QA.dim1,QB.dim1); Ia.init(n); Ib.init(n);
00467
00468     for (n=ia=ib=0; ia<QA.dim1 && ib<QB.dim1 ;) {
00469
00470         c=QA.recCompareP(ia, QB.rec(ib));
00471
00472         if (c<0) ia++; else
00473             if (c>0) ib++;
00474         else {
00475             Ia[n]=ia;
00476             Ib[n]=ib; ia++; ib++; n++;
00477         }
00478     }
00479
00480     Ia.Resize(n); Ib.Resize(n);
00481
00482     return 0;
00483 }
00484
00485 void Sab2Cell(
00486     const wbMatrix<int> &SA,
00487     const wbMatrix<int> &SB,
00488     const wbvector<unsigned> &Ia,
00489     const wbvector<unsigned> &Ib,
00490     const wbvector<unsigned> &Id,
00491     const wbvector<unsigned> &ika,
00492     const wbvector<unsigned> &ikb,
00493     mxArray* ST, int fid_SAB, int fid_Sab
00494 ){
00495     unsigned i,k,l,ia,ib,ng=Id.len,d2=ika.len+ikb.len;
00496     wbvector<int> ss, sk(d2);
00497     double *d, *SD, *sd;
00498
00499     mxSetFieldByNumber(ST,0,fid_SAB, Mx::Array<double>( 2,ng).Return(SD));
00500     mxSetFieldByNumber(ST,0,fid_Sab, Mx::Array<double>(d2,ng).Return(sd));
00501
00502     for (l=k=0; k<ng; k++) {
00503
00504         ia=Ia[l]; ib=Ib[l]; l+=Id[k];
00505
00506         if (ia>=SA.dim1 || ib>=SB.dim1)
00507             wbdie(FLINE, "Index out of bounds.");
00508
00509         ss.init(SA.dim2, SA.rec(ia)); ss.get(ika, sk.data);
00510         ss.init(SB.dim2, SB.rec(ib)); ss.get(ikb, sk.data+ika.len);
00511
00512         i=k*2;
00513         SD[i ]=(double)sk.prod(unsigned(0),ika.len-1);
00514         SD[i+1]=(double)sk.prod(ika.len,sk.len-1);
00515
00516         d=sd+k*d2;
00517
00518         for (i=0; i<d2; i++) d[i]=(double)sk[i];
00519     }
00520
00521     return;
00522 }
00523
00524 void QtoCell(
00525     const wbMatrix<double> &Q,
00526     const wbvector<unsigned> &dab,
00527     mxArray *S,
00528     int field_nr,
00529     const wbvector<unsigned> &P
00530 ){
00531     unsigned i,j,k,m,d1=Q.dim1,d2=n=dab.len;
00532     double *dd;
00533     mxArray *C;
00534
00535     wbvector<unsigned> D2(n);
00536
00537     if (field_nr>=0) {
00538         mxSetFieldByNumber(S,0,field_nr, mxCreateCellMatrix(1,n));
00539         C=mxGetFieldByNumber(S,0,field_nr);
00540     } else { S=mxCreateCellMatrix(1,n); C=S; }
00541
00542     if (Q.dim1==0 || dab.len==0) {
00543         for (i=0; i<n; i++)

```

```

00544         mxSetCell(C,i,mxCreateDoubleMatrix(0,dab[i],mxREAL));
00545         return;
00546     }
00547
00548     D2[0]=0; for (j=1; j<n; j++) D2[j]=D2[j-1]+dab[j-1];
00549
00550     if (D2[j-1]+dab[j-1]!=Q.dim2) wbdie(FLINE, "Dimension mismatch.");
00551     if (P.len && P.len!=n) wbdie(FLINE, "Invalid permutation.");
00552
00553     for (k=0; k<n; k++) {
00554         d2=dab[k];
00555
00556         mxSetCell(C,k, Mx::Array<double>(d1,d2).Return(dd));
00557         m = P.len ? D2[P[k]] : D2[k];
00558
00559         for (j=0; j<d2; ++j)
00560             for (i=0; i<d1; ++i) { dd[d1*j+i]=Q(i,m+j); }
00561     }
00562 };
00563
00564 void Data2vec(
00565     mxArray *AD, const wbMatrix<int> sda, wbvector<double> &Avec) {
00566
00567     unsigned i,d,k,N;
00568     wbvector<int> SD;
00569
00570     if (sda.dim1==0 || sda.dim2==0) { Avec.init(); return; }
00571
00572     SD.init(sda.dim1);
00573     for (i=0; i<SD.len; i++) SD[i]=sda.recProd(i);
00574
00575     N=(unsigned)SD.sum();
00576
00577     Avec.init(N);
00578     for (k=i=0; i<SD.len; ++i) { d=(unsigned)SD[i];
00579         Mx::Array<double> A(FL,mxGetCell(AD,i));
00580         memcpy(Avec.data+k, A.data, d*sizeof(double));
00581         k+=d;
00582     }
00583 };
00584
00585 void QI2Cell(
00586     const wbMatrix<double> &Q,
00587     const mxArray *AQ,
00588     mxArray* C,
00589     int field_nr,
00590     const wbvector<widx_t> &P
00591 ){
00592     unsigned i,m,M;
00593     wbvector<widx_t> iP, dqa, iv;
00594     wbindex iq,ir;
00595
00596     wbMatrix<double> Qref;
00597
00598     if (Q.data==NULL) {
00599         mxSetFieldByNumber(C, 0, field_nr,
00600             mxCreateDoubleMatrix(0,0,mxREAL)); return;
00601     }
00602
00603     getIPerm(P,iP);
00604
00605     check_qdim(AQ,m,dqa);
00606     getQsub (AQ,m,dqa, iP, Qref);
00607
00608     M=Q.dim1; iv.init(M);
00609
00610     if (Q.dim2==Qref.dim2) {
00611         matchIndex(Q,Qref,iq,ir); m=iq.len;
00612         for (i=0; i<m; i++) iv[iq[i]]=ir[i]+1;
00613     }
00614
00615     mxSetFieldByNumber(C,0,field_nr,
00616         Mx::Array<double>(M,1).copyFromTR(iv.data)
00617     );
00618 };
00619 };
00620
00621 void DtoCell(
00622     const wbMatrix<int> &SA,
00623     const wbMatrix<int> &SB,
00624     const wbvector<unsigned> &Ia,
00625     const wbvector<unsigned> &Ib,
00626     const wbMatrix<unsigned> &IG,
00627     const wbvector<unsigned> &ika,
00628     const wbvector<unsigned> &ikb,
00629     mxArray* ST, int fid1, int fid2
00630 ){

```

```

00631     unsigned k,l,ia,ib,ng=IG.dim1;
00632     wbvector<size_t> uu, sk(ika.len+ikb.len);
00633
00634     mxArray *D=mxCreateCellMatrix(ng,1);
00635     double *sd;
00636
00637     mxSetFieldByNumber(ST,0,fid1, D);
00638     mxSetFieldByNumber(ST,0,fid2, Mx::Array<double>(ng,2).Return(sd));
00639
00640     for (k=0; k<ng; ++k) {
00641
00642         l=IG(k,1); ia=Ia[l]; ib=Ib[l];
00643
00644         if (ia>=SA.dim1 || ib>=SB.dim1)
00645             wbdie(FLINE,"Index out of bounds.");
00646
00647         uu.initT(SA.dim2, SA.rec(ia)); uu.get(ika, sk.data);
00648         uu.initT(SB.dim2, SB.rec(ib)); uu.get(ikb, sk.data+ika.len);
00649
00650         sd[k] = (double)sk.prod(unsigned(0),ika.len-1);
00651         sd[k+ng] = (double)sk.prod(ika.len,sk.len-1);
00652
00653         mxSetCell(D,k, Mx::Array<double>(sk).Return());
00654     };
00655 };
00656
00657 void Sab2Site2Cell(
00658     const wbMatrix<int> &SA,
00659     const wbMatrix<int> &SB,
00660     const wbvector<unsigned> &Ia,
00661     const wbvector<unsigned> &Ib,
00662     const wbMatrix<unsigned> &IG,
00663     const wbvector<unsigned> &ika,
00664     const wbvector<unsigned> &ikb,
00665     const wbvector<unsigned> &ikP,
00666     mxArray* ST, int fid_Sab, int fid_SAB
00667 ){
00668     unsigned k,l,ia,ib, ng=IG.dim1, dk=ika.len+ikb.len; int *s1,*s2;
00669     wbMatrix<int> sd(ng,dk);
00670     wbvector<int> uu;
00671     double *SD;
00672
00673     mxSetFieldByNumber(ST,0,fid_SAB, Mx::Array<double>(2,ng).Return(SD));
00674
00675     for (k=0; k<ng; ++k) {
00676
00677         l=IG(k,1); ia=Ia[l]; ib=Ib[l];
00678
00679         if (ia>=SA.dim1 || ib>=SB.dim1)
00680             wbdie(FLINE,"Index out of bounds.");
00681
00682         s1=sd.rec(k); s2=s1+ika.len;
00683
00684         uu.init(SA.dim2, SA.rec(ia)); uu.get(ika, s1);
00685         uu.init(SB.dim2, SB.rec(ib)); uu.get(ikb, s2);
00686
00687         SD[2*k] = (double)Wb::prodRange(s1,ika.len);
00688         SD[2*k+1] = (double)Wb::prodRange(s2,ikb.len);
00689     }
00690
00691     sd.colPermute(ikP);
00692     sd.mat2mxs(ST,fid_Sab);
00693 };
00694
00695 #endif
00696

```

5.20 spectral.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : QSpace NRG routines
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,

```

```

00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the license for the specific language governing permissions and
00017  * limitations under the license.
00018  * ----- */
00019
00020 #ifndef __WB_SPECTRAL_HCC__
00021 #define __WB_SPECTRAL_HCC__
00022
00023 // default smoothing: norm-preserving log-gauss broadening
00024 #define LOG_GAUSS_BRD 2
00025
00026 namespace Wb {
00027
00028 template <class TS>
00029 void Fourier(
00030     const wbvector<double> &om,
00031     const wbMatrix<TS> &A,
00032     const wbvector<double> &tt,
00033     wbMatrix<wbcomplex> &az,
00034     char domflag=0,
00035     double alpha=0.,
00036     char vflag=1
00037 );
00038
00039 template <class TS>
00040 void dispSumRule(
00041     const wbvector<double> &om,
00042     const wbMatrix<TS> &A
00043 );
00044
00045 };
00046
00047 void KKimag(
00048     const wbMatrix<double> &Rp,
00049     const wbMatrix<double> &Rn,
00050     const wbvector<double> &OM,
00051     wbMatrix<double> &Ip,
00052     wbMatrix<double> &In
00053 );
00054
00055 template <class TS>
00056 class Spectral {
00057 public:
00058
00059     Spectral()
00060         : sigma(0.6), sigma2(0.5), alpha(0.25), Emin(1E-8), Emax(10.),
00061           eps(1E-6), nlog(128), l0(0), fac(1.), Delta(0.) {};
00062
00063     void init() {
00064         Ap.init(); An.init(); Ap_buf.init(); An_buf.init();
00065         Ar.init(); A0.init(); mspec.init();
00066         istr.init();
00067     };
00068
00069     void init(const char *istr, unsigned Nops,
00070             double delta=0., double T_=0, unsigned Niter=0
00071     );
00072
00073     void initBuf() {
00074         if (Ap.dim1!=An.dim1) wblog(FL,
00075             "ERR severe dimension mismatch (%d,%d).", Ap.dim1, An.dim1);
00076
00077         Ap_buf.init(Ap.dim1, Ap.dim2);
00078         An_buf.init(An.dim1, An.dim2);
00079     };
00080
00081     void calcSpectralMoments(const char *F, int L, unsigned n) {
00082         if (n) {
00083             if (n>4) wblog(F,L,"WRN %s with n=%d !?",FCT,n);
00084             else wblog(F,L,"<i> calculate %d spectral moment(s)",n);
00085             mspec.init(Ap.dim1,n+1);
00086         }
00087     };
00088
00089     void Add(
00090         const wbararray<double> &Om, const wbararray<TS> &CC, unsigned s,
00091         char dblock=0,
00092         char mflag=0
00093     );
00094
00095     void Add2buf(const wbararray<double> &Om, const wbararray<TS> &CC, unsigned s);
00096
00097     void crossAddBuf(const char minor=1, char vflag=1);
00098
00099     void saveIter(const char *F, int L, unsigned iter);
00100
00101     void getOmega(wbvector<double> &om, unsigned &linfit) const;

```



```

00102 void getOmega(wbvector<double> &om) const {
00103     unsigned linfit=0; getOmega(om,linfit);
00104 };
00105
00106 void applyIROPfac(const char *F, int L, const wbvector<double> &fac);
00107
00108 void getRawData(
00109     wbvector<double> &om, wbMatrix<TS> &A,
00110     unsigned linfit=0, const char *isbuf=""
00111 ) const;
00112
00113 void getPARTIAL(
00114     wbarray<TS> &A1, wbarray<TS> &A2) const;
00115
00116 void pairRawData();
00117
00118 void detailedBalance(
00119     const double T, wbvector<double> &om, wbMatrix<TS> &aa) const;
00120
00121 void dispSumRule();
00122 void dispSumRule(const wbvector<double> &om, const wbMatrix<TS> &A);
00123
00124 void getSmoothSpec(
00125     wbvector<double> &om, wbMatrix<TS> &A, char vflag=1);
00126
00127 void getSmoothSpec(
00128     wbMatrix<TS> &A, const wbvector<double> &om, char vflag=1);
00129
00130 void KKreal(
00131     wbMatrix<wbcomplex> &GZ,
00132     wbvector<double> &OM,
00133     wbvector<double> &DOM
00134 ) const;
00135
00136 wbMatrix<TS> Ap, An;
00137 wbMatrix<TS> Ap_buf, An_buf;
00138 wbvector<TS> Ar;
00139 wbMatrix<TS> AO;
00140 wbMatrix<TS> mspec;
00141
00142 wbarray<TS> AP, AN;
00143
00144 wbstring istr;
00145 double Temp;
00146
00147 double sigma, sigma2, alpha, Emin, Emax, eps;
00148 unsigned nlog;
00149
00150 protected:
00151 private:
00152
00153     double l0, fac, Delta;
00154
00155     double OM2IDX(const double &om) const {
00156         return ((log(fabs(om))-l0)*fac);
00157     }
00158
00159     inline unsigned om2idx(double om) const {
00160         if (Delta==0) { if (om==0.) return 0; }
00161         else {
00162             if (om> Delta) om-=Delta; else
00163             if (om<-Delta) om+=Delta; else return 0;
00164         }
00165
00166         int k = (int)floor(OM2IDX(om));
00167         if (k<1) k=1; else
00168         if (k>=(int)Ap.dim2) { k=Ap.dim2-1; }
00169
00170         return (unsigned)k;
00171     }
00172
00173     inline double idx2om(const unsigned &k) const {
00174         return exp( (double(k)+0.5)/fac + l0 );
00175     }
00176
00177     int crossAddBuf_aux(
00178         TS *a, const TS *b, const unsigned M,
00179         const char minor=1, char vflag=1
00180     );
00181
00182     TS foldLogGaussian(
00183         const double om, const TS *aa, const unsigned n
00184     );
00185
00186     TS foldGaussian(
00187         const double om, const double b,
00188         const double *oo, const TS *aa, const unsigned n

```

```

00189     );
00190 };
00191
00192 template <class TS>
00193 class RSpecData {
00194 public:
00195     RSpecData() { };
00196
00197     RSpecData& Append(const double *op, const TS *cp, unsigned n) {
00198         om.Append(n,op);
00199         cc.Append(n,cp); return *this;
00200     };
00201
00202     mxArray* mxCreateStruct(unsigned m, unsigned n) const {
00203         const char *flds[]={ "om","cc"};
00204         return mxCreateStructMatrix(m,n,2,flds);
00205     };
00206     void add2MxStruct(mxArray *S, unsigned i, char tflag=0) const {
00207         mxSetFieldByNumber(S,i,0,om.toMx(tflag));
00208         mxSetFieldByNumber(S,i,1,cc.toMx(tflag));
00209     };
00210
00211     wbvector<double> om;
00212     wbvector<TS> cc;
00213
00214 protected:
00215 private:
00216
00217 };
00218
00219 template <class TS>
00220 class TDSpectral {
00221 public:
00222
00223     TDSpectral() : raw(0), l0(0), fac(1.), check_avg_om(0) {};
00224
00225     void init(const char *F, int L,
00226             const char *istr0, const wbvector<double> &t0,
00227             unsigned Nnrg, unsigned Nop,
00228             double Emin=1E-8, double Emax=10., unsigned nl=512,
00229             char vflag=1
00230             );
00231
00232     void checkAvgOm() {
00233         check_avg_om=1;
00234         AW.init(2,AP.dim(2),AP.dim(3));
00235     };
00236     wbarray<TS> getAvgOm();
00237
00238     void Add(
00239         const wbarray<double> &Om, const wbarray<TS> &CC,
00240         unsigned iter, unsigned s
00241     );
00242
00243     void addPartialFourier();
00244
00245     void getOmega(wbvector<double> &om, unsigned &linfit) const;
00246     void getOmega(wbvector<double> &om) const {
00247         unsigned linfit=0; getOmega(om,linfit);
00248     }
00249
00250     void applyIROPFac(
00251         const char *F, int L, const wbvector<double> &fac, char rixs=0);
00252
00253     void getSpecData(
00254         wbvector<double> &om, wbarray<TS> &A,
00255         unsigned linfit=0
00256     ) const;
00257
00258     void getSpecData(
00259         wbvector<double> &om, wbMatrix<TS> &A,
00260         unsigned linfit=0
00261     );
00262
00263     void mergeNRGData();
00264
00265     void getSmoothSpec_t(double sigma, double eps,
00266         char bswitch = LOG_GAUSS_BRD, char vflag=1);
00267
00268     void Fourier(
00269         wbvector<double> &om, wbMatrix<wbcomplex> &ST,
00270         double alpha, double Lambda, char vflag=1
00271     );
00272
00273     void Fourier(
00274         wbvector<double> &om, wbMatrix<TS> &A,
00275

```

```

00276     const wbvector<double> &tt,
00277     wbMatrix<wbcomplex> &at,
00278     double alpha=0., char vflag=1
00279 );
00280
00281 void dispSumRule();
00282
00283 wbararray<TS> AP,AN;
00284 wbMatrix<TS> Ap,An;
00285 wbMatrix<wbcomplex> At;
00286 wbvector<double> tt;
00287 wbstring istr;
00288
00289 mxArray *toMx() const;
00290 int init(
00291     const mxArray *S,
00292     double &emin, double &emax, unsigned &nlog
00293 );
00294
00295 unsigned raw;
00296 wbvector< RSpecData<TS> > RAW;
00297
00298 protected:
00299 private:
00300
00301     double l0,fac;
00302     unsigned N;
00303
00304     char check_avg_om;
00305     wbararray<TS> AW;
00306
00307     double OM2IDX(const double &om) const {
00308         return ((log(fabs(om))-10)*fac);
00309     }
00310
00311     inline unsigned om2idx(const double &om) const {
00312         if (om==0.) return 0;
00313
00314         int k = (int)floor(OM2IDX(om));
00315         if (k<1) k=1; else if (k>=(int)N) k=N-1;
00316
00317         return (unsigned)k;
00318     }
00319
00320     inline double idx2om(const unsigned &k) const {
00321         return exp( ((double)k+0.5)/fac + 10 );
00322     }
00323
00324     void initLG(
00325         wbMatrix<double> &wlg, unsigned n, double sigma,
00326         char bswitch = LOG_GAUSS_BRD, char vflag=1
00327     );
00328
00329     void foldLogGaussian(
00330         const wbMatrix<double> &wlg,
00331         TS *aa, TS a0, unsigned i0
00332     );
00333 };
00334
00335 template <class TS>
00336 void Spectral<TS>::init (
00337     const char *istr0,
00338     unsigned Nops,
00339     double d,
00340     double T,
00341     unsigned Niter
00342 ){
00343     unsigned Nom;
00344     double l2;
00345
00346     if (Emin<=0. || Emax<=0. || Emin>=0.1*Emax || nlog==0) wblog(FL,
00347         "ERR %s() invalid range E = [%g .. %g @ %d]%s", myname, Emin, Emax, nlog,
00348         (Emin>0 && Emax>0 && Emin<Emax) ? " (require Emin<Emax/10)":"");
00349     if (d<0) wblog(FL,"ERR invalid Delta (%g neg!)",d);
00350
00351     if (eps<Emin) wblog(FL,"NB! eps=%g < min(|om|)=%g", eps, Emin);
00352     if (alpha!=0.25) {
00353         if (alpha==0.5) wblog(FL,
00354             "WRN %s() using symmetric log-Gaussian broadening",FCT);
00355         else wblog(FL,"WRN %s() using "
00356             "log-Gaussian with shift alpha=%g !?",FCT,alpha);
00357     }
00358
00359     Delta=d;
00360
00361     l0 =log(Emin);
00362     l2 =log(Emax);

```

```

00363     fac=log10(M_E)*double(nlog);
00364     Nom=(unsigned)((l2-l0)*fac)+1;
00365
00366     Ap.init(Nops,Nom);
00367     An.init(Nops,Nom); Ar.init(Nops);
00368
00369     if (int(Niter)>0) {
00370         AP.init(Nom,Nops,Niter);
00371         AN.init(Nom,Nops,Niter);
00372     }
00373
00374     istr=istr0; Temp=T;
00375 };
00376
00377 template <class TS>
00378 void Spectral<TS>::Add(
00379     const wbararray<double> &Om,
00380     const wbararray<TS> &CC,
00381     unsigned s,
00382     char dblock,
00383     char mflag
00384 ){
00385     unsigned i,k, n=Om.SIZE.prod();
00386     double x, emin=0.1*idx2om(0);
00387     TS q, r=0;
00388
00389     double bfac=(!mflag && Temp>0 ? 10/Temp : 0);
00390
00391     double zfac=(Temp>0 ? 1/Temp : 0);
00392
00393     if (!Om.sameSize(CC)) {
00394         Om.info("Om"); CC.info("CC");
00395         wblog(FL,"ERR size mismatch between Om and CC!");
00396     }
00397     if (dblock) {
00398         if (Om.SIZE.len!=2 || Om.SIZE[0]!=Om.SIZE[1]) wblog(FL,
00399             "ERR %s() quadratic block expected (%s)",
00400             FCT,Om.sizeStr().data);
00401     }
00402
00403     if (s>=Ap.dim1 || s>=Ar.len) wblog(FL,
00404         "ERR index out of bounds (%d/%d;%d)",s, Ap.dim1,Ar.len);
00405
00406     for (i=0; i<n; ++i) { x=Om.data[i]; k=om2idx(x);
00407         if (x<0. || (x<=emin && s%2))
00408             An(s,k)+=CC.data[i];
00409         else Ap(s,k)+=CC.data[i];
00410
00411         if (x!=0) {
00412             q=CC.data[i]/x;
00413             if (bfac>0) { x=bfac*fabs(x); if (x<10) {
00414                 double w=exp(-x*x);
00415                 q*=(1-w);
00416             }}
00417             r+=q;
00418         }
00419         else if (mflag && !(s%2)) {
00420             r+=zfac*CC.data[i];
00421         }
00422     }
00423
00424     Ar[s]+=r;
00425
00426     if (mspec.dim2) {
00427         double dbl; unsigned j;
00428
00429         if (mspec.dim1!=Ap.dim1) {
00430             wblog(FL,
00431                 "WRN mspec must be initialized AFTER ASpec!\n"
00432                 "resize (%d->%d x %d)", mspec.dim1, Ap.dim1, mspec.dim2
00433             );
00434             mspec.Resize(Ap.dim1,mspec.dim2);
00435         }
00436
00437         for (i=0; i<n; i++)
00438             for (dbl=CC.data[i], j=0; j<mspec.dim2; j++) {
00439                 if (j) dbl*=Om.data[i];
00440                 mspec(s,j)+=dbl;
00441             }
00442     }
00443 };
00444
00445 template <class TS>
00446 void Spectral<TS>::Add2buf(
00447     const wbararray<double> &Om, const wbararray<TS> &CC,
00448     unsigned s
00449 ){

```

```

00450     unsigned i,k,n=Om.SIZE.prod();
00451
00452     if (!Om.sameSize(CC)) {
00453         Om.info("Om"); CC.info("CC");
00454         wblog(FL, "ERR Size mismatch between Om and CC!");
00455     }
00456
00457     if (s>Ap_buf.dim1)
00458         wblog(FL, "ERR Index out of bounds (%d,%d)",s, Ap_buf.dim1);
00459
00460     for (i=0; i<n; i++) {
00461         k=om2idx(Om.data[i]);
00462         if (Om.data[i]<0.)
00463             An_buf(s,k)+=CC.data[i];
00464         else Ap_buf(s,k)+=CC.data[i];
00465     }
00466 };
00467
00468 template <class TS>
00469 void Spectral<TS>::crossAddBuf(
00470     const char minor,
00471     char vflag
00472 ){
00473     unsigned s, Nom=Ap.dim2;
00474     int i;
00475
00476     if (!Ap.sameSize(Ap_buf) || !An.sameSize(An_buf) ||
00477         !Ap.sameSize(An))
00478     ){
00479         Ap.info("Ap"); Ap_buf.info("Ap_buf");
00480         An.info("An"); An_buf.info("An_buf");
00481         wblog(FL, "ERR Spectral - severe size mismatch.");
00482     }
00483
00484     for (s=0; s<Ap.dim1; s++) { i=0;
00485
00486         i+=crossAddBuf_aux(Ap.rec(s), Ap_buf.rec(s), Nom,minor,vflag);
00487         i+=crossAddBuf_aux(An.rec(s), An_buf.rec(s), Nom,minor,vflag);
00488
00489         if (i>1) {
00490             wblog(FL, "WRN empty buffer (%d/%d) !?",s,Ap.dim1);
00491             continue;
00492         }
00493     }
00494 };
00495
00496 template <class TS>
00497 int Spectral<TS>::crossAddBuf_aux(
00498     TS *a,
00499     const TS *b,
00500     const unsigned Nom,
00501     const char minor,
00502     char vflag
00503 ){
00504     unsigned i, i1, i2, amin, amax, bmin, bmax;
00505     double t, tfac;
00506
00507     if (minor<1 || minor>2) wblog(FL,
00508         "ERR invalid minor flag %d", minor);
00509
00510     for (i=0; i<Nom; ++i) { if (a[i]!=0.) break; }; amin=i;
00511     for (i=Nom-1; i>=amin; --i) { if (a[i]!=0.) break; }; amax=i;
00512     for (i=0; i<Nom; ++i) { if (b[i]!=0.) break; }; bmin=i;
00513     for (i=Nom-1; i>=bmin; --i) { if (b[i]!=0.) break; }; bmax=i;
00514
00515     if (bmin>bmax) {
00516         return vflag;
00517     }
00518
00519     if (amin>amax) {
00520         for (i=bmin; i<=bmax; i++) a[i]=b[i];
00521         return 0;
00522     }
00523
00524     i1=MAX(amin,bmin); i2=MIN(amax,bmax);
00525
00526     if (i1==i2) { a[i1]=0.5*(a[i1]+b[i2]); return 0; }
00527
00528     tfac = 1. / double(i2-i1);
00529
00530     for (i=0; i<Nom; ++i) {
00531         if (i<i1 || i>i2) a[i]+=b[i];
00532         else {
00533             t = tfac * double(i-i1);
00534             if (minor==1)
00535                 a[i] = (1.-t)*a[i] + t*b[i];

```

```

00537         else a[i] = t*a[i] + (1.-t)*b[i];
00538     }
00539 }
00540
00541 return 0;
00542 };
00543
00544 template <class TS>
00545 void Spectral<TS>::saveIter(const char *F, int L, unsigned iter){
00546     unsigned n=Ap.numel();
00547
00548     if (AP.SIZE.len!=3 || AP.SIZE!=AN.SIZE || n!=AP.SIZE[0]*AP.SIZE[1])
00549         wblog(F_L,
00550             "ERR %s() invalid initialization of AP/AN [%s; %s; %dx%d]",
00551             FCT,AP.sizeStr().data, AN.sizeStr().data, Ap.dim1, Ap.dim2
00552         );
00553     if (iter>=AP.SIZE[2]) wblog(F_L,
00554         "ERR %s() iter out of bounds AP/AN [%s; %d/%d]",
00555         FCT,AP.sizeStr().data, iter+1, AP.SIZE[2]
00556     );
00557
00558     memcpy(AP.data+iter*n, Ap.data, n*sizeof(Ap.data[0]));
00559     memcpy(AN.data+iter*n, An.data, n*sizeof(An.data[0]));
00560 }
00561
00562 template <class TS>
00563 void Spectral<TS>::getOmega(wbvector<double> &om, unsigned &linfit) const {
00564     unsigned i, Nom=Ap.dim2;
00565     double dbl;
00566
00567     if (linfit) linfit=unsigned(fac);
00568     if (linfit>Nom) wblog(FL,"ERR Nom<linfit (%d,%d) !?",Nom,linfit);
00569
00570     om.init(2*Nom);
00571
00572     dbl = idx2om(linfit) / (linfit+0.5);
00573
00574     for (i=0; i<linfit; ++i) om[Nom-1-i] = -(om[Nom+i] = dbl*(i+0.5));
00575     for ( ; i<Nom; ++i) om[Nom-1-i] = -(om[Nom+i] = idx2om(i) );
00576 }
00577
00578 template <class TS>
00579 void Spectral<TS>::applyIROPfac(
00580     const char *F, int L, const wbvector<double> &fac) {
00581     unsigned i1,i2,s,m, n=fac.len;
00582     double x;
00583
00584     if (2*n!=Ap.dim1 || 2*n!=An.dim1 || 2*n!=Ar.len) wblog(FL,
00585         "ERR %s() size mismatch (%d,%d,%d/2*d)",
00586         FCT,Ap.dim1,An.dim1,Ar.len,n);
00587
00588     for (s=0; s<n; ++s) { if (fac[s]!=1) {
00589         if (fac[s]<1 || fac[s]!=round(fac[s])) wblog(FL,
00590             "ERR %s() got fac[%d]=%g !?",FCT,s+1,fac[s]);
00591         wblog(FL," * applying IROP factor 1/%g to op(%d)",fac[s],s+1);
00592
00593         x=1/fac[s]; i1=2*s; i2=2*s+1;
00594         Ap.rowTimes(i1,x); An.rowTimes(i1,x); Ar[i1]*=x;
00595         Ap.rowTimes(i2,x); An.rowTimes(i2,x); Ar[i2]*=x;
00596     }}
00597
00598     if (!Ap_buf.isEmpty() || !An_buf.isEmpty()) {
00599         if (!Ap_buf.sameSize(Ap) || !An_buf.sameSize(An)) wblog(FL,
00600             "ERR %s() severe size mismatch %dx%d <> %dx%d <> %dx%d !?", FCT,
00601             Ap_buf.dim1,Ap_buf.dim2,An_buf.dim1,An_buf.dim2,Ap.dim1,Ap.dim2
00602         );
00603         for (s=0; s<n; ++s) { if (fac[s]>1) { x=1/fac[s]; i1=2*s; i2=2*s+1;
00604             Ap_buf.rowTimes(i1,x); An_buf.rowTimes(i1,x);
00605             Ap_buf.rowTimes(i2,x); An_buf.rowTimes(i2,x);
00606         }}
00607     }
00608
00609     if (!AP.isEmpty() || !AN.isEmpty()) {
00610         if (AP.SIZE.len!=3 || !AP.sameSize(AN) || AP.SIZE[1]!=2*n)
00611             wblog(FL,
00612                 "ERR %s() size mismatch (%s; %s; %dx%d) !?", FCT,
00613                 AP.sizeStr().data, AN.sizeStr().data, Ap.dim1,Ap.dim2
00614             );
00615
00616         TS *ap=AP.data, *an=AN.data;
00617         unsigned n2=2*AP.SIZE[0]; m=n*AP.SIZE[2];
00618
00619         for (i1=0; i1<m; ++i1, ap+=n2, an+=n2) { s=i1/n;
00620             if (fac[s]>1) { x=1/fac[s];

```

```

00624         Wb::timesRange(ap, x, n2);
00625         Wb::timesRange(an, x, n2);
00626     }
00627 }
00628 }
00629
00630 if (!A0.isEmpty() || !mspec.isEmpty()) { wblog(FL,
00631     "WRN IROPfac yet not applied to A0 and mspec (%d,%d)",
00632     A0.isEmpty(), mspec.isEmpty());
00633     wblog(FL, " * A0: %dx%d", A0.dim1, A0.dim2);
00634     wblog(FL, " * sp: %dx%d", mspec.dim1, mspec.dim2);
00635 }
00636 };
00637
00638 template <class TS>
00639 void Spectral<TS>::getRawData(
00640     wvector<double> &om, wbMatrix<TS> &A, unsigned linfit,
00641     const char *isbuf
00642 ) const {
00643     unsigned i,s,Nom=Ap.dim2;
00644
00645     getOmega(om, linfit);
00646
00647     A.init(Ap.dim1, 2*Nom);
00648
00649     if (A.dim2!=om.len || !Ap.sameSize(An)) wblog(FL,
00650     "ERR severe dimension mismatch (%d,%d).", A.dim2, om.len);
00651
00652     if (isbuf && isbuf[0]) {
00653         if (strcmp(isbuf,"buf"))
00654             wblog(FL, "ERR invalid flag '%s'", isbuf);
00655
00656         if (!Ap.sameSize(Ap_buf)) {
00657             if (Ap_buf.isEmpty())
00658                 wblog(FL, "ERR buffer not set !?"); else
00659                 wblog(FL, "ERR severe size mismatch !?");
00660         }
00661
00662         for (s=0; s<Ap.dim1; s++) { A(s,Nom)=NAN;
00663             for (i=0; i<Nom; ++i) {
00664                 A(s,Nom-i-1) = An_buf(s,i);
00665                 A(s,Nom+i) = Ap_buf(s,i);
00666             }
00667         }
00668     }
00669     else {
00670         for (s=0; s<Ap.dim1; s++) { A(s,Nom)=NAN;
00671             for (i=0; i<Nom; ++i) {
00672                 A(s,Nom-i-1) = An(s,i);
00673                 A(s,Nom+i) = Ap(s,i);
00674             }
00675         }
00676     }
00677
00678     if (linfit) {
00679         for (s=0; s<A.dim1; s++) {
00680             Wb::set2avg(A.rec(s)+Nom, linfit);
00681             Wb::set2avg(A.rec(s)+Nom-linfit, linfit);
00682         }
00683     }
00684 };
00685
00686 template <class TS>
00687 void Spectral<TS>::pairRawData() {
00688     unsigned i,j,k, m=Ap.dim1, n=Ap.dim2;
00689
00690     if (!Ap.sameSize(An)) wblog(FL, "ERR severe size mismatch!");
00691     if (Ap.dim1%2) wblog(FL,
00692     "ERR Ap has odd number of rows !? (%d)", Ap.dim1);
00693
00694     for (i=0; i<m; i+=2)
00695     for (k=i/2, j=0; j<n; j++) {
00696         Ap(k,j)=Ap(i,j)+Ap(i+1,j);
00697         An(k,j)=An(i,j)+An(i+1,j);
00698     }
00699
00700     Ap.Resize(m/2,n);
00701     An.Resize(m/2,n);
00702
00703     if (mspec.dim1==0) return;
00704
00705     m=mspec.dim1; n=mspec.dim2;
00706
00707     if (m%2) wblog(FL,
00708     "ERR mspec has odd number of rows !? (%d)", m);
00709
00710     for (i=0; i<m; i+=2)

```

```

00711     for (k=i/2, j=0; j<n; j++) mspec(k,j)=mspec(i,j)+mspec(i+1,j);
00712
00713     mspec.Resize(m/2,n);
00714 }
00715
00716 template <class TS>
00717 void Spectral<TS>::getPARTIAL(
00718     wbararray<TS> &A1, wbararray<TS> &A2
00719 ) const {
00720
00721     if (AP.isEmpty() && AN.isEmpty()) {
00722         A1.init(); A2.init(); return;
00723     }
00724
00725     if (AP.SIZE.len!=3 || AP.SIZE!=AN.SIZE || AP.SIZE[1]%2) wblog(FL,
00726         "ERR %s() invalid initialization of AP/AN [%s; %s; %dx%d]",
00727         FCT,AP.sizeStr().data, AN.sizeStr().data, Ap.dim1, Ap.dim2
00728     );
00729
00730     int i,j, m=AP.SIZE[1]/2, n=AP.SIZE[0], K=AP.SIZE[2], n1=n-1, n2=2*n*m;
00731     const TS *ap=AP.data, *an=AN.data;
00732     TS *a1, *a2;
00733
00734     A1.init(2*n,m,K); a1=A1.data;
00735     A2.init(2*n,m,K); a2=A2.data;
00736
00737     for (K*=m, j=0; j<K; ++j) {
00738         if (j>=m) {
00739             for (i=0; i<n; ++i) { a1[i]=an[n1-i] - an[n1-i-n2]; }; a1+=n; an+=n;
00740             for (i=0; i<n; ++i) { a1[i]=ap[ i] - ap[ i-n2]; }; a1+=n; ap+=n;
00741             for (i=0; i<n; ++i) { a2[i]=an[n1-i] - an[n1-i-n2]; }; a2+=n; an+=n;
00742             for (i=0; i<n; ++i) { a2[i]=ap[ i] - ap[ i-n2]; }; a2+=n; ap+=n;
00743         }
00744         else {
00745             for (i=0; i<n; ++i) { a1[i]=an[n1-i]; }; a1+=n; an+=n;
00746             for (i=0; i<n; ++i) { a1[i]=ap[ i]; }; a1+=n; ap+=n;
00747             for (i=0; i<n; ++i) { a2[i]=an[n1-i]; }; a2+=n; an+=n;
00748             for (i=0; i<n; ++i) { a2[i]=ap[ i]; }; a2+=n; ap+=n;
00749         }
00750     }
00751 };
00752
00753 template <class TS>
00754 void Spectral<TS>::detailedBalance(
00755     const double T,
00756     wbvector<double> &om, wbMatrix<TS> &aa
00757 ) const {
00758
00759     unsigned i,j,k1,k2,m,n;
00760     wbvector<double> f1,f2;
00761     wbMatrix<TS> ar;
00762
00763     if (T<0) wblog(FL,"ERR T=%g !?", T);
00764
00765     getRawData(om,ar);
00766
00767     n=Ap.dim2; f1.init(2*n);
00768
00769     if (T==0) {
00770         for (i=0; i<n; i++) f1[i]=1;
00771     }
00772     else {
00773         double beta=1./T;
00774         for (i=0; i<om.len; i++) f1[i]=1./(1.+exp(beta*om[i]));
00775     }
00776
00777     f1.flip(f2);
00778
00779     m=Ap.dim1; n=ar.dim2;
00780     aa.init(2*m,n);
00781
00782     for (i=0; i<m; ++i) { k1=2*i; k2=k1+1;
00783         for (j=0; j<n; j++) {
00784             aa(k1,j)=ar(i,j)*f2[j];
00785             aa(k2,j)=ar(i,j)*f1[j];
00786         }
00787     }
00788 }
00789
00790 template <class TS>
00791 void Spectral<TS>::dispSumRule() {
00792
00793     unsigned i; double w=0;
00794     wbvector<TS> sp=Ap.recSum(), sn=An.recSum();
00795     Wb::termcolor Tc(Wb::TCOLS::BLUE);
00796
00797     wblog(FL, "\r%60s\n"

```



```

00798         "    sum %sp_raw : %s%N"
00799         "    + %sn_raw : %s ", "",
00800         istr.data, sp.toStrf("%8.5f").data,
00801         istr.data, sn.toStrf("%8.5f").data
00802     );
00803     sp+=sn;
00804
00805     if (!A0.isEmpty()) {
00806         if (A0.dim1!=sp.len) wblog(FL,
00807             "ERR size mismatch (%d/%dx%d)", sp.len,A0.dim1,A0.dim2);
00808         sp+=A0.recSum();
00809     }
00810
00811     if (sp.len) { w=Wb::real(sp[0]); }
00812     for (i=0; i<sp.len; ++i) { if (Wb::abs(sp[i]-w)>1E-2) break; }
00813
00814     if (i==sp.len && fabs(w)>1E-4 && fabs(w-round(w))<1E-4) { sp-=w;
00815         wblog(FL, "\r%60s\r    %s\\", "", Tc.el);
00816         printf("total%+7g: %s", -w, sp.toStrf("%8.2g").data);
00817         printf("%s\n", Tc.em);
00818     }
00819     else {
00820         printf("          %stotal : ", Tc.el);
00821         for (i=0; i<sp.len; ++i) { w=round(Wb::real(sp[i]));
00822             if (Wb::abs(sp[i]-w)<1E-5) {
00823                 if (w!=0)
00824                     printf("%2g%+5.0E ", w, Wb::real(sp[i])-w);
00825                 else printf("%8.1E ", Wb::real(sp[i]));
00826             }
00827             else { printf("%8.5f ", Wb::real(sp[i])); }
00828         }; printf("%s\n", Tc.em);
00829     }
00830 };
00831
00832 template <class TS>
00833 void Spectral<TS>::dispSumRule(
00834     const wbvector<double> &om, const wbMatrix<TS> &A
00835 ){
00836     wbvector<TS> s(A.dim1);
00837     unsigned i; double m=0;
00838
00839     if (om.len!=A.dim2) wblog(FL,
00840         "ERR Dimension mismatch (%d,%d)", om.len, A.dim2);
00841
00842     for (unsigned i=0; i<A.dim1; ++i)
00843         s[i] = Wb::IntTrapez(om.data, A.rec(i), om.len);
00844
00845     wblog(FL, "\r%60s\r    "
00846         "int AA(om) : %s", "", s.toStrf("%8.5f").data);
00847
00848     if (!A0.isEmpty()) {
00849         if (A0.dim1!=s.len) wblog(FL,
00850             "ERR size mismatch (%d/%dx%d)", s.len,A0.dim1,A0.dim2);
00851         s+=A0.recSum();
00852     }
00853
00854     if (s.len) {
00855         for (m=round(Wb::real(s[0])), i=0; i<s.len; ++i) {
00856             if (Wb::abs(s[i]-m)>1E-2) break;
00857         }
00858         if (m && i==s.len) { s-=m;
00859             char ms[8]; Wb::termcolor Tc(Wb::TCOLS::BLUE);
00860             if (m) {
00861                 i=snprintf(ms,8,"%+3g",-m);
00862                 if (int(i)<0) wblog(FL, "WRN %s() i=%d", FCT,i);
00863             }
00864             else { *ms=0; }
00865             wblog(FL, "\r%60s\r    %s* total%-3s: %s%s",
00866                 "", Tc.el, ms, s.toStrf("%8.2g").data, Tc.em);
00867         }
00868     }
00869 };
00870
00871 template <class TS>
00872 void Spectral<TS>::getSmoothSpec(
00873     wbvector<double> &om, wbMatrix<TS> &A, char vflag
00874 ){
00875     unsigned linfit=0;
00876
00877     getOmega(om, linfit);
00878     getSmoothSpec(A, om, vflag);
00879 };
00880
00881 template <class TS>
00882 void Spectral<TS>::getSmoothSpec(
00883     wbMatrix<TS> &A, const wbvector<double> &omega,
00884     char vflag

```

```

00885 ){
00886     unsigned i,s,Nom=Ap.dim2;
00887     wvector<double> OM(Nom), I;
00888     double x, dbl, om, aom;
00889     TS w1=0., w2=0.;
00890
00891     if (fac<=0 || sigma<=0 || sigma2<=0 || eps<=0 || Nom<4)
00892     wblog(FL,"ERR %s()\n"
00893         "fac=%.3g, sigma=%.3g, sigma2=%.3g, eps=%.3g, Nom=%d !?",
00894         FCT,fac,sigma,sigma2, eps,Nom
00895     );
00896
00897     for (i=0; i<OM.len; i++) OM[i]=idx2om(i);
00898
00899     if (vflag) {
00900         double w1=omega.min(), w2=omega.max(), w0=omega.aMin();
00901         if (w1==w2)
00902             sprintf_str(" %.3g .. %.3g", w0,w2);
00903         else sprintf_str(" %.3g .. %.3g .. %.3g",w1,w0,w2);
00904
00905         wblog(FL," * "
00906             "%s() %g bins/dec (len=%d)\n"
00907             "sigma=%.3g [eps=%.3g @ %.3g]\n"
00908             "|omega| range: %s",
00909             myname,fac/log10(M_E),Nom,sigma,eps,sigma2,str);
00910         if (fabs((w0-idx2om(0))/w0)>0.01 ||
00911             fabs((w2-idx2om(Nom-1))/w2)>0.01) wblog(FL,
00912             "|emin...emax|= %.3g .. %.3g (bins)",idx2om(0),idx2om(Nom-1));
00913         if (vflag && (Ap.dim1<8 || vflag>1)) { dispSumRule(); }
00914     }
00915
00916     A.init(Ap.dim1, omega.len);
00917
00918     for (s=0; s<Ap.dim1; ++s)
00919     for (i=0; i<omega.len; ++i) { om=omega[i]; aom=fabs(om);
00920
00921         if (aom>eps) x=1.;
00922         else {
00923             if (aom==0.) x=0.;
00924             else {
00925                 dbl = (log(aom)-log(eps))/sigma;
00926                 x = exp(-dbl*dbl);
00927                 if (x<1E-10) x=0.;
00928             }
00929         }
00930
00931         if (x!=0.)
00932         w1 = foldLogGaussian(aom, om>0 ? Ap.rec(s) : An.rec(s), Nom);
00933
00934         if (x!=1.) {
00935             double b=MAX(sigma2*eps,om*exp(-sigma*sigma/4));
00936
00937             w2 = foldGaussian( om, b, OM.data, Ap.rec(s), Nom)
00938                 + foldGaussian(-om, b, OM.data, An.rec(s), Nom);
00939         }
00940
00941         A(s,i) = x*w1 + (1.-x)*w2;
00942     }
00943
00944     if (vflag && (Ap.dim1<8 || vflag>1)) dispSumRule(omega,A);
00945 };
00946
00947 template <class TS>
00948 TS Spectral<TS>::foldLogGaussian(
00949     const double om, const TS *aa, const unsigned n
00950 ){
00951     double k, dbl, efac;
00952     TS xa=0.;
00953
00954     if (sigma<=0. || fac<=0.) wblog(FL,
00955         "ERR sigma=%g, fac=%g !?", sigma, fac);
00956
00957     k=OM2IDX(om)-0.5;
00958     efac=1./(fac*sigma);
00959
00960     double dx=alpha*sigma;
00961
00962     for (unsigned i=0; i<n; ++i) {
00963         dbl=efac*(k-double(i))-dx;
00964         xa += aa[i]*exp(-dbl*dbl);
00965     }
00966     xa /= (sigma*sqrt(M_PI)*om);
00967
00968     if (alpha!=0.25) {
00969         xa *= exp(+sigma*sigma*(alpha-0.25));
00970     }
00971

```

```

00972     return xa;
00973 };
00974
00975 template <class TS>
00976 TS Spectral<TS>::foldGaussian(
00977     const double om, const double b,
00978     const double *oo, const TS *aa, const unsigned n
00979 ){
00980     double dbl, efac;
00981     TS xa=0.;
00982
00983     if (b<=0.) wblog(FL, "ERR b=%g !?", b);
00984
00985     efac = 1./b;
00986
00987     for (unsigned i=0; i<n; i++) {
00988         dbl=efac*(om-oo[i]);
00989         xa += aa[i]*exp(-dbl*dbl);
00990     }
00991
00992     xa *= 1. / (b*sqrt(M_PI));
00993
00994     return xa;
00995 };
00996
00997 template <class TS>
00998 void Spectral<TS>::KKreal(
00999     wbMatrix<wbcomplex> &GZ, wbvector<double> &OM, wbvector<double> &DOM
01000 ) const {
01001
01002     unsigned i,j,s, d=Ap.dim1, Nom;
01003     wbMatrix<double> II, RR;
01004     double doi, doj;
01005
01006     getRawData(OM, II); Nom=OM.len;
01007     Wb::getDiff(OM,DOM);
01008
01009     #if 0
01010         wblog(FL,"TST Get smooth data ...");
01011
01012         getSmoothSpec(OM, II, 1E-6, 0.3, 0); Nom=OM.len;
01013         Wb::getDiff(OM,DOM);
01014
01015         for (s=0; s<d; s++)
01016             for (i=0; i<Nom; i++) II(s,i) *= DOM[i];
01017     #endif
01018
01019     RR.init(d,Nom);
01020
01021     for (i=0; i<Nom; ++i)
01022         for (j=i+1; j<Nom; ++j) {
01023
01024             doi=DOM[i]/(OM[j]-OM[i]);
01025             doj=DOM[j]/(OM[i]-OM[j]);
01026
01027             for (s=0; s<d; s++) {
01028                 RR(s,i) += II(s,j)*doi;
01029                 RR(s,j) += II(s,i)*doj;
01030             }
01031         }
01032
01033     #if 0
01034         wbMatrix<double> Ip2, In2;
01035         wblog(FL,"TST backtransform real->imag");
01036
01037         RR*=(1/M_PI); RR.put("RR","base",'r');
01038
01039         KKimag(Rp,Rn,OM,Ip2,In2);
01040
01041         Ip2.put("Ip","base",'r');
01042         In2.put("In","base",'r');
01043
01044         wblog(FL,"ERR");
01045     #endif
01046
01047     II *= M_PI;
01048
01049     GZ.set(RR,II);
01050 }
01051
01052 void KKimag(
01053     const wbMatrix<double> &RR,
01054     const wbvector<double> &OM,
01055     wbMatrix<double> &II
01056 ){
01057     unsigned i,j,s, d=RR.dim1, Nom=RR.dim2;
01058     double doi, doj;

```

```

01059     wbvector<double> DOM;
01060
01061     Wb::getDiff(OM, DOM); II.init(d,Nom);
01062
01063     if (OM.len!=Nom) wblog(FL,
01064         "ERR severe dimension mismatch (%d,%d) !?", OM.len, Nom);
01065
01066     for (i=0; i<Nom; ++i)
01067     for (j=i+1; j<Nom; ++j) {
01068         doi=DOM[i]/(OM[i]-OM[j]); doj=DOM[j]/(OM[j]-OM[i]);
01069         for (s=0; s<d; s++) {
01070             II(s,i) += RR(s,j)*doi;
01071             II(s,j) += RR(s,i)*doj;
01072         }
01073     }
01074
01075     II *= (1./M_PI);
01076 }
01077
01078 template <class TS>
01079 void TDSpectral<TS>::init(const char *F, int L, const char *istr0,
01080     const wbvector<double> &t0,
01081     unsigned Nnrg, unsigned Nop,
01082     double Emin, double Emax, unsigned nl, char vflag
01083 ){
01084     double l2;
01085
01086     if (Emin<=0. || Emax<=0. || Emin>=0.1*Emax || nl==0) wblog(F,L,
01087         "ERR invalid parameter set [%g %g %d]s", Emin, Emax, nl,
01088         (Emin>0 && Emax>0 && Emin<Emax)? " (require Emin<Emax/10)": "");
01089
01090     l0 = log(Emin);
01091     l2 = log(Emax);
01092
01093     fac= log10(M_E)*double(nl);
01094
01095     N = (unsigned)((l2-l0)*fac)+1;
01096
01097     AP.init(N,Nop,Nnrg);
01098     AN.init(N,Nop,Nnrg); if (check_avg_om) {
01099     AW.init(2,Nop,Nnrg); }
01100
01101     if (vflag) {
01102         wblog(F_L,"%3s TDSpectral: %.2g .. %.2g (%d/dec; 2x%d)",
01103             istr0 ? istr0 : "<i>", Emin, Emax, nl, N);
01104     }
01105
01106     istr=istr0; tt=t0;
01107 };
01108
01109 template <class TS>
01110 wbarray<TS> TDSpectral<TS>::getAvgOm() {
01111
01112     if (AW.rank()!=3 || AW.dim(1)!=2) wblog(FL,
01113         "ERR invalid AW data structure [%s] !?",AW.sizeStr().data);
01114
01115     unsigned i=0, n=AW.numel();
01116     wbarray<TS> aw(AW); if (aw.isEmpty()) return aw;
01117     TS *a=AW.data, *b=aw.data;
01118
01119     for (; i<n; i+=2) { b[i] = ((a[i+1]!=0.) ? a[i]/a[i+1] : a[i]); }
01120
01121     return aw;
01122 };
01123
01124 template <class TS>
01125 void TDSpectral<TS>::Add(
01126     const wbarray<double> &Om,
01127     const wbarray<TS> &CC, unsigned iter, unsigned s
01128 ){
01129     static int iter_last=-1;
01130     unsigned i,k,n=Om.numel();
01131     TS *ap, *an;
01132
01133     if (!Om.sameSize(CC)) {
01134         Om.info("Om"); CC.info("CC");
01135         wblog(FL,"ERR size mismatch between Om and CC!");
01136     }
01137
01138     if (s>=AP.dim(2) || iter>=AP.dim(3)) wblog(FL,
01139         "ERR index out of bounds (op=%d/%d; k=%d/%d)",
01140         s+1,AP.dim(2),iter+1,AP.dim(3));
01141
01142     ap=&AP(0,s,iter);
01143     an=&AN(0,s,iter);
01144
01145     for (i=0; i<n; ++i) {

```

```

01146         k=om2idx(Om.data[i]);
01147         if (Om.data[i]<0.)
01148             an[k]+=CC.data[i];
01149         else ap[k]+=CC.data[i];
01150     }
01151
01152     if (raw) {
01153         if (iter_last!=(int)iter) {
01154             wblog(FL, "%NTST iter=%d", iter);
01155             if (RAW.len) addPartialFourier();
01156             RAW.initDef(AP.dim(2));
01157             wblog(FL, "TST done");
01158         }
01159
01160         RAW[s].Append(Om.data, CC.data, n);
01161
01162         iter_last=iter;
01163     }
01164
01165     if (check_avg_om) {
01166         if (AW.rank()!=3 || AW.dim(1)!=2 ||
01167             AW.dim(2)!=AP.dim(2) || AW.dim(3)!=AP.dim(3)) wblog(FL,
01168             "ERR size mismatch for AW [%s] ? [%s]",
01169             AW.sizeStr().data, AP.sizeStr().data
01170         );
01171         AW(0,s,iter)+=CC.dotProd(Om);
01172         AW(1,s,iter)+=CC.sum();
01173     }
01174 };
01175
01176 template <class TS> inline
01177 void TDSpectral<TS>::addPartialFourier() {
01178
01179     if (!RAW.len) return;
01180
01181     unsigned s=0, m=RAW.len;
01182     wbMatrix<wbcomplex> at;
01183     wbMatrix<TS> aa;
01184
01185     if (m!=AP.dim(2)) wblog(FL,
01186         "ERR severe size mismatch (%d/%d)", m, AP.dim(2));
01187
01188     if (At.isEmpty()) At.init(AP.dim(2), tt.len);
01189
01190     for (s=0; s<m; ++s) {
01191         const RSpecData<TS> &R=RAW[s];
01192         if (R.om.len!=R.cc.len) wblog(FL,
01193             "ERR RAW[%d].len=%d/%d !?", s, R.om.len, R.cc.len);
01194
01195         aa.init2ref(1, R.cc.len, R.cc.data);
01196
01197         Wb::Fourier(R.om, aa, tt, at);
01198         At.recAddP(s, at.data);
01199     }
01200
01201     RAW.init();
01202 };
01203
01204 template <class TS>
01205 void TDSpectral<TS>::getOmega(
01206     wbvector<double> &om, unsigned &linfit
01207 ) const {
01208
01209     unsigned i;
01210     double dbl;
01211
01212     if (linfit) linfit=unsigned(fac);
01213     if (linfit>N) wblog(FL, "ERR linfit>N (%d,%d) !?", N, linfit);
01214
01215     om.init(2*N);
01216
01217     dbl = idx2om(linfit) / (linfit+0.5);
01218
01219     for (i=0; i<linfit; i++) om[N-1-i] = -(om[N+i] = dbl*(i+0.5));
01220     for ( ; i<N; i++) om[N-1-i] = -(om[N+i] = idx2om(i) );
01221 }
01222
01223 template <class TS>
01224 void TDSpectral<TS>::applyIROPfac(
01225     const char *F, int L, const wbvector<double> &fac, char rixs) {
01226
01227     unsigned i,k=0, n=fac.len;
01228     double x;
01229
01230     for (i=0; i<n; ++i) { if (fac[i]!=1) {
01231         if (fac[i]<1 || fac[i]!=round(fac[i])) wblog(FL,
01232             "ERR %s() got fac[%d]=%g !?", FCT, i+1, fac[i]);

```

```

01233     else {
01234         wblog(FL, " * applying IROP factor 1/%g%s to op(%d)",
01235             fac[i], rixs? "^2":"", i+1);
01236         ++k;
01237     }
01238 }
01239 if (!k) return;
01240
01241 if (!Ap.isEmpty() || !An.isEmpty()) {
01242     if (n!=Ap.dim1 || n!=An.dim1) wblog(FL,
01243         "ERR %s() size mismatch (%d,%d/2*%d)", FCT, Ap.dim1, An.dim1, n);
01244
01245     for (i=0; i<n; ++i) {
01246         if (fac[i]!=1) { x=1/fac[i]; if (rixs) { x*=x; }
01247             Ap.rowTimes(i,x);
01248             An.rowTimes(i,x);
01249         }
01250     }
01251 }
01252
01253 if (!AP.isEmpty() || !AN.isEmpty()) {
01254     if (AP.SIZE.len!=3 || !AP.sameSize(AN) || AP.SIZE[1]!=n)
01255         wblog(FL, "ERR %s() size mismatch (%s; %s /%d) !?",
01256             FCT, SSTR(AP), SSTR(AN), n);
01257
01258     TS *ap=AP.data, *an=AN.data;
01259     unsigned n0=AP.SIZE[0], m=n*AP.SIZE[2];
01260
01261     for (i=0; i<m; ++i, ap+=n0, an+=n0) { k=i%n;
01262         if (fac[k]>1) { x=1/fac[k]; if (rixs) { x*=x; }
01263             Wb::timesRange(ap, x, n0);
01264             Wb::timesRange(an, x, n0);
01265         }
01266     }
01267 }
01268 };
01269
01270 template <class TS>
01271 void TDSpectral<TS>::getSpecData(
01272     wvector<double> &om, warray<TS> &A,
01273     unsigned linfit
01274 ) const {
01275     unsigned i,r,s, d1=AP.dim(2), d2=AP.dim(3);
01276
01277     getOmega(om, linfit);
01278     A.init(2*N, d1, d2);
01279
01280     if (om.len!=2*N || !AP.sameSize(AN)) wblog(FL,
01281         "ERR severe dimension mismatch (%d,%d).", 2*N, om.len);
01282
01283     for (r=0; r<d1; r++)
01284         for (s=0; s<d2; s++) {
01285             for (i=0; i<N; i++) {
01286                 A(N-i-1, r, s) = AN(i, r, s);
01287                 A(N+i, r, s) = AP(i, r, s);
01288             }
01289         }
01290
01291     if (linfit) {
01292         for (r=0; r<d1; r++)
01293             for (s=0; s<d2; s++) {
01294                 Wb::set2avg(A.ref(r,s)+N, linfit);
01295                 Wb::set2avg(A.ref(r,s)+N-linfit, linfit);
01296             }
01297     }
01298 };
01299
01300 template <class TS>
01301 void TDSpectral<TS>::getSpecData(
01302     wvector<double> &om, wMatrix<TS> &A,
01303     unsigned linfit
01304 ) {
01305     unsigned i,r;
01306
01307     getOmega(om, linfit);
01308
01309     if (Ap.isEmpty()) mergeNRGData();
01310     A.init(Ap.dim1, 2*N);
01311
01312     if (A.dim2!=om.len || !Ap.sameSize(An)) wblog(FL,
01313         "ERR Severe dimension mismatch (%d,%d).", A.dim2, om.len);
01314
01315     for (r=0; r<A.dim1; r++)
01316         for (i=0; i<N; i++) {
01317             A(r, N-i-1) = An(r, i);
01318             A(r, N+i) = Ap(r, i);

```

```

01320     }
01321
01322     if (linfit) {
01323         for (r=0; r<A.dim1; r++) {
01324             Wb::set2avg(A.rec(r)+N, linfit);
01325             Wb::set2avg(A.rec(r)+N-linfit, linfit);
01326         }
01327     }
01328 }
01329
01330 template <class TS>
01331 void TDSpectral<TS>::Fourier(
01332     wvector<double> &om,
01333     wbMatrix<wbcomplex> &ST,
01334     double alpha, double Lambda, char vflag
01335 ){
01336     unsigned k,it,ik,is,io,l=0,no, nt=tt.len, ns=AP.dim(2), nk=AP.dim(3);
01337
01338     warray<wbcomplex> TKS(nt,nk,ns), KST;
01339     wbMatrix<double> aTK(nt,nk);
01340     warray<TS> AR;
01341     double ek,oi,*at; TS kso,*ap;
01342     wbcomplex z,*st;
01343
01344     if (vflag) wblog(FL,
01345         "<i> %s (damping alpha=%g, Lambda=%g)",FCT,alpha,Lambda);
01346
01347     getSpecData(om,AR);
01348     ap=AR.data; no=om.len;
01349
01350     if (alpha==0) aTK.set(1); else
01351     for (ik=0; ik<nk; ++ik) {
01352         ek=pow(Lambda, -double(ik)/2.) * 0.5*(Lambda+1);
01353         for (it=0; it<nt; it++)
01354             aTK(it,ik)=exp(-fabs(alpha*ek*tt[it]));
01355     }
01356
01357     for (ik=0; ik<nk; ik++)
01358     for (is=0; is<ns; is++)
01359     for (io=0; io<no; io++, l++) if (ap[l]!=0.) { kso=ap[l]; oi=om[io];
01360         for (it=0; it<nt; it++)
01361             TKS(it,ik,is)+=kso*expi(oi*tt[it]);
01362     }
01363
01364     TKS.permute(KST,"2 3 1");
01365     ST.init(ns,nt);
01366
01367     for (is=0; is<ns; ++is)
01368     for (it=0; it<nt; ++it) {
01369         st=KST.ref(is,it); z=0;
01370         if (alpha==0)
01371             for (k=0; k<nk; ++k) z+=st[k];
01372         else for (at=aTK.rec(it), k=0; k<nk; ++k) z+=at[k]*st[k];
01373         ST(is,it)=z;
01374     }
01375
01376 };
01377
01378 template <class TS>
01379 void TDSpectral<TS>::Fourier(
01380     wvector<double> &om, wbMatrix<TS> &A,
01381     const wvector<double> &tt,
01382     wbMatrix<wbcomplex> &at,
01383     double alpha, char vflag
01384 ){
01385     if (Ap.isEmpty()) { mergeNRGData(); dispSumRule(); }
01386     getSpecData(om,A);
01387
01388     Wb::Fourier(om,A,tt,at,0,alpha,vflag);
01389 };
01390
01391 template <class TS>
01392 void Wb::Fourier(
01393     const wvector<double> &om, const wbMatrix<TS> &A,
01394     const wvector<double> &tt,
01395     wbMatrix<wbcomplex> &az,
01396     char domflag,
01397     double alpha,
01398     char vflag
01399 ){
01400     unsigned i,j,l,m,n,b1,b2, nom, nt=tt.len, D=512, stop=0;
01401     wbMatrix<wbcomplex> ez;
01402     wbcomplex z;
01403     double f,ot;
01404
01405     wvector< wbMatrix<wbcomplex> > AZ;
01406     wvector<double> dom(om.len);

```

```

01407
01408     m=A.dim1; nom=om.len;
01409
01410     if (nom!=A.dim2) wblog(FL,
01411         "ERR %s - size mismatch (%d/%d)",nom,A.dim2);
01412     if (!tt.len || !m || !nom) {
01413         wblog(FL,"WRN Empty data [%d %d %d] - return",tt.len,m,nom);
01414         return;
01415     }
01416     if (nom<2) wblog(FL,"ERR %s - omega too short (%d/2)",nom);
01417
01418     n=om.len-1;
01419     for (i=1; i<n; i++) dom[i]=0.5*(om[i+1]-om[i-1]);
01420     dom[0]=om[1]-om[0]; dom[n]=om[n]-om[n-1];
01421
01422     for (i=0; i<dom.len; i++) if (dom[i]<=0) { wblog(FL,
01423         "WRN omega not strictly ascending !?"); break; }
01424
01425     for (i=0; i<tt.len; i++) if (tt[i]<0) { wblog(FL,
01426         "WRN negative tt values !?"); break; }
01427
01428     if (domflag) {
01429         wbMatrix<double> B(A);
01430
01431         for (i=0; i<B.dim1; ++i)
01432             for (j=0; j<B.dim2; ++j) B(i,j)*=dom[j];
01433
01434         Wb::Fourier(om,B,tt,az,0,alpha,vflag);
01435         return;
01436     }
01437
01438     if (vflag) wblog(FL," * %s() spectral data "
01439         "(alpha=%4g, %s)",FCT,alpha, domflag? "func":"disc");
01440
01441     AZ.initDef(nt<D ? 1 : nt/D);
01442
01443     for (l=b1=0; !stop; l++,b1+=D) {
01444         b2=b1+D-1; if (b2+D>nt) { b2=nt-1; stop=1; }
01445         n=b2-b1+1;
01446
01447         if (n*nom*sizeof(double)/(1<<20)>900) {
01448             wblog(FL,"WRN trying to allocate %.6g MB !?",
01449                 n*nom*sizeof(double)/double(1<<20));
01450         }
01451
01452         ez.init(nom,n);
01453         for (i=0; i<nom; i++)
01454             for (j=0; j<n; j++) { ot=om[i]*tt[b1+j];
01455                 ez(i,j).init_expi(ot);
01456
01457                 if (alpha>0) {
01458                     f=::exp(-::fabs(alpha*dom[i]*tt[b1+j]));
01459                     ez(i,j)*=f;
01460                 }
01461             }
01462         Wb::MatProd(A,ez,AZ[l]);
01463     }
01464
01465     az.CAT(2,AZ);
01466 };
01467
01468 template <class TS>
01469 void TDSpectral<TS>::mergeNRGData() {
01470
01471     unsigned i,k, d1=AP.dim(1), d2=AP.dim(2), d3=AP.dim(3);
01472     unsigned s=d1*d2, S=s*d3;
01473     TS *Dp,*Dn,*dp,*dn;
01474
01475     if (!AP.sameSize(AN)) wblog(FL, "ERR severe size mismatch!");
01476
01477     Ap.init(d2,d1); dp=Ap.data; Dp=AP.data;
01478     An.init(d2,d1); dn=An.data; Dn=AN.data;
01479
01480     for (i=0; i<S; i++) { k=i%s;
01481         dp[k]+=Dp[i];
01482         dn[k]+=Dn[i];
01483     }
01484 };
01485
01486 template <class TS>
01487 void TDSpectral<TS>::dispSumRule() {
01488
01489     unsigned i=0; double m=0;
01490     wbvector<TS> sp,sn;
01491     wbararray<TS> x;
01492
01493     AP.sum("1,3",x); sp.init(x.numel(),x.data);

```



```

01494     AN.sum("1,3",x); sn.init(x.numel(),x.data);
01495
01496     wblog(FL," * om>0: %s\nom<0: %s",
01497           sp.toStrf("%8.5f").data, sn.toStrf("%8.5f").data);
01498
01499     sp+=sn;
01500
01501     if (sp.len) {
01502         m=Wb::round(Wb::real(sp[0]));
01503         for (i=0; i<sp.len; i++) if (Wb::abs(sp[i]-m)>1E-2) break;
01504     }
01505
01506     Wb::termcolor Tc(Wb::TCOLS::BLUE);
01507     if (i==sp.len) { sp=TS(m); wblog(FL,
01508         " * %stotal %s%s (%g)",Tc.el,sp.toStrf("%8.2g").data,Tc.em,m); }
01509     else wblog(FL,
01510         " * %stotal %s%s", Tc.el,sp.toStrf("%8.5f").data,Tc.em);
01511 };
01512
01513 template <class TS>
01514 void Wb::dispSumRule(
01515     const wbvector<double> &om,
01516     const wbMatrix<TS> &A
01517 ){
01518     unsigned i; double m=0;
01519     wbvector<double> s(A.dim1);
01520     Wb::termcolor Tc(Wb::TCOLS::BLUE);
01521
01522     if (om.len!=A.dim2) wblog(FL,
01523         "ERR Dimension mismatch (%d,%d)", om.len, A.dim2);
01524
01525     for (unsigned i=0; i<A.dim1; i++)
01526         s[i] = Wb::IntTrapez(om.data, A.rec(i), om.len);
01527
01528     wblog(FL,"SUM %s", s.toStrf("%8.5f").data);
01529
01530     if (s.len) {
01531         for (m=Wb::round(s[0]), i=1; i<s.len; ++i) {
01532             if (Wb::abs(s[i]-m)>1E-2) { break; }
01533         }
01534         if (i==s.len) { s-=m;
01535             wblog(FL,"%s * total-%g = %s%s",
01536                 Tc.el,m,s.toStrf("%8.2g").data,Tc.em);
01537         }
01538     }
01539 }
01540
01541 template <class TS>
01542 void TDSpectral<TS>::getSmoothSpec_t(
01543     double sigma, double eps, char bswitch, char vflag
01544 ) {
01545     unsigned i,s;
01546     wbMatrix<double> Ap0(Ap), An0(An), wlg;
01547
01548     if (fac<=0 || sigma<=0 || N<4) wblog(FL,
01549         "ERR invalid paramters (fac=%g, sigma=%g, N=%d)",fac,sigma,N);
01550     if (fac*sigma<1) wblog(FL,
01551         "WRN sigma < discrete log spacing (%.3g / %.3g)",sigma,1/fac);
01552     if (vflag) wblog(FL,
01553         "<i> %s() %dx%d bins (sigma=%g)",myname,AP.dim(2),N,sigma);
01554
01555     if (Ap.isEmpty()) { mergeNRGData(); };
01556     if (vflag) { dispSumRule(); }
01557
01558     Ap.set(0); An.set(0);
01559     initLG(wlg,N,sigma,bswitch,vflag);
01560
01561     for (s=0; s<Ap.dim1; ++s)
01562         for (i=0; i<Ap.dim2; ++i) {
01563             foldLogGaussian(wlg, Ap.rec(s), Ap0(s,i), i);
01564             foldLogGaussian(wlg, An.rec(s), An0(s,i), i);
01565         }
01566
01567     if (eps) {
01568         if (vflag>1) {
01569             wblog(FL," * %s() eps=%.3g/%.3g",myname,eps,exp(10)); }
01570
01571         unsigned j,l, i0=0, i1=0, i2=0;
01572         unsigned nE=(1<12);
01573
01574         double eps0=0.05*eps, eps1=1.5*eps, x1=5., xfac=x1/double(nE);
01575         double x=1./sqrt(M_PI), dbl, lfac=1/xfac;
01576
01577         wbvector<double> om(Ap.dim2), dom(om.len);
01578         wbvector<double> gauss_lin(2*nE+1), erf_om(om.len);
01579
01580         if (om.len<10) wblog(FL,"ERR %s() om.len = %d !?",FCT,om.len);

```

```

01581     for (i=0; i<om.len; ++i) {
01582         om[i]=idx2om(i);
01583
01584         if (!i0 && i && om[i]>eps0) { i0=i; }
01585
01586         if (!i1 && i && (om[i]-om[i-1])>eps1) { i1=i; }
01587
01588         if (i>1) {
01589             dom[i-1] = 0.5*( om[i] - om[i-2] ); }
01590     }
01591     dom[i-1] = 2*dom[i-2];
01592     dom[0] = 0.5*( om[0] + om[1] );
01593
01594     if (i1) { if (vflag) {
01595         sprintf_str("(%.1g < |om| < %.1g (%.0f%%)",om[i0],om[i1],
01596             100*((i1-i0)/double(om.len)));
01597         wblog(FL,"--> Gaussian broadening for %s",str);
01598     }}
01599     else {
01600         i1=om.len-1; sprintf_str("(%.3g; eps=%.3g)",om[i1],eps);
01601         wblog(FL,"WRN Gaussian broadening for all omega %s",str);
01602     }
01603
01604     i2=i1+10; if (i2>om.len) { i2=om.len-1; }
01605
01606     for (i=0; i<erf_om.len; ++i) {
01607         erf_om[i] = (0.5*( std::erf( om[i]/eps ) + 1 ));
01608     }
01609
01610     for (i=0; i<gauss_lin.len; ++i) {
01611         dbl = i*xfac - x1;
01612         gauss_lin[i] = x*exp(-dbl*dbl);
01613     }
01614     dom*=(1/eps);
01615
01616     Ap0=Ap; An0=An;
01617     for (s=0; s<Ap.dim1; ++s)
01618     for (i=0; i<i1; ++i) { Ap(s,i) = An(s,i) = 0; }
01619
01620     for (s=0; s<Ap.dim1; ++s) {
01621         for (x=0., i=0; i<i0; ++i) { x += (Ap0(s,i) + An0(s,i)); }
01622         for (i=0; i<i2; ++i) {
01623             Ap(s,i) += ( dbl = x*(erf_om[i] - (i? erf_om[i-1] : 0.5)) );
01624             An(s,i) += dbl;
01625         }
01626     }
01627
01628     for (s=0; s<Ap.dim1; ++s)
01629     for (i=i0; i<i1; ++i) {
01630         for (j=0; j<i2; ++j) {
01631             dbl = (om[j]-om[i])/eps;
01632             if (fabs(dbl)<x1 ) { l = (dbl + x1) * lfac + 0.5;
01633             if (l<gauss_lin.len) {
01634                 Ap(s,j) += Ap0(s,i) * (dbl = gauss_lin[l] * dom[j]);
01635                 An(s,j) += An0(s,i) * dbl;
01636             }}
01637
01638             dbl = (om[j]+om[i])/eps;
01639             if (fabs(dbl)<x1 ) { l = (dbl + x1) * lfac + 0.5;
01640             if (l<gauss_lin.len) {
01641                 An(s,j) += Ap0(s,i) * (dbl = gauss_lin[l] * dom[j]);
01642                 Ap(s,j) += An0(s,i) * dbl;
01643             }}
01644         }
01645     }
01646 }
01647
01648 };
01649
01650 template <class TS>
01651 void TDSpectral<TS>::initLG(
01652     wbMatrix<double> &wlg,
01653     unsigned n, double sigma, char bswitch, char vflag
01654 ){
01655     unsigned i,j;
01656     double dbl,efac,s=0;
01657
01658     if (sigma<=0. || fac<=0.) wblog(FL,"ERR sigma=%g, fac=%g",sigma,fac);
01659
01660     efac=1./(fac*sigma); wlg.init(2,n);
01661
01662     switch (bswitch) {
01663     case 0:
01664         strcpy(str,"symmetric in log. space");
01665         s=0; break;
01666     case 1:
01667         strcpy(str,"symmetric log-gauss");

```

```

01668         s=sigma/2.;
01669         break;
01670     case 2:
01671         strcpy(str,"norm preserving log-gauss");
01672         s=sigma/4.;
01673         break;
01674     default:
01675         wblog(FL,"ERR invalid bswitch=%d",bswitch);
01676     }
01677     if (vflag) wblog(FL,"--> %s() using %s",FCT,str);
01678
01679     for (i=0; i<n; i++) {
01680         dbl=efac*double(i)+s; wlg(0,i)=exp(-dbl*dbl);
01681         dbl=efac*double(i)-s; wlg(1,i)=exp(-dbl*dbl);
01682     }
01683
01684     wlg*=(1./wlg.sum());
01685
01686     for (i=0; i<2; ++i)
01687         for (j=0; j<n; ++j) if (fabs(wlg(i,j))<1E-20) wlg(i,j)=0;
01688 };
01689
01690 template <class TS> inline
01691 void TDSpectral<TS>::foldLogGaussian(
01692     const wbMatrix<double> &wlg,
01693     TS *aa, TS a0, unsigned i0
01694 ){
01695     unsigned i, n=wlg.dim2;
01696     const double *wp=wlg.rec(0), *wn=wlg.rec(1);
01697     double wi, nrm=0.;
01698
01699     for (i=0; i<n; ++i) {
01700         nrm += (i0>i ? wp[i0-i] : wn[i-i0]); }
01701     nrm=a0/nrm;
01702
01703     for (i=0; i<n; ++i) {
01704         wi=( i0>i ? wp[i0-i] : wn[i-i0] );
01705         if (wi) aa[i]+=nrm*wi;
01706     }
01707 };
01708
01709 template <class TS>
01710 mxArray* TDSpectral<TS>::toMx() const {
01711     mxArray *S=mxCreateStructMatrix(1,1,0,NULL);
01712
01713     mxAddField2Scalar(FL,S,"name",istr.toMx());
01714     mxAddField2Scalar(FL,S,"AP", AP.toMx());
01715     mxAddField2Scalar(FL,S,"AN", AN.toMx());
01716     mxAddField2Scalar(FL,S,"fac",numtoMx(fac));
01717     mxAddField2Scalar(FL,S,"l0", numtoMx(l0 ));
01718
01719     if (raw) {
01720         mxAddField2Scalar(FL,S,"RAW",RAW.toMx()); }
01721
01722     return S;
01723 };
01724
01725 template <class TS>
01726 int TDSpectral<TS>::init(const mxArray *S,
01727     double &emin, double &emax, unsigned &nlog
01728 ){
01729     if (!mxIsStruct(S) || !Mx::IsScalar(S) ||
01730         mxGetFieldNumber(S,"name")<0 ||
01731         mxGetFieldNumber(S,"AP" )<0 ||
01732         mxGetFieldNumber(S,"AN" )<0 ||
01733         mxGetFieldNumber(S,"fac" )<0 ||
01734         mxGetFieldNumber(S,"l0" )<0
01735     ) return 1;
01736
01737     istr.init(mxGetField(S,0,"name" ));
01738     AP .init(mxGetField(S,0,"AP"));
01739     AN .init(mxGetField(S,0,"AN"));
01740     mxGetNumber(mxGetField(S,0,"fac"),fac);
01741     mxGetNumber(mxGetField(S,0,"l0"), l0 );
01742
01743     if (AP.SIZE!=AN.SIZE || AP.isEmpty() || AN.isEmpty()) return 2;
01744     if (fac<=0) return 3;
01745
01746     N=AP.dim(1);
01747
01748     emin=exp(l0);
01749     emax=idx2om(N);
01750     nlog=(unsigned)(fac/log10(M_E) + 0.5);
01751
01752     wblog(FL,
01753         "<i> TDSpectral `%' from RAW data: %dx%d bins [x%d]",
01754         istr.data, AP.dim(1), AP.dim(2), AP.dim(3));

```

```

01755
01756     return 0;
01757 };
01758
01759 #endif

```

5.21 util.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : QSpace utility routines
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_UTIL_CC__
00021 #define __WB_UTIL_CC__
00022
00023 /* ----- */
00024 // NB! this requires
00025 //     fabs() => [std]::fabs
00026 //     sqrt() => [std]::sqrt(), etc.
00027 // since Wb::fabs() is only defined for MPFR (!)
00028 // and so compiler complains that MPFR routine got double // Wb,Jul05,22
00029 /* ----- */
00030
00031 int Wb::Rational(
00032     double &x0,
00033     long &P, long &Q,
00034     double *dx,
00035     unsigned *nmax,
00036     wvector<double>* aa,
00037     unsigned niter,
00038     long pqmax,
00039
00040     double eps,
00041
00042     double eps2,
00043
00044     char vflag
00045 ){
00046     unsigned i; int err=0;
00047     long p[niter+2], q[niter+2];
00048     double e,xi,x=0, x=x0, a[niter+1];
00049
00050     if (eps<eps2 || eps2<=0) wblog(FL,
00051         "ERR %s() invalid eps=%g, eps2=%g",FCT,eps,eps2);
00052
00053     if (pqmax<=0) { pqmax=long(MIN(1E12, 1/eps)); }
00054     if (pqmax<=0 || pqmax>(1L<<50)) {
00055         wblog(FL,"ERR %s() pqmax=%ld !?",FCT,pqmax);
00056     }
00057
00058     if (::fabs(x0)>pqmax || (x!=0 && ::fabs(1/x0)>pqmax)) {
00059         if (vflag) wblog(FL,"WRN %s() double out of bounds long int",FCT);
00060         if (dx) { (*dx)=0; }
00061         if (nmax) { (*nmax)=0; }; if (aa) { aa->init(); }
00062         return -1;
00063     }
00064
00065     p[0]=1; p[1]=a[0]=xi::round(x);
00066     q[0]=0; q[1]=1;
00067
00068     if (vflag) printf(
00069         "\n %s() finding rational approximation for %.16g ... \n"
00070         "\n %6d: %4g | %8ld / %8ld \n",FCT,x, 0,a[0],p[1],q[1]);
00071
00072     if (::fabs(xi-x)<eps) {
00073         P=p[1]; Q=q[1];
00074         if (dx) { (*dx)=::fabs(xi-x); }
00075         if (nmax) { (*nmax)=0; }; if (aa) { aa->init(1,a); }

```

```

00076     x0=xi; return 0;
00077 }
00078
00079 if (::fabs(x)>1) {
00080     eps *=::fabs(x);
00081 }
00082
00083 r=::fabs(xi-x0);
00084
00085 for (i=0; i<niter; ++i) {
00086     if ((e=::fabs(x-a[i]))<eps || r<eps2) break;
00087
00088     x=1./(x-a[i]); a[i+1]=::round(x);
00089     p[i+2]=a[i+1]*p[i+1]+p[i];
00090     q[i+2]=a[i+1]*q[i+1]+q[i];
00091
00092     xi=double(p[i+2])/q[i+2];
00093     r=::fabs(xi-x0);
00094
00095     if (vflag) {
00096         printf("%6d: %4g | %8ld / %8ld = %20.16g @ %8.3g (%.3g)\n",
00097             i+1, a[i+1], p[i+2], q[i+2], xi, r, x-a[i+1]);
00098     }
00099     if (labs(p[i+2])>pqmax || labs(q[i+2])>pqmax) {
00100         ++i; err=1; break;
00101     }
00102 }
00103 P=p[i+1]; Q=q[i+1]; if (Q<0) { P=-P; Q=-Q; }
00104 if (i>=niter) err=2;
00105
00106 if (vflag) {
00107     if (i<niter) printf("\n "
00108         "Rational representation: %ld/%ld\n\n",P,Q);
00109     else printf("\n "
00110         "Failed to find rational representation (%d @ %g)\n\n",
00111         niter,eps
00112     );
00113 }
00114
00115 if (dx) (*dx)=r;
00116 if (aa) aa->init(i,a);
00117 if (nmax) (*nmax)=i;
00118
00119 if (!err) x0=xi;
00120 return err;
00121 };
00122
00123 template<class T>
00124 double Wb::FixRational(
00125     const char *F    QS_UNUSED_VAR,
00126     int L            QS_UNUSED_VAR,
00127     T *d             QS_UNUSED_VAR,
00128     unsigned n       QS_UNUSED_VAR,
00129     char rflag       QS_UNUSED_VAR,
00130     unsigned niter   QS_UNUSED_VAR,
00131     long pqmax       QS_UNUSED_VAR,
00132     double eps       QS_UNUSED_VAR,
00133     double eps2      QS_UNUSED_VAR,
00134     double *rz       QS_UNUSED_VAR,
00135     double *ra       QS_UNUSED_VAR,
00136     char vflag       QS_UNUSED_VAR
00137 ) {
00138     return 0;
00139 };
00140
00141 template<>
00142 double Wb::FixRational(
00143     const char *F, int L, double *A, unsigned n,
00144     char rflag, unsigned niter,
00145     long pqmax,
00146     double eps,
00147     double eps2,
00148     double *r2z,
00149     double *r2m,
00150     char vflag
00151 ) {
00152     unsigned i,m,m2, nmax=0, nmissd=0; int e1,e2; long p,q, p2,q2;
00153     double x,Ai,Ai2,a2,d2,a=0,r=0,r2=0,r2max=0,z2=0,err2=0, eps1=eps;
00154
00155     for (i=0; i<n; ++i) {
00156         x=::fabs(A[i]); if (a<x) { a=x; }
00157
00158         x=::fabs(A[i]-::round(A[i])); if (r<x) { r=x; }
00159         if (x<eps) { d2=(x*x);
00160             A[i]=::round(A[i]);
00161             if (A[i]) { err2+=d2; } else { z2+=d2; }
00162             if (r2max<d2) { r2max=d2; }

```

```

00163     }
00164 }
00165 if (r2z) { (*r2z)=z2; } else { err2+=z2; }
00166
00167 if (r<eps || int(niter)<1) {
00168     if (r2m) { (*r2m)=r2max; }
00169     return err2;
00170 }
00171 if (a>1) { epsi*=a; }
00172
00173 for (i=0; i<n; ++i) { a=Ai=A[i]; e2=-1;
00174     e1=Rational(a,p,q,&r,&m,NULL,niter,pqmax,eps,eps2,vflag>1);
00175
00176     if (rflag && (e1 || abs(p)>999 || abs(q)>999)) {
00177         if ( (a2=Ai2=Ai+Ai) > eps) {
00178             e2=Rational(a2,p2,q2,&r2,&m2,NULL,niter,pqmax,
00179                 a2<0.01 ? eps *::fabs(Ai) : eps,
00180                 a2<0.01 ? eps2*::fabs(Ai) : eps2, vflag>1);
00181         }
00182     }
00183
00184     if (!e1 && !e2) {
00185         double q1=::fabs(A[i]*A[i]-Ai2), q2=::fabs(a2-Ai2);
00186         if (q1>q2) { if (vflag>1) {
00187             PRINTF(" ==> picking sqrt() over plain (e=%g / %g)\n",
00188                 ::sqrt(q2), ::sqrt(q1)); }
00189             e1=-2;
00190         }
00191     }
00192
00193     if (!e1) {
00194         if (nmax<m) { nmax=m; }
00195         if (r>epsi) { if (q<0) { p=-p; q=-q; }; wblog(F_L,
00196             "WRN fixing number %.16g = %ld/%ld (%d; %g)",Ai,p,q,m,r);
00197         }
00198         A[i]=a;
00199         d2=r*r; err2+=d2; if (r2max<d2) { r2max=d2; }
00200         continue;
00201     }
00202     if (e2) {
00203         nmissd++; continue;
00204     }
00205
00206     if (nmax<m2) { nmax=m2; }
00207     a=::sqrt(a2); if (Ai<0) { a=-a; }
00208     r=::fabs(a-Ai);
00209
00210     if (r>epsi) {
00211         wblog(F_L,"WRN fixing %.16g = %ssqrt(%ld/%ld) (%.2g)",
00212             A[i], A[i]<0 ? "-":"", p2,q2,r);
00213         wblog(FL,"TST %s() pqmax=%ld, eps=%g, eps2=%g,niter=%d",
00214             FCT,pqmax,eps,eps2,niter);
00215         double a2=Ai+Ai;
00216         Rational(a2,p2,q2,&r2,&m2,NULL,niter,pqmax,eps,eps2,'v');
00217     }
00218     d2=r*r; err2+=d2; if (r2max<d2) { r2max=d2; }
00219     A[i]=a;
00220 }
00221
00222 if (r2m) { (*r2m)=r2max; }
00223
00224 if (nmissd && vflag) wblog(F_L,
00225     "WRN %s() %d/%d values unaltered (%d)\r\\",FCT,nmissd,n,nmax);
00226
00227 return err2;
00228 };
00229
00230 wstring Wb::rat2Str(
00231     const char *F, int L, double d, unsigned niter,
00232     long pqmax,
00233     double eps,
00234     double eps2,
00235     char vflag
00236 ){
00237     wstring sout(32);
00238     long p;
00239
00240     if (::fabs(d-(p=:round(d)))<eps) {
00241         if (sout.printf(F_L,"%.16g",d)>8) {
00242             sout.printf(F_L,"%s%ld", p || d>=0 ? "":"-", p);
00243         }; return sout;
00244     }
00245
00246     if (int(niter)<1) {
00247         sout.printf(F_L,"%.5g",d);
00248         return sout;
00249     }

```

```

00250
00251     int e1, e2=-1; unsigned m,m2; long q, p2,q2;
00252     double r,r2, d_ =d, d2, d2_, a=:fabs(d); if (a>1) { eps*=a; }
00253
00254     e1=Rational(d,p,q,&r,&m,NULL,niter,pqmax,eps,eps2,vflag>1);
00255
00256     if (e1 || abs(p)>999 || abs(q)>999) {
00257         if ((d2_ =d_*d_)>eps) { d2=d2_;
00258             e2=Rational(d2,p2,q2,&r2,&m2,NULL,niter,pqmax,
00259                 d2<0.01 ? eps *a : eps,
00260                 d2<0.01 ? eps2*a : eps2, vflag>1
00261             );
00262         }
00263
00264         if (!e1 && !e2) {
00265             double q1=:fabs(d*d-d2_), q2=:fabs(d2-d2_);
00266             if (q1>q2) { if (vflag>1) {
00267                 PRINTF(" ==> picking sqrt() over plain (e=%.3g / %.3g)\n",
00268                     :sqrt(q2), :sqrt(q1)); }
00269             e1=-2;
00270         }
00271     }
00272
00273     if (!e1) {
00274         sout.printf(F_L,"%ld/%ld",p,q);
00275         return sout;
00276     }
00277     if (e2) {
00278         sout.printf(F_L,"%0.5g",d);
00279         return sout;
00280     }
00281
00282     wstring sx(24);
00283
00284     char sgn[]="-"; if (d>=0) { sgn[0]=0; }
00285     sout.data[0]=0;
00286
00287     if (q2==1) {
00288         sout.printf(F_L,"%s%s", sgn,Wb::surdStrf(sx,"%ld",p2));
00289     }
00290     else if (p2==1) {
00291         sout.printf(F_L,"%s1/%s",sgn,Wb::surdStrf(sx,"%ld",q2));
00292     }
00293     else { r=:sqrt(double(q2));
00294         if (:fabs(r-:round(r))<eps) {
00295             sout.printf(F_L,"%s%s/%g",sgn,Wb::surdStrf(sx,"%ld",p2),r);
00296         }
00297     }
00298     if (!sout.data[0]) { r=:sqrt(double(p2));
00299         if (:fabs(r-:round(r))<eps)
00300             { sout.printf(F_L,"%s%g/%s",sgn,r,Wb::surdStrf(sx,"%ld",q2)); }
00301         else { sout.printf(F_L,"%s%s",sgn,Wb::surdStrf(sx,"(%ld/%ld)",p2,q2)); }
00302     }
00303     return sout;
00304 };
00305
00306 template<>
00307 double Wb::SkipZeros(double *d, unsigned n, double eps){
00308     unsigned i=0; double s2=0;
00309     for (; i<n; ++i) {
00310         if (:fabs(d[i])<eps) { s2+=(d[i]*d[i]); d[i]=0; }
00311     }
00312     return s2;
00313 };
00314
00315 template<>
00316 double Wb::SkipZeros(long double *d, unsigned n, double eps){
00317     unsigned i=0; double s2=0;
00318     for (; i<n; ++i) {
00319         if (:fabs(d[i])<eps) { s2+=(d[i]*d[i]); d[i]=0; }
00320     }
00321     return s2;
00322 };
00323
00324 template<>
00325 double Wb::SkipZeros(wbcomplex *z, unsigned n, double eps){
00326     unsigned i=0; double s2=0;
00327     for (; i<n; ++i) {
00328         if (:fabs(z[i].r)<eps) { s2+=(z[i].r*z[i].r); z[i].r=0; }
00329         if (:fabs(z[i].i)<eps) { s2+=(z[i].i*z[i].i); z[i].i=0; }
00330     }
00331     return s2;
00332 };
00333
00334
00335
00336

```

```

00337     return s2;
00338 };
00339
00340 inline wstring Wb::size2Str(
00341     double n,
00342     char p
00343 ){
00344     if (!n) { return "0"; }
00345
00346     unsigned l=0, m=16; size_t x;
00347     char s[m], ustr[8], fmt[m];
00348
00349     x=size_t(1)<<30; if (n>x) { strcpy(ustr,"GB"); } else {
00350     x=size_t(1)<<20; if (n>x) { strcpy(ustr,"MB"); } else {
00351     x=size_t(1)<<10; if (n>x) { strcpy(ustr,"kB"); } else {
00352     x=size_t(1);
00353         strcpy(ustr,"bytes"); } } }
00354
00355     if (p<1 || p>9) {
00356         if (p=='c') { p=2; }
00357         else p=3;
00358     }
00359
00360     if (p>5) { snprintf(fmt,16,"%.%dg %s",p); }
00361     else { snprintf(fmt,16,"%.%dg%1.1s",p); }
00362     l=snprintf(s,m,fmt,n/x,ustr);
00363
00364     if (l>=m) wblog(FL,
00365         "ERR %s() string out of bounds (%d/%d)",FCT,l,m);
00366     return s;
00367 };
00368
00369 inline const char* Wb::filename(const char *s) {
00370     if (s==NULL) { wblog(FL,"WRN %s() got NULL string",FCT); }
00371     else {
00372         for (int i=strlen(s)-1; i>=0; --i) {
00373             if (s[i]=='/' || s[i]=='\\') return (s+i+1); }
00374     }
00375     return s;
00376 };
00377
00378 size_t Wb::getFileSize(const char* f) {
00379     struct stat S;
00380     if (!f || !f[0] || stat(f, &S)!=0) { return 0; }
00381     return S.st_size;
00382 };
00383
00384 #ifdef __APPLE__
00385
00386 double Wb::GET_mtime(const char *F, int L, const char *f, const struct stat &S) {
00387
00388     if (S.st_mtimespec.tv_nsec>1E9) wblog(F,L,
00389         "WRN %s got mtime %ld @ %.3g", f?f:"", f? "() ":"",
00390         S.st_mtimespec.tv_sec, 1E-9*S.st_mtimespec.tv_nsec
00391     );
00392     return (S.st_mtimespec.tv_sec + 1E-9*double(S.st_mtimespec.tv_nsec));
00393 };
00394
00395 #else
00396
00397 double Wb::GET_mtime(const char *F, int L, const char *f, const struct stat &S) {
00398
00399     if (S.st_mtim.tv_sec!=S.st_mtime || S.st_mtim.tv_nsec>1E9)
00400         wblog(F,L,"WRN %s() got difference in mtime (%ld @ %.3g)",
00401             f?f:"",S.st_mtim.tv_sec, S.st_mtime, 1E-9*S.st_mtim.tv_nsec
00402         );
00403     return S.st_mtim.tv_sec + 1E-9*double(S.st_mtim.tv_nsec);
00404 };
00405
00406 #endif
00407
00408 #ifdef __APPLE__
00409
00410 double Wb::GET_atime(const char *F, int L, const char *f, const struct stat &S) {
00411
00412     if (S.st_atimespec.tv_nsec>1E9) wblog(F,L,
00413         "WRN %s got atime %ld @ %.3g", f?f:"", f? "() ":"",
00414         S.st_atimespec.tv_sec, 1E-9*S.st_atimespec.tv_nsec
00415     );
00416     return S.st_atimespec.tv_sec + 1E-9*double(S.st_atimespec.tv_nsec);
00417 };
00418
00419 #else
00420
00421 double Wb::GET_atime(const char *F, int L, const char *f, const struct stat &S) {
00422
00423     if (S.st_atim.tv_sec!=S.st_atime || S.st_atim.tv_nsec>1E9)

```



```

00424         wblog(F,L,"WRN %s() got difference in atime (%ld @ %.3g)",
00425         f?f:" ",S.st_atim.tv_sec, S.st_atime, 1E-9*S.st_atim.tv_nsec
00426         );
00427         return S.st_atim.tv_sec + 1E-9*double(S.st_atim.tv_nsec);
00428     };
00429
00430 #endif
00431
00432 #ifdef __APPLE__
00433
00434 double Wb::GET_ctime(const char *F, int L, const char *f, const struct stat &S) {
00435
00436     if (S.st_ctimespec.tv_nsec>1E9) wblog(F,L,
00437     "WRN %s got ctime %ld @ %.3g", f?f:" ", f?":":"",
00438     S.st_ctimespec.tv_sec, 1E-9*S.st_ctimespec.tv_nsec
00439     );
00440     return S.st_ctimespec.tv_sec + 1E-9*double(S.st_ctimespec.tv_nsec);
00441 };
00442
00443 #else
00444
00445 double Wb::GET_ctime(const char *F, int L, const char *f, const struct stat &S) {
00446
00447     if (S.st_ctim.tv_sec!=S.st_ctime || S.st_ctim.tv_nsec>1E9)
00448     wblog(F,L,"WRN %s() got difference in ctime (%ld @ %.3g)",
00449     f?f:" ",S.st_ctim.tv_sec, S.st_ctime, 1E-9*S.st_ctim.tv_nsec
00450     );
00451     return S.st_ctim.tv_sec + 1E-9*double(S.st_ctim.tv_nsec);
00452 };
00453
00454 #endif
00455
00456 double Wb::getFileTime(const char* f, char w) {
00457
00458     struct stat S;
00459     if (!f || !f[0] || stat(f, &S)!=0) { return 0; }
00460
00461     switch (w) {
00462     case 'm': return GET_mtime(FLF,S);
00463     case 'a': return GET_atime(FLF,S);
00464     case 'c': return GET_ctime(FLF,S);
00465
00466     case 'M': return S.st_mtime;
00467     case 'A': return S.st_atime;
00468     case 'C': return S.st_ctime;
00469
00470     default: wblog(FL,"ERR %s() invalid w=%c<%d>",FCT,w,w);
00471     }
00472
00473     return 0;
00474 };
00475
00476 bool Wb::fexist(const char *f, char type) {
00477
00478     if (!f || !f[0]) wblog(FL,"ERR %s() invalid (empty) file name",FCT);
00479
00480     struct stat S;
00481     if (stat(f, &S)!=0) { return 0; }
00482
00483     if (type=='f') return (S.st_mode & S_IFREG); else
00484     if (type=='d') return (S.st_mode & S_IFDIR); else
00485     return 1;
00486 };
00487
00488 bool Wb::isFile(const char *f) {
00489     if (f && f[0]) {
00490         struct stat S;
00491         if (stat(f,&S)==0) { return (S.st_mode & S_IFREG); }
00492     }
00493     return 0;
00494 };
00495
00496 bool Wb::isDir(const char *f) {
00497     if (f && f[0]) {
00498         struct stat S;
00499         if (stat(f,&S)==0) { return (S.st_mode & S_IFDIR); }
00500     }
00501     return 0;
00502 };
00503
00504 wstring Wb::hostname(unsigned len) {
00505
00506     wstring s(MAX(16U,len));
00507     int e=gethostname(s.data,s.len);
00508
00509     if (e && errno==ENAMETOOLONG) { unsigned i=0;
00510         for (; i<s.len; ++i) {

```

```

00511         if (s.data[i]=='.') { s.data[i]=0; break; }
00512     }
00513     if (i && i<s.len) { e=0; }
00514 }
00515 if (e) { unsigned i=0;
00516     for (; i<s.len; ++i) {
00517         if (!isprint(s.data[i])) { s.data[i]=0; break; }
00518     }
00519     if (i<4) { strcpy(s.data,"(host!?)"); } else
00520     if (i>=s.len) {
00521         s.data[s.len-1]=0;
00522     }
00523     wblog(FL,"WRN %s() received e=%d (%s)",FCT,errno,s.data);
00524 }
00525 return s;
00526 };
00527
00528 wbststring Wb::hostid(char pflag) {
00529     wbststring hid=hostname(pflag ? 24 : 16);
00530
00531     unsigned i=0, k=0, n=hid.len; char *s=hid.data;
00532     if (!n || !s[0] || !isalpha(s[0])) {
00533         wblog(FL,"WRN %s() received hostname '%s'",FCT,s);
00534         return hid;
00535     }
00536     for (; i<n; ++i) {
00537         if (s[i]=='-') { k=i+1; } else
00538         if (s[i]=='.') { if (i>4 && k+3<i) { s[i]=0; n=i; break; } } else
00539         if (!s[i]) { n=i; break; }
00540     }
00541     if (i<n) { n=i; } else
00542     if (i==hid.len) { s[i-1]=0;
00543         wblog(FL,"WRN %s() received '%s'",FCT,s);
00544         return hid;
00545     }
00546     if (k && k+1<n) {
00547         for (i=k; i<n; ++i) { s[i-k]=s[i]; }
00548         s[i-k]=0; i-=k;
00549     }
00550     if (pflag && i+2<hid.len) { char fmt[]="/%d";
00551         if (pflag=='x' || pflag=='X') { fmt[2]=pflag; }
00552         snprintf(s+i, hid.len-i, fmt, getpid());
00553     }
00554     return hid;
00555 };
00556
00557 template <class TI> inline
00558 TI Wb::sub2ind(const TI *s, const TI *I, unsigned n){
00559     if (n) {
00560         unsigned j=n-1; TI k=I[j];
00561         for (--j; j<n; --j) k=k*s[j]+I[j];
00562         return k;
00563     }
00564     else { wblog(FL,"ERR %s() got null set !?", FCT); return -1; }
00565 };
00566
00567 template <class T1, class T2> inline
00568 void Wb::ind2sub(T1 k, const T1 *s, T2 *I, unsigned n) {
00569     unsigned x,i=0;
00570     for (; i<n; ++i) {
00571         if (s[i]) { x=k/s[i]; I[i]=k-x*s[i]; k=x; }
00572         else {
00573             wblog(FL,"ERR %s() got null-size %s",FCT,
00574                 SSTR(wbvector<T1>(n,s,'r')));
00575         }
00576     }
00577     if (k) { wblog(FL,
00578         "ERR %s() index out of bounds\n(%d => [%s; %s])",FCT,
00579         (wbvector<T2>(n,I,'r')+1).toStr().data,
00580         SSTR(wbvector<T1>(n,s,'r')));
00581     }
00582 };
00583
00584 int Wb::atoi(const char *s, int &iout,
00585     unsigned n,
00586     char white
00587 ){
00588     if (!s || !s[0]) { return -1; }
00589     unsigned i=0; if (int(n)<=0) n=9;
00590     if (white) {
00591         for (; s[i]; ++i) { if (s[i]!=' ') break; }
00592         if (i) {

```

```

00598         if (!s[i]) { return -2; }
00599         s+=i; i=0;
00600     }
00601 }
00602
00603 if (white<0) {
00604     if (s[0]=='+' || s[0]=='-') { if (!s[++i]) return -3; }
00605 }
00606
00607 for (; s[i] && i<n; ++i) {
00608     if (!isdigit(s[i])) return -4;
00609 }
00610 if (i==n && isdigit(s[i])) return -5;
00611
00612 iout=::atoi(s);
00613 return i;
00614 };
00615
00616 int Wb::isUIntString(const char *s, char white) {
00617     if (!s) { return -1; }
00618
00619     if (white) {
00620         unsigned i=0, n=0, m=0;
00621
00622         for (; s[i]; ++i) {
00623             if (isdigit(s[i])) ++m; else
00624             if (isspace(s[i])) { if (m) { ++n; m=0; } }
00625             else return -(100+n);
00626         }
00627         return (m ? n+1 : n);
00628     }
00629     else {
00630         unsigned i=0, n=0;
00631
00632         while (isspace(*s)) ++s;
00633         while (isdigit(s[n])) ++n;
00634         for (i=n; isspace(s[i]); ++i) {};
00635
00636         if (!s[i])
00637             return n;
00638         else return -(10+n);
00639     }
00640 };
00641
00642 int Wb::dstrlen_utf8(const char *s) {
00643     int q=0;
00644     if (!s || !s[0]) { return q; }
00645
00646     for (int i=0, l=strlen(s); i<l && s[i]; ++i) {
00647         if (s[i]<0 || s[i]>127) {
00648             if ((s[i] & 0xE0) == 0xC0) { q+=1; } else
00649             if ((s[i] & 0xF0) == 0xE0) { q+=2; } else
00650             if ((s[i] & 0xF8) == 0xF0) { q+=3; } else
00651             { q=0; break; }
00652         }
00653     }
00654     return q;
00655 };
00656
00657 template <class T>
00658 wstring Wb::bits(const T &x, char compact) {
00659     wstring s_;
00660     int i, j=0, n=sizeof(T), l=9*n; { s_.init(l); if (!l) return s_; }
00661     char *s; unsigned char c;
00662     const char *sx = (char*)(&x);
00663
00664     if (compact) {
00665         for (i=0; i<n; ++i) { if (sx[i]) j=i; }
00666         n=j+1; l=9*n;
00667     }
00668     s=s_.data+l-1; s[0]=0;
00669
00670     for (i=0; i<n; ++i, s-=9) { c=sx[i];
00671         for (j=1; j<=8; ++j, c>>1) { s[-j]=(c&1 ? '1' : '-'); }
00672         if (i+1<n) s[-j]=' ';
00673     }
00674
00675     return s_;
00676 };
00677
00678 void Wb::IOstat::print(const char *istr,
00679     const char *hstr, const char *xstr1, const char *xstr2
00680 ){
00681     if (hstr && hstr[0]) printf("\n I/O stats for %-9s "
00682         "#read%-6s size_read #write%-5s size_write\n",
00683         hstr, xstr1?xstr1:"", xstr2?xstr2:"");
00684 }

```

```

00685     char s[64];
00686     snprintf(s, 30, "%4ld", aux1);
00687     snprintf(s+30, 30, "%4ld", aux2);
00688
00689     printf("      %-19s %6d %4s %10s %6d %4s %10s\n", istr ? istr:"",
00690         nread, aux1 ? s : "", size2Str(sread).data,
00691         nwrite, aux2 ? s+30 : "", size2Str(swrite).data
00692     );
00693 };
00694
00695 bool Wb::is_finite(const wbcomplex *x, size_t n) {
00696     for (size_t i=0; i<n; ++i) { if (!x[i].isfinite()) { return 0; }}
00697     return 1;
00698 };
00699
00700 wbstring cpu_time::toStr(char flag) {
00701
00702     size_t l=0, n=32; char tstr[n];
00703
00704     cpu_time dt=since();
00705     double t=(flag=='c' ? dt.tcpu : dt.tsys);
00706
00707     if (t<0) {
00708         wblog(FL, "WRN %s() got negative time (%ld)", FCT, long(t)); t=-t; }
00709
00710     if (t<60) {
00711         l=snprintf(tstr, n, "%.4g sec", t); }
00712     else if (t<1E14) {
00713         int s,m,h,d; long x=t,
00714         fac=24*3600; d=t/fac; x-=d*fac;
00715         fac=3600; h=x/fac; x-=h*fac;
00716         fac=60; m=x/fac; x-=m*fac; s=x; x-=s;
00717
00718         if (d)
00719             l=snprintf(tstr, n, "%d-%02d:%02d:%02d", d, h, m, s);
00720         else if (h || m>9)
00721             l=snprintf(tstr, n, "%02d:%02d:%02d", h, m, s);
00722         else l=snprintf(tstr, n, "%02d:%02d", m, s);
00723
00724         if (t>2E9) l=-1;
00725     }
00726     else {
00727         l=snprintf(tstr, n, "(WRN %.3g yrs (%ld sec))",
00728             t/(3600*24*365), long(t));
00729         l=-1;
00730     }
00731
00732     if (l>=n) {
00733         long tl=(flag=='c' ? dt.tcpu : dt.tsys);
00734         double yrs=double(tl)/(3600*24*365);
00735         wblog(FL,
00736             "WRN cpu_time() %s '%s'\n'%c' dt=%ld => %.12g yrs (%d) !?\n"
00737             "%15.12g %15.12g %% now [CPU/SYS]\n"
00738             "%15.12g %15.12g %% then",
00739             int(l)<0 ? "possibly invalid CPU time":"string out of bounds\n",
00740             tstr, flag, tl, yrs, l, dt.time('c'), dt.time('s'), tcpu, tsys
00741         );
00742     }
00743
00744     return tstr;
00745 };
00746
00747 #ifdef __APPLE__
00748
00749 double Mac::getMemSize(const char *tag) {
00750
00751     mach_msg_type_number_t count = HOST_VM_INFO_COUNT;
00752     vm_statistics_data_t S;
00753
00754     if (host_statistics(
00755         mach_host_self(),
00756         HOST_VM_INFO,
00757         (host_info_t)&S,
00758         &count
00759     )) wblog(FL, "ERR %s() %s", FCT, strerror(errno));
00760
00761     long n = getpagesize(),
00762         sused = S.wire_count + S.active_count,
00763         sfree = S.inactive_count + S.free_count;
00764
00765     if (!tag || !tag[0]) wblog(FL, "ERR %s() got empty tag !?", FCT);
00766
00767     if (!strcmp(tag, "total")) { return n*(sused+sfree); }
00768     if (!strcmp(tag, "free" )) { return n*(sfree); }
00769     if (!strcmp(tag, "usage")) {
00770         return 100*double(sused)/(sfree+sused);
00771     }

```

```

00772
00773     wblog(FL,"ERR %s() tag '%s' not yet defined",FCT,tag);
00774     return 0;
00775 };
00776
00777 size_t Mac::getProcMemSize(pid_t p, char res) {
00778     struct task_basic_info S;
00779     mach_msg_type_number_t len = TASK_BASIC_INFO_COUNT;
00780
00781     task_t tid = MACH_PORT_NULL;
00782     task_for_pid(current_task(), p ? p : getpid(), &tid);
00783
00784     if (task_info(
00785         tid, TASK_BASIC_INFO, (task_info_t)&S, &len)
00786     ){ wblog(FL,"ERR %s() %s",FCT,strerror(errno)); }
00787
00788     return (res ? S.resident_size : S.virtual_size);
00789 };
00790
00791 size_t Wb::getProcSize(
00792     const char *tag QS_UNUSED_VAR,
00793     const pid_t &p
00794 ) { return Mac::getProcMemSize(p); };
00795
00796 #else
00797
00798 size_t Wb::getProcSize(const char *tag, const pid_t &p) {
00799     const int n=32; char f[n];
00800     int i=snprintf(f,n,"/proc/%d/status",p);
00801     if (i>n) wblog(FL,
00802         "ERR %s() string out of bounds (%s; %d)",FCT,f,n);
00803     return Wb::getProcSize(tag,f);
00804 };
00805
00806 size_t Wb::getProcSize(const char *tag, const char *f) {
00807     char *s, *line=NULL, u[8];
00808     size_t val; unsigned i=0, m=0; size_t len=0; ssize_t n;
00809
00810     static unsigned ncall=0;
00811
00812     FILE *fid; ++ncall;
00813     if (!f || !f[0] || !(fid=fopen(f,"r"))) {
00814         wblog(FL,"WRN failed to access %s '%s' (%d)",f?f:"","tag?tag:",ncall);
00815         return 0;
00816     }
00817
00818     while ((n=getline(&line,&len,fid))>=0) { ++i;
00819         if (n) line[n-1]=0;
00820         if ((s=strstr(line,tag))) { u[0]=0; ++m;
00821             sscanf(s,"%*s %lu %s",&val,u);
00822             if (strlen(u)==2 && u[1]=='B') {
00823                 if (u[0]=='k' || u[0]=='K') { val*=(1<<10); } else
00824                 if (u[0]=='M') { val*=(1<<20); } else
00825                 if (u[0]=='G') { val*=(1<<30); }
00826                 else wblog(FL,"WRN %s() got unit '%s'",FCT,u);
00827             }
00828             else wblog(FL,
00829                 "WRN %s() invalid unit >%s< (%d)",FCT,u,strlen(u));
00830             break;
00831         }
00832     }
00833
00834     if (line) { free(line); line=NULL; }
00835     fclose(fid); fid=NULL;
00836
00837     if (!m) wblog(FL,"WRN %s() '%s' not found in %s",FCT,tag,f);
00838
00839     return val;
00840 };
00841
00842 long Wb::getCpuInfo(const char *tag, const char *f) {
00843     char *s, *line=NULL;
00844     int m=0, val=0, i=0, vmax=-1; size_t len=0; ssize_t n;
00845
00846     FILE *fid=fopen(f,"r");
00847     if (fid==NULL) {
00848         wblog(FL,"WRN failed to access %s '%s'",f?f:"","tag?tag:");
00849         return 0;
00850     }
00851
00852     if (!strcmp(tag,"processor")) {
00853         while ((n=getline(&line,&len,fid))>=0) { ++i;
00854             if (n) line[n-1]=0;
00855             if ((s=strstr(line,tag))) { ++m;

```

```

00859         sscanf(s,"%*s : %d",&val); if (vmax<val) vmax=val;
00860     }
00861 }
00862
00863     if (vmax+1!=m) wblog(FL,
00864         "WRN %s() inconsistent ncores = %d/%d !?",FCT,vmax,m);
00865     val=m;
00866 }
00867 else wblog(FL,"ERR %s() invalid tag %s -> %s",FCT,f,tag);
00868
00869     if (line) { free(line); line=NULL; }
00870     fclose(fid); fid=NULL;
00871
00872     if (!m) wblog(FL,"WRN %s() '%s' not found in %s",FCT,tag,f);
00873
00874     return val;
00875 };
00876
00877 #endif
00878
00879 #ifdef __APPLE__
00880
00881 size_t wbsys::getMemTot() { return Mac::getMemSize("total"); };
00882 size_t wbsys::getMemFree() { return Mac::getMemSize("free"); };
00883
00884 int wbsys::getNumCores() {
00885     int n=-1;
00886
00887     size_t l=4;
00888     int e=sysctl((int [2]){ CTL_HW, HW_NCPU },2,&n,&l,NULL,0);
00889
00890     if (e || n<1) {
00891         wblog(FL,"ERR %s() sysctl returned e=%d (n=%d)\n%s",
00892             FCT,e,n, e? strerror(errno): "");
00893     }
00894     return n;
00895 };
00896
00897 int wbsys::getCacheLineSize() {
00898     size_t n=-1;
00899     size_t l=sizeof(n);
00900     int e=sysctl((int [2]){ CTL_HW, HW_CACHELINE },2,&n,&l,NULL,0);
00901
00902     if (e || n<1) {
00903         wblog(FL,"ERR %s() sysctl returned e=%d (n=%d)\n%s",
00904             FCT,e,n, e? strerror(errno): "");
00905     }
00906     return n;
00907 };
00908
00909 #else
00910
00911 size_t wbsys::getMemTot() {
00912     struct sysinfo I; sysinfo(&I); return I.totalram;
00913 };
00914
00915 size_t wbsys::getMemFree() {
00916     struct sysinfo I; sysinfo(&I); return I.freeram;
00917 };
00918
00919 size_t wbsys::getSwapTot() {
00920     struct sysinfo I; sysinfo(&I); return I.totalswap;
00921 };
00922
00923 size_t wbsys::getSwapFree() {
00924     struct sysinfo I; sysinfo(&I); return I.freeswap;
00925 };
00926
00927 int wbsys::getNumCores() {
00928
00929     return sysconf(_SC_NPROCESSORS_ONLN);
00930 };
00931
00932 int wbsys::getCacheLineSize() {
00933     int n=sysconf(_SC_LEVEL1_DCACHE_LINESIZE);
00934     if (n<8) { wblog(FL,"ERR %s() got L1d cache-linesize ls=%d !?",FCT,n); }
00935     return n;
00936 }
00937
00938 #endif
00939
00940 void Wb::ResSummary(const char *F, int L, const char *istr) {
00941
00942     static int do_cleanup=1;
00943     if (F==NULL && L) { do_cleanup=L; return; }
00944
00945     #ifndef __APPLE__

```

```

00946     if (do_cleanup) { do_cleanup=0; wbttop PS;
00947         wbstring s1(PS.VmSize2Str()), s2(PS.VmPeak2Str());
00948         if (istr) { sprintf_str("%s() ",istr); } else { str[0]=0; }
00949
00950         wblog(F_L," * %sVMEM size: %s / %s", str, s1.data, s2.data);
00951         wblog(F_L," * %scputime : %s",str, STR2(gCPUTime,'c'));
00952     }
00953 #else
00954     if (do_cleanup) { do_cleanup=0;
00955         if (istr) { sprintf_str("%s() ",istr); } else { str[0]=0; }
00956         wblog(F_L," * %sVMEM size / cputime (skipped for OSX)", str);
00957     }
00958 #endif
00959 };
00960
00961 void Wb::MemStat(const char *F, int L, char lflag) {
00962     wbttop().MemStat(F,L,lflag);
00963 };
00964
00965 inline wbstring wbsys::MemTot2Str() {
00966     return Wb::size2Str(getMemTot()); };
00967
00968 inline wbstring wbsys::MemFree2Str() {
00969     return Wb::size2Str(getMemFree()); };
00970
00971 inline wbstring wbsys::SwapTot2Str() {
00972     return Wb::size2Str(getSwapTot()); };
00973
00974 inline wbstring wbsys::SwapFree2Str() {
00975     return Wb::size2Str(getSwapFree()); };
00976
00977 #ifdef __APPLE__
00978
00979 char wbsys::checkSwapSpace(const char *F, int L) {
00980
00981     static double xref=0.25;
00982
00983     struct xsw_usage S; size_t l=sizeof(S);
00984     int e=sysctl((int [2]){ CTL_VM, VM_SWAPUSAGE },2,&S,&l,NULL,0);
00985
00986     if (e) wblog(F_L,
00987         "ERR %s() sysctl returned e=%d\n(%s)",FCT,e,strerror(errno));
00988
00989     if (S.xsu_total<=0 || double(S.xsu_avail)/S.xsu_total>xref) { return 0; }
00990
00991     wblog(F_L,"WRN free swap space: %s / %s @ %.3g",
00992         Wb::size2Str(S.xsu_avail).data,
00993         Wb::size2Str(S.xsu_total).data, xref); xref-=0.05;
00994     return 1;
00995 };
00996
00997 #else
00998
00999 char wbsys::checkSwapSpace(const char *F, int L) {
01000     static double xref=0.25;
01001     double x=getSwapTot(); if (x<=0) return 0;
01002     x=getSwapFree()/x; if (x>xref) { return 0; }
01003
01004     xref-=0.05; wblog(F_L,"WRN free swap space: %s / %s",
01005         SwapFree2Str().data, SwapTot2Str().data);
01006     return 1;
01007 };
01008 #endif
01009
01010 inline wbstring wbttop::VmSize2Str() const {
01011     return Wb::size2Str(getVmSize()); };
01012
01013 inline wbstring wbttop::VmPeak2Str() const {
01014     return Wb::size2Str(getVmPeak()); };
01015
01016 int wbttop::runningLarge(const char *F, int L,
01017     double th0,
01018     double fac
01019 ){
01020     static double thr=th0;
01021     int r=0; size_t s0, M;
01022
01023     s0=wbsys::getMemTot();
01024     M=getVmPeak();
01025
01026     if (M<thr*s0) { wbsys::checkSwapSpace(F_L); }
01027     else {
01028         wblog(F_L,"%s() %s is using %s",FCT,myname,VmPeak2Str().data);
01029         thr*=fac; r=1;
01030     }
01031     doflush(); wblogBuf.flush();
01032

```

```

01033     return r;
01034 };
01035
01036 void Wbtop::MemStat(const char *F, int L, char lflag) const {
01037     static size_t vm_size_max=0;
01038     static size_t vm_size_last=0;
01039
01040     if (lflag=='m' || lflag=='M') {
01041         size_t q=getVmSize();
01042         if (vm_size_max*(lflag=='m' ? 1.10 : 1)<q)
01043             { vm_size_max=q; }
01044         else { lflag=0; }
01045     }
01046     else if (!lflag && getVmPeak()>0.66*systemem) {
01047         size_t q=getVmSize();
01048         if (!vm_size_last ||
01049             fabs((q-vm_size_last)/double(vm_size_last))>0.05
01050         ){ vm_size_last=q; lflag=2; }
01051     }
01052
01053     if (lflag) {
01054         double gfac=1/double(1<<30);
01055         wblog(F_L,"SYS %s() %.1fG (%.1f/%.0fG)",FCT,
01056             gfac*getVmSize(), gfac*getVmPeak(), gfac*systemem);
01057         doflush();
01058     }
01059 };
01060
01061 template <class T>
01062 size_t Wb::findfirst_sorted(const char *F, int L,
01063     const T* d0, const T* dd, size_t m, size_t N, size_t M, char lex
01064 ){
01065     if (!N || !M || m>=M) { if (F) wblog(F,L,
01066         "ERR %s() got empty/invalid input (%dx%d; %d",FCT,N,M,m);
01067         return -1;
01068     }
01069
01070     int q=Wb::cmpRange(dd,d0,m,lex);
01071     if (q>=0) {
01072         if (q>0) { if (F) wblog(F,L,
01073             "ERR %s() empty match (<first)",FCT);
01074             return -1;
01075         }
01076         return 0;
01077     }
01078
01079     size_t i1=0, i2=N-1, i=(i2+i1)/2;
01080     int q2=Wb::cmpRange(dd+i2*M,d0,m,lex);
01081
01082     if (q2<0) { if (F) wblog(F,L,
01083         "ERR %s() empty match (>last)",FCT);
01084         return N;
01085     }
01086
01087     while (i>i1) {
01088         q=Wb::cmpRange(dd+i*M,d0,m,lex);
01089         if (q<0) { i1=i; }
01090         else {
01091             if (q>0 && q!=q2) wblog(F_L,"ERR %s() "
01092                 "got unsorted data (upper i=%d: %d/%d) !?",FCT,i,q,q2);
01093             q2=q; i2=i;
01094         }
01095         i=(i1+i2)/2;
01096     }
01097
01098     if (q2) { if (F) wblog(F_L,
01099         "ERR %s() empty match (i=%d/%d @ %d,%d)",FCT,i,N,q,q2);
01100         return -1;
01101     }
01102     return i2;
01103 };
01104
01105 template <class T>
01106 size_t Wb::findlast_sorted(const char *F, int L,
01107     const T* d0, const T* dd, size_t m, size_t N, size_t M, char lex
01108 ){
01109     if (!N || !M || m>=M) { if (F) wblog(F,L,
01110         "ERR %s() got empty/invalid input (%dx%d; %d",FCT,N,M,m);
01111         return -1;
01112     }
01113
01114     int q1=Wb::cmpRange(dd,d0,m,lex);
01115
01116     if (q1>0) { if (F) wblog(F,L,
01117         "ERR %s() empty match (<first)",FCT);
01118         return -1;
01119     }

```



```

01120
01121     size_t i1=0, i2=N-1, i=(i2+i1)/2;
01122     int q=Wb::cmpRange(dd+i2*M,d0,m,lex);
01123
01124     if (q<=0) {
01125         if (q<0) { if (F) wblog(F,L,
01126             "ERR %s() empty match (>last)",FCT);
01127             return N;
01128         }
01129         return i2;
01130     }
01131
01132     while (i>i1) {
01133         q=Wb::cmpRange(dd+i*M,d0,m,lex);
01134         if (q>0) { i2=i; }
01135         else {
01136             if (q<0 && q!=q1) wblog(F_L,"ERR %s() "
01137                 "got unsorted data (lower i=%d: %d/%d) !?",FCT,i,q,q1);
01138             q1=q; i1=i;
01139         }
01140         i=(i1+i2)/2;
01141     }
01142
01143     if (q1) { if (F) wblog(F_L,
01144         "ERR %s() empty match (i=%d/%d @ %d,%d)",FCT,i,N,q1,q);
01145         return -1;
01146     }
01147     return i1;
01148 };
01149
01150 void Wb_srand() {
01151
01152     struct timespec tnow;
01153     clock_gettime(CLOCK_REALTIME,&tnow);
01154
01155     unsigned long low, high;
01156     __asm__ __volatile__ ("rdtsc" : "=a" (low), "=d" (high) : );
01157
01158     ::srand(::rand() ^ unsigned(getpid()) ^ high ^ low);
01159     ::srand(::rand() ^ tnow.tv_sec ^ tnow.tv_nsec);
01160 };
01161
01162 #ifdef QS_USING_MPFR
01163
01164 namespace Wb {
01165
01166 template<> inline
01167 quad& rand::rand_(quad &x) { return (x.Rand()); };
01168
01169 template<> inline
01170 quad& rand::randb(quad &x) {
01171     x.Rand(); x*=2; x-=1; return x;
01172 };
01173
01174 template<> inline
01175 quad& rand::rands(quad &x) {
01176     x.Rand(); x*=2; x-=1;
01177     return (x.Atanh());
01178 };
01179
01180 };
01181
01182 template <> inline
01183 Wb::quad WbUtil<Wb::quad>::eps() {
01184     Wb::quad x;
01185     return x.eps();
01186 };
01187
01188 #endif
01189
01190 template <> inline
01191 wcomplex WbUtil<wcomplex>::eps() { return DBL_EPSILON; };
01192
01193 #endif
01194

```

5.22 util.hh

```

00001 /* -----
00002 * Project : QSpace tensor library (v4.0)
00003 * Class   : QSpace utility routines
00004 *
00005 * Copyright 2024 Andreas Weichselbaum

```

```

00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_UTIL_HH__
00021 #define __WB_UTIL_HH__
00022
00023 /* ----- */
00024 namespace Wb {
00025 /* ----- */
00026
00027     int Rational(
00028         double &x,
00029         long &P, long &Q,
00030         double *dx=NULL,
00031         unsigned *nmax=NULL,
00032         wbvector<double>* aa=NULL,
00033         unsigned niter=6,
00034         long pqmax=-1,
00035         double eps=1E-8,
00036         double eps2=1E-14,
00037         char vflag=0
00038     );
00039
00040     template<class T>
00041     double FixRational(
00042         const char *F, int L, T *d, unsigned n, char rflag=0,
00043         unsigned niter=6,
00044         long pqmax=-1,
00045         double eps=1E-8,
00046         double eps2=1E-14,
00047         double *rz=NULL, double *ra=NULL, char vflag=0
00048     );
00049
00050     wbstring rat2Str(
00051         const char *F, int L, double d, unsigned niter=6,
00052         long pqmax=99999, double eps=1E-8, double eps2=1E-14, char vflag=0
00053     );
00054
00055     template<class T>
00056     double SkipZeros(
00057         T *d      QS_UNUSED_VAR,
00058         unsigned n QS_UNUSED_VAR,
00059         double eps QS_UNUSED_VAR =1e-14
00060     ){ return 0; };
00061
00062     void ResSummary(const char *F, int L, const char *istr=NULL);
00063
00064 #ifndef __WB_MEM_TRACK_HH__
00065     void MemStat(const char *F=0, int L=0, char lflag=0);
00066 #else
00067     void MemStat(const char *F, int L, char lflag);
00068 #endif
00069
00070 #ifndef __APPLE__
00071     long getCpuInfo(const char *tag, const char *f="/proc/cpuinfo");
00072     size_t getProcSize(const char *tag, const char *f="/proc/meminfo");
00073 #endif
00074     size_t getProcSize(const char *tag, const pid_t &p);
00075     size_t getProcSize(const char *tag, const char *f);
00076
00077     wbstring hostname(unsigned len=16);
00078     wbstring hostid(char pflag=0);
00079
00080     wbstring size2Str(double n, char p=4);
00081     const char* filename(const char *s);
00082
00083     size_t getFileSize(const char* f);
00084     double getFileInfo(const char* f, char w='m');
00085
00086     double GET_mtime(const char *F, int L, const char *f, const struct stat &S);
00087     double GET_atime(const char *F, int L, const char *f, const struct stat &S);
00088     double GET_ctime(const char *F, int L, const char *f, const struct stat &S);
00089
00090     bool fexist(const char *f, char type='*');
00091     bool isFile(const char *f);
00092     bool isDir(const char *f);

```

```

00093
00094 int mkdir(
00095     const char *F, int L, const char *path,
00096     char priv=0
00097 ){
00098     mode_t p=(S_IRWXU
00099         | S_IXGRP
00100         | S_IXOTH
00101     );
00102     if (!priv) p |= (
00103         S_IRGRP
00104         | S_IROTH
00105     );
00106
00107     int e;
00108
00109     #pragma omp critical (manage_WB_FILE_IO)
00110     { e::mkdir(path,p); }
00111
00112     if (e && F) wblog(F,L,
00113         "ERR %s() failed to create directory (e=%d)\n%s",FCT,e,path);
00114     return e;
00115 };
00116
00117 template <class TI>
00118 TI sub2ind(const TI *s, const TI *I, unsigned n);
00119
00120 template <class T1, class T2> inline
00121 void ind2sub(T1 k, const T1 *s, T2 *I, unsigned n);
00122
00123 int atoi(const char *s, int &iout, unsigned n, char white=0);
00124
00125 int isUIntString(const char *s, char white=1);
00126
00127 template <class T>
00128 size_t findfirst_sorted(const char *F, int L,
00129     const T* d0, const T* dd, size_t m, size_t N, size_t M,
00130     char lex=1
00131 );
00132
00133 template <class T>
00134 size_t findlast_sorted(const char *F, int L,
00135     const T* d0, const T* dd, size_t m, size_t N, size_t M,
00136     char lex=1
00137 );
00138
00139 template <class T0, class T> inline
00140 size_t findfirst_sorted(const char *F, int L,
00141     const T0* d0, const T* dd, size_t m, size_t N, size_t M, char lex=1
00142 ){
00143     T d2[m];
00144     for (unsigned i=0; i<m; ++i) d2[i]=T(d0[i]);
00145     return findfirst_sorted(F,L,d2,dd,m,N,M,lex);
00146 };
00147
00148 template <class T0, class T> inline
00149 size_t findlast_sorted(const char *F, int L,
00150     const T0* d0, const T* dd, size_t m, size_t N, size_t M, char lex=1
00151 ){
00152     T d2[m];
00153     for (unsigned i=0; i<m; ++i) d2[i]=T(d0[i]);
00154     return findlast_sorted(F,L,d2,dd,m,N,M,lex);
00155 };
00156
00157 int dstrlen_utf8(const char *s);
00158
00159 template <class T>
00160 wstring bits(const T &x, char compact=1);
00161
00162 class IOstat {
00163 public:
00164
00165     IOstat()
00166         : nread(0), nwrite(0), sread(0), swrite(0), aux1(0), aux2(0) {};
00167
00168     IOstat& init() {
00169         nread=nwrite=0; sread=swrite=aux1=aux2=0;
00170         return *this;
00171     }
00172
00173     void gotread (size_t s=0) { nread +=1; sread +=s; }
00174     void gotwrite(size_t s=0) { nwrite+=1; swrite+=s; }
00175
00176     bool isEmpty() {
00177         return (!nread && !nwrite && !sread && !swrite && !aux1 && !aux2);
00178     };
00179

```

```

00180     void print(const char *istr,
00181               const char *hstr=0, const char *xstr1=0, const char *xstr2=0);
00182
00183     unsigned nread, nwrite;
00184     size_t sread, swrite, aux1, aux2;
00185
00186     protected:
00187     private:
00188 };
00189
00190 template <class T, class T2>
00191 class iterator {
00192 public:
00193
00194     iterator(T* p_) : i(0), p(p_) {
00195         if (!p_) wblog(FL, "ERR %s() got p=NULL !?", FCT);
00196         n=p->numel();
00197     };
00198
00199     iterator& operator++() { ++i; return *this; }
00200     bool end() { return i>=n; }
00201
00202     size_t iter () const { return i; }
00203     size_t numel() const { return n; }
00204
00205     T2& operator[](size_t k) {
00206         if (!p) wblog(FL, "ERR %s() object not yet initialized !?", FCT);
00207         return (*p)[k];
00208     }
00209     T2& at() {
00210         if (i>=n) wblog(FL,
00211             "ERR %s() index out of bounds (%d/%d) !?", FCT, i, n);
00212         return (*p)[i];
00213     };
00214
00215     size_t i, n;
00216     T* p;
00217
00218     protected:
00219     private:
00220 };
00221
00222 template <class T, class T2>
00223 class iterator<T, T2*> {
00224 public:
00225
00226     iterator(T* p_) : i(0), p(p_) {
00227         if (!p_) wblog(FL, "ERR %s() got p=NULL !?", FCT);
00228         n=p->numel();
00229     };
00230
00231     iterator& operator++() { ++i; return *this; }
00232     bool end() { return i>=n; }
00233
00234     size_t iter () const { return i; }
00235     size_t numel() const { return n; }
00236
00237     T2* operator[](size_t k) {
00238         if (!p) wblog(FL, "ERR %s() object not yet initialized !?", FCT);
00239         return (*p)[k];
00240     }
00241     T2* at() {
00242         if (i>=n) wblog(FL,
00243             "ERR %s() index out of bounds (%d/%d) !?", FCT, i, n);
00244         return (*p)[i];
00245     };
00246
00247     size_t i, n;
00248     T* p;
00249
00250     protected:
00251     private:
00252 };
00253
00254 template <class T, class T2>
00255 class citerator {
00256 public:
00257
00258     citerator(const T* p_) : i(0), p(p_) {
00259         if (!p_) wblog(FL, "ERR %s() got p=NULL !?", FCT);
00260         n=p->numel();
00261     };
00262
00263     citerator& operator++() { ++i; return *this; }
00264     bool end() { return i>=n; }
00265
00266     size_t iter () const { return i; }

```

```

00267     size_t numel() const { return n; }
00268
00269     const T& operator[](size_t k) {
00270         if (!p) wblog(FL, "ERR %s() object not yet initialized !?", FCT);
00271         return (*p)[k];
00272     }
00273     const T& at() {
00274         if (i >= n) wblog(FL,
00275             "ERR %s() index out of bounds (%d/%d) !?", FCT, i, n);
00276         return (*p)[i];
00277     };
00278
00279     size_t i, n;
00280     const T* p;
00281
00282 protected:
00283 private:
00284 };
00285
00286 template <class T, class T2>
00287 class citerator<T, T2*> {
00288 public:
00289     citerator(const T* p_) : i(0), p(p_) {
00290         if (!p_) wblog(FL, "ERR %s() got p=NULL !?", FCT);
00291         n = p->numel();
00292     };
00293
00294     citerator& operator++() { ++i; return *this; }
00295     bool end() { return i >= n; }
00296
00297     size_t iter() const { return i; }
00298     size_t numel() const { return n; }
00299
00300     const T2* operator[](size_t k) {
00301         if (!p) wblog(FL, "ERR %s() object not yet initialized !?", FCT);
00302         return (*p)[k];
00303     }
00304     const T2* at() {
00305         if (i >= n) wblog(FL,
00306             "ERR %s() index out of bounds (%d/%d) !?", FCT, i, n);
00307         return (*p)[i];
00308     };
00309
00310     size_t i, n;
00311     const T* p;
00312
00313 protected:
00314 private:
00315 };
00316
00317 class rand {
00318 public:
00319     rand() {};
00320
00321     double get_dfac() { return dfac; }
00322
00323     template <class T>
00324     T& rand_(T &x);
00325
00326     template <class T>
00327     T& rand_(T &x, double fac);
00328
00329     template <class T>
00330     T& rand(T &x);
00331
00332     template <class T>
00333     T& randb(T &x);
00334
00335     template <class T>
00336     void scale_rand(T *x, size_t n, double fac) {
00337         for (size_t i=0; i<n; ++i) { x[i]*=fac; }
00338     }
00339
00340 protected:
00341 private:
00342     static double dfac;
00343 };
00344
00345 double rand::dfac = (1.-1./double(long(1)<<52)) / double(RAND_MAX);
00346
00347 template<> inline
00348 double& rand::rand_(double &x) { return (x=dfac*::rand()); };
00349
00350

```

```

00354 template<> inline
00355 int& rand::rand_(int &x) { return (x=(100*dfac)*::rand()); };
00356
00357 template<> inline
00358 unsigned& rand::rand_(unsigned&x) { return (x=(100*dfac)*::rand()); };
00359
00360 template<> inline
00361 long& rand::rand_(long &x) { return (x=(100*dfac)*::rand()); };
00362
00363 template<> inline
00364 size_t& rand::rand_(size_t &x) { return (x=(100*dfac)*::rand()); };
00365
00366 template<> inline
00367 int& rand::rand_(int&x, double fac) {
00368     return (x=(fac*dfac)*::rand());
00369 };
00370
00371 template<> inline
00372 unsigned& rand::rand_(unsigned &x, double fac) {
00373     return (x=(fac*dfac)*::rand());
00374 };
00375
00376 template<> inline
00377 long& rand::rand_(long &x, double fac) {
00378     return (x=(fac*dfac)*::rand());
00379 };
00380
00381 template<> inline
00382 size_t& rand::rand_(size_t &x, double fac) {
00383     return (x=(fac*dfac)*::rand());
00384 };
00385
00386 template<> inline
00387 double& rand::randb(double &x) { return (x=(2*dfac)*::rand()-1.); }
00388
00389 template<> inline
00390 int& rand::randb(int &x) { return (x=(199*dfac)*::rand()-99); }
00391
00392 template<> inline
00393 double& rand::rands(double &x) {
00394     x=(2*dfac)*::rand()-1;
00395     if (fabs(x)>=1) wblog(FL,"ERR %s() got x=%.4g (1%+.4g) !?",FCT,x,x-1);
00396     return (x=atanh(x));
00397 };
00398
00399 template<> inline
00400 int& rand::rands(int &x) {
00401     double d=dfac*::rand();
00402     if (fabs(d)>=1) wblog(FL,"ERR %s() got d=%.4g (1%+.4g) !?",FCT,d,d-1);
00403     return (x=(100*atanh(d)+0.5));
00404 };
00405
00406 template<> inline
00407 void rand::scale_rand(int *x, size_t n, double fac) {
00408     fac/=double(99);
00409     for (size_t i=0; i<n; ++i) { x[i]=((double(x[i]))*fac); }
00410 };
00411
00412 template <class T,
00413 typename std::enable_if< std::is_floating_point<T>::value , T>::type* = nullptr >
00414 bool is_finite(const T *x, size_t n) {
00415     for (size_t i=0; i<n; ++i) { if (!::isfinite(x[i])) { return 0; } }
00416     return 1;
00417 };
00418
00419 bool is_finite(const wcomplex *x, size_t n);
00420
00421 template <class T,
00422 typename std::enable_if< is_integral<T>::value , T>::type* = nullptr >
00423
00424 bool is_finite(
00425     const T *x QS_UNUSED_VAR,
00426     size_t n QS_UNUSED_VAR) { return 1; }
00427
00428 };
00429
00430 #ifdef __APPLE__
00431
00432 namespace Mac {
00433     double getMemSize(const char *tag);
00434     size_t getProcMemSize(
00435         pid_t p=0,
00436         char res=1
00437     );
00438 };
00439
00440 #endif

```

```

00441
00442     void wb_srand();
00443
00444 template <class T>
00445 class WbUtil {
00446     public:
00447
00448         static void adjust_tnorm(char &tnorm);
00449
00450         static constexpr bool isPOD() {
00451             return std::is_trivially_copyable<T>::value;
00452         };
00453
00454         static bool isFloat();
00455         static bool isInt();
00456
00457         static constexpr char isComplex() { return 0; };
00458
00459         static char hasConj() { return isComplex(); };
00460         static char isReal() {
00461             char q=isComplex(); if (q>=0) { q=(q? 0 : 1); }
00462             return q;
00463         };
00464
00465         T eps();
00466
00467     protected:
00468     private:
00469 };
00470
00471
00472 template <class T> inline
00473 void WbUtil<T>::adjust_tnorm(char &tnorm){ tnorm=0; };
00474
00475 template <> inline
00476 void WbUtil<wbcomplex>::adjust_tnorm(char &tnorm QS_UNUSED_VAR){};
00477
00478 template <> inline
00479 constexpr char WbUtil<wbcomplex>::isComplex() { return 1; };
00480
00481 template <> inline
00482 constexpr char WbUtil<void>::isComplex() { return -1; };
00483
00484 #ifdef MATLAB_MEX_FILE
00485 template <> inline
00486 constexpr char WbUtil<mxComplexDouble>::isComplex() { return 2; };
00487 #endif
00488
00489 #ifdef _COMPLEX_H
00490 template <> inline
00491 constexpr char WbUtil<complex<double> >::isComplex() { return 4; };
00492 template <> inline
00493 constexpr char WbUtil<complex<float > >::isComplex() { return 8; };
00494 #endif
00495
00496 template <class T> inline
00497 bool WbUtil<T>::isFloat(){ return 0; };
00498
00499 template <> inline bool WbUtil<double>::isFloat(){ return 1; };
00500 template <> inline bool WbUtil<long double>::isFloat(){ return 1; };
00501 template <> inline bool WbUtil<float>::isFloat(){ return 1; };
00502 template <> inline bool WbUtil<wbcomplex>::isFloat(){ return 1; };
00503
00504 template <class T> inline
00505 bool WbUtil<T>::isInt(){ return 0; }
00506
00507 template <> inline bool WbUtil<int>::isInt(){ return 1; }
00508 template <> inline bool WbUtil<char>::isInt(){ return 1; }
00509 template <> inline bool WbUtil<long>::isInt(){ return 1; }
00510
00511 template <> inline bool WbUtil<unsigned>::isInt(){ return 1; }
00512 template <> inline bool WbUtil<unsigned char>::isInt(){ return 1; }
00513 template <> inline bool WbUtil<unsigned long>::isInt(){ return 1; }
00514
00515 template <> inline double WbUtil<double>::eps(){ return DBL_EPSILON; }
00516 template <> inline float WbUtil<float >::eps(){ return FLT_EPSILON; }
00517
00518 template <> inline
00519 long double WbUtil<long double>::eps(){ return LDBL_EPSILON; }
00520
00521 #include <sys/time.h>
00522 #include <sys/resource.h>
00523
00524 class cpu_time {
00525     public:
00526
00527

```

```

00528     cpu_time() { tcpu=time('c'); tsys=time('s'); };
00529
00530     void init() { tcpu=time('c'); tsys=time('s'); };
00531
00532     cpu_time since() {
00533         cpu_time dt; dt.tcpu-=tcpu; dt.tsys-=tsys;
00534         return dt;
00535     };
00536
00537     static double time(char flag) {
00538         struct rusage result;
00539         getrusage(RUSAGE_SELF,&result);
00540         if (flag=='c') {
00541             return (
00542                 double(result.ru_utime.tv_sec)
00543                 + 1e-6 * result.ru_utime.tv_usec
00544             );
00545         } else {
00546             return (
00547                 double(result.ru_stime.tv_sec)
00548                 + 1e-6 * result.ru_stime.tv_usec
00549             );
00550         }
00551     };
00552
00553     wstring toStr(char flag='c');
00554     double tcpu, tsys;
00555
00556 protected:
00557 private:
00558 };
00559
00560     cpu_time gCPUTime;
00561
00562 namespace wbsys {
00563
00564     wstring MemTot2Str();
00565     wstring MemFree2Str();
00566     wstring SwapTot2Str();
00567     wstring SwapFree2Str();
00568
00569     size_t getMemTot();
00570     size_t getMemFree();
00571     size_t getSwapTot();
00572     size_t getSwapFree();
00573
00574     char checkSwapSpace(const char *F=NULL, int L=0);
00575
00576     int getNumCores();
00577     int getCacheLineSize();
00578 };
00579
00580 class wbttop {
00581
00582 public:
00583
00584     wbttop() {
00585         pid=getpid(); ppid=getppid();
00586         system=wbsys::getMemTot();
00587     };
00588
00589     wstring VmSize2Str() const;
00590     size_t getVmSize() const { return Wb::getProcSize("VmSize",pid); };
00591
00592     wstring VmPeak2Str() const;
00593     size_t getVmPeak() const { return Wb::getProcSize("VmPeak",pid); };
00594
00595     int runningLarge(const char *F=NULL, int L=0,
00596                     double th0=0.80,
00597                     double fac=1.2
00598     );
00599
00600     void MemStat(const char *F=0, int L=0, char lflag=0) const;
00601
00602     pid_t pid, ppid;
00603     size_t system;
00604
00605 protected:
00606 private:
00607 };
00608
00609 #endif
00610

```


5.23 wbarry.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : wbarry (array class, col-major)
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_ARRAY_COL_MAJOR_CC__
00021 #define __WB_ARRAY_COL_MAJOR_CC__
00022
00023 //-----//
00024 // wbarry - N-D Array class
00025 //
00026 //     generic N-dimensional class that allows permutations
00027 //     and reshaping; index order is column-major.
00028 //
00029 // Wb,Sep14,09 ; Wb,Aug11,05
00030 //-----//
00031
00032 template<class T>
00033 windex& wbarry<T>::ind2sub(size_t k, windex &I) const {
00034     const size_t *s=SIZE.data, n=SIZE.len;
00035     size_t i,r, e=0, *idx;
00036
00037     I.init(n); idx=I.data;
00038     for (i=0; i<n; ++i) {
00039         if (!s[i]) wblog(FL,
00040             "ERR wbarry::%s() got size %s",FCT,SSTR_(this));
00041
00042         r=k/s[i]; idx[i]=k-r*s[i]; k=r;
00043         if (idx[i]>=s[i]) e++;
00044     }
00045
00046     if (e || k) wblog(FL,
00047         "ERR wbarry::%s() out of bounds (%d => [%s; %s])",
00048         FCT, STR(I+1), SSTR_(this)
00049     );
00050
00051     return I;
00052 };
00053
00054 template<class T>
00055 void wbarry<T>::setRand(const char pnflag) {
00056     static int firstcall=1; size_t i,s=numel(); double fac;
00057
00058     if (WbUtil<T>::isFloat())
00059         fac = 1. / (double)RAND_MAX;
00060     else fac = 100. / (double)RAND_MAX;
00061
00062     if (firstcall) { wb_srand(); firstcall=0; }
00063     for (i=0; i<s; ++i) data[i] = (T)(fac*::rand());
00064
00065     if (pnflag) {
00066         fac *= (0.5*(double)RAND_MAX);
00067         for (i=0; i<s; ++i) data[i] -= fac;
00068     }
00069 }
00070
00071 template<class T>
00072 double wbarry<T>::SkipTiny(double eps_) {
00073     T zero=T(0), x2=zero, eps=T(eps_);
00074     if (eps!=zero) { size_t i=0, n=numel(); T a;
00075         for (; i<n; ++i) { if (data[i]!=zero) {
00076             a=Wb::abs(data[i]); if (a<eps) { x2+=Wb::norm2(a); data[i]=zero; }
00077         }}
00078     }
00079     return double(x2);
00080 };
00081
00082
00083 template<> inline
00084 double wbarry<wbcomplex>::SkipTiny(double eps) {
00085     double a,x2=0;

```

```

00086     for (size_t s=numel(), i=0; i<s; ++i) {
00087         a=fabs(data[i].r); if (a>0 && a<eps) { x2+=a*a; data[i].r=0; }
00088         a=fabs(data[i].i); if (a>0 && a<eps) { x2+=a*a; data[i].i=0; }
00089     }
00090     return std::sqrt(x2);
00091 }
00092
00093 template<> inline
00094 wbararray<double>& wbararray<double>::SkipTiny_float(double ref) {
00095     Wb::chopTiny_float(data,numel(),ref);
00096     return *this;
00097 };
00098
00099 template<> inline
00100 wbararray<wbcomplex>& wbararray<wbcomplex>::SkipTiny_float(wbcomplex ref) {
00101     Wb::chopTiny_float((double*)data,2*numel(),ref.r);
00102     return *this;
00103 };
00104
00105 template<> inline
00106 double wbararray<wbcomplex>::SkipTiny_imag(double eps) {
00107     return Wb::chopTiny_imag(data,numel(),eps);
00108 };
00109
00110 template<class T>
00111 wbvector<T>& wbararray<T>::normDim2(unsigned id, wbvector<T> &xk) const {
00112
00113     size_t i, n, N=numel(); if (!N) { return xk.init(); }
00114     unsigned k, r=SIZE.len, l=r-1;
00115     const size_t *s=SIZE.data; T *x2;
00116     WBINDEX I(r);
00117
00118     if (id>=r) { wblog(FL,"ERR %s() index out of bounds (%d/%d)",FCT,id,r); }
00119     n=SIZE[id]; xk.init(n); x2=xk.data;
00120
00121     for (i=0; i<N; ++i) {
00122         x2[I[id]] += Wb::norm2(data[i]);
00123
00124         k=0; ++I[0];
00125         while (I[k]>=s[k] && k<l) { I[k]=0; ++I[++k]; }
00126     }
00127
00128     return xk;
00129 };
00130
00131 template<class T>
00132 wbararray<T>& wbararray<T>::SkipTrailingZeroSpace(
00133     unsigned k,
00134     size_t n,
00135     double eps
00136 ) {
00137     wbvector<T> x2; normDim2(k,x2); if (!x2) { return *this; }
00138
00139     size_t N=SIZE[k];
00140
00141     if (long(n)<0) { n=0; }
00142     if (n<N) {
00143         size_t i=N-1;
00144         double eps2=x2.max(); if (eps2<1) { eps2=1; }
00145         eps2*=(eps*eps);
00146
00147         for (; i<N; --i) { if (x2[i]>eps2) { break; } }
00148         if ((++i)<N) { if (i<n) { i=n; }
00149             wbvector<size_t> S(SIZE); S[k]=i;
00150             Resize(S);
00151         }
00152     }
00153     return *this;
00154 };
00155
00156 template<class T> inline
00157 T wbararray<T>::normDiff2(const wbararray<T> &B, char sflag) const {
00158
00159     T x2=0;
00160     if (sameSize(B)) {
00161         x2=Wb::rangeNormDiff2(data, B.data, numel());
00162         return x2;
00163     }
00164
00165     if (!sflag || SIZE.len!=2 || B.SIZE.len!=2) {
00166         if (sflag) wblog(FL,"ERR %s() size mismatch\n"
00167             "(sflag=%d for matrices only: %s / %s",FCT,sflag,SSTR_(this),SSTR(B));
00168         else wblog(FL,"ERR size mismatch (%s; %s)",SSTR_(this),SSTR(B));
00169     }
00170
00171     char b1=1, b2=1;
00172     size_t i,j, d1=SIZE[0], d2=SIZE[1], D1=B.SIZE[0], D2=B.SIZE[1];

```

```

00173     const T* a=data, *b=B.data;
00174
00175     if (d1>D1) { SWAP(d1,D1); b1=0; }
00176     if (d2>D2) { SWAP(d2,D2); b2=0; }
00177
00178     for (j=0; j<D2; ++j) {
00179         if (j<d2) {
00180             for (i=0; i<d1; ++i) { x2+=Wb::norm2(a[i]-b[i]); }
00181             if (b1)
00182                 { for (; i<D1; ++i) { x2+=Wb::norm2(b[i]); } }
00183             else { for (; i<D1; ++i) { x2+=Wb::norm2(a[i]); } }
00184             a+=SIZE[0]; b+=B.SIZE[0];
00185         }
00186         else if (b2) {
00187             if (j==d2) { D1=B.SIZE[0]; }
00188             for (i=0; i<D1; ++i) { x2+=Wb::norm2(b[i]); }; b+=D1;
00189         }
00190         else {
00191             if (j==d2) { D1=SIZE[0]; }
00192             for (i=0; i<D1; ++i) { x2+=Wb::norm2(a[i]); }; a+=D1;
00193         }
00194     }
00195     return x2;
00196 };
00197
00198 template<class T> inline
00199 T wbararray<T>::normDiff2(const wvector<T> &b) const {
00200     if (!isVector() || numel()!=b.len) wblog(FL,
00201         "ERR size mismatch (%s; %d)",SSTR_(this),b.len);
00202     return Wb::rangeNormDiff2(data, b.data, numel());
00203 };
00204
00205 template<class T>
00206 wbararray<T>& wbararray<T>::plus(
00207     const wbararray &B, wbararray &C, T bfac, char iflag
00208 ) const {
00209
00210     if (isEmpty() && B.isEmpty()) { return C.init(); }
00211
00212     if (C.isRef()) wblog(FL,"ERR %s() got ref (considered const)",FCT);
00213
00214     if (SIZE==B.SIZE) { if (!bfac) { C=*this; } else {
00215         size_t i=0, n=numel(); const T *b=B.data; T* c;
00216
00217         C.SIZE=SIZE;
00218         C.NEW_DATA(SIZE.prod(0),NULL,0,0);
00219         c=C.data;
00220
00221         if (bfac==T(+1)) for (; i<n; ++i) { c[i] = data[i] + b[i]; } else
00222         if (bfac==T(-1)) for (; i<n; ++i) { c[i] = data[i] - b[i]; }
00223         else for (; i<n; ++i) { c[i] = data[i] + b[i]*bfac; }
00224     } }
00225     else if (!B && !bfac) { C.init(*this); }
00226     else if (!iflag || (SIZE.len && B.SIZE.len)) {
00227         wblog(FL,"ERR %s() size mismatch %s / %s",FCT,SSTR_(this),SSTR(B));
00228     }
00229     else {
00230         if (SIZE.len) { C=*this; }
00231         else {
00232             if (!bfac) { C.init(B.SIZE); } else
00233             if (bfac==T(1)) { C.init(B); }
00234             else {
00235                 size_t i=0, n=B.numel(); const T* b=B.data; T* c;
00236                 C.SIZE=B.SIZE;
00237                 C.NEW_DATA(B.SIZE.prod(0),NULL,0,0);
00238                 c=C.data;
00239
00240                 if (bfac==T(-1))
00241                     { for (; i<n; ++i) { c[i] =-b[i]; } }
00242                 else { for (; i<n; ++i) { c[i] = b[i]*bfac; } }
00243             }
00244         }
00245     }
00246     return C;
00247 };
00248
00249 template<class T>
00250 template<class TB>
00251 wbararray<T>& wbararray<T>::Plus(
00252     const wbararray<TB> &B, TB bfac,
00253     char iflag,
00254     T afac
00255 ){
00256     if (isEmpty()) {
00257         if (B.isEmpty()) { return *this; }
00258         if (iflag || afac==T(0)) {
00259             if (iflag && afac && afac!=T(1)) { wblog(FL,

```

```

00260         "ERR %s() got empty A with afac=%s",FCT,NSTR(afac));
00261     }
00262     if (bfac==TB(1)) { return init(B); } else
00263     if (!bfac) { return init(B.SIZE); }
00264     else {
00265         SIZE=B.SIZE;
00266         NEW_DATA(B.SIZE.prod(0),NULL,0,0);
00267     }
00268 }
00269 }
00270 else { iflag=0; }
00271
00272 if (isRef()) wblog(FL,"ERR %s() got ref (considered const)",FCT);
00273 if (SIZE!=B.SIZE) wblog(FL,
00274     "ERR %s() size mismatch %s / %s",FCT,SSTR_(this), SSTR(B));
00275
00276 Wb::cpyRange(data, B.data, numel(), iflag? T(0) : afac, bfac);
00277
00278 return *this;
00279 };
00280
00281 template<class T> inline
00282 char wbararray<T>::sameUptoFac(
00283     const wbararray<T> &B, T* fac_, double eps
00284 ) const {
00285
00286     size_t i, n=numel(); T a,fac=1;
00287
00288     if (SIZE!=B.SIZE) return 1;
00289     if (!n) return 0;
00290
00291     a=this->aMax(&i);
00292     if (a<=eps) {
00293         if (a==0 || B.aMax(&i)>eps) { if (fac_) (*fac_)=0; return 0; }
00294         else return 1;
00295     }
00296     if (Wb::abs(B.data[i])<=eps) return 2;
00297
00298     fac=data[i]/B.data[i]; if (fac_) (*fac_)=fac;
00299
00300     for (i=0; i<n; ++i) {
00301         a=Wb::abs(data[i]-fac*B.data[i]);
00302         if (a>eps) return 3;
00303     }
00304     return 0;
00305 };
00306
00307 template<class T> inline
00308 bool wbararray<T>::sameAs(const wbararray &B, double eps) const {
00309     if (this!=&B) {
00310         if (SIZE!=B.SIZE) { return 0; }
00311         if (data!=B.data) {
00312             if (eps) {
00313                 for (size_t i=0, n=numel(); i<n; ++i) {
00314                     if (Wb::abs(data[i]-B.data[i])>eps) { return 0; }
00315                 }
00316             } else { return (!memcmp(data, B.data, numel()*sizeof(T))); }
00317         }
00318     }
00319     return 1;
00320 };
00321
00322 template<class T> inline
00323 wbvector<T>& wbararray<T>::getCol(size_t k, wbvector<T> &v) const {
00324     const T* x=col(k);
00325     return v.init(SIZE[0],x);
00326 };
00327
00328 template<class T> inline
00329 wbvector<T>& wbararray<T>::getRow(size_t k, wbvector<T> &v) const {
00330     const T* x=row(k);
00331     return v.initStride(SIZE[1],x,SIZE[0]);
00332 };
00333
00334 template<class T> inline
00335 wbararray<T>& wbararray<T>::getCol(size_t k, wbararray<T> &a) const {
00336     const T* x=col(k);
00337     size_t s[2]={ SIZE[0], 1 }; a.SIZE.init(2,s); a.NEW_DATA(s[0],x);
00338     return a;
00339 };
00340
00341 template<class T> inline
00342 wbararray<T>& wbararray<T>::getRow(size_t k, wbararray<T> &a) const {
00343     const T* x=row(k);
00344     size_t s[2]={ 1, SIZE[1] }; a.SIZE.init(2,s);
00345
00346     a.NEW_DATA(s[1],NULL,0,0);

```

```

00347     Wb::cpyStride(a.data, x, 1, NULL, s[1], -1, SIZE[0], 0);
00348
00349     return a;
00350 };
00351
00352 template<class T> inline
00353 wbvector<T> wbararray<T>::getDiag() const {
00354
00355     wbvector<T> x;
00356
00357     if (!SIZE.len) { return x; }
00358     if (SIZE.len!=2) wblog(FL,
00359         "ERR %s() invalid matrix %s", FCT, SSTR_(this));
00360
00361     size_t i=0, j=0, m=SIZE[0]+1, n=MIN(SIZE[0], SIZE[1]);
00362     if (n) { x.init(n);
00363         for (; i<n; ++i, j+=m) { x[i]=data[j]; }
00364     }
00365     return x;
00366 };
00367
00368 template<class T>
00369 void wbararray<T>::getMatSize(
00370     const char *F, int L, size_t &dim1, size_t &dim2) const {
00371
00372     size_t i, r=SIZE.len, r2=r/2;
00373     const size_t *const &s=SIZE.data;
00374
00375     if (r%2) wblog(F_L,
00376         "ERR %s() even rank object required (%s)", FCT, SSTR_(this));
00377     if (r) {
00378         dim1=s[0]; dim2=s[r2];
00379         for (i=1; i<r2; i++) { dim1*=s[i]; dim2*=s[i+r2]; }
00380     } else { dim1=dim2=0; }
00381 };
00382
00383 template<class T>
00384 void wbararray<T>::getMatSize(size_t &dim1, size_t &dim2, unsigned m) const {
00385
00386     unsigned i=0, r=SIZE.len;
00387     const size_t *const &s=SIZE.data;
00388
00389     if (m>r) { m=r; }
00390     if (m) { dim1=s[0]; for (++i; i<m; ++i) { dim1*=s[i]; } }
00391     else { dim1=(r?1:0); }
00392
00393     if (i<r) { dim2=s[i]; for (++i; i<r; ++i) { dim2*=s[i]; } }
00394     else { dim2=(r?1:0); }
00395 };
00396
00397 template<class T>
00398 void wbararray<T>::getMatSize(const ctrIdx &ic,
00399     size_t &dimc,
00400     size_t &dimk,
00401     unsigned r, size_t *dom
00402 ) const {
00403
00404     unsigned i, j;
00405
00406     if (!ic) wblog(FL, "ERR %s() got empty ctrIdx %s", FCT, STR(ic));
00407     if (int(r)<0) {
00408         if (dom) wblog(FL, "ERR %s() invalid usage (%d,%p)", FCT, r, dom);
00409         r=SIZE.len;
00410     }
00411     else if (r>SIZE.len) {
00412         sprintf_str("%s() rank out of bounds (r=%d/%ld)", FCT, r, SIZE.len);
00413         wblog(FL, dom ? "ERR %s": "WRN %s", str);
00414         r=SIZE.len;
00415     }
00416
00417     wbvector<char> mm(r);
00418     dimc=dimk=1;
00419
00420     for (j=0; j<ic.len; ++j) { i=ic[j];
00421         if (i>r) { char s[128];
00422             snprintf(s, 127, "%s() ctrIdx out of bounds (%s/%d)", FCT, STR(ic), r);
00423             if (dom || i>99)
00424                 wblog(FL, "ERR %s", s);
00425             else wblog(FL, "WRN %s", s);
00426         }
00427         else {
00428             if (++mm[i]!=1) wblog(FL,
00429                 "ERR %s() non-unique ctrIdx (%s/%d)", FCT, STR(ic), r);
00430             dimc *= SIZE[i];
00431         }
00432     }
00433

```

```

00434     for (i=0; i<r; ++i) { if (!mm[i]) { dimk *= SIZE[i]; }}
00435
00436     if (dom) { for (*dom=1; i<SIZE.len; ++i) { *dom *= SIZE[i]; }}
00437 };
00438
00439 template<class T>
00440 char wbararray<T>::fitsSize(const wbararray<T> &B, char strict) const {
00441
00442     unsigned i=0, n=MIN(SIZE.len,B.SIZE.len);
00443     const SPIDX_T *s1=SIZE.data, *s2=B.SIZE.data;
00444
00445     for (; i<n; ++i) { if (s1[i]>s2[i]) return 0; }
00446
00447     if (SIZE.len==B.SIZE.len) { return 1; }
00448     if (strict) { return 0; }
00449
00450     if (SIZE.len<B.SIZE.len) {
00451         for (n=B.SIZE.len; i<n; ++i) { if (!s2[i]) return 0; }
00452         return 2;
00453     }
00454     else {
00455         for (n=SIZE.len; i<n; ++i) { if (s1[i]>1) return 0; }
00456         return 3;
00457     }
00458 };
00459
00460 template<class T>
00461 template<class TB>
00462 bool wbararray<T>::sameSize(const wbararray<TB> &B,
00463     char lflag
00464 ) const {
00465
00466     if (!lflag) { return (SIZE==B.SIZE); }
00467     if (numel()!=B.numel()) { return 0; }
00468     if (!data) { return 1; }
00469
00470     unsigned i=0, j=0, ra=SIZE.len, rb=B.SIZE.len;
00471     const size_t *sa=SIZE.data, *sb=B.SIZE.data;
00472
00473     if (!ra || !rb) wblog(FL,"ERR %s() got rank r=%d/%d",FCT,ra,rb);
00474
00475     if (lflag=='l') { lflag=3; } else
00476     if (lflag=='L') { lflag=7; } else
00477     if (lflag<1 || lflag>7) { wblog(FL,
00478         "ERR %s() invalid lflag=%s",FCT,cSTR(lflag)); }
00479
00480     if (lflag&2) {
00481         while (ra) { if (sa[ra-1]==1) { --ra; } else { break; }}
00482         while (rb) { if (sb[rb-1]==1) { --rb; } else { break; }}
00483     }
00484
00485     if (lflag&1) {
00486         for (; i<ra; ++i) { if (sa[i]!=1) { break; }}
00487         for (; j<rb; ++j) { if (sb[j]!=1) { break; }}
00488     }
00489
00490     if (lflag&4) {
00491         while (i<ra && j<rb) {
00492             if (sa[i]!=sb[j]) { return 0; }
00493             for (++i; i<ra; ++i) { if (sa[i]!=1) break; }
00494             for (++j; j<rb; ++j) { if (sb[j]!=1) break; }
00495         }
00496         if (i<ra || j<rb) { return 0; }
00497     }
00498     else {
00499         ra-=i; rb-=j; if (ra!=rb) { return 0; }
00500         sa+=i; sb+=j;
00501         for (i=0; i<ra; ++i) { if (sa[i]!=sb[i]) return 0; }
00502     }
00503
00504     return 1;
00505 };
00506
00507 template<class T> inline
00508 bool wbararray<T>::sameSize(const size_t *s, unsigned n) const {
00509
00510     unsigned i=0, m=MIN(n,unsigned(SIZE.len));
00511     for (i=0; i<m; ++i) { if (s[i]!=SIZE.data[i]) return 0; }
00512     for (; i<n; ++i) { if (s[i]!=1) return 0; }
00513     return 1;
00514 };
00515
00516 template<class T> inline
00517 bool wbararray<T>::sameSize1(const size_t *s2, unsigned n2) const {
00518
00519     const size_t *s1=SIZE.data;
00520     unsigned i=0, j=0, n1=SIZE.len;

```

```

00521
00522     while (1) {
00523         for (; i<n1 && s1[i]==1; ++i) {};
00524         for (; j<n2 && s2[j]==1; ++j) {};
00525         if (i==n1 || j==n2 || s1[i]!=s2[j]) break;
00526         ++i; ++j;
00527     }
00528
00529     return ((i<n1 || j<n2 || (!n1)^(!n2)) ? 0 : 1);
00530 };
00531
00532 template<class T> inline
00533 bool wbararray<T>::equal (const wbararray<T> &B, T eps) const {
00534     size_t i,s=numel(); if (SIZE!=B.SIZE) { return 0; }
00535     if (eps)
00536         { for (i=0; i<s; ++i) { if (fabs(data[i]-B.data[i])>eps) return 0; }}
00537     else { if (memcmp(data,B.data,s*sizeof(T))) { return 0; }}
00538     return 1;
00539 };
00540
00541 template<class T> inline
00542 bool wbararray<T>::unequal (const wbararray<T> &B, T eps) const {
00543     return !equal(B,eps);
00544 };
00545
00546 template<class T> inline
00547 bool wbararray<T>::equal (
00548     const wbararray<T> &B, double eps, double &maxdiff) const {
00549
00550     if (SIZE!=B.SIZE) return 0;
00551
00552     size_t i,s=numel(); double d;
00553
00554     for (maxdiff=0, i=0; i<s; i++) {
00555         d = Wb::abs( data[i] - B.data[i] );
00556         if (maxdiff<d) maxdiff=d;
00557     }
00558
00559     return (maxdiff<eps);
00560 };
00561
00562 template<class T> inline
00563 bool wbararray<T>::unequal (
00564     const wbararray<T> &B, double eps, double &maxdiff) const {
00565     return !(this->equal(B,eps,maxdiff));
00566 };
00567
00568 template<class T>
00569 bool wbararray<T>::isDiag_aux(const T eps, const char* task) const {
00570
00571     size_t i=0;
00572     wIndex I(SIZE);
00573
00574     char isDiag=!strcmp(task,"isDiag");
00575     char isIdty=!strcmp(task,"isIdty");
00576
00577     if (!isDiag && !isIdty) wblog(FL,"ERR %s() invalid task '%s'",FCT,task);
00578     if (SIZE.len%2) wblog(FL,"ERR %s() for even-rank objects only (%s)",
00579         task,SSTR_(this));
00580
00581     while (++I) {
00582         if (!I.isdiag()) {
00583             if (Wb::abs(data[i])>eps) return 0;
00584         }
00585         else {
00586             if (isIdty && (Wb::abs(data[i]-T(1)))>eps) return 0;
00587         }
00588         ++i;
00589     }
00590
00591     return 1;
00592 };
00593
00594 template<class T>
00595 bool wbararray<T>::isDiag_aux(double *epsp, const char* task) const {
00596     if (!epsp) wblog(FL,"ERR %s() got null eps",FCT);
00597     if (!task) wblog(FL,"ERR %s() got null task",FCT);
00598
00599     size_t i,k, s=numel(), r=SIZE.len, l=r-1;
00600     WINDEX I(r); bool is=1; double a, eps=0, eref=*epsp;
00601
00602     char isDiag=!strcmp(task,"isDiag");
00603     char isIdty=!strcmp(task,"isIdty");
00604     if (!isDiag && !isIdty) wblog(FL,"ERR invalid task '%s'", task);
00605
00606     if (!isRank(2)) { info("this"); wblog(FL,
00607         "ERR %s only applies to rank-2 tensors (%s).",task,SSTR_(this)); }

```

```

00608
00609     for (i=0; i<s; i++) {
00610         if (I[0]!=I[1]) { a=Wb::abs(data[i]);
00611             if (is && eref<a) is=0;
00612             if (eps<a) eps=a;
00613         }
00614         else if (isIdty) { a=Wb::abs(data[i]-T(1));
00615             if (is && eref<a) is=0;
00616             if (eps<a) eps=a;
00617         }
00618
00619         k=0; I[0]++;
00620         while (I[k]>=SIZE[k] && k<1) { I[k]=0; ++I[++k]; }
00621     }
00622
00623     *epsp=eps; return is;
00624 };
00625
00626 template<class T>
00627 bool wbararray<T>::isProptoId(T &x, const T eps) const {
00628
00629     if (!data) { return 0; }
00630
00631     size_t i=0;
00632     wbIndex I(SIZE);
00633
00634     if (SIZE.len%2) wblog(FL,
00635         "ERR %s() for even-rank objects only (%s)",FCT,SSTR_(this));
00636     if (!x && data) { x=data[0]; }
00637
00638     while (++I) {
00639         if (!I.isdiag())
00640             { if (Wb::abs(data[i] )>eps) return 0; }
00641         else { if (Wb::abs(data[i]-x)>eps) return 0; }; ++i;
00642     }
00643
00644     if (i && !data) wblog(FL,
00645         "ERR %s() invalid empty array (len=%d)",FCT,i);
00646
00647     return 1;
00648 };
00649
00650 template<class T>
00651 bool wbararray<T>::isOpS(size_t *n) const {
00652
00653     size_t i,r=SIZE.len, r2=r/2;
00654
00655     if (r==0) return 1;
00656     if (r%2) return 0;
00657
00658     for (i=0; i<r2; i++) if (SIZE[i]!=SIZE[i+r2]) return 0;
00659
00660     if (n) { (*n)=Wb::prodRange(SIZE.data,r2); }
00661
00662     return 1;
00663 }
00664
00665 template<class T>
00666 wbararray<T>& wbararray<T>::balanceOp(
00667     const char *F, int L, T &dref, double &dscale
00668 ){
00669     size_t i,j,l,n=0;
00670
00671     if (!isOpS(&n)) wblog(F_L,
00672         "ERR %s() requires operator (%s)",FCT,SSTR_(this));
00673
00674     if (n<=1) { dref=0; dscale=1; return *this; }
00675     dref=0; dscale=0;
00676
00677     for (l=i=0; i<n; i++, l+=(n+1)) dref+=data[l];
00678     dref /= n;
00679
00680     for (l=i=0; i<n; i++)
00681         for (j=0; j<n; j++, l++) { if (i==j) data[l]-=dref;
00682             dscale += Wb::abs2(data[l]);
00683         }
00684     dscale = std::sqrt(dscale)/n;
00685
00686     if (dscale!=0) (*this)*=(1./dscale);
00687     else dscale=1;
00688
00689     return *this;
00690 }
00691
00692 template<class T>
00693 wbararray<T>& wbararray<T>::Symmetrize(
00694     const char *F, int L,

```



```

00695     double *delta,
00696     char cflag,
00697     char tflag,
00698     char fflag
00699 ) {
00700     size_t i,j,n=0; T d1,d2; double e,E=0,dmax=0;
00701
00702     if (!isOpS(&n)) wblog(F,L,
00703         "ERR %s() (generalized) square matrix required (%s)",
00704         FCT,SSTR_(this));
00705     if (tflag && !delta) return *this;
00706
00707     if (cflag && typeid(T)!=typeid(wbcomplex)) cflag=0;
00708
00709     for (i=0; i<n; i++)
00710     for (j=i; j<n; j++) { d1=data[i+j*n]; d2=data[j+i*n];
00711         dmax=MAX(dmax,Wb::abs(d1),Wb::abs(d2));
00712         if (i!=j) {
00713             e=(cflag ? Wb::abs(d1-Wb::CONJ(d2)) : Wb::abs(d1-d2));
00714             E=MAX(E,e);
00715         }
00716     }
00717
00718     dmax=MAX(1.,dmax);
00719     e=E; if (dmax!=0) e/=dmax;
00720
00721     if (delta) {
00722         if (dmax!=0) {
00723             if (e>(*delta)) { tflag=1;
00724 #ifdef MATLAB_MEX_FILE
00725                 this->put(FL,"M_");
00726 #endif
00727                 sprintf_str("%s() non-symmetric operator (%d)\n"
00728                     "eps=%.3g/%.3g @ dmax=%.3g",FCT,cflag,e,*delta,dmax);
00729
00730                 if (fflag) wblog(F,L,"ERR %s",str);
00731                 else wblog(F,L,"WRN %s",str);
00732             }
00733             *delta=(e);
00734         }
00735         else if (e==0) *delta=0; else *delta=1;
00736     }
00737
00738     if (tflag==0) {
00739         if (cflag) {
00740             for (i=0; i<n; i++)
00741             for (j=i+1; j<n; j++) {
00742                 d1=0.5*(data[i+j*n]+Wb::CONJ(data[j+i*n]));
00743                 data[i+j*n]=d1; data[j+i*n] = Wb::CONJ(d1);
00744             }
00745         }
00746         else {
00747             for (i=0; i<n; i++)
00748             for (j=i+1; j<n; j++) {
00749                 d1=0.5*(data[i+j*n]+data[j+i*n]);
00750                 data[i+j*n] = data[j+i*n] = d1;
00751             }
00752         }
00753     }
00754
00755     return *this;
00756 };
00757
00758 template<class T>
00759 wbarry<T>& wbarry<T>::swapRows(size_t i1, size_t i2) {
00760
00761     if (rank()!=2) wblog(FL,
00762         "ERR %s() for matrices only (%s)",FCT,SSTR_(this));
00763     if (i1>=SIZE[0] || i2>=SIZE[0]) wblog(FL,"ERR %s() "
00764         "index out of bounds (%d,%d; %s)",FCT,i1+1,i2+1,SSTR_(this));
00765
00766     if (i1!=i2) { size_t dim1=SIZE[0], dim2=SIZE[1]; if (dim2) {
00767         T x, *d1=data+i1, *d2=data+i2;
00768         for (size_t j=0; j<dim2; ++j, d1+=dim1, d2+=dim1) {
00769             x=(*d1); *d1=(*d2); *d2=x;
00770         }
00771     }}
00772     return *this;
00773 };
00774
00775 template<class T>
00776 wbarry<T>& wbarry<T>::swapCols(size_t j1, size_t j2) {
00777
00778     if (rank()!=2) wblog(FL,
00779         "ERR %s() for matrices only (%s)",FCT,SSTR_(this));
00780     if (j1>=SIZE[1] || j2>=SIZE[1]) wblog(FL,"ERR %s() "
00781         "index out of bounds (%d,%d; %s)",FCT,j1+1,j2+1,SSTR_(this));

```

```

00782
00783     if (j1!=j2) { size_t dim1=SIZE[0]; if (dim1) {
00784         T x, *d1=data+dim1*j1, *d2=data+dim1*j2;
00785         for (size_t i=0; i<dim1; ++i) {
00786             x=d1[i]; d1[i]=d2[i]; d2[i]=x;
00787         }
00788     }}
00789     return *this;
00790 };
00791
00792 template<class T>
00793 wbararray<T>& wbararray<T>::setCol(size_t k, size_t k0) {
00794     if (rank()!=2) wblog(FL,
00795         "ERR %s() for matrices only (%s)",FCT,SSTR_(this));
00796     if (k>=SIZE[1] || k0>=SIZE[1]) wblog(FL,
00797         "ERR %s() index out of bounds (%d,%d; %s)",
00798         FCT,k+1,k0+1,SSTR_(this));
00799
00800     size_t dim1=SIZE[0];
00801     if (k!=k0 && dim1) {
00802         MEM_COPY<T>(ref(OU,k),dim1,ref(OU,k0));
00803     }
00804
00805     return *this;
00806 };
00807
00808 template<class T>
00809 wbararray<T>& wbararray<T>::setCol(
00810     size_t k, const T* v, T fac, size_t stride) {
00811
00812     if (rank()!=2) wblog(FL,
00813         "ERR %s() for matrices only (%s)",FCT,SSTR_(this));
00814     if (k>=SIZE[1]) wblog(FL,
00815         "ERR %s() index out of bounds (%d; %s)",FCT,k+1,SSTR_(this));
00816
00817     size_t i=0, dim1=SIZE[0]; T *d=data+k*dim1;
00818     if (dim1 && d!=v) {
00819         if (fac== 1) { for (; i<dim1; ++i, v+=stride) { d[i]= (*v); }} else
00820         if (fac== -1) { for (; i<dim1; ++i, v+=stride) { d[i]= -(v); }}
00821         else { for (; i<dim1; ++i, v+=stride) { d[i]= fac*(v); }}
00822     }
00823     return *this;
00824 };
00825
00826 template<class T>
00827 wbararray<T>& wbararray<T>::setCol(size_t k, const wbvector<T> &v) {
00828
00829     if (rank()!=2) wblog(FL,
00830         "ERR %s() for matrices only (%s)",FCT,SSTR_(this));
00831     if (k>=SIZE[1]) wblog(FL,
00832         "ERR %s() index out of bounds (%d: %s)",FCT,k+1,SSTR_(this));
00833
00834     size_t dim1=SIZE[0];
00835     if (v.len!=dim1) wblog(FL,"ERR size mismatch (%d/%d)",v.len,dim1);
00836     if (dim1) MEM_COPY<T>(data+k*dim1, dim1, v.data);
00837     return *this;
00838 };
00839
00840 template<class T>
00841 template<class T2>
00842 wbararray<T>& wbararray<T>::setCol(size_t k, const wbvector<T2> &v) {
00843
00844     if (rank()!=2) wblog(FL,
00845         "ERR %s() for matrices only (%s)",FCT,SSTR_(this));
00846     if (k>=SIZE[1]) wblog(FL,
00847         "ERR %s() index out of bounds (%d: %s)",FCT,k+1,SSTR_(this));
00848
00849     size_t dim1=SIZE[0]; T* d=data+k*dim1;
00850     if (dim1) {
00851         size_t i=0, n=(v.len<dim1 ? v.len: dim1);
00852         for (; i<n; ++i) { d[i]=T(v.data[i]); }
00853         for (; i<dim1; ++i) { d[i]=T(0); }
00854     }
00855     return *this;
00856 };
00857
00858 template<class T>
00859 wbararray<T>& wbararray<T>::setRow(size_t k, const T* d0) {
00860
00861     if (rank()!=2) wblog(FL,
00862         "ERR %s() for matrices only (%s)",FCT,SSTR_(this));
00863     if (k>=SIZE[0]) wblog(FL,
00864         "ERR %s() index out of bounds (%d; %s)",FCT,k+1,SSTR_(this));
00865
00866     size_t i=0, dim1=SIZE[0], dim2=SIZE[1];
00867     T* d=data+k;
00868     for (; i<dim2; i++, d+=dim1) d[i]=d0[i];

```

```

00869
00870     return *this;
00871 };
00872
00873 template<class T>
00874 wbvector<T>& wbarry<T>::colNorm2(wbvector<T> &a) const {
00875
00876     if (rank()!=2) wblog(FL,
00877         "ERR %s() for matrices only (%s)",FCT,SSTR_(this));
00878
00879     size_t i,j, dim1=SIZE[0], dim2=SIZE[1];
00880     const T* d0=data; T x2;
00881
00882     a.init(dim2);
00883     for (j=0; j<dim2; j++, d0+=dim1) {
00884         for (x2=T(), i=0; i<dim1; i++) x2+=(Wb::CONJ(d0[i])*d0[i]);
00885         a[j]=x2;
00886     }
00887     return a;
00888 };
00889
00890 template<class T>
00891 T wbarry<T>::colNorm2(size_t k) const {
00892
00893     size_t j=0, dim1=SIZE[0], dim2=SIZE[1];
00894     const T* d=data+k*dim1;
00895     T a=0;
00896
00897     if (rank()!=2) wblog(FL,
00898         "ERR %s() for matrices only (%s)",FCT,SSTR_(this));
00899     if (k>=dim2) wblog(FL,
00900         "ERR %s() index out of bounds (%d/%d; %s)",FCT,k,dim2,SSTR_(this));
00901
00902     for (; j<dim2; ++j) { a+=Wb::CONJ(d[j])*d[j]; }
00903     return a;
00904 };
00905
00906 template<class T>
00907 T wbarry<T>::NormalizeCol(size_t k, char tnorm, char qflag) {
00908
00909     if (rank()!=2) wblog(FL,
00910         "ERR %s() matrix type required (%s)",FCT,SSTR_(this));
00911
00912     const size_t dim1=SIZE[0], dim2=SIZE[1];
00913     if (k>=dim2) wblog(FL,
00914         "ERR %s() index out of bounds (%d/%d)",FCT,k,dim2);
00915
00916     T *d=data+dim1*k, x2=Wb::overlap(d,d,dim1,1,tnorm);
00917     x2=Wb::sqrt(x2);
00918
00919     if (x2!=0) Wb::timesRange(d,1/x2,dim1);
00920     else if (!qflag) wblog(FL,"ERR %s() got vector with norm 0!",FCT);
00921
00922     return x2;
00923 };
00924
00925 template<class T>
00926 wbarry<T>& wbarry<T>::ColProject(
00927     size_t k1, size_t k2, char nflag, char tnorm
00928 ) {
00929
00930     if (rank()!=2) wblog(FL,
00931         "ERR %s() matrix type required (%s)",FCT,SSTR_(this));
00932
00933     const size_t dim1=SIZE[0], dim2=SIZE[1];
00934     if (k1>=dim2 || k2>=dim2) wblog(FL,
00935         "ERR %s() index out of bounds (%d,%d/%d)",FCT,k1,k2,dim2);
00936     if (k1==k2) wblog(FL,"WRN %s() got k1=k2=%d",FCT,k1+1);
00937
00938     gs_project_range(
00939         data+dim1*k1, data+dim1*k2, dim1, 1,
00940         nflag, tnorm
00941     );
00942     return *this;
00943 };
00944
00945 template<class T>
00946 bool wbarry<T>::isOrthogonalCol(size_t k0, char tnorm) const {
00947
00948     if (rank()!=2) wblog(FL,
00949         "ERR %s() matrix type required (%s)",FCT,SSTR_(this));
00950
00951     const size_t dim1=SIZE[0], dim2=SIZE[1];
00952     if (k0>=dim2) wblog(FL,
00953         "ERR %s() index out of bounds (%d/%d)",FCT,k0,dim2);
00954     if (!dim1) return 0;
00955

```

```

00956     const T *d=data, *d0=data+dim1*k0;
00957     T nrm2=Wb::overlap(d0,d0,dim1,1,tnorm);
00958
00959     for (size_t k=0; k<dim2; k++, d+=dim1) {
00960         if (k!=k0 && Wb::abs(Wb::overlap(d,d0,dim1,1,tnorm)/nrm2)>1E-10)
00961             return 0;
00962     }
00963
00964     return 1;
00965 };
00966
00967 template<class T>
00968 bool wbararray<T>::isOrthoCols(T *x2, char tnorm, T eps) const {
00969
00970     bool q=0;
00971     const size_t *s=SIZE.data;
00972
00973     if (SIZE.len==2 && s[1] && s[0]>=s[1]) {
00974         wbararray<T> E;
00975         Wb::MatProd(*this,*this,E, tnorm? 'T':'C');
00976
00977         if (x2)
00978             { q=E.isProptoId(*x2,eps); }
00979         else { q=E.isDiagMatrix(); }
00980     }
00981
00982     return q;
00983 };
00984
00985 template<class T>
00986 unsigned wbararray<T>::QRdecomp(
00987     const char *F, int L, wbararray<T> &Q, wbararray<T> &R, T eps
00988 ){
00989     if (isEmpty()) { Q.init(); R.init(); return 0; }
00990     if (rank()!=2 || !numel()) wblog(F_L,
00991         "ERR %s() matrix type required (%s)",FCT,SSTR_(this));
00992
00993     size_t dim1=SIZE[0], dim2=SIZE[1], n=MIN(dim1,dim2);
00994
00995     if (dim1<=1 || dim2<=1) {
00996         Q.init(dim1,n); R.init(n,dim2);
00997
00998         if (!dim1 || !dim2) return n;
00999         if (dim1==1) { Q[0]=1; R=*this; return n; }
01000         if (dim2==1) { Q=*this;
01001             if (R[0]=this->norm()!=0) { Q*=(T(1)/R[0]); }
01002             else { Q[0]=1; }
01003             return n;
01004         }
01005     }
01006
01007     if (allEqual(0)) {
01008         Q.init(dim1,1); R.init(1,dim2); Q[0]=1;
01009         return 1;
01010     }
01011
01012     size_t i,j,k=0;
01013     WBPPerm P;
01014     wbararray<T> U(dim1,n);
01015
01016     R=*this;
01017
01018     for (k=0; k<n; ++k) {
01019         if (R.Householder(F_L,k,U.col(k),&P,eps)<0) break;
01020     }
01021
01022     if (!R.isFinite() || !U.isFinite()) wblog(FL,"ERR %s() ",FCT);
01023
01024     if (k<n) {
01025         if (!k) wblog(FL,"ERR %s() resulted in k=%d/%d !?",FCT,k,n);
01026         R.Resize(k,dim2); n=k;
01027     }
01028     else if (n<dim1) {
01029         R.Resize(n,dim2);
01030     }
01031
01032     Q.initIdentityB(dim1,n);
01033
01034     T zero=0, *x;
01035     const T *u=U.col(n-1);
01036
01037     for (k=n; k>0; u-=dim1) { --k; x=Q.data;
01038         for (j=0; j<n; ++j, x+=dim1) {
01039             Wb::householder(u+k,x+k,dim1-k,eps);
01040         }
01041     }
01042

```

```

01043     x=Q.data;
01044     for (j=0; j<n; ++j, x+=dim1) {
01045         for (i=0; i<dim1; ++i) { if (Wb::abs(x[i])>eps) {
01046             if (x[i]<zero) { T *y=R.data+j;
01047                 for (i=0; i<dim1; ++i) { x[i]=-x[i]; }
01048                 for (k=0; k<dim2; ++k, y+=n) { y[0]=-y[0]; }
01049             }
01050             break;
01051         } }
01052     }
01053
01054     return n;
01055 };
01056
01057 template<class T>
01058 int wbarry<T>::Householder(
01059     const char *F, int L, size_t k,
01060     T *u, WPERM *P,
01061     T eps
01062 ) {
01063     if (rank()!=2) wblog(F_L,
01064         "ERR %s() matrix type required (%s)",FCT,SSTR_(this));
01065
01066     size_t i=0, j=0, k2=k, dim1=SIZE[0], dim2=SIZE[1];
01067     if (k>=MIN(dim1,dim2)) wblog(F_L,"ERR %s() "
01068         "index out of bounds (%d; %s)",FCT,k+1,SSTR_(this));
01069
01070     T nrm, x2=0., x0max=0., *x;
01071
01072     for (; i<k; ++i) u[i]=0.;
01073
01074     if (P) {
01075         if (!k) P->init(dim2); else
01076         if (P->len!=dim2) wblog(F_L,"ERR %s() "
01077             "got invalid P for (k=%d: %d/%d) !?",FCT,k+1,P->len,dim2);
01078         T x2max=0.;
01079
01080         for (x=data, j=0; j<dim2; ++j, x+=dim1) {
01081             if (!(P->data[j])) {
01082                 for (x2=0., i=k; i<dim1; ++i) { x2 += (Wb::CONJ(x[i])*x[i]); }
01083                 if (x2max<x2) { x2max=x2; k2=j; }
01084             }
01085             else {
01086                 if (P->data[j]>k) wblog(F_L,
01087                     "ERR %s() invalid P=[%s] (%d/%d/%d)",FCT,STR_(P),j,k,dim2);
01088                 if (x0max<(x2=Wb::abs(x[P->data[j]-1]))) { x0max=x2; }
01089             }
01090         }; x2=x2max;
01091
01092         x=data+k2*dim1;
01093
01094         for (i=k; i<dim1; ++i) { u[i]=x[i]; };
01095     }
01096     else {
01097         for (x=data, j=0; j<k; ++j, x+=dim1) {
01098             if (x[j]<0) wblog(F_L,"ERR %s() x[%d]=%d",FCT,j,x[j]);
01099             if (x0max<(x2=Wb::abs(x[j]))) { x0max=x2; }
01100         }
01101         for (x2=0., i=k; i<dim1; ++i) { u[i]=x[i];
01102             x2+=(Wb::CONJ(x[i])*x[i]);
01103         }
01104     }
01105
01106     eps*=(x0max<x2 ? x2 : x0max);
01107
01108     nrm=Wb::sqrt(x2); if (nrm<=eps) {
01109         if (!P && k+1<dim2) wblog(F_L,
01110             "ERR %s() check cols (%d/%d)",FCT,k,dim2);
01111         return -1;
01112     }
01113
01114     u[k] += (x[k]>0 ? +nrm : -nrm);
01115
01116     nrm=T(1)/Wb::sqrt( x2 - Wb::CONJ(x[k])*x[k] + Wb::CONJ(u[k])*u[k] );
01117     if (!isfinite(nrm)) wblog(F_L,"ERR %s() nrm=%g",FCT,nrm);
01118
01119     for (i=k; i<dim1; ++i) { u[i]*=nrm; }
01120
01121     if (P) {
01122         for (x=data, j=0; j<dim2; ++j, x+=dim1) {
01123             if (!(P->data[j])) {
01124                 Wb::householder(u+k,x+k,dim1-k,eps); }
01125             }
01126         P->data[k2]=(k+1);
01127     }
01128     else {
01129         for (x=data+k*dim1, j=k; j<dim2; ++j, x+=dim1) {

```

```

01130         Wb::householder(u+k,x+k,dim1-k,eps);
01131     }
01132 }
01133
01134     return k2;
01135 };
01136
01137 template<class T> inline
01138 void Wb::householder(const T *u, T *a, size_t n, const T& eps) {
01139     size_t i=0;
01140     T ua=0.; for (; i<n; ++i) { ua += (Wb::CONJ(u[i])*a[i]); }
01141
01142     if (ua) { ua*=2.;
01143         for (i=0; i<n; ++i) { a[i] -= (u[i]*ua); }
01144         if (eps>0.) { ua=0.;
01145             for (i=0; i<n; ++i) ua += Wb::CONJ(a[i])*a[i];
01146             if (ua<eps) { for (i=0; i<n; ++i) a[i]=0.; }
01147         }
01148     }
01149 }
01150 };
01151
01152 template<class T>
01153 unsigned wbararray<T>::eigTriDiag(
01154     const char *F, int L, wvector<T> &E, wbararray<T> *U, T eps
01155 ) const {
01156     unsigned D;
01157
01158     if (!isMatrix()) wlog(F_L,
01159         "ERR %s() matrix expected (%s)",FCT,SSTR_(this));
01160
01161     D=dim(1); E.init(D); if (U) U->initIdentity(D);
01162     if (D<=1) {
01163         if (D) { E.data[0]=data[0]; }
01164         return 0;
01165     }
01166
01167     wvector<T> d_(D), e_(D);
01168
01169     int i,k,m,l; unsigned iter=0;
01170     T s,r,p,g,f,da,ea,c,b, *d=d_.data, *e=e_.data;
01171
01172     for (l=D-1, i=0; i<l; ++i) {
01173         if (Wb::abs(data[(i+1)+i*D] - data[i+(i+1)*D])>eps) wlog(F_L,
01174             "ERR %s() got non-symmetric matrix\nM(%d,%d+1): %.4g / %.4g !?",
01175             FCT,i+1,i+2, double(data[(i+1)+i*D]), double(data[i+(i+1)*D]));
01176         if (i>1 && (Wb::abs(data[i])>eps || Wb::abs(data[i*D])>eps)) wlog(F_L,
01177             "ERR %s() got non-triangular matrix\nM(%d,1): %.4g !?",
01178             FCT,i+1, double(data[i]));
01179
01180         d[i]=data[i*D+i]; e[i]=0.5*(data[i*D+i+1] + data[(i+1)*D+i]);
01181     }; d[i]=data[i*D+i];
01182
01183     for (l=0; l<int(D); ++l) { iter=0;
01184         do {
01185             for (m=l; m<int(D-1); ++m) {
01186                 ea=Wb::abs(e[m]); da=Wb::abs(d[m]) + Wb::abs(d[m+1]);
01187                 if (T(ea+da)==da) break;
01188             }
01189             if (m!=l) {
01190                 if (++iter > 30) wlog(F_L,
01191                     "ERR %s() too many iterations (iter=%d) !?",FCT,iter);
01192                 g=(d[l+1] - d[l]) / (T(2)*e[l]);
01193                 r=NR::pythag(g,T(1));
01194                 g=d[m]-d[l]+e[l]/(g+NR::sign(r,g));
01195                 s=c=1; p=0;
01196
01197                 for (i=m-1; i>=l; --i) {
01198                     f=s*e[i];
01199                     b=c*e[i]; e[i+1]=(r=NR::pythag(f,g));
01200                     if (r==0) { d[i+1]-=p; e[m]=0; break; }
01201
01202                     s=f/r;
01203                     c=g/r;
01204                     g=d[i+1]-p;
01205                     r=(d[i]-g)*s + 2*c*b;
01206                     d[i+1]=g + (p=s*r);
01207                     g=c*r-b;
01208
01209                     if (U) { wbararray<T> &z=(*U);
01210                         for (k=0; k<int(D); ++k) { f=z(k,i+1);
01211                             z(k,i+1) = s*z(k,i) + c*f;
01212                             z(k,i) = c*z(k,i) - s*f;
01213                         }
01214                     }
01215                 }
01216             }

```

```

01217
01218         if (r==0 && i>=1) continue;
01219         d[l]-=p; e[l]=g; e[m]=0;
01220     }
01221     } while (m!=1);
01222 }
01223
01224 wbpem P;
01225 d_.sort(E,P);
01226 if (U) U->ColPermute(P);
01227
01228 return 0;
01229 };
01230
01231 template<class T>
01232 wbararray<T>& wbararray<T>::ColPermute(const wbpem &P, char iflag){
01233
01234     if (rank()!=2) wblog(FL,
01235         "ERR %s() matrix type required (%s)",FCT,SSTR_(this));
01236     if (P.len!=SIZE[1]) wblog(FL,
01237         "ERR %s() size mismatch (%d/%d)",FCT,P.len,SIZE[1]);
01238
01239     size_t i=0, dim1=SIZE[0], dim2=SIZE[1];
01240     wbararray<T> X(*this);
01241
01242     for (; i<P.len; i++) {
01243         if (P[i]>=dim2) wblog(FL,
01244             "ERR %s() index out of bounds (%d/%d)",FCT,P[i],dim2);
01245         if (iflag==0)
01246             MEM_COPY<T>(data+i*dim1, dim1, X.data+P[i]*dim1);
01247         else MEM_COPY<T>(data+P[i]*dim1, dim1, X.data+i*dim1);
01248     }
01249
01250     return *this;
01251 };
01252
01253 template<class T>
01254 wbararray<T>& wbararray<T>::SignCol(size_t k, const T *d0, char tnorm) {
01255
01256     if (rank()!=2) wblog(FL,
01257         "ERR %s() matrix type required (%s)",FCT,SSTR_(this));
01258
01259     const size_t dim1=SIZE[0], dim2=SIZE[1];
01260     if (k>=dim2) wblog(FL,
01261         "ERR %s() index out of bounds (%d/%d)",FCT,k,dim2);
01262
01263     T *d=data+dim1*k, x=Wb::overlap(d0,d,dim1,1,tnorm);
01264     if (x<0) {
01265         for (size_t i=0; i<dim1; i++) { d[i]=-d[i]; }
01266     }
01267
01268     return *this;
01269 };
01270
01271 template<class T>
01272 wbararray<T>& wbararray<T>::FlipSignCol(size_t k) {
01273
01274     if (rank()!=2) wblog(FL,
01275         "ERR %s() matrix type required (%s)",FCT,SSTR_(this));
01276
01277     const size_t dim1=SIZE[0], dim2=SIZE[1];
01278     if (k>=dim2) wblog(FL,
01279         "ERR %s() index out of bounds (%d/%d)",FCT,k,dim2);
01280
01281     T *d=data+dim1*k;
01282     for (size_t i=0; i<dim1; i++) { d[i]=-d[i]; }
01283
01284     return *this;
01285 };
01286
01287 template<class T>
01288 wbararray<T>& wbararray<T>::SignConventionCol(
01289     size_t k0,
01290     double deps
01291 ){
01292
01293     if (rank()!=2) wblog(FL,
01294         "ERR %s() matrix type required (%s)",FCT,SSTR_(this));
01295
01296     const size_t dim1=SIZE[0], dim2=SIZE[1];
01297     size_t i,k,k1,k2; T *d, eps=
01298         ((typeid(T)!=typeid(double) && typeid(T)!=typeid(float)) ? 0 : deps);
01299
01300     if (k>=dim2) wblog(FL,
01301         "ERR %s() index out of bounds (%d/%d)",FCT,k,dim2);
01302
01303     if (int(k0)>=0) { k1=k0; k2=k0+1; }

```

```

01304     else { k1=0; k2=dim2; }
01305
01306     for (k=k1; k<k2; k++) { d=data+dim1*k;
01307         for (i=0; i<dim1; i++) {
01308             if (d[i]> eps) break;
01309             if (d[i]<=-eps) {
01310                 for (i=0; i<dim1; i++) { d[i]=-d[i]; }
01311                 break;
01312             }
01313         }
01314     }
01315
01316     return *this;
01317 };
01318
01319 template<class T>
01320 wbararray<T>& wbararray<T>::NormalizeCols(
01321     const char *F, int L, double *amin, double *amax, char tnorm
01322 ){
01323     if (!SIZE.len) return *this;
01324
01325     size_t i,j,m, n=SIZE[0];
01326     T *d=data, z=0; double a;
01327     wvector<T> nn;
01328
01329     if (SIZE.len>1) for (m=SIZE[1], i=2; i<SIZE.len; i++) m*=SIZE[i];
01330     else m=1;
01331
01332     nn.init(m);
01333
01334     for (j=0; j<m; j++, d+=n, z=0) {
01335         z=Wb::overlap(d,d,n,1,tnorm); z=Wb::sqrt(z);
01336
01337         a=Wb::abs(z); if (a==0) {
01338             wblog(F,L,"ERR |sum^2| of column returns %.3g!%s",a,
01339                 tnorm && (typeid(T)==typeid(wbcomplex)) ? " (using t-norm)": "");
01340         }
01341         nn[j]=z;
01342     }
01343
01344     if (amin) {
01345         a=nn.aMin(); if ((*amin)>0 && a<(*amin))
01346             wblog(F,L,"WRN %s() nMin=%.3g (%.3g)",FCT,a,*amin);
01347         (*amin)=a;
01348     }
01349     if (amax) {
01350         a=nn.aMax(); if ((*amax)>0 && a>(*amax))
01351             wblog(F,L,"WRN %s() nMax=%.3g (%.3g)",FCT,a,*amax);
01352         (*amax)=a;
01353     }
01354
01355     for (d=data, j=0; j<m; j++, d+=n)
01356         for (z=T(1)/nn[j], i=0; i<n; i++) { d[i]*=z; }
01357
01358     return *this;
01359 };
01360
01361 template<class T>
01362 wbararray<T>& wbararray<T>::OrthoNormalizeCols(
01363     const char *F, int L,
01364     char tnorm,
01365     char qflag,
01366     double eps,
01367     unsigned np
01368 ){
01369     if (!SIZE.len) return *this;
01370
01371     size_t i,j,k,l,p,m, n=SIZE[0];
01372     T z, *d=0, *v=data; double a;
01373
01374     char cflag=(typeid(T)==typeid(wbcomplex));
01375     char xflag=(qflag=='x' || qflag=='X');
01376
01377     if (SIZE.len<2) m=1; else
01378         for (m=SIZE[1], i=2; i<SIZE.len; i++) m*=SIZE[i];
01379
01380     if (!m || !n) return *this;
01381
01382     for (l=k=0; k<m; ++k, ++l, v+=n) {
01383         for (p=0; p<np; ++p) {
01384             for (d=data, j=0; j<l; ++j, d+=n) {
01385                 z=Wb::overlap(d,v,n,1,tnorm); if (z!=T(0)) {
01386                     for (i=0; i<n; ++i) v[i]-=z*d[i]; }
01387             }
01388         }
01389
01390         z=Wb::overlap(v,v,n,1,tnorm);

```



```

01391     a=Wb::abs(z);
01392
01393     if (a>eps) { a=Wb::sqrt(1/a);
01394         if (xflag && l<k)
01395             { for (i=0; i<n; ++i) d[i]=v[i]*a; }
01396         else { for (i=0; i<n; ++i) v[i]*=a; }
01397     }
01398     else {
01399         if (xflag) --l;
01400         else if (qflag) {
01401             for (i=0; i<n; ++i) v[i]=0;
01402         }
01403         else {
01404             wblog(F_L,"ERR |sum^2| of column returns %.3g!%s",
01405                 a, tnorm && cflag ? " (using t-norm)" : "");
01406         }
01407     }
01408 }
01409
01410 if (xflag && l<k) {
01411     if (SIZE.len!=2) wblog(F_L,"ERR %s() qflag='%c' must be "
01412         "used with rank-2 tensors (%d)",FCT,qflag,qflag,SIZE.len);
01413     if (!l) wblog(F_L,"ERR %s() got all null-vectors (%g)",FCT,k);
01414     SIZE[1]=1;
01415 }
01416
01417 return *this;
01418 };
01419
01420 template<class T>
01421 T wbarry<T>::norm2Cols(long j1, long j2) const {
01422
01423     T x2=0; long i,j, dim1,dim2;
01424     if (SIZE.len!=2) wblog(F_L,
01425         "ERR %s() got size %s array",FCT,SSTR_(this));
01426     dim1=SIZE[0]; dim2=SIZE[1];
01427
01428     if (j1<0) { j1+=dim2; }
01429     if (j2<0) { j2+=dim2; }; if (j1>j2 || !dim1) { return x2; }
01430     if (j1<0 || j2>=dim2) wblog(F_L,
01431         "ERR %s() index out of bounds (%ld,%ld/%ld)",FCT,j1,j2,dim2);
01432
01433     for (j=j1; j<=j2; ++j)
01434         for (i=0; i<dim1; ++i) { x2+=Wb::norm2(data[i+j*dim1]); }
01435     return x2;
01436 };
01437
01438 template<class T>
01439 T wbarry<T>::norm2Recs(long i1, long i2) const {
01440
01441     T x2=0; long i,j,dim1,dim2;
01442     if (SIZE.len!=2) wblog(F_L,
01443         "ERR %s() got size %s array",FCT,SSTR_(this));
01444     dim1=SIZE[0]; dim2=SIZE[1];
01445
01446     if (i1<0) { i1+=dim1; }
01447     if (i2<0) { i2+=dim1; }; if (i1>i2 || !dim2) { return x2; }
01448     if (i1<0 || i2>=dim1) wblog(F_L,
01449         "ERR %s() index out of bounds (%ld,%ld/%ld)",FCT,i1,i2,dim1);
01450
01451     for (j=0; j<dim2; ++j)
01452         for (i=i1; i<=i2; ++i) { x2+=Wb::norm2(data[i+j*dim1]); }
01453     return x2;
01454 };
01455
01456 template<class T>
01457 bool wbarry<T>::isSym_aux(
01458     const char *F, int L, const char *fct,
01459     const wbarry<T> &B, double eps, double* xref,
01460     const char symflag,
01461     const char lflag
01462 ) const {
01463
01464     WBINDEX S1,S2;
01465     size_t i,j,k,dim1,dim2, r=SIZE.len, r2=r/2;
01466     double x;
01467
01468     const wbarry<T> &A = (*this);
01469     bool issame = (&A==&B);
01470     wbarry<T> A2,B2;
01471
01472     if (r%2 || r!=B.SIZE.len) {
01473         if (lflag) sprintf_str(
01474             "%s %s() only applies to even-rank objects (%ld;%ld)",
01475             shortFL(F,L),fct, r, B.SIZE.len);
01476         return 0;
01477     }

```

```

01478     if (isEmpty() && B.isEmpty()) return 1;
01479
01480     dim1=dim2=1;
01481     for (i=0; i<r2; i++) { j=i+r2; dim1*=SIZE[i]; dim2*=SIZE[j];
01482         if (SIZE[i]!=B.SIZE[j] || SIZE[j]!=B.SIZE[i]) {
01483             if (lflag) sprintf_str(
01484                 "%s %s() size mismatch [%s; %s].", SHORT_FL, fct,
01485                 SSTR(this), SSTR(B));
01486             return 0;
01487         }
01488     }
01489
01490     A.groupIndizes_DREF(A2,r2,r2);
01491     B.groupIndizes_DREF(B2,r2,r2);
01492
01493     if (xref) *xref=0;
01494     if (eps<0) {
01495         eps=-eps; x=A2.aMax(); if (x>1) eps*=x;
01496     }
01497
01498     if (symflag=='s') {
01499         if (eps==0) {
01500             for (i=0; i<dim1; i++) { k = (issame ? i : 0);
01501                 for (j=k; j<dim2; j++) {
01502                     if (A2(i,j)!=Wb::CONJ(B2(j,i))) {
01503                         if (xref) {
01504                             x=Wb::abs(A2(i,j)-Wb::CONJ(B2(j,i)));
01505                             *xref = MAX(*xref,x);
01506                         }
01507                         else return 0;
01508                     }
01509                 }
01510             }
01511         }
01512         else if (symflag=='a') {
01513             for (i=0; i<dim1; i++) { k = (issame ? i : 0);
01514                 for (j=k; j<dim2; j++) {
01515                     x=Wb::abs(A2(i,j)-Wb::CONJ(B2(j,i)));
01516                     if (xref) *xref=MAX(*xref,x); else if (x>eps) return 0;
01517                 }
01518             }
01519         }
01520         else if (symflag=='a') {
01521             if (eps==0) {
01522                 for (i=0; i<dim1; i++) { k = (issame ? i : 0);
01523                     for (j=k; j<dim2; j++) {
01524                         if (A2(i,j)!=-Wb::CONJ(B2(j,i))) {
01525                             if (xref) {
01526                                 x = Wb::abs(A2(i,j) + Wb::CONJ(B2(j,i)));
01527                                 *xref = MAX(*xref,x);
01528                             }
01529                             else return 0;
01530                         }
01531                     }
01532                 }
01533             }
01534             else {
01535                 for (i=0; i<dim1; i++) { k = (issame ? i : 0);
01536                     for (j=k; j<dim2; j++) {
01537                         x = Wb::abs(A2(i,j) + Wb::CONJ(B2(j,i)));
01538                         if (x>eps) { if (xref) *xref=MAX(*xref,x); else return 0; }
01539                     }
01540                 }
01541             }
01542             if (xref && (*xref)>eps) return 0;
01543             return 1;
01544         }
01545     }
01546
01547     template<class T>
01548     char wbararray<T>::hasGroupSize(size_t s1, size_t s2) const {
01549
01550         size_t i,s=1,s0=1;
01551
01552         for (i=0; i<SIZE.len; i++) {
01553             s*=SIZE[i]; if (s0==s1 && (!s || s!=s1)) break;
01554             s0=s;
01555         }
01556         for (s=1; i<SIZE.len; i++) s*=SIZE[i];
01557
01558         return ((s0!=s1 || s!=s2) ? 0 : 1);
01559     };
01560
01561     template<class T> inline
01562     bool wbararray<T>::isComplex() const { return 0; };
01563
01564     template<> inline

```

```

01565 bool wbararray<wbcomplex>::isComplex() const {
01566     for (size_t s=numel(), i=0; i<s; i++)
01567         if (data[i].i!=0.) return 1;
01568     return 0;
01569 };
01570
01571
01572 template<class T> inline
01573 bool wbararray<T>::isZero(double eps, char bflag) const {
01574     size_t i, s=numel(); if (!s) { return 1; }
01575
01576     if (eps==0) {
01577         for (i=0; i<s; ++i) { if (data[i]!=0.) return 0; }
01578     }
01579     else {
01580         if (bflag) {
01581             double e2=eps*eps, n2=0.;
01582             for (i=0; i<s; ++i) {
01583                 n2+=Wb::norm2(data[i]); if (n2>e2) { return 0; }
01584             }
01585         }
01586         else {
01587             for (i=0; i<s; ++i) { if (Wb::abs(data[i])>eps) { return 0; } }
01588         }
01589     }
01590     return 1;
01591 }
01592
01593
01594
01595 template<class T> inline
01596 size_t wbararray<T>::nnz(const T eps, WBINDEX *I) const {
01597     size_t i,n, s=numel(); T a;
01598
01599     for (i=n=0; i<s; ++i) {
01600         a=data[i]; if (a<0) { a=-a; };
01601         if (a>eps) { ++n; }
01602     }
01603
01604     if (I) {
01605         I->init(n);
01606         for (i=n=0; i<s; ++i) {
01607             a=data[i]; if (a<0) a=-a;
01608             if (a>eps) I->data[n++]=i;
01609         }
01610     }
01611     return n;
01612 }
01613
01614
01615
01616 template<class T> inline
01617 void wbararray<T>::SetRef (
01618     const widx_t *I, size_t len, const T* d0
01619 ){
01620     size_t i,k,s, S=numel(), r=SIZE.len, l=len-1;
01621     size_t nz=r-len;
01622
01623     if (len>r || r==0 || (len && data==NULL)) wblog(FL,
01624         "ERR %s() index out of bounds ([%s]) for %s !?",FCT,
01625         (WBINDEX(len,I)+1).toStr().data, SSTR_(this));
01626     for (i=0; i<len; ++i) if (I[i]>=SIZE[i+nz]) wblog(FL,
01627         "ERR %s() index out of bounds (%s; %s)",FCT,
01628         (WBINDEX(len,I)+1).toStr().data, SSTR_(this));
01629
01630     if (!nz || !len) wblog(FL,"ERR invalid reference (len=%d/%d)",len,r);
01631
01632     for (s=SIZE[0], i=1; i<nz; i++) s*=SIZE[i];
01633     for (k=I[l], i=l-1; i<l; i--) { k = k*SIZE[i+nz] + I[i]; }
01634     k*=s;
01635
01636     if (k+s>S || d0==NULL) wblog(FL,
01637         "ERR index out of bounds ( %d+%d / %d; 0x%lX)\n"
01638         "having I=[%s] for %s array",k,s,S,d0,
01639         (WBINDEX(len,I)+1).toStr().data, SSTR_(this)
01640     );
01641
01642     MEM_COPY<T>(data+k,s,d0);
01643 };
01644
01645 template<class T> inline
01646 wbararray<T>& wbararray<T>::Squeeze() {
01647     size_t r=SIZE.len; if (r>1) {
01648         unsigned i=0, l=0; size_t *s=SIZE.data;
01649         for (; i<r; ++i) { if (s[i]!=1) {
01650             if (l!=i) { s[l]=s[i]; }

```

```

01652         ++l;
01653     }}
01654     if (l<r) {
01655         SIZE.len=(l? l:1);
01656     }
01657 }
01658 return *this;
01659 };
01660
01661 template<class T> inline
01662 int wbararray<T>::skipSingletons(const char *F, int L,
01663 unsigned r2
01664 ) {
01665     int m=0;
01666     unsigned i, r=SIZE.len;
01667
01668     if (r2==r || (!r && r2<=0)) { return m; }
01669     if (!r) { return (m=-r2); }
01670
01671     if (int(r2)<=0) { r2=-r2; if (!r2) { r2=1; }
01672     if (r2<r) {
01673         for (i=r-1; i>=r2; --i) { if (SIZE[i]!=1) break; }
01674
01675         m=r-1-i;
01676         if (m) { SIZE.Shorten2(i+1); }
01677     }}
01678     else if (r2<r) { int e=0;
01679         for (i=r2; i<r; ++i) { if (SIZE[i]!=1) { ++e; if (F) wblog(F_L,
01680             "ERR %s() non-singleton at %d/%d in [%s]",FCT,i+1,r2,STR(SIZE));
01681         }}
01682         if (e) { m=-e; }
01683         else { m=r-r2; SIZE.Shorten2(r2); }
01684     }
01685     else { m=-r2;
01686         if (F) wblog(F_L,"ERR %s() rank out of bounds (%d/%d)",FCT,r2,r);
01687     }
01688
01689     return m;
01690 };
01691
01692 template<class T> inline
01693 wbararray<T>& wbararray<T>::appendSingletons(
01694     const char *F, int L, unsigned r, unsigned m) {
01695
01696     unsigned i, l=SIZE.len;
01697     if (l<r) { unsigned s=1;
01698         if (!l) { s=0;
01699         }
01700         SIZE.Resize(r); for (i=l; i<r; ++i) { SIZE[i]=s; }
01701     }
01702     else if (l>r) {
01703         size_t *s=SIZE.data;
01704
01705         if (l==2 && r==1 && s[0]==1) {
01706             s[0]=s[1]; SIZE.len=1;
01707         }
01708         else {
01709             if (r>2) { r+=m; } else
01710             if (!r) wblog(FL,"ERR %s() got r=%d+%d",FCT,r,m);
01711
01712             if (l>r) {
01713                 for (i=r; i<l; ++i) { if (s[i]!=1) { wblog(F_L,
01714                     "ERR %s() got r=%d->%d for %s",FCT,l,r,SSTR_(this));
01715                 }}
01716                 SIZE.len=r;
01717             }
01718         }
01719     }
01720
01721     return *this;
01722 };
01723
01724 template<class T> inline
01725 void wbararray<T>::prependSingletons(unsigned r) {
01726
01727     if (!SIZE.len) wblog(FL,"ERR %s() called on empty array",FCT);
01728
01729     if (r>SIZE.len) {
01730         size_t i, l=SIZE.len-1, m=r-SIZE.len;
01731         SIZE.Resize(r);
01732         for (i=l; i<r; --i) SIZE[i+m]=SIZE[i];
01733         for (i=0; i<m; ++i) SIZE[i]=1;
01734     }
01735     else if (r<SIZE.len) wblog(FL,
01736         "ERR %s() invalid extended rank !? (%d->%d)",FCT,SIZE.len,r);
01737 };
01738

```

```

01739 template<class T> inline
01740 wbararray<T>& wbararray<T>::ExpandOM(const char *F, int L, unsigned r0,
01741 const wvector<unsigned> &S,
01742 const unsigned *sx
01743 ){
01744     if (!SIZE) {
01745         if (!S && !sx) { return *this; }
01746         wblog(FL,"ERR %s() got empty vector with S=%s",FCT,STR(S));
01747     }
01748     if (!S) { if (!sx) {
01749         if (int(r0)<=0 || r0==SIZE.len) { return *this; }}
01750         wblog(FL,"ERR %s() got empty S=[%s] for %s",FCT,STR(S),SSTR_(this));
01751     }
01752     if (sx && S.isEqual(sx)) { sx=NULL; }
01753
01754     unsigned i=0, l=SIZE.len;
01755     size_t M=1;
01756
01757     if (int(r0)<=0) { r0=l-S.len;
01758         if (int(r0)<2) wblog(FL,
01759             "ERR %s() requesting %d/%d OM indices",FCT,S.len,l);
01760     }
01761     else if (r0+S.len<l) wblog(F_L,
01762         "ERR %s() rank out of bounds %d+%d / %d",FCT,r0,S.len,l);
01763
01764     M=( r0<l ? numOM(r0) : 1 );
01765
01766     if (M!=S.prod()) wblog(FL,
01767         "ERR %s() got OM size mismatch %s @ %d <> %s",FCT,
01768         SSTR_(this), r0, SSTR(S));
01769
01770     if (r0<=2 && M>1) wblog(FL,"ERR %s() "
01771         "got OM for rank-%d array %s (l=%d)",FCT,r0,SSTR_(this), S.len);
01772
01773     if (sx) { M=1;
01774         for (; i<S.len; ++i) { M*=sx[i]; if (sx[i]!=S[i]) {
01775             if (sx[i]<S[i]) { wblog(FL,"ERR %s() "
01776                 "cannot zero-pad to smaller size\nS=[%s] -> [%s]",
01777                 FCT,STR(S),STR(wvector<unsigned>(S.len,sx,'r')));
01778             }
01779         }}
01780     if (M>1 && r0<=2) { wblog(FL,"ERR %s() "
01781         "got OM for rank-%d array %s (M=%ld)",FCT,r0,SSTR_(this),M);
01782     }
01783 }
01784
01785 if (l!=r0+1 && l!=r0+S.len) { unsigned j=0;
01786     for (i=r0; i<l && j<S.len; ++i, ++j) { if (SIZE[i]!=S[j]) {
01787         while (S[j]==1 && ++j<S.len) {
01788             if (SIZE[i]==S[j]) break;
01789         }
01790     }}
01791     for (; i<l; ++i) { if (SIZE[i]!=1) break; }
01792     for (; j<S.len; ++j) { if (S[j]!=1) break; }
01793
01794     if (i<l || j<S.len) { wblog(FL,
01795         "WRN %s() got OM size reshape %s @ %d <> %s",
01796         FCT, SSTR_(this), r0, SSTR(S));
01797     }
01798 }
01799
01800 wvector<size_t> SX; SIZE.resize(r0+S.len,SX);
01801 for (i=0; i<S.len; ++i) { SX[r0+i]=S[i]; }
01802 Reshape(SX);
01803
01804 if (sx) {
01805     for (i=0; i<S.len; ++i) { SX[r0+i]=sx[i]; }
01806     Resize(SX);
01807 }
01808
01809 return *this;
01810 };
01811
01812 template<class T> inline
01813 int wbararray<T>::ExpandOM(
01814     const char *F, int L, wbararray<T> &B,
01815     unsigned r
01816 ) {
01817     int rval=0, eq=0;
01818
01819     if (isEmpty() || B.isEmpty()) { return (rval=-1); }
01820
01821     if (!r) { unsigned R=MAX(SIZE.len,B.SIZE.len);
01822         if (!isScalar() || !B.isScalar() || R>2) { wblog(FL,
01823             "ERR %s() unexpected empty/scalar QSpaces (%s / %s, r=%d)",
01824             FCT, SSTR_(this), SSTR(B),R);

```

```

01826     }
01827     if ( SIZE.len<R) { appendSingletons(R); }
01828     if (B.SIZE.len<R) { B.appendSingletons(R); }
01829     return rval;
01830 }
01831
01832 if ((eq=SIZE.isEqual(B.SIZE))) {
01833     #ifndef WB_SKIP_ASSERT
01834         if (r<=2 && r<SIZE.len) { unsigned i=r;
01835             for (; i<SIZE.len; ++i) { if (SIZE[i]!=1) { wblog(FL,
01836                 "ERR %s() got OM space %s for rank r=%d",FCT,SSTR_(this),r);
01837             }}
01838         }
01839     #endif
01840     return rval;
01841 }
01842
01843 if (SIZE.len<r || B.SIZE.len<r) wblog(F_L,"ERR %s() "
01844     "rank out of bounds (r=%d/%d/%d)",FCT,r,SIZE.len,B.SIZE.len);
01845 if (memcmp(SIZE.data,B.SIZE.data,r*sizeof(SIZE[0]))) wblog(F_L,
01846     "ERR %s() size mismatch %s / %s (r=%d)",FCT,SSTR_(this),SSTR(B),r);
01847
01848 unsigned i, R=MAX(SIZE.len,B.SIZE.len);
01849
01850 if (SIZE.len!=B.SIZE.len) {
01851     if ( SIZE.len==r) { appendSingletons(R); rval|=1; } else
01852     if (B.SIZE.len==r) { B.appendSingletons(R); rval|=2; }
01853     else {
01854         wblog(F_L,"ERR %s() rank mismatch (r=%d/%d)",
01855             FCT,SIZE.len,B.SIZE.len);
01856     }
01857     eq=SIZE.isEqual(B.SIZE);
01858 }
01859
01860 if (r<3) {
01861     for (i=r; i<R; ++i) { if ( SIZE[i]!=1) { break; }}
01862     if (i<R || !eq) wblog(FL,"ERR %s() "
01863         "got OM space %s / %s for rank r=%d",FCT,SSTR_(this),SSTR(B),r);
01864 }
01865 else if (!eq) {
01866     wvector<size_t> S(SIZE); unsigned q=0;
01867     for (i=r; i<SIZE.len; ++i) {
01868         if (S[i]<B.SIZE[i]) { S[i]=B.SIZE[i]; ++q; }}
01869     if (q) { this->Resize(S); rval|=4; }
01870
01871     S=B.SIZE; q=0;
01872     for (i=r; i<SIZE.len; ++i) {
01873         if (S[i]<SIZE[i]) { S[i]=SIZE[i]; ++q; }}
01874     if (q) { B.Resize(S); rval|=8; }
01875 }
01876
01877 return rval;
01878 };
01879
01880 template<class T>
01881 template<class TI> inline
01882 int wbararray<T>::aMax(T &x, TI &k) const {
01883     int e=0; size_t n=numel(); x=0; k=0;
01884     if (n) { size_t i=0; T a;
01885         for (; i<n; ++i) { if (x<(a=Wb::abs(data[i]))) { x=a; k=i; }
01886     }
01887     else { k=-1; e=1; }
01888     return e;
01889 };
01890
01891 template<class T>
01892 template<class TI> inline
01893 T wbararray<T>::aMax(TI &i_, TI &j_, const wbararray *B) const {
01894     T x=0; i_=j_=0;
01895     if (SIZE.len!=2) wblog(FL,
01896         "ERR %s() invalid usage (s=%s)",FCT,SSTR_(this));
01897
01898     if (!B) {
01899         size_t k; int e=aMax(x,k);
01900         if (e)
01901             { wblog(FL, "WRN %s() got empty array (e=%d)",FCT,e); }
01902         else { i_=k*SIZE[0]; j_=k/SIZE[0]; }
01903         return x;
01904     }
01905
01906     if (B->SIZE.len!=2) wblog(FL,
01907         "ERR %s() invalid usage (s=%s)",FCT,SSTR_(B));
01908
01909     TI na=SIZE[0], nb=B->SIZE[0],
01910         ma=SIZE[1], mb=B->SIZE[1], n=MIN(na,nb), m=MIN(ma,mb), i,j;
01911
01912     if (n && m) { T a, b;

```

```

01913         for (i=0; i<n; ++i) {
01914             for (j=0; j<m; ++j) {
01915                 a=Wb::abs( data[i+j*na]);
01916                 b=Wb::abs(B->data[i+j*nb]); if (a<b) { a=b; }
01917                 if (x<a) { x=a; i_=i; j_=j; }
01918             }
01919         }
01920     else { wblog(FL, "WRN %s() got empty array (%dx%d)",FCT,n,m); }
01921
01922     return x;
01923 };
01924
01925 template<class T> inline
01926 double wbararray<T>::maxDiff (const wbararray<T> &B) const {
01927     size_t i,s=numel();
01928     double dmax=0;
01929
01930     if (SIZE!=B.SIZE) {
01931         sprintf_str("%s:%d data SIZE mismatch ([%s, %s])",
01932             FL,SSTR_(this),SSTR(B));
01933         return NAN;
01934     }
01935
01936     for (i=0; i<s; i++)
01937         dmax=MAX(dmax, Wb::abs(data[i]-B.data[i]));
01938
01939     return dmax;
01940 };
01941
01942 template<class T> inline
01943 void wbararray<T>::TimesEl(const wbararray<T> &B, char conj) {
01944
01945     if (!sameSize(B)) wblog(FL,
01946         "ERR %s() size mismatch: %s <> %s",FCT,SSTR_(this),SSTR(B));
01947
01948     Wb::TimesElRange(data,B.data,numel(),conj);
01949 };
01950
01951 template<class T>
01952 wbararray<T>& wbararray<T>::timesEl(
01953     const wbararray &B, wbararray &C, T bfac, T cfac, char conj) const {
01954
01955     if (!sameSize(B)) wblog(FL,
01956         "ERR %s() size mismatch\n%s <> %s",FCT,SSTR_(this),SSTR(B));
01957
01958     if (!cfac || C.isEmpty()) {
01959         if (bfac) { C=*this;
01960             Wb::TimesElRange(C.data,B.data,numel(),conj);
01961             if (bfac!=T(1)) { C*=bfac; }
01962         }
01963         else C.init(SIZE);
01964     }
01965     else if (!sameSize(C)) { wblog(FL,
01966         "ERR %s() size mismatch\n%s <> %s",FCT,SSTR_(this),SSTR(C)); }
01967     else {
01968         if (cfac!=T(1)) { C*=cfac; }
01969         if (bfac) {
01970             Wb::timesElRange_add(C.data, data,B.data,numel(),bfac,conj);
01971         }
01972     }
01973     return C;
01974 };
01975
01976 template<class T>
01977 wbararray<T>& wbararray<T>::timesEl_OM(
01978     unsigned r, const wbararray &B, wbararray &C, char conj
01979 ) const {
01980
01981     if (!r || r>SIZE.len) wblog(FL,
01982         "ERR %s() invalid rank (r=%d/%d)",FCT,r,SIZE.len);
01983     if (!sameSize(B)) wblog(FL,
01984         "ERR %s() size mismatch %s / %s",FCT,SSTR_(this),SSTR(B));
01985
01986     unsigned i=0; size_t N=1, M=1;
01987     const size_t *sa=SIZE.data, *sb=B.SIZE.data;
01988
01989     bool self=( &C==this || &C==&B || C.data==data || C.data==B.data);
01990
01991     for (; i<r; ++i) {
01992         if (sa[i]!=sb[i]) wblog(FL,"ERR %s() "
01993             "size mismatch %s / %s (i=%d/%d)",FCT,SSTR_(this),SSTR(B),i+1,r);
01994         N*=sa[i];
01995     }
01996     for (; i<SIZE.len; ++i) { M*=sa[i]; }
01997
01998     if (!self) {
01999         C.init_bare(wbvector<size_t>(r,SIZE.data,'r'));

```

```

02000     }
02001
02002     Wb::timesElRange_OM(C.data, data,B.data,N,M,conj);
02003
02004     if (self)
02005         { C.SIZE.len=r; }
02006
02007     return C;
02008 };
02009
02010 template<class T> inline
02011 T wbararray<T>::dotProd(const wbararray<T> &B, char conj) const {
02012     T x=T(0);
02013     if (!sameSize(B)) wblog(FL,
02014         "ERR %s() size mismatch: %s <> %s",FCT,SSTR_(this),SSTR(B));
02015
02016     return Wb::dotProd(data,B.data,numel(),x,conj);
02017 };
02018
02019 template<class T> inline
02020 T wbararray<T>::weightedAvg(const wbararray<T> &B) const {
02021     size_t i,s=numel(); T w,x=0,n2=0;
02022
02023     if (!sameSize(B)) wblog(FL,
02024         "ERR %s() size mismatch: %s <> %s",FCT,SSTR_(this),SSTR(B));
02025
02026     for (i=0; i<s; i++) { w=B.data[i]; x+=(data[i]*w); n2+=w*Wb::CONJ(w); }
02027
02028     if (n2<=0) wblog(FL,
02029         "ERR weighted avg with total weight %.4g",n2);
02030     return x/std::sqrt(n2);
02031 };
02032
02033 template<class T>
02034 wbararray<T>& wbararray<T>::tensorProd(
02035     const wbararray<T> &B_, wbararray<T> &C,
02036     const char aflag0, const char bflag0,
02037     const char kflag
02038 ) const {
02039
02040     if (this==&C || &B_==&C) {
02041         wbararray<T> X; tensorProd(B_,X,aflag0,bflag0,kflag);
02042         return X.save2(C);
02043     }
02044
02045     wbararray<T> A,B;
02046     opFlags<T> aflag(aflag0), bflag(bflag0);
02047     size_t ra=SIZE.len, rb=B_.SIZE.len;
02048
02049     aflag.applyOrRef(FL,*this,A);
02050     bflag.applyOrRef(FL, B_, B);
02051
02052     if (ra!=rb) {
02053         if (ra>2 || rb>2) wblog(FL,
02054             "ERR %s() rank mismatch (r=%d/%d)",FCT,ra,rb);
02055         if (!ra || !rb) { return C.init(); }
02056
02057         if (ra==1) { A.ExpandDiagonal(); ra= SIZE.len; } else
02058         if (rb==1) { B.ExpandDiagonal(); rb=B.SIZE.len; }
02059
02060         if (ra!=rb) wblog(FL,
02061             "ERR %s() rank mismatch (%d/%d)",FCT,ra,rb);
02062     }
02063
02064     const size_t r=A.SIZE.len, l=r-1, *sa=A.SIZE.data, *sb=B.SIZE.data;
02065     WBINDEX Ia(r), Ib(r), Ja(r), Jb(r), S;
02066
02067     size_t i=0,k=0,ia,ib,N;
02068
02069     if (kflag) {
02070         S.init(r);
02071         for (; i<r; ++i) { S[i]=sa[i]*sb[i]; }
02072     }
02073     else {
02074         S.init(2*r);
02075         for (; i<r; ++i) { S[k++]=sa[i]; S[k++]=sb[i]; }
02076     }
02077
02078     C.init(S); N=S.prod(); Ja[l]=Jb[l]=0;
02079
02080     for (k=1,i=0; i<N; ++i) {
02081         for (--k; k<r; --k) {
02082             Ja[k] = ( Ja[k+1] + Ia[k+1] ) * sa[k];
02083             Jb[k] = ( Jb[k+1] + Ib[k+1] ) * sb[k];
02084         }
02085
02086         ia = Ja[0] + Ia[0];

```



```

02087         ib = Jb[0] + Ib[0];
02088         C.data[i] = A.data[ia] * B.data[ib];
02089
02090         k=0;
02091         if ( (++Ia[k]) >= sa[k] ) { Ia[k]=0; ++Ib[k]; }
02092         while (Ib[k]>=sb[k] && k<1) {
02093             Ib[k]=Ia[k]=0; ++k;
02094             if ( (++Ia[k]) >= sa[k] ) { Ia[k]=0; ++Ib[k]; }
02095         }
02096     }
02097
02098     return C;
02099 };
02100
02101 template<class T>
02102 wbararray<T>& wbararray<T>::Cat(
02103     unsigned dim,
02104     const wbvector< wbararray<T> > &aa
02105 ){
02106     wbvector< const wbararray<T>* > ap(aa.len);
02107     for (unsigned i=0; i<aa.len; i++) ap[i]=&aa[i];
02108     Cat(dim,ap); return *this;
02109 };
02110
02111 template<class T>
02112 wbararray<T>& wbararray<T>::Cat(
02113     unsigned dim,
02114     const wbvector< wbararray<T>* > &ap
02115 ){
02116     if (ap.len==0) { init(); return *this; }
02117     if (ap.len==1) { *this=(*ap[0]); return *this; }
02118
02119     size_t i,j=0,k,n, e=0, N, *s2=0;
02120     const size_t r=ap[0]->SIZE.len, l=r-1, *sref=ap[0]->SIZE.data;
02121     const WBINDEX &S0=ap[0]->SIZE;
02122     WBINDEX S(S0), I(r), IO(ap.len), idx;
02123
02124     if (dim<1) wblog(FL,"ERR invalid index (not 1-based; %d/%d)",dim,r);
02125     dim--;
02126
02127     if (dim<r) N=sref[dim];
02128
02129     for (i=1; i<ap.len; i++) {
02130         if (ap[i]==NULL || ap[i]->data==NULL) continue;
02131
02132         if (r != ap[i]->SIZE.len) e++; else {
02133             s2=ap[i]->SIZE.data;
02134             for (j=0; j<r; j++) if (j!=dim && s2[j]!=sref[j]) e++;
02135         }
02136         if (e) wblog(FL,"ERR size mismatch %d/%d: %s <> %s (%d)",
02137             i+1,ap.len, SSTR_(ap[0]),SSTR_(ap[i]),dim+1);
02138
02139         IO[i]=N; if (dim<r) N+=s2[dim];
02140     }
02141
02142     if (dim>=r) { T *dd;
02143         S.Resize(dim+1); S[dim]=ap.len; init(S); dd=data; n=ap[0]->numel();
02144         for (i=0; i<ap.len; i++, dd+=n) {
02145             MEM_COPY<T>(dd,n,ap[i]->data);
02146         }
02147         return *this;
02148     }
02149
02150     idx.init(N);
02151     for (k=i=0; i<ap.len; i++) {
02152         if (ap[i]==NULL || ap[i]->data==NULL) continue;
02153         for (n=ap[i]->SIZE.data[dim], j=0; j<n; j++, k++) idx[k]=i;
02154     }
02155
02156     S[dim]=N; init(S); n=numel();
02157
02158     for (i=0; i<n; i++) {
02159         widx_t q=idx[I[dim]];
02160         const size_t *s=ap[q]->SIZE.data;
02161
02162         for (j=0,k=1; k<r; k--) {
02163             if (j) j*=s[k];
02164             j += (k!=dim ? I[k] : (I[k]-IO[q]));
02165         }
02166
02167         data[i]=ap[q]->data[j];
02168
02169         k=0; I[0]++;
02170         while (I[k]>=SIZE[k] && k<1) { I[k]=0; ++I[++k]; }
02171     }
02172
02173     return *this;

```

```

02174 };
02175
02176 template<class T>
02177 wbararray<T>& wbararray<T>::BlockCat(
02178     const wbararray< const wbararray<T>* > &ap,
02179     WBINDEX *D1_, WBINDEX *D2_
02180 ){
02181     size_t i,j,m,n,i0,j0,d1,d2,M,N;
02182     WBINDEX D1,D2;
02183
02184     if (ap.isEmpty()) {
02185         init(); if (D1_) D1_>init(); if (D2_) D2_>init();
02186         return *this;
02187     }
02188
02189     if (ap.rank()!=2) wblog(FL,
02190         "ERR %s() invalid input matrix (%s)",FCT,SSTR(ap));
02191
02192     m=ap.SIZE[0]; n=ap.SIZE[1]; D1.init(m); D2.init(n);
02193     for (i=0; i<m; i++)
02194         for (j=0; j<n; j++) if (ap(i,j) && !ap(i,j)->isEmpty()) {
02195             const wbararray<T> &a=(*ap(i,j));
02196             if (a.rank()!=2) wblog(FL,
02197                 "ERR %s() invalid block rank (%d,%d): %s",FCT,i+1,j+1,SSTR(a));
02198
02199             if (D1[i]) {
02200                 if (D1[i]!=a.SIZE[0]) wblog(FL,
02201                     "ERR %s() block-size mismatch (%d,%d: %s <> %dx..",
02202                     FCT, i+1, j+1, SSTR(a), D1[i]);
02203             }
02204             else D1[i]=a.SIZE[0];
02205
02206             if (D2[j]) {
02207                 if (D2[j]!=a.SIZE[0]) wblog(FL,
02208                     "ERR %s() block-size mismatch (%d,%d: %s <> ..x%d",
02209                     FCT, i+1, j+1, SSTR(a), D2[j]);
02210             }
02211             else D2[j]=a.SIZE[0];
02212         }
02213
02214     if (D1_) D1_>init();
02215     if (D2_) D2_>init();
02216
02217     M=D1.sum(); N=D2.sum();
02218     init(M,N); if (isEmpty()) return *this;
02219
02220     for (j0=j=0; j<n; j++, j0+=d2) { d2=D2[j];
02221         for (i0=i=0; i<m; i++, i0+=d1) { d1=D1[i];
02222             ap(i,j)->copyStride(ref(i0,j0),M);
02223         }
02224     }
02225     return *this;
02226 };
02227
02228 template<class T>
02229 wbararray<T>& wbararray<T>::blockTrace(
02230     size_t D1,
02231     wbararray<T> &a, size_t D2
02232 ) const {
02233
02234     if (rank()!=2) wblog(FL,
02235         "ERR %s() only applies to rank-2 tensors (%d)",FCT,rank());
02236
02237     size_t i, dim1=SIZE[0], dim2=SIZE[1], m=(D1 ? dim1/D1 : 0);
02238     if (int(D2)<0) { D2=D1; }
02239
02240     if (!D1 || !D2 || dim1%D1 || dim2%D2 || (dim2/D2)!=m) wblog(FL,
02241         "ERR %s() invalid block specs (%d/%d; %d/%d; %d)",
02242         D1,dim1, D2,dim2, m);
02243
02244     a.init(D1,D2);
02245     for (i=0; i<m; ++i) { addBlock(i*D1, i*D2, D1, D2, a); }
02246
02247     return a;
02248 };
02249
02250 template<class T>
02251 wbararray<T>& wbararray<T>::BlockDiag(const wvector< wbararray<T> > &D) {
02252
02253     size_t i,d1=0,d2=0, i0=0, j0=0, D1=0, D2=0;
02254
02255     for (i=0; i<D.len; i++) {
02256         if (!D[i].isMatrix()) wblog(FL,
02257             "ERR %s() matrices expected (%s)",FCT,SSTR(D[i]));
02258         D1+=D[i].SIZE[0];
02259         D2+=D[i].SIZE[1];
02260     }

```

```

02261
02262     init(D1,D2);
02263     for (i=0; i<D.len; i++, i0+=d1,j0+=d2) {
02264         d1=D[i].SIZE[0];
02265         d2=D[i].SIZE[1];
02266         Wb::cpyStride(ref(i0,j0), D[i].data,d1,d2,D1);
02267     }
02268
02269     return *this;
02270 };
02271
02272 template<class T>
02273 wbararray<T>& wbararray<T>::addBlock(
02274     size_t i, size_t j,
02275     size_t n, size_t m,
02276     wbararray<T> &a
02277 ) const {
02278
02279     if (rank()!=2) wblog(FL,
02280         "ERR %s() only applies to rank-2 tensors (%s)",FCT,SSTR_(this));
02281     size_t dim1=SIZE[0], dim2=SIZE[1];
02282
02283     if (i+n>dim1 || j+m>dim2) wblog(FL,
02284         "ERR %s() index out of bounds (%d:%d/%d; %d:%d/%d)",
02285         i+1,i+n,dim1,j+1,j+m,dim2);
02286
02287     if (a.isEmpty()) a.init(n,m);
02288     Wb::cpyStride(a.data, ref(i,j), n,m, -1,dim1,'+');
02289
02290     return a;
02291 };
02292
02293 template<class T>
02294 T wbararray<T>::norm2block(
02295     size_t i0, size_t j0,
02296     size_t n, size_t m
02297 ) const {
02298
02299     T x2=0;
02300
02301     if (rank()!=2) wblog(FL,
02302         "ERR %s() only applies to rank-2 tensors (%s)",FCT,SSTR_(this));
02303     size_t dim1=SIZE[0], dim2=SIZE[1];
02304
02305     if (i0+n>dim1 || j0+m>dim2) wblog(FL,
02306         "ERR %s() index out of bounds (%d:%d/%d; %d:%d/%d)",
02307         i0+1,i0+n,dim1, j0+1,j0+m,dim2);
02308
02309     if (m && n) {
02310         size_t i,j; const T* d = data + i0 + j0*dim1;
02311         for (j=0; j<m; ++j, d+=dim1) {
02312             for (i=0; i<n; ++i) { x2+=Wb::norm2(d[i]); }
02313         }
02314
02315         return x2;
02316     };
02317
02318 template<class T>
02319 wbararray<T>& wbararray<T>::getBlock(
02320     size_t i, size_t j,
02321     size_t n, size_t m,
02322     wbararray<T> &a
02323 ) const {
02324
02325     if (rank()!=2) wblog(FL,
02326         "ERR %s() only applies to rank-2 tensors (%d)",FCT,rank());
02327
02328     size_t dim1=SIZE[0], dim2=SIZE[1];
02329
02330     if (i+n>dim1 || j+m>dim2) wblog(FL,
02331         "ERR %s() index out of bounds (%d..%d/%d; %d..%d/%d)",
02332         i+1,i+n,dim1,j+1,j+m,dim2);
02333
02334     a.init(n,m); Wb::cpyStride(a.data, ref(i,j),n,m,-1,dim1);
02335     return a;
02336 };
02337
02338 template<class T>
02339 void wbararray<T>::copyStride(T* dd, size_t stride, T afac) const {
02340
02341     if (SIZE.len!=2) wblog(FL,"ERR %s() "
02342         "only intended for rank-2 tensors (r=%d)",FCT,rank());
02343
02344     size_t i, j=0, dim1=SIZE[0], dim2=SIZE[1];
02345     T *d0=data;
02346
02347     if (afac==T(1)) {

```

```

02348         for (; j<dim2; ++j, dd+=stride, d0+=dim1) {
02349             MEM_COPY<T>(dd,dim1,d0);
02350         }
02351     }
02352     else if (afac==T(-1)) {
02353         for (; j<dim2; ++j, dd+=stride, d0+=dim1) {
02354             for (i=0; i<dim1; ++i) { dd[i]= -d0[i]; }
02355         }
02356     else if (afac) {
02357         for (; j<dim2; ++j, dd+=stride, d0+=dim1) {
02358             for (i=0; i<dim1; ++i) { dd[i]=afac*d0[i]; }
02359         }
02360     else {
02361         for (; j<dim2; ++j, dd+=stride) {
02362             for (i=0; i<dim1; ++i) { dd[i]=0; }
02363         }
02364     };
02365
02366     template<class T>
02367     void wbararray<T>::addStride(T* dd, size_t stride, const T afac) const {
02368
02369         if (SIZE.len==2) wblog(FL,"ERR %s() "
02370             "only intended for rank-2 tensors (r=%d)",FCT,rank());
02371
02372         size_t i=0, j=0, dim1=SIZE[0], dim2=SIZE[1];
02373         T *a=data;
02374
02375         if (afac==T(1)) {
02376             for (; j<dim2; ++j, a+=dim1, dd+=stride) {
02377                 for (i=0; i<dim1; ++i) { dd[i] += a[i]; }
02378             }
02379         else if (afac==T(-1)) {
02380             for (; j<dim2; ++j, a+=dim1, dd+=stride) {
02381                 for (i=0; i<dim1; ++i) { dd[i] -= a[i]; }
02382             }
02383         else if (afac) {
02384             for (; j<dim2; ++j, a+=dim1, dd+=stride) {
02385                 for (i=0; i<dim1; ++i) { dd[i]+=(afac*a[i]); }
02386             }
02387     };
02388
02389     template<class T> inline
02390     wbararray<T>& wbararray<T>::Reshape(const wbvector<size_t> &S, bool lflag) {
02391
02392         if (numel()!=S.prod(0)) wblog(FL,
02393             "ERR %s() length mismatch (%s => %s)",FCT, SSTR_(this), SSTR(S));
02394
02395         if (!lflag) {
02396             unsigned i=0, j=0, e=0;
02397             const size_t *s1=SIZE.data, *s2=S.data; size_t sb;
02398
02399             while (!e) {
02400                 for (; i<SIZE.len; ++i) { if (SIZE[i]!=1) { break; } }
02401                 for (; j<S.len; ++j) { if (S[j] !=1) { break; } }
02402                 if (i>=SIZE.len || j>=S.len) { break; }
02403
02404                 if (s1[i]==s2[j]) { ++i; ++j; }
02405                 else if (s1[i]<s2[j]) {
02406                     for (sb=s1[i++]; i<SIZE.len; ++i) {
02407                         if ((sb*s1[i]) >= s2[j]) {
02408                             if (sb==s2[j]) { ++i; ++j; } else { ++e; }
02409                             break;
02410                         }
02411                     }
02412                 }
02413                 else if (s1[i]>s2[j]) {
02414                     for (sb=s2[j++]; j<S.len; ++j) {
02415                         if ((sb*s2[j]) >= s1[i]) {
02416                             if (sb==s1[i]) { ++i; ++j; } else { ++e; }
02417                             break;
02418                         }
02419                     }
02420                 }
02421             }
02422             if (e || i<SIZE.len || j<S.len) wblog(FL,"WRN %s() "
02423                 "got non-trivial reshape %s -> %s",FCT,SSTR_(this),SSTR(S));
02424         }
02425         SIZE=S; return *this;
02426     };
02427
02428     template<class T>
02429     void wbararray<T>::GroupIndizes(size_t K) {
02430         size_t i,n;
02431
02432         if (!K || SIZE.len%K) wblog(FL,
02433             "ERR %s() invalid block specs: mod(%d,%d)",FCT,SIZE.len,K);

```

```

02435
02436     n=SIZE.len/K;
02437
02438     for (i=0; i<n; ++i)
02439         SIZE[i]=Wb::prodRange(SIZE.data+i*K, K);
02440
02441     SIZE.Resize(n);
02442 }
02443
02444 template<class T> inline
02445 void wbararray<T>::groupIndices_P(
02446     const WBINDEX &I, int pos, wperm &P,
02447     size_t *s1, size_t *s2
02448 ) const {
02449
02450     size_t i,j,j1=0,j2=0,l, m=I.len, n=SIZE.len;
02451     wperm_t *p; widxt_t *idx;
02452     char mark[n ? n : 1];
02453
02454     if (m>n) wblog(FL,
02455         "ERR %s() invalid group index (len=%d/%d)",FCT,m,n);
02456     if (!n) {
02457         if (s1) { *s1=0; }
02458         if (s2) { *s2=0; }; return;
02459     }
02460     P.init(n); p=P.data; idx=I.data;
02461
02462     if (pos==1) { j1=0; j2=m; } else
02463     if (pos==2) { j1=n-m; j2=0; }
02464     else wblog(FL,"ERR %s() invalid pos=%d",FCT,pos);
02465
02466     memset(mark,0,n*sizeof(char));
02467
02468     for (j=j1, i=0; i<m; ++i, ++j) { l=p[j]=idx[i];
02469         if (l>n) wblog(FL,"ERR %s() index not unique (%d/%d)",FCT,l,n);
02470         if (++mark[l]>1) wblog(FL,
02471             "ERR %s() index not unique (%d/%d)",FCT,l,n);
02472     }
02473     for (j=j2, i=0; i<n; ++i) { if (!mark[i]) { p[j++]=i; } }
02474
02475     if (s1 || s2) {
02476         for (j1=j2=1, i=0; i<n; ++i) {
02477             if (mark[i])
02478                 { j1*=SIZE[i]; }
02479             else { j2*=SIZE[i]; }
02480         }
02481         if (s1) { *s1 = (pos==1 ? j1 : j2); }
02482         if (s2) { *s2 = (pos==1 ? j2 : j1); }
02483     }
02484 };
02485
02486 template<class T>
02487 wbararray<T>& wbararray<T>::transpose(const char *F, int L, wbararray<T> &A) const {
02488
02489     unsigned r=rank(); wperm P;
02490
02491     if (r%2) wblog(F_L,"ERR %s() "
02492         "applies to even-rank arrays only! (%s)",FCT,SSTR_(this));
02493
02494     P.initTranspose(r);
02495     permute(A,P);
02496
02497     return A;
02498 };
02499
02500 template<class T>
02501 wbararray<T>& wbararray<T>::flipLR(const char *F, int L, wbararray &B) const {
02502
02503     if (rank()!=2) wblog(F_L,
02504         "ERR %s() requires matrix (%s)",FCT,SSTR_(this));
02505     if (&B==this || B.data==data) wblog(F_L,
02506         "ERR %s() requires different objects!",FCT);
02507     B.init(SIZE);
02508
02509     if (SIZE[1]) {
02510         widxt_t i=0, j=0, dim1=SIZE[0], dim2=SIZE[1];
02511         const T *a=data; T *b=B.data+dim1*(dim2-1);
02512         for (; j<dim2; ++j, a+=dim1, b-=dim1) {
02513             for (i=0; i<dim1; ++i) b[i]=a[i];
02514         }
02515     }
02516
02517     return B;
02518 };
02519
02520 template<class T>
02521 wbararray<T>& wbararray<T>::FlipLR(const char *F, int L) {

```

```

02522
02523     if (rank()!=2) wblog(F_L,
02524         "ERR %s() requires matrix (%s)",FCT,SSTR_(this));
02525
02526     widx_t i=0, j=0, dim1=SIZE[0], dim2=SIZE[1], d2=dim2/2;
02527
02528     if (dim2>1) {
02529         T x, *a=data, *b=data+dim1*(dim2-1);
02530         for (; j<d2; ++j, a+=dim1, b-=dim1) {
02531             for (i=0; i<dim1; ++i) {
02532                 x=a[i]; a[i]=b[i]; b[i]=x;
02533             }
02534         }
02535     }
02536
02537     return *this;
02538 };
02539
02540 template<class T>
02541 wbararray<T>& wbararray<T>::flipUD(const char *F, int L, wbararray &B) const {
02542
02543     if (rank()!=2) wblog(F_L,
02544         "ERR %s() requires matrix (%s)",FCT,SSTR_(this));
02545     if (&B==this || B.data==data) wblog(F_L,
02546         "ERR %s() requires different objects!",FCT);
02547     B.init(SIZE);
02548
02549     if (SIZE[0]) {
02550         widx_t i=0, j=0, dim1=SIZE[0], dim2=SIZE[1], l=dim1-1;
02551         const T *a=data; T *b=B.data;
02552         for (; j<dim2; ++j, a+=dim1, b+=dim1) {
02553             for (i=0; i<dim1; ++i) b[i]=a[l-i];
02554         }
02555     }
02556
02557     return B;
02558 };
02559
02560 template<class T>
02561 wbararray<T>& wbararray<T>::FlipUD(const char *F, int L) {
02562
02563     if (rank()!=2) wblog(F_L,
02564         "ERR %s() requires matrix (%s)",FCT,SSTR_(this));
02565
02566     widx_t i=0, j=0, dim1=SIZE[0], dim2=SIZE[1], d2=dim1/2, l=dim1-1;
02567
02568     if (dim1>1) {
02569         T x, *a=data;
02570         for (; j<dim2; ++j, a+=dim1) {
02571             for (i=0; i<d2; ++i) {
02572                 x=a[i]; a[i]=a[l-i]; a[l-i]=x;
02573             }
02574         }
02575     }
02576
02577     return *this;
02578 };
02579
02580 template<class T>
02581 wbararray<T>& wbararray<T>::MatPermute(const wperm &P, char iflag){
02582
02583     if (isIdentityPerm(P)) return *this;
02584     if (!isSMatrix()) wblog(F_L,
02585         "ERR %s() expects square matrix (%s)",FCT,SSTR_(this));
02586
02587     wbararray<T> X(*this);
02588     const wperm_t *p=P.data;
02589     size_t i,j, dim1=SIZE[0], dim2=SIZE[1];
02590
02591     if (!iflag) {
02592         for (i=0; i<dim1; i++)
02593             for (j=0; j<dim2; j++) { data[i+dim1*j]=X.data[p[i]+dim1*p[j]]; }
02594     }
02595     else {
02596         for (i=0; i<dim1; i++)
02597             for (j=0; j<dim2; j++) { data[p[i]+dim1*p[j]]=X.data[i+dim1*j]; }
02598     }
02599
02600     return *this;
02601 };
02602
02603 template<class T>
02604 wbararray<T>& wbararray<T>::Permute(
02605     const wperm &P, char iflag, char rcpy) {
02606
02607     if (!P) { return *this; }
02608

```

```

02609     if (P.len>SIZE.len) { appendSingletons(P.len); }
02610
02611     if (!requiresDataPerm(P)) {
02612         SIZE.Permute(P,iflag);
02613         return *this;
02614     }
02615
02616     if (isRef()) {
02617         if (rcpy=='c' || rcpy=='C' || rcpy=='i' || rcpy=='I') {
02618             wbarry<T> X;
02619             return permute(X,P,iflag).save2(*this);
02620         }
02621         else if (rcpy!='r' && rcpy!='R') {
02622             wblog(PFL, "ERR %s() got ref (rcpy=%s)",FCT,cSTR(rcpy));
02623         }
02624     }
02625
02626     wbarry<T> X(*this);
02627     X.permute(*this,P,iflag);
02628
02629     return *this;
02630 };
02631
02632 template<class T>
02633 wbarry<T>& wbarry<T>::Permute(const char* sidx, char iflag, char rcpy) {
02634     wbperm P(sidx);
02635     if (P.isEmpty()) wblog(FL, "WRN got empty permutation '%s'", sidx);
02636     return Permute(P,iflag,rcpy);
02637 };
02638
02639 template<class T>
02640 wbarry<T>& wbarry<T>::permute(
02641     wbarry<T> &A,
02642     const char* sidx,
02643     char iflag
02644 ) const {
02645
02646     wbperm P(sidx);
02647
02648     if (P.isEmpty()) {
02649         wblog (FL, "WRN got empty permutation '%s'", sidx);
02650         return A.init(*this);
02651     }
02652     return permute(A,P,iflag);
02653 };
02654
02655 template<class T>
02656 wbarry<T>& wbarry<T>::permute(
02657     wbarry<T> &A,
02658     const wbperm &P0,
02659     char iflag
02660 ) const {
02661
02662     if (!P0) { return A=(*this); }
02663     if (&A==this) {
02664         wbarry<T> X;
02665         return permute(X,P0,iflag).save2(A);
02666     }
02667
02668     wbperm P;
02669
02670 #ifdef WB_CLOCK
02671     Wb::Clock clk("arr:permute",0);
02672 #endif
02673
02674     if (iflag!=0 && iflag !='I') wblog(FL,
02675         "ERR %s() invalid iflag '%c'< %d>",FCT,iflag,iflag);
02676
02677     P.init(P0,iflag,SIZE.len);
02678
02679     if (!requiresDataPerm(P)) {
02680         A=(*this); A.SIZE.Select(P);
02681         return A;
02682     }
02683
02684 #ifdef QS_USING_OMP
02685     int np=MAX(OMP_NUM_THREADS,QSP_NUM_THREADS);
02686 #else
02687     int np=0;
02688 #endif
02689
02690     A.init_bare(SIZE);
02691     SIZE.permute(A.SIZE,P);
02692
02693     wbarry_permute__(A.data,*this,P,np);
02694
02695     return A;

```

```

02696 };
02697
02698 template<class T>
02699 wbararray<T>& wbararray<T>::select0(
02700     const WBPERM &P,
02701     unsigned dim,
02702     wbararray<T> &A
02703 ) const {
02704
02705     if (&A==this) {
02706         wbararray<T> X(A); X.select0(P,dim,A);
02707         return A;
02708     }
02709
02710     const unsigned r=SIZE.len, l=r-1; size_t i,j,k,d;
02711     WBINDEX S,I;
02712
02713     if (dim>=SIZE.len) wblog(FL,
02714         "ERR %s() dimension out of bounds (%d/%d)",FCT,dim+1,SIZE.len);
02715
02716     S=SIZE; d=SIZE[dim]; S[dim]=P.len;
02717
02718     for (i=0; i<P.len; i++) if (P[i]>=d) wblog(FL,
02719         "ERR %s() index out of bounds (%d/%d)",FCT,P[i],d);
02720
02721     A.init(S); I.init(SIZE.len);
02722     widx_t *ip=I.data, *s=S.data; const wperm_t *p=P.data;
02723
02724     for (d=A.numel(), i=0; i<d; i++) {
02725         for (j=0, k=1; k<r; k--) {
02726             if (j) j*=SIZE[k];
02727             j+=(k!=dim ? ip[k] : p[ip[k]]);
02728         }
02729
02730         A.data[i]=data[j];
02731
02732         k=0; ip[0]++;
02733         while (ip[k]>=s[k] && k<l) { ip[k]=0; ++ip[++k]; }
02734     }
02735
02736     return A;
02737 }
02738
02739 template<class T>
02740 wbararray<T>& wbararray<T>::selectSqueeze(
02741     wbararray<T> &A, unsigned p,
02742     unsigned dim
02743 ) const {
02744
02745     if (&A==this) {
02746         wbararray<T> X(A); X.selectSqueeze(A,p,dim);
02747         return A;
02748     }
02749
02750     const unsigned r=SIZE.len, l=r-1; size_t i,j,k,s;
02751     WBINDEX S,I;
02752
02753     if (dim>=SIZE.len) wblog(FL,
02754         "ERR %s() dimension out of bounds (%d/%d)",FCT,dim+1,SIZE.len);
02755
02756     if (p>=SIZE[dim]) wblog(FL,
02757         "ERR %s() index out of bounds (%d/%d)",FCT,p,SIZE[dim]);
02758
02759     if (SIZE.len==1) {
02760         A.init(1); A.data[0]=data[p];
02761         return A;
02762     }
02763
02764     S=SIZE; S[dim]=1; A.init(S); s=A.numel(); I.init(SIZE.len);
02765
02766     for (i=0; i<s; i++) {
02767         for (j=0, k=1; k<r; k--) {
02768             if (j) j*=SIZE[k];
02769             j+=(k!=dim ? I[k] : p);
02770         }
02771
02772         A.data[i]=data[j];
02773
02774         k=0; I[0]++;
02775         while (I[k]>=S[k] && k<l) { I[k]=0; ++I[++k]; }
02776     }
02777
02778     for (i=dim+1; i<S.len; i++) A.SIZE[i-1]=A.SIZE[i];
02779     A.SIZE.len--; A.SIZE[A.SIZE.len]=0;
02780
02781     return A;
02782 }

```



```

02783
02784 template<class T>
02785 void wbararray<T>::setBlock(
02786     const wbvector< WBINDEX > &DB,
02787     const wbindex &IB,
02788     const wbararray<T> &A
02789 ){
02790     size_t i,j,ik,k,s, r=SIZE.len, l=r-1;
02791     wbindex I,I1(r),I2(r);
02792
02793     if (DB.len!=r || IB.len!=r) wblog(FL,
02794         "ERR %s() size mismatch (r: %d %d / %d)",FCT,DB.len,IB.len,r);
02795
02796     if (A.SIZE.len!=r) {
02797         for (i=r; i<A.SIZE.len; ++i) { if (A.SIZE[i]!=1) break; }
02798         if (i!=A.SIZE.len) wblog(FL,
02799             "ERR %s() size mismatch (r=%d / %d)",FCT,A.SIZE.len,r);
02800     }
02801
02802     for (k=0; k<r; ++k) {
02803         const WBINDEX &Dk=DB[k]; ik=IB[k];
02804
02805         if (ik>=Dk.len) wblog(FL,
02806             "ERR %s() block index out of bounds (%d: %d/%d)",
02807             FCT,k+1, ik+1, Dk.len);
02808         if (Dk[ik] != A.SIZE[k]) wblog(FL,
02809             "ERR %s() block size mismatch (%d: %d/%d)",
02810             FCT,k+1, Dk[ik], A.SIZE[k]);
02811
02812         I1[k]=Wb::addRange(Dk.data,ik);
02813         I2[k]=I1[k]+Dk[ik];
02814
02815         if (I2[k]>SIZE[k]) wblog(FL,
02816             "ERR %s() index out of bounds (%d: %d/%d)",
02817             FCT,k+1, I2[k], SIZE[k]);
02818     }
02819
02820     I=I1; s=A.numel();
02821
02822     for (i=0; i<s; ++i) {
02823         for (j=I[1],k=1-1; k<r; k--) j = j*SIZE[k] + I[k];
02824
02825         data[j]=A.data[i];
02826
02827         k=0; I[0]++;
02828         while (I[k]>=I2[k] && k<l) { I[k]=I1[k]; ++I[++k]; }
02829     }
02830 }
02831
02832 template<class T>
02833 void wbararray<T>::addBlock(
02834     const WBINDEX &IO,
02835     const wbararray<T> &A,
02836     const char dflag
02837 ){
02838     size_t i,j,k, s=A.numel(), r=SIZE.len, l=r-1;
02839     unsigned ra=(dflag ? r/2 : r), la=ra-1;
02840     const widx_t *S=A.SIZE.data;
02841     WBINDEX I(ra);
02842
02843     if (IO.len!=r || A.SIZE.len!=ra || (dflag && r%2)) wblog(FL,
02844         "ERR %s() size mismatch (%d/%d, %s; %c<%d>)",
02845         FCT,IO.len, A.SIZE.len, SSTR_(this), dflag
02846     );
02847     for (i=0; i<r; i++) if (IO[i]+S[i]>=ra ? i-ra : i)>SIZE[i])
02848         wblog(FL,"ERR %s() index out of bounds (%d: %s - %s %s)",
02849             FCT,i+1,IO.toStr().data,SSTR(A),SSTR_(this)
02850     );
02851
02852     for (i=0; i<s; i++) {
02853         for (j=0,k=1; k<r; k--) { if (j) j*=SIZE[k];
02854             j+=(IO[k]+I[k<ra ? k : k-ra]);
02855         }
02856
02857         data[j] += A.data[i];
02858
02859         k=0; I[0]++;
02860         while (I[k]>=S[k] && k<la) { I[k]=0; ++I[++k]; }
02861     }
02862 };
02863
02864 template<class T>
02865 T wbararray<T>::min() const {
02866     size_t i,n=numel(); T x;
02867
02868     if (n==0) {
02869         wblog(FLINE,"WRN min() of empty wbararray !?");

```

```

02870         memset (&x,0,sizeof(T)); return x;
02871     }
02872
02873     for (x=data[0], i=1; i<n; i++)
02874         if (x>data[i]) x=data[i];
02875
02876     return x;
02877 }
02878
02879 template<class T>
02880 T wbararray<T>::max() const {
02881     size_t i,n=numel(); T x;
02882
02883     if (n==0) {
02884         wblog(FLINE,"WRN max() of empty wbararray !?");
02885         memset (&x,0,sizeof(T)); return x;
02886     }
02887
02888     for (x=data[0], i=1; i<n; i++)
02889         if (x<data[i]) x=data[i];
02890
02891     return x;
02892 }
02893
02894 template<class T>
02895 T wbararray<T>::aMin(char zflag, size_t *k_) const {
02896
02897     size_t i=0, k=-1,n=numel(); T a,x=0;
02898
02899     if (!n) {
02900         if (k_) { (*k_)=-1; wblog(FL,"WRN %s() got empty array",FCT); }
02901         return 0;
02902     }
02903
02904     if (zflag==0) {
02905         x=Wb::abs(data[i++]); k=0; if (x!=0) {
02906             for (; i<n; ++i) { a=Wb::abs(data[i]); if (x>a) {
02907                 x=a; k=i; if (x==0) break;
02908             }}
02909         }
02910     }
02911     else {
02912         for (; i<n; ++i) {
02913             a=Wb::abs(data[i]); if (a) { x=a; k=i; break; }
02914         }
02915         for (; i<n; ++i) { a=Wb::abs(data[i]);
02916             if (a && x>a) { x=a; k=i; }
02917         }
02918     }
02919     if (k_) (*k_)=k;
02920     return x;
02921 };
02922
02923 template<class T>
02924 wstring wbararray<T>::toStr() const {
02925     wstring s;
02926     unsigned l=2; size_t n=numel();
02927     if (n==1) { s.init(24+4*SIZE.len);
02928         l=snprintf(s.data,s.len, SIZE.len==2? "%s (%s)" : "%s [%s]",
02929             STR(wbvector<T>(numel(),data,'r')), SSTR_(this));
02930     }
02931     else if (n || SIZE.len) { s.init(5 + 10*(numel()+SIZE.len));
02932         l=snprintf(s.data,s.len,"[%s] (%s)",
02933             STR(wbvector<T>(numel(),data,'r')), SSTR_(this));
02934     }
02935     else { s="[]"; }
02936
02937     if (l>s.len) wblog(FL,
02938         "WRN %s() string out of bounds (%d/%d)",FCT,l,s.len);
02939     return s;
02940 };
02941
02942 template<class T>
02943 wstring Wb::sizeStrM(
02944     unsigned r, const T* sd,
02945     unsigned m, const T* sm, const char *sep, const char *sepM
02946 ) {
02947     unsigned i=0, j, l=0, n=0, ndims=r+m;
02948     char *s; size_t x;
02949     wstring sout;
02950
02951     if (r>1 || m>1)
02952         { n+=(ndims-(r && m ? 2 : 1))*(sep ? strlen(sep) : 1); }
02953     if (m) { n+=(sepM ? strlen(sepM) : 1); }
02954
02955     for (; i<ndims; ++i) {
02956         x=(i<r ? sd[i] : sm[i-r]); if (x<0) { x=-x; ++n; }

```

```

02957         for (j=0; j<64; ++j) { if (!(x>>1)) break; }
02958         n+=(1+ceil(3*(double(i)/10)));
02959     }; if (n<8) n=8;
02960
02961     sout.init(n); s=sout.data;
02962
02963     for (i=0; i<r && l<n; ++i) {
02964         l+=snprintf(s+l,n-l,"%s%d",i? (sep? sep:" "):"", long(sd[i]));
02965     }
02966     for (i=0; i<m && l<n; ++i) {
02967         l+=snprintf(s+l,n-l,"%s%d", i? (sep? sep:" "): (sepM? sepM:"|"),
02968             long(sm[i]));
02969     }
02970
02971     if (l>=n) wblog(FL,
02972         "WRN %s() string out of bounds (%d/%d)\n'%s'",FCT,l,n,s);
02973     return sout;
02974 };
02975
02976 template<class T>
02977 void wbararray<T>::print_ref (const char *sb, const char *nl) const {
02978
02979     if (isRef() || mtype || (sptr && sptr->mtype)) {
02980         int l=0, n=32; char str[n]; str[0]=0;
02981
02982         if (sptr) {
02983             l+=snprintf(str,n,"%s",STR_(sptr));
02984             if (mtype!=sptr->mtype && l<n)
02985                 l+=snprintf(str+l,n-l," / %s !?",Wb::MTYPE_STR[mtype]);
02986         }
02987         else { l+=snprintf(str,n,"%s",Wb::MTYPE_STR[mtype]); }
02988
02989         printf(" %s%s%s", sb?sb:"", str, sb?sb:"", nl?nl:"");
02990
02991         if (l>=n) { printf("\n");
02992             wblog(FL,"WRN %s() string out of bounds (%d/%d)",FCT,l,n);
02993         }
02994     }
02995     else if ((nl && nl[0])) { printf("%s",nl); }
02996
02997     if (sptr && data!=sptr->data) {
02998         if (!nl && !nl[0]) { PRINTF("\n"); }
02999         wblog(FL, "WRN %s() data=%p / %p !?",FCT,data,sptr->data);
03000     }
03001 };
03002
03003 template<class T>
03004 void wbararray<T>::info(const char *F, int L,
03005     const char* istr, unsigned k,
03006     unsigned nlt, unsigned nlb
03007 ) const {
03008
03009     if (nlt) {
03010         printf("\n"); --nlt; if (nlt>1) {
03011             printf("\n"); --nlt; }
03012
03013         if (nlt) { --nlt;
03014             printf("%-20s %-8s %-8s %s\n",
03015                 "",
03016                 "name","size","data type");
03017         }
03018         if (nlt>4) nlt=4;
03019         for (unsigned i=0; i<nlt; ++i) { printf("\n"); }
03020     }
03021
03022     char s[32];
03023     if (int(k)>=0)
03024         { snprintf(s,32,"%s[%d]",istr?istr:"arr",k+1); }
03025     else { snprintf(s,32,"%s",istr?istr:""); }
03026
03027     #pragma omp critical (__ensure_sequential_wblog__)
03028     { printf("%-20s %-8s %-8s %s",shortFL(F_L),s,SSTR_(this),TSTR(T));
03029         print_ref();
03030     }
03031
03032     if (nlb) {
03033         if (nlb>10) nlb=10;
03034         for (unsigned i=0; i<nlb; ++i) { printf("\n"); }
03035     }
03036 };
03037
03038 template<class T>
03039 void wbararray<T>::info(const char* istr) const {
03040
03041     unsigned l=0, n=16; char s[n];
03042     size_t b=numel()*sizeof(T);
03043

```

```

03044     if (b<(1<10)) l=snprintf(s,n,"%ld bytes ",b); else
03045     if (b<(1<20)) l=snprintf(s,n,"%lg kB",b/double(1<10)); else
03046         l=snprintf(s,n,"%lg MB",b/double(1<20));
03047     if (l>=n) wblog(FL,"ERR %s() string out of bounds (%d/%d)",FCT,l,n);
03048
03049     #pragma omp critical (__ensure_sequential_wblog__)
03050     { printf(" %-12s %-10s %12s @ %12p %s",
03051         istr? istr:"", SSTR_(this), s, data, TSTR(T));
03052         print_ref();
03053     }
03054 };
03055
03056 template<class T>
03057 void wbararray<T>::print(
03058     const char *F, int L, const char* istr, const char* fmt0
03059 ) const {
03060
03061     if (F) {
03062         wstring tstr;
03063         if (typeid(T)==typeid(double)) tstr="double";
03064         else tstr.init(((char*)(typeid(T).name())));
03065
03066         wblog(F,L,"%s = [%s] %s array \\",
03067             istr[0]? istr:"ans", SSTR_(this), tstr.data);
03068         print_ref();
03069     }
03070     else info(istr);
03071
03072     printdata(istr, fmt0);
03073 };
03074
03075 template<class T>
03076 int wbararray<T>::printdata(
03077     const char *istr,
03078     const char *fmt0
03079 ) const {
03080
03081     unsigned i,j,k, r=SIZE.len, l=r-1, s=numel();
03082     wstring fmt;
03083
03084     if (!fmt0[0]) {
03085         if (typeid(T)==typeid(double) || typeid(T)==typeid(float))
03086             fmt=" %12.5g";
03087         else fmt.init2Fmt((T)0, 6);
03088     } else fmt=fmt0;
03089
03090     if (s>1) {
03091         if (SIZE.len<2) {
03092             printf("\n");
03093             for (i=0; i<s; i++) printf(fmt.data,data[i]);
03094             if (s) printf("\n\n");
03095         }
03096         else {
03097             WBINDEX I(r);
03098
03099             wbararray<T> A; wperm P(r); P[0]=1; P[1]=0;
03100             permute(A,P);
03101
03102             for (k=2, i=0; i<s; i++) {
03103                 if (k) { if (i) printf("\n");
03104                     if (k>1) {
03105                         if (r>2) {
03106                             printf("\n %s(:,:) ",istr);
03107                             for (j=2; j<r; j++) printf(",%ld",I[j]+1);
03108                             printf(") = [\n\n");
03109                         }
03110                         else printf("%s = [\n",istr);
03111                     }
03112                 }
03113
03114                 printf(fmt.data,A.data[i]);
03115
03116                 k=0; I[0]++;
03117                 while (I[k]>=A.SIZE[k] && k<1) { I[k]=0; ++I[++k]; }
03118             }
03119             printf("\n];\n");
03120         }
03121     }
03122     else if (s==1) {
03123         printf(" %s = ",istr);
03124         printf(fmt.data,data[0]); printf("\n");
03125     }
03126     else printf("\n");
03127
03128     return (int)s;
03129 };
03130

```

```

03131 template<>
03132 int wbarry<wbcomplex>::printdata(
03133     const char *istr, const char *fmt0
03134 ) const {
03135
03136     unsigned i,j,k, r=SIZE.len, l=r-1, s=numel();
03137     wstring fmt;
03138
03139     fmt=((fmt0 && fmt0[0]) ? fmt0 : " %8.4g%+8.4gi");
03140
03141     if (s>1) {
03142         if (SIZE.len<2) {
03143             printf("\n");
03144             for (i=0; i<s; i++) printf(fmt.data,data[i].r,data[i].i);
03145             if (s) printf("\n\n");
03146         }
03147         else {
03148             WBINDEX I(r);
03149
03150             wbarry<wbcomplex> A; wperm P(r); P[0]=1; P[1]=0;
03151             permute(A,P);
03152
03153             for (k=2, i=0; i<s; i++) {
03154                 if (k) { if (i) printf("\n");
03155                     if (k>1) {
03156                         if (r>2) {
03157                             printf("\n %s(:,:),",istr);
03158                             for (j=2; j<r; j++) printf(",%ld",I[j]+1);
03159                             printf(") = [\n\n");
03160                         }
03161                         else printf("%s = [\n",istr);
03162                     }
03163                 }
03164
03165                 printf(fmt.data,A.data[i].r,A.data[i].i);
03166
03167                 k=0; I[0]++;
03168                 while (I[k]>=A.SIZE[k] && k<1) { I[k]=0; ++I[++k]; }
03169             }
03170             printf("\n];\n");
03171         }
03172     }
03173     else if (s==1) {
03174         printf(" %s = ",istr);
03175         printf(fmt.data,data[0].r,data[0].i); printf("\n");
03176     }
03177     else printf("\n");
03178
03179     return (int)s;
03180 };
03181
03182 template<class T>
03183 template<class TB, class TC>
03184 wbarry<TC>& wbarry<T>::comm(
03185     const wbarry<TB> &B, wbarry<TC> &C, char aflag, char bflag
03186 ) const {
03187
03188     if ((void*)&C==(void*)this || (void*)&C==(void*)&B) {
03189         wbsparry<TC> X; comm(B,X,aflag,bflag);
03190         return X.save2(C);
03191     }
03192
03193     if (!isMatrix() || !B.isMatrix() || SIZE[0]!=B.SIZE[1] ||
03194         SIZE[0]!=SIZE[1] || B.SIZE[0]!=B.SIZE[1]) wblog(FL,
03195         "ERR %s() invalid operators for [A,B] (%s; %s)",
03196         FCT,SSTR_(this), SSTR(B)
03197     );
03198
03199     Wb::MatProd(*this,B,C,aflag,bflag);
03200     Wb::MatProd(B,*this,C,bflag,aflag,-1.,1.);
03201
03202     return C;
03203 };
03204
03205 template<class T>
03206 template<class TB, class TC>
03207 wbarry<TC>& wbarry<T>::acomm(
03208     const wbarry<TB> &B, wbarry<TC> &C, char aflag, char bflag
03209 ) const {
03210
03211     if (!isMatrix() || !B.isMatrix() ||
03212         SIZE[0]!=SIZE[1] || B.SIZE[0]!=B.SIZE[1]) wblog(FL,
03213         "ERR %s() invalid operators for [A,B] (%s; %s)",
03214         FCT,SSTR_(this), SSTR(B)
03215     );
03216
03217     Wb::MatProd(*this,B,C,aflag,bflag);

```

```

03218     Wb::MatProd(B, *this, C, bflag, aflag, +1., 1.);
03219
03220     return C;
03221 };
03222
03223 template<class T>
03224 template<class TB, class TC>
03225 wbararray<TC>& wbararray<T>::contractMat(
03226     const char *F, int L, unsigned i1,
03227     const wbararray<TB> &M, unsigned i2,
03228     wbararray<TC> &X
03229 ) const {
03230
03231     unsigned r=SIZE.len, l=r-1;
03232     wvector<size_t> S; wperm P;
03233     wbararray<T> A;
03234     wbararray<T> MA;
03235
03236     if (!M.isMatrix()) wblog (F_L,
03237         "ERR %s() rank 2 object required (%d)", FCT, M.SIZE.len);
03238     if (i1<1 || i1>SIZE.len || i2<1 || i2>2) wblog(F_L,
03239         "ERR %s() invalid contraction indizes [%d %d, %d %d]", i1, i2,
03240         FCT, rank(), M.rank());
03241
03242     if (i1==1) {
03243         toMatrixRef(MA, i1-1, 1, P); SIZE.select(P, S); l=0;
03244         if (i2==2)
03245             { Wb::MatProd(M, MA, X, 'T'); S[l]=M.SIZE[0]; }
03246         else { Wb::MatProd(M, MA, X, 'T'); S[l]=M.SIZE[1]; }
03247     }
03248     else {
03249         toMatrixRef(MA, i1-1, 2, P); SIZE.select(P, S);
03250         if (i2==1)
03251             { Wb::MatProd(MA, M, X, 'N'); S[l]=M.SIZE[1]; }
03252         else { Wb::MatProd(MA, M, X, 'N', 'T'); S[l]=M.SIZE[0]; }
03253     }
03254
03255     X.Reshape(S).Permute(P, 'I');
03256
03257     return X;
03258 };
03259
03260 template<class T>
03261 template<class TB> inline
03262 wbararray<T>& wbararray<T>::Contract(char *idx1,
03263     const wbararray<TB> &B, char *idx2,
03264     const wperm &pfinal
03265 ){
03266     wbararray<T> A(*this);
03267
03268     WBINDEX i1, i2;
03269     i1 = Str2Idx(idx1, 1);
03270     i2 = Str2Idx(idx2, 1);
03271
03272     A.contract(i1, B, i2, *this, pfinal);
03273     return *this;
03274 }
03275
03276 template<class T>
03277 template<class TB, class TC> inline
03278 wbararray<TC>& wbararray<T>::contract(const char *F, int L,
03279     const char *idx1, const wbararray<TB> &B, const char *idx2,
03280     wbararray<TC> &Q, const wperm &pfinal
03281 ) const {
03282
03283     wvector<unsigned> i1, i2;
03284     i1 = Str2Idx(idx1, 1);
03285     i2 = Str2Idx(idx2, 1);
03286
03287     return contract(F, L, i1, B, i2, Q, pfinal);
03288 }
03289
03290 template<class T>
03291 template<class TB> inline
03292 wbararray<T>& wbararray<T>::Contract(const wvector<unsigned> &i1,
03293     const wbararray<TB> &B, const wvector<unsigned> &i2,
03294     const wperm &pfinal
03295 ) {
03296     wbararray<T> A(*this); A.contract(i1, B, i2, *this, pfinal);
03297     return *this;
03298 }
03299
03300 template<class T>
03301 template<class TB, class TC>
03302 wbararray<TC>& wbararray<T>::contract(
03303     const char *F, int L, const iTags &ta,
03304     const wbararray<TB> &B, const iTags &tb, wbararray<TC> &C, iTags &tc,

```

```

03305     const wperm &pfinal, T afac, TC cfac
03306 ) const {
03307
03308     ctrIdx ia,ib;
03309
03310     ta.getCtr(F_L,tb,ia,ib,&tc);
03311     contract(F_L,ia,B,ib,C,pfinal,afac,cfac);
03312
03313     tc.Permute(pfinal);
03314     return C;
03315 };
03316
03317 template<class TA>
03318 template<class TB, class TC>
03319 wbarry<TC>& wbarry<TA>::contract(
03320     const char *F, const int L,
03321     const ctrIdx &ica, const wbarry<TB> &B, const ctrIdx &icb,
03322     wbarry<TC> &C0, const wperm &P,
03323     TA afac,
03324     TC cfac
03325 ) const {
03326
03327     if ((void*)(&C0)==(void*)this || (void*)(&C0)==(void*)(&B)) {
03328         wbarry<TC> Ci;
03329         if (cfac) {
03330             bool q1=((void*)(&C0)==(void*)this),
03331                 q2=((void*)(&C0)==(void*)(&B));
03332             wblog(FL,"ERR %s() got cfac=%s with self-reference (%s)",
03333                 FCT, NSTR(cfac), q1 && q2 ? "A=B=C" : (q1 ? "A=C" : "B=C"));
03334         }
03335         contract(F,L,ica,B,icb,Ci,P,afac,cfac);
03336         return Ci.save2(C0);
03337     }
03338
03339 #ifdef WB_CLOCK
03340     Wb::Clock clk("arr:contract",0);
03341 #endif
03342
03343     unsigned i,s; size_t one=1;
03344     char aflag, bflag;
03345     wvector<size_t> S1,S2;
03346     wbarry<TA> MA;
03347     wbarry<TB> MB;
03348     wbarry<TC> Ci;
03349     wperm p1,p2;
03350
03351     wvector<unsigned> i1(ica);
03352     wvector<unsigned> i2(icb);
03353
03354     char gotC_=1;
03355     if (C0.isEmpty()) {
03356         if (isEmpty() || B.isEmpty()) { return C0; }
03357         if (cfac && cfac!=TC(1)) wblog(FL,
03358             "ERR %s() got cfac=%s with empty C",FCT,NSTR(cfac));
03359         if (C0.mtype && C0.mtype!=Wb::MEX_RETURN) wblog(FL,
03360             "WRN %s() unexpected mt=%s",FCT,Wb::MTYPE_STR[C0.mtype]);
03361         SWAP(Ci.mtype,C0.mtype);
03362         gotC_=0;
03363     }
03364
03365     if (i1.isEmpty() || i2.len!=i1.len) wblog(F,L,
03366         "ERR %s() index set length mismatch [%s] / [%s]",FCT,STR(i1),STR(i2));
03367     for (s=SIZE.len, i=0; i<i1.len; ++i) { if (i1[i]>=s) wblog(F,L,
03368         "ERR %s() index out of bounds (A @ [%s]/%d)",FCT,STR(i1+1),s); }
03369     for (s=B.SIZE.len, i=0; i<i2.len; ++i) { if (i2[i]>=s) wblog(F,L,
03370         "ERR %s() index out of bounds (B @ [%s]/%d)",FCT,STR(i2+1),s); }
03371     for (i=0; i<i1.len; ++i) {
03372         if (SIZE[i1[i]]!=B.SIZE[i2[i]]) wblog(F,L,
03373             "ERR wbarry::%s() incompatible data size\n[%s] @ %d <> [%s] @ %d",
03374             FCT,SSTR_(this),i1[i]+1,SSTR(B),i2[i]+1); }
03375
03376     toMatrixRef(FL,MA,i1,2,aflag);     SIZE.getI(i1,S1);
03377     B.toMatrixRef(FL,MB,i2,1,bflag);    B.SIZE.getI(i2,S2);
03378
03379     if ((i=S1.len+S2.len)<2) {
03380         if (!i) { S2.init2val(2,one); }
03381     }
03382
03383     if (!afac) {
03384         WBINDEX S(S1,S2); if (P) S.Permute(P);
03385         if (gotC_) {
03386             if (C0.SIZE!=S) wblog(F_L,
03387                 "ERR %s() size mismatch %s / %s",FCT,SSTR(S),SSTR(C0));
03388             C0*=cfac;
03389         }
03390         else {
03391             Ci.init(S).save2(C0);

```

```

03392     }
03393     return C0;
03394 }
03395
03396 char isPtrans = P.isCyclic2F(S1.len,S2.len,'i');
03397
03398 if (aflag<0 || aflag>1) wblog(FL,
03399 "ERR %s() invalid aflag=%d !?",FCT,aflag);
03400 if (bflag<0 || bflag>1) wblog(FL,
03401 "ERR %s() invalid bflag=%d !?",FCT,bflag);
03402
03403 if (isPtrans) { aflag=!aflag; bflag=!bflag; }
03404
03405 if (ica.conj && ISCOMPLX_(TA)) {
03406     if (aflag)
03407         { aflag='C'; }
03408     else { aflag='N'; MA.Instantiate().Conj(); }
03409 } else { aflag=(aflag ? 'T':'N'); }
03410
03411 if (icb.conj && ISCOMPLX_(TB)) {
03412     if (bflag)
03413         { bflag='C'; }
03414     else { bflag='N'; MB.Instantiate().Conj(); }
03415 } else { bflag=(bflag ? 'T':'N'); }
03416
03417 if (!isPtrans) {
03418     Wb::MatProd(MA,MB,Ci, aflag, bflag);
03419     Ci.Reshape(UVEC(S1,S2)).Permute(P);
03420 }
03421 else {
03422     Wb::MatProd(MB,MA,Ci, bflag, aflag);
03423     Ci.Reshape(UVEC(S2,S1));
03424 }
03425
03426 if (gotC_) {
03427     if (!Ci.sameSize(C0)) wblog(F,L,
03428 "ERR %s() size mismatch\n%s cannot add %s / %s",
03429 FCT,SHORT_FL,SSTR(Ci),SSTR(C0));
03430     return C0.Plus(Ci, TC(afac), 0, cfac);
03431 }
03432 else {
03433     if (cfac && cfac!=TC(1)) wblog(FL,
03434 "WRN %s() got cfac=%s with empty input",FCT,NSTR(cfac));
03435     if (afac!=TA(1)) { Ci*=afac; }
03436     return Ci.save2(C0);
03437 }
03438 };
03439
03440 template<class T>
03441 wbararray<T>& wbararray<T>::trace(
03442 unsigned il, unsigned i2, wbararray<T> &C) const {
03443
03444     size_t i,j,k,d,q,s,r,l;
03445     WBINDEX S,I0,I;
03446
03447     if (&C==this) wblog(FL,
03448 "ERR %s() output space coincides with *this",FCT);
03449
03450     if (il==i2 || !il || !i2 || il>SIZE.len || i2>SIZE.len ||
03451         SIZE[i1-1]!=SIZE[i2-1]) {
03452         wblog(FL,"ERR %s() invalid trace indizes (%d,%d; %s)",
03453             FCT,il, i2, SSTR_(this)); return C;
03454     }
03455
03456     il--; i2--;
03457
03458     d=SIZE[i1];
03459
03460     S.init(SIZE.len-2);
03461     for (k=i0; k<SIZE.len; ++k) if (i!=i1 && i!=i2) S[k++]=SIZE[i];
03462     if (S.len==0) { S.init(1); S[0]=1; }
03463
03464     if (C.isEmpty()) C.init(S); else {
03465         if (S!=C.SIZE) wblog(FL,
03466 "ERR %s() size mismatch of I/O array\n%s: [%s] vs. [%s]",
03467             FCT,SHORT_FL,SSTR(C), STR(S)
03468         );
03469     }
03470
03471     I.init(S.len); s=S.prod(0); I0.init(SIZE.len);
03472     r=SIZE.len-1; l=S.len-1;
03473
03474     for (i=0; i<s; ++i) {
03475         for (q=k=0; k<SIZE.len; ++k) if (k!=i1 && k!=i2) I0[k]=I[q++];
03476         for (q=0; q<d; ++q) {
03477             I0[i1]=I0[i2]=q;
03478             for (j=I0[r],k=r-1; k<r; k--) j = j*SIZE[k] + I0[k];

```



```

03479
03480         C.data[i]+=data[j];
03481     }
03482
03483     k=0; I[0]++;
03484     while (I[k]>=S[k] && k<l) { I[k]=0; ++I[++k]; }
03485 }
03486
03487 return C;
03488 }
03489
03490 template<class T>
03491 wbarry<T>& wbarry<T>::trace(
03492     const ctrIdx &I1, const ctrIdx &I2, wbarry<T> &C) const {
03493
03494     size_t i,j,k,l, kc,kx, lc,lx,l0, Nc,Nx, *sx, *sc;
03495     const size_t *s0=SIZE.data;
03496
03497     unsigned r0=SIZE.len, *mx;
03498     const unsigned *i1=I1.data, *i2=I2.data;
03499
03500     WBINDEX Sx, Sc, I0, Ix, Ic; widx_t *i0, *ix, *ic;
03501     wvector<unsigned> Mx;
03502
03503     if (&C==this) wblog (FL,
03504         "ERR %s() output space coincides with *this !?",FCT);
03505     if (I1.len!=I2.len || 2*I1.len>r0) wblog (FL,
03506         "ERR %s() invalid index sets (%s <> %s)",FCT,STR(I1),STR(I2));
03507
03508     Mx.init(r0);      mx=Mx.data;
03509     Sx.init(I1.len);  sx=Sx.data;
03510
03511     for (i=0; i<I1.len; ++i) {
03512         j=i1[i]; if (j>=r0 || mx[j]++) { break; }
03513         j=i2[i]; if (j>=r0 || mx[j]++) { break; }
03514
03515         sx[i]=s0[i1[i]]; if (sx[i]!=s0[i2[i]]) { break; }
03516     }
03517     if (i<I1.len) wblog (FL,
03518         "ERR %s() invalid index set\nhaving %s with %s <> %s",
03519         FCT,SSTR_(this),STR(I1),STR(I2)
03520     );
03521
03522     if (!I1.len) {
03523         if (C.isEmpty()) C=(*this); else C+=(*this);
03524         return C;
03525     }
03526
03527     k=r0-2*I1.len; Sc.init(k?k:1); sc=Sc.data;
03528     if (!k) { sc[0]=1; }
03529     else { for (l=i=0; i<r0; ++i) { if (!mx[i]) sc[l++] = s0[i]; } }
03530
03531     Nx=Sx.prod(0); lx=Sx.len-1;
03532     Nc=Sc.prod(0); lc=Sc.len-1; l0=r0-1;
03533
03534     if (C.isEmpty()) C.init(Sc); else {
03535         if (Sc!=C.SIZE) wblog(FL,
03536             "ERR %s() size mismatch of I/O array\n%s: [%s] vs. [%s]",
03537             FCT,SHORT_FL,SSTR(C), STR(Sc)
03538         );
03539     }
03540
03541     I0.init(r0);      i0=I0.data;
03542     Ic.init(Sc.len);  ic=Ic.data;
03543     Ix.init(Sx.len);  ix=Ix.data;
03544
03545     for (kc=0; kc<Nc; ++kc) {
03546         for (l=i=0; i<r0; ++i) if (!mx[i]) i0[i]=ic[l++];
03547
03548         Ix.set(0);
03549         for (kx=0; kx<Nx; ++kx) {
03550             for (i=0; i<Ix.len; ++i) {
03551                 i0[i1[i]] = i0[i2[i]] = ix[i];
03552             }
03553             for (k=i0[l0], i=l0-1; i<l0; i--) { k = k*s0[i] + i0[i]; }
03554
03555             C.data[kc] += data[k];
03556
03557             i=0; ++ix[0];
03558             while (ix[i]>=sx[i] && i<lx) { ix[i]=0; ++ix[++i]; }
03559         }
03560
03561         i=0; ic[0]++;
03562         while (ic[i]>=sc[i] && i<lc) { ic[i]=0; ++ic[++i]; }
03563     }
03564
03565     return C;

```

```

03566 };
03567
03568 template<class T>
03569 wbararray<T> TestContract (
03570     const wbararray<T> &A, const WBINDEX i1,
03571     const wbararray<T> &B, const WBINDEX i2,
03572     const wbpperm &pfinal
03573 ){
03574     WBINDEX Si,Sj,Sk, I,J,K, Ia,Ib,Ic, Iq; WBPPerm Pa,Pb;
03575     size_t i,j,k, si,sj,sk, li,lj,lk, q, ra=A.SIZE.len, rb=B.SIZE.len;
03576     wbararray<T> C;
03577
03578     T dbl;
03579
03580     if (i1.isEmpty() || i2.len!=i1.len) wblog (FL,
03581         "ERR %s() invalid index set {[%s], [%s]}",
03582         FCT,i1.toStr().data, i2.toStr().data);
03583     if (i1.anyGE(ra) || i2.anyGE(rb)) wblog (FL,
03584         "ERR %s() index out of range\n[%s; %d], [%s; %d]",
03585         FCT,i1.toStr().data,ra,i2.toStr().data,rb);
03586
03587     return A;
03588
03589     for (k=0; k<i1.len; ++k) {
03590         if (A.SIZE[i1[k]]!=B.SIZE[i2[k]]) wblog (FL,
03591             "ERR %s() incompatible objects {[%s], [%s]}\n[%s] <> [%s] at %d",
03592             FCT,i1.toStr().data, i2.toStr().data, SSTR(A), SSTR(B), k);
03593     }
03594
03595     Si=A.SIZE.Skip(i1); si=Si.len>0 ? Si.prod(0) : 1; li=Si.len-1;
03596     Sj=A.SIZE [i1];    sj=Sj.len>0 ? Sj.prod(0) : 1; lj=Sj.len-1;
03597     Sk=B.SIZE.Skip(i2); sk=Sk.len>0 ? Sk.prod(0) : 1; lk=Sk.len-1;
03598
03599     Pa = Index(0,ra-1).Move2end (i1);
03600     Pb = Index(0,rb-1).Move2front(i2);
03601
03602     Ia.init(ra); Ib.init(rb); C.init(Si.append(Sk));
03603
03604     if (C.numel()==0) C.init(1);
03605
03606     I.init(Si.len);
03607     for (i=0; i<si; ++i) {
03608
03609         K.init(Sk.len);
03610         for (k=0; k<sk; ++k) {
03611
03612             J.init(Sj.len);
03613             for (dbl=0, j=0; j<sj; ++j) {
03614                 Iq=I.append(J); for (q=0; q<Iq.len; ++q) Ia[Pa[q]]=Iq[q];
03615                 Iq=J.append(K); for (q=0; q<Iq.len; ++q) Ib[Pb[q]]=Iq[q];
03616
03617                 if (j==sj-1) Ic=I.append(K);
03618
03619                 dbl += A(Ia) * B(Ib);
03620
03621                 if (J.len>0) {
03622                     q=0; J[0]++;
03623                     while (J[q]>=Sj[q] && q<lj) { J[q]=0; ++J[++q]; }
03624                 }
03625             }
03626             C(Ic) = dbl;
03627
03628             if (K.len>0) {
03629                 q=0; K[0]++;
03630                 while (K[q]>=Sk[q] && q<lk) { K[q]=0; ++K[++q]; }
03631             }
03632         }
03633         if (I.len>0) {
03634             q=0; I[0]++;
03635             while (I[q]>=Si[q] && q<li) { I[q]=0; ++I[++q]; }
03636         }
03637     }
03638
03639     if (pfinal) C.Permute(pfinal);
03640
03641     return C;
03642 };
03643
03644 int tensor_ref_::check(const char *F, int L) const {
03645     int q=0;
03646     if (Ar) {
03647         if (!Az) { q=1; }
03648         else { char s[64]; q=-1;
03649             snprintf(s,64,"%s() multiple refs set (%p/%p)",FCT,Ar,Az);
03650             if (F) wblog(F,L,"ERR %s",s);
03651             else wblog(F_L,"WRN %s",s);
03652         }
03653     }

```

```

03653     }
03654     else if (Az) { q=2; }
03655
03656     if (q<=0 && F) wblog(FL,"ERR %s() invalid array ref (q=%d)",FCT,q);
03657     return q;
03658 };
03659
03660 int tensor_ref_::rank(unsigned r) const {
03661     int q=-1;
03662     if (Ar) {
03663         if (Az) wblog(FL,"ERR %s() multiple refs set (%p/%p)",FCT,Ar,Az);
03664         q=(Ar->SIZE.len==r);
03665     }
03666     else if (Az) { q=(Az->SIZE.len==r); }
03667     return q;
03668 };
03669
03670 bool tensor_ref_::sameSize(const wbvector<size_t> &S) const {
03671     bool q=0;
03672     if (Ar) {
03673         if (Az) wblog(FL,"ERR %s() multiple refs set (%p/%p)",FCT,Ar,Az);
03674         q=(Ar->SIZE.isEqual(S));
03675     }
03676     else if (Az) { q=(Az->SIZE.isEqual(S)); }
03677     return q;
03678 };
03679
03680 wstring tensor_ref_::sizeStr() const {
03681     wstring s;
03682     if (Ar) {
03683         if (Az) wblog(FL,"ERR %s() multiple refs set (%p/%p)",FCT,Ar,Az);
03684         s=SSTR_(Ar);
03685     }
03686     else if (Az) { s=SSTR_(Az); }
03687     else { s="[]"; }
03688
03689     return s;
03690 };
03691
03692
03693 const wbvector<size_t>* tensor_ref_::getSIZE() const {
03694
03695     const wbvector<size_t> *s=NULL;
03696     if (Ar) {
03697         if (Az) wblog(FL,"ERR %s() multiple refs set (%p/%p)",FCT,Ar,Az);
03698         s=&(Ar->SIZE);
03699     }
03700     else if (Az) { s=&(Az->SIZE); }
03701
03702     return s;
03703 };
03704
03705 template<>
03706 void tensor_ref_::contract(
03707     const char *F, int L, const ctrIdx &ia,
03708     const tensor_ref_ &b, const ctrIdx &ib, wbarry<double> &C
03709 ) const {
03710
03711     if (Az || b.Az) wblog(FL,
03712         "ERR %s() got complex array refs (%p/%p)",FCT,Az,b.Az);
03713     if (!Ar || !b.Ar) wblog(FL,
03714         "ERR %s() got NULL array refs (%p/%p)",FCT,Ar,b.Ar);
03715     Ar->contract(F_L,ia,*b.Ar,ib,C,wbperm(),1.,0.);
03716 };
03717
03718 template<>
03719 void tensor_ref_::contract(
03720     const char *F, int L, const ctrIdx &ia,
03721     const tensor_ref_ &b, const ctrIdx &ib, wbarry<wbcomplex> &C
03722 ) const {
03723
03724     wbperm P; wbcomplex zero(0);
03725     check(F_L); b.check(F_L);
03726
03727     if (Ar) {
03728         if (b.Ar)
03729             { Ar->contract(F_L,ia,*b.Ar,ib,C, P,1.,zero); }
03730         else { Ar->contract(F_L,ia,*b.Az,ib,C, P,1.,zero); }
03731     }
03732     else {
03733         if (b.Ar)
03734             { Az->contract(F_L,ia,*b.Ar,ib,C, P,1.,zero); }
03735         else { Az->contract(F_L,ia,*b.Az,ib,C, P,1.,zero); }
03736     }
03737 };
03738
03739 char tensorRef_::check(const char* F, int L) const {

```

```

03740     char q=0, r;
03741     if (!(S && !R) || !it) { q|=1; if (F) wblog(F_L,
03742         "ERR %s() got uninitialized data (%d/%d)",FCT,S.len,it.len); }
03743     if (S.len && S.len!=it.len) { q|=2; if (F) wblog(F_L,
03744         "ERR %s() length mismatch (S: %d/%d)",FCT,S.len,it.len); }
03745     if (!(r=R.rank(it.len))) { q|=2; if (F) wblog(F_L,
03746         "ERR %s() length mismatch (A: %d/%d)",FCT,r,it.len); }
03747     if (ID.len && ID.len!=it.len) { q|=4; if (F) wblog(F_L,
03748         "ERR %s() length mismatch (ID: %d/%d)",FCT,ID.len,it.len); }
03749     if (S.len && r>0 && !R.sameSize(S)) { q|=8; if (F) wblog(F_L,
03750         "ERR %s() SIZE mismatch %s / %s",FCT,SSTR(S),SSTR(R)); }
03751     return q;
03752 };
03753
03754 char tensorRef_::Check(const char* F, int L, unsigned &k) {
03755     if (!id) { id=(++k); }
03756     if (!R) { if (F) wblog(F_L,"ERR %s() got NULL array reference",FCT); }
03757     else if (!S.len) { S.init2ref(*R.getSIZE()); }
03758
03759     return check(F_L);
03760 };
03761
03762 template<class T>
03763 int tensorRef_::contract(const char *F, int L,
03764     const tensorRef_ &b, tensorRef_ &c,
03765     double *flops,
03766     wbararray<T> *C
03767 ) const {
03768
03769     int nc=0;
03770
03771     if (this==&c || &b==&c) { tensorRef_ x;
03772         nc=contract(F,L,b,x,flops,C);
03773         x.save2(c); return nc;
03774     }
03775
03776     unsigned i,k,l=0;
03777
03778     ctrIdx ia, ib;
03779     wvector<char> ma, mb;
03780
03781     if ((i=(check() | b.check()))) wblog(F_L,
03782         "ERR %s() %s input\n%s\n%s", FCT,
03783         i&1 ? "got empty":"inconsistent", STR_(this),STR(b));
03784
03785     nc=this->it.getCtr(
03786         flops? NULL : (F? F : __FILE__),
03787         flops? 1 : (L? L : __LINE__),
03788         b.it, ia,ib,
03789         &c.it, &ma, &mb
03790     );
03791
03792     if (nc<1 && !flops) wblog(F_L,
03793         "ERR %s() invalid contraction (empty match)",FCT);
03794
03795     if (ma.len!=S.len || mb.len!=b.S.len) wblog(F_L,"ERR %s() "
03796         "length mismatch %d/%d ; %d/%d",FCT,ma.len,S.len,mb.len,b.S.len);
03797     for (i=0; i<ia.len; ++i) { if (S[ia[i]]!=b.S[ib[i]]) { wblog(F_L,
03798         "ERR %s() size mismatch\n%-40s @ %s\n%-40s @ %s",
03799         FCT, STR_(this), STR(ia), STR(b), STR(ib));
03800     }}
03801
03802     l=c.it.len;
03803     c.S.init(l); c.ID.init(l);
03804
03805     k=(id<8);
03806     for (l=i=0; i<S.len; ++i) { if (!ma[i]) {
03807         if (c.it[l]!= it[i]) wblog(F_L,"ERR %s() %d/%d: %s / %s",
03808             FCT,i+1, S.len,STR(c.it[l]), STR(it[i]));
03809         c.ID[l]=(ID.len ? ID[i] : k+i);
03810         c.S[l]= S[i]; ++l;
03811     }}
03812
03813     k=(b.id<8);
03814     for (i=0; i<b.S.len; ++i) { if (!mb[i]) {
03815         if (c.it[l]!=b.it[i]) wblog(F_L,"ERR %s() %d/%d: %s / %s",
03816             FCT,i+1,b.S.len,STR(c.it[l]),STR(b.it[i]));
03817         c.ID[l]=k+i;
03818         c.S[l]=b.S[i]; ++l;
03819     }}
03820     if (l!=c.S.len) wblog(F_L,"ERR %s() %d/%d",FCT,l,c.S.len);
03821
03822     if (flops) { double f=R.xflops()*b.R.xflops();
03823         if (f==2) { f=4; }
03824
03825         for (i=0; i<ia.len; ++i) { f *= S[ia[i]]; }
03826         for (i=0; i<c.S.len; ++i) { f *= c.S[i]; }

```

```

03827     *flops = f;
03828 }
03829
03830 if (C) {
03831     ia.conj= conj;
03832     ib.conj=b.conj;
03833
03834     R.contract(F_L,ia,b.R,ib,*C);
03835     c.R.init(*C);
03836 }
03837
03838 return nc;
03839 };
03840
03841 double tensorRefs::getOptimalCtrOrder(const char *F, int L,
03842     wperm &P,
03843     wperm *pfin,
03844     char useP,
03845     char vflag
03846 ) {
03847
03848     double fmin=-1, flops=0, f;
03849
03850     if (len<2 || len>8) wblog(FL,
03851         "ERR %s() got %d tensors (max. 8 supported)",FCT,len);
03852
03853     Check(F_L);
03854
03855     unsigned ip=0,i0=0, j; int nc; wperm_t *p;
03856     wbMatrix<wperm_t> PP;
03857     tensorRef_ R, Rmin;
03858
03859     if (vflag) {
03860         if (vflag=='v') vflag=1; else
03861         if (vflag=='V') vflag=3; else
03862         if (vflag>8) wblog(FL,"WRN %s() got vflag=%s",FCT,cSTR(vflag));
03863     }
03864     if (useP) {
03865         if (P) { if (P.len!=len) wblog(FL,
03866             "ERR %s() size mismatch (%d/%d)",FCT,P.len,len);
03867         } else { useP=0; }
03868     }
03869     if (useP)
03870         { PP.init(1,len,P.data); }
03871     else { Wb::getPerms(PP,len); }
03872     p=PP.data;
03873
03874     for (; ip<PP.dim1; ++ip, p+=PP.dim2) {
03875         if (p[0]>p[1]) { continue; }
03876
03877         nc=data[p[0]].contract(FL,data[p[1]], R,f);
03878         if (!nc) { continue; }
03879         flops=f;
03880
03881         for (j=2; j<len; ++j) {
03882             nc=R.contract(FL,data[p[j]], R,f);
03883             if (!nc) { break; }
03884             flops+=f;
03885         }
03886
03887         if (!nc) { continue; }
03888
03889         if (!ip || (fmin>flops)) { fmin=flops; R.save2(Rmin); i0=ip; }
03890
03891         if (vflag>1) { PRINTF(
03892             "%2d [%s] %-64s @%8ld flops\n",
03893             ip, STR(wbperm().init(PP.dim2,p,'r')), STR(R), size_t(flops));
03894         }
03895     }
03896     if (vflag) { print();
03897         wblog(FL,"--> %s() picking ip=%d/%ld\n",FCT,i0,PP.dim1);
03898     }
03899
03900     P.init(PP.dim2,PP.rec(i0));
03901
03902     if (pfin) {
03903         Rmin.ID.Sort(*pfin);
03904     }
03905
03906     return fmin;
03907 };
03908
03909 void tensorRefs::print(const char *F, int L) const {
03910
03911     if (F || !len) wblog(F_L,
03912         "--- tensorRefs listing%s",len? "":" (empty)");
03913

```

```

03914     if (len) { unsigned i=0; char istr[16];
03915         PRINTF("\n");
03916         for (; i<len; ++i) {
03917             snprintf(istr,16," %2d/%1d :",i+1,len);
03918             data[i].print(istr, F?"v':0);
03919         }
03920         PRINTF("\n");
03921     }
03922 };
03923
03924 template<class T> inline
03925 bool wbarrray<T>::requiresDataPerm(const wperm &P, char lflag) const {
03926
03927     if (!P.len) { return 0; }
03928
03929     const wperm_t *p=P.data; const size_t *s=SIZE.data;
03930     unsigned i,j, j_=-1;
03931
03932     if (P.len!=SIZE.len) {
03933         if (lflag<0 || lflag>3) wblog(FL,
03934             "WRN %s() got lflag=%s",FCT,cSTR(lflag));
03935         if (!(lflag & (P.len<SIZE.len ? 1 : 2))) { wblog(FL,"ERR %s() "
03936             "invalid permutation (len=%d/%d; %d)",FCT,P.len,SIZE.len,lflag);
03937         }
03938     }
03939     if (numel()<=1) { return 0; }
03940
03941     for (i=0; i<P.len; ++i) { if ((j=p[i])<SIZE.len && s[j]>1) {
03942         if (j_<j || int(j_)<0)
03943             { j_=j; }
03944         else { return 1; }
03945     }}
03946
03947     return 0;
03948 };
03949
03950 template<class T> inline
03951 void wbarrray<T>::toMatrixRef(
03952     wbarrray<T> &A,
03953     const wbvector<unsigned> &I,
03954     int pos,
03955     wperm &P
03956 ) const {
03957     size_t s1,s2;
03958     groupIndizes_P(I,pos,P,&s1,&s2);
03959
03960     if (!requiresDataPerm(P)) { A.init2ref(*this); }
03961     else { permute(A,P); }
03962
03963     A.Reshape(s1,s2);
03964 };
03965
03966 template<class T> inline
03967 bool wbarrray<T>::toMatrixRef(const char *F, int L,
03968     wbarrray<T> &A,
03969     const wbvector<unsigned> &I,
03970     int pos,
03971     char &tflag
03972 ) const {
03973     size_t i,s1,s2; wperm P;
03974
03975     int k[2]={pos, pos==1 ? 2 : 1};
03976
03977     for (i=0; i<2; ++i) {
03978         groupIndizes_P(I,k[i],P,&s1,&s2);
03979
03980         if (!requiresDataPerm(P)) {
03981             tflag = (pos==k[i] ? 0 : 1);
03982             A.init2ref(*this);
03983             A.Reshape(s1,s2);
03984             return 0;
03985         }
03986     }
03987
03988     if (&A==this) wblog(F_L,"ERR %s() Wb::overlapping output space",FCT);
03989
03990     toMatrixRef(A,I,pos,P); tflag=0;
03991
03992     return 1;
03993 };
03994
03995 template<class T>
03996 void wbarrray<T>::toMatrixRef(
03997     wbarrray<T> &A,
04000     unsigned K,

```

```

04001     const wbpem &P0
04002 ) const {
04003
04004     size_t i,s1,s2; wbpem P;
04005
04006     if (K>SIZE.len) wblog(FL,
04007         "ERR %s() index out of bounds (%d/%d)",FCT,K,SIZE.len);
04008
04009     if (!P0.len) { P.Index(SIZE.len); }
04010     else {
04011         if (P0.len!=SIZE.len) wblog(FL,
04012             "ERR %s() rank mismatch (len=%d/%d)",FCT,P0.len,SIZE.len);
04013         P=P0;
04014     }
04015
04016     for (s1=1,i=0; i<K; ++i) { s1*=SIZE[P[i]]; }
04017     for (s2=1; i<P.len; ++i) { s2*=SIZE[P[i]]; }
04018
04019     if (!requiresDataPerm(P))
04020         { A.init2ref(*this); }
04021     else { permute(A,P); }
04022
04023     A.Reshape(s1,s2);
04024 };
04025
04026 template<class T>
04027 void wbarry<T>::toMatrixRefM(
04028     wbarry<T> &A,
04029     unsigned K,
04030     const wbpem &P0,
04031     unsigned R,
04032     wbpem pfin
04033 ) const {
04034
04035     size_t i,m=1,s1,s2; wbpem P;
04036     char use_ref=0;
04037
04038     if (K>R) wblog(FL,
04039         "ERR %s() index out of bounds (K=%d/%d/%d)",FCT,K,R,SIZE.len);
04040     if (R<2 || R>SIZE.len) wblog(FL,
04041         "ERR %s() invalid rank (R=%d/%d; K=%d)",FCT,R,SIZE.len,K);
04042     if (SIZE.len>R+1) wblog(FL,"WRN %s() "
04043         "got multi-dimensional OM (%s @ %d; K=%d)",FCT,SSTR_(this),R,K);
04044
04045     if (!P0.len) { P.Index(R); }
04046     else {
04047         if (P0.len!=R) wblog(FL,
04048             "ERR %s() rank mismatch (len=%d/%d/%d)",FCT,P0.len,R,SIZE.len);
04049         P=P0;
04050     }
04051
04052     for (s1=1,i=0; i<K; ++i) { s1*=SIZE[P[i]]; }
04053     for (s2=1; i<P.len; ++i) { s2*=SIZE[P[i]]; }
04054     for ( ; i<SIZE.len; ++i) { m *=SIZE[ i ]; }
04055
04056     use_ref=( requiresDataPerm(P) ? 0:1 );
04057     if (pfin.len) {
04058         if (pfin.len!=3) wblog(FL,"ERR %s() invalid pfin='%s'",FCT,STR(pfin));
04059         if (pfin[0]!=0 || m!=1) { use_ref=0; }
04060     }
04061
04062     if (use_ref)
04063         { A.init2ref(*this); }
04064     else { permute(A,P); }
04065
04066     A.Reshape(s1,s2,m);
04067
04068     if (pfin.len) {
04069         A.Permute(pfin);
04070     }
04071 };
04072
04073 template<class T> inline
04074 wbarry<T>& wbarry<T>::save2(wbMatrix<T> &M) {
04075
04076     if (M.data) { M.init(); }
04077
04078     if (SIZE.len!=2) {
04079         if (SIZE.len) wblog(FL,"ERR invalid rank-%d",SIZE.len);
04080         return M;
04081     }
04082
04083     check_consistency(FL);
04084     if (sptr) { M.data=sptr->save_dref(FL); }
04085
04086     M.dim1=SIZE[1]; M.dim2=SIZE[0]; SIZE.init();
04087

```

```

04088     return M;
04089 };
04090
04091 #ifdef QS_USING_MPFR
04092
04093 template<> inline
04094 mxArray* wbararray<Wb::quad>::toMx() const {
04095
04096     const char *fields[]={"S","data"};
04097     mxArray *a=mxCreateStructMatrix(1,1,2,fields);
04098
04099     wvector<Wb::quad> X(numel(),data,'r');
04100
04101     mxSetFieldByNumber(a,0,0, SIZE.toMx());
04102     mxSetFieldByNumber(a,0,1, X.toMx());
04103
04104     return a;
04105 };
04106
04107 template<> inline
04108 wbararray<Wb::quad>& wbararray<Wb::quad>::init(
04109     const char *F, int L, const mxArray *S,
04110     char ref QS_UNUSED_VAR,
04111     char vec QS_UNUSED_VAR
04112 ){
04113     if (!S || !mxIsStruct(S)) wblog(FL,"ERR %s() "
04114         "got invalid input '%s' / mpfr",FCT,S? mxGetClassName(S):"null");
04115
04116     wvector<size_t> sz(FL,mxGetFieldByNumber(S,0,0));
04117     wvector<Wb::quad> X(mxGetFieldByNumber(S,0,1));
04118
04119     if (X.len!=sz.prod(0)) wblog(FL,
04120         "ERR %s() size mismatch %d/%d",FCT,(int)X.len,(int)sz.prod(0));
04121     if (X.len) init(sz,X.data);
04122
04123     return *this;
04124 };
04125
04126 #endif
04127
04128 template<class T>
04129 void cell2mat(
04130     const wbMatrix< wbarrRef<T> > &C,
04131     wbararray<T> &M,
04132     WBINDEX &D1, WBINDEX &D2
04133 ){
04134
04135     size_t i,j,m,r,s,d1=0,d2=0,i0,j0;
04136     T *d0, *dd;
04137     double dc,fac;
04138
04139     D1.init(C.dim1);
04140     D2.init(C.dim2);
04141
04142     for (i=0; i<C.dim1; ++i)
04143     for (j=0; j<C.dim2; ++j) { if (C(i,j).D==NULL) continue;
04144
04145         WBINDEX &S = C(i,j).D->SIZE;
04146
04147         if (S.len==2) { d1=S[0]; d2=S[1]; } else
04148         if (S.len==1) { d1=d2=S[0]; }
04149         else wblog(FL,
04150             "ERR %s() requires 1/2-D arrays (got rank-%d).",FCT,S.len);
04151
04152         if (C(i,j).tflag) SWAP(d1,d2);
04153
04154         if (D2[j]==0) D2[j]=d2; else
04155         if (d2!=D2[j]) wblog(FL,
04156             "ERR %s() dimension mismatch (%d,%d: %d/%d).",FCT,i,j,d2,D2[j]);
04157
04158         if (D1[i]==0) D1[i]=d1; else
04159         if (d1!=D1[i]) wblog(FL,
04160             "ERR %s() dimension mismatch (%d,%d: %d/%d).",FCT,i,j,d1,D1[i]);
04161     }
04162
04163     M.init(D1.sum(),D2.sum());
04164
04165     for (j0=j=0; j<C.dim2; ++j, j0+=d2) { d2=D2[j];
04166     for (i0=i=0; i<C.dim1; ++i, i0+=d1) { d1=D1[i];
04167
04168         if (C(i,j).D==NULL) continue;
04169
04170         d0=(C(i,j).D->data); dd=M.ref(i0,j0);
04171         fac=C(i,j).fac;
04172         dc =C(i,j).dc;
04173
04174         if (C(i,j).D->SIZE.len==1) {

```



```

04175         if (C(i,j).tflag) wblog(FL,
04176             "WRN %s() tflag irrelevant for diag-matrix (%d,%d)",
04177             FCT,i+1,j+1);
04178         if (d1!=d2) wblog(FL,"ERR d1 != d2 !? (%,%d)",d1,d2);
04179
04180         for (r=0; r<d1; ++r, dd+=(M.dim1+1))
04181             dd[r]+=(dc+fac*d0[r]);
04182
04183         continue;
04184     }
04185
04186     if (C(i,j).tflag==0) {
04187
04188         if (fac==1.) {
04189             for (s=0; s<d2; ++s, dd+=M.dim1, d0+=d1)
04190                 for (r=0; r<d1; ++r) dd[r]+= d0[r];
04191         }
04192         else if (fac==-1.) {
04193             for (s=0; s<d2; ++s, dd+=M.dim1, d0+=d1)
04194                 for (r=0; r<d1; ++r) dd[r]-= d0[r];
04195         }
04196         else if (fac!=0.) {
04197             for (s=0; s<d2; ++s, dd+=M.dim1, d0+=d1)
04198                 for (r=0; r<d1; ++r) dd[r]+= fac*d0[r];
04199         }
04200     }
04201     else {
04202
04203         if (fac==1.) {
04204             for (s=0; s<d2; ++s, dd+=M.dim1)
04205                 for (r=0; r<d1; ++r) dd[r]+= d0[s+r*d2];
04206         }
04207         else if (fac==-1.) {
04208             for (s=0; s<d2; ++s, dd+=M.dim1)
04209                 for (r=0; r<d1; ++r) dd[r]-= d0[s+r*d2];
04210         }
04211         else if (fac!=0.) {
04212             for (s=0; s<d2; ++s, dd+=M.dim1)
04213                 for (r=0; r<d1; ++r) dd[r]+= fac*d0[s+r*d2];
04214         }
04215     }
04216     if (dc!=0.) {
04217         if (d1!=d2) wblog(FL,
04218             "WRN %s() constant diagonal shift \n"
04219             "requires square block (%d,%d: %d,%d)",FCT,i+1,j+1,d1,d2);
04220
04221         dd=M.ref(i0,j0); m=MIN(d1,d2);
04222
04223         for (r=0; r<m; ++r, dd+=(M.dim1+1))
04224             dd[r]+=dc;
04225     }
04226 }}
04227 };
04228
04229 template<class T>
04230 void wbarry<T>::getReal(wbarry<double> &R) const {
04231     R.initT(*this);
04232 };
04233
04234 template<>
04235 void wbarry<wbcomplex>::getReal(wbarry<double> &R) const {
04236     size_t i=0, n=numel();
04237     R.init(SIZE); for (; i<n; ++i) { R.data[i]=(double)data[i].r; }
04238 };
04239
04240 template<class T>
04241 void wbarry<T>::getImag(wbarry<double> &I QS_UNUSED_VAR) const {
04242     wblog(FL,"ERR wbarry::getImag not defined for type %s",TSTR(T));
04243 };
04244
04245 template<>
04246 void wbarry<double>::getImag(wbarry<double> &I) const {
04247     I.init(SIZE);
04248 };
04249
04250 template<>
04251 void wbarry<wbcomplex>::getImag(wbarry<double> &I) const {
04252     size_t i=0, n=numel();
04253     I.init(SIZE); for (; i<n; ++i) { I.data[i]=data[i].i; }
04254 };
04255
04256 template<class T>
04257 void wbarry<T>::Conj() { return; }
04258
04259 template<>
04260 void wbarry<wbcomplex>::Conj() {
04261     size_t i=0, n=numel();

```

```

04262     for (; i<n; ++i) { data[i].i = -data[i].i; }
04263 };
04264
04265 template<class T>
04266 void wbarry<T>::set(
04267     const wbarry<double> &R QS_UNUSED_VAR,
04268     const wbarry<double> &I QS_UNUSED_VAR
04269 ) { wblog(FL, "ERR wbarry::set(R,I) not defined for type %s", TSTR(T)); };
04270
04271 template<>
04272 void wbarry<wbcomplex>::set(
04273     const wbarry<double> &R_, const wbarry<double> &I_) {
04274
04275     size_t i=0, n;
04276     const double *R=R_.data, *I=I_.data;
04277
04278     if (R) { init(R_.SIZE); n=numel();
04279         if (I) {
04280             if (!I_.sameSize(R_)) wblog(FL, "ERR %s() "
04281                 "dimension mismatch %s / %s", FCT, SSTR(R_), SSTR(I_));
04282             for (; i<n; ++i) { data[i].set(R[i], I[i]); }
04283         }
04284         else { for (; i<n; ++i) { data[i].set(R[i], 0.); } }
04285     }
04286     else if (I) {
04287         init(I_.SIZE); n=numel();
04288         for (; i<n; ++i) { data[i].set(0., I[i]); }
04289     }
04290     else { init(); }
04291 };
04292
04293 template <typename T>
04294 void wbperm_helper<T>::permute(const T *src, T *dest, int np
04295 ) const {
04296
04297     if (rk<2) {
04298         if (rk==1) { std::copy_n (src, sz[0], dest); }
04299         return;
04300     }
04301
04302     widx_t m=2*m_blk, m2=m*m;
04303
04304     if (!src || !dest) wblog(FL,
04305         "ERR %s() got null input (s=%p, d=%p; rk=%d)", FCT, src, dest, rk);
04306
04307     if (numel<8192) {
04308         widx_t idest=-1;
04309         wbIndex I(rk, sz);
04310         while (++I) { dest[++idest]=src[I.serial(stride)]; }
04311         return;
04312     }
04313
04314     if (l1 && (l1>1 || stride[0]!=1)) wblog(FL,
04315         "ERR %s() unexpected l1=%d with stride %d (l2=%d, rk=%d)",
04316         FCT, l1, stride[0], l2, rk);
04317
04318     if (np<1) { np=1; }
04319
04320     if (l1) {
04321         const unsigned k=2;
04322
04323         widx_t idest=-1, isrc=0, len=sz[k-1], step=stride[k-1];
04324         wbIndex I(rk-k, sz+k);
04325
04326         if (rk<3) wblog(FL,
04327             "ERR %s() got rk=%d with l1=%d, l2=%d !?", FCT, rk, l1, l2);
04328         if (sz[0]<2) wblog(FL, "ERR %s() unexpected sz[0]=%d", FCT, sz[0]);
04329
04330         if (np>1) { np=std::min(4,np); if ((int)len<4*np) { np=1; } }
04331
04332         while (++I) { ++idest;
04333             isrc = I.serial(stride+k);
04334             #pragma omp parallel for num_threads(np)
04335             for (widx_t i=0; i<len; ++i) {
04336                 std::copy_n(
04337                     src+isrc+i*step, sz[0], dest+(i+idest*len)*sz[0]);
04338             }
04339         }
04340         return;
04341     }
04342
04343     unsigned i, l=0;
04344     int r_ = rk-(l1+2); if (r_<1) { r_ = 1; }
04345
04346     widx_t isrc, idest;
04347     widx_t sz_[3*r_], *stride_s = sz_+r_, *stride_d = sz_+2*r_;
04348

```

```

04349     widx_t N_=0, M_;
04350     widx_t M=sz[l2], N=sz[l1];
04351
04352     for (idest=1, i=0; i<=l1; ++i) { idest*=sz[i]; }
04353     for (; i<rk; ++i) {
04354         if (i!=l2) { sz[l] = sz[i];
04355             stride_s[l] = stride[i];
04356             stride_d[l] = idest;
04357             ++l;
04358         }
04359         else {
04360             N_=idest;
04361         }
04362         idest*=sz[i];
04363     }
04364
04365     M_=stride[l1];
04366     if (!N_) wblog(FL,"ERR %s() failed to set N_",FCT);
04367
04368     if (np>1) {
04369         if (M*N > (1<14) && M>=2*m) {
04370             int np_=M/m; if (np>np_) { np=np_; }
04371         }
04372     }
04373
04374     wbararray<T> Blk(m2,np);
04375     wbIndex I_(l,sz_);
04376
04377     while (++I_) {
04378         isrc = I_.serial(stride_s);
04379         idest = I_.serial(stride_d);
04380
04381         #pragma omp parallel for num_threads(np)
04382         for (widx_t I=0; I<M; I+=m) {
04383             const T* s_ = T* d_, *blk = Blk.data;
04384             #ifdef QS_USING_OMP
04385                 blk += omp_get_thread_num()*m2;
04386             #endif
04387
04388             widx_t i,j,J,jM_,n_,m_ = std::min(m,M-I);
04389             for (J=0; J<N; J+=m) { n_ = std::min(m,N-J);
04390                 s_ = src + isrc + J*M_ + I;
04391                 for (j=0; j<n_; ++j) { jM_=j*M_;
04392                     for (i=0; i<m_; ++i) { blk[j*m+i] = s_[jM_+i]; } }
04393
04394                 d_ = dest + idest + I*N_ + J;
04395                 for (i=0; i<m_; ++i) {
04396                     for (j=0; j<n_; ++j) { d_[i*N_+j] = blk[j*m+i]; } }
04397             } }
04398     }
04399 };
04400
04401 template <typename T>
04402 mxArray* wbpem_helper<T>::toMx() const {
04403
04404     const char *fields[] = { "rk","numel","sz","stride","blk" };
04405     mxArray *S=mxCreateStructMatrix(1,1,5,fields);
04406     wvector<double> x(MAX(3U,rk));
04407
04408     x.len=2; x[0]=rk; x[1]=rk_;
04409     mxSetFieldByNumber(S,0,0,x.toMx());
04410
04411     x.len=1; x[0]=numel;
04412     mxSetFieldByNumber(S,0,1,x.toMx());
04413
04414     if (rk) {
04415         x.len=rk;
04416         Wb::cpyRange(x.data,sz, rk); mxSetFieldByNumber(S,0,2,x.toMx());
04417         Wb::cpyRange(x.data,stride,rk); mxSetFieldByNumber(S,0,3,x.toMx());
04418     }
04419
04420     x.len=3; x[0]=l1; x[1]=l2; x[2]=m_blk;
04421     mxSetFieldByNumber(S,0,4,x.toMx());
04422
04423     return S;
04424 };
04425
04426 #endif
04427

```

5.24 wbararray.hh

```
00001 /* -----
```

```

00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : wbarrray (array class, col-major)
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_ARRAY_COL_MAJOR_HH__
00021 #define __WB_ARRAY_COL_MAJOR_HH__
00022
00023 // #define CHECK_ELEMENT_RANGE
00024
00025 namespace Wb {
00026     template<class T>
00027     wbsttring sizeStrM(
00028         unsigned r, const T* sd, unsigned m, const T* sm,
00029         const char *sepl="x", const char *sepM="_");
00030
00031     template<class T> inline
00032     void householder(const T *u, T *x, size_t n, const T& eps);
00033 };
00034
00035 template <class T>
00036 class wbarrray {
00037 public:
00038     wbarrray() : mtype(Wb::MEM_DEF), sptr(NULL), data(NULL) { };
00039
00040     wbarrray(size_t d1) : wbarrray() { init(d1); };
00041     wbarrray(size_t d1, size_t d2) : wbarrray() { init(d1,d2); };
00042     wbarrray(size_t d1, size_t d2, size_t d3) : wbarrray() { init(d1,d2,d3); };
00043     wbarrray(size_t d1, size_t d2, size_t d3, size_t d4)
00044         : wbarrray() { init(d1,d2,d3,d4); };
00045
00046     wbarrray(char *sidx) : wbarrray() { init(Str2Idx(sidx)); };
00047     wbarrray(const wvector<unsigned> &S) : wbarrray() { init(S); };
00048
00049     wbarrray(const wvector<size_t> &S, T *d0, char ref=0) : wbarrray() {
00050         if (ref)
00051             { INIT2REF(d0,&S); }
00052         else { NEW(S,d0); }
00053     };
00054
00055     wbarrray(const wbarrray &a, char ref=0)
00056         : wbarrray() { init(a,ref); };
00057
00058     template<class T2>
00059     wbarrray(const wbarrray<T2> &a) : wbarrray() {
00060         NEW(a.SIZE); init_T(a.data);
00061     };
00062
00063     wbarrray(const mxArray* a, char ref=0, char vec=0)
00064         : wbarrray() { init(a,ref,vec); };
00065
00066     wbarrray(const char *F, int L, const mxArray* a,
00067         char ref=0,
00068         char vec=0)
00069         : wbarrray() { init(F,L,a,ref,vec); };
00070
00071     wbarrray(const wMatrix<T> &a, char ref=0)
00072         : wbarrray() { init(a,ref); };
00073
00074     virtual ~wbarrray() {
00075         DELETE_DATA();
00076     };
00077
00078     wbarrray& initX() {
00079         DELETE_DATA('!'); SIZE.init();
00080         return *this;
00081     };
00082
00083     wbarrray& init() {
00084         DELETE_DATA();
00085         SIZE.init(); return *this;
00086     };
00087
00088 };

```

```

00089
00090 wbararray& init(size_t d1, const T* d=NULL) {
00091     if (d1) { NEW(wbvector<size_t>(1,d1,'r'),d); } else { init(); }
00092     return *this;
00093 };
00094
00095 wbararray& init(size_t d1, size_t d2, const T* d=NULL) {
00096     size_t s[2]= {d1,d2}; NEW(wbvector<size_t>(2,s,'r'),d);
00097     return *this;
00098 };
00099
00100 wbararray& init(size_t d1, size_t d2, size_t d3, const T* d=NULL) {
00101     size_t s[3]= {d1,d2,d3}; NEW(wbvector<size_t>(3,s,'r'),d);
00102     return *this;
00103 };
00104
00105 wbararray& init(size_t d1, size_t d2, size_t d3, size_t d4, const T* d=NULL) {
00106     size_t s[4]= {d1,d2,d3,d4}; NEW(wbvector<size_t>(4,s,'r'),d);
00107     return *this;
00108 };
00109
00110 wbararray& init2ref(const wbararray &a) {
00111     return INIT2REF(a.data,&a.SIZE);
00112 };
00113
00114 wbararray& initScalar(T x=0, unsigned r=2) {
00115     if (r==2) { init(1,1,&x); } else
00116     if (r==1) { init(1); data[0]=x; } else
00117     if (!r) wblog(FL,"ERR %s() got r=%d (x=%g)",FCT,r,double(x));
00118     else { wbvector<size_t> S; NEW(S.init2val(r,1)); data[0]=x; }
00119     return *this;
00120 };
00121
00122 wbararray& init(const wbararray &a, char ref=0) {
00123     if (this!=&a) {
00124         if (!ref) { NEW(a.SIZE, a.data); }
00125         else { DELETE_DATA(); SIZE=a.SIZE;
00126             a.check_consistency(FL);
00127             if (a.sptr) {
00128                 (sptr=a.sptr)->add_dref();
00129                 data=a.data;
00130             }
00131         }
00132     }
00133     return *this;
00134 };
00135
00136 wbararray& set(const wbararray &A) { return init(A); }
00137
00138 template<class T_>
00139 wbararray& set(const wbararray &A, T_ fac) {
00140     if (fac==T_(1) ) { init(A); }
00141     else if (!A || !fac) { NEW(A.SIZE); }
00142     else {
00143         size_t i=0, n=A.SIZE.prod(0);
00144         const T *a=A.data;
00145         SIZE=A.SIZE; NEW_DATA(n,NULL,0,0);
00146
00147         if (fac==T_(-1))
00148             { for (; i<n; ++i) data[i]= -a[i]; }
00149         else { for (; i<n; ++i) data[i]=fac*a[i]; }
00150     }
00151     return *this;
00152 };
00153
00154 wbararray& init(const wbMatrix<T> &a, char ref=0) {
00155     if (ref) {
00156         size_t s[]={ a.dim2, a.dim1 }; SIZE.init(2,s);
00157         INIT2REF(a.data);
00158     }
00159     else {
00160         size_t i,j, l=0; init(a.dim1,a.dim2);
00161         for (i=0; i<a.dim1; ++i)
00162             for (j=0; j<a.dim2; ++j, ++l) { data[i+a.dim1*j]=a.data[l]; }
00163     }
00164     return *this;
00165 };
00166
00167 wbindex& ind2sub(size_t k, wbindex &I) const;
00168
00169 void adjustMMat(
00170     const char* F, int L,
00171     char flag, const T fac, wbararray &A
00172 ) const {
00173
00174     if (!isMatrix()) wblog(F,L,
00175         "ERR %d - rank-2 tensor required (%d)",FCT,SIZE.len);

```

```

00176         if (!strchr("NTC",flag)) wblog(F,L,
00177             "ERR %d - invalid flag %c<%d>",FCT,flag,flag);
00178
00179         size_t s=numel();
00180
00181         if (flag=='C' && typeid(T)!=typeid(wbcomplex)) flag='T';
00182
00183         if (fac!=T(1) || flag!='N') { size_t i;
00184             if (flag!='N') permute(A,"2 1"); else A=(*this);
00185             if (flag=='C') for (i=0; i<s; i++) A[i]=Wb::CONJ(A[i]);
00186             if (fac!=T(1)) for (i=0; i<s; i++) A[i]*=fac;
00187         }
00188         else { A.init2ref(*this); }
00189     }
00190
00191     void adjustDMat(
00192         const char* F, int L,
00193         char flag, const T fac, wbararray &A
00194     ) const {
00195
00196         if (SIZE.len!=1) wblog(F,L,
00197             "ERR %d - 1D representation required (%d)",FCT,SIZE.len);
00198         if (!strchr("NTC",flag)) wblog(F,L,
00199             "ERR %d - invalid flag %c<%d>",FCT,flag,flag);
00200
00201         size_t n=SIZE[0];
00202
00203         if (flag=='C' && typeid(T)!=typeid(wbcomplex)) flag='T';
00204
00205         if (fac!=T(1) || flag=='C') { size_t i; A=(*this);
00206             if (flag=='C') for (i=0; i<n; i++) A[i]=Wb::CONJ(A[i]);
00207             if (fac!=T(1)) for (i=0; i<n; i++) A[i]*=fac;
00208         }
00209         else { A.init2ref(*this); }
00210     }
00211
00212     void adjustVec(
00213         const char* F, int L, char &flag, wbararray &A, char pos
00214     ) const { A=(*this); A.adjustVec(F,L,flag,pos); };
00215
00216     void adjustVec(const char* F, int L, char &flag, char pos) {
00217         if (SIZE.len!=1) wblog(F,L,
00218             "ERR %s() vector expected (%d)",FCT,SIZE.len);
00219         if (!strchr("NTC",flag)) wblog(F,L,
00220             "ERR %s() invalid flag %c<%d>",FCT,flag,flag);
00221
00222         if (flag=='C' && typeid(T)!=typeid(wbcomplex)) flag='T';
00223         if (flag=='C') {
00224             for (size_t s=numel(), i=0; i<s; ++i)
00225                 data[i]=Wb::CONJ(data[i]);
00226         }
00227         flag='N';
00228
00229         SIZE.Resize(2);
00230         if (pos==1) { SIZE[1]=SIZE[0]; SIZE[0]=1; } else
00231         if (pos==2) { SIZE[1]=1; } else
00232             wblog(F,L,"ERR %s() invalid pos=%d",FCT,pos);
00233     };
00234
00235     void adjustCMat(const char* F, int L,
00236         T cfac, size_t s1, size_t s2, char cforce
00237     ){
00238         if (cfac!=T(0)) {
00239             if (data==NULL) {
00240                 if (cforce) wblog(F,L,
00241                     "WRN C = A*B + c*[ ] with c=%s !?", Wb::num2Str(cfac).data);
00242             }
00243             else if (!isMatrix() || SIZE[0]!=s1 || SIZE[1]!=s2) {
00244                 wblog(F,L,"ERR %s() dimension mismatch: C=(%s) =? (%d,%d).",
00245                     FCT, SSTR_(this), s1, s2); return;
00246             }
00247             else {
00248                 if (cfac!=T(1)) {
00249                     size_t i, s=SIZE.prod(0);
00250                     for (i=0; i<s; i++) data[i]*=cfac;
00251                 }
00252                 return;
00253             }
00254         }
00255         init();
00256     };
00257
00258     void setBlock(
00259         const wbvector< WBINDEX > &D,
00260         const wbindex &IB,
00261         const wbararray &A
00262     );

```

```

00263
00264 void addBlock(
00265     const WBINDEX &I,
00266     const wbararray &A, const char dflag=0
00267 );
00268
00269 wbararray& addBlock(
00270     size_t i, size_t j,
00271     size_t n, size_t m,
00272     wbararray &a
00273 ) const;
00274
00275 T norm2block(
00276     size_t i, size_t j,
00277     size_t n, size_t m
00278 ) const;
00279
00280 wbararray& getBlock(
00281     size_t i, size_t j,
00282     size_t n, size_t m,
00283     wbararray &a
00284 ) const;
00285
00286 wbararray& BlockDiag(const wbvector< wbararray > &D);
00287
00288 wbararray& blockTrace(
00289     size_t D,
00290     wbararray &a, size_t D2=-1
00291 ) const;
00292
00293 void copyStride(T* dd, size_t stride, T afac=1) const;
00294 void addStride (T* dd, size_t stride, T afac=1) const;
00295
00296 template<class T0>
00297 wbararray& init_T(const T0 *dd) {
00298     if (SIZE.len) {
00299         for (size_t n=SIZE.prod(), i=0; i<n; ++i) { data[i]=T(dd[i]); }
00300         return *this;
00301     };
00302
00303     template<class T0>
00304     wbararray& init_T(const char *F, int L, const T0 *dd) {
00305         for (size_t n=SIZE.prod(), i=0; i<n; ++i) { data[i]=T(dd[i]);
00306             if (T0(data[i])!=dd[i]) wblog(F_L,
00307                 "ERR type conversion changes value (%g->%g) !?",
00308                 double(dd[i]), double(data[i])
00309             );
00310         }
00311         return *this;
00312     };
00313
00314     wbararray& initT(const wbararray<T> &a) { return init(a); };
00315
00316     template<class T0>
00317     wbararray& initT(const wbararray<T0> &a) {
00318         NEW(a.SIZE); init_T(a.data);
00319         return *this;
00320     };
00321
00322     template <class T0>
00323     wbararray& initT(const char *F, int L, const wbararray<T0> &a) {
00324         NEW(a.SIZE); init_T(F_L,a.data); return *this;
00325     };
00326
00327     wbararray& init(char *s, T* d=NULL) {
00328         init(Str2Idx(s), d); return *this;
00329     };
00330
00331     wbararray& init(const WBINDEX &S, T* d=NULL) {
00332         return NEW(S,d);
00333     };
00334
00335     template <class IT>
00336     wbararray& init(const wbvector<IT> &S_, T* d=NULL) {
00337         WBINDEX S(S_.len);
00338         for (size_t i=0; i<S.len; ++i) { S.data[i]=size_t(S_.data[i]); }
00339         return NEW(S,d);
00340     };
00341
00342     wbararray& init_bare(const WBINDEX &S) {
00343         size_t len=S.prod(0);
00344         if (len)
00345             { SIZE=S; NEW_DATA(len,NULL,0,0); }
00346         else { init(); }
00347         return *this;
00348     };
00349

```

```

00350     wbararray& init_bare(size_t d1, size_t d2) {
00351         wvector<size_t> S(2); S[0]=d1; S[1]=d2;
00352         return init_bare(S);
00353     };
00354
00355     wbararray& init(const char *F, int L,
00356         const mxArray *a, char ref=0, char vec=0);
00357
00358     wbararray& init(const char *F, int L,
00359         const mxArray *a, wbpperm &P0);
00360
00361     wbararray& init(const mxArray *a, char ref=0, char vec=0) {
00362         return init(FL,a,ref,vec); }
00363     wbararray& init(const mxArray *a, wbpperm &P0) {
00364         return init(FL,a,P0); }
00365
00366     wbararray& init2ref(const T *x, const WBINDEX &S) {
00367         SIZE=S; return INIT2REF(x);
00368     };
00369     wbararray& init2ref(const T *x) {
00370         return INIT2REF(x);
00371     };
00372
00373     wbararray& init2ref(size_t d1, const T *x) {
00374         SIZE.init(1,&d1);
00375         return INIT2REF(x);
00376     };
00377     wbararray& init2ref(size_t d1, size_t d2, const T *x) {
00378         size_t s[] = {d1,d2}; SIZE.init(2,s);
00379         return INIT2REF(x);
00380     };
00381     wbararray& init2ref(size_t d1, size_t d2, size_t d3, const T *x) {
00382         size_t s[] = {d1,d2,d3}; SIZE.init(3,s);
00383         return INIT2REF(x);
00384     };
00385
00386     int isRef() const { return (sptr && sptr->isRef()); };
00387
00388     wbararray& Instantiate(const char *F=NULL, int L=0) {
00389         if (isRef() && data) {
00390             if (!sptr) wblog(F_L,"ERR %s() got null sptr !?",FCT);
00391             T *d0=data; NEW_DATA(SIZE.prod(0),d0);
00392         }
00393         else if (F) wblog(F,L,
00394             "WRN %s() no need to instantiate %s",SSTR_(this));
00395
00396         return *this;
00397     };
00398
00399     wbararray& unRef() { return Instantiate(); };
00400
00401     void init2Vec(const T *d0, size_t len, char ref=0) {
00402         SIZE.init(1,&len);
00403         if (data!=d0) { NEW_DATA(len,d0,ref); }
00404     };
00405
00406     void init2ref(const wbararray &A, size_t s1, size_t s2) {
00407         if (s1*s2!=A.SIZE.prod(0)) wblog(FL,
00408             "ERR %s() cannot reshape array %s into %dx%d",
00409             SSTR_(this), s1, s2
00410         );
00411
00412         SIZE.init(2); SIZE[0]=s1; SIZE[1]=s2;
00413         INIT2REF(A.data);
00414     };
00415
00416     void initTst();
00417
00418     void init2Vec(const wvector<T> &a) {
00419         WBINDEX S(1); S[0]=a.len;
00420         NEW(S, a.data);
00421     };
00422
00423     wbararray& initIdentity(size_t d, char dflag=0, T dval=1);
00424     wbararray& initIdentity(const WBINDEX &S, char dflag=0);
00425     wbararray& Reduce2Id();
00426
00427     wbararray& initIdentityB(
00428         size_t d1, size_t d2, size_t k=0);
00429
00430     wbararray& initIdentityB3(
00431         size_t d1, size_t d2, size_t D, size_t i0, T x=1);
00432
00433     wbararray& Expand2Projector(const T eps=0);
00434     wbararray& ExpandDiagonal(unsigned i1, unsigned i2);
00435     wbararray& ExpandDiagonal();
00436

```



```

00437     void Diag2Vec();
00438     wbararray& Reduce2Diag(T eps=0);
00439
00440     wbararray& initDiag(unsigned n, const T *d){
00441         init(n,n);
00442         for (size_t i=0; i<n; ++i) data[i+i*n]=d[i];
00443         return *this;
00444     };
00445
00446     wbararray& initDiag(const wbvector<T> &d){
00447         return initDiag(d.len, d.data); };
00448
00449     void initVec(size_t d, const T val=0) {
00450         init(d); if (val) set(val);
00451     };
00452
00453     wbararray& resize(const wbvector<size_t> &S, wbararray &B) const;
00454
00455     wbararray& resizeNumCols(size_t n, wbararray &B) const {
00456         if (!n && isEmpty()) { return B.init(); }
00457         if (SIZE.len!=2) wblog(FL,
00458             "ERR %s() for matrices only (got %s)",FCT,SSTR_(this));
00459         wbvector<size_t> S(SIZE); S[1]=n;
00460         return resize(S,B);
00461     };
00462
00463     void Resize(const wbvector<size_t> &S, const T* d0=NULL) {
00464         if (d0) { NEW(S,d0); } else
00465         if (S!=SIZE) {
00466             wbararray<T> X;
00467             resize(S,X).save2(*this);
00468         };
00469     };
00470
00471     void Resize(size_t s1, size_t s2) {
00472         size_t s[2]={s1,s2}; wbararray X;
00473         if (SIZE.len!=2) wblog(FL,"ERR rank-2 required (%d)",SIZE.len);
00474         resize(wbvector<size_t>(2,s),X).save2(*this);
00475     };
00476
00477     wbararray& Resize1(unsigned i, size_t Si) {
00478         if (i>=SIZE.len) wblog(FL,
00479             "ERR %s() index out of bounds (%d/%d)",FCT,i,SIZE.len);
00480         if (SIZE[i]!=Si) {
00481             wbvector<size_t> S(SIZE); wbararray X; S[i]=Si;
00482             resize(S,X).save2(*this);
00483         }
00484         return *this;
00485     };
00486
00487     wbararray& ResizeM(unsigned r, const wbvector<size_t> &Sm);
00488
00489     wbararray& Enlarge(
00490         size_t dim,
00491         size_t D,
00492         size_t istart=0
00493     );
00494
00495     void swap(wbararray &A) {
00496         SWAP(sptr, A.sptr );
00497         SWAP(data, A.data );
00498         SWAP(mtype,A.mtype); SIZE.swap(A.SIZE);
00499     };
00500
00501     wbararray& save2(wbararray &A) {
00502         if (this!=&A) {
00503             if (A.mtype) wblog(FL,"ERR %s() got %s / %s",
00504                 FCT, Wb::MTYPE_STR[mtype], Wb::MTYPE_STR[A.mtype]);
00505             this->swap(A); if (data!=A.data) { init(); }
00506         }
00507         return A;
00508     };
00509
00510     wbararray& save2(wbMatrix<T>&);
00511
00512     wbararray& Append2(
00513         const char *F, int L, wbararray &B, unsigned dim);
00514     wbararray& Append2(wbararray &B, unsigned dim){
00515         return Append2(0,0,B,dim);
00516     }
00517
00518     void Append2(
00519         const char *F, int L,
00520         wbararray &B, unsigned dim,
00521         wbararray<double> &W, double eps
00522     );
00523     void Append2(wbararray &B, unsigned dim, wbararray &W, double eps){

```

```

00524         return Append2(0,0,B,dim,W,eps);
00525     };
00526
00527     void rand(const UVEC &d) { init(d); setRand(); };
00528     void setRand(const char pnflag=0);
00529
00530     wbararray& Reset(const T *dref) {
00531         size_t n=SIZE.prod();
00532         if (dref) { memcpy(data,dref,sizeof(T)*n); }
00533         else { MEM_SET<T>(data,n); }
00534         return *this;
00535     };
00536
00537     wbararray& set(const T& c) {
00538         size_t n=SIZE.prod();
00539         if (c)
00540             for (size_t i=0; i<n; ++i) data[i]=c; }
00541         else { MEM_SET<T>(data,n); }
00542         return *this;
00543     };
00544
00545     wbararray& operator= (const wbararray &a) { return init(a); };
00546
00547     template <class T0>
00548     wbararray<T>& operator= (const wbararray<T0> &a) {
00549         if ((void*)this!=(void*)&a) {
00550             NEW(a.SIZE);
00551             for (size_t n=a.numel(), i=0; i<n; ++i) data[i]=T(a.data[i]);
00552         }
00553         return *this;
00554     };
00555
00556     void SetRef(const widx_t *I, size_t len, const T* d0);
00557     void SetRef(size_t k, const T* d0) { SetRef(&k,1,d0); };
00558
00559     const T* ref(const widx_t *I, size_t n) const {
00560         if (!n) return NULL;
00561         return data+serial_index(I,n);
00562     };
00563     T* ref(const widx_t *I, size_t n) {
00564         if (!n) return NULL;
00565         return data+serial_index(I,n);
00566     };
00567
00568     T* col(size_t k) {
00569         if (SIZE.len!=2 || k>=SIZE[1]) wblog(FL,
00570             "ERR %s() index out of bounds (%s; %d)",FCT,SSTR_(this),k);
00571         return (data+k*SIZE[0]);
00572     };
00573     const T* col(size_t k) const {
00574         if (SIZE.len!=2 || k>=SIZE[1]) wblog(FL,
00575             "ERR %s() index out of bounds (%s; %d)",FCT,SSTR_(this),k);
00576         return (data+k*SIZE[0]);
00577     };
00578
00579     T* row(size_t k) {
00580         if (SIZE.len!=2 || k>=SIZE[0]) wblog(FL,
00581             "ERR %s() index out of bounds (%d / %s)",FCT,k,SSTR_(this));
00582         return (data+k);
00583     };
00584     const T* row(size_t k) const {
00585         if (SIZE.len!=2 || k>=SIZE[0]) wblog(FL,
00586             "ERR %s() index out of bounds (%d / %s)",FCT,k,SSTR_(this));
00587         return (data+k);
00588     };
00589
00590     wbvector<T>& getCol(size_t k, wbvector<T> &v) const;
00591     wbararray<T>& getCol(size_t k, wbararray<T> &a) const;
00592
00593     wbvector<T>& getRow(size_t k, wbvector<T> &v) const;
00594     wbararray<T>& getRow(size_t k, wbararray<T> &a) const;
00595
00596     wbvector<T> getDiag() const;
00597
00598     T* ref(const WBINDEX &I) { return ref(I.data,I.len); };
00599     const T* ref(const WBINDEX &I) const {
00600         return ref(I.data,I.len);
00601     };
00602
00603     const T* ref(widx_t i) const { return ref(&i,1); }
00604     T* ref(widx_t i) { return ref(&i,1); }
00605
00606     const T* ref(widx_t i1, widx_t i2) const {
00607         widx_t I[]={i1,i2}; return ref(I,2);
00608     };
00609     T* ref(widx_t i1, widx_t i2) {
00610         widx_t I[]={i1,i2}; return ref(I,2);

```

```

00611     };
00612
00613     const T* ref(widx_t i1, widx_t i2, widx_t i3) const {
00614         widx_t I[]={i1,i2,i3}; return ref(I,3);
00615     };
00616     T* ref(widx_t i1, widx_t i2, widx_t i3) {
00617         widx_t I[]={i1,i2,i3}; return ref(I,3);
00618     };
00619
00620     const T& operator[] (size_t i) const {
00621         return data[i];
00622     };
00623     T& operator[] (size_t i) {
00624         return data[i];
00625     }
00626
00627     const T& operator() (size_t i) const {
00628         if (SIZE.len!=1) wblog(FL,
00629             "ERR %s requires vector type (%s)",FCT,SSTR_(this));
00630         return data[i];
00631     };
00632     T& operator() (size_t i) {
00633         if (SIZE.len!=1) wblog(FL,
00634             "ERR %s requires vector type (%s)",FCT,SSTR_(this));
00635         return data[i];
00636     };
00637
00638     const T& operator() (size_t i, size_t j) const {
00639         if (SIZE.len!=2) wblog(FL,
00640             "ERR %s(i,j) requires matrix type (%s)",FCT,SSTR_(this));
00641         return data[ i + j*SIZE[0] ];
00642     };
00643     T& operator() (size_t i, size_t j) {
00644         if (SIZE.len!=2) wblog(FL,
00645             "ERR %s(i,j) requires matrix type (%s)",FCT,SSTR_(this));
00646         return data[ i + j*SIZE[0] ];
00647     };
00648
00649     const T& operator() (size_t i, size_t j, size_t k) const {
00650         if (SIZE.len!=3) wblog(FL,
00651             "ERR %s(i,j,k) requires 3D-object (%s)",FCT,SSTR_(this));
00652         return data[ i + SIZE[0] * (j + SIZE[1]*k) ];
00653     };
00654     T& operator() (size_t i, size_t j, size_t k) {
00655         if (SIZE.len!=3) wblog(FL,
00656             "ERR %s(i,j,k) requires 3D-object (%s)",FCT,SSTR_(this));
00657         return data[ i + SIZE[0] * (j + SIZE[1]*k) ];
00658     };
00659
00660     const T& operator() (const WBINDEX &I) const {
00661         if (I.len!=SIZE.len) wblog(FL,
00662             "ERR %s() invalid index [%s]",FCT,STR(I));
00663         return data[serial_index(I.data,I.len)];
00664     };
00665     T& operator() (const WBINDEX &I) {
00666         if (I.len!=SIZE.len) wblog(FL,
00667             "ERR invalid index [%s] having %s",STR(I),SSTR_(this));
00668         return data[serial_index(I.data,I.len)];
00669     };
00670
00671     const T& element(const WBINDEX &I) const {
00672         if (I.len!=SIZE.len) wblog(FL,"ERR %s() invalid index [%s] "
00673             "having %s",FCT,STR(I),SSTR_(this));
00674         for (size_t i=0; i<I.len; i++) if (I.data[i]>=SIZE.data[i])
00675             wblog(FL,"ERR %s() index out of bounds (%s; %s)",
00676                 FCT,STR(I),SSTR_(this));
00677         return data[serial_index(I.data,I.len)];
00678     };
00679     T& element(const WBINDEX &I) {
00680         if (I.len!=SIZE.len) wblog(FL,"ERR %s() invalid index [%s] "
00681             "having %s",FCT,STR(I),SSTR_(this));
00682         for (size_t i=0; i<I.len; i++) if (I.data[i]>=SIZE.data[i])
00683             wblog(FL,"ERR %s() index out of bounds (%s; %s)",
00684                 FCT,STR(I),SSTR_(this));
00685         return data[serial_index(I.data,I.len)];
00686     };
00687
00688     size_t numel() const {
00689         size_t N=0;
00690         if (SIZE.len) {
00691             const size_t *s=SIZE.data; size_t i=0;
00692             N=s[i]; for (++i; i<SIZE.len; ++i) { N*=s[i]; }
00693         }
00694         return N;
00695     };
00696
00697     size_t numOM(unsigned r0) const {

```

```

00698     size_t M=1;
00699     if (!SIZE.len) { return (M=0); }
00700     if (r0<2) wblog(FL,"ERR %s() invalid r0=%d/%d",FCT,r0,SIZE.len);
00701     if (r0<SIZE.len) {
00702         const size_t *s=SIZE.data; size_t i=r0;
00703         M=s[i]; for (++i; i<SIZE.len; ++i) { M*=s[i]; }
00704
00705         if (r0==2 && M>1) wblog(FL,
00706             "ERR %s() got OM=%d for %s @ r0=%d",FCT,M,SSTR_(this),r0);
00707     }
00708     return M;
00709 };
00710
00711 wbsting sizeStr(unsigned stride=0, const char *sep2="") const {
00712     if (SIZE.len)
00713         { return SIZE.toStrf("","x",stride,sep2); }
00714     else { return wbsting("[]"); }
00715 };
00716
00717 wbarray& Squeeze();
00718
00719 int skipSingletons(const char *F, int L, unsigned r=-1);
00720 int skipSingletons(unsigned r=-1) {
00721     return skipSingletons(FL,r);
00722 };
00723
00724 wbarray& appendSingletons(
00725     const char *F, int L, unsigned r, unsigned m=0);
00726 wbarray& appendSingletons(unsigned r, unsigned m=0) {
00727     return appendSingletons(0,0,r,m); };
00728
00729 void prependSingletons(unsigned r);
00730
00731 wbarray& ExpandOM(const char *F, int L, unsigned r0,
00732     const wvector<unsigned> &S,
00733     const unsigned *sx=NULL);
00734
00735 wbarray& ExpandOM(const char *F, int L, unsigned r0,
00736     unsigned m, const unsigned *s, const unsigned *sx=NULL
00737 ) { return ExpandOM(F,L,r0, wvector<unsigned>(m,s,'r'), sx); };
00738
00739 int ExpandOM(const char *F, int L, wbarray &B, unsigned r0);
00740
00741 wbarray& FuseOM(const char *F, int L, unsigned r, unsigned mc=-1) {
00742     if (!SIZE.len) { return *this; }
00743
00744     unsigned i, m=1;
00745     for (i=r; i<SIZE.len; ++i) { m*=SIZE[i]; }
00746
00747     if (r<2 && m>1) wblog(F_L,
00748         "ERR %s() requesting OM m=%d for rank r=%d/%d",FCT,m,r,SIZE.len);
00749     if (r>SIZE.len) wblog(F_L,
00750         "ERR %s() rank out of bounds (r=%d/%d; m=%d)",FCT,r,SIZE.len,m);
00751     if (int(mc)>=0 && m!=mc && (m!=1 || mc>1)) wblog(F_L,"ERR %s() "
00752         "got OM mismatch %s @ %d/%d (r=%d)",FCT,SSTRM(SIZE,r),m,mc,r);
00753
00754     if (m==1 && r)
00755         { SIZE.len=r; }
00756     else { SIZE[r]=m; SIZE.len=r+1; }
00757
00758     return *this;
00759 };
00760
00761 size_t length() const {
00762     size_t l=0; const size_t *s=SIZE.data;
00763     for (unsigned i=0; i<SIZE.len; ++i) {
00764         if (l<s[i]) { l=s[i]; } else
00765             if (!s[i]) { l=0; break; }
00766     }
00767     return l;
00768 }
00769
00770 size_t size(unsigned i) const { return SIZE[i]; };
00771
00772 size_t dim (unsigned i) const {
00773     if (i==0 || i>SIZE.len) wblog(FL,
00774         "ERR index out of bounds (%d/%d)", i-1, SIZE.len);
00775     return SIZE[i-1];
00776 };
00777 size_t dim0(unsigned i) const {
00778     if (i>=SIZE.len) wblog(FL,
00779         "ERR index out of bounds (%d/%d)", i, SIZE.len);
00780     return SIZE[i];
00781 };
00782
00783 size_t dim1() const {
00784     size_t D=0;

```

```

00785         if (SIZE.len==1) { D=SIZE[0]; } else
00786         if (SIZE.len) {
00787             unsigned i=0, n=SIZE.len, z=0, l=-1; const size_t *s=SIZE.data;
00788             for (; i<n; ++i) {
00789                 if (s[i]>1) {
00790                     if (l>i) { l=i; } else
00791                         wblog(FL,"ERR %s() invalid vector %s",FCT,SSTR_(this));
00792                 }
00793                 else if (!s[i]) { ++z; }
00794             }
00795
00796             if (!z) { D=s[l<n ? l : 0]; } else
00797             if (n>2 || l<n) {
00798                 wblog(FL,"ERR %s() invalid vector %s",FCT,SSTR_(this));
00799             }
00800         }
00801         return D;
00802     };
00803
00804     size_t dim2() const {
00805         if (!SIZE.len) return 0;
00806         if (SIZE.len!=2) wblog(FL,
00807             "ERR %s() got rank-%d array (%s) !?",FCT,SIZE.len,SSTR_(this));
00808         if (SIZE[0]!=SIZE[1]) wblog(FL,
00809             "ERR %s() got non-square matrix (%s) !?",FCT,SSTR_(this));
00810         return SIZE[0];
00811     };
00812
00813     void getMatSize(const char *F, int L, size_t &dim1, size_t &dim2) const;
00814     void getMatSize(size_t &dim1, size_t &dim2, unsigned m) const;
00815     void getMatSize(const CtrIdx &ic, size_t &dimc, size_t &dimk,
00816         unsigned r=-1, size_t *dom=NULL) const;
00817
00818     unsigned rank() const { return SIZE.len; };
00819
00820     bool isRank(unsigned r) const {
00821         bool q=1;
00822         if (r!=SIZE.len) { unsigned i=r;
00823             for (; i<SIZE.len; ++i) { if (SIZE[i]!=1) break; }
00824             if (i!=SIZE.len) { q=0; }
00825         }
00826         return q;
00827     };
00828
00829     bool isRankM(unsigned r, unsigned m) const {
00830         bool q=1;
00831         if (SIZE.len==r) { return q; }
00832         if (r>2 && m) { if (SIZE.len==r+1 || SIZE.len==r+m ) { return q; } }
00833         return (q=0);
00834     };
00835
00836     bool isOpS(size_t *n=NULL) const;
00837     bool isOpS(unsigned &n) const { bool q;
00838         size_t N; q=isOpS(&N); n=N;
00839         if (size_t(n)!=N) wblog(FL,
00840             "ERR %s() unsigned out of bounds (%d/%ld)",FCT,n,N);
00841         return q;
00842     };
00843
00844     wbararray< SkipTiny_float>(const T eps QS_UNUSED_VAR =1e-14) { return *this; };
00845     double SkipTiny_imag (double eps QS_UNUSED_VAR =1e-14) { return 0; };
00846
00847     double SkipTiny(double eps_=1E-14);
00848
00849     wbararray<T>& SkipTrailingZeroSpace(
00850         unsigned k,
00851         size_t n=0,
00852         double eps=1e-14);
00853
00854     bool isMatrix() const { return SIZE.len==2; }
00855     bool isMatrix(size_t d1, size_t d2) {
00856         return (SIZE.len==2 && SIZE[0]==d1 && SIZE[1]==d2);
00857     };
00858
00859     bool isSMatrix(unsigned d=-1) const {
00860         bool q=0;
00861         if (SIZE.len==2 && SIZE[0]==SIZE[1] &&
00862             (int(d)<0 || d==SIZE[0])) { q=1; }
00863         return q;
00864     };
00865
00866     bool isSquare() const {
00867         if (SIZE.len%2) return 0;
00868         else {
00869             for (unsigned i=0, n=SIZE.len/2; i<n; ++i) {
00870                 if (SIZE[i]!=SIZE[n+i]) return 0; }
00871         }

```

```

00872         return 1;
00873     };
00874
00875     bool isEmpty() const { return (data==NULL); };
00876     bool isComplex() const;
00877     bool isZero(double eps=0., char flag=0) const;
00878
00879     bool isFinite() const {
00880         bool q=1; size_t i=0, n=numel();
00881         for (; i<n; ++i) { if (!isfinite(data[i])) { q=0; break; }
00882             }; return q;
00883     };
00884
00885     explicit operator bool() const { return !isEmpty(); }
00886     bool operator! () const { return isEmpty(); }
00887
00888     wbarray& Symmetrize(
00889         const char *F, int L,
00890         double *delta=NULL, char cflag=1, char tflag=0, char fflag=0
00891     );
00892
00893     wbarray& swapRows(size_t i1, size_t i2);
00894     wbarray& swapCols(size_t j1, size_t j2);
00895
00896     wbarray& setCol(size_t k, size_t k0 );
00897     wbarray& setCol(size_t k, const T* d, T fac=1, size_t stride=1);
00898     wbarray& setRow(size_t k, const T* d);
00899
00900     wbarray& setCol(size_t k, const wbvector<T> &v);
00901     template<class T2>
00902     wbarray<T>& setCol(size_t k, const wbvector<T2> &v);
00903
00904     wbvector<T>& colNorm2(wbvector<T> &a) const;
00905
00906     T colNorm2(size_t k) const;
00907
00908     wbvector<T>& normDim2(unsigned k, wbvector<T> &xk) const;
00909     wbvector<T>& normDim(unsigned k, wbvector<T> &xk) const {
00910         normDim2(k,xk);
00911         for (size_t n=xk.len, i=0; i<n; ++i) { xk[i]=Wb::sqrt(xk[i]); }
00912         return xk;
00913     };
00914
00915     unsigned QRdecomp(
00916         const char *F, int L, wbarray<T> &Q, wbarray<T> &R,
00917         T eps=1E-15);
00918
00919     int Householder(
00920         const char *F, int L,
00921         size_t k, T *u,
00922         WBPERM *P=NULL,
00923         T eps=1E-15);
00924
00925     unsigned eigTriDiag(
00926         const char *F, int L, wbvector<T> &E,
00927         wbarray<T> *U=NULL, T eps=1E-15) const;
00928
00929     wbarray& ColProject(
00930         size_t k1, size_t k2, char nflag=0, char tnorm=0);
00931
00932     wbarray& ColPermute(const wbperm &P, char iflag=0);
00933
00934     bool isOrthogonalCol(size_t k, char tnorm=0) const;
00935     bool isOrthoCols(T *x2=NULL, char tnorm=0, T eps=1E-14) const;
00936
00937     wbarray& SignCol(size_t k, const T *d0, char tnorm=0);
00938     wbarray& FlipSignCol(size_t k);
00939     wbarray& SignConventionCol(
00940         size_t k=-1,
00941         double eps=1E-12
00942     );
00943
00944     T NormalizeCol(size_t k, char tnorm=0, char qflag=0);
00945     wbarray& NormalizeCols(
00946         const char *F, int L,
00947         double *amin=NULL, double *amax=NULL, char tnorm=0
00948     );
00949
00950     wbarray& OrthoNormalizeCols(
00951         const char *F=NULL, int L=0, char tnorm=0,
00952         char qxflag=0, double eps=1E-14, unsigned np=1
00953     );
00954
00955     wbarray& balanceOp(
00956         const char *F, int L, T &xref, double &xscale
00957     );
00958

```

```

00959     bool isProptoId(T &x, T eps=1E-14) const;
00960
00961     bool isProptoId(T *x_=NULL, T eps=1E-14) const {
00962         bool q=0; T x=T(0);
00963         if (isProptoId(x,eps)) { q=1; if (x_) { (*x_)=x; }}
00964         return q;
00965     };
00966
00967     bool isDiagMatrix(double eps=1E-14) const {
00968         return isDiag_aux(eps,"isDiag"); }
00969
00970     bool isIdentityMatrix(double eps=1E-14) const {
00971         if (data && Wb::abs(data[0]-double(1))<eps) {
00972             if ((SIZE.len%2)==0) {
00973                 if (SIZE.allEqual(1)) { return 1; }
00974                 else { T one(1);
00975                     return isProptoId(one,eps);
00976                 }
00977             }
00978         }
00979         return 0;
00980     };
00981
00982     bool isDiagMatrix(double *eps) const {
00983         return isDiag_aux(eps,"isDiag"); }
00984
00985     bool isIdentityMatrix(double *eps) const {
00986         return isDiag_aux(eps,"isIdty"); }
00987
00988     size_t nnz(const T eps=0, WBINDEX *I=NULL) const;
00989
00990     double sparsity(const T eps=0) const {
00991         size_t n=numel();
00992         return (n ? nnz(eps)/double(n) : 0);
00993     };
00994
00995     bool isHConj(
00996         const wbararray &B, double eps=1E-12, double* xref=NULL
00997     ) const { return isSym_aux(FLF, B, eps, xref, 's'); };
00998
00999     bool isHConj(
01000         double eps=1E-12, double* xref=NULL
01001     ) const { return isSym_aux(FLF,*this,eps,xref,'s'); };
01002
01003     bool isAHerm(
01004         const wbararray &B, double eps=1E-12, double* xref=NULL
01005     ) const { return isSym_aux(FLF, B, eps, xref, 'a'); };
01006
01007     bool isAHerm(
01008         double eps=1E-12, double* xref=NULL
01009     ) const { return isSym_aux(FLF,*this,eps,xref,'a'); };
01010
01011     char hasGroupSize(size_t s1, size_t s2) const;
01012
01013     template<class TI>
01014     int aMax(T &x, TI &k) const;
01015
01016     T aMax() const {
01017         T x; size_t k; int e=aMax(x,k);
01018         if (e) wblog(FL,"WRN %s() got empty array (e=%d)",FCT,e);
01019         return x;
01020     };
01021
01022     template<class TI>
01023     T aMax(TI &i, TI &j, const wbararray *B=NULL) const;
01024
01025     double maxDiff(const wbararray &B) const;
01026     T froNorm2(const wbararray &B) const;
01027     T norm2() const;
01028     T norm() const { return Wb::sqrt(norm2()); };
01029
01030     T hasNorm2(const T &x2) const;
01031
01032     T normCol2(size_t k) const;
01033
01034     T norm2Recs(long i1, long i2) const;
01035     T norm2Cols(long j1, long j2) const;
01036
01037     wbvector<T>& norm2Cols(wbvector<T> &x2) const;
01038     wbvector<T> normCols() const;
01039
01040     T scalarProd(const wbararray &B) const;
01041     T sum() const;
01042
01043     wbararray& sum(const WBINDEX &I, wbararray&) const;
01044     wbararray& sum(const char *sidx, wbararray &A) const {
01045         sum(Str2Idx(sidx,1), A);

```

```

01046         return A;
01047     };
01048
01049     char fitsSize(const wbarray &, char strict=0) const;
01050
01051     size_t getSize(unsigned i, char lflag=1) const {
01052         size_t s=0;
01053         if (i<SIZE.len) { s=SIZE[i]; } else
01054             if (lflag && SIZE.len) {
01055                 for (i=0; i<SIZE.len; ++i) { if (!SIZE[i]) { break; }}
01056                 if (i==SIZE.len) { s=1; }
01057             }
01058         return s;
01059     };
01060
01061     wbvector<size_t>& getSize(
01062         const wbvector<unsigned> &I, wbvector<size_t> &S
01063     ) const { return SIZE.select(I,S); };
01064
01065     size_t getSizeM(unsigned r) {
01066         size_t M=1;
01067         if (r>2 && SIZE.len==r+1) { M=SIZE[r]; }
01068         else if (SIZE.len!=r) wblog(FL,
01069             "ERR %s() unexpected rank r=%d/%d",FCT,r,SIZE.len);
01070         return M;
01071     };
01072
01073     bool sameSize(const size_t *s, unsigned n) const;
01074
01075     bool sameSize(size_t d1, size_t d2) const {
01076         size_t s[2]={d1,d2}; return sameSize(s,2);
01077     };
01078
01079     template <class Ts>
01080     bool sameSizeM(unsigned r, unsigned m, const Ts *s) const {
01081         bool rval=0; unsigned n=r+m;
01082         if (n==SIZE.len) { unsigned i=0;
01083             for (; i<m; ++i) { if (SIZE[r+i]!=s[i]) { break; }}
01084             if (i>=m) { rval=1; }
01085         }
01086         return rval;
01087     };
01088
01089     template<class TB>
01090     bool sameSize(const wbarray<TB> &, char lflag=2) const;
01091
01092     bool sameSize1(const size_t *s, unsigned n) const;
01093
01094     template<class TB>
01095     bool hasSameSize(
01096         unsigned ia, const wbarray<TB> &B, unsigned ib, char lflag=1
01097     ) const {
01098         size_t sa=getSize(ia,lflag), sb=B.getSize(ib,lflag);
01099         return (sa==sb);
01100     };
01101
01102     char sameUptoFac(
01103         const wbarray &B, T *fac=NULL, double eps=1e-12
01104     ) const;
01105
01106     bool sameAs(const wbarray &B, double eps=1e-12) const;
01107
01108     bool operator== (const wbarray &B) const { return sameAs(B); };
01109     bool operator!= (const wbarray &B) const { return !sameAs(B); };
01110
01111     T normDiff2(const wbarray &B, char sflag=0) const;
01112     T normDiff (const wbarray &B, char sflag=0) const {
01113         return Wb::sqrt(normDiff2(B,sflag)); };
01114
01115     T normDiff2(const wbvector<T> &B) const;
01116     T normDiff (const wbvector<T> &B) const { return Wb::sqrt(normDiff2(B)); };
01117
01118     void TimesEl(const wbarray &, char conj=0);
01119     wbarray& timesEl(
01120         const wbarray &B, wbarray &C, T bfac=1, T cfac=0, char conj=0) const;
01121     wbarray& timesEl_OM(unsigned r,
01122         const wbarray &B, wbarray &C, char conj=0) const;
01123
01124     T dotProd(const wbarray &, char conj=0) const;
01125     T weightedAvg(const wbarray &) const;
01126
01127     void TensorProd(const wbarray &B,
01128         const char aflag='N', const char bflag='N', const char kflag=0
01129     ){
01130         wbarray X(*this);
01131         X.tensorProd(B,*this,aflag,bflag,kflag);
01132     };

```



```

01133
01134     wbararray& tensorProd(
01135         const wbararray &, wbararray &,
01136         const char aflag='N', const char bflag='N', const char kflag=0
01137     ) const;
01138
01139     wbararray& kron(
01140         const wbararray &B, wbararray &X, char aflag='N', char bflag='N'
01141     ) const { return tensorProd(B,X,aflag,bflag,'k'); };
01142
01143     wbararray& Kron(const wbararray &B, char aflag='N', char bflag='N') {
01144         wbararray X; tensorProd(B,X,aflag,bflag,'k');
01145         return X.save2(*this);
01146     };
01147
01148     wbararray& Cat(unsigned dim, const wvector< wbararray* > &ap);
01149     wbararray& Cat(unsigned dim, const wvector< wbararray > &ap);
01150
01151     wbararray& BlockCat(
01152         const wbararray< wbararray > &aa,
01153         WBINDEX *D1=NULL, WBINDEX *D2=NULL
01154     ){
01155         wbararray< const wbararray* > ap(aa.SIZE);
01156         for (size_t n=aa.numel(), i=0; i<n; ++i) { ap[i]=&aa[i]; }
01157         return BlockCat(ap,D1,D2);
01158     };
01159
01160     wbararray& BlockCat(
01161         const wbararray< const wbararray* > &ap,
01162         WBINDEX *D1=NULL, WBINDEX *D2=NULL
01163     );
01164
01165     void operator+= (T x) {
01166         if (isRef()) wblog(FL,"ERR %s() reference is considered const",FCT);
01167         for (size_t n=numel(), i=0; i<n; ++i) { data[i]+=x; }
01168     };
01169
01170     void operator-= (T x) {
01171         if (isRef()) wblog(FL,"ERR %s() reference is considered const",FCT);
01172         for (size_t n=numel(), i=0; i<n; ++i) { data[i]-=x; }
01173     };
01174
01175     template <class T_>
01176     wbararray& Times(const T_ a, char rcpy=0) {
01177         if (a!=T_(1)) {
01178             if (!rcpy || !isRef()) { return (*this)*=a; }
01179             else {
01180                 wbararray<T> X; this->times(a,X);
01181                 X.save2(*this);
01182             }
01183         }; return *this;
01184     };
01185
01186     template <class T_>
01187     wbararray& times(const T_ a, wbararray<T> &X) const {
01188         if (&X==this) { X*=a; } else
01189         if (a==T_(1)) { X.init(*this); } else
01190         if (!a) { X.init(SIZE); } else { X.set(*this,a); }
01191         return X;
01192     };
01193
01194     template <class T_>
01195     wbararray& operator*=(const T_ a) {
01196         if (a!=T_(1)) { size_t i=0, n=numel();
01197             if (isRef()) wblog(FL,"ERR %s() got ref (considered const)",FCT);
01198             if (!a) { MEM_SET<T>(data,n); } else
01199             if (a==T_(-1))
01200                 { for (; i<n; ++i) { data[i] = -data[i]; } }
01201             else { for (; i<n; ++i) { data[i] *= a; } }
01202         }
01203         return *this;
01204     };
01205
01206     template <class T_>
01207     wbararray& operator/=(const T_ &a) {
01208         if (SIZE && a!=T_(1)) { size_t i=0, n=numel();
01209             if (isRef()) wblog(FL,"ERR %s() reference is considered const",FCT);
01210             if (!a) wblog(FL,"ERR %s() got div/0",FCT);
01211
01212             if (a==T_(-1)) for (; i<n; ++i) { data[i] = -data[i]; } else
01213             if (n<4 || WbUtil<T>::isInt()) { for (; i<n; ++i) { data[i]/=a; } }
01214             else {
01215                 T x=1/T(a);
01216                 if (fabs(double(x*a-1))>1E-12)
01217                     { for (; i<n; ++i) { data[i]/=a; } }
01218                 else { for (; i<n; ++i) { data[i]*=x; } }
01219             }
01220         }
01221     };

```

```

01220     }
01221     return *this;
01222 };
01223
01224 void operator+= (const wbararray &B) { Plus(B); };
01225 void operator-= (const wbararray &B) { Plus(B,-1); };
01226
01227 wbararray& plus(
01228     const wbararray &B, wbararray &C, T bfac=1, char iflag=0) const;
01229
01230 template<class TB>
01231 wbararray& Plus(
01232     const wbararray<TB> &B, TB bfac=TB(1),
01233     char iflag=0, T afac=T(1));
01234
01235 bool equal (const wbararray&, T eps) const;
01236 bool unequal(const wbararray&, T eps) const;
01237 bool equal (const wbararray&, double eps, double& maxdiff) const;
01238 bool unequal(const wbararray&, double eps, double& maxdiff) const;
01239
01240 bool allEqual(const T &x) {
01241     for (size_t n=numel(), i=0; i<n; ++i) { if (data[i]!=x) return 0; }
01242     return 1;
01243 };
01244 bool allLE(const T &x) {
01245     for (size_t n=numel(), i=0; i<n; ++i) { if (data[i]>x) return 0; }
01246     return 1;
01247 };
01248
01249 char isVector() const {
01250     if (SIZE.len==1) return 1;
01251     if (SIZE.len==2 || (SIZE[0]!=1 && SIZE[1]!=1)) return 0;
01252     if (SIZE[0]==1 && SIZE[1]==1) return -1;
01253     else return 2;
01254 };
01255
01256 unsigned isScalar() const {
01257     unsigned rval=0;
01258     if (SIZE.len && SIZE.allEqual(1)) { rval=SIZE.len; }
01259     return rval;
01260 };
01261
01262 bool requiresDataPerm(const wbperm &P, char lflag=3) const;
01263
01264 void markUnequalZero(const T m=1, const T eps=0) {
01265     size_t i=0, n=numel(); T z(0);
01266     if (eps==z)
01267         { for (; i<n; ++i) data[i] = (data[i]!=T(0) ? m:z); }
01268     else { for (; i<n; ++i) data[i] = (Wb::abs(data[i])>eps ? m:z); }
01269 };
01270
01271 void toMatrixRef(
01272     wbararray &A,
01273     const UVEC &I,
01274     int pos,
01275     wbperm &P
01276 ) const;
01277
01278 bool toMatrixRef(const char *F, int L,
01279     wbararray &A,
01280     const UVEC &I,
01281     int pos,
01282     char &tflag
01283 ) const;
01284
01285 void toMatrixRef(
01286     wbararray &A, unsigned K, const wbperm &P0
01287 ) const;
01288
01289 void toMatrixRefM(
01290     wbararray &A, unsigned K, const wbperm &P0, unsigned R,
01291     wbperm pfin=wbperm()
01292 ) const;
01293
01294 void toMatrixRef(
01295     wbararray &A, unsigned i, int pos, wbperm &P
01296 ) const { return toMatrixRef( A, UVEC(1,&i), pos, P); };
01297
01298 void toMatrixRef(
01299     wbararray &A, const char* sidx, int pos, wbperm &P
01300 ) const { return toMatrixRef( A, Str2Idx(sidx,1), pos, P); };
01301
01302 wbararray& Reshape(const wbvector<size_t> &, bool lflag=0);
01303
01304 wbararray& Reshape(size_t s1, size_t s2, bool lflag=0) {
01305     WBINDEX S(2); S.data[0]=s1; S.data[1]=s2;
01306     return Reshape(S,lflag);

```

```

01307     };
01308     wbarry& Reshape(size_t s1, size_t s2, size_t s3, bool lflag=0) {
01309         WBINDEX S(3); S.data[0]=s1; S.data[1]=s2; S.data[2]=s3;
01310         return Reshape(S,lflag);
01311     };
01312
01313     wbarry<T>& Reshape1(const size_t *s, unsigned n) {
01314         if (!sameSize1(s,n)) wblog(FL,
01315             "ERR %s() size mismatch (%s <> %s)",ECT,
01316             SSTR_(this), SSTR(wbvector<size_t>(n,s)));
01317         SIZE.init(n,s); return *this;
01318     };
01319     wbarry& Reshape1(size_t s1, size_t s2) {
01320         size_t s[2]={s1,s2}; return Reshape1(s,2);
01321     };
01322     wbarry& Reshape1(size_t s1, size_t s2, size_t s3) {
01323         size_t s[3]={s1,s2,s3}; return Reshape1(s,3);
01324     };
01325
01326     void GroupIndizes(size_t K);
01327
01328     wbarry& groupIndizes_DREF(
01329         wbarry &A, unsigned k1, unsigned k2
01330     ) const {
01331         size_t i, s1=1, s2=1;
01332
01333         if (k1+k2!=SIZE.len) wblog(FL,
01334             "ERR invalid index group %d+%d=%d !?",k1,k2,SIZE.len);
01335
01336         k2=SIZE.len;
01337
01338         for (i=0; i<k1; ++i) { s1*=SIZE[i]; }
01339         for ( ; i<k2; ++i) { s2*=SIZE[i]; }
01340
01341         A.SIZE.init(2); A.SIZE[0]=s1; A.SIZE[1]=s2;
01342
01343         if (sptr!=A.sptr) {
01344             A.INIT2REF(data);
01345         }
01346
01347         return A;
01348     };
01349
01350     void GroupIndizes(unsigned k1, unsigned k2) {
01351         GroupIndizes_DREF(*this,k1,k2);
01352     };
01353
01354     void groupIndizes_P(
01355         const WBINDEX&, int, wbperm&,
01356         size_t* =NULL, size_t* =NULL
01357     ) const;
01358
01359     wbarry& transpose(const char *F, int L, wbarry &A) const;
01360     wbarry& Transpose(const char *F=0, int L=0) {
01361         wbarry X; transpose(F,L,X);
01362         return X.save2(*this);
01363     };
01364
01365     wbarry& flipLR(const char *F, int L, wbarry &A) const;
01366     wbarry& FlipLR(const char *F=0, int L=0);
01367
01368     wbarry& flipUD(const char *F, int L, wbarry &A) const;
01369     wbarry& FlipUD(const char *F=0, int L=0);
01370
01371     wbarry& MatPermute(const wbperm &P, char iflag=0);
01372
01373     wbarry& Permute(const char* s, char iflag=0, char rcpy=0);
01374     wbarry& Permute(const wbperm&, char iflag=0, char rcpy=0);
01375
01376     wbarry& permute(wbarry&, const char* s, char iflag=0) const;
01377     wbarry& permute(wbarry&, const wbperm&, char iflag=0) const;
01378
01379     wbarry& select0(
01380         const WBPERM &, unsigned dim, wbarry&) const;
01381     wbarry& Select0(const UVEC &I, unsigned dim) {
01382         wbarry A(*this); A.select0(I,dim,*this);
01383         return *this;
01384     };
01385
01386     wbarry& select0(
01387         size_t i1, size_t i2, unsigned dim, wbarry &Q
01388     ) const {
01389         if (dim>=SIZE.len) wblog(FL,
01390             "ERR dim out of bounds (%d/%d)",dim+1,SIZE.len);
01391         if (i1>=SIZE[dim] || i2>=SIZE[dim]) wblog(FL,
01392             "ERR index out of bounds (%d,%d/%d)",i1,i2,SIZE[dim]);
01393

```

```

01394         if (i2<i1) {
01395             Q.init(); Q.SIZE=SIZE; Q.SIZE[dim]=0;
01396             return Q;
01397         }
01398
01399         wbindex I; I.Index(i1,i2);
01400         return select0(I,dim,Q);
01401     };
01402
01403     wbararray& selectSqueeze(
01404         wbararray &A, unsigned p,
01405         unsigned dim) const;
01406
01407     T min() const;
01408     T max() const;
01409
01410     T aMin(char zflag=0, size_t *k=NULL) const;
01411
01412     void info(const char* ="") const;
01413     void info(const char *F, int L,
01414         const char* ="", unsigned k=-1, unsigned nlt=0, unsigned nlb=0) const;
01415
01416     void print(
01417         const char *F, int L,
01418         const char *istr="", const char *fmt=""
01419     ) const;
01420
01421     void print(const char *istr="", const char *fmt="") const {
01422         print(0,0,istr,fmt);
01423     };
01424
01425     int printdata(const char *istr="ans", const char *fmt="") const;
01426     void print_ref(const char *bstr=" ***", const char *nl="\n") const;
01427
01428     wbststring toStr() const;
01429
01430     void contractDiag(
01431         unsigned ic, const wbararray &B,
01432         T a, T b, wbararray &C, char Iflag=0, char bflag='N'
01433     ) const;
01434
01435     template<class TB, class TC>
01436     wbararray<TC>& contractMat(
01437         const char *F, int L, unsigned i1,
01438         const wbararray<TB>&M, unsigned i2,
01439         wbararray<TC>&) const;
01440
01441     template<class TB, class TC>
01442     wbararray<TC>& contractMat(
01443         const char *F, int L, unsigned i1,
01444         const wbararray<TB>&M, wbararray<TC> &C
01445     ) const { return contractMat(F,L,i1,M,1,C); };
01446
01447     wbararray& ContractMat(const char *F, int L,
01448         unsigned i1, const wbararray &M, unsigned i2=1
01449     ){ wbararray A;
01450         return contractMat(F,L,i1,M,i2,A).save2(*this);
01451     };
01452
01453     template<class TB, class TC>
01454     wbararray<TC>& contract(const char *F, int L,
01455         const ctrIdx&, const wbararray<TB> &B,
01456         const ctrIdx&, wbararray<TC> &C,
01457         const wbperm &pfinal=wbperm(),
01458         T afac=1, TC cfac=1
01459     ) const;
01460
01461     template<class TB, class TC>
01462     wbararray<TC>& contract(
01463         const char *F, int L, const iTags &ta,
01464         const wbararray<TB> &B, const iTags &tb, wbararray<TC> &C, iTags &tc,
01465         const wbperm &pfinal=wbperm(),
01466         T afac=1, TC cfac=1
01467     ) const;
01468
01469     template<class TB>
01470     wbararray<T>& Contract(
01471         const wbvector<unsigned>&, const wbararray<TB>&,
01472         const wbvector<unsigned>&,
01473         const wbperm &pfinal=wbperm()
01474     );
01475
01476     template<class TB>
01477     wbararray<T>& Contract(
01478         char*, const wbararray<TB>&, char*,
01479         const wbperm &pfinal=wbperm()
01480     );

```

```

01481
01482     template<class TB, class TC>
01483     wbararray<TC>& contract(const char *F, const int L,
01484         const char*, const wbararray<TB>&, const char*, wbararray<TC>&,
01485         const wbperm &pfinal=wbperm()
01486     ) const;
01487
01488     template<class TB, class TC>
01489     wbararray<TC>& contract(
01490         char *i1, const wbararray<TB> &B, char *i2, wbararray<TC> &Q,
01491         const wbperm &pfinal=wbperm()
01492     ) const { return contract(FL,i1,B,i2,Q,pfinal); };
01493
01494     template<class TB, class TC>
01495     wbararray<TC>& contract(
01496         const wbvector<unsigned> &i1, const wbararray<TB> &B0,
01497         const wbvector<unsigned> &i2,
01498         wbararray<TC> &Q0, const wbperm &P
01499     ) const { return contract(FL,i1,B0,i2,Q0,P); };
01500
01501     template<class TB, class TC>
01502     wbararray<TC>& contract(const char *F, int L,
01503         unsigned i1, const wbararray<TB>&B, unsigned i2,
01504         wbararray<TC>&C, const wbperm &pfinal=wbperm()
01505     ) const {
01506
01507         if (!i1 || !i2) wblog(F,L,
01508             "ERR %s() expecting 1-based indices (%d,%d)",FCT,i1,i2);
01509         --i1; --i2;
01510         wbvector<unsigned> I1(1,&i1), I2(1,&i2);
01511
01512         return contract(I1,B,I2,C, pfinal);
01513     };
01514
01515     template<class TB, class TC>
01516     wbararray<TC>& contract(
01517         unsigned i1, const wbararray<TB>&B, unsigned i2,
01518         wbararray<TC>&C, const wbperm &pfinal=wbperm()
01519     ) const { return contract(FL,i1,B,i2,C,pfinal); }
01520
01521     template<class TB, class TC>
01522     wbararray<TC>& comm(
01523         const wbararray<TB>&, wbararray<TC>&, char aflag='N', char bflag='N'
01524     ) const;
01525
01526     template<class TB, class TC>
01527     wbararray<TC>& acomm(
01528         const wbararray<TB>&, wbararray<TC>&, char aflag='N', char bflag='N'
01529     ) const;
01530
01531     wbararray operator+(const wbararray &B) const {
01532         wbararray C;
01533         return plus(B,C);
01534     };
01535
01536     wbararray operator-(const wbararray &B) const {
01537         wbararray C;
01538         return plus(B,C,-1);
01539     };
01540
01541     wbararray operator*(const wbararray &B) const {
01542         wbararray C;
01543         wbvector<unsigned> ica(1), icb(1); ica[0]=1; icb[0]=0;
01544         return contract(FL,ica,B,icb,C);
01545     };
01546
01547     T trace() const;
01548     wbararray& trace(unsigned i1, unsigned i2, wbararray &C) const;
01549     wbararray& trace(const ctrIdx &I1, const ctrIdx &I2, wbararray<T> &C) const;
01550
01551     mxArray* toMx() const { return toMx_Struct(); }
01552     mxArray* toMx_Struct() const;
01553     mxArray* toMx_base() const;
01554
01555     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
01556     void add2MxStruct(mxArray *S, unsigned i, char tflag=0) const;
01557
01558     void put(const char *vname="ans", const char* ws="base") const {
01559         mxArray *a=toMx();
01560         mxPutAndDestroy(FL,a,vname,ws);
01561     };
01562
01563     void put(const char *F, int L,
01564         const char *vname="ans", const char* ws="base"
01565     ) const {
01566         mxArray *a=toMx();
01567         wblog(F,L,"I/O putting array '%s' to %s.",vname,ws);

```

```

01568         mxPutAndDestroy(F,L,a,vname,ws);
01569     };
01570
01571     void getReal(wbarray<double> &R) const;
01572     void getImag(wbarray<double> &I) const;
01573     void Conj();
01574
01575     void set(const wbarray<double> &R, const wbarray<double> &I);
01576
01577     const wbarray& opA(char tflag, wbarray& aux) {
01578         if (!isMatrix()) wblog(FL,
01579             "ERR cannot use opA() with rank-%d object.", SIZE.len);
01580
01581         if (tflag=='N') { return *this; } else
01582         if (tflag=='T' || tflag=='C') {
01583             permute(aux, wperm("2 1")); if (tflag=='C') aux.Conj();
01584             return aux;
01585         }
01586         wblog(FL, "ERR invalid flag %c<%d>", tflag, tflag);
01587         return *this;
01588     };
01589
01590     bool isSym_aux(
01591         const char *file, int line, const char *fct,
01592         const wbarray &B, double eps, double* xref,
01593         const char symflag='s',
01594         const char lflag=1
01595     ) const;
01596
01597     wbvector<size_t> SIZE;
01598
01599     Wb::MTYPE mtype;
01600
01601     Wb::sptr<T> *sptr;
01602
01603     T *data;
01604
01605 protected:
01606
01607     bool isDiag_aux(const T eps, const char* task) const;
01608     bool isDiag_aux(
01609         double *eps,
01610         const char* task) const;
01611
01612     size_t serial_index(const widx_t *I, size_t n) const;
01613
01614 private:
01615
01616     void print_rec (UVEC&, size_t, const char*, const char*) const;
01617
01618     void make2D() {
01619         if (SIZE.len<2) {
01620             SIZE.Resize(2); if (SIZE[0]) { SIZE[1]=1; }
01621         }
01622         else if (SIZE.len>2) wblog(FL,
01623             "ERR make2D() - invalid usage (got rank-%d)",SIZE.len);
01624     };
01625
01626     void MemErrMsg(const char* file, int line, size_t s) {
01627         wblog(file,line, "ERR Out of memory? (%dx%d)", s, sizeof(T));
01628     };
01629
01630     int check_consistency(const char *F=0, int L=0) const {
01631         int e=0;
01632
01633         if (sptr) {
01634             if (!data || sptr->data!=data) { e|=1; }
01635             if (sptr->check_consistency(0)) { e|=2; }
01636             if (!SIZE.len || SIZE.min()==0) { e|=4; }
01637
01638             if (e && F) wblog(F,L,
01639                 "ERR Wb::sptr = %s\n%p / %p (r=%ld, e=%d)",
01640                 STR_(sptr), sptr, data, SIZE.len, e
01641             );
01642
01643             if (mtype!=sptr->mtype) {
01644                 if (mtype || sptr->mtype!=Wb::MEM_REF) wblog(F,L,
01645                     "WRN Wb::sptr mtype = %s / %s !?",
01646                     Wb::MTYPE_STR[mtype], Wb::MTYPE_STR[sptr->mtype]);
01647             }
01648         }
01649         else {
01650             if (SIZE.len && SIZE.min()) { e|=8; }
01651             if (data) { e|=16; }
01652
01653             if (e && F) wblog(F,L,
01654                 "ERR Wb::sptr inconsistency: %p / %p",sptr,data

```

```

01655         );
01656     }
01657     return e;
01658 };
01659
01660 void DELETE_DATA(char mflag=0) {
01661     if (sptr) { check_consistency(FL);
01662         if (sptr->rm_dref(mflag)) {
01663             WB_DELETE_1(sptr);
01664         }
01665         else { sptr=NULL; }
01666         data=NULL;
01667
01668         mtype=Wb::MEM_DEF;
01669     }
01670     else if (data) wblog(FL, "ERR %s() %p / %p !?", FCT, data, sptr);
01671 };
01672
01673 wbarrray& NEW_DATA(size_t n, const T* d0=NULL, char ref=0, char init=1) {
01674     if (!n) {
01675         DELETE_DATA(); return *this;
01676     }
01677
01678     if (!ref) {
01679         if (!sptr) WB_NEW_1(sptr);
01680
01681         if (mtype && WBLOG_MMEX) { wblog(FL,
01682             " | %s ... %p -> %s", Wb::MTYPE_STR[mtype],
01683             this, Wb::size2Str(n*sizeof(T)).data);
01684         }
01685
01686         if (d0 || init)
01687             { data=sptr->malloc_data(n,d0,mtype); }
01688         else { data=sptr->malloc_base(n, 0,mtype); }
01689     }
01690     else {
01691         RENEW_SPTR();
01692         data=(sptr->init2ref(n,d0));
01693     }
01694     return *this;
01695 };
01696
01697 void RENEW_SPTR() {
01698     if (sptr) { DELETE_DATA(); }
01699     WB_NEW_1(sptr);
01700 };
01701
01702 wbarrray& INIT2REF(const T *d0, const wvector<size_t> *S=NULL) {
01703     if (S) { SIZE=(*S); }
01704     size_t n=SIZE.prod(0);
01705
01706     if (d0) { if (!sptr || !isRef() || d0!=data) {
01707         RENEW_SPTR();
01708         data=(sptr->init2ref(n,d0));
01709     } }
01710     else if (n) wblog(FL,
01711         "ERR data/size inconsistency: %p / %p => %s", data, d0, SSTR_(this));
01712
01713     return *this;
01714 };
01715
01716 wbarrray& NEW(const wvector<size_t> &S, const T* d0=NULL) {
01717     SIZE=S;
01718     NEW_DATA(SIZE.prod(0), d0);
01719     return *this;
01720 };
01721
01722 wbarrray& NEW(const wvector<size_t> &S, size_t n, const T* d0=NULL) {
01723     SIZE.init(n,S); NEW_DATA(SIZE.prod(0), d0);
01724     return *this;
01725 };
01726
01727 T& dummy() const {
01728     static T d; memset(&d,0,sizeof(T));
01729     return d;
01730 };
01731
01732 };
01733
01734 template <class T>
01735 class wperm_helper {
01736 public:
01737     wperm_helper()
01738         : rk_(0), rk(0), numel(0), sz(NULL), stride(NULL),

```

```

01742     l1(0), l2(0), m_blk(0) {};
```

```

01743
01744     wbperm_helper(const wbvector<widx_t> &sz_, const wbperm &perm)
01745     : rk_(sz_.len), rk(0), numel(0),
01746       sz( new widx_t [2*rk_] ),
01747       stride(sz+rk_), l1(0), l2(0), m_blk(0) {
01748
01749         unsigned i,j;
01750
01751         widx_t stride_[rk_+1]; stride_[0]=1;
01752         for (i=1; i<=rk_; ++i) { stride_[i] = stride_[i-1]*sz_[i-1]; }
01753         numel = stride_[i-1];
01754
01755         if (rk_!=perm.len) wblog(FL,
01756             "ERR %s() rank mismatch (r=%d/%d)",FCT,rk_,perm.len);
01757         if (!numel) { rk=1; return; }
01758
01759         for (i=0; i<rk_; ++i) { j=perm[i];
01760             sz[i] = sz_[j];
01761             stride[i] = stride_[j];
01762         }
01763
01764         for (i=1; i<rk_; ++i) {
01765             if (sz[rk]==1) {
01766                 sz[rk] *= sz[i];
01767                 stride[rk] = stride[i];
01768             }
01769             else if (sz[i]==1 || stride[i] == stride[rk] * sz[rk]) {
01770                 sz[rk] *= sz[i];
01771             }
01772             else if ((++rk)<i) {
01773                 sz[rk] = sz[i];
01774                 stride[rk] = stride[i];
01775             }
01776         }
01777
01778         if (++rk>=2) { widx_t s0;
01779             l1=(stride[0]==1 ? 1 : 0);
01780
01781             s0=(l1 ? sz[0] : 1);
01782             for (i=0; i<rk; ++i) { if (stride[i]==s0) { l2=i; break;}}
01783
01784             if (l2<=l1) {
01785                 wblog(FL,"ERR %s() failed to identify l2=%d "
01786                     "(rk=%d, l1=%d, s0=%ld)",FCT,l2,rk,l1,s0);
01787             }
01788         }
01789
01790         m_blk = wbsys::getCacheLineSize()/sizeof(T);
01791     };
01792
01793     ~wbperm_helper () {
01794         if (sz) { delete [] sz; sz=0; }
01795     };
01796
01797     void permute(const T *src, T *dest, int np=1) const;
01798
01799     wbperm_helper& save2(wbperm_helper &X) {
01800         X.rk_=rk_; X.rk=rk; X.numel=numel; rk_=rk=0;
01801         X.sz=sz; X.stride=stride; sz=stride=NULL;
01802         X.l1=l1; X.l2=l2; X.m_blk=m_blk;
01803         return X;
01804     };
01805
01806     mxArray* toMx() const;
01807
01808     unsigned rk_;
01809     unsigned rk;
01810     widx_t numel;
01811     widx_t *sz;
01812     widx_t *stride;
01813
01814     unsigned l1, l2;
01815
01816     unsigned m_blk;
01817 };
01818
01819 template <class T>
01820 void wbarry_permute__(T *B_data,
01821     const wbarry<T> &A, const wbvector<wperm_t> &P, int np=1
01822 );
01823
01824 template <class T>
01825 void wbarry_permute__(T *B_data,
01826     const wbarry<T> &A, const wbvector<wperm_t> &P, int np
01827 ){
01828     wbperm_helper<T> PH(A.SIZE,P);

```



```

01829     PH.permute(A.data,B_data,np);
01830 };
01831
01832 #ifdef QS_USING_HPTT
01833
01834 template <>
01835 void wbararray_permute__( double *B_data,
01836     const wbararray<double> &A, const wbvector<wperm_t> &P_, int np
01837 ){
01838     wbvector<int> Sz(A.SIZE), P(P_);
01839
01840     auto plan = hptt::create_plan(P.data, Sz.len,
01841         1., A.data, Sz.data, NULL,
01842         0., B_data, NULL,
01843         hptt::ESTIMATE,np>=1 ? np : 1);
01844     plan->execute();
01845 };
01846
01847 template <>
01848 void wbararray_permute__(wbcomplex *B_data,
01849     const wbararray<wbcomplex> &A, const wbvector<wperm_t> &P_, int np
01850 ){
01851     wbvector<int> Sz(A.SIZE), P(P_);
01852     auto plan = hptt::create_plan(P.data, Sz.len,
01853         1., (hptt::DoubleComplex*)A.data, Sz.data, NULL,
01854         0., (hptt::DoubleComplex*)B_data, NULL,
01855         hptt::ESTIMATE,np>=1 ? np : 1);
01856     plan->execute();
01857 };
01858
01859 #endif
01860
01861 class tensor_ref_ {
01862
01863 public:
01864     tensor_ref_() : Ar(NULL), Az(NULL) {};
01865     tensor_ref_(const tensor_ref_ &b) : Ar(b.Ar), Az(b.Az) {};
01866
01867     tensor_ref_& init(const tensor_ref_ &b) {
01868         Ar=b.Ar; Az=b.Az;
01869         return *this;
01870     };
01871     tensor_ref_& operator=(const tensor_ref_ &b) { return init(b); };
01872
01873     bool operator==(const wbararray<double> *A) const {
01874         return (Ar && Ar==A); };
01875     bool operator==(const wbararray<wbcomplex> *A) const {
01876         return (Az && Az==A); };
01877
01878     tensor_ref_& init(const wbararray<double> &A) { Ar=&A; Az=NULL;
01879         return *this; };
01880     tensor_ref_& init(const wbararray<wbcomplex> &A) { Az=&A; Ar=NULL;
01881         return *this; };
01882
01883     explicit operator bool() const { return (Ar || Az); };
01884     bool operator! () const { return (!Ar && !Az); };
01885
01886     int check(const char *F=NULL, int L=0) const;
01887     int rank(unsigned r) const;
01888
01889     bool sameSize(const wbvector<size_t> &S) const;
01890
01891     double xflops() const { return (Az ? 2 : 1); };
01892
01893     wstring sizeStr() const;
01894     const wbvector<size_t>* getSIZE() const;
01895
01896     template<class TC>
01897     void contract(
01898         const char *F, int L, const ctrIdx &ia,
01899         const tensor_ref_ &b, const ctrIdx &ib, wbararray<TC> &C) const;
01900
01901     const wbararray<double> *Ar;
01902     const wbararray<wbcomplex> *Az;
01903
01904 protected:
01905 private:
01906 };
01907
01908 class tensorRef_ {
01909 public:
01910     tensorRef_ () : id(0), conj(0) {};
01911
01912     tensorRef_ (const tensorRef_ &B) : id(B.id), conj(B.conj), R(B.R) {
01913         it=B.it; S=B.S; ID=B.ID; };
01914
01915     template<class T>

```

```

01916     tensorRef_(const wbararray<T> &A, const iTags *t_=NULL) {
01917         init(A,t_); }
01918
01919     tensorRef_& operator=(const tensorRef_ &B) { return init(B); };
01920
01921     tensorRef_& init(const tensorRef_ &B) {
01922         id=B.id; conj=B.conj; R=B.R; it=B.it; S=B.S; ID=B.ID;
01923         return *this;
01924     };
01925
01926     tensorRef_& save2(tensorRef_ &B) {
01927         B.id=id; B.conj=conj; B.R=R;
01928         S.save2(B.S); it.save2(B.it); ID.save2(B.ID);
01929         return B;
01930     };
01931
01932     template<class T>
01933     tensorRef_& init(const wbararray<T> &A, const iTags *t_=NULL, char c=0) {
01934         R.init(A);
01935         S.init2ref(A.SIZE); ID.init(); id=0; conj=c;
01936
01937         if (!t_) { it.init(); }
01938         else {
01939             if (t_>len!=S.len) { unsigned r=t_>len;
01940                 if (r>S.len || r<2 || (r==2 && A.numOM(r)!=1)) { wblog(FL,
01941                     "ERR %s() unexpected rank r=%d/%d (%s; %s)",
01942                     FCT, r,S.len, SSTR(A), STR_(t_));
01943                 }
01944             }
01945             it.init(*t_);
01946         }
01947         return *this;
01948     };
01949
01950     size_t numel() const { return S.prod(0); };
01951
01952     char check(const char* F=NULL, int L=0) const;
01953     char Check(const char* F, int L, unsigned &id_);
01954
01955     int UpdateItagsCtr(const char *F, int L,
01956         const ctrIdx &ia, tensorRef_ &b,
01957         const ctrIdx &ib, const char *tag
01958     ) {
01959         int q=it.UpdateItagsCtr(F,L,ia,b.it,ib,tag);
01960
01961         conj=ia.conj;
01962         b.conj=ib.conj;
01963
01964         return q;
01965     };
01966
01967     template<class T>
01968     int contract(const char *F, int L,
01969         const tensorRef_ &b, tensorRef_ &c, double *flops,
01970         wbararray<T> *C) const;
01971
01972     int contract(const char *F, int L,
01973         const tensorRef_ &b, tensorRef_ &c, double &flops
01974     ) const { return contract(F,L,b,c,&flops, (wbararray<double>*)NULL); }
01975
01976     template<class T>
01977     int contract(const char *F, int L,
01978         const tensorRef_ &b, tensorRef_ &c, wbararray<T> &C
01979     ) const { return contract(F,L,b,c,NULL, &C); }
01980
01981     wbststring toStr() const {
01982         wbststring s(ID.len ? 128 : 256);
01983         unsigned l, n=s.len;
01984
01985         l=snprintf(s.data,s.len,"%-16s", SSTR(S)); if (l<n) {
01986             l+=snprintf(s.data+l,n-l,ID.len ? "%-24s " : "%s",STR(it)); }
01987         for (unsigned i=0; i<ID.len && l<n; ++i) {
01988             l+=snprintf(s.data+l,n-l,"%s%d.%d",i?" ":"",ID[i]>>8,ID[i]&255); }
01989
01990         if (l>=n) wblog(FL,"ERR %s() "
01991             "string out of bounds (%d/%d)%N%N%N%N",FCT,l,n,s.data);
01992         return s;
01993     };
01994
01995     const tensorRef_& print(const char* istr=NULL, char vflag=1) const {
01996         char q=(vflag ? check() : 0);
01997         PRINTF("%s %-50s%\n", istr? istr: "          ", STR_(this),
01998             q ? (q&1 ? "empty data" : "length mismatch") : "");
01999         return *this;
02000     };
02001
02002     unsigned id;

```

```

02003
02004     char conj;
02005
02006     tensor_ref_ R;
02007
02008     iTags it;
02009     wbvector<size_t> S;
02010
02011     wbvector<unsigned> ID;
02012
02013     protected:
02014     private:
02015 };
02016
02017 class tensorRefs : public wbvector<tensorRef_> {
02018     public:
02019         tensorRefs(unsigned l=0) : wbvector<tensorRef_>(l) {};
02020
02021         char check(const char *F, int L) const {
02022             char q=0;
02023             for (unsigned i=0; i<len; ++i) { q|=data[i].check(F,L); }
02024             return q;
02025         };
02026
02027         char Check(const char *F, int L) {
02028             char q=check(F,L);
02029             if (!q) {
02030                 unsigned i=0, id=0;
02031                 for ( ; i<len; ++i) { if (id<data[i].id) { id=data[i].id; } }
02032                 for (i=0; i<len; ++i) { data[i].Check(F_L,id); }
02033             }
02034             return q;
02035         };
02036
02037         void print() { print(NULL,0); }
02038         void print(const char *F, int L) const;
02039
02040         double getOptimalCtrOrder(
02041             const char *F, int L, wbperm &P,
02042             wbperm *pfin=NULL, char useP=0, char vflag=0);
02043
02044     protected:
02045     private:
02046 };
02047
02048 template <class T>
02049 class wbarrRef {
02050     public:
02051         wbarrRef() : dc(0.), fac(1.), tflag(0), D(NULL) {};
02052
02053         wbarrRef(const wbarrRef<T> &r) {
02054             memcpy(this, &r, sizeof(r));
02055         };
02056
02057         wbarrRef<T>& operator=(const wbarrRef<T> &r) {
02058             memcpy(this, &r, sizeof(r));
02059             return *this;
02060         };
02061
02062         void print(const char *vname="ans") {
02063             printf("\nwbarRef %s\n", vname);
02064             printf("... wbararray : 0x%lX (%s%s)\n", D,
02065                 D && D->SIZE.len==1 ? "vec of length ":"", D? SSTR_(D): "");
02066             printf("... dc      : %g\n", dc);
02067             printf("... fac      : %g\n", fac);
02068             printf("... tflag    : %d\n", tflag);
02069         };
02070
02071         double dc;
02072         double fac;
02073         bool tflag;
02074
02075         wbarray<T> *D;
02076
02077     protected:
02078     private:
02079 };
02080
02081 namespace NR {
02082
02083 template<class T> inline
02084 T pythag(T a, T b) {
02085     if (a<0) a=-a;
02086     if (b<0) b=-b;
02087 }
02088
02089

```

```

02090     if (a>b)
02091     { T x=b/a; return a*Wb::sqrt(T(1)+x*x); } else if (b==0) return 0;
02092     else { T x=a/b; return b*Wb::sqrt(T(1)+x*x); }
02093 };
02094
02095 template<class T> inline
02096 T sign(const T &a, const T &b) {
02097     return (b>=0 ? (a>=0 ? a : -a) : (a>=0 ? -a : a));
02098 };
02099
02100 };
02101
02102 template<class T>
02103 wbararray<T> TestContract(
02104     const wbararray<T> &A, C_UVEC i1,
02105     const wbararray<T> &B, C_UVEC i2, const wbperm &pfinal=wbperm());
02106
02107 template<class T>
02108 void cell2mat(
02109     const wbMatrix< wbarrRef<T> > &C,
02110     wbararray<T> &M, WBINDEX &D1, WBINDEX &D2
02111 );
02112
02113 template<class T> inline
02114 size_t wbararray<T>::serial_index(const widx_t *I, size_t len) const {
02115
02116     size_t i,k, r=SIZE.len, l=r-1, nz=r-len;
02117
02118     if (len==0) return -1;
02119     if (!r || len>r || (len && data==NULL)) wblog(FL,
02120         "ERR %s[%s] having %s array",FCT,
02121         STR(WBINDEX(len,I)+1), SSTR_(this));
02122
02123 #ifdef CHECK_ELEMENT_RANGE
02124     for (i=0; i<len; i++) if (I[i]>=SIZE[i+nz]) {
02125         wblog(FL,"ERR %s[%s] index out of bounds (%s)",FCT,
02126             STR(WBINDEX(len,I)+1), SSTR_(this));
02127     }
02128 #endif
02129
02130     for (k=I[l-nz], i=l-1; i<r; i--) {
02131         k = k+SIZE[i] + ((i>=nz) ? I[i-nz] : 0);
02132     }
02133
02134 #ifdef CHECK_ELEMENT_RANGE
02135     if (k>=numel()) wblog(FL,
02136         "ERR %s() index out of bounds (%d; %s)",FCT,k,SSTR_(this));
02137 #endif
02138
02139     return k;
02140 };
02141
02142 template<class T>
02143 mxArray* wbararray<T>::toMx_Struct() const {
02144
02145     size_t i,dim1,dim2,r=SIZE.len, s=numel();
02146     WBINDEX I(r);
02147     T A;
02148
02149     if (r==2) { dim1=SIZE[0]; dim2=SIZE[1]; } else
02150     if (r==1) { dim1=SIZE[0]; dim2=(dim1 ? 1 : 0); } else
02151     if (r==0) { dim1=dim2=0; }
02152     else wblog(FL,"ERR %s() not implemented yet for rank>2 (%d)",SIZE.len);
02153
02154     mxArray *S=A.mxCreateStruct(dim1,dim2);
02155
02156     for (i=0; i<s; ++i) data[i].add2MxStruct(S,i);
02157
02158     return S;
02159 };
02160
02161 template<class T>
02162 mxArray* wbararray<T>::mxCreateStruct(unsigned m, unsigned n) const {
02163     return mxCreateCellMatrix(m,n);
02164 }
02165
02166 template<class T>
02167 void wbararray<T>::add2MxStruct(mxArray *S, unsigned i, char tflag) const {
02168
02169     if (tflag) { size_t s=0;
02170         if (S==NULL || (s=mxGetNumberOfElements(S))<1 || i>=s) wblog(FL,
02171             "ERR %s() must follow mxCreateStruct()\n%lx, %d/%d",FCT,S,i+1,s);
02172     }
02173
02174     mxSetCell(S,i,toMx());
02175 };
02176

```

```

02177 template <class T> inline
02178 mxArray* wbararray<T>::toMx_base() const {
02179
02180     mxArray *a=NULL;
02181
02182     if (sptr && sptr->mtype) { check_consistency(FL);
02183         if (sptr->mtype==Wb::MEX_RETURN) {
02184             size_t l=P2X.BUF.size();
02185             a=P2X.Return(0,0,data);
02186
02187             if (a) {
02188                 size_t s1=numel(), s2=mxGetNumberOfElements(a);
02189                 if (s1==s2) {
02190                     if (WBLOG_MMEX) wblog(FL,
02191                         "2Mx %s (PSX @ %d)",STR_(sptr),l);
02192                     mxSetDimensions(a,SIZE.data,SIZE.len);
02193                 }
02194                 else wblog(FL,
02195                     "TST %s() numel=%ld/%ld %p/%p (PSX @ %ld)",
02196                     FCT,s1,s2,mxGetData(a),data,l
02197                 );
02198             }
02199             else {
02200                 wblog(FL,"WRN sptr=%s not / no longer in P2X @ %ld",STR_(sptr),l);
02201             }
02202         }
02203         else {
02204             if (sptr->mtype!=Wb::MEM_REF)
02205                 wblog(FL,"TST %s() got %s",FCT,Wb::MTYPE_STR[sptr->mtype]);
02206         }
02207     }
02208
02209     if (!a) { a=cpyRange2Mx(data,SIZE); }
02210
02211     return a;
02212 };
02213
02214 template <> mxArray* wbararray<unsigned>::toMx() const { return toMx_base(); }
02215 template <> mxArray* wbararray<int>::toMx() const { return toMx_base(); }
02216 template <> mxArray* wbararray<char>::toMx() const { return toMx_base(); }
02217 template <> mxArray* wbararray<double>::toMx() const { return toMx_base(); }
02218 template <> mxArray* wbararray<wbcomplex>::toMx() const { return toMx_base(); }
02219
02220 template<class T>
02221 wbararray<T>& wbararray<T>::resize(
02222     const wbvector<size_t> &S, wbararray<T> &B) const {
02223
02224     if (SIZE.len!=S.len) wblog(FL,
02225         "ERR length mismatch (%d/%d)",SIZE.len,S.len);
02226     if (this==&B) wblog(FL,"WRN %s() got call onto self",FCT);
02227
02228     if (S==SIZE) { B=(*this); return B; }
02229     else if (!S) { B.init(); return B; }
02230     else { B.NEW(S); }
02231
02232     size_t i,j,k, r=S.len, l=r-1, *s1=SIZE.data, *s2=B.SIZE.data;
02233     WBINDEX I(r); T *b=B.data;
02234
02235     size_t s[r];
02236     for (i=0; i<r; ++i) { s[i] = MIN( S[i], SIZE[i] ); }
02237
02238     while (I[l]<s[l]) {
02239         for (i=j=I[l], k=l-1; k<r; --k) {
02240             i = i*s1[k] + I[k];
02241             j = j*s2[k] + I[k];
02242         }
02243
02244         b[j]=data[i];
02245
02246         k=0; I[0]++;
02247         while (I[k]>=s[k] && k<l) { I[k]=0; ++I[++k]; }
02248     }
02249
02250     return B;
02251 };
02252
02253 template<class T>
02254 wbararray<T>& wbararray<T>::ResizeM(unsigned r, const wbvector<size_t> &Sm) {
02255
02256     unsigned i=0, m=Sm.len;
02257     wbvector<size_t> S(SIZE);
02258     wbararray<T> X;
02259
02260     if (r+m!=S.len) wblog(FL,
02261         "ERR %s() size mismatch %d+%d / %d",FCT,r,m,S.len);
02262     if (int(r)<=2 && Sm.anyUnequal(1)) wblog(FL,
02263         "ERR %s() unexpected OM %s",FCT,SSTRM_(r,S.data,m,Sm.data));

```

```

02264
02265     for (; i<m; ++i) {
02266         if (S[r+i]>Sm[i]) wblog(FL,"WRN %s() "
02267             "reducing OM (%d/%d+%d): %d->%d",FCT,i+1,r,m,S[r+i],Sm[i]);
02268         S[r+i]=Sm[i];
02269     }
02270     return resize(S,X).save2(*this);
02271 };
02272
02273 template<class T>
02274 wbararray<T>& wbararray<T>::Enlarge(
02275     size_t dim,
02276     size_t D,
02277     size_t istart
02278 ){
02279     WBINDEX S(SIZE);
02280     wbararray<T> X;
02281     wbIndex I(S);
02282
02283     if (!dim || dim>S.len) wblog(FL,
02284         "ERR %s() dimension out of bounds (%d/%d)",FCT,dim,S.len);
02285     if (D<S[--dim]) wblog(FL,
02286         "ERR %s() envalid new size (%d: %d/%d)",FCT,dim+1,S[dim],D);
02287     if (istart+S[dim]>=D) wblog(FL,
02288         "ERR %s() index out of bounds (%d+%d/%d)",FCT,istart+1,S[dim],D);
02289
02290     S[dim]=D; X.init(S);
02291     if (istart) {
02292         for (size_t i=0; ++I; ++i) {
02293             I.data[dim]+=istart; X(I)=data[i];
02294             I.data[dim]-=istart;
02295         }
02296     } else { for (size_t i=0; ++I; ++i) X(I)=data[i]; }
02297
02298     X.save2(*this);
02299     return *this;
02300 };
02301
02302 template<class T>
02303 wbararray<T>& wbararray<T>::Append2(
02304     const char *F, int L,
02305     wbararray<T> &B,
02306     unsigned dim
02307 ){
02308     size_t i=0, D=0, ib=0;
02309
02310     if (B.isEmpty()) { save2(B); return B; }
02311
02312     if (SIZE.len!=B.SIZE.len) wblog(F_L,
02313         "ERR %s() incompatible objects (%s <> %s)",FCT,SSTR_(this),SSTR(B));
02314     if (dim>=SIZE.len) wblog(F_L,
02315         "ERR %s() dimension out of bounds (%d/%d)",FCT,dim+1,SIZE.len);
02316
02317     for (i=0; i<SIZE.len; ++i) {
02318         if (i!=dim) { if (SIZE[i]!=B.SIZE[i]) wblog(F_L,
02319             "ERR %s() size mismatch (%s <> %s @ %d/%d; %d)",
02320             FCT,SSTR_(this),SSTR(B),i+1,SIZE.len,dim+1);
02321         }
02322         else { D=SIZE[i]+B.SIZE[i]; ib=B.SIZE[i]; }
02323     }
02324
02325     WBINDEX S(SIZE); S[dim]=D;
02326     wbIndex Ia(SIZE), Ib(B.SIZE);
02327     size_t &ia=Ia.data[dim];
02328     wbararray<T> X(S);
02329
02330     for (i=0; ++Ib; ++i) X(Ib)=B.data[i];
02331     for (i=0; ++Ia; ++i) {
02332         ia+=ib; X(Ia)=data[i];
02333         ia-=ib;
02334     }
02335
02336     X.save2(B); init();
02337     return B;
02338 };
02339
02340 template<class T>
02341 void wbararray<T>::Append2(
02342     const char *F, int L,
02343     wbararray<T> &B, unsigned dim,
02344     wbararray<double> &W, double eps
02345 ){
02346     size_t i,n, D=0, ib=0;
02347     const double *w=W.data;
02348
02349     if (this->isEmpty()) return;
02350     if (dim>=SIZE.len) wblog(F_L,

```

```

02351         "ERR %s() dimension out of bounds (%d/%d)", FCT, dim+1, SIZE.len);
02352
02353     if (B.isEmpty()) {
02354         WBINDEX S(SIZE); S[dim]=0;
02355         B.init(S);
02356     }
02357
02358     if (SIZE.len!=B.SIZE.len) wblog(F_L,
02359         "ERR %s() incompatible objects (%s <> %s)",
02360         FCT, SSTR_(this), SSTR(B));
02361     for (i=0; i<SIZE.len; ++i) { if (i!=dim) {
02362         if (SIZE[i]!=B.SIZE[i]) wblog(F_L,
02363             "ERR %s() size mismatch (%s <> %s; %d; %d/d)",
02364             FCT, SSTR_(this), SSTR(B), dim+1, i+1, SIZE.len);
02365     }}
02366
02367     wbindex J(SIZE[dim]);
02368     size_t m=0, *j=J.data; n=W.numel();
02369
02370     if (!W.isVector() || n!=SIZE[dim]) wblog(F_L,
02371         "ERR %s() invalid weights W (%s <> %s @ %d)",
02372         FCT, SSTR(W), SSTR_(this), dim+1);
02373     for (i=0; i<n; ++i) { if (w[i]<eps) { j[m++]=i; }}
02374
02375     if (!m) return;
02376     if (m==n) { Append2(F_L, B, dim); return; }
02377
02378     J.len=m;
02379     D=m+B.SIZE[dim]; ib=B.SIZE[dim];
02380
02381     WBINDEX S(SIZE); S[dim]=m;
02382     wbindex Ix(S), Ib(B.SIZE); S[dim]=D;
02383     wbindex Ia(S.len), J2;
02384     wbararray<T> X(S);
02385
02386     size_t &ia=Ia.data[dim], &ix=Ix.data[dim];
02387
02388     for (i=0; ++Ib; ++i) { X(Ib)=B.data[i]; }
02389     for (i=0; ++Ix; ++i) { Ia.setp(Ix.data); ia=j[ia];
02390         ix+=ib; X(Ix)=(*this)(Ia);
02391         ix-=ib;
02392     }
02393
02394     X.save2(B);
02395     J.invert(SIZE[dim], J2); Select0(J2, dim);
02396 };
02397
02398 template<class T>
02399 wbararray<T>& wbararray<T>::init(
02400     const char *F, int L, const mxArray *a, char ref, char vec) {
02401
02402     if (!a) { init(); return *this; }
02403
02404     unsigned r=mxGetNumberOfDimensions(a);
02405     const size_t *sz=mxGetDimensions(a);
02406
02407     if (!mxIsNumChar(a)) wblog(F, L,
02408         "ERR got rank-%d array %s", r, mxGetClassName(a));
02409
02410     wvector<size_t> S(r, sz); init();
02411
02412     if (vec && r==2) {
02413         if (S[1]==1) { S.len=1; } else
02414         if (S[0]==1) { S[0]=S[1]; S.len=1; }
02415     }
02416
02417     Mx::Array<T> A(a);
02418
02419     if (ref && A.data) {
02420         INIT2REF((T*)A.data, &S);
02421     }
02422     else {
02423         init_bare(S); A.copy_to(data);
02424         if (ref && WBLOG_TCAST) wblog(F, L,
02425             "WRN ignoring ref (%s)", A.toStrT().data);
02426     }
02427
02428     if (data && !Wb::is_finite(data, 1)) wblog(F, L,
02429         "WRN %s() encountered nan or inf in mex input data", FCT);
02430
02431     return *this;
02432 };
02433
02434 template<class T>
02435 wbararray<T>& wbararray<T>::init(
02436     const char *F, int L, const mxArray *a, wbpperm &P
02437 ) {

```

```

02438     if (!a || mxIsEmpty(a)) { init(); return *this; }
02439     if (P.isIdentityPerm()) { return init(F,L,a); }
02440
02441     unsigned i, r=mxGetNumberOfDimensions(a);
02442     const size_t *sz=mxGetDimensions(a);
02443     size_t len=1;
02444
02445     if (!mxIsNumeric(a)) wblog(F,L,
02446         "ERR got rank-%d array %s",r,mxGetClassName(a));
02447     if (!r || !sz || P.len!=r) wblog(FL,
02448         "ERR %s() got %d/%d @ sz=%p !?",FCT,P.len,r,sz);
02449
02450     SIZE.init(r);
02451     for (i=0; i<r; ++i) { SIZE[i]=sz[P[i]]; len*=sz[i]; }
02452
02453     NEW_DATA(len,NULL,0,0);
02454
02455     Mx::Array<T>(a).copyTo(data,P);
02456
02457     return *this;
02458 };
02459
02460 template<class T> inline
02461 void wbararray<T>::initTst() {
02462
02463     size_t i,j,k, r=SIZE.len, l=r-1, s=numel(), x;
02464     WBINDEX I(r);
02465
02466     for (i=0; i<s; i++) {
02467         for (j=I[l], k=l-1; k<r; k--) j = j*SIZE[k] +I[k];
02468         for (x=I[0]+1, k=1; k<r; k++) x = x*10 + (I[k]+1);
02469
02470         data[j]=(T)x;
02471
02472         k=0; I[0]++;
02473         while (I[k]>=SIZE[k] && k<l) { I[k]=0; ++I[++k]; }
02474     }
02475 }
02476
02477 template<class T> inline
02478 wbararray<T>& wbararray<T>::initIdentity(size_t d, char dflag, T dval) {
02479
02480     if (dflag) { init(d).set(dval); }
02481     else {
02482         init(d,d);
02483         for (size_t i=0; i<d; ++i) {
02484             data[i+d*i]=dval;
02485         }
02486     }
02487     return *this;
02488 };
02489
02490 template<class T> inline
02491 wbararray<T>& wbararray<T>::initIdentity(
02492     const WBINDEX &S, char dflag
02493 ){
02494     size_t i,d,m=S.len/2;
02495     size_t const* const s=S.data;
02496
02497     if (S.len%2) wblog(FL,
02498         "ERR %s() requires even rank object (%s)",FCT,SSTR_(this));
02499     for (i=0; i<m; i++) if (s[i]!=s[i+m]) wblog(FL,
02500         "ERR %s() requires symmetric object (%s)",FCT,SSTR_(this));
02501     if (m>1 && dflag) wblog(FL,
02502         "ERR %s() rank-2 tensor required with dflag (%s)",FCT,SSTR_(this));
02503
02504     if (!m) { init(); return *this; }
02505     if (dflag) { init(s[0]).set(T(1)); return *this; }
02506     if (m==1 && s[0]==1) { init(S).set(T(1)); return *this; }
02507
02508     init(S); for (d=s[0], i=1; i<m; i++) d*=s[i];
02509     for (i=0; i<d; i++) data[i+d*i]=1;
02510
02511     return *this;
02512 };
02513
02514 template<class T> inline
02515 wbararray<T>& wbararray<T>::Reduce2Id() {
02516     if (SIZE.len==2) {
02517         size_t i, j, *s=SIZE.data; T *x=data;
02518
02519         for (j=0; j<s[1]; ++j) {
02520             for (i=0; i<s[0]; ++i, ++x) { *x = (i==j ? 1 : 0); }
02521         }
02522     else if (SIZE.len) wblog(FL,
02523         "ERR %s() matrix expected (got %s)",FCT,SSTR_(this));
02524

```



```

02525     return *this;
02526 };
02527
02528 template<class T> inline
02529 wbararray<T>& wbararray<T>::initIdentityB(
02530     size_t d1, size_t d2, size_t k
02531 ){
02532     init(d1,d2); if (d1 && d2) {
02533         if (int(k)>=0) {
02534             if (k<d2) {
02535                 size_t j=k, dx=d1+1; T *x=data+j*d1;
02536                 for (; j<d2 && (j-k)<d1; ++j, x+=dx) { x[0]=1; }
02537             }
02538             else wblog(FL,
02539                 "WRN %s() got all-zero block (%dx%d w/ k=%d)",FCT,d1,d2, k);
02540         }
02541         else { k=-k;
02542             if (k<d1) {
02543                 size_t i=k, dx=d1+1; T *x=data+i;
02544                 for (; i<d1 && (i-k)<d2; ++i, x+=dx) { x[0]=1; }
02545             }
02546             else wblog(FL,
02547                 "WRN %s() got all-zero block (%dx%d w/ k=%d)",FCT,d1,d2,-k);
02548         }
02549     }
02550     return *this;
02551 };
02552
02553 template<class T> inline
02554 wbararray<T>& wbararray<T>::initIdentityB3(
02555     size_t d1, size_t d2, size_t D,
02556     size_t i0,
02557     T one
02558 ){
02559     size_t i=0, d12=d1*d2, d12_=d12+1;
02560
02561     if (i0+d12>D) wblog(FL,"ERR %s() "
02562         "index out of bounds (%ld*%ld) x %ld @ %ld",FCT,d1,d2,D,i0+1);
02563
02564     init(d1,d2,D);
02565     for (T *x = data + i0*d12; i<d12; ++i, x+=d12_) { (*x)=one; }
02566
02567     return *this;
02568 };
02569
02570 template<class T> inline
02571 void wbararray<T>::Diag2Vec() {
02572     size_t i,m;
02573     WBINDEX S(1);
02574
02575     if (!isSMatrix()) wblog(FL,
02576         "ERR Calling %s for rank-%d object (%s).",
02577         FCT, SIZE.len, SSTR_(this));
02578
02579     m=S[0]=SIZE[0];
02580     for (i=1; i<m; i++) data[i]=data[i+i*m];
02581
02582     Resize(S);
02583 };
02584
02585 template<class T> inline
02586 wbararray<T>& wbararray<T>::Reduce2Diag(T eps) {
02587
02588     if (!SIZE.len) { return *this; }
02589     else if (!isSMatrix()) wblog(FL,
02590         "ERR %s() symmetric matrix required\n"
02591         "(got rank-%d object; %s)",FCT,rank(),SSTR_(this)
02592     );
02593
02594     unsigned i, j=0, n=SIZE[0];
02595     wbararray<T> X(n,1); T *d0=data;
02596
02597     X.init(n,1);
02598     for (; j<n; ++j, d0+=n) {
02599         X.data[j]=d0[j];
02600         for (i=0; i<n; ++i) { if (i!=j) {
02601             if (Wb::abs(d0[i])>eps) wblog(FL,
02602                 "ERR %s() got off-diagonal matrix element (%g/%g)",
02603                 FCT,double(d0[i]),double(eps)
02604             );
02605         } }
02606     }
02607
02608     return X.save2(*this);
02609 };
02610
02611 template<class T>

```

```

02612 void wbararray<T>::contractDiag(
02613     unsigned ic, const wbararray<T> &B,
02614     T a, T b, wbararray<T> &C, char lflag, char bflag
02615 ) const {
02616
02617     if (B.SIZE.len!=2) wblog(FL,"ERR invalid rank-%d",B.SIZE.len);
02618     if (ic<1 || ic>B.SIZE.len) wblog(FL,
02619         "ERR invalid contraction index (ic=%d)",ic);
02620     if (!strchr("NTCc",bflag)) wblog(FL,
02621         "ERR invalid flag %c<%d>",bflag,bflag);
02622
02623     size_t i,k, s=numel(), r=B.SIZE.len, l=r-1;
02624     size_t n=B.SIZE.min(), m=B.SIZE[0];
02625     WBINDEX I_(r);
02626     wvector<T> h_(n);
02627     wbararray<T> X;
02628
02629     size_t *I=I_.data, *S=S_.data;
02630     T *h=h_.data;
02631
02632     ic--;
02633     if (n!=B.SIZE[ic]) wblog(FL,"ERR dimension mismatch (%d/%d)",n,B.SIZE[ic]);
02634     if (n==0) return;
02635
02636     for (i=0; i<n; i++) h[i]=B.data[i+i*m];
02637     if (bflag=='C' || bflag=='c') h_.Conj();
02638
02639     X=*this; if (a!=1) X*=a;
02640
02641     for (i=0; i<s; i++) {
02642         if (lflag)
02643             X.data[i]/=(b-h[I[ic]]+1E-33);
02644         else X.data[i]*=(b-h[I[ic]]);
02645
02646         k=0; I[0]++;
02647         while (I[k]>=S[k] && k<l) { I[k]=0; ++I[++k]; }
02648     }
02649
02650     if (C.isEmpty()) X.save2(C); else {
02651         if (C.SIZE!=B.SIZE) wblog(FL,"ERR size mismatch in C: %s <> %s",
02652             SSTR_(this), SSTR(C));
02653         C+=X;
02654     }
02655 };
02656
02657 template<class T>
02658 wbararray<T>& wbararray<T>::ExpandDiagonal() {
02659
02660     size_t i,n=numel();
02661     wbararray<T> X;
02662
02663     if (!isVector())
02664         wblog(FL,"ERR %s() got %s array",FCT,SSTR_(this));
02665
02666     save2(X); init(n,n);
02667     for (i=0; i<n; ++i) data[i+n*i]=X.data[i];
02668
02669     return *this;
02670 };
02671
02672 template<class T>
02673 wbararray<T>& wbararray<T>::ExpandDiagonal(unsigned i1, unsigned i2) {
02674
02675     if (i1>=B.SIZE.len || i2>=B.SIZE.len || i1==i2) wblog(FL,
02676         "ERR %s() invalid indices (%d %d; %d)",FCT,i1+1,i2+1,B.SIZE.len);
02677     if (B.SIZE[i1]!=1 && B.SIZE[i2]!=1) wblog(FL,
02678         "ERR %s() array should be diagonal in (%s; %d %d)",
02679         FCT, SSTR_(this), i1+1, i2+1);
02680     if (B.SIZE[i1]==1 && B.SIZE[i2]==1) { return *this; }
02681
02682     if (B.SIZE[i1]!=1) SWAP(i1,i2);
02683
02684     size_t i,j,k,*s0,*s, r=B.SIZE.len, l=r-1;
02685     WBINDEX S=B.SIZE;
02686     wbararray<T> X;
02687     windex I(r);
02688
02689     save2(X); s0=X.SIZE.data;
02690     S[i1]=S[i2]; init(S); s=S.SIZE.data;
02691
02692     for (i=0; I[l]<s0[l]; i++) {
02693         for (j=0,k=l; k<r; k--) {
02694             if (j) { j*=s[k]; }
02695             j += (k!=i1 ? I[k] : I[i2]);
02696         }
02697         data[j] = X.data[i];
02698     }

```

```

02699         k=0; I[0]++;
02700         while (I[k]>=s0[k] && k<1) { I[k]=0; ++I[++k]; }
02701     }
02702     return *this;
02703 };
02704
02705 template<class T>
02706 wbararray<T>& wbararray<T>::Expand2Projector(T eps) {
02707
02708     size_t i,j,m,n=numel();
02709     WBINDEX S(2);
02710     wbvector<char> mark(n);
02711
02712     if (n==0) { if (SIZE.len>1) SIZE[0]=0; return *this; }
02713     if (!isVector()) wblog(FL,
02714         "ERR %s() array must be 1-dim vector (got %s)",
02715         FCT, SSTR_(this)
02716     );
02717
02718     for (m=i=0; i<n; i++) if (Wb::abs(data[i])>eps) { mark[i]=1; m++; }
02719
02720     S[0]=n; S[1]=m; init(S);
02721
02722     for (i=j=0; i<n; i++)
02723         if (mark[i]) { data[i+(j++)*n]=1; }
02724
02725     return *this;
02726 };
02727
02728 template<class T> inline
02729 T wbararray<T>::froNorm2(const wbararray<T> &B) const {
02730     T x=0;
02731
02732     if (!sameSize(B)) wblog(FL,"ERR %s() size mismatch (%s/%s)",
02733         FCT, SSTR_(this), SSTR(B));
02734
02735     for (size_t n=numel(), i=0; i<n; i++) x+=Wb::CONJ(data[i])*B.data[i];
02736     return x;
02737 };
02738
02739 template<class T> inline
02740 T wbararray<T>::norm2() const {
02741     T d=0;
02742     for (size_t n=numel(), i=0; i<n; ++i) d+=data[i]*data[i];
02743     return d;
02744 };
02745
02746 template<> inline
02747 wcomplex wbararray<wcomplex>::norm2() const {
02748     double d=0;
02749
02750     for (size_t n=numel(), i=0; i<n; ++i)
02751         d+=data[i].abs2();
02752
02753     return d;
02754 };
02755
02756 template<class T> inline
02757 T wbararray<T>::hasNorm2(const T &x2) const {
02758     T q2=0; size_t i=0, n=numel();
02759     for (; i<n; ++i) {
02760         if ((q2+=Wb::norm2(data[i]))>x2) { return q2; }
02761     }
02762     return -q2;
02763 };
02764
02765 template<class T> inline
02766 T wbararray<T>::normCol2(size_t k) const {
02767
02768     if (SIZE.len!=2) wblog(FL,
02769         "ERR %s() invalid matrix %s",FCT,SSTR_(this));
02770     if (k>=SIZE[1]) wblog(FL,
02771         "ERR %s() index out of bounds (%d/%d)",FCT,k,SIZE[1]);
02772
02773     size_t i=0, dim1=SIZE[0];
02774     const T *d=data+k*dim1; T d2=0;
02775
02776     for (i=0; i<dim1; ++i) { d2+=Wb::abs2(d[i]); }
02777     return d2;
02778 };
02779
02780 template<class T> inline
02781 wbvector<T>& wbararray<T>::norm2Cols(wbvector<T> &x2_) const {
02782
02783     if (!SIZE.len) { if (x2_.len) { x2_.init(); }; return x2_; }
02784     if (SIZE.len!=2) wblog(FL,
02785         "ERR %s() invalid matrix %s",FCT,SSTR_(this));

```

```

02786
02787     x2_.init(SIZE[1]); {
02788         T* x2=x2_.data; size_t i,j, l=0, dim1=SIZE[0], dim2=SIZE[1];
02789         for (j=0; j<dim2; ++j)
02790             for (i=0; i<dim1; ++i, ++l) { x2[j]+=Wb::norm2(data[l]); }
02791     }
02792     return x2_;
02793 };
02794
02795 template<class T> inline
02796 wbvector<T> wbararray<T>::normCols() const {
02797
02798     wbvector<T> x_; norm2Cols(x_);
02799     if (x_) {
02800         size_t i=0; T *x=x_.data;
02801         for (; i<x_.len; ++i) { x[i]=Wb::sqrt(x[i]); }
02802     }
02803     return x_;
02804 };
02805
02806 template<class T> inline
02807 T wbararray<T>::scalarProd(const wbararray<T> &B) const {
02808     T x=0; size_t i=0, n=numel();
02809
02810     if (!sameSize(B)) wblog(FI,
02811         "ERR %s() size mismatch: %s <> %s",FCT,SSTR_(this),SSTR(B));
02812     for (; i<n; i++) { x+=data[i]*B.data[i]; }
02813     return x;
02814 }
02815
02816 template<> inline
02817 wbcomplex wbararray<wbcomplex>::scalarProd(
02818     const wbararray<wbcomplex> &B
02819 ) const {
02820
02821     wbcomplex x=0; size_t i=0, n=numel();
02822
02823     if (!sameSize(B)) wblog(FI,
02824         "ERR %s() size mismatch: %s <> %s",FCT,SSTR_(this),SSTR(B));
02825     for (; i<n; ++i) { x+=data[i]*B.data[i].conj(); }
02826
02827     return x;
02828 };
02829
02830 template<class T> inline
02831 T wbararray<T>::sum() const {
02832     T x=0;
02833
02834     for (size_t n=numel(), i=0; i<n; i++)
02835         x+=data[i];
02836
02837     return x;
02838 }
02839
02840 template<class T> inline
02841 wbararray<T>& wbararray<T>::sum(
02842     const WBINDEX &K,
02843     wbararray<T>&A
02844 ) const {
02845
02846     size_t i,j,k,q, r=SIZE.len, l=r-1, s=numel();
02847     WBINDEX S=SIZE;
02848     wbvector<char> mark(r);
02849     char *m=mark.data;
02850     wbindex I;
02851
02852     for (k=i=0; i<K.len; i++) {
02853         if (K[i]<SIZE.len) { mark[K[i]]++; k++;
02854             if (mark[K[i]]>1) wblog(FI,
02855                 "ERR %s() index not unique (%s; %d)",
02856                 FCT,STR(K+1),SIZE.len);
02857             }
02858         else wblog(FI,
02859             "ERR %s() index out of bounds (%s; %d/%d)",
02860             FCT, STR(K+1), K[i],SIZE.len);
02861     }
02862
02863     for (k=i=0; i<mark.len; i++) if (!mark[i]) S[k++]=SIZE[i];
02864     if (k) S.len=k;
02865     else { A.init(1,1); A[0]=Wb::addRange(data,s); return A; }
02866
02867     A.init(S); I.init(SIZE.len);
02868     for (q=1; q<r; q--) if (!m[q]) break;
02869
02870     for (i=0; i<s; i++) {
02871         for (j=I[q], k=q-1; k<r; k--) if (!m[k]) j=j*SIZE[k]+I[k];
02872     }

```

```

02873         A.data[j]+=data[i];
02874
02875         k=0; I[0]++;
02876         while (I[k]>=SIZE[k] && k<1) { I[k]=0; ++I[++k]; }
02877     }
02878
02879     return A;
02880 }
02881
02882 template<class T> inline
02883 T wbarrray<T>::trace() const {
02884     T x=0;
02885
02886     if (SIZE.len%2) wblog(FL,
02887         "ERR %s() got object with odd rank %d",FCT,SIZE.len);
02888     if (!isSquare()) wblog(FL,
02889         "WRN %s() of non-square array %s",FCT,SSTR_(this));
02890
02891     if (SIZE.len) {
02892         size_t n=SIZE[0], m;
02893
02894         if (SIZE.len==2) { m=SIZE[1]; }
02895         else {
02896             unsigned i=1, r2=SIZE.len/2; m=SIZE[r2];
02897             for (; i<r2; ++i) {
02898                 n*=SIZE[i];
02899                 m*=SIZE[r2+i];
02900             }
02901         }
02902
02903         if (m>n) { m=n; }
02904         for (size_t i=0; i<m; ++i) { x+=data[i+i*n]; }
02905     }
02906
02907     return x;
02908 };
02909
02910 #endif
02911

```

5.25 wbarrray_blas.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : wbarrray (array class, col-major)
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the license for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_ARRAY_BLAS_CC__
00021 #define __WB_ARRAY_BLAS_CC__
00022
00023 /* ===== */
00024 // NB! call DZGEMM after all checks are made (such as size, rank, ...)
00025 // C = (alpha=afac) * op(A)*op(B) + (beta=cfac)*C
00026 // e.g., also C must already be fully allocated // DZGEMM_C_ALLOCATED
00027 /* ===== */
00028
00029 template<class TA, class TB, class TC >
00030 void DZGEMM(
00031     const wbarrray<TA> &A,
00032     const wbarrray<TB> &B, wbarrray<TC> &C,
00033     const unsigned k,
00034     char aflag='N', char bflag='N',
00035     TA afac=1, TC cfac=0
00036 );
00037
00038 #ifdef QS_USING_MPFR
00039
00040 template<> inline
00041 void DZGEMM(

```

```

00042     const wbararray<Wb::quad> &A,
00043     const wbararray<Wb::quad> &B, wbararray<Wb::quad> &C,
00044     const unsigned k, char aflag, char bflag,
00045     Wb::quad afac, Wb::quad cfac
00046 ){
00047     wblog(FL,"ERR %s() not yet defined for data type Wb::quad");
00048     wblog(FL,"ERR %s() %p %p %p, %d %d %d, %p %p",
00049     FCT,&A, &B, &C, k, aflag, bflag, &afac, &cfac);
00050 };
00051
00052 #endif
00053
00054 template<> inline
00055 void DZGEMM(
00056     const wbararray<double> &A,
00057     const wbararray<double> &B, wbararray<double> &C,
00058     const unsigned k,
00059     char aflag, char bflag,
00060     double afac, double cfac
00061 ){
00062     if (A.SIZE.len!=2 || B.SIZE.len!=2 || C.SIZE.len!=2) wblog(FL,
00063     "ERR %s() requires matrices (got: %s * %s = %s)",
00064     FCT, SSTR(A), SSTR(B), SSTR(C));
00065
00066     unsigned
00067     a1=A.SIZE[0],
00068     b1=B.SIZE[0],
00069     c1=C.SIZE[0], c2=C.SIZE[1];
00070
00071     if (!a1 || !b1 || !c1 || !c2) {
00072         if (A.SIZE[1]==b1 && B.SIZE[1]==c2 && a1==c1) {
00073             if (cfac!=double(1)) C*=cfac;
00074             return;
00075         }
00076         wblog(FL,"ERR %s() got empty matrices (A: %s, B: %s; %c%c)",
00077         FCT,SSTR(A),SSTR(B),aflag,bflag); return;
00078     }
00079
00080 #ifndef WB_CLOCK
00081     unsigned a2=A.SIZE[1], b2=B.SIZE[1];
00082
00083     stat_dgemm.account(a1*a2*(a2==b1 ? b2 : b1));
00084
00085     Wb::Clock clk("dgemm",1);
00086 #endif
00087
00088     dgemm(
00089         aflag, bflag, (pINT)c1, (pINT)c2, (pINT)k,
00090         afac, A.data, (pINT)a1, B.data, (pINT)b1,
00091         cfac, C.data, (pINT)c1
00092     );
00093 };
00094
00095 template<> inline
00096 void DZGEMM(
00097     const wbararray<wbcomplex> &A,
00098     const wbararray<wbcomplex> &B, wbararray<wbcomplex> &C,
00099     const unsigned k,
00100     char aflag, char bflag,
00101     wbcomplex afac, wbcomplex cfac
00102 ){
00103     if (A.SIZE.len!=2 || B.SIZE.len!=2) wblog(FL,
00104     "ERR %s() matrices required (%s * %s)",FCT,
00105     A.sizeStr().data, B.sizeStr().data);
00106
00107     unsigned
00108     a1=A.SIZE[0],
00109     b1=B.SIZE[0],
00110     c1=C.SIZE[0], c2=C.SIZE[1];
00111
00112     if (!a1 || !b1 || !c1 || !c2) {
00113         if (A.SIZE[1]==b1 && B.SIZE[1]==c2 && a1==c1) {
00114             if (cfac!=double(1)) C*=cfac;
00115             return;
00116         }
00117         wblog(FL,"WRN %s() got empty matrices (A: %s, B: %s)",
00118         FCT, A.sizeStr().data, B.sizeStr().data); return;
00119     }
00120
00121 #ifndef WB_CLOCK
00122     unsigned a2=A.SIZE[1], b2=B.SIZE[1];
00123
00124     stat_dgemm.account(a1*a2*(a2==b1 ? b2 : b1));
00125     Wb::Clock clk("zgemm",1);
00126 #endif
00127
00128     zgemm (

```

```

00129     aflag, bflag, (pINT)c1, (pINT)c2, (pINT)k,
00130     afac, A.data, (pINT)a1, B.data, (pINT)b1,
00131     cfac, C.data, (pINT)c1
00132 );
00133
00134 #ifndef WB_CLOCK
00135 { Wb::Clock *clk;
00136   if (clk.stop())
00137       { Wb::Clock("zgeNX",1,0,&Wb::Clocks,&clk,0); }
00138   else { Wb::Clock("zgeNN",1,0,&Wb::Clocks,&clk,0); }
00139
00140   clk.ncall+=2;
00141   clk.tcpu+= (long unsigned)
00142       (double(a1*a2*b2)*(4E-9*CLOCKS_PER_SEC)+1.5);
00143 }
00144 #endif
00145 };
00146
00147 template<> inline
00148 void DZGEMM(
00149     const wbarry<wbcomplex> &A,
00150     const wbarry<double> &B0, wbarry<wbcomplex> &C,
00151     const unsigned k,
00152     char aflag, char bflag,
00153     wbcomplex afac, wbcomplex cfac
00154 ){
00155     wbarry<wbcomplex> B; B.set(B0, wbarry<double>());
00156     DZGEMM(A,B,C,k,aflag,bflag,afac,cfac);
00157 };
00158
00159 template<> inline
00160 void DZGEMM(
00161     const wbarry<double> &A0,
00162     const wbarry<wbcomplex> &B, wbarry<wbcomplex> &C,
00163     const unsigned k,
00164     char aflag, char bflag,
00165     double afac, wbcomplex cfac
00166 ){
00167     wbarry<wbcomplex> A; A.set(A0, wbarry<double>());
00168     DZGEMM(A,B,C,k,aflag,bflag,wbcomplex(afac,0),cfac);
00169 };
00170
00171 template<> inline
00172 void DZGEMM(
00173     const wbarry<double> &A,
00174     const wbarry<double> &B, wbarry<wbcomplex> &C,
00175     const unsigned k,
00176     char aflag, char bflag, double afac, wbcomplex cfac
00177 ){
00178     wbarry<double> C0(C.SIZE);
00179     DZGEMM(A,B,C0,k,aflag,bflag,afac);
00180
00181     C.Plus(C0,double(1),0,cfac);
00182 };
00183
00184 template<class T>
00185 void Wb::VMatProd(
00186     const wbarry<T> &A0, const wbarry<T> &B0, wbarry<T> &C,
00187     char aflag, char bflag, const T afac, const T cfac,
00188     const char cforce
00189 ){
00190     char isvec=0; wbarry<T> A,B;
00191
00192     if (A0.SIZE.len==1) {
00193         isvec++; A0.adjustVec(FL,aflag,A,1);
00194     }
00195     else A.init2ref(A0);
00196
00197     if (B0.SIZE.len==1) {
00198         isvec++; B0.adjustVec(FL,bflag,B,2);
00199     }
00200     else B.init2ref(B0);
00201
00202     Wb::MatProd(A,B,C,aflag,bflag,afac,cfac,cforce);
00203
00204     if (isvec) { C.skipSingletons(); }
00205 };
00206
00207 template<class T>
00208 void Wb::MatVProd(
00209     const wbarry<T> &A, const wbvector<T> &v, wbarry<T> &x,
00210     char aflag, char bflag, const T afac, const T cfac,
00211     const char cforce
00212 ){
00213     wbarry<T> B;
00214     opFlags<T> Aflag(aflag), Bflag(bflag);
00215

```

```

00216     if (Bflag.conj()) { B.init(v.len,1,v.data).Conj(); }
00217     else {
00218         if (Bflag.trans()) wblog(FL,
00219             "WRN %s() ignoring bflag=%c",FCT,bflag);
00220         B.init2ref(v.len,1,v.data);
00221     }
00222
00223     Wb::MatProd(A,B,x,aflag,'N',afac,cfac,cforce);
00224     x.skipSingletons();
00225 };
00226
00227 template<class T>
00228 T Wb::VMatVprod(
00229     const wbararray<T> &v1, const wbararray<T> &M, const wbararray<T> &v2
00230 ){
00231     unsigned i,dim1,dim2;
00232     const T *p1=v1.data, *p2=v2.data, *dd=M.data;
00233     T x=0;
00234
00235     M.getMatSize(FL,dim1,dim2);
00236
00237     if (v1.numel()!=dim1 || v2.numel()!=dim2) wblog(FL,
00238         "ERR %s() severe size mismatch: %s x %s x %s",FCT,
00239         v1.sizeStr().data, M.sizeStr().data, v2.sizeStr().data);
00240
00241     for (i=0; i<dim2; i++, dd+=dim1) {
00242         x+=Wb::overlap(p1,dd,dim1)*p2[i];
00243     }
00244
00245     return x;
00246 };
00247
00248 template<class T>
00249 T Wb::VMatVprod(
00250     const wbvector<T> &v1, const wbararray<T> &M, const wbvector<T> &v2
00251 ){
00252     unsigned i,dim1,dim2;
00253     const T *p1=v1.data, *p2=v2.data, *dd=M.data;
00254     T x=0;
00255
00256     M.getMatSize(FL,dim1,dim2);
00257
00258     if (v1.len!=dim1 || v2.len!=dim2) wblog(FL,
00259         "ERR %s() severe size mismatch: <%d| %s |%d>",
00260         FCT, v1.len, M.sizeStr().data, v2.len);
00261
00262     for (i=0; i<dim2; i++, dd+=dim1) {
00263         x+=Wb::overlap(p1,dd,dim1)*p2[i];
00264     }
00265
00266     return x;
00267 };
00268
00269 template<class TA, class TB, class TC>
00270 wbararray<TC>& Wb::MatProd(
00271     const wbararray<TA> &A, const wbararray<TB> &B, wbararray<TC> &C,
00272     char aflag, char bflag, TA afac, TC cfac,
00273     char cforce
00274 ){
00275     if ((void*)&A==(void*)&C || (void*)&B==(void*)&C) {
00276         wbararray<TC> XC; if (cfac!=TC(0)) XC.init(C);
00277         Wb::MatProd(A,B,XC,aflag,bflag,afac,cfac,cforce);
00278         XC.save2(C); return C;
00279     }
00280
00281     unsigned a1,a2,b1,b2;
00282     char iflag=0;
00283
00284     #ifdef WB_CLOCK
00285         Wb::Clock clk("MatProd",1);
00286     #endif
00287
00288     if (A.rank()!=2 || B.rank()!=2) wblog(FL,
00289         "ERR %s() rank-2 tensors required (%s; %s)",FCT,SSTR(A),SSTR(B));
00290
00291     a1=A.SIZE[0]; a2=A.SIZE[1]; if (aflag!='N') SWAP(a1,a2);
00292     b1=B.SIZE[0]; b2=B.SIZE[1]; if (bflag!='N') SWAP(b1,b2);
00293
00294     if (a2!=b1) wblog(FL,
00295         "ERR %s() size mismatch %dx%d * %dx%d",FCT,a1,a2,b1,b2);
00296
00297     if (cfac!=TC(0)) {
00298         if (C.data==NULL) { iflag=1;
00299             if (cforce) wblog(FL,
00300                 "WRN C = A*B + c*[] with c=%s",toStr(double(cfac)).data);
00301         }
00302         else if (!C.isMatrix() || C.SIZE[0]!=a1 || C.SIZE[1]!=b2) {

```



```

00303         wblog(FL,"ERR %s() dimension mismatch: C=(%s) =? (%d,%d).",
00304             FCT, SSTR(C), a1, b2); iflag=1;
00305     }
00306 } else iflag=1;
00307
00308 if (iflag) {
00309     C.init(a1,b2);
00310 }
00311
00312 DZGEMM(A,B,C,a2,aflag,bflag,afac,cfac);
00313
00314 return C;
00315 };
00316
00317 template<class TA, class TB, class TC>
00318 void Wb::DMatProd(
00319     const wbararray<TA> &A0, const wbararray<TB> &B0, wbararray<TC> &C,
00320     char aflag, char bflag, const TA afac, const TC cfac,
00321     const char cforce
00322 ){
00323     if (A0.SIZE.len==2 && B0.SIZE.len==2) {
00324         Wb::MatProd(A0,B0,C, aflag,bflag, afac,cfac, cforce);
00325         return;
00326     }
00327
00328 #ifdef WB_CLOCK
00329     Wb::Clock clk("MatProd",1);
00330 #endif
00331
00332     if (A0.SIZE.len<1 || A0.SIZE.len>2 || B0.SIZE.len<1 || B0.SIZE.len>2)
00333         wblog(FL,"ERR %s() rank <= 2 required (got %d,%d)", FCT,
00334             A0.SIZE.len,B0.SIZE.len);
00335
00336     unsigned i,j,k,sa,a1,a2,sb,b1,b2;
00337     wbararray<TA> A;
00338     wbararray<TB> B;
00339
00340     if (A0.SIZE.len==1 && B0.SIZE.len==2) {
00341         A0.adjustDMat(FL,aflag, afac, A);
00342         B0.adjustMMat(FL,bflag, 1. , B);
00343
00344         sa=A.SIZE[0]; b1=B.SIZE[0]; b2=B.SIZE[1]; sb=b1*b2;
00345         if (sa!=b1) wblog(FL,
00346             "ERR %s() sever size mismatch (%d * %dx%d)",
00347             FCT,sa,b1,b2);
00348
00349         C.adjustCMat (FL,cfac,b1,b2,cforce);
00350
00351         if (C.isEmpty()) { C=B;
00352             for (k=j=0; j<b2; j++)
00353                 for ( i=0; i<b1; i++, k++) C.data[k]*=A.data[i];
00354         }
00355         else {
00356             for (k=j=0; j<b2; j++)
00357                 for ( i=0; i<b1; i++, k++) C.data[k]+=A.data[i]*B.data[k];
00358         }
00359     }
00360     else
00361         if (A0.SIZE.len==2 && B0.SIZE.len==1) {
00362             TB bfac; Wb::safeConvert(FL,afac,bfac);
00363
00364             A0.adjustMMat(FL,aflag, 1. , A);
00365             B0.adjustDMat(FL,bflag, bfac, B);
00366
00367             a1=A.SIZE[0]; a2=A.SIZE[1]; sa=a1*a2; sb=B.SIZE[0];
00368             if (a2!=sb) wblog(FL,
00369                 "ERR %s() sever size mismatch (%dx%d * %d)",
00370                 FCT,a1,a2,sb);
00371
00372             C.adjustCMat (FL,cfac,a1,a2,cforce);
00373             if (C.isEmpty()) { C=A;
00374                 for (k=j=0; j<a2; j++)
00375                     for ( i=0; i<a1; i++, k++) C.data[k]*=B.data[j];
00376             }
00377             else {
00378                 for (k=j=0; j<a2; j++)
00379                     for ( i=0; i<a1; i++, k++) C.data[k]+=A.data[k]*B.data[j];
00380             }
00381         }
00382     }
00383     else wblog(FL,"ERR !?");
00384 };
00385
00386 template<
00387 wbararray<double>& wbarInverse(const char *F, int L, wbararray<double> &M) {
00388
00389     if (M.rank()!=2 || M.SIZE[1]>M.SIZE[0]) wblog(F_L,

```

```

00390     "ERR %s() invalid rank-2 tensor (%s)",FCT,M.sizeStr().data);
00391
00392     pINT e=0, l=-1, m=M.SIZE[0], n=M.SIZE[1], N=m*n;
00393     wbvector<pINT> ipiv(MIN(m,n));
00394     wbvector<double> work(1);
00395
00396     if (n<m) wblog(FL,
00397         "ERR DGETRI() requires dim1>=dim2 (%dx%d)",m,n);
00398
00399     dgetrf(m,n,M.data,m,ipiv.data,e);
00400     if (e) wblog(FL,"ERR DGETRF() returned e=%d !?",FCT,e);
00401
00402     dgetri(n,M.data,m,ipiv.data,work.data,1,e);
00403     if (e) wblog(FL,"ERR DGETRI() returned e=%d !?",FCT,e);
00404
00405     l=MIN(pINT(work[0]),N); work.init(1);
00406     dgetri(n,M.data,m,ipiv.data,work.data,1,e);
00407     if (e) {
00408         if (e<0)
00409             wblog(FL,"ERR DGETRI() got invalid argument #&d",-e);
00410         else wblog(FL,"ERR DGETRI() got singular matrix (i=%d)",e);
00411     }
00412
00413     return M;
00414 };
00415
00416 template<>
00417 wbararray<double>& wbInverse(
00418     const char *F, int L,
00419     const wbararray<double>&M, wbararray<double> &Minv
00420 ){ Minv=M; return wbInverse(F,L,Minv); };
00421
00422 inline void wbEigenS(
00423     const wbararray<double> &M, wbararray<double> &V, wbvector<double> &E
00424 ){
00425     unsigned n, r=M.SIZE.len, r2=r/2;
00426     pINT ni=0, q=0;
00427     double nd=0;
00428
00429     if (r==0) { V.init(); E.init(); return; }
00430
00431     if (r%2) wblog(FL,"ERR %s() even-rank required (%d)",FCT,r);
00432     if (!M.isHConj(-1E-12)) {
00433         MXPut(FL,"ans").add(M,"M").add(V,"V").add(E,"E").add(r,"r");
00434         wblog(FL,"ERR %s() got non-symmetric %s matrix",
00435             FCT, M.sizeStr().data);
00436     }
00437
00438     n=Wb::prodRange(M.SIZE.data,r2);
00439
00440     V=M; E.init(n); if (n==0) return;
00441
00442     if (r2>1) {
00443         V.SIZE[r2]=n; V.SIZE.len=r2+1;
00444     }
00445
00446     dsyevd('V','L',n,V.data,n,E.data,&nd,-1,&ni,-1,q);
00447     if (nd<1 || ni<1 || q) wblog(FL,
00448         "ERR DSYEVD returned lwork=%g/%d (e=%d) !?",nd,ni,q);
00449
00450     wbvector<double> wd((pINT)nd);
00451     wbvector<pINT> wi(ni);
00452
00453     dsyevd(
00454         'V',
00455         'L',
00456         n,
00457         V.data,
00458         n,
00459         E.data,
00460         wd.data, wd.len,
00461         wi.data, wi.len,
00462         q
00463     );
00464
00465     if (q>0) { pINT q_=q; V.Reset(M.data);
00466         dsyevd('V','U',n,V.data,n,E.data,wd.data,wd.len,wi.data, wi.len,q);
00467
00468         if (!q) wblog(FL,
00469             "WRN DSYEVD() ok only for uplo=L->U (i=%d/%d) !?",q_,n);
00470         else wblog(FL,
00471             "ERR DSYEVD() failed to converge (info=%d/%d) !?\n"
00472             "(D=%d @ l=%ld/%ld)",q_,q,n,E.len,wd.len,wi.len);
00473     }
00474
00475     if (q) {
00476         MXPut(FL,0,"tmpfile").add(M,"M").add(q,"q");

```

```

00477         wblog(FL, "ERR DSYEVD() returned "
00478             "info=%d (D=%d; l=%ld/%ld) !?", q, n, wd.len, wi.len);
00479     }
00480 };
00481
00482 inline void wbEigenS(
00483     const wbarry<wbcomplex> &M, wbarry<wbcomplex> &V, wvector<double> &E
00484 ) {
00485     unsigned n, r=M.SIZE.len, r2=r/2;
00486     pINT ni=0, q=0;
00487     wbcomplex nz=0; double nr=0;
00488
00489     if (r==0) { V.init(); E.init(); return; }
00490
00491     if (r%2) wblog(FL, "ERR %s() even-rank required (%d)", FCT, r);
00492     if (!M.isHConj(-1E-12)) {
00493         MXPput(FL, "ans").add(M, "M").add(V, "V").add(E, "E").add(r, "r");
00494         wblog(FL, "ERR %s() got non-hermitian %s matrix", FCT, SSTR(M));
00495     }
00496
00497     n=Wb::prodRange(M.SIZE.data, r2);
00498
00499     V=M; E.init(n);
00500     if (n<=1) {
00501         if (n) {
00502             double x=V.SkipTiny_imag(1E-12);
00503             if (x<0) wblog(FL, "ERR %s() got complex data @ %.3g", FCT, x);
00504             E.data[0]=V.data[0]; V.data[0]=1;
00505         }
00506         return;
00507     }
00508
00509     if (r2>1) {
00510         V.SIZE[r2]=n; V.SIZE.len=r2+1;
00511     }
00512
00513     zheevd('V', 'U', n, V.data, n, E.data, &nz, -1, &nr, -1, &ni, -1, q);
00514     if (nz.r<1 || nr<1 || ni<1 || q) {
00515         MXPput(FL, 0, "tmpfile").add(M, "M").add(q, "q");
00516         wblog(FL, "ERR ZHEEVD returned lwork=%g/%g/%d (e=%d)", nz.r, nr, ni, q);
00517     }
00518
00519     wvector<wbcomplex> wz(nz.r);
00520     wvector<double> wr(nr);
00521     wvector<pINT> wi(ni);
00522
00523     zheevd(
00524         'V',
00525         'U',
00526         n,
00527         V.data,
00528         n,
00529         E.data,
00530         wz.data, wz.len,
00531         wr.data, wr.len,
00532         wi.data, wi.len,
00533         q
00534     );
00535
00536     if (!q) { return; }
00537     if (q<0) wblog(FL, "ERR %s() ZHEEVD returned e=%d", FCT, q);
00538
00539     V=M; E.init(n);
00540
00541     wblog(FL, "WRN ZHEEVD returned i=%d / falling back to ZHEEV", q);
00542     MXPput(FL, 0, "tmpfile").add(M, "M").add(q, "q");
00543
00544     if (n<128) { ni=2*n-1; }
00545     else {
00546         ni=ilaenv(1, "zhetr", "U", (pINT)n, (pINT)n, (pINT)n, (pINT)n);
00547         if (ni>n) wblog(FL, "WRN %s() got lwork=(%d+2)*%d", FCT, ni, n);
00548         ni=(ni+1)*n;
00549     }
00550     if (size_t(ni)>wz.len) { wz.init(ni); }
00551
00552     nr=3*n-2; if (nr>wr.len) { wr.init(nr); }
00553
00554     zheev(
00555         'V',
00556         'U',
00557         n,
00558         V.data,
00559         n,
00560         E.data,
00561         wz.data, ni,
00562         wr.data,
00563         q

```

```

00564     );
00565
00566     if (q) wblog(FL,"ERR %s() ZHEEV returned e=%d",FCT,q);
00567 };
00568
00569 void wbEigen_CS (
00570     wbararray<wbcomplex> &M,
00571     wbararray<wbcomplex> &V,
00572     wbvector<wbcomplex> &E,
00573     char wjob,
00574     char issym,
00575     char tnorm,
00576     char qflag
00577 ){
00578     unsigned n=0, r=M.SIZE.len, r2=r/2;
00579     char jobvl='N', jobvr='V', vflag=1; pINT q=0;
00580
00581     if (wjob) { wjob=toupper(wjob); jobvr=jobvl='N';
00582         switch (wjob) {
00583             case 'L': jobvl='V'; break;
00584             case 'R': jobvr='V'; break;
00585             case 'N': vflag=0; break;
00586             default : wblog(FL,"ERR invalid wjob=%c<%d>",wjob,wjob);
00587         }
00588     }
00589
00590     if (r==0) { V.init(); E.init(); return; }
00591
00592     if (!M.isOps(n)) wblog(FL,
00593         "ERR %s() (generalized) square matrix required (%s)",
00594         FCT,M.sizeStr().data);
00595
00596     E.init(n); if (vflag) V.init(n,n);
00597
00598     if (n<=0) {
00599         if (n) { E[0]=M[0]; V[0]=1; }
00600         return;
00601     }
00602
00603     if (r2>1) {
00604         V.SIZE[r2]=n; V.SIZE.len=r2+1;
00605     }
00606
00607     if (issym) { tnorm=1;
00608         double x=1E-12;
00609         if (issym=='T') M.Symmetrize(FL,&x,0);
00610         else if (issym=='H') M.Symmetrize(FL,&x,1);
00611         else wblog(FL,"ERR invalid issym=%c<%d>",issym,issym);
00612
00613         if (x>1E-8) {
00614             #ifdef MATLAB_MEX_FILE
00615                 M.put("M_");
00616             #endif
00617             wblog(FL,"ERR operator not symmetric (%.3g)",x);
00618         }
00619         else if (x>1E-12) wblog(FL,
00620             "WRN operator not quite symmetric (%.3g)",x);
00621     }
00622
00623     wbvector<wbcomplex> aux;
00624     wbvector<double> aux2(2*n);
00625
00626     unsigned lwork=MAX(2U,n/4)*n;
00627
00628     if (n>127) {
00629         zgeev (
00630             jobvl, jobvr, n, M.data, n, E.data, V.data, n, V.data, n,
00631             aux.data, -1, aux2.data, q);
00632         lwork=(unsigned)aux[0].r; {
00633             unsigned b=lwork/n; if (b<2 || b>n)
00634                 wblog(FL,"WRN %s() got lwork %g*&d",FCT,double(lwork)/n,n);
00635         }
00636     }
00637
00638     aux.init(lwork);
00639
00640     #ifdef WB_CLOCK
00641         Wb::Clock clk("zgeev",1);
00642     #endif
00643
00644     { wbararray<wbcomplex> X(M);
00645         zgeev (
00646             jobvl,
00647             jobvr,
00648             n,
00649             X.data,
00650             n,

```

```

00651     E.data,
00652     V.data,
00653     jobvl=='V' ? n : 1,
00654     V.data,
00655     jobvr=='V' ? n : 1,
00656     aux.data,
00657     aux.len,
00658     aux2.data,
00659     q
00660 ); }
00661
00662 if (q) {
00663     MXPut(FL,0,"tmpfile").add(M,"M").add(q,"q");
00664     wblog(FL,"ERR %s() CGEEV returned %d (%d/%d)",FCT,q,aux.len,n);
00665 }
00666
00667 if (tnorm) {
00668     wbEigen_CS_regen(M,V,E,wjob);
00669
00670     wbararray<wbcomplex> X; double eps=10E-8;
00671     Wb::MatProd(V,V,X,'T');
00672
00673     if (!X.isIdentityMatrix(&eps)) {
00674 #ifndef MATLAB_MEX_FILE
00675         printf("\n"); M.put(FL,"M_"); V.put(FL,"V_"); E.put(FL,"E_",'t');
00676         wvector<wbcomplex> Es(E); Es.Sort(); Es.put("Es_"); X.put("X_");
00677 #endif
00678         if (qflag)
00679             wblog(FL,"WRN eig() U matrix not quite orthogonal (%.3g)",eps); else
00680             wblog(FL,"ERR eig() U matrix not quite orthogonal (%.3g)",eps);
00681     }
00682 }
00683 }
00684
00685 template<class T>
00686 void wbEigen_CS_regen(
00687     const wbararray<T> &M0,
00688     wbararray<T> &V,
00689     wvector<T> &E,
00690     char wjob
00691 ){
00692     if (E.len==0) return;
00693     if (!V.isRank(2) || E.len!=V.SIZE[1] || V.SIZE[0]!=V.SIZE[1])
00694         wblog(FL,"ERR invalid usage (%s; %d)",V.sizeStr().data,E.len);
00695
00696     unsigned i,i0=0, n=E.len;
00697     WBINDEX S(2); S[0]=n;
00698     double a, eps=1E-8, nmin=0, nmax=0;
00699
00700     wpperm P; E.Sort(P); V.Select0(P,1);
00701
00702     V.NormalizeCols(FL,&nmin,&nmax,'T');
00703
00704     if (nmin>1+1E-14) wblog(FL,
00705         "ERR zggeev() orthogonal matrix with t-norm = %.3g",nmin); else
00706     if (nmin<1E-4) wblog(FL,
00707         "ERR zggeev() orthogonal matrix with t-norm = %.3g",nmin); else
00708     if (nmin<0.01) wblog(FL,
00709         "WRN zggeev() orthogonal matrix with t-norm = %.3g",nmin);
00710
00711     for (i=1; i<n; i++) {
00712         a=Wb::absdiff_(E[i],E[i-1]);
00713
00714         if (i+1<n) { if (a<eps) continue; }
00715         else { if (a<eps) i++; }
00716
00717         if (i>i0+1) { T eref; double escale, x=eps;
00718
00719             wblog(FL,"TST fixing degenerate block: %d:%d (%d)  \r\\",i0+1,i,n);
00720
00721             wbararray<T> U,X,M2,u2; wvector<T> e2;
00722
00723             S[1]=i-i0; U.init2ref(V.ref(i0),S);
00724             Wb::MatProd(M0,U,X); Wb::MatProd(U,X,M2,'T');
00725
00726             M2.Symmetrize(FL,&x,0);
00727             if (x>eps) wblog(FL,
00728                 "WRN matrix not quite symmetric (x=%.4g) !?",x);
00729             M2.balanceOp(FL,eref,escale);
00730
00731             wbEigen_CS (M2,u2,e2,wjob,0,0);
00732
00733             e2*=escale; e2+=eref;
00734             Wb::cpyRange(E.data+i0,e2.data,e2.len);
00735
00736             Wb::MatProd(U,u2,X);
00737             memcpy(U.data,X.data,sizeof(T)*X.numel());

```

```

00738
00739     U.OrthoNormalizeCols(FL, 'T', T(1E-14), 2);
00740 }
00741     i0=i;
00742 }
00743
00744 };
00745
00746 template<class T>
00747 void wbEigen_CS_regen_trial(
00748     const wbararray<T> &M0,
00749     wbararray<T> &V,
00750     wbvector<T> &E
00751 ){
00752     if (E.len==0) return;
00753     if (!V.isRank(2) || E.len!=V.SIZE[1] || V.SIZE[0]!=V.SIZE[1])
00754         wblog(FL, "ERR invalid usage (%s; %d)", V.sizeStr().data, E.len);
00755
00756     unsigned i, j, k, i0=0, n=E.len; double a, eps=1E-8;
00757
00758     wbpem P; E.Sort(P); V.Select0(P, 1);
00759     wbcomplex *vr, *v, z, z2;
00760     unsigned count=0;
00761
00762     wbararray<T> U, X, M2, u2; wbvector<T> e2;
00763     double eps0, eps2;
00764
00765     Wb::MatProd(M0, V, X); Wb::MatProd(V, X, M2, 'T');
00766     eps0=eps; M2.isDiagMatrix(&eps0);
00767     wbararray<T> V0(V);
00768
00769     for (i=1; i<n; i++) {
00770         a=Wb::abs(E[i]-E[i-1]);
00771         if (a<eps) {
00772             for (j=i0; j<i; j++) { vr=V.ref(j); v=V.ref(i);
00773                 z=z2=0; for (k=0; k<n; k++) { z+=vr[k]*v[k]; z2+=(vr[k]*vr[k]); }
00774                 z=z/z2; for (k=0; k<n; k++) v[k]=z*vr[k];
00775
00776                 z=z2=0; for (k=0; k<n; k++) { z+=vr[k]*v[k]; z2+=(vr[k]*vr[k]); }
00777                 z=z/z2; for (k=0; k<n; k++) v[k]=z*vr[k];
00778             }
00779
00780             wblog(FL, "TST fixing degenerate block: %d-%d (%.3g)", i0, j, std::sqrt(z2.abs())); count++;
00781
00782             }
00783         else i0=i;
00784     }
00785
00786     if (count) {
00787         Wb::MatProd(M0, V, X); Wb::MatProd(V, X, M2, 'T');
00788         eps2=eps; M2.isDiagMatrix(&eps2);
00789         wblog(FL, "TST fixing degenerate block: %.6g => %.6g", eps0, eps2);
00790         V0.put("V0_"); V.put("V_"); M0.put("M_"); E.put("E_");
00791         wblog(FL, "ERR");
00792     }
00793 }
00794 };
00795
00796 template<class T> inline
00797 void GESVD_old(
00798     wbararray<T> &A QS_UNUSED_VAR,
00799     wbararray<T> &U QS_UNUSED_VAR,
00800     wbvector<double> &S QS_UNUSED_VAR,
00801     wbararray<T> &Vd QS_UNUSED_VAR
00802 ){
00803     wblog(FL, "ERR %s() not defined for datatype %s",
00804         FCT, getName(typeid(T)).c_str());
00805 };
00806
00807 template<> inline
00808 void GESVD_old(
00809     wbararray<double> &A,
00810     wbararray<double> &U,
00811     wbvector<double> &S,
00812     wbararray<double> &Vd
00813 ){
00814     unsigned dim1=A.SIZE[0], dim2=A.SIZE[1], k=S.len;
00815     pINT i;
00816
00817     pINT lwork=MAX3(1U, 3*MIN(dim1, dim2)+MAX(dim1, dim2), 5*MIN(dim1, dim2));
00818
00819     if (MIN(dim1, dim2)>=8) {
00820         int e=0; double w;
00821         dgesvd(
00822             'S', 'S', dim1, dim2, A.data, dim1, S.data,
00823             U.data, dim1, Vd.data, k,
00824             &w, -1, i);

```

```

00825     i=(pINT)w;
00826
00827     if (i<lwork) e=1; else
00828     if (i>dim1*dim2 && i>(1<<20)) {
00829         unsigned m=max(dim1,dim2); e=(i<(m*m) ? 2 : 3);
00830     }
00831     if (e) { wblog(FL,
00832         "WRN dgesvd() lwork=%d/%d (%.4g; %dx%d; e=%d)",
00833         i,lwork,double(i)/double(dim1*dim2),dim1,dim2,e);
00834     }
00835     lwork=i;
00836 }
00837
00838 wvector<double> W(lwork);
00839
00840 dgesvd(
00841     'S','S', dim1, dim2, A.data, dim1, S.data,
00842     U.data, dim1,
00843     Vd.data, k,
00844     W.data, W.len, i
00845 );
00846
00847 if (i) { if (i<0) { i=-i; wblog(FL,
00848     "ERR %s() dgesvd: invalid argument %d\nhint: %s",FCT,i,
00849     i==5 || i==8 ? "check input data for NAN or INF values":"??");
00850 }
00851 else { wblog(FL,
00852     "ERR %s() dgesvd returned e=%d\n"
00853     "hint: check input data for NAN or INF values",FCT,i);
00854 }}
00855 };
00856
00857 template<> inline
00858 void GESVD_old(
00859     wbararray<wbcomplex> &A,
00860     wbararray<wbcomplex> &U,
00861     wvector<double> &S,
00862     wbararray<wbcomplex> &Vd
00863 ){
00864     unsigned dim1=A.SIZE[0], dim2=A.SIZE[1], k=S.len;
00865     wvector<wbcomplex> W, RW(5*MIN(dim1,dim2));
00866     pINT i, lwork=MAX(1U, 2*MIN(dim1,dim2)+MAX(dim1,dim2));
00867
00868     if (MIN(dim1,dim2)>=8) { wbcomplex w;
00869         zgesvd(
00870             'S','S', dim1, dim2, A.data, dim1, S.data,
00871             U.data, dim1, Vd.data, k,
00872             &w, -1, RW.data, i
00873         );
00874
00875         i=unsigned(w.r);
00876         if (i<lwork || (i>10*dim1*dim2 && lwork>1E6)) {
00877             unsigned m=MIN(dim1,dim2); wblog(FL,"WRN zgesvd(%dx%d) "
00878                 "lwork = %d -> %d (~%d*%d)",dim1,dim2,lwork,i,i/m,m);
00879         }
00880         lwork=i;
00881     }
00882
00883     W.init(lwork);
00884
00885     zgesvd(
00886         'S','S', dim1, dim2, A.data, dim1, S.data,
00887         U.data, dim1,
00888         Vd.data, k,
00889         W.data, W.len, RW.data, i
00890     );
00891
00892     if (i) {
00893         if (i<0) { i=-i;
00894             wblog(FL,"ERR %s() zgesvd: invalid argument %d\nhint: %s",FCT,i,
00895             i==5 || i==8 ? "check input data for NAN or INF values":"??");
00896         }
00897         else wblog(FL,"ERR %s() zgesvd returned i=%d",FCT,i);
00898     }
00899 };
00900
00901 template<class T> inline
00902 int GESVD(
00903     wbararray<T> &A,
00904     wbararray<T> &U, wvector<double> &S, wbararray<T> &Vd,
00905     char lflag=0
00906 );
00907
00908 template<> inline
00909 int GESVD(
00910     wbararray<double> &A,
00911     wbararray<double> &U,

```

```

00912     wbvector<double> &S,
00913     wbararray<double> &Vd, char lflag
00914 ){
00915     unsigned dim1=A.SIZE[0], dim2=A.SIZE[1], k=S.len;
00916     pINT q, ni=8*MIN(dim1,dim2);
00917     double nd;
00918
00919     wbvector<double> wd;
00920     wbvector<pINT> wi(ni);
00921
00922     #ifdef WB_CLOCK
00923         Wb::Clock clk("dgesvd",1);
00924     #endif
00925
00926     dgesdd('S',dim1,dim2,A.data,dim1,S.data,U.data,dim1,Vd.data,k,
00927         &nd,-1,wi.data,q);
00928     if (nd<1 || q) wblog(FL,
00929         "ERR DGESDD() returned lwork=%g (e=%d) !?",nd,q);
00930     wd.init(nd);
00931
00932     dgesdd(
00933         'S', dim1, dim2, A.data, dim1, S.data,
00934         U.data, dim1,
00935         Vd.data, k,
00936         wd.data, wd.len, wi.data,
00937         q
00938     );
00939
00940     if (q) {
00941         if (q<0) { q=-q; wblog(FL,
00942             "ERR DGESDD() invalid argument %d\n%s",q, q==5 || q==8 ?
00943             "hint: check input data for NAN or INF values" : ""); }
00944         else if (lflag) {
00945             wblog(FL,"WRN DGESDD() did not converge (info=%d) !?",q);
00946         }
00947         else {
00948             MXPput(FL,0,"tmpfile").add(A,"A").add(q,"q");
00949             wblog(FL,"ERR DGESDD() did not converge (info=%d) !?",q);
00950         }
00951     }
00952     return q;
00953 };
00954
00955 template<> inline
00956 int GESVD(
00957     wbararray<wbcomplex> &A,
00958     wbararray<wbcomplex> &U,
00959     wbvector<double> &S,
00960     wbararray<wbcomplex> &Vd, char lflag
00961 ){
00962     unsigned dim1=A.SIZE[0], dim2=A.SIZE[1], k=S.len;
00963     pINT q, n1=MIN(dim1,dim2), n2=MAX(dim1,dim2);
00964
00965     pINT nd = n1*(3*n1+6 >= 2*n2 ? 5*n1+7 : 2*(n1+n2)+1);
00966
00967     #ifdef WB_CLOCK
00968         Wb::Clock clk("zgesvd",1);
00969     #endif
00970
00971     wbcomplex nz;
00972     wbvector<wbcomplex> wz;
00973     wbvector<double> wd(nd);
00974     wbvector<pINT> wi(8*n1);
00975
00976     if (!dim1 || !dim2) wblog(FL,
00977         "ERR %s() got %dx%d matrix!?",FCT,dim1,dim2);
00978
00979     zgesdd('S',dim1,dim2,A.data,dim1,S.data,U.data,dim1,Vd.data,k,
00980         &nz, -1, wd.data, wi.data, q);
00981     if (nz.r<1 || q) wblog(FL,
00982         "ERR ZGESDD() returned lwork=%g (e=%d) !?",nz.r,q);
00983     wz.init(nz.r);
00984
00985     zgesdd(
00986         'S', dim1, dim2, A.data, dim1, S.data, U.data, dim1, Vd.data, k,
00987         wz.data, wz.len, wd.data, wi.data,
00988         q
00989     );
00990
00991     if (q) {
00992         if (q<0) { q=-q; wblog(FL,
00993             "ERR ZGESDD() invalid argument %d\n%s",q, q==5 || q==8 ?
00994             "hint: check input data for NAN or INF values" : ""); }
00995         else if (lflag) {
00996             wblog(FL,"WRN ZGESDD() failed to converge (info=%d) !?",q);
00997         }
00998         else {

```



```

00999         wblog(FL,"ERR ZGESDD() failed to converge (info=%d) !?",q);
01000     }
01001 }
01002 return q;
01003 };
01004
01005 template<class T>
01006 void wbsvd(
01007     const wbararray<T> &A,
01008     wbararray<T> &U, wvector<double> &S,
01009     wbararray<T> &Vc,
01010     const WBINDEX &I,
01011     char bare
01012 ){
01013     unsigned dim1,dim2,k, r=A.SIZE.len, l=I.len;
01014     int itry=0, ntry=2, q=0;
01015     wbararray<T> X,Vd;
01016     wperm P;
01017
01018     static unsigned ncall=0;
01019     static int nout=0; ++ncall;
01020
01021     for (; itry<ntry; ++itry) {
01022         if (l==0) {
01023             if (r!=2) wblog(FL,
01024                 "ERR %s() rank-2 required, use index otherwise (%d)",FCT,r);
01025             X.init2ref(A);
01026         }
01027         else {
01028             if (l>=r) wblog(FL,
01029                 "ERR %s() invalid dimensions (%s; %d/%d)",FCT,I.toStr().data,l,r);
01030             A.toMatrixRef(X,I,l,P);
01031         }
01032
01033         if (X.isRef()) X.Instantiate();
01034
01035         dim1=X.SIZE[0]; dim2=X.SIZE[1]; k=MIN(dim1,dim2);
01036
01037         U.init(dim1,k); S.init(k); Vd.init(k,dim2);
01038         if (!dim1 || !dim2) return;
01039
01040         if (itry==0) {
01041             q=GESVD(X,U,S,Vd,'l');
01042             if (!q) break;
01043         }
01044         else {
01045             char vn[8]; snprintf(vn,8,"Isvd%d", (++nout)%10);
01046             MXPut Ix(FL,vn,"tmpfile");
01047             Ix.add(ncall,"ncall").add(q,"q").add(X,"A");
01048
01049             wblog(FL,"WRN GESVD returned i=%d / falling back to ZGESVD",q);
01050             GESVD_old(X,U,S,Vd);
01051             Ix.add(U,"U").add(S,"S").add(Vd,"Vd");
01052         }
01053     }
01054 }
01055
01056 #ifdef SVD_BUG_SAFEGUARD
01057 { double a=Wb::sqrt(A.norm2()), s=Wb::sqrt(S.norm2());
01058     double e=fabs(a-s)/(a*sqrt(double(dim1)*double(dim2)));
01059
01060     if (e>1E-10) { char s_[128];
01061         if (l==0) { X.init2ref(A); } else { A.toMatrixRef(X,I,l,P); }
01062         if (X.isRef()) X.Instantiate();
01063
01064         X.Transpose();
01065         dim2=X.SIZE[0]; dim1=X.SIZE[1]; k=MIN(dim1,dim2);
01066
01067         Vd.init(dim2,k); S.init(k); U.init(k,dim1);
01068
01069         GESVD(X,Vd,S,U); Vd.Transpose(); U.Transpose();
01070
01071         s=Wb::sqrt(S.norm2());
01072         a=fabs(a-s)/(a*sqrt(double(dim1)*double(dim2)));
01073         if (ISCOMPLX_(T))
01074             strcpy(s_,"ZGESVD");
01075         else strcpy(s_,"DGESVD");
01076         snprintf(s_+10,118,"%dx%d @%.3g -> %.2g",dim1,dim2,e,a);
01077
01078         if (a<1E-10)
01079             wblog(FL,"WRN fixed %s bug: %s (ok)", s_,s_+10);
01080         else wblog(FL,"ERR failed to fix %s bug\n%s",s_,s_+10);
01081     }
01082 }
01083 #endif
01084
01085 if (l) {

```

```

01086     WBINDEX q,Q; A.SIZE.select(P,Q);
01087     q.init(l+1, Q.data ); q[l]=S.len; U.Reshape(q);
01088     q.init(r-1+1,Q.data+1-1); q[0]=S.len; Vd.Reshape(q);
01089 }
01090
01091     if (bare) { Vd.save2(Vc); }
01092     else {
01093         P.initFirstTo(Vd.SIZE.len-1,Vd.SIZE.len);
01094         Vd.permute(Vc,P);
01095     }
01096 };
01097
01098 #endif
01099

```

5.26 wbararray_blas.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : wbararray (array class, col-major)
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_ARRAY_BLAS_HH__
00021 #define __WB_ARRAY_BLAS_HH__
00022
00023 // ----- //
00024 // also link wblas.hh for wrapper routines to Fortran library
00025 // extern "C" { ... }
00026 // Wb,Sep16,09
00027 // ----- //
00028
00029 namespace Wb {
00030
00031 template<class TA, class TB, class TC>
00032 wbararray<TC>& MatProd(
00033     const wbararray<TA> &A, const wbararray<TB> &B,
00034     wbararray<TC> &C,
00035     char aflag='N', char bflag='N',
00036     TA afac=1., TC cfac=0., char cforce=0
00037 );
00038
00039 template<class T>
00040 void VMatProd(
00041     const wbararray<T> &A0, const wbararray<T> &B0, wbararray<T> &C,
00042     char aflag='N', char bflag='N', const T afac=1., const T cfac=0.,
00043     const char cforce=1
00044 );
00045
00046 template<class T>
00047 void MatVProd(
00048     const wbararray<T> &A, const wvector<T> &v, wbararray<T> &x,
00049     char aflag='N', char bflag='N', const T afac=1., const T cfac=0.,
00050     const char cforce=1
00051 );
00052
00053 template<class T> T VMatVprod(
00054     const wbararray<T> &v1, const wbararray<T> &M, const wbararray<T> &v2);
00055
00056 template<class T> T VMatVprod(
00057     const wvector<T> &v1, const wbararray<T> &M, const wvector<T> &v2);
00058
00059 template<class TA, class TB, class TC>
00060 void DMatProd(
00061     const wbararray<TA> &A0, const wbararray<TB> &B0, wbararray<TC> &C,
00062     char aflag='N', char bflag='N', const TA afac=1., const TC cfac=0.,
00063     const char cforce=1
00064 );
00065
00066 template<class TA, class TB, class TC>

```

```

00067 void DMatProd(
00068     const wvector<TA> &A0, const wbarry<TB> &B0, wbarry<TC> &C,
00069     char aflag='N', char bflag='N', const TA afac=1., const TC cfac=0.,
00070     const char cforce=1
00071 ){
00072     wbarry<TA> A; A.init2Vec(A0.data,A0.len,'r');
00073     DMatProd(A,B0,C,aflag,bflag,afac,cfac,cforce);
00074 };
00075
00076 template<class TA, class TB, class TC>
00077 void DMatProd(
00078     const wbarry<TA> &A0, const wvector<TB> &B0, wbarry<TC> &C,
00079     char aflag='N', char bflag='N', const TA afac=1., const TC cfac=0.,
00080     const char cforce=1
00081 ){
00082     wbarry<TB> B; B.init2Vec(B0.data,B0.len,'r');
00083     DMatProd(A0,B,C,aflag,bflag,afac,cfac,cforce);
00084 };
00085
00086 };
00087
00088 template<class T>
00089 wbarry<T>& wbInverse(
00090     const char *F, int L,
00091     const wbarry<T> &M QS_UNUSED_VAR, wbarry<T> &Minv
00092 ){
00093     wblog(F_L,"ERR %s() not yet defined for T=%s",FCT,TSTR(T));
00094     return Minv;
00095 }
00096
00097 template<class T>
00098 wbarry<T>& wbInverse(const char *F, int L, wbarry<T>&M);
00099
00100 void wbEigenS(
00101     const wbarry<double>&M,
00102     wbarry<double>&U, wvector<double> &E
00103 );
00104
00105 void wbEigenS(
00106     const wbarry<wbcomplex>&M,
00107     wbarry<wbcomplex>&U, wvector<double> &E
00108 );
00109
00110 void wbEigen_CS(
00111     wbarry<wbcomplex> &M,
00112     wbarry<wbcomplex>&U, wvector<wbcomplex> &E,
00113     char wjob='R', char issym='H', char tnorm=1, char qflag=0
00114 );
00115
00116 template<class T>
00117 void wbEigen_CS_regen(
00118     const wbarry<T> &M0, wbarry<T> &V, wvector<T> &E,
00119     char wjob
00120 );
00121
00122 #define SVD_BUG_SAFE_GUARD
00123
00124 template<class T>
00125 void wbSVD(
00126     const wbarry<T> &A,
00127     wbarry<T> &U, wvector<double> &S, wbarry<T> &Vc,
00128     const WBINDEX &I=WBINDEX(),
00129     char bare=0
00130 );
00131
00132 template <class TQ>
00133 void wbEigenS (
00134     const wbsparry<double>&M,
00135     wbsparry<double>&U, wvector<double> &E
00136 ){
00137     wbarry<double> Mf,Uf; M.toFull(Mf);
00138     wbEigenS(Mf,Uf,E); U.init(FL,Uf,1E-14);
00139 };
00140
00141 template<class T>
00142 class opFlags {
00143 public:
00144     opFlags() : p(0), type(0) {};
00145     opFlags(const opFlags &f) : p(f.p), type(0) {};
00146
00147     opFlags(const char *F, int L, char f)
00148         : p(0), type(0) { init(F,L,f); };
00149
00150     opFlags(const opFlags &f1, const opFlags &f2)
00151         : p(0), type(0) { init(f1,f2); };
00152
00153

```

```

00154     opFlags<T>& operator= (char f) { init(f); return *this; }
00155     opFlags<T>& operator= (const opFlags<T> &x) { p=x.p; return *this; }
00156
00157     opFlags(char f) : p(0), type(0) { init(FL,f); };
00158     operator char() { return p; }
00159
00160     opFlags<T>& init(const char *F, int L, char f);
00161     opFlags<T>& init(char f) { return init(0,0,f); };
00162
00163     opFlags<T>& init(const opFlags<T> &f1, const opFlags<T> &f2){
00164     p=(f1.p ^ f2.p); return *this; };
00165
00166     char trans() const { return (p & TRANS_PAT); }
00167     char conj () const { return (p & CONJ_PAT); }
00168     char idle () const { return (!(p & TRANS_PAT) && !(p & CONJ_PAT)); }
00169
00170     void applyOrRef(
00171         const char *F, int L,
00172         const wbararray<T> &A0, wbararray<T> &A, const T fac=1
00173     ) const;
00174
00175     template <class T2>
00176     T2& applyOrRef(const char *F, int L, const T2 &A0, const T fac=1) const;
00177
00178     template <class T2>
00179     const T2& applyConjOrRef(const char *F, int L, const T2 &A0, const T fac=1) const;
00180
00181     template <class T2>
00182     char apply(const char *F, int L, T2 &A, const T fac=1) const;
00183
00184     char toChar() const {
00185         char c, tflag=(p & TRANS_PAT), cflag=(p & CONJ_PAT);
00186         if (tflag)
00187             c=(cflag ? 'C' : 'T');
00188         else c=(cflag ? 'c' : 'N');
00189         return c;
00190     }
00191
00192     mxArray *toMx() const { return numtoMx(this->toChar()); };
00193
00194     void info(const char *F, int L, const char *istr="") {
00195         wblog(F,L,"<i> opFlags<%s> %s: %d -> [%d %d]",
00196             TSTR(T), istr && istr[0] ? istr : "",
00197             p, trans(), conj());
00198     };
00199
00200     char p;
00201
00202 private:
00203
00204     T type;
00205
00206     static char TRANS_PAT;
00207     static char CONJ_PAT;
00208 };
00209
00210 template <class T> char opFlags<T>::TRANS_PAT = 1;
00211 template <class T> char opFlags<T>::CONJ_PAT = 2;
00212
00213 template <class T> inline
00214 opFlags<T>& opFlags<T>::init(
00215     const char *file, int line, char f
00216 ) {
00217     p=0;
00218
00219     if (f>16) {
00220         if (!strchr("ENCtC",f)) wblog(__FILE__, "ERR invalid flag %c<%d>",f,f);
00221         if (strchr("CT",f)) p |= TRANS_PAT;
00222         if (f=='E') type=(T)'E';
00223
00224         if (typeid(T)==typeid(wbcomplex))
00225             if (strchr("Cc",f)) p |= CONJ_PAT;
00226     }
00227     else {
00228         p=f;
00229         if (p & CONJ_PAT && typeid(T)!=typeid(wbcomplex))
00230             p &= ~CONJ_PAT;
00231     }
00232
00233     return *this;
00234 };
00235
00236 template <class T> inline
00237 void opFlags<T>::applyOrRef(
00238     const char *F, int L,
00239     const wbararray<T> &A0, wbararray<T> &A, const T fac
00240 ) const {

```

```

00241
00242     unsigned r=A0.SIZE.len;
00243     char p2=p;
00244
00245     if (p && ((p & ~TRANS_PAT) & ~CONJ_PAT))
00246         wblog(F,L,"ERR invalid flags (%d)",p);
00247
00248     if (!r) { A.init(); return; }
00249     if (r>2 && r%2) wblog(F,L,"ERR invalid rank-%d object (odd)",r);
00250
00251     if (r==1) p2 &= ~TRANS_PAT;
00252
00253     if (p2 || fac!=T(1)) {
00254         if (p2 & TRANS_PAT) {
00255             wbpperm P; P.initTranspose(r);
00256             A0.permute(A,P);
00257         } else A=A0;
00258
00259         if (p2 & CONJ_PAT) A.Conj();
00260
00261         if (fac!=T(1)) A*=fac;
00262     }
00263     else A.init2ref(A0);
00264 };
00265
00266 template <class T>
00267 template <class T2> inline
00268 T2& opFlags<T>::applyOrRef(
00269     const char *F, int L, const T2 &A, const T fac
00270 ) const {
00271
00272     if (p && ((p & ~TRANS_PAT) & ~CONJ_PAT))
00273         wblog(F,L,"ERR invalid flags (%d)",p);
00274
00275     if (!p && fac==1) return A;
00276
00277     T2 *a = new T2;
00278     if (p & TRANS_PAT) { A.transpose(F_L,*a); } else { (*a)=A; }
00279     if (p & CONJ_PAT) a->Conj();
00280     if (fac!=1) (*a)*=fac;
00281
00282     return (*a);
00283 };
00284
00285 template <class T>
00286 template <class T2> inline
00287 const T2& opFlags<T>::applyConjOrRef(
00288     const char *F, int L, const T2 &A, const T fac
00289 ) const {
00290
00291     char c=(p & CONJ_PAT);
00292     if (p && (p & ~TRANS_PAT & ~CONJ_PAT))
00293         wblog(F,L,"ERR invalid flags (%d)",p);
00294
00295     if (!c && fac==1) return A;
00296     else {
00297         T2 *a = new T2(A);
00298         if (c) a->Conj();
00299         if (fac!=1) (*a)*=fac;
00300         return (*a);
00301     }
00302 };
00303
00304 template <class T>
00305 template <class T2> inline
00306 char opFlags<T>::apply(
00307     const char *F, int L, T2 &A, const T fac
00308 ) const {
00309
00310     if (p && ((p & ~TRANS_PAT) & ~CONJ_PAT))
00311         wblog(F,L,"ERR invalid flags (%d)",p);
00312
00313     if (p || fac!=1) {
00314         if (p & TRANS_PAT) A.Transpose(F_L);
00315         if (p & CONJ_PAT) A.Conj();
00316         if (fac!=1) A*=fac;
00317         return 1;
00318     }
00319     return 0;
00320 };
00321
00322 #endif
00323

```

5.27 wbbits.cc

```

00001 #ifndef __WB_BITSET_CC__
00002 #define __WB_BITSET_CC__
00003
00004 // ----- //
00005 namespace Wb {
00006 // ----- //
00007
00008 template <class T>
00009 wbsting bitstr(const T &x) { // NB! i is 0-based
00010     Bitset B(8*sizeof(T)); memcpy(B.data, (void*)&x, B.len);
00011     return B.toStr();
00012 };
00013
00014 wbsting Bitset::toStr() const {
00015     wbsting s(nbits+len+1, ' ');
00016     unsigned i=0, l=0;
00017     s[l++]='['; s[s.len-2]=']';
00018     for (unsigned k=0; k<len; ++k) {
00019         const char &b=data[k]; char r=1;
00020         for (; r && i<nbits; r<=1, ++i) { s[l++]=( b&r ? '1' : '-' ); }
00021         if (!r && i<nbits) s[l++]='|';
00022     }
00023     if (l>s.len) wblog(FL,
00024         "ERR %s() index out of bounds (%d/%d)", FCT, l, s.len);
00025     return s;
00026 };
00027
00028 mxArray* Bitset::toMx() const {
00029     mxArray *a=mxCreateStruct(1,1);
00030     add2MxStruct(a,0);
00031     return a;
00032 };
00033
00034 mxArray* Bitset::mxCreateStruct(unsigned m, unsigned n) const {
00035     const char *fields[]={"nbits", "bset", "data"};
00036     return mxCreateStructMatrix(m,n,3,fields);
00037 };
00038
00039 void Bitset::add2MxStruct(mxArray *S, unsigned i) const {
00040     mxSetFieldByNumber(S,i,0, numtoMx(nbits));
00041     mxSetFieldByNumber(S,i,1, toStr().toMx());
00042     wbsting v(len);
00043     for (unsigned j=0; j<len; ++j) v.data[j]=(unsigned char)data[j];
00044     mxSetFieldByNumber(S,i,2, v.toMx());
00045 };
00046
00047 void Bitset::put(const char *file, int line,
00048     const char *vname, const char *ws
00049 ) const {
00050     mxArray *a=toMx();
00051     int i=mxPutVariable(ws,vname,a);
00052     if (i) wblog(__FL__,
00053         "ERR failed to write variable '%s' (%d)", vname,i);
00054     if (file) wblog(file,line,
00055         "I/O putting variable '%s' to %s", vname,ws);
00056     mxDestroyArray(a);
00057 };
00058
00059 template <class T>
00060 wbsting bitset<T>::toStr() const {
00061     unsigned len=sizeof(T);
00062     wbsting s(9*len+1, ' ');
00063     unsigned i=1, l=0; T r=1;
00064
00065     s[l++]='['; s[s.len-2]=']';
00066     for (; r; r<=1, ++i) {
00067         s[l++]=( data&r ? '1' : '-' );
00068         if (i%8==0 && l+2<s.len) { s[l++]='|'; }
00069     }
00070     return s;
00071 };
00072
00073 template <class T>
00074 mxArray* bitset<T>::toMx() const {
00075     mxArray *a=mxCreateStruct(1,1);
00076     add2MxStruct(a,0);
00077     return a;
00078 };
00079
00080 template <class T>

```

```

00086 mxArray* bitset<T>::mxCreatestruct(unsigned m, unsigned n) const {
00087     const char *fields[]={"nbits","bset","data"};
00088     return mxCreatestructMatrix(m,n,3,fields);
00089 };
00090
00091 template <class T>
00092 void bitset<T>::add2MxStruct(mxArray *S, unsigned i) const {
00093     mxSetFieldByNumber(S,i,0, numtoMx(8*sizeof(T)));
00094     mxSetFieldByNumber(S,i,1, toString().toMx());
00095     mxSetFieldByNumber(S,i,2, numtoMx(data));
00096 };
00097
00098 template <class T>
00099 void bitset<T>::put(const char *file, int line,
00100     const char *vname, const char *ws
00101 ) const {
00102
00103     mxArray *a=toMx();
00104     int i=mexPutVariable(ws,vname,a);
00105
00106     if (i) wblog(__FILE__,
00107         "ERR failed to write variable '%s' (%d)",vname,i);
00108     if (file) wblog(file,line,
00109         "I/O putting variable '%s' to %s",vname,ws);
00110     mxDestroyArray(a);
00111 };
00112
00113 };
00114
00115 #endif
00116

```

5.28 wbbitsset.hh

```

00001 #ifndef __WB_BITSET_HH__
00002 #define __WB_BITSET_HH__
00003
00004 // -----
00005 namespace Wb {
00006 // -----
00007
00008 // arbitrary length bitset
00009 class Bitset;
00010
00011 template <class T> class bitset;
00012
00013 template <class T>
00014 void SetBit(T &x, unsigned i) {
00015     unsigned nbits=8*sizeof(T);
00016     if (i>=nbits) wblog(FL,
00017         "ERR %s() bit index out of range (%d/%d)",FCT,i,nbits);
00018     T r=1; r<=i; x|=r;
00019 };
00020
00021 template <class T> inline
00022 T setbit(const T &x0, unsigned i) { T x=x0; SetBit(x,i); return x; };
00023
00024 template <class T>
00025 void UnsetBit(T &x, unsigned i) {
00026     unsigned nbits=8*sizeof(T);
00027     if (i>=nbits) wblog(FL,
00028         "ERR %s() bit-index out of range (%d/%d)",FCT,i,nbits);
00029     T r=1; r=~(r<=i); x&=r;
00030 };
00031
00032 template <class T> inline
00033 T unsetbit(const T &x0, unsigned i) { T x=x0; UnsetBit(x,i); return x; };
00034
00035 template <class T>
00036 bool issetbit(const T &x, unsigned i) {
00037     unsigned nbits=8*sizeof(T);
00038     if (i>=nbits) wblog(FL,
00039         "ERR %s() bit-index out of range (%d/%d)",FCT,i,nbits);
00040     T r=1; r<=i;
00041     return (x&r ? 1 : 0);
00042 };
00043
00044 template <class T>
00045 wstring bitstr(const T &x);
00046
00047 class Bitset: public wvector<char> {
00048 public:
00049

```

```

00050
00051 Bitset(unsigned n=0 ) : nbits(0) { NEW_BITSET(n); };
00052 Bitset(const Bitset &b)
00053 : wvector<char>(), nbits(0) { NEW_BITSET(b.nbits, b.data); };
00054
00055 Bitset& init(unsigned n=0) { return NEW_BITSET(n); };
00056 Bitset& init(unsigned n, const char* b) { return NEW_BITSET(n,b); };
00057
00058 Bitset& setall(bool t) {
00059     memset(data, t ? 255:0, len);
00060     if (t && len) { (data[len-1])>=(8*len-nbits); }
00061     return *this;
00062 };
00063
00064 Bitset& set(unsigned i) {
00065     if (i>=nbits) wblog(FL,
00066         "ERR %s() bit-index out of range (%d/%d)",FCT,i,nbits);
00067     unsigned k=i/8; data[k] |= (1<(i-8*k));
00068     return *this;
00069 };
00070
00071 Bitset& unset(unsigned i) {
00072     if (i>=nbits) wblog(FL,
00073         "ERR %s() bit-index out of range (%d/%d)",FCT,i,nbits);
00074     unsigned k=i/8; data[k] &= ~(1<(i-8*k));
00075     return *this;
00076 };
00077
00078 bool isset(unsigned i) {
00079     if (i>=nbits) wblog(FL,
00080         "ERR %s() bit-index out of range (%d/%d)",FCT,i,nbits);
00081     unsigned k=i/8;
00082     return (data[k] & (1<(i-8*k)) ? 1 : 0);
00083 };
00084
00085 unsigned nnz() const {
00086     unsigned i,n=0;
00087     for (i=0; i<len; ++i) {
00088         const char &b=data[i];
00089         for (char r=1; r; r<=1) { if (b&r) ++n; }
00090     }
00091     return n;
00092 };
00093
00094 bool operator==(const Bitset &b) const {
00095     if (len!=b.len || nbits!=b.nbits) return 0;
00096     if (data!=b.data) {
00097         for (unsigned i=0; i<len; i++)
00098             if (data[i]!=b.data[i]) return 0;
00099     }
00100     return 1;
00101 };
00102
00103 bool operator!=(const Bitset &b) const { return !((*this)==b); };
00104
00105 wstring toStr() const;
00106
00107 mxArray* toMx() const;
00108 mxArray* mxCreateStruct(unsigned m, unsigned n) const;
00109 void add2MxStruct(mxArray *S, unsigned i) const;
00110
00111 void put(const char *vname, const char *ws="base"
00112 ) const { put(0,0,vname,ws); };
00113
00114 void put(const char *file, int line,
00115     const char *vname, const char *ws="base"
00116 ) const;
00117
00118 unsigned nbits;
00119
00120 protected:
00121
00122 Bitset& NEW_BITSET(unsigned n, const char* d=NULL) {
00123     nbits=n; if (n) RENEW(((n-1)/8)+1,d);
00124     return *this;
00125 }
00126
00127 };
00128
00129 template <class T>
00130 class bitset {
00131
00132 public:
00133
00134     bitset() : data(0) {};
00135     bitset(const bitset<T> &b) : data(0) { data=b.data; };
00136

```



```

00137     bitset& setall(bool t) {
00138         memset(&data, t ? 255:0, sizeof(T));
00139         return *this;
00140     };
00141
00142     bitset& set(unsigned i) {
00143         T r=1; r<=i;
00144         if (r) { data |= r; } else wblog(FI,
00145             "WRN %s() bit-index out of range (%d/%d)",FCT,i,8*sizeof(T));
00146         return *this;
00147     };
00148
00149     bitset& unset(unsigned i) {
00150         T r=1; r<=i;
00151         if (r) { data &=~r; } else wblog(FI,
00152             "WRN %s() bit-index out of range (%d/%d)",FCT,i,8*sizeof(T));
00153         return *this;
00154     };
00155
00156     bool isset(unsigned i) {
00157         T r=1; r<=i;
00158         if (!r) wblog(FI,
00159             "WRN %s() bit-index out of range (%d/%d)",FCT,i,8*sizeof(T));
00160         return (data&r ? 1 : 0);
00161     };
00162
00163     unsigned nnz() const {
00164         unsigned n=0;
00165         for (T r=1; r; r<=1) { if (data&r) ++n; }
00166         return n;
00167     };
00168
00169     bool operator==(const bitset &b) const { return (data==b.data); };
00170     bool operator!=(const bitset &b) const { return (data!=b.data); };
00171
00172     wstring toStr() const;
00173
00174     mxArray* toMx() const;
00175     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
00176     void add2MxStruct(mxArray *S, unsigned i) const;
00177
00178     void put(const char *vname, const char *ws="base"
00179 ) const { put(0,0,vname,ws); };
00180
00181     void put(const char *file, int line,
00182         const char *vname, const char *ws="base"
00183 ) const;
00184
00185     T data;
00186
00187     protected:
00188
00189 };
00190
00191 };
00192
00193 #endif
00194

```

5.29 wbbblas.hh

```

00001 #ifndef __WB_BLAS_HH__
00002 #define __WB_BLAS_HH__
00003
00004 // ----- //
00005 // NB! matlab's mwbblas/mwlapack have their own header files
00006 // specifically, starting with matlab-R2009, the LDA type
00007 // (e.g. in DGEMM) is no longer int but ptrdiff_t
00008 // for compatibility reasons with 64bit linux (*@&$)!
00009 //
00010 // trying to avoid matlabs blas.h and lapack.h as it defines
00011 // other troublesome stuff, eg. second as wrapper for fortran _second,
00012 // but the latter is defined/used with maps!)*&#
00013
00014 // #define pINT int // for matlab <= R2008
00015 // #define pINT ptrdiff_t // for matlab >= R2009
00016 // NB! all int, also info variable, replaced by pINT (!)
00017
00018 extern "C" {
00019
00020 #define dgemm dgemm_
00021 void dgemm (
00022

```

```

00023     const char    &,
00024     const char    &,
00025     const pINT    &,
00026     const pINT    &,
00027     const pINT    &,
00028     const double  &,
00029     double [],
00030     const pINT    &,
00031     double [],
00032     const pINT    &,
00033     const double  &,
00034     double [],
00035     const pINT    &
00036 );
00037
00038 #define zgemm zgemm_
00039 void zgemm (
00040
00041     const char    &,
00042     const char    &,
00043     const pINT    &,
00044     const pINT    &,
00045     const pINT    &,
00046     const wcomplex &,
00047     wcomplex [],
00048     const pINT    &,
00049     wcomplex [],
00050     const pINT    &,
00051     const wcomplex &,
00052     wcomplex [],
00053     const pINT    &
00054 );
00055
00056 #define ilaenv ilaenv_
00057
00058 int ilaenv(
00059     const pINT &,
00060     const char *,
00061     const char *,
00062     const pINT &,
00063     const pINT &,
00064     const pINT &,
00065     const pINT &
00066 );
00067
00068 #define dgetri dgetri_
00069 void dgetri(
00070     const pINT    &,
00071     double [],
00072     const pINT    &,
00073     pINT [],
00074     double [],
00075     const pINT    &,
00076     pINT    &
00077 );
00078
00079 #define dgetrf dgetrf_
00080 void dgetrf(
00081     const pINT    &,
00082     const pINT    &,
00083     double [],
00084     const pINT    &,
00085     pINT [],
00086     pINT    &
00087 );
00088
00089 #define dsyevd dsyevd_
00090
00091 void dsyevd(
00092     const char    &,
00093     const char    &,
00094     const pINT    &,
00095     double [],
00096     const pINT    &,
00097     double [],
00098     double [],
00099     const pINT    &,
00100     pINT [],
00101     const pINT    &,
00102     pINT    &
00103 );
00104
00105 #define zheev zheev_
00106
00107 void zheev (
00108     const char    &,
00109     const char    &,

```

```

00110  const pINT    &,
00111  wbcomplex[],
00112  const pINT    &,
00113  double [],
00114  wbcomplex[], const pINT &,
00115  double [],
00116  pINT    &
00117  );
00118
00119  #define zheevd zheevd_
00120
00121  void zheevd(
00122  const char &,
00123  const char &,
00124  const pINT &,
00125  wbcomplex[],
00126  const pINT &,
00127  double [],
00128  wbcomplex[], const pINT &,
00129  double [], const pINT &,
00130  pINT [], const pINT &,
00131  pINT    &
00132  );
00133
00134  #define dgeev dgeev_
00135  void dgeev (
00136
00137  const char &,
00138  const char &,
00139  const pINT &,
00140  double [],
00141  const pINT &,
00142  double [],
00143  double [],
00144  double [],
00145  const pINT &,
00146  double [],
00147  const pINT &,
00148  double [],
00149  const pINT &,
00150  pINT    &
00151  );
00152
00153  #define zgeev zgeev_
00154  void zgeev (
00155
00156  const char &,
00157  const char &,
00158  const pINT &,
00159  wbcomplex [],
00160  const pINT &,
00161  wbcomplex [],
00162  wbcomplex [],
00163  const pINT &,
00164  wbcomplex [],
00165  const pINT &,
00166  wbcomplex [],
00167  const pINT &,
00168  double [],
00169  pINT    &
00170  );
00171
00172  #define dgesvd dgesvd_
00173
00174  void dgesvd (
00175  const char &,
00176  const char &,
00177  const pINT &,
00178  const pINT &,
00179  double [],
00180  const pINT &,
00181  double [],
00182  double [],
00183  const pINT &,
00184  double [],
00185  const pINT &,
00186
00187  double [],
00188  const pINT &,
00189  pINT    &
00190  );
00191
00192  #define dgesdd dgesdd_
00193
00194  void dgesdd (
00195  const char &,
00196  const pINT &,

```

```

00197 const pINT &,
00198 double [],
00199 const pINT &,
00200 double [],
00201 double [],
00202 const pINT &,
00203 double [],
00204 const pINT &,
00205
00206 double [],
00207 const pINT &,
00208 pINT [],
00209 pINT &
00210 );
00211
00212 #define zgesvd zgesvd_
00213
00214 void zgesvd (
00215 const char &,
00216 const char &,
00217 const pINT &,
00218 const pINT &,
00219 wbcomplex[],
00220 const pINT &,
00221 double [],
00222 wbcomplex[],
00223 const pINT &,
00224 wbcomplex[],
00225 const pINT &,
00226 wbcomplex[],
00227 const pINT &,
00228 wbcomplex[],
00229 pINT &
00230 );
00231
00232 #define zgesdd zgesdd_
00233
00234 void zgesdd (
00235 const char &,
00236 const pINT &,
00237 const pINT &,
00238 wbcomplex[],
00239 const pINT &,
00240 double [],
00241 wbcomplex[],
00242 const pINT &,
00243 wbcomplex[],
00244 const pINT &,
00245 wbcomplex[],
00246 const pINT &,
00247 double[],
00248 pINT [],
00249 pINT &
00250 );
00251
00252 }
00253
00254 class STAT_DGEMM {
00255 public:
00256     STAT_DGEMM() { nn.init(32); ss.init(32); }
00257     ~STAT_DGEMM() {
00258         unsigned i; double fac=nn.max(); fac=1./(fac!=0 ? fac:1);
00259         if (nn.max()<1) { return; }
00260
00261         wblog(FL,"%NDGEMM statistics");
00262         printf("\n idx_n avg.dim count cost (^2.5)\n\n");
00263         for (i=0; i<nn.len; i++) if (nn[i])
00264             printf(" %3d %8.2f %12d %9g\n", i,
00265                 pow(double(ss[i])/nn[i], 1/3.), nn[i],
00266                 pow(double(ss[i])/nn[i], 2.5/3) * nn[i]*fac);
00267         printf("\n");
00268     };
00269
00270     void account(unsigned n0) {
00271         unsigned i,n=n0;
00272         for (i=0; i<nn.len; i++) { n+=1; if (n==0) break; }
00273         if (i>=nn.len) i=nn.len-1;
00274         nn[i]++; ss[i]+=n0;
00275     };
00276
00277     wbvector<unsigned> nn;
00278     wbvector<unsigned long> ss;
00279
00280 private:
00281
00282 };
00283

```

```

00284 #ifdef WB_CLOCK
00285     STAT_DGEMM stat_dgemm;
00286 #endif
00287
00288 #endif
00289

```

5.30 wbclock.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : QSpace clock routines
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_CLOCK_CC__
00021 #define __WB_CLOCK_CC__
00022
00023 /* ----- */
00024 /* ----- */
00025 // see also $JUL/WbModules::sec2str() // Wb,Sep22,16
00026 // tags: sec2str, int2time, num2time, tstamp
00027
00028 inline wbstring Wb::sec2Str(double t) {
00029     wbstring st(16);
00030     unsigned l;
00031
00032     if (t<100) { l=snprintf(st.data,st.len,"%0.5g",t); } else
00033     if (t<3600) {
00034         double s=fmod(t,60), m=(t-s)/60;
00035         if (::fabs(s-::round(s))<1e-6)
00036             l=snprintf(st.data,st.len,"%02.0f:%02f", m,s); }
00037     else { l=snprintf(st.data,st.len,"%02.0f:%02.3f",m,s); }
00038 }
00039 else {
00040     double s=fmod(t,60);
00041     unsigned q=::round((t-s)/60), m,h,d;
00042
00043     m=q%60; q=(q-m)/60;
00044     h=q%24; d=(q-h)/24;
00045
00046     if (d)
00047         l=snprintf(st.data,st.len,"%d-%02d:%02d:%02.0f",d,h,m,s);
00048     else l=snprintf(st.data,st.len, "%02d:%02d:%02.0f", h,m,s);
00049 }
00050
00051 if (l>=st.len) wblog(FL,"WRN %s() string out of bounds "
00052     "(len=%d/%d)\nstr = %s",FCT,l,st.len,st.data);
00053 return st;
00054 };
00055
00056 void Wb::pause(const char *F, int L, double tsec) {
00057     if (tsec<=0) { return; }
00058
00059     if (F) wblog(FL," * %s() for %g secs",FCT,tsec); else
00060     if (tsec>99) wblog(F_L,"WRN %s() for %g secs !?",FCT,tsec);
00061
00062     timespec t; double s=floor(tsec), n=floor((tsec-s)*1E9);
00063     t.tv_sec=s; t.tv_nsec=n;
00064
00065     nanosleep(&t,0);
00066 };
00067
00068 void Wb::get_Clock_name(wbstring &name, const char *s, char use_tag) {
00069
00070     if (use_tag) {
00071         unsigned l, n=16+(s ? strlen(s) : 0); char sx[n];
00072         l=snprintf(sx,n,"%0.12s:",
00073             #ifdef PROG_TAG
00074                 PROG_TAG

```

```

00075         #elif defined(myname)
00076             myname
00077         #else
00078             "???"
00079         #endif
00080     );
00081     if (s && l<n) {
00082         if (l && !strncmp(sx,s,l))
00083             { wblog(FL,"WRN %s() '%s' :? '%s'",FCT,sx,s); }
00084         else { l+=snprintf(sx+l,n-l,"%s",s); }
00085     }
00086     name=sx;
00087 }
00088 else { name=(s ? s : ""); }
00089 };
00090
00091 Wb::Clock* Wb::ClockSet::insert(Wb::Clock* clk, char lflag) {
00092
00093     if (!clk) { wblog(FL,"ERR %s() got null clock",FCT); }
00094     if (!clk->name) { wblog(FL,"WRN %s() got empty clock name",FCT); }
00095
00096     auto q=buf.insert({ clk->name.data, clk });
00097
00098     if (!q.second && !lflag) wblog(FL,
00099         "ERR %s() got existing clock entry '%s'",FCT,clk->name.data);
00100     if (q.first->second!=clk) wblog(FL,
00101         "ERR %s() clk inconsistency %p / %p",FCT,q.first->second,clk);
00102     return q.first->second;
00103 };
00104
00105 int Wb::ClockSet::erase(Wb::Clock* clk) {
00106     if (!clk) { wblog(FL,"ERR %s() got null clock",FCT); }
00107     if (!clk->name) {
00108         wblog(FL,"ERR %s() got empty clock name",FCT);
00109     }
00110
00111     return buf.erase(clk->name.data);
00112 };
00113
00114 Wb::Clock* Wb::ClockSet::use(const char *istr, char mode) {
00115     Wb::Clock* clk;
00116
00117     if (!istr) wblog(FL,"ERR %s() got null clock string", FCT);
00118     if (!istr[0]) wblog(FL,"WRN %s() got empty clock string",FCT);
00119
00120     auto i=buf.find(istr);
00121     if (i!=buf.end()) { clk=i->second; }
00122     else {
00123         clk = new Wb::Clock(istr,mode&2,0,NULL,NULL,0);
00124         auto q=buf.insert({ istr, clk });
00125         if (!q.second) wblog(FL,"WRN %s() failed to insert clock",FCT);
00126     }
00127
00128     if (mode>1) {
00129         if (mode&1) { clk->resume(); }
00130         if (mode>3) wblog(FL,"WRN %s() got mode = %s",FCT,cSTR(mode));
00131     }
00132     return clk;
00133 };
00134
00135 void Wb::ClockSet::init(char vflag) {
00136     unsigned n=buf.size();
00137
00138     if (vflag>3) {
00139         if (vflag=='v') { vflag=1; } else
00140         if (vflag=='V') { vflag=2; } else
00141         if (vflag=='q') { vflag=0; } else
00142             wblog(FL,"WRN %s() got vflag=%s",FCT,cSTR(vflag));
00143     }
00144
00145     for (auto i=buf.begin(); i!=buf.end(); ++i) {
00146         if (!vflag) { i->second->init(); }
00147         delete i->second;
00148     }
00149     if (n) { PRINTF("\n"); }
00150 };
00151
00152 unsigned Wb::ClockSet::info(char vflag) const {
00153     for (auto i=buf.begin(); i!=buf.end(); ++i) {
00154         i->second->info(NULL,vflag);
00155     }
00156     return buf.size();
00157 };
00158
00159 int Wb::ClockSet::info_u(unsigned u, char rflag) {
00160     unsigned m=0;
00161

```

```

00162     for (auto i=buf.begin(); i!=buf.end(); ++i) {
00163         Wb::Clock &clk = *(i->second);
00164         if ((u && (clk.user & u)) || u==clk.user) { ++m;
00165             clk.info(); if (rflag) {
00166                 clk.reset(); }
00167         }
00168     }
00169     return m;
00170 };
00171
00172 int Wb::ClockSet::reset_u(unsigned u, char iflag) {
00173     unsigned m=0;
00174
00175     for (auto i=buf.begin(); i!=buf.end(); ++i) {
00176         Wb::Clock &clk = *(i->second);
00177         if ((u && (clk.user & u)) || u==clk.user) { ++m;
00178             if (iflag) { clk.info(); }
00179             clk.reset();
00180         }
00181     }
00182     return m;
00183 };
00184
00185 Wb::Clock* Wb::ClockSet::get(const char *istr, char use_tag) {
00186     wstring s;
00187     get_Clock_name(s,istr,use_tag);
00188
00189     auto i=buf.find(s.data());
00190     return ( i!=buf.end() ? i->second : NULL);
00191 };
00192
00193 int Wb::ClockSet::reset(const char *istr, char use_tag) {
00194
00195     int q=0;
00196     Wb::Clock *clk=get(istr,use_tag);
00197     if (clk) { clk->reset(); ++q; }
00198     return q;
00199 };
00200
00201 mxArray* Wb::ClockSet::toMx() const {
00202     unsigned i=0, n=buf.size();
00203     mxArray* C=mxCreateCellMatrix(n,3);
00204
00205     for (auto it=buf.begin(); it!=buf.end(); ++it, ++i) {
00206         const Wb::Clock &ci=*(it->second);
00207         mxSetCell(C, i, ci.name.toMx());
00208         mxSetCell(C, n+i, numtoMx(ci.gettime('c')));
00209         mxSetCell(C, 2*n+i, numtoMx(ci.gettime( )));
00210     }
00211     return C;
00212 };
00213
00214 mxArray* Wb::Clock::toMxS(const char *F, int L) const {
00215     return MXPut(F,L)
00216         .add(name,"istr")
00217         .add(sec2Str(gettime('c')), "cpu")
00218         .add(sec2Str(gettime( )), "wall")
00219         .toMx();
00220 };
00221
00222 mxArray* Wb::Clock::toMx(const char *F, int L) const {
00223     return MXPut(F,L).add(name,"istr")
00224         .addP(numtoMx(gettime('c')), "cpu ")
00225         .addP(numtoMx(gettime( )), "wall")
00226         .toMx();
00227 };
00228
00229 void Wb::save_and_clear_Profiling() {
00230
00231     mxArray *S=mxCreateStructMatrix(1,1,0,NULL);
00232
00233     for (auto I=gwb_Profs.begin(); I!=gwb_Profs.end(); ++I) {
00234         unsigned j=0, n=I->second.size(); double x[4];
00235         wbMatrix<double> M(n,4);
00236         for (auto J=I->second.begin(); J!=I->second.end(); ++J, ++j) {
00237             x[0]=J->first;
00238             x[1]=J->second.ncall;
00239             x[2]=J->second.dt;
00240             x[3]=J->second.aux; if (x[1]>1) { x[2]/=x[1]; x[3]/=x[1]; }
00241             M.recSet(j,x);
00242         }
00243         mxAddField2Scalar(FL,S,I->first.c_str(),M.SortRecs().toMx());
00244     }
00245
00246     const unsigned flen=128;
00247     unsigned i=0, l=0; char fout[flen]; fout[0]=fout[flen-1]=0;
00248     const char *s;

```

```

00249
00250     s=getenv("Wb_PROFILE_FOUT");
00251     if (s && s[0])
00252     { l=snprintf(fout,flen,"%s",s); }
00253     else {
00254         const mxArray *a=mexGetVariablePtr("caller","profile_fout");
00255         if (a && mxGetString(a,fout,flen-1)) { l=-1; }
00256     }
00257     if (l>=flen) { fout[flen-1]=0; wblog(FL,
00258     "WRN %s() string out of bounds (%d/%d) !?\ncheck %s",FCT,l,flen,
00259     int(l)>0? "variable `profile_fout`":"env `WB_PROFILE_FOUT`");
00260     fprintf(stdout,"\n%s (%ld)\n",fout,strlen(fout)); fout[0]=0;
00261     }
00262
00263     if (fout[0]) { Wb::matFile F;
00264         if (F.open(FL,fout,"w",0)) {
00265             unsigned n=mxGetNumberOfFields(S);
00266             for (i=0; i<n; ++i) {
00267                 F.put(FL,
00268                     mxGetFieldNameByNumber(S,i),
00269                     mxGetFieldByNumber(S,0,i)
00270                 );
00271             }
00272             F.close();
00273         }
00274         else {
00275             wblog(FL,"WRN %s() invalid file name\n%s",FCT,fout);
00276             fout[0]=0;
00277         }
00278     }
00279
00280     static int ncall=0;
00281
00282     if (l || ++ncall==1) { strcpy(fout,"Iprof"); }
00283     else { sprintf(fout,flen,"Iprof%02d",ncall); }
00284
00285     mxPutAndDestroy(FL,S,fout,"base");
00286 };
00287
00288 #endif
00289

```

5.31 wbclock.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : QSpace clock routines
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_CLOCK_HH__
00021 #define __WB_CLOCK_HH__
00022
00023 #include <set>
00024
00025 namespace Wb {
00026     void pause(const char *F, int L, double tsec);
00027     void pause(double tsec) { pause(0,0,tsec); }
00028     wstring sec2Str(double t);
00029 };
00030
00031 #define CHRONO_NOW std::chrono::high_resolution_clock::now
00032
00033 #define CHRONO_CLOCK std::chrono::time_point<std::chrono::high_resolution_clock>
00034 #define CHRONO_DIFF2SEC(A) (1E-9 * \
00035 double(std::chrono::duration_cast<std::chrono::nanoseconds>(A).count()))
00036
00037 #define WBCLK_USE_TAG 1
00038
00039 namespace Wb {

```



```

00040
00041 void get_Clock_name(wbstring &name, const char *s, char use_tag);
00042
00043 class ClockSet {
00044
00045 public:
00046
00047 ~ClockSet() { init(); }
00048
00049 void init(char vflag=1);
00050
00051 int info_u(unsigned u, char rflag=1);
00052 int reset_u(unsigned u, char iflag=0);
00053
00054 int reset(const char *istr, char uset_tag=1);
00055
00056 unsigned info(char vflag=0) const;
00057
00058 Clock* use(const char *istr, char mode);
00059
00060 Clock* use(const char *F, int L, char mode) {
00061     return use(shortFL(F,L),mode);
00062 };
00063
00064 Clock* get(const char *s, char use_tag=1);
00065
00066 Clock* insert(Wb::Clock* clk, char lflag=0);
00067
00068 int erase(Wb::Clock* clk);
00069
00070 mxArray* toMx() const;
00071
00072 map<std::string, Wb::Clock*> buf;
00073
00074 protected:
00075 private:
00076 };
00077
00078 ClockSet Clocks;
00079
00080 class Clock {
00081
00082 public:
00083
00084 Clock()
00085 : ttot(0), ctot(0), tneg(0), tref(), cref(0), nz(0), ncall(0),
00086   clk_flags(0), user(0), gcs(NULL) {};
00087
00088 Clock(const char *F, int L, char use_tag=1, unsigned u=0)
00089 : Clock(L>0? shortFL(F,L) : F, L>0 ? use_tag : L, u, &Clocks) {};
00090
00091 Clock(const char *istr, char use_tag=0)
00092 : Clock(istr,use_tag,0,&Clocks) {};
00093
00094 Clock(const char *istr, char use_tag, unsigned u,
00095        ClockSet *gcs=NULL,
00096        Clock **g=NULL,
00097        char rflag=1
00098 ) : Clock() { user=u;
00099     if (istr) {
00100         if (use_tag) { clk_flags|=WBCLK_USE_TAG; }
00101
00102         init(istr,use_tag); if (rflag) { resume(); }
00103         gcs=gcs_; if (!gcs && g) { gcs=&Wb::Clocks; }
00104         if (gcs) {
00105             Clock *x=gcs->use(name.data,0);
00106             if (g) { (*g)=x; }
00107         }
00108     }
00109     else if (g || gcs) wblog(FL,"ERR %s() invalid usage",FCT);
00110 };
00111
00112 ~Clock() { done(); }
00113
00114 void done() {
00115     if (ncall) { stop(); }
00116     if (gcs) {
00117         Clock *g = gcs->use(name.data,0);
00118         g->Add(*this);
00119     }
00120     else if (WBLOG_CLK) { info(); }
00121     init();
00122 }
00123
00124 void init() {
00125     ttot=ctot=tneg=0; tref=CHRONO_CLOCK();
00126     nz=ncall=clk_flags=user=0; cref=0; gcs=NULL;

```

```

00127     name.init();
00128 };
00129
00130 void reset() {
00131     ttot=ctot=tneg=0; tref=CHRONO_CLOCK();
00132     nz=ncall=0; cref=0;
00133 };
00134
00135 void init(const char *s, char use_tag=1) {
00136     reset();
00137     get_Clock_name(name,s,use_tag);
00138 };
00139
00140 void start() {
00141     cref=clock(); ctot=ttot=tneg=0; nz=0; ncall=1;
00142     tref=CHRONO_NOW();
00143 };
00144
00145 void resume() {
00146     if (!ncall) { start(); } else
00147     if ((ncall%2)==0) {
00148         cref=clock(); ++ncall;
00149         tref=CHRONO_NOW();
00150     }
00151 };
00152
00153 int stop(const char *F=NULL, int L=0) {
00154     if (ncall%2) { double tw,tc; ++ncall;
00155         ttot+=(tw=get_dt_wall());
00156         ctot+=(tc=double(clock()-cref)/CLOCKS_PER_SEC);
00157
00158         if (tw<=0 || tc<=0) { ++nz; if (tw) tneg-=tw; }
00159         return 1;
00160     }
00161     else {
00162         if (!ncall) wblog(FL,
00163             "WRN clock '%s' not yet started",name.data? name.data:"");
00164         return 0;
00165     }
00166 };
00167
00168 double gettime(char cflag=0) const;
00169
00170 void Add(const Clock &t) {
00171     if (ncall%2) { wblog(FL,"WRN %s() clock still running",FCT); }
00172     ttot += t.ttot;
00173     ctot += t.ctot; ncall+=2;
00174 };
00175
00176 int Switch(const char *istr, Clock **g=NULL, char rflag=1) {
00177     if (!name.data || !gcs) wblog(FL,
00178         "ERR %s() invalid usage (not yet initialized)",FCT);
00179     int q=stop(); Clock *x;
00180
00181     reset();
00182
00183     init(istr,clk_flags&1); if (rflag) { resume(); }
00184
00185     x=gcs->use(name.data,0);
00186     if (g) { (*g)=x; }
00187
00188     return q;
00189 };
00190
00191 mxArray* toMx (const char *F=NULL, int L=0) const;
00192 mxArray* toMxS(const char *F=NULL, int L=0) const;
00193
00194 void info(const char *istr=NULL, char vflag=0) const;
00195
00196 double ttot, ctot, tneg;
00197
00198 CHRONO_CLOCK tref;
00199 clock_t cref;
00200
00201 size_t nz;
00202
00203 size_t ncall;
00204
00205 wstring name;
00206
00207 unsigned clk_flags;
00208
00209 unsigned user;
00210
00211 ClockSet *gcs;
00212
00213 protected:

```

```

00214 private:
00215
00216     double get_dt_wall() const {
00217         return ( CHRONO_DIFF2SEC( CHRONO_NOW() - tref) );
00218     };
00219
00220     double getTickFreq() const;
00221 };
00222
00223 };
00224
00225 double Wb::Clock::gettime(char cflag) const {
00226     double dt=0;
00227
00228
00229     if (cflag) {
00230         dt=ctot; if (ncall%2) {
00231             dt+=double(clock()-cref)/CLOCKS_PER_SEC; }
00232     }
00233     else {
00234         dt=ttot; if (ncall%2) {
00235             dt+=get_dt_wall(); }
00236     }
00237
00238     return dt;
00239 };
00240
00241 void Wb::Clock::info(const char *istr, char vflag) const {
00242
00243     static int first_call=1;
00244
00245     if (vflag>8) {
00246         if (vflag=='v') { vflag=1; } else
00247         if (vflag=='V') { vflag=2; } else
00248             wblog(FL,"WRN %s() unexpected vflag=%s",FCT,cSTR(vflag));
00249     }
00250     if (!ncall && vflag<2) { return; }
00251
00252     unsigned l=0, n=16; char s[n];
00253     double tc=gettime('c'), tw=gettime();
00254
00255     if (nz) {
00256         double p0=1-nz/(double)(ncall/2);
00257         l=snprintf(s,16,"%5.1f%%", 100*p0);
00258     } else s[0]=0;
00259
00260     if (clk_flags && l<n) {
00261         l+=snprintf(s+l,n-l,"%d", clk_flags); }
00262
00263     if (first_call) { first_call=0;
00264         PRINTF("\n   %-28s Count   CPU-time   Wall-time       / call\n",
00265             "Wb::Clocks");
00266     }
00267
00268     PRINTF(" %c %-26s%8ld %10s %10s %9.3g %s\n",
00269         (ncall%2) ? '*' : ' ', istr? istr:name.data, ncall/2,
00270         sec2Str(tc).data, sec2Str(tw).data,
00271         tw/(ncall/2), s
00272     );
00273 };
00274
00275 double Wb::Clock::getTickFreq() const {
00276
00277     unsigned i,j,m=512,n=32;
00278     clock_t c1,c2,nc=0;
00279
00280     CHRONO_CLOCK t1,t2;
00281     double dt=0;
00282
00283     for (i=j=0; i<n; ++i) {
00284         t1=CHRONO_NOW(); c1=clock(); while (c1==clock()) { if (++j>m) break; };
00285         t2=CHRONO_NOW(); c2=clock();
00286
00287         if (i) {
00288             nc+=(c2-c1);
00289             dt+=CHRONO_DIFF2SEC( t2-t1 );
00290         }
00291     }
00292
00293     return (dt/nc);
00294 };
00295
00296 namespace Wb {
00297     class Profile;
00298
00299     class line_stat__ {
00300     public:

```

```

00301         line_stat__() : dt(0), aux(0), ncall(0) {};
00302
00303         void add(double dt_, double dx) { dt+=dt_; aux+=dx; ++ncall; };
00304
00305         double dt, aux;
00306         size_t ncall;
00307
00308     protected:
00309     private:
00310 };
00311
00312 void save_and_clear_Profiling();
00313 };
00314
00315 map< std::string, map<int, Wb::line_stat__ > > gwb_Profs;
00316
00317 namespace Wb {
00318 class Profile {
00319 public:
00320
00321     Profile(const char *F, int L, const char *istr=0) {
00322         unsigned i=0, l=0, n=32; char fl[32];
00323         if (!F || !F[0] || !L) wblog(FL,"ERR %s() invalid empty F !?",FCT);
00324
00325         for (; F[i]; ++i) { if (F[i]=='/') { l=i+1; }
00326         for (i=0; F[l] && i<n; ++l, ++i) {
00327             if (isalpha(F[l])) { fl[i]=F[l]; } else
00328             if ((F[l]=='.' || F[l]=='_') && i) { fl[i]='_'; } else { fl[i]='X'; }
00329         }; l=i;
00330
00331         if (l<n) { l+=snprintf(fl+l,n-l,"%03d",L); }
00332         if (l<n && istr && istr[0]) { l+=snprintf(fl+l,n-l,"%s",istr); }
00333         if (l>=n) { fl[n-l]=0; wblog(FL,
00334             "ERR %s() string out of bounds (%d/%d)\n%s",FCT,l,n,fl);
00335         }
00336         std::string fl_=fl;
00337
00338         #pragma omp critical (gwb_Profs__)
00339         { auto im=gwb_Profs.find(fl_);
00340           if (im==gwb_Profs.end()) {
00341               gwb_Profs[fl_]=map<int, Wb::line_stat__ >();
00342               if ((im=gwb_Profs.find(fl_))==gwb_Profs.end())
00343               { wblog(FL,"ERR %s() map entry failed !?",FCT); }
00344           }
00345           M=&(im->second);
00346         }
00347         tic();
00348     };
00349
00350 void tic() { tref=CHRONO_NOW(); };
00351 void toc(int L, double x=0) {
00352     double dt=CHRONO_DIFF2SEC( CHRONO_NOW() - tref );
00353
00354     #pragma omp critical (gwb_ProfL__)
00355     { auto im=M->find(L);
00356       if (im==M->end()) { (*M)[L]=Wb::line_stat__();
00357         if ((im=M->find(L))==M->end())
00358         { wblog(FL,"ERR %s() map entry failed !?",FCT); }
00359       }
00360       im->second.add(dt,x);
00361     }
00362 };
00363
00364 CHRONO_CLOCK tref;
00365
00366 protected:
00367 private:
00368
00369     map<int, Wb::line_stat__> *M;
00370 };
00371
00372 };
00373
00374 #endif
00375

```

5.32 wbcomplex.cc

```

00001 #ifndef __WB_COMPLEX_CC__
00002 #define __WB_COMPLEX_CC__
00003
00004 /* ----- */
00005 /* ----- */

```

```

00006 // complex number class
00007 // Wb,Dec20,01
00008
00009 wbstring wbcomplex::toStr(const char *fmt) const {
00010     wbstring zstr(32), zfmt(16);
00011     unsigned k,n;
00012
00013     double in=abs(), rn=0;
00014     if (in) { rn=r/in; in=i/in;
00015         if (std::fabs(rn)<1E-12) rn=0;
00016         if (std::fabs(in)<1E-12) in=0;
00017     }
00018     else { in=i; rn=i; }
00019
00020     if (!rn) {
00021         if (in) {
00022             zfmt.printf(FL,"%si",fmt);
00023             zstr.printf(FL,zfmt.data,i); return zstr;
00024         }
00025         else {
00026             zstr.printf(FL,fmt,r); return zstr;
00027         }
00028     }
00029     else if (!in) { zstr.printf(FL,fmt,r); return zstr; }
00030
00031     for (n=strlen(fmt), k=0; k<n; ++k) { if (fmt[k]!='+') break; }
00032     if (k<n) {
00033         zfmt.printf(FL,"%s%i",fmt,fmt);
00034     }
00035     else {
00036         zfmt.printf(FL,"%s+%si",fmt,fmt);
00037         zfmt[n]='%'; zfmt[n+1]='+';
00038     }
00039
00040     zstr.printf(FL,zfmt.data,r,i);
00041     return zstr;
00042 };
00043
00044 inline wbstring toStr(const wbcomplex &z, const char* fmt) {
00045     return z.toStr(fmt);
00046 }
00047
00048 inline wbstring toStr(const double &d, const char* fmt) {
00049     size_t l=0, n=32; char s[n];
00050     l=snprintf(s,n,fmt,d);
00051     if (l>=n) wblog(FL,"ERR %s() string out of bounds (%d/%d)",FCT,l,n);
00052     return s;
00053 };
00054 };
00055
00056 #endif
00057

```

5.33 wbcomplex.hh

```

00001 #ifndef __WB_COMPLEX_HCC__
00002 #define __WB_COMPLEX_HCC__
00003
00004 class wbcomplex;
00005
00006 /* ----- */
00007 /* ----- */
00008 // complex number class
00009 // Wb,Dec20,01
00010
00011 class wbcomplex {
00012
00013 public:
00014
00015     wbcomplex() : r(0.), i(0.) {};
00016
00017     wbcomplex(const wbcomplex &z) = default;
00018
00019     wbcomplex(double r0, double i0=0.) : r(r0), i(i0) {};
00020
00021     ~wbcomplex() = default;
00022
00023     wbcomplex& operator= (const wbcomplex &z) = default;
00024
00025     wbcomplex& operator= (const double &dbl) {
00026         r=dbl; i=0.; return *this;
00027     };
00028

```

```

00029     explicit operator bool() const { return (r || i); };
00030     bool operator! () const { return (!r && !i); }
00031
00032     operator double() const {
00033         if (i!=0) wblog(FL,
00034             "ERR got double(%4g%+.4gi) type conversion !?",r,i);
00035         return r;
00036     };
00037
00038     wbcomplex& init(double r0, double i0) { r=r0; i=i0; return *this; };
00039     wbcomplex& set(double r0, double i0) { r=r0; i=i0; return *this; };
00040
00041     wbcomplex& setRand(double fac=2., double shift=-1.);
00042
00043     wbcomplex& init_expi(double ph) {
00044         r=std::cos(ph); i=std::sin(ph);
00045         return *this;
00046     };
00047
00048     mxComplexDouble& explicit_copy2(mxComplexDouble &zm) const {
00049         zm.real=r; zm.imag=i;
00050         return zm;
00051     };
00052
00053     bool isinf() const { return (std::isinf(r) || std::isinf(i)); };
00054     bool isnan() const { return (std::isnan(r) || std::isnan(i)); };
00055
00056     bool isfinite() const { return (std::isfinite(r) && std::isfinite(i)); };
00057     bool isnormal() const { return (std::isnormal(r) && std::isnormal(i)); };
00058
00059     bool operator== (const wbcomplex &z) const {
00060         return (float(r)==float(z.r) && float(i)==float(z.i));
00061     };
00062     bool operator!= (const wbcomplex &z) const {
00063         return (float(r)!=float(z.r) || float(i)!=float(z.i));
00064     };
00065
00066     bool operator== (const double &dbl) const {
00067         return (i==0. && float(r)==float(dbl)); };
00068     bool operator!= (const double &dbl) const {
00069         return (i!=0. || float(r)!=float(dbl)); };
00070
00071     bool operator> (const wbcomplex &z) const {
00072         return (r>z.r || (r==z.r && i>z.i) ? 1 : 0); };
00073
00074     bool operator>= (const wbcomplex &z) const {
00075         return (r>z.r || (r==z.r && i>=z.i) ? 1 : 0); };
00076
00077     bool operator< (const wbcomplex &z) const {
00078         return (r<z.r || (r==z.r && i<z.i) ? 1 : 0); };
00079
00080     bool operator<= (const wbcomplex &z) const {
00081         return (r<z.r || (r==z.r && i<=z.i) ? 1 : 0); };
00082
00083     wbcomplex& operator+= (const wbcomplex &z) {
00084         r+=z.r; i+=z.i; return *this; };
00085     wbcomplex& operator-= (const wbcomplex &z) {
00086         r-=z.r; i-=z.i; return *this; };
00087     wbcomplex& operator*= (const wbcomplex &z) {
00088         double r_;
00089         r_ = r*z.r - i*z.i;
00090         i = r*z.i + i*z.r; r=r_; return *this;
00091     };
00092     wbcomplex& operator/= (const wbcomplex &z);
00093
00094     wbcomplex& operator*= (double dbl) { r*=dbl; i*=dbl; return *this; };
00095     wbcomplex& operator+= (double dbl) { r+=dbl; return *this; };
00096     wbcomplex& operator-= (double dbl) { r-=dbl; return *this; };
00097
00098     wbcomplex operator+ (const wbcomplex &z) const {
00099         return wbcomplex(r+z.r, i+z.i); };
00100     wbcomplex operator- (const wbcomplex &z) const {
00101         return wbcomplex(r-z.r, i-z.i); };
00102     wbcomplex operator* (const wbcomplex &z) const {
00103         return wbcomplex( r*z.r-i*z.i, i*z.r+r*z.i); };
00104     wbcomplex operator/ (const wbcomplex &z) const;
00105
00106     wbcomplex operator+ (const double &dbl) const {
00107         return wbcomplex(r+dbl,i); };
00108     wbcomplex operator- (const double &dbl) const {
00109         return wbcomplex(r-dbl,i); };
00110     wbcomplex operator* (const double &dbl) const {
00111         return wbcomplex(r*dbl, i*dbl); };
00112     wbcomplex operator/ (const double &dbl) const {
00113         if (dbl==0.) wblog(FL, "ERR DIV/0! (%g+%gi)/%g",r,i,dbl);
00114         return wbcomplex(r/dbl, i/dbl);
00115     };

```

```

00116
00117     wbcomplex operator/= (double dbl) {
00118         if (dbl==0.) wblog(FL,"ERR DIV/0! (%g+%gi)/%g",r,i,dbl);
00119         r/=dbl; i/=dbl; return *this;
00120     };
00121
00122     wbcomplex times(const wbcomplex &) const;
00123
00124     wbcomplex ConjProd(const wbcomplex &z) const {
00125         return wbcomplex(r*z.r+i*z.i, -i*z.r+r*z.i);
00126     };
00127
00128     void Conj() { i=-i; };
00129     wbcomplex conj() const { return wbcomplex(r,-i); };
00130
00131     double abs2() const { return double(r*r+i*i); };
00132     double abs() const;
00133
00134     wbcomplex sqrt() const;
00135
00136     wbcomplex sign() const {
00137         wbcomplex z;
00138         double a=abs(); if (a) { z.init(r/a, i/a); }
00139         return z;
00140     };
00141
00142     double arg() const;
00143     double arg_degree() { return (180./M_PI)*arg(); }
00144
00145     wbcomplex round() const {
00146         return wbcomplex( ::round(r), ::round(i) );
00147     };
00148
00149     wstring toStr(const char *fmt="%.4g") const;
00150     void put (const char *vname="ans", const char *ws="base") const;
00151
00152     double r,i;
00153
00154 protected:
00155
00156 private:
00157 };
00158
00159 inline wbcomplex wbcomplex::operator/ (const wbcomplex &b) const {
00160     double x, den;
00161
00162     if (fabs(b.r)>=fabs(b.i)) {
00163         if (b.r==0) { wblog(FL,"WRN DIV/0"); return wbcomplex(Inf,Inf); }
00164         x=b.i/b.r;
00165         den = b.r + x*b.i;
00166         return wbcomplex((r+x*i)/den, (i-x*r)/den);
00167     }
00168     else {
00169         x=b.r/b.i;
00170         den = b.i + x*b.r;
00171
00172         return wbcomplex((x*r+i)/den, (x*i-r)/den);
00173     }
00174 };
00175
00176 inline wbcomplex& wbcomplex::operator/= (const wbcomplex &z) {
00177     double r_, dbl=1./z.abs2();
00178     if (z==0.) wblog(FL,"ERR DIV/0! (%g+i%g)/(%g+i%g)",r,i,z.r,z.i);
00179
00180     r_ = (r * z.r + i * z.i) * dbl;
00181     i = (i * z.r - r * z.i) * dbl; r = r_; return *this;
00182 };
00183
00184 inline wbcomplex wbcomplex::times(const wbcomplex &z) const {
00185     double a,b; a=r*z.r; b=i*z.i;
00186     wbcomplex zz(a-b, (r+i)*(z.r+z.i)-a-b);
00187     return zz;
00188 };
00189
00190 inline wbcomplex& wbcomplex::setRand(double fac, double shift) {
00191     static char first_call=1;
00192
00193     if (first_call) { wb_srand(); first_call=0; }
00194     fac/=(double)RAND_MAX;
00195
00196     if (shift==0.)
00197     { r=fac*rand(); i=fac*rand(); }
00198     else { r=fac*rand()+shift; i=fac*rand()+shift; }
00199
00200     return *this;
00201 };
00202

```

```

00203
00204 inline double wbcomplex::abs() const {
00205     double a=0.;
00206     double x=fabs(r), y=fabs(i);
00207
00208     if (!x) { a=y; } else
00209     if (!y) { a=x; } else if (x>y)
00210         { a=y/x; a = x*std::sqrt(1.+a*a); }
00211     else { a=x/y; a = y*std::sqrt(1.+a*a); }
00212
00213     return a;
00214 };
00215
00216 inline wbcomplex wbcomplex::sqrt() const {
00217     wbcomplex z;
00218     if (!r && !i) { return z; }
00219
00220     double ar=fabs(r), ai=fabs(i), w, x;
00221     if (ar>=ai)
00222         { x=ai/ar; w=std::sqrt(ar)*std::sqrt(0.5*(1.+std::sqrt(1+x*x))); }
00223     else { x=ar/ai; w=std::sqrt(ai)*std::sqrt(0.5*(x +std::sqrt(1+x*x))); }
00224
00225     if (r>=0.) { z.init( w, i/(2.*w) ); } else
00226     if (i>=0.) { z.init( i/(2.*w), w ); } else { z.init(-i/(2.*w),-w ); }
00227
00228     return z;
00229 };
00230
00231 inline double wbcomplex::arg() const {
00232     double phi=0.;
00233
00234     if (!i) { if (r<0) { phi+=M_PI; }; } else
00235     if (!r) { phi=( i>0? +0.5 : -0.5 )*M_PI; }
00236     else {
00237         phi=atan(i/r);
00238         if (r<0) { phi+=(i>0 ? M_PI : -M_PI); }
00239     }
00240     return phi;
00241 };
00242
00243 void wbcomplex::put(const char *vname, const char *ws) const {
00244
00245     mxArray *a=NULL;
00246     if (i)
00247         { a=Mx::Array<wbcomplex>(1,1).copyFromTR(this); }
00248     else { a=Mx::Array<double>(1,1).copyFromTR(&r ); }
00249
00250     int e=mexPutVariable(ws,vname,a);
00251     if (e==1) wblog(FL,
00252         "ERR failed to put variable %s to workspace '%s'",vname,ws);
00253
00254     mxDestroyArray(a);
00255 };
00256
00257 inline wbcomplex operator+(const wbcomplex &z) { return z; }
00258 inline wbcomplex operator-(const wbcomplex &z) {
00259     return wbcomplex(-z.r, -z.i);
00260 };
00261 inline wbcomplex operator*(double d, const wbcomplex &z) {
00262     return wbcomplex(z.r*d, z.i*d);
00263 };
00264 inline wbcomplex operator+(double d, const wbcomplex &z) {
00265     return wbcomplex(d+z.r, +z.i);
00266 };
00267 inline wbcomplex operator-(double d, const wbcomplex &z) {
00268     return wbcomplex(d-z.r, -z.i);
00269 };
00270 inline wbcomplex operator/(double d, const wbcomplex &z) {
00271     return wbcomplex(d,0)/z;
00272 };
00273
00274 inline wbcomplex exp(const wbcomplex &z) {
00275     double r=exp(z.r);
00276     return wbcomplex(r*cos(z.i), r*sin(z.i));
00277 };
00278
00279 inline wbcomplex expi(const double &x) {
00280     return wbcomplex(cos(x), sin(x));
00281 };
00282
00283 inline wbsttring toStr(const wbcomplex &z, const char* fmt="%.4g");
00284 inline wbsttring toStr(const double &d, const char* fmt="%.4g");
00285
00286 namespace Wb {
00287
00288 inline bool isnan(const wbcomplex &z) { return z.isnan(); };
00289 inline bool isnan(const double &d) { return std::isnan(d); }

```



```

00290
00291 inline bool isinf(const wbcomplex &z) { return z.isinf(); }
00292 inline bool isinf(const double &d) { return std::isinf(d); }
00293
00294 inline bool isfinite(const wbcomplex &z) { return z.isfinite(); };
00295 inline bool isnormal(const wbcomplex &z) { return z.isnormal(); };
00296
00297 template <class T> inline T real(const T &d) { return d; };
00298 template <class T> inline T imag(const T &d QS_UNUSED_VAR) { return 0; };
00299 template <class T> inline T conj(const T &d) { return d; };
00300
00301 template <class T> inline T CONJ(const T &d) { return d; };
00302 template <class T> inline T abs (const T &a) { return (a<0 ? -a : a); };
00303 template <class T> inline T abs2(const T &d) { return d*d; };
00304 template <class T> inline T norm (const T &a){ return (a<0 ? -a : a); };
00305 template <class T> inline T norm2(const T &d){ return d*d; };
00306
00307 inline double real(const wbcomplex &z) { return z.r; };
00308 inline double imag(const wbcomplex &z) { return z.i; };
00309 inline wbcomplex conj(const wbcomplex &z) { return wbcomplex(z.r,-z.i); };
00310
00311 inline wbcomplex round(const wbcomplex &z) { return z.round(); }
00312 inline double round(const double &d) { return std::round(d); }
00313
00314 template <class T> inline
00315 T sqrt(const T &d) { return std::sqrt(d); }
00316
00317 template <> inline
00318 wbcomplex sqrt(const wbcomplex &d) { return d.sqrt(); }
00319
00320 template <> inline
00321 double abs(const double &a) { return std::fabs(a); };
00322 template <> inline
00323 double norm(const double &a) { return std::fabs(a); };
00324
00325 template <> inline
00326 wbcomplex CONJ(const wbcomplex &a) { return a.conj(); };
00327
00328 inline double abs (const wbcomplex &a) { return a.abs(); };
00329 inline double abs2 (const wbcomplex &a) { return a.abs2(); };
00330 inline double norm (const wbcomplex &a) { return a.abs(); };
00331 inline double norm2(const wbcomplex &a) { return a.abs2(); };
00332
00333 #ifdef __WB_MPFR_HH__
00334
00335 template <unsigned P> inline
00336 Wb::mpfr__<P> abs(const Wb::mpfr__<P> &a) { return a.abs(); };
00337 template <unsigned P> inline
00338 Wb::mpfr__<P> norm(const Wb::mpfr__<P> &a) { return a.abs(); };
00339
00340 template <unsigned P> inline
00341 Wb::mpfr__<P> abs2(const Wb::mpfr__<P> &a) { return a.abs2(); };
00342 template <unsigned P> inline
00343 Wb::mpfr__<P> norm2(const Wb::mpfr__<P> &a) { return a.abs2(); };
00344
00345 #endif
00346
00347 template <class T> inline double absdiff_f(
00348     const T &d1, const T &d2
00349 ){ return double(std::fabs(float(d1)-float(d2))); };
00350
00351 template <> inline double absdiff_f(
00352     const wbcomplex &d1, const wbcomplex &d2
00353 ){
00354     float dr=float(d1.r)-float(d2.r), di=float(d1.i)-float(d2.i);
00355     return std::sqrt(double(dr*dr+di*di));
00356 };
00357
00358 };
00359
00360 #endif
00361

```

5.34 wbeigs.hh

```

00001 #ifndef __WB_EIGS_HH__
00002 #define __WB_EIGS_HH__
00003
00004 // ----- //
00005 // ----- //
00006
00007 #define AUPDFUN  dsaupd
00008 #define EUPDFUN  dseupd

```

```

00009 #define AUPDSTR "DSAUPD"
00010 #define EUPDSTR "DSEUPD"
00011
00012 extern "C" {
00013
00014 #define dsaupd dsaupd_
00015 void dsaupd (
00016
00017     const int    &,
00018     const char   &,
00019     const int    &,
00020     const char[2] ,
00021     const int    &,
00022     const double &,
00023     double[],
00024     const int    &,
00025     double[],
00026     const int    &,
00027     const int[11] ,
00028     const int[11] ,
00029     double[],
00030     double[],
00031     const int    &,
00032
00033     const int    &
00034 );
00035
00036 #define dseupd dseupd_
00037 void dseupd (
00038
00039     const int    &,
00040     const char   &,
00041     const int[]  ,
00042     const double[],
00043     const double[],
00044     const int    &,
00045     const double &,
00046
00047     const char   &,
00048     const int    &,
00049     const char[2] ,
00050     const int    &,
00051     const double &,
00052     double[],
00053     const int    &,
00054     double[],
00055     const int    &,
00056     const int[11] ,
00057     const int[11] ,
00058     double[],
00059     double[],
00060     const int    &,
00061     const int    &
00062 );
00063
00064 }
00065
00066 char* info2str(int info) {
00067     char s[64];
00068     switch (info) {
00069         case -1: strcpy(s, "N<=0"); break;
00070         case -2: strcpy(s, "NEV<=0"); break;
00071         case -3: strcpy(s, "NCV<=NEV || NCV>N"); break;
00072         case -4: strcpy(s, "MAXIT<0"); break;
00073         case -5: strcpy(s, "invalid WHICH"); break;
00074         case -6: strcpy(s, "invalid BMAT"); break;
00075         case -7: strcpy(s,
00076             "length of private work array WORKL not sufficient"); break;
00077         case -8: strcpy(s, "error return from trid. eigenvalue "
00078             "calculation (from lapack::dstegr)"); break;
00079         case -9: strcpy(s, "starting vector is zero"); break;
00080         case -10: strcpy(s, "invalid MODE=IPARAM[6]"); break;
00081         case -11: strcpy(s, "IPARAM[6]==1 && BMAT=='G' are incompatible"); break;
00082         case -12: strcpy(s, "invalid IPARAM[0] (must be equal to 0 or 1)"); break;
00083         case -13: strcpy(s, "NEV && WHICH=='BE' are incompatible."); break;
00084         case -9999: strcpy(s, "could not build an Arnoldi factorization."); break;
00085         case -15: strcpy(s, "HOWMNY must be one of 'A' or 'S' if RVEC==true"); break;
00086         case -16: strcpy(s, "HOWMNY = 'S' not yet implemented"); break;
00087         case -17: strcpy(s, "data inconsistency between DSAUPD and DSEUPD"); break;
00088         default: sprintf(s, "unknown info=%d", info);
00089     }
00090
00091     return s;
00092 }
00093
00094 #endif
00095

```

5.35 wbfft.hh

```

00001
00002 #ifndef __WB_FFTW_C__
00003 #define __WB_FFTW_C__
00004
00005 #include <rfftw.h>
00006
00007 namespace Wb {
00008
00009 int FFT2D (
00010     wbMatrix<double> &X2,
00011     wbMatrix<double> &F2,
00012     char dir
00013 ){
00014
00015     int info=-1, thesame=(X2==F2);
00016     wbvector<int> dim(2);
00017
00018     if (dir=='F') {
00019         if (X2.isEmpty()) wblog(FL,
00020             "ERR %s - invalid dim [%d,%d]",FCT,X2.dim1,X2.dim2);
00021
00022         dim[0] = X2.dim1;
00023         dim[1] = X2.dim2;
00024
00025         if (!thesame) F2=X2;
00026
00027         F2.Resize(dim[0],dim[1]+2);
00028
00029         info=FFT_ND(dim,F2[0],dir);
00030     }
00031     else
00032     if (dir=='B') {
00033         if (F2.isEmpty()) wblog(FL,
00034             "ERR %s - invalid dim [%d,%d]",FCT,F2.dim1,F2.dim2);
00035
00036         dim[0] = F2.dim1;
00037         dim[1] = F2.dim2-2;
00038
00039         if (!thesame) X2 = F2;
00040
00041         info = FFT_ND (dim, X2[0], dir);
00042
00043         X2.Resize(dim[0], dim[1]);
00044     }
00045     else
00046     wblog(FL,"ERR %s - invalid dir=%c<%d>",FCT,dir,dir);
00047
00048     return info;
00049 }
00050
00051 int FFT_ND (
00052     wbvector<int> &dim,
00053     double *zz,
00054     char dir
00055 ){
00056
00057     unsigned i; int s=dim.prod(), info=-1;
00058
00059     if (dim.len==0 || s<=0) wblog(FL,
00060         "ERR %s - invalid dimensions [%s]",FCT,dim.toStr().data);
00061
00062     if (dim.last()%2) wblog(FL,
00063         "ERR %s - invalid dimensions [%s]\n"
00064         "leading/last dimension must be even!",FCT,dim.toStr().data);
00065
00066     if (dir=='F') {
00067
00068         rfftwnd_plan plan;
00069
00070         plan = rfftwnd_create_plan (
00071             dim.len, dim.data, FFTW_FORWARD, FFTW_ESTIMATE | FFTW_IN_PLACE);
00072
00073         rfftwnd_one_real_to_complex (plan, zz, NULL);
00074
00075         rfftwnd_destroy_plan(plan);
00076         info=0;
00077     }
00078
00079     else if (dir=='B') {
00080
00081         double fact; int ntot, nlast=dim.last();
00082         rfftwnd_plan plan;
00083
00084         plan = rfftwnd_create_plan (
00085             dim.len, dim.data, FFTW_BACKWARD, FFTW_ESTIMATE | FFTW_IN_PLACE);

```

```

00086
00087     rfftwnd_one_complex_to_real (plan, (fftw_complex*)zz, NULL);
00088
00089     for (ntot=1,i=0; i<dim.len; i++) ntot *= dim[i];
00090     fact = 1./ntot;
00091     ntot = (ntot / nlast) * (nlast+2);
00092     for (i=0; i<ntot; i++) zz[i] *= fact;
00093
00094     rfftwnd_destroy_plan(plan);
00095     info = 0;
00096 }
00097 else wblog(FL,"ERR %s - invalid dir=%c<%d>",dir,dir);
00098
00099 return info;
00100 }
00101
00102 }
00103
00104 #endif
00105

```

5.36 wbindx.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : QSpace index routines
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_INDEX_CC__
00021 #define __WB_INDEX_CC__
00022
00023 /* ----- */
00024 /* ----- */
00025 // if toend==0 : add towards the end, i.e. P=[I,...] (default)
00026 // else P=[...,I].
00027
00028 int wbindx::extend2Perm(widx_t N, wbpem &P, char toend) const {
00029
00030     widx_t i,k;
00031     wvector<char> mark(N);
00032
00033     for (i=0; i<len; ++i) {
00034         if (data[i]>=N) { sprintf_str(
00035             "%s:%d %s() index out of bounds (%ld/%ld)",FLF,data[i],N);
00036             return 1;
00037         }
00038         if (mark[data[i]]++) { sprintf_str(
00039             "%s:%d %s() index not unique",FLF);
00040             return 1;
00041         }
00042     }
00043
00044     P.init(N);
00045
00046     if (!toend) {
00047         memcpy(P.data,data,len*sizeof(widx_t));
00048
00049         for (k=len, i=0; i<N; ++i)
00050             if (!mark[i]) P[k++]=i;
00051     }
00052     else {
00053         memcpy(P.data+N-len,data,len*sizeof(widx_t));
00054
00055         for (k=i=0; i<N; ++i)
00056             if (!mark[i]) P[k++]=i;
00057     }
00058
00059     return 0;
00060 };

```

```

00061
00062 wbindx& wbindx::invert(widx_t n, wbindx &I, char uflag) const {
00063
00064     wbvector<char> M(n); char *m=M.data;
00065     widx_t i,l;
00066
00067     for (i=0; i<len; i++) { l=data[i];
00068         if (l<n) m[l]=1; else wblog(FL,
00069             "ERR %s() index out of bounds (%d: %d/%d)",FCT,i,l,n);
00070     }
00071
00072     l=M.nnz();
00073     if (uflag && l!=len) wblog(FL,
00074         "ERR %s() index not unique (%d/%d)",FCT,l,len);
00075
00076     I.init(n-l);
00077     if (I.len) {
00078         for (l=i=0; i<n; ++i) if (!m[i]) I.data[l++]=i;
00079     }
00080
00081     return I;
00082 };
00083
00084 wbindx wbindx::flipIdx(widx_t ndim) const {
00085     wbindx a(len);
00086
00087     if (ndim==0) {
00088         if (len==0) return *this; else { dbstop(FL);
00089             wblog(FL,"ERR wbindx - invalid ndim=%d (%d)", ndim, len); }
00090     }
00091
00092     for (widx_t m=ndim-1, i=0; i<len; i++) {
00093         if (i>m) wblog(FL,"ERR wbindx - invalid ndim=%d (%d)",ndim,i);
00094         a.data[i]=m-data[i];
00095     }
00096     return a;
00097 };
00098
00099 inline char wbindx::isUnique(widx_t imax, widx_t offset) const {
00100     wbvector<char> mark(imax);
00101
00102     if (offset==0) {
00103         for (widx_t i=0; i<len; i++) {
00104             if (data[i]<imax) { if (++mark[data[i]])>1) return 0; }
00105             else return 0;
00106         }
00107     }
00108     else {
00109         widx_t i=0, i0=offset; imax+=offset;
00110         for (; i<len; i++) {
00111             if (data[i]>=i0 && data[i]<imax) {
00112                 if (++mark[data[i]-i0])>1) return 0;
00113             }
00114             else return 0;
00115         }
00116     }
00117
00118     return 1;
00119 };
00120
00121 inline wbindx& wbindx::init(const ctrIdx &ic) {
00122     initT(ic.len,ic.data);
00123     return *this;
00124 };
00125
00126 inline wbindx& wbindx::init(
00127     const char *F, int L, const mxArray *a, widx_t offset
00128 ){
00129     if (!a) { init(); return *this; }
00130
00131     const size_t *sz=mxGetDimensions(a);
00132
00133     Mx::Array<double> A(a);
00134     wbvector<double> x;
00135
00136     if (A.rank!=2 || (sz[0]!=1 && sz[1]!=1)) wblog(F_L,
00137         "ERR %s() numeric vector required\n%s",FCT,STR(A));
00138
00139     init(A.len); if (!A.len) { return *this; }
00140     if (A.data)
00141         { x.init(A.len,A.data,'r'); }
00142     else { x.init(A.len); A.rcopy_to(x.data,0,'!'); }
00143
00144     for (widx_t i=0; i<A.len; ++i) { data[i]=widx_t(x.data[i]);
00145
00146         if (double(data[i])!=x.data[i]) wblog(F_L,
00147             "ERR %s() type mismatch (%d: %d %g)",i+1,data[i],x.data[i]);

```

```

00148
00149     if (offset) {
00150         if (data[i]>=offset) { data[i]-=offset; }
00151         else if (offset==1) wblog(F_L,"ERR %s() "
00152             "1-based index required (%d: %d)",FCT,i+1,data[i]);
00153         else wblog(F_L,"ERR %s() "
00154             "index out of bounds (%d: %d/%d)",FCT,i+1,data[i],offset
00155         );
00156     }
00157 }
00158
00159 return *this;
00160 };
00161
00162 template <class T>
00163 template <class T2>
00164 void groupIndex<T>::getRecs0(
00165     const wbMatrix<T2> &AB, wbMatrix<T2> &X, T m
00166 ) const {
00167
00168     widx_t i0=0; if (swidx_t(m)<0) { m=-m; i0=AB.dim2-m; }
00169
00170     if (swidx_t(i0)<0 || i0+m>AB.dim2) wblog(FL,
00171         "ERR %s() size out of bounds (%d+%d/%d)",FCT,i0+m,AB.dim2);
00172
00173     X.init(D.len,m); if (!D.len || !m) return;
00174
00175     if (I0.len!=D.len+1) wblog(FL,
00176         "ERR %s() unexpected size %d/%d",FCT,I0.len-1,D.len);
00177     if (AB.dim1!=P.len) wblog(FL,"ERR %s() size mismatch "
00178         "(%dx%d <> %dx%d)",FCT,P.len, m, AB.dim1, AB.dim2);
00179
00180     for (widx_t i=0; i<D.len; ++i) {
00181         X.recSetP(i,AB.ref(I0.data[i],i0));
00182     }
00183 };
00184
00185 int ctrIdx::init(const char *F, int L, const char *s, unsigned offset) {
00186
00187     int i, n=Str2Idx(FL,s,(wbvector<unsigned>&)(*this),offset,'x');
00188
00189     if (!n) {
00190         if (F) wblog(F,L,"ERR %s() got empty/null string !?",FCT);
00191         return -1;
00192     }
00193
00194     if (n<0) { int m=0; i=-n-1;
00195
00196         for (; s[i]; ++i) { if (!isspace(s[i])) {
00197             if (CTR_IS_SEP_ANY(s[i])) { ++m; } else break;
00198         }}
00199         if (m<=1 && CTR_IS_CONJ(s[i])) { conj=1;
00200             for (++i; s[i]; ++i) { if (!isspace(s[i])) break; }
00201         }
00202
00203         if (s[i]) { if (F) wblog(F,L,"ERR %s() "
00204             "invalid ctr-index set '%s' (%d: %c<%d>)",FCT,s,i+1,s[i],s[i]);
00205             return -2;
00206         }
00207     }
00208
00209     if (!len) { if (F) wblog(F,L,
00210         "ERR %s() got empty ctr-index set '%s' !?",FCT,s);
00211     }
00212     else if (!wbvector<unsigned>::isUnique()) { if (F) wblog(F,L,
00213         "ERR %s() invalid index set '%s' [%s]",
00214         FCT,STR_(this),wbvector<unsigned>::toStr().data);
00215         return -3;
00216     }
00217     else if (F) {
00218         if (len>32 || anyGT(62)) wblog(F,L,"WRN "
00219             "got ctr-index [%s] (len=%d)",wbvector<unsigned>::toStr().data,len);
00220     }
00221
00222     return len;
00223 };
00224
00225 ctrIdx& ctrIdx::init(const char *F, int L, const mxArray *a) {
00226
00227     if (!a) wblog(FL,"ERR %s() got null mxArray",FCT);
00228     if (mxIsChar(a)) {
00229         wbstring s(a); int e=init(F,L,s.data);
00230         if (e<=0) wblog(FL,
00231             "ERR %s() '%s' => [%s] @ e=%d !?",FCT,s.data,STR_(this),e);
00232     }
00233     else {
00234         wbvector<unsigned>::init(F,L,a);

```

```

00235         if (anyEqual(0)) wblog(FL,
00236             "ERR %s() contraction index must be 1-based",FCT);
00237         (*this)--1;
00238         conj=0;
00239     }
00240     return *this;
00241 };
00242
00243 template<class TQ, class TD>
00244 ctrIdx& ctrIdx::initOp(const char *F, int L,
00245     itag_ t1, const QSpace<TQ,TD> &B, const char *sopt) {
00246
00247     unsigned l=0, r=-1;
00248
00249     if (!sopt || (l=strlen(sopt))>8 || strcmp(sopt,"-op",3)) wblog(F_L,
00250         "ERR %s() invalid ctr-string '%s'",FCT,sopt?sopt:"");
00251
00252     if (B.isOperator(&r)<=0 || r>3 || r!=B.itags.len) wblog(F_L,
00253         "ERR %s() got '%s' with QSpace %s !?",FCT,sopt,STR(B.itags));
00254
00255     init(l);
00256     if (sopt[l-1]!='*') { data[0]=0; conj=1; } else { data[0]=1; }
00257
00258     newtags.initOp(t1,B.itags,sopt+1);
00259
00260     return *this;
00261 };
00262
00263 int ctrIdx::extend2Perm(widx_t N, wbpem &P, char toend) const {
00264
00265     widx_t i,k;
00266     wvector<char> mark(N);
00267
00268     for (i=0; i<len; ++i) {
00269         if (data[i]>=N) { sprintf_str(
00270             "%s:%d %s() index out of bounds (%d/%ld)",FLF,data[i],N);
00271             return 1;
00272         }
00273         if (mark[data[i]]++) { sprintf_str(
00274             "%s:%d %s() index not unique",FLF);
00275             return 1;
00276         }
00277     }
00278
00279     P.init(N);
00280
00281     if (!toend) {
00282         for (k=0; k<len; ++k) { P.data[k]=data[k]; }
00283         for (i=0; i<N; ++i) { if (!mark[i]) P[k++]=i; }
00284     }
00285     else {
00286         for (k=i=0; i<N; ++i) { if (!mark[i]) P[k++]=i; }
00287         for (i=0; i<len; ++i) { P.data[i+k]=data[i]; }
00288     }
00289
00290     return 0;
00291 };
00292
00293 wbpem&
00294 ctrIdx::extend2Perm( unsigned ra, unsigned ma,
00295     const ctrIdx &icb__, unsigned rb, unsigned mb,
00296     const wbpem &pab, char conj, unsigned mc,
00297     wbpem &Pbc,
00298     ctrIdx &kb,
00299     ctrIdx &kcb,
00300     char Bflag
00301 ) const {
00302
00303     ctrIdx ka,
00304         kca;
00305
00306     unsigned i=0, l=0, m=0, rca=ra-len, rcb=rb-icb__.len, rc=rca+rcb;
00307
00308     ctrIdx ica,icb; wbpem p,ipab;
00309     if (Bflag)
00310         { icb__.sort(icb,p); this->wvector<unsigned>::permute(ica,p); }
00311     else { this->sort(ica,p); icb__.wvector<unsigned>::permute(icb,p); }
00312
00313     if (pab.len)
00314         ipab.init(pab,'i');
00315     else ipab.init(rc);
00316
00317     if (len!=icb.len || ra<len || rb<icb.len) wblog(FL,"ERR %s() "
00318         "invalid ctrIdx data (%d/%d, %d/%d) !?",FCT,ra,len,rb,icb.len);
00319     if (ra>127 || rb>127 || rc>127) wblog(FL,
00320         "ERR %s() got rank %d/%d/%d !?",FCT,ra,rb,rc);
00321     if (ipab.len!=rc) wblog(FL,"ERR %s() "

```

```

00322     "invalid permutation pab=[%s] (%d)",FCT,STR(pab),rc);
00323
00324     ica.invert(ra,ka);
00325     icb.invert(rb,kb);
00326
00327     kca.initT(rca, ipab.data, !conj);
00328     kcb.initT(rcb, ipab.data+rca, !conj);
00329
00330     if (Bflag) {
00331         wperm Pa(ra+(mb?1:0)+(mc?1:0)); m=ra;
00332
00333         for (; l<len; ++l) { Pa[l]=ica[l]; }
00334         if (mb) Pa[l++]=m++;
00335
00336         kca.Sort(p); ka.Permute(p);
00337         for (i=0; i<ka.len; ++i, ++l) { Pa[l]=ka[i]; }
00338         if (mc) Pa[l++]=m++;
00339
00340         if (l!=Pa.len || !Pa.isValidPerm()) wblog(FL,
00341             "ERR %s() got invalid Pa=[%s] (%d/%d)",FCT,STR(Pa),l,Pa.len);
00342         Pa.invert(Pbc);
00343     }
00344     else {
00345         wperm Pb(rb+(ma?1:0)+(mc?1:0)); m=rb;
00346
00347         for (; l<len; ++l) { Pb[l]=icb[l]; }
00348         if (ma) Pb[l++]=m++;
00349
00350         kcb.Sort(p); kb.Permute(p);
00351         for (i=0; i<kb.len; ++i, ++l) { Pb[l]=kb[i]; }
00352         if (mc) Pb[l++]=m++;
00353
00354         if (l!=Pb.len || !Pb.isValidPerm()) wblog(FL,
00355             "ERR %s() got invalid Pb=[%s] (%d/%d)",FCT,STR(Pb),l,Pb.len);
00356
00357         Pb.invert(Pbc);
00358         ka.save2(kb);
00359         kca.save2(kcb);
00360     }
00361
00362     return Pbc;
00363 };
00364
00365 int ctrIdx::checkUniqueS(const char *F, int L, const unsigned r) const {
00366
00367     if (len) {
00368         unsigned i=1;
00369         for (; i<len; ++i) {
00370             if (data[i-1]>=data[i]) {
00371                 if (data[i-1]>data[i]) { if (F) wblog(F,L,
00372                     "ERR %s() ctrIdx not sorted [%s]",FCT,STR_(this));
00373                     return 1;
00374                 }
00375                 if (data[i-1]==data[i]) wblog(F_L,
00376                     "ERR %s() ctrIdx not sorted [%s]",FCT,STR_(this));
00377             }
00378         }
00379
00380         if (int(r)>0) {
00381             if (data[i-1]>=r) wblog(F_L,
00382                 "ERR %s() ctrIdx out of bounds (%s; %d) !?",FCT,STR_(this),r);
00383         }
00384     }
00385
00386     return 0;
00387 };
00388
00389 ctrIdx& ctrIdx::invert(unsigned r, ctrIdx &I) const {
00390
00391     wvector<char> M(r); char *m=M.data;
00392     unsigned i=0, l;
00393
00394     for (; i<len; ++i) { l=data[i];
00395         if (l>=r) wblog(FL,
00396             "ERR %s() index out of bounds (%d: %d/%d)",FCT,i,l,r);
00397         if ((++m[l])>1) wblog(FL,
00398             "ERR %s() index not unique: %s",FCT,toStr().data
00399         );
00400     }
00401
00402     I.init(r-len);
00403     if (I.len) {
00404         for (l=i=0; i<r; ++i) { if (!m[i]) I.data[l++]=i; }
00405     }
00406
00407     I.conj=conj;
00408

```



```

00409     return I;
00410 };
00411
00412 wbststring ctrIdx::toStr(char xflag) const {
00413     wbststring s;
00414
00415     if (!xflag && this->anyGT(60)) {
00416         unsigned l, n=4*len+4; s.init(n);
00417
00418         l=snprintf(s.data,n,"%s%s",
00419             ((wbvector<unsigned>&)(*this)+1).toStrf("%d","").data,
00420             conj? "":"*");
00421         if (l>=n) wblog(FL,
00422             "ERR %s() string out of bounds (%d/%d)",FCT,l,n
00423         );
00424     }
00425     else {
00426         unsigned i=0, n=len+(conj ? 2 : 1); s.init(n);
00427         for (; i<len; ++i) {
00428             if (data[i]< 9) { s.data[i]='1'+ data[i]; } else
00429             if (data[i]<35) { s.data[i]='A'+(data[i]- 9); }
00430             else { s.data[i]='a'+(data[i]-35); }
00431         }
00432         if (conj) { s.data[i++]='*'; }
00433         s.data[i]=0;
00434     }
00435
00436     return s;
00437 };
00438
00439 itag_& itag_::init(const char* F, int L, const char *s, unsigned n) {
00440
00441     if (!s || !s[0] || !n) { t=0; return *this; }
00442
00443     unsigned i=0, l=0, n_=ITAG_LEN;
00444     char *q=(char*)(&t); t=0;
00445
00446     if (int(n)<0 || n>n_) n=n_;
00447
00448     while (s[i]!=' ') { ++i; };
00449     if (i) wblog(FL,"WRN %s() ignoring leading blanks (%d)",FCT,i);
00450
00451     for (; i<n && s[i]; ++i) {
00452         if (IS_CHAR_ITAG(s[i])) { q[l++]=s[i]; }
00453         else break;
00454     }
00455
00456     while (s[i]==CC_ITAG) { ++i;
00457         q[0] ^= char(128);
00458     }
00459
00460     if (i<n && s[i]) {
00461         if (i>=n_)
00462             sprintf_str("invalid itag '%s' (len=%li/%d)",s,strlen(s),n);
00463         else sprintf_str("invalid itag '%s' (len=%li/%d; i=%d)",s,strlen(s),n,i);
00464         if (F) wblog(FL,"ERR %s",str); else wblog(F_L,"WRN %s",str);
00465         t=0; return *this;
00466     }
00467     if (!t && F) wblog(F,L,"WRN %s() invalid itag '%s'",FCT,s);
00468
00469     return *this;
00470 };
00471
00472 template<class TQ, class TD>
00473 itag_& itag_::init(const char *F, int L,
00474     const wbvector< QSpace<TQ,TD> > &A,
00475     wbindex ia
00476 ){
00477     t=0; if (!A.len) return *this;
00478
00479     unsigned k=0, l=0; itag_ x;
00480
00481     if (!ia.len) {
00482         unsigned rk,r_=-1;
00483         for (; k<A.len; ++k) {
00484             if (int(rk=A[k].isOperator(&r_,'x'))<=0) wblog(FL,
00485                 "ERR %s() invalid rank-%d operator \n(%d: %d/%d, '%s')",
00486                 FCT,A[k].itags.len,k+1,rk,r_,A[k].otype2Str().data
00487             );
00488         }
00489         ia.Index(r_); k=0;
00490     }
00491     if (!ia.len) wblog(FL,"ERR %s() got empty index set !?",FCT);
00492
00493     for (; k<A.len; ++k) {
00494         const iTags &it=A[k].itags;
00495         for (l=0; l<ia.len; ++l) {

```

```

00496         if (ia[l]>=it.len) wblog(F_L,
00497             "ERR %s() index out of bounds (%d/%d)",FCT,ia[l],it.len);
00498         (x=it[ia[l]]).deConj();
00499
00500         if (!t) t=x.t; else
00501         if (x.t && x.t!=t) wblog(F_L,"ERR %s() got itags "
00502             "mismatch (%s [%s]; %s)",FCT, A[k].itags.toStr().data,
00503             (ia+1).toStr().data, this->toStr().data
00504         );
00505     }
00506 }
00507 return *this;
00508 };
00509
00510 bool itag_::isEmpty(char lenient) const {
00511
00512     if (!lenient) { return (t ? 0 : 1); }
00513     if (lenient==1 || lenient=='l') {
00514         return isEmpty();
00515     }
00516
00517     char mflag=(lenient=='m' ? 1 : (lenient=='M' ? 2 : 0));
00518     if (mflag==1) {
00519         if (t & ~(IDT)255) { return 0; }
00520     }
00521     else {
00522         unsigned i=(mflag ? 1 : 0);
00523         const char *s=(const char*)&t;
00524         for (; i<ITAG_LEN; ++i) { if (s[i]&127) return 0; }
00525
00526         if (lenient==2 || lenient=='L') { return 1; }
00527     }
00528     if (mflag) {
00529         char c=((char&t) & 127;
00530         return (!c || IS_CHAR_MARK(c) ? 1 : 0);
00531     }
00532     wblog(F_L,"WRN %s() invalid flag '%s'",FCT,cSTR(lenient));
00533     return isEmpty();
00534 };
00535
00536 bool itag_::isEmpty_m() const {
00537
00538     if (t & ~(IDT)255) { return 0; }
00539     else {
00540         char c=((char&t) & 127;
00541         return (!c || IS_CHAR_MARK(c) ? 1 : 0);
00542     }
00543 };
00544
00545 itag_& itag_::SetFlag(unsigned l) {
00546     if (l>=ITAG_LEN) wblog(F_L,
00547         "ERR %s() index out of bounds (%d/%d)",FCT,l,ITAG_LEN);
00548     ((char*)&t)[l] |= 128;
00549     return *this;
00550 };
00551
00552 itag_& itag_::SetFlags(unsigned l, char check) {
00553
00554     char *s = (char*)&t;
00555
00556     if (l==1) { s[0] |= 128; return *this; }
00557
00558     if (int(l)<0) { unsigned q=(int(l)==-1);
00559         l=( itag_::flag_id & ~((unsigned char)3) );
00560         if (q) { itag_::iter_flag_id(); }
00561
00562         if (int(l)<=0) wblog(F_L,
00563             "ERR %s() invalid usage (l=%d / %d; %d)",FCT,l,itag_::flag_id,q);
00564     }
00565     else if (int(l)<=0) {
00566         wblog(F_L,"ERR %s() invalid usage (l=%d)",FCT,l);
00567     }
00568
00569     unsigned i=0, n=ITAG_LEN;
00570
00571     if (l>=(1U<n)) wblog(F_L,
00572         "ERR %s() index out of bounds (%d/%d)",FCT,l,1<n);
00573     if (l&1) wblog(F_L,"WRN %s() got flags %d",FCT,l);
00574
00575     for (; i<n; ++i) {
00576         if (l & (1<i)) {
00577             if (check && (s[i] & 128)) wblog(F_L,
00578                 "WRN %s() flag %d already set (%s)",FCT,i,STR_(this));
00579             s[i] |= 128;
00580         }
00581         else if (i) {
00582             if (check && (s[i] & 128)) wblog(F_L,

```

```

00583         "WRN %s() unsetting flag %d (%s)",FCT,i,STR_(this));
00584         s[i] &= 127;
00585     }
00586 }
00587 return *this;
00588 };
00589
00590 itag_& itag_::SetSI(const char *F, int L,
00591     const char *tag, unsigned k, char mflag, char nfmt) {
00592     unsigned l, n=ITAG_LEN+1;
00593     char s[n];
00594     if (!tag) wblog(F_L,"ERR %s() got null tag",FCT);
00595     if (nfmt==2) { l=snprintf(s,n,"%s%02d",tag,k); }
00596     else if (nfmt<=1) { l=snprintf(s,n,"%s%d",tag,k); }
00597     else {
00600         unsigned m=8; char fmt[m];
00601         l=snprintf(fmt,m,"%s%s%0d",nfmt);
00602         if (l>=m) wblog(F_L,
00603             "ERR %s() fmt string out of bounds (%s; %d/%d)",FCT,fmt,l,m);
00604         l=snprintf(s,n,fmt,tag,k);
00605     }
00606     if (mflag & ~char(7)) wblog(F_L,
00607         "ERR %s() invalid mflag=%s",FCT,cSTR(mflag));
00608     if (mflag&3) {
00609         char m=getMark();
00610         if (m && l<n) { s[l]=m; if (++l<n) { s[l]=0; } }
00611     }
00612     if (mflag&1 || ((mflag&2) && isConj())) {
00613         if (l+1<n) { s[l]=CC_ITAG; s[l+1]=0; }
00614         else { ++l; }
00615     }
00616     if (l>=n) { s[n-1]=0; wblog(F_L,"ERR %s() "
00617         "string out of bounds (%s, %d/%d/%d)",FCT,s,l,n,ITAG_LEN); }
00618     return init(s);
00619 };
00620
00621 #ifndef NOMEX
00622 itag_& itag_::init(const char* F, int L, const mxArray *a) {
00623     if (!a) { t=0; return *this; }
00624     if (mxIsChar(a)) { wbsting s(a); init(F,L,s.data); }
00625     else wblog(F_L,
00626         "ERR %s() invalid QSpace.itag upon input (%s)",
00627         FCT,mxGetClassName(a));
00628     return *this;
00629 };
00630 #endif
00631
00632 char itag_::sameAs(const itag_ &B, char lflag) const {
00633     char q=0;
00634     IDT c128=128, a=t, b=B.t;
00635     bool cflag = (a & c128) != (b & c128);
00636     a &= ~c128;
00637     b &= ~c128;
00638     if (a==b) { q=(a ? 3 : 4); } else
00639     if (lflag) {
00640         if (!b) { if (! isEmpty_m()) { q=1; } } else
00641         if (!a) { if (!B.isEmpty_m()) { q=2; } }
00642     }
00643     if (cflag) { q=-q; }
00644     return q;
00645 };
00646
00647 unsigned itag_::to_str(char *sout, unsigned len,
00648     char cflag) const {
00649     unsigned e=0, l=0, i=0, m=0, n=ITAG_LEN;
00650     const char *st=(const char *)(&t), c128=char(128);
00651     if (int(len)<=int(n)) wblog(F_L,
00652         "ERR %s() string out of bounds (%d/%d)",FCT,len,n);
00653     for (; i<n && st[i]; ++i) { sout[i]=st[i];

```

```

00670         if (sout[i]<0) { sout[i]+=c128; m|=(1<<i); }
00671         if (sout[i])
00672             { if (!IS_CHAR_ITAG(sout[i])) { e+=10; }}
00673         else { break; }
00674     }
00675
00676     for (l=i, ++i; i<n; ++i) { if (st[i]) {
00677         if (st[i]==c128) { m|=(1<<i); } else { e+=100; }
00678     }}
00679
00680     if (e) { sout[l]=0; wblog(FL,
00681         "WRN %s() got non-char itag (%s; e=%d) !?",FCT,sout,e); }
00682     if ((m&1) && !cflag) { m-=1; }
00683
00684     if (m>1 && l<len) {
00685         if (!(m&2))
00686             { l+=snprintf(sout+l,len-l,"%d",m>2); }
00687         else { l+=snprintf(sout+l,len-l,"%d",m>1); }
00688     }
00689     if (m&1 && l<len) { sout[l++]=CC_ITAG; }
00690
00691     if (l>=len) { sout[len-1]=0; wblog(FL,
00692         "ERR %s() string out of bounds (%s / %d)",FCT,sout,l); }
00693     sout[l]=0;
00694
00695     return l;
00696 };
00697
00698 int itag_::isValid() const {
00699
00700     unsigned l=0, i=0, n=ITAG_LEN, nx=2;
00701     const char *s=(const char *)(&t), x128=~char(128);
00702     char x;
00703
00704     for (; i<n; ++i) { x=(s[i] & x128);
00705         if (x) { if (!IS_CHAR_ITAG(x)) return 0; } else break;
00706         if (s[i]<0 && i>=nx) return 0;
00707     }; l=i;
00708
00709     for (; i<n; ++i) { if (s[i]) {
00710         if (i<nx) { if (s[i] & x128) return 0; }
00711         else return 0;
00712     }}
00713
00714     return (l ? 2: 1);
00715 };
00716
00717 bool itag_::isConj_x(const itag_ &x, char lflag) const {
00718
00719     if (lflag<=0) { return isConj(x); }
00720     if (lflag>3) {
00721         if (lflag=='l') lflag=1; else
00722         if (lflag=='L') lflag=3; else
00723         wblog(FL,"ERR %s() invalid lflag=%d !?",FCT,lflag);
00724     }
00725
00726     unsigned i=1;
00727     const char *a=(const char *)(&t), *b=(const char *)(&x.t);
00728
00729     if (!(lflag & 2)) {
00730         if ((a[0]^char(128)) != b[0]) return 0;
00731         for (; i<ITAG_LEN; ++i) { if (a[i]!=b[i]) break; }
00732     }
00733     else {
00734         if (tolower(a[0]^char(128)) != tolower(b[0])) return 0;
00735         for (; i<ITAG_LEN; ++i) { if (tolower(a[i])!=tolower(b[i])) break; }
00736     }
00737
00738     if (i<ITAG_LEN) {
00739         if (!(lflag & 1) ||
00740             (a[i] && a[i]!='~') || (b[i] && b[i]!='~')) return 0;
00741         for (++i; i<ITAG_LEN; ++i) { if (a[i] || b[i]) return 0; }
00742     }
00743
00744     return 1;
00745 };
00746
00747 itag_ & itag_::AppendChar(char q, const char *qs) {
00748
00749     unsigned i=0, n=ITAG_LEN;
00750     char *s=(char *)(&t);
00751
00752     if (!IS_CHAR_ITAG(q)) wblog(FL,"ERR %s() invalid char %s",FCT,cSTR(q));
00753
00754     for (; i<n; ++i) { if (!(s[i] & char(127))) { break; }}
00755
00756     if (!i) { s[i]+=q; }

```

```

00757     else {
00758         char c=(s[i-1] & char(127));
00759         if (qs && strchr(qs,c)) { --i; }
00760         else if (i<n) {
00761             if (IS_CHAR_MARK(c)) {
00762                 s[i] = (s[i-1] & char(127)) + (s[i] & char(128));
00763                 --i;
00764             }
00765         }
00766         else wblog(FL,"ERR %s() itag out of bounds (%s)",FCT,STR_(this));
00767
00768         s[i] = q + (s[i] & char(128));
00769     }
00770
00771     return *this;
00772 };
00773
00774 itag_& itag_::MarkDual(char m, bool always) {
00775     unsigned i=0, n=ITAG_LEN;
00776     char *s=(char*)(&t);
00777
00778     if (!IS_CHAR_ITAG(m)) wblog(FL,"ERR %s() invalid char %s",FCT,cSTR(m));
00779
00780     for (; i<n; ++i) { if (!(s[i] & 127)) { break; }}
00781
00782     if (i || always) {
00783         if (i && (s[i-1] & 127)==m) {
00784             s[i-1] &= 128;
00785         }
00786         else if (i<n) {
00787             s[i] = (s[i] & 128) + m;
00788         }
00789         else { wblog(FL,
00790             "ERR %s() itag string out of bounds (%s)\n(length must "
00791             "not exceed %d chars for %s)",FCT,STR_(this),n-1,myname);
00792         }
00793     }
00794     return *this;
00795 };
00796
00797 itag_& itag_::PrependChar(char q, const char *qs) {
00798     unsigned i=0, n=ITAG_LEN;
00799     char *s=(char*)(&t), x=(s[0] & 127);
00800
00801     if (!IS_CHAR_ITAG(q)) wblog(FL,
00802         "ERR %s() invalid char %s",FCT,cSTR(q));
00803
00804     if (!qs || !strchr(qs,x)) {
00805         for (; i<n; ++i) { if (!(s[i] & char(127))) { break; }}
00806         if (i==n) wblog(FL,"ERR %s() itag out of bounds (%s)",FCT,STR_(this));
00807         for (; i; --i) {
00808             s[i] = (s[i-1] & 127) + (s[i] & 128);
00809         }
00810     }
00811     s[i] = q + (s[i] & 128);
00812
00813     return *this;
00814 };
00815
00816 int itag_::CheckFirstChar(char q, const char *qs) {
00817     char *s=(char*)(&t), x=(s[0] & 127);
00818     if (!IS_CHAR_ITAG(q)) wblog(FL,"ERR %s() invalid char %s",FCT,cSTR(q));
00819
00820     if (qs && strchr(qs,x)) {
00821         s[0] = q + (s[0] & 128);
00822         return 1;
00823     }
00824
00825     return 0;
00826 };
00827
00828 unsigned iTags::init(const char *F, int L, const char *s) {
00829     if (s && s[0]) {
00830         unsigned i,l=0, i0=0, n=0;
00831         for (i=0; s[i]; ++i) {
00832             if (!IS_CHAR_ITAGS(s[i])) wblog(F_L,
00833                 "ERR %s() invalid itag >%s<\n"
00834                 "[required: printable string with no space]",FCT,s);
00835             if (IS_ITAGS_SEP(s[i])) ++l;
00836         }
00837         wvector<itag_>::init(++l);
00838         for (l=0, n=i, i=0; i<=n; ++i) {

```

```

00844         if (IS_ITAGS_SEP(s[i]) || i==n) {
00845             data[l].init(F_L,s+i0,i-i0);
00846             ++l; i0=i+1;
00847         }
00848     }
00849 }
00850 else wbvector<itag_>::init();
00851
00852 return len;
00853 };
00854
00855 iTags& iTags::init_alpha(
00856     unsigned r, unsigned s,
00857     const char *tag,
00858     char t,
00859     unsigned k
00860 ){
00861     unsigned i=0, l, n=10; char st[n];
00862
00863     init(r+s);
00864     for (; i<len; ++i) {
00865         l=snprintf(st,n,"%s%c%d",tag ? tag : QS_ITAG_,t+i,k);
00866         if (i>=r && ++l<n) { st[l-1]=CC_ITAG; st[l]=0; }
00867         if (l>=n) wblog(FL,
00868             "ERR %s() string out of bounds (%s: %d/%d)",FCT,st,l,n);
00869         data[i].init(st);
00870     }
00871
00872     return *this;
00873 };
00874
00875 #ifdef LD_CLEBSCH_QS
00876
00877 unsigned iTags::init_qdir(const char *s) {
00878
00879     QDir qdir(FL,s); init(qdir.len);
00880
00881     for (unsigned i=0; i<qdir.len; ++i) {
00882         if (qdir[i]<0) data[i].Conj();
00883     }
00884
00885     return qdir.len;
00886 };
00887
00888 #endif
00889
00890 #ifndef NOMEMX
00891 unsigned iTags::init(const char *F, int L, const mxArray *a) {
00892
00893     if (!a || mxIsEmpty(a)) { init(); return len; }
00894
00895     if (mxIsChar(a)) { wbsting sz(a); init(sz.data); }
00896     else if (mxIsCell(a)) {
00897         unsigned i=0, n=mxGetNumberOfElements(a);
00898         wbvector<itag_>::init(n);
00899         for (; i<n; ++i) data[i].init(F_L,mxGetCell(a,i));
00900     }
00901     else wblog(FL,"ERR invalid QSpace.itag upon input");
00902
00903     return len;
00904 };
00905 #endif
00906
00907 iTags& iTags::initOp(
00908     itag_ t, const iTags &b, const char *ot) {
00909
00910     if (!b.isOp()) wblog(FL,"ERR %s() invalid usage (%s)",FCT,STR(b));
00911     init(b);
00912
00913     if (!t.isEmpty()) { t.deConj();
00914         for (unsigned i=0; i<2; ++i) {
00915             if (data[i].isConj()) { (data[i]=t).Conj(); }
00916             else { data[i]=t; }
00917         }
00918     }
00919
00920     if (len>2 && data[2].isEmpty()) {
00921         unsigned i=0;
00922         char s[10]; strncpy(s, ot?ot:"op", 8); s[8]=0;
00923
00924         for (; s[i]; ++i) { if (s[i]==CC_ITAG) { s[i]=0; break; } }
00925
00926         if (data[2].isConj()) { s[i]=CC_ITAG; s[i+1]=0; }
00927         data[2].init(s);
00928     }
00929
00930     return *this;

```

```

00931 };
00932
00933 unsigned iTags::Init(
00934     const char *F, int L, const char *tag, unsigned k, char lflag
00935 ){
00936     size_t l=0, n=64; char s[n]; s[0]=0;
00937     iTags ix(*this);
00938
00939     if (!tag) wblog(F_L,"ERR %s() invalid tag (empty)",FCT);
00940     if (!strcmp(tag,"nrg:",4)) {
00941         const char *t=tag+4;
00942         if (strlen(t)!=2) wblog(FL,"ERR %s() invalid tag '%s'",FCT,tag);
00943
00944         if (int(k)<0) wblog(FL,
00945             "ERR %s() invalid k=%d for '%s'",FCT,k,tag);
00946
00947         if (t[0]=='H' && strchr("KD",t[1])) {
00948             l=snprintf(s,n,"%c%02d;%c%02d*",t[1],k,t[1],k);
00949         }
00950         else if (t[0]=='A' && strchr("KD",t[1])) {
00951             if (k)
00952                 l=snprintf(s,n,"K%02d;%c%02d*;s%02d",k-1,t[1],k,k); }
00953             else { l=snprintf(s,n,"L00;%c%02d*;s%02d",t[1],k,k); }
00954         }
00955         else wblog(FL,"ERR %s() invalid tag '%s'",FCT,tag);
00956     }
00957     else if (!strncasecmp(tag,"op:",3) && int(k)<0) {
00958         int i=0, n1=0, n2=-1; const char *t=tag+3;
00959         unsigned r=-k;
00960
00961         if (lflag<0) {
00962             lflag = (tag[0]=='O'? 1:0) + (tag[1]=='P'? 2:0);
00963         }
00964
00965         if (r<2 || (lflag ? r>4 : r>3)) wblog(FL,
00966             "ERR %s() invalid rank-%d operator",FCT,r);
00967
00968         for (; t[i]; ++i) {
00969             if (t[i]==':') {
00970                 n1=i++; while (t[i]) { ++i; }; n2=i-n1-1;
00971                 if (n1>8 || n2>8) wblog(FL,"ERR %s() "
00972                     "itag string out of bounds ('%s' %d,%d/8)",FCT,t,n1,n2);
00973                 break;
00974             }
00975         }
00976         if (n2>=0) { char fmt[20];
00977             if (r>3) wblog(FL,
00978                 "ERR %s() got rank-%d QSpace (l=%d)",FCT,r,lflag);
00979
00980             if (r==2) {
00981                 snprintf(fmt,20,"%%%d.%%ds;%%d.%%ds*",n1,n1,n1,n1);
00982                 l=snprintf(s,n,fmt,t,t);
00983             }
00984             else {
00985                 snprintf(fmt,20,"%%%d.%%ds;%%d.%%ds*;%s*",n1,n1,n1,n1);
00986                 l=snprintf(s,n,fmt,t,t,t+n1+1);
00987             }
00988         }
00989         else if (r==3)
00990             { l=snprintf(s,n,"%s;%s*;op*",t,t); }
00991         else { l=snprintf(s,n,"%s;%s*",t,t); }
00992
00993         if (r==3 && !data[2].isConj() && l>1 && l<n) { s[--l]=0; }
00994     }
00995     else wblog(F_L,"ERR %s() unexpected tag '%s'",FCT,tag);
00996     if (l>=n) wblog(FL,"ERR %s() string out of bounds (%s/%d)",FCT,l,n);
00997     n=0;
00998
00999     init(F,L,s);
01000
01001     if (len<Ix.len) {
01002
01003         for (unsigned i=0; i<len; ++i) {
01004             if (data[i].isConj(Ix.data[i],0)) { ++n; }
01005             else { Ix[i]=data[i]; }
01006         }
01007         if (!n) { Ix.save2(*this); }
01008     }
01009     else if (Ix.len && !Ix.sameConj(*this)) {
01010         n=99;
01011     }
01012
01013     if (n) wblog(FL,"ERR inconsistent conj labels "
01014         "%s => %s; e=%d",STR(Ix), STR_(this), n);
01015     return len;
01016 };
01017

```

```

01018 iTags& iTags::Init(unsigned r, int k, const char **ss) {
01019
01020     unsigned i=0, l, n=9; char st[n+1];
01021     init(r);
01022
01023     for (; i<r; ++i) { l=snprintf(st,n,"%s", ss[i]);
01024         if (k>=0 && l<n) {
01025             if (l>1 && st[l-2]==CC_ITAG) { --l;
01026                 l+=snprintf(st+l,n-l,"%d%c",k,CC_ITAG)-1; }
01027             else l+=snprintf(st+l,n-l,"%d",k);
01028         }
01029         if (l>=n) { st[n]=0; wblog(FL,
01030             "ERR %s() string out of bounds (%s /%d)",FCT,st,n); }
01031         data[i].init(st);
01032     }
01033
01034     return *this;
01035 };
01036
01037 bool iTags::isOp(int r, char lflag) const {
01038
01039     bool rval=0; if (r>=0) {
01040         if (r<2 || r>4) wblog(FL,"WRN %s() unexpected rank r=%d",FCT,r);
01041         if (len!=unsigned(r)) { return rval; }
01042     }
01043     else if (len<2 || len>4) { return rval; }
01044
01045     if (len==4) {
01046         for (unsigned i=2; i<4; ++i) {
01047             if (!data[i].isConj() ||
01048                 !data[i].isConj(data[i-2], lflag=='L'? 0:1)) {
01049                 return rval; }
01050         }
01051         return (rval=1);
01052     }
01053
01054     if (!lflag) { unsigned i=1;
01055         for (; i<len; ++i) { if (!data[i].isConj()) { return rval; }}
01056     }
01057     else if (lflag!='L' && lflag!='L') {
01058         wblog(FL,"WRN %s() unexpected lflag=%s",FCT,cSTR(lflag));
01059     }
01060
01061     rval = data[0].isConj(data[1], lflag=='L'? 0:1);
01062     return rval;
01063 };
01064
01065 bool iTags::isOpX(unsigned r) const {
01066     if (len<2 || len%2 || (int(r)>=0 && len!=2*r)) return 0;
01067     for (unsigned n=len/2, i=0; i<n; ++i) {
01068         if (!data[i].isConj(data[i+n]) || !data[i+n].isConj()) return 0;
01069     }
01070     return 1;
01071 };
01072
01073 bool iTags::isAtensor(unsigned r) const {
01074     if (int(r)>=0) {
01075         if (r<3) wblog(FL,"ERR %s() r=%d !?",FCT,r);
01076         if (len!=r) return 0;
01077     }
01078     else if (len<3) { return 0; }
01079
01080     if (data[2].isConj()) return 0;
01081     if (data[0].isConj() && data[1].isConj()) return 0;
01082     return 1;
01083 };
01084
01085 unsigned iTags::Set(const char* F, int L,
01086     const char *tag, unsigned r,
01087     const char *tom,
01088     unsigned m,
01089     bool conjOM
01090 ) {
01091     unsigned rval=0, i,l;
01092     if (!len && int(r)<=0 && int(m)<=0) { return rval; }
01093
01094     if (int(r)<0) {
01095         if (int(m)<=0) { r=len; } else
01096         if (m+2<len) { r=len-m; } else
01097         wblog(F_L,"ERR %s() requesting %d OM legs for rank-%d",FCT,m,len-m);
01098     } else
01099     if (r>len) wblog(F_L,
01100         "ERR %s() rank out of bounds (%d/%d)",FCT,r,len);
01101
01102     if (int(m)<0) { if (tom) { m=len-r; }}
01103     else {
01104         if (m && r< 2) wblog(F_L,"ERR %s() "

```



```

01105         "requesting %d OM leg%s for rank-%d",FCT,m,m==1?"":"s",r);
01106
01107         if ((l=r+m)<len) wblog(F_L,
01108             "ERR %s() asking for lower rank (%d+%d / %d)",FCT,r,m,len);
01109         if (l>len) { this->Resize(l); }
01110     }
01111
01112     if (!tag || (r<len && int(m)>0 && !tom)) wblog(F_L,
01113         "ERR %s() got null strings (%p / %p)",FCT,tag,tom);
01114     if (int(m)<0) { m=0; }
01115
01116     for (i=0; i<r; ++i) {
01117         if (data[i].isEmpty('m')) { ++rval;
01118             data[i].SetSI(F_L,tag,i+1);
01119         }
01120     }
01121
01122     for (i=0; i<m; ++i) {
01123         if (!data[r+i]) { ++rval;
01124             data[r+i].SetSI(F_L,tom,i+1, conjOM ? 1 : 0);
01125         }
01126         else wblog(F_L,
01127             "ERR %s() non-empty OM itags (%s; %d+%d)",FCT,STR_(this),r,m);
01128     }
01129
01130     return rval;
01131 };
01132
01133 wbsting iTags::toStr(char vflag) const {
01134
01135     unsigned k, n=ITAG_LEN;
01136     size_t l=0, N=(n*len>28 ? 2*n*len+8 : 64);
01137     char s[N];
01138
01139     if (vflag) { s[0]='{' ; s[1]=' ' ; l=2; }
01140     s[N-1]=0;
01141
01142     for (k=0; k<len; ++k) {
01143         if (l+4>=N) wblog(FL,"ERR %s() "
01144             "string out of bounds (%s; %d*%d @ %d/%d)",FCT,s,len,n,l,N);
01145         if (k) { s[l++]=',' ; if (vflag) { s[l++]=' ' ; } }
01146         l+=data[k].to_str(s+l,N-1);
01147     }
01148     if (l>=N || (vflag && l+6>N)) wblog(FL,"ERR %s() "
01149         "string out of bounds (%s; %d*%d @ %d/%d)",FCT,s,len,n,l,N);
01150     if (vflag) { s[l]=' ' ; s[l+1]='}' ; l+=2; }
01151     s[l]=0;
01152
01153     return s;
01154 };
01155
01156 int iTags::getCtr(const char *F, int L, const iTags &B,
01157     ctrIdx &ia, ctrIdx &ib,
01158     iTags *C,
01159     wvector<char> *ma_, wvector<char> *mb_
01160 ) const {
01161
01162     int nc=0;
01163     wvector<char> ma(len), mb(B.len);
01164     unsigned i,j, it=0, l=0, cflag=(ia.conj ^ ib.conj);
01165     int q, e=0;
01166
01167     for (; it<2; ++it) {
01168         q=(cflag ? SGN(+1) : SGN(-1));
01169
01170         for (i=0; i< len; ++i) {
01171             for (j=0; j<B.len; ++j) {
01172                 if (SGN(data[i].sameAs(B.data[j]))==q) {
01173                     if (!ma[i] && !mb[j])
01174                         { ma[i]=j+1; mb[j]=i+1; ++nc; }
01175                     else { ++e; }
01176                 }
01177             }
01178         }
01179         if (nc || e) { break; }
01180         else { cflag=!cflag; }
01181     }
01182
01183     if (e) { unsigned n=64, e1=0, e2=0; char s[n];
01184         for (i= 0; i< len; ++i) {
01185             for (j=i+1; j< len; ++j) { if ( data[i]== data[j]) { ++e1; } } }
01186             for (i= 0; i<B.len; ++i) {
01187                 for (j=i+1; j<B.len; ++j) { if (B.data[i]==B.data[j]) { ++e2; } } }
01188             if (e1) {
01189                 l=snprintf(s,n,"got non-unique itag%s in A",e1>1 || e2>1? "s":"" );
01190                 if (e2 && l<n) { l+=snprintf(s+l,n-l," and B"); }
01191             }

```

```

01192     }
01193     else if (e2)
01194         { l=snprintf(s,n,"got non-unique itag%s in B",e2>1? "s:"); }
01195     else { l=snprintf(s,n,"got non-unique itags"); }
01196
01197     wblog(F_L,"ERR %s() %s (e=%d, %d/%d)\n A: %s\n B: %s",
01198     FCT,s,e,e1,e2,STR_(this),STR(B));
01199 }
01200
01201 ia.wbvector<unsigned>::init(nc);
01202 ib.wbvector<unsigned>::init(nc);
01203
01204 if (nc) {
01205     for (i=0; i<len; ++i) { if (ma[i]) {
01206         ia[l]=i;
01207         ib[l]=ma[i]-1; ++l;
01208     }}
01209
01210     if (it) {
01211         wblog(F_L,"WRN contract::match() check missing conj flag\n"
01212         "A: %-15s -> %s\n"
01213         "B: %-15s -> %s", STR_(this), STR(ia), STR(B), STR(ib));
01214         mexWRN("applying conj(A) flag");
01215         ia.Conj();
01216     }
01217 }
01218 else if (F || !L) { wblog(F_L,
01219 "ERR %s() empty set of matching itags\n      %s%s\n X %s%s", FCT,
01220 STR_(this), ia.conj ? "*:" : "",
01221 STR(B),      ib.conj ? "*:" : "");
01222 }
01223
01224 if (C) { C->init( len + B.len - 2*nc );
01225 if (C->len) { l=0; itag_ *c=C->data;
01226
01227     for (i=0; i<len; ++i) { if (!ma[i]) {
01228         c[l]= data[i]; if (ia.conj) { c[l].Conj(); }; ++l;
01229     }}
01230     for (i=0; i<B.len; ++i) { if (!mb[i]) {
01231         c[l]=B.data[i]; if (ib.conj) { c[l].Conj(); }; ++l;
01232     }}
01233     if (l!=C->len) wblog(F_L,"ERR %s() %d/%d",FCT,l,C->len);
01234 }}
01235
01236 if (ma_) { ma.save2(*ma_); }
01237 if (mb_) { mb.save2(*mb_); }
01238
01239 return nc;
01240 };
01241
01242 int iTags::UpdateItagsCtr(const char *F, int L,
01243     const ctrIdx &ia, iTags &B,
01244     const ctrIdx &ib,
01245     const char *tag
01246 ){
01247     int q, s=0, e=0; unsigned i=0, k=0, l;
01248
01249     if (!ia || ia.len!=ib.len) wblog(F_L,
01250     "ERR %s() invalid usage (ia.len=%d/%d)",FCT,ia.len,ib.len);
01251     if (ia.len>15) wblog(F_L,
01252     "ERR %s() contracting %d>15 indices not supported",FCT,ia.len);
01253
01254     if (ia.conj) { this->Conj(); }
01255     if (ib.conj) { B.Conj(); }
01256
01257     for (; i<ia.len; ++i) {
01258         if (ia[i]>= len) wblog(F_L,
01259         "ERR %s() index out of bounds (A: %s / %d)",FCT,STR(ia), len);
01260         if (ib[i]>=B.len) wblog(F_L,
01261         "ERR %s() index out of bounds (B: %s / %d)",FCT,STR(ib),B.len);
01262
01263         itag_ &a=this->data[ia[i]], &b=B.data[ib[i]];
01264         q=a.sameAs(b,'1');
01265
01266         if (!q && (l=strlen(QS_ITAG_))) {
01267             if (!strcmp((const char*)a, QS_ITAG_,l)) { a.init_tag(); } else
01268             if (!strcmp((const char*)b, QS_ITAG_,l)) { b.init_tag(); }
01269             else { l=0; }
01270
01271             if (!l || !(q=a.sameAs(b,'1'))) {
01272                 e+=l; continue;
01273             }
01274         }
01275
01276         if (i) { if (SGN(q)!=s) {
01277             e+=(1<<4); continue;
01278         }}

```

```

01279     else { s=SGN(q); }
01280
01281     if ((q==abs(q))==3) {
01282         if (a.isEmpty_m())
01283             { q|=4; }
01284         else { continue; }
01285     }
01286
01287     if (q==1) { b.tSet(a); } else
01288     if (q==2) { a.tSet(b); } else
01289     if (q& 4) {
01290         unsigned n=9; char s[n];
01291         if (!tag) wblog(F_L,"ERR %s() got null tag (%p)",FCT,tag);
01292         l=snprintf(s,n,"%s%d",tag,++k);
01293         if (q>4 && l+1<n) {
01294             char c=((char&)a)&127;
01295             if (c) { s[l]=c; s[+1]=0; }
01296         }
01297         if (l>=n) { s[n-1]=0; wblog(FL,
01298             "ERR %s() string out of bounds (%s: %d/%d)",FCT,s,l,n); }
01299         a.Set(s); b.Set(s);
01300     }
01301     else wblog(F_L,"ERR %s() q=%d",FCT,q);
01302 }
01303
01304 if (s>=0) { e|=(1<<8); }
01305
01306 if (e && (F || L)) {
01307     sprintf_str("' %s' @ %s / '%s' @ %s (e=%d)",
01308         STR_(this), STR(ia), STR(B), STR(ib), e);
01309     if (e&15) wblog(F_L,"ERR %s() itag mismatch %s",FCT,str);
01310     else if (unsigned((e>=4)&15) + 1 >= ia.len)
01311         wblog(F_L,"ERR %s() missing conj-flag %s", FCT,str);
01312     else wblog(F_L,"ERR %s() itag conj mismatch %s",FCT,str);
01313 }
01314
01315 if (!e) {
01316     FlagItagsCtr(F_L,ia,B,ib,0,-1);
01317 }
01318
01319 return e;
01320 };
01321
01322 int iTags::FlagItagsCtr(
01323     const char *F, int L, const ctrIdx &ia,
01324     iTags &B,
01325     const ctrIdx &ib,
01326     char all,
01327     char sgn
01328 ) {
01329
01330     int rval=0;
01331     unsigned i,j,l, id=0;
01332     wvector<char> ma(len), mb(B.len);
01333
01334     if (!ia.len || !ib.len) {
01335         if (!len && !B.len && !ia.len && !ib.len) { return rval; }
01336     }
01337
01338     if (sgn)
01339         { sgn=SGN(sgn); }
01340     else { sgn=((!ia.conj) ^ (!ib.conj) ? SGN(+1) : SGN(-1)); }
01341
01342     for (i=0; i<ma.len; ++i) {
01343         if (!data[i].isEmpty()) { ma[i]|=1; }
01344         for (j=i+1; j<ma.len; ++j) {
01345             if (data[i]==data[j]) {
01346                 if (!(ma[i]&2)) { ma[i]|=2; id+=1; }
01347                 if (!(ma[j]&2)) { ma[j]|=2; id+=1; }
01348             }
01349         }
01350     }
01351     for (i=0; i<mb.len; ++i) {
01352         if (!B.data[i].isEmpty()) { mb[i]|=1; }
01353         for (j=i+1; j<mb.len; ++j) {
01354             if (B.data[i]==B.data[j]) {
01355                 if (!(mb[i]&2)) { mb[i]|=2; id+=256; }
01356                 if (!(mb[j]&2)) { mb[j]|=2; id+=256; }
01357             }
01358         }
01359     }
01360
01361     if (ia.len!=ib.len) wblog(FL,
01362         "ERR %s() invalid pair of ctr-indices %s / %s",FCT,STR(ia),STR(ib));
01363     for (i=0; i<ia.len; ++i) {
01364         if (ia[i]>=ma.len || ma[ia[i]]>3) wblog(F_L,
01365             "ERR %s() invalid ctr-index ica = %s (r=%d)",FCT,STR(ia),ma.len);

```

```

01366     ma[ia[i]]+=((1+ib[i])<<2);
01367 }
01368 for (i=0; i<ib.len; ++i) {
01369     if (ib[i]>mb.len || mb[ib[i]]>3) wblog(F_L,
01370         "ERR %s() invalid ctr-index icb = %s (r=%d)",FCT,STR(ib),mb.len);
01371     mb[ib[i]]+=((1+ia[i])<<2);
01372 }
01373
01374 for (j=0; j<B.len; ++j) { if (mb[j]<4 && ((mb[j]&1) || all)) {
01375     for (i=0; i< len; ++i) { if (ma[i]<4 && ((ma[i]&1) || all) &&
01376         SGN(data[i].sameAs(B.data[j]))==sgn) {
01377         B.data[j].SetFlags(-1);
01378         if (mb[j]&2) { mb[j]-=2; id-=256; }
01379         ++rval; break;
01380     }}
01381 }
01382
01383 if (int(id)<=0) {
01384     if (id) wblog(FL,"ERR %s() got id=%d !?",FCT,id);
01385     return rval;
01386 }
01387
01388 for (i=0; i<ia.len; ++i) { j=ia[i]; l=ib[i];
01389     if (l>B.len) wblog(FL,
01390         "ERR %s() index out of bounds (%d/%d)",FCT,l,B.len);
01391     if (ma[j]&2 || mb[l]&2) {
01392         if (SGN(data[j].sameAs(B.data[l]))!=sgn) wblog(FL,
01393             "ERR %s() invalid ctr-indices %s / %s (sgn=%d)",
01394             FCT,STR_(this),STR(B),sgn);
01395
01396         data[j].SetFlags(-2); if (ma[j]&2) { ma[j]-=2; id-= 1; }
01397         B.data[l].SetFlags(-1); if (mb[l]&2) { mb[l]-=2; id-=256; }
01398         rval+=2;
01399     }
01400 }
01401
01402 if (int(id)<=0) {
01403     if (id) wblog(FL,"ERR %s() got id=%d !?",FCT,id);
01404     return rval;
01405 }
01406
01407 for (i=0; i<len; ++i) {
01408     for (j=i+1; j<len; ++j) { if (data[i]==data[j]) {
01409         if (ma[i]>=4 || ma[j]>=4) {
01410             wblog(FL,"ERR %s() A: (%d,%d) %d/%d",FCT, i,ma[i],ma[j]); }
01411         if ((ma[j]&1) || all) { data[j].SetFlags(-1); }
01412     }}
01413 }
01414
01415 for (i=0; i<B.len; ++i) {
01416     for (j=i+1; j<B.len; ++j) { if (B.data[i]==B.data[j]) {
01417         if (mb[i]>=4 || mb[j]>=4) {
01418             wblog(FL,"ERR %s() B: (%d,%d) %d/%d",FCT, i,mb[i],mb[j]); }
01419         if ((mb[j]&1) || all) { B.data[j].SetFlags(-1); }
01420     }}
01421 }
01422
01423 #ifndef WB_SKIP_ASSERT
01424 if (all) { CtrIdx ia_,ib_;
01425     this->getCtr(F,L,B,ia_,ib_);
01426     if (ia_.len!=ib_.len) { wblog(F_L,"ERR %s() ctr-index mismatch\n"
01427         "    A: %s | %s -> %s\n"
01428         "    B: %s | %s -> %s", FCT,
01429         STR_(this),STR(ia),STR(ia_), STR(B),STR(ib),STR(ib_));
01430     }
01431 }
01432 #endif
01433
01434 return rval;
01435 };
01436
01437 int iTags::findRegEx(const char *pat) const {
01438
01439     regex_t r; int e, k=-1;
01440
01441     unsigned i, m=0, l=ITAG_LEN+2;
01442     char s[l+1]; s[l]=0;
01443     wbsting ptx;
01444
01445     if (!pat || !pat[0]) wblog(FL,"ERR got empty itag regexp");
01446
01447     if (strstr(pat,"\\d") && strlen(pat)<32) { unsigned j=0; str[0]=0;
01448         for (i=0; pat[i]; ++i) {
01449             if (pat[i]=='\\' && pat[i+1]=='d') { strcpy(str+j,"[0-9]"); j+=5; ++i; }
01450             else if (pat[i]=='[') {
01451                 if (strstr(pat+i,"\\d")) wblog(FL,
01452                     "WRN failed to translate non-posix \\d -> [0-9]\n"

```

```

01453         "in itag regexp '%s'",FCT,pat));
01454         strcpy(str+j,pat+i); break;
01455     }
01456     else { str[j++] = pat[i]; }
01457 }
01458 str[j]=0; ptx=str;
01459 pat=ptx.data;
01460 }
01461
01462 if ((e=regcomp(&r,pat, REG_EXTENDED | REG_NOSUB))) {
01463     return -1;
01464 }
01465
01466 for (i=0; i<len; ++i) {
01467     data[i].to_str(s,l);
01468     if (!regexec(&r,s,0,NULL,0)) {
01469         if (!m || !data[k].sameAs(data[i])) { k=i; ++m; }
01470     }
01471 }
01472 regfree(&r);
01473
01474 if (m>1) {
01475     return -m;
01476 }
01477
01478 return (k+1);
01479 };
01480
01481 int iTags::RegEx_replace(const char *F, int L, const char *rstr) const {
01482
01483     int nrep=0;
01484     if (!rstr || !rstr[0]) { return nrep; }
01485     if (strcmp(rstr,"s/",2) || !rstr[2]) wblog(F_L,
01486         "ERR invalid itag regex '%s'\nexpecting s/*/*/*",rstr);
01487
01488     unsigned j, i=0, i1=0, i2=0, e=0;
01489     char icalse=0, gflag=0; const char *s=rstr+2;
01490
01491     for (; s[i]; ++i) { if (s[i]=='/') { i1=(++i); break; } }
01492     for (; s[i]; ++i) { if (s[i]=='/') { i2=(++i); break; } }
01493     for (; s[i]; ++i) {
01494         if (s[i]=='i') { ++icalse; } else
01495         if (s[i]=='g') { ++gflag; } else { break; }
01496     }
01497     if (!i1 || !i2 || s[i] || icalse>1 || gflag>1) wblog(F_L,
01498         "ERR invalid itag regex '%s'\nexpecting s/*/*/*",rstr);
01499
01500     char rst_[strlen(rstr+2)+1];
01501     strcpy(rst_,rstr+2); rst_[i1-1]=rst_[i2-1]=0;
01502
01503     wbsting t_(ITAG_LEN+1);
01504     char c, *t=t_.data; t[ITAG_LEN]=0;
01505
01506     for (i=0; i<len; ++i) { char *s=(char *) (data+i);
01507         memcpy(t,s,ITAG_LEN);
01508         for (j=0; j<ITAG_LEN; ++j) { t[j] &= 127; }
01509
01510         if (t_.RegEx_replace(F_L,rst_,rst_+i1,icalse,gflag)) { ++nrep;
01511             t=t_.data;
01512             for (j=0; j<ITAG_LEN && t[j]; ++j) {
01513                 c=(t[j]&127); if (c!=t[j]) { ++e; }
01514                 s[j] = (s[j]&128) + c;
01515             }
01516             if (e || (j==ITAG_LEN && t[j])) wblog(FL,"ERR %s() "
01517                 "new itag out of bounds (%s; %d/%d)",FCT,t,strlen(t),ITAG_LEN);
01518             for (; j<ITAG_LEN; ++j) { s[j] &= 128; }
01519         }
01520     }
01521     return nrep;
01522 };
01523
01524 template <class T>
01525 sparseIndex2D<T>& sparseIndex2D<T>::Setup(char use_rmaj) {
01526
01527     if (IJ.dim2!=2) wblog(FL,
01528         "ERR %s() got invalid IJ (%s)",FCT,IJ.sizeStr().data);
01529     if (S.len!=2) wblog(FL,
01530         "ERR %s() got invalid S (len=%d)",FCT,S.len);
01531
01532     const T *s=S.data;
01533
01534     if (IJ.dim1<=1) { P.init(IJ.dim1);
01535         cIdx.init(s[1]+1);
01536         if (IJ.dim1) { T *idx=IJ.data;
01537             if (idx[0]>=s[0] || idx[1]>=s[1]) wblog(FL,
01538                 "ERR %s() index out of bounds (%ld,%ld) having (%dx%d) !?",
01539                 FCT,idx[0],idx[1],s[0],s[1]);

```

```

01540         for (T i=idx[l]+1; i<=s[l]; ++i) cIdx[i]=1;
01541     }
01542     else P.init();
01543     return *this;
01544 }
01545
01546 T j=0, l=0, lmax=IJ.diml-1, *idx=IJ.data;
01547
01548 if (!use_rmaj || s[0]>128 || (s[0]>1 && s[1]<(1<<24))) {
01549     cIdx.init(s[l]+1); rmaj=0;
01550     IJ.SortRecs(P,+1, -1);
01551
01552     while (1) {
01553         for (; j<=idx[l]; ++j) { cIdx[j]=1; }
01554         for (; l<lmax && idx[l]==idx[3]; ++l, idx+=2) {
01555             if (idx[0]>=idx[2]) wblog(FL,
01556                 "ERR %s() IJ not sorted or unique!",FCT);
01557             ++l; if (l>=lmax) { ++l; break; }
01558
01559             if (idx[l]>idx[3]) wblog(FL,"ERR %s() IJ(:,2) not sorted!",FCT);
01560             if (idx[0]>=s[0]) wblog(FL,
01561                 "ERR %s() index out of bounds (%ld/%ld) !?",FCT,idx[0],s[0]);
01562             ++l; idx+=2;
01563         }
01564     }
01565     else {
01566         cIdx.init(s[0]+1); rmaj=1;
01567         IJ.SortRecs(P,+1, +1);
01568
01569         while (1) {
01570             for (; j<=idx[0]; ++j) { cIdx[j]=1; }
01571             for (; l<lmax && idx[0]==idx[2]; ++l, idx+=2) {
01572                 if (idx[l]>=idx[3]) wblog(FL,
01573                     "ERR %s() IJ not sorted or unique!",FCT);
01574                 ++l; if (l>=lmax) { ++l; break; }
01575
01576                 if (idx[0]>idx[2]) wblog(FL,"ERR %s() IJ(:,2) not sorted!",FCT);
01577                 if (idx[l]>=s[l]) wblog(FL,
01578                     "ERR %s() index out of bounds (%ld/%ld) !?",FCT,idx[l],s[l]);
01579                 ++l; idx+=2;
01580             }
01581         }
01582
01583         for (; j<cIdx.len; ++j) { cIdx[j]=1; }
01584
01585         return *this;
01586     };
01587
01588 template <class T>
01589 inline T sparseIndex2D<T>::cidx(T j) const {
01590     #ifndef WB_SKIP_ASSERT
01591         if (j>=cIdx.len) wblog(FL,
01592             "ERR %s() index out of bounds (%d/%d)",FCT,j,cIdx.len);
01593     #endif
01594     return cIdx.data[j];
01595 };
01596
01597 template <class T>
01598 inline T sparseIndex2D<T>::ridx(T ic) const {
01599     #ifndef WB_SKIP_ASSERT
01600         if (ic>=IJ.diml) wblog(FL,
01601             "ERR %s() index out of bounds (%d/%d)",FCT,ic,IJ.diml);
01602     #endif
01603     return IJ.data[2*ic];
01604 };
01605
01606 #endif
01607

```

5.37 wbindex.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : QSpace index routines
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *

```

```

00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_INDEX_HH__
00021 #define __WB_INDEX_HH__
00022
00023 // ----- //
00024 // index class
00025 // Wb, Apr19, 06
00026 // ----- //
00027
00028 class wbindex : public wbvector<widx_t> {
00029 public:
00030
00031     wbindex(widx_t n=0, char iflag=0) {
00032         if (iflag) Index(n); else init(n);
00033     };
00034
00035     wbindex(const ctrIdx& ic) { init(ic); };
00036
00037     explicit wbindex(widx_t n, const widx_t* d)
00038     : wbvector<widx_t>(n,d) {};
00039
00040     explicit wbindex(const char* s, unsigned offset=0) {
00041         int i=Str2Idx(s,*this,offset);
00042         if (i<=0) wblog(FL,"ERR %s() invalid index '%s' (%d)",FCT,s,i);
00043     };
00044
00045     wbindex& init(widx_t n=0, const widx_t *d=NULL) {
00046         wbvector<widx_t>::init(n,d);
00047         return *this;
00048     };
00049
00050     wbindex& init(const ctrIdx &ic);
00051
00052     wbindex& init(
00053         const char *F, int L, const mxArray *a, widx_t offset=0);
00054
00055     widx_t init(
00056         const char *F, int L, const mxArray *a,
00057         const char *istr, char check_type=1
00058     ){ return wbvector<widx_t>::init(F,L,a,istr,check_type); };
00059
00060     wbindex& init(const mxArray *a, widx_t offset=0) {
00061         return init(0,0,a,offset);
00062     };
00063
00064     wbindex& initGroup(
00065         const wbvector<widx_t> &P, wbvector<widx_t> &D
00066     ){
00067         widx_t i,d, k=0, ig=0, m=D.sum();
00068         if (m!=P.len) wblog(FL,
00069             "ERR %s() invalid group index (%d/%d)",FCT,m,P.len);
00070         init(P.len+D.len);
00071
00072         for (; ig<D.len; ++ig) {
00073             data[k+ig]=d=D[ig]; m=ig+1;
00074             for (i=0; i<d; ++i, ++k) data[k+m]=P[k];
00075         }
00076
00077         return *this;
00078     };
00079
00080     wbindex& initGroup0(
00081         const wbvector<widx_t> &P, wbvector<widx_t> &D
00082     ){
00083         widx_t l=0, i=0; init(D.len);
00084         for (; i<D.len; ++i) {
00085             if (l>=P.len) wblog(FL,"ERR %s() index out of bounds",FCT,l,P.len);
00086             data[i]=P[l]; l+=D[i];
00087         }
00088         return *this;
00089     };
00090
00091     void initStr(const char* s, widx_t offset=0) {
00092         int i=Str2Idx(s,*this,offset);
00093         if (i<=0) wblog(FL,"ERR %s() invalid index '%s' (%d)",FCT,s,i);
00094     };
00095
00096     bool isIndex(widx_t n=0) {
00097         if (n) {
00098             if (swidx_t(n)>0) {

```

```

00100         for (widx_t i=0; i<len; ++i) {
00101             if (data[i]!=(n+i)) return 0; }
00102     }
00103     else {
00104         n=(1-n-len); if (swidx_t(n)<0) return 0;
00105         for (widx_t i=0; i<len; ++i) {
00106             if (data[i]!=(n+i)) return 0; }
00107     }
00108 }
00109 else {
00110     for (widx_t i=0; i<len; ++i) { if (data[i]!=i) return 0; }
00111 }
00112 return 1;
00113 };
00114
00115 wbindex& Index(widx_t n) {
00116     init(n);
00117     if (n) { for (widx_t i=0; i<n; ++i) data[i]=i; }
00118     return *this;
00119 };
00120
00121 wbindex& Index_ex(widx_t n, widx_t ix) {
00122     widx_t i,k;
00123
00124     if (ix>=n) wblog(FL,
00125         "WRN index to be excluded of no relevance (%d/%d)",ix,n);
00126
00127     init(ix<n ? n-1 : n); if (n==0) return *this;
00128
00129     for (k=i=0; i<n; i++) if (i!=ix) data[k++]=i;
00130
00131     return *this;
00132 };
00133
00134 wbindex& Index(widx_t i1, widx_t i2) {
00135     if (i1>i2) { init(); return *this; }
00136     init(i2-i1+1);
00137     for (widx_t i=0; i<len; ++i) data[i]=i+i1;
00138
00139     return *this;
00140 };
00141
00142 wbindex& Index(widx_t i1, widx_t i2, int p) {
00143     if (i1>i2) { init(); return *this; }
00144
00145     widx_t n=i2-i1+1; p=(-p)%int(n);
00146     if (!p || n==1) { return Index(i1,i2); }
00147     if (p<0) p+=n;
00148
00149     init(n); {
00150         widx_t i, ix=i1+p, *d=data;
00151         for (i=ix; i<=i2; ++i, ++d) { (*d)=i; }
00152         for (i=i1; i< ix; ++i, ++d) { (*d)=i; }
00153     }
00154     return *this;
00155 };
00156
00157 wbindex& BlockIndex(const wvector<widx_t> &D, char sub=0) {
00158     widx_t i,j,d,l; init(D.sum());
00159
00160     if (sub==0) {
00161         for (l=i=0; i<D.len; ++i)
00162             for (d=D[i], j=0; j<d; ++j,++l) data[l]=i;
00163     }
00164     else {
00165         for (l=i=0; i<D.len; ++i)
00166             for (d=D[i], j=0; j<d; ++j,++l) data[l]=j;
00167     }
00168
00169     return *this;
00170 };
00171
00172 wbindex& BlockIndex(widx_t n, widx_t D) {
00173     widx_t i,j,l; init(n*D);
00174
00175     for (l=i=0; i<n; ++i)
00176         for ( j=0; j<D; ++j,++l) data[l]=i;
00177
00178     return *this;
00179 };
00180
00181 wbindex& BlockIndexI(widx_t n, widx_t D) {
00182     widx_t i,j,l; init(n*D);
00183
00184     for (l=j=0; j<D; ++j)
00185         for ( i=0; i<n; ++i,++l) data[l]=i;
00186

```



```

00187         return *this;
00188     };
00189
00190     wbindex& IndexTranspose(widx_t m, widx_t n) {
00191         widx_t i,j,l;
00192         init(n*m); if (!n || !m) return *this;
00193
00194         for (l=j=0; j<m; ++j) {
00195             data[l++]=j;
00196             for (i=1; i<n; ++i,++l) data[l]=data[l-1]+m;
00197         }
00198
00199         return *this;
00200     };
00201
00202     wbindex& operator= (const char* s) {
00203         int i=Str2Idx(s,*this,0);
00204         if (i<=0) wblog(FL,"ERR %s() invalid index '%s' (%d)",FCT,s,i);
00205         return *this;
00206     };
00207
00208     int extend2Perm(widx_t N, wbperm &P, char toend=0) const;
00209
00210     void toPerm(wbperm &P) const {
00211         P.initT(len, data);
00212
00213         if (!P.isValidPerm()) { print("this");
00214             wblog(FL, "WRN No valid perm."); }
00215     };
00216
00217     wbindex& invert(widx_t N, wbindex &I2, char uflag=1) const;
00218
00219     wbindex& Invert(widx_t N, char uflag=1) {
00220         wbindex X; save2(X);
00221         return X.invert(N,*this,uflag);
00222     };
00223
00224     wbindex flipIdx(widx_t ndim) const;
00225     char isUnique(widx_t imax, widx_t offset=0) const;
00226
00227     mxArray* toMx(const char tflag=0) const {
00228         return toMx_offset(1,tflag);
00229     };
00230
00231 protected:
00232 private:
00233
00234 };
00235
00236 wbperm::wbperm (const wbindex &P) {
00237     RENEW(P.len,P.data);
00238     if (!isValidPerm()) { dispInvalidPerm(FL); }
00239 };
00240
00241 class wbIndex : public wbvector<widx_t> {
00242 public:
00243
00244     wbIndex() : wbvector<widx_t>() {};
00245     wbIndex(const wbvector<widx_t> &S) { init(S); };
00246
00247     template<class TI>
00248     wbIndex(widx_t n, const TI *S) { init(n,S); };
00249
00250     wbIndex(const wbIndex &I) : wbvector<widx_t>() {
00251         if (!I.isEmpty()) {
00252             wbvector<widx_t>::init(I.len); SIZE=I.SIZE;
00253             if (len) --data[0];
00254         }
00255     };
00256
00257     wbIndex& init() {
00258         wbvector<widx_t>::init(); SIZE.init();
00259         return *this;
00260     };
00261
00262     wbIndex& init(const wbvector<widx_t> &S, bool pp=1) {
00263         const widx_t* const &s=S.data;
00264         for (widx_t k=0; k<S.len; ++k) if (!s[k]) {
00265             wbvector<widx_t>::init(); SIZE.init();
00266             return *this;
00267         }
00268
00269         wbvector<widx_t>::init(S.len); SIZE=S;
00270
00271         if (len) {
00272             if (pp)

```

```

00274         { --data[0]; }
00275     else { data[len-1]=SIZE[len-1]; }
00276 }
00277 return *this;
00278 };
00279
00280 template<class TI>
00281 wbIndex& init(unsigned n, const TI *S, bool pp=1) {
00282
00283     unsigned k=0; for (; k<n; ++k) { if (!S[k]) { break; }}
00284     if (k<n || (!n && !S)) {
00285         wbvector<widx_t>::init(); SIZE.init();
00286         return *this;
00287     }
00288
00289     if (n) {
00290         wbvector<widx_t>::init(n); SIZE.initT(n,S);
00291         if (!S) { wblog(FL,"ERR %s() got n=%d with null S",FCT,n); }
00292     }
00293     else {
00294         wbvector<widx_t>::init(1);
00295         SIZE.init(1); SIZE[0]=1;
00296     }
00297
00298     if (pp)
00299         { --data[0]; }
00300     else { data[len-1]=SIZE[len-1]; }
00301
00302     return *this;
00303 };
00304
00305 wbIndex& reset(bool pp=1) {
00306     if (len!=SIZE.len) wblog(FL,
00307         "ERR %s() severe size mismatch (%d/%d)",FCT,len,SIZE.len);
00308
00309     if (len) { wbvector<widx_t>::set(0);
00310         if (pp)
00311             { --data[0]; }
00312         else { data[len-1]=SIZE[len-1]; }
00313     }
00314     return *this;
00315 };
00316
00317 wbIndex& set(wbvector<widx_t> &I) {
00318     widx_t k=0, *const &i=I.data, *const &s=SIZE.data;
00319     if (I.len!=len) wblog(FL,
00320         "ERR wbIndex::%s() size mismatch (%d/%d)",FCT,I.len,len);
00321     for (; k<len; k++) if (i[k]>=s[k]) {
00322         wblog(FL,"ERR wbIndex::%s() index out of bounds (%d: %d/%d)",
00323             FCT,k+1,i[k],s[k]);
00324         data[k]=i[k];
00325     }
00326     return *this;
00327 };
00328
00329 bool isValid() const {
00330     if (len!=SIZE.len) return 0;
00331     for (widx_t i=0; i<len; ++i) { if (data[i]>=SIZE.data[i]) return 0; }
00332     return 1;
00333 };
00334
00335 void checkValid(const char *F, int L) const {
00336     if (!isValid()) wblog(F,L,
00337         "ERR %s() invalid hyperindex\n%s",FCT,STR_(this));
00338 };
00339
00340 bool operator++() {
00341     widx_t k=0, l=len-1, *const &s=SIZE.data;
00342
00343     if (len!=SIZE.len) wblog(FL,
00344         "ERR wbIndex::++ size mismatch (len=%d/%d)",len,SIZE.len);
00345     if (!len) { return 0; }
00346
00347     k=0; ++data[0];
00348     while (data[k]==s[k] && k<l) { data[k]=0; ++data[++k]; }
00349
00350     if (data[k]>=s[k]) {
00351         if (k!=l) wblog(FL,"WRN wbIndex::++ "
00352             "premature stop (%d/%d: %s)",k+1,len,STR_(this));
00353         return 0;
00354     }
00355     return 1;
00356 };
00357
00358 bool operator--() {
00359     widx_t k=0, l=len-1, *const &s=SIZE.data;
00360

```

```

00361         if (len!=SIZE.len) wblog(FL,
00362             "ERR wbIndex:-- size mismatch (%d/%d)",len,SIZE.len);
00363         if (!len) { return 0; }
00364
00365         k=0; data[0]--;
00366         while (swidx_t(data[k])==-1 && k<1) { data[k]=s[k]-1; --data[++k]; }
00367
00368         if (data[k]>=s[k]) {
00369             if (k!=1) wblog(FL,"WRN wbIndex:-- "
00370                 "premature stop (%d/%d: %s)",k+1,len,STR_(this));
00371             return 0;
00372         }
00373         return 1;
00374     };
00375
00376     bool isEmpty() const {
00377         if (len!=SIZE.len) wblog(FL,"ERR %d/%d",len,SIZE.len);
00378         if (!len) return 1;
00379         else {
00380             const widx_t *s=SIZE.data;
00381             for (widx_t i=0; i<len; i++) if (!s[i]) return 1;
00382         }
00383         return 0;
00384     };
00385
00386     bool gotmore() const {
00387         if (len!=SIZE.len) wblog(FL,"ERR %d/%d",len,SIZE.len);
00388         for (widx_t i=0; i<len; ++i) {
00389             if (data[i]+1<SIZE.data[i]) return 1; else
00390             if (data[i]>=SIZE.data[i]) wblog(FL,
00391                 "WRN %s() index out of bounds (%s)",FCT,STR_(this)
00392             );
00393         }
00394         return 0;
00395     };
00396
00397     bool isdiag() const {
00398         if (len%2) wblog(FL,"ERR %s() applies to "
00399             "even-rank objects only (%s)",FCT,SSTR_(this));
00400         for (widx_t r=len/2, i=0; i<r; ++i) {
00401             if (data[i]!=data[i+r]) { return 0; }
00402         }
00403         return 1;
00404     };
00405
00406     widx_t numel() const { return SIZE.prod(0); };
00407
00408     widx_t serial() const {
00409         widx_t l, k=len-1;
00410         const widx_t *s=SIZE.data;
00411
00412         if (!len) wblog(FL,"ERR wbIndex::%s() got empty object",FCT);
00413         for (l=data[k--]; k<len; --k) { l = l*s[k] + data[k]; }
00414         return l;
00415     };
00416
00417     widx_t serial(const widx_t *stride) const {
00418         widx_t l=0;
00419         if (!len) wblog(FL,"ERR wbIndex::%s() got empty object",FCT);
00420         if (!stride) wblog(FL,"ERR wbIndex::%s() got null strides",FCT);
00421         for (unsigned k=0; k<len; ++k) { l += data[k]*stride[k]; }
00422         return l;
00423     };
00424
00425     wstring sizeStr() const { return SIZE.toStrf("", "x"); };
00426     wstring toStr() const {
00427         char s[128]; snprintf(s,128,"[%s] %s",
00428             wvector<widx_t>::toStr().data, SSTR(SIZE));
00429         return s;
00430     };
00431
00432     mxArray* toMx() const {
00433         const char *fields[]={"S","idx"};
00434         mxArray *S=mxCreateStructMatrix(1,1,2,fields);
00435
00436         mxSetFieldByNumber(S,0,0, SIZE.toMx());
00437         mxSetFieldByNumber(S,0,1, ((*this)+1).toMx());
00438         return S;
00439     };
00440
00441     wvector<widx_t> SIZE;
00442
00443 protected:
00444 private:
00445
00446 };
00447

```

```

00448 template <class T=widx_t>
00449 class groupIndex {
00450
00451 public:
00452
00453     groupIndex() {};
00454
00455     groupIndex(
00456         const wbvector<T> &P_, const wbvector<T> &D_,
00457         const wperm_t **p=NULL
00458     ){ P=P_; D=D_; setup(); if (p) (*p)=P.data; };
00459
00460     groupIndex(const wbvector<T> &DI, T ng=-1) { init(DI,ng); }
00461
00462     void init() { P.init(); D.init(); IO.init(); };
00463
00464     groupIndex& init(
00465         const wbvector<T> &P_, const wbvector<T> &D_
00466     ){ P=P_; D=D_; setup(); return *this; };
00467
00468     groupIndex& initX(
00469         wbvector<wperm_t> &P_, wbvector<T> &D_,
00470         const wperm_t **p=NULL
00471     ){
00472         P_.save2(P); D_.save2(D); setup(); if (p) (*p)=P.data;
00473         return *this;
00474     };
00475
00476     groupIndex& init(const wbvector<T> &DI, T ng=-1) {
00477         if (DI.isEmpty()) { init(); return *this; }
00478
00479         if (long(ng)<0) {
00480             T n=0,l=0;
00481             for (; l<DI.len; ++n, l+=(1+DI[l]));
00482             if (l!=DI.len) wblog(FL,
00483                 "ERR %s() invalid group index (%d/%d)",FCT,l,DI.len);
00484             ng=n;
00485         }
00486         P.init(DI.len-ng); D.init(ng);
00487
00488         T i,d,k=0,l=0; T *dp=P.data;
00489         while (l<DI.len) { d=DI.data[l++];
00490             for (i=0; i<d; ++i, ++k, ++l) { dp[k]=DI.data[l]; }
00491         }
00492         if (k!=P.len || l!=DI.len) wblog(FL,
00493             "ERR %s() %d/%d %d/%d",FCT,k,P.len,l,DI.len);
00494         setup(); return *this;
00495     };
00496
00497     groupIndex& init(T ng) {
00498         D.init(ng); IO.init(ng+1); P.init();
00499         return *this;
00500     };
00501
00502     groupIndex& init_g(T ig, T nz) {
00503         if (ig>D.len || ig+1>=IO.len) wblog(FL,"ERR %s() "
00504             "index out of bounds (%d/%d,%d)",FCT,ig,D.len,IO.len-1);
00505         if (ig==0) IO[ig]=0;
00506         IO[ig+1]=nz; D[ig]=nz-IO[ig];
00507         return *this;
00508     };
00509
00510     groupIndex& setD(T i, T nz) {
00511         if (i>=D.len) wblog(FL,"ERR %s() "
00512             "index out of bounds (%d/%d)",FCT,i,D.len);
00513         D[i]=nz; return *this;
00514     };
00515
00516     T init_ic() {
00517         if (IO.len!=D.len+1) wblog(FL,"ERR %s() "
00518             "IO not yet initialized !? (%d/%d+1)",FCT,IO.len,D.len+1);
00519         T i=0; IO[0]=0;
00520         for (; i<D.len; ++i) { IO[i+1]=IO[i]+D[i]; }
00521         return IO[i];
00522     };
00523
00524     groupIndex& setup(char full=0) {
00525         T n=D.cumsum(IO,'N'); if (n!=P.len) wblog(FL,
00526             "ERR %s() severe size mismatch (%d/%d)",FCT,n,P.len);
00527         if (full) { Ig.init(P.len);
00528             for (T l=0, j=0, d=0, i=0; i<D.len; ++i, l+=d) {
00529                 for (d=D[i], j=0; j<d; ++j) { Ig[P[l+j]]=i; }}
00530         }
00531         return *this;
00532     };
00533
00534     T dim(T ig) {

```

```

00535         if (ig>=D.len) wblog(FL,
00536             "ERR %s() index out of bounds (%d/%d)",FCT,ig,D.len);
00537         return D.data[ig];
00538     };
00539
00540     T numGroups() const { return D.len; };
00541
00542     void getGroup(T ig, T* &p0, T &d) const {
00543         if (ig>=D.len || I0.len!=D.len+1) wblog(FL,
00544             "ERR %s() index out of bounds (%d/%d/%d)",FCT,ig,I0.len-1,D.len);
00545         p0=P.data+I0.data[ig];
00546         d=D.data[ig];
00547     };
00548
00549     T getGroup(T ig, T &j, T &j2) const {
00550         if (long(ig)==-1) return 0;
00551         if (ig>=D.len || I0.len!=D.len+1) wblog(FL,
00552             "ERR %s() index out of bounds (%d/%d/%d)",FCT,ig,I0.len-1,D.len);
00553         j =I0.data[ig ];
00554         j2=I0.data[ig+1]; return ig+1;
00555     };
00556
00557     T getGroup(T ig) const {
00558         if (long(ig)==-1) return 0;
00559         if (ig>=I0.len) wblog(FL,"ERR %s() "
00560             "index out of bounds (%d/%d/%d)",FCT,ig,I0.len-1,D.len);
00561         return I0.data[ig];
00562     };
00563
00564     template <class T2>
00565     void getRecs0(const wbMatrix<T2> &, wbMatrix<T2> &, T m) const;
00566
00567     T groupIter(T &ig, T &j, T &j2) const {
00568         if (long(ig)>=0) { ++ig; } else { ig=0;
00569             if (I0.len!=D.len+1) wblog(FL,"ERR %s() "
00570                 "index out of bounds (%d/%d/%d)",FCT,ig,I0.len-1,D.len);
00571         }
00572         if (ig<D.len) {
00573             j =I0.data[ig ];
00574             j2=I0.data[ig+1];
00575         }
00576         else {
00577             ig=-1;
00578         }
00579
00580         return ig+1;
00581     };
00582
00583     mxArray* toMx() const {
00584         const char *fields[]={"P","D","I0","pp"};
00585         mxArray *S=mxCreateStructMatrix(1,1,4,fields);
00586
00587         mxSetFieldByNumber(S,0,0,(P+1).toMx());
00588         mxSetFieldByNumber(S,0,1,D.toMx());
00589         mxSetFieldByNumber(S,0,2,(I0+1).toMx());
00590
00591         wbvector<T> p;
00592         mxArray *c=mxCreateCellMatrix(1,D.len);
00593
00594         for (T i=0; i<D.len; ++i) {
00595             p.init(D[i],P.data+I0[i]);
00596             mxSetCell(c,i,p.toMx());
00597         }
00598         mxSetFieldByNumber(S,0,3,c);
00599
00600         return S;
00601     };
00602
00603     wbvector<T> D;
00604     wbperm P;
00605
00606     wbvector<T> I0;
00607
00608     wbvector<T> Ig;
00609
00610 protected:
00611 private:
00612 };
00613
00614 #define IDT unsigned long
00615 #define ITAG_LEN sizeof(IDT)
00616
00617 #define CC_ITAG '*'
00618
00619 #define QS_MARK_DUAL '\n'
00620 #define IS_CHAR_MARK(c) ((c)>=32 && (c)<= QS_MARK_DUAL)
00621

```

```

00622 #define IS_ITAGS_SEP(c) ((c)==' ' || (c)==';' || (c)=='|')
00623 #define IS_CHAR_ITAGS(c) ((c)>32 && (c)<127)
00624 #define IS_CHAR_ITAG(c) (((c)>32 && (c)<127) && (c)!=CC_ITAG && !IS_ITAGS_SEP(c))
00625
00626 #define IT2STR(a) (a).itags.toStr().data
00627 #define IT2STR__ itags.toStr().data
00628
00629 class itag_ {
00630
00631 public:
00632
00633     itag_() : t(0) {};
00634     itag_(const char *F, int L, const char *s) { init(F,L,s); };
00635     itag_(const char *s) { init(FL,s); };
00636     itag_(const itag_ &x) : t(x.t) {};
00637
00638     itag_& init(const char* F, int L, const char *s, unsigned n=-99);
00639
00640     itag_& init(const char *s=NULL) { return init(FL,s); };
00641
00642     itag_& init_tag() {
00643         char *s = (char*)&t, c128=char(128);
00644         unsigned k=0, n=ITAG_LEN; for (; k<n; ++k) { s[k]&=c128; }
00645         return *this;
00646     };
00647
00648     itag_& init_qdir(char c) { t=0;
00649         if (c=='-' || c==CC_ITAG) Conj();
00650         else if (c!='+') wblog(FL,"ERR %s() invalid c=%c<%d>",FCT,c,c);
00651         return *this;
00652     };
00653
00654     template<class TQ, class TD>
00655     itag_& init(const char *F, int L,
00656         const wbvector< QSpace<TQ,TD> > &A, wbindx ia);
00657
00658 #ifndef NOMEX
00659     itag_& init(const char* F, int L, const mxArray *a);
00660 #endif
00661
00662     itag_& operator=(unsigned n) { t=n; return *this; }
00663     itag_& operator=(const char *s) { return init(FL,s); }
00664     itag_& operator=(const itag_ &x) { t=x.t; return *this; }
00665
00666     bool operator==(const itag_ &x) const { return t==x.t; };
00667     bool operator!=(const itag_ &x) const { return t!=x.t; };
00668
00669     char sameAs(const itag_ &x, char lflag=0) const;
00670
00671     bool isConj(const itag_ &x, char full=1) const {
00672         IDT c128=128;
00673         return (full ? (t^c128)==x.t : (t&c128) ^ (x.t & c128) );
00674     };
00675
00676     bool isConj_x(const itag_ &x, char lflag) const;
00677     int isValid() const;
00678
00679     bool isEmpty() const {
00680         return (t & ~(IDT)(128) ? 0 : 1);
00681     };
00682
00683     bool isEmpty(char lenient) const;
00684     bool isEmpty_m() const;
00685
00686     char getMark() const {
00687         char c=((char&)t) & 127;
00688         if (c && !IS_CHAR_MARK(c)) { c=0; }
00689         return c;
00690     };
00691
00692     explicit operator bool() const { return (t!=0); };
00693     bool operator! () const { return !t; };
00694
00695     explicit operator const char*() const {
00696         return (const char*)&t;
00697     };
00698
00699     itag_& Set(const char *s) { char c=isConj();
00700         init(FL,s);
00701         if (isConj()) wblog(FL,
00702             "ERR %s() unexpected usage (%s; %d)",FCT,STR_(this),c);
00703         if (c) Conj();
00704         return *this;
00705     };
00706
00707     itag_& tSet(const itag_ &x) {
00708         if (isConj() ^ x.isConj()) { t=x.t; Conj(); } else { t=x.t; }

```

```

00709     return *this;
00710 };
00711
00712 itag_ & Set_(const itag_ &x, char keep=0) {
00713     if (isEmpty() && !keep) { t=x.t; } else
00714     if (isConj() ^ x.isConj()) { Conj(); }
00715     return *this;
00716 };
00717
00718 itag_ & Conj() {
00719     ((char&)t) ^= char(128);
00720     return *this;
00721 };
00722
00723 itag_ & setConj() {
00724     ((char&)t) |= char(128);
00725     return *this;
00726 };
00727
00728 bool isConj() const {
00729     return (((char&)t)<0);
00730 };
00731
00732 itag_ & deConj() {
00733     char &c=((char&)t); if (c<0) { c+=char(128); }
00734     return *this;
00735 };
00736
00737 itag_ conj() const { itag_ x(*this); return x.Conj(); }
00738 itag_ deconj() const { itag_ x(*this); return x.deConj(); }
00739
00740 itag_ & SetFlag (unsigned l);
00741
00742 itag_ & SetFlags(unsigned l=-1, char check=1);
00743
00744 itag_ & UnsetFlag(unsigned l) {
00745     if (l>=ITAG_LEN) wblog(FL,
00746         "ERR %s() index out of bounds (%d/%d)",FCT,l,ITAG_LEN);
00747     ((char*)&t)[l] &= 127;
00748     return *this;
00749 };
00750
00751 itag_ & UnsetFlags() {
00752     char *s = (char*)&t;
00753     for (unsigned l=1; l<ITAG_LEN; ++l) { if (s[l]<0) {
00754         s[l]+=char(128);
00755     } }
00756     return *this;
00757 };
00758
00759 bool GotFlag(unsigned l) const {
00760     if (l>=ITAG_LEN) wblog(FL,
00761         "ERR %s() index out of bounds (%d/%d)",FCT,l,ITAG_LEN);
00762     return ( ((char*)&t)[l] < 0 );
00763 };
00764
00765 unsigned GotFlags(unsigned l=1) const {
00766     unsigned q=0;
00767     const char *s = (char*)&t;
00768     for (; l<ITAG_LEN; ++l) { if (s[l]<0) { q|=(1<<l); } }
00769     return q;
00770 };
00771
00772 itag_ & SetSI( const char *F, int L,
00773     const char *tag, unsigned k,
00774     char mflag=6,
00775     char nfmt=1);
00776
00777 itag_ & AppendChar(char q, const char *qs=NULL);
00778 itag_ & PrependChar(char q, const char *qs=NULL);
00779 int CheckFirstChar(char q, const char *qs=NULL);
00780
00781 itag_ & MarkDual(char m=QS_MARK_DUAL, bool always=1);
00782
00783 itag_ & AddTilde() { return AppendChar('~',"KD~"); };
00784 itag_ & SetM() { return AppendChar('~',"KD~"); };
00785
00786 itag_ & SetK() {
00787     if (CheckFirstChar('K',"AEX")>0)
00788     { return *this; }
00789     else { return AppendChar('K',"KD~"); }
00790 };
00791 itag_ & SetD() {
00792     if (CheckFirstChar('D',"AEX")>0)
00793     { return *this; }
00794     else { return AppendChar('D',"KD~"); }
00795 };

```

```

00796
00797 unsigned to_str(char *s, unsigned len, char cflag=1) const;
00798
00799 wbststring toStr(char cflag=1) const {
00800     wbststring s(ITAG_LEN+4);
00801     to_str(s.data,s.len,cflag);
00802     return s;
00803 };
00804
00805 mxArray* toMx() const { return toStr().toMx(); };
00806
00807 IDT t;
00808
00809 static void Reset(unsigned char q=-1) {
00810     if (char(q)<0) {
00811         if (itag_::flag_id & 2) { return; }
00812         else { q=4; }
00813     }
00814     if (q&1) wblog(FL,"ERR %s() invalid q=%d",FCT,q);
00815     if (q<4) { q+=4; }
00816     itag_::flag_id=q;
00817 };
00818
00819 static void iter_flag_id() {
00820     #pragma omp critical (iter_flag_id__)
00821     if ((itag_::flag_id+=4)<4) { itag_::flag_id+=4; }
00822 };
00823
00824 protected:
00825 private:
00826
00827     static unsigned char flag_id;
00828 };
00829
00830 unsigned char itag_::flag_id = 4;
00831
00832 class iTags : public wbvector<itag_> {
00833 public:
00834
00835     iTags(unsigned n=0) : wbvector<itag_>(n) {};
00836     iTags(const char *F, int L, const char *s) { init(F,L,s); };
00837     iTags(const char *s) { init(FL,s); };
00838     iTags(const iTags &b) { wbvector<itag_>::init(b); };
00839
00840     unsigned init(const char* F, int L, const char *s);
00841     unsigned init(const char *s) { return init(FL,s); };
00842
00843     iTags& init(unsigned n=0) {
00844         return (iTags&)wbvector<itag_>::init(n);
00845     };
00846     iTags& init(const iTags &b) {
00847         return (iTags&)wbvector<itag_>::init(b);
00848     };
00849     iTags& operator=(const iTags &b) {
00850         return (iTags&)wbvector<itag_>::init(b);
00851     };
00852
00853     iTags& Update(const iTags &b) {
00854         if (len!=b.len) { wblog(FL,
00855             "ERR %s() itag length mismatch (%d/%d) '%s' / '%s'",
00856             FCT,len,b.len,STR_(this),STR(b));
00857         }
00858         for (unsigned i=0; i<len; ++i) {
00859             if (!data[i].isEmpty()) { if (data[i]!=b.data[i]) {
00860                 wblog(FL,"ERR %s() itag mismatch '%s' / '%s'",
00861                     FCT,STR_(this),STR(b),len,b.len);
00862             } }
00863             else if (data[i].isConj() ^ b.data[i].isConj()) {
00864                 wblog(FL,"ERR %s() itag conj mismatch '%s' / '%s'",
00865                     FCT,STR_(this),STR(b),len,b.len);
00866             }
00867         }
00868         return init(b);
00869     };
00870
00871     iTags& tSet(const iTags &b) {
00872         if (len!=b.len) { wblog(FL,
00873             "ERR %s() itag length mismatch (%d/%d) '%s' / '%s'",
00874             FCT,len,b.len,STR_(this),STR(b));
00875         }
00876         for (unsigned i=0; i<len; ++i) { data[i].tSet(b.data[i]); }
00877         return *this;
00878     };
00879
00880 #ifndef LD_CLEBSCH_QS
00881     unsigned init_qdir(const char *s);

```



```

00883 #endif
00884 #ifndef NOMEX
00885     unsigned init(const char* F, int L, const mxArray *a);
00886 #endif
00887
00888     iTags& init3() {
00889         wvector<itag_>::init(3); data[2].Conj();
00890         return *this;
00891     };
00892
00893     iTags& init(size_t l1, const itag_* d1, size_t l2, const itag_* d2) {
00894         wvector<itag_>::init(l1,d1,l2,d2);
00895         return *this;
00896     };
00897
00898     unsigned Init(const char* F, int L,
00899         const char *s,
00900         unsigned k=-1,
00901         char lflag=-1);
00902
00903     iTags& Init(unsigned r, int k, const char **st);
00904
00905     iTags& initOp(itag_ t, const iTags &b, const char *ot="op");
00906
00907     iTags& init_alpha(
00908         unsigned r, unsigned s, const char *tag,
00909         char t,
00910         unsigned k);
00911
00912     unsigned Set(const char* F, int L, const char *tag,
00913         unsigned r=-1,
00914         const char *tom=NULL,
00915         unsigned m=-1,
00916         bool conjOM=1
00917     );
00918
00919     iTags& SetConj(const char* F, int L, unsigned i) {
00920         if (len) {
00921             if (i>=len) wblog(F_L,
00922                 "ERR %s() index out of bounds (%d/%d)",FCT,i,len);
00923             data[i].Conj();
00924         }; return *this;
00925     };
00926
00927     iTags& Conj() {
00928         for (unsigned i=0; i<len; ++i) { data[i].Conj(); }
00929         return *this;
00930     };
00931
00932     iTags& SetFlag(const char* F, int L, unsigned k, unsigned l) {
00933         if (len) {
00934             if (k>=len) wblog(F_L,
00935                 "ERR %s() index out of bounds (%d/%d)",FCT,k,len);
00936             data[k].SetFlag(l);
00937         }; return *this;
00938     };
00939
00940     iTags& SetFlags(
00941         const char* F, int L, unsigned k, unsigned l=-1, char check=1) {
00942         if (len) {
00943             if (k>=len) wblog(F_L,
00944                 "ERR %s() index out of bounds (%d/%d)",FCT,k,len);
00945             data[k].SetFlags(l,check);
00946         }; return *this;
00947     };
00948
00949     iTags& UnsetFlag(const char* F, int L, unsigned k, unsigned l) {
00950         if (len) {
00951             if (k>=len) wblog(F_L,
00952                 "ERR %s() index out of bounds (%d/%d)",FCT,k,len);
00953             data[k].UnsetFlag(l);
00954         }; return *this;
00955     };
00956
00957     iTags& UnsetFlags() {
00958         for (unsigned k=0; k<len; ++k) {
00959             data[k].UnsetFlags();
00960         }; return *this;
00961     };
00962
00963     iTags& init_tags() {
00964         for (unsigned k=0; k<len; ++k) data[k].init_tag();
00965         return *this;
00966     };
00967
00968     bool operator==(const iTags &b) const {
00969         if (len!=b.len) return 0;

```

```

00970         for (unsigned i=0; i<len; ++i) {
00971             if (data[i].sameAs(b.data[i], 'l') <= 0) return 0;
00972         }
00973         return 1;
00974     };
00975     bool operator!=(const iTags &b) const {
00976         return !((*this)==b);
00977     };
00978
00979     int findRegEx(const char *r) const;
00980     int RegEx_replace(const char *F, int L, const char *rstr) const;
00981
00982     int getCtr(
00983         const char *F, int L, const iTags &b,
00984         ctrIdx &ia, ctrIdx &ib, iTags *c=NULL,
00985         wbvector<char> *ma=NULL, wbvector<char> *mb=NULL
00986     ) const;
00987
00988     int UpdateITagsCtr(const char *F, int L,
00989         const ctrIdx &ia, iTags &b,
00990         const ctrIdx &ib,
00991         const char *tag
00992     );
00993
00994     int FlagITagsCtr(
00995         const char *F, int L, const ctrIdx &ia,
00996         iTags &B, const ctrIdx &ib,
00997         char all=1,
00998         char sgn=0
00999     );
01000
01001     bool sameConj(const iTags &b) const {
01002         if (len!=b.len) { return 0; }
01003         for (unsigned i=0; i<len; ++i) {
01004             if (data[i].isConj(b.data[i], 0)) { return 0; }
01005         }
01006         return 1;
01007     };
01008
01009     bool isOp(int r=-1, char lflag=0) const;
01010
01011     bool isOpX(unsigned r=-1) const;
01012
01013     bool isAtensor(unsigned r=-1) const;
01014
01015     bool got_qdir(const char *qdir, char fflag=0) const {
01016         unsigned i=0, n=(qdir? strlen(qdir) : 0);
01017         if (n<2) wblog(FL,
01018             "ERR %s() invalid qdir=%s !?", FCT, qdir? qdir: "(null)");
01019         if (len<n || (fflag && len!=n)) return 0;
01020
01021         for (; i<n; ++i) {
01022             if (qdir[i]=='+' ) { if ( data[i].isConj()) return 0; } else
01023             if (qdir[i]=='-' ) { if (!data[i].isConj()) return 0; }
01024             else wblog(FL, "ERR %s() invalid qdir='%s' !?", FCT, qdir);
01025         }
01026         return 1;
01027     };
01028
01029     unsigned SkipLast(unsigned n=1) {
01030         if (n<len) { len-=n; }
01031         else {
01032             if (n>len) wblog(FL,
01033                 "ERR %s() range out of bounds (%d/%d) !?", FCT, n, len);
01034             init();
01035         }
01036         return len;
01037     };
01038
01039     wbstring toStr(char vflag=1) const;
01040
01041     const char* toStrk(
01042         char *sout, unsigned l, unsigned k,
01043         const char *st="(itag?)", char cflag=0
01044     ) const {
01045         if (k>=len || data[k].isEmpty())
01046             { sprintf(sout, l, "%s", st?st: ""); }
01047         else { sprintf(sout, l, "%s", data[k].toStr(cflag).data); }
01048         return sout;
01049     };
01050
01051     mxArray* toMx() const {
01052         if (len) {
01053             mxArray* a=mxCreateCellMatrix(1, len);
01054             for (unsigned i=0; i<len; ++i) {
01055                 mxSetCell(a, i, data[i].toMx());
01056             }

```

```

01057         return a;
01058     }
01059     return mxCreateCellMatrix(0,0);
01060 };
01061
01062 int got_op_labels(const char *F=NULL, int L=0) const {
01063     int q=0, x=0;
01064     unsigned i=0, l=len/2;
01065     for (; i<l; ++i) {
01066         if (!(x=data[i].isValid())) wblog(F,L,
01067             "ERR %s() got invalid itags (%s)",FCT,STR_(this));
01068         if (data[i].isConj() || data[i].conj()!=data[i+1]) return 0;
01069         if (q>0) { if (q!=x) q=-1; } else
01070             if (!q) { q=x; }
01071     }
01072     return q;
01073 };
01074
01075 unsigned gotLabels() const {
01076     unsigned m=0;
01077     for (int q, k=0; k<int(len); ++k) {
01078         q=data[k].isValid();
01079         if (q>1) { ++m; } else if (!q) return 0;
01080     }
01081     return m;
01082 };
01083
01084 protected:
01085 private:
01086
01087     unsigned checkUnique(const char *F=NULL, int L=0) const {
01088         if (len) {
01089             unsigned i,j;
01090             for (i=0; i<len; ++i)
01091                 for (j=i+1; j<len; ++j) if (data[i].t==data[j].t) {
01092                     if (F) wblog(F,L,
01093                         "WRN got non-unique itags (%s)",STR_(this));
01094                     return 0;
01095                 }
01096         }
01097         return len;
01098     };
01099
01100 };
01101
01102 #define CTR_IS_NUM(c)      (((c)>='1' && (c)<='9') || ((c)>='a' && (c)<='z'))
01103
01104 #define CTR_IS_SEP_ANY(c) (isspace(c) || (c)==',' || (c)==';' )
01105 #define CTR_IS_CONJ(c)    ((c)=='*')
01106 #define CTR_IS_OTHER(c)   (CTR_IS_SEP_ANY(c) || CTR_IS_CONJ(c))
01107
01108 #define CTR_IS_VALID(c)   (CTR_IS_NUM(c) || CTR_IS_OTHER(c))
01109
01110 class ctrIdx : public wbvector<unsigned> {
01111 public:
01112
01113     ctrIdx() : conj(0) {};
01114
01115     ctrIdx(unsigned l, unsigned *d, char c=0)
01116         : wbvector<unsigned>(l,d), conj(c) {};
01117
01118     ctrIdx(const char *F, int L, const char *s, char c=0)
01119         : conj(c) { init(F,L,s); };
01120
01121     ctrIdx(const char *F, int L, const mxArray *a)
01122         : conj(0) { init(F,L,a); };
01123
01124     ctrIdx(const ctrIdx &I)
01125         : wbvector<unsigned>(I), conj(I.conj), newtags(I.newtags) {};
01126
01127     ctrIdx(const wbvector<unsigned> &I, char c=0)
01128         : wbvector<unsigned>(I), conj(c) {};
01129
01130     ctrIdx& init(unsigned l=0, unsigned *d=NULL, char c=0) {
01131         wbvector<unsigned>::init(l,d); conj=c;
01132         return *this;
01133     };
01134
01135     template <class T>
01136     ctrIdx& initT(unsigned l=0, T* d=NULL, char c=0) {
01137         wbvector<unsigned>::initT(l,d); conj=c;
01138         return *this;
01139     };
01140
01141     ctrIdx& init(const wbvector<unsigned> &I, char c=0) {
01142         wbvector<unsigned>::init(I); conj=c;
01143     };

```

```

01144         return *this;
01145     };
01146
01147     ctrIdx& init1(unsigned k, char c=0) {
01148         wbvector<unsigned>::init(1);
01149         data[0]=k; conj=c;
01150         return *this;
01151     };
01152
01153     ctrIdx& init2(unsigned k1, unsigned k2, char c=0) {
01154         wbvector<unsigned>::init(2);
01155         if (k1==k2) wblog(FL,"ERR %s() got k=(%d,%d) !?",FCT,k1,k2);
01156         data[0]=k1; data[1]=k2; conj=c;
01157         return *this;
01158     };
01159
01160     template<class TQ, class TD>
01161     ctrIdx& initOp(const char *F, int L,
01162         itag_ t1, const QSpace<TQ,TD> &B, const char *sopt);
01163
01164     int init(const char *F, int L, const char *s, unsigned offset=1);
01165
01166     ctrIdx& init(const char *F, int L, const mxArray *a);
01167     ctrIdx& init(const mxArray *a) { return init(FL,a); }
01168
01169     ctrIdx& Index(unsigned l, char c=0) {
01170         wbvector<unsigned>::init(1); conj=c;
01171         for (unsigned i=0; i<l; ++i) data[i]=i;
01172         return *this;
01173     };
01174
01175     int checkUniqueS(const char *F=0, int L=0, const unsigned r=-1) const;
01176
01177     ctrIdx& Sort(wbperm &P, unsigned r=-1) {
01178         if (len) {
01179             if (!isSorted())
01180                 { wbvector<unsigned>::Sort(P); }
01181             else { P.init(); }
01182             if (int(r)>=0) checkUniqueS(FL,r);
01183         }
01184         else { P.init(); }
01185         return *this;
01186     };
01187
01188     ctrIdx& sort(ctrIdx &I, wbperm &P, unsigned r=-1) const {
01189         if (len) {
01190             if (!isSorted())
01191                 { wbvector<unsigned>::sort(I,P); }
01192             else { I.init(*this); P.init(); }
01193             if (int(r)>=0) I.checkUniqueS(FL,r);
01194         }
01195         else { I.init(); P.init(); }
01196         return I;
01197     };
01198
01199     ctrIdx& Sort_(ctrIdx &J, unsigned r=-1) {
01200         if (len!=J.len) wblog(FL,
01201             "ERR %s() length mismatch (%s/%s) !?",FCT,STR_(this),STR(J));
01202         if (len) {
01203             if (!isSorted()) {
01204                 wbperm P; wbvector<unsigned>::Sort(P);
01205                 J.wbvector<unsigned>::Permute(P);
01206             }
01207             if (int(r)>=0) { checkUniqueS(FL,r); }
01208         }
01209         return *this;
01210     };
01211
01212     ctrIdx& invert(unsigned r, ctrIdx &I) const;
01213     ctrIdx& Invert(unsigned r) {
01214         ctrIdx I; save2(I);
01215         return I.invert(r,*this);
01216     };
01217
01218     ctrIdx& Conj() {
01219         conj=(conj ? 0 : 1);
01220         return *this;
01221     };
01222
01223     int extend2Perm(widx_t N, wbperm &P, char toend=0) const;
01224
01225     wbperm& extend2Perm( unsigned ra, unsigned ma,
01226         const ctrIdx &icb, unsigned rb, unsigned mb,
01227         const wbperm &pab, char conj, unsigned mc,
01228         wbperm &Pbc,
01229         ctrIdx &kb,
01230         ctrIdx &kcb,

```

```

01231     char Bflag=1
01232 ) const;
01233
01234     ctrIdx& save2(ctrIdx &J) {
01235         wbvector<unsigned>::save2(J);
01236         J.conj=conj; return J;
01237     };
01238
01239     mxArray* toMx() const { return toStr().toMx(); };
01240
01241     wbsting toStr(char xflag=0) const;
01242
01243     char conj;
01244
01245     iTags newtags;
01246
01247 protected:
01248 private:
01249
01250 };
01251
01252 bool isCtrIdx(const mxArray *a) {
01253     unsigned i=0, r=60;
01254
01255     if (Mx::IsDbIVector(0,0,a)) { wbvector<double> x(FL,a);
01256         if (mxGetM(a)!=1 || x.len>r) return 0;
01257
01258         for (; i<x.len; ++i) {
01259             if (x[i]<0 || double(int(x[i]))!=x[i] || x[i]>r) return 0;
01260         }
01261         if (!x.wbvector<double>::isUnique()) return 0;
01262         return 1;
01263     }
01264     else if (mxIsChar(a)) { ctrIdx idx; wbsting s(a);
01265         if (idx.init(0,0,s.data)<=0) return 0;
01266         for (; i<idx.len; ++i) { if (idx[i]>r) return 0; }
01267         if (!idx.wbvector<unsigned>::isUnique()) return 0;
01268         return 1;
01269     }
01270     return 0;
01271 };
01272 };
01273
01274 template <class T=SPIDX_T>
01275 class sparseIndex2D {
01276
01277 public:
01278
01279     sparseIndex2D() : rmaj(0) {};
01280
01281     sparseIndex2D& Setup(char use_rmaj=0);
01282
01283     T cidx(T j) const;
01284     T ridx(T ic) const;
01285
01286     wbsting sizeStr() const {
01287         wbsting s;
01288         if (S.len) { s=S.toStrf("","x"); } else { s="[]"; }
01289         return s;
01290     };
01291
01292     char rmaj;
01293     wbMatrix<T> IJ;
01294     wbvector<T> S;
01295
01296     wbperm P;
01297
01298     wbvector<T> cIdx;
01299
01300 protected:
01301 private:
01302
01303 };
01304
01305 #endif
01306

```

5.38 wbio.c

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : wblog (logging routines)
00004  *

```

```

00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_WBIO_C__
00021 #define __WB_WBIO_C__
00022
00023 /* ----- */
00024 /* ----- */
00025 // generate temporary file $TMPDIR/<p=myprog><f=">-<pid>.<x=log>
00026 // in $TMPDIR (must be defined in environment)
00027 // Wb, Apr30, 20
00028
00029 wbstring Wb::tmpfile(const char *F, int L,
00030     const char *t,
00031     const char *x,
00032     const char *f
00033 ) {
00034
00035     wbstring fout(64);
00036
00037     unsigned l; const char *p=getenv("TMPDIR");
00038     if (!p) wblog(F_L, "ERR TMPFILE not defined (null)"); else
00039     if (p[0]!='/' ) wblog(F_L, "ERR invalid TMPFILE '%s', p);
00040
00041     l=snprintf(fout.data, fout.len, "%s/%s-%d", p,
00042         f && f[0] ? f : myname, t? t:"", getpid(), x? x : ".log");
00043
00044     if (l>=fout.len) wblog(F_L,
00045         "ERR %s() string out of bounds (%d/%d)", FCT, l, fout.len);
00046
00047     return fout;
00048 };
00049
00050 void Wb::tmpFile::init(const char *F, int L,
00051     const char *f,
00052     const char *x,
00053     const char *mode
00054 ) {
00055
00056     if (fid) wblog(F_L,
00057         "ERR %s() already got initialized fid=%d", FCT, fid);
00058
00059     fout=tmpfile(F_L, f, x);
00060     fid=fopen(fout.data, mode && mode[0] ? mode : "a+");
00061
00062     if (!fid) wblog(F_L,
00063         "ERR %s() failed to open file\n'%s'", FCT, fout.data);
00064     else {
00065         fprintf(fid, "\n%-18s %5s %s open %s\n\n",
00066             shortFL(PF_L), "", Wb::TimeStamp('T').data, Wb::TimeStamp('D').data);
00067     }
00068
00069     if (F) wblog(PF_L, "I/O opening tmpfile '%s'", fout.data);
00070 };
00071
00072 int Wb::tmpFile::sepline(const char *x, unsigned n) {
00073     unsigned l=0;
00074
00075     if (!n || !ncall) { return l; }
00076     if (int(n)<0) { n=72; }
00077     else if (int(n)>90) n=90;
00078
00079     unsigned m=(x ? strlen(x) : 0); char s[n+1];
00080
00081     if (m<=l) { memset(s, m? x[0] : '-', n); }
00082     else {
00083         while (l+m<=n) { memcpy(s+l, x, m); l+=m; }
00084         if (l<n) { memcpy(s+l, x, n-l); }
00085     }
00086
00087     if (!omp_in_parallel()) { this->flush(); }
00088
00089     l=n; s[l]=0; {
00090         if (!fid) wblog(F_L, "ERR %s() tmpFile not yet opened !?", FCT);
00091         #pragma omp critical (__tmpFile_IO__)

```

```

00092     { fprintf(fid,"%s\n",s); }
00093 }
00094
00095 ncall=0;
00096
00097 return 1;
00098 };
00099
00100 int Wb::tmpFile::blogf(const char *F, int L, const char *fmt, ...) {
00101     va_list args; Wb::ARGV wd(&args);
00102     va_start(args,fmt); unsigned l=0;
00103
00104     char sx[8];
00105     snprintf(sx,8,"%d/%d",omp_get_thread_num(), omp_get_num_threads());
00106
00107     if ((++ncall)<1) { ncall=1; }
00108
00109     if (fid) {
00110         #pragma omp critical (__tmpFile_IO__)
00111         { l+=fprintf(fid,"%-18s %5s %s ",
00112             shortFL(F_L,18), sx, Wb::TimeStamp('T').data);
00113             l+=vfprintf(fid,fmt,args);
00114             l+=fprintf(fid,"\n");
00115         }
00116     }
00117     else {
00118         fprintf(stdout,"\n%s %s "
00119             "ERR %s() tmpFile closed / not yet opened !?\n\n",shortFLT, FCT);
00120     }
00121
00122     return 1;
00123 };
00124
00125
00126 void WbPrintMatrixC (
00127     const wbMatrix<wbcomplex> &M, const char *istr, int space
00128 ){
00129     unsigned i,j,l=0,n=64; char dstr[n], sfmt[64];
00130
00131     int rflag, iflag;
00132
00133     PRINTF("%s%s", istr, istr[0] ? "\n" : "");
00134
00135     if (space<1) snprintf(sfmt,n," %s");
00136     else          snprintf(sfmt,n," %%%ds", space);
00137
00138     for (i=0; i<M.dim1; i++) {
00139         for (j=0; j<M.dim2; j++) {
00140
00141             rflag = std::isnan(M(i,j).r);
00142             if (!rflag) {
00143                 if (M(i,j).r!=0.) {
00144                     rflag = (fabs(M(i,j).r)==1.) ? 1 : 2;
00145                     if (M(i,j).r<0) rflag = -rflag;
00146                 }
00147             }
00148
00149             iflag = std::isnan(M(i,j).i);
00150             if (!iflag) {
00151                 if (M(i,j).i!=0.) {
00152                     iflag = (fabs(M(i,j).i)==1.) ? 1 : 2;
00153                     if (M(i,j).i<0) iflag = -iflag;
00154                 }
00155             }
00156
00157             if (rflag) {
00158                 l=snprintf(dstr,n,"% g",M(i,j).r);
00159                 if (iflag) {
00160                     if (abs(iflag)>1)
00161                         l+=snprintf(dstr+l,n-l,"%+gi", M(i,j).i);
00162                     else l+=snprintf(dstr+l,n-l,"%ci", iflag==1? '+' : '-');
00163                 }
00164             }
00165             else {
00166                 if (iflag)
00167                     if (abs(iflag)>1)
00168                         l=snprintf(dstr,n,"% gi", M(i,j).i);
00169                     else l=snprintf(dstr,n,"%ci", iflag==1? ' ' : '-');
00170                 else { l=snprintf(dstr,n,"%g", M(i,j).r); }
00171             }
00172             PRINTF(sfmt,dstr);
00173         }
00174         PRINTF("\n");
00175     }
00176     PRINTF("\n");
00177 };
00178

```

```

00179 template<class T>
00180 int Str2Idx(
00181     const char *F, int L,
00182     const char* s_, wvector<T> &idx,
00183     T offset,
00184     char cmpct
00185 ){
00186     if (!s_) {
00187         wblog(F_L,"ERR %s() got null input !?",FCT);
00188         return -1;
00189     }
00190     if (!s_[0]) { idx.init(); return 0; }
00191
00192     int i,j, n=0, e=0, l=strlen(s_),
00193         ms=1,
00194         mx=0,
00195         ma=0;
00196
00197     char c, sx=0, bflag=0;
00198     char s[l+1], *s1=NULL, *s2=s;
00199     long int x;
00200
00201     strcpy(s,s_);
00202
00203     for (i=0; i<l && !e; ++i) { if (!isspace(c=s_[i])) {
00204         if (isalnum(c)) { ++ma; ms=0;
00205             if ((++n)>1 && !sx) { sx=''; }
00206             for (++i; i<l; ++i) { if (isalnum(s_[i])) ++ma; else break; }
00207             --i; continue;
00208         }
00209         else { ++mx; s[i]=' '; }
00210
00211         if (c=='\n' || c=='\r') { ++ms;
00212             if (n && ms==1)
00213                 { if (sx) { if (sx!=c) e=__LINE__; } else { sx=c; }}
00214             else { e=__LINE__; }
00215         }
00216         else if (c=='[') { if (n || bflag&1) e=__LINE__; else bflag|=1; }
00217         else if (c==']') { if (bflag&2) e=__LINE__; else { bflag|=2;
00218             for (++i; i<l && s_[i]; ++i) {
00219                 if (!isspace(s_[i])) { e=__LINE__; break; }
00220             }
00221         }}
00222         else { e=__LINE__; }
00223     }}
00224
00225     if (!n && !mx) { idx.init(); return 0; }
00226
00227     if (bflag && bflag!=3) {
00228         if (e) { if (e>0) e=-e; } else
00229             e=__LINE__;
00230     }
00231
00232     if (e<0) { if (F) wblog(F,L,
00233         "ERR %s() invalid '%s' (i=%d/%d; b=%d)",FCT,s_,i,l,bflag);
00234         return -i;
00235     }
00236
00237     if (e) { l=i; }
00238
00239     if (n==1 && cmpct) { n=ma; idx.init(n);
00240         for (j=i=0; i<l; ++i) {
00241             if (isalnum(s[i])) { c=s[i];
00242                 if (unsigned(j)>=idx.len) wblog(FL,
00243                     "ERR %s() j=%d/%d !?",FCT,l,idx.len);
00244                 if (c>='0' && c<='9') { idx[j++]=c-'0'; } else
00245                 if (c>='A' && c<='Z') { idx[j++]=c-'A'+10; } else
00246                 if (c>='a' && c<='z') { idx[j++]=c-'a'+36; }
00247                 else { e=__LINE__; ++i; break; }
00248             }
00249         }
00250     }
00251     else { idx.init(n);
00252
00253         for (n=0; n<=int(idx.len); ++n) { s1=s2;
00254             x=strtol(s1,&s2,0); if (s1==s2 || n==int(idx.len)) { break; }
00255
00256             if ((long int)T(x)!=x) wblog(F_L,
00257                 "ERR %s() invalid '%s' %g/%ld !?",FCT,s_,double(T(x)),x);
00258             idx[n]=T(x);
00259         }
00260         if (n!=int(idx.len)) { e=__LINE__; }
00261         if (!e) {
00262             if (s1!=s2) { l=s2-s; e=__LINE__; } else {
00263                 for (; *s2; ++s2) { if (!isspace(*s2)) { l=s2-s; e=__LINE__; }}}
00264         }
00265     }

```



```

00266
00267     if (e<0) { if (F) wblog(F,L,
00268         "ERR %s() invalid '%s' (%d/%d;%d)",FCT,s_,i,l,bflag);
00269         return -l;
00270     }
00271
00272     if (offset && e>=int(0)) {
00273         for (i=0; i<int(idx.len); ++i) {
00274             if (idx[i]>=offset) { idx[i]-=offset; } else { e=-__LINE__; }}
00275     }
00276
00277     if (e<0) {
00278         if (F) wblog(F,L,"ERR %s() invalid '%s' "
00279             "(e=%d; %d/%d, %d @ %d)",FCT,s_,-e,idx.len,l,bflag,offset);
00280         return -l;
00281     }
00282
00283     if (e) {
00284         return -l;
00285     }
00286
00287     return idx.len;
00288 };
00289
00290 template<class T>
00291 int Str2Idx(
00292     const char* s, wvector<T> &I, T offset, char cmpct
00293 ){ return Str2Idx(FL,s,I,offset,cmpct); };
00294
00295 template<class T>
00296 int Str2Idx(
00297     const char* F, int L,
00298     wvector<T> &I, const mxArray *a, T offset, char cmpct
00299 ){
00300     if (!a) wblog(FL,"ERR %s() got null mxArray",FCT);
00301
00302     if (mxIsChar(a)) { wstring s(a);
00303         int n=Str2Idx(0,0,s.data,I,offset,cmpct);
00304         if (n<0) wblog(F_L,
00305             "ERR %s() '%s' => [%s] @ e=%d !?",FCT,s.data,STR(I),n);
00306         return I.len;
00307     }
00308     else {
00309         I.init(F,L,a);
00310         if (offset) {
00311             if (I.anyLT(offset)) wblog(FL,
00312                 "ERR %s() index incompatible with offset=%d",FCT,offset);
00313             I-=offset;
00314         }
00315         return 0;
00316     }
00317 };
00318
00319 #endif
00320

```

5.39 wbio.h

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : wblog (logging routines)
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_WBIO_H__
00021 #define __WB_WBIO_H__
00022
00023 /* ----- */
00024 // tags: tmpfile, tempfile, tempmat, TMPDIR // Wb,Jun06,19
00025

```

```

00026 namespace Wb {
00027
00028     wbsting tmpfile(const char *F, int L,
00029         const char *t=NULL, const char *x=NULL, const char *f=NULL);
00030
00031     class tmpFile {
00032     public:
00033         tmpFile() : fid(NULL), ncall(-1) { };
00034
00035         tmpFile(const char *F, int L,
00036             const char *f=NULL, const char *x=NULL, const char *mode=NULL)
00037             : fid(NULL), ncall(-1) { init(F,L,f,x,mode); };
00038
00039         ~tmpFile() { if (fid) {
00040             wblog(PFL,
00041                 "I/O closing tmpfile '%s'",fout.data? fout.data:"(null!)");
00042             fprintf(fid,"\n%-18s %5s %s close %s\n\n",
00043                 shortFL(PFL),"", Wb::TimeStamp('T').data, Wb::TimeStamp('D').data);
00044             fclose(fid); fid=NULL;
00045         } };
00046
00047         void init(const char *F, int L,
00048             const char *f=NULL, const char *x=NULL, const char *mode=NULL);
00049
00050         int blogf(const char *F, int L, const char *fmt, ...);
00051
00052         int sepline(const char *w="-", unsigned len=-1);
00053
00054         void flush() { if (fid) fflush(fid); };
00055
00056         FILE *fid;
00057         int ncall;
00058         wbsting fout;
00059
00060     protected:
00061     private:
00062 };
00063
00064 };
00065
00066 template<class T>
00067 int Str2Idx(
00068     const char *F, int L,
00069     const char* s, wbsting<T> &idx, T offset=0, char cmpct=0
00070 );
00071
00072 template<class T>
00073 int Str2Idx(
00074     const char* s, wbsting<T> &idx, T offset=0, char cmpct=0
00075 );
00076
00077 template<class T> int Str2Idx(
00078     const char* s, wbsting<T> &idx, unsigned offset=0, char cmpct=0
00079 ){ return Str2Idx(s,idx,T(offset),cmpct); };
00080
00081 wbsting<unsigned> Str2Idx(
00082     const char *F, int L, const char* s, unsigned offset=0
00083 ){
00084     wbsting<unsigned> I;
00085     int e=Str2Idx(F_L,s,I,offset);
00086     if (e<0) wblog(FL,
00087         "ERR %s() invalid '%s' (%d;%d)",FCT,s?s:"null",-e,offset);
00088     return I;
00089 };
00090
00091 wbsting<unsigned> Str2Idx(const char* s, unsigned offset=0) {
00092     return Str2Idx(FL,s,offset);
00093 };
00094
00095 template<class T>
00096 int Str2Idx(
00097     const char* F, int L,
00098     wbsting<T> &I, const mxArray *a, T offset=0, char cmpct=0
00099 );
00100
00101 template<class T>
00102 int Str2Idx(
00103     wbsting<T> &I, const mxArray *a, T offset=0, char cmpct=0
00104 ){ return Str2Idx(FL,I,a,offset,cmpct); };
00105
00106 void WbPrintMatrixC (
00107     const wbsting<wbcomplex> &M, const char *istr="", int space=10);
00108
00109 #endif

```

5.40 wblib.h

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  *
00004  * Copyright 2024 Andreas Weichselbaum
00005  *
00006  * Licensed under the Apache License, Version 2.0 (the "License");
00007  * you may not use this file except in compliance with the License.
00008  * You may obtain a copy of the License at
00009  *
00010  *     http://www.apache.org/licenses/LICENSE-2.0
00011  *
00012  * Unless required by applicable law or agreed to in writing, software
00013  * distributed under the License is distributed on an "AS IS" BASIS,
00014  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00015  * See the License for the specific language governing permissions and
00016  * limitations under the License.
00017  * ----- */
00018
00019 #ifndef __WB_LIB_MEX_HCC__
00020 #define __WB_LIB_MEX_HCC__
00021
00022 /* ----- //
00023  * header file for mex programs
00024  * A. Weichselbaum (C) Jan 2006
00025  * ----- */
00026
00027 #define PP_STR__(a) #a
00028 #define PP_STRFY(a) PP_STR__(a)
00029
00030 #ifndef QS_USING_OMP
00031 #   ifdef _OPENMP
00032 #       define QS_USING_OMP 1
00033 #   else
00034 #       warning compiling QSpace without openMP
00035 #   endif
00036 #elif QS_USING_OMP<=0
00037 #undef QS_USING_OMP
00038 #endif
00039
00040 #if __linux__
00041     #define QS_UNUSED_VAR __attribute__((unused))
00042 #elif __unix__
00043     #warning got unix
00044 #elif __APPLE__
00045     #define QS_UNUSED_VAR __attribute__((unused))
00046 #else
00047     #error GOT UNSUPPORTED OPERATING SYSTEM
00048 #endif
00049
00050 #include <assert.h>
00051 #include <execinfo.h>
00052
00053 #include <time.h>
00054 #include <signal.h>
00055 #include <unistd.h>
00056 #include <fcntl.h>
00057 #include <regex.h>
00058 #include <regex>
00059
00060 #include <cstdio>
00061 #include <cstdlib>
00062 #include <chrono>
00063
00064 #include <exception>
00065
00066 #include <cmath>
00067
00068 #include <ctype.h>
00069 #include <climits>
00070 #include <stdarg.h>
00071 #include <string.h>
00072
00073 #include <string>
00074 #include <iostream>
00075 #include <typeinfo>
00076
00077 #include <sys/ipc.h>
00078 #include <sys/msg.h>
00079 #include <sys/types.h>
00080 #include <sys/stat.h>
00081
00082 #if __linux__
00083 #include <sys/sysinfo.h>
00084 #endif
00085

```

```

00086 #include <map>
00087 #include <queue>
00088 #include <unordered_map>
00089 #include <algorithm>
00090 #include <vector>
00091
00092 #ifdef QS_USING_OMP
00093 #include <omp.h>
00094 #else
00095     int omp_get_max_threads() { return 1; }
00096     int omp_get_num_threads() { return 1; }
00097     int omp_get_thread_num() { return 0; }
00098     int omp_in_parallel() { return 0; }
00099     int omp_get_level() { return 0; }
00100 #endif
00101
00102 #if __APPLE__
00103     #ifndef exp10
00104         inline double exp10(double x) { return pow(double(10),x); }
00105     #endif
00106     #ifndef sighandler_t
00107         #define sighandler_t sig_t
00108     #endif
00109
00110     #include <mach/mach_host.h>
00111     #include <mach/task.h>
00112
00113     #include <sys/sysctl.h>
00114     #undef setbit
00115     #undef isset
00116 #endif
00117
00118 #ifdef QS_USING_HPTT
00119 #include "hptt.h"
00120 #undef I
00121 #endif
00122
00123     unsigned ARG_CHECK=1;
00124
00125 #define STRLEN 1023
00126     char str[STRLEN+1];
00127
00128 #define sprintf_str(...) snprintf(str,STRLEN, __VA_ARGS__)
00129
00130 #define MX_CHECK_HELPER_NARGS(n1,n2,m) \
00131     if (nargin>0 && \
00132         checkHelpVersion(argin[0], nargout ? argout : NULL)) { return; }; \
00133     if (nargin<n1 || (n2>=0 && nargin>n2)) { \
00134         strcpy(str,"ERR invalid number of input args"); unsigned l=strlen(str); \
00135         if (nargin<n1) \
00136             { sprintf(str+1,32," (%d / %d required)",nargin,n1); } \
00137         else { sprintf(str+1,32," (%d / %d %s)",nargin,n2, \
00138             n1==n2? "expected":"at most"); }; \
00139         if (nargin || nargout) wblog(FL,str); else usage(FL,str); \
00140     }; \
00141     if (m>=0 && nargout>m) { \
00142         sprintf_str("ERR invalid number of output args (%d)",nargout); \
00143         if (nargin || nargout) wblog(FL,str); else usage(FL,str); \
00144     }
00145
00146 template <class T> class wbvector;
00147 template <class T> class wbMatrix;
00148 template <class T> class wbarray;
00149 template <class TD> class wbsparray;
00150 template <class TQ, class TD> class QSpace;
00151 template <class T> class sparseIndex2D;
00152
00153 class bitset;
00154 class wbcomplex;
00155 class wbindex;
00156 class wbperm;
00157 class wbstring;
00158
00159 class iTags;
00160 class ctrIdx;
00161
00162 namespace CG {
00163     class FileLock;
00164 }
00165
00166 namespace Wb {
00167     class Clock;
00168     class ClockSet;
00169 }
00170
00171 using namespace std;

```

```

00173
00174 #ifdef MATLAB_MEX_FILE
00175 class MXPut;
00176 #endif
00177
00178 #define ENABLE_IF_COMPLEX(T__) typename \
00179     std::enable_if< WbUtil<T__>::isComplex(), T__ >::type* = nullptr
00180
00181 #define ENABLE_IF_isPOD(T__) typename \
00182     std::enable_if< WbUtil<T__>::isPOD(), T__ >::type* = nullptr
00183
00184 #define ENABLE_IF_noPOD(T__) typename \
00185     std::enable_if< !WbUtil<T__>::isPOD(), T__ >::type* = nullptr
00186
00187 #define ENABLE_IF_isINT(T__) typename \
00188     std::enable_if<std::is_integral<T__>::value>* = nullptr
00189
00190 #if 1
00191     #define widx_t size_t
00192     #define swidx_t long
00193     #define wperm_t size_t
00194     #define swperm_t long
00195 #else
00196     #define widx_t unsigned
00197     #define swidx_t int
00198     #define wperm_t unsigned
00199     #define swperm_t int
00200 #endif
00201
00202 #if 1
00203     #define SPIDX_T size_t
00204     #define sSPIDX_T long
00205 #else
00206     #define SPIDX_T unsigned
00207     #define sSPIDX_T int
00208 #endif
00209
00210 #define WBINDEX wbvector<widx_t>
00211 #define WBPERM wbvector<wperm_t>
00212 #define WBIDXMAT wbMatrix<widx_t>
00213
00214 #define UVEC wbvector<unsigned>
00215 #define C_UVEC const wbvector<unsigned>
00216 #define DMAT wbMatrix<double>
00217 #define C_DMAT const wbMatrix<double>
00218 #define UMAT wbMatrix<unsigned>
00219 #define C_UMAT const wbMatrix<unsigned>
00220 #define IMAT wbMatrix<int>
00221 #define C_IMAT const wbMatrix<int>
00222 #define C_TMAT const wbMatrix<T>
00223 #define C_MEX const mxArray
00224 #define C_UINT const unsigned
00225 #define C_CHAR const char
00226
00227 #define DMAT0 wbMatrix<double>()
00228
00229 #define __FILE__ (file ? file : __FILE__), (line ? line : __LINE__)
00230
00231 #define F_L (F ? F : __FILE__), (F ? L : __LINE__)
00232 #define F_LF (F ? F : __FILE__), (F ? L : __LINE__), __FUNCTION__
00233 #define F_L_F (F ? F : __FILE__), (F ? L : __LINE__), (fct ? fct : __FUNCTION__)
00234
00235 #define FSTR __FILE__ ": "
00236 #define FLINE __FILE__, __LINE__
00237 #define FL __FILE__, __LINE__
00238 #define FLF __FILE__, __LINE__, __FUNCTION__
00239 #define FCT __FUNCTION__
00240 #define FCL __FUNCTION__, __LINE__
00241
00242 std::string getName(const std::type_info &type_id, char vflag=1);
00243
00244 #define PFL shortFL(__FILE__, __LINE__, -1, ""), WBL_GOT_SHORTFL
00245 #define PF_L shortFL(F?F:__FILE__, F?L:__LINE__, -1, ""), WBL_GOT_SHORTFL
00246
00247 #ifndef TIME
00248 #define TIME Wb::TimeStamp('T').data
00249 #endif
00250
00251 #ifndef shortFLT
00252 #define shortFLT shortFL(__FILE__, __LINE__), Wb::TimeStamp('T').data
00253 #define SHORT_FL shortFL(__FILE__, __LINE__)
00254 #endif
00255
00256 #ifndef PSTR
00257     #ifdef QS_USING_OMP
00258         #define PSTR Wb::ompID2Str().data
00259     #else

```

```

00260     #define PSTR ""
00261 #endif
00262 #endif
00263
00264 #ifndef Inf
00265 #define Inf FP_INFINITE
00266 #endif
00267
00268 #if !defined(MAX)
00269     template <class T> inline
00270     const T& MAX(const T&a, const T&b) { return ((b)>(a) ? (b) : (a)); }
00271 #endif
00272
00273     template <class T> inline
00274     const T& MAX3(const T&a, const T&b, const T&c) {
00275         if ((b)>(a))
00276             { return ((c)>(b) ? (c) : (b)); }
00277         else { return ((c)>(a) ? (c) : (a)); }
00278     }
00279
00280 #if !defined(MIN)
00281     template <class T> inline
00282     const T& MIN(const T&a, const T&b) { return ((b)<(a) ? (b) : (a)); };
00283 #endif
00284
00285     template <class T> inline
00286     const T& MIN3(const T&a, const T&b, const T&c) {
00287         if ((b)<(a))
00288             { return ((c)<(b) ? (c) : (b)); }
00289         else { return ((c)<(a) ? (c) : (a)); }
00290     };
00291
00292 #if !defined(SWAP)
00293     template <class T>
00294     inline void SWAP(T &a, T &b) { T x=a; a=b; b=x; }
00295 #endif
00296
00297 #if !defined(NSTR)
00298 #define NSTR(a)      Wb::num2Str(a ).data
00299 #endif
00300 #if !defined(NSTRf)
00301 #define NSTRf(a,f) Wb::num2Str(a,f).data
00302 #endif
00303
00304 #if !defined(RSTR)
00305 #define RSTR(a,n) wbstring().initrep(a,n).data
00306 #endif
00307
00308 #if !defined(SEC2STR)
00309 #define SEC2STR(a) Wb::sec2Str(a).data
00310 #endif
00311
00312 #if !defined(TSTR)
00313 #define TSTR(a) getName(typeid(a),1).c_str()
00314 #endif
00315
00316 #if !defined(sTSTR)
00317 #define sTSTR(a) getName(typeid(a),0).c_str()
00318 #endif
00319
00320 #if !defined(SSTR)
00321 #define SSTR(a) (a).sizeStr().data
00322 #endif
00323
00324 #if !defined(SSTR_)
00325 #define SSTR_(a) (a)->sizeStr().data
00326 #endif
00327
00328 #if !defined(SSTRM)
00329 #define SSTRM(S,r) (S).toStrf("%d","x",r,"_").data
00330 #endif
00331
00332 #if !defined(SSTRM_)
00333 #define SSTRM_(r,sd,m,sm) Wb::sizeStrM(r,sd,m,sm).data
00334 #endif
00335
00336 #if !defined(QSTR)
00337 #define QSTR(a) (a).qStr().data
00338 #endif
00339
00340 #if !defined(QSTR_)
00341 #define QSTR_(a) (a)->qStr().data
00342 #endif
00343
00344 #if !defined(STR)
00345 #define STR(a) (a).toStr().data
00346 #endif

```

```

00347
00348 #if !defined(STR_)
00349 #define STR_(a) (a)->toStr().data
00350 #endif
00351
00352 #if !defined(STR2)
00353 #define STR2(a,b) (a).toStr(b).data
00354 #endif
00355
00356 #if !defined(STR2_)
00357 #define STR2_(a,b) (a)->toStr(b).data
00358 #endif
00359
00360 #if !defined(cSTR)
00361 #define cSTR(a) Wb::char2Str(a).data
00362 #endif
00363
00364 #if !defined(I2STR)
00365 #define I2STR(a) Wb::int2Str(a).data
00366 #endif
00367
00368 #if !defined(RATS)
00369 #define RATS(a) Wb::rat2Str(__FILE__, __LINE__, a).data
00370 #endif
00371
00372 #if !defined(STRREP)
00373 #define STRREP(str,n) wbsting().initrep(str,n).data
00374 #endif
00375
00376 #if !defined(BITS)
00377 #define BITS(a) Wb::bits(a).data
00378 #endif
00379
00380 #if !defined(POW)
00381     template <class T>
00382     inline T POW(const T &a, const unsigned &n) {
00383         T x = n ? a : 1;
00384         for (unsigned i=1; i<n; i++) x*=n;
00385         return x;
00386     }
00387 #endif
00388
00389 #if !defined(SGN)
00390     template <class T>
00391     inline int SGN(const T &a) {
00392         return ((T(0)<a)-(a<T(0)));
00393     }
00394 #endif
00395
00396 #if !defined(NUMCMP)
00397     template <class T>
00398     inline char NUMCMP(const T &a, const T &b) {
00399         if (a<b) return -1; else
00400         if (a>b) return +1;
00401         return 0;
00402     }
00403
00404     template <class Ta, class Tb>
00405     inline char NUMCMP(const Ta &a, const Tb &b) {
00406         if (a<b) return -1; else
00407         if (a>b) return +1;
00408         return 0;
00409     }
00410 #endif
00411
00412 #define ISREAL(x) WbUtil<x>::isReal()
00413 #define ISCOMPLX_(x) WbUtil<x>::isComplex()
00414
00415 #ifdef TST_MAC
00416     #include "Include/nomex.hh"
00417
00418     #include <Accelerate/Accelerate.h>
00419
00420 #elif defined(NOMEX)
00421     #include "nomex.hh"
00422 #else
00423
00424     #include <mex.h>
00425     #include <mat.h>
00426
00427     #define QS_VERSION 4.0
00428     #define QS_VERSION_SUB 0
00429     #define QS_VERSION_SUB_ ""
00430
00431     #ifdef MEX_MLVER
00432         #define MEX_MLVER_STR PP_STRFY(MEX_MLVER)
00433     #else

```

```

00434     #define MEX_MLVER_STR "(unknown)"
00435 #endif
00436
00437 #ifdef HOST_NAME
00438     #define HOST_NAME_STR PP_STRFY(HOST_NAME)
00439 #else
00440     #define HOST_NAME_STR "(host)"
00441 #endif
00442
00443 #define mxIsNumChar(a) (mxIsNumeric(a) || mxIsChar(a) || mxIsLogical(a))
00444
00445 #endif
00446
00447 #ifdef MAIN
00448     #undef printf
00449 #endif
00450
00451 #ifndef PROG
00452 #ifdef MATLAB_MEX_FILE
00453     #define PROG mexFunctionName()
00454 #else
00455     #define PROG "(?prog?)"
00456 #endif
00457 #endif
00458
00459     enum WBL_COLOR_SCHEME { WLC_OFF,
00460                             WLC_DARK,
00461                             NUM_WBL_COLOR_SCHEME };
00462
00463 namespace Wb {
00464     class myname__ {
00465     public:
00466         myname__() { unsigned i=0, k=0;
00467             #ifdef PROG
00468                 snprintf(data, 60, "%s", PROG);
00469             #elif defined MATLAB_MEX_FILE
00470                 snprintf(data, 60, "%s", mexFunctionName());
00471             #else
00472                 snprintf(data, 60, "(?myname?)");
00473             #endif
00474
00475             for (i=0; data[i]; ++i) { if (data[i]=='/') k=i+1; }
00476             if (k && data[k]) {
00477                 for (i=0; data[k]; ++k, ++i) { data[i]=data[k]; }
00478                 data[i]=0;
00479             }
00480
00481             #ifdef PROG_TAG
00482                 snprintf(data+60, 4, "%.3s", PROG_TAG);
00483             #else
00484                 snprintf(data+60, 4, "%.3s", data);
00485             #endif
00486         };
00487
00488         char data[64];
00489
00490     private:
00491     };
00492
00493     myname__ myName;
00494
00495 #define myname Wb::myName.data
00496 #define mytag Wb::myName.data+60
00497
00498     WBL_COLOR_SCHEME useCol = WLC_DARK;
00499
00500     int get_WB_VERBOSE(const char *F=0, int L=0);
00501     int got_DBSTOP(const char *F=0, int L=0);
00502     int got_DESKTOP();
00503     int is_DEPLOYED(const char *F=0, int L=0);
00504
00505     int envVRB= 0;
00506     int envDKT=-1;
00507
00508     int envDBG= 0;
00509
00510     int my_caller_tid=0;
00511 };
00512
00513
00514 void dbstop(const char* file, int line);
00515 void ExitMsg(const char* s="", char xflag=0);
00516 void doflush();
00517
00518 #ifdef LD_CLEBSCH_QS
00519 #ifndef QS_SKIP_MPFR
00520     #define QS_USING_MPFR

```



```

00521 #endif
00522 #endif
00523
00524     #define QS_ITAG_ "__"
00525
00526 #ifdef __WBDEBUG__
00527 #define __WB_MEM_CHECK__
00528 #undef WB_SKIP_ASSERT
00529 #elif defined __WB_MEM_CHECK__
00530 #define __WBDEBUG__
00531 #endif
00532
00533 namespace Wb {
00534 class ARGV {
00535 public:
00536     ARGV(va_list *vl=NULL) : args(vl) {};
00537     ~ARGV() { if (args) { va_end(*args); args=NULL; } };
00538
00539 private:
00540     va_list *args;
00541 };
00542 };
00543
00544 namespace wblog {
00545     const int SLEN=256;
00546 };
00547
00548 #ifdef MATLAB_MEX_FILE
00549 #define PRINTF(...) \
00550     { if (omp_get_level()==0) \
00551         { printf(__VA_ARGS__); } \
00552         else { fprintf(stdout,__VA_ARGS__); } \
00553     }
00554 #else
00555 #define PRINTF printf
00556 #endif
00557
00558 #include "wblibx.h"
00559
00560 #include "wblog.h"
00561
00562 #ifdef QS_USING_OMP
00563 #include "wbomp.hh"
00564 #endif
00565
00566 #include "memtrack.hh"
00567
00568 #include "util.hh"
00569 #include "memlib.hh"
00570 #include "mxlib.hh"
00571
00572 #include "wbcomplex.hh"
00573
00574 #ifdef QS_USING_MPFR
00575 #include <gmp.h>
00576 #include <mpfr.h>
00577 #include "wbmpfr.hh"
00578 #endif
00579
00580 #include "gathered.hh"
00581 #include "wbsort.hh"
00582
00583 #include "wbvector.hh"
00584 #include "wbstring.hh"
00585 #include "wbbitset.hh"
00586 #include "wbio.h"
00587
00588 #include "wbperm.hh"
00589 #include "wbindex.hh"
00590
00591 #include "wbclock.hh"
00592 #include "wbMatrix.hh"
00593
00594 #include "mexlib.hh"
00595
00596 #include "wbMatIO.hh"
00597
00598 #include "wbarrray.hh"
00599 #include "wbsparray.hh"
00600
00601 #include "wbutil.hh"
00602
00603 #include "wblas.hh"
00604 #include "wbMatrix_blas.hh"
00605 #include "wbarray_blas.hh"
00606
00607 #include "wbopts.hh"

```

```

00608
00609 #ifdef DBG_GCX_LOCKS
00610     Wb::tmpFile LKF(FL, "-lks");
00611 #endif
00612
00613 #ifdef DBG_QSX_BUF
00614     Wb::tmpFile BFF(FL, "-buf");
00615 #endif
00616
00617 #ifdef LD_CLEBSCH_QS
00618     #include "clebsch.hh"
00619     #include "QSpace_aux.hh"
00620     #include "QSpace.hh"
00621     #include "mpsortho.hh"
00622 #endif
00623
00624     #include "wblog.c"
00625
00626 #ifdef QS_USING_MPFR
00627     #include "wbmpfr.cc"
00628 #endif
00629
00630 #ifdef QS_USING_OMP
00631     #include "wbomp.cc"
00632 #endif
00633
00634     #include "wbsort.cc"
00635     #include "wbstring.cc"
00636     #include "wbbitset.cc"
00637     #include "memlib.cc"
00638     #include "wbcomplex.cc"
00639     #include "gathered.cc"
00640     #include "util.cc"
00641
00642 #ifdef LD_CLEBSCH_QS
00643     #include "QSpace_aux.cc"
00644     #include "QSpace.cc"
00645 #endif
00646
00647     #include "wbvector.cc"
00648     #include "wbperm.cc"
00649     #include "wbindex.cc"
00650     #include "wbMatrix.cc"
00651     #include "wbarray.cc"
00652     #include "wbspararray.cc"
00653
00654     #include "wbclock.cc"
00655     #include "mexlib.cc"
00656     #include "matlib.cc"
00657     #include "wbMatrix_blas.cc"
00658     #include "wbarray_blas.cc"
00659
00660     #include "wbio.c"
00661     #include "wbmsg.hh"
00662
00663 #ifdef LD_CLEBSCH_QS
00664     #include "clebsch.cc"
00665     #include "clebsch_aux.cc"
00666     #include "clebsch_io.cc"
00667     #include "clebsch_old.cc"
00668     #include "mpsortho.cc"
00669 #endif
00670
00671 namespace Wb {
00672
00673     class gpara__ {
00674     public:
00675         gpara__() {
00676             #ifdef __WBDEBUG__
00677                 wblog(FL, "TST %s() setting up global %s", myname, FCT);
00678             #endif
00679                 tlast=time(NULL);
00680
00681                 wb_srand();
00682
00683                 #ifdef __WB_MPFR_HH__
00684                     gmp_randinit_default(Wb::wb_rstate);
00685                 #endif
00686
00687                 envDKT=got_DESKTOP();
00688                 my_caller_tid=omp_get_thread_num();
00689
00690                 init();
00691             };
00692
00693             void init(char force=0) {
00694                 time_t tnow=time(NULL), dt=tnow-tlast;

```

```

00695
00696     #ifdef __WBDEBUG__
00697         wblog(FL,"TST gpara__::%s() %s ENV (dt=%ld)", FCT,
00698             dt>=1 ? "checking":"skipping", dt);
00699     #endif
00700
00701     if (force || dt>=1) { tlast=tnow; } else { return; }
00702
00703     memset(str,0,STRLEN+1);
00704     envVRB=get_WB_VERBOSE();
00705     envDBG=get_DBSTOP();
00706
00707     #ifdef QS_USING_OMP
00708         Wb::GetNumThreads(FL,OMP_NUM_THREADS,"OMP_NUM_THREADS");
00709         Wb::GetNumThreads(FL,QSP_NUM_THREADS,"QSP_NUM_THREADS");
00710
00711         sp_num_threads=MAX(OMP_NUM_THREADS,QSP_NUM_THREADS);
00712     #endif
00713
00714     my_caller_tid=omp_get_thread_num();
00715 };
00716
00717 private:
00718     time_t tlast;
00719 };
00720
00721 gpara__ gpara;
00722
00723 class Cleanup {
00724 public:
00725     Cleanup() {
00726         gpara.init();
00727     };
00728
00729     void Check() {
00730         wblog::check_ERR_pending();
00731     };
00732
00733     ~Cleanup() {
00734         wblog::myIO.clear();
00735         if (gwb_Profs.size()) { Wb::save_and_clear_Profiling(); }
00736     };
00737
00738 private:
00739 };
00740
00741 };
00742
00743 #ifdef MAIN
00744     #ifdef MATLAB_MEX_FILE
00745         #warning got MAIN with MATLAB_MEX_FILE
00746         #pragma message "\n\n \
00747             " Don't define MAIN with MEX files, this kills the whole matlab seesion!\n" \
00748             " See ExitMsg() in wblib.h (!)\n\n \
00749             ""
00750     #endif
00751
00752     void ExitMsg(const char* s, char xflag) {
00753         if (s && s[0])
00754             { printf("\n%s !? %s\n",shortFL(FL),s); }
00755         else { printf("\n"); }
00756
00757         if (Wb::envDBG) {
00758             dbstop(FL); if (xflag) { exit(-1); }
00759         }
00760         else {
00761             if (xflag) {
00762                 exit(-1);
00763             }
00764             else {
00765                 mexErrMsgIdAndTxt("Wb:MEX:wblog",s);
00766             }
00767         }
00768     }
00769 }
00770
00771 void doflush() { fflush(0); };
00772
00773 #else
00774
00775     #ifdef DBSTOP
00776         void ExitMsg(const char* s, char xflag) {
00777             if (s && s[0]) printf("%s >%s<\n",shortFL(FL),s ? s:"");
00778             dbstop(FL);
00779             printf("\n%s dbstop() and on we go ...\n", shortFL(FL));
00780             mexErrMsgIdAndTxt("Wb:ERR:mex",s);
00781         }

```

```

00782     #else
00783         void ExitMsg(const char* s, char xflag QS_UNUSED_VAR) {
00784             printf("\n");
00785             if (Wb::is_DEPLOYED()) {
00786                 Wb::print_backtrace(FL, "from deployed code"); printf("\n"); }
00787
00788             mexErrMsgIdAndTxt("Wb:ERR:mex", s);
00789             fprintf(stdout, "%s TST %s()\n", shortFL(FL), FCT);
00790         }
00791     #endif
00792
00793     void doflush() {
00794         fflush(0);
00795
00796         #if 0
00797
00798             if (!omp_in_parallel()) {
00799                 mexEvalString("pause(0);");
00800             }
00801         #endif
00802
00803     };
00804
00805 #endif
00806
00807 #endif
00808

```

5.41 wblibx.h

```

00001 #ifndef __WB_WBLIBX_HH__
00002 #define __WB_WBLIBX_HH__
00003
00004 /* ----- */
00005 /* ----- */
00006
00007     void usage(const char *file=NULL, int line=0, const char* estr=NULL);
00008     int isHelpIndicator(const char *s);
00009
00010 #ifdef MATLAB_MEX_FILE
00011     int isHelpIndicator(const mxArray *a0);
00012     int checkHelpVersion(const mxArray *a0, mxArray **argout=NULL);
00013 #else
00014     int checkHelpVersion(const char *s);
00015 #endif
00016
00017     void wbdie(const char *file, int line, const char* istr);
00018
00019     void dbstop(const char* file, int line);
00020
00021 namespace Wb {
00022     char* memsize2Str(double x, char *s, unsigned l=-1);
00023 }
00024
00025     void version_info_toMx(mxCArray *S);
00026
00027 #ifndef MEX_EXT
00028 #if __linux__
00029     #define MEX_EXT mexa64
00030 #elif __unix__
00031     #define MEX_EXT mexa64
00032 #elif __APPLE__
00033     #define MEX_EXT mexmac*
00034 #else
00035     #define MEX_EXT unknown
00036 #endif
00037 #endif
00038
00039 namespace Wb {
00040     class VersionInfo {
00041     public:
00042         VersionInfo() {
00043             fctn[0]=tag[0]=matlab[0]=mpfr[0]=compiler[0]=os[0] = 0;
00044             qspace[0]=git[0]=flags[0]=compiled[0] = 0;
00045             init();
00046         };
00047
00048         VersionInfo& init();
00049         wbstring toStr() const;
00050
00051         void print() const;
00052         mxArray* toMx() const;
00053

```

```

00054     char fctn[32], tag[8], matlab[16], mpfr[16], compiler[16], os[24];
00055     char qspace[16], git[16], flags[128], compiled[64];
00056
00057     private:
00058
00059     inline unsigned str_cpy(char *s, unsigned n, const char *a);
00060 };
00061 }
00062
00063 template <class T>
00064 class is_pointer_ { public: char q() { return 0; }; };
00065
00066 template <class T>
00067 class is_pointer_<T*> { public: char q() { return 1; }; };
00068
00069 template <class T>
00070 class is_pointer_<T**> { public: char q() { return 2; }; };
00071
00072 template <class T>
00073 class is_pointer_<T***> { public: char q() { return 3; }; };
00074
00075 template <class T>
00076 inline char ispointer(const T& x QS_UNUSED_VAR) { return is_pointer_<T>().q(); };
00077
00078 #endif
00079

```

5.42 wblog.c

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : wblog (logging routines)
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_LOG_C__
00021 #define __WB_LOG_C__
00022
00023 /* ----- */
00024 /* ----- */
00025
00026 #ifndef LIBQS_SETUP
00027
00028 void usage(const char *F, int L, const char* estr) {
00029
00030     int l=0;
00031     while (isspace(USAGE[l])) { ++l; }
00032
00033     if (USAGE[l]) { printf("\n%s\n",l?"": " ",USAGE); }
00034     else {
00035         const char *f=mexFunctionName();
00036         if (f && f[0]) {
00037             char cmd[256]; l=snprintf(cmd,256,
00038                                     "if exist('%s.m','file')==2, fprintf(1,'\n'); help %s;\n"
00039                                     "else fprintf(1,'"
00040                                     "  ERR failed to find help %s.m file for mex function %s()\n');"
00041                                     "end", f,f,f,f);
00042
00043             if (l<256) {
00044                 if (F && F[0]) { mexPrintf("\n  usage %s\n",shortFL(F,L)); }
00045                 mexEvalString(cmd);
00046             }
00047             else wblog(FL,"WRN %s() got f=%s (l=%d/256) !?",FCT,f,l);
00048         }
00049     }
00050
00051     if (estr && estr[0]) { wbdie(F,L,estr); }
00052 };
00053
00054 #endif

```

```

00055
00056 const char* shortFL(const char *F, int L,
00057     unsigned n,
00058     const char *P, char sep
00059 ) {
00060
00061     static char fstr[512]="", *fs=fstr;
00062     unsigned i,l, j=0, nx=(P? 40 : 32);
00063     char *sout=fstr, sx[nx];
00064
00065     if (int(n)<0) { n=WBL_HLEN; }
00066     else if (n<16 || n>=nx) {
00067         if (!n && !F && !L)
00068             { memset(fstr,0,512); return sout; }
00069         else { wblog(FL,"ERR %s() got n=%d/%d\n",FCT,n+1,nx); }
00070     }
00071
00072 #pragma omp critical (using_shortFL)
00073 { if (fs+n-fstr<512)
00074     { sout=fs; fs+=(n+1); }
00075     else { sout=fs=fstr; }
00076 }
00077
00078 if (!F || !F[0]) { sout[0]=0; return sout; }
00079 if (F[0]=='/' || F[0]=='.' ) { l=strlen(F);
00080     for (i=l-1; i<l; --i) { if (F[i]=='.' ) { break; }}
00081     for ( ; i<l; --i) { if (F[i]=='/') { j=i+1; break; }}
00082 }
00083
00084 if (P && !P[0]) {
00085     P=mytag; if (!P || !P[0]) { P=myname; }
00086 #ifdef PROG
00087     if (P && P[0]) { l=strlen(P);
00088         if (!strncmp(F+j,P, l>3? 3:l)) { P=NULL; }
00089     }
00090 #endif
00091 }
00092
00093 if (!P || !P[0]) {
00094     if (L>0)
00095         { l=snprintf(sx,nx,"% .24s:%d",F+j,L); }
00096     else { l=snprintf(sx,nx,"% .24s:", F+j ); }
00097 }
00098 else {
00099     if (L>0)
00100         { l=snprintf(sx,nx,"% .3s%c%.20s:%d",P,sep,F+j,L); }
00101     else { l=snprintf(sx,nx,"% .3s%c%.20s:", P,sep,F+j ); }
00102 }
00103
00104 if (l>=nx) wblog(FL,
00105     "WRN %s() string out of bounds (%d/%d)\n%s",FCT,l,nx,sx);
00106
00107 if (l<=n) { strcpy(sout,sx); }
00108 else {
00109     for (i=l-1; i<l; --i) { if (sx[i]=='.' ) { break; }}
00110     for (j=i; j<l; --j) { if (sx[j]=='_' || sx[j]=='-') { break; }}
00111     if (j<l && (l-j)<n-3) {
00112         i=n+j-1;
00113         strncpy(sout,sx,i);
00114         sout[i]='\n';
00115         strcpy(sout+i+1,sx+j+1);
00116     }
00117     else if (i<l && (l-i)<n-8) {
00118         j=n+i-1;
00119         strncpy(sout,sx,j);
00120         sout[j]='\n';
00121         strcpy(sout+j+1,sx+i+1);
00122     }
00123     else { i=5;
00124         strncpy(sout,sx,i); sout[i]='\n';
00125         strcpy(sout+i+1,sx+l-n+i+1);
00126     }
00127 }
00128 return sout;
00129 };
00130
00131 const char* tmpmatFL(
00132     const char *F, int L, const char *vn, char *s, unsigned n) {
00133
00134     if (!s || int(n)<8) wblog(FL,"ERR %s() invalid input (%p, %d)",FCT,s,n);
00135     unsigned l=snprintf(s,n,"./tmp");
00136
00137     if (l<n && F && F[0]) { unsigned i=0, k=0;
00138         for (; F[i]; ++i) { if (F[i]=='/') { k=i+1; } }
00139         i=l; l+=snprintf(s+l,n-l,"%s",F+k);
00140         for (; s[i] && i<n; ++i) { if (s[i]=='.' ) { s[i]=0; l=i; break; }}
00141     }

```

```

00142
00143     if (l<n) { l+=snprintf(s+l,n-l,"%d",L); }
00144 #ifdef QS_USING_OMP
00145     if (l<n) {
00146         int j=omp_get_thread_num();
00147         if (j!=Wb::my_caller_tid) { l+=snprintf(s+l,n-l,"%d",j); }
00148     }
00149 #endif
00150
00151     if (l<n && vn && vn[0]) { l+=snprintf(s+l,n-l,"%s",vn); }
00152     if (l<n) { l+=snprintf(s+l,n-l,".mat"); }
00153
00154     if (l>=n) wblog(F_L,"WRN %s() "
00155         "string out of bounds (%d/%d)\n%s",FCT,l,n,s?s:"(null)");
00156     return s;
00157 };
00158
00159 unsigned Wb::VersionInfo::str_cpy(char *s, unsigned n, const char *a) {
00160     unsigned i=0; s[0]=0;
00161     if (a) {
00162         for (; i<n; ++i) { if (!(s[i]=a[i])) break; }
00163         if (i>=n) { wblog(F_L,
00164             "WRN %s() out of bounds (%d/%d)\n%s",FCT,strlen(a),n,a);
00165         }
00166     }
00167     return i;
00168 };
00169
00170 Wb::VersionInfo& Wb::VersionInfo::init() {
00171
00172     unsigned i,l,m,n; char *s;
00173
00174     str_cpy(fctn,sizeof(fctn),myname);
00175 #ifdef PROG_TAG
00176     str_cpy(tag, sizeof(tag), PROG_TAG);
00177 #endif
00178
00179     n=str_cpy(matlab,sizeof(matlab),MEX_MLVER_STR);
00180
00181     s=matlab;
00182     for (l=i=0; i<n; ++i) { if (s[i]=='/') { l=i+1; }}
00183     if (l) {
00184         for (i=l; i<=n; ++i) { if (!(s[i-l]=s[l])) break; }
00185     }
00186
00187     l=sizeof(os);
00188     #if __linux__
00189         snprintf(os,l,"linux / %s", PP_STRFY(MEX_EXT));
00190     #elif __unix__
00191         snprintf(os,l,"unix / %s", PP_STRFY(MEX_EXT));
00192     #elif __APPLE__
00193         snprintf(os,l,"macOS / %s", PP_STRFY(MEX_EXT));
00194     #else
00195         snprintf(os,l,"unknown / %s",PP_STRFY(MEX_EXT));
00196     #endif
00197
00198 #ifdef QS_USING_MPFR
00199     str_cpy(mpfr,sizeof(mpfr),MPFR_VERSION_STRING);
00200 #endif
00201
00202     n=sizeof(compiler); l=6;
00203
00204     #if defined(__clang__)
00205         snprintf(compiler,n,"clang/%d.%d.%d",
00206             __clang_major__, __clang_minor__, __clang_patchlevel__);
00207     #elif defined(__GNUC__)
00208         snprintf(compiler,n,"gcc/%d.%d.%d",
00209             __GNUC__, __GNUC_MINOR__, __GNUC_PATCHLEVEL__);
00210     #endif
00211
00212 #ifdef LD_CLEBSCH_QS
00213     n=sizeof(qspace);
00214     l=snprintf(qspace,n,"%1f",double(QS_VERSION));
00215
00216     i=strlen(QS_VERSION_SUB_);
00217     if (QS_VERSION_SUB != 0) {
00218         { l+=snprintf(qspace+l,n-l,"%g",double(QS_VERSION_SUB)); }
00219         if (i) { l+=snprintf(qspace+l,n-l,"%s",QS_VERSION_SUB_); }
00220     }
00221 #ifdef QS_VERSION_XSTR
00222     if (l<n) {
00223         l+=snprintf(qspace+l,n-l," / %s",PP_STRFY(QS_VERSION_XSTR));
00224     }
00225 #endif
00226 #endif
00227 #ifdef __QS_GIT_BRANCH__
00228     str_cpy(git,sizeof(git),PP_STRFY(__QS_GIT_BRANCH__));

```

```

00229 #endif
00230
00231     n=sizeof(flags); l=0;
00232
00233 #ifdef MAIN
00234     l+=snprintf(flags+l,n-l,"%s","MAIN");
00235 #endif
00236 #ifdef MATLAB_MEX_FILE
00237     l+=snprintf(flags+l,n-l,"%s",l? " ", ":", "MEX_FILE");
00238 #endif
00239
00240     i=1; m=0;
00241     l+=snprintf(flags+l,n-l," using");
00242
00243 #ifdef LD_CLEBSCH_QS
00244     if (l<n) { l+=snprintf(flags+l,n-l," QS" ); ++m; }
00245 #endif
00246 #ifdef QS_USING_OMP
00247     if (l<n) { l+=snprintf(flags+l,n-l," OMP" ); ++m; }
00248 #endif
00249 #ifdef QS_USING_MPFR
00250     if (l<n) { l+=snprintf(flags+l,n-l," MPFR"); ++m; }
00251 #endif
00252 #ifdef QS_USING_HPTT
00253     if (l<n) { l+=snprintf(flags+l,n-l," HPTT"); ++m; }
00254 #endif
00255
00256     if (m)
00257     { i=1; m=0; if (l+3<n) { strcpy(flags+l," "); l+=2; } }
00258     else { l=i; m=1; }
00259
00260 #ifdef WB_SKIP_ASSERT
00261     if (l<n) {
00262         l+=snprintf(flags+l,n-l,"%s",m? " ", ":", "SKIP_ASSERT");
00263     }; ++m;
00264 #endif
00265 #ifdef MEX_CXX_FLAGS
00266     if (l<n) {
00267         l+=snprintf(flags+l,n-l,"%s",m? " ", ":", PP_STRFY(MEX_CXX_FLAGS));
00268     }; ++m;
00269 #endif
00270 #ifdef __WBDEBUG__
00271     if (l<n) {
00272         l+=snprintf(flags+l,n-l,"%s",m? " ", ":", "DEBUG");
00273     }; ++m;
00274 #endif
00275 #ifdef DBSTOP
00276     if (l<n) {
00277         l+=snprintf(flags+l,n-l,"%s",m? " ", ":", "DBSTOP");
00278     }; ++m;
00279 #endif
00280 #ifdef DBG_GCX_LOCKS
00281     if (l<n) { l+=snprintf(flags+l,n-l,
00282         "%s=%g",m? " ", ":", "DBG_GCX_LOCKS", double(DBG_GCX_LOCKS));
00283     }; ++m;
00284 #endif
00285 #ifdef DBG_QSX_BUF
00286     if (l<n) { l+=snprintf(flags+l,n-l,
00287         "%s=%g",m? " ", ":", "DBG_QSX_BUF", double(DBG_QSX_BUF));
00288     }; ++m;
00289 #endif
00290 #ifdef __WB_MEM_CHECK__
00291     if (l<n) {
00292         l+=snprintf(flags+l,n-l,"%s",m? " ", ":", "MEM_CHECK");
00293     }; ++m;
00294 #endif
00295
00296     if (m)
00297     { i=1; m=0; }
00298     else { l=i; }
00299
00300     l=snprintf(compiled,sizeof(compiled),"s on s",
00301         #ifdef __DATE_COMP__
00302             PP_STRFY(__DATE_COMP__),
00303         #else
00304             __DATE__,
00305         #endif
00306         HOST_NAME_STR
00307     );
00308
00309     return *this;
00310 };
00311
00312 void Wb::VersionInfo::print() const { PRINTF("\n%s\n",toStr().data); };
00313
00314 wbsting Wb::VersionInfo::toStr() const {
00315

```



```

00316     wbsting sout; unsigned i=0,l,n=512;
00317     sout.init(n);
00318
00319     const unsigned m=8; char q[2], *sx=sout.data;
00320
00321     const char *field[m] = {
00322         "Matlab","Compiler","System","MPFR","QSpace","Git-branch","Flags","Compiled"
00323     };
00324     const char *data[m] = {
00325         matlab, compiler, os, mpfr, qspace, git, flags, compiled
00326     };
00327
00328     l=snprintf(sx,n,"Version info for function '%s'",fctn);
00329     if (*tag)
00330         { l+=snprintf(sx+l,n-l," (%s)\n\n",tag); }
00331     else { l+=snprintf(sx+l,n-l,"\n\n"); }
00332
00333     q[0]=q[1]=0;
00334
00335     for (; i<m; ++i) { if (data[i] && l<n) {
00336         if (i==0) { *q='R'; } else
00337         if (i==3 || i==4)
00338             { if (*data[i]) { *q='v'; } else continue; }
00339         else { *q=0; }
00340
00341         l+=snprintf(sx+l,n-l," %-13s %s\n",field[i],q,data[i]);
00342     }}
00343
00344     if (l<n)
00345         { if (l) { sout.len=l; }}
00346     else { wblog(FL,"WRN %s() string out of bounds (%d/%d)",FCT,l,n); }
00347
00348     return sout;
00349 };
00350
00351 mxArray* Wb::VersionInfo::toMx() const {
00352
00353     unsigned i=0; const unsigned n=10;
00354
00355     const char *fields[n] = {
00356         "function","tag","matlab","system","MPFR","compiler","QSpace",
00357         "git_branch", "flags", "compiled"
00358     };
00359     mxArray *S=mxCreateStructMatrix(1,1,n,fields);
00360
00361     const char *data[n] = {
00362         fctn, tag, matlab, os, mpfr, compiler, qspace, git,
00363         flags, compiled
00364     };
00365
00366     for (; i<n; ++i) {
00367         mxSetFieldByNumber(S,0,i,wbsting(data[i]).toMx());
00368     }
00369
00370     return S;
00371 };
00372
00373 void banner(unsigned n, const char *s, const char *istr, const char *fstr){
00374
00375     unsigned i,r=0,l=0,m=0, s1=0, s2=0;
00376
00377     if (s && s[0]) r=strlen(s);
00378     else wblog(FL,"ERR cannot repeat empty string!");
00379
00380     if (istr && istr[0]) { s1=strlen(istr); printf("%s",istr); }
00381     if (fstr && fstr[0]) { s2=strlen(fstr); }
00382
00383     m=s1+s2; if (n>=m) l=(n-m)/r;
00384     else wblog(FL,"ERR overall length too short (%d/%d)",n,m);
00385
00386     for (i=0; i<l; i++) printf("%s",s);
00387     for (i=l*r+s1, l=n-s2; i<l; i++) printf("%s",s);
00388
00389     if (fstr && fstr[0]) { printf("%s",fstr); }
00390     printf("\n");
00391 };
00392
00393 void wbdie(const char *F, int L, const char* istr) {
00394
00395     PRINTF("\n%s ERR\n\n%s\n", shortFL(F,L), istr);
00396
00397     #ifdef QS_USING_OMP
00398         throw Wb::LogException(ERR);
00399     #else
00400         ExitMsg();
00401     #endif
00402 };

```

```

00403
00404 void wbdie(
00405     const char *F, int L,
00406     const char *istr1,
00407     const char *istr2
00408 ){
00409     wblog(F,L,"ERR %s%N%N%s%N", istr1, istr2);
00410     #ifdef QS_USING_OMP
00411         throw Wb::LogException(ERR);
00412     #else
00413         ExitMsg();
00414     #endif
00415 };
00416
00417 Wb::LogException::LogException(WBLOG_TYPE l, const char *s)
00418 : type(l), nrefs(NULL), count(0), ith(-1), nth(-1) {
00419
00420     if (!s || !s[0]) { istr[0]=0; } else {
00421         if (snprintf(istr,64,"%s",s)>=64) {
00422             istr[61]=istr[62]='.'; istr[63]=0;
00423         }
00424     }
00425
00426     if (l) { nrefs = new int; (*nrefs)=1; count=1;
00427         ith=omp_get_thread_num();
00428         nth=omp_get_num_threads();
00429         #ifdef DBSTOP
00430             fprintf(stdout,"%s %s dbstop() having Wb::LogException "
00431                 "nrefs=%d, istr='%s'; tid=%d/%d\n",
00432                 shortFLT,*nrefs,istr,ith,nth); fflush(0);
00433             dbstop(FL);
00434         #endif
00435     }
00436 };
00437
00438 void Wb::LogException::report(const char *F, int L, const char *func) {
00439     if (type || count) {
00440         unsigned l, n=128; char s[n];
00441
00442         l=snprintf(s,n,
00443             "%s ERR %s() at nerr=%d",shortFL(F_L),func?func:FCT,count);
00444         if (l<n) {
00445             if (ith<=1 && nth<=1)
00446                 l+=snprintf(s+l,n-l," (serial @ %d/%d)",ith,nth);
00447             else l+=snprintf(s+l,n-l," (parallel section thread %d/%d)",ith,nth);
00448         }
00449
00450         if (type) { PRINTF("\n%s\n",s); throw(*this); } else
00451         if (count) { ExitMsg(s); }
00452     }
00453     init();
00454 };
00455
00456 wstring Wb::LogException::toStr() const {
00457     wstring sout(128);
00458
00459     unsigned l, n=sout.len; char *s=sout.data;
00460
00461     l=snprintf(s,n,"%s: ",istr[0] ? istr : "");
00462     if (l<n) {
00463         if (type<NUM_TYPE)
00464             { l+=snprintf(s+l,n-l,"type=%s",WBLOG_TYPE_STR[type]); }
00465         else { l+=snprintf(s+l,n-l,"type=%d !?",unsigned(type)); }
00466     }
00467     if (l<n) {
00468         l+=snprintf(s+l,n-l," %s (i=%d/%d; nerr=%d)",
00469             ith<=1 && nth<=1 ? "serial":"parallel section", ith,nth,count);
00470     }
00471     if (l<n) {
00472         if (nrefs)
00473             { l+=snprintf(s+l,n-l," *nrefs=%d",*nrefs); }
00474         else { l+=snprintf(s+l,n-l," *nrefs=null"); }
00475     }
00476     return sout;
00477 };
00478
00479 void init_header(
00480     char *hstr, unsigned hlen,
00481     const char *F, int L,
00482     const char *time_stamp, const char *tag, int xcol,
00483     unsigned lenFL=21
00484 ){
00485     unsigned i,k=0,n;
00486
00487     if (xcol && hlen>12) {
00488         i=Wb::termcolor::ID("hdr");
00489         if (int(i)>0) {

```

```

00490         if (i<8)
00491             { i=snprintf(hstr,hlen,"\\e[3%dm",i); }
00492         else { i=snprintf(hstr,hlen,"\\e[38;5;%dm",i); }
00493     }
00494     else {
00495         i=snprintf(hstr,hlen,"\\e[0m");
00496     }
00497     hstr+=i; hlen-=i;
00498 }
00499
00500 if (F) {
00501     if (F[0]=='/' || (F[0]=='.' && F[1]=='/')) {
00502         for (i=0; F[i]; ++i) { if (F[i]=='/') k=i+1; }
00503     }
00504
00505     if (L==WBL_GOT_SHORTFL)
00506         { i=snprintf(hstr,hlen,"%.100s", F+k ); }
00507     else { i=snprintf(hstr,hlen,"%.100s:%d",F+k,L); }
00508
00509     if (i>=hlen) wblog(FL,"ERR %s() "
00510         "string out of bounds (%d/%d)%N%N'%s'",FCT,i,hlen,hstr);
00511
00512     n=strlen(hstr); if (n>lenFL) {
00513         i=lenFL/2; hstr[i]=hstr[i+1]='.'; i+=2;
00514         for (k=n-lenFL/2+2; k<n; ++k, ++i) hstr[i]=hstr[k];
00515         hstr[i]=0;
00516     }
00517 }
00518 else { i=snprintf(hstr,hlen,"(null):%d", L); }
00519
00520 i=strlen(hstr); for (; i<lenFL; ++i) { hstr[i]=' '; }
00521
00522 n=strlen(time_stamp+17); if (n>32) { n=32; }
00523 if (i+n+12<hlen) {
00524     memcpy(hstr+i,time_stamp+17,n); i+=n;
00525     hstr[i]=' '; hstr[++i]=0;
00526     hstr+=i; hlen-=i; i=0;
00527 }
00528 else wblog(FL,"ERR %s() "
00529     "string out of bounds (%d/%d)%N%N'%s'",FCT,i,hlen,hstr);
00530
00531 if (xcol) {
00532     if (xcol>0) {
00533         if (xcol<8)
00534             { i=snprintf(hstr,hlen,"\\e[3%dm",xcol); }
00535         else { i=snprintf(hstr,hlen,"\\e[38;5;%dm",xcol); }
00536     }
00537     else if (xcol<0) { i=snprintf(hstr,hlen,"\\e[0m"); }
00538 }
00539
00540 if (i<hlen) {
00541     if (tag && tag[0])
00542         { i+=snprintf(hstr+i,hlen-i," %s ",tag); }
00543     else { hstr[i]=' '; hstr[++i]=0; }
00544 }
00545 if (i>=hlen) wblog(FL,"ERR %s() "
00546     "string out of bounds (%d/%d)%N%N'%s'",FCT,i,hlen,hstr);
00547 };
00548
00549 int check_update_header(
00550     char *hstr, const char* fmt, const char *istr=NULL, int lenFL=21
00551 ){
00552     int n,k=0,i=0,q=0;
00553
00554     for (; fmt[i]; ++i) {
00555         if (isalpha(fmt[i])) q|=1; else if (fmt[i]=='.') q|=2; else
00556         if (isdigit(fmt[i])) q|=8; else if (fmt[i]=='_') q|=16; else
00557         if (fmt[i]=='/') { q|=4; k=i+1; } else break;
00558     }
00559
00560     if ((q&3)!=3 || fmt[i]!=':') return 0;
00561     n=i++;
00562
00563     for (; fmt[i]; ++i) if (!isdigit(fmt[i])) break;
00564     if (i==n+1 || isalpha(fmt[i])) return 0;
00565
00566     if (istr && istr[0]) {
00567         strcpy(hstr,istr);
00568         n=strlen(hstr); hstr+=n; lenFL-=n;
00569     }
00570
00571     if (i-k<lenFL) {
00572         memcpy(hstr,fmt+k,i-k);
00573         for (k=i-k; k<lenFL; k++) hstr[k]=' ';
00574     }
00575     else {
00576         memcpy(hstr,fmt+k,lenFL/2); k=lenFL/2;

```

```

00577     hstr[k]=hstr[k+1]='.';
00578     memcpy(hstr+k+2, fmt+i-k+2,k-2); k=2*k;
00579 }
00580
00581 if (k<=lenFL) {
00582     for (; hstr[k]; k++) if (hstr[k]!=' ') break;
00583     if (k>lenFL) {
00584         for (n=k-lenFL; hstr[k]; k++) hstr[k-n]=hstr[k];
00585         hstr[k-n]=0;
00586     }
00587 }
00588
00589 for (; fmt[i]; ++i) {
00590     if (!isspace(fmt[i]) && fmt[i]!=':' && fmt[i]!='(' && fmt[i]!=')')
00591         break;
00592 }
00593
00594 return i;
00595 };
00596
00597 void wbSetLogLevel(unsigned l) {
00598     wblogf(stdout,0,0,"log_level",l);
00599 };
00600
00601 int wblogf(FILE *fid,
00602     const char* file, int line, const char *fmt, va_list args
00603 ) {
00604     #ifdef QS_USING_OMP
00605         Wb::ompGuard myLK(wblog_lock,l);
00606     #endif
00607
00608     Wb::SBUF S; int l; Wb::LogException e;
00609
00610     WBL_COLOR_SCHEME xcol=WLC_OFF;
00611     if (Wb::useCol && (fid==stdout || fid==stderr)) { xcol=WLC_DARK; }
00612
00613     try { l=wblogs(S,xcol,file,line,fmt,args); }
00614     catch (Wb::LogException &e_) { e=e_; l=-11; }
00615     catch (...) { l=-12; }
00616
00617     { S.flush(fid, l>1); }
00618
00619     if (e.type) { throw(e); } else
00620     if (l<-10) {
00621         sprintf_str("ERR wblog.h:%d encountered l=%d !?",__LINE__,l);
00622         ExitMsg(str);
00623     }
00624
00625     return l;
00626 };
00627
00628 int wblogl(const char* file, int line, const char *fmt, ...) {
00629
00630     va_list args; Wb::ARGV wd(&args);
00631     va_start(args,fmt);
00632
00633     WBL_COLOR_SCHEME xcol=WLC_OFF;
00634     if (Wb::useCol) { xcol=WLC_DARK; }
00635
00636     Wb::SBUF S; int l; Wb::LogException e;
00637     try { l=wblogs(S,xcol,file,line,fmt,args,l); }
00638     catch (Wb::LogException &e_) { e=e_; l=-11; }
00639     catch (...) { l=-12; }
00640
00641     S.flush(l>3 ? stderr : stdout,l>1);
00642
00643     if (e.type) { throw(e); } else
00644     if (l<-10) {
00645         sprintf_str("ERR wblog.h:%d encountered l=%d !?",__LINE__,l);
00646         ExitMsg(str);
00647     }
00648
00649     return l;
00650 };
00651
00652 int wblogs(
00653     Wb::SBUF &Sb, WBL_COLOR_SCHEME xcol_,
00654     const char* file, int line,
00655     const char *fmt, va_list args,
00656     char Hflag
00657 ){
00658     static int log_level=1;
00659     static int then=0;
00660
00661     int rval=0, xcol=0;
00662
00663     unsigned i,j,k,l,m, nesc=0, hlen=128, flen=128;

```

```

00664     const unsigned nt=32;
00665
00666     char isfmt=0, hflag=1, bflag=0, iflag=0, eflag=0, wflag=0, fwd=0;
00667
00668     char fstr[flen];
00669     char time_stamp[nt], tag[8];
00670
00671     char c, *cp, cb, log_header[hlen];
00672
00673     if (file==NULL && line<=0) {
00674         if (!strcmp(fmt,"log_level")) {
00675             log_level=(unsigned)va_arg(args,int);
00676             return 0;
00677         }
00678     }
00679
00680     if (log_level<0) { return 0; }
00681
00682     if (!fmt) wblog(FL,"ERR wblog got null fmt");
00683
00684     if (fmt[0]=='\\') { hflag=0; fmt+=1; bflag=1; }
00685
00686     for (; *fmt; ++fmt) { if (*fmt=='\n') Sb.cat("\n"); else break; }
00687
00688     tag[0]=0;
00689
00690     eflag=wblog_check_tag(fmt,"ERR",tag); if (!eflag) {
00691     wflag=wblog_check_tag(fmt,"WRN",tag); if (!wflag) {
00692         if (fmt[0]=='\b') { fmt+=1; }
00693         else {
00694             unsigned i=0, k,l;
00695             for (; fmt[i]; ++i) { if (fmt[i]!='\n') break; }
00696             for (k=i, l=k+3; i<l && fmt[i]; ++i) {
00697                 if (fmt[i]=='%' || !isprint(fmt[i])) break; }
00698
00699             if (i==l && fmt[i]==' ') {
00700                 memcpy(tag,fmt+k,3); tag[3]=0; iflag=i+1;
00701             }
00702
00703             if (xcol_ && iflag) {
00704                 xcol=Wb::termcolor::ID(tag);
00705             }
00706         }
00707     }
00708     else { iflag=wflag; }
00709
00710     if ((log_level & 15)==0) {
00711         if (!eflag && !iflag) return 0; }
00712
00713     if (xcol_ && !xcol) {
00714         xcol=Wb::termcolor::ID(tag);
00715     }
00716
00717     if (eflag || iflag) fmt+=(eflag+iflag);
00718     if (eflag) Sb.cat("\n");
00719
00720     #pragma omp critical (got_CPTR_TIME)
00721     { time_t curr_time=time(NULL);
00722       struct tm *tblock=localtime(&curr_time);
00723
00724       l=strftime(time_stamp,nt,"%a, %d %b %Y %T",tblock);
00725
00726       if (!l) {
00727           snprintf(time_stamp,nt,"Sat 22 Jul 1972 12:00:00%s",
00728             WBLOG_MSEC ? ".000" : "");
00729       }
00730       else if (WBLOG_MSEC && l+4<nt) {
00731           using namespace std::chrono;
00732           time_point<system_clock> t=system_clock::now();
00733
00734           snprintf(time_stamp+l,nt-l,"%03ld",
00735             size_t(duration_cast<milliseconds>(t.time_since_epoch()).count())%1000);
00736       }
00737
00738       if (then && then!=tblock->tm_yday) {
00739           char dstr[nt];
00740           l=strftime(dstr,nt,"%d-%b-%Y %T", tblock);
00741           Sb.catf("\n> TODAY %s (%s)\n\n",l? dstr:"??",myname);
00742       }
00743       then=tblock->tm_yday;
00744     }
00745
00746     if (!fmt[0] && !eflag && !iflag) {
00747         return 0;
00748     }
00749 }
00750

```

```

00751
00752 init_header(log_header,hlen,file,line,time_stamp,tag,
00753             xcol ? xcol : (xcol_ ? -1 : 0));
00754
00755 for (;;) {
00756     if (fmt[0]=='%' && fmt[1]!='N') { Sb.cat("\n"); fmt+=2; }
00757     else break;
00758 }
00759
00760 if (hflag) { Sb.cat(log_header); hflag=0; }
00761
00762 i=isfmt=0;
00763
00764 while ((c=(*fmt++)) {
00765     if (c=='\f') { fwd|=1; continue; }
00766     if (c=='\n') { fwd ^= 2; } else
00767     if (c=='\"') { fwd ^= 4; }
00768
00769     fstr[i++]=c;
00770
00771     if (i>124) { fstr[i-1]=0;
00772         snprintf(str,512,
00773             "ERR fstr out of bounds %d/128 (%s'%s)",i,fstr,fmt-1);
00774         ExitMsg(str);
00775     }
00776
00777     if (c=='%') {
00778         if (fmt[0]=='%' && !isfmt) { fstr[i]=fmt[0]; ++fmt; ++i; } else
00779         if (i<2 || fstr[i-2]!='\\') { ++isfmt; }
00780         continue;
00781     }
00782
00783     if (c=='\n') {
00784         fstr[i-1]=0; Sb.catf("%s\n",fstr);
00785         if (fmt[0]=='\n') {
00786             for (j=1; fmt[j]; ++j) { if (fmt[j]!='\n') break; }
00787             if (!fmt[j]) { Sb.cat(fmt); break; }
00788         }
00789         else {
00790             fmt+=check_update_header(log_header,fmt);
00791             Sb.cat(log_header);
00792             i=isfmt=0;
00793         }
00794     }
00795     else if (c=='\e' || (c=='\\' && *fmt=='e')) { ++nesc; }
00796
00797     if (!isfmt || isdigit(c) || strchr("%+.lL",c)) { continue; }
00798
00799     if (c=='s') {
00800         fstr[i]=0;
00801         if (i>2 && fstr[i-2]=='%') {
00802             fstr[i-2]=0; Sb.cat(fstr);
00803             strcpy(fstr,"%s"); i=2;
00804         }
00805
00806         char *s0=va_arg(args, char*);
00807
00808         if (!s0) { Sb.catf(fstr,"(null)"); } else {
00809             for (k=0; s0[k]>=30; ++k);
00810
00811             if ((!s0[k] && k<4) || i>2) { Sb.catf(fstr,s0); }
00812             else {
00813                 char ck=s0[k]; if (ck) { k+=strlen(s0+k); }
00814                 char sl_[k+1]; char *sl=sl_;
00815                 strcpy(sl,s0);
00816                 for (k=0; sl[k] || !k;) {
00817                     while (sl[k]) {
00818                         if (sl[k]=='\n') {
00819                             Sb.catf("\n%s",log_header);
00820                             sl+=(k+1); k=0;
00821                         }
00822                         else if (isspace(sl[k])) { ++k; }
00823                         else break;
00824                     }
00825                 }
00826
00827                 k=(fwd ? 0 : check_update_header(log_header,sl,"> "));
00828                 if (k) { sl+=k; k=0;
00829                     if (!ck && *fmt==' ') { ++fmt; }
00830
00831                     Sb.catf("\r%s",log_header);
00832                 }
00833
00834                 for (k=0; sl[k]; ++k) {
00835                     if (sl[k]=='\f') { sl[k]='\n'; } else
00836                     if (sl[k]=='\n') break;
00837                 }

```

```

00838
00839         if (s1[k]) {
00840             for (j=k+1; s1[j]; ++j) { if (s1[j]!='\n') break; }
00841             if (!s1[j]) { Sb.catf("%s",s1); break; }
00842
00843             s1[j-1]=0;
00844             Sb.catf("%s\n",s1);
00845             Sb.cat(log_header);
00846             s1+=j; k=0;
00847         }
00848         else {
00849             j=strlen(s1); if (j) {
00850                 if (s1[j-1]=='\\') { bflag|=2;
00851                     s1[j-1]=0;
00852                 }
00853                 Sb.catf(fstr,s1); break;
00854             }
00855         }
00856     }}
00857     i=isfmt=0;
00858 }
00859 else if (strchr("dXp",c)) {
00860     fstr[i]=0;
00861     m=0; if (i>1) if (fstr[i-2]=='l') m++;
00862     if (m) Sb.catf(fstr, va_arg(args, long));
00863     else Sb.catf(fstr, va_arg(args, int ));
00864
00865     i=isfmt=0;
00866 }
00867 else if (c=='D') {
00868     char s[nt]; int j,l,q;
00869
00870     if (i>1 && fstr[i-2]=='l') {
00871         fstr[i-2]='s'; fstr[i-1]=0;
00872         l=snprintf(s,nt,"%ld",va_arg(args,long));
00873     }
00874     else {
00875         fstr[i-1]='s'; fstr[i]=0;
00876         l=snprintf(s,nt,"%d", va_arg(args,int ));
00877     }
00878
00879     j=l+(1-(s[0]!='-' ? 1 : 2))/3;
00880     if (j>1 && j<int(nt)) {
00881         s[j]=0; q=(l-1)%3;
00882         for (--l, --j; l>0 && j>1; --j) {
00883             s[j]=s[l--]; if (l%3==q) { s[--j]=','; }
00884         }
00885     }
00886     Sb.catf(fstr,s);
00887     i=isfmt=0;
00888 }
00889 else if (c=='c') {
00890     fstr[i]=0;
00891     Sb.catf(fstr, va_arg(args, int ));
00892     i=isfmt=0;
00893 }
00894 else if (strchr("eEgf",c)) {
00895     fstr[i]=0;
00896     m=0; if (i>1) if (fstr[i-2]=='L') m++;
00897     if (m) Sb.catf(fstr, va_arg(args, long double));
00898     else Sb.catf(fstr, va_arg(args, double));
00899     i=isfmt=0;
00900 }
00901 else if (c=='N') {
00902     fstr[i-2]=0;
00903     Sb.catf("%s\n",fstr);
00904     i=isfmt=0;
00905 }
00906 else if (c=='Y') {
00907     fstr[i-2]=0; l=16;
00908     char c_; c_=time_stamp[l];
00909     time_stamp[l]=0; Sb.catf("%s%s",fstr,time_stamp);
00910     time_stamp[l]=c_;
00911
00912     i=isfmt=0;
00913 }
00914 else if (c=='R') {
00915     k=i; fstr[--i]=0;
00916
00917     for (; i<flen; --i) { if (fstr[i]=='%') break; }
00918
00919     if (i>=flen) {
00920         Sb.catf("%s:%d ERR invalid format '%s'\n", FLINE, fmt-k);
00921     }
00922     else {
00923         fstr[i]=0; Sb.catf("%s",fstr);
00924     }

```

```

00925         if (sscanf(fstr+i+1,"%d",&k)<=0) {
00926             Sb.catf("%s:%d ERR invalid format '%s'\n", FLINE, fmt-k);
00927         }
00928         else {
00929             snprintf(fstr,flen,"%s",va_arg(args, char *));
00930             for (; k>0; --k) { Sb.catf("%s",fstr); }
00931         }
00932     }
00933
00934     i=isfmt=0;
00935 }
00936 else if (c=='B') {
00937     k = i; fstr[--i]=0;
00938
00939     for (; int(i)>=0; i--)
00940         if (fstr[i]=='%') break;
00941
00942     if (int(i)<0)
00943         Sb.catf("%s:%d ERR Invalid format >%s<", FLINE, fmt-k);
00944     else {
00945         fstr[i] = 0;
00946         Sb.catf("%s",fstr);
00947
00948         i = sscanf(fstr+i+1, "%d", &k);
00949         if (i<=0) k=8;
00950
00951         m = va_arg(args, int);
00952         cp = (char*)&m;
00953
00954         if (k>0)
00955             for (i=1=0; i<((k-1)/8+1); i++) {
00956                 for (cb=1, j=0; j<CHAR_BIT; ++j, cb<=1) {
00957                     if (cp[i] & cb) Sb.catf("1");
00958                     else Sb.catf("0");
00959                     if ((++j)>=k) break;
00960                 }
00961             }
00962     }
00963     i=isfmt=0;
00964 }
00965 else {
00966     fstr[i]=0; i=isfmt=0;
00967     Sb.catf(" »%s« ERR-fmt [..%s]\n",fstr,fmt);
00968     if (xcol) { Sb.catf("\e[0m"); }
00969     return -1;
00970 }
00971 }
00972 }
00973 fstr[i]=0;
00974
00975 if (i && (fstr[i-1]=='\\')) { fstr[i-1]=0; bflag|=2; }
00976 if (!(bflag & 2)) strcat(fstr,"\n");
00977
00978 Sb.catf("%s",fstr);
00979 if (xcol) { Sb.catf("\e[0m"); }
00980
00981 if (nesc && !xcol_) {
00982     Sb.skipEscCols();
00983 }
00984
00985 if (eflag) { rval|=8; }
00986 if (wflag) { rval|=4; }
00987 if (iflag) { rval|=2; }; rval|=1;
00988
00989 #ifdef DBG_GCX_LOCKS
00990     if (!Hflag && (eflag || wflag)) { LKF.flush(); }
00991 #endif
00992
00993 if (eflag) { if (Hflag) { ++wblog::ERR_pending; } else {
00994     #ifdef DBSTOP
00995     { unsigned n=wblog::myIO.size();
00996         if (n) {
00997             fprintf(stdout,"\n"
00998                 "Got %d pending entr%s in I/O buffer:\n\n",n,n==1?"y":"ies");
00999             for (const wblog::stdio_buf &b : wblog::myIO) {
01000                 b.print_stdout(stdout);
01001             }
01002         }
01003         Sb.print(FL,"DBSTOP"); fflush(stdout);
01004     }
01005     #endif
01006     throw Wb::LogException(ERR);
01007 } }
01008 #ifdef DBSTOP
01009     else if (wflag) {
01010         Sb.print(FL,"DBSTOP");
01011         dbstop(file ? file : __FILE__, file ? line : __LINE__);

```



```

01012     }
01013 #endif
01014
01015     if (!Hflag) { wblog::check_ERR_pending(); }
01016     return rval;
01017 };
01018
01019 void wblog::check_ERR_pending() {
01020     if (wblog::ERR_pending) { char s[32];
01021         int q=wblog::ERR_pending; wblog::ERR_pending=0;
01022         snprintf(s,32,"pending error%s (e=%d)",q>1? "s":"",q);
01023         ExitMsg(s);
01024     }
01025 }
01026 };
01027
01028 unsigned wblog_check_tag(const char *fmt, const char *t0, char *tag) {
01029
01030     const char *s;
01031     int i,k,l=strlen(t0); if (!l) { return 0; }
01032
01033     s=strstr(fmt,t0); if (!s) { return 0; }
01034     k=s-fmt;
01035
01036     if (isalnum(s[l])) { return 0; }
01037     for (i=0; i<k; ++i) {
01038         if (isspace(fmt[i])) { return 0; }
01039     }
01040
01041     memcpy(tag,t0,l); tag[l]=0;
01042
01043     l+=k; if (fmt[l]) { ++l; }
01044
01045     return l;
01046 };
01047
01048 char wblog_findtoken(const char *istr, const char *tok, int maxoffset) {
01049
01050     char f=1; const char *c=strstr(istr,tok);
01051
01052     if (c==0) return 0;
01053     if (maxoffset>=0) if (c>istr+maxoffset) return 0;
01054
01055     if (c>istr)
01056         if (isalnum(*(c-1))) f=0;
01057         if (isalnum(*(c+3))) f=0;
01058
01059     return f;
01060 }
01061
01062 void Wb::SBUF::flush(FILE *fid, char fflag) {
01063
01064     if (!slen || !sbuf) { return; }
01065
01066     if (l && sbuf[0]) {
01067
01068         #ifdef MATLAB_MEX_FILE
01069         if (fid==stdout || fid==stderr) { int tid=omp_get_thread_num();
01070
01071             if (tid!=Wb::my_caller_tid) {
01072                 wblog::myIO.push_back(wblog::stdio_buf(*this,fid,tid));
01073             }
01074             else {
01075                 wblog::myIO.clear();
01076
01077                 if (Wb::envDKT==1) { mexPrintf("%s",sbuf); }
01078                 else {
01079                     PRINTF("%s",sbuf);
01080                     fflush(stdout);
01081                 }
01082             }
01083         }
01084         else { fprintf(fid,"%s",sbuf); fflush(fid); }
01085         #else
01086         { fprintf(fid,"%s",sbuf); fflush(fid); }
01087         #endif
01088
01089         if (fflag && (fid==stdout || fid==stderr)) { doflush(); }
01090     }
01091
01092     if (sbuf) { sbuf[0]=0; l=0; }
01093 };
01094
01095 void Wb::SBUF::print(const char *F, int L, const char *istr) {
01096
01097     if (!l) return;
01098     if (!slen || l>slen) { fprintf(stderr,

```

```

01099     "\n\n%s %s got l=%d/%d !?\n\n",shortFLT,l,slen);
01100     return;
01101 }
01102
01103 unsigned n=1+64, j=(sbuf[0]=='\n' ? 1:0); char s[n];
01104 char *sj=sbuf+j, c=sbuf[1]; sbuf[1]=0;
01105
01106 if (istr) { snprintf(s,n,
01107     "\n%s (%s in %s)\n%s", shortFL(F_L),istr,myname,sj);
01108 }
01109 else if (F) { snprintf(s,n,"\n%s\n%s",shortFL(F,L),sj); }
01110 else { snprintf(s,n,"\rTST> ` %s'",sj); }
01111
01112 PRINTF("\e[3%dm%s\e[0m\n",j?1:5,s);
01113
01114 sbuf[1]=c;
01115 };
01116
01117 void Wb::SBUF::increase_size(const char *F, int L, unsigned n) {
01118
01119     if (!slen || !sbuf || l>=slen) {
01120         error_bounds(F_L,FCT,"");
01121         ExitMsg("");
01122     }
01123     else if (ref) { char s[64];
01124         snprintf(s,64,"\n\n%s "
01125             "ERR cannot increase size %d -> %d for ref=%d data\n\n",
01126             shortFL(F_L),slen,n,ref);
01127         ExitMsg(s);
01128     }
01129     else if (wblog::SLEN<=32) { char s[64];
01130         snprintf(s,64,"\n\n%s ERR got %p @ %d, L=%d !?\n\n",
01131             shortFL(F_L),sbuf,slen,wblog::SLEN);
01132         ExitMsg(s);
01133     }
01134     else if (n>=slen) {
01135
01136         unsigned nx=(1+(n+1)/wblog::SLEN)*wblog::SLEN;
01137         char *sx = new char[nx];
01138         if (!sx) { char s[64]; snprintf(s,64,
01139             "\n\n%s ERR allocation error (n=%d)\n\n",shortFL(F_L),nx);
01140             ExitMsg(s);
01141         }
01142         sbuf[slen-1]=0;
01143         strcpy(sx,sbuf); delete [] sbuf; sbuf=sx; slen=nx;
01144
01145         #if 0
01146         nx=strlen(sbuf);
01147         fprintf(stderr,"%s TST increasing sbuf[ %d -> %d ] having "
01148             "l=%d+%d (%d)\n",SHORT_FL,nx+1,slen, l,n-1,n);
01149         if (Wb::envVRB & 8)
01150             fprintf(stderr,"%s' (%X)\n\n",sbuf,Wb::envVRB);
01151         #endif
01152     }
01153 };
01154
01155 void Wb::SBUF::error_bounds(
01156     const char *F, int L, const char *fct, const char *fmt) {
01157
01158     if (sbuf && slen) {
01159         sbuf[slen-1]=0;
01160         fprintf(stderr,
01161             "\n\n%s ERR %s() string out of bounds (l=%d+%ld/%d):\n\n%s\n\n",
01162             shortFL(F_L),fct?fct:"(fct)",l,fmt ? strlen(fmt):-1,slen,sbuf);
01163         if (fmt) fprintf(stderr," ERR fmt = '%s'\n\n",fmt);
01164         if (l>=slen) ExitMsg("");
01165     }
01166     else {
01167         char s[128]; snprintf(s,128,
01168             "\n\n%s ERR %s() got uninitialized %p @ l=%d+%ld/%d !?\n\n",
01169             shortFL(F_L),fct?fct:"(fct)",sbuf,l,fmt ? strlen(fmt):-1,slen);
01170         ExitMsg(s);
01171     }
01172 };
01173
01174 int Wb::SBUF::cat(const char *s) {
01175     int n=0;
01176
01177     if (s && s[0]) {
01178         n=strlen(s); if (l+n>=slen) { increase_size(F_L,l+n); }
01179         strcpy(sbuf+l,s); l+=n;
01180     }
01181     return n;
01182 };
01183
01184 int Wb::SBUF::catf(const char *fmt, ...) {
01185

```

```

01186     if (!fmt || !fmt[0]) { return 0; }
01187
01188     va_list args; Wb::ARGV wd(&args);
01189     va_start(args, fmt);
01190
01191     unsigned iter=0, n=strlen(fmt)+32;
01192     if (l+n>=slen) { increase_size(FL, l+n); }
01193     if (l>=slen) {
01194         fprintf(stdout, "%s %s ERR got l=%d/%d (%d)", shortFLT, l, slen, n);
01195         fflush(stdout);
01196     }
01197
01198     while (1) {
01199         n=vsnprintf(sbuf+l, slen-l, fmt, args);
01200         if (l+n<slen) { break; } else increase_size(FL, l+n);
01201
01202         va_end(args); va_start(args, fmt);
01203         if (++iter>2) fprintf(stdout, "%s %s ERR got iter=%d", shortFLT, iter);
01204     }
01205
01206     if (iter>1) {
01207         fprintf(stderr, "%s TST increasing sbuf[%d->%d] having "
01208             "l=%d+%d (%d)\n", SHORT_FL, wblog::SLEN, slen, l, n, l+n);
01209         if (Wb::envVRB & 8)
01210             fprintf(stderr, "\n`s' (%X)\n\n", sbuf, Wb::envVRB);
01211     }
01212
01213     l+=n; return n;
01214 };
01215
01216 int Wb::SBUF::skipEscCols() {
01217     int nesc=0, nskip=0;
01218     if (!sbuf) { return (nesc=-1); }
01219     unsigned i=0, k=0, j=0;
01220
01221     for (; sbuf[i]; ++i) {
01222         if (sbuf[i]=='\e' && sbuf[i+1]=='[') { j=i+2; } else
01223         if (sbuf[i]=='\\' && sbuf[i+1]=='e' && sbuf[i+2]=='[') { j=j+3; }
01224         if (!j) { if (k<i) { sbuf[k]=sbuf[i]; }; ++k; }
01225         else {
01226             for (; sbuf[j]; ++j) {
01227                 if (!isdigit(sbuf[j]) && sbuf[j]!=';') { break; }
01228             }
01229             if (sbuf[j]=='m')
01230                 { ++nesc; i=j; }
01231             else { ++nskip; if (k<i) { sbuf[k]=sbuf[i]; }; ++k; }
01232             j=0;
01233         }
01234     }
01235
01236     if (k<i) { sbuf[k]=0; l=k; }
01237
01238     if (!nesc) { nesc=-nskip; }
01239     return nesc;
01240 };
01241
01242 char* Wb::surdStrf(wbstring &s, const char *fmt, ...) {
01243     va_list args; Wb::ARGV wd(&args);
01244     va_start(args, fmt);
01245     if (!s) { s.init(32); }
01246     return surdStrf(s.data, s.len, fmt, args);
01247 };
01248
01249 char* Wb::surdStrf(char *s, unsigned n, const char *fmt, va_list args) {
01250
01251     if (!s || n<8) wblog(FL, "ERR %s() invalid usage (s=%g, n=%d)", FCT, s, n);
01252     if (!fmt || !fmt[0]) { s[0]=0; return s; }
01253
01254     unsigned i, k, l;
01255
01256     k=std::wcrctomb(s, u'\u221A', NULL);
01257
01258     i=l=k+1;
01259     l+=vsnprintf(s+l, n-l, fmt, args);
01260
01261     for (; i<n && s[i]; ++i) { if (s[i]<'0' || s[i]>'9') break; }
01262
01263     if (l+(s[i] ? 1:0)>=n) wblog(FL,
01264         "WRN %s() string out of bounds (%d/%d)\n`s'", FCT, l, n, s);
01265
01266     if (s[i]) {
01267         s[k]='('; s[l]=')'; i=0;
01268     }
01269     else {
01270         for (i=k; i; --i) { s[i]=s[i-1]; }
01271         s[i++]=' ';
01272     }

```

```

01273     return s+i;
01274 };
01275
01276 Wb::termcolor& Wb::termcolor::init(unsigned k) {
01277
01278     if (em) { delete [] em; em=e1=NULL; }
01279
01280     if (!Wb::useCol) { em=e1 = new char[1]; em[0]=0; }
01281     else {
01282         unsigned l, n=(k<8 ? 12 : 18);
01283         em = new char[n];
01284         l=snprintf(em,n,"\e[0m");
01285         e1 = em + (++l);
01286
01287         if (k< 8) { l+=snprintf(e1,n-1,"\e[3%dm",k); } else
01288         if (k<256) { l+=snprintf(e1,n-1,"\e[38;5;%dm",k); }
01289         else wblog(FL,"ERR %s() color index out of bounds (%d)",FCT,k);
01290
01291         if (l>=n) wblog(FL,
01292             "WRN %s() termcolor() string out of bounds (%d/%d)",FCT,l,n);
01293     }
01294
01295     return *this;
01296 };
01297
01298 int Wb::termcolor::ID(const char *tag) {
01299
01300     int i=0;
01301
01302     if (Wb::useCol && tag && tag[0]) {
01303         switch (tag[0]) {
01304             case 'h':
01305                 if (!strcmp(tag,"hdr")) { i=240; }
01306                 break;
01307             case 'E':
01308                 if (!strcmp(tag,"ERR")) { i= 1; } else
01309                 if (!strcmp(tag,"ENV")) { i=240; }
01310                 break;
01311             case 'W':
01312                 if (!strcmp(tag,"WRN")) { i= 9; }
01313                 break;
01314             case ' ':
01315                 if (!strcmp(tag," *")) { i=243; } else
01316                 if (!strcmp(tag," * ")) { i=246; }
01317                 break;
01318             case 'T':
01319                 if (!strcmp(tag,"TST")) { i=246; }
01320                 break;
01321             case 'N':
01322                 if (!strcmp(tag,"NB!")) { i= 34; }
01323                 break;
01324             case 'D':
01325                 if (!strcmp(tag,"DBG")) { i=130; }
01326                 break;
01327             case 'X':
01328                 if (tag[1]=='X') {
01329                     if (tag[2]=='E') { i=1; } else
01330                     if (tag[2]=='W') { i=9; } else
01331                     if (tag[2]=='G') { i=2; } else
01332                     if (tag[2]=='Y') { i=3; } else
01333                     if (tag[2]=='B') { i=4; }
01334                 }
01335                 break;
01336             }}
01337
01338     return i;
01339 };
01340
01341 #endif
01342

```

5.43 wblog.h

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : wblog (logging routines)
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *

```

```

00011 *      http://www.apache.org/licenses/LICENSE-2.0
00012 *
00013 * Unless required by applicable law or agreed to in writing, software
00014 * distributed under the License is distributed on an "AS IS" BASIS,
00015 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016 * See the License for the specific language governing permissions and
00017 * limitations under the License.
00018 * ----- */
00019
00020 #ifndef __WB_LOG_H__
00021 #define __WB_LOG_H__
00022
00023 // e.g. set in matlab via
00024 // » setenv('WB_VERBOSE',num2str(1<10 + 1<21));
00025 // » setenvb WB_VERBOSE 10 21
00026 // read in wblib.h -> envVRB=gathered.cc -> get_WB_VERBOSE()
00027 // default 0xF1; // 0xF1 = 241: bit 0 and 4..7
00028
00029 #define WBL_MSEC__      (1U<10)
00030 #define WBL_TCAST__    (1U<12)
00031
00032 #define WBL_CLK__      (1U<20)
00033 #define WBL_CTR__      (1U<21)
00034 #define WBL_MMEX__     (1U<22)
00035
00036 #define WBL_IO__       (1U<23)
00037 #define WBL_RESIZE__   (1U<24)
00038 #define WBL_RLARGE__   (1U<25)
00039
00040 #define WBLOG_MSEC      (Wb::envVRB & WBL_MSEC__ )
00041 #define WBLOG_TCAST     (Wb::envVRB & WBL_TCAST__ )
00042 #define WBLOG_CLK       (Wb::envVRB & (WBL_CLK__ | 192))
00043 #define WBLOG_CTR       (Wb::envVRB & WBL_CTR__ )
00044 #define WBLOG_MMEX      (Wb::envVRB & WBL_MMEX__ )
00045 #define WBLOG_IO        (Wb::envVRB & WBL_IO__ )
00046 #define WBLOG_RESIZE    (Wb::envVRB & WBL_RESIZE__ )
00047 #define WBLOG_RLARGE    (Wb::envVRB & WBL_RLARGE__ )
00048
00049 #define WBL_HLEN 20
00050 #define WBL_GOT_SHORTFL -99
00051
00052 const char* shortFL(
00053     const char *F, int L=-1, unsigned n=WBL_HLEN,
00054     const char *P=NULL, char sep=':');
00055
00056 const char* tmpmatFL(
00057     const char *F, int L, const char *vn, char *s, unsigned n);
00058
00059 enum WBLOG_TYPE { OK, WRN, ERR, NUM_TYPE };
00060
00061 const char* WBLOG_TYPE_STR[NUM_TYPE] = { "OK","WRN","ERR" };
00062
00063 namespace Wb {
00064
00065     struct LogException : public exception {
00066     public:
00067         LogException (WBLOG_TYPE l=OK, const char *s=NULL);
00068
00069         LogException (const LogException &e)
00070             : type(e.type), nrefs(e.nrefs), count(e.count), ith(e.ith), nth(e.nth) {
00071             if (nrefs) { ++(*nrefs); ++count;
00072             }
00073             istr[63]=0;
00074             strncpy(istr,e.istr,64);
00075             if (istr[63]) { istr[61]=istr[62]='.'; istr[63]=0; }
00076         };
00077
00078         ~LogException() noexcept {
00079             if (nrefs) {
00080                 if (--(*nrefs)<1) {
00081                     delete nrefs; nrefs=NULL;
00082                 }
00083             } else {
00084             }
00085         };
00086
00087         LogException& operator=(const LogException &e) {
00088             type=e.type; count=e.count; ith=e.ith; nth=e.nth;
00089             nrefs=e.nrefs; if (nrefs) { ++(*nrefs);
00090             }
00091             istr[63]=0;
00092             strncpy(istr,e.istr,64);
00093             if (istr[63]) { istr[61]=istr[62]='.'; istr[63]=0; }
00094             return *this;
00095         };
00096
00097         LogException& operator++() {

```

```

00098         ++count;
00099
00100         int n=omp_get_num_threads();
00101         if (nth<n) { nth=n; ith=omp_get_thread_num(); }
00102
00103         return *this;
00104     };
00105
00106     LogException& operator+=(const LogException &e) {
00107         if (!type)
00108             { (*this)=e; count=1; }
00109         else { ++count; }
00110
00111         int n=omp_get_num_threads();
00112         if (nth<n) { nth=n; ith=omp_get_thread_num(); }
00113
00114         return *this;
00115     };
00116
00117     bool operator!() const { return (!type && !count); }
00118     explicit operator bool() const { return (type || count); }
00119
00120     LogException& init() {
00121         if (nrefs) { --(nrefs);
00122             if (*nrefs<1) { delete nrefs; nrefs=NULL; }
00123         }
00124         type=OK; count=ith=nth=0; istr[0]=0;
00125         return *this;
00126     };
00127
00128     const char* what() const throw () { return istr; };
00129
00130     wstring toStr() const;
00131     void report(const char *F=0, int L=0, const char *func=0);
00132
00133     WBLOG_TYPE type;
00134     int *nrefs;
00135     char istr[64];
00136     unsigned count;
00137     int ith, nth;
00138
00139 private:
00140 };
00141
00142 char* surdStrf(wstring &s, const char *fmt, ...);
00143 char* surdStrf(char *s, unsigned n, const char *fmt, va_list args);
00144
00145 enum class TCOLS {
00146     BLACK, RED, GREEN, YLW, BLUE, PINK, CYAN, LGRAY, DGRAY,
00147     LRED, LGREEN, LYLW, LBLUE, LPNK, LCYAN, WHITE
00148 };
00149
00150 class termcolor {
00151 public:
00152     termcolor(const char *tag)
00153         : em(NULL), el(NULL) { int k=ID(tag); init(k); };
00154
00155     termcolor(Wb::TCOLS k)
00156         : em(NULL), el(NULL) { init(unsigned(k)); };
00157
00158     termcolor(unsigned k)
00159         : em(NULL), el(NULL) { init(k); };
00160
00161     ~termcolor() {
00162         if (em) { delete [] em; }
00163         em=el=NULL;
00164     }
00165
00166     termcolor& init(unsigned k);
00167
00168     static int ID(const char *tag);
00169
00170     char *em, *el;
00171
00172 private:
00173 };
00174
00175 class SBUF {
00176 public:
00177     SBUF(unsigned l0=wblog::SLEN, char *s=NULL)
00178         : l(0), slen(0), ref(0), sbuf(NULL) {
00179         init(l0,s);
00180     };
00181
00182     ~SBUF() { if (sbuf) {

```

```

00185     #pragma omp critical (__ensure_sequential_wblog__)
00186     { flush(stderr,1); }
00187     if (!ref) { delete [] sbuf; }
00188     sbuf=NULL;
00189 };
00190
00191 SBUF& init(unsigned l0=0, char *s=NULL) { l=0;
00192     if (sbuf) { if (!ref) { delete [] sbuf; }; sbuf=NULL; }
00193     slen=l0; ref=0;
00194     if (slen) {
00195         if (s) { sbuf=s; ref=1; }
00196         else {
00197             sbuf = new char[slen];
00198             if (!sbuf) { sprintf_str("ERR %s() "
00199                 "failed to allocate sbuf (l=%d)",__FUNCTION__,slen);
00200                 ExitMsg(str);
00201             }
00202         }
00203         memset(sbuf,0,slen);
00204     }
00205     return *this;
00206 };
00207
00208 int cat(const char *s);
00209 int catf(const char *fmt, ...);
00210
00211 void error_bounds(const char *F, int L, const char *fct, const char *fmt);
00212 void increase_size(const char *F, int L, unsigned n);
00213
00214 void flush(FILE *fid=stdout, char fflag=0);
00215 void print(const char *F=0, int L=0, const char *istr=0);
00216
00217 int skipEscCols();
00218
00219 unsigned l, slen, ref;
00220
00221 char *sbuf;
00222
00223 private:
00224 };
00225
00226 };
00227
00228 namespace wblog {
00229
00230 class stdio_buf {
00231 public:
00232     stdio_buf(Wb::SBUF &S, FILE *f_, int t_) : s(NULL), fid(0), tid(0), ref(0) {
00233         if (f_!=stdin && f_!=stdout) {
00234             fprintf(stderr,"\\nERR %s() invalid fid=%p\\n'%s'\\n\\n",
00235                 FCT,f_,S.sbuf?S.sbuf:"");
00236         }
00237         if (S.sbuf) {
00238             s=S.sbuf; fid=f_; tid=t_; ref=1;
00239             S.sbuf=0; S.slen=S.l=0;
00240         }
00241     };
00242
00243     stdio_buf(const stdio_buf &b) : s(NULL), fid(0), ref(0) {
00244         if (!b.ref) fprintf(stderr,"\\nERR %s() got b.ref=%d\\n\\n",FCT,b.ref);
00245         s=b.s; fid=b.fid; tid=b.tid;
00246     };
00247
00248     void print_stdout(FILE *f=NULL) const {
00249         if (Wb::envVRB &256) {
00250             unsigned i=0, m=0; for (; s[i]; ++i) {
00251                 if (s[i]!='\\n') { if (++m>2) break; } else m=0;
00252             }
00253             if (m>2) { i-=2; char *x = (char*)s;
00254                 if (tid<10) x[i]=tid + '0'; else
00255                 if (tid<36) x[i]=tid-10+'a'; else
00256                 if (tid<62) x[i]=tid-36+'A'; else { x[i]='*'; }; x[i+1]=' ';
00257             }
00258         }
00259         if (f) { fprintf(f,"%s",s); }
00260         else {
00261             if (Wb::envDKT==1)
00262                 { mexPrintf("%s",s); }
00263             else { PRINTF("%s",s); }
00264         }
00265     };
00266
00267     ~stdio_buf() { if (s && !ref) {
00268         if (Wb::my_caller_tid!=omp_get_thread_num()) {
00269             fprintf(stderr,"WRN %s() got tid=%d/%d !?\\nWRN '%s'\\n",
00270                 FCT,omp_get_thread_num(),Wb::my_caller_tid,s);
00271         }
00272     }

```

```

00272         else {
00273             print_stdout();
00274             delete [] s;
00275             fflush(fid);
00276         }
00277     };
00278
00279 private:
00280     const char *s;
00281     FILE *fid;
00282     int tid;
00283     char ref;
00284 };
00285
00286     std::deque<stdio_buf> myIO;
00287
00288     void check_ERR_pending();
00289     int ERR_pending=0;
00290
00291 };
00292
00293     void wbSetLogLevel(unsigned l);
00294
00295     int wblogs(Wb::SBUF &S,
00296               WBL_COLOR_SCHEME xcol,
00297               const char* file, int line,
00298               const char *fmt, va_list args,
00299               char Hflag=0
00300             );
00301
00302     int wblogs(
00303         Wb::SBUF &S, WBL_COLOR_SCHEME xcol,
00304         const char* file, int line, const char *fmt, ...
00305     ){
00306         va_list args; Wb::ARGV wd(&args);
00307         va_start(args,fmt);
00308         return wblogs(S,xcol,file,line,fmt,args);
00309     };
00310
00311     int wblogf(FILE *fid,
00312               const char* file, int line, const char *fmt, va_list args);
00313
00314     int wblogf(FILE *fid,
00315               const char* file, int line, const char *fmt, ...) {
00316
00317         va_list args; Wb::ARGV wd(&args);
00318         va_start(args,fmt);
00319
00320         int l=wblogf(fid,file,line,fmt,args);
00321
00322         return l;
00323     };
00324
00325     int wblogl(const char* file, int line, const char *fmt, ...);
00326
00327 #define wblog(...) wblogf(stdout, __VA_ARGS__)
00328
00329     int wb_printf(const char *fmt, ...) {
00330
00331         unsigned l=0, n=256;
00332         Wb::SBUF S(n); {
00333             va_list args; Wb::ARGV wd(&args);
00334             va_start(args,fmt);
00335             l=vsprintf(S.sbuf,n,fmt,args);
00336         }
00337         if (l>=n) fprintf(stderr,
00338             "\nERR string out of bounds (%d/%d)\nERR %s\n\n",l,n,S.sbuf);
00339
00340         #pragma omp critical (__ensure_sequential_wblog__)
00341         { S.flush(); }
00342
00343         return l;
00344     };
00345
00346     unsigned wblog_check_tag(const char *fmt, const char *t, char *tag);
00347
00348     char wblog_findtoken(const char *istr, const char *tok, int maxoffset=-1);
00349
00350     void banner(
00351         unsigned n, const char *s,
00352         const char *istr=NULL, const char *fstr=NULL
00353     );
00354
00355 #endif
00356

```


5.44 wbMatlO.hh

```

00001 #ifndef __WB_MatIO_HCC__
00002 #define __WB_MatIO_HCC__
00003
00004 // IO handling of mat files (MatLab binaries)
00005
00006 namespace Wb {
00007
00008 class matFile {
00009 public:
00010
00011     matFile() : mfp(NULL) {};
00012
00013     matFile(const char *F, int L, const char *fname, const char *rw
00014 ) : mfp(NULL) { open(F,L,fname,rw,1); };
00015
00016     matFile(const matFile &X) : mfp(NULL) {
00017         wblog(FL,"ERR do not copy matFile object! (0x%lX)",&X);
00018     };
00019
00020 ~matFile() {
00021     init();
00022 };
00023
00024 void close() { init(); };
00025 void init() {
00026     if (mfp) if (matClose(mfp)) {
00027         wblog(F.data,L,"WRN Could not close file %s%s /%s !?",
00028             strlen(fname.data)>20 ? "\n" : "", fname.data, rw.data);
00029     }
00030     mfp=NULL; F.init(); L=0; fname.init(); rw.init();
00031 };
00032
00033 int Open(const char *F, int L, const char *FName, const char *RW
00034 ){ return open(F,L,FName,RW,1); };
00035
00036 int open(
00037     const char *file, int line, const char *FName, const char *RW,
00038     char xflag=0
00039 ){
00040     if (mfp) init();
00041     F=file; L=line; fname=FName;
00042     if (!RW || !RW[0]) wblog(FL,
00043         "ERR %s() got empty read/write flag",FCT);
00044
00045     if (!strcmp(RW,"w")) {
00046         rw="w7.3";
00047     }
00048     else if (!strcmp(RW,"w:std")) {
00049         rw="wz";
00050     }
00051     else { rw=RW; }
00052
00053     mfp=matOpen(fname.data, rw.data);
00054     if (!mfp) {
00055         if (xflag) { unsigned n=strlen(fname.data);
00056             if (n<50)
00057                 wblog(F.data,L,"ERR failed to %s access%s%s"
00058                     "\n(hint: non-existing directory?)",
00059                     rw.data, n<30 ? ":" : " file\n",fname.data);
00060             else {
00061                 char *x,cwd[256]; cwd[0]=0; x=getcwd(cwd,256);
00062                 wblog(F.data,L,"ERR failed to access (%s) ... %N%N"
00063                     "   mat: %s !?%N"
00064                     "   cwd: %s%N"
00065                     " (hint: non-existing directory or no rw permission)%N",
00066                     rw.data, fname.data, x ? cwd : "(pwd)");
00067             }
00068         }
00069         init(); return 0;
00070     }
00071     return 1;
00072 };
00073
00074 matFile& put(
00075     const char *file, int line,
00076     const char *vname, const mxArray* a
00077 ){
00078     int i=matPutVariable(mfp,vname,a);
00079     if (i) {
00080         wblog(file ? file : F.data, line ? line : L,
00081             "ERR matPutVariable() returned %d \n(%s,%s; 0x%lX) !?\n"
00082             "hint: possibly out of disc space", i,
00083             fname.data ? fname.data : "(undefined)",
00084             vname ? vname : "(empty)", mfp
00085         );
00086     }
00087 }

```

```

00086         }
00087         return *this;
00088     };
00089     matFile& put(const char *vname, const mxArray* a){
00090         return put(0,0,vname,a);
00091     };
00092
00093     int existsVar(const char* vn) const {
00094         mxArray *a=matGetVariableInfo(mfp,vn);
00095         int r=(a!=NULL);
00096         mxDestroyArray(a);
00097         return r;
00098     };
00099
00100     int deleteVar(const char* vn) const {
00101         int i=matDeleteVariable(mfp,vn);
00102         return i;
00103     };
00104
00105     int deleteVar(const char *f, int L, const char* vn) const {
00106         int i=matDeleteVariable(mfp,vn);
00107         if (i) wblog(f,L,
00108             "ERR Failed to delete variable '%s'\n"
00109             "from file '%s' (0x%lX; %d/%d ?)",
00110             vn, fname.data, mfp, existsVar(vn), i);
00111         return i;
00112     };
00113
00114     MATFile *mfp;
00115
00116     wbsting F, fname, rw;
00117     int L;
00118
00119     protected:
00120     private:
00121 };
00122
00123 }
00124
00125 #endif
00126

```

5.45 wbMatrix.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : wbMatrix (matrix class, row-major)
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_MATRIX_ROW_MAJOR_CC__
00021 #define __WB_MATRIX_ROW_MAJOR_CC__
00022
00023 /* ----- */
00024 /* ----- */
00025 // quick integer data type // Wb,Dec02,11
00026
00027 template<class T> inline
00028 bool wbMatrix<T>::thisIsInt() const { return 0; };
00029
00030 template<> inline
00031 bool wbMatrix<int>::thisIsInt() const { return 1; };
00032
00033 template<> inline
00034 bool wbMatrix<unsigned>::thisIsInt() const { return 1; };
00035
00036 template<> inline
00037 bool wbMatrix<size_t>::thisIsInt() const { return 1; };
00038
00039 template<> inline

```

```

00040 bool wbMatrix<char>::thisIsInt() const { return 1; };
00041
00042 template <> inline
00043 bool wbMatrix<unsigned char>::thisIsInt() const { return 1; };
00044
00045 template <>
00046 bool wbMatrix<double>::isNormal() const {
00047     for (size_t n=dim1*dim2, i=0; i<n; ++i)
00048         if (data[i] && !isnormal(data[i])) return 0;
00049     return 1;
00050 };
00051
00052 template <>
00053 bool wbMatrix<float>::isNormal() const {
00054     for (size_t n=dim1*dim2, i=0; i<n; ++i)
00055         if (data[i] && !isnormal(data[i])) return 0;
00056     return 1;
00057 };
00058
00059 template <>
00060 bool wbMatrix<wbcomplex>::isNormal() const {
00061     for (size_t n=dim1*dim2, i=0; i<n; ++i) {
00062         if (data[i].r && !isnormal(data[i].r)) return 0;
00063         if (data[i].i && !isnormal(data[i].i)) return 0;
00064     }
00065     return 1;
00066 };
00067
00068 template <> bool wbMatrix<unsigned>::isNormal() const { return 1; };
00069 template <> bool wbMatrix<size_t>::isNormal() const { return 1; };
00070 template <> bool wbMatrix<int>::isNormal() const { return 1; };
00071 template <> bool wbMatrix<char>::isNormal() const { return 1; };
00072 template <> bool wbMatrix<long>::isNormal() const { return 1; };
00073
00074 template <class T> inline
00075 wbMatrix<T>& wbMatrix<T>::init(
00076     const char *F, int L, const mxArray *a,
00077     char tflag, char ref, char tcheck
00078 ){
00079     if (!a || mxIsEmpty(a)) { init(); return *this; }
00080
00081     unsigned r=mxGetNumberOfDimensions(a);
00082     const size_t *S=mxGetDimensions(a);
00083
00084     if (r!=2 || !mxIsNumChar(a)) wblog(F,L,
00085         "ERR got rank-%d array '%s'",r,mxGetClassName(a));
00086
00087     size_t d1=(tflag ? S[1]:S[0]), d2=(tflag ? S[0]:S[1]);
00088
00089     Mx::Array<T> A(F,L,a,!tflag);
00090     if (!ref || !A.data) { if (ref) wblog(F,L,"WRN invalid ref: %s",STR(A));
00091         RENEW(d1,d2);
00092         A.copyTo(data,tcheck);
00093     }
00094     else { init2ref(d1,d2,A.data); }
00095
00096     return *this;
00097 };
00098
00099 template <class T> inline
00100 wbMatrix<T>& wbMatrix<T>::init2ref(size_t r, size_t c, const T* d) {
00101     if (data && !isref) { init(); }; isref=1;
00102     dim1=r; dim2=c; data=(T*)d;
00103     return *this;
00104 };
00105
00106 template <class T> inline
00107 wbMatrix<T>& wbMatrix<T>::init2ref(const wbvector<T> &v, const char tflag) {
00108     if (data && !isref) { init(); }; isref=1; data=v.data;
00109     if (tflag)
00110         { dim1=v.len; dim2=1; }
00111     else { dim1=1; dim2=v.len; }
00112     return *this;
00113 };
00114
00115 template <class T> inline
00116 wbMatrix<T>& wbMatrix<T>::init2ref(const wbMatrix<T> &M) {
00117     if (data && !isref) { init(); }; isref=1;
00118     dim1=M.dim1; dim2=M.dim2; data=M.data;
00119     return *this;
00120 };
00121
00122 template <class T>
00123 template <class TA, class TB>
00124 wbMatrix<T>& wbMatrix<T>::init(
00125     const char *F, int L,
00126     const wbMatrix<TA> &M, const wbvector<TB> &V, char dim

```

```

00127 ){
00128     size_t i,j,n1=M.dim1,n2=M.dim2;
00129     if (abs(dim)==2) {
00130         if (n1!=V.len) wblog(F_L,
00131             "ERR %s() size mismatch %d/%dx%d",FCT,V.len,n1,n2);
00132         RENEW(n1,n2+1,0,0);
00133         if (dim>0) {
00134             for (i=0; i<n1; ++i) {
00135                 for (j=0; j<n2; ++j) (*this)(i,j)=(T)M(i,j);
00136                 (*this)(i,j)=(T)V[i];
00137             }
00138         }
00139         else {
00140             for (i=0; i<n1; ++i) {
00141                 (*this)(i,0)=(T)V[i];
00142                 for (j=0; j<n2; ++j) (*this)(i,j+1)=(T)M(i,j);
00143             }
00144         }
00145     }
00146     else if (abs(dim)==1) {
00147         if (n2!=V.len) wblog(F_L,
00148             "ERR %s() size mismatch %dx%d/%d",FCT,n1,n2,V.len);
00149         RENEW(n1+1,n2,0,0);
00150         if (dim>0) {
00151             for (i=0; i<n1; ++i)
00152                 for (j=0; j<n2; ++j) (*this)(i+1,j)=(T)M(i,j);
00153             for (j=0; j<n2; ++j) (*this)(i,j)=(T)V[j];
00154         }
00155         else {
00156             for (j=0; j<n2; ++j) (*this)(0,j)=(T)V[j];
00157             for (i=0; i<n1; ++i)
00158                 for (j=0; j<n2; ++j) (*this)(i+1,j)=(T)M(i,j);
00159         }
00160     }
00161     else wblog(F_L,"ERR %s() invalid dim=%d",FCT,dim);
00162
00163     return *this;
00164 };
00165
00166 template <class T> inline
00167 int matchSortedIdxU(
00168     const char *F, int L,
00169     C_TMAT &QA, C_TMAT &QB, windex &Ia, windex &Ib,
00170     widx_t m,
00171     char lex
00172 ){
00173     widx_t m1=0, m2=0; int i=0;
00174     if (QA.dim2!=QB.dim2) wblog(FL,"ERR %s() dimension mismatch "
00175         "%dx%d <> %d/%d",FCT,QA.dim1,QA.dim2,QB.dim1,QB.dim2);
00176
00177     try {
00178         i=Wb::matchSortedIdx(
00179             QA.data, QA.dim2, QA.dim1,
00180             QB.data, QB.dim2, QB.dim1, Ia,Ib, m, lex, &m1, &m2
00181         );
00182         if (m1 || m2) wblog(F_L,"ERR %s() "
00183             "got non-unique/sorted input records (%d,%d)",FCT,m1,m2);
00184     }
00185     catch (...) {
00186 #ifdef MATLAB_MEX_FILE
00187         MXPut(FL,"q").add(QA,"A").add(QB,"B").add(int(m),"m").add(lex,"lex");
00188         wblog(FL,"ERR %s()",FCT);
00189 #endif
00190     }
00191
00192     return i;
00193 };
00194
00195 template<class T> inline
00196 bool wbMatrix<T>::quickCheckSorted(
00197     char dir,
00198     char lex,
00199     const char *F, int L,
00200     size_t n
00201 ) const {
00202
00203     char c; dir=(dir>0 ? +1 : -1);
00204
00205     for (size_t i=1; i<dim1; i++) {
00206         c=recCompare(i,i-1,-1,lex);
00207         if (c) {
00208             if (c!=dir) {
00209                 if (F) wblog(F,L,
00210                     "ERR %s() input records not sorted %s (%d)",
00211                     FCT, dir>0 ? "ascendingly":"descendingly", i);
00212                 else return 0;
00213             }

```

```

00214         if (int(--n)<=0) break;
00215     }
00216 }
00217
00218     return 1;
00219 };
00220
00221 template<>
00222 WBINDEX& wbMatrix<size_t>::toIndex(
00223     WBINDEX &I, const WBINDEX *S) const {
00224
00225     const size_t *idx=data;
00226     if (S==NULL) {
00227         WBINDEX SX(dim2); widx_t *s=SX.data; size_t i=0,j;
00228
00229         for (; i<dim1; ++i, idx+=dim2) {
00230             for (j=0; j<dim2; ++j) {
00231                 if (s[j]<idx[j]) s[j]=idx[j];
00232             }
00233         }
00234         SX+=1; return toIndex(I,&SX);
00235     }
00236
00237     I.init(dim1); if (!dim1) return I;
00238     if (!dim2) wblog(FL,
00239         "ERR %s() got empty index matrix (%dx%d)",FCT,dim1,dim2);
00240
00241     size_t i=0,j, l=dim2-1;
00242     const widx_t *s=S->data;
00243
00244     for (; i<dim1; ++i, idx+=dim2) { widx_t &q=I.data[i];
00245         for (j=l, q=idx[j--]; j<l; --j) { q = q*size_t(s[j]) + size_t(idx[j]); }
00246     }
00247
00248     return I;
00249 };
00250
00251 template<>
00252 WBIDXMAT& wbMatrix<size_t>::toIndex2D(
00253     const WBINDEX &S, const wbindex &Ic,
00254     WBIDXMAT &IJ, char pos
00255 ) const {
00256
00257     wbvector<char> M(S.len);
00258     size_t i; char *m=M.data;
00259
00260     if (S.len!=dim2) wblog(FL,
00261         "ERR %s() size mismatch (%dx%d/%d)",FCT,dim1,dim2,S.len);
00262     if (!dim2) wblog(FL,
00263         "ERR %s() got empty index matrix (%dx%d)",FCT,dim1,dim2);
00264
00265     for (i=0; i<Ic.len; ++i) {
00266         if (Ic[i]>=S.len) wblog(FL,"ERR %s() index out of bounds "
00267             "%s; %d",FCT,STR(Ic+1),S.len);
00268         if ((++m[Ic[i]])>1) wblog(FL,"ERR %s() index not unique "
00269             "%s; %d",FCT,STR(Ic+1),S.len);
00270     };
00271 }
00272
00273 if ((--pos)>1) wblog(FL,
00274     "ERR %s() index out of founds (pos=%d)",FCT,pos);
00275
00276 if (dim2==2 && Ic.len==1) {
00277     if (int(Ic[0])==pos) { IJ=(*this); }
00278     else {
00279         IJ.init(dim1,dim2);
00280         size_t dl2=dim1*dim2; widx_t *d=IJ.data;
00281         for (i=0; i<dl2; i+=2) {
00282             d[i]=data[i+1];
00283             d[i+1]=data[i];
00284         }
00285     }
00286     return IJ;
00287 }
00288
00289 IJ.init(dim1,2); if (!dim1) return IJ;
00290
00291 wbindex Ik(S.len-Ic.len);
00292 unsigned j=0, k=0, lk=Ik.len-1, lc=Ic.len-1;
00293 const size_t *idx=data, *s=S.data;
00294 const widx_t *ic=Ic.data; widx_t *ik=Ik.data;
00295
00296 for (i=0; i<S.len; ++i) { if (!m[i]) { ik[j++]=i; }}
00297
00298 for (i=0; i<dim1; ++i, idx+=dim2, k+=2) {
00299     if (Ik.len) {
00300         widx_t &I=IJ.data[k+(1-pos)]; I=idx[ik[lk]];

```

```

00301         for (j=lk-1; j<lk; --j) { if (I) I*=s[ik[j]]; I+=idx[ik[j]]; }
00302     }
00303     if (Ic.len) {
00304         widx_t &J=IJ.data[k+pos]; J=idx[ic[lc]];
00305         for (j=lc-1; j<lc; --j) { if (J) J*=s[ic[j]]; J+=idx[ic[j]]; }
00306     }
00307 }
00308
00309 return IJ;
00310 };
00311
00312 template<>
00313 wbMatrix<double>& wbMatrix<double>::SkipTiny_float(double x) {
00314     Wb::chopTiny_float(data,dim1*dim2,x);
00315     return *this;
00316 }
00317
00318 template<>
00319 wbMatrix<wbcomplex>& wbMatrix<wbcomplex>::SkipTiny_float(wbcomplex x) {
00320     Wb::chopTiny_float((double*)data,2*dim1*dim2,x.r);
00321     return *this;
00322 }
00323
00324 template <class T> inline
00325 size_t wbMatrix<T>::maxRec(char lex, size_t *m_) const {
00326     if (!dim1 || !dim2) { wblog(FL,
00327         "WRN %s() got empty matrix (%d,%d)",FCT,dim1,dim2);
00328         return 0;
00329     }
00330
00331     size_t i=1, k=0, m=1; char c;
00332     for (; i<dim1; i++) {
00333         c=recCompare(k,i,-1,lex);
00334         if (c<0) { k=i; m=1; }
00335         else if (c==0) { m++; }
00336     }
00337
00338     if (m_) (*m_)=m;
00339     return k;
00340 };
00341
00342
00343 template <> inline
00344 size_t wbMatrix<double>::maxRec_float(
00345     char lex, size_t *m_, double xref
00346 ) const {
00347
00348     wbMatrix<double> M(*this);
00349     Wb::chopTiny_float(M.data,dim1*dim2,xref);
00350     return M.maxRec(lex,m_);
00351 };
00352
00353 template <class T> inline
00354 T wbMatrix<T>::recMax(size_t r, size_t *k) const {
00355
00356     if (r>=dim1) wblog(FL,
00357         "ERR %s() index out of bounds (%d/%d)",FCT,r,dim1);
00358     if (!dim2) { wblog(FL,
00359         "WRN %s() from empty matrix (%dx%d)",FCT,dim1,dim2);
00360         return 0;
00361     }
00362
00363     T *d=data+r*dim2, x=d[0]; if (k) (*k)=0;
00364
00365     for (size_t i=1; i<dim2; ++i) {
00366         if (x<d[i]) { x=d[i]; if (k) (*k)=i; }
00367     }
00368
00369     return x;
00370 };
00371
00372 template <class T> inline
00373 T wbMatrix<T>::colMax(size_t c, size_t *k) const {
00374
00375     if (c>=dim2) wblog(FL,
00376         "ERR %s() index out of bounds (%d/%d)",FCT,c,dim2);
00377     if (!dim1) { if (k) (*k)=0; wblog(FL,
00378         "WRN %s() from empty matrix (%dx%d)",FCT,dim1,dim2);
00379         return 0;
00380     }
00381
00382     T *d=data+c, x=d[0]; d+=dim2; if (k) (*k)=0;
00383
00384     for (size_t i=1; i<dim1; ++i, d+=dim2) {
00385         if (x<d[0]) { x=d[0]; if (k) (*k)=i; }
00386     }
00387

```

```

00388     return x;
00389 };
00390
00391 template <class T> inline
00392 double wbMatrix<T>::recMaxA(size_t r, size_t *k) const {
00393     if (k) (*k)=0;
00394
00395     if (r>=dim1) wblog(FL,
00396         "ERR %s() index out of bounds (%d/%d)",FCT,r,dim1);
00397     if (!dim2) { wblog(FL,
00398         "WRN %s() from empty matrix (%dx%d)",FCT,dim1,dim2);
00399         return 0;
00400     }
00401
00402     T *d=data+r*dim2;
00403     double a=ABS(d[0]), amax=a;
00404
00405     for (size_t i=1; i<dim2; i++) {
00406         a=ABS(d[i]); if (amax<a) { amax=a; if (k) (*k)=i; }
00407     }
00408
00409     return amax;
00410 };
00411
00412 template <class T>
00413 double wbMatrix<T>::maxRelDiff(const wbMatrix<T> &M) const {
00414
00415     size_t i, s=dim1*dim2;
00416     double maxdiff=0., maxval=0.;
00417
00418     if (!sameSize(M)) wblog(FL,
00419         "ERR %s() incompatible matrix objects",FCT);
00420
00421     for (i=0; i<s; i++) {
00422         maxdiff=MAX(maxdiff, fabs(double(M.data[i]-data[i])));
00423         maxval=MAX(maxval,
00424             MAX( fabs(double(data[i])), fabs(double(M.data[i])) ) );
00425     }
00426
00427     return ((maxval==0.) ? 0. : maxdiff/maxval);
00428 };
00429
00430 template <class T>
00431 T wbMatrix<T>::normDiff2(const wbMatrix<T> &M, size_t *k) const {
00432
00433     if (!sameSize(M)) wblog(FL,
00434         "ERR %s() incompatible matrix objects (%dx%d, %dx%d)",
00435         FCT,dim1,dim2,M.dim1,M.dim2);
00436     T fac=1;
00437
00438     return Wb::rangeNormDiff2(M.data, data, dim1*dim2, fac, k);
00439 };
00440
00441 template <class T>
00442 template <class T2>
00443 double wbMatrix<T>::normDiff(const wbMatrix<T2> &M) const {
00444
00445     if (!sameSize(M)) wblog(FL,
00446         "ERR %s() incompatible matrix objects (%dx%d, %dx%d)",
00447         FCT,dim1,dim2,M.dim1,M.dim2);
00448
00449     double x, x2=0;
00450     for (size_t n=numel(), i=0; i<n; ++i) {
00451         x=(data[i]-M.data[i]); x2+=(x*x);
00452     }
00453
00454     return sqrt(x2);
00455 };
00456
00457 template<class T>
00458 bool wbMatrix<T>::isOrthoRows(T *x, char tnorm, T eps) const {
00459
00460     if (!dim1 || !dim2 || dim1>dim2) return 0;
00461
00462     wbarray<T> E, A(*this,'r');
00463     Wb::MatProd(A,A,E, tnorm? 'T':'C');
00464
00465     if (x)
00466         return E.isProptoId(*x,eps);
00467     else return E.isDiagMatrix();
00468 };
00469
00470 template<class T>
00471 bool wbMatrix<T>::isOrthoRows(
00472     const wbMatrix<T> &B_, T *x, char tnorm, T eps) const {
00473
00474     if (dim1>dim2 || B_.dim1>B_.dim2) return 0;

```

```

00475     if (!(*this) && !B_) return 0;
00476
00477     wbarray<T> E, A(*this,'r'), B(B_,'r');
00478     Wb::MatProd(A,B,E, tnorm? 'T':'C');
00479
00480     if (x)
00481         return E.isProptoId(*x,eps);
00482     else return E.isDiagMatrix();
00483 };
00484
00485 template <class T> inline
00486 wbMatrix<T>& wbMatrix<T>::NormalizeCol(size_t k) {
00487
00488     if (k>=dim2) wblog(FL,"ERR %s() index out of bounds (%d/%d)",FCT,k,dim2);
00489     if (!dim1) return *this;
00490
00491     T *d=data+k, x2 = Wb::overlap(d,d,dim1,dim2);
00492     Wb::timesRange(d,1/Wb::sqrt(x2),dim1,dim2);
00493
00494     return *this;
00495 };
00496
00497 template <class T> inline
00498 wbMatrix<T>& wbMatrix<T>::set2Cols(const size_t i1, const size_t i2){
00499     const wbMatrix<T> M; save2(M);
00500     M.getCols(i1,i2,*this); return *this;
00501 };
00502
00503 template <class T> inline
00504 wbMatrix<T>& wbMatrix<T>::getCols(
00505     size_t j1, size_t j2,
00506     wbMatrix<T> &M
00507 ) const {
00508
00509     if (&M==this) {
00510         wbMatrix<T> X; getCols(j1,j2,X);
00511         return X.save2(M);
00512     }
00513
00514     if (j1==0 && j2+1==dim2) return M.init(*this);
00515
00516     if (j1>=dim2 || j2>=dim2) {
00517         if (int(j1-j2)==1 && (j1==0 || j1==dim2)) {
00518             return M.init(dim1,0);
00519         }
00520         else wblog(FL,
00521             "ERR index out of bounds (%dx%d: %d,%d)",dim1,dim2,j1,j2);
00522     }
00523     if (j1>j2) return M.init(dim1,0);
00524
00525     size_t i=0, m=j2-j1+1; const T *p=data+j1;
00526
00527     M.init(dim1,m);
00528
00529     for (; i<dim1; ++i)
00530         MEM_COPY<T>(M.data+i*m, m, p+i*dim2);
00531
00532     return M;
00533 };
00534
00535 template <class T> inline
00536 wbvector<T>& wbMatrix<T>::getCol(
00537     const size_t k, wbvector<T> &v
00538 ) const {
00539
00540     size_t i=0; const T *p=data+k;
00541
00542     if (k>=dim2) wblog(FL,
00543         "ERR %s() index out of bounds (%dx%d/%d)",FCT,dim1,dim2,k);
00544     v.init(dim1);
00545     for (; i<dim1; ++i, p+=dim2) { v[i]=p[0]; }
00546
00547     return v;
00548 };
00549
00550 template <class T>
00551 template <class TI> inline
00552 wbMatrix<T>& wbMatrix<T>::getCols(
00553     size_t n, const TI *I, wbMatrix<T> &M) const {
00554
00555     wbMatrix<T> X(dim1,n); T *d0=data, *d=X.data;
00556     size_t i=0, j;
00557
00558     for (j=0; j<n; ++j) if (I[j]>=dim2) wblog(FL,
00559         "ERR %s() index out of bounds (%d: %d/%d)",FCT,j,I[j],dim2);
00560
00561     for (; i<dim1; ++i, d+=n, d0+=dim2) {

```



```

00562     for (j=0; j<n; ++j) d[j]=d0[I[j]]; }
00563
00564     X.save2(M); return M;
00565 };
00566
00567 template <class T> inline
00568 wbvector<T>& wbMatrix<T>::getRec(
00569     const size_t j0, wbvector<T> &v,
00570     char ref
00571 ) const {
00572
00573     if (j0>=dim1) wblog(FL,
00574         "ERR %s() index out of bounds (%d/%d)",FCT,j0,dim1);
00575     return v.init(dim2,rec(j0),ref);
00576 };
00577
00578 template <class T>
00579 wbMatrix<T>& wbMatrix<T>::getRecs(
00580     const WBINDEX &I, wbMatrix<T> &M
00581 ) const {
00582
00583     size_t i=0; M.init(I.len,dim2);
00584     for (; i<I.len; ++i) {
00585         if (I[i]>=dim1) wblog(FL,"ERR index out of bounds (%d/%d)",I[i],dim1);
00586         MEM_CPY<T>(M.data+i*dim2, dim2, data+I[i]*dim2);
00587     }
00588     return M;
00589 };
00590
00591 template <class T>
00592 wbMatrix<T>& wbMatrix<T>::getRecs(
00593     size_t i1, size_t i2, wbMatrix<T> &M,
00594     size_t m
00595 ) const {
00596
00597     if (i1>=dim1 || i2>=dim1) wblog(FL,
00598         "ERR index out of bounds (%d,%d/%dx%d)",i1,i2,dim1,dim2);
00599
00600     if (m==0) {
00601         if (i1>i2) { M.init(0,dim2); return M; }
00602         M.init(i2-i1+1,dim2);
00603         MEM_CPY<T>(M.data, M.dim1*dim2, data+i1*dim2);
00604     }
00605     else {
00606         bool lflag=(int(m)<0); m=abs(int(m));
00607         if (m>dim2) wblog(FL,
00608             "ERR %s() m=%d/%d out of bounds",FCT,m,dim2);
00609
00610         M.init(i1<=i2 ? i2-i1+1 : 0, m);
00611         if (M.dim1) {
00612             const T* d0=data+i1*dim2; if (lflag) d0+=(dim2-m);
00613
00614             for (size_t i=0; i<M.dim1; ++i, d0+=dim2) {
00615                 MEM_CPY<T>(M.data+i*m, m, d0);
00616             }
00617         }
00618     }
00619
00620     return M;
00621 };
00622
00623 template <class T> inline
00624 wbMatrix<T>& wbMatrix<T>::getBlocks(
00625     size_t j1, size_t j2, const size_t D,
00626     wbMatrix<T> &M
00627 ) const {
00628
00629     size_t i,isnn, D2=2*D, s=D;
00630     T *p2,*p0;
00631
00632     if (D*(j1+1)>dim2 || D*(j2+1)>dim2 || (D ? dim2%D : 0)) wblog(FL,
00633         "ERR invalid block parameters (%d,%d,%d,%d).",j1, j2, D, dim2);
00634
00635     M.init(dim1,D2);
00636
00637     isnn=(j2==(j1+1));
00638     if (isnn) s+=s;
00639
00640     p0=data+D*j1; p2=M.data;
00641     for (i=0; i<dim1; i++) MEM_CPY<T>(p2+i*D2, s, p0+i*dim2);
00642
00643     if (!isnn) {
00644         p0=data+D*j2; p2=M.data+D;
00645         for (i=0; i<dim1; i++) MEM_CPY<T>(p2+i*D2, s, p0+i*dim2);
00646     }
00647
00648     return M;

```

```

00649 }
00650
00651 template <class T>
00652 void wbMatrix<T>::ColKron(
00653     const wbMatrix<T> &B,
00654     int d_
00655 ){
00656     if (&B==this) {
00657         wbMatrix<T> X(B); ColKron(X,d_);
00658         return;
00659     }
00660
00661     size_t i,j, da=dim1, d2=dim2, db=B.dim1;
00662     size_t d=( d_<0 ? B.dim2 : (size_t)d_);
00663
00664     if (d>B.dim2) wblog(F,
00665         "ERR %s() index out of bounds (%d/%d)", FCT,d,B.dim2);
00666
00667     if (B.isEmpty() || d==0) return;
00668     if (isEmpty()) { *this = B; return; }
00669
00670     Repmat(db,1, 0,B.dim2);
00671
00672     T *dd, *d0;
00673     for (i=0; i<db; ++i) {
00674         d0=B.data+i*B.dim2; dd = data + i*da*dim2 + d2;
00675         for (j=0; j<da; ++j, dd+=dim2) MEM_COPY<T>(dd,d,d0);
00676     }
00677 };
00678
00679 template <class T>
00680 wbMatrix<T>& wbMatrix<T>::repmat(
00681     size_t m, size_t n, wbMatrix<T> &Q,
00682     size_t pad1, size_t pad2
00683 ) const {
00684
00685     if (this==&Q) {
00686         wbMatrix<T> X(*this);
00687         return X.repmat(m,n,Q,pad1,pad2);
00688     }
00689
00690     size_t r,i,j;
00691     size_t d1=dim1+pad1, d2=dim2+pad2, D1=m*d1, D2=n*d2;
00692
00693     Q.init(D1,D2);
00694
00695     for (i=0; i<m; i++)
00696     for (j=0; j<n; j++) {
00697         T *d = Q.data + (i*n*d1+j)*d2;
00698         for (r=0; r<dim1; r++, d+=D2) MEM_COPY<T>(d, dim2, data+r*dim2);
00699     }
00700
00701     return Q;
00702 }
00703
00704 template <class T> inline
00705 wbMatrix<T>& wbMatrix<T>::getBlock(
00706     size_t j1, const size_t D, wbMatrix<T> &M
00707 ) const {
00708     T *d, *d0=data+D*j1;
00709
00710     if (D*(j1+1)>dim2 || (D ? dim2%D : 0)) wblog(F,
00711         "ERR %s() invalid block parameters (%d+%d/%d)",FCT,j1,D,dim2);
00712     M.init(dim1,D); d=M.data;
00713
00714     for (size_t i=0; i<dim1; i++, d+=D, d0+=dim2)
00715         MEM_COPY<T>(d,D,d0);
00716
00717     return M;
00718 };
00719
00720 template <class T> inline
00721 wbMatrix<T>& wbMatrix<T>::getBlock(
00722     size_t i, size_t j,
00723     size_t n, size_t m,
00724     wbMatrix<T> &M
00725 ) const {
00726
00727     if (i+n>dim1 || j+m>dim2) wblog(F,
00728         "ERR %s() index out of bounds (%d..%d/%d; %d..%d/%d)",
00729         i+1,i+n,dim1,j+1,j+m,dim2
00730     );
00731
00732     M.init(n,m); Wb::cpyStride(M.data,ref(i,j),m,n,-1,dim2);
00733     return M;
00734 };
00735

```

```

00736 template <class T>
00737 template <class T2>
00738 wbMatrix<T>& wbMatrix<T>::getBlock(
00739     size_t k,
00740     size_t D,
00741     const wbMatrix<T2>& R, size_t k2, size_t D2,
00742     wbMatrix<T> &M
00743 ) const {
00744
00745     size_t i,j;
00746     const T *d0; const T2 *d2; T *d;
00747
00748     if (D*(k+1)>dim2 || !D || dim2%D
00749         || D2*(k2+1)>R.dim2 || !D2 || R.dim2%D2) wblog(FL,
00750         "ERR invalid block setting (%d,%d/%d; %d,%d/%d).",
00751         k,D,dim2, k2,D2,R.dim2);
00752     if (dim1!=R.dim1) wblog(FL,
00753         "ERR %s() dimension mismatch (%d/%d)",FCT,dim1,R.dim1);
00754
00755     M.init(dim1,D+D2); d=M.data; d0=data+D*k; d2=R.data+D2*k2;
00756
00757     for (i=0; i<dim1; i++) {
00758         MEM_COPY<T>(d,D,d0); d0+=dim2; d+=D;
00759         for (j=0; j<D2; j++) d[j]=T(d2[j]);
00760         d2+=R.dim2; d+=D2;
00761     }
00762
00763     return M;
00764 };
00765
00766 template <class T> inline
00767 void wbMatrix<T>::setBlock(
00768     size_t k, const size_t D,
00769     const T* d0
00770 ){
00771     T *d = data+k*D;
00772
00773     if ((k+1)*D>dim2 || (D ? dim2%D : 0)) wblog(FL,
00774     "ERR invalid block parameters (%d,%d,%d).",k, D, dim2);
00775
00776     for (size_t i=0; i<dim1; i++, d+=dim2)
00777         MEM_COPY<T>(d,D,d0);
00778 };
00779
00780 template <class T> inline
00781 void wbMatrix<T>::SetBlock(
00782     size_t i0, size_t j0,
00783     const wbMatrix<T> &M
00784 ){
00785     if (i0+M.dim1>dim1 || j0+M.dim2>dim2) wblog(FL,"ERR %s() index "
00786     "out of bounds (%d/%d)",FCT,i0+1,M.dim1,dim1,j0+1,M.dim2,dim2);
00787     const T* d0=M.data; T* d=ref(i0,j0);
00788
00789     for (size_t i=0; i<M.dim1; i++, d+=dim2, d0+=M.dim2)
00790         MEM_COPY<T>(d,M.dim2,d0);
00791 };
00792
00793 template <class T>
00794 wbMatrix<T>& wbMatrix<T>::blockSum(size_t D, wbMatrix<T> &Q) const {
00795
00796     size_t i,r,rk;
00797     const T *q0; T *q;
00798
00799     if (&Q==this) wblog(FL,"ERR Output space same as input space!");
00800     if (!D || dim2%D) wblog(FL,"ERR Invalid block size (%d/%d)",D,dim2);
00801
00802     rk=dim2/D; if (rk==1) { Q=(*this); return Q; }
00803
00804     Q.init(dim1,D);
00805
00806     for (i=0; i<dim1; i++) {
00807         q0=rec(i); q=Q.rec(i);
00808         for (r=0; r<rk; r++, q0+=D) Wb::addRange(q0,q,D);
00809     }
00810
00811     return Q;
00812 }
00813
00814 template <class T> inline
00815 void wbMatrix<T>::blockSum(
00816     size_t j1,
00817     size_t j2,
00818     const size_t D,
00819     wbMatrix<double> &Q
00820 ) const {
00821
00822     size_t i,k;

```

```

00823     double *q1, *q2, *q;
00824
00825     if (D*(j1+1)>dim2 || D*(j2+1)>dim2 || (D ? dim2%D : 0)) wblog(FL,
00826 "ERR Invalid block parameters (%d,%d,%d,%d).",j1, j2, D, dim2);
00827
00828     Q.init(dim1,D);
00829     for (k=0; k<dim1; k++) {
00830         q=rec(k); q1=q+D*j1; q2=q+D*j2;
00831         q=Q.rec(k);
00832         for (i=0; i<D; i++) q[i]=q1[i]+q2[i];
00833     }
00834 }
00835
00836 template <class T> inline
00837 void wbMatrix<T>::blockSum(
00838     const wbindex &J,
00839     const size_t D,
00840     wbMatrix<double> &Q
00841 ) const {
00842
00843     size_t i,j,r=(D ? dim2/D : 0);
00844     double *q;
00845
00846     if (D ? dim2%D : 0) wblog(FL,"ERR Invalid D=%d (%d)", D, dim2);
00847     for (i=0; i<J.len; i++) if (J[i]>=r)
00848         wblog(FL,"ERR Block index out of range (%d; %d,%d)",J[i],D,dim2);
00849
00850     Q.init(dim1,D);
00851
00852     for (i=0; i<dim1; i++) {
00853         q=Q.rec(i);
00854         for (j=0; j<J.len; j++) Wb::addRange(Q.ref(i,J[j]*D), q, D);
00855     }
00856 }
00857
00858 template <class T> inline
00859 void wbMatrix<T>::recSet(size_t k,
00860     const T* a, const wbindex &ia,
00861     const T* b, const wbindex &ib
00862 ){
00863     if (ia.len+ib.len!=dim2) wblog(FL,"ERR %s() size mismatch "
00864 "%dx%d/ (%d: %d+%d)",FCT,dim1,dim2,k,ia.len,ib.len);
00865     if (k>=dim1) wblog(FL,
00866 "ERR %s() index out of bounds (%dx%d: %d)",FCT,dim1,dim2,k);
00867
00868     if (dim2) { size_t i,l=0; T *d=rec(k);
00869         for (i=0; i<ia.len; ++i, ++l) { d[l]=a[ia[i]]; }
00870         for (i=0; i<ib.len; ++i, ++l) { d[l]=b[ib[i]]; }
00871     }
00872 };
00873
00874 template <class T> inline
00875 void wbMatrix<T>::recSetB(
00876     size_t r, size_t j, size_t D,
00877     const T *q0, const T *q2, const T a
00878 ){
00879     T *q = data + r*dim2 + j*D;
00880
00881     if (r>=dim1 || (j+1)*D>dim2 || (D ? dim2%D : 0)) wblog(FL,
00882 "ERR setRecB() index out of bounds (%d/%d; %d/%d %d)",
00883     r,dim1, j,dim2, D);
00884
00885     if (q2==NULL)
00886         for (size_t i=0; i<D; i++) q[i]=q0[i];
00887     else if (a) {
00888         if (a==+1) for (size_t i=0; i<D; i++) q[i]=q0[i]+q2[i]; else
00889         if (a==-1) for (size_t i=0; i<D; i++) q[i]=q0[i]-q2[i];
00890         else for (size_t i=0; i<D; i++) q[i]=q0[i]+a*q2[i];
00891     }
00892 }
00893
00894 template <class T> inline
00895 void wbMatrix<T>::recSetB(
00896     size_t r, const wbindex &J, size_t D, const T *d0
00897 ){
00898     size_t i,j,k; const widxt *jp=J.data;
00899     T *d = data + r*dim2;
00900
00901     if (r>=dim1) wblog(FL,
00902 "ERR %s() index out of bounds (%d/%d)",FCT,r,dim1);
00903     if (J.len*D!=dim2) wblog(FL,
00904 "ERR %s() size mismatch ([%s] %d/%d)",FCT,STR(J+1),D,dim2);
00905
00906     if (D==1) {
00907         for (j=0; j<J.len; j++) d[j]=d0[jp[j]];
00908     }
00909     else {

```

```

00910         for (j=0; j<J.len; j++, d+=D) { k=jp[j]*D;
00911             for (i=0; i<D; i++) { d[i]=d0[k+i]; }
00912         }
00913     }
00914 }
00915
00916 template <class T> inline
00917 void wbMatrix<T>::recSet(size_t i1, size_t i2) {
00918
00919     if (i1>=dim1 || i2>=dim1) wblog(FL,
00920         "ERR index out of bounds (%d,%d; %d)", i1, i2, dim1);
00921
00922     if (i1!=i2)
00923         MEM_COPY<T>(data+i1*dim2, dim2, data+i2*dim2);
00924 }
00925
00926 template <class T> inline
00927 void wbMatrix<T>::recSet(size_t i, const wbvector<T> &v) {
00928
00929     if (i>=dim1) wblog(FL, "ERR %s() index out of bounds (%d/%d)", FCT, i, dim1);
00930     if (v.len!=dim2) wblog(FL, "ERR size mismatch (%d/%d)", v.len, dim2);
00931     MEM_COPY<T>(data+i*dim2, dim2, v.data);
00932 };
00933
00934 template <class T>
00935 template <class T2>
00936 void wbMatrix<T>::recSetT(size_t i, const wbvector<T2> &v) {
00937
00938     size_t j=0, n=(v.len<dim2 ? v.len : dim2); T *d = data + i*dim2;
00939
00940     if (i>=dim1) wblog(FL, "ERR %s() index out of bounds (%d/%d)", FCT, i, dim1);
00941     if (v.len>dim2) wblog(FL, "ERR size mismatch (%d/%d)", v.len, dim2);
00942
00943     for (; j<n; ++j) { d[j]=T(v.data[j]); }
00944     for (; j<dim2; ++j) { d[j]=T(0); }
00945 };
00946
00947 template <class T> inline
00948 void wbMatrix<T>::recSet (
00949     size_t i, const wbMatrix<T> &M, size_t j
00950 ){
00951     if (i>=dim1 || j>=M.dim1) wblog(FL,
00952         "ERR index out of bounds (%d/%d; %d/%d)", i, dim1, j, M.dim1);
00953     if (M.dim2!=dim2) wblog(FL,
00954         "ERR incompatible objects (%d/%d)", M.dim2, dim2);
00955
00956     MEM_COPY<T>(data+i*dim2, dim2, M.data+j*M.dim2);
00957 }
00958
00959 template <class T> inline
00960 void wbMatrix<T>::recSet (
00961     size_t i, const wbvector<T> &v1, const wbvector<T> &v2
00962 ){
00963     if (i>=dim1) wblog(FL,
00964         "ERR index out of bounds (%d/%d)", i, dim1);
00965     if (v1.len+v2.len!=dim2) wblog(FL,
00966         "ERR incompatible objects (%d+%d; %d)", v1.len, v2.len, dim2);
00967
00968     MEM_COPY<T>(data+i*dim2, dim2, v1.len, v1.data, v2.data);
00969 }
00970
00971 template <class T>
00972 void wbMatrix<T>::recSet(size_t i, const T* v, T x) {
00973     if (i>=dim1) wblog(FL,
00974         "ERR record index out of bounds (%d/%s)", i, SSTR_(this));
00975     T *d=data+i*dim2; i=0;
00976     if (d!=v) {
00977         if (x==T(+1)) { for (; i<dim2; ++i) d[i]= v[i]; } else
00978         if (x==T(-1)) { for (; i<dim2; ++i) d[i]=-v[i]; } else
00979         if (x==T( 0)) { for (; i<dim2; ++i) d[i]= T(0); }
00980         else          { for (; i<dim2; ++i) d[i]=x*v[i]; }
00981     }
00982 };
00983
00984 template <class T> inline
00985 void wbMatrix<T>::recSetP(size_t i1, const T* v, size_t n) {
00986     if (i1>=dim1) wblog(FL,
00987         "ERR record index out of bounds (%d/%d)", i1, dim1);
00988     if (long(n)<0) n=dim2; else if (n>dim2) wblog(FL,
00989         "ERR %s() rec-length out of bounds (%d/%d)", FCT, n, dim2);
00990     T *p=data+i1*dim2; if (p!=v) MEM_COPY<T>(p, n, v);
00991 };
00992
00993 template <class T> inline
00994 void wbMatrix<T>::recSetP(size_t i1,
00995     const T *v1, size_t n1,
00996     const T *v2, size_t n2

```

```

00997 ){
00998     if (i1>=dim1) {
00999         if (n1 || n2) wblog(FL,
01000             "ERR record index out of bounds (%d/%d)", i1, dim1);
01001         return;
01002     }
01003     if (n1+n2>dim2) wblog(FL,
01004         "ERR %s() size out of bounds (%d+%d/%d)", FCT, n1, n2, dim2);
01005
01006     size_t i; T *p=data+i1*dim2;
01007     for (i=0; i<n1; ++i) { p[i]=v1[i]; }; p+=n1;
01008     for (i=0; i<n2; ++i) { p[i]=v2[i]; };
01009 };
01010
01011 template <class T> inline
01012 void wbMatrix<T>::recSetP(size_t l,
01013     const T *v1, const windex &i1, const T *v2, const windex &i2
01014 ){
01015     if (l>=dim1) {
01016         if (i1.len || i2.len) wblog(FL,
01017             "ERR record index out of bounds (%d/%d)", l, dim1);
01018         return;
01019     }
01020     if (i1.len+i2.len>dim2) wblog(FL,
01021         "ERR %s() size out of bounds (%d+%d/%d)", FCT, i1.len, i2.len, dim2);
01022
01023     size_t i=0; T *x=data+l*dim2; if (i1.len) {
01024         for (; i<i1.len; ++i) { x[i]=v1[i1.data[i]]; }; x+=i1.len; i=0; }
01025         for (; i<i2.len; ++i) { x[i]=v2[i2.data[i]]; };
01026 };
01027
01028 template <class T> inline
01029 void wbMatrix<T>::recAddP(size_t i1, const T* v) {
01030     T *d = data+i1*dim2;
01031     for (size_t i=0; i<dim2; i++) d[i]+=v[i];
01032 }
01033
01034 template <class T>
01035 template<class T2> inline
01036 void wbMatrix<T>::setCol(size_t k, const wbvector<T2> &v) {
01037     if (v.len!=dim1) wblog(FL,
01038         "ERR %s() size mismatch (%d/%d)", FCT, v.len, dim1);
01039     setCol(k, v.data);
01040 };
01041
01042 template <class T>
01043 template<class T2> inline
01044 void wbMatrix<T>::setCol(size_t k, const T2 *v, T fac, size_t stride) {
01045
01046     if (k>=dim2) {
01047         if (dim2) wblog(FL,
01048             "ERR %s() index out of bounds (%d/%d)", FCT, k, dim2);
01049         else wblog(FL,
01050             "ERR %s() got empty matrix (%dx%d/%d)", FCT, dim1, dim2, k);
01051     }
01052
01053     T *d=data+k; size_t i=0;
01054     if (fac==1)
01055         { for (; i<dim1; ++i, d+=dim2, v+=stride) { (*d)= T(*v); }}
01056     else if (fac==-1)
01057         { for (; i<dim1; ++i, d+=dim2, v+=stride) { (*d)=-T(*v); }}
01058     else { for (; i<dim1; ++i, d+=dim2, v+=stride) { (*d)=fac*T(*v); }}
01059 };
01060
01061 template <class T> inline
01062 void wbMatrix<T>::setCol(size_t k, const T &x) {
01063
01064     if (k>=dim2) {
01065         if (dim2) wblog(FL,
01066             "ERR %s() index out of bounds (%d/%d)", FCT, k, dim2);
01067         else wblog(FL,
01068             "ERR %s() got empty matrix (%dx%d/%d)", FCT, dim1, dim2, k);
01069     }
01070
01071     T *d=data+k;
01072     for (size_t i=0; i<dim1; ++i, d+=dim2) (*d)=x;
01073 };
01074
01075 template <class T>
01076 template <class T2> inline
01077 void wbMatrix<T>::setColT (size_t k, const wbvector<T2> &v) {
01078     if (v.len!=dim1) wblog(FL,
01079         "ERR %s() size mismatch (%d/%d)", FCT, v.len, dim1);
01080     setColT(k, v.data);
01081 };
01082
01083 template <class T>

```

```

01084 template <class T2> inline
01085 void wbMatrix<T>::setColT(size_t k, const T2 *v, T fac, size_t stride) {
01086
01087     T *d=data+k; size_t i=0;
01088     if (k>=dim2) wblog(FL,
01089         "ERR %s() index out of bounds (%d/%d)",FCT,k,dim2);
01090
01091     if (fac==1)
01092         { for (; i<dim1; ++i, d+=dim2, v+=stride) { (*d)= T(*v); }}
01093     else if (fac==-1)
01094         { for (; i<dim1; ++i, d+=dim2, v+=stride) { (*d)=-T(*v); }}
01095     else { for (; i<dim1; ++i, d+=dim2, v+=stride) { (*d)=fac*T(*v); }}
01096 };
01097
01098 template <class T> inline
01099 wbMatrix<T>& wbMatrix<T>::colTimes(size_t i, T x) {
01100
01101     if (i>=dim2) wblog(FL,
01102         "ERR %s() index out of bounds (%s/%d)",FCT,SSTR_(this),i);
01103     if (x!=T(1)) { T *d=data+i; i=0;
01104         if (x==T( 0)) { for (; i<dim1; ++i, d+=dim2) d[0]= T(0); } else
01105         if (x==T(-1)) { for (; i<dim1; ++i, d+=dim2) d[0]=-d[0]; }
01106         else { for (; i<dim1; ++i, d+=dim2) d[0]*=x; }
01107     }
01108     return *this;
01109 };
01110
01111 template <class T> inline
01112 wbMatrix<T>& wbMatrix<T>::rowTimes(size_t i, T x) {
01113
01114     if (i>=dim1) wblog(FL,
01115         "ERR %s() index out of bounds (%d /%s",FCT,i,SSTR_(this));
01116     if (x!=T(1)) { T *d=data+i*dim2; i=0;
01117         if (x==T( 0)) { for (; i<dim2; ++i) d[i]= T(0); } else
01118         if (x==T(-1)) { for (; i<dim2; ++i) d[i]=-d[i]; }
01119         else { for (; i<dim2; ++i) d[i]*=x; }
01120     }
01121     return *this;
01122 };
01123
01124 template <class T>
01125 size_t wbMatrix<T>::skipNanRecs(wbindex &I QS_UNUSED_VAR) {
01126     wblog(FL,"WRN %s() irrelevant for type <%s>",FCT,TSTR(T));
01127     return 0;
01128 }
01129
01130 template <>
01131 size_t wbMatrix<double>::skipNanRecs(wbindex &I) {
01132
01133     size_t i,j,k=0,skipped=0;
01134     double *d=data;
01135     I.init(dim1);
01136
01137     for (i=0; i<dim1; i++, d+=dim2) {
01138         for (j=0; j<dim2; j++) if (std::isnan(d[j])) break;
01139         if (j<dim2) { skipped++; continue; }
01140
01141         if (k<i) recSet(k,i);
01142         I[k++]=i;
01143     }
01144
01145     if (k==0)
01146         { Resize(0,dim2); I.init(); }
01147     else { dim1=k; I.len=k; }
01148
01149     return skipped;
01150 };
01151
01152 template <class T> inline
01153 wbMatrix<T>& wbMatrix<T>::recPermute(
01154     const wbperm &P, wbMatrix<T> &M, char iflag
01155 ) const {
01156
01157     if (P.len!=dim1 || !P.isValidPerm()) wblog(FL,
01158         "ERR %s() invalid permutation [%s; %d]",FCT,STR(P),dim1);
01159     if (&M==this) wblog(FL,"ERR %s() got same object!",FCT);
01160
01161     if (M.dim1!=dim1 || M.dim2!=dim2) {
01162         M.init(dim1,dim2);
01163     }
01164
01165     if (iflag) {
01166         const T *d=data;
01167         for (size_t i=0; i<P.len; ++i, d+=dim2)
01168             MEM_COPY<T>(M.data+P[i]*dim2, dim2, d);
01169     }
01170     else {

```

```

01171         T *d=M.data;
01172         for (size_t i=0; i<P.len; ++i, d+=dim2)
01173             MEM_COPY<T>(d, dim2, data+P[i]*dim2);
01174     }
01175
01176     return M;
01177 };
01178
01179 template <class T> inline
01180 wbMatrix<T>& wbMatrix<T>::colPermute(
01181     const wbperm &P, wbMatrix<T> &M,
01182     char iflag
01183 ) const {
01184
01185     size_t i,j;
01186     const wperm_t *p=P.data;
01187     const T* d0; T* d2;
01188
01189     if (!P.isValidPerm(0,0,dim2)) wblog(FL,
01190         "ERR %s() invalid permutation [%s; %d]", FCT,STR(P), dim2);
01191     M.init(dim1,dim2); d2=M.data; d0=data;
01192
01193     if (iflag) {
01194         for (i=0; i<dim1; i++, d0+=dim2, d2+=dim2)
01195             for (j=0; j<dim2; j++) d2[p[j]]=d0[j];
01196     }
01197     else {
01198         for (i=0; i<dim1; i++, d0+=dim2, d2+=dim2)
01199             for (j=0; j<dim2; j++) d2[j]=d0[p[j]];
01200     }
01201
01202     return M;
01203 };
01204
01205 template <class T> inline
01206 void wbMatrix<T>::blockPermute(const wbperm &P, wbMatrix<T> &M
01207 ) const {
01208
01209     if (this==&M) {
01210         wbMatrix<T> X(*this); X.blockPermute(P,M);
01211         return;
01212     }
01213
01214     size_t i,j,D; const T *d0=data; T *d;
01215
01216     if (!P.len || !P.isValidPerm()) wblog(FL,
01217         "ERR invalid permutation (%d;%d)", P.len, dim2);
01218     if (dim2%P.len) wblog(FL,
01219         "ERR block dimension mismatch ( %d = %d*?? )", dim2, P.len);
01220
01221     M.init(dim1,dim2); d=M.data; D=dim2/P.len;
01222
01223     for (i=0; i<dim1; i++, d0+=dim2)
01224         for (j=0; j<P.len; j++, d+=D) MEM_COPY<T>(d, D, d0+D*P[j]);
01225 }
01226
01227 template <class T> inline
01228 wbMatrix<T>& wbMatrix<T>::cols2Front(
01229     const WBINDEX &I1, wbMatrix<T> &M
01230 ) const {
01231
01232     wbperm P; P.init2Front(I1,dim2);
01233     return colPermute(P,M);
01234 };
01235
01236 template <class T> inline
01237 wbMatrix<T>& wbMatrix<T>::cols2End(
01238     const WBINDEX &I1, wbMatrix<T> &M
01239 ) const {
01240
01241     wbperm P; P.init2End(I1,dim2);
01242     return colPermute(P,M);
01243 };
01244
01245 template <class T>
01246 WBIDXMAT& wbMatrix<T>::toBlockIndex(
01247     widx_t D, WBIDXMAT &II_,
01248     wbvector< wbMatrix<T> > *QQ_
01249 ) const {
01250
01251     if (!D || dim2%D)
01252         wblog(FL,"ERR %s() invalid blocksize (%d/%d)",dim2,D);
01253
01254     size_t i,m=dim2/D;
01255
01256     wbvector< wbMatrix<T> > QQ;
01257     wbvector< wbindex > II;

```



```

01258     WBINDEX dd;
01259     wbperm P;
01260
01261     wbvector< WBINDEX const* > Ip;
01262
01263     QQ.initDef(m);
01264     II.initDef(m); Ip.init(m);
01265
01266     for (i=0; i<m; i++) {
01267         getBlock(i,D,QQ[i]).groupRecs(P,dd);
01268         II[i].BlockIndex(dd).Permute(P,'i');
01269         Ip[i]=(&II[i]);
01270     }
01271
01272     II_.CAT(2,Ip);
01273     if (QQ_) QQ.save2(*QQ_);
01274
01275     return II_;
01276 };
01277
01278 template <class T>
01279 template <class T2>
01280 void wbMatrix<T>::toBlockIndex(widx_t D,
01281     wbMatrix<T2> &R, widx_t DR_,
01282     WBIDXMAT *IB_,
01283     WBIDXMAT *I2_,
01284     WBIDXMAT *SS_,
01285     wbvector< wbMatrix<T> > *QQ_
01286 ) const {
01287
01288     if (dim1!=R.dim1) wblog(FL,
01289         "ERR %s() size mismatch (%d/%d)",FCT,dim1,R.dim1);
01290     if (!D || dim2%D) wblog(FL,
01291         "ERR %s() invalid blocksize (%d/%d)",FCT,dim2,D);
01292
01293     if (dim1==0 || dim2==0) {
01294         if (IB_) { IB_>init(); }
01295         if (I2_) { I2_>init(); }
01296         if (SS_) { SS_>init(); }
01297         if (QQ_) { QQ_>init(); }; return;
01298     }
01299
01300     size_t i,DR, m=dim2/D;
01301
01302     wbvector< wbMatrix<T> > QQ;
01303     wbvector< WBINDEX > IB,I2,SB;
01304     WBINDEX dd;
01305     wbMatrix<T2> Ri;
01306     wbperm pp;
01307
01308     if (int(DR_)<=0) DR=R.dim2/m; else DR=DR_;
01309     if (m*DR_!=R.dim2) wblog(FL,
01310         "ERR block size mismatch (expecting %d*d = %d / %d)",
01311         m,DR_,m*DR_,R.dim2
01312     );
01313
01314     QQ.init(m);
01315     IB.init(m); I2.init(m); SB.init(m);
01316
01317     for (i=0; i<m; i++) {
01318         getBlock(i,D, R,i,DR, QQ[i])
01319         .groupRecs(pp,dd, D,
01320             IB[i], I2[i], SB[i],
01321             'p'
01322         );
01323     }
01324
01325
01326     if (IB_) { IB_>CAT(2,IB); }
01327     if (I2_) { I2_>CAT(2,I2); }
01328     if (SS_) { SS_>CAT(2,SB); }; if (QQ_) QQ.save2(*QQ_);
01329 };
01330
01331 template <class T>
01332 void wbMatrix<T>::recPrint(
01333     size_t k, const char *istr0, char mflag) const {
01334
01335     size_t l=0, n=64; char s[n];
01336     wbvector<T> d;
01337
01338     if (k>=dim1) wblog(FL,
01339         "ERR %s() index out of bounds (%d/%d)",FCT,k,dim1);
01340
01341     l=snprintf(s,n,"%32s.rec(%ld)", istr0, k);
01342     if (l>=n) wblog(FL,"ERR %s() string out of bounds (%ld/%ld)",FCT,l,n);
01343
01344     d.init(dim2, (*this).rec(k));

```

```

01345     d.print(s,mflag);
01346
01347     return;
01348 };
01349
01350 template <class T>
01351 wbstring wbMatrix<T>::rec2Str(
01352     size_t k,
01353     const char *fmt,
01354     const char *sep,
01355     size_t stride,
01356     const char *sep2
01357 ) const {
01358
01359     if (k>=dim1) wblog(FL,
01360         "ERR %s() index out of bounds (%d/%d)",FCT,k,dim1);
01361     if (!dim2) return "";
01362
01363     wbvector<T> d; d.init2ref(dim2, data+k*dim2);
01364     return d.toStrf(fmt,sep,stride,sep2);
01365 };
01366
01367 template <class T>
01368 wbstring wbMatrix<T>::toStr(
01369     const char *fmt0, const char *sep, const char *rsep,
01370     size_t stride, const char *sep2
01371 ) const {
01372
01373     size_t j, i=16+dim1*dim2*16;
01374     wbstring s,fmt;
01375
01376     if (isComplex()) { i*=2; }
01377     if (i<128) { i=128; }
01378     s.init(i);
01379
01380     if (fmt0 && fmt0[0])
01381         { fmt=fmt0; }
01382     else { fmt.init2Fmt((T)0); }
01383
01384     for (i=0; i<dim1; ++i) { if (i) s.push(FL,rsep);
01385     for (j=0; j<dim2; ++j) { if (j) s.push(FL,sep);
01386         s.pushf(FL,fmt.data, data[i*dim2+j]);
01387         if (stride && ((j+1)%stride)==0 && j+1<dim2) {
01388             s.push(FL,sep2);
01389         }
01390     }}
01391
01392     return s;
01393 };
01394
01395 template <>
01396 wbstring wbMatrix<wbcomplex>::toStr(
01397     const char *fmt0, const char *sep, const char *rsep,
01398     size_t stride, const char *sep2
01399 ) const {
01400
01401     wbstring s;
01402     unsigned i,j; char fmt[16], zfmt[8]="%s", s1[32];
01403
01404     snprintf(fmt,16,"%s",fmt0 && fmt0[0] ? fmt0 : "%.4g");
01405
01406     for (i=0; fmt[i]; ++i) { if (fmt[i]!='%') {
01407         if (isdigit(fmt[++i])) { unsigned l=fmt[i]-'0';
01408             for (j=i+1; fmt[j]; ++j) {
01409                 if (isdigit(fmt[j])) { l=10*l+(fmt[j]-'0'); } else break;
01410             }
01411             snprintf(zfmt,8,"%%%ds ",l);
01412         }
01413         break;
01414     }}
01415
01416     s.init(32+(4+i)*dim1*dim2);
01417
01418     for (i=0; i<dim1; ++i) { if (i) s.push(FL,rsep);
01419     for (j=0; j<dim2; ++j) { if (j) s.push(FL,sep);
01420         snprintf(s1,32,"%s",(*this)(i,j).toStr(fmt).data);
01421         s.pushf(FL,zfmt,s1);
01422         if (stride && ((j+1)%stride)==0 && j+1<dim2) {
01423             s.push(FL,sep2);
01424         }
01425     }}
01426
01427     return s;
01428 };
01429
01430 template <class T>
01431 void wbMatrix<T>::toMxStruct(mxArray* S, const char *vname) const {

```

```

01432
01433     mxArray *a=Mx::Array<double>(dim1,dim2,'t').copyFromTR(data);
01434
01435     int fid = mxAddField2Scalar(FL,S,vname);
01436     mxSetFieldByNumber(S,0,fid, a);
01437 };
01438
01439 template <class T>
01440 void wbMatrix<T>::mat2mxs(mxArray* S, char fid) const {
01441     int n;
01442
01443     if (!S) wblog(FL,"ERR %s() got null",FCT);
01444     if (!mxIsStruct(S)) wblog(FL,
01445         "ERR %s() got non-structure %s",FCT,mxGetClassName(S));
01446     if ((n=mxGetNumberOfElements(S))!=1) wblog(FL,
01447         "ERR %s() single struct required (%d)",FCT,n);
01448     if (fid>=(n=mxGetNumberOfFields(S))) wblog(FL,
01449         "ERR %s() field index out of bounds (%d/%d)",FCT,fid,n);
01450
01451     mxArray *a=Mx::Array<double>(dim2,dim1).copyFromTR(data);
01452     mxSetFieldByNumber(S,0,fid, a);
01453 };
01454
01455 #ifndef QS_USING_OMP
01456
01457 template <class T>
01458 void getSortPerm_OMP(
01459     const wbMatrix<T> &A, wbperm &P, char dir, char lex
01460 ){
01461     if (!A.dim2) { P.init(); return; }
01462
01463     wbRecs<T> R(A,P,dir,lex);
01464
01465     #ifdef WB_CLK_SPARSE
01466         Wb::Clock clk("mat:sort:recs-omp",0);
01467     #endif
01468
01469     if (A.dim1<128) {
01470         sort(P.data, P.data+A.dim1, R);
01471         return;
01472     }
01473
01474     int id=0, i, l;
01475     int mp=MIN(
01476         1 << unsigned(floor(log2(double(A.dim1)))-6),
01477         omp_get_max_threads()
01478     );
01479
01480     double nsub=double(A.dim1)/mp;
01481
01482     wbindex idx(mp+1); wbperm PX(P.len);
01483     for (i=0; i<mp; ++i) { idx[i]=size_t(i*nsub+0.5); }
01484     idx[i]=A.dim1;
01485
01486     #pragma omp parallel for
01487     for (i=0; i<mp; ++i) {
01488         size_t i1=idx.data[i], i2=idx.data[i+1];
01489         sort(P.data+i1, P.data+i2, R);
01490         id=MAX(id,omp_get_thread_num());
01491     }
01492
01493     while (idx.len>2) { int m2=mp/2; P.swap(PX);
01494
01495     #pragma omp parallel for
01496         for (i=0; i<m2; ++i) {
01497             size_t k=2*i, i1=idx.data[k], i2=idx.data[k+1], i3=idx.data[k+2];
01498             merge(
01499                 PX.data+i1, PX.data+i2,
01500                 PX.data+i2, PX.data+i3, P.data+i1, R
01501             );
01502             id=MAX(id,omp_get_thread_num());
01503         }
01504
01505         if (mp%2) {
01506             size_t i1=idx.data[mp-3], i2=idx.data[mp-1], i3=idx.data[mp];
01507             memcpy(PX.data+i1,P.data+i1,(i2-i1)*sizeof(size_t));
01508             merge(
01509                 PX.data+i1, PX.data+i2,
01510                 PX.data+i2, PX.data+i3, P.data+i1, R
01511             );
01512             idx[mp-1]=idx[mp]; idx.len=(mp--);
01513         }
01514
01515         for (l=1, i=2; i<mp; i+=2, ++l) { idx[l]=idx[i]; }
01516         idx[l]=idx[mp]; mp=l; idx.len=l+1;
01517     }
01518

```

```

01519 };
01520
01521 #endif
01522
01523 template <class T> inline
01524 wbMatrix<T>& wbMatrix<T>::SortRecs(
01525     wbpem &P, char dir, char lex
01526 ){
01527     if (!dim1) { P.init(); return *this; }
01528     static int use_omp=-1;
01529
01530 #ifdef LD_CLEBSCH_QS
01531     if (use_omp<0) {
01532         use_omp=(
01533             sp_num_threads > 0 ? 1 : 0
01534         );
01535
01536         if (use_omp && CG_VERBOSE>6)
01537             wblog(FL,"OMP %s() using openMP on top of STL",FCT);
01538     }
01539
01540     char isLarge=(
01541         (dim1>(1<<30) && CG_VERBOSE>5) ||
01542         (dim1>(1<<28) && CG_VERBOSE>8) );
01543
01544     if (isLarge) {
01545         if (use_omp>0) sprintf_str(
01546             "(parallel mode @ %d; %d)",omp_get_max_threads(),use_omp);
01547         else strcpy (str,"(serial mode)");
01548         wblog(FL,"TST %s() got %.3g x %d entries %s",
01549             FCT,double(dim1),int(dim2),str);
01550     }
01551 #else
01552     if (use_omp<0) use_omp=0;
01553 #endif
01554
01555     if (use_omp<=0 || dim1<128) {
01556         Wb::hpsort(data,dim2,dim1, P,dir,lex);
01557         return *this;
01558     }
01559     else {
01560
01561         #ifdef QS_USING_OMP
01562             getSortPerm_OMP(*this,P,dir,lex);
01563         #else
01564             wblog(FL,"ERR %s()",FCT);
01565         #endif
01566
01567         return recPermute(P);
01568     }
01569
01570 #ifdef LD_CLEBSCH_QS
01571     if (isLarge) wblog(FL,"TST %s() done",FCT);
01572 #endif
01573     return *this;
01574 };
01575
01576 template <class T>
01577 template <class T2>
01578 void wbMatrix<T>::groupRecs_rdeg(
01579     wbpem &P, WBINDEX &D, const wbMatrix<T2> &R,
01580     wbvector<widxt> *Ib,
01581     wbvector<widxt> *I2,
01582     wbvector<widxt> *Sb,
01583     char iflag
01584 ){
01585     if (dim1==0) { P.init(); D.init(); return; }
01586
01587     SortRecs(P);
01588     groupSortedRecs(D);
01589
01590     if (R.dim1!=P.len) wblog(FL,
01591         "ERR %s() dimension mismatch (%d/%d)",FCT,R.dim1,P.len);
01592     if (R.dim2==0) { wblog(FL,
01593         "WRN %s() got empty reference space",FCT); return;
01594     }
01595
01596     size_t i,j,d,l=0, m=D.len;
01597     WBINDEX dd,v, I(D.max());
01598     wbvector< WBINDEX > SS;
01599     wbvector< wbindex > II;
01600     wbMatrix<T2> Q;
01601     wbpem pp;
01602
01603     if (I2 || Sb) { II.init(m); if (Sb) SS.init(m); }
01604
01605     for (i=0; i<m; ++i, l+=d) { d=D[i];

```

```

01606         if (d>1) {
01607             I.len=d; for (j=0; j<d; j++) I[j]=P[l+j];
01608             R.getRecs(I,Q).groupRecs(pp,dd);
01609             for (j=0; j<d; j++) P[l+j]=I[pp[j]];
01610
01611             if (I2 || Sb) {
01612                 II[i].BlockIndex(dd); if (Sb)
01613                     SS[i].init(pp.len).set(dd.len);
01614             }
01615         }
01616         else {
01617             if (d!=1) wblog(FL,"ERR d=%g !?",d);
01618             if (I2 || Sb) {
01619                 II[i].init(1).set(0); if (Sb)
01620                     SS[i].init(1).set(1);
01621             }
01622             continue;
01623         }
01624     }
01625
01626     if (Ib) {
01627         wbindex J; J.BlockIndex(D); J.save2(*Ib);
01628         if (Ib->len!=P.len) wblog(FL,"ERR size mismatch %d/%d",Ib->len,P.len);
01629         if (iflag) Ib->Permute(P,iflag);
01630     }
01631
01632     if (I2) {
01633         wbvector< WBINDEX* > J(II.len);
01634         for (i=0; i<II.len; ++i) { J[i]=(WBINDEX*)&II[i]; }; I2->Cat(J);
01635         if (I2->len!=P.len) wblog(FL,"ERR size mismatch %d/%d",I2->len,P.len);
01636         if (iflag) I2->Permute(P,iflag);
01637     }
01638
01639     if (Sb) {
01640         Sb->Cat(SS);
01641         if (Sb->len!=P.len) wblog(FL,"ERR size mismatch %d/%d",Sb->len,P.len);
01642         if (iflag) Sb->Permute(P,iflag);
01643     }
01644 };
01645
01646 template <class T>
01647 void wbMatrix<T>::groupRecs(
01648     wbpem &P, WBINDEX &D, size_t nc,
01649     WBINDEX &Ib,
01650     WBINDEX &I2,
01651     WBINDEX &Sb,
01652     char iflag
01653 ){
01654     if (dim1==0) { P.init(); D.init(); return; }
01655
01656     SortRecs(P);
01657
01658     if (nc>dim2) wblog(FL,"ERR dimension out of bounds (%d/%d)",nc,dim2);
01659     groupSortedRecs(D,nc,Ib,I2,Sb);
01660
01661     if (iflag) {
01662         Ib.Permute(P,iflag);
01663         I2.Permute(P,iflag);
01664         Sb.Permute(P,iflag);
01665     }
01666 };
01667
01668 template <class T>
01669 void wbMatrix<T>::groupRecs(
01670     wbpem &P, WBINDEX &D,
01671     size_t m,
01672     char lex,
01673     wbindex *Ig,
01674     wbMatrix<T> *X
01675 ){
01676     if (!dim1) { P.init(); D.init(); return; }
01677     if (!dim2) {
01678         P.init(dim1); D.init(1); D[0]=dim1;
01679         if (Ig) { Ig->init(dim1).set(0); }
01680
01681         if (int(m)<0) dim1=1;
01682         else if (m) wblog(FL,
01683             "ERR %s() m=%d out of bounds (%dx%d)",FCT,m,dim1,dim2);
01684         return;
01685     }
01686
01687     #ifdef WB_CLK_SPARSE
01688         Wb::Clock clk("mat:sort:recs",0);
01689     #endif
01690
01691     if (X) {
01692         if (dim1>1) { X->init(*this); } else { X=NULL; }

```

```

01693     }
01694
01695     if (!isSorted(+l,lex))
01696     { SortRecs(P,+l,lex); }
01697     else { P.init(dim1); }
01698
01699 #ifdef WB_CLK_SPARSE
01700     clk.Switch("mat:group:recs");
01701 #endif
01702
01703     if (int(m)<0) { groupSortedRecs(D, 0,-1,lex); }
01704     else if (m) { groupSortedRecs(D,'k',m,lex); }
01705     else { D.init(1); D[0]=dim1; }
01706
01707     if (X && (dim1==X->dim1)) { X->save2(*this);
01708         P.init(dim1);
01709         D.init2val(dim1,1);
01710     }
01711
01712     if (Ig) {
01713         size_t i,j,l=0,d=0; widx_t *ig; wperm_t *p=P.data;
01714         Ig->init(P.len); ig=Ig->data;
01715         for (i=0; i<D.len; ++i, l+=d) { d=D.data[i];
01716             for (j=0; j<d; ++j) ig[p[l+j]]=i; }
01717     }
01718 };
01719
01720 template <class T>
01721 void wbMatrix<T>::groupRecs(
01722     wperm &P, WBINDEX &D,
01723     const WBINDEX *I, char lex
01724 ){
01725     if (dim1==0) { P.init(); D.init(); return; }
01726
01727     SortRecs(P,+l,lex);
01728
01729     if (!I) { groupSortedRecs(D,0,-1,lex); }
01730     else {
01731         wbMatrix<T> X; this->cols2Front(*I,X);
01732         X.groupSortedRecs(D,'k',I->len,lex);
01733     }
01734 };
01735
01736 template <class T>
01737 void wbMatrix<T>::groupSortedRecs(
01738     WBINDEX &d,
01739     char keepall,
01740     size_t m,
01741     char lex
01742 ){
01743     size_t i,ig,n=dim1;
01744     char c, cref=0;
01745
01746     if (int(m)<0) { m=dim2; }
01747     else if (m>dim2) wblog(FL,"ERR number out of bounds (%d/%d)",m,dim2);
01748     if (!dim2) {
01749         if (!dim1) wblog(FL,
01750             "WRN %s() for %dx%d matrix !?",FCT,dim1,dim2);
01751         d.init(1); d[0]=dim1; init(); return;
01752     }
01753     else if (m==0) {
01754         if (m<dim2 && !keepall) wblog(FL,
01755             "WRN %s() keeping all since (m=%d)<%d",FCT,m,dim2);
01756         d.init(1); d[0]=dim1; return;
01757     }
01758
01759     d.init(n); if (n==0) return;
01760
01761     ig=0; d[ig]++;
01762     for (i=1; i<n; i++) {
01763         c=recCompare(i,i-1,m,lex);
01764         if (c) {
01765             if ((++ig)!=i) { if (!keepall)
01766                 MEM_COPY<T>(data+ig*dim2, dim2, data+i*dim2); }
01767
01768             if (c!=cref) {
01769                 if (cref) { MXPut(FL,"a").add(*this,"M").add(m,"m");
01770                     wblog(FL,"ERR recs not sorted %d/%d (%d, m=%d/%d)",
01771                         c,cref,i,m,dim2);
01772                 }
01773                 else cref=c;
01774             }
01775         }
01776         d[ig]++;
01777     }
01778 }
01779

```

```

01780     d.Resize(ig+1);
01781     if (!keepall) Resize(ig+1,dim2);
01782 };
01783
01784 template <class T>
01785 void wbMatrix<T>::groupSortedRecs(
01786     WBINDEX &d,
01787     size_t m,
01788     char lex,
01789     WBINDEX &Ib,
01790     WBINDEX &I2,
01791     WBINDEX &Sb
01792 ){
01793     size_t i,j,i2,i0=0, ig=0;
01794     char c,c2, cref=0;
01795
01796     if (int(m)<0) { m=dim2; }
01797     else if ((!m && dim2) || m>dim2) wblog(FL,
01798         "ERR number out of bounds (%d/%d)",m,dim2);
01799
01800     if (!dim1 || !dim2) {
01801         d.init(); init(); Ib.init(); I2.init(); Sb.init();
01802         return;
01803     }
01804
01805     Ib.init(dim1); d.init(dim1);
01806     I2.init(dim1);
01807     Sb.init(dim1);
01808
01809     for (d[ig]++, i=1; i<=dim1; i++, d[ig]++) {
01810         if (i<dim1)
01811             c=recCompare(i,i-1,m,lex);
01812         else c=99;
01813
01814         if (c) {
01815             for (i2=0, j=i0+1; j<i; j++) {
01816                 c2=recCompare(j,j-1,-1,lex); if (c2) { i2++;
01817                     if (c2!=cref) {
01818                         if (cref) wblog(FL,
01819                             "ERR input recs not sorted (%d: %d/%d).",i,c2,cref);
01820                         else cref=c2;
01821                     }
01822                 }
01823                 I2[j]=i2;
01824             }
01825             for (i2++, j=i0; j<i; j++) { Ib[j]=ig; Sb[j]=i2; }
01826             i0=i; if (i>=dim1) break;
01827
01828             if (c!=cref) {
01829                 if (cref) wblog(FL,
01830                     "ERR input recs not sorted (%d: %d/%d).",i,c,cref);
01831                 else cref=c;
01832             }
01833
01834             if ((++ig)!=i)
01835                 MEM_COPY<T>(data+ig*dim2, dim2, data+i*dim2);
01836         }
01837     }
01838
01839     d.len=ig+1;
01840     dim1=ig+1;
01841 };
01842
01843 template <> inline
01844 wbMatrix<double>& wbMatrix<double>::sortRecs_float(wbperm &P, char dir) {
01845
01846     if (dim1==0) { P.init(); return *this; }
01847
01848     char lex=1;
01849     wbMatrix<double> X(*this); X.SkipTiny_float();
01850
01851     X.SortRecs(P,dir,lex);
01852
01853     return Set2Recs(P);
01854 };
01855
01856 template <class T>
01857 char wbMatrix<T>::findRecsInSet(
01858     const wbMatrix<T> &S, wbvector<int> &I) const {
01859
01860     size_t j,l,k,d, e=0; int ns=(int)S.dim1, ifound;
01861     WBINDEX dg;
01862     wbMatrix<T> QQ(*this);
01863     wbperm is;
01864
01865     QQ.groupRecs(is,dg); I.init(dim1);
01866

```

```

01867     for (l=k=0; k<QQ.dim1; k++) {
01868         for (ifound=0; ifound<ns; ifound++) {
01869             if (QQ.recEqual(k, S.rec(ifound)))
01870                 break;
01871         }
01872         if (ifound>=ns) { ifound=-1; e++; }
01873     }
01874     for (d=dg[k], j=0; j<d; j++)
01875         I[is[l++]]=ifound;
01876 }
01877
01878 return e;
01879 }
01880
01881 template <class T>
01882 size_t wbMatrix<T>::findUniqueRecSorted1(size_t n, T eps) const {
01883
01884     if (long(n)>=0 && (!n || n>dim2)) wblog(FL,
01885         "ERR %s() invalid n=%d/%d",FCT,n,dim2);
01886     if (dim1<=1) { return (dim1 ? 0:-1); }
01887
01888     if (eps==0) {
01889         char c=0, c0=0;
01890         for (size_t l=dim1-1, i=0; i<l; ++i) {
01891             c=recCompare(i,i+1,n);
01892             if (!c || !c0) { c0=c; continue; }
01893             else { return i; }
01894         }
01895         if (dim1==2 && c) return 0;
01896     }
01897     else {
01898         T c=0, c0=0;
01899         for (size_t l=dim1-1, i=0; i<l; ++i) {
01900             c=recDiff2(i,i+1,n);
01901             if (c<eps || c0<eps) { c0=c; continue; }
01902             else { return i; }
01903         }
01904         if (dim1==2 && (c>eps)) return 0;
01905     }
01906     return -1;
01907 };
01908
01909 template <class T>
01910 int wbMatrix<T>::findRecSorted(const T* r, size_t n, char lex) const {
01911
01912     if (int(n)>=0 && (!n || n>dim2)) wblog(FL,
01913         "ERR %s() invalid n=%d/%d",FCT,n,dim2);
01914
01915     if (dim1<3) {
01916         for (size_t i=0; i<dim1; ++i) {
01917             if (!recCompareP(i,r,n,lex)) return i;
01918         }; return -1;
01919     }
01920
01921     size_t k1=0, k2=dim1-1, k=(k2-k1)/2;
01922     char c, cref=recCompare(k1,k2,n,lex);
01923
01924     if (cref==0) {
01925         wblog(FL,"WRN all recs the same (%dx%d) ???",dim1,dim2);
01926         return (recCompareP(0,r,n,lex) ? -1 : 0);
01927     }
01928     for (size_t m=MAX(size_t(1), (dim1-2)/4), i=1; i<dim1; i+=m)
01929         if ((c=recCompare(i-1,i,n,lex))==-cref) wblog(FL,
01930             "ERR records not sorted (%d/%d) !?",c,cref);
01931
01932     c=recCompareP(k1,r,n,lex); if (c!= cref) return (c ? -1 : 0);
01933     c=recCompareP(k2,r,n,lex); if (c!=-cref) return (c ? -1 : dim1-1);
01934     c=recCompareP(k ,r,n,lex);
01935
01936     while (1) {
01937         if (c==cref)
01938             { k1=k; k+=(k2-k1)/2; if (k==k1) return -1; } else
01939             if (c) { k2=k; k-=(k2-k1)/2; if (k==k2) return -1; } else
01940                 return k;
01941
01942         if (recCompare(k1,k2,n,lex)==-cref) wblog(FL,
01943             "ERR records not sorted !?");
01944         c=recCompareP(k,r,n,lex);
01945     }
01946
01947     return -1;
01948 };
01949
01950 template <class T>
01951 int wbMatrix<T>::findRec(const T* r, size_t n, char lex) const {
01952
01953     for (size_t k=0; k<dim1; k++)

```



```

01954     if (!recCompareP(k,r,n,lex)) return k;
01955
01956     return -1;
01957 }
01958
01959 template <class T>
01960 size_t wbMatrix<T>::UnionRecs(const wbMatrix<T> &B0, wbindex &Ia){
01961     size_t l=0,ia=0,ib=0, na=(dim2 ? dim1 : 0), nb=B0.dim1;
01962     char c;
01963
01964     wbMatrix<T> A(*this), B(B0);
01965     wbperm P1,P2; wbindex I;
01966
01967     if (A.isEmpty() || B.isEmpty()) { init(); return na; }
01968
01969     if (A.dim2 != B.dim2) wblog(FL,
01970         "ERR %s() severe dimension mismatch (%d/%d)",
01971         FCT,A.dim2,B.dim2);
01972
01973     A.SortRecs(P1); I.init(na);
01974     B.SortRecs(P2);
01975
01976     while (ia<na && ib<nb) { c=A.recCompareP(ia, B.rec(ib));
01977         if (c<0) ia++; else
01978             if (c>0) ib++;
01979         else {
01980             do { I[l++]=ia++; } while (ia<na && A.recEqual(ia-1,ia));
01981             do {         ib++; } while (ib<nb && B.recEqual(ib-1,ib));
01982         }
01983     }
01984
01985     if (l) I.len=l; else I.init();
01986
01987     A.Set2Recs(I); P1.get(I,Ia);
01988
01989     return (na-dim1);
01990 }
01991
01992 template <class T>
01993 bool wbMatrix<T>::gotRecOverlap(const wbMatrix<T> &B0) const {
01994
01995     size_t ia=0,ib=0, na=(dim2 ? dim1 : 0), nb=B0.dim1;
01996     char c;
01997
01998     wbMatrix<T> A(*this), B(B0);
01999     wbperm P1,P2;
02000
02001     if (A.isEmpty() || B.isEmpty()) return 0;
02002
02003     if (A.dim2!=B.dim2) wblog(FL,
02004         "ERR %s() severe dimension mismatch (%d/%d)",FCT,A.dim2,B.dim2);
02005
02006     A.SortRecs(P1);
02007     B.SortRecs(P2);
02008
02009     while (ia<na && ib<nb) { c=A.recCompareP(ia, B.rec(ib));
02010         if (c<0) ia++; else
02011             if (c>0) ib++; else return 1;
02012     }
02013
02014     return 0;
02015 };
02016
02017 template <class T>
02018 bool wbMatrix<T>::gotRecOverlapSA(const wbMatrix<T> &B) const {
02019
02020     size_t ia=0,ib=0, na=(dim2 ? dim1 : 0), nb=B.dim1;
02021     char c;
02022
02023     if (isEmpty() || B.isEmpty()) return 0;
02024
02025     if (dim2!=B.dim2) wblog(FL,
02026         "ERR %s() severe dimension mismatch (%dx%d/%dx%d)",
02027         FCT,dim1,dim2,B.dim1,B.dim2);
02028
02029     while (ia<na && ib<nb) { c=recCompareP(ia, B.rec(ib));
02030         if (c<0) ia++; else
02031             if (c>0) ib++; else return 1;
02032     }
02033
02034     return 0;
02035 };
02036
02037 template <class T>
02038 size_t wbMatrix<T>::findValsCol(
02039     const size_t k, const wbvector<T> &v, wbindex &Ia
02040 ) const {

```

```

02041     wbvector<T> vk; getCol(k,vk);
02042     vk.findValues(v,Ia);
02043     return Ia.len;
02044 }
02045
02046 template <class T> inline
02047 void matchIndexU(const char *F, int L,
02048     const wbMatrix<T> &QA,
02049     const wbMatrix<T> &QB,
02050     wbindex &IB, const char force
02051 ){
02052     size_t ma,mb; wbindex Ia, Ib;
02053
02054     matchIndex(QA,QB,Ia,Ib,1,&ma,&mb);
02055     if (mb) wblog(F,L,"ERR index B not unique (%d)",mb);
02056
02057     if (force) { wbpperm iP;
02058         if (Ia.len!=QA.dim1) wblog(F,L,
02059             "ERR failed to find all records (%d/%d)",Ia.len,QA.dim1);
02060
02061         getIPerm(Ia,iP); Ib.select(iP,IB);
02062     }
02063     else {
02064         IB.init(QA.dim1).set(size_t(-1));
02065         IB.Set(Ia,Ib);
02066     }
02067 }
02068
02069 template <class T>
02070 int matchIndex(
02071     const wbMatrix<T> &QA, const wbMatrix<T> &QB,
02072     wbindex &Ia, wbindex &Ib,
02073     char lex,
02074     widx_t *ma, widx_t *mb,
02075     T eps
02076 ){
02077     size_t i;
02078
02079     wbMatrix<T> Q1(QA), Q2(QB);
02080     wbpperm P1, P2;
02081     wbindex ix1, ix2;
02082
02083     if (QA.dim2!=QB.dim2 && QA.dim2 && QB.dim2) wblog(FL,
02084         "ERR %s severe dimension mismatch (%s <> %s)",
02085         FCT,SSTR(QA),SSTR(QB));
02086
02087     if (QA.isEmpty() || QB.isEmpty()) {
02088         Ia.init(); Ib.init(); return 0;
02089     }
02090
02091     Q1.SortRecs(P1);
02092     Q2.SortRecs(P2);
02093
02094     i=matchSortedIdx(Q1,Q2,ix1,ix2,-1,lex,ma,mb,eps);
02095
02096     P1.get(ix1,Ia);
02097     P2.get(ix2,Ib);
02098
02099     return (int)i;
02100 };
02101
02102 template <class T>
02103 int wbMatrix<T>::getDiff(
02104     const wbMatrix<T> &B, wbindex &Ia, wbindex *Ib) const {
02105
02106     wbMatrix<T> Q1(*this), Q2(B);
02107     wbpperm P1,P2;
02108     wbindex ix1,ix2;
02109
02110     if (dim2!=B.dim2) wblog(FL,
02111         "ERR %s() severe dimension mismatch (%d/%d)",
02112         FCT,dim2, B.dim2
02113     );
02114
02115     if (isEmpty() || B.isEmpty()) {
02116         Ia.init(); if (Ib) Ib->init();
02117         return 0;
02118     }
02119
02120     Q1.SortRecs(P1);
02121     Q2.SortRecs(P2);
02122
02123     Q1.getDiffSorted(Q2,ix1, Ib ? &ix2 : NULL);
02124     P1.get(ix1, Ia); if (Ib) {
02125         P2.get(ix2,*Ib); }
02126
02127     return Ia.len;

```

```

02128 };
02129
02130 template <class T>
02131 int wbMatrix<T>::getDiffSorted(
02132     const wbMatrix<T> &B, wbindex &Ia, wbindex *IB
02133 ) const {
02134
02135     size_t ia,ib,la,lb; widx_t *Ib=NULL; char c;
02136
02137     if (dim2!=B.dim2) wblog(FL,
02138         "ERR %s() dimension mismatch (%d/%d)",FCT,dim2,B.dim2);
02139
02140     Ia.init(dim1);
02141     if (IB) { IB->init(B.dim1); Ib=IB->data; }
02142
02143     for (la=lb=ia=ib=0; ia<dim1 && ib<B.dim1; ) {
02144         c=recCompareP(ia, B.rec(ib));
02145         if (c<0) { Ia[la++]=ia++; } else
02146         if (c>0) { if (Ib) { Ib[lb++]=ib; }; ++ib; }
02147         else {
02148             while ((++ia)<dim1) {
02149                 c=recCompare(ia-1,ia);
02150                 if (c) {
02151                     if (c>0) wblog(FL,
02152                         "ERR %s() expecting ascending order (%d)",FCT,ia);
02153                     break;
02154                 }
02155             }
02156             while ((++ib)<B.dim1) {
02157                 c=B.recCompare(ib-1,ib);
02158                 if (c) {
02159                     if (c>0) wblog(FL,
02160                         "ERR %s() expecting ascending order (%d)",FCT,ib);
02161                     break;
02162                 }
02163             }
02164         }
02165     }
02166
02167     while (ia<dim1) { Ia[la++]=ia++; }
02168     Ia.Resize(la);
02169
02170     if (Ib) {
02171         while (ib<B.dim1) { Ib[lb++]=ib++; }
02172         IB->Resize(lb);
02173     }
02174
02175     return Ia.len;
02176 };
02177
02178 #endif
02179

```

5.46 wbMatrix.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : wbMatrix (matrix class, row-major)
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the license for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_MATRIX_ROW_MAJOR_HH__
00021 #define __WB_MATRIX_ROW_MAJOR_HH__
00022
00023 /* -----
00024  * wbMatrix - row-major matrix class
00025  * AW (C) Jan 2006
00026  * ----- */
00027
00028 template <class T>

```

```

00029 class wbMatrix {
00030
00031     public:
00032
00033         wbMatrix(size_t r=0, size_t c=0)
00034             : data(NULL), dim1(r), dim2(c), isdiag(0), isref(0) {
00035             if (r || c) NEW_DATA();
00036         };
00037
00038         wbMatrix(size_t r, size_t c, T *d, const char ref=0)
00039             : data(NULL), dim1(r), dim2(c), isdiag(0), isref(0) {
00040             if (ref) { data=d; isref=1; }
00041             else if (r || c) NEW_DATA(d);
00042         };
00043
00044         wbMatrix(const wbMatrix &M)
00045             : data(NULL), dim1(M.dim1), dim2(M.dim2), isdiag(M.isdiag), isref(0) {
00046             NEW_DATA(M.data);
00047         };
00048
00049         template <class T2>
00050         wbMatrix(const wbMatrix<T2> &M)
00051             : data(NULL), dim1(0), dim2(0), isdiag(0), isref(0) {
00052             if (M.isdiag) wblog(FL,"ERR %s() got isdiag=%d",FCT,M.isdiag);
00053             initT(M);
00054         };
00055
00056         wbMatrix(const mxArray *a, char tflag=0, char ref=0)
00057             : data(NULL), dim1(0), dim2(0), isdiag(0),isref(0) {
00058             init(FL,a,tflag,ref);
00059         };
00060
00061         wbMatrix(const char *F, int L, const mxArray *a)
00062             : data(NULL), dim1(0), dim2(0), isdiag(0),isref(0) {
00063             init(F,L,a);
00064         };
00065
00066         virtual ~wbMatrix() { if (data && !isref) {
00067             WB_DELETE(data);
00068         } };
00069
00070         wbMatrix& init(size_t r=0, size_t c=0, const T* d=NULL) {
00071             if (isref) {
00072                 isref=0; dim1=dim2=0; data=0;
00073             }
00074             RENEW(r,c,d); return *this;
00075         };
00076
00077         wbMatrix& init(
00078             size_t r, size_t c, const T *d0, size_t stride
00079         ){
00080             unsigned i=0, j=0; T* d;
00081
00082             if (int(stride)<0) wblog(FL,
00083                 "ERR %s() got stride=%ld !?",FCT,stride);
00084             if (!d0) wblog(FL,"ERR %s() got null space !?",FCT);
00085
00086             RENEW(r,c,NULL,0); d=data;
00087             for (; i<dim1; ++i, d+=dim2, d0+=stride) {
00088                 for (j=0; j<dim2; ++j) d[j]=d0[j];
00089             }
00090             return *this;
00091         };
00092
00093         wbMatrix& initIdentity(size_t n) {
00094             size_t i=0, n2=n*n; RENEW(n,n,NULL,0);
00095             for (; i<n2; i+=(n+1)) { data[i]=T(1); }
00096         };
00097
00098         wbMatrix& init2val(size_t r, size_t c, const T &d) {
00099             RENEW(r,c,NULL,0); set(d);
00100             return *this;
00101         };
00102
00103         wbMatrix& initDef(size_t r=0, size_t c=0, T* d=NULL) {
00104             RENEW(r,c,NULL,0);
00105             if (d) for (size_t n=r*c, i=0; i<n; ++i) data[i]=d[i];
00106             return *this;
00107         };
00108
00109         template<class T2>
00110         wbMatrix<T2>& initT(size_t r, size_t c, T2* d) {
00111             RENEW(r,c,0,0);
00112             for (size_t n=r*c, i=0; i<n; ++i) data[i]=(T)d[i];
00113             return *this;
00114         };
00115

```

```

00116     template<class T2>
00117     wbMatrix<T>& initT(const wbMatrix<T2> &B) {
00118         if (B.isdiag) wblog(FL,"ERR %s() got isdiag=%d",FCT,B.isdiag);
00119         RENEW(B.dim1,B.dim2,0,0);
00120         for (size_t n=dim1*dim2, i=0; i<n; ++i) data[i]=(T)B.data[i];
00121         return *this;
00122     };
00123
00124     template<class T2>
00125     wbMatrix<T>& initT(const char *F, int L, const wbMatrix<T2> &B) {
00126         if (B.isdiag) wblog(FL,"ERR %s() got isdiag=%d",FCT,B.isdiag);
00127         RENEW(B.dim1,B.dim2,0,0);
00128         for (size_t n=dim1*dim2, i=0; i<n; ++i) { data[i]=(T)B.data[i];
00129             if (T2(data[i])!=B.data[i]) wblog(F,L,
00130                 "ERR type conversion changes value (%g,%g)",
00131                 double(B.data[i]), double(T2(data[i])))
00132             );
00133         }
00134         return *this;
00135     };
00136
00137     template<class TA, class TB>
00138     wbMatrix<T>& init(const char *F, int L,
00139         const wbMatrix<TA> &B, const wbvector<TB> &V, char dim=2
00140     );
00141
00142     template<class T2>
00143     wbMatrix<T>& initDim(const wbMatrix<T2> &B) {
00144         RENEW(B.dim1, B.dim2);
00145         return *this;
00146     };
00147
00148     wbMatrix& init(const wbMatrix &B) {
00149         RENEW(B.dim1, B.dim2, B.data); isdiag=B.isdiag;
00150         return *this;
00151     };
00152
00153     wbMatrix& init(
00154         const char *F, int L, const mxArray *a,
00155         char tflag=0, char ref=0, char tcheck=1);
00156
00157     void init(const mxArray *a) { init(FL,a); };
00158
00159     void init0(const mxArray *a) {
00160         init(FL,a,1,0,1);
00161     };
00162
00163     void initTST() {
00164         size_t i,j,k=0;
00165         for (i=0; i<dim1; ++i)
00166             for (j=0; j<dim2; ++j) data[k++]=10*(i+1)+(j+1);
00167     };
00168
00169     wbMatrix& init2ref(size_t r, size_t c, const T* d);
00170     wbMatrix& init2ref(const wbvector<T> &v, const char tflag=0);
00171     wbMatrix& init2ref(const wbMatrix &M);
00172
00173     wbMatrix& unRef();
00174
00175     void no_ref_(const char *F, int L, const char *fct) {
00176         if (isref) { wblog(F_L,
00177             "ERR %s() got reference (isref=%d)",fct,isref);
00178         }
00179     };
00180
00181     void swap(wbMatrix &B, char ref=0) {
00182         if (this!=&B) {
00183             if (!ref && (isref || B.isref)) wblog(FL,
00184                 "ERR %s() got arrays with isref=(%d,%d)",FCT,isref,B.isref);
00185             SWAP(data, B.data );
00186             SWAP(dim1, B.dim1 );
00187             SWAP(dim2, B.dim2 );
00188             SWAP(isref, B.isref );
00189             SWAP(isdiag, B.isdiag);
00190         }
00191     };
00192
00193     wbMatrix& save2(wbMatrix &A, char ref=0) {
00194         if (this!=&A) {
00195             if (!ref) { no_ref_(FLF); }
00196             this->swap(A,ref); if (data!=A.data) { init(); }
00197         }
00198         return A;
00199     };
00200
00201     void copyStride(T* dd, size_t stride) const {
00202         T *d0=data;

```

```

00203         for (size_t i=0; i<dim1; ++i, d0+=dim2, dd+=stride)
00204             MEM_COPY<T>(dd, dim2, d0);
00205     };
00206
00207     wbMatrix& operator= (const wbMatrix &B) {
00208         if (this!=&B) {
00209             RENEW(B.dim1, B.dim2, B.data);
00210             isdiag=B.isdiag;
00211         }
00212         return *this;
00213     };
00214
00215     template<class TB>
00216     wbMatrix& operator= (const wbMatrix<TB> &B) {
00217         init(B.dim1, B.dim2);
00218         for (size_t i=0, n=B.numel(); i<n; ++i) { data[i]=T(B.data[i]); }
00219         isdiag=B.isdiag;
00220         return *this;
00221     };
00222
00223     explicit operator bool() const {
00224         if (data)
00225             { if ( dim1 && dim2) { return 1; }}
00226         else { if (!dim1 || !dim2) { return 0; }}
00227
00228         wblog(FL,"ERR unexpected matrix %gx%g (%p)",dim1,dim2,data);
00229         return 0;
00230     };
00231     bool operator! () const { return !bool(*this); }
00232
00233     bool operator==(const T &x) const {
00234         for (size_t n=dim1*dim2, i=0; i<n; ++i)
00235             { if (data[i]!=x) return 0; }
00236         return 1;
00237     };
00238
00239     bool operator==(const wbMatrix &B) const {
00240         if (this!=&B) {
00241             if (dim1!=B.dim1 || dim2!=B.dim2) return 0;
00242             if (data!=B.data) {
00243                 for (size_t n=dim1*dim2, i=0; i<n; ++i)
00244                     if (data[i]!=B.data[i]) return 0;
00245             }
00246         }
00247         return 1;
00248     };
00249
00250     bool operator!=(const wbMatrix &B) const {
00251         return !(*this==B);
00252     };
00253
00254     bool deepEqualP(const wbMatrix &B) const;
00255
00256     bool anyEqual(const T& x) const {
00257         for (size_t n=dim1*dim2, i=0; i<n; ++i) {
00258             if (data[i]==x) return 1; }
00259         return 0;
00260     };
00261
00262     bool allEqual(const T& x) const {
00263         for (size_t n=dim1*dim2, i=0; i<n; ++i) {
00264             if (data[i]!=x) return 0; }
00265         return 1;
00266     };
00267
00268     bool anyUnequal(const T& x) const { return !allEqual(x); };
00269
00270     bool anyLT(const T& x) const {
00271         for (size_t n=dim1*dim2, i=0; i<n; ++i) {
00272             if (data[i]<x) return 1; }
00273         return 0;
00274     };
00275
00276     bool anyGT(const T& x) const {
00277         for (size_t n=dim1*dim2, i=0; i<n; ++i) {
00278             if (data[i]>x) return 1; }
00279         return 0;
00280     };
00281
00282     bool col_anyGT(size_t j, const T& x) const {
00283         if (dim1) {
00284             if (j>=dim2) wblog(FL,
00285                 "ERR %s() index out of bounds (%d/%d)",FCT,j,dim2);
00286             for (size_t i=0; i<dim1; ++i, j+=dim2) {
00287                 if (data[j]>x) return 1;
00288             }
00289         }

```

```

00290         return 0;
00291     };
00292
00293     wbMatrix& operator+= (const wbMatrix &B) {
00294         size_t i,s=dim1*dim2;
00295
00296         if (isdiag) if (!B.isdiag) isdiag=0;
00297
00298         if (dim1!=B.dim1 || dim2!=B.dim2) wblog(FL,
00299             "ERR Dimension mismatch (%dx%d + %dx%d).",dim1,dim2,B.dim1,B.dim2);
00300
00301         for (i=0; i<s; ++i) data[i]+=B.data[i];
00302         return *this;
00303     };
00304
00305     wbMatrix operator+ (const wbMatrix &B) const {
00306         wbMatrix Mout(*this); Mout+=B;
00307         return Mout;
00308     };
00309
00310     wbMatrix operator+ (const T &x) const {
00311         wbMatrix Mout(*this); Mout+=x;
00312         return Mout;
00313     };
00314
00315     void operator-= (const wbMatrix &B) {
00316         if (isdiag) if (!B.isdiag) { isdiag=0; }
00317         if (dim1!=B.dim1 || dim2!=B.dim2) wblog(FL,
00318             "ERR size mismatch (%dx%d + %dx%d).",dim1,dim2,B.dim1,B.dim2);
00319
00320         for (size_t i=0, n=dim1*dim2; i<n; ++i) {
00321             data[i]-=B.data[i];
00322         }
00323     };
00324
00325     wbMatrix operator- (const wbMatrix &B) const {
00326         wbMatrix Mout(*this); Mout-=B;
00327         return Mout;
00328     };
00329
00330     void operator*= (const T c) {
00331         size_t i, n=dim1*dim2;
00332
00333         if (c==T(1)) { return; } else
00334         if (c==T( 0)) { set((T)0); } else
00335         if (c==T(-1)) { for (i=0; i<n; ++i) data[i]=-data[i]; }
00336         else { for (i=0; i<n; ++i) data[i]*=c; }
00337     };
00338
00339     wbMatrix& operator+= (const T c) { if (c) {
00340         for (size_t n=dim1*dim2, i=0; i<n; ++i) data[i]+=c; }
00341         return *this;
00342     };
00343
00344     void operator-= (const T c) { if (c) {
00345         for (size_t n=dim1*dim2, i=0; i<n; ++i) data[i]-=c; }
00346     };
00347
00348     wbMatrix operator* (const T x) const {
00349         wbMatrix X(*this); if (x!=1) X*=x;
00350         return X;
00351     };
00352
00353     wbMatrix& add (const wbMatrix &B, const T& c=1, const char zflag=0);
00354     wbMatrix& minus(const wbMatrix &B, const T& c=1, const char zflag=0);
00355     wbMatrix& times(T fac, wbMatrix &X) const {
00356         X=*this; X*=fac; return X;
00357     };
00358
00359     wbMatrix& rowTimes(size_t i, T fac);
00360     wbMatrix& colTimes(size_t j, T fac);
00361
00362     T prod() const {
00363         size_t i,n=dim1*dim2; T x=0;
00364
00365         if (n==0) {
00366             wblog(FL,"WRN %s() of null object !?",FCT);
00367             return x;
00368         }
00369
00370         for (x=data[0], i=1; i<n; ++i)
00371             if (x<data[i]) x*=data[i];
00372
00373         return x;
00374     };
00375
00376     T min() const {

```

```

00377     size_t i,n=dim1*dim2; T x=0;
00378
00379     if (n==0) { wblog(FL,
00380         "WRN %s() of null object !?",FCT); return x; }
00381     for (x=data[0], i=1; i<n; ++i) { if (x>data[i]) x=data[i]; }
00382
00383     return x;
00384 };
00385
00386 T max() const {
00387     size_t i,n=dim1*dim2; T x=0;
00388
00389     if (n==0) { wblog(FL,
00390         "WRN %s() of null object !?",FCT); return x; }
00391     for (x=data[0], i=1; i<n; ++i) { if (x<data[i]) x=data[i]; }
00392
00393     return x;
00394 };
00395
00396 T aMax() const {
00397     T m=0; if (dim1 && dim2) {
00398         size_t i=0, n=dim1*dim2; T a;
00399         for (; i<n; ++i) { a=Wb::abs(data[i]); if (m<a) { m=a; } }
00400     }
00401     else wblog(FL,"WRN %s() got empty object",FCT);
00402     return m;
00403 };
00404
00405 T aMax(unsigned &i_, unsigned &j_) const {
00406     T m=0, a;
00407     if (!dim1 && !dim2) wblog(FL,"ERR %s() got empty object",FCT);
00408
00409     size_t i,j,l=0;
00410     for (i=0; i<dim1; ++i)
00411         for (j=0; j<dim2; ++j, ++l) { if (m<(a=Wb::abs(data[i]))) {
00412             m=a; i_=i; j_=j;
00413         } }
00414     return m;
00415 };
00416
00417 T sum() const {
00418     T x=0;
00419     size_t i, n=dim1*dim2;
00420     for (i=0; i<n; ++i) x+=data[i];
00421     return x;
00422 };
00423
00424 T cumsum0prodRec(wbvector<T> &cs, char xflag=0) const {
00425     wbvector<T> x;
00426     return recProd(x).cumsum_(cs,xflag);
00427 };
00428
00429 T cumsum0prodRec(
00430     wbvector<T> &cs, unsigned i1, unsigned i2, char xflag=0
00431 ) const {
00432
00433     wbvector<T> x(dim1);
00434     if (i2>=dim2) wblog(FL,
00435         "ERR %s() index out of bounds (%d/%d)",FCT,i2+1,dim2);
00436
00437     if (i1<=i2) {
00438         unsigned i=0, j, n=i2-i1+1; const T* d=data+i1;
00439         for (; i<dim1; ++i, d+=dim2) {
00440             x[i]=d[0];
00441             for (j=1; j<n; ++j) { x[i]*=d[j]; }
00442         }
00443     }
00444
00445     return x.cumsum_(cs,xflag);
00446 };
00447
00448 T norm2() const {
00449     T x2=0; size_t i, n=dim1*dim2;
00450     for (i=0; i<n; ++i) x2+=Wb::norm2(data[i]);
00451     return x2;
00452 };
00453 T norm() const { return Wb::sqrt(norm2()); }
00454
00455 double normReal() const;
00456 double normImag() const;
00457
00458 T norm2Cols(long j1, long j2) const {
00459     T x2=0; long i,j;
00460
00461     if (j1<0) { j1+=dim2; }
00462     if (j2<0) { j2+=dim2; }; if (j1>j2 || !dim1) { return x2; }
00463     if (j1<0 || j2>=dim2) wblog(FL,

```



```

00464         "ERR %s() index out of bounds (%ld,%ld/%ld)",FCT,j1,j2,dim2);
00465
00466         for (i=0; i<dim1; ++i)
00467             for (j=j1; j<=j2; ++j) { x2+=Wb::norm2(data[i*dim2+j]); }
00468         return x2;
00469     };
00470
00471     T norm2Recs(long i1, long i2) const {
00472         T x2=0; long i,j;
00473
00474         if (i1<0) { i1+=dim1; }
00475         if (i2<0) { i2+=dim1; }; if (i1>i2 || !dim2) { return x2; }
00476         if (i1<0 || i2>=dim1) wblog(FL,
00477             "ERR %s() index out of bounds (%ld,%ld/%ld)",FCT,i1,i2,dim1);
00478
00479         for (i=i1; i<=i2; ++i)
00480             for (j=0; j<dim2; ++j) { x2+=Wb::norm2(data[i*dim2+j]); }
00481         return x2;
00482     };
00483
00484     wbvector<T>& norm2Cols(wbvector<T> &x2_) const {
00485         x2_.init(dim2); T* x2=x2_.data; size_t i,j, l=0;
00486         for (i=0; i<dim1; ++i)
00487             for (j=0; j<dim2; ++j, ++l) { x2[j]+=Wb::norm2(data[l]); }
00488         return x2_;
00489     };
00490
00491     wbvector<T> norm2Cols() const {
00492         wbvector<T> x2; return norm2Cols(x2);
00493     };
00494
00495     wbvector<T> normCols() const {
00496         wbvector<T> x_; norm2Cols(x_); T *x=x_.data;
00497         for (size_t i=0; i<dim2; ++i) { x[i]=Wb::sqrt(x[i]); }
00498         return x_;
00499     };
00500
00501     size_t maxRec(char lex=1, size_t *m=NULL) const;
00502     size_t maxRec_float(char lex=1, size_t *m=NULL,
00503         T xref QS_UNUSED_VAR =-1
00504         ) const { return maxRec(lex,m); };
00505
00506     T recMax(size_t r, size_t *k=NULL) const;
00507     T colMax(size_t c, size_t *k=NULL) const;
00508
00509     T colMax(size_t c, unsigned &k) const { T x;
00510         size_t K; x=colMax(c,&K); k=K;
00511         if (size_t(k)!=K) wblog(FL,
00512             "ERR %s() unsigned out of bounds (%d/%ld)",FCT,k,K);
00513         return x;
00514     };
00515
00516     wbMatrix& NormalizeCol(size_t k);
00517
00518     bool isOrthoRows(T *x=NULL, char tnorm=0, T eps=1E-14) const;
00519     bool isOrthoRows(const wbMatrix &B,
00520         T *x=NULL, char tnorm=0, T eps=1E-14) const;
00521
00522     double recMaxA(size_t r, size_t *k=NULL) const;
00523
00524     size_t skipNanRecs(wbindex &I);
00525
00526     wbMatrix& ResizeXRecs(size_t r);
00527     wbMatrix& Resize(size_t r, size_t c, const T* dx=NULL);
00528     wbMatrix& resize(size_t m, size_t n, wbMatrix &M) const;
00529
00530     wbMatrix& Resize2Mult(const WBINDEX &M);
00531
00532     wbMatrix& transpose(wbMatrix &M) const;
00533     wbMatrix& Transpose() {
00534         wbMatrix X; save2(X); return X.transpose(*this);
00535     };
00536
00537     wbMatrix& Reshape(size_t r, size_t c);
00538
00539     template <class T2>
00540     wbMatrix<T>& AddCols(size_t n, const T2* d0=NULL) {
00541         Resize(dim1,dim2+n);
00542         if (d0) Wb::cpyStride(data+dim2-n, d0, n, dim1, dim2);
00543         return *this;
00544     };
00545
00546     template <class T2>
00547     wbMatrix<T>& AddRows(size_t n, const T2* d0=NULL) {
00548         Resize(dim1+n,dim2);
00549         if (d0) MEM_COPY<T>(data+(dim1-n)*dim2, n*dim2, d0);
00550         return *this;
00551     };

```

```

00551     };
00552
00553     bool isEmpty() const { return data==NULL; };
00554     bool isNormal() const;
00555
00556     bool isFinite() const {
00557         return Wb::is_finite(data,dim1*dim2);
00558     };
00559
00560     bool isSquare() const { return (dim1==dim2); };
00561     bool isSquare(size_t d) const { return (dim1==dim2 && d==dim1); };
00562
00563     bool isScalar() const { return (dim1==1 && dim2==1); };
00564     bool isVector() const { return (dim1==1 || dim2==1); };
00565
00566     bool isDiag() const;
00567     bool isDiagMatrix(T eps=1E-14) const;
00568
00569     bool isIdentity(double eps, double &maxdiff) const;
00570     bool isProptoId(T &x, T eps=1E-14) const;
00571
00572     bool isHConj(
00573         const wbMatrix &B, double eps=1E-12, double *xref=NULL
00574     ) const { return isSym_aux(B, eps, xref, 's'); };
00575
00576     bool isHConj(
00577         double eps=1E-12, double *xref=NULL
00578     ) const { return isSym_aux(*this,eps,xref,'s'); };
00579
00580     bool isAHerm(
00581         const wbMatrix &B, double eps=1E-12, double *xref=NULL
00582     ) const { return isSym_aux(B, eps, xref, 'a'); };
00583
00584     bool isAHerm(
00585         double eps=1E-12, double *xref=NULL
00586     ) const { return isSym_aux(*this,eps,xref,'a'); };
00587
00588     bool isComplex() const;
00589
00590     bool isUnique() const;
00591
00592     bool isUniqueSorted(char dir=0, char lex=1) const;
00593
00594     int isSorted(
00595         char dir=0,
00596         char lex=1,
00597         size_t m=-1
00598     ) const;
00599
00600     bool quickCheckSorted(
00601         char dir,
00602         char lex=1,
00603         const char *F=NULL, int L=0,
00604         size_t n=2
00605     ) const;
00606
00607     bool thisIsInt() const;
00608
00609     int getDiff(
00610         const wbMatrix &B, wbindex &Ia, wbindex *Ib=NULL) const;
00611
00612     int getDiffSorted(
00613         const wbMatrix &B, wbindex &Ia, wbindex *Ib=NULL) const;
00614
00615     size_t length() const {
00616         return dim1>dim2 ? (dim2 ? dim1 : 0) : (dim1 ? dim2 : 0);
00617     };
00618
00619     size_t numel() const { return dim1*dim2; };
00620
00621     void flipSign() {
00622         size_t i,s=dim1*dim2;
00623         for (i=0; i<s; ++i) data[i]=-data[i];
00624     };
00625
00626     wbMatrix& Symmetrize() {
00627         size_t i,j,r,s;
00628         if (dim1!=dim2) wblog(FL,
00629             "ERR Symmetrize() called with %dx%d matrix !?",dim1,dim2);
00630
00631         for (j=0; j<dim2; ++j)
00632             for (i=j+1; i<dim1; ++i) {
00633                 r=i*dim2+j; s=j*dim2+i;
00634                 data[r]=data[s]=0.5*(data[r]+data[s]);
00635             }
00636
00637         return *this;

```

```

00638     };
00639
00640     void set(const T &x) {
00641         for (size_t s=dim1*dim2, i=0; i<s; ++i) { data[i]=x; }
00642         isdiag=0;
00643     };
00644
00645     wbMatrix& setRand(double fac=1., double shift=0.);
00646     void setDiagRand(double fac=1., double shift=0.);
00647
00648     double maxRelDiff(const wbMatrix &M) const;
00649
00650     template <class T2>
00651     double normDiff(const wbMatrix<T2> &M) const;
00652
00653     T normDiff2(const wbMatrix &M, size_t *k=NULL) const;
00654     T normDiff (const wbMatrix &M, size_t *k=NULL) const {
00655         return Wb::sqrt(normDiff2(M,k));
00656     };
00657
00658     bool sameSize(const wbMatrix B) const {
00659         return (dim1==B.dim1 && dim2==B.dim2);
00660     };
00661
00662     WBINDEX& toIndex(WBINDEX &I, const WBINDEX *S QS_UNUSED_VAR =NULL)
00663     ) const {
00664         wblog(FL,"ERR %s() not defined for type '%s'",FCT,
00665             TSTR(T)); return I;
00666     };
00667
00668     WBIDXMAT& toIndex2D(
00669         const WBINDEX &S QS_UNUSED_VAR,
00670         const wbindex &ic QS_UNUSED_VAR, WBIDXMAT &IJ,
00671         char pos QS_UNUSED_VAR =1
00672     ) const {
00673         wblog(FL,"ERR %s() not defined for type '%s'",FCT,
00674             TSTR(T)); return IJ;
00675     };
00676
00677     wbMatrix& SkipTiny_float(T ref QS_UNUSED_VAR ==-1) {
00678         return *this;
00679     };
00680
00681     void reset() {
00682         if (dim1 && dim2) MEM_SET<T>(data,dim1*dim2);
00683         isdiag=0;
00684     }
00685
00686     size_t nnz() const {
00687         size_t n=0, i=0,s=dim1*dim2;
00688         for (i=0; i<s; ++i) { if (data[i]!=T(0)) ++n; }
00689         return n;
00690     };
00691
00692     size_t nnz(T eps) const {
00693         size_t n=0;
00694         if (!eps) { n=nnz(); } else {
00695             size_t i,s=dim1*dim2;
00696             for (i=0; i<s; ++i) { if (Wb::abs(data[i])>eps) ++n; }
00697         }
00698         return n;
00699     };
00700
00701     wbMatrix& CAT(C_UINT d12, const wbMatrix** M, C_UINT len);
00702     wbMatrix& CAT(C_UINT d12, const wbvector< wbMatrix > &M0);
00703     wbMatrix& CAT(C_UINT d12, wbvector< wbMatrix const* > M);
00704
00705     wbMatrix& CAT(C_UINT d12, const wbvector< wbvector<T> > &M);
00706     wbMatrix& CAT(C_UINT d12, const wbvector< wbvector<T> const* > &M);
00707
00708     wbMatrix& cat(C_UINT d12, const wbMatrix&);
00709     wbMatrix& cat(C_UINT d12, const wbMatrix&, const wbMatrix&);
00710     wbMatrix& cat(C_UINT d12, const wbMatrix&, const wbMatrix&, const wbMatrix&);
00711
00712     wbMatrix& Cat(C_UINT d12,
00713         const wbMatrix&, const wbMatrix&);
00714     wbMatrix& Cat(C_UINT d12,
00715         const wbMatrix&, const wbMatrix&, const wbMatrix&);
00716     wbMatrix& Cat(C_UINT d12,
00717         const wbMatrix&, const wbMatrix&, const wbMatrix&, const wbMatrix&);
00718
00719     wbMatrix<T>& CAT(const unsigned dim,
00720         wbvector< wbMatrix<T> const* > M,
00721         wbvector< WBINDEX const* > I
00722     );
00723
00724     wbMatrix& Cat(const unsigned dim,

```

```

00725     const wbMatrix &M1, const wbindex &I1,
00726     const wbMatrix &M2, const wbindex &I2
00727 ){
00728     wbvector< wbMatrix<T> const* > M(2);
00729     wbvector< WBINDEX const* > I(2);
00730     M.data[0]=&M1; I.data[0]=&I1;
00731     M.data[1]=&M2; I.data[1]=&I2;
00732     return CAT(dim,M,I);
00733 };
00734
00735 void split(C_UINT d12, wbvector< wbMatrix* > &M) const;
00736 void split(C_UINT d12, wbMatrix** M, C_UINT len) const;
00737 void split(C_UINT d12, wbMatrix &M1, wbMatrix &M2, wbMatrix &M3) const;
00738
00739 const T& operator() (size_t i, size_t j, const T&x) const {
00740     if (i<dim1 && j<dim2) return data[i*dim2+j];
00741     else return x;
00742 };
00743 T& operator() (size_t i, size_t j, const T&x) {
00744     if (i<dim1 && j<dim2) return data[i*dim2+j];
00745     else return x;
00746 };
00747
00748 const T& operator() (size_t i, size_t j) const { return data[i*dim2+j]; };
00749 T& operator() (size_t i, size_t j) { return data[i*dim2+j]; };
00750
00751 const T& el(size_t i, size_t j) const {
00752     if (i>=dim1 || j>=dim2) wblog(FL,"ERR %s() "
00753     "index out of bounds (%d,%d; %dx%d)",FCT,i+1,j+1,dim1,dim2);
00754     return data[i*dim2+j];
00755 };
00756 T& el(size_t i, size_t j) {
00757     if (i>=dim1 || j>=dim2) wblog(FL,"ERR %s() "
00758     "index out of bounds (%d,%d; %dx%d)",FCT,i+1,j+1,dim1,dim2);
00759     return data[i*dim2+j];
00760 };
00761
00762 const T& els(long i, long j) const {
00763     if (i<0) i+=dim1;
00764     if (j<0) j+=dim2;
00765     if (i>=dim1 || j>=dim2) wblog(FL,"ERR %s() "
00766     "index out of bounds (%d,%d; %dx%d)",FCT,i+1,j+1,dim1,dim2);
00767     return data[i*dim2+j];
00768 };
00769 T& els(long i, long j) {
00770     if (i>=dim1 || j>=dim2) wblog(FL,"ERR %s() "
00771     "index out of bounds (%d,%d; %dx%d)",FCT,i+1,j+1,dim1,dim2);
00772     return data[i*dim2+j];
00773 };
00774
00775 const T* operator() (size_t i) const { return data+i*dim2; };
00776 T* operator() (size_t i) { return data+i*dim2; };
00777
00778 const T& operator[] (size_t i) const { return data[i]; };
00779 T& operator[] (size_t i) { return data[i]; };
00780
00781 const T* ref(size_t i, size_t j=0) const {
00782     if (int(j)<0) j=dim2+j;
00783     if (i>=dim1 || j>dim2) wblog(FL,"ERR %s() "
00784     "index out of bounds (%ld/%ld, %ld/%ld)",FCT,i,dim1,j,dim2);
00785     return data+(i*dim2+j);
00786 };
00787 T* ref(size_t i, size_t j=0) {
00788     if (int(j)<0) j=dim2+j;
00789     if (i>=dim1 || j>dim2) wblog(FL,"ERR %s() "
00790     "index out of bounds (%ld/%ld, %ld/%ld)",FCT,i,dim1,j,dim2);
00791     return data+(i*dim2+j);
00792 };
00793
00794 const T* rec(size_t i) const {
00795     if (i>=dim1) wblog(FL,"ERR index out of bounds (%d/%d)",i,dim1);
00796     return data+i*dim2;
00797 };
00798
00799 T* rec(size_t i) {
00800     if (i>=dim1) wblog(FL,
00801     "ERR %s() index out of bounds (%d/%d)",FCT,i,dim1);
00802     return data+i*dim2;
00803 };
00804
00805 void getDiag(wbvector<T> &D) const {
00806     size_t i, n=MIN(dim1,dim2); D.init(n);
00807     for (i=0; i<n; ++i) D[i]=data[i*dim2+i];
00808 };
00809 wbvector<T> getDiag() const {
00810     wbvector<T> D; getDiag(D); return D;
00811 };

```

```

00812
00813 void appendRows (size_t n, const T* =NULL);
00814 void appendRow(const wbvector<T> &v);
00815
00816 mxArray* toMx (char raw=0) const;
00817 mxArray* toMxP(char flag=0) const;
00818 mxArray* toMx_base(char raw=0) const;
00819 mxArray* toMx_base_d(char raw=0) const;
00820
00821 mxArray* toMx_Struct() const;
00822 mxArray* toMxP_S() const;
00823 mxArray* toMxP_C() const;
00824
00825 mxArray* toMxT(char raw QS_UNUSED_VAR =0) const { return toMx(); }
00826
00827 mxArray* mxCreateStruct(unsigned m, unsigned n) const;
00828 void add2MxStruct(mxArray *S, unsigned i, char tst=0) const;
00829
00830 void toMxStruct(mxArray* S, const char *vname) const;
00831 void mat2mx (mxArray* &a) const { a=toMx(); };
00832 void mat2mxs(mxArray* a, char field_nr) const;
00833
00834 void info(const char *istr="ans") const;
00835 void print(const char *istr="", char mflag=0) const;
00836 void Print(const char *istr="", char mflag=0) const;
00837
00838 void recPrint(size_t i, const char *istr="", char mflag=0) const;
00839
00840 void printdata(
00841     const char *istr, const char *dbl_fmt=" %8.4g", const char *rsep="\n") const;
00842
00843 wbstring toStr() const { return toStr("", " ", ";", "0,"); };
00844
00845 wbstring toStr(
00846     const char *fmt, const char *sep=" ", const char *rsep=";",
00847     size_t stride=0, const char *sep2=""
00848 ) const;
00849
00850 wbstring rec2Str(
00851     size_t k, const char *fmt="", const char *sep=" ",
00852     size_t stride=0, const char *sep2=""
00853 ) const;
00854
00855 wbstring sizeStr() const {
00856     wbstring s(16);
00857     size_t l=snprintf(s.data,s.len,"%ldx%ld",dim1,dim2);
00858     if (l>=s.len) wblog(FL,
00859         "WRN %s() string out of bounds (%ld/%ld)",FCT,l,s.len);
00860     return s;
00861 };
00862
00863 void put(const char *F, int L,
00864     const char *vname="ans", const char *ws="caller", const char raw=0
00865 ) const {
00866     mxArray *a=toMx(raw);
00867     mxPutAndDestroy(F_L,a,vname,ws);
00868 };
00869
00870 void put(const char *vname="ans", const char *ws="caller",
00871     const char raw=0) const { put(0,0,vname,ws,raw); }
00872
00873 void putg(const char *vname="ans", const char raw=0
00874 ) const { put(vname, "global",raw); return; };
00875
00876 void putx(const char *vname="ans", const char raw=0
00877 ) const { put(vname, "caller", raw); return; };
00878
00879 void put0(const char *vname="ans", const char *ws="caller") const {
00880     put(vname,ws,'r');
00881 }
00882
00883 void getReal(wbMatrix<double> &R) const;
00884 void getImag(wbMatrix<double> &I) const;
00885 void set(const wbMatrix<double> &R, const wbMatrix<double> &I);
00886 wbMatrix& Conj();
00887
00888 void setRec(size_t r, const T x) {
00889     T* d=data+r*dim2;
00890     if (r>=dim1) wblog(FL,"ERR index out of bounds (%d/%d)",r,dim1);
00891     for (size_t i=0; i<dim2; ++i) d[i]=x;
00892 };
00893
00894 void setRec(size_t r, const T x1, const T x2) {
00895     size_t i=0, m=dim2%2, n=dim2-m; T* d=data+r*dim2;
00896
00897     if (r>=dim1) wblog(FL,
00898         "ERR index out of bounds (%d/%d)",r,dim1);

```

```

00899
00900     for (; i<n; i+=2) { d[i]=x1; d[i+1]=x2; }
00901     if (m) d[i]=x1;
00902 };
00903
00904 void recSet(size_t idest, size_t isrc);
00905
00906 void recSet(size_t, const wbvector<T>&);
00907
00908 template<class T2>
00909 void recSetT(size_t, const wbvector<T2>&);
00910
00911 void recSet(size_t, const wbMatrix&, size_t);
00912 void recSet(size_t i, const wbvector<T>&, const wbvector<T>&);
00913
00914 void recSet (size_t, const T*, T bfac=1);
00915 void recSetP(size_t, const T*, size_t n=-1);
00916 void recSetP(size_t, const T*, size_t, const T*, size_t);
00917 void recSetP(size_t,
00918     const T*, const wbindex &i1, const T*, const wbindex &i2);
00919
00920 void recSetB(size_t, size_t, size_t D, const T*,
00921     const T* =NULL, const T bfac=1);
00922 void recSetB(size_t, const wbindex&, size_t D, const T*);
00923
00924 void recSet(size_t k,
00925     const wbvector<T> &a, const wbindex &ia,
00926     const wbvector<T> &b, const wbindex &ib
00927 ){ return recSet(k,a.data,ia,b.data,ib); };
00928
00929 void recSet(size_t k,
00930     const T* a, const wbindex &ia,
00931     const T* b, const wbindex &ib
00932 );
00933
00934 template<class T2>
00935 void setCol(size_t k, const wbvector<T2> &v);
00936
00937 template<class T2>
00938 void setCol(size_t k, const T2 *v, T fac=1, size_t stride=1);
00939
00940 void setCol(size_t k, const T& x);
00941
00942 template <class T2>
00943 void setColT(size_t k, const wbvector<T2> &v);
00944 template <class T2>
00945 void setColT(size_t k, const T2* v, T fac=1, size_t stride=1);
00946
00947 void setLastCol(const T& x) { setCol(dim2-1,x); };
00948
00949 void recAddP(size_t, const T*);
00950 int recLess (size_t, size_t, char lex=1) const;
00951 int recLessE(size_t, size_t, char lex=1) const;
00952 int recEqual(size_t, size_t) const;
00953 int recEqual(size_t j1, const T *d) const;
00954
00955 T recDiff2(size_t i1, size_t i2, size_t n=-1) const;
00956 T recDiff2P(size_t i1, const T* b, size_t n=-1) const;
00957
00958 bool recAllEqual(const T *v) {
00959     for (size_t i=0; i<dim1; ++i) { if (recCompareP(i,v)) return 0; }
00960     return 1;
00961 };
00962
00963 bool recAllEqual() {
00964     for (size_t i=1; i<dim1; ++i) { if (recCompare(i,0)) return 0; }
00965     return 1;
00966 };
00967
00968 bool recIsZero(size_t r) {
00969     T *d=data+r*dim2;
00970     if (r>=dim1) wblog(FL,"ERR index out of bounds (%d/%d)",r,dim1);
00971     for (size_t i=0; i<dim2; ++i) if (d[i]!=0) return 0;
00972     return 1;
00973 };
00974
00975 char recCompare(
00976     size_t, size_t, size_t m=-1, char lex=1, T eps=0) const;
00977 char recCompare(
00978     size_t, const wbvector<T> &r2, char lex=1) const;
00979 char recCompareP(
00980     size_t, const T*, size_t m=-1, char lex=1, T eps=0) const;
00981
00982 void set2Rec(size_t i) { recSet(0,i); Resize(1,dim2); };
00983
00984 wbMatrix& Set2Recs(const WBINDEX &I) {
00985     wbMatrix X; save2(X);

```

```

00986         return X.getRecs(I,*this);
00987     };
00988
00989     wbvector<T>& getRec(size_t j0, wbvector<T> &v, char ref=0) const;
00990     wbvector<T> getRec(size_t j0) const {
00991         wbvector<T> v; return getRec(j0,v);
00992     };
00993
00994     size_t UnionRecs(const wbMatrix &B, wbindex &Ia);
00995
00996     bool gotRecOverlap (const wbMatrix &B) const;
00997     bool gotRecOverlapSA(const wbMatrix &B) const;
00998
00999     size_t findValsCol(
01000         const size_t k, const wbvector<T> &v, wbindex &Ia) const;
01001
01002     wbMatrix& getRecs(const WBINDEX &, wbMatrix &M) const;
01003     wbMatrix getRecs(const WBINDEX &I) const {
01004         wbMatrix M; getRecs(I,M);
01005         return M;
01006     };
01007
01008     wbMatrix& getRecs(
01009         size_t j1, size_t j2, wbMatrix &M, size_t m=0) const;
01010
01011     wbMatrix getRecs(
01012         size_t i1, size_t i2, size_t m=0) const {
01013         wbMatrix M; getRecs(i1,i2,M,m);
01014         return M;
01015     };
01016
01017     wbvector<T>& getCol(const size_t c, wbvector<T> &v) const;
01018     wbvector<T> getCol(const size_t c) const {
01019         wbvector<T> v; getCol(c,v); return v;
01020     };
01021
01022     wbMatrix getCols(const size_t j1, const size_t j2) const {
01023         wbMatrix M; getCols(j1,j2,M);
01024         return M;
01025     };
01026
01027     wbMatrix& getCols(size_t j1, size_t j2, wbMatrix &M) const;
01028
01029     template <class TI>
01030     wbMatrix& getCols(size_t n, const TI *I, wbMatrix<T> &M) const;
01031
01032     wbMatrix& getCols(const wbperm &p, wbMatrix &M) const {
01033         return getCols(p.len,p.data,M);
01034     };
01035
01036     wbMatrix& getCols(
01037         const wbindex &I, wbMatrix &M, char iflag=0) const {
01038
01039         if (!iflag) { return getCols(I.len,I.data,M); }
01040         else {
01041             wbindex I2; I.invert(dim2,I2);
01042             return getCols(I2.len,I2.data,M);
01043         }
01044     };
01045
01046     wbMatrix& set2Cols(
01047         const size_t j1, const size_t j2);
01048
01049     wbMatrix& set2Cols(const wbindex &J, char iflag=0) {
01050         wbMatrix X; this->save2(X); X.getCols(J,*this,iflag);
01051         return *this;
01052     };
01053
01054     wbMatrix& recPermute(const wbperm &P, wbMatrix &M, char iflag=0) const;
01055     wbMatrix& recPermute(const wbperm &P, char iflag=0) {
01056         wbMatrix M(*this);
01057         return M.recPermute(P,*this,iflag);
01058     };
01059
01060     wbMatrix& colPermute(const wbperm &P, wbMatrix &M, char iflag=0) const;
01061     wbMatrix& colPermute(const wbperm &P, char iflag=0) {
01062         wbMatrix M; save2(M); return M.colPermute(P,*this,iflag);
01063     };
01064
01065     wbMatrix& FlipRecs() { wbperm P(dim1,'r'); return recPermute(P); };
01066     wbMatrix& FlipCols() { wbperm P(dim2,'r'); return colPermute(P); };
01067
01068     wbMatrix& cols2Front(
01069         const WBINDEX &I1, wbMatrix &M) const;
01070     wbMatrix& cols2End(
01071         const WBINDEX &I1, wbMatrix &M) const;
01072

```

```

01073 void blockPermute(const wbperm &P, wbMatrix &M) const;
01074 void BlockPermute(const wbperm &P){
01075     wbMatrix X; save2(X); X.blockPermute(P,*this);
01076 };
01077
01078 wbvector<T>& recProd(wbvector<T> &p) const;
01079 T recProd(size_t) const;
01080
01081 T recSum (size_t) const;
01082 wbvector<T>& recSum(wbvector<T> &) const;
01083 wbvector<T>& recSumA(wbvector<T> &) const;
01084 wbvector<T> recSum() const { wbvector<T> s; return recSum(s); };
01085 T colSum (size_t) const;
01086
01087 wbMatrix& repmat (
01088     size_t m, size_t n, wbMatrix &Q,
01089     size_t pad1=0, size_t pad2=0
01090 ) const;
01091
01092 wbMatrix& Repmat (
01093     size_t m, size_t n,
01094     size_t pad1=0, size_t pad2=0
01095 ){ wbMatrix X(*this); return X.repmat(m,n,*this,pad1,pad2); };
01096
01097 void ColKron(
01098     const wbMatrix &Q,
01099     int d=-1
01100 );
01101
01102 wbMatrix& getBlocks(
01103     size_t j1, size_t j2,
01104     const size_t D, wbMatrix &M
01105 ) const;
01106
01107 wbMatrix& getBlock(
01108     size_t j1,
01109     const size_t D, wbMatrix &M) const;
01110
01111 wbMatrix getBlock(
01112     size_t j1,
01113     const size_t D
01114 ) const {
01115     wbMatrix M; getBlock(j1,D,M);
01116     return M;
01117 };
01118
01119 template <class T2>
01120 wbMatrix<T>& getBlock(
01121     size_t k, size_t D,
01122     const wbMatrix<T2>& R, size_t k2, size_t D2,
01123     wbMatrix &M
01124 ) const;
01125
01126 wbMatrix& getBlock(
01127     size_t i, size_t j,
01128     size_t n, size_t m,
01129     wbMatrix &M
01130 ) const;
01131
01132 wbMatrix& blockSum(size_t D, wbMatrix&) const;
01133 void blockSum(size_t, size_t, const size_t, wbMatrix<double>&) const;
01134 void blockSum(const wbindex &J, const size_t, wbMatrix<double>&) const;
01135
01136 void setBlock(size_t k, const size_t D, const T*);
01137 void SetBlock(size_t i0, size_t j0, const wbMatrix &M);
01138
01139 WBIDXMAT& toBlockIndex(
01140     widx_t D, WBIDXMAT &II,
01141     wbvector< wbMatrix > *QI=NULL
01142 ) const;
01143
01144 template <class T2>
01145 void toBlockIndex(widx_t D,
01146     wbMatrix<T2> &R, widx_t DR,
01147     WBIDXMAT *II=NULL, WBIDXMAT *IS=NULL,
01148     WBIDXMAT *SS=NULL, wbvector< wbMatrix > *QI=NULL
01149 ) const;
01150
01151 void groupRecs(
01152     wbperm &P, WBINDEX &d,
01153     size_t m=-1,
01154     char lex=1,
01155     wbindex *Ig=NULL,
01156     wbMatrix<T> *X=NULL
01157 );
01158
01159 void groupRecs(wbindex &Ig, widx_t m=-1, char lex=1) {

```



```

01160     wbperm P; wbvector<widx_t> d;
01161     groupRecs(P,d,m,lex,&Ig);
01162 };
01163
01164 void groupRecs(
01165     groupIndex<widx_t> &IG, size_t m=-1, char lex=1,
01166     wbindex *Ig=NULL
01167 ){
01168     wbperm P; wbvector<widx_t> d;
01169     groupRecs(P,d,m,lex,Ig);
01170     IG.initX(P,d);
01171 };
01172
01173 void groupRecs(
01174     groupIndex<widx_t> &IG, size_t m, char lex, const wperm_t **p
01175 ){
01176     wbperm P; wbvector<widx_t> d;
01177     groupRecs(P,d,m,lex);
01178     IG.initX(P,d,p);
01179 };
01180
01181 void groupRecs(
01182     wbperm &P, WBINDEX &d, wbMatrix &B,
01183     size_t m=-1, char lex=1, wbindex *Ig=NULL
01184 ) const { B=*this; B.groupRecs(P,d,m,lex,Ig); };
01185
01186 void groupRecs(
01187     wbperm &P, WBINDEX &d,
01188     const WBINDEX *I, char lex=1
01189 );
01190
01191 template <class T2>
01192 void groupRecs_rdeg(wbperm &P, WBINDEX &D, const wbMatrix<T2> &R,
01193     WBINDEX *Ib=NULL,
01194     WBINDEX *I2=NULL,
01195     WBINDEX *Sb=NULL,
01196     char iflag=0
01197 );
01198
01199 void groupRecs(
01200     wbperm &P, WBINDEX &D, size_t nc,
01201     WBINDEX &Ib,
01202     WBINDEX &I2,
01203     WBINDEX &Sb,
01204     char iflag=0
01205 );
01206
01207 void groupSortedRecs(
01208     WBINDEX &d, char keepall=0, size_t m=-1, char lex=1);
01209
01210 void groupSortedRecs(
01211     WBINDEX &d, const WBINDEX *I);
01212
01213 void groupSortedRecs(
01214     WBINDEX &d,
01215     size_t mc,
01216     char lex,
01217     WBINDEX &Ib,
01218     WBINDEX &I2,
01219     WBINDEX &Sb
01220 );
01221
01222 void makeUnique() { if (dim1<2) return;
01223     wbperm P; WBINDEX D;
01224     groupRecs(P,D);
01225 };
01226 void makeUnique(wbperm &P, WBINDEX &D) {
01227     if (dim1<2) {
01228         P.init(dim1); D.init(dim1); if (dim1) D[0]=1;
01229         return;
01230     }
01231     groupRecs(P,D);
01232 };
01233
01234 void makeUnique(wbindex &I) {
01235     if (dim1<2) { I.init(dim1); }
01236     else {
01237         size_t i=0, l=0, d; wbperm P; WBINDEX D;
01238         groupRecs(P,D); I.init(D.len);
01239         for (; i<D.len; ++i, l+=d) { d=D[i]; I[i]=P[l]; }
01240     }
01241 };
01242
01243 void findUnique(wbindex &I) const {
01244     if (dim1<2) { I.init(dim1); return; }
01245
01246     wbMatrix X(*this);

```

```

01247     wbperm P; WBINDEX D; X.groupRecs(P,D);
01248     I.init(D.numEQ(T(1)));
01249     for (size_t d, l=0, j=0, i=0; i<D.len; ++i, l+=d) {
01250         if ((d=D[i])==1) I[j++]=P[l];
01251     }
01252 };
01253
01254 size_t findUniqueRecSorted1(size_t n=-1, T eps=T(1E-12)) const;
01255
01256 void makeUnique_ig(wbindex &Iu){
01257     wbperm P; wbvector<widx_t> D; groupRecs(P,D);
01258     Iu.initGroup(P,D);
01259 };
01260 void makeUnique(groupIndex<widx_t> &IG){ groupRecs(IG); };
01261
01262 wbMatrix& SortRecs(wbperm &P,
01263     char dir=+1,
01264     char lex=1
01265 );
01266
01267 wbMatrix& sortRecs(
01268     wbMatrix &B, wbperm &P, char dir=+1, char lex=1
01269 ){
01270     B=*this;
01271     return B.SortRecs(P,dir,lex);
01272 };
01273
01274 wbMatrix& SortRecs(char dir=+1, char lex=1) {
01275     wbperm P; return SortRecs(P,dir,lex);
01276 };
01277
01278 wbMatrix& sortRecs_float(wbperm &P, char dir QS_UNUSED_VAR =+1) {
01279     P.init(dim1); return *this;
01280 };
01281 wbMatrix& sortRecs_float() { wbperm P; return sortRecs_float(P); };
01282
01283 char findRecsInSet(const wbMatrix &S, wbvector<int> &I) const;
01284 int findRecSorted(const T* r, size_t n=-1, char lex=1) const;
01285 int findRec(const T* r, size_t n=-1, char lex=1) const;
01286
01287 T *data;
01288
01289 size_t dim1, dim2;
01290
01291 char isdiag, isref;
01292
01293 protected:
01294
01295 void NEW_DATA(T* d=NULL) { size_t s=dim1*dim2;
01296
01297     if (data) { WB_DELETE(data); }
01298     if (s) {
01299         WB_NEW(data,s);
01300         MEM_COPY<T>(data,s,d);
01301     }
01302     else data=NULL;
01303 };
01304
01305 void RENEW(
01306     const size_t &d1, const size_t &d2, const T* dd=NULL, char iflag=1
01307 ){
01308     const size_t s=d1*d2; isdiag=0;
01309
01310     if (isref) {
01311         if (d1==0 || d2==0) {
01312             if (d1 || d2) wblog(FL,
01313                 "WRN wbMatrix is declared as reference "
01314                 "(&dx%d => %dx%d; %d)",dim1,dim2,d1,d2,isref
01315             );
01316
01317             dim1=d1; dim2=d2; isref=0;
01318
01319             data=NULL;
01320
01321             return;
01322         }
01323         else {
01324             wblog(FL,"ERR wbMatrix is declared as reference (&dx%d; %d)",
01325                 d1,d2, isref);
01326         }
01327     }
01328
01329     if (d1!=dim1 || d2!=dim2) {
01330
01331         dim1=d1; dim2=d2; if (data) {
01332             if (dd==data) wblog(FL,
01333                 "ERR init space equals *this (use Resize instead)");

```

```

01334         WB_DELETE(data);
01335     }
01336     if (d1==0 || d2==0) return;
01337     WB_NEW(data,s);
01338 }
01339
01340     if (!iflag && !dd) return;
01341     if (s) MEM_COPY<T>(data,s,dd);
01342 };
01343
01344 private:
01345
01346     bool isSym_aux(
01347         const wbMatrix &B, double eps, double *xref,
01348         const char symflag='s'
01349     ) const;
01350 };
01351
01352 template <class T>
01353 class wbRecs {
01354
01355     const T* data;
01356     size_t n,lda; bool asc, lex;
01357
01358 public:
01359
01360     wbRecs(const wbMatrix<T> &A, wbperm &P, char dir_=1, char lex_=1)
01361         : data(A.data), n(A.dim1), lda(A.dim2),
01362         asc(dir_> 0 ? 1 : 0), lex(lex_> 0 ? 1 : 0)
01363     {
01364         P.init(n);
01365     };
01366
01367     ~wbRecs() {};
01368
01369     bool operator()(const size_t &pa, const size_t &pb) const {
01370
01371         if (pa>n || pb>n) wblog(FL,
01372             "ERR %s() index out of bounds (%d,%d; %d)",FCT,pa,pb,n);
01373
01374         const T *a=data+pa*lda, *b=data+pb*lda;
01375         if (lex) { size_t i=0;
01376             if (asc) {
01377                 for (; i<lda; ++i) {
01378                     if (a[i]<b[i]) return 1;
01379                     if (a[i]>b[i]) return 0;
01380                 }
01381             }
01382             else {
01383                 for (; i<lda; ++i) {
01384                     if (a[i]<b[i]) return 0;
01385                     if (a[i]>b[i]) return 1;
01386                 }
01387             }
01388         }
01389         else { size_t i=lda-1;
01390             if (asc) {
01391                 for (; i<lda; --i) {
01392                     if (a[i]<b[i]) return 1;
01393                     if (a[i]>b[i]) return 0;
01394                 }
01395             }
01396             else {
01397                 for (; i<lda; --i) {
01398                     if (a[i]<b[i]) return 0;
01399                     if (a[i]>b[i]) return 1;
01400                 }
01401             }
01402         }
01403         if (asc) return (pa<pb);
01404         else return (pb<pa);
01405     };
01406 };
01407
01408 template <class T>
01409 void getSortPerm_OMP(
01410     const wbMatrix<T> &A, wbperm &P, char dir, char lex);
01411
01412 template <class T>
01413 bool wbMatrix<T>::deepEqualP(const wbMatrix<T> &B) const {
01414     if (dim1!=B.dim1 || dim2!=B.dim2) return 0;
01415
01416     if (data!=B.data) {
01417         size_t i=0, n=numel();
01418         for (; i<n; ++i) { if (data[i]==B.data[i]) continue;
01419             if (!data[i] || !B.data[i]) return 0; else
01420             if (!(*data[i]==(*B.data[i]))) return 0;

```

```

01421     }
01422 }
01423
01424     return 1;
01425 };
01426
01427 template <class T> inline
01428 bool wbMatrix<T>::isDiag() const {
01429     size_t i,j, l=0;
01430
01431     for (i=0; i<dim1; ++i)
01432     for (j=0; j<dim2; ++j, ++l) { if (i!=j && data[l]) return 0; }
01433
01434     return 1;
01435 };
01436
01437 template <class T>
01438 bool wbMatrix<T>::isDiagMatrix(T eps) const {
01439
01440     if (!dim1 || dim1!=dim2) return 0;
01441     size_t i,j, l=0;
01442
01443     for (i=0; i<dim1; ++i)
01444     for (j=0; j<dim2; ++j, ++l) { if (i!=j && Wb::abs(data[l])>eps) return 0; }
01445
01446     return 1;
01447 };
01448
01449 template <class T>
01450 bool wbMatrix<T>::isIdentity(double eps, double &maxdiff) const {
01451
01452     if (!dim1 || dim1!=dim2) return 0;
01453     size_t i,j,l=0; double d; maxdiff=0;
01454
01455     for (i=0; i<dim1; ++i)
01456     for (j=0; j<dim2; ++j, ++l) {
01457         d = ABS( data[l] - (i!=j ? 0 : 1) );
01458         if (maxdiff<d) maxdiff=d;
01459     }
01460
01461     return (maxdiff<eps);
01462 };
01463
01464 template <class T>
01465 bool wbMatrix<T>::isProptoId(T &x, T eps) const {
01466
01467     if (!dim1 || dim1!=dim2) return 0;
01468     size_t i,j,l=0;
01469
01470     for (i=0; i<dim1; ++i)
01471     for (j=0; j<dim2; ++j, ++l) {
01472         if (Wb::abs(data[l] - (i!=j ? 0 : data[0]))>eps) { return 0; }
01473     }
01474     x=data[0]; return 1;
01475 };
01476
01477 template <class T> inline
01478 bool wbMatrix<T>::isSym_aux(
01479     const wbMatrix<T> &B, double eps, double *xref,
01480     const char symflag
01481 ) const {
01482
01483     const wbMatrix<T> &A>(*this);
01484     char issame = (&A==&B);
01485     size_t i,j,k;
01486     double x;
01487
01488     if (A.dim1!=B.dim2 || A.dim2!=B.dim1) return 0;
01489     if (xref) *xref=0;
01490
01491     if (symflag=='s') {
01492         if (eps==0) {
01493             for (i=0; i<dim1; ++i) { k = (issame ? i : 0);
01494             for (j=k; j<dim2; ++j) {
01495                 if (A(i,j)!=Wb::CONJ(B(j,i))) {
01496                     if (xref) {
01497                         x = Wb::abs(A(i,j) - Wb::CONJ(B(j,i)));
01498                         xref[0] = MAX(*xref,x);
01499                     }
01500                     else return 0;
01501                 }
01502             }
01503         }
01504     }
01505     else {
01506         for (i=0; i<dim1; ++i) { k = (issame ? i : 0);
01507         for (j=k; j<dim2; ++j) {
01508             x = Wb::abs(A(i,j) - Wb::CONJ(B(j,i)));

```

```

01508         if (x>eps) { if (xref) xref[0]=MAX(*xref,x); else return 0; }
01509     }}
01510 }
01511 }
01512 else if (symflag=='a') {
01513     if (eps==0) {
01514         for (i=0; i<dim1; ++i) { k = (issame ? i : 0);
01515             for (j=k; j<dim2; ++j) {
01516                 if (A(i,j)!=-Wb::CONJ(B(j,i))) {
01517                     if (xref) {
01518                         x = Wb::abs(A(i,j) + Wb::CONJ(B(j,i)));
01519                         xref[0] = MAX(*xref,x);
01520                     }
01521                     else return 0;
01522                 }
01523             }}
01524     }
01525     else {
01526         for (i=0; i<dim1; ++i) { k = (issame ? i : 0);
01527             for (j=k; j<dim2; ++j) {
01528                 x = Wb::abs(A(i,j) + Wb::CONJ(B(j,i)));
01529                 if (x>eps) { if (xref) xref[0]=MAX(*xref,x); else return 0; }
01530             }}
01531     }
01532 }
01533 else wblog(FL,"ERR invalid symflag=%c<%d>",symflag,symflag);
01534
01535 if (xref && (*xref)>eps) return 0;
01536
01537 return 1;
01538 }
01539
01540 template <class T> inline
01541 bool wbMatrix<T>::isComplex() const { return 0; };
01542
01543 template<> inline
01544 bool wbMatrix<wbcomplex>::isComplex() const {
01545     for (size_t s=dim1*dim2, i=0; i<s; ++i)
01546         if (data[i].i!=0.) return 1;
01547
01548     return 0;
01549 }
01550
01551 template <class T> inline
01552 bool wbMatrix<T>::isUnique() const {
01553
01554     wbMatrix<T> Q(*this);
01555     wbperm P;
01556
01557     if (!dim2) { return (dim1<=1); }
01558
01559     Q.SortRecs(P);
01560     return Q.isUniqueSorted();
01561 };
01562
01563 template <class T> inline
01564 bool wbMatrix<T>::isUniqueSorted(char dir, char lex) const {
01565
01566     if (dim1<=1 || !dim2) return 1;
01567     size_t i=1; char c=0;
01568
01569     if (!dir) { if (dim1<=2) return 1;
01570                 c=recCompare(1,0,-1,lex); if (!c) { return 0; }
01571                 i=2;
01572             }
01573     else if (abs(int(dir))>1) {
01574         if (dir=='A') c=+1; else
01575         if (dir=='D') c=-1; else
01576         wblog(FL,"ERR %s() invalid c='%c'<%d> !?",FCT,c,c);
01577     }
01578     else { c=dir; }
01579
01580     for (; i<dim1; ++i) {
01581         if (recCompare(i,i-1,-1,lex)!=c) return 0;
01582     }
01583
01584     return 1;
01585 };
01586
01587 template <class T> inline
01588 int wbMatrix<T>::isSorted(char dir, char lex, size_t m) const {
01589
01590     if (!dim1 || !dim2) { return -1; }
01591     if (dim1==1) { return 3; }
01592
01593     size_t j=1, ndeg=0; char c=0;
01594

```

```

01595     if (!dir) {
01596         for (; j<dim1; ++j, ++ndeg) {
01597             if ((c=recCompare(j,j-1,m,lex))) { dir=c; ++j; break; }}
01598     }
01599     else if (abs(int(dir))>1) {
01600         wblog(FL,"ERR %s() invalid direction s=%d",FCT,dir);
01601     }
01602
01603     for (; j<dim1; ++j) {
01604         if ((c=recCompare(j,j-1,m,lex))) { if (c!=dir) return 0; }
01605         else { ++ndeg; }
01606     }
01607
01608     return (1 | (ndeg ? 0 : 2) | (c<0 ? 4 : 0));
01609 };
01610
01611 template <class T> inline
01612 wbMatrix<T>& wbMatrix<T>::ResizeXRecs(size_t nr) {
01613
01614     no_ref_(FLF);
01615
01616     if (nr) {
01617         if (nr<=dim1 && nr/double(dim1)>0.10) dim1=nr;
01618         else Resize(nr,dim2);
01619     }
01620     else init(0,dim2);
01621
01622     return *this;
01623 };
01624
01625 template <class T> inline
01626 wbMatrix<T>& wbMatrix<T>::resize(
01627     size_t m, size_t n, wbMatrix<T> &M
01628 ) const {
01629
01630     if (this==&M) return M.Resize(m,n);
01631
01632     M.init(m,n);
01633     m=MIN(m,dim1); n=MIN(n,dim2); if (m==0 || n==0) return M;
01634
01635     for (size_t i=0; i<m; ++i)
01636         MEM_COPY<T>(M.data+i*M.dim2, n, data+i*dim2);
01637
01638     return M;
01639 };
01640
01641 template <class T> inline
01642 wbMatrix<T>& wbMatrix<T>::Resize(size_t nr, size_t nc, const T* dx) {
01643
01644     size_t d1,d2,s=nr*nc; T *d0=data;
01645     no_ref_(FLF);
01646
01647     d1=MIN(dim1,nr); d2=MIN(dim2,nc);
01648
01649     if (dim1==nr && dim2==nc) return *this;
01650     if (d1==0 || d2==0) { RENEW(nr,nc); return *this; }
01651
01652     WB_NEW(data,s);
01653
01654     if (nc==dim2) {
01655         MEM_COPY<T>(data,d1*dim2,d0);
01656         if (nr>dim1) {
01657             if (dx)
01658                 MEM_COPY<T>(data+dim1*dim2, (nr-dim1)*dim2,dx);
01659             else MEM_SET<T>(data+dim1*dim2, (nr-dim1)*dim2);
01660         }
01661     }
01662     else {
01663         size_t i; T *p=data, *p0=d0;
01664         for (i=0; i<d1; ++i, p+=nc, p0+=dim2) {
01665             MEM_COPY<T>(p,d2,p0); if (d2<nc) {
01666                 MEM_SET<T>(p+d2,nc-d2); }
01667         }
01668         if (nr>dim1) {
01669             if (dx) wblog(FL,"ERR %s() "
01670                 "allows no reference data if dim2=%d->%d",FCT,dim2,nc);
01671             MEM_SET<T>(data+dim1*dim2, (nr-dim1)*dim2);
01672         }
01673     }
01674
01675     dim1=nr; dim2=nc;
01676     WB_DELETE(d0);
01677
01678     return *this;
01679 };
01680
01681 template <class T> inline

```

```

01682 wbMatrix<T>& wbMatrix<T>::Resize2Mult(const WBINDEX &M) {
01683
01684     size_t m=M.sum();
01685
01686     if (M.len!=dim1) wblog(FL,
01687         "ERR %s() size mismatch (%d/%dx%d)",FCT,M.len,dim1,dim2);
01688     if (!m || !dim2) { return init(m,dim2); } else
01689     if (int(m)<0) wblog(FL,"ERR %s() got m=%d !?",FCT,m);
01690
01691     size_t i,j;
01692     wbMatrix<T> X(m,dim2);
01693     const T *x0=data; T *x=X.data;
01694
01695     for (i=0; i<M.len; ++i, x0+=dim2) { m=M.data[i];
01696     for (j=0; j<m; ++j, x+=dim2) MEM_COPY<T>(x, dim2, x0); }
01697
01698     return X.save2(*this);
01699 };
01700
01701 template <class T> inline
01702 wbMatrix<T>& wbMatrix<T>::unRef() {
01703
01704     if (!isref) return *this;
01705     if (!dim1 || !dim2) {
01706         if (data) wblog(FL,"ERR data=%lX (%dx%d) !?",data,dim1,dim2);
01707         isref=0; return *this;
01708     }
01709
01710     size_t s=dim1*dim2;
01711     T const* const d0=data;
01712
01713     WB_NEW(data,s);
01714
01715     MEM_COPY<T>(data,s,d0);
01716     isref=0;
01717
01718     return *this;
01719 };
01720
01721 template <class T> inline
01722 wbMatrix<T>& wbMatrix<T>::transpose(wbMatrix<T> &M) const {
01723
01724     if (this==&M) return M.Transpose();
01725
01726     size_t i,j,k=0; M.init(dim2,dim1);
01727
01728     if (isdiag) {
01729         M.isdiag=1; k=MIN(dim1,dim2);
01730         for (i=0; i<k; ++i) M.data[i*dim1+i]=data[i*dim2+i];
01731         return M;
01732     }
01733
01734     for (j=0; j<dim2; ++j)
01735     for (i=0; i<dim1; ++i) M.data[k++]=data[i*dim2+j];
01736
01737     return M;
01738 };
01739
01740 template <class T> inline
01741 wbMatrix<T>& wbMatrix<T>::Reshape(size_t nr, size_t nc) {
01742
01743     if (dim1!=nr || dim2!=nc) {
01744         if (dim1*dim2!=nr*nc) wblog(FL,
01745             "ERR %s() size not preserved (%dx%d => %dx%d)",
01746             FCT,dim1,dim2,nr,nc
01747         );
01748         dim1=nr; dim2=nc;
01749     }
01750     return *this;
01751 };
01752
01753 template <class T> inline
01754 wbMatrix<T>& wbMatrix<T>::add(
01755     const wbMatrix<T> &M,
01756     const T& c,
01757     const char zflag
01758 ){
01759     size_t i,s=dim1*dim2;
01760
01761     if (data==NULL && zflag) {
01762         RENEW(M.dim1, M.dim2, M.data); (*this)*=c;
01763         isdiag=M.isdiag;
01764         return *this;
01765     }
01766
01767     if (dim1!=M.dim1 || dim2!=M.dim2)
01768         wbdie(FL,"Dimension mismatch.");

```

```

01769
01770     if (isdiag) if (!M.isdiag) isdiag=0;
01771
01772     if (c==T(+1)) { for (i=0; i<s; ++i) data[i]+=M.data[i]; } else
01773     if (c==T( 0)) { return *this; } else
01774     if (c==T(-1)) { for (i=0; i<s; ++i) data[i]-=M.data[i]; }
01775     else { for (i=0; i<s; ++i) data[i]+=(c * M.data[i]); }
01776
01777     return *this;
01778 }
01779
01780 template <class T> inline
01781 wbMatrix<T>& wbMatrix<T>::minus(
01782     const wbMatrix<T> &M,
01783     const T& c,
01784     const char zflag
01785 ){
01786     size_t i,s=dim1*dim2;
01787
01788     if (data==NULL && zflag) {
01789         RENEW(M.dim1, M.dim2, M.data); (*this)*=(-c);
01790         isdiag=M.isdiag;
01791         return *this;
01792     }
01793
01794     if (dim1!=M.dim1 || dim2!=M.dim2)
01795         wbdie(FL,"Dimension mismatch.");
01796
01797     if (isdiag) if (!M.isdiag) isdiag=0;
01798
01799     if (c==T(+1)) { for (i=0; i<s; ++i) data[i]-=M.data[i]; } else
01800     if (c==T( 0)) { return *this; } else
01801     if (c==T(-1)) { for (i=0; i<s; ++i) data[i]+=M.data[i]; }
01802     else { for (i=0; i<s; ++i) data[i]-=(c * M.data[i]); }
01803
01804     return *this;
01805 }
01806
01807 template <class T> inline
01808 wbMatrix<T>& wbMatrix<T>::cat(const unsigned dim,
01809     const wbMatrix<T> &M2
01810 ){
01811     const wbMatrix<T>* M0[] = {this, &M2};
01812     wbvector< wbMatrix<T> const* > M; M.init(2,M0);
01813     return CAT(dim,M);
01814 };
01815
01816 template <class T> inline
01817 wbMatrix<T>& wbMatrix<T>::cat(const unsigned dim,
01818     const wbMatrix<T> &M2, const wbMatrix<T> &M3
01819 ){
01820     const wbMatrix<T>* M0[] = {this, &M2, &M3};
01821     wbvector< wbMatrix<T> const* > M; M.init(3,M0);
01822     return CAT(dim,M);
01823 };
01824
01825 template <class T> inline
01826 wbMatrix<T>& wbMatrix<T>::cat(const unsigned dim,
01827     const wbMatrix<T> &M2, const wbMatrix<T> &M3, const wbMatrix<T> &M4
01828 ){
01829     const wbMatrix<T>* M0[] = {this, &M2, &M3, &M4};
01830     wbvector< wbMatrix<T> const* > M; M.init(4,M0);
01831     return CAT(dim,M);
01832 };
01833
01834 template <class T> inline
01835 wbMatrix<T>& wbMatrix<T>::Cat(const unsigned dim,
01836     const wbMatrix<T> &M1, const wbMatrix<T> &M2
01837 ){
01838     wbMatrix<T> const* M0[] = {&M1, &M2};
01839     wbvector< wbMatrix<T> const* > M; M.init(2,M0);
01840     return CAT(dim,M);
01841 };
01842
01843 template <class T> inline
01844 wbMatrix<T>& wbMatrix<T>::Cat(const unsigned dim,
01845     const wbMatrix<T> &M1, const wbMatrix<T> &M2,
01846     const wbMatrix<T> &M3
01847 ){
01848     wbMatrix<T> const* M0[] = {&M1, &M2, &M3};
01849     wbvector< wbMatrix<T> const* > M; M.init(3,M0);
01850     return CAT(dim,M);
01851 };
01852
01853 template <class T> inline
01854 wbMatrix<T>& wbMatrix<T>::Cat(const unsigned dim,
01855     const wbMatrix<T> &M1, const wbMatrix<T> &M2,

```



```

01856     const wbMatrix<T> &M3, const wbMatrix<T> &M4
01857 ){
01858     wbMatrix<T> const* M0[] = {&M1, &M2, &M3, &M4};
01859     wbvector< wbMatrix<T> > const* > M; M.init(4,M0);
01860     return CAT(dim,M);
01861 };
01862
01863 template <class T> inline
01864 wbMatrix<T>& wbMatrix<T>::CAT(const unsigned dim,
01865     wbMatrix<T> const* M[], const unsigned len
01866 ){
01867     wbvector< wbMatrix<T> > const* > MM;
01868     MM.init(len,M);
01869
01870     return CAT(dim,MM);
01871 };
01872
01873 template <class T> inline
01874 wbMatrix<T>& wbMatrix<T>::CAT(const unsigned dim,
01875     const wbvector< wbMatrix<T> > &M0
01876 ){
01877     wbvector< wbMatrix<T> > const* > M(M0.len);
01878     for (unsigned i=0; i<M0.len; ++i) M[i]=&M0[i];
01879
01880     return CAT(dim,M);
01881 };
01882
01883 template <class T> inline
01884 wbMatrix<T>& wbMatrix<T>::CAT(const unsigned dim,
01885     const wbvector< wbvector<T> > &V0
01886 ){
01887     wbvector< wbMatrix<T> > M0(V0.len);
01888     wbvector< wbMatrix<T> > const* > M(V0.len);
01889
01890     for (size_t i=0; i<V0.len; ++i) {
01891         if (dim==1)
01892             M0[i].init2ref(1,V0[i].len,V0[i].data);
01893         else M0[i].init2ref(V0[i].len,1,V0[i].data);
01894
01895         M[i]=&M0[i];
01896     }
01897
01898     return CAT(dim,M);
01899 };
01900
01901 template <class T> inline
01902 wbMatrix<T>& wbMatrix<T>::CAT(const unsigned dim,
01903     const wbvector< wbvector<T> > const* > &V0
01904 ){
01905     wbvector< wbMatrix<T> > M0(V0.len);
01906     wbvector< wbMatrix<T> > const* > M(V0.len);
01907
01908     for (size_t i=0; i<V0.len; ++i) {
01909         if (dim==1)
01910             M0[i].init2ref(1,V0[i]->len,V0[i]->data);
01911         else M0[i].init2ref(V0[i]->len,1,V0[i]->data);
01912
01913         M[i]=&M0[i];
01914     }
01915
01916     return CAT(dim,M);
01917 };
01918
01919 template <class T>
01920 wbMatrix<T>& wbMatrix<T>::CAT(const unsigned dim,
01921     wbvector< wbMatrix<T> > const* > M
01922 ){
01923     wbMatrix<T> X;
01924     size_t i,k;
01925
01926     if (isdiag) { wblog(FL,"WRN unsetting isdiag"); isdiag=0; }
01927
01928     if (M.len==0) { this->init(); return *this; }
01929
01930     for (k=i=0; i<M.len; ++i) {
01931         if (M[i]==this) {
01932             if (!k++) { this->save2(X); }
01933             M[i]=&X;
01934         }
01935     }
01936
01937     for (k=i=0; i<M.len; ++i) {
01938         if (M[i] && (M[i]->dim1 || M[i]->dim2)) {
01939             if (k<i) { M[k]=M[i]; } ++k;
01940             if (M[i]->isdiag) wblog(FL,
01941                 "ERR %s() got diagonal matrix (i=%d)",FCT, i+1
01942             );

```

```

01943     }
01944 }
01945 if (k<M.len) {
01946     if (k) M.len=k;
01947     else { this->init(); return *this; }
01948 }
01949
01950 if (dim==1) {
01951     size_t m,d1,D1,d2=0;
01952
01953     for (D1=k=0; k<M.len; ++k) { D1+=(M[k]->dim1);
01954         if (k==0) d2=(M[k]->dim2); else
01955             if (d2!=(M[k]->dim2)) wblog(FL,
01956                 "ERR dimension mismatch (%d/%d)",d2,M[k]->dim2
01957             );
01958     }
01959
01960     this->init(D1,d2);
01961
01962     for (m=k=0; k<M.len; ++k) { d1=(M[k]->dim1);
01963         MEM_COPY<T>(data+d2*m, d1*d2, M[k]->data);
01964         m+=d1;
01965     }
01966 }
01967 else if (dim==2) {
01968     size_t i,m,d1=0,d2,D2;
01969     T *p, *p2;
01970
01971     for (D2=k=0; k<M.len; ++k) { D2+=(M[k]->dim2);
01972         if (k==0) d1=(M[k]->dim1); else
01973             if (d1!=(M[k]->dim1)) wblog(FL,
01974                 "ERR dimension mismatch (%d/%d)",d1,M[k]->dim1
01975             );
01976     }
01977
01978     this->init(d1,D2);
01979
01980     for (m=k=0; k<M.len; ++k) { d2=(M[k]->dim2);
01981         p=data+m; p2=(M[k]->data);
01982
01983         for (i=0; i<d1; ++i)
01984             MEM_COPY<T>(p+D2*i, d2, p2+d2*i);
01985
01986         m+=d2;
01987     }
01988 }
01989 else wblog(FL,"ERR invalid cat dim=%d",dim);
01990
01991 return *this;
01992 };
01993
01994 template <class T>
01995 wbMatrix<T>& wbMatrix<T>::CAT(
01996     const unsigned dim,
01997     wbvector< wbMatrix<T> const* > M,
01998     wbvector< WBINDEX const* > I
01999 ){
02000     wbMatrix<T> X;
02001     size_t i,k;
02002
02003     if (M.len!=I.len) wblog(FL,
02004         "ERR %s() size mismatch %d/%d",FCT,M.len,I.len);
02005     if (!M.len) { this->init(); return *this; }
02006
02007     for (k=i=0; i<M.len; ++i) {
02008         if (M[i]==this) {
02009             if (!(k++)) { this->save2(X); }
02010             M[i]=&X;
02011         }
02012     }
02013
02014     for (k=i=0; i<M.len; ++i) {
02015         if (M[i] && (M[i]->dim1 || M[i]->dim2 || I[i]->len)) {
02016             if (k<i) { M[k]=M[i]; I[k]=I[i]; }; ++k;
02017             if (M[i]->isdiag) wblog(FL,
02018                 "ERR %s() got diagonal matrix (i=%d)",FCT, i+1
02019             );
02020         }
02021     }
02022     if (k<M.len) {
02023         if (k) { M.len=k; I.len=k; }
02024         else { this->init(); return *this; }
02025     }
02026
02027     if (isdiag) isdiag=0;
02028
02029     if (dim==1) {

```

```

02030     size_t d1,d2=0, D1=0;
02031
02032     for (k=0; k<M.len; ++k) { D1+=(I[k]->len);
02033         if (k==0) d2=(M[k]->dim2); else
02034             if (d2!=(M[k]->dim2)) wblog(FL,
02035                 "ERR dimension mismatch (vcat: dim2=%d/%d)",d2,M[k]->dim2
02036             );
02037     }
02038
02039     this->init(D1,d2);
02040     T *x=data;
02041
02042     for (k=0; k<M.len; ++k) {
02043         const T* x0=M[k]->data;
02044         const widx_t *Ik=I[k]->data; D1=M[k]->dim1; d1=I[k]->len;
02045
02046         for (i=0; i<d1; ++i, x+=d2) {
02047             if (Ik[i]>=D1) wblog(FL,
02048                 "ERR %s() index out of bounds (%d/%d)",FCT,Ik[i]+1,D1);
02049             MEM_COPY<T>(x, d2, x0+Ik[i]*d2);
02050         }
02051     }
02052 }
02053 else if (dim==2) {
02054     size_t i,d1=0,d2,D1,D2=0;
02055
02056     for (k=0; k<M.len; ++k) { D2+=(d2=M[k]->dim2);
02057         if (k==0) d1=(I[k]->len); else
02058             if (d1!=(I[k]->len)) wblog(FL,
02059                 "ERR dimension mismatch (%d/%d)",d1,I[k]->len
02060             );
02061     }
02062
02063     this->init(d1,D2);
02064     T *x=data;
02065
02066     for (k=0; k<M.len; ++k) {
02067         const T* x0=M[k]->data;
02068         const widx_t *Ik=I[k]->data; D1=M[k]->dim1; d2=(M[k]->dim2);
02069
02070         for (i=0; i<d1; ++i) {
02071             if (Ik[i]>=D1) wblog(FL,
02072                 "ERR %s() index out of bounds (%d/%d)",FCT,Ik[i]+1,D1);
02073             MEM_COPY<T>(x+i*D2, d2, x0+Ik[i]*d2);
02074         }; x+=d2;
02075     }
02076 }
02077 else wblog(FL,"ERR invalid cat dim=%d",dim);
02078
02079 return *this;
02080 };
02081
02082 template <class T> inline
02083 void wbMatrix<T>::split(
02084     const unsigned dim,
02085     wbvector< wbMatrix<T>* > &M) const {
02086
02087     split(dim,M.data,M.len);
02088     return;
02089 }
02090
02091 template <class T> inline
02092 void wbMatrix<T>::split(
02093     const unsigned dim,
02094     wbMatrix<T> &M1,
02095     wbMatrix<T> &M2,
02096     wbMatrix<T> &M3
02097 ) const {
02098
02099     wbMatrix<T>* M[] = {&M1,&M2,&M3};
02100     split(dim,M,3);
02101     return;
02102 }
02103
02104 template <class T>
02105 void wbMatrix<T>::split(
02106     const unsigned dim, wbMatrix<T>** M, const unsigned len
02107 )const {
02108
02109     unsigned k;
02110
02111     for (k=0; k<len; ++k) {
02112         if (M[k]==this)
02113             wbdie(FL, "This matrix itself is included in split list!");
02114     }
02115
02116     if (dim==1) {

```

```

02117     size_t m,d1,D1;
02118
02119     for (D1=k=0; k<len; ++k) D1+=(*M[k]).dim1;
02120
02121     if (D1!=dim1)
02122         wbdie(FL, "Dimension mismatch.");
02123
02124     for (k=0; k<len; ++k) {
02125         if (M[k]->dim2!=dim2) M[k]->init(M[k]->dim1, dim2);
02126     }
02127
02128     for (m=k=0; k<len; ++k) { d1=M[k]->dim1;
02129         MEM_CPY<T>(M[k]->data, d1*dim2, data+dim2*m);
02130         m+=d1;
02131     }
02132 }
02133 else if (dim==2) {
02134     size_t i,m,d2,D2;
02135     T *p, *p2;
02136
02137     for (D2=k=0; k<len; ++k) D2+=M[k]->dim2;
02138
02139     if (D2!=dim2)
02140         wblog(FL,"ERR dimension mismatch (%d/%d)",D2,dim2);
02141
02142     for (k=0; k<len; ++k) {
02143         if (M[k]->dim1!=dim1) M[k]->init(dim1,M[k]->dim2);
02144     }
02145
02146     for (m=k=0; k<len; ++k) {
02147         d2=M[k]->dim2; p=data+m; p2=M[k]->data;
02148
02149         for (i=0; i<dim1; ++i)
02150             MEM_CPY<T>(p2+d2*i, d2, p+D2*i);
02151
02152         m+=d2;
02153     }
02154 }
02155 else wblog(FL,"ERR invalid cat index (%d).", dim);
02156 };
02157
02158 template <class T> inline
02159 int wbMatrix<T>::recLess(size_t i1, size_t i2, char lex) const {
02160     size_t i;
02161     T *r1=data+i1*dim2, *r2=data+i2*dim2;
02162
02163     if (i1>=dim1 || i2>=dim1) wblog(FL,
02164         "ERR %s() index out of bounds (%d,%d/%d)",FCT,i1,i2,dim1);
02165
02166     if (lex>0) {
02167         for (i=0; i<dim2; ++i)
02168             if (r1[i]<r2[i]) return 1; else
02169                 if (r1[i]>r2[i]) return 0;
02170     }
02171     else {
02172         for (i=dim2-1; i<dim2; --i)
02173             if (r1[i]<r2[i]) return 1; else
02174                 if (r1[i]>r2[i]) return 0;
02175     }
02176
02177     return 0;
02178 }
02179 }
02180
02181 template <class T> inline
02182 int wbMatrix<T>::recLessE(size_t i1, size_t i2, char lex) const {
02183     size_t i;
02184     T *r1=data+i1*dim2, *r2=data+i2*dim2;
02185
02186     if (i1>=dim1 || i2>=dim1) wblog(FL,
02187         "ERR %s() index out of bounds (%d,%d/%d)",FCT,i1,i2,dim1);
02188
02189     if (lex) {
02190         for (i=0; i<dim2; ++i)
02191             if (r1[i]<r2[i]) return 1; else
02192                 if (r1[i]>r2[i]) return 0;
02193     }
02194     else {
02195         for (i=dim2-1; i<dim2; --i)
02196             if (r1[i]<r2[i]) return 1; else
02197                 if (r1[i]>r2[i]) return 0;
02198     }
02199
02200     return 1;
02201 }
02202 }
02203

```

```

02204 template <class T> inline
02205 int wbMatrix<T>::recEqual(size_t i1, const T *d) const {
02206     if (i1>=dim1) wblog(FL,
02207         "ERR %s() index out of bounds (%d/%d)",FCT,i1,dim1);
02208     if (data==d) return 1;
02209     else return (!memcmp(data+i1*dim2, d, dim2*sizeof(T)));
02210 };
02211
02212 template <class T> inline
02213 int wbMatrix<T>::recEqual(size_t i1, size_t i2) const {
02214     if (i1>=dim1 || i2>=dim1) wblog(FL,
02215         "ERR %s() index out of bounds (%d,%d/%d)",FCT,i1,i2,dim1);
02216     if (i1==i2) return 1;
02217     else return (!memcmp(data+i1*dim2, data+i2*dim2, dim2*sizeof(T)));
02218 };
02219
02220 template <class T> inline
02221 T wbMatrix<T>::recDiff2(size_t i1, size_t i2, size_t n) const {
02222     if (i1>=dim1 || i2>=dim1) wblog(FL,
02223         "ERR %s() index out of bounds (%d,%d/%d)",i1,i2,dim1);
02224     if (i1==i2) return 0;
02225     return Wb::rangeNormDiff2(
02226         data+i1*dim2, data+i2*dim2, int(n)<0 ? dim2 : n
02227     );
02228 };
02229
02230 template <class T> inline
02231 T wbMatrix<T>::recDiff2P(size_t i1, const T* b, size_t n) const {
02232     if (i1>=dim1) wblog(FL,
02233         "ERR %s() index out of bounds (%d/%d)",i1,dim1);
02234     return Wb::rangeNormDiff2(
02235         data+i1*dim2, b, int(n)<0 ? dim2 : n
02236     );
02237 };
02238
02239 template <class T> inline
02240 char wbMatrix<T>::recCompare(
02241     size_t i1, size_t i2,
02242     size_t m,
02243     char lex,
02244     T eps
02245 ) const {
02246
02247     if (i1>=dim1 || i2>=dim1) wblog(FL,
02248         "ERR %s() index out of bounds (%d,%d/%d)",FCT,i1,i2,dim1);
02249     if (!dim2) wblog(FL,
02250         "ERR %s() got nothing to compare (dim2=%d)",FCT,dim2);
02251
02252     if (int(m)<0) m=dim2; else
02253     if (!m || m>dim2) wblog(FL,"ERR size out of bounds (%d/%d)",m,dim2);
02254
02255     i1*=dim2; i2*=dim2;
02256     if (lex<=0) {
02257         i1+=(dim2-m);
02258         i2+=(dim2-m);
02259     }
02260
02261     if (eps<=T(0))
02262         return Wb::recCompare(data+i1, data+i2, m, lex);
02263     else return Wb::recCompare(data+i1, data+i2, m, lex, eps);
02264 };
02265
02266 template <class T> inline
02267 char wbMatrix<T>::recCompare(
02268     size_t i1, const wbvector<T> &v, char lex
02269 ) const {
02270
02271     if (v.len!=dim2) wblog(FL,
02272         "ERR %s() size mismatch (%d,%d)",FCT,v.len,dim2);
02273     return Wb::recCompare(data+i1*dim2, v.data, dim2, lex);
02274 };
02275
02276 template <class T> inline
02277 char wbMatrix<T>::recCompareP(
02278     size_t i1, const T* v,
02279     size_t m,
02280     char lex,
02281     T eps
02282 ) const {
02283
02284     if (i1>=dim1) wblog(FL,
02285         "ERR %s() index out of bounds (%d/%d)",FCT,i1,dim1);
02286     if (!dim2) wblog(FL,
02287         "ERR %s() got nothing to compare (dim2=%d)",FCT,dim2);
02288
02289     if (int(m)<0) m=dim2; else
02290     if (!m || m>dim2) wblog(FL,"ERR size out of bounds (%d/%d)",m,dim2);

```

```

02291
02292     il*=dim2;
02293     if (lex<=0) il+=(dim2-m);
02294
02295     return Wb::recCompare(data+il, v, m, lex, eps);
02296 };
02297
02298 template <class T> inline
02299 wbvector<T>& wbMatrix<T>::recProd(wbvector<T> &pp) const {
02300     size_t i=0, j; T *d=data, p;
02301
02302     pp.init(dim1);
02303
02304     if (!dim2) wblog(FL,"ERR %s() records of length 0",FCT);
02305
02306     for (; i<dim1; ++i, d+=dim2) {
02307         for (p=d[0], j=1; j<dim2; ++j) { p*=d[j]; } pp[i]=p; }
02308
02309     return pp;
02310 };
02311
02312 template <class T> inline
02313 T wbMatrix<T>::recProd(size_t i) const {
02314
02315     if (i>=dim1 || dim2==0) { wblog(FL,
02316         "ERR %s() got empty record length (%dx%d; i)",FCT,dim1,dim2,i);
02317         return 0;
02318     }
02319
02320     const T *d=data+i*dim2; T p=d[0];
02321     for (size_t j=1; j<dim2; ++j) { p*=d[j]; }
02322     return p;
02323 };
02324
02325 template <class T>
02326 inline T wbMatrix<T>::colSum(size_t c) const {
02327     T *d=data+c, x=0;
02328
02329     if (c>=dim2) wblog(FL,"ERR index out of bounds (%d/%d)", c, dim2);
02330     if (dim1==0) wblog(FL,
02331         "WRN Sum over column of length zero (%d/%d).", c, dim2);
02332
02333     for (size_t i=0; i<dim1; ++i, d+=dim2) x+=(*d);
02334     return x;
02335 }
02336
02337 template <class T> inline
02338 T wbMatrix<T>::recSum(size_t i) const {
02339
02340     if (i>=dim1) wblog(FL,
02341         "ERR %s() index out of bounds (%d,%dx%d)",FCT,i+1,dim1,dim2);
02342     if (!dim2) wblog(FL,
02343         "ERR %s() got empty object (%d,%dx%d)",i,dim1,dim2);
02344
02345     return Wb::addRange(rec(i),dim2);
02346 };
02347
02348 template <class T> inline
02349 wbvector<T>& wbMatrix<T>::recSum(wbvector<T> &s) const {
02350
02351     s.init(dim1);
02352     if (!dim2) { if (dim1) wblog(FL,
02353         "WRN %s() got empty object (%dx%d)",FCT,dim1,dim2);
02354         return s;
02355     }
02356
02357     size_t i=0; const T* d=data;
02358     for (; i<dim1; ++i, d+=dim2) { s[i]=Wb::addRange(d,dim2); }
02359
02360     return s;
02361 };
02362
02363 template <class T> inline
02364 wbvector<T>& wbMatrix<T>::recSumA(wbvector<T> &s) const {
02365
02366     s.init(dim1);
02367     if (!dim2) { if (dim1) wblog(FL,
02368         "WRN %s() got empty object (%dx%d)",FCT,dim1,dim2);
02369         return s;
02370     }
02371
02372     size_t i=0; const T* d=data; double q;
02373     for (; i<dim1; ++i, d+=dim2) {
02374         q=Wb::addRange2(d,dim2);
02375         s[i]=std::sqrt(q);
02376     }
02377

```

```

02378     return s;
02379 };
02380
02381 template <class T>
02382 wbMatrix<T>& wbMatrix<T>::setRand(double fac, double shift) {
02383     size_t i,s=dim1*dim2;
02384     static char first_call=1;
02385
02386     if (first_call) { wb_srand(); first_call=0; }
02387     fac/=(double)RAND_MAX; isdiag=0;
02388
02389     if (shift==0.)
02390         for (i=0; i<s; ++i) data[i]=(T)(fac*rand());
02391     else for (i=0; i<s; ++i) data[i]=(T)(fac*rand()+shift);
02392
02393     return *this;
02394 }
02395
02396 template<>
02397 wbMatrix<wbcomplex>& wbMatrix<wbcomplex>::setRand(double fac, double shift) {
02398     size_t i,s=dim1*dim2;
02399     static char first_call=1;
02400
02401     if (first_call) { wb_srand(); first_call=0; }
02402     fac/=(double)RAND_MAX; isdiag=0;
02403
02404     if (shift==0.) {
02405         for (i=0; i<s; ++i)
02406             data[i].set(fac*rand(), fac*rand());
02407     }
02408     else {
02409         for (i=0; i<s; ++i)
02410             data[i].set(fac*rand()+shift, fac*rand()+shift);
02411     }
02412
02413     return *this;
02414 }
02415
02416 template<>
02417 wbMatrix<wbcomplex>& wbMatrix<wbcomplex>::Symmetrize() {
02418     size_t i,j,r,s;
02419     if (dim1!=dim2) wblog(FL,
02420         "ERR Symmetrize() called with %dx%d matrix !?",dim1,dim2);
02421
02422     for (j=0; j<dim2; ++j)
02423         for (i=j+1; i<dim1; ++i) { r=i*dim2+j; s=j*dim2+i;
02424             data[r]=data[s]=wbcomplex(
02425                 0.5*(data[r].r+data[s].r),
02426                 0.5*(data[r].i-data[s].i)
02427             );
02428             data[s].Conj();
02429         }
02430
02431     return *this;
02432 };
02433
02434 template <class T>
02435 inline void wbMatrix<T>::setDiagRand(double fac, double shift) {
02436     size_t i, s=MIN(dim1,dim2);
02437     static char first_call=1;
02438
02439     if (data==NULL) return;
02440
02441     if (first_call) { wb_srand(); first_call=0; }
02442     fac/=(double)RAND_MAX;
02443
02444     MEM_SET<T>(data,dim1*dim2);
02445
02446     if (shift==0.)
02447         for (i=0; i<s; ++i) data[i*dim2+i]=(T)(fac*rand());
02448     else for (i=0; i<s; ++i) data[i*dim2+i]=(T)(fac*rand()+shift);
02449
02450     isdiag=1;
02451 }
02452
02453 template <class T>
02454 mxArray* wbMatrix<T>::toMx_base_d(char raw) const {
02455     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
02456     return Mx::Array<double>(d1,d2,!raw).copyFromTR(data);
02457 };
02458
02459 template <class T>
02460 mxArray* wbMatrix<T>::toMx_base(char raw) const {
02461     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
02462     return Mx::Array<double>(d1,d2,!raw).copyFromTR(data);
02463 };
02464

```

```

02465 template <>
02466 mxArray* wbMatrix<wbcomplex>::toMx_base(char raw) const {
02467     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
02468     return Mx::Array<wbcomplex>(d1,d2,!raw).copyFromTR(data);
02469 };
02470
02471 template <>
02472 mxArray* wbMatrix<char>::toMx_base(char raw) const {
02473     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
02474     return Mx::Array<mxChar>(d1,d2,!raw).copyFromTR(data);
02475 };
02476
02477 template <>
02478 mxArray* wbMatrix<long>::toMx_base(char raw) const {
02479     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
02480     if (double(aMax())<1E15)
02481         return Mx::Array<double>(d1,d2,!raw).copyFromTR(data);
02482     else return Mx::Array<int64_T>(d1,d2,!raw).copyFromTR(data);
02483 };
02484
02485 template <>
02486 mxArray* wbMatrix<unsigned long>::toMx_base(char raw) const {
02487     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
02488     if (double(aMax())<1E15)
02489         return Mx::Array<double>(d1,d2,!raw).copyFromTR(data);
02490     else return Mx::Array<uint64_T>(d1,d2,!raw).copyFromTR(data);
02491 };
02492
02493 template <class T> inline
02494 mxArray* wbMatrix<T>::toMx(char raw) const {
02495     if (raw) wblog(FL,
02496         "WRN raw flag will be ignored for typeid=%s",TSTR(T));
02497     return toMx_Struct();
02498 };
02499
02500 template <class T> inline
02501 mxArray* wbMatrix<T>::toMxP(char flag) const {
02502
02503     if (!flag || flag=='s' || flag=='S') { return toMxP_S(); }
02504     if (flag=='c' || flag=='C') { return toMxP_C(); }
02505
02506     wblog(FL,"ERR %s() invalid flag (%s)",FCT,cSTR(flag));
02507     return 0;
02508 };
02509
02510 template <> inline
02511 mxArray* wbMatrix<unsigned>::toMx(char raw) const {
02512     return toMx_base(raw);
02513 };
02514 template <> inline
02515 mxArray* wbMatrix<unsigned>::toMxT(char raw) const {
02516     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
02517     return Mx::Array<uint32_T>(d1,d2,!raw).copyFromTR(data);
02518 };
02519
02520 template <> inline
02521 mxArray* wbMatrix<int>::toMx(char raw) const {
02522     return toMx_base(raw);
02523 };
02524 template <> inline
02525 mxArray* wbMatrix<int>::toMxT(char raw) const {
02526     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
02527     return Mx::Array<int32_T>(d1,d2,!raw).copyFromTR(data);
02528 };
02529
02530 template <> inline
02531 mxArray* wbMatrix<unsigned long>::toMx(char raw) const {
02532     return toMx_base(raw);
02533 };
02534 template <> inline
02535 mxArray* wbMatrix<unsigned long>::toMxT(char raw) const {
02536     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
02537     return Mx::Array<uint64_T>(d1,d2,!raw).copyFromTR(data);
02538 };
02539
02540 template <> inline
02541 mxArray* wbMatrix<long>::toMx(char raw) const {
02542     return toMx_base(raw);
02543 };
02544 template <> inline
02545 mxArray* wbMatrix<long>::toMxT(char raw) const {
02546     size_t d1=(raw ? dim2:dim1), d2=(raw ? dim1:dim2);
02547     return Mx::Array<int64_T>(d1,d2,!raw).copyFromTR(data);
02548 };
02549
02550 template <> inline
02551 mxArray* wbMatrix<unsigned char>::toMx(char raw) const {

```



```

02552     return toMx_base(row);
02553 };
02554 template <> inline
02555 mxArray* wbMatrix<unsigned char>::toMxT(char row) const {
02556     size_t d1=(row ? dim2:dim1), d2=(row ? dim1:dim2);
02557     return Mx::Array<uint8_T>(d1,d2,!row).copyFromTR(data);
02558 };
02559
02560 template <> inline
02561 mxArray* wbMatrix<char>::toMx(char row) const {
02562     return toMx_base(row);
02563 };
02564 template <> inline
02565 mxArray* wbMatrix<char>::toMxT(char row) const {
02566     size_t d1=(row ? dim2:dim1), d2=(row ? dim1:dim2);
02567     return Mx::Array<int8_T>(d1,d2,!row).copyFromTR(data);
02568 };
02569
02570 template <> inline
02571 mxArray* wbMatrix<double>::toMx(char row) const {
02572     return toMx_base(row);
02573 };
02574
02575 template <> inline
02576 mxArray* wbMatrix<wbcomplex>::toMx(char row) const {
02577     return toMx_base(row);
02578 };
02579
02580 template<class T>
02581 mxArray* wbMatrix<T>::toMx_Struct() const {
02582
02583     mxArray *S=data->mxCreateStruct(dim1,dim2);
02584
02585     size_t i,j;
02586     for (i=0; i<dim1; ++i)
02587         for (j=0; j<dim2; ++j) data[j+i*dim2].add2MxStruct(S,i+j*dim1);
02588
02589     return S;
02590 };
02591
02592 template<class T>
02593 mxArray* wbMatrix<T>::toMxP_S() const {
02594
02595     mxArray *S;
02596     if (data) { size_t i,j;
02597         S=(*data)->mxCreateStruct(dim1,dim2);
02598
02599         for (i=0; i<dim1; ++i)
02600             for (j=0; j<dim2; ++j) data[j+i*dim2]->add2MxStruct(S,i+j*dim1);
02601     }
02602     else { S=mxCreateStructMatrix(dim1,dim2,0,NULL); }
02603
02604     return S;
02605 };
02606
02607 template<class T>
02608 mxArray* wbMatrix<T>::toMxP_C() const {
02609
02610     mxArray *S;
02611
02612     if (data) { size_t i,j;
02613         S=(*data)->mxCreateCell(dim1,dim2);
02614
02615         for (i=0; i<dim1; ++i)
02616             for (j=0; j<dim2; ++j) data[j+i*dim2]->add2MxCell(S,i+j*dim1);
02617     }
02618     else { S=mxCreateCellMatrix(dim1,dim2); }
02619
02620     return S;
02621 };
02622
02623 template<class T>
02624 mxArray* wbMatrix<T>::mxCreateStruct(unsigned m, unsigned n) const {
02625     return mxCreateCellMatrix(m,n);
02626 }
02627
02628 template<class T>
02629 void wbMatrix<T>::add2MxStruct(mxArray *S, unsigned i, char tst) const {
02630
02631     if (tst) {
02632         size_t s=0;
02633         if (!S || (s=mxGetNumberOfElements(S))<1 || i>=s) wblog(FL,
02634             "ERR %s() must follow mxCreateCell()\n%lx, %d/%d",
02635             FCT,S,i+1,s);
02636     }
02637
02638     mxSetCell(S,i,toMx());

```

```

02639 }
02640
02641 template <class T>
02642 void wbMatrix<T>::info(const char *istr) const {
02643
02644     size_t l=0, n=16; char s[n];
02645     snprintf(s,n,"%lix%li",dim1,dim2);
02646     if (l>=n) wblog(FL,"ERR %s() string out of bounds (%d/%d)",FCT,l,n);
02647     fprintf(stdout," %12s %10s @ 0x%p double array\n", istr, s, data);
02648 };
02649
02650 template <class T>
02651 void wbMatrix<T>::print(const char *istr, char mflag) const {
02652
02653     mxArray *a=toMx_base(0);
02654
02655     if (istr && istr[0]) {
02656         if (mflag) { wb_printf("\n%s = [\n",istr); }
02657         else {
02658             wb_printf("\n%s = [%dx%d double]\n\n",istr,dim1,dim2);
02659         }
02660     } else { wb_printf("\n"); }
02661
02662     Wb::CallMatlab(0,NULL,1,&a,"disp");
02663     wb_printf("%s\n", mflag && istr && istr[0] ? " ":"");
02664
02665     mxDestroyArray(a);
02666 };
02667
02668 template <class T>
02669 void wbMatrix<T>::Print(const char *istr, char mflag) const {
02670
02671     size_t i,j;
02672
02673     if (!mflag) {
02674         if (istr[0])
02675             printf("\n%s = [%ldx%ld double]\n\n", istr, dim1, dim2);
02676         else printf("\n");
02677     }
02678     else
02679         printf("\n%s = [\n", istr[0] ? istr : "ans");
02680
02681     for (i=0; i<dim1; ++i) {
02682         for (j=0; j<dim2; ++j) printf(" %8.3g", (double)data[i*dim2+j]);
02683         printf("\n");
02684     }
02685
02686     if (mflag) printf("];\n");
02687 }
02688
02689 template <class T>
02690 void wbMatrix<T>::printdata(
02691     const char *istr, const char *dbl_fmt, const char *rsep
02692 ) const {
02693
02694     size_t i=0,j;
02695     if (istr && istr[0]) printf("%s = [\n ",istr); else printf("[");
02696
02697     for (; i<dim1; ++i) { if (i) printf("%s ",rsep);
02698         for (j=0; j<dim2; ++j) { printf(dbl_fmt, double(data[i*dim2+j])); }
02699
02700         printf(" ]\n");
02701     }
02702
02703 template <class T>
02704 void wbMatrix<T>::appendRow(const wbvector<T> &v) {
02705
02706     if (dim1) {
02707         if (dim2!=v.len) wblog(FL,
02708             "ERR Dimension mismatch (%dx%d : %d)", dim1,dim2, v.len);
02709
02710         appendRows(1, v.data);
02711     }
02712     else RENEW(1,v.len,v.data);
02713 };
02714
02715 template <class T>
02716 void wbMatrix<T>::appendRows(size_t n, const T* v) {
02717
02718     size_t s=dim1*dim2, ds=n*dim2;
02719     T *d0=data;
02720
02721     dim1+=n; if (!n || !dim2) return;
02722
02723     WB_NEW(data,s+ds);
02724     MEM_COPY<T>(data,s+ds,s,d0,v);
02725     WB_DELETE(d0);

```

```

02726 };
02727
02728 template<class T>
02729 void wbMatrix<T>::getReal(wbMatrix<double> &R) const {
02730     if (typeid(T) != typeid(double)) {
02731         R.init(dim1, dim2);
02732         for (size_t n=dim1*dim2, i=0; i<n; ++i) { R.data[i]=double(data[i]); }
02733     }
02734     else { R.init(*this); }
02735     return;
02736 };
02737
02738 template<class T>
02739 void wbMatrix<T>::getImag(wbMatrix<double> &I QS_UNUSED_VAR) const {
02740     wblog(FL, "ERR wbMatrix::getImag not defined for type %s.", TSTR(T));
02741 };
02742
02743 template<class T>
02744 void wbMatrix<T>::set(
02745     const wbMatrix<double> &R QS_UNUSED_VAR,
02746     const wbMatrix<double> &I QS_UNUSED_VAR
02747 ) {
02748     wblog(FL, "ERR wbMatrix::set(R,I) not defined for type %s.", TSTR(T));
02749 };
02750
02751 template<class T>
02752 wbMatrix<T>& wbMatrix<T>::Conj() { return *this; };
02753
02754 template<class T>
02755 double wbMatrix<T>::normReal() const { return Wb::sqrt(norm2()); };
02756
02757 template<class T>
02758 double wbMatrix<T>::normImag() const { return 0; };
02759
02760 template<>
02761 double wbMatrix<wbcomplex>::normReal() const {
02762     double x=0;
02763     for (size_t n=dim1*dim2, i=0; i<n; ++i) x+=Wb::norm2(data[i].r);
02764     return Wb::sqrt(x);
02765 };
02766
02767 template<>
02768 double wbMatrix<wbcomplex>::normImag() const {
02769     double x=0;
02770     for (size_t n=dim1*dim2, i=0; i<n; ++i) x+=Wb::norm2(data[i].i);
02771     return Wb::sqrt(x);
02772 };
02773
02774 template<>
02775 wbMatrix<wbcomplex>& wbMatrix<wbcomplex>::Conj() {
02776     for (size_t s=dim1*dim2, i=0; i<s; ++i)
02777         data[i].i = -data[i].i;
02778     return *this;
02779 }
02780
02781 template<>
02782 void wbMatrix<wbcomplex>::getReal(wbMatrix<double> &R) const {
02783     R.init(dim1, dim2);
02784
02785     if (isdiag) {
02786         size_t i,k,s=MIN(dim1,dim2);
02787         for (i=0; i<s; ++i) { k=i*dim2+i; R.data[k]=data[k].r; }
02788         R.isdiag=isdiag;
02789     }
02790     else {
02791         for (size_t s=dim1*dim2, i=0; i<s; ++i)
02792             R.data[i]=data[i].r;
02793     }
02794 }
02795
02796 template<>
02797 void wbMatrix<wbcomplex>::getImag(wbMatrix<double> &I) const {
02798     I.init(dim1, dim2);
02799
02800     if (isdiag) {
02801         size_t i,k,s=MIN(dim1,dim2);
02802         for (i=0; i<s; ++i) { k=i*dim2+i; I.data[k]=data[k].i; }
02803         I.isdiag=isdiag;
02804     }
02805     else {
02806         for (size_t s=dim1*dim2, i=0; i<s; ++i)
02807             I.data[i]=data[i].i;
02808     }
02809 }
02810
02811 template<>
02812 void wbMatrix<wbcomplex>::set(

```

```

02813     const wbMatrix<double> &R,
02814     const wbMatrix<double> &I
02815 ){
02816     if (!R.sameSize(I)) wblog(FL,
02817     "ERR Dimension mismatch (%dx%d; %dx%d)!", R.dim1, R.dim2, I.dim1, I.dim2);
02818
02819     init(R.dim1, R.dim2);
02820
02821     for (size_t s=dim1*dim2, i=0; i<s; ++i)
02822     data[i].set(R.data[i], I.data[i]);
02823 }
02824
02825 template <class T> inline
02826 void matchIndexU(const char *F, int L,
02827     const wbMatrix<T> &QA, const wbMatrix<T> &QB,
02828     wbindex &Ib, const char force=1
02829 );
02830
02831 template<class T>
02832 int matchIndex(C_TMAT &QA, C_TMAT &QB, wbindex &Ia, wbindex &Ib,
02833     char lex=1,
02834     widx_t *ma=NULL, widx_t *mb=NULL,
02835     T eps=0
02836 );
02837
02838 template <class T> inline
02839 int matchIndex(
02840     const wbvector<T> &qA, const wbvector<T> &qB,
02841     wbindex &Ia, wbindex &Ib,
02842     char lex=1,
02843     widx_t *ma=NULL, widx_t *mb=NULL,
02844     T eps=0
02845 ){
02846     wbMatrix<T> QA, QB; int r;
02847     QA.init2ref(qA, 't');
02848     QB.init2ref(qB, 't'); if (QA.dim2!=1 || QB.dim2!=1) wblog(FL, "ERR");
02849
02850     r=matchIndex(QA, QB, Ia, Ib, lex, ma, mb, eps);
02851     return r;
02852 };
02853
02854 template <class T> inline
02855 int matchSortedIdx(
02856     const T *da, size_t lda, size_t na,
02857     const T *db, size_t ldb, size_t nb, wbindex &Ia, wbindex &Ib,
02858     widx_t m=-1,
02859     char lex=1
02860 ){
02861     widx_t m1=0, m2=0;
02862     return Wb::matchSortedIdx(da, lda, na, db, ldb, nb, Ia, Ib, m, lex, &m1, &m2);
02863 };
02864
02865 template <class T> inline
02866 int matchSortedIdx(C_TMAT &QA, C_TMAT &QB, wbindex &Ia, wbindex &Ib,
02867     widx_t m=-1,
02868     char lex=1,
02869     widx_t *ma=NULL, widx_t *mb=NULL,
02870     T eps=0
02871 ){
02872     return Wb::matchSortedIdx(
02873         QA.data, QA.dim2, QA.dim1,
02874         QB.data, QB.dim2, QB.dim1, Ia, Ib, m, lex, ma, mb, eps
02875     );
02876 };
02877
02878 template <class T> inline
02879 int matchSortedIdx(C_TMAT &QA, C_TMAT &QB,
02880     wbindex &Ia, wbindex &Ib, wbvector<widx_t> &D,
02881     widx_t m=-1,
02882     char lex=1,
02883     T eps=0
02884 ){
02885     return Wb::matchSortedIdx(
02886         QA.data, QA.dim2, QA.dim1,
02887         QB.data, QB.dim2, QB.dim1, Ia, Ib, D, m, lex, eps
02888     );
02889 };
02890
02891 template <class T> inline
02892 int matchSortedIdxU(
02893     const char *F, int L,
02894     C_TMAT &QA, C_TMAT &QB, wbindex &Ia, wbindex &Ib,
02895     widx_t m=-1, char lex=1
02896 );
02897
02898 template <class T> inline
02899 int matchSortedIdxU(

```

```

02900      C_TMAT &QA, C_TMAT &QB, wbindex &Ia, wbindex &Ib, widx_t m=-1
02901  ){ return matchSortedIdxU(FL,QA,QB,Ia,Ib,m); };
02902
02903 #endif
02904

```

5.47 wbMatrix_blas.cc

```

00001  /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : wbMatrix (matrix class, row-major)
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *      http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_BLAS_CC__
00021 #define __WB_BLAS_CC__
00022
00023 // ----- //
00024 // ----- //
00025 // wrapper routine
00026
00027 template<class TA, class TB, class TC>
00028 void MMDIAG(
00029     const wbMatrix<TA> &A,
00030     const wbMatrix<TB> &B, wbMatrix<TC> &C,
00031     char aflag='N', char bflag='N',
00032     const TA afac=1., const TC cfac=0.
00033 );
00034
00035 template<class TA, class TB, class TC>
00036 wbMatrix<TC>& MMDIAG(
00037     const wbvector<TA> &A,
00038     const wbMatrix<TB> &B, wbMatrix<TC> &C,
00039     char aflag='N', char bflag='N',
00040     const TA afac=1., const TC cfac=0.
00041 );
00042
00043 template<class TA, class TB, class TC>
00044 wbMatrix<TC>& MMDIAG(
00045     const wbMatrix<TA> &A,
00046     const wbvector<TB> &B, wbMatrix<TC> &C,
00047     char aflag='N', char bflag='N',
00048     const TA afac=1., const TC cfac=0.
00049 );
00050
00051 template<class TA, class TB, class TC>
00052 void DZGEMM_VEC(
00053     const wbMatrix<TA> &A,
00054     const wbvector<TB> &B,
00055     wbvector<TC> &C, int m, int n,
00056     char aflag='N', const TA afac=1., const TC cfac=0.
00057 );
00058
00059 template<class TD>
00060 void DZGEMM(
00061     const wbMatrix<TD> &A,
00062     const wbMatrix<TD> &B, wbMatrix<TD> &C,
00063     const unsigned k,
00064     char aflag='N', char bflag='N',
00065     const TD afac=1., const TD cfac=0.
00066 );
00067
00068 template<>
00069 inline void DZGEMM<double>(
00070     const wbMatrix<double> &A,
00071     const wbMatrix<double> &B, wbMatrix<double> &C,
00072     const unsigned k,
00073     char aflag, char bflag,
00074     const double afac, const double cfac
00075 ){

```

```

00076
00077 #ifdef WB_CLOCK
00078     Wb::Clock clk("dgemm",1);
00079     if (aflag=='N')
00080         stat_dgemm.account(A.dim1*A.dim2*(A.dim2==B.dim1 ? B.dim2 : B.dim1));
00081     else stat_dgemm.account(A.dim1*A.dim2*(A.dim1==B.dim1 ? B.dim2 : B.dim1));
00082 #endif
00083
00084     dgemm(bflag, aflag, int(C.dim2), int(C.dim1), int(k),
00085         afac, B.data, int(B.dim2),
00086         A.data, int(A.dim2),
00087         cfac, C.data, int(C.dim2)
00088     );
00089 }
00090
00091 template<>
00092 inline void DZGEMM<wbcomplex>(
00093     const wbMatrix<wbcomplex> &A,
00094     const wbMatrix<wbcomplex> &B, wbMatrix<wbcomplex> &C,
00095     const unsigned k,
00096     char aflag, char bflag,
00097     const wbcomplex afac, const wbcomplex cfac
00098 ){
00099     #ifdef WB_CLOCK
00100         Wb::Clock clk("zgemm",1);
00101         if (bflag=='N')
00102             stat_dgemm.account(A.dim1*A.dim2*(A.dim2==B.dim1 ? B.dim2 : B.dim1));
00103         else stat_dgemm.account(A.dim1*A.dim2*(A.dim1==B.dim1 ? B.dim2 : B.dim1));
00104     #endif
00105
00106     zgemm (
00107         bflag, aflag, (int)C.dim2, (int)C.dim1, (int)k,
00108         afac, B.data, (int)B.dim2, A.data, (int)A.dim2,
00109         cfac, C.data, (int)C.dim2
00110     );
00111
00112     #ifdef WB_CLOCK
00113     {
00114         Wb::Clock *clk;
00115         if (clk.stop())
00116             { Wb::Clock("zgeNX",1,0,&Wb::Clocks,&clk,0); }
00117         else { Wb::Clock("zgeNN",1,0,&Wb::Clocks,&clk,0); }
00118
00119         clk.ncall+=2;
00120         clk.tcpu+= (long unsigned)
00121             (double(A.dim1*A.dim2*B.dim2)*(4E-9*CLOCKS_PER_SEC)+1.5);
00122     }
00123     #endif
00124 }
00125
00126 namespace Wb {
00127
00128 void MatProd(
00129     const wbMatrix<double> &A,
00130     const wbMatrix<wbcomplex> &B, wbMatrix<wbcomplex> &C,
00131     char aflag='N', char bflag='N',
00132     const double afac=1., const double cfac=0.,
00133     const char i00flag=0
00134 ){
00135     wbMatrix<double> R,I, rC, iC;
00136
00137     B.getReal(R);
00138     B.getImag(I);
00139
00140     if (bflag=='C') I.flipSign();
00141
00142     if (cfac!=0.) { C.getReal(rC); C.getImag(iC); }
00143
00144     MatProd(A, R, rC, aflag, bflag, afac, cfac, i00flag);
00145     MatProd(A, I, iC, aflag, bflag, afac, cfac, i00flag);
00146
00147     C.set(rC,iC);
00148 };
00149
00150 void MatProd(
00151     const wbMatrix<wbcomplex> &A,
00152     const wbMatrix<double> &B, wbMatrix<wbcomplex> &C,
00153     char aflag='N', char bflag='N',
00154     const double afac=1., const double cfac=0.,
00155     const char i00flag=0
00156 ){
00157     wbMatrix<double> R,I, rC, iC;
00158
00159     A.getReal(R);
00160     A.getImag(I);
00161
00162     if (aflag=='C') I.flipSign();

```

```

00163
00164     if (cfac!=0.) { C.getReal(rC); C.getImag(iC); }
00165
00166     MatProd(R, B, rC, aflag, bflag, afac, cfac, i00flag);
00167     MatProd(I, B, iC, aflag, bflag, afac, cfac, i00flag);
00168
00169     C.set(rC,iC);
00170 };
00171
00172 template<class T>
00173 void MatProd(
00174     const wbMatrix<T> &A,
00175     const wbMatrix<T> &B, wbMatrix<T> &C,
00176     char aflag, char bflag,
00177     const T afac, const T cfac,
00178     const char i00flag
00179 ){
00180     unsigned a1=A.dim1, a2=A.dim2, b1=B.dim1, b2=B.dim2;
00181     const char flags[]="NTCntc";
00182
00183     #ifdef WB_CLOCK
00184         Wb::Clock clk("MatProd",1);
00185     #endif
00186
00187     if (!strchr(flags,aflag) || !strchr(flags,bflag)) wblog(FL,
00188         "ERR %s() invalid flags %c<%d>, %c<%d>",
00189         FCT,aflag,aflag,bflag,bflag);
00190
00191     if (aflag!='N') SWAP(a1,a2);
00192     if (bflag!='N') SWAP(b1,b2);
00193
00194     if (a2!=b1) {
00195         wblog(FL, "ERR %s() dimension mismatch: (%d,%d) * (%d,%d) ?",
00196             FCT,a1,a2,b1,b2); return;
00197     }
00198
00199     if (cfac!=0.) {
00200         if (C.data==NULL && i00flag) {
00201             if (cfac!=1.) wblog(FL,
00202                 "WRN C = A*B + c*[] with c=%s !?", toStr(cfac).data);
00203             C.init(a1,b2);
00204         }
00205         else if (C.dim1!=a1 || C.dim2!=b2) { wblog(FL,
00206             "ERR %s() dimension mismatch: C={%d,%d}=? (%d,%d)",
00207             FCT,C.dim1,C.dim2,a1,b2); return;
00208         }
00209     }
00210
00211     if (a1==0 || b2==0) {
00212         if (cfac==0.) C.init(a1,b2);
00213         return;
00214     }
00215
00216     if (a2==0) { wblog(FL,
00217         "ERR %s() cannot multiply (%dx%d)*(%dx%d)!",
00218         FCT,a1,a2,b1,b2); return; }
00219
00220     if (&C==&A || &C==&B) { wblog(FL,
00221         "ERR I/O spaces must be distinct!\n[%71X %71X %71X]",
00222         &A, &B, &C); return; }
00223
00224     if (cfac==0.)
00225         C.init(a1,b2);
00226
00227     if (A.isdiag || B.isdiag)
00228         MMDIAG(A,B,C, aflag,bflag,afac,cfac);
00229     else DZGEMM(A,B,C,a2,aflag,bflag,afac,cfac);
00230 };
00231
00232 template<class TA, class TB, class TC>
00233 void MatProd(
00234     const wbMatrix<TA> &A,
00235     const wbvector<TB> &B, wbvector<TC> &C,
00236     char aflag, const TC afac, const TC cfac,
00237     const char i00flag
00238 ){
00239     unsigned m=A.dim1, n=A.dim2;
00240     if (aflag!='N') SWAP(m,n);
00241
00242     if (B.len!=n) wblog(FL,
00243         "ERR severe size mismatch Ax=b (%dx%d, %d)",m,n,B.len);
00244
00245     if (!C.data) {
00246         if (cfac!=0. && i00flag) wblog(FL,
00247             "WRN C = A*B + c*[] with c=%s !?",toStr(cfac).data);
00248         C.init(m);
00249     }

```

```

00250     else if (C.len!=m) {
00251         if (cfac==0.) C.init(m); else wblog(FL,
00252             "ERR severe size mismatch Ax=b (%d, %d)", m, C.len);
00253     }
00254
00255     DZGEMM_VEC(A,B,C,m,n,aflag,afac,cfac);
00256 };
00257
00258 };
00259
00260 template<>
00261 inline void DZGEMM_VEC(
00262     const wbMatrix<double> &A,
00263     const wbvector<double> &B,
00264     wbvector<double> &C, int m, int n,
00265     char aflag, const double afac, const double cfac
00266 ){
00267     dgemm (
00268         'N', aflag, 1, m, n, afac,
00269         B.data, 1, A.data, A.dim2, cfac,
00270         C.data, 1
00271     );
00272 }
00273
00274 template<>
00275 inline void DZGEMM_VEC(
00276     const wbMatrix<wbcomplex> &A,
00277     const wbvector<wbcomplex> &B,
00278     wbvector<wbcomplex> &C, int m, int n,
00279     char aflag, const wbcomplex afac, const wbcomplex cfac
00280 ){
00281     zgemm (
00282         'N', aflag, 1, m, n, afac,
00283         B.data, 1, A.data, A.dim2, cfac,
00284         C.data, 1
00285     );
00286 }
00287
00288 template<>
00289 inline void DZGEMM_VEC(
00290     const wbMatrix<double> &A,
00291     const wbvector<wbcomplex> &B,
00292     wbvector<wbcomplex> &C, int m, int n,
00293     char aflag, const double afac, const wbcomplex cfac
00294 ){
00295     wbvector<double> b,cr,ci;
00296
00297     if (cfac && cfac!=1.) { C*=cfac; }
00298
00299     B.getReal(b); C.getReal(cr); DZGEMM_VEC(A,b,cr,m,n,aflag,afac,1.);
00300     B.getImag(b); C.getImag(ci); DZGEMM_VEC(A,b,ci,m,n,aflag,afac,1.);
00301
00302     C.set(cr,ci);
00303 };
00304
00305 inline void DZGEMM_aux(
00306     const wbMatrix<double> &A,
00307     const wbMatrix<double> &B, wbMatrix<double> &C,
00308     const unsigned k,
00309     char aflag='N', char bflag='N',
00310     const double afac=1., const double cfac=0.
00311 ){
00312     if (A.isdiag || B.isdiag)
00313         MMDIAG(A, B, C, aflag, bflag, afac, cfac);
00314     else DZGEMM(A, B, C, k, aflag, bflag, afac, cfac);
00315 }
00316
00317 template<class TA, class TB, class TC>
00318 inline void MMDIAG(
00319     const wbMatrix<TA> &A,
00320     const wbMatrix<TB> &B, wbMatrix<TC> &C,
00321     char aflag, char bflag,
00322     const TA afac, const TC cfac
00323 ){
00324     unsigned i, s=C.dim1*C.dim2, dmax;
00325
00326     if (!A.isdiag && !B.isdiag) wblog(FL,
00327         "ERR Wrong call - elements not diagonal.");
00328
00329     if (A.isdiag && A.dim1>1 && A.dim2>1)
00330         if (A(1,0)!=0. || A(0,1)!=0.) wblog(FL,
00331             "ERR Input A is NOT diag even though isdiag=%d", A.isdiag);
00332
00333     if (B.isdiag && B.dim1>1 && B.dim2>1)
00334         if (B(1,0)!=0. || B(0,1)!=0.) wblog(FL,
00335             "ERR Input B is NOT diag even though isdiag=%d", B.isdiag);
00336

```



```

00337     if (cfac!=1.) for (i=0; i<s; i++) C.data[i]*=cfac;
00338
00339     if (A.isdiag && !B.isdiag) {
00340         MMDIAG(A.getDiag(), B, C, aflag, bflag, afac);
00341     }
00342     else
00343     if ((!A.isdiag) && B.isdiag) {
00344         MMDIAG(A, B.getDiag(), C, aflag, bflag, afac);
00345     }
00346     else {
00347         wbvector<TA> adiag;
00348         wbvector<TB> bdiag;
00349         A.getDiag(adiag); B.getDiag(bdiag);
00350
00351         if (aflag=='C')
00352             for (i=0; i<adiag.len; i++) adiag[i]=Wb::conj(adiag[i]);
00353
00354         if (bflag=='C')
00355             for (i=0; i<bdiag.len; i++) bdiag[i]=Wb::conj(bdiag[i]);
00356
00357         dmax=MIN( MIN(A.dim1,A.dim2), MIN(B.dim1,B.dim2) );
00358         for (i=0; i<dmax; i++) C(i,i) += afac * adiag[i] * bdiag[i];
00359     }
00360
00361     if (C.isdiag) if (!A.isdiag || !B.isdiag) C.isdiag=0;
00362 }
00363
00364 template<class TA, class TB, class TC>
00365 inline wbMatrix<TC>& MMDIAG(
00366     const wbvector<TA> &A,
00367     const wbMatrix<TB> &B, wbMatrix<TC> &C,
00368     char aflag, char bflag,
00369     const TA afac, const TC cfac
00370 ){
00371     unsigned i,j, m=B.dim1, n=B.dim2;
00372     char acc,btr;
00373     TA dbl;
00374
00375     if (bflag=='C' && typeid(TB)!=typeid(wbcomplex)) bflag='T';
00376
00377     if (!strchr("NTC",aflag) || !strchr("NTC",bflag)) wblog(FL,
00378         "ERR Invalid flag(s) %c<%d>, %c<%d>",aflag,aflag,bflag,bflag);
00379
00380     acc=(aflag=='C');
00381     btr=(bflag!='N'); if (btr) SWAP(m,n);
00382
00383     if (A.len!=m) wblog(FL,
00384         "ERR Size mismatch (diag(%d) * %dx%d)", A.len,m,n);
00385     if (&C==&B) wblog(FL,"ERR Output space C equal to input space.");
00386
00387     if (C.isEmpty() || (cfac==0. && (C.dim1!=m || C.dim2!=n)))
00388         C.init(m,n);
00389     else {
00390         if (C.dim1!=m || C.dim2!=n) wblog(FL,
00391             "ERR Size mismatch (%dx%d; %dx%d)",C.dim1,C.dim2, m,n);
00392         if (cfac!=1.) {
00393             if (cfac==0.) C.reset();
00394             else C*=cfac;
00395         }
00396     }
00397
00398     for (i=0; i<C.dim1; i++) {
00399         dbl=A[i]; if (acc) dbl=Wb::conj(dbl);
00400         dbl*=afac; if (dbl==0.) continue;
00401
00402         switch (bflag) {
00403             case 'N': for (j=0; j<C.dim2; j++) C(i,j)+=(dbl*B(i,j)); break;
00404             case 'T': for (j=0; j<C.dim2; j++) C(i,j)+=(dbl*B(j,i)); break;
00405             case 'C': for (j=0; j<C.dim2; j++) C(i,j)+=(dbl*Wb::conj(B(j,i)));
00406         }
00407     }
00408
00409     return C;
00410 }
00411
00412 template<class TA, class TB, class TC>
00413 inline wbMatrix<TC>& MMDIAG(
00414     const wbMatrix<TA> &A,
00415     const wbvector<TB> &B, wbMatrix<TC> &C,
00416     char aflag, char bflag,
00417     const TA afac, const TC cfac
00418 ){
00419     unsigned i,j, m=A.dim1, n=A.dim2;
00420     char atr,bcc;
00421     TB dbl;
00422
00423     if (aflag=='C' && typeid(TA)!=typeid(wbcomplex)) aflag='T';

```

```

00424
00425     if (!strchr("NTC",aflag) || !strchr("NTC",bflag)) wblog(FL,
00426 "ERR Invalid flag(s) %c<%d>, %c<%d>",aflag,aflag,bflag,bflag);
00427
00428     atr=(aflag!='N'); if (atr) SWAP(m,n);
00429     bcc=(bflag=='C');
00430
00431     if (B.len!=n) wblog(FL,
00432 "ERR Size mismatch (%dx%d * diag(%d))", m,n, B.len);
00433     if (&C==&A) wblog(FL,"ERR Output space C equal to input space.");
00434
00435     if (C.isEmpty() || (cfac==0. && (C.dim1!=m || C.dim2!=n)))
00436         C.init(m,n);
00437     else {
00438         if (C.dim1!=m || C.dim2!=n) wblog(FL,
00439 "ERR Size mismatch (%dx%d; %dx%d)",C.dim1,C.dim2, m,n);
00440         if (cfac!=1.) {
00441             if (cfac==0.) C.reset();
00442             else C*=cfac;
00443         }
00444     }
00445
00446     for (j=0; j<C.dim2; j++) {
00447         dbl=B[j]; if (bcc) dbl=Wb::conj(dbl);
00448         dbl*=afac; if (dbl==0.) continue;
00449
00450         switch (aflag) {
00451             case 'N': for (i=0; i<C.dim1; i++) C(i,j)+=(dbl*A(i,j)); break;
00452             case 'T': for (i=0; i<C.dim1; i++) C(i,j)+=(dbl*A(j,i)); break;
00453             case 'C': for (i=0; i<C.dim1; i++) C(i,j)+=(dbl*Wb::conj(A(j,i)));
00454         }
00455     }
00456
00457     return C;
00458 }
00459
00460 inline void WbEigenSymmetric (
00461     const wbMatrix<double> &M, wbMatrix<double> &V, wbvector<double> &E
00462 ){
00463     unsigned n=M.dim1; pINT ni=0, q=0;
00464     double nd=0;
00465
00466     if (M.dim1!=M.dim2) wblog(FL,
00467 "ERR WbEigenSymmetric requires square matrix (%d,%d).",
00468 M.dim1, M.dim2);
00469
00470     V=M; E.init(n); if (n==0) return;
00471
00472     dsyevd('V','U',n,V.data,n,E.data,&nd,-1,&ni,-1,q);
00473     if (nd<1 || ni<1 || q) wblog(FL,
00474 "ERR DSYEVD returned lwork=%g/%d (e=%d) !?",nd,ni,q);
00475
00476     wbvector<double> wd(nd);
00477     wbvector<pINT> wi(ni);
00478
00479     dsyevd(
00480         'V',
00481         'L',
00482         n,
00483         V.data,
00484         n,
00485         E.data,
00486         wd.data, wd.len,
00487         wi.data, wi.len,
00488         q
00489     );
00490
00491     if (q) wblog(FL,"ERR DSYEV returned e=%d !?", q);
00492 }
00493
00494 inline void WbEigenSymmetric (
00495     const wbMatrix<wbcomplex> &M, wbMatrix<wbcomplex> &V, wbvector<double> &E
00496 ){
00497     unsigned n=M.dim1; pINT q, ni=0;
00498     wbcomplex nz=0; double nr=0;
00499
00500     if (M.dim1!=M.dim2) wblog(FL,
00501 "ERR %s() requires square matrix (%d,%d)",FCT,M.dim1,M.dim2);
00502     if (!M.isHConj(1E-12,&nr)) wblog(FL,
00503 "ERR %s() got non-hermitian matrix (%.3g)",FCT,nr);
00504     nr=0;
00505
00506     V=M; E.init(n); if (!n) return;
00507
00508     zheevd('V','U',n,V.data,n,E.data,&nz,-1,&nr,-1,&ni,-1,q);
00509     if (nz.r<1 || nr<1 || ni<1 || q) wblog(FL,
00510 "ERR ZHEEVD returned lwork=%g/%g/%d (e=%d) !?",nz.r,nr,ni,q);

```

```

00511
00512     wbvector<wbcomplex> wz(nz.r);
00513     wbvector<double> wr(nr);
00514     wbvector<pINT> wi(ni);
00515
00516     zheevd(
00517         'V',
00518         'L',
00519         n,
00520         V.data,
00521         n,
00522         E.data,
00523         wz.data, wz.len,
00524         wr.data, wr.len,
00525         wi.data, wi.len,
00526         q
00527     );
00528
00529     if (q<0) { wblog(FL,"ERR %s() ZHEEVD returned e=%d",FCT,q); }
00530     if (q>0) {
00531         V=M; E.init(n);
00532
00533         wblog(FL,"WRN %s() ZHEEVD returned i=%d / falling back to ZHEEV",FCT,q);
00534         MXPut(FL,0,"tmpfile").add(M,"M").add(q,"q");
00535
00536         if (n<128) { ni=2*n-1; }
00537         else {
00538             ni=ilaenv(1,"zhetr", "U", (pINT)n, (pINT)n, (pINT)n, (pINT)n);
00539             if (ni>n) wblog(FL,"WRN %s() got lwork=(%d+2)*%d",FCT,ni,n);
00540             ni=(ni+1)*n;
00541         }
00542         if (size_t(ni)>wz.len) { wz.init(ni); }
00543
00544         nr=3*n-2; if (nr>wr.len) { wr.init(nr); }
00545
00546         zheev(
00547             'V',
00548             'L',
00549             n,
00550             V.data,
00551             n,
00552             E.data,
00553             wz.data, ni,
00554             wr.data,
00555             q
00556         );
00557         wblog(FL,"ERR ZHEEV returned e=%d", q);
00558     }
00559
00560     V.Conj();
00561 };
00562
00563 template<class T>
00564 inline void GESVD_M(
00565     wbMatrix<T> &A    QS_UNUSED_VAR,
00566     wbMatrix<T> &U    QS_UNUSED_VAR,
00567     wbvector<double> &S    QS_UNUSED_VAR,
00568     wbMatrix<T> &Vt    QS_UNUSED_VAR
00569 ){ wblog(FL,"ERR GESVD_M not defined for datatype %s",TSTR(T)); };
00570
00571 template<>
00572 inline void GESVD_M(
00573     wbMatrix<double> &A,
00574     wbMatrix<double> &U,
00575     wbvector<double> &S,
00576     wbMatrix<double> &Vt
00577 ){
00578     unsigned M=A.dim1, N=A.dim2, K=S.len;
00579     wbvector<double> wd;
00580     double nd;
00581     pINT q, ni=8*MIN(M,N);
00582     wbvector<pINT> wi(ni);
00583
00584     #ifdef WB_CLOCK
00585     Wb::Clock clk("dgesvd",1);
00586     #endif
00587
00588     dgesdd('S',N,M,A.data,N,S.data,Vt.data,N,U.data,K,&nd,-1,wi.data,q);
00589     if (nd<1 || q) wblog(FL,
00590         "ERR DGESDD returned lwork=%g (e=%d) !?",nd,q);
00591     wd.init(nd);
00592
00593     dgesdd(
00594         'S', N, M, A.data, N, S.data,
00595         Vt.data, N,
00596         U.data, K,
00597         wd.data, wd.len, wi.data,

```

```

00598     q
00599 );
00600
00601 if (q) wblog(FL, "ERR DGESDD returned e=%d !?", q);
00602 };
00603
00604 template<
00605 inline void GESVD_M(
00606     wbMatrix<wbcomplex> &A,
00607     wbMatrix<wbcomplex> &U,
00608     wbvector<double> &S,
00609     wbMatrix<wbcomplex> &Vt
00610 ){
00611     unsigned M=A.dim1, N=A.dim2, K=S.len;
00612     wbvector<wbcomplex> wz;
00613
00614     pINT q, n=MIN(M,N);
00615     wbcomplex nz;
00616
00617     wbvector<double> wd(n*(5*n+7));
00618     wbvector<pINT> wi(12*n);
00619
00620     if (!M || !N) wblog(FL, "ERR %s() got %dx%d matrix!?", FCT, M, N);
00621
00622 #ifdef WB_CLOCK
00623     Wb::Clock clk("zgesvd", 1);
00624 #endif
00625
00626     zgesdd(
00627         'S', N, M, A.data, N, S.data, Vt.data, N, U.data, K,
00628         &nz, -1, wd.data, wi.data, q);
00629     if (nz.r<1 || q) wblog(FL,
00630         "ERR ZGESDD() returned lwork=%g (e=%d) !?", nz.r, q);
00631     wz.init(nz.r);
00632
00633     zgesdd(
00634         'S', N, M, A.data, N, S.data, Vt.data, N, U.data, K,
00635         wz.data, wz.len, wd.data, wi.data,
00636         q
00637     );
00638
00639     if (q) wblog(FL, "ERR ZGESDD returned e=%d !?", q);
00640 };
00641
00642 template<class T>
00643 inline void wbSVD(
00644     const wbMatrix<T> &A0,
00645     wbMatrix<T> &U,
00646     wbvector<double> &S,
00647     wbMatrix<T> &Vt
00648 ){
00649     wbMatrix<T> A(A0);
00650
00651     unsigned M=A.dim1, N=A.dim2, K=MIN(M,N);
00652
00653     U.init(M,K); S.init(K); Vt.init(K,N);
00654     if (!M || !N) return;
00655
00656     GESVD_M(A, U, S, Vt);
00657
00658 #ifdef SVD_BUG_SAFE_GUARD
00659     { double a=Wb::sqrt(A0.norm2()), s=Wb::sqrt(S.norm2()), e=fabs(a-s)/a;
00660         if (e>1e-10) {
00661             wblog(FL, "WRN %s() need to fix SVD bug: %dx%d @ e=%3g !?", FCT, M, N, e);
00662         }
00663     }
00664 #endif
00665 };
00666
00667 #endif
00668

```

5.48 wbMatrix_blas.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : wbMatrix (matrix class, row-major)
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at

```

```

00010 *
00011 *      http://www.apache.org/licenses/LICENSE-2.0
00012 *
00013 * Unless required by applicable law or agreed to in writing, software
00014 * distributed under the License is distributed on an "AS IS" BASIS,
00015 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016 * See the License for the specific language governing permissions and
00017 * limitations under the License.
00018 * ----- */
00019
00020 #ifndef __WB_MATRIX_BLAS_HH__
00021 #define __WB_MATRIX_BLAS_HH__
00022
00023 // ----- //
00024 // ----- //
00025
00026 void WbEigenSymmetric (const wbMatrix<double>&M,
00027     wbMatrix<double>&U, wbvector<double> &E);
00028
00029 namespace Wb {
00030
00031 template<class T>
00032 void MatProd(
00033     const wbMatrix<T> &A,
00034     const wbMatrix<T> &B, wbMatrix<T> &C,
00035     char aflag='N', char bflag='N',
00036     const T afac=1., const T cfac=0.,
00037     const char i00flag=0
00038 );
00039
00040 template<class TA, class TB, class TC>
00041 void MatProd(
00042     const wbMatrix<TA> &A,
00043     const wbvector<TB> &B, wbvector<TC> &C,
00044     char aflag='N', const TA afac=1., const TC cfac=0.,
00045     const char i00flag=0
00046 );
00047
00048 template<class T>
00049 void SVD(
00050     const wbMatrix<T> &A,
00051     wbMatrix<T> &U,
00052     wbMatrix<double> &S,
00053     wbMatrix<T> &Vt
00054 );
00055
00056 };
00057
00058 #endif
00059

```

5.49 wbmpfr.hh

```

00001 /* -----
00002 * Project : QSpace tensor library (v4.0)
00003 * Class   : QSpace MPFR routines
00004 *
00005 * Copyright 2024 Andreas Weichselbaum
00006 *
00007 * Licensed under the Apache License, Version 2.0 (the "License");
00008 * you may not use this file except in compliance with the License.
00009 * You may obtain a copy of the License at
00010 *
00011 *      http://www.apache.org/licenses/LICENSE-2.0
00012 *
00013 * Unless required by applicable law or agreed to in writing, software
00014 * distributed under the License is distributed on an "AS IS" BASIS,
00015 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016 * See the License for the specific language governing permissions and
00017 * limitations under the License.
00018 * ----- */
00019
00020 #ifndef __WB_MPFR_HH__
00021 #define __WB_MPFR_HH__
00022
00023 /* ----- *
00024     wrapper class for MPFR (multiple precision floating point library)
00025     Wb, Feb01, 12
00026
00027 -----
00028 Docu: see $SRC/MPFR/doc/mpfr.pdf
00029 -----
00030

```

```

00031 MPFR basics
00032 * before assigning, MPFR variables need to be initialized.
00033 * when done with a variable, it needs to be cleared out.
00034 * variable will have the same allocated space during all its life.
00035
00036 * MPFR functions may create intermediate caches
00037   => use mpfr_free_cache() to free
00038   It is strongly advised to do that before terminating a thread
00039   to avoid memory leaks.
00040
00041 * The semantics of a calculation in MPFR is specified as follows:
00042   Compute the requested operation exactly (with "infinite accuracy"),
00043   and round the result to the precision of the destination variable,
00044   with the given rounding mode.
00045
00046 * if two variables are used to store only a few significant bits,
00047   and their product is stored in a variable with large precision,
00048   then MPFR will still compute the result with full precision.
00049
00050 -----
00051 General rules:
00052 * all MPFR functions expect output arguments before input arguments;
00053   typically: output var, input vars ..., rounding
00054 * ok to use the same variable for both input and output in the same expression
00055 * functions returning int: ternary value
00056   0: value stored in output is the exact result of given mathematical function.
00057   pos. (neg): value stored in output is greater (lower) than the exact result.
00058
00059 -----
00060 MPFR rounding modes (similar to GMP rounding modes
00061 GMP_RNDD, GMP_RNDU, GMP_RNDN, ...):
00062
00063   MPFR_RNDN: nearest: round to nearest (roundTiesToEven in IEEE 754-2008)
00064   MPFR_RNDZ: zero : round toward zero (roundTowardZero in IEEE 754-2008)
00065   MPFR_RNDU: up : round toward plus infity (roundTowardPositive in IEEE 754-2008)
00066   MPFR_RNDD: down : round toward minus infity (roundTowardNegative in IEEE 754-2008)
00067   MPFR_RNDA: away 0 : round away from zero.
00068
00069 -----
00070 Initialization routines can clearance
00071
00072   void mpfr_init(mpfr_t x)
00073   => initialize x to default precision and set its value to NaN
00074
00075   void mpfr_inits2(mpfr_t x1, mpfr_t x2, ..., (mpfr_ptr)0)
00076   => initialize set of variables to default precision
00077   => must be terminated by null-pointer (whose type must also be mpfr_ptr)
00078
00079   void mpfr_init2(mpfr_t x, mpfr_prec_t prec)
00080   void mpfr_inits2(mpfr_prec_t prec, mpfr_t x1, mpfr_t x2, ..., (mpfr_ptr)0)
00081   => same as previous init() functions, but with specified exact precision
00082
00083   void mpfr_clear(mpfr_t x)
00084   => free the space occupied by the significand of x.
00085   Make sure to call this function for all mpfr_t variables
00086   when you are done with them.
00087
00088   void mpfr_clears(mpfr_t x1, mpfr_t x2, ..., (mpfr_ptr)0)
00089   => clear sequence of variables
00090   => must be terminated by null-pointer (whose type must also be mpfr_ptr)
00091
00092   void mpfr_set_default_prec(mpfr_prec_t prec)
00093   => if MPFR was compiled with --enable-thread-safe,
00094   the default precision is local to each thread.
00095
00096   mpfr_prec_t mpfr_get_default_prec(void)
00097   => return the current default MPFR precision in bits.
00098
00099   void mpfr_set_prec (mpfr_t x, mpfr_prec_t prec)
00100   mpfr_prec_t mpfr_get_prec (mpfr_t x)
00101   => set/get variable specific precision
00102
00103 * ----- */
00104
00105 #define DEF_RND GMP_RNDN
00106
00107 namespace Wb {
00108   gmp_randstate_t wb_rstate;
00109
00110   template <unsigned P=128>
00111   class mpfr__ {
00112   public:
00113     mpfr__(double val=0) {
00114       mpfr_init2(f,P); init(FL,val);

```

```

00118     #pragma omp atomic
00119     ++num_vars;
00120 };
00121
00122 mpfr__(const char *F, int L, double val) {
00123     mpfr_init2(f,P); init(F_L,val);
00124     #pragma omp atomic
00125     ++num_vars;
00126 };
00127
00128 mpfr__(const mpfr__ &b) {
00129     mpfr_init2(f,P); mpfr_set(f,b.f,DEF_RND);
00130     #pragma omp atomic
00131     ++num_vars;
00132 };
00133
00134 ~mpfr__() {
00135     mpfr_clear(f);
00136
00137     #pragma omp atomic
00138     --num_vars;
00139
00140     if (!num_vars) {
00141         Cleanup(FL);
00142     }
00143 };
00144
00145 void Cleanup(const char *F=NULL, int L=0) const;
00146
00147 mpfr__& init (const char *F, int L, double val, char lflag=0);
00148 mpfr__& init_s(const char *F, int L, const char *s, unsigned base=10);
00149 mpfr__& init_d(const char *F, int L, double val, char dcheck=1);
00150
00151 mpfr__& set_pi () { mpfr_const_pi (f,DEF_RND); return *this; };
00152 mpfr__& set_log2() { mpfr_const_log2(f,DEF_RND); return *this; };
00153 mpfr__& set_e   () { init(FL,1); mpfr_exp(f,f,DEF_RND); return *this; };
00154
00155 mpfr__& set_nan () { mpfr_set_nan(f); return *this; };
00156 mpfr__& set_inf (int s=+1) { mpfr_set_inf(f,s); return *this; };
00157 mpfr__& set_zero(int s=+1) { mpfr_set_zero(f,s); return *this; };
00158
00159 int isnan () const { return mpfr_nan_p(f); };
00160 int isinf () const { return mpfr_inf_p(f); };
00161 int iszero () const { return mpfr_zero_p(f); };
00162 int isnumber () const { return mpfr_number_p(f); };
00163 int isnonzero() const { return mpfr_regular_p(f); };
00164
00165 int sign() { return mpfr_sgn(f); };
00166
00167 void swap(mpfr__<P> &b) { mpfr_swap(f,b.f); };
00168
00169 int cmp(const mpfr__ &b ) const { return mpfr_cmp (f,b.f); };
00170 int cmp(const double &val) const { return mpfr_cmp_d (f,val); };
00171 int cmp(const int &val) const { return mpfr_cmp_si (f,val); };
00172 int cmp(const long &val) const { return mpfr_cmp_si (f,val); };
00173 int cmp(const unsigned &val) const { return mpfr_cmp_ui (f,val); };
00174 int cmp(const unsigned long &val) const { return mpfr_cmp_ui (f,val); };
00175
00176 bool operator> (const mpfr__ &x) const { return (cmp(x)> 0); };
00177 bool operator> (const double &x) const { return (cmp(x)> 0); };
00178 bool operator> (const int &x) const { return (cmp(x)> 0); };
00179 bool operator> (const long &x) const { return (cmp(x)> 0); };
00180 bool operator> (const unsigned &x) const { return (cmp(x)> 0); };
00181 bool operator> (const unsigned long &x) const { return (cmp(x)> 0); };
00182
00183 bool operator< (const mpfr__ &x) const { return (cmp(x)< 0); };
00184 bool operator< (const double &x) const { return (cmp(x)< 0); };
00185 bool operator< (const int &x) const { return (cmp(x)< 0); };
00186 bool operator< (const long &x) const { return (cmp(x)< 0); };
00187 bool operator< (const unsigned &x) const { return (cmp(x)< 0); };
00188 bool operator< (const unsigned long &x) const { return (cmp(x)< 0); };
00189
00190 bool operator>= (const mpfr__ &x) const { return (cmp(x)>=0); };
00191 bool operator>= (const double &x) const { return (cmp(x)>=0); };
00192 bool operator>= (const int &x) const { return (cmp(x)>=0); };
00193 bool operator>= (const long &x) const { return (cmp(x)>=0); };
00194 bool operator>= (const unsigned &x) const { return (cmp(x)>=0); };
00195 bool operator>= (const unsigned long &x) const { return (cmp(x)>=0); };
00196
00197 bool operator<= (const mpfr__ &x) const { return (cmp(x)<=0); };
00198 bool operator<= (const double &x) const { return (cmp(x)<=0); };
00199 bool operator<= (const int &x) const { return (cmp(x)<=0); };
00200 bool operator<= (const long &x) const { return (cmp(x)<=0); };
00201 bool operator<= (const unsigned &x) const { return (cmp(x)<=0); };
00202 bool operator<= (const unsigned long &x) const { return (cmp(x)<=0); };
00203
00204 bool operator!= (const mpfr__ &x) const { return (cmp(x)!=0); };

```

```

00205 bool operator!=(const double &x) const { return (cmp(x)!=0); };
00206 bool operator!=(const int &x) const { return (cmp(x)!=0); };
00207 bool operator!=(const long &x) const { return (cmp(x)!=0); };
00208 bool operator!=(const unsigned &x) const { return (cmp(x)!=0); };
00209 bool operator!=(const unsigned long &x) const { return (cmp(x)!=0); };
00210
00211 bool operator==(const mpfr__ &x) const { return (cmp(x)==0); };
00212 bool operator==(const double &x) const { return (cmp(x)==0); };
00213 bool operator==(const int &x) const { return (cmp(x)==0); };
00214 bool operator==(const long &x) const { return (cmp(x)==0); };
00215 bool operator==(const unsigned &x) const { return (cmp(x)==0); };
00216 bool operator==(const unsigned long &x) const { return (cmp(x)==0); };
00217
00218 mpfr__ operator= (double val) { return init(FL,val); };
00219 mpfr__ operator= (const mpfr__ &b) {
00220     mpfr_set(f,b.f,DEF_RND);
00221     return *this;
00222 };
00223
00224 mpfr__ times(const mpfr__ &b, mpfr__ &c) const {
00225     mpfr_mul(c.f,f,b.f,DEF_RND); return c; };
00226 mpfr__ divby(const mpfr__ &b, mpfr__ &c) const {
00227     mpfr_div(c.f,f,b.f,DEF_RND); return c; };
00228
00229 mpfr__ operator*= (const mpfr__ &b) {
00230     mpfr_mul(f,f,b.f,DEF_RND); return *this; };
00231 mpfr__ operator/= (const mpfr__ &b) {
00232     mpfr_div(f,f,b.f,DEF_RND); return *this; };
00233 mpfr__ operator+= (const mpfr__ &b) {
00234     mpfr_add(f,f,b.f,DEF_RND); return *this; };
00235 mpfr__ operator-= (const mpfr__ &b) {
00236     mpfr_sub(f,f,b.f,DEF_RND); return *this; };
00237
00238 mpfr__ operator*= (double val) { mpfr__ b(FL,val); return (*this)*=b; };
00239 mpfr__ operator/= (double val) { mpfr__ b(FL,val); return (*this)/=b; };
00240 mpfr__ operator+= (double val) { mpfr__ b(FL,val); return (*this)+=b; };
00241 mpfr__ operator-= (double val) { mpfr__ b(FL,val); return (*this)-=b; };
00242
00243 mpfr__ operator* (const mpfr__ &b) const {
00244     mpfr__ x; mpfr_mul(x.f,f,b.f,DEF_RND); return x; };
00245 mpfr__ operator/ (const mpfr__ &b) const {
00246     mpfr__ x; mpfr_div(x.f,f,b.f,DEF_RND); return x; };
00247 mpfr__ operator+ (const mpfr__ &b) const {
00248     mpfr__ x; mpfr_add(x.f,f,b.f,DEF_RND); return x; };
00249 mpfr__ operator- (const mpfr__ &b) const {
00250     mpfr__ x; mpfr_sub(x.f,f,b.f,DEF_RND); return x; };
00251
00252 mpfr__ operator* (int n) const {
00253     mpfr__ x; mpfr_mul_si(x.f,f,n,DEF_RND); return x; };
00254 mpfr__ operator/ (int n) const {
00255     mpfr__ x; mpfr_div_si(x.f,f,n,DEF_RND); return x; };
00256 mpfr__ operator+ (int n) const {
00257     mpfr__ x; mpfr_add_si(x.f,f,n,DEF_RND); return x; };
00258 mpfr__ operator- (int n) const {
00259     mpfr__ x; mpfr_sub_si(x.f,f,n,DEF_RND); return x; };
00260
00261 mpfr__ operator* (unsigned n) const {
00262     mpfr__ x; mpfr_mul_ui(x.f,f,n,DEF_RND); return x; };
00263 mpfr__ operator/ (unsigned n) const {
00264     mpfr__ x; mpfr_div_ui(x.f,f,n,DEF_RND); return x; };
00265 mpfr__ operator+ (unsigned n) const {
00266     mpfr__ x; mpfr_add_ui(x.f,f,n,DEF_RND); return x; };
00267 mpfr__ operator- (unsigned n) const {
00268     mpfr__ x; mpfr_sub_ui(x.f,f,n,DEF_RND); return x; };
00269
00270 mpfr__ operator* (long n) const {
00271     mpfr__ x; mpfr_mul_si(x.f,f,n,DEF_RND); return x; };
00272 mpfr__ operator/ (long n) const {
00273     mpfr__ x; mpfr_div_si(x.f,f,n,DEF_RND); return x; };
00274 mpfr__ operator+ (long n) const {
00275     mpfr__ x; mpfr_add_si(x.f,f,n,DEF_RND); return x; };
00276 mpfr__ operator- (long n) const {
00277     mpfr__ x; mpfr_sub_si(x.f,f,n,DEF_RND); return x; };
00278
00279 mpfr__ operator* (unsigned long n) const {
00280     mpfr__ x; mpfr_mul_ui(x.f,f,n,DEF_RND); return x; };
00281 mpfr__ operator/ (unsigned long n) const {
00282     mpfr__ x; mpfr_div_ui(x.f,f,n,DEF_RND); return x; };
00283 mpfr__ operator+ (unsigned long n) const {
00284     mpfr__ x; mpfr_add_ui(x.f,f,n,DEF_RND); return x; };
00285 mpfr__ operator- (unsigned long n) const {
00286     mpfr__ x; mpfr_sub_ui(x.f,f,n,DEF_RND); return x; };
00287
00288 mpfr__ Sqrt() { mpfr_sqrt(f,f,DEF_RND); return *this; };
00289 mpfr__ Exp() { mpfr_exp(f,f,DEF_RND); return *this; };
00290 mpfr__ Log() { mpfr_log(f,f,DEF_RND); return *this; };
00291 mpfr__ Log2() { mpfr_log2(f,f,DEF_RND); return *this; };

```



```

00292     mpfr_& Atan() { mpfr_atan(f,f,DEF_RND); return *this; };
00293
00294     mpfr_& Tanh() { mpfr_tanh(f,f,DEF_RND); return *this; };
00295     mpfr_& Atanh() { mpfr_atanh(f,f,DEF_RND); return *this; };
00296
00297     mpfr_& Rand() {
00298         mpfr_urandomb(f,wb_rstate);
00299         return *this;
00300     };
00301
00302     mpfr_& Log10() { mpfr_log10(f,f,DEF_RND); return *this; };
00303     mpfr_& Round() { mpfr_round(f,f); return *this; };
00304
00305     mpfr_ sqrt() const {
00306         mpfr_<P> x(*this); mpfr_sqrt(x.f,x.f,DEF_RND); return x; };
00307     mpfr_ exp() const {
00308         mpfr_<P> x(*this); mpfr_exp(x.f,x.f,DEF_RND); return x; };
00309     mpfr_ log() const {
00310         mpfr_<P> x(*this); mpfr_log(x.f,x.f,DEF_RND); return x; };
00311     mpfr_ log2() const {
00312         mpfr_<P> x(*this); mpfr_log2(x.f,x.f,DEF_RND); return x; };
00313     mpfr_ atan() const {
00314         mpfr_<P> x(*this); mpfr_atan(x.f,x.f,DEF_RND); return x; };
00315     mpfr_ log10() const {
00316         mpfr_<P> x(*this); mpfr_log10(x.f,x.f,DEF_RND); return x; };
00317     mpfr_ round() const {
00318         mpfr_<P> x(*this); mpfr_round(x.f,x.f); return x; };
00319
00320     explicit operator double() const { return mpfr_get_d(f,DEF_RND); };
00321
00322     explicit operator wbcomplex() const;
00323
00324     explicit operator bool() const {
00325         return (mpfr_zero_p(f) ? 0 : 1);
00326     };
00327
00328     double getval_d() const { return mpfr_get_d(f,DEF_RND); };
00329
00330     bool operator< (double val) const { return (getval_d()< val); };
00331     bool operator<= (double val) const { return (getval_d()<=val); };
00332     bool operator> (double val) const { return (getval_d()> val); };
00333     bool operator>= (double val) const { return (getval_d()>=val); };
00334
00335     int prec(unsigned base=2) const {
00336         int p=mpfr_get_prec(f);
00337         if (base!=2) { p = p* (::log(2)/::log(base)) + 1; }
00338         return p;
00339     };
00340
00341     mpfr_ eps() {
00342         mpfr_ x;
00343         mpfr_set_uj_2exp(x.f, 1, -prec(2), DEF_RND);
00344         return x;
00345     };
00346
00347     mpfr_& FlipSign() {
00348         mpfr_setsign(f,f, mpfr_signbit(f) ? 0 : 1, DEF_RND);
00349         return *this;
00350     };
00351
00352     mpfr_& Abs() {
00353         mpfr_setsign(f,f, 0, DEF_RND);
00354         return *this;
00355     };
00356
00357     mpfr_ abs() const { return mpfr_(*this).Abs(); };
00358     mpfr_ fabs() const { return mpfr_(*this).Abs(); };
00359     mpfr_ abs2() const {
00360         mpfr_ x(*this); x*=(*this);
00361         return x;
00362     };
00363
00364     void print(const char *vname="", const char *istr=0, char sep=0) const {
00365         if (sep) printf("-----"
00366             "0----+----1----+----2----+----3----+----4----+----"
00367             "5----+----6----+----7----\n"
00368         );
00369         if (vname && vname[0]) printf("%6s = ",vname);
00370         else printf(" ");
00371
00372         mpfr_out_str(stdout,
00373             10,
00374             0,
00375             f,
00376             GMP_RNDU
00377         );
00378

```

```

00379         if (!istr || !istr[0]) {
00380             long p,q; unsigned niter; double r, x=getval_d();
00381             int e=Rational(x,p,q,&r,&niter,NULL,6,1000000,1E-10,1E-12);
00382             if (!e && p) { char s[32];
00383                 if (q!=1) snprintf(s,32," = %ld/%ld",p,q);
00384                 else snprintf(s,32," = %ld",p);
00385                 istr=s;
00386             }
00387         }
00388         printf(" %s\n", istr && istr[0] ? istr : "");
00389     };
00390
00391     size_t to_str(char *s, size_t n_, int base=10) const;
00392     wstring to_str(int base=10) const;
00393
00394     void toStr(
00395         char *s, unsigned n, int base=10, int nd=-1, char align=0) const;
00396
00397     wstring& toStr(
00398         wstring &sout,int base=10, int nd=-1, char align=0) const;
00399
00400     mpfr_t f;
00401
00402     static unsigned long num_vars;
00403
00404     private:
00405
00406 };
00407
00408     typedef mpfr__<128> quad;
00409     typedef mpfr__<128> quad2;
00410
00411     typedef mpfr__<192> quad3;
00412     typedef mpfr__<256> quad4;
00413     typedef mpfr__<256> qquad;
00414
00415     template<>
00416     unsigned long quad::num_vars=0;
00417
00418     quad mpfr_dummy_quad;
00419
00420     template <unsigned P>
00421     mpfr__<P> operator+(const mpfr__<P> &a) { return mpfr__<P>(a); };
00422
00423     template <unsigned P>
00424     mpfr__<P> operator-(const mpfr__<P> &a) { return mpfr__<P>(a).FlipSign(); };
00425
00426     template <class T, unsigned P>
00427     mpfr__<P> operator*(const T& x_, const mpfr__<P> &a) {
00428         mpfr__<P> x(x_); return (x*=a);
00429     };
00430
00431     template <class T, unsigned P>
00432     mpfr__<P> operator+(const T& x_, const mpfr__<P> &a) {
00433         mpfr__<P> x(x_); return (x+=a);
00434     };
00435
00436     template <class T, unsigned P>
00437     mpfr__<P> operator-(const T& x_, const mpfr__<P> &a) {
00438         mpfr__<P> x(x_); return (x-=a);
00439     };
00440
00441     template <class T, unsigned P>
00442     mpfr__<P> operator/(const T& x_, const mpfr__<P> &a) {
00443         mpfr__<P> x(x_); return (x/=a);
00444     };
00445
00446     template <unsigned P> inline
00447     int isnan(const mpfr__<P> &a) { return a.isnan(); };
00448
00449     template <unsigned P> inline
00450     int isinf(const mpfr__<P> &a) { return a.isinf(); };
00451
00452     template <unsigned P> inline
00453     int iszero(const mpfr__<P> &a) { return a.iszero(); };
00454
00455     template <unsigned P> inline
00456     int isnumber(const mpfr__<P> &a) { return a.isnumber(); };
00457
00458     template <unsigned P> inline
00459     int isfinite(const mpfr__<P> &a) { return a.isfinite(); };
00460
00461     template <unsigned P> inline
00462     mpfr__<P> fabs (const mpfr__<P> &x) { return x.fabs(); };
00463
00464     template <unsigned P> inline
00465     mpfr__<P> sqrt (const mpfr__<P> &x) { return x.sqrt(); };

```

```

00466
00467 template <unsigned P> inline
00468 mpfr__<P> exp (const mpfr__<P> &x) { return x.exp(); }
00469
00470 template <unsigned P> inline
00471 mpfr__<P> log (const mpfr__<P> &x) { return x.log(); }
00472
00473 template <unsigned P> inline
00474 mpfr__<P> log2 (const mpfr__<P> &x) { return x.log2(); }
00475
00476 template <unsigned P> inline
00477 mpfr__<P> atan (const mpfr__<P> &x) { return x.atan(); }
00478
00479 template <unsigned P> inline
00480 mpfr__<P> log10(const mpfr__<P> &x) { return x.log10(); }
00481
00482 template <unsigned P> inline
00483 mpfr__<P> round(const mpfr__<P> &x) { return x.round(); }
00484
00485 class cc_quad {
00486
00487 public:
00488
00489     cc_quad(long double l=0) : f(l) { if (l!=0) {
00490         long double r1=std::sqrt(l), r2=std::sqrt(f);
00491
00492         printf("1> %22.20Lf => %22.20Lf (%.3Lg)\n",l,r1,r1*r1-l);
00493         printf("2> %22.20Lf => %22.20Lf (%.3Lg)\n", f,r2,r2*r2-f);
00494
00495         r2=std::sqrt(f);
00496         printf("3> %22.20Lf => %22.20Lf (%.3Lg)\n\n",f,r2,r2*r2-f);
00497
00498     }};
00499
00500     cc_quad sqrt() const { cc_quad x(*this); x.f=std::sqrt(f); return x; };
00501     cc_quad fabs() const { cc_quad x(*this); x.f=std::fabs(f); return x; };
00502     cc_quad abs() const { cc_quad x(*this); x.f=std::fabs(f); return x; };
00503
00504     cc_quad operator+(const cc_quad& b) const { return (f+b.f); };
00505     cc_quad operator-(const cc_quad& b) const { return (f-b.f); };
00506     cc_quad operator*(const cc_quad& b) const { return (f*b.f); };
00507     cc_quad operator/(const cc_quad& b) const { return (f/b.f); };
00508
00509     cc_quad operator+=(const cc_quad& b) { return (f+=b.f); };
00510     cc_quad operator-=(const cc_quad& b) { return (f-=b.f); };
00511     cc_quad operator*=(const cc_quad& b) { return (f*=b.f); };
00512     cc_quad operator/=(const cc_quad& b) { return (f/=b.f); };
00513
00514     long double f;
00515
00516 private:
00517
00518 };
00519
00520     cc_quad sqrt(const cc_quad &a) { return a.sqrt(); }
00521
00522 };
00523
00524 #endif
00525

```

5.50 wbmsg.hh

```

00001 #ifndef __WB_MSG_HH__
00002 #define __WB_MSG_HH__
00003
00004 /* https://stackoverflow.com/questions/24785230/difference-between-msgget-and-mq-open
00005  * Wb,Oct06,22
00006
00007 The msgXXX family is inherited from SysV. The mq_XXX family was
00008 created by POSIX. Both families are however part of POSIX.
00009
00010 The two families have somewhat different feature sets the most
00011 important being that the SysV variant allows for the specification
00012 of different message types. This provides for flexibility in the
00013 way messages are processed and in assigning priorities. The POSIX
00014 family in contrast always returns the oldest message with the
00015 highest priority.
00016
00017 As a rule the POSIX family is the better choice unless there is
00018 something about SysV that you really need to use. The POSIX
00019 interface is also less cumbersome. An added bonus (on Linux) is
00020 that the POSIX mqd_t type representing a queue identifier is

```

```

00021 actually a file descriptor. This allows for the extremely
00022 convenient use of POSIX queues with select, poll and similar
00023 facilities.
00024
00025 Basicly, msgget, msgsnd, msgrcv are System V IPC, while mq_open,
00026 mq_send, mq_receive are POSIX IPC.
00027
00028 POSIX IPC is designed after System V IPC. So a lot of old systems
00029 support only System V IPC while new systems begin to support POSIX
00030 IPC as well. And, because POSIX IPC can learn the advantages and
00031 disadvantages from System V IPC, POSIX IPC may be designed and
00032 implemented better. A notable difference is that
00033 all the POSIX IPC interfaces are thread safe.
00034
00035 Comments: SysV interfaces are also thread safe. There would be no
00036 point to them otherwise
00037 Well, I didn't confirm it but just read it from
00038 docs.oracle.com/cd/E19455-01/806-4750/6jdqdf1tf/index.html that
00039 SysV IPC may not thread safe: Unlike the System V IPC interfaces,
00040 the POSIX IPC interfaces are all multithread safe
00041
00042 # IPC benchmarking
00043 # https://stackoverflow.com/questions/2854004/ipc-speed-and-compare
00044 Pipe benchmark                36,539 msg/s
00045 FIFOs (named pipes) benchmark 26,246 msg/s
00046 Message Queues benchmark      67,920 msg/s
00047 Shared Memory benchmark       3,821,893 msg/s  100x faster!
00048 TCP sockets benchmark         22,483 msg/s
00049 Unix domain sockets benchmark 40,683 msg/s
00050 ZeroMQ benchmark              15,414 msg/s
00051
00052 https://www.qnx.com/developers/docs/6.5.OSPl.update/..
00053 com.qnx.doc.neutrino_sys_arch/ipc.html#Shared_memory
00054
00055 Shared memory offers the highest bandwidth IPC available. Once a
00056 shared-memory object is created, processes with access to the
00057 object can use pointers to directly read and write into it
00058
00059 */
00060
00061 // ----- //
00062 // Messaging class -> automates deletion of message queue!!
00063 // Wb, Dec03, 2003
00064
00065 #define WB_MSG_PRIV 0666
00066
00067 typedef struct __wb_msgbuf {
00068     long    type;
00069     time_t  time;
00070     char    text[128];
00071     double  val;
00072 } wbMsgBuf;
00073
00074 const size_t WBMSGSZ = sizeof(wbMsgBuf) - sizeof(long);
00075
00076 class wbMsg {
00077 public:
00078
00079     wbMsg(const char* file, int line,
00080           const char *istr="Message queue", int key_=0)
00081     {
00082         info=istr;
00083
00084         key = (key_>0) ? key_ : getpid();
00085         msgflg = WB_MSG_PRIV;
00086
00087         if ((msqid = msgget(key, IPC_CREAT | msgflg)) < 0) {
00088             wblog(FL,"ERR msgget returns %d", -msqid);
00089             exit(-1);
00090         }
00091
00092         if (istr && istr[0]) wblog(file,line,"%NMSG create queue "
00093             "%s (key=%d (0x%X), qid=%d)", info.data,key,key,msqid);
00094     };
00095
00096     ~wbMsg () {
00097         if (info.data && info.data[0]) wblog (FL,"MSG remove queue "
00098             "%s (key=%d (0x%X), qid=%d)", info.data,key,key,msqid);
00099         msgctl (msqid, IPC_RMID, 0);
00100     };
00101
00102     int Rec(const char*,int, wbMsgBuf &rbuf,
00103            unsigned flag=IPC_NOWAIT);
00104 };
00105
00106
00107

```

```

00108     int Snd(const char*,int, wBMsgBuf &sbuf,
00109             const char *istr, double val=0., unsigned type=1
00110             );
00111
00112     wBstring info;
00113     int msqid, msgflg;
00114     key_t key;
00115
00116     protected:
00117     private:
00118 };
00119
00120 int wBMsg::Rec (
00121     const char* file, int line,
00122     wBMsgBuf &rbuf,
00123     unsigned flag
00124 ){
00125
00126     memset(&rbuf,0,sizeof(wBMsgBuf));
00127
00128     int i = msgrcv(msqid, &rbuf, WBMSGSZ, 1, flag);
00129     if (i < 0) {
00130         if (i!=-1)
00131             wBlog(file,line,
00132                 "ERR can't read message port %d (%d)", msqid, i);
00133         return -i;
00134     }
00135
00136     sprintf_str("%s", ctime(&rbuf.time));
00137     str[strlen(str)-1] = 0;
00138
00139     wBlog (file,line,"%NNB! RECEIVED MESSAGE %s (%d; 0x%lX %d)\n\n"
00140         "   time : %s\n"
00141         "   text  : %s\n"
00142         "   val   : %g\n"
00143         "   type  : %ld\n",
00144         info.data ? info.data : "", key, key, msqid,
00145         str, rbuf.text, rbuf.val, rbuf.type);
00146
00147     return 0;
00148 }
00149
00150 int wBMsg::Snd (
00151     const char* file, int line,
00152     wBMsgBuf &sbuf,
00153     const char *istr, double val,
00154     unsigned type
00155 ){
00156
00157     int i;
00158
00159     memset(&sbuf,0,sizeof(wBMsgBuf));
00160
00161     sbuf.type = type;
00162     sbuf.time = time((time_t*)NULL);
00163     strncpy(sbuf.text,istr,127);
00164     sbuf.val = val;
00165
00166     i = msgsnd(msqid, &sbuf, WBMSGSZ, IPC_NOWAIT);
00167     if (i<0) {
00168         wBlog(FL,"ERR can't send to message port %d (err=%d)", msqid, i);
00169         return -1;
00170     }
00171
00172     sprintf_str("%s", ctime(&sbuf.time));
00173     str[strlen(str)-1] = 0;
00174
00175     wBlog (file,line,
00176         "Message sent ... \n"
00177         "   type  : %ld\n"
00178         "   time  : %s\n"
00179         "   text  : %s\n"
00180         "   val   : %g",
00181         sbuf.type, str, sbuf.text, sbuf.val);
00182
00183     return 0;
00184 }
00185
00186 int WbMSGsnd (
00187     const char* file, int line,
00188     unsigned key,
00189     char *text, double val,
00190     unsigned type=1
00191 ){
00192
00193     wBMsgBuf sbuf;
00194     int i, msqid;

```

```

00195
00196     if ((msqid = msgget(key, IPC_CREAT | WB_MSG_PRIV)) < 0) {
00197         wblog(FL, "ERR can't read message port %d (err=%d)", key, -msqid);
00198         return -1;
00199     }
00200
00201     sbuf.type = type;
00202     sbuf.time = time((time_t*)NULL);
00203     strcpy(sbuf.text, text);
00204     sbuf.val = val;
00205
00206     i = msgsnd(msqid, &sbuf, WBMSGSZ, IPC_NOWAIT);
00207     if (i < 0) {
00208         wblog(FL, "ERR can't send to message port %d (err=%d)", msqid, i);
00209         return -1;
00210     }
00211
00212     sprintf_str("%s", ctime(&sbuf.time));
00213     str[strlen(str)-1] = 0;
00214
00215     wblog(file, line,
00216         "Message sent (%d; 0x%lX %d) ...\n"
00217         "    type   : %ld\n"
00218         "    time   : %s\n"
00219         "    text   : %s\n"
00220         "    val    : %g", key, key, msqid,
00221         sbuf.type, str, sbuf.text, sbuf.val);
00222
00223     return 0;
00224 }
00225
00226 #endif
00227

```

5.51 wbomp.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : QSpace OMP routines
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_OMP_HH__
00021 #define __WB_OMP_HH__
00022
00023 #ifdef QS_USING_OMP
00024
00025     int OMP_NUM_THREADS=0;
00026
00027     int QSP_NUM_THREADS=0;
00028
00029 namespace Wb {
00030
00031     wstring ompID2Str(char vflag=0);
00032
00033     int ompStatus(const char *F, int L);
00034
00035     class ompNLock {
00036     public:
00037
00038         ompNLock(const char *s=NULL)
00039             : level(0), owner(-1), istr(0), nactive(0), ntot(0) {
00040             omp_init_nest_lock(&_nlk);
00041             if (s && s[0]) { set_istr(s); }
00042         };
00043
00044         ~ompNLock() {
00045             if (level || nactive) {
00046                 wblog(FL, "WRN %s() got lock @ l=%d, n=%d (id=%d/%d) !?",

```

```

00048         FCT,level,nactive,omp_get_thread_num(),omp_get_num_threads());
00049         if (level==1 && !nactive) { release(); }
00050     }
00051     if (istr) { delete [] istr; istr=0; }
00052     owner=-1;
00053     omp_destroy_nest_lock(&_nlk);
00054 };
00055
00056 void set_istr(const char *s);
00057
00058 wstring toStr(const char *istr=NULL) const;
00059
00060 wstring get_istr() const;
00061
00062 int acquire();
00063
00064 int test_acquire();
00065
00066 int release();
00067
00068 int level;
00069 int owner;
00070
00071 char *istr;
00072
00073 int nactive;
00074
00075 long ntot;
00076
00077 private:
00078     omp_nest_lock_t _nlk;
00079
00080     ompNLock(const ompNLock &);
00081     void operator=(const ompNLock &);
00082
00083 #ifdef LD_CLEBSCH_QS
00084     wstring check_BUF_size() const;
00085 #endif
00086 };
00087
00088 class ompGuard {
00089 public:
00090     ompGuard() : lp(0) { };
00091
00092     ompGuard(const char *F, int L, ompNLock &lk, int lmax=99)
00093         : lp(0) {
00094         if (omp_get_num_threads()>1) {
00095             acquire(F,L,lk,lmax);
00096         }
00097     };
00098
00099     ompGuard(ompNLock &lk, int lmax=99) : lp(0) {
00100         if (omp_get_num_threads()>1) {
00101             acquire(0,0,lk,lmax);
00102         }
00103     };
00104
00105     void acquire(ompNLock &lk, int lmax=99) { acquire(0,0,lk,lmax); };
00106     void acquire(const char *F, int L, ompNLock &lk, int lmax=99);
00107
00108     ~ompGuard() { if (lp) { lp->release(); } };
00109
00110     int level() { return (lp ? lp->level : -1); };
00111
00112 private:
00113     ompNLock *lp;
00114
00115     ompGuard(const ompGuard &) {
00116         wblog(FL,"ERR %s() forbidden copy",FCT); };
00117     void operator=(const ompGuard &) {
00118         wblog(FL,"ERR %s() forbidden copy",FCT); };
00119 };
00120
00121 class nthreads_aux {
00122 public:
00123     nthreads_aux() : np_curr(0), np_max(0) { };
00124
00125     nthreads_aux& increment(const char *fl) {
00126         int q=omp_get_num_threads();
00127         if (np_max>q) {
00128             Wb::termcolor Tc(1);
00129             fprintf(stdout,"TST %s%s() having nq=%d->%d%s\n",
00130                 Tc.e1,fl? fl:"(null)",np_max,q,Tc.em);

```

```

00135         np_max=q;
00136     }
00137     if (np_max<(++np_curr)) {
00138         Wb::termcolor Tc(5);
00139         fprintf(stdout,"TST %s%s() having nq=%d->%d%s\n",
00140             Tc.el,fl? fl:"(null)",np_max,np_curr,Tc.em);
00141         np_max=np_curr;
00142     }
00143     return *this;
00144 };
00145
00146 nthreads_aux& decrement(const char *fl) {
00147     if (--np_curr<0) { Wb::termcolor Tc(5);
00148         fprintf(stdout,"%sWRN %s() having nq=%d/%d%s\n",
00149             Tc.el,fl? fl:"(null)",np_curr,np_max,Tc.em);
00150     }
00151     return *this;
00152 };
00153
00154 private:
00155     int np_curr, np_max;
00156 };
00157
00158 map< std::string, nthreads_aux > np_hist;
00159
00160 class checkMaxThreads {
00161 public:
00162     checkMaxThreads(const char *F, int L) {
00163         std::string fl=shortFL(F,L);
00164         #pragma omp critical (checkMaxThreads__)
00165         { np_hist[fl].increment(fl.c_str()); im=np_hist.find(fl); }
00166     };
00167
00168 ~checkMaxThreads() {
00169     #pragma omp critical (checkMaxThreads__)
00170     { im->second.decrement(im->first.c_str()); }
00171 };
00172
00173 private:
00174
00175     map< std::string, nthreads_aux >::iterator im;
00176 };
00177
00178 class thread_xlink;
00179 map< int, thread_xlink* > txmap;
00180
00181 class thread_xlink {
00182 public:
00183     thread_xlink(const char *F=NULL, int L=0) : tx(0) {
00184         t0=omp_get_thread_num();
00185         nt=omp_get_num_threads();
00186
00187         if (F) { fl=shortFL(F,L); }
00188     };
00189
00190 ~thread_xlink() {
00191     auto it = txmap.find(t0);
00192     if (it!=txmap.end()) {
00193         #pragma omp critical (thread_xlink__)
00194         txmap.erase(t0);
00195     }
00196 };
00197
00198 void init(const char *F=NULL, int L=0) {
00199     if (nt<=1) {
00200         t0=omp_get_thread_num();
00201         nt=omp_get_num_threads();
00202     }
00203     else {
00204         unsigned t2=omp_get_thread_num(); if (t2!=t0) {
00205             wblog(F_L,"WRN %s() thread-id changed %d->%d / %d->%d",
00206                 FCT, t0,t2,nt,omp_get_num_threads());
00207         }
00208     }
00209     if (!tx) { if (F || fl.empty()) fl=shortFL(F,L);
00210         #pragma omp critical (thread_xlink__)
00211         txmap[t0]=this;
00212     }
00213     tx=(1<t0);
00214 };
00215
00216 int add(const char *F, int L, size_t h);
00217 int add(const char *F, int L, const ompNLock *lk);
00218
00219
00220
00221

```



```

00222     static int check_deadlocks();
00223
00224     unsigned t0, nt;
00225
00226     size_t tx;
00227     std::string fl;
00228
00229 private:
00230 };
00231
00232 };
00233
00234 Wb::ompNLock wblog_lock("wblog");
00235 Wb::ompNLock rclog_lock("rclog");
00236 Wb::ompNLock mxapi_lock("mxapi");
00237
00238 Wb::ompNLock XS_buf("XS_buf");
00239
00240 Wb::ompNLock CS_buf("CS_buf");
00241
00242 Wb::ompNLock CG_locks("CG_locks");
00243
00244 #else
00245 #endif
00246
00247 #endif
00248

```

5.52 wbopts.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : QSpace MEX routine options class
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_OPTS_HCC__
00021 #define __WB_OPTS_HCC__
00022
00023 /* ----- */
00024 /* ----- */
00025 /* class to manage optional input arguments */
00026
00027 class OPTS {
00028
00029 public:
00030
00031     OPTS () : mto(0) {};
00032
00033     OPTS (const mxArray** ain, const unsigned n)
00034     : mto(0) { init(ain,n); };
00035
00036     OPTS (const char *fname)
00037     : mto(0) { init(fname); };
00038
00039     void init (const mxArray** ain, const unsigned n);
00040     void init (const char *fname);
00041
00042     ~OPTS () { init(); }
00043     void init() {
00044         name.init();
00045
00046         if (MAT.mfp && aa.len)
00047             for (unsigned i=0; i<aa.len; ++i) mxDestroyArray(aa[i]);
00048
00049         aa.init();
00050     };
00051
00052     int len() { return ( MAT.mfp ? -1 : (int)aa.len ); };
00053

```

```

00054     template <class T>
00055     bool getOpt(const char*, int,
00056                const char* vn, T &x, char force=0, const char *istr=0);
00057
00058     template <class T>
00059     bool getOpt(const char*, int,
00060                const char* vn, wbvector<T> &x, char force=0, const char *istr=0);
00061
00062     template <class T>
00063     bool getOpt(const char*, int,
00064                const char* vn, wbMatrix<T> &x, char force=0, const char *istr=0);
00065
00066     bool getOpt(const char*, int, const char* vn, const char *istr=0);
00067
00068     template <class T>
00069     bool getOpt(const char* vn, T &x, char force=0, const char *istr=0) {
00070         return getOpt(NULL,0, vn,x,force,istr);
00071     };
00072
00073     template <class T>
00074     bool getOpt(const char* vn,
00075                wbvector<T> &x, char force=0, const char *istr=0) {
00076         return getOpt(NULL,0,vn,x,force,istr);
00077     };
00078
00079     template <class T>
00080     bool getOpt(const char* vn,
00081                wbMatrix<T> &x, char force=0, const char *istr=0) {
00082         return getOpt(NULL,0,vn,x,force,istr);
00083     };
00084
00085     bool getOpt(const char* vn, const char *istr=0) {
00086         return getOpt(NULL,0,vn,istr);
00087     };
00088
00089     void checkAnyLeft(const char *MAT=NULL, int L=0);
00090
00091 protected:
00092 private:
00093
00094     wbvector<mxArray*> aa;
00095     wbvector<wbstring> name;
00096
00097     unsigned mto;
00098
00099     Wb::matFile MAT;
00100
00101     int findOpt(const char* vn);
00102 };
00103
00104 int OPTS::findOpt(const char* vn) {
00105
00106     unsigned i, n=name.len;
00107     int k=-1;
00108
00109     for (i=0; i<n; ++i)
00110         if (name[i].len) if (!strcmp(name[i].data,vn)) {
00111             if (k<0) k=i; else ++mto;
00112         };
00113
00114     return k;
00115 }
00116
00117 void OPTS::init(const mxArray** ain, const unsigned l) {
00118
00119     unsigned n=128; char s[n];
00120     if (MAT.mfp) wblog(FL,
00121         "ERR OPTS already initialized by file\n%s",MAT.fname.data);
00122
00123     aa.init(l, (mxArray**)ain);
00124     name.init(l);
00125
00126     for (unsigned i=0; i<aa.len; ++i) {
00127         if (!mxIsChar(aa[i])) { continue; } else
00128             if (mxGetString(aa[i],s,n)) { wblog(FL,
00129                 "WRN failed to read OPTS string (len<%d?)",n);
00130                 continue;
00131             }
00132
00133         name[i].init(s);
00134         aa[i]=NULL;
00135     }
00136 };
00137
00138 void OPTS::init (const char *fname) {
00139
00140     if (aa.len) wblog(FL,

```

```

00141         "ERR OPTS already set by mxArray[] (n=%d)\ngot %s",aa.len,fname);
00142
00143     MAT.Open(FL, fname, "r");
00144 };
00145
00146 bool OPTS::getOpt(
00147     const char *F, int L, const char *vn, const char *istr
00148 ){
00149     unsigned r=0;
00150
00151     if (MAT.mfp) {
00152         mxArray *a = matGetVariable(MAT.mfp, vn); r=(a!=NULL);
00153         if (a) {
00154             double dbl=0.;
00155             if (mxGetNumber(a,dbl) { wblog(FL,"ERR %s",str); }
00156             if (dbl==0) r=0;
00157         }
00158         mxDestroyArray(a);
00159     }
00160     else {
00161         int i=findOpt(vn); r=(i>=0);
00162         if (r) name[i].init();
00163     }
00164
00165     if (r && F) {
00166         if (istr && istr[0])
00167             { wblog(F,L," * %s",istr); }
00168         else { wblog(F,L," * %s",vn); }
00169     }
00170
00171     return r;
00172 };
00173
00174 template <class T>
00175 bool OPTS::getOpt(
00176     const char *F, int L, const char* vn, T &x0,
00177     char force, const char *istr
00178 ){
00179     unsigned k,e=0;
00180     mxArray *a; T x=0;
00181
00182     if (MAT.mfp) {
00183         a = matGetVariable(MAT.mfp, vn);
00184         if (!a) {
00185             if (force) wblog(F_L,
00186                 "ERR %s() option '%s' required (%s)",FCT,vn, istr?istr:"");
00187             return 0;
00188         }
00189     }
00190     else {
00191         int i=findOpt(vn);
00192         if (i<0) {
00193             if (force) wblog(F_L,
00194                 "ERR %s() option '%s' required (%s)",FCT,vn, istr?istr:"");
00195             return 0;
00196         }
00197
00198         k=(unsigned) (i+1);
00199
00200         if (k>=name.len) { ++e; } else
00201         if (aa[k]==NULL) { ++e; }
00202         if (e) wblog(F_L,"ERR %s() missing value for '%s'",FCT,vn);
00203
00204         a=aa[k];
00205
00206         name[k-1].init(); aa[k]=NULL;
00207     }
00208
00209     if (mxGetNumber(a,x)) wblog(F_L,
00210         "ERR %s() failed to read value for '%s'\n%s",FCT,vn,str);
00211
00212     if (F && x0!=x) {
00213         if (istr && istr[0])
00214             { wblog(F,L," * %-8s: %s [%s]", vn, NSTR(x), istr); }
00215         else { wblog(F,L," * %-8s: %s", vn, NSTR(x)); }
00216     }
00217
00218     x0=x;
00219
00220     if (MAT.mfp) mxDestroyArray(a);
00221
00222     return 1;
00223 };
00224
00225 template<>
00226 bool OPTS::getOpt(const char *F, int L,
00227     const char* vn,

```

```

00228     wbsting &x, char force, const char *istr
00229 ){
00230     int lflag=0;
00231     if (MAT.mfp) { unsigned n=128; char s[n]; s[0]=0;
00232         mxArray *a = matGetVariable(MAT.mfp, vn);
00233         if (!a) {
00234             if (force) wblog(F_L,"ERR %s() missing input for '%s'",FCT,vn);
00235             return 0;
00236         }
00237
00238         if (!mxIsChar(a)) wblog(F_L,
00239             "ERR %s() invalid type `%s' for '%s'",FCT,mxGetClassName(a),vn);
00240         if (mxGetString(a,s,n)) wblog(F_L,
00241             "ERR %s() failed to read string for '%s' (len<%d?)",FCT,vn,n);
00242
00243         if (x.data && !strcmp(x.data,s)) { ++lflag; }
00244         x=s; mxDestroyArray(a);
00245     }
00246     else {
00247         int i; unsigned k,e=0;
00248
00249         i=findOpt(vn);
00250         if (i<0) {
00251             if (force) wblog(F_L,"ERR %s() option '%s' required",FCT,vn);
00252             return 0;
00253         }
00254         k=unsigned(i+1);
00255
00256         if (k>=name.len) { ++e; } else
00257         if (aa[k]!=NULL) { ++e; }
00258         if (e) wblog(F_L,"ERR %s() missing value for option '%s'",FCT,vn);
00259
00260         if (x.data && !strcmp(x.data,name[k].data)) ++lflag;
00261         x=name[k];
00262
00263         name[k-1].init(); name[k].init();
00264     }
00265
00266     if (F && lflag) {
00267         if (istr && istr[0])
00268             { wblog(F,L," * %-8s: %s [%s]",vn, x.data, istr); }
00269         else { wblog(F,L," * %-8s: %s", vn, x.data); }
00270     }
00271
00272     return 1;
00273 };
00274
00275 template<>
00276 bool OPTS::getOpt(const char *F, int L,
00277     const char* vn, mxArray* &a, char force, const char *istr
00278 ){
00279     if (MAT.mfp) {
00280         a=matGetVariable(MAT.mfp, vn);
00281         if (a) aa.Append((mxArray*)a);
00282     }
00283     else {
00284         int i=findOpt(vn);
00285
00286         if (i<0) a=NULL;
00287         else {
00288             unsigned e=0, k=unsigned(i+1);
00289
00290             if (k>=name.len) { ++e; } else
00291             if (aa[k]==NULL) { ++e; }
00292             if (e) wblog(F_L,"ERR %s() value missing for '%s'",FCT,vn);
00293
00294             a=aa[k];
00295
00296             name[k-1].init(); aa[k]=NULL;
00297         }
00298     }
00299
00300     if (!a) {
00301         if (force) wblog(F_L,"ERR %s() missing value for '%s'",FCT,vn);
00302         return 0;
00303     }
00304
00305     if (F) {
00306         if (istr && istr[0])
00307             { wblog(F,L," * %-8s: (%s) [%s]",vn, mxGetClassName(a), istr); }
00308         else { wblog(F,L," * %-8s: (%s)", vn, mxGetClassName(a)); }
00309     }
00310
00311     return 1;
00312 };
00313
00314 template<>

```

```

00315 bool OPTS::getOpt(
00316     const char *F, QS_UNUSED_VAR,
00317     int L, QS_UNUSED_VAR,
00318     const char* vn, itag_ &t, char force, const char *istr
00319 ){
00320     wstring q;
00321     getOpt(vn,q,force,istr);
00322     if (q) { t.init(0,0,q.data); return 1; }
00323     return 0;
00324 };
00325
00326 template<class T>
00327 bool OPTS::getOpt(const char *F, int L,
00328     const char* vn, wvector<T>&x_, char force, const char *istr
00329 ){
00330     int lflag=0;
00331     if (MAT.mfp) {
00332         mxArray *a = matGetVariable(MAT.mfp, vn);
00333         if (!a) {
00334             if (force) wblog(F_L,"ERR %s() missing input for '%s'",FCT,vn);
00335             return 0;
00336         }
00337
00338         if (!mxIsDouble(a)) wblog(F_L,
00339             "ERR %s() invalid type '%s' for '%s'",FCT,mxGetClassName(a),vn);
00340
00341         wvector<T> x; x.init(FL,a);
00342         if (x!=x_) { x.save2(x_); ++lflag; }
00343         mxDestroyArray(a);
00344     }
00345     else {
00346         int i; unsigned k,e=0;
00347
00348         i=findOpt(vn);
00349         if (i<0) {
00350             if (force) { wblog(F_L,"ERR %s() missing input %s",FCT,vn); }
00351             return 0;
00352         }
00353         k=unsigned(i+1);
00354
00355         if (k>=name.len) { ++e; } else
00356         if (aa[k]==NULL) { ++e; }
00357         if (e) wblog(F_L,"ERR %s() missing value for '%s' (vector)",FCT,vn);
00358
00359         wvector<T> x; x.init(FL,aa[k]);
00360         if (x!=x_) { x.save2(x_); ++lflag; }
00361
00362         name[k-1].init(); aa[k]=NULL;
00363     }
00364
00365     if (F && lflag) {
00366         if (istr && istr[0])
00367             { wblog(F,L," * %-8s: [%s] (%s)", vn, x_.toStr().data, istr); }
00368         else { wblog(F,L," * %-8s: [%s]", vn, x_.toStr().data); }
00369     }
00370
00371     return 1;
00372 }
00373
00374 template<class T>
00375 bool OPTS::getOpt(const char *F, int L,
00376     const char* vn, wMatrix<T>&x_, char force, const char *istr
00377 ){
00378     int lflag=0;
00379     if (MAT.mfp) {
00380         mxArray *a = matGetVariable(MAT.mfp, vn);
00381         if (!a) {
00382             if (force) wblog(F_L,
00383                 "ERR %s() missing value for %s (matrix)",FCT,vn);
00384             return 0;
00385         }
00386
00387         if (!mxIsDouble(a)) wblog(F_L,
00388             "ERR %s() invalid type '%s' for '%s'",FCT,mxGetClassName(a),vn);
00389
00390         wMatrix<T> x; x.init(FL,a);
00391         if (x!=x_) { x.save2(x_); ++lflag; }
00392         mxDestroyArray(a);
00393     }
00394     else {
00395         int i; unsigned k,e=0;
00396
00397         i=findOpt(vn);
00398         if (i<0) {
00399             if (force) wblog(F_L,
00400                 "ERR %s() missing input %s (matrix)",FCT,vn);
00401             return 0;

```

```

00402     }
00403     k=unsigned(i+1);
00404
00405     if (k>=name.len) { ++e; } else
00406     if (aa[k]==NULL) { ++e; }
00407     if (e) wblog(F_L,
00408         "ERR %s() missing input data for '%s' (matrix)",FCT,vn);
00409
00410     wbMatrix<T> x; x.init(FL,aa[k]);
00411     if (x!=x_) { x.save2(x_); ++lflag; }
00412
00413     name[k-1].init(); aa[k]=NULL;
00414 }
00415
00416 if (F && lflag) {
00417     if (istr && istr[0])
00418     { wblog(F,L," * %-8s: [%s] (%s)", vn, x_.toStr().data, istr); }
00419     else { wblog(F,L," * %-8s: [%s]",      vn, x_.toStr().data); }
00420 }
00421
00422 return 1;
00423 }
00424
00425 void OPTS::checkAnyLeft(const char *F, int L) {
00426     unsigned i,e=0;
00427
00428     if (MAT.mfp) {
00429         MAT.init();
00430         for (i=0; i<aa.len; ++i) mxDestroyArray(aa[i]);
00431         return;
00432     }
00433
00434     for (i=0; i<name.len; ++i) { if (name[i].len || aa[i]) {
00435         if ((++e)==1) printf("\n");
00436
00437         if (name[i].len) {
00438             printf("%5d: '%s'\n", i+1, name[i].data);
00439         }
00440         else if (aa[i]) {
00441             int m=mxGetM(aa[i]), n=mxGetN(aa[i]);
00442             if (mxIsCell(aa[i]))
00443                 printf("%5d: {%dx%d %s}\n",i+1,m,n,mxGetClassName(aa[i]));
00444             else printf("%5d: {%dx%d %s}\n",i+1,m,n,mxGetClassName(aa[i]));
00445         }
00446     } }
00447
00448     if (e) { PRINTF("\n"); if (mto) {
00449         wblog(F_L,"WRN some options appear more than once (m=%d)",mto); }
00450         wblog(F_L,"ERR unused/unrecognized options");
00451     }
00452 }
00453 };
00454
00455 #endif
00456

```

5.53 wbperm.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : QSpace permute routines
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_PERM_CC__
00021 #define __WB_PERM_CC__
00022
00023 // ----- //
00024 // ----- //
00025

```

```

00026 char wbperm::isValidPerm(const char *F, int L, wperm_t r) const {
00027
00028     char rval=0, id=0;
00029
00030     if (swperm_t(r)>=0 && r!=len) { if (F) wblog(F,L,
00031         "ERR invalid permutation [%s; %d]",toStr().data,r);
00032         return rval;
00033     }
00034
00035     if (len) { wvector<char> mark(len);
00036         for (wperm_t i=0; i<len; ++i) {
00037             if (i==data[i]) { ++id; } else
00038                 if (data[i]>=len || mark[data[i]]++) { if (F) wblog(F,L,
00039                     "ERR invalid permutation [%s]",toStr().data);
00040                     return rval;
00041                 }
00042         }
00043     }
00044     return (rval=( unsigned(id)==len? 2:1));
00045 };
00046
00047 wbperm& wbperm::init(const wbperm &P, char iflag, unsigned r) {
00048     if (&P!=this) {
00049         if (iflag)
00050             { P.invert(*this); }
00051         else { RENEW(P.len,P.data); }
00052     }
00053     else if (iflag) { Invert(); }
00054
00055     if (int(r)>=0) { Extend(r); }
00056
00057     return *this;
00058 };
00059
00060 wbperm& wbperm::invert(wbperm &iP) const {
00061     if (&iP==this) {
00062         wbperm X; this->invert(X).save2(iP);
00063         return iP;
00064     }
00065
00066     #ifndef SKIP_WB_ASSERT
00067     isValidPerm(FL);
00068     #endif
00069
00070     iP.RENEW(len);
00071
00072     for (wperm_t *p=iP.data, i=0; i<len; ++i) {
00073         if (data[i]>=len) wblog(FL,"ERR %s() "
00074             "permutation out of bounds (%d/%d)",FCT,data[i],len);
00075         p[data[i]]=i;
00076     }
00077     return iP;
00078 };
00079
00080 wbperm& wbperm::Extend(wperm_t r) {
00081
00082     if (r!=len) {
00083         if (int(r)<=0) wblog(FL,"ERR %s() requesting r=%d/%d",FCT,r,len);
00084
00085         if (r>len) {
00086             wbperm P(r);
00087             if (len) { memcpy(P.data,data,len*sizeof(wperm_t)); }
00088             P.save2(*this);
00089         }
00090         else {
00091             for (unsigned i=r; i<len; ++i) {
00092                 if (data[i]!=i) wblog(FL,
00093                     "ERR %s() invalid r=%d having %s",FCT,r,STR_(this));
00094             }
00095             Shorten2(r);
00096         }
00097         return *this;
00098     };
00099
00100     wbperm& wbperm::Complete(wperm_t l) {
00101         if (l<len) {
00102             if (l) {
00103                 unsigned i=0; wvector<char> mark(len);
00104                 for (; i<l; ++i) {
00105                     if (data[i]>=len) wblog(FL,"ERR %s() invalid partial perm\n"
00106                         "(index out of bounds i=%d: %d/%d)",FCT,i+1,data[i],len);
00107                     if (++mark[data[i]]!=1) wblog(FL,"ERR %s() "
00108                         "invalid partial perm (non-unique: %s)",FCT,STR_(this));
00109                 }
00110                 for (i=0; i<len; ++i) { if (!mark[i]) { data[l++]=i; } }
00111             }
00112             else { init(l=len); }

```

```

00113     }
00114     else if (l>len) wblog(FL,
00115         "ERR %s() invalid usage (l=%d/%d)",FCT,l,len);
00116     return *this;
00117 };
00118
00119 int wperm::initStr(
00120     const char *F, int L, const char *s, wperm_t offset
00121 ) {
00122     if (!s) { init(); return -1; }
00123     while (isspace(*s) && *s) ++s;
00124     if (!*s) { init(); return 0; }
00125     unsigned i=0, n=0;
00126     while (isdigit(s[n])) ++n;
00127     for (i=n; isspace(s[i]); ++i) {};
00128     if (!s[i]) { init(n);
00129         for (i=0; i<n; ++i) { data[i]=s[i]-'0'; }
00130         if (int(offset)>=0) {
00131             for (i=0; i<n; ++i) {
00132                 if (data[i]<offset) { if (F) wblog(FL,"ERR %s() "
00133                     "invalid offset ('%s': %d !?)",FCT,s,offset);
00134                     else return -2;
00135                 }
00136                 data[i]-=offset;
00137             }
00138         }
00139         else {
00140             char mark[n+1]; memset(mark,0,n+1);
00141             for (i=0; i<n; ++i) {
00142                 if (data[i]>n || ++mark[data[i]]!=1) { return -4; }
00143             }
00144             if (!mark[0]) {
00145                 for (i=0; i<n; ++i) { --data[i]; }
00146             }
00147             return len;
00148         }
00149     }
00150     else {
00151         if (isdigit(s[i]) || isspace(s[i]) || s[i]==' ' || s[i]==';') {
00152             i=Str2Idx(s,*this,offset);
00153         }
00154         else { i=-(i+1); }
00155         if (int(i)<0) { if (F) wblog(FL,
00156             "ERR %s() invalid permutation (i=%d, offset=%d)"
00157             "%N%N s = '%s' %N",FCT,-i,offset,s);
00158             else return -3;
00159         }
00160     }
00161     i=isValidPerm(F ? F : NULL,F ? L : 0);
00162     return (i ? len : -4);
00163 };
00164
00165 wperm& wperm::init_trafo(const wperm &p1, const wperm &p2) {
00166     if (!p1.len) {
00167         if (p2.len) return init(p2);
00168         else return init();
00169     }
00170     if (!p2.len) { return init(p1,'i'); }
00171     if (p1.len!=p2.len) wblog(FL,
00172         "ERR %s() length mismatch (%d/%d)",FCT,p1.len,p2.len);
00173     if (this==&p1 || this==&p2) {
00174         wperm X; X.init_trafo(p1,p2);
00175         X.save2(*this); return *this;
00176     }
00177     init(p1,'i').Permute(p2);
00178     return *this;
00179 };
00180
00181 wperm& wperm::init2Front(const wvector<wperm_t> &I, wperm_t N) {
00182     wperm_t i,j;
00183     wvector<char> mark(N); char *m=mark.data;
00184     if (I.len>N) wblog(FL,"ERR index too long (%d/%d)",I.len,N);
00185     init(N);

```



```

00200     for (i=0; i<I.len; i++) { j=I[i];
00201         if (j>=N) wblog(FL,"ERR index out of bounds (%d/%d)",j,N);
00202         data[i]=j; if ((++m[j])>1) {
00203             wblog(FL,"ERR %s() index not unique (%d/%d)",FCT,j,N); }
00204     }
00205
00206     if (i<N)
00207     for (j=0; j<N; j++) { if (!m[j]) data[i++]=j; }
00208
00209     return *this;
00210 };
00211
00212 wpper& wpper::init2End(const wvector<wperm_t> &I, wperm_t N) {
00213     wperm_t i,j, l=N-I.len;
00214     wvector<char> mark(N); char *m=mark.data;
00215     if (I.len>N) wblog(FL,"ERR index too long (%d/%d)",I.len,N);
00216
00217     init(N);
00218     for (i=0; i<I.len; i++) { j=I[i];
00219         if (j>=N) wblog(FL,"ERR index out of bounds (%d/%d)",j,N);
00220         data[l+i]=j; if ((++m[j])>1) {
00221             wblog(FL,"ERR %s() index not unique (%d/%d)",FCT,j,N); }
00222     }
00223
00224     if (l)
00225     for (i=j=0; j<N; j++) { if (!m[j]) data[i++]=j; }
00226
00227     return *this;
00228 };
00229
00230 wpper& wpper::init2FrontB(wperm_t m, wperm_t N) {
00231
00232     if (!m || m>N) wblog(FL,
00233         "ERR %s() m out of bounds (%d/%d)",FCT,m,N);
00234
00235     wperm_t i,l=N-m; init(N);
00236
00237     for (i=0; i<m; ++i) { data[i]=i+l; }
00238     for ( ; i<N; ++i) { data[i]=i-m; }
00239
00240     return *this;
00241 };
00242
00243 wpper& wpper::init2EndB(wperm_t m, wperm_t N) {
00244
00245     if (!m || m>N) wblog(FL,
00246         "ERR %s() m out of bounds (%d/%d)",FCT,m,N);
00247
00248     wperm_t i,l=N-m; init(N);
00249
00250     for (i=0; i<l; ++i) { data[i]=i+m; }
00251     for ( ; i<N; ++i) { data[i]=i-l; }
00252
00253     return *this;
00254 };
00255
00256 wpper& wpper::init2Front(wperm_t k, wperm_t N) {
00257     if (k>=N) wblog(FL,
00258         "ERR %s() index out of bounds (%d/%d)",FCT,k+1,N);
00259     wperm_t i=0; RENEW(N); data[0]=k;
00260     for (; i<k; ++i) { data[i+1]=i; }; ++i;
00261     for (; i<N; ++i) { data[i ]=i; };
00262     return *this;
00263 };
00264
00265 wpper& wpper::init2End(wperm_t k, wperm_t N) {
00266     if (k>=N) wblog(FL,
00267         "ERR %s() index out of bounds (%d/%d)",FCT,k+1,N);
00268     wperm_t i=0; RENEW(N); data[N-1]=k;
00269     for (; i<k; ++i) { data[i ]=i; }; ++i;
00270     for (; i<N; ++i) { data[i-1]=i; };
00271     return *this;
00272 };
00273
00274 wpper& wpper::Cycle(wperm_t k1, wperm_t k2) {
00275     if (k1>=len || k2>=len) wblog(FL,
00276         "ERR %s() index out of bounds [%d %d]/%d !?",FCT,k1,k2,len);
00277     if (k1==k2) return *this;
00278
00279     wperm_t x1=data[k1], k=k1;
00280     if (k1<k2)
00281         { for (; k<k2; ++k) { data[k]=data[k+1]; }}
00282     else { for (; k>k2; --k) { data[k]=data[k-1]; }}
00283     data[k]=x1;
00284
00285     return *this;
00286 };

```

```

00287
00288 wbpPerm & wbpPerm::initCycle(wperm_t r, wperm_t k1, wperm_t k2) {
00289
00290     if (k1>=r || k2>=r) wblog(FL,
00291         "ERR %s() index out of bounds [%d %d]/%d !?",FCT,k1,k2,len);
00292     if (k1==k2) { return Index(r); }
00293
00294     RENEW(r);
00295
00296     if (k1<k2) { wperm_t k=0;
00297         for (; k<k1; ++k) { data[k]=k; }
00298         for (; k<k2; ++k) { data[k]=k+1; }; data[k++]=k1;
00299         for (; k<r; ++k) { data[k]=k; }
00300     }
00301     else { wperm_t k=r-1;
00302         for (; k>k1; --k) { data[k]=k; }
00303         for (; k>k2; --k) { data[k]=k-1; }; data[k--]=k1;
00304         for (; k<r; --k) { data[k]=k; }
00305     }
00306
00307     return *this;
00308 };
00309
00310 wbpPerm & wbpPerm::Rotate(swperm_t k) {
00311
00312     if (!len) wblog(FL,
00313         "ERR %s() got empty permutation %d/%d !?",FCT,k,len);
00314     if ((k%len)<0) k+=len;
00315     if (k) {
00316         wbpPerm P0(*this);
00317         for (wperm_t i=0; i<len; ++i) { data[(i+k)%len]=P0[i]; }
00318     }
00319     return *this;
00320 };
00321
00322 wbpPerm & wbpPerm::initRotate(wperm_t r, swperm_t k) {
00323
00324     if (!r) {
00325         if (k) wblog(FL,
00326             "ERR %s() got empty permutation %d/%d !?",FCT,k,r);
00327         return Index(r);
00328     }
00329
00330     if ((k%r)<0) k+=r;
00331     if (!k) return Index(r);
00332
00333     RENEW(r);
00334     for (wperm_t i=0; i<len; ++i) { data[(i+k)%len]=i; }
00335     return *this;
00336 };
00337
00338 bool wbpPerm::sameAs(const wbpPerm &b) const {
00339     if (!len) {
00340         if (b.len)
00341             { return b.isIdentityPerm(); }
00342         else { return 1; }
00343     }
00344     if (!b.len)
00345         { return isIdentityPerm(); }
00346     else { return (*this)==b; }
00347 };
00348
00349 bool wbpPerm::isIdentityPerm() const {
00350     bool id=1; if (len) {
00351         for (unsigned i=0; i<len; ++i) { if (data[i]!=i) { id=0; break; } };
00352     }
00353     return id;
00354 };
00355
00356 bool wbpPerm::isIdentityPerm(const char *F, int L, wperm_t r) const {
00357     if (int(r)>=0 && len!=r) { if (F) wblog(F,L,
00358         "ERR %s() invalid permutation (len=%d/%d)",FCT,len,r);
00359         return 0;
00360     }
00361     bool q=1; char mark[len]; memset(mark,0,len);
00362     wperm_t i=0;
00363
00364     for (; i<len; ++i) {
00365         if (data[i]==i) { ++mark[i]; } else { q=0; break; }
00366     }
00367     for (; i<len; ++i) {
00368         if (data[i]>=len) wblog(F_L, "ERR %s() invalid permutation\n"
00369             "(index out of range %d/%d)",FCT,data[i],len);
00370         if ((++mark[data[i]])>1) wblog(F_L,"ERR %s() "
00371             "invalid permutation (non-unique index)",FCT);
00372     }
00373     return q;

```

```

00374 };
00375
00376 bool wbperm::isReversePerm() const {
00377     if (len==0) return 0;
00378     for (wperm_t m=len-1, i=0; i<len; i++) if (data[i]!=m-i) return 0;
00379     return 1;
00380 };
00381
00382 bool wbperm::isCyclic2F(wperm_t m, wperm_t n,
00383     char iflag
00384 ) const {
00385
00386     if (swperm_t(n)<0) { n = (len>m ? (len-m) : 0); }
00387     if (!len) { return (m || n ? 0 : 1); }
00388     if (len!=m+n) wblog(FL,
00389         "ERR %s() length mismatch (%d / %d + %d)",FCT,len,m,n);
00390
00391     if (iflag) { SWAP(m,n); }
00392
00393     wperm_t i=0;
00394     for (; i<m; ++i) { if (data[i]!=i+n) { return 0; }}
00395     for (; i<len; ++i) { if (data[i]!=i-m) { return 0; }}
00396     return 1;
00397 };
00398
00399 bool wbperm::isOpTranspose() const {
00400
00401     if (len!=3 && len%2) { return 0; }
00402
00403     for (wperm_t i=0, r=len/2; i<r; ++i) {
00404         if (data[i]!=i+r || data[i+r]!=i) return 0;
00405     }
00406     return 1;
00407 };
00408
00409 wbperm& wbperm::Permute(const wbperm &P, char iflag, wperm_t r){
00410
00411     if (P.len && !P.isIdentityPerm(FL,r)) {
00412         if (!len || isIdentityPerm(FL,r)) {
00413             if (iflag)
00414                 { P.invert(*this); }
00415             else { init(P); }
00416         }
00417         else {
00418             wbperm X(*this);
00419             wperm_t i=0, *x=X.data, *p=P.data;
00420
00421             if (iflag)
00422                 { for (; i<r; ++i) { data[p[i]] = x[i]; }}
00423             else { for (; i<r; ++i) { data[i] = x[p[i]]; }}
00424         }
00425     }
00426     return *this;
00427 };
00428
00429 #endif
00430

```

5.54 wbperm.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : wbvector (template vector class)
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_PERM_HH__
00021 #define __WB_PERM_HH__
00022
00023 // ----- //

```

```

00024 // permutation class
00025 // Wb, Apr19, 06
00026 // ----- //
00027
00028 class wbperm : public wbvector<wperm_t> {
00029
00030 public:
00031
00032     wbperm() {};
00033
00034     wbperm(wperm_t n, char reverse=0) { Index(n,reverse); };
00035
00036     wbperm(const wbperm &P, char iflag=0, unsigned r=-1)
00037         : WBPERM() { init(P,iflag,r); };
00038
00039     wbperm(const wbperm &P, size_t r) { init_pad(P,r); };
00040
00041     template<class T>
00042     wbperm(const wbvector<T> &b, wperm_t offset=0) {
00043         init(FL,b,offset);
00044     };
00045
00046     wbperm(const wbindex &P);
00047
00048     wbperm(const char *s, wperm_t offset=1) {
00049         initStr(FL,s,offset);
00050     }
00051
00052     wperm_t ell(wperm_t i) const {
00053         if (len) {
00054             if (i>=len) wblog(FL,
00055                 "ERR element index out of bounds (%d/%d)",i+1/len);
00056             return data[i];
00057         }
00058         else return i;
00059     };
00060
00061     wbperm& operator= (const wbperm &P) { return init(P); };
00062     wbperm& operator= (const char *s) {
00063         initStr(FL,s,0); return *this;
00064     };
00065
00066     int initStr(
00067         const char *F, int L, const char *s, wperm_t offset=1);
00068
00069     wbperm& init(const wbperm &P, char iflag=0, unsigned r=-1);
00070     wbperm& Extend(wperm_t r);
00071
00072     wbperm& init_trafo(const wbperm &p1_, const wbperm &p2_);
00073
00074     wbperm& init_pad(const wbperm &P, wperm_t r) {
00075         if (r<P.len) wblog(FL,
00076             "ERR invalid %s construction (%d/%d)",FCT,r,P.len);
00077         RENEW(r); memcpy(data,P.data,P.len*sizeof(P.data[0]));
00078         for (wperm_t i=P.len; i<r; i++) data[i]=i;
00079         if (!isValidPerm()) dispInvalidPerm(FL);
00080         return *this;
00081     };
00082
00083     wbperm& init(unsigned n=0, char reverse=0) {
00084         return Index(n,reverse);
00085     };
00086
00087     wbperm& init(unsigned n, const wperm_t* d, char ref=0) {
00088         wbvector<wperm_t>::init(n,d,ref); isValidPerm(FL);
00089         return *this;
00090     };
00091
00092     wbperm& init(const char *F, int L, const mxArray* a){
00093         return init(F,L,a,mxIsChar(a) ? 1 : 0);
00094     };
00095
00096     wbperm& init(const char *F, int L,
00097         const mxArray* a, wperm_t offset
00098     ){
00099         if (mxIsNumeric(a)) {
00100             WBPERM::init(F,L,a); if (offset) { operator--(offset); }
00101             isValidPerm(F_L);
00102         }
00103         else if (mxIsChar(a)) {
00104             int l=mxGetNumberOfElements(a)+1;
00105             char s[l]; mxGetString(a,s,l);
00106             initStr(F_L,s,offset);
00107         }
00108         else wblog(FL,
00109             "ERR %s() invalid input (%s) !?",FCT,mxGetClassName(a));
00110         return *this;

```

```

00111     };
00112
00113     template<class T>
00114     wpperm& init(
00115         const char *F, int L, const wvector<T> &b, wperm_t offset=0
00116     ){
00117         initT(b.len,b.data);
00118         if (offset) { (*this)-=offset; }; isValidPerm(F_L);
00119         return *this;
00120     };
00121
00122     wpperm& initTranspose(wperm_t r) {
00123         WPERM::init(r); wperm_t r2=r/2;
00124         if (r%2) wblog(FL,"ERR %s() even input rank required (%d)",FCT,r);
00125         for (wperm_t i=0; i<r2; ++i) { data[i]=r2+i; data[r2+i]=i; }
00126         return *this;
00127     };
00128
00129     wpperm& initPerm2(wperm_t r) {
00130         WPERM::init(r); wperm_t r2=r/2;
00131         if (r%2) wblog(FL,"ERR %s() even input rank required (%d)",FCT,r);
00132         for (wperm_t l=0,i=0; i<r2; ++i) {
00133             data[l++]=i;
00134             data[l++]=r2+i;
00135         }
00136         return *this;
00137     };
00138
00139     wpperm& initFirstTo (wperm_t k, wperm_t N) {
00140         Index(N); mvFirstTo(k);
00141         return *this;
00142     };
00143
00144     wpperm& initLastTo (wperm_t k, wperm_t N) {
00145         Index(N); mvLastTo(k);
00146         return *this;
00147     };
00148
00149     wpperm& init2Front(const WPERM &I, wperm_t N);
00150     wpperm& init2End  (const WPERM &I, wperm_t N);
00151
00152     wpperm& init2FrontB(wperm_t m, wperm_t N);
00153     wpperm& init2EndB  (wperm_t m, wperm_t N);
00154
00155     wpperm& init2Front(wperm_t k, wperm_t N);
00156     wpperm& init2End  (wperm_t k, wperm_t N);
00157
00158     wpperm& Cycle(wperm_t k1, wperm_t k2);
00159     wpperm& initCycle(wperm_t n, wperm_t k1, wperm_t k2);
00160
00161     wpperm& Index(wperm_t n, char reverse=0) {
00162         RENEW(n); if (n==0) return *this;
00163         if (reverse)
00164             for (wperm_t l=n-1, i=0; i<n; ++i) data[i]=l-i;
00165         else for (wperm_t i=0; i<n; ++i) data[i]=i;
00166         return *this;
00167     };
00168
00169     wpperm& Index(wperm_t i1, wperm_t i2) {
00170         if (i1>i2) { RENEW(0); return *this; }
00171
00172         RENEW(i2-i1+1);
00173         for (wperm_t i=0; i<len; ++i) data[i]=i+i1;
00174
00175         return *this;
00176     };
00177
00178     char isValidPerm(const char *F, int L, wperm_t r=-1) const;
00179     char isValidPerm(wperm_t r=-1) const { return isValidPerm(0,0,r); };
00180
00181     wpperm& Complete(wperm_t l);
00182
00183     wpperm& Invert() {
00184         wpperm iP; this->invert(iP).save2(*this);
00185         return *this;
00186     };
00187
00188     wpperm& invert(wpperm &iP) const;
00189
00190     wpperm& Rotate(swperm_t l);
00191     wpperm& initRotate(wperm_t n, swperm_t k);
00192
00193     wpperm& Permute(const wpperm &P, char iflag, wperm_t r);
00194     wpperm& Permute(const wpperm &P, char iflag=0) {
00195         return Permute(P, iflag, len>P.len ? len : P.len);
00196     };
00197

```

```

00198     bool sameAs(const wbperm &b) const;
00199     bool isIdentityPerm() const;
00200     bool isIdentityPerm(const char *F, int L, wperm_t r=-1) const;
00201
00202     bool isReversePerm() const;
00203     bool isCyclic2F(wperm_t m, wperm_t n=-1, char iflag=0) const;
00204
00205     bool isOpTranspose() const;
00206
00207     explicit operator bool() const {
00208         return (isEmpty() || isIdentityPerm() ? 0 : 1); };
00209     bool operator! () const { return !bool(*this); }
00210
00211     wbperm& times(const wbperm &p2, wbperm &Pout) const {
00212         if (!isValidPerm()) wblog(FL,
00213             "ERR %s() invalid permutation (%s)",__FUNCTION__,toStr().data);
00214         if (!p2.isValidPerm()) wblog(FL,
00215             "ERR %s() invalid permutation (%s)",__FUNCTION__,STR(p2));
00216         if (len!=p2.len) wblog(FL,
00217             "ERR %s() incompatible permutations (%d/%d)",
00218             __FUNCTION__,len,p2.len);
00219
00220         wbperm P(len);
00221         for (wperm_t i=0; i<len; i++) P.data[i]=data[p2.data[i]];
00222
00223         P.save2(Pout); return Pout;
00224     };
00225
00226     wbperm& flipIdx(wbperm &P) const;
00227     wbperm& FlipIdx();
00228     wbperm flipIdx() const {
00229         wbperm P; flipIdx(P);
00230         return P;
00231     };
00232
00233     void ToRaw() { ToRaw(len); };
00234     void ToRaw(wperm_t n) {
00235         if (n<len) wblog(FL,"WRN ToRaw() with n=%d/%d !?",n,len);
00236         if (n==0) return; else n--;
00237
00238         Flip();
00239
00240         for (wperm_t i=0; i<len; i++) {
00241             if (i>n) wblog(FL,"ERR wbperm - invalid n=%d (%d)",n,i);
00242             data[i]=n-data[i];
00243         }
00244     }
00245
00246     wstring toStr() const {
00247         wstring s_;
00248         unsigned i=0; char *s;
00249
00250         if (len<10) {
00251             s_.init(len+1); s=s_.data;
00252             for (; i<len; ++i) { s[i]='1'+data[i]; }
00253             s[i]=0;
00254         }
00255         else {
00256             unsigned l=0, n=len*(log10(double(len))+2);
00257             s_.init(n); s=s_.data;
00258             for (; i<len && l<n; ++i) {
00259                 l+=snprintf(s+l,n-l, " %d", (int)(data[i]+1));
00260             }
00261             if (l+3>=n) wblog(FL,"WRN %s() "
00262                 "string out of bounds (%d/%d; %d/%d)",FCT,l,n,i+1,len);
00263             else { s[l]=' '; s[l+1]=0; }
00264         }
00265         return s_;
00266     };
00267
00268     mxArray* toMx(const char tflag=0) const {
00269         return toMx_offset(l,tflag);
00270     };
00271
00272 protected:
00273 private:
00274
00275     void dispInvalidPerm(const char* F, int L) const {
00276         wblog(F,L,"WRN invalid permutation [%s]",toStr().data); };
00277
00278 };
00279
00280
00281 int isValidPerm(const char *s, wperm_t r=-1) {
00282     int q=0; wbperm P; int i;
00283     if ((i=P.initStr(0,0,s))<0) { q=-1; } else
00284     if (P.isValidPerm(r)) { q=P.len; } else { q=-2; }

```

```

00285     return q;
00286 };
00287
00288 int isValidPerm(const mxArray *a, wperm_t r=-1) {
00289     int q=0;
00290
00291     if (!a) { q=-11; } else
00292     if (mxIsChar(a)) {
00293         wbststring S(FL,a); if (S) {
00294             q=isValidPerm(S.data,r);
00295         }
00296     }
00297     else if (!mxIsNumeric(a)) { q=-12; }
00298     else if (!Mx::IsVector(a)) {
00299         if (mxGetNumberOfElements(a) q=-13;
00300     }
00301     else {
00302         wvector<double> X(FL,a);
00303         if (int(r)>=0 && X.len!=r) { q=-3; } else
00304         if (X) {
00305             unsigned i=0, j, n=X.len; double *x=X.data;
00306             wvector<char> m(n+1);
00307             for (; i<n; ++i) { j=x[i];
00308                 if (x[i]!=j || j>n || ++m[j]!=1) { break; }
00309             }
00310             q=(i<n || (m[0] && m[n]) ? -4 : n);
00311         }
00312     }
00313     return q;
00314 };
00315
00316 wperm& wperm::flipIdx(wperm &P) const {
00317     if (&P==this) wblog(FL,
00318         "ERR flipIdx(P) calls itself\nused flipIdx() instead!");
00319
00320     P.RENEW(len); if (!len) return P;
00321
00322     for (wperm_t m=len-1, i=0; i<len; i++)
00323         P.data[i]=m-data[i];
00324
00325     return P;
00326 };
00327
00328 inline wperm& wperm::FlipIdx() {
00329     for (wperm_t m=len-1, i=0; i<len; ++i) {
00330         if (data[i]>m) wblog(FL,"ERR index out of bounds (%d/%d)",data[i],m);
00331         data[i]=m-data[i];
00332     }
00333
00334     return *this;
00335 };
00336
00337 #endif
00338
00339 #endif
00340

```

5.55 wbsort.cc

```

00001 #ifndef __WB_REC_HPSORT_CC__
00002 #define __WB_REC_HPSORT_CC__
00003
00004 /* ----- *
00005 * Heap sort algorithm for vector array
00006 * Reference: Numerical Recipies C, p337f
00007 * AW (C) Jan 2006
00008 * ----- *
00009 * mergesort (ranks #2, vs. #3 for hpsort)
00010 *   + on average faster than hpsort
00011 *   + stable (does not reverse order within degenerate subspaces)
00012 *   - requires worst case an additional space of size N
00013 *   + in-place merge-sort: implemented in C++ STL (!) [stable_sort]
00014 *   + easily parallelizes!
00015 *   * while quicksort is faster in typical cases, it is
00016 *     (i) not stable, and (ii) has a worst-case performance of N^2
00017 * hpsort, for comparsion
00018 *   * is NOT stable (ie. may reverse order within degenerate subspaces)
00019 *   * slower on average, while in-place
00020 *   * still performs MEM_CPY even on already sorted array (!?)
00021 *   * reverse sort is actually faster than sorting a sorted array (!?)
00022 * Wb,Dec29,11
00023 * ----- */
00024

```

```

00025 // by including P, this ensures to keep initial order of degenerate
00026 // subspaces, in that what is actually is sorted is [ra,(1:N)']
00027
00028 template<class T>
00029 void Wb::hpsort(
00030     wbvector<T> &ra,
00031     wbperm &P,
00032     char dir
00033 ){
00034     size_t i,j,l,ir, N=ra.len; wperm_t ia,*p;
00035     T rra;
00036
00037     char q, Q=(dir>0 ? +1 : -1);
00038
00039     P.Index(N); p=P.data; if (N<2) return;
00040
00041     l = (N>1);
00042     ir= N-1;
00043
00044     for (;;) {
00045         if(l>0) {
00046             rra=ra[--l];
00047             ia=p[l];
00048         } else {
00049             rra=ra[ir];
00050             ra[ir]=ra[0];
00051
00052             ia=p[ir]; p[ir]=p[0];
00053
00054             if ((--ir)==0) {
00055                 ra[0]=rra;
00056                 p[0]=ia;
00057                 break;
00058             }
00059         }
00060
00061         i=1;
00062
00063         j=l+l+1;
00064         while (j<=ir) {
00065             if (j<ir) {
00066                 q=NUMCMP (ra[j],ra[j+1]); if (q==0) q=NUMCMP (p[j],p[j+1]);
00067                 if (q!=Q) j++;
00068             }
00069
00070             q=NUMCMP (rra,ra[j]); if (q==0) q=NUMCMP (ia,p[j]);
00071             if (q!=Q) {
00072                 ra[i]=ra[j];
00073                 p[i]=p[j];
00074
00075                 i = j;
00076                 j = ((j+1)<<1) - 1;
00077             }
00078             else break;
00079         }
00080         ra[i]=rra;
00081         p[i]=ia;
00082     }
00083
00084     return;
00085 };
00086
00087 template<class T>
00088 int Wb::hpsort(
00089     T *A,
00090     size_t lda,
00091     size_t N,
00092     wbperm &P,
00093     char dir,
00094     char lex
00095 ){
00096     size_t i,j,l, ia,ir, *p;
00097
00098     char q, Q=(dir>0 ? +1 : -1);
00099
00100     P.Index(N); p=P.data; if (N<2) return 0;
00101
00102     l=(N>1); ir=N-1;
00103     T x[lda];
00104
00105     for (;;) {
00106         if (l>0) {
00107             ia=p[--l]; MEM_COPY<T>(x,lda,A+l*lda);
00108         }
00109         else {
00110             ia=p[ir]; MEM_COPY<T>(x,lda,A+ir*lda);
00111             p[ir]=p[0]; MEM_COPY<T>(A+ir*lda,lda,A);

```



```

00112
00113         if ((--ir)==0) {
00114             p[0]=ia; MEM_COPY<T>(A,lda,x);
00115             break;
00116         }
00117     }
00118
00119     i=1; j=2*1+1;
00120
00121     while (j<=ir) { T *aj=A+j*lda;
00122         if (j<ir) {
00123             q=Wb::recCompare(aj,aj+lda,lda,lex);
00124             if (q==0) q=NUMCMP(p[j],p[j+1]);
00125             if (q!=Q) { ++j; aj+=lda; }
00126         }
00127
00128         q=Wb::recCompare(x,aj,lda,lex); if (q==0) q=NUMCMP(ia,p[j]);
00129         if (q!=Q) {
00130             MEM_COPY<T>(A+i*lda,lda,A+j*lda);
00131             p[i]=p[j];
00132
00133             i=j;
00134             j=((j+1)<<1)-1;
00135         }
00136         else break;
00137     }
00138     p[i]=ia; MEM_COPY<T>(A+i*lda,lda,x);
00139 }
00140
00141 return 0;
00142 };
00143
00144 template<class T>
00145 int Wb::hpsort(
00146     wbMatrix<T> &ra, wbperm &P, char dir, char lex
00147 ){ return hpsort(ra.data, ra.dim2, ra.dim1, P, dir, lex); };
00148
00149 template<class T>
00150 void Wb::hpsort(wbvector<T> &ra) { wbperm P; hpsort(ra,P); }
00151
00152 template <class T> inline
00153 size_t Wb::matchSortedIdx_grp(
00154     const T *A, int lda, size_t na,
00155     const T *B, int ldb, size_t nb,
00156     wbindex &gra, wbindex &grb, size_t &mtot,
00157     int m,
00158     char lex,
00159     T eps
00160 ){
00161     size_t ia,ib,iga=0,igb=0,ng=0,m1,m2,*p1,*p2;
00162     int nwrn=0; char c, cr=0;
00163     const T *a,*b;
00164
00165     if (m<0) { m=lda;
00166         if (lda!=ldb) wblog(FL,
00167             "WRN %s() got different record length (%d/%d)",FCT,lda,ldb);
00168     }
00169
00170     if (!m) wblog(FL,"ERR finite m required "
00171         " (m=%d; %dx%d <> %dx%d)",m,lda,na,ldb,ldb);
00172     if (m>lda || m>ldb || lda<=0 || ldb<=0) wblog(FL,
00173         "ERR m out of bounds (m=%d; %d/%d)",m,lda,ldb);
00174     if (!A || !B) wblog(FL,
00175         "ERR %s() got null data (0x%lX, 0x%lX)",FCT,A,B);
00176
00177     gra.init(2*na); grb.init(2*nb); mtot=0;
00178
00179     if (lex<=0) {
00180         A+=(lda-m);
00181         B+=(ldb-m);
00182     }
00183
00184     a=A; b=B;
00185     for (ia=1; ia<na; ++ia, a+=lda) {
00186         c=Wb::recCompare(a,a+lda,m,lex,eps,&nwrn);
00187         if (c) { cr=c; break; }
00188     }
00189     if (!cr) {
00190         for (ib=1; ib<nb; ++ib, b+=ldb) {
00191             c=Wb::recCompare(b,b+ldb,m,lex,eps,&nwrn);
00192             if (c) { cr=c; break; }
00193         };
00194         if (!cr) { cr=-1; }}
00195
00196     for (a=A,b=B, ia=ib=0; ia<na && ib<nb;) {
00197         c=Wb::recCompare(a,b,m,lex,eps,&nwrn);
00198     }

```

```

00199         if (c== cr) { ++ia; a+=lda; continue; } else
00200         if (c== -cr) { ++ib; b+=ldb; continue; } else
00201         if (c) wblog(FI, "ERR %s() c=%d/%d !?", FCT, c, cr);
00202
00203         m1=m2=1; ++ng;
00204         p1=gra.data+(iga++); gra[iga]=ia++; a+=lda;
00205         p2=grb.data+(igb++); grb[igb]=ib++; b+=ldb;
00206
00207         while (ia<na) {
00208             c=Wb::recCompare(a-lda,a,m,lex,eps,&nwrn);
00209             if (!c) { gra[iga]=ia++; a+=lda; ++m1; }
00210             else {
00211                 if (c!=cr) wblog(FI,
00212                     "ERR %s() got unsorted recs (%d: %d/%d)", FCT, ia+1, c, cr);
00213                 break;
00214             }
00215         }
00216         while (ib<nb) {
00217             c=Wb::recCompare(b-ldb,b,m,lex,eps,&nwrn);
00218             if (!c) { grb[igb]=ib++; b+=ldb; ++m2; }
00219             else {
00220                 if (c!=cr) wblog(FI,
00221                     "ERR %s() got unsorted recs (%d: %d/%d)", FCT, ib+1, c, cr);
00222                 break;
00223             }
00224         }
00225         mtot+=m1*m2; (*p1)=m1; (*p2)=m2;
00226     }
00227
00228     gra.Shorten2(iga);
00229     grb.Shorten2(igb);
00230
00231     return ng;
00232 };
00233
00234 template <class T> inline
00235 size_t Wb::matchSortedIdx(
00236     const T *A, int lda, size_t na,
00237     const T *B, int ldb, size_t nb, wbindex &Ia, wbindex &Ib,
00238     int m, char lex,
00239     widxt *ma_,
00240     widxt *mb_,
00241     T eps
00242 ){
00243     size_t i,j,k=0, ig=0,iga=0,igb=0, m1,m2,mt=0,ma=0,mb=0, ng,*p1,*p2;
00244     wbindex gra,grb;
00245
00246     ng=Wb::matchSortedIdx_grp(A,lda,na,B,ldb,nb,gra,grb,mt,m,lex,eps);
00247     Ia.init(mt); Ib.init(mt); mt=0;
00248
00249     for (; ig<ng; ++ig) { m1=gra[iga]; m2=grb[igb];
00250         if (m1==1 && m2==1) {
00251             Ia[k]=gra[iga+1]; iga+=2;
00252             Ib[k]=grb[igb+1]; igb+=2; ++k;
00253         }
00254         else {
00255             p1=gra.data+(iga+1); iga+=(m1+1);
00256             p2=grb.data+(igb+1); igb+=(m2+1);
00257
00258             ma+=(m1-1);
00259             mb+=(m2-1); mt+=(m1*m2-1);
00260
00261             for (i=0; i<m1; ++i)
00262                 for (j=0; j<m2; ++j, ++k) { Ia[k]=p1[i]; Ib[k]=p2[j]; }
00263         }
00264     }
00265
00266     if (ma_) { (*ma_)=ma; }
00267     if (mb_) { (*mb_)=mb; }
00268
00269     return mt;
00270 };
00271
00272 template <class T> inline
00273 size_t Wb::matchSortedIdx(
00274     const T *A, int lda, size_t na,
00275     const T *B, int ldb, size_t nb,
00276     wbindex &Ia, wbindex &Ib, wvector<widxt> &D,
00277     int m, char lex, T eps
00278 ){
00279     size_t i,j,k=0, ig=0,iga=0,igb=0, m1,m2,mt=0, ng,*p1,*p2;
00280     wbindex gra, grb;
00281
00282     ng=Wb::matchSortedIdx_grp(A,lda,na,B,ldb,nb,gra,grb,mt,m,lex,eps);
00283     Ia.init(mt); Ib.init(mt); D.init(ng); mt=0;
00284
00285     for (; ig<ng; ++ig) { m1=gra[iga]; m2=grb[igb];

```

```

00286     if (m1==1 && m2==1) {
00287         Ia[k]=gra[iga+1]; iga+=2;
00288         Ib[k]=grb[igb+1]; igb+=2; D[ig]=1; ++k;
00289     }
00290     else {
00291         p1=gra.data+(iga+1); iga+=(m1+1);
00292         p2=grb.data+(igb+1); igb+=(m2+1);
00293
00294         D[ig]=m1*m2; mt+=(D[ig]-1);
00295
00296         for (i=0; i<m1; ++i)
00297             for (j=0; j<m2; ++j, ++k) { Ia[k]=p1[i]; Ib[k]=p2[j]; }
00298     }
00299 }
00300
00301 return mt;
00302 };
00303
00304 #endif
00305

```

5.56 wbsort.hh

```

00001 #ifndef __WB_REC_HPSORT_HH__
00002 #define __WB_REC_HPSORT_HH__
00003
00004 namespace Wb {
00005
00006 /* ----- */
00007 // generalized matrix/record sort
00008 // sorted by rows/records of length m, with actual leading length lda
00009 /* ----- */
00010
00011 template<class T>
00012 int hpsort(
00013     T *A,
00014     size_t lda,
00015     size_t N,
00016     wbperm &P,
00017     char dir=+1,
00018     char lex=+1
00019 );
00020
00021 template<class T>
00022 int hpsort(
00023     wbMatrix<T> &ra,
00024     wbperm &P,
00025     char dir=+1,
00026     char lex=+1
00027 );
00028
00029 template<class T>
00030 void hpsort(
00031     wbvector<T> &ra,
00032     wbperm &P,
00033     char dir=+1
00034 );
00035
00036 template<class T>
00037 void hpsort(wbvector<T> &ra);
00038
00039 template <class T> inline
00040 size_t matchSortedIdx_grp(
00041     const T *da, int lda, size_t na,
00042     const T *db, int ldb, size_t nb,
00043     wbindex &gra, wbindex &grb, size_t &mtot,
00044     int m,
00045     char lex,
00046     T eps=0
00047 );
00048
00049 template <class T> inline
00050 size_t matchSortedIdx(
00051     const T *da, int lda, size_t na,
00052     const T *db, int ldb, size_t nb, wbindex &Ia, wbindex &Ib,
00053     int m, char lex,
00054     widx_t *ma=NULL,
00055     widx_t *mb=NULL,
00056     T eps=0
00057 );
00058
00059 template <class T> inline
00060 size_t matchSortedIdx(

```

```

00061     const T *da, int lda, size_t na,
00062     const T *db, int ldb, size_t nb,
00063     wbindex &Ia, wbindex &Ib, wbvector<widx_t> &D,
00064     int m, char lex, T eps=0
00065 );
00066
00067 };
00068
00069 #endif
00070

```

5.57 wbsparray.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : wbsparray (sparse array of arbitrary dimension)
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_SPARRAY_CC__
00021 #define __WB_SPARRAY_CC__
00022
00023 // ----- //
00024 // check whether A=this fits into B for all dimensions
00025 // return value
00026 // 0 A=this does not fit into B (is larger in at least one dimension)
00027 // else A fits, where
00028 // 1 A has same SIZE.len (strict)
00029 // 2 B.SIZE.len is longer than in A (B has non-zero extra dimension)
00030 // 3 A.SIZE.len is longer than in B (A has trailing singleton dimensions)
00031 // 4 A and B have diagonal format
00032 // Wb,Jun22,20
00033
00034 template <class TD>
00035 char wbsparray<TD>::fitsSize(const wbsparray<TD> &B, char strict) const {
00036
00037     bool i1=isDiag(), i2=B.isDiag();
00038     if (i1 || i2) {
00039         if (!i1 || !i2) wblog(FL,
00040             "ERR %s() got mixed diag format (%d/%d)",FCT,i1,i2);
00041         return (D.len<=B.D.len ? 4 : 0);
00042     }
00043
00044     unsigned i=0, n=MIN(SIZE.len,B.SIZE.len);
00045     const SPIDX_T *s1=SIZE.data, *s2=B.SIZE.data;
00046
00047     for (; i<n; ++i) { if (s1[i]>s2[i]) return 0; }
00048
00049     if (SIZE.len==B.SIZE.len) { return 1; }
00050     if (strict) { return 0; }
00051
00052     if (SIZE.len<B.SIZE.len) {
00053         for (n=B.SIZE.len; i<n; ++i) { if (!s2[i]) return 0; }
00054         return 2;
00055     }
00056     else {
00057         for (n=SIZE.len; i<n; ++i) { if (s1[i]>1) return 0; }
00058         return 3;
00059     }
00060 };
00061
00062 template <class TD>
00063 char wbsparray<TD>::sameSize(const wbsparray<TD> &B, char strict) const {
00064
00065     if (SIZE.len!=B.SIZE.len && !strict) {
00066         if ( isDiag(FL)) return (B.isSMatrix(FL, D.len) ? 11 : 0);
00067         if (B.isDiag(FL)) return ( isSMatrix(FL,B.D.len) ? 12 : 0);
00068
00069         char s=sameSizeUp2Singletons(B.SIZE);
00070         if (s) {

```

```

00071         if (s>2)
00072             return 23;
00073         else {
00074             return (SIZE.len>B.SIZE.len ? 21 : 22);
00075         }
00076     }
00077     else return 0;
00078 }
00079
00080 if (SIZE==B.SIZE) {
00081     if (!SIZE.len && D.len)
00082         return (D.len==B.D.len ? 13 : 0);
00083     else return 1;
00084 }
00085
00086 return 0;
00087 };
00088
00089 template <class TD> inline
00090 char wbsparray<TD>::sameSizeUp2Singletons(
00091     const wbvector<SPIDX_T> &S, const char strict) const {
00092
00093     if (SIZE==S) return 1;
00094     if (numel()!=S.prod(0)) return 0;
00095
00096     if (isDiag()) {
00097         if (S.len<2 || S[0]!=S[1] || S[0]!=D.len) return 0;
00098         if (S.len==2) return 1;
00099         if (!strict) {
00100             for (unsigned i=2; i<S.len; ++i) if (S[i]!=1) return 0;
00101             return 2;
00102         }
00103         return 0;
00104     }
00105
00106     { char ok=1;
00107       unsigned n,N, i=0; const SPIDX_T *s;
00108
00109       if (SIZE.len<S.len)
00110           { n=SIZE.len; N= S.len; s= S.data; }
00111       else { n= S.len; N=SIZE.len; s=SIZE.data; }
00112
00113       for (; i<n; ++i) { if (SIZE[i]!=S[i]) { ok=0; break; } }
00114       if (ok) {
00115           for (; i<N; ++i) if (s[i]!=1) { ok=0; break; } }
00116
00117       if (ok) return 2;
00118     }
00119
00120     if (!strict) {
00121         unsigned i=0, j=0, ra=SIZE.len, rb=S.len;
00122         const SPIDX_T *sa=SIZE.data, *sb=S.data;
00123
00124         for (; i<ra; ++i) { if (sa[i]!=1) {
00125             for (; j<rb; ++j) { if (sb[j]!=1) break; }
00126             if (j<rb && sa[i]==sb[j]) { ++j; }
00127             else return 0;
00128         } }
00129         for (; j<rb; ++j) { if (sb[j]!=1) return 0; }
00130         return 3;
00131     }
00132
00133     return 0;
00134 };
00135
00136 template <class TD> inline
00137 char wbsparray<TD>::matchWithSingletons(
00138     const char *F, int L, const wbvector<SPIDX_T> &S, wbvector<int> &M
00139 ) const {
00140
00141     unsigned i=0, j=0, e=0, ra=SIZE.len, rb=S.len;
00142     const SPIDX_T *sa=SIZE.data, *sb=S.data;
00143
00144     if (!S.len) {
00145         if (SIZE.len) wblog(FL,"ERR %s() got %s <> ()",FCT,SSTR_(this));
00146         M.init(); return 0;
00147     }
00148
00149     M.init(S.len).set(-1);
00150
00151     for (; i<ra; ++i) { if (sa[i]!=1) {
00152         for (; j<rb; ++j) { if (sb[j]!=1) break; }
00153         if (j<rb && sa[i]==sb[j]) { M[j++]=i; }
00154         else { ++e; break; }
00155     } }
00156
00157     if (!e)

```

```

00158     for (; j<rb; ++j) { if (sb[j]!=1) { ++e; break; }}
00159
00160     if (e) {
00161         if (F) wblog(F,L,"ERR %s() got size mismatch (%s <> %s)",
00162             FCT, SSTR_(this), SSTR(S));
00163         return e;
00164     }
00165
00166     return 0;
00167 };
00168
00169 template <class TD> inline
00170 bool wbsparray<TD>::hasSize(SPIDX_T d1) const {
00171     return (SIZE.len==1 && SIZE[0]==d1);
00172 };
00173
00174 template <class TD> inline
00175 bool wbsparray<TD>::hasSize(SPIDX_T d1, SPIDX_T d2) const {
00176     return (SIZE.len==2 && SIZE[0]==d1 && SIZE[1]==d2);
00177 };
00178
00179 template <class TD> inline
00180 bool wbsparray<TD>::hasSize(SPIDX_T d1, SPIDX_T d2, SPIDX_T d3) const {
00181     return (SIZE.len==3 && SIZE[0]==d1 && SIZE[1]==d2 && SIZE[2]==d3);
00182 };
00183
00184 template <class TD> inline
00185 wbsparray<TD>& wbsparray<TD>::setRand(double x, char sflag){
00186
00187     SPIDX_T n=numel(); if (!n) return *this;
00188     n=MIN(n,SPIDX_T(x*pow(double(n),1./rank(FL))));
00189
00190     init_nnz(n); if (!n) return *this;
00191
00192     SPIDX_T i,j, m=SIZE.len, *I=IDX.data;
00193     windex I1; Wb::rand R;
00194
00195     for (i=0; i<n; ++i, I+=m) {
00196         for (j=0; j<m; ++j) {
00197             R.rand_(I[j], SIZE[j]);
00198             if (I[j]>=SIZE[j]) wblog(FL,"ERR %s() index out of bounds "
00199                 "(%d,%d: %d: %s)",FCT,i+1,j+1,I[j],SSTR_(this)
00200             );
00201         }
00202     }
00203
00204     IDX.makeUnique(I1); D.Select(I1);
00205     setRand_data(x,sflag);
00206
00207     Compress(); return *this;
00208 };
00209
00210 template <class TD> inline
00211 wbsparray<TD>& wbsparray<TD>::setRand_data(double x, char sflag) {
00212
00213     if (D.len) {
00214         SPIDX_T i=0, n=D.len; Wb::rand R;
00215
00216         if (sflag) {
00217             if (sflag=='d' || sflag=='D') {
00218                 for (; i<n; ++i) { R.rands(D.data[i]); }
00219             }
00220             else {
00221                 for (; i<n; ++i) { R.randb(D.data[i]); }
00222             }
00223         }
00224         else {
00225             for (; i<n; ++i) { R.rand_(D.data[i]); }
00226         }
00227
00228         if (x!=1) { R.scale_rand(D.data,D.len, x); }
00229     }
00230
00231     return *this;
00232 };
00233
00234 template <class TD> inline
00235 wbsparray<TD>& wbsparray<TD>::Resize(const wbvector<SPIDX_T> &S_) {
00236
00237     if (SIZE.len!=S_.len) wblog(FL,
00238         "ERR %s() invalid size (%d/%d)",FCT,S_.len,SIZE.len);
00239     if (!D.len) { return init(S_); }
00240     if (SIZE.len!=IDX.dim2) wblog(FL,
00241         "ERR %s() got size %s !?",FCT,SSTR_(this));
00242
00243     SPIDX_T i,j, l=0, N=IDX.dim1, m=IDX.dim2;
00244     const SPIDX_T *idx=IDX.data, *s=S_.data;

```

```

00245     wbsparray<TD> X; TD *x=D.data;
00246
00247     for (j=0; j<m; ++j) { if (s[j]<SIZE[j]) break; }
00248     if (j==m) {
00249         SIZE=S_; return *this;
00250     }
00251
00252     for (i=0; i<N; ++i, idx+=m) {
00253         for (j=0; j<m; ++j) { if (idx[j]>=s[j]) break; }
00254         if (j==m && x[i]!=0) ++l;
00255     }
00256
00257     if (l) {
00258         save2(X); init(S_,l); l=0; idx=X.IDX.data; x=X.D.data;
00259         for (i=0; i<N; ++i, idx+=m) {
00260             for (j=0; j<m; ++j) { if (idx[j]>=s[j]) break; }
00261             if (j==m && x[i]!=0) {
00262                 for (j=0; j<m; ++j) { IDX(l,j)=idx[j]; }
00263                 D[l++]=x[i];
00264             }
00265         }
00266     }
00267     else init(S_);
00268
00269     return *this;
00270 };
00271
00272 template <class TD> inline
00273 SPIDX_T wbsparray<TD>::findRecSortedP(
00274     SPIDX_T *idx, SPIDX_T n, char lex
00275 ){
00276     unsigned i=0;
00277
00278     if (long(n)<0) n=IDX.dim2;
00279     else if (n!=SIZE.len) {
00280         if (lex>0) {
00281             if (n<SIZE.len) {
00282                 for (i=n; i<SIZE.len; ++i) { if (SIZE[i]!=1) { i=-1; break; } }
00283             }
00284             else if (n>SIZE.len) {
00285                 for (i=SIZE.len; i<n; ++i) { if (idx[i]!=0) { i=-1; break; } }
00286                 n=SIZE.len;
00287             }
00288         }
00289         else {
00290             wblog(FL,"ERR %s() got n=%d/%d (%d)",FCT,n,SIZE.len,lex);
00291         }
00292     }
00293
00294     if (int(i)<0 || !n || !SIZE.len) wblog(FL,
00295         "ERR %s() invalid index [%s] (%s)",
00296         FCT, wvector<SPIDX_T>(n,idx,'r'), SSTR_(this)
00297     );
00298
00299     return IDX.findRecSorted(idx,n,lex);
00300 };
00301
00302 template<class TD> inline
00303 char wbsparray<TD>::sameUptoFac(
00304     const wbsparray<TD> &B, TD *fac_, char lex, TD eps
00305 ) const {
00306
00307     widx_t i=-1, n=D.len; TD a0,b0,fac=1;
00308     const TD *a=D.data, *b=B.D.data;
00309     wbindx Ia,Ib;
00310     char iA=0, iB=0, sab=sameSize(B);
00311
00312     if (!sab) return 1;
00313     if (!n) {
00314         if (fac_) { (*fac_)=(B.D.len ? 0. : 1.); }
00315         return (B.D.len ? 0 : 2);
00316     }
00317
00318     if (sab>=20) wblog(FL,"ERR %s() got singletons (%s <> %s)",
00319         FCT,SSTR_(this), SSTR(B));
00320
00321     if (sab>=10) {
00322         iA=((sab-10) & 1);
00323         iB=((sab-10) & 2);
00324     }
00325
00326     a0=D.aMax(&i);
00327
00328     if (swidx_t(i)<0) wblog(FL,
00329         "ERR %s() got zero data (%g;%d)",FCT,double(a0),i
00330     );
00331

```

```

00332     if (iB) {
00333         if (iA) b0=B.D[i];
00334         else {
00335             const SPIDX_T *I=IDX.ref(i);
00336             if (IDX.dim2!=2) wblog(FL,
00337                 "ERR %s() %dx%d !?",FCT,IDX.dim1,IDX.dim2);
00338             b0=(I[0]==I[1] ? B.D[I[0]] : TD(0.));
00339         }
00340     }
00341 }
00342 else {
00343     b0=B.value(IDX.rec(i),IDX.dim2);
00344 }
00345
00346 if (a0<=eps) {
00347     if (fac_) { (*fac_)=((a0==0 && b0) || Wb::abs(b0)>eps ? 0. : 1.); }
00348     return (Wb::abs(b0)>eps ? 0 : 3);
00349 }
00350 if (Wb::abs(b0)<=eps) { return 4; }
00351
00352 fac=a[i]/b0; if (fac_) (*fac_)=fac;
00353
00354 if (sab>=10) {
00355     if (iA && iB) {
00356         for (i=0; i<D.len; ++i) { if (Wb::abs(a[i]-fac*b[i])>eps) return 11; }
00357         return 0;
00358     }
00359     else if (iA) {
00360         const SPIDX_T *I=B.IDX.data; wvector<char> mark(D.len);
00361         if (B.IDX.dim2!=2) wblog(FL,
00362             "ERR %s() %dx%d !?",FCT,B.IDX.dim1,B.IDX.dim2);
00363         for (i=0; i<B.D.len; ++i, I+=2) {
00364             if (I[0]==I[1]) { mark.el(I[0])=1;
00365                 if (Wb::abs(a[I[0]]-fac*b[i])>eps) return 12; }
00366             else if (Wb::abs(b[i])>eps) return 13;
00367         }
00368         for (i=0; i<D.len; ++i) {
00369             if (!mark[i] && Wb::abs(a[i])>eps) return 14;
00370         }
00371         return 0;
00372     }
00373     else {
00374         const SPIDX_T *I=IDX.data; wvector<char> mark(B.D.len);
00375         if (IDX.dim2!=2) wblog(FL,
00376             "ERR %s() %dx%d !?",FCT,IDX.dim1,IDX.dim2);
00377         for (i=0; i<D.len; ++i, I+=2) {
00378             if (I[0]==I[1]) { mark.el(I[0])=1;
00379                 if (Wb::abs(a[i]-fac*b[I[0]])>eps) return 21; }
00380             else if (Wb::abs(a[i])>eps) return 22;
00381         }
00382         for (i=0; i<B.D.len; ++i) {
00383             if (!mark[i] && Wb::abs(b[i])>eps) return 23;
00384         }
00385         return 0;
00386     }
00387 }
00388
00389 matchSortedIdxU(FL,IDX,B.IDX,Ia,Ib,-1,lex);
00390 for (n=Ia.len, i=0; i<n; ++i) {
00391     if (Wb::abs(a[Ia[i]]-fac*b[Ib[i]])>eps) return 31;
00392 }
00393
00394 Ia.Invert( IDX.dim1,'u');
00395 for (i=0; i<Ia.len; ++i) { if (Wb::abs(a[Ia[i]])>eps) return 32; }
00396
00397 Ib.Invert(B.IDX.dim1,'u');
00398 for (i=0; i<Ib.len; ++i) { if (Wb::abs(b[Ib[i]])>eps) return 33; }
00399
00400 return 0;
00401 };
00402
00403 template<class TD> inline
00404 bool wbspararray<TD>::isZero(TD eps, char bflag) const {
00405
00406     if (!D.len) return 1;
00407     Wb::scale_eps(eps,D.data,D.len);
00408
00409     if (eps==0) {
00410         for (SPIDX_T i=0; i<D.len; ++i) if (D[i]!=0) return 0;
00411     }
00412     else {
00413         if (bflag) { return (D.norm()<eps); }
00414         else {
00415             for (SPIDX_T i=0; i<D.len; ++i) {
00416                 if (Wb::abs(D[i])>eps) return 0; }
00417         }
00418     }

```



```

00419
00420     return 1;
00421 };
00422
00423 template <class TD>
00424 bool wbsparray<TD>::isDiagMatrix(TD eps) const {
00425
00426     if (!isDiag()) { unsigned r=rank(FL);
00427
00428         if (r!=2) return 0;
00429         if (IDX.dim2==1) return 1;
00430
00431         SPIDX_T i=0; const SPIDX_T *id=IDX.data;
00432
00433         if (IDX.dim2!=2) sperror_this(FLF);
00434         Wb::scale_eps(eps,D.data,D.len);
00435
00436         if (eps==0) {
00437             for (; i<IDX.dim1; ++i, id+=2) { if (id[0]!=id[1]) return 0; }
00438         }
00439         else {
00440             for (; i<IDX.dim1; ++i, id+=2) if (id[0]!=id[1]) {
00441                 if (Wb::abs(D[i])>eps) return 0;
00442             }
00443         }
00444     }
00445
00446     return 1;
00447 };
00448
00449 template <class TD>
00450 wvector<TD>& wbsparray<TD>::getDiag(
00451     const char *F, int L, wvector<TD> &dd) const {
00452
00453     if (isDiag(F_L)) { dd.init(D); }
00454     else {
00455         SPIDX_T i=0, n=-1; const SPIDX_T *I=IDX.data;
00456
00457         if (!isSMatrix(F_L,&n) || IDX.dim2!=2) wblog(F_L,
00458             "ERR %s() requires rank-2 tensor (%s)",FCT,SSTR_(this));
00459         dd.init(n);
00460
00461         for (; i<IDX.dim1; ++i, I+=2) {
00462             if (*I==I[1]) { dd.el(*I) += D[i]; }
00463         }
00464     }
00465     return dd;
00466 };
00467
00468 template <class TD>
00469 TD wbsparray<TD>::getDiag(const char *F, int L, SPIDX_T k) const {
00470
00471     TD x=0;
00472
00473     if (isDiag(F_L)) {
00474         if (sSPIDX_T(k)<0) { k+=D.len; }
00475         if (k>=D.len) wblog(FL,
00476             "ERR %s() index out of bounds (%d/%d)",FCT,k,D.len);
00477         x=D[k];
00478     }
00479     else {
00480         SPIDX_T i=0, n=-1; const SPIDX_T *I=IDX.data;
00481
00482         if (!isSMatrix(F_L,&n) || IDX.dim2!=2) wblog(F_L,
00483             "ERR %s() requires rank-2 tensor (%s)",FCT,SSTR_(this));
00484         if (sSPIDX_T(k)<0) { k+=n; }
00485         if (k>=n) wblog(FL,
00486             "ERR %s() index out of bounds (%d/%d)",FCT,k,n);
00487
00488         for (; i<IDX.dim1; ++i, I+=2) {
00489             if (*I==I[1] && *I==k) { x+=D[i]; }
00490         }
00491     }
00492     return x;
00493 };
00494
00495 template <class TD>
00496 wbsparray<TD>& wbsparray<TD>::diag2reg(
00497     const char *F, int L, wbsparray<TD> &A, unsigned r
00498 ) const {
00499
00500     if (int(r)<0 || r==2) {
00501         if (!D.len) { checkSize(F_L); A=(*this); return A; }
00502         if (!isDiag(FL)) sperror_this(F_LF);
00503
00504         SPIDX_T i=0, n=D.len, *I; const SPIDX_T s[]={n,n};
00505         wvector<SPIDX_T> S(2,s);

```

```

00506
00507     A.init(S,n); A.D=D; I=A.IDX.data;
00508     for (; i<n; ++i, I+=2) { I[0]=I[1]=i; }
00509 }
00510 else {
00511     if (!r) wblog(FL,"ERR %s() invalid rank (%d)",FCT,r);
00512     if (!isScalar()) sperror_this(F_LF);
00513     A.SIZE.init(r).set(1);
00514     A.IDX.init(D.len,r); A.D=D; A.isref=0;
00515 }
00516
00517 return A;
00518 };
00519
00520 template <class TD>
00521 wbspararray<TD>& wbspararray<TD>::diag2reg(
00522     const char *F, int L, unsigned r
00523 ){
00524     if (isref) wblog(FL,"ERR %s() got reference (%d)",FCT,isref);
00525
00526     if (int(r)<0 || r==2) {
00527         if (!D.len) { checkSize(F_L); return *this; }
00528         if (!isDiag(FL)) sperror_this(F_LF);
00529
00530         SPIDX_T i=0, n=D.len, *I;
00531         SIZE.init(2); SIZE[0]=SIZE[1]=n;
00532         IDX.init(n,SIZE.len);
00533
00534         for (I=IDX.data; i<n; ++i, I+=2) { I[0]=I[1]=i; }
00535     }
00536     else {
00537         if (!r) wblog(FL,"ERR %s() invalid rank (%d)",FCT,r);
00538         if (!isScalar()) sperror_this(F_LF);
00539         SIZE.init(r).set(1);
00540         IDX.init(D.len,r);
00541     }
00542
00543     return *this;
00544 };
00545
00546 template <class TD>
00547 bool wbspararray<TD>::isProptoId(TD &q, TD eps) const {
00548
00549     if (!D.len) return 0;
00550     if (isDiag(FL)) {
00551         if (q==0) q=D[0];
00552         for (SPIDX_T i=1; i<D.len; ++i) { if (Wb::abs(D[i]-q)>eps) return 0; }
00553         return 1;
00554     }
00555
00556     unsigned j, r=rank(FL), r2=r/2;
00557     SPIDX_T i=0,d=1; const SPIDX_T *id=IDX.data, *s=SIZE.data;
00558
00559     if (IDX.dim2!=r) sperror_this(FLF);
00560     Wb::scale_eps(eps,D.data,D.len);
00561
00562     if (!r || r%2) return 0;
00563     for (j=0; j<r2; ++j) { d*=s[j];
00564         if (!s[j] || s[j]!=s[j+r2]) return 0;
00565     }
00566
00567     if (double(eps)<=0) {
00568         if (D.len!=d) return 0;
00569         for (; i<IDX.dim1; ++i, id+=r) {
00570             for (j=0; j<r2; ++j) { if (id[j]!=id[j+r2]) return 0; }
00571             if (i || q!=0) { if (Wb::abs(D[i]-q)>eps) return 0; }
00572             else q=D[i];
00573         }
00574     }
00575     else {
00576         SPIDX_T n=0;
00577         for (; i<IDX.dim1; ++i, id+=r) {
00578             for (j=0; j<r2; ++j) { if (id[j]!=id[j+r2]) {
00579                 if (Wb::abs(D[i]-q)>eps) return 0;
00580                 else break;
00581             } }
00582             if (j==r2) {
00583                 if ((++n)!=1 || q!=0) { if (Wb::abs(D[i]-q)>eps) return 0; }
00584                 else q=D[i];
00585             }
00586         }
00587         if (!n || n!=d) return 0;
00588     }
00589
00590     return 1;
00591 };
00592

```

```

00593 template <class TD>
00594 char wbsparray<TD>::isIdentity(TD eps) const {
00595
00596     if (!D.len) return 1;
00597     if (isDiag(FL)) {
00598         for (SPIDX_T i=1; i<D.len; ++i) { if (Wb::abs(D[i]-1)>eps) return 2; }
00599         return 0;
00600     }
00601
00602     unsigned r=rank(FL), r2=r/2;
00603     SPIDX_T j,i=0,d=1; const SPIDX_T *id=IDX.data, *s=SIZE.data;
00604
00605     if (IDX.dim2!=r) sperror_this(FLF);
00606     Wb::scale_eps(eps,D.data,D.len);
00607
00608     if (!r || r%2) return 3;
00609     for (j=0; j<r2; ++j) { d*=s[j];
00610         if (!s[j] || s[j]!=s[j+r2]) return 4;
00611     }
00612
00613     if (eps<=0) {
00614         if (D.len!=d) return 5;
00615         for (; i<IDX.dim1; ++i, id+=r) {
00616             for (j=0; j<r2; ++j) { if (id[j]!=id[j+r2]) return 6; }
00617             if (Wb::abs(D[i]-1)>eps) return 7;
00618         }
00619     }
00620     else {
00621         SPIDX_T n=0;
00622         for (; i<IDX.dim1; ++i, id+=r) {
00623             for (j=0; j<r2; ++j) { if (id[j]!=id[j+r2]) {
00624                 if (Wb::abs(D[i])>eps) return 8;
00625                 else break;
00626             } }
00627             if (j==r2) { ++n;
00628                 if (Wb::abs(D[i]-1)>eps) return 9;
00629             }
00630         }
00631         if (!n || n!=d) return 10;
00632     }
00633     return 0;
00634 };
00635
00636 template <class TD>
00637 char wbsparray<TD>::isIdentity(
00638     const wbpperm &P, TD *dval_, TD eps) const {
00639
00640     TD dval=1;
00641
00642     if (!D.len) return 1;
00643     if (isDiag(FL)) {
00644         if (dval_) { (*dval_)=dval=D.avg(); eps*=dval; }
00645         for (SPIDX_T i=1; i<D.len; ++i) { if (Wb::abs(D[i]-dval)>eps) return 2; }
00646         return 0;
00647     }
00648
00649     unsigned j1,j2, r=rank(FL);
00650     const SPIDX_T *id=IDX.data;
00651
00652     if (IDX.dim2!=r) sperror_this(FLF);
00653
00654     if (P.len) {
00655         if (P.len!=r || r<2) wblog(FL,"ERR %s() "
00656             "invalid permutation P=[%s] (r=%d)",FCT,STR(P),r);
00657         j1=P[0]; j2=P[1];
00658     }
00659     else {
00660         unsigned i=0; j1=j2=SIZE.len;
00661         for (; i<SIZE.len; ++i) { if (SIZE[i]>1) { j1=i; break; } }
00662         for (++i; i<SIZE.len; ++i) { if (SIZE[i]>1) { j2=i; break; } }
00663         for (++i; i<SIZE.len; ++i) { if (SIZE[i]>1) { break; } }
00664         if (j1==SIZE.len) {
00665             if (SIZE.allEqual(1)) {
00666                 if (D.len!=1) return 3;
00667                 dval=D[0]; if (dval_) { (*dval_)=dval; }
00668                 return 0;
00669             }
00670             else return 4;
00671         }
00672         if (j2==SIZE.len || i<SIZE.len) return 5;
00673     }
00674
00675     if (j1>=SIZE.len || j2>=SIZE.len || j1==j2) wblog(FL,
00676         "ERR %s() got (%d,%d/%d)",FCT,j1,j2,SIZE.len);
00677     if (SIZE[j1]!=SIZE[j2]) return 6;
00678
00679

```

```

00680     Wb::scale_eps(eps,D.data,D.len);
00681
00682     if (dval_) {
00683         SPIDX_T i=0, l=0; dval=0;
00684         for (; i<IDX.dim1; ++i, id+=r) {
00685             if (id[j1]==id[j2]) { dval+=D[i]; ++l; }
00686         }
00687         if (l!=SIZE[j1]) return 7;
00688         if (l) { dval/=l; eps*=dval; }; (*dval_)=dval;
00689         id=IDX.data;
00690     }
00691
00692     if (eps<=TD(0)) { if (D.len!=SIZE[j1]) return 8;
00693         for (SPIDX_T i=0; i<IDX.dim1; ++i, id+=r) {
00694             if (id[j1]!=id[j2]) return 12;
00695             if (Wb::abs(D[i]-dval)>eps) return 11;
00696         }
00697     }
00698     else {
00699         SPIDX_T i=0, l=0;
00700         for (; i<IDX.dim1; ++i, id+=r) {
00701             if (id[j1]==id[j2]) { ++l;
00702                 if (Wb::abs(D[i]-dval)>eps) return 11; }
00703             else { if (Wb::abs(D[i])>eps) return 12; }
00704         }
00705         if (!l || l!=SIZE[j1]) return 7;
00706     }
00707
00708     return 0;
00709 };
00710
00711 template <class TD>
00712 bool wbsparray<TD>::isSym_aux(
00713     const char *F, int L, const char *fct,
00714     const wbsparray<TD> &B0, TD eps, TD* xref,
00715     const char symflag,
00716     const char lflag
00717 ) const {
00718
00719     wbsparray<TD> B; B0.transpose(F_L,B);
00720     SPIDX_T i,r,s; widxt m,ma,mb;
00721     wbindx Ia,Ib;
00722     wvector<SPIDX_T> Ja(IDX.dim1), Jb(B.IDX.dim1);
00723     TD x;
00724
00725     checkSize(FL,"A: "); if (&B!=this) checkSize(FL,"B: ");
00726
00727     if (IDX.dim2!=B.IDX.dim2 || IDX.dim2%2) {
00728         if (lflag) sprintf_str(
00729             "%s %s() only applies to even-rank objects (%ld;%ld).",
00730             shortFL(F,L),fct, SIZE.len, B.SIZE.len);
00731         return 0;
00732     }
00733     if (isEmpty() && B.isEmpty()) return 1;
00734
00735     m=matchIndex(IDX,B.IDX,Ia,Ib,l,&ma,&mb);
00736     if (m) wblog(FL,"ERR %s() input sparse matrices "
00737         "not compressed (%d,%d,%d) !?",FCT,m,ma,mb);
00738
00739     if (xref) (*xref)=0;
00740
00741     if (symflag=='s') {
00742         for (i=0; i<Ia.len; ++i) { r=Ia[i]; s=Ib[i];
00743             if ((++Ja[r])>1 || (++Jb[s])>1) {
00744                 wblog(FL,"ERR %s()",FCT);
00745             }
00746             x=Wb::abs(D[r]-Wb::CONJ(B.D[s])); if (xref) *xref=MAX(*xref,x);
00747             if (x>eps) return 0;
00748         }
00749     }
00750     else if (symflag=='a') {
00751         for (i=0; i<Ia.len; ++i) { r=Ia[i]; s=Ib[i];
00752             if ((++Ja[r])>1 || (++Jb[s])>1) {
00753                 wblog(FL,"ERR %s()",FCT);
00754             }
00755             x=Wb::abs(D[r]+Wb::CONJ(B.D[s])); if (xref) *xref=MAX(*xref,x);
00756             if (x>eps) return 0;
00757         }
00758     }
00759     else wblog(FL,"ERR invalid symflag=%c<%d>",symflag,symflag);
00760
00761     for (i=0; i<Ja.len; ++i) { if (!Ja[i] && Wb::abs(D[i])>eps) return 0; }
00762
00763     if (&B0!=this)
00764         for (i=0; i<Jb.len; ++i) { if (!Jb[i] && Wb::abs(B.D[i])>eps) return 0; }
00765
00766     return 1;

```

```

00767 };
00768
00769 template <class TD>
00770 TD wbsparray<TD>::dotProd(
00771     const char *F, int L, const wbsparray<TD> &B, char tnorm
00772 ) const {
00773
00774     TD x=TD();
00775     char cflag = ((tnorm || WbUtil<TD>::hasConj())>0) ? 1 : 0;
00776     char iA=isDiag(), iB=B.isDiag();
00777
00778     if (iA || iB) {
00779         if ((!iA && SIZE.len!=2) || (!iB && B.SIZE.len!=2) || dim()!=B.dim())
00780             wblog(FL,"ERR %s() size mismatch (%s <> %s)",
00781                 FCT,SSTR_(this),SSTR(B)
00782             );
00783
00784         if (iA && iB) {
00785             const TD *a=D.data, *b=B.D.data; SPIDX_T i=0;
00786             if (cflag)
00787                 { for (; i<D.len; ++i) x+=Wb::CONJ(a[i])*b[i]; }
00788             else { for (; i<D.len; ++i) x+= a[i] *b[i]; }
00789         }
00790         else if (iA)
00791             { x=B.dotProd_full_diag(*this, cflag ? 2:0); }
00792         else { x= dotProd_full_diag(B, cflag ? 1:0); }
00793
00794         return x;
00795     }
00796
00797     if (SIZE!=B.SIZE) wblog(F_L,"ERR %s() "
00798         "got size mismatch (%s <> %s)",FCT,SSTR_(this),SSTR(B));
00799     if (!SIZE.len) { return x; }
00800
00801     SPIDX_T ia=0, ib=0, na=IDX.dim1, nb=B.IDX.dim1, l=0, lnext=0;
00802     const SPIDX_T *Ia=IDX.data, *Ib=B.IDX.data, Davg=Wb::sqrt(SIZE.prod());
00803     const TD *a=D.data, *b=B.D.data;
00804     unsigned m=SIZE.len;
00805     char q=0, lex=0;
00806
00807     while (ia<na && ib<nb) {
00808         q=Wb::recCompare(Ia,Ib,m,lex);
00809
00810         if (q<0) { ++ia; Ia+=m; } else
00811         if (q>0) { ++ib; Ib+=m; } else {
00812             if (cflag)
00813                 { x+=Wb::CONJ(a[ia])*b[ib]; }
00814             else { x+= a[ia] *b[ib]; }
00815             ++ia; ++ib; Ia+=m; Ib+=m;
00816
00817             if ((++l)>=lnext) { lnext+=Davg;
00818                 if (ia<na) { q=Wb::recCompare(Ia-m,Ia,m,lex);
00819                     if (q>0) wblog(F_L,"ERR %s() A: "
00820                         "input sparse IDX not sorted (%ld/%ld) !?",FCT,ia,na); }
00821                 if (ib<nb) { q=Wb::recCompare(Ib-m,Ib,m,lex);
00822                     if (q>0) wblog(F_L,"ERR %s() B: "
00823                         "input sparse IDX not sorted (%ld/%ld) !?",FCT,ib,nb);
00824                 }
00825             }
00826         }
00827     }
00828
00829     return x;
00830 };
00831
00832 template <class TD> inline
00833 TD wbsparray<TD>::dotProd_full_diag(const wbsparray<TD> &B, char cflag
00834 ) const {
00835
00836     const TD *a=D.data, *b=B.D.data;
00837     const SPIDX_T *I=IDX.data;
00838     TD x=TD();
00839
00840     if (IDX.dim2!=2 || B.IDX.dim1 || B.D.len!=SIZE[0]) wblog(FL,
00841         "ERR %s() %s <> %s !?",FCT, SSTR_(this), SSTR(B)
00842     );
00843
00844     if (cflag==0) {
00845         for (SPIDX_T i=0; i<D.len; ++i, I+=2) {
00846             if (I[0]==I[1]) { x += a[i] * b[I[0]]; }
00847         }
00848     }
00849     else if (cflag==1) {
00850         for (SPIDX_T i=0; i<D.len; ++i, I+=2) {
00851             if (I[0]==I[1]) { x += Wb::CONJ(a[i]) * b[I[0]]; }
00852         }
00853     }

```

```

00854     else if (cflag==2) {
00855         for (SPIDX_T i=0; i<D.len; ++i, I+=2) {
00856             if (I[0]==I[1]) { x += a[i] * Wb::CONJ(b[I[0]]); }
00857         }
00858     }
00859     else wblog(FL, "ERR %s() invalid cflag=%d", FCT, cflag);
00860
00861     return x;
00862 };
00863
00864 template <class TD>
00865 wbsparray<TD>& wbsparray<TD>::TimesEl (
00866     const wbsparray<TD> &B, char tnorm
00867 ) {
00868     char cflag = ((tnorm || WbUtil<TD>::hasConj())>0) ? 1 : 0;
00869     char iA=0, iB=0, sab=sameSize(B);
00870
00871     if (!sab || sab>=20) wblog(FL, "ERR %s() "
00872         "size mismatch (%s <> %s)", FCT, SSTR_(this), SSTR(B));
00873
00874     if (sab>=10) { wbsparray<TD> X;
00875
00876         iA=((sab-10) & 1);
00877         iB=((sab-10) & 2);
00878
00879         if (iA && iB) {
00880             SPIDX_T i=0; const TD *a=D.data; TD *x;
00881             X=B; x=X.D.data;
00882             if (cflag)
00883                 { for (; i<D.len; ++i) x[i]*=Wb::CONJ(a[i]); }
00884             else { for (; i<D.len; ++i) x[i]*= a[i]; }
00885         }
00886         else if (iA)
00887             { B.timesEl_full_diag(*this,X, cflag ? 2:0); }
00888         else { timesEl_full_diag(B, X, cflag ? 1:0); }
00889     }
00890     else {
00891         wbindex Ia,Ib; SPIDX_T i=0, m,ma=0,mb=0;
00892         const TD *a=D.data, *b=B.D.data;
00893
00894         m=matchIndex(IDX,B.IDX,Ia,Ib,1,&ma,&mb);
00895         if (m) wblog(FL, "ERR %s() input sparse "
00896             "objects not compressed (%d,%d,%d) !?", FCT, m,ma,mb);
00897
00898         wbsparray<TD> X(SIZE, Ia.len);
00899         TD *x=X.D.data;
00900
00901         for (i=0; i<Ia.len; ++i) { X.IDX.recSetP(i, IDX.rec(Ia[i])); }
00902         if (cflag)
00903             { for (i=0; i<Ia.len; ++i) { x[i]=Wb::CONJ(a[Ia[i]])*b[Ib[i]]; }}
00904         else { for (i=0; i<Ia.len; ++i) { x[i]= a[Ia[i]] *b[Ib[i]]; }}
00905
00906         X.save2(*this).Compress();
00907     }
00908
00909     return *this;
00910 };
00911
00912 template <class TD> inline
00913 wbsparray<TD>& wbsparray<TD>::timesEl_full_diag(
00914     const wbsparray<TD> &B, wbsparray<TD> &X, char cflag
00915 ) const {
00916
00917     const TD *a=D.data, *b=B.D.data;
00918     const SPIDX_T *I=IDX.data;
00919
00920     X.initDiag(B.D.len); TD *x=X.D.data;
00921
00922     if (IDX.dim2!=2 || B.IDX.dim1 || B.D.len!=IDX.dim1) wblog(FL,
00923         "ERR %s() (%d,%d)x%d <> (%d,%d)x%d !?", FCT,
00924         D.len, IDX.dim1, IDX.dim2, B.D.len, B.IDX.dim1, B.IDX.dim2
00925     );
00926
00927     if (cflag==0) {
00928         for (SPIDX_T i=0; i<IDX.dim1; ++i, I+=2) { if (I[0]==I[1]) {
00929             x[I[0]]=a[i]*b[I[0]];
00930         }}
00931     }
00932     else if (cflag==1) {
00933         for (SPIDX_T i=0; i<IDX.dim1; ++i, I+=2) { if (I[0]==I[1]) {
00934             x[I[0]]=Wb::CONJ(a[i])*b[I[0]];
00935         }}
00936     }
00937     else if (cflag==2) {
00938         for (SPIDX_T i=0; i<IDX.dim1; ++i, I+=2) { if (I[0]==I[1]) {
00939             x[I[0]]=a[i]*Wb::CONJ(b[I[0]]);
00940         }}
00941     }

```

```

00941     }
00942     else wblog(F_L, "ERR %s() invalid cflag=%d", FCT, cflag);
00943     return X;
00944 };
00945
00946 template <class TD> inline
00947 wbsparray<TD>& wbsparray<TD>::setCol(
00948     const char *F, int L, SPIDX_T k, const wbsparray<TD> &v
00949 ){
00950     unsigned rv=v.rank(F_L);
00951
00952     if (rank(F_L)!=2 || rv<1 || rv>2 || SIZE[0]!=v.SIZE[0] ||
00953         (rv==2 && v.SIZE[1]!=1)
00954         ) wblog(F_L, "ERR %s() invalid input (%s <> %s)",
00955             FCT, SSTR_(this), SSTR(v)
00956         );
00957
00958     if (k>=SIZE[1]) wblog(F_L,
00959         "ERR %s() index out of bounds (%d/%d)", FCT, k, SIZE[1]);
00960     if ((IDX.dim1 && IDX.dim2!=2) || v.IDX.dim2!=rv) wblog(F_L,
00961         "ERR %s() %dx%d <> %dx%d/%d !?", FCT, IDX.dim1, IDX.dim2,
00962         v.IDX.dim2, v.IDX.dim2, rv);
00963
00964     SPIDX_T i=0, l=0, l2, n=IDX.dim1, *I2=IDX.data+1;
00965
00966     for (; i<n; ++i, I2+=2) { if ((*I2)!=k) {
00967         if (l<i) { setRec(l,i); }; ++l;
00968     }}
00969
00970     if (l<i && F) wblog(F_L,
00971         "WRN %s() *this already contains data at col=%d/%d (%d/%d)",
00972         FCT, k, SIZE[1], l, SIZE[0]
00973     );
00974     if (!l) {
00975         if (v.IDX.dim2==1)
00976             { v.IDX.resize(v.IDX.dim1, 2, IDX); }
00977         else { IDX=v.IDX; }
00978         IDX.setLastCol(k);
00979
00980         D=v.D; return *this;
00981     }
00982
00983     n=v.D.len; l2=l+n;
00984     if (l2) {
00985         if (l2<=IDX.dim1)
00986             { REDSIZE_NNZ(l2); }
00987         else { IDX.Resize(l2, IDX.dim2); D.Resize(l2); }
00988     }
00989     else { IDX.init(0, IDX.dim2); D.init(); }
00990
00991     if (n) {
00992         const SPIDX_T *Iv=v.IDX.data;
00993         Wb::cpyRange(D.data+1, v.D.data, n); I2=IDX.rec(l);
00994         for (i=0; i<n; ++i, I2+=2, Iv+=rv) {
00995             I2[0]=(*Iv);
00996             I2[1]=k;
00997         }
00998     }
00999     return *this;
01000 };
01001
01002 template <class TD> inline
01003 TD wbsparray<TD>::Normalize(char tnorm, char qflag){
01004     TD x = Wb::sqrt(Wb::overlap(D.data, D.data, D.len, 1, tnorm));
01005
01006     if (Wb::abs(x)>TD(1E-15)) Wb::timesRange(D.data, TD(1)/x, D.len);
01007     else if (!qflag) {
01008         wblog(F_L, "ERR %s() got vector with norm %.3g !?", FCT, double(x));
01009     }
01010
01011     return x;
01012 };
01013
01014 template <class TD>
01015 TD wbsparray<TD>::NormalizeCol(SPIDX_T k, char tnorm, char qflag){
01016
01017     if (rank()!=2 || IDX.dim2!=2) wblog(F_L,
01018         "ERR %s() requires rank-2 (got %s)", FCT, SSTR_(this));
01019     if (k>=SIZE[1]) wblog(F_L,
01020         "ERR %s() index out of bounds (%d/%d)", FCT, k, SIZE[1]);
01021     checkSize(F_LF);
01022
01023     SPIDX_T i=0, n=IDX.dim1, *I2=IDX.data+1;
01024     opFlags<TD> xflags(tnorm ? 'N' : 'C');
01025     TD x=0;
01026
01027     if (xflags.conj())

```

```

01028         { for (; i<n; ++i, I2+=2) if ((*I2)==k) { x+=Wb::CONJ(D[i])*D[i]; }}
01029     else { for (; i<n; ++i, I2+=2) if ((*I2)==k) { x+=D[i]*D[i]; }}
01030
01031     x=Wb::sqrt(x);
01032     if (Wb::abs(x)>1E-15) { x=1/x;
01033         for (i=0; i<n; ++i, I2+=2) if ((*I2)==k) { D[i]*=x; }
01034     }
01035     else if (!qflag) {
01036         wblog(FL,"ERR %s() got vector with norm %.3g !?",FCT,x);
01037     }
01038
01039     return x;
01040 };
01041
01042 template <class TD>
01043 wbsparray<TD>& wbsparray<TD>::QRdecomp(const char *F, int L,
01044     wbsparray<TD> &Q,
01045     char posR,
01046     TD eps
01047 ){
01048     if (rank()!=2 || SIZE.len!=2 || IDX.dim2!=2) wblog(F_L,
01049         "ERR %s() requires rank-2 tensor (%s)",FCT,SSTR_(this));
01050
01051     SPIDX_T d1=SIZE[0], d2=SIZE[1], *idx=IDX.data;
01052     if (!d1 || !d2) { Q.init(); return *this; }
01053
01054     SPIDX_T i,l, k=0, k_, l1=0, l2=0, iu=0, N=IDX.dim1;
01055     TD x2, c2=0, e2=0, eps2=eps*eps;
01056     wbsparray<TD> u,X;
01057     int dk=0;
01058
01059     eps2*=Wb::sqrt(TD(MAX(d1,d2)));
01060
01061     if (eps2>TD(1E-8)) wblog(F_L,
01062         "ERR %s() got eps=%.3g !?",FCT,double(eps));
01063
01064     if (!N) { SPIDX_T one=1;
01065         Q.initIdentity(d1,one); init(one,d2);
01066         return *this;
01067     }
01068
01069     if (N==1) {
01070         wvector<SPIDX_T> s(2); wbsparray<TD> R;
01071         s[0]=d1; s[1]=1; Q.init(s,1); Q.IDX(0,0)=IDX(0,0);
01072         s[0]=1; s[1]=d2; R.init(s,1); R.IDX(0,1)=IDX(0,1);
01073
01074         if (posR && D[0]<0)
01075             { Q.D[0]=-1; R.D[0]=-D[0]; }
01076         else { Q.D[0]= 1; R.D[0]= D[0]; }
01077
01078         return R.save2(*this);
01079     }
01080
01081     wvector< wbsparray<TD> > U(d2);
01082
01083     wvector<char> sR;
01084     if (posR) { posR=1; sR.init2val(d2,1); }
01085
01086     wbsparray<TD> A_(*this);
01087
01088     for (; k<=d2; ++k) {
01089         N=IDX.dim1;
01090
01091         if (k) {
01092             if (l1>=N) l1=N-1;
01093             if (IDX(l1,1)>=k_) {
01094                 while (l1>0) { if (IDX(--l1,1)<k_) { ++l1; break; } }
01095             }
01096             if (IDX(l1,1)!=k_) wblog(FL,
01097                 "ERR %s() QR out of pace (%ld/%ld) !?",FCT,IDX(l1,1),k_);
01098             l2=l1;
01099         }
01100
01101         idx=IDX.rec(l2); k_=idx[1]; dk=-1;
01102
01103         while (l2<N) {
01104             while (l2<N && idx[0]<k && idx[1]==k_) { ++l2; idx+=2; }
01105
01106             if ((++dk)==0 && posR && k) {
01107                 const TD &rk=D[l2-1];
01108
01109                 if (idx[-2]!=k-1) {
01110                     MXPut(FL,"q").add(A_,"A")
01111                         .add(*this,"R").add(IDX,"IDX").add(D,"D")
01112                         .add(l1,"l1").add(l2,"l2").add(k,"k").add(k_,"k_")
01113                         .add(c2,"c2").add(e2,"e2").add(eps,"eps");
01114                     wblog(FL,

```



```

01115         "ERR %s() got R(%ld,%ld,%ld)=%.4g (l1=%ld, l2=%ld)",
01116         FCT,idx[-2],k-1,k_,double(rk),l1,l2-1
01117     );
01118 }
01119 if (Wb::norm2(rk)<=eps2) {
01120     sprintf_str("got R(%ld,%ld)=%.4g",k-1,k_,double(rk));
01121     if (rk) wblog(FL,"WRN %s() %s",FCT,str);
01122     else wblog(FL,"ERR %s() %s",FCT,str);
01123 }
01124 if (rk<0) { sR[k-1]=-1; posR=-1; }
01125 }
01126
01127 if (l2>=N && idx[-2]<k) { l1=l2; break; }
01128 if (idx[l1]!=k_) {
01129     k_=idx[l1]; continue;
01130 }
01131
01132 l1=l2; c2=Wb::norm2(D[l2]); idx+=2; ++l2;
01133 while (l2<N && idx[l1]==k_) { idx+=2; c2+=Wb::norm2(D[l2++]); }
01134
01135 if (c2<=eps2 || (l1 && dk==0)) { e2+=c2;
01136     if (c2>100*eps2) wblog(FL,"ERR %s() "
01137         "got c2=%.3g (%.3g) !?",FCT,double(c2),double(eps2));
01138     for (l=11; l<l2; ++l) D[l]=0;
01139     if (l2<N) {
01140         k_=idx[l1]; continue;
01141     }
01142     else { l1=l2; break; }
01143 }
01144 else {
01145     break;
01146 }
01147 }; if (l1>=N) break;
01148
01149 if (k_<k || k>=d2) {
01150     MXPut(FL,"q").add(A_, "A")
01151         .add(*this,"R").add(IDX,"IDX").add(D,"D")
01152         .add(l1,"l1").add(l2,"l2").add(k,"k").add(k_,"k_")
01153         .add(c2,"c2").add(e2,"e2").add(eps,"eps");
01154     wblog(FL,"ERR %s() k_=%ld/%ld/%ld !?",FCT,k_,k,d2);
01155 }
01156
01157 x2=getHouseholderVec(FL,k,k_,u,eps2);
01158
01159 if (x2<eps2*100) {
01160     if (x2>eps2) wblog(FL,"WRN %s() got x2 @ %.3g / %.3g "
01161         "[%ldx%ld; %d]",FCT,double(x2),double(eps2),d1,d2,dk);
01162     continue;
01163 }
01164
01165 LProject1(u,X,-2);
01166 Plus(FL,X);
01167 u.save2(U[iu++]);
01168 }
01169
01170 if (!k || k>d2) wblog(FL,"ERR %s() got k=%ld/%ld !?",FCT,k,d2);
01171
01172 if (k<d1) {
01173     N=IDX.dim1; idx=IDX.data; c2=0;
01174     for (l=0; l<N; ++l, idx+=2) {
01175         if (idx[0]>=k) { c2+=Wb::norm2(D[l]); }
01176     }
01177     if (c2>eps2) { MXPut(FL,"q").add(*this,"R").add(c2,"c2");
01178         wblog(FL,"ERR %s() got non-triangular R matrix (%.3g) !?",
01179             FCT,double(c2));
01180     }
01181     SPIDX_T s[2]={k,d2};
01182     Resize(wbvector<SPIDX_T>(2,s));
01183 }
01184 e2+=SkipTiny(eps);
01185
01186 Q.initIdentity(d1,MIN(d1,k));
01187 if (iu) {
01188     for (k=iu-1; k<iu; --k) {
01189         Q.LProject1(U[k],X,-2);
01190         Q.Plus(FL,X);
01191     }
01192 }
01193 e2+=Q.SkipTiny(eps);
01194
01195 if (posR<0) {
01196     TD *x=Q.D.data; N=Q.IDX.dim1; idx=Q.IDX.data;
01197     for (i=0; i<N; ++i, idx+=2) { if (sR[idx[l1]]<0) x[i]=-x[i]; }
01198     x=D.data; N=IDX.dim1; idx=IDX.data;
01199     for (i=0; i<N; ++i, idx+=2) { if (sR[idx[0]]<0) x[i]=-x[i]; }
01200 }
01201

```

```

01202     return *this;
01203 };
01204
01205 template <class TD>
01206 wbspararray<TD>& wbspararray<TD>::OrthoNormalizeColsQR(
01207     const char *F, int L,
01208     TD eps
01209 ){
01210     wbspararray<TD> Q;
01211
01212     QRdecomp(F_L,Q,'+',eps);
01213
01214     if (SIZE.len==2 || D.len) {
01215         SPIDX_T l, n=D.len, d2=SIZE[1]; char e=0; int lmax=-1;
01216         wvector<TD> dd(d2); TD eps=1E-12;
01217         wvector<int> id; id.init2val(d2,-1);
01218
01219         for (l=0; l<n; ++l) { if (Wb::norm(D[l])>eps) {
01220             dd[IDX(l,1)]=D[l];
01221             id[IDX(l,1)]=IDX(l,0);
01222         }}
01223
01224         for (l=0; l<d2 && !e; ++l) {
01225             if (id[l]>lmax) {
01226                 if (id[l]==lmax+1) { lmax=id[l];
01227                     if (dd[l]<0) { e=1;
01228                         sprintf_str("dd[%ld]=%.3g",l,double(dd[l]));
01229                     }
01230                 }
01231                 else { e=2; sprintf_str("%ld: %d -> %d",l,lmax,id[l]); }
01232             }
01233         }
01234
01235         if (e) { MXPut(FL,"q")
01236             .add(*this,"R").add(Q,"Q").add(SIZE,"S")
01237             .add(IDX,"IDX").add(D,"D").add(dd,"dd").add(id,"id");
01238             wblog(FL,"ERR %s() %s (eps=%g) !?",FCT,str,double(eps));
01239         }
01240     }
01241
01242     return Q.save2(*this);
01243 };
01244
01245 template <class TD>
01246 wbspararray<TD>& wbspararray<TD>::OrthoNormalizeColsGS(
01247     const char *F, int L,
01248     char tnorm,
01249     char qflag,
01250     TD eps,
01251     unsigned np
01252 ){
01253     if (rank()!=2) wblog(FL,
01254         "ERR %s() requires rank-2 tensor (%s)",FCT,SSTR_(this));
01255
01256     SPIDX_T i,j,ip;
01257     char xflag=(qflag=='x' || qflag=='X');
01258     wvector< wbspararray<TD> > X;
01259     wvector<SPIDX_T> IO;
01260     TD a,z;
01261
01262     splitSparseCM(F_L,X, xflag ? eps : TD(0));
01263
01264     WbUtil<TD>().adjust_tnorm(tnorm);
01265
01266     for (i=0; i<X.len; ++i) {
01267         for (ip=0; ip<np; ++ip) {
01268             for (j=0; j<i; ++j) { if (X[j].D.len) {
01269                 z=X[i].dotProd(FL,X[j],tnorm); if (z!=0) {
01270                     X[i].Plus(FL,X[j],-z); }
01271             }}
01272         }
01273
01274         z=X[i].norm2(tnorm); a=Wb::abs(z);
01275
01276         if (a>eps) { X[i]*=(TD(1)/Wb::sqrt(a)); }
01277         else {
01278             if (qflag) {
01279                 X[i].initz();
01280             }
01281             else if (!xflag) {
01282                 MXPut(FL,"x").add(*this,"V").add(X,"X")
01283                     .add(qflag,"qflag").add(xflag,"xflag")
01284                     .add(i+1,"i").add(a,"a").add(eps,"eps");
01285                 wblog(F_L,"ERR |U(:,%d/%d)| = %.3g (%g) %s !?", i+1, X.len,
01286                     double(a), double(eps), tnorm ? " (using t-norm)" : ""
01287                 );
01288             }
01289         }
01290     }

```

```

01289     }
01290 }
01291
01292 wvector< wbsparray<TD>* > xp(X.len);
01293 if (xflag) {
01294     for (j=i=0; i<X.len; ++i) { if (X[i].D.len) {
01295         xp[j++]=(X[i]);
01296     }}
01297     if (j<i) {
01298         if (!j) wblog(F_L,"ERR %s() got all null-vectors",FCT);
01299         xp.len=j;
01300     }
01301 }
01302 else {
01303     for (i=0; i<X.len; ++i) { xp[i]=(X[i]); }
01304 }
01305
01306 initCAT(FL,xp,&I0);
01307 if (!IDX.dim1) wblog(F_L,"ERR %s() got all null-vectors",FCT);
01308
01309 return *this;
01310 };
01311
01312 template <class TD>
01313 template <class T2>
01314 wvector<T2>& wbsparray<TD>::norm2vec(unsigned l,
01315     wvector<T2> &a, char tnorm
01316 ) const {
01317     if (l>=SIZE.len) wblog(FL,
01318         "ERR %s() index out of bounds (l=%d/%d)",FCT,l,SIZE.len);
01319     a.init(SIZE[l]);
01320
01321     unsigned m=IDX.dim2;
01322     SPIDX_T i=0, *ip=IDX.data+l;
01323     T2 *ad=a.data;
01324
01325     if (tnorm)
01326         for (; i<D.len; ++i, ip+=m) { ad[*ip] += D[i]*D[i]; }
01327     else { for (; i<D.len; ++i, ip+=m) { ad[*ip] += Wb::CONJ(D[i])*D[i]; } }
01328
01329     return a;
01330 };
01331
01332 template <class TD>
01333 TD wbsparray<TD>::trace() const {
01334     TD x=0;
01335
01336     unsigned r=rank(FL); if (!r) return x;
01337
01338     if (r==2 && !IDX.dim1) return D.sum();
01339     if (r%2 || r!=SIZE.len || r!=IDX.dim2) sperror_this(FLF);
01340
01341     SPIDX_T i=0; unsigned r2=r/2, n=r2*sizeof(SPIDX_T);
01342     const SPIDX_T *idx=IDX.data;
01343
01344     for (; i<r2; ++i) {
01345         if (SIZE[i]!=SIZE[i+r2]) wblog(FL,
01346             "ERR %s() got non-symmetric tensor (%s)",FCT,SSTR(this));
01347     }
01348
01349     for (i=0; i<IDX.dim1; ++i, idx+=r) {
01350         if (memcmp(idx,idx+r2,n)==0) x+=D[i];
01351     }
01352
01353     return x;
01354 };
01355
01356 template <class TD>
01357 template <class T2>
01358 wvector<T2>& wbsparray<TD>::trace(
01359     unsigned k, wvector<T2> &A
01360 ) const {
01361     SPIDX_T i=0;
01362     unsigned r=IDX.dim2, r2=(r-1)/2, n=r2*sizeof(SPIDX_T);
01363     const SPIDX_T *idx=IDX.data+(k?0:1), *sz=SIZE.data+(k?0:1);
01364     int l=(k ? r-1 : -1);
01365
01366     if (k && k+1!=SIZE.len) wblog(FL,
01367         "ERR %s() only accepts k=1 or k=rank (%d/%d)",FCT,k+1,r);
01368     if (SIZE.len%2!=1) wblog(FL,"ERR %s() "
01369         "requires odd-rank tensor (r=%d; k=%d)",FCT,SIZE.len,k);
01370     if (SIZE.len!=IDX.dim2 || IDX.dim1!=D.len) sperror_this(FLF);
01371
01372     A.init(SIZE[k]);

```

```

01376     T2 *a=A.data;
01377
01378     for (; i<r2; ++i) {
01379         if (sz[i]!=sz[i+r2]) wblog(FL,"ERR %s() "
01380             "got non-symmetric tensor (%s; k=%d)",FCT,SSTR_(this),k+1);
01381     }
01382
01383     for (i=0; i<IDX.dim1; ++i, idx+=r) {
01384         if (memcmp(idx,idx+r2,n)==0) {
01385             if (idx[l]>=A.len) wblog(FL,"ERR %s() "
01386                 "index out of bounds (%d,%d: %d/%d)",FCT,i,l,idx[l],A.len);
01387             a[idx[l]]+=D[i];
01388         }
01389     }
01390
01391     return A;
01392 };
01393
01394 bool mxIsWbssparray(
01395     const char *F, int L, const mxArray *a, unsigned k
01396 ){
01397
01398     if (!a || mxIsEmpty(a)) { return 0; }
01399
01400     SPIDX_T n=mxGetNumberOfElements(a);
01401     if (k>=n) { if (F) wblog(F,L,
01402         "ERR sparse::%s() index exceeds dimension (%d/%d)",FCT,k,n);
01403         return 0;
01404     }
01405
01406     int e[3] {
01407         mxGetFieldNumber(a,"S"),
01408         mxGetFieldNumber(a,"idx"), mxGetFieldNumber(a,"data")
01409     };
01410
01411     if (e[0]<0 || e[1]<0 || e[2]<0) { if (F) wblog(FL,
01412         "ERR %s() invalid sparse array (missing fields%s%s)",FCT,
01413         e[0]?" S:":"", e[1]?" idx:":"", e[2]?" data:"");
01414         return 0;
01415     }
01416
01417     return 1;
01418 };
01419
01420 bool mxIsWbssparray(
01421     const char *F, int L, const mxArray *a, unsigned k,
01422     const mxArray **as_, const mxArray **ad_, const mxArray **ai_
01423 ){
01424
01425     if (!a || mxIsEmpty(a)) { return 0; }
01426
01427     SPIDX_T n=mxGetNumberOfElements(a);
01428     if (k>=n) { if (F) wblog(F,L,
01429         "ERR sparse::%s() index exceeds dimension (%d/%d)",FCT,k,n);
01430         return 0;
01431     }
01432
01433     const mxArray
01434         *as=mxGetField(a,k,"S"),
01435         *ai=mxGetField(a,k,"idx"),
01436         *ad=mxGetField(a,k,"data");
01437
01438     if (!as || !ad) { if (F) {
01439         if (as && mxGetNumberOfElements(as)) wblog(F_L,
01440             "ERR %s() invalid sparse array (0x%lX, 0x%lX: S (%d)",
01441             FCT,as,ad, as ? mxGetNumberOfElements(as):-1);
01442         if (ad && mxGetNumberOfElements(ad)) wblog(F_L,
01443             "ERR %s() invalid sparse array (0x%lX, 0x%lX: data (%d)",
01444             FCT,as,ad, ad ? mxGetNumberOfElements(ad):-1);
01445     }
01446         return 0;
01447     }
01448
01449     if (as_) (*as_)=as;
01450     if (ad_) (*ad_)=ad;
01451     if (ai_) (*ai_)=ai;
01452
01453     return 1;
01454 };
01455
01456 template <class TD>
01457 wbsparray<TD>& wbsparray<TD>::toScalar(const char *F, int L, unsigned r) {
01458
01459     if (D.len>1 || (SIZE.len && !SIZE.allEqual(1))) wblog(F_L,
01460         "ERR %s() invalid scalar %s",FCT,info2Str().data);
01461
01462     if (!D.len) { D.init(1); D[0]=0; }

```

```

01463
01464     if (int(r)<=0) {
01465         if (SIZE.data) SIZE.init();
01466         if (IDX.dim1 || IDX.dim2) IDX.init();
01467     }
01468     else {
01469         if (SIZE.len) { if (SIZE.len!=r) wblog(FL,
01470             "ERR %s() got rank mismatch (%d/%d)",FCT,SIZE.len,r); }
01471         else {
01472             SIZE.init(r).set(1);
01473             IDX.init(1,r);
01474         }
01475     }
01476
01477     return *this;
01478 };
01479
01480 template <class TD>
01481 TD wbsparray<TD>::getScalar(const char *F, int L) const {
01482
01483     if (D.len>1 || (SIZE.len && !SIZE.allEqual(1))) wblog(F_L,
01484         "ERR %s() invalid scalar %s",FCT,info2Str().data);
01485
01486     return (D.len ? D[0] : TD(0));
01487 };
01488
01489 template <class TD>
01490 wbvector<TD>& wbsparray<TD>::getCol(SPIDX_T k, wbvector<TD> &v) const {
01491
01492     if (isDiag(FL)) {
01493         if (sSPIDX_T(k)<0) { k+=D.len; }
01494         if (k>=D.len) wblog(FL,
01495             "ERR %s() index out of bounds (%d/ %s)",FCT,k,SSTR_(this));
01496         v.init(D.len); v[k]=D[k];
01497     }
01498     else {
01499         if (SIZE.len!=2) wblog(FL,"ERR %s() invalid S=%d",FCT,SSTR_(this));
01500         if (sSPIDX_T(k)<0) { k+=SIZE[1]; }
01501         if (k>=SIZE[1]) wblog(FL,
01502             "ERR %s() index out of bounds (%d/ %s)",FCT,k,SSTR_(this));
01503
01504         SPIDX_T i=0, n=SIZE[0]; const SPIDX_T *I=IDX.data;
01505         v.init(n);
01506
01507         for (; i<IDX.dim1; ++i, I+=2) { if (I[1]==k) {
01508             if (*I>=n) wblog(FL,"ERR %s() "
01509                 "index out of bounds (%d/%d/ %s)",FCT,*I,n,SSTR_(this));
01510             v[*I]+=D[i];
01511         } }
01512     }
01513     return v;
01514 };
01515
01516 template <class TD>
01517 wbvector<TD>& wbsparray<TD>::getRow(SPIDX_T k, wbvector<TD> &v) const {
01518
01519     if (isDiag(FL)) {
01520         if (sSPIDX_T(k)<0) { k+=D.len; }
01521         if (k>=D.len) wblog(FL,
01522             "ERR %s() index out of bounds (%d/ %s)",FCT,k,SSTR_(this));
01523         v.init(D.len); v[k]=D[k];
01524     }
01525     else {
01526         if (SIZE.len!=2) wblog(FL,"ERR %s() invalid S=%d",FCT,SSTR_(this));
01527         if (sSPIDX_T(k)<0) { k+=SIZE[0]; }
01528         if (k>=SIZE[0]) wblog(FL,
01529             "ERR %s() index out of bounds (%d/ %s)",FCT,k,SSTR_(this));
01530
01531         SPIDX_T i=0, n=SIZE[1]; const SPIDX_T *I=IDX.data;
01532         v.init(n);
01533
01534         for (; i<IDX.dim1; ++i, I+=2) { if (*I==k) {
01535             if (I[1]>=n) wblog(FL,"ERR %s() "
01536                 "index out of bounds (%d/%d/ %s)",FCT,I[1],n,SSTR_(this));
01537             v[I[1]]+=D[i];
01538         } }
01539     }
01540     return v;
01541 };
01542
01543 template <class TD>
01544 int wbsparray<TD>::checkTrailingSingletons(
01545     const char *F, int L, unsigned *r
01546 ) const {
01547
01548     if (SIZE.len!=IDX.dim2) wblog(F_L,
01549         "ERR %s() size mismatch (%d/%d)",FCT,SIZE.len,IDX.dim2);

```

```

01550     if (!SIZE.len) {
01551         if (r && int(*r)>=0) wblog(FL,
01552             "ERR %s() out of bounds (%d/%d)",FCT,*r,SIZE.len);
01553         return 0;
01554     }
01555
01556     unsigned l=SIZE.len-1;
01557     for (; l<SIZE.len; --l) { if (SIZE[l]!=1) break; }; ++l;
01558
01559     if (r) {
01560         if (int(*r)>=0) {
01561             if ((*r)>SIZE.len) wblog(FL,
01562                 "ERR %s() out of bounds (%d/%d)",FCT,*r,SIZE.len);
01563             if (l>(*r)) wblog(FL,"ERR %s() "
01564                 "got non-singletons for dim>=%d (%d,%d)",FCT,*r,l,SIZE.len);
01565             l=(*r);
01566         }
01567         else (*r)=l;
01568     }
01569
01570     return (SIZE.len-l);
01571 };
01572
01573 template <class TD>
01574 int wbsparray<TD>::skipTrailingSingletons(
01575     const char *F, int L, wbsparray<TD> &C, unsigned r
01576 ) const {
01577
01578     C.init();
01579
01580     if (SIZE.len && checkTrailingSingletons(F_L,&r)) {
01581         unsigned m=SIZE.len-r;
01582
01583         #ifndef WB_SKIP_ASSERT
01584         SPIDX_T i=0, n=IDX.dim1, zero=0;
01585         for (; i<n; ++i) if (Wb::anyUnequal(IDX.ref(i,r),m,zero)) {
01586             wblog(F_L,"ERR %s() IDX out of bounds (%d/%dx%d: %d/%d) !?",
01587                 FCT,i+1,IDX.dim1,IDX.dim2,r,SIZE.len);
01588         }
01589         #endif
01590
01591         C.SIZE.init(r,SIZE.data);
01592         IDX.resize(IDX.dim1,r, C.IDX);
01593         C.D=D;
01594
01595         return m;
01596     }
01597
01598     if (int(r)>=0) wblog(FL,
01599         "ERR %s() out of bounds (%d/%d)",FCT,r,SIZE.len);
01600
01601     return 0;
01602 };
01603
01604 template <class TD>
01605 int wbsparray<TD>::SkipTrailingSingletons(
01606     const char *F, int L, unsigned r
01607 ){
01608
01609     if (SIZE.len && checkTrailingSingletons(F_L,&r)) {
01610         unsigned m=SIZE.len-r;
01611
01612         #ifndef WB_SKIP_ASSERT
01613         SPIDX_T i=0, n=IDX.dim1, zero=0;
01614         for (; i<n; ++i) if (Wb::anyUnequal(IDX.ref(i,r),m,zero)) {
01615             wblog(F_L,"ERR %s() IDX out of bounds (%d/%dx%d: %d/%d) !?",
01616                 FCT,i+1,IDX.dim1,IDX.dim2,r,SIZE.len);
01617         }
01618         #endif
01619
01620         if (r) SIZE.len=r; else SIZE.init();
01621         IDX.Resize(IDX.dim1,r);
01622
01623         return m;
01624     }
01625
01626     if (int(r)>=0) wblog(FL,
01627         "ERR %s() out of bounds (%d/%d)",FCT,r,SIZE.len);
01628
01629     return 0;
01630 };
01631
01632 template <class TD> inline
01633 bool wbsparray<TD>::hasSingletons(const wbindex &I) const {
01634
01635     if (!I.len) wblog(FL,"ERR %s() got empty index set",FCT);
01636     for (unsigned i=0; i<I.len; ++i) {

```

```

01637         if (I[i]>=SIZE.len || SIZE[I[i]]!=1) return 0;
01638     }
01639
01640     return 1;
01641 };
01642
01643 template <class TD>
01644 wbsparray<TD>& wbsparray<TD>::skipSingletons(
01645     const char *F, int L, const wbindex &I, wbsparray<TD> &A
01646 ) const {
01647
01648     if (!I.len) { A=*this; return A; }
01649     if (&A==this) {
01650         wbsparray<TD> X; A.save2(X);
01651         return X.skipSingletons(F,L,I,A);
01652     }
01653
01654     if (!hasSingletons(I)) wblog(F_L,"ERR %s() got S=(%s) "
01655         "given I=[%s] !?", FCT,SSTR_(this), STR(I));
01656
01657     wbindex I2; I.invert(IDX.dim2,I2,'u');
01658
01659     SIZE.select(I2,A.SIZE); A.isref=0;
01660     IDX.getCols(I2,A.IDX); A.D=D;
01661
01662     return A;
01663 };
01664
01665 template <class TD>
01666 wbsparray<TD>& wbsparray<TD>::AddTrailingSingletons(
01667     const char *F, int L, unsigned r
01668 ){
01669     unsigned rk=rank(F_L); if (r==rk) { return *this; }
01670
01671     if (int(r)<0) wblog(F_L,"ERR %s() invalid rank (%d/%d)",FCT,r,rk);
01672     if (r<rk) wblog(F_L,"ERR %s() cannot reduce rank (%d/%d)",FCT,r,rk);
01673
01674     if (isDiag()) {
01675         if (IDX.data) wblog(F_L,
01676             "ERR %s() got IDX data (%s) !?",FCT,IDX.dim1,IDX.dim2);
01677         SIZE.init(r); SIZE[0]=SIZE[1]=D.len;
01678         for (unsigned i=2; i<r; ++i) SIZE[i]=1;
01679         IDX.init(D.len,r);
01680         for (SPIDX_T *I=IDX.data, i=0; i<D.len; ++i, I+=r) {
01681             I[0]=I[1]=i;
01682         }
01683     }
01684     else {
01685         if (SIZE.len) {
01686             unsigned j=SIZE.len; SIZE.Resize(r);
01687             for (; j<r; ++j) SIZE[j]=1;
01688         }
01689
01690         if (IDX.dim1) {
01691             if (!SIZE.len) wblog(F_L,
01692                 "ERR %s() got empty size (%d/%d)",FCT,SIZE.len,r);
01693             IDX.Resize(IDX.dim1,r);
01694         }
01695     }
01696
01697     return *this;
01698 };
01699
01700 template <class TD>
01701 char wbsparray<TD>::checkSize2(
01702     const char *F, int L, const wbsparray<TD> &B
01703 ) const {
01704
01705     char sab=sameSize(B);
01706     this->checkSize(F_L,"A:"); B.checkSize(F_L,"B:");
01707
01708     if (sab==1) {
01709         if (IDX.dim2!=B.IDX.dim2){ wblog(F_L,
01710             "ERR sparse::%s() IDX size mismatch (%dx%d <> %dx%d)",
01711             FCT,IDX.dim1,IDX.dim2,B.IDX.dim1,B.IDX.dim2);
01712             return -1;
01713         }
01714     }
01715     else if (sab<=0) {
01716         wblog(F_L,"ERR %s() mismatch %s <> %s (%d)",
01717             FCT,SSTR_(this),SSTR(B),sab);
01718     }
01719
01720     return sab;
01721 };
01722
01723 template <class TD> inline

```

```

01724 char wbsparray<TD>::checkSize(
01725     const char *F, int L, const char *fct, const char *istr
01726 ) const {
01727
01728     if (SIZE.len) {
01729         if (IDX.data) {
01730             if (IDX.dim1!=D.len || IDX.dim2!=SIZE.len) {
01731                 if (F) sperror_this(F_L_F,istr); else return 2;
01732             }
01733         }
01734         else if (D.len) {
01735             if (SIZE.len==2 && SIZE[0]==D.len && SIZE[1]==D.len) {
01736                 wblog(FL,"WRN %s() got SIZE for diag-format (%s; d=%d)",
01737                     FCT,SSTR_(this),D.len);
01738             }
01739             else { if (F) sperror_this(F_L_F,istr); else return 3; }
01740         }
01741     }
01742     else {
01743         if (!IDX.isEmpty()) {
01744             if (F) sperror_this(F_L_F,istr); else return 1;
01745         }
01746     }
01747     return 0;
01748 };
01749
01750 template <class TD> inline
01751 void wbsparray<TD>::check_IDX_range(const char *F, int L) const {
01752
01753     if (!SIZE.len) {
01754         if (IDX.dim1 || IDX.dim2) sperror_this(F_L_F);
01755         return;
01756     }
01757
01758     if (IDX.dim1!=D.len ||
01759         IDX.dim2!=SIZE.len && (IDX.dim1 || IDX.dim2))
01760     ) sperror_this(F_L_F);
01761
01762     if (IDX.dim1) {
01763         SPIDX_T i=0; unsigned j=0, m=IDX.dim2;
01764         const SPIDX_T *s=SIZE.data, *I=IDX.data;
01765
01766         for (; i<IDX.dim1; ++i, I+=m) {
01767             for (j=0; j<m; ++j) {
01768                 if (I[j]>=s[j]) wblog(FL,
01769                     "ERR %s() index out of bounds (%d: %d <> %s)",
01770                     FCT,j+1,I[j]+1, SSTR_(this)
01771                 );
01772             }
01773         }
01774     }
01775 };
01776
01777 template <class TD>
01778 wbsparray<TD>& wbsparray<TD>::make2D(const wbindex &ic, wbsparray<TD> &B,
01779     char pos
01780 ) const {
01781
01782     if (B.D.len) { B.init(); }; B.D=D;
01783     IDX.toIndex2D(SIZE,ic,B.IDX,pos);
01784
01785     SPIDX_T s2, s1=SIZE.prod(ic.data,ic.len,s2);
01786
01787     if (pos==1)
01788     { SPIDX_T s[2]={s1,s2}; B.SIZE.init(2,s); }
01789     else { SPIDX_T s[2]={s2,s1}; B.SIZE.init(2,s); }
01790
01791     B.Compress(); return B;
01792 };
01793
01794 template <class TD>
01795 sparseIndex2D<SPIDX_T>& wbsparray<TD>::toIndex2D(
01796     const wbindex &ic, sparseIndex2D<SPIDX_T> &a,
01797     char pos,
01798     char use_rmaj
01799 ) const {
01800
01801     unsigned i,j;
01802     if (pos==1) { i=0; j=1; } else { i=1; j=0; }
01803
01804     a.S.init(2);
01805     a.S[i]=SIZE.prod(ic.data,ic.len,a.S[j]);
01806
01807 #ifndef WB_SKIP_ASSERT
01808     { unsigned k; double s1=1, s2=1; wvector<char> mark(SIZE.len);
01809         for (k=0; k<ic.len; ++k) { ++mark[ic[k]]; }
01810         for (k=0; k<SIZE.len; ++k) { (mark[k] ? s1 : s2)*=SIZE[k]; }

```



```

01811     if (double(a.S[i])!=s1 || double(a.S[j]!=s2)) wblog(FL,
01812         "ERR %s() type range out of bounds !?\n(%gx%g <> %ldx%ld)",
01813         FCT,s1,s2,a.S[0],a.S[1]
01814     );
01815 }
01816 #endif
01817
01818     IDX.toIndex2D(SIZE,ic,a.IJ,pos);
01819
01820     return a.Setup(use_rmaj);
01821 };
01822
01823 template <class TD>
01824 mxArray* wbsparray<TD>::toMxSp() const {
01825
01826     if (!Wb::isBaseType(typeid(TD))) {
01827         double x;
01828         for (SPIDX_T i=0; i<D.len; ++i) { x=double(D[i]);
01829             if (fabs(double(TD(x)-D[i])/x)>1E-20) {
01830                 MXPut(FL,"ans").add(*this,"data").add(i+1,"i").add(x,"x");
01831                 wblog(FL,"ERR %s() got data type `%s' (d[%d]=%g @ %.3g)",
01832                     FCT,TSTR(TD), i+1,x,double(TD(x)-D[i])
01833                 );
01834             }
01835         }
01836     }
01837
01838     if (isScalar()) {
01839         if (D.len) {
01840             if (D.len>1) sperror_this(FLF);
01841             return numtoMx(D[0]);
01842         }
01843         return numtoMx(0);
01844     }
01845
01846     if (isDiag(FL)) {
01847         wbMatrix<SPIDX_T> IJ; SPIDX_T d1=1,d2=1;
01848         get2DIndex(IJ,d1,d2);
01849
01850         return Wb::mxCreateSparse(FL,d1,d2,IJ, D);
01851     }
01852
01853     if (!SIZE.len) {
01854         if (IDX.isEmpty() && D.isEmpty())
01855             return Wb::mxCreateSparse(FL,0,0,IDX,D);
01856         else wblog(FL,
01857             "ERR %s() invalid wbsparray (%s;%d)",FCT,SSTR_(this),D.len);
01858     }
01859
01860     if (SIZE.len==2) {
01861         return Wb::mxCreateSparse(FL,SIZE[0],SIZE[1],IDX,D);
01862     }
01863
01864     if (SIZE.len==1) {
01865         wbMatrix<SPIDX_T> IX(IDX.dim1,2);
01866         SPIDX_T i=0, *I=IX.data;
01867
01868         if (IDX.dim2!=1) sperror_this(FLF);
01869         for (; i<IDX.dim1; ++i, I+=2) {
01870             I[0]=IDX.data[i]; I[1]=0;
01871         }
01872
01873         return Wb::mxCreateSparse(FL,SIZE[0],1,IX,D);
01874     }
01875
01876     wblog(FL,"ERR %s() requires rank-2 wbsparray (%s)",
01877         FCT,SSTR_(this)); return 0;
01878 };
01879
01880 template <class TD>
01881 mxArray* wbsparray<TD>::mxCreateStructX(
01882     unsigned m, unsigned n, int ma
01883 ) const {
01884     const char *fields[]={ "S","idx","data","info","A" };
01885     return mxCreateStructMatrix(m,n,ma!=-99 ? 5:4,fields);
01886 };
01887
01888 template <class TD>
01889 void wbsparray<TD>::add2MxStructX(mxArray *a, unsigned i, int ma) const {
01890
01891     char isId=isIdentityMatrix(), isc=isScalar();
01892
01893     mxSetFieldByNumber(a,i,0, SIZE.toMx());
01894
01895     if (isc) {
01896         mxSetFieldByNumber(a,i,1, IDX.toMx());
01897         mxSetFieldByNumber(a,i,2, D.toMx('t'));

```

```

01898     mxSetFieldByNumber(a,i,3, wbsttring("scalar").toMx());
01899 }
01900 else if (!IDX.isEmpty() && IDX.allEqual(0) && D.allEqual(0)) {
01901     mxSetFieldByNumber(a,i,1,mxCreateSparse(IDX.dim1,IDX.dim2,0,mxREAL));
01902     mxSetFieldByNumber(a,i,2,mxCreateSparse(1,D.len,0,mxREAL));
01903     mxSetFieldByNumber(a,i,3, wbsttring("init").toMx());
01904 }
01905 else if (isId) {
01906     mxSetFieldByNumber(a,i,2, numtoMx(D[0]));
01907     mxSetFieldByNumber(a,i,3, wbsttring("identity").toMx());
01908 }
01909 else if (isDiag(FL)) {
01910     mxSetFieldByNumber(a,i,2, D.toMx('t'));
01911     mxSetFieldByNumber(a,i,3, wbsttring("diag").toMx());
01912 }
01913 else if (IDX.dim2!=1 && isDiagMatrix()) {
01914     mxSetFieldByNumber(a,i,1, IDX.getCols(0,0).toMx());
01915     mxSetFieldByNumber(a,i,2, D.toMx('t'));
01916 }
01917 else {
01918     mxSetFieldByNumber(a,i,1, IDX.toMx());
01919     mxSetFieldByNumber(a,i,2, D.toMx('t'));
01920 };
01921
01922 if (ma!=-99) {
01923     wbMatrix<SPIDX_T> IJ; SPIDX_T d1=1,d2=1;
01924
01925     if (!SIZE.len) { d1=d2=0; }
01926     else ma=get2DIndex(IJ,d1,d2,ma);
01927
01928     mxSetFieldByNumber(a,i,4,
01929         Wb::mxCreateSparse(FL,d1,d2,IJ,D)
01930     );
01931 }
01932 };
01933
01934 template <class TD>
01935 mxArray* wbsparray<TD>::mxCreateSTRUCT(
01936     unsigned m, unsigned n
01937 ) const {
01938     const char *fields[]={"S","idx","data","A"};
01939     return mxCreateStructMatrix(m,n,4,fields);
01940 };
01941
01942 template <class TD>
01943 mxArray* wbsparray<TD>::add2MxSTRUCT(mxArray *a, unsigned i) const {
01944
01945     wbMatrix<SPIDX_T> IJ; SPIDX_T d1=1,d2=1; char tflag='t';
01946     get2DIndex(IJ,d1,d2);
01947
01948     mxSetFieldByNumber(a,i,0, SIZE.toMx());
01949     mxSetFieldByNumber(a,i,1, IDX.toMx());
01950
01951 #ifdef QS_USING_MPFR
01952     if (typeid(TD)==typeid(Wb::quad)) {
01953         tflag=30;
01954     }
01955 #endif
01956
01957     mxSetFieldByNumber(a,i,2, D.toMx(tflag));
01958
01959     mxSetFieldByNumber(a,i,3,Wb::mxCreateSparse(FL,d1,d2,IJ,D));
01960
01961     return a;
01962 };
01963
01964 template <class TD> inline
01965 mxArray* wbsparray<TD>::mxCreateStruct(unsigned m, unsigned n) const {
01966     const char *fields[]={"S","data"};
01967     return mxCreateStructMatrix(m,n,2,fields);
01968 };
01969
01970 #ifdef QS_USING_MPFR
01971
01972 template <> inline mxArray*
01973 wbsparray<Wb::quad>::mxCreateStruct(unsigned m, unsigned n) const {
01974     const char *fields[]={"S","idx","data"};
01975     return mxCreateStructMatrix(m,n,3,fields);
01976 };
01977
01978 #endif
01979
01980 template <class TD>
01981 void wbsparray<TD>::add2MxStruct(mxArray *a, unsigned i, int ma) const {
01982
01983     wbMatrix<SPIDX_T> IJ;
01984     SPIDX_T d1,d2; get2DIndex(IJ,d1,d2,ma);

```

```

01985     int e=0;
01986
01987     mxSetFieldByNumber(a,i,0, SIZE.toMx());
01988     mxSetFieldByNumber(a,i,1, Wb::mxCreateSparse (FL,d1,d2,IJ,D,NULL,&e));
01989
01990     if (e && !isDiag()) {
01991         static unsigned ic=0; char s[8];
01992         snprintf(s,8,"i%02d",ic++);
01993         MXPput (FL,s).add(*this,"A").addP(toMX(),"A0").add(e,"e");
01994         wblog(FL,"ERR %s() got e=%d",FCT,e);
01995     }
01996 };
01997
01998 #ifdef QS_USING_MPFR
01999
02000 template <>
02001 void wbsparray<Wb::quad>::add2MxStruct (
02002     mxArray *a, unsigned i, int ma QS_UNUSED_VAR
02003 ) const {
02004
02005     mxSetFieldByNumber(a,i,0, SIZE.toMx ());
02006     mxSetFieldByNumber(a,i,1, IDX .toMxT());
02007     mxSetFieldByNumber(a,i,2, D .toMx ());
02008 };
02009
02010 #endif
02011
02012 template <class TD> inline
02013 mxArray* wbsparray<TD>::mxCreateCell(unsigned m, unsigned n) const {
02014     return mxCreateCellMatrix(m,n);
02015 };
02016
02017 template <class TD>
02018 void wbsparray<TD>::add2MxCell(mxArray *a, unsigned i) const {
02019
02020     if (rank()>2) wblog(FL,
02021         "ERR %s() cell array expects rank <=2 (%s)",FCT,SSTR_(this));
02022     mxSetCell(a,i,toMxSp());
02023
02024 };
02025
02026 template <class TD>
02027 mxArray* wbsparray<TD>::IDtoMx() const {
02028
02029     const char *fields[]={"S","nnz","norm","stat"};
02030     mxArray *S=mxCreateStructMatrix(1,1,4,fields);
02031
02032     const char *field2[]={"istat","idx","d"};
02033     mxArray *as=mxCreateStructMatrix(1,1,3,field2);
02034
02035     if (isDiag()) {
02036         SPIDX_T s[2]={D.len,D.len};
02037         mxSetFieldByNumber(S,0,0, wvector<SPIDX_T>(2,s).toMx());
02038     } else
02039     mxSetFieldByNumber(S,0,0, SIZE.toMx());
02040
02041     mxSetFieldByNumber(S,0,1, numtoMx(nnz()));
02042     mxSetFieldByNumber(S,0,2, numtoMx(double(norm())));
02043
02044     if (D.len>3) {
02045         unsigned i=0; const unsigned m=3;
02046         const TD *d=D.data; TD a;
02047
02048         wbindx I(m); widxt *k=I.data;
02049         wvector<double> x_(m); double *x=x_.data;
02050         x[0]=d[0];
02051         x[1]=Wb::abs(d[0]);
02052         x[2]=d[0];
02053
02054         for (++i; i<D.len; ++i) { a=Wb::abs(d[i]);
02055             if (x[0]>d[i]) { x[0]=d[i]; k[0]=i; }
02056             if (x[1]>a) { x[1]=a; k[1]=i; }
02057             if (x[2]<d[i]) { x[2]=d[i]; k[2]=i; }
02058         }
02059
02060         mxSetFieldByNumber(as,0,0, I .toMx());
02061         mxSetFieldByNumber(as,0,1, IDX.getRecs(I).toMx());
02062         mxSetFieldByNumber(as,0,2, x_ .toMx());
02063     }
02064     else if (D.len) {
02065         mxSetFieldByNumber(as,0,1, IDX.toMx());
02066         mxSetFieldByNumber(as,0,2, D .toMx());
02067     }
02068
02069     mxSetFieldByNumber(S,0,3,as);
02070
02071     return S;

```

```

02072 };
02073
02074 template <class TD>
02075 wbsparray<TD>& wbsparray<TD>::initCAT(
02076     const char *F, int L, wbvector< wbsparray<TD>* > &X,
02077     wbvector<SPIDX_T> *I0, char del
02078 ){
02079     SPIDX_T i,id,l, r=0, i0=-1, i2=0, n=0, nz=0;
02080     wbvector<SPIDX_T> S;
02081
02082     for (i=0; i<X.len; ++i) {
02083         if (!X[i] || X[i]->isEmpty()) continue;
02084         if (X[i]==this) {
02085             wbsparray<TD> A; A.initCAT(F,L,X,I0,del);
02086             return A.save2(*this);
02087         }
02088         X[i]->checkSize(F_LF);
02089         nz+=X[i]->nnz();
02090
02091         if ((++n)==1) { i0=i; r=X[i]->SIZE.len+1;
02092             S.init(r,X[i]->SIZE);
02093         }; i2=i;
02094
02095         if (i!=i0 && !X[i0]->sameSize(*X[i])) wblog(F_L,
02096             "ERR %s() severe size mismatch (%d: %s <> %d: %s)",FCT,
02097             i0+1, SSTR_(X[i0]), i+1, SSTR_(X[i])
02098         );
02099     }
02100     if (long(i0)<0) { init(); return *this; }
02101
02102     char gotI0=(I0 && I0->len);
02103     if (gotI0 && I0->len!=X.len+1) wblog(F_L,
02104         "ERR %s() size mismatch (%d/%d+1)",FCT,I0->len,X.len);
02105
02106     S.last()=(gotI0 ? I0->last() : n);
02107     init(S,nz);
02108
02109     for (id=-1, l=0, i=i0; i<=i2; ++i) {
02110         if (X[i] && !X[i]->isEmpty()) {
02111             if (gotI0) { id=I0->data[i];
02112                 if (id>=S.last()) { wblog(F_L,
02113                     "ERR %s() index out of bounds (%d/%d)",FCT,id,S.last());
02114                 }
02115             }
02116             else ++id;
02117
02118             if (X[i]->nnz()) l+=catRecs(l,*X[i], id);
02119         }
02120     }
02121
02122     if (l!=D.len) wblog(F_L,"ERR %s() %d/%d !?",FCT,l,D.len);
02123
02124     if (del) for (i=0; i<X.len; ++i) {
02125         if (X[i]) { WB_DELETE_1(X[i]); }
02126         else break;
02127     }
02128
02129     Compress(); return *this;
02130 };
02131
02132 template <class TD>
02133 SPIDX_T wbsparray<TD>::catRecs(
02134     SPIDX_T i0, const wbsparray<TD> &a, SPIDX_T id
02135 ){
02136     SPIDX_T n=a.IDX.dim1;
02137
02138     if (IDX.dim2!=a.IDX.dim2+1) wblog(FL,
02139         "ERR %s() size mismatch (%d+1/%d)",FCT,IDX.dim2,a.IDX.dim2);
02140     if (i0+n>IDX.dim1) wblog(FL,
02141         "ERR %s() index out of bounds (%d+%d = %d ?)",FCT,i0,n,IDX.dim1);
02142
02143     if (!n && a.D.len==1) {
02144         if (IDX.dim2!=1 || IDX.dim1!=D.len) wblog(FL,"ERR %s() "
02145             "%s <> %dx%d (%d)",FCT,SSTR_(this),IDX.dim1,IDX.dim2,i0);
02146         if (a.D[0])
02147             { IDX(i0,0)=id; D[i0]=a.D[0]; return 1; }
02148         else { return 0; }
02149     }
02150
02151     if (a.D.len!=n) wblog(FL,"ERR %s() %s <> %dx%d (%d)",
02152         FCT, SSTR_(a), a.IDX.dim1, a.IDX.dim2, i0);
02153
02154     Wb::cpyRange(D.data+i0, a.D.data, n);
02155     Wb::cpyStride(
02156         IDX.rec(i0), a.IDX.data, a.IDX.dim2, a.IDX.dim1,
02157         IDX.dim2
02158     );

```

```

02159
02160     for (SPIDX_T l=IDX.dim2-1, i=0; i<n; ++i) {
02161         IDX(i0+i,l)=id;
02162     }
02163
02164     return n;
02165 };
02166
02167 template <class TD>
02168 wbsparray<TD>& wbsparray<TD>::Cat(
02169     const char *F, int L,
02170     const wbsparray<TD> &B, unsigned k
02171 ){
02172     if (this==&B) wblog(FL,"ERR %s() got same object !?",FCT);
02173     if ( ! isEmpty()) { (*this)=B; return *this; }
02174     if (B.isEmpty()) { return *this; }
02175
02176     if (!SIZE.len || !B.SIZE.len) wblog(F_L,
02177         "ERR %s() got empty size (%s <> %s @ %d) !?",
02178         FCT, SSTR_(this), SSTR(B), k);
02179     checkSize(FLF); B.checkSize(FLF);
02180
02181     if (!k || (k>SIZE.len && k>99)) wblog(F_L,
02182         "ERR %s() k out of bounds (%d/%d)",FCT,k,SIZE.len);
02183     if (k>SIZE.len) AddTrailingSingletons(FL,k);
02184     --k;
02185
02186     SPIDX_T i=0, n=MIN(SIZE.len, B.SIZE.len); int e=0;
02187     SPIDX_T D1=SIZE[k], d1=D.len, d2=B.D.len, d12=d1+d2;
02188
02189     for (; i<n; ++i) { if (i!=k) {
02190         if (SIZE[i]!=B.SIZE[i]) { e=1; break; }
02191     } }
02192     if (e==0) { if (i==k) ++i;
02193         for (; i< SIZE.len; ++i) { if ( SIZE[i]!=1) { e=2; break; } }
02194         for (; i<B.SIZE.len; ++i) { if (B.SIZE[i]!=1) { e=3; break; } }
02195     }
02196     if (e) wblog(F_L,
02197         "ERR %s() size mismatch (%s <> %s @ %d; e=%d)",
02198         FCT, SSTR_(this), SSTR(B), k, e
02199     );
02200
02201     SIZE[k]+=(B.SIZE.len>k ? B.SIZE[k] : 1);
02202     D.Resize(d12, B.D.data);
02203
02204     if (!IDX.dim1) {
02205         if (B.IDX.dim2==SIZE.len)
02206             { IDX=B.IDX; }
02207         else { B.IDX.resize(B.IDX.dim1,SIZE.len, IDX); }
02208     }
02209     else {
02210         if (B.IDX.dim2==IDX.dim2) {
02211             IDX.Resize(d12,IDX.dim2, B.IDX.data);
02212         }
02213         else {
02214             SPIDX_T j, *a, n=MIN(IDX.dim2,B.IDX.dim2);
02215             const SPIDX_T *b=B.IDX.data;
02216
02217             IDX.Resize(d12,IDX.dim2); a=IDX.ref(d1,0);
02218             for (i=0; i<d2; ++i, a+=IDX.dim2, b+=B.IDX.dim2) {
02219                 for (j=0; j<n; ++j) { a[j]=b[j]; }
02220             }
02221             for (SPIDX_T i=d1; i<d12; ++i) IDX(i,k)+=D1;
02222         }
02223
02224     Compress(); return *this;
02225 };
02226
02227 template <class TD>
02228 wbsparray<TD>& wbsparray<TD>::initCAT(
02229     const char *F, int L, const wbvector< const wbsparray<TD>* > &X,
02230     unsigned k
02231 ){
02232     if (!X.len) { init(); return *this; }
02233
02234     SPIDX_T i,j, i0=-1, i2=0, n=0;
02235     wbvector<SPIDX_T> S0, DD(X.len), dd(X.len);
02236
02237     for (i=0; i<X.len; ++i) { if (X[i] && !X[i]->isEmpty()) {
02238         const wbvector<SPIDX_T> &S2=X[i]->SIZE;
02239         if (X[i]==this) {
02240             wbsparray<TD> A; A.initCAT(F,L,X,k);
02241             return A.save2(*this);
02242         }
02243
02244         if (S2.len!=X[i]->IDX.dim2) X[i]->sperror_this(F_L);
02245         X[i]->checkSize(F_LF);

```

```

02246
02247     if ((++n)==1) { i0=i; S0=S2;
02248         if (!k || k>S2.len) wblog(FL,"ERR %s() k out of bounds "
02249             "(%s; %d)",FCT,SSTR(S2),k);
02250         --k;
02251     }
02252     else {
02253         if (S0.len!=S2.len) wblog(FL,"ERR %s() rank mismatch "
02254             "(%d: %s <> %d/%d: %s; %d)",FCT,i0+1,SSTR(S0),
02255             i+1,X.len, SSTR(S2), k+1);
02256         for (j=0; j<S0.len; ++j) {
02257             if (j!=k && S0[j]!=S2[j]) wblog(FL,"ERR %s() size mismatch "
02258                 "(%d: %s <> %d/%d: %s; %d/%d)",FCT,i0+1,SSTR(S0),
02259                 i+1,X.len, SSTR(S2), j+1,k+1);
02260         }
02261     }
02262
02263     DD[i]=S2[k];
02264     dd[i]=X[i]->D.len; i2=i;
02265 }
02266
02267 SPIDX_T dall=dd.sum(), Dall=DD.sum();
02268
02269 if (!n) { init(); return *this; }
02270 S0[k]=Dall; init(S0,dall); if (!dall) return *this;
02271
02272 SPIDX_T *pi=IDX.data,N,s, d1=0, d2, D1=0, m=S0.len;
02273 TD *pd=D.data;
02274
02275 for (i=i0; i<=i2; ++i, d1+=n, D1+=N) { n=dd[i]; N=DD[i];
02276     s=n*m; MEM_COPY<SPIDX_T>(pi,s,X[i]->IDX.data); pi+=s;
02277     s=n ; MEM_COPY<TD>(pd,s,X[i]->D.data); pd+=s;
02278     for (d2=d1+n, j=d1; j<d2; ++j) IDX(j,k)+=D1;
02279 }
02280 if (d1!=IDX.dim1) wblog(FL,"ERR %s() %d/%d",FCT,d1,IDX.dim1);
02281
02282 Compress(); return *this;
02283 };
02284
02285 template <class TD>
02286 wbspararray<TD>& wbspararray<TD>::init(
02287     const char *F, int L, const mxArray *a, unsigned k
02288 ){
02289     init(); if (!a) {
02290         if (k) wblog(FL,"ERR %s() got k=%d for null mxArray !?",FCT,k);
02291         return *this;
02292     }
02293
02294     if (mxIsEmpty(a) && mxIsDouble(a)) {
02295         SPIDX_T i,r=mxGetNumberOfDimensions(a);
02296         if (r) {
02297             const size_t *sp=mxGetDimensions(a);
02298             wvector<SPIDX_T> S(r);
02299             for (i=0; i<r; ++i) S[i]=(SPIDX_T)sp[i];
02300             init(S,0);
02301         }
02302         return *this;
02303     }
02304
02305     if (mxIsStruct(a)) {
02306         const mxArray *as,*ad,*ai;
02307         if (!mxIsWbspararray(F_L,a,k,&as,&ad,&ai))wblog(FL,
02308             "ERR %s() invalid sparse array",FCT);
02309
02310         SIZE.init(F_L,as);
02311         IDX .init(F_L,ai);
02312         D .init(F_L,ad);
02313
02314         check_IDX_range(F_L);
02315
02316         if (IDX.isEmpty()) {
02317             if (!SIZE.isEmpty() && !IDX.dim2) {
02318                 if (IDX.dim1) wblog(FL,
02319                     "ERR %s() got size %s !?",FCT,SSTR(this));
02320                 IDX.init(IDX.dim1,SIZE.len);
02321             }
02322         }
02323
02324         if (!isDiag()) {
02325             if (IDX.dim1!=D.len || (IDX.dim1 && IDX.dim2!=SIZE.len))
02326                 sperror_this(F_L);
02327         }
02328
02329         Compress();
02330         return *this;
02331     }
02332 }

```

```

02333
02334     if (mxIsCell(a)) {
02335         a=mxGetCell(a,k); if (!a) return *this;
02336     }
02337     else if (k) wblog(FL,"ERR %s() got k=%d !?",FCT,k);
02338
02339     if (mxIsSparse(a)) {
02340
02341         SPIDX_T j,k,l,m,n,d, nnz=mxGetNzmax(a);
02342         mwIndex *Ir,*Jc;
02343
02344         if (!mxIsDouble(a) || mxGetNumberOfDimensions(a)!=2) wblog(FL,
02345             "ERR %s() invalid input (got %s)",FCT,mxTypeSize2Str(a).data);
02346
02347         m=mxGetM(a); n=mxGetN(a); initz(m,n,nnz);
02348         Ir=mxGetIr(a);
02349         Jc=mxGetJc(a);
02350
02351         if (mxIsComplex(a)) {
02352             if (typeid(TD)!=typeid(wbcomplex)) wblog(FL,
02353                 "ERR %s() got complex sparse array for %s",FCT,TSTR(TD));
02354             wbcomplex *dz=(wbcomplex*)mxGetComplexDoubles(a);
02355             for (l=j=0; j<n; ++j) { d=SPIDX_T(Jc[j+1]-Jc[j]);
02356                 for (k=0; k<d; ++k,++l) { D[l]=dz[l];
02357                     IDX(l,0)=SPIDX_T(Ir[l]); IDX(l,1)=j;
02358                 }}
02359         }
02360
02361         else {
02362             double *dr=mxGetDoubles(a);
02363             for (l=j=0; j<n; ++j) { d=SPIDX_T(Jc[j+1]-Jc[j]);
02364                 for (k=0; k<d; ++k,++l) { D[l]=dr[l];
02365                     IDX(l,0)=SPIDX_T(Ir[l]); IDX(l,1)=j;
02366                 }}
02367         }
02368
02369         if (l && l!=nnz) wblog(FL,
02370             "ERR severe sparse inconsistency (mex: %d,%d)",l,nnz);
02371     }
02372     else {
02373
02374         if (!mxIsDouble(a)) wblog(FL,
02375             "ERR %s() got '%s' data",FCT,mxGetClassName(a));
02376         if (typeid(TD)==typeid(wbcomplex)) wblog(FL,
02377             "WRN %s() ignores complex input data",FCT);
02378
02379         SPIDX_T r=mxGetNumberOfDimensions(a);
02380         const size_t *sp=mxGetDimensions(a);
02381         const double *dr=mxGetDoubles(a);
02382
02383         wbvector<SPIDX_T> S(r), I(r);
02384         SPIDX_T i,k,N, nz=0, iz=0, l=r-1, *s=S.data, *ip=I.data;
02385
02386         for (i=0; i<r; ++i) S[i]=(SPIDX_T)sp[i];
02387
02388         N=S.prod(0);
02389         for (i=0; i<N; ++i) { if (dr[i]!=0) ++nz; }
02390         init(S,nz);
02391
02392         for (i=0; i<N; ++i) {
02393             if (dr[i]!=0) {
02394                 if (iz>nz) wblog(FL,
02395                     "ERR %s() index out of bounds (%d/%d) !?",FCT,iz+1,nz);
02396                 D[iz]=dr[i]; IDX.recSetP(iz,ip); ++iz;
02397             }
02398
02399             k=0; ++ip[0];
02400             while (ip[k]>=s[k] && k<1) { ip[k]=0; ++ip[++k]; }
02401         }
02402     }
02403
02404     return *this;
02405 };
02406
02407 template <class TD>
02408 wbsparray<TD>& wbsparray<TD>::init(
02409     const char *F, int L, const wbararray<TD> &A, TD eps
02410 ){
02411     unsigned k, r=A.rank(), l=r-1;
02412     wbvector<SPIDX_T> I(r);
02413     SPIDX_T i,N, nz=0, iz=0, *ip=I.data; const size_t *s=A.SIZE.data;
02414
02415     N=A.numel();
02416     for (i=0; i<N; ++i) { if (Wb::abs(A.data[i])>eps) ++nz; }
02417     init(A.SIZE,nz);
02418
02419     for (i=0; i<N; ++i) {

```

```

02420         if (Wb::abs(A.data[i])>eps) {
02421             if (iz>=nz) wblog(FL,
02422                 "ERR %s() index out of bounds (%d/%d) !?",FCT,iz+1,nz);
02423             D[iz]=A.data[i]; IDX.recSetP(iz,ip); ++iz;
02424         }
02425
02426         k=0; ++ip[0];
02427         while (ip[k]>=s[k] && k<l) { ip[k]=0; ++ip[++k]; }
02428     }
02429
02430     return *this;
02431 };
02432
02433 template <class TD>
02434 template <class TA>
02435 wbspararray<TA>& wbspararray<TD>::toFull(wbspararray<TA> &A) const {
02436
02437     if (isDiag()) { A.initDiag(D); }
02438     else {
02439         unsigned k, r=rank(FL), l=r-1;
02440         SPIDX_T i,j; const SPIDX_T *s=SIZE.data, *ip=IDX.data;
02441
02442         A.init(SIZE);
02443         if (IDX.dim1 && A.data!=NULL) {
02444             for (i=0; i<IDX.dim1; ++i, ip+=IDX.dim2) {
02445                 for (j=ip[l], k=l-1; k<r; --k) { j=j*s[k]+ip[k]; }
02446                 A.data[j]=TA(D.data[i]);
02447             }
02448         }
02449     }
02450     return A;
02451 };
02452
02453 template <class TD>
02454 template <class T2>
02455 wbvector<T2>& wbspararray<TD>::toFull(wbvector<T2> &A) const {
02456
02457     if (SIZE.len) {
02458         unsigned m=IDX.dim2, r=rank(FL);
02459         SPIDX_T i=0; const SPIDX_T *s=SIZE.data, *ip=0;
02460
02461         for (; i<r; ++i) { if (s[i]!=1) {
02462             if (!ip) { ip=IDX.data+i; }
02463             else wblog(FL,
02464                 "ERR %s() invalid usage (got %s tensor)",
02465                 FCT,SSTR_(this)
02466             );
02467         }}
02468         if (!ip) ip=IDX.data;
02469
02470         A.init(numel());
02471         for (i=0; i<D.len; ++i, ip+=m) {
02472             if ((*ip)>=A.len) wblog(FL,
02473                 "ERR %s() invalid sparse setting (%d/%d)",FCT,*ip,A.len);
02474             if (A[*ip]) wblog(FL,"ERR %s() "
02475                 "invalid sparse setting (%d: %g)",FCT,*ip,double(A[*ip]));
02476             A[*ip]=D.data[i];
02477         }
02478     }
02479     else { A.init(1);
02480         if (D.len) {
02481             if (D.len>1) wblog(FL,
02482                 "ERR %s() invalid scalar (len=%d)",FCT,D.len);
02483             A.data[0]=D.data[0];
02484         }
02485         else { A.data[0]=0; }
02486     }
02487     return A;
02488 };
02489
02490 template <class TD> inline
02491 wbspararray<TD>& wbspararray<TD>::reshape(
02492     const wbvector<SPIDX_T> &S, wbspararray<TD> &A
02493 ) const {
02494
02495     if (S==SIZE) { A=*this; return A; }
02496
02497     if (!S.len && (SIZE.allEqual(1) && D.len<=1)) {
02498         A.init(); A.D=D;
02499         return A;
02500     }
02501
02502     if (isDiag()) {
02503         wbspararray<TD> X; diag2reg(FL,X);
02504         return X.reshape(S,A);
02505     }
02506

```



```

02507     if (sameSizeUp2Singletons(S)) {
02508         unsigned j; SPIDX_T i=0, *I; const SPIDX_T *I0=IDX.data; int *ms;
02509
02510         wvector<int> Ms; matchWithSingletons(FL,S,Ms);
02511         A.init(S); A.D=D; A.IDX.init(IDX.dim1,S.len);
02512         ms=Ms.data; I=A.IDX.data;
02513
02514         for (; i<IDX.dim1; ++i, I+=S.len, I0+=IDX.dim2) {
02515             for (j=0; j<S.len; ++j) {
02516                 if (ms[j]>=0) { I[j]=I0[ms[j]]; }
02517             }
02518         }
02519         return A;
02520     }
02521
02522     if (numel()!=S.prod(0)) wblog(FL,
02523         "ERR %s() got size mismatch (%s => %s)",
02524         FCT,SSTR_(this), SSTR(S)
02525     );
02526
02527     A.init(S); A.D=D; A.IDX.init(IDX.dim1,S.len);
02528
02529     if (!IDX.isEmpty()) {
02530         SPIDX_T i,k, *I=A.IDX.data; unsigned r0=SIZE.len, r=S.len;
02531         const SPIDX_T *I0=IDX.data, *S0=SIZE.data, *S=A.SIZE.data;
02532
02533         for (i=0; i<IDX.dim1; ++i, I0+=r0, I+=r) {
02534             k=Wb::sub2ind(S0,I0,r0);
02535             Wb::ind2sub(k,S,I,r);
02536         }
02537     }
02538
02539     return A;
02540 };
02541
02542 template <class TD>
02543 template <class DB> inline
02544 wbsparray<TD>& wbsparray<TD>::init(
02545     const char *F, int L,
02546     SPIDX_T d1, SPIDX_T d2, const wbsparray<DB> &B
02547 ){
02548     SPIDX_T s[] = {d1,d2};
02549     wvector<SPIDX_T> S; S.init(2,s);
02550
02551     if (B.numel()!=S.prod(0)) wblog(F_L,"ERR %s() size mismatch "
02552         "(%s <> %s)",FCT,SSTR(S),SSTR(B));
02553     (*this)=B; return Reshape(S);
02554 };
02555
02556 template <class TD>
02557 template <class DB> inline
02558 wbsparray<TD>& wbsparray<TD>::init(
02559     const char *F, int L,
02560     SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, const wbsparray<DB> &B
02561 ){
02562     SPIDX_T s[] = {d1,d2,d3};
02563     wvector<SPIDX_T> S; S.init(3,s);
02564
02565     if (B.numel()!=S.prod(0)) wblog(F_L,"ERR %s() size mismatch "
02566         "(%s <> %s)",FCT,SSTR(S),SSTR(B));
02567     (*this)=B; return Reshape(S);
02568 };
02569
02570 template <class TD>
02571 template <class DB> inline
02572 wbsparray<TD>& wbsparray<TD>::init(
02573     const char *F, int L,
02574     SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T d4, const wbsparray<DB> &B
02575 ){
02576     SPIDX_T s[] = {d1,d2,d3,d4};
02577     wvector<SPIDX_T> S; S.init(4,s);
02578
02579     if (B.numel()!=S.prod(0)) wblog(F_L,"ERR %s() size mismatch "
02580         "(%s <> %s)",FCT,SSTR(S),SSTR(B));
02581     (*this)=B; return Reshape(S);
02582 };
02583
02584 template <class TD>
02585 wbsparray<TD>& wbsparray<TD>::splitLast(SPIDX_T k, wbsparray &a) const {
02586
02587     if (&a==this) {
02588         wblog(FL,"ERR %s() output same as input space",FCT);
02589     }
02590
02591     unsigned r=SIZE.len, l=r-1;
02592     SPIDX_T *I=IDX.data+l, i=0, n=0;
02593

```

```

02594     if (r<2) {
02595         wblog(FL, "ERR %s() got S=%s (k=%d)", FCT, SSTR_(this), k);
02596     }
02597
02598     if (k>=SIZE[1]) wblog(FL,
02599         "ERR %s() index out of bounds (%s /%d)", FCT, SSTR_(this), k);
02600     if (IDX.dim2!=r) wblog(FL, "ERR %s() %d/%d", FCT, IDX.dim2, r);
02601
02602     for (; i<IDX.dim1; ++i, I+=r) { if (*I==k) ++n; }
02603
02604     a.SIZE.init(1, SIZE.data);
02605     a.init(a.SIZE, n);
02606
02607     if (n) {
02608         for (I=IDX.data+1, n=i=0; i<IDX.dim1; ++i, I+=r) {
02609             if (*I==k) {
02610                 a.IDX.recSetP(n, IDX.rec(i));
02611                 a.D[n++]=D[i];
02612             }
02613         }
02614     }
02615     return a;
02616 };
02617
02618 template <class TD>
02619 wbspararray<TD>& wbspararray<TD>::select0(
02620     const WBINDEX &P, unsigned dim,
02621     wbspararray<TD> &X
02622 ) const {
02623
02624     if (&X==this) {
02625         wbspararray<TD> Q; X.save2(Q);
02626         return Q.select0(P, dim, X);
02627     }
02628     if (dim>=SIZE.len) wblog(FL,
02629         "ERR %s() dimension out of bounds (%d/%d)", FCT, dim+1, SIZE.len);
02630
02631     SPIDX_T i, l, d0=SIZE[dim], *j;
02632     sSPIDX_T *I;
02633
02634     wbvector<sSPIDX_T> I_(d0); I_.set(-1); I=I_.data;
02635
02636     for (i=0; i<P.len; ++i) {
02637         if (P[i]>=d0) wblog(FL,
02638             "ERR %s() index out of bounds (%d/%d)", FCT, P[i], d0);
02639         if (I[P[i]]>=0) wblog(FL,
02640             "ERR %s() input index not unique (%d/%d)", FCT, P[i], d0);
02641         I[P[i]]=i;
02642     }
02643
02644     for (j=IDX.data+dim, l=i=0; i<IDX.dim1; ++i, j+=IDX.dim2) {
02645         if (I[*j]>=0) ++l;
02646     }; X.init(SIZE, l); X.SIZE[dim]=P.len;
02647
02648     for (j=IDX.data+dim, l=i=0; i<IDX.dim1; ++i, j+=IDX.dim2) {
02649         if (I[*j]>=0) {
02650             X.IDX.recSetP(l, IDX.rec(i)); X.IDX(1, dim)=I[*j];
02651             X.D[l++]=D[i];
02652         }
02653     }
02654
02655     return X;
02656 };
02657
02658 template <class TD>
02659 wbspararray<TD>& wbspararray<TD>::getBlock(const char *F, int L,
02660     SPIDX_T i1, SPIDX_T i2, SPIDX_T j1, SPIDX_T j2,
02661     wbspararray<TD> &B
02662 ) const {
02663
02664     SPIDX_T k1, k2, dim1, dim2, i=0, l=0, n=IDX.dim1;
02665
02666     if (!isRank(2)) wblog(F_L,
02667         "ERR %s() only applies to rank-2 tensors (%d)", FCT, SIZE.len);
02668
02669     if (!SIZE.len) { dim1=dim2=D.len; }
02670     else {
02671         if (SIZE.len!=2 || IDX.dim2!=2) wblog(F_L, "ERR %s() "
02672             "got invalid rank-2 tensor (%ld,%ld)!", FCT, SIZE.len, IDX.dim2);
02673         dim1=SIZE[0]; dim2=SIZE[1];
02674     }
02675
02676     if (sSPIDX_T(i2)<0) i2+=dim1;
02677     if (sSPIDX_T(j2)<0) j2+=dim2;
02678
02679     if (sSPIDX_T(i1)<0 || i2>=dim1) wblog(F_L, "ERR %s() \n"
02680         "index out of bounds (1: %ld .. %ld / %ld)", FCT, i1, i2, dim1);

```

```

02681     if (sSPIDX_T(j1)<0 || j2>=dim2) wblog(F_L,"ERR %s() \n"
02682         "index out of bounds (l: %ld .. %ld / %ld)", FCT,j1,j2,dim2);
02683
02684     k1=(i2>=i1 ? i2-i1+1 : 0);
02685     k2=(j2>=j1 ? j2-j1+1 : 0);
02686
02687     B.init(k1,k2); if (!k1 || !k2) return B;
02688
02689     if (!SIZE.len) {
02690         k1=MAX(i1,j1); k2=MIN(i2,j2); l=(k2>k1 ? k2-k1 : 0);
02691         if (l) { B.init_nnz(l); SPIDX_T *idx=B.IDX.data;
02692             for (; i<l; ++i, ++k1, idx+=2) {
02693                 idx[0]=k1-i1;
02694                 idx[1]=k1-j1;
02695                 B.D.data[i]=D.data[k1];
02696             }
02697         }
02698         return B;
02699     }
02700
02701     const SPIDX_T *i0=IDX.data; SPIDX_T *idx;
02702
02703     for (; i<n; ++i, i0+=2) {
02704         if (i0[1]>=j1) {
02705             if (i0[1]<=j2) { if (i0[0]>=i1 && i0[0]<=i2) ++l; }
02706             else break;
02707         }
02708     }
02709
02710     if (l) { B.init_nnz(l); idx=B.IDX.data; i0=IDX.data; l=0;
02711         for (i=0; i<n; ++i, i0+=2) {
02712             if (i0[1]>=j1) {
02713                 if (i0[1]<=j2) { if (i0[0]>=i1 && i0[0]<=i2) {
02714                     idx[0]=i0[0]-i1;
02715                     idx[1]=i0[1]-j1;
02716                     B.D.data[l]=D.data[i]; idx+=2; ++l;
02717                 } } else break;
02718             }
02719         }
02720     }
02721
02722     return B;
02723 };
02724
02725 template <class TD>
02726 wbsparray<TD>& wbsparray<TD>::splitBlock(const char *F, int L,
02727     sSPIDX_T i1, sSPIDX_T i2, sSPIDX_T j1, sSPIDX_T j2,
02728     wbsparray<TD> &B
02729 ){
02730
02731     SPIDX_T k1,k2,dim1,dim2, j,i_, i=0, l=0, n=IDX.dim1;
02732
02733     if (!isRank(2) || SIZE.len!=2) wblog(F_L,
02734         "ERR %s() only applies to rank-2 tensors (%d)",FCT,SIZE.len);
02735
02736     B.init(SIZE); if (!D.len) return B;
02737     dim1=SIZE[0]; dim2=SIZE[1];
02738
02739     if (i2<0) i2+=dim1;
02740     if (j2<0) j2+=dim2;
02741
02742     if (i1<0 || i2>=dim1) wblog(F_L,"ERR %s() \n"
02743         "index out of bounds (l: %ld .. %ld / %ld)", FCT,i1,i2,dim1);
02744     if (j1<0 || j2>=dim2) wblog(F_L,"ERR %s() \n"
02745         "index out of bounds (l: %ld .. %ld / %ld)", FCT,j1,j2,dim2);
02746
02747     k1=(i2>=i1 ? i2-i1+1 : 0);
02748     k2=(j2>=j1 ? j2-j1+1 : 0); if (!k1 || !k2) return B;
02749
02750     SPIDX_T *j0, *i0=IDX.data, *idx;
02751
02752     for (; i<n; ++i, i0+=2) { if (i0[1]>=j1) break; }; i_=i;
02753     for (; i<n; ++i, i0+=2) {
02754         if (i0[1]<=j2) { if (i0[0]>=i1 && i0[0]<=i2) ++l; }
02755         else break;
02756     }
02757
02758     if (l) {
02759         B.init_nnz(l); idx=B.IDX.data; l=0; j=i_; i0=IDX.data+2*i;
02760         for (; i<n; ++i, i0+=2) {
02761             if (i0[1]<=j2 && i0[0]>=i1 && i0[0]<=i2) {
02762                 idx[0]=i0[0];
02763                 idx[1]=i0[1]; idx+=2;
02764                 B.D.data[l]=D.data[i]; ++l;
02765             }
02766             else {
02767                 if (j<i) { j0=IDX.data+2*j;

```

```

02768             j0[0]=i0[0];
02769             j0[1]=i0[1]; D[j]=D[i];
02770         }; ++j;
02771     }
02772 }
02773 }
02774
02775 REDSIZE_NNZ(j);
02776 return B;
02777 };
02778
02779 template <class TD>
02780 TD wbsparray<TD>::getHouseholderVec(const char *F, int L,
02781     SPIDX_T k,
02782     SPIDX_T j,
02783     wbsparray<TD> &u,
02784     TD eps2
02785 ) const {
02786     TD x2=0, b2=0;
02787
02788     if (!isRank(2) || SIZE.len!=2 || IDX.dim2!=2) wblog(F_L,
02789         "ERR %s() only applies to rank-2 tensors (%s; %d,%d)",
02790         FCT,SSTR_(this),SIZE.len,IDX.dim2);
02791
02792     SPIDX_T dim1=SIZE[0], dim2=SIZE[1], i1,i2, i=0, l=0, n=IDX.dim1;
02793     const SPIDX_T *i0=IDX.data;
02794     char gotk=0;
02795
02796     if (k>=dim2) wblog(F_L,"ERR %s() "
02797         "col-index out of bounds (%d/%d)",FCT,k,dim2);
02798     u.init(dim1); if (!dim1) return x2;
02800
02801     for (; i<n; ++i, i0+=2) { if (i0[1]>=j) break; }
02802     for (; i<n; ++i, i0+=2) { if (i0[0]>=k) break; }
02803     i1=i; if (i<n && i0[0]==k && i0[1]==j) gotk=1;
02804     for (; i<n; ++i, i0+=2) {
02805         if (i0[1]==j) { b2+=Wb::norm2(D[i]); ++l; }
02806         else break;
02807     }
02808     i2=i;
02809
02810     if (b2<=eps2) {
02811         if (b2==0) {
02812             wblog(F_L,"WRN %s() got 0 vector (%s) !?",FCT,SSTR_(this));
02813             return x2;
02814         }
02815         wblog(F_L,"WRN %s() got b2=%.3g !?",FCT,double(b2));
02816     }
02817
02818     u.init_nnz(1+(gotk?0:1));
02819
02820     SPIDX_T *idx=u.IDX.data; i0=IDX.rec(i1);
02821
02822     if (gotk) { l=0; } else { u.D[0]=0; idx[0]=k; l=1; }
02823
02824     for (i=i1; i<i2; ++i, i0+=2) {
02825         u.D[l]=D.data[i]; idx[l]=i0[0]; ++l;
02826     }
02827
02828     x2=Wb::norm2(u.D[0]);
02829     if (u.D[0]<0) { u.D[0]-=Wb::sqrt(b2); } else { u.D[0]+=Wb::sqrt(b2); }
02830     x2=b2-x2+Wb::norm2(u.D[0]);
02831
02832     u*=(TD(1)/Wb::sqrt(x2));
02833
02834     return x2;
02835 };
02836
02837 template <class TD>
02838 wbsparray<TD>& wbsparray<TD>::IDX_Shift(sSPIDX_T i1, sSPIDX_T i2) {
02839
02840     if (!isRank(2) || SIZE.len!=2 || IDX.dim2!=2) wblog(F_L,
02841         "ERR %s() only applies to rank-2 tensors (%s; %d,%d)",
02842         FCT,SSTR_(this),SIZE.len,IDX.dim2
02843     );
02844
02845     SPIDX_T i=0, n=IDX.dim1, *I=IDX.data;
02846     SPIDX_T dim1=SIZE[0]+i1, dim2=SIZE[1]+i2; SIZE[0]=dim1; SIZE[1]=dim2;
02847
02848     if (i1>=0 && i2>=0) {
02849         for (; i<n; ++i, I+=2) { I[0]+=i1; I[1]+=i2; }
02850     }
02851     else {
02852         if (sSPIDX_T(dim1)<0) wblog(F_L,
02853             "ERR %s() index out of bounds (%ld/%ld)",FCT,i1,SIZE[0]);
02854         if (sSPIDX_T(dim2)<0) wblog(F_L,

```

```

02855         "ERR %s() index out of bounds (%ld/%ld)", FCT, i2, SIZE[1]);
02856
02857     for (; i<n; ++i, I+=2) { I[0]+=i1; I[1]+=i2;
02858         if (I[0]>=dim1 || I[1]>=dim2) wblog(FL,
02859             "ERR %s() index out of bounds (%ld,%ld; %s)",
02860             FCT, I[0], I[1], SSTR_(this)
02861         );
02862     }
02863 }
02864
02865     return *this;
02866 };
02867
02868 template <class TD>
02869 SPIDX_T wbsparray<TD>::LContractVec_omp(
02870     const wbsparray<TD> &b, wvector<TD> &x, char tnorm) const {
02871
02872     SPIDX_T i, k0=0, d2, l=0;
02873
02874     if (!isMatrix()) wblog(FL,
02875         "ERR %s() only applies to rank-2 tensors (%d)", FCT, SIZE.len);
02876
02877     if (!isEmpty() && !b.isEmpty()) {
02878         if (b.IDX.dim2!=1 || b.SIZE.len!=1) wblog(FL,
02879             "ERR %s() expecting sparse VECTOR\n(got b: IDX=%s; SIZE=%s)",
02880             FCT, SSTR(b.IDX), STR(b.SIZE));
02881         if (dim0(0)!=b.SIZE[0]) wblog(FL,
02882             "ERR %s() size mismatch (%d/%d)", FCT, SSTR_(this), SSTR(b));
02883     }
02884
02885     if (!D.len || !b.D.len) { x.init(); return l; }
02886
02887     if (SIZE.len) {
02888         indexSparseRef<TD> ISP(FL, *this);
02889         d2=ISP.numCols(); x.init(d2);
02890
02891         #ifndef WB_SKIP_ASSERT
02892         SPIDX_T d1=ISP.numRows();
02893         if (d1!=SIZE[0] || d2!=SIZE[1] || !d1 || !d2) wblog(FL,
02894             "ERR %s() got size inconsistency (%dx%d / %s)",
02895             FCT, d1, d2, SSTR_(this));
02896         #endif
02897
02898         int np=1;
02899         if (sp_num_threads>1) {
02900             np=IDX.dim1/1024;
02901             if (np>sp_num_threads) np=sp_num_threads;
02902             if (np>int(d2)) np=d2;
02903         }
02904
02905         for (; k0<d2; ++k0) { if (ISP.nnz(k0)) break; }
02906
02907         #pragma omp parallel for num_threads(np) reduction(+:l)
02908         for (SPIDX_T k=k0; k<d2; ++k) {
02909             x[k]=ISP.LContractVec(b,k,tnorm);
02910             if (x[k]) l+=1;
02911         }
02912
02913         if (l!=x.nnz()) {
02914             MXPut(FL, "q").add(x, "x").add(l, "l"); SPIDX_T n=0;
02915             for (i=0; i<x.len; ++i) {
02916                 if (x[i]) printf("%6ld : %+.4g\n", i, ++n, double(x[i])); else
02917                 if (x[i]!=0) printf("%6ld : %+.4g (!)\n", i, ++n, double(x[i]));
02918             }
02919             wblog(FL, "ERR %s() %ld/%ld !?", FCT, l, x.nnz());
02920         }
02921     }
02922     else {
02923         const SPIDX_T *idx=b.IDX.data;
02924         d2=D.len;
02925         x.init(d2); k0=b.IDX[0]; l=b.IDX.dim1;
02926         for (i=0; i<l; ++i) {
02927             if (idx[i]>=d2 || (i && idx[i-1]>=idx[i])) wblog(FL,
02928                 "ERR %s() b.IDX out of bounds (%ld/%ld) !?", FCT, idx[i], d2);
02929             x[idx[i]]=b.D[i]*D[idx[i]];
02930         }
02931     }
02932
02933     return l;
02934 };
02935
02936 template <class TD>
02937 wbsparray<TD> & wbsparray<TD>::LProject1(
02938     const wbsparray<TD> &B, wbsparray<TD> &X, TD xfac, char tnorm) const {
02939
02940     if (!D.len) { X=(*this); return X; }
02941     if (!B.D.len) { X.init(SIZE); return X; }

```

```

02942
02943     wbvector<TD> x;
02944     SPIDX_T i, d1,d2;
02945     SPIDX_T l=LContractVec_omp(B,x,tnorm);
02946
02947     if (xfac!=1) {
02948         if (xfac) x*=xfac;
02949         else wblog(FL,"ERR %s() got xfac=0 !?",FCT);
02950     }
02951
02952     d1=B.D.len; d2=x.len;
02953     if (SIZE.len) { X.init(SIZE, 1*d1); }
02954     else {
02955         SPIDX_T s[2]={d2,d2};
02956         X.init(wbvector<SPIDX_T>(2,s), 1*d1);
02957     }
02958
02959     const SPIDX_T *I=B.IDX.data;
02960     SPIDX_T *idx=X.IDX.data, r=X.IDX.dim2; l=0;
02961     const TD *b=B.D.data; TD *xb=X.D.data;
02962
02963     for (SPIDX_T k=0; k<d2; ++k) { if (x[k]) {
02964         for (i=0; i<d1; ++i, ++l, idx+=r) {
02965             xb[l]=x[k]*b[i]; idx[0]=I[i]; idx[1]=k;
02966         }
02967     }}
02968
02969     if (l!=X.IDX.dim1) wblog(FL,"ERR %s() %ld/%ld !?",FCT,l,X.IDX.dim1);
02970     return X;
02971 };
02972
02973 template <class TD> inline
02974 wbspararray<TD>& wbspararray<TD>::Permute(const wbperm &P, char iflag){
02975
02976     if (!SIZE.len && !IDX.dim1 && D.len<=1) return *this;
02977     if (!P.isEmpty()) {
02978         SIZE.Permute(P,iflag); IDX.colPermute(P,iflag);
02979         Sort();
02980     }
02981     return *this;
02982 };
02983
02984 template <class TD> inline
02985 wbspararray<TD>& wbspararray<TD>::permute(
02986     const wbperm &P, wbspararray<TD>&B, char iflag
02987 ) const {
02988
02989     if (this==&B) return B.Permute(P,iflag);
02990
02991     if (P.isEmpty() || (!SIZE.len && !IDX.dim1 && D.len<=1)) { B=(*this); }
02992     else {
02993         SIZE.permute(B.SIZE,P,iflag); B.D=D; B.isref=0;
02994         IDX.colPermute(P,B.IDX,iflag);
02995         B.Sort();
02996     }
02997     return B;
02998 };
02999
03000 template <class TD> inline
03001 wbspararray<TD>& wbspararray<TD>::Permute(const char* s, char iflag){
03002     return Permute(Str2Idx(s,1),iflag);
03003 };
03004
03005 template <class TD> inline
03006 wbspararray<TD>& wbspararray<TD>::permute(
03007     const char* s, wbspararray<TD>&Q, char iflag
03008 ) const { return permute(Q,Str2Idx(s,1),iflag); };
03009
03010 template <class TD> inline
03011 wbspararray<TD>& wbspararray<TD>::ColPermute(const wbperm &P0, char iflag){
03012
03013     if (isIdentityPerm(P0)) return *this;
03014
03015     if (SIZE.len!=2 || IDX.dim2!=2) wblog(FL,
03016         "ERR %s() requires rank-2 (%s; %d)",FCT,SSTR_(this),IDX.dim2);
03017     if (P0.len!=SIZE[1]) wblog(FL,
03018         "ERR %s() size mismatch (%d/%d)",FCT,P0.len,SIZE[1]);
03019
03020     SPIDX_T k,i=0, l=IDX.dim2-1; wbperm P;
03021
03022     if (!iflag)
03023         P0.invert(P);
03024     else P.wbvector<wbperm_t>::init2ref(P0);
03025
03026     for (; i<IDX.dim1; ++i) { k=IDX(i,1);
03027         if (k>=P.len) wblog(FL,"ERR %s() %d/%d !?",FCT,k,P.len);
03028         IDX(i,1)=P[k];

```

```

03029     }
03030     return Sort();
03031 };
03032
03033 template <class TD> inline
03034 wbsparray<TD>& wbsparray<TD>::MatPermute(const wperm &P0, char iflag){
03035
03036     if (isIdentityPerm(P0)) return *this;
03037
03038     SPIDX_T d=-1;
03039     if (!isSMatrix(FL,&d)) wblog(FL,
03040         "ERR %s() requires rank-2 (%s; %d)",FCT,SSTR_(this),IDX.dim2);
03041     if (P0.len!=d) wblog(FL,
03042         "ERR %s() size mismatch (%d/%d)",FCT,P0.len,d);
03043
03044     if (isDiag()) { D.Permute(P0); }
03045     else {
03046         wperm P;
03047         if (!iflag)
03048             P0.invert(P);
03049         else P.wbvector<wperm_t>::init2ref(P0);
03050
03051         SPIDX_T *I=IDX.data, n=IDX.numel(); wperm_t *p=P.data;
03052         for (SPIDX_T i=0; i<n; ++i) {
03053             if (I[i]>=P.len) wblog(FL,"ERR %s() %d/%d !?",FCT,I[i],P.len);
03054             I[i]=p[I[i]];
03055         }
03056     }
03057
03058     return Sort();
03059 };
03060
03061 template <class TD>
03062 void wbsparray<TD>::setSize_kron(
03063     const char *F, int L,
03064     const wbvector<SPIDX_T> &sa, const wbvector<SPIDX_T> &sb,
03065     char kflag
03066 ){
03067     wbvector<SPIDX_T> S;
03068     if (sa.len!=sb.len) wblog(F_L,
03069         "ERR %s() rank mismatch (%d/%d)",FCT,sa.len,sb.len);
03070
03071     if (kflag) { S.init(sa.len); SPIDX_T *s=S.data;
03072         for (unsigned i=0; i<sa.len; ++i) {
03073             s[i]=sa[i]*sb[i];
03074         }
03075     }
03076     else { S.init(sa.len+sb.len); SPIDX_T *s=S.data;
03077         for (unsigned i=0, l=0; i<sa.len; ++i) {
03078             s[l++]=sa[i];
03079             s[l++]=sb[i];
03080         }
03081     }
03082     S.save2(SIZE);
03083 };
03084
03085 template <class TD>
03086 void wbsparray<TD>::setSize_kron(const char *F, int L, char kflag) {
03087
03088     if (kflag) {
03089         unsigned l=0, i=0, r=IDX.dim2; SPIDX_T *s=SIZE.data;
03090         if (SIZE.len!=2*r) wblog(F_L,
03091             "ERR %s() unexpected size (%dx%d <> %d/(%d/2))",
03092             FCT,IDX.dim1,IDX.dim2,D.len,SIZE.len
03093         );
03094         if (r) {
03095             for (; i<r; ++i,l+=2) { s[i]=s[l]*s[l+1]; }
03096             SIZE.len=r;
03097         }
03098     }
03099     else if (SIZE.len!=IDX.dim2) sperror_this(F_L);
03100 };
03101
03102 template <class TD>
03103 wbsparray<TD>& wbsparray<TD>::setRec_kron(
03104     SPIDX_T l, TD x, unsigned r, SPIDX_T *i1, SPIDX_T *i2,
03105     unsigned kflag
03106 ){
03107     if (l>IDX.dim1) wblog(FL,"ERR %s() index "
03108         "out of bounds (%d,%d <> %dx%d)",FCT,l,r,IDX.dim1,IDX.dim2);
03109
03110     if (kflag) {
03111         if (l==0 && (SIZE.len!=2*r || IDX.dim2!=r))
03112             wblog(FL,"ERR %s() 2*%d/2*%d/%d",FCT,r,IDX.dim2,SIZE.len);
03113         SPIDX_T *q=IDX.rec(l);
03114         for (unsigned i=0; i<r; ++i) {
03115             q[i]=i1[i] + SIZE[2*i]*i2[i];

```

```

03116     }
03117 }
03118 else {
03119     if (l==0 && (SIZE.len!=2*r || SIZE.len!=IDX.dim2))
03120         wblog(FL,"ERR %s() index out of bounds (%d,%d <> %dx%d)",
03121             FCT,l,r,IDX.dim1,IDX.dim2
03122         );
03123
03124     SPIDX_T *q=IDX.rec(l);
03125     for (unsigned j=0, i=0; i<r; ++i) {
03126         q[j++]=i1[i];
03127         q[j++]=i2[i];
03128     }
03129 }
03130 D[l]=x; return *this;
03131 };
03132
03133 template <class TD>
03134 wbspararray<TD>& wbspararray<TD>::transpose(
03135     const char *F, int L, wbspararray<TD> &A
03136 ) const {
03137
03138     unsigned r=rank(); wbperr P;
03139
03140     if (r%2) wblog(F_L,"ERR %s() "
03141         "applies to even-rank arrays only! (%s)",FCT,SSTR_(this));
03142
03143     P.initTranspose(r);
03144     permute(P,A);
03145
03146     return A.Sort();
03147 };
03148
03149 template <class TD> inline
03150 wbspararray<TD>& wbspararray<TD>::tensorProd(const char *F, int L,
03151     const wbspararray<TD>& B0, wbspararray<TD>& C,
03152     char aflag0, char bflag0,
03153     char kflag
03154 ) const {
03155
03156     char iA=isDiag(FL), iB=B0.isDiag(FL);
03157
03158     if (iA || iB) {
03159         if (iA && iB) {
03160             if (kflag) {
03161                 C.initDiag(D.len*B0.D.len); D.tensorProd(B0.D,C.D);
03162                 return C;
03163             }
03164             wbspararray<TD> A2; diag2reg(FL,A2);
03165             wbspararray<TD> B2; B0.diag2reg(FL,B2);
03166             return A2.tensorProd(F,L,B2,C,aflag0,bflag0,kflag);
03167         }
03168         else if (iA) {
03169             wbspararray<TD> A2; diag2reg(FL,A2, B0.rank());
03170             return A2.tensorProd(F,L,B0,C,aflag0,bflag0,kflag);
03171         }
03172         else {
03173             wbspararray<TD> B2; B0.diag2reg(FL,B2, rank());
03174             return tensorProd(F,L,B2,C,aflag0,bflag0,kflag);
03175         }
03176     }
03177
03178     if ((void*)this==(void*)&C || (void*)&B0==(void*)&C) {
03179         wbspararray<TD> X; tensorProd(F_L,B0,X,aflag0,bflag0,kflag);
03180         return X.save2(C);
03181     }
03182
03183     unsigned ra=rank(), rb=B0.rank();
03184
03185     if (!ra || ra%2 || ra!=rb) wblog(F_L,"ERR %s() invalid input to "
03186         "kron(%s,%s)",FCT,SSTR_(this),SSTR(B0));
03187     if (SIZE.len!=IDX.dim2 || B0.SIZE.len!=B0.IDX.dim2) wblog(FL,
03188         "ERR %s() data inconsistency (%s -> %dx%d, %s -> %dx%d)",FCT,
03189         SSTR_(this), IDX.dim1, IDX.dim2,
03190         SSTR(B0),B0.IDX.dim1,B0.IDX.dim2);
03191
03192     opFlags<TD> aflag(aflag0), bflag(bflag0);
03193     wbspararray<TD> A(*this),B(B0);
03194
03195     if (!A.isCompressed()) wblog(FL,"ERR %s() A not sorted/unique",FCT);
03196     if (!B.isCompressed()) wblog(FL,"ERR %s() B not sorted/unique",FCT);
03197
03198     aflag.apply(F_L,A);
03199     bflag.apply(F_L,B);
03200
03201     SPIDX_T i,j, l=0, na=A.IDX.dim1, nb=B.IDX.dim1;
03202     C.init_kron(SIZE,B.SIZE,na*nb,kflag);

```



```

03203
03204     if (C.IDX.dim1) {
03205         for (i=0; i<na; ++i)
03206             for (j=0; j<nb; ++j,++l) {
03207                 C.setRec_kron(
03208                     l, A.D[i]*B.D[j], ra, A.IDX.rec(i), B.IDX.rec(j), kflag
03209                 );
03210             }
03211     }
03212
03213     C.setSIZE_kron(F_L,kflag);
03214     C.Compress (FL);
03215
03216     return C;
03217 };
03218
03219 template <class TD> inline
03220 wbsparray<TD>& wbsparray<TD>::tensorProdX(const char *F, int L,
03221     const wbsparray& B, wbsparray& C,
03222     char conja, const TD &afac, char conjb, const TD &cfac,
03223     const wbpem *Pfinal
03224 ) const {
03225
03226     if (cfac && cfac!=1) { C.D*=cfac; }
03227     if (afac==0) { return C; }
03228
03229     char iA=isDiag(FL), iB=B.isDiag(FL), noconj=1;
03230     unsigned l=0, ra=(iA ? 2 : SIZE.len), rb=(iB ? 2 : B.SIZE.len);
03231
03232     const SPIDX_T *Ia=IDX.data, *Ib=B.IDX.data;
03233     SPIDX_T ia=0, ib=0, ic=0, na=D.len, nb=B.D.len, *Ic;
03234     wbsparray<TD> X;
03235
03236     if ((conja || conjb)) {
03237         if (ISCOMPLX_(TD)) { noconj=0; }
03238         else { conja=conjb=0; }
03239     }
03240
03241     X.SIZE.init(ra+rb); Ic=X.SIZE.data;
03242     if (iA) { Ic[0]=Ic[l]=na; }
03243     else { for (; l<ra; ++l) { Ic[l]=SIZE[l]; }; l=0; }; Ic+=ra;
03244     if (iB) { Ic[0]=Ic[l]=nb; }
03245     else { for (; l<rb; ++l) { Ic[l]=B.SIZE[l]; }}
03246     X.init_nnz(na*nb); Ic=X.IDX.data;
03247
03248     for (; ib<nb; ++ib) {
03249         for (ia=0; ia<na; ++ia, ++ic) {
03250             if (iA) { Ic[0]=Ic[l]=ia; }
03251             else { for (l=0; l<ra; ++l) { Ic[l]=Ia[l]; }; Ia+=ra; }
03252             Ic+=ra;
03253
03254             if (iB) { Ic[0]=Ic[l]=ib; }
03255             else { for (l=0; l<rb; ++l) { Ic[l]=Ib[l]; }; Ib+=rb; }
03256             Ic+=rb;
03257
03258             if (noconj) { X.D[ic]=D[ia]*B.D[ib]; }
03259             else { X.D[ic] =
03260                 (conja ? Wb::CONJ( D[ia]) : D[ia])
03261                 * (conjb ? Wb::CONJ(B.D[ib]) : B.D[ib]);
03262             }
03263         }
03264     }
03265
03266     if (Pfinal && !Pfinal->isIdentityPerm()) { X.Permute(*Pfinal); }
03267
03268     if (afac!=1) { X*=afac; }
03269     if (cfac) {
03270         if (X.SIZE!=C.SIZE) wblog(FL,"ERR %s() "
03271             "got size mismatch (%s <> %s) !?",FCT,SSTR(X),SSTR(C));
03272         C.Plus(F_L,X);
03273     }
03274     else X.save2(C);
03275
03276     return C;
03277 };
03278
03279 template <class TD>
03280 wbsparray<TD>& wbsparray<TD>::Plus(
03281     const char *F, int L,
03282     const wbsparray<TD> &B, TD bfac, TD afac,
03283     TD eps
03284 ){
03285     if (isref) wblog(FL,"ERR %s() got isref=%d !?",FCT,isref);
03286
03287     if ((void*)this==(void*)(&B)) {
03288         TD x=afac+bfac;
03289         if (x== 0) { D.init(); IDX.init(0,IDX.dim2); isref=0; } else

```

```

03290         if (x==-1) { for (SPIDX_T i=0; i<D.len; ++i) D[i]=-D[i]; } else
03291         if (x!=+1) { for (SPIDX_T i=0; i<D.len; ++i) D[i]*=x; }
03292         return *this;
03293     }
03294
03295     if (isEmpty()) { (*this)=B; (*this)*=bfac; return *this; }
03296     if (B.isEmpty()) { (*this)*=afac; return *this; }
03297
03298     char ida=isDiag(), idb=B.isDiag();
03299     if (ida || idb) {
03300         if (ida && idb) {
03301             if (D.len!=B.D.len) wblog(FL,
03302                 "ERR %s() size mismatch (%ld/%ld)", FCT, D.len, B.D.len);
03303             for (SPIDX_T i=0; i<D.len; ++i) { D[i]=afac*D[i]+bfac*B.D[i]; }
03304             return *this;
03305         }
03306         else if (ida) { diag2reg();
03307             return Plus(F, L, B, bfac, afac, eps);
03308         }
03309         else if (idb) {
03310             wbsparray<TD> B2; B.diag2reg(FL, B2);
03311             return Plus(F, L, B2, bfac, afac, eps);
03312         }
03313     }
03314
03315     if (SIZE!=B.SIZE || !SIZE.len) wblog(FL,
03316         "ERR %s() got size mismatch (%s <> %s)", FCT, SSTR_(this), SSTR(B));
03317
03318     if (afac!=1) {
03319         if (afac== 0) { D.init(); IDX.init(0, IDX.dim2); isref=0; } else
03320         if (afac== -1) { for (SPIDX_T i=0; i<D.len; ++i) D[i]=-D[i]; } else
03321         if (afac!=+1) { for (SPIDX_T i=0; i<D.len; ++i) D[i]*=afac; }
03322     }
03323
03324     if (bfac==0) return *this;
03325
03326     SPIDX_T ia=0, ib=0, l=0, nx, na=IDX.dim1, nb=B.IDX.dim1;
03327     const SPIDX_T *Ia=IDX.data, *Ib=B.IDX.data;
03328     unsigned m=SIZE.len;
03329     char lex=0;
03330
03331     wbsparray<TD> X(SIZE);
03332     const TD *a=D.data, *b=B.D.data; TD *x, e2=0;
03333
03334     wbvector<char> qq_(na+nb); char q, *qq=qq_.data;
03335
03336     #ifndef WB_SKIP_ASSERT
03337     SPIDX_T Dmax=SIZE.max();
03338     for (ia=1; ia<na; ia+=Dmax) {
03339         if (Wb::recCompare(Ia+(ia-1)*m, Ia+ia*m, m, lex)>=0) wblog(FL,
03340             "ERR %s() A: input sparse IDX not sorted (%ld/%ld) !?", FCT, ia, na);
03341     }
03342     for (ib=1; ib<nb; ib+=Dmax) {
03343         if (Wb::recCompare(Ib+(ib-1)*m, Ib+ib*m, m, lex)>=0) wblog(FL,
03344             "ERR %s() B: input sparse IDX not sorted (%ld/%ld) !?", FCT, ib, nb);
03345     }
03346     ia=ib=0;
03347     #endif
03348
03349     while (ia<na && ib<nb) {
03350         qq[l++]=q=Wb::recCompare(Ia+ia*m, Ib+ib*m, m, lex);
03351
03352         if (q<=0) { ++ia; }
03353         if (q>=0) { ++ib; }
03354     }
03355
03356     if (ia<na)
03357         { for (; ia<na; ++ia) { qq[l++]=-1; } }
03358     else { for (; ib<nb; ++ib) { qq[l++]="+1; } }
03359
03360     X.init_nnz(1); x=X.D.data; nx=X.IDX.dim1; ia=ib=1=0;
03361
03362     for (; i<nx; ++i) {
03363         if (qq[i]<0) {
03364             x[l]=a[ia]; if (afac!=1) x[l]*=afac;
03365             if (Wb::abs(x[l])>eps) X.IDX.recSetP(l++, Ia); else e2+=Wb::norm2(x[l]);
03366             ++ia; Ia+=m;
03367         }
03368         else if (qq[i]>0) {
03369             x[l]=b[ib]; if (bfac!=1) x[l]*=bfac;
03370             if (Wb::abs(x[l])>eps) X.IDX.recSetP(l++, Ib); else e2+=Wb::norm2(x[l]);
03371             ++ib; Ib+=m;
03372         }
03373         else {
03374             x[l]=a[ia]; if (afac!=1) x[l]*=afac;
03375             x[l]+=(bfac!=1 ? bfac*b[ib] : b[ib]);
03376             if (Wb::abs(x[l])>eps) X.IDX.recSetP(l++, Ia); else e2+=Wb::norm2(x[l]);

```

```

03377         ++ia; ++ib; Ia+=m; Ib+=m;
03378     }
03379 }
03380
03381 X.REDSIZE_NNZ(1);
03382
03383 return X.save2(*this);
03384 };
03385
03386 template <class TD>
03387 double wbsparray<TD>::Compress(
03388     const char *F, int L, TD eps, char lex, const wbperm *Pc
03389 ){
03390     wbperm P; WBINDEX dg; rank(F_L);
03391
03392     checkSize(F_L);
03393     if (!IDX.dim1) {
03394         double e2=0;
03395         if (Pc && Pc->len!=2) wblog(FL,
03396             "ERR %s() invalid permutation (%s)",FCT,Pc->toStr().data);
03397         if (double(eps)>0) {
03398             for (SPIDX_T k=0; k<D.len; ++k) {
03399                 TD &x=D.data[k];
03400                 if (x && Wb::abs(x)<=eps) { e2+=double(Wb::norm2(x)); x=0; }
03401             }
03402         }
03403         return e2;
03404     }
03405
03406     if (!Pc || Pc->isIdentityPerm()) {
03407         if (double(eps)<=0 && IDX.isUniqueSorted(+1,lex)) { return 0; }
03408         IDX.groupRecs(P,dg,-1,lex);
03409     }
03410     else {
03411         wbMatrix<SPIDX_T> X; IDX.colPermute(*Pc,X);
03412         X.groupRecs(P,dg,-1,lex);
03413     }
03414
03415 #ifdef WB_CLK_SPARSE
03416     Wb::Clock clk("sparse:cmpr",0);
03417 #endif
03418
03419     SPIDX_T i,j,k,d; wperm_t *p=P.data;
03420     wbvector<TD> D0; D.save2(D0); D.init(dg.len);
03421     const TD *d0=D0.data;
03422
03423     for (j=k=0; j<dg.len; ++j,++k) {
03424         TD &x=D.data[k]; x=d0[p[0]];
03425         for (d=dg.data[j], i=1; i<d; ++i) { x+=d0[p[i]]; }
03426
03427         p+=d;
03428         if (x==0) { --k; } else if (k<j) { IDX.recSet(k,j); }
03429     }
03430
03431     REDSIZE_NNZ(k);
03432
03433     return (double)SkipTiny(eps);
03434 };
03435
03436 template <class TD>
03437 int wbsparray<TD>::splitSparseCM(
03438     const char *F, int L, wbvector< wbsparray<TD> > &X,
03439     TD eps,
03440     char ref,
03441     char tnorm
03442 ){
03443     iterSparseCMref<TD> ISP(F_L,*this);
03444     SPIDX_T i=0, n=ISP.numIter(); if (!n) { X.init(); return X.len; }
03445
03446     if (eps>0) {
03447         wbvector<char> mark(n); SPIDX_T l=0;
03448         for (; i<n; ++i) { if (ISP.norm2(i,tnorm)>=eps) { mark[i]=1; ++l; }}
03449         X.init(l); i=l=0;
03450         for (; i<n; ++i) { if (mark[i]) { ISP.getIter(i,X[l++],ref); }}
03451     }
03452     else {
03453         X.init(n);
03454         for (; i<n; ++i) { ISP.getIter(i,X[i],ref); }
03455     }
03456
03457     return X.len;
03458 };
03459
03460 template <class T>
03461 inline int checkContract(const char* F, int L,
03462     const wbvector<T> &SA, const ctrIdx &ica,
03463     const wbvector<T> &SB, const ctrIdx &icb,

```

```

03464     const wbpemr *pfinal, wvector<T> *SC
03465 ){
03466     int e=0;
03467
03468     if (ica.len!=icb.len) {
03469         if (F) wblog(F,L,
03470             "ERR %s() invalid index set (length mismatch, [%s], [%s])",
03471             FCT,STR(ica),STR(icb));
03472         return 1;
03473     }
03474     if (!ica.len) {
03475         if (F) wblog(F,L,"ERR contract() got empty ic[ab] !?");
03476         return 1;
03477     }
03478
03479     for (unsigned i=0; i<ica.len; ++i) {
03480         if ((ica[i]<SA.len ? SA[ica[i]] : 1) !=
03481             (icb[i]<SB.len ? SB[icb[i]] : 1))
03482             wblog(F,L,
03483                 "ERR invalid contraction @ i=%d/%d ... !?\n"
03484                 "having: %s (%s) * %s (%s)",i+1,ica.len,
03485                 SSTR(SA), STR(ica), SSTR(SB), STR(icb)
03486             );
03487     }
03488
03489     if (SC) {
03490         wvector<T> sC; sC.initI(SA,ica,SB,icb);
03491
03492         if (SC->isEmpty()) { sC.save2(*SC); }
03493         else {
03494             if (pfinal && pfinal->len) sC.Permute(*pfinal);
03495             if ((*SC)!=sC) { e=6; if (F) wblog(F,L,
03496                 "ERR %s() severe size mismatch (C: %s <> %s)",
03497                 FCT, SSTR(sC), SSTR(SC));
03498             }
03499         }
03500
03501         if (!SC->prod(0)) return -1;
03502     }
03503
03504     return e;
03505 };
03506
03507 template <class TD> inline
03508 char wbsparray<TD>::contract_scalar(
03509     const char* F, int L, const ctrIdx &ica,
03510     const wbsparray<TD> &B, const ctrIdx &icb, wbsparray<TD> &C,
03511     const wbpemr &pfinal, const TD& afac, const TD& cfac
03512 ) const {
03513
03514     char isa=isScalar(), isb=B.isScalar(), gotC=(cfac && !C.isEmpty());
03515     unsigned r=-1;
03516     if (!isa || !isb) return 0;
03517
03518     TD x=(D.len && B.D.len ? afac*D[0]*B.D[0] : TD(0));
03519     if (gotC && C.D.len) { x+=cfac*C.D[0]; }
03520
03521     if (ica.len!=icb.len) wblog(F,L,"ERR %s() length mismatch: "
03522         "%s <> %s",FCT, STR(ica), STR(icb));
03523     if (gotC && (!C.isScalar() || C.D.len>1)) wblog(F,L,
03524         "ERR %s() invalid scalar %s",FCT,C.info2Str("C").data);
03525     if (SIZE.len!=IDX.dim2 || B.SIZE.len!=B.IDX.dim2) wblog(F,L,
03526         "ERR %s() invalid scalars %s <> %s", FCT,
03527         info2Str("A").data, B.info2Str("B").data);
03528
03529     if (SIZE.len) {
03530         if (B.SIZE.len) {
03531             checkContract(F,L, SIZE,ica,B.SIZE,icb,
03532                 (gotC? &pfinal : NULL), (gotC? &C.SIZE : NULL));
03533             r=(SIZE.len + B.SIZE.len - 2*ica.len);
03534         }
03535         else {
03536             wvector<SPIDX_T> Sb(2); Sb[0]=Sb[1]=B.D.len;
03537             checkContract(F,L, SIZE,ica,Sb,icb,
03538                 (gotC? &pfinal : NULL), (gotC? &C.SIZE : NULL));
03539             r=(SIZE.len + 2 - 2*ica.len);
03540         }
03541     }
03542     else {
03543         if (B.SIZE.len) {
03544             wvector<SPIDX_T> Sa(2); Sa[0]=Sa[1]=D.len;
03545             checkContract(F,L,Sa,ica,B.SIZE,icb,
03546                 (gotC? &pfinal : NULL), (gotC? &C.SIZE : NULL));
03547             r=(2 + B.SIZE.len - 2*ica.len);
03548         }
03549     }
03550

```

```

03551     C.initScalar(x,r);
03552
03553     return 1;
03554 };
03555
03556 template <class TD>
03557 wbsparray<TD>& wbsparray<TD>::contract_diag_diag(
03558     const char* F, int L, const ctrIdx &ica,
03559     const wbsparray<TD> &B, const ctrIdx &icb, wbsparray<TD> &C,
03560     const wbpperm &pfinal, const TD& afac, const TD& cfac
03561 ) const {
03562
03563     SPIDX_T i=0, ra=2, rb=2; int e=0;
03564
03565     if (SIZE.len || B.SIZE.len) wblog(F_L,
03566         "ERR %s() (%s) <> (%s)",FCT,SSTR_(this),SSTR(B));
03567     if (D.len!=B.D.len) wblog(F_L,
03568         "ERR %s() size mismatch (%d/%d)",FCT,D.len,B.D.len);
03569
03570     if (!ica.len) e=-1; else
03571     if (ica.len!=icb.len || ica.len>2) e=-2; else {
03572         for (; i<ica.len; ++i) if (ica[i]>=ra || icb[i]>=rb) e=3;
03573     }
03574     if (!e && ica.len==2 && (ica[0]==ica[1] || icb[0]==icb[1])) e=4;
03575     if (e) wblog(F_L,
03576         "ERR %s() invalid contraction %s (%s) <> %s (%s)",
03577         FCT,SSTR_(this),STR(ica),SSTR(B),STR(icb)
03578     );
03579
03580     if (ISCOMPLX_(TD) && (ica.conj || icb.conj))
03581     wblog(F_L,"ERR %s() complex not yet implemented",FCT);
03582
03583     wvector<TD> x(D.len);
03584     if (afac) {
03585         for (i=0; i<D.len; ++i) { x[i]=D[i]*B.D[i]; }
03586     }
03587
03588     if (ica.len==2) { TD q=afac*x.sum();
03589         if (cfac) {
03590             if (!C.isScalar()) wblog(F_L,
03591                 "ERR %s() invalid scalar C (%s)",FCT,SSTR(C));
03592             C.D*=cfac; C[0]+=q;
03593         }
03594         else C.initScalar(q);
03595     }
03596     if (!pfinal.isEmpty()) wblog(F_L,
03597         "ERR %s() got pfinal=[%s] for scalar !?",FCT,STR(pfinal)
03598     );
03599 }
03600 else {
03601     if (cfac) {
03602         if (!C.isDiag() || C.D.len!=D.len) wblog(F_L,
03603             "ERR %s() invalid diag C (%s; %d) @ cfac=%.3g",
03604             FCT,SSTR(C),D.len,double(cfac));
03605         C.D*=cfac; C.D.Plus(x,afac);
03606     }
03607     else {
03608         if (afac) x*=afac;
03609         C.initDiag(D.len); x.save2(C.D);
03610     }
03611
03612     if (!pfinal.isEmpty() && !pfinal.isValidPerm(0,0,2))
03613     wblog(F_L,"ERR %s() got pfinal=[%s] !?",FCT,STR(pfinal));
03614 }
03615
03616     return C;
03617 };
03618
03619 template <class TD>
03620 char wbsparray<TD>::contract_check_2full(
03621     const char* F, int L, const ctrIdx &ica,
03622     const wbsparray<TD> &B, const ctrIdx &icb, wbsparray<TD> &C,
03623     const wbpperm &pfinal, const TD& afac, const TD& cfac,
03624     char flag, TD eps, SPIDX_T *nnz_C
03625 ) const {
03626
03627     if (flag=='s' || flag=='S'
03628         || typeid(TD)!=typeid(double))
03629     return 0;
03630
03631     if (flag && flag!='f' && flag!='F')
03632     wblog(F_L,"ERR %s() invalid flag=%c<%d>",FCT,flag,flag);
03633
03634     if (!flag || nnz_C!=NULL) {
03635         SPIDX_T NA=numel(), NB=B.numel(), M=SIZE.prod(ica.data,ica.len);
03636         SPIDX_T NC=(NA/M)*(NB/M);
03637         double pa=nnz()/double(NA), pb=B.nnz()/double(NB);

```

```

03638
03639     double pc=M*pa*pb; if (pc>1) pc=1;
03640
03641     if (!NA || !NB || !M) wblog(F_L,
03642         "ERR %s() got\n%s [%s] <> %s [%s] (%d)",FCT,SSTR_(this),
03643         ica.toStr().data, SSTR(B), STR(icb), M);
03644
03645     if ((pc>0.50 && NC<(1<<24)) ||
03646         (pc>0.20 && NC<(1<<16)) ||
03647         (pc>0.10 && NC<(1<<12))
03648     ) { flag='F';
03649         wvector<SPIDX_T> sC; sC.initI(SIZE,ica,B.SIZE,icb);
03650     }
03651
03652     if (nnz_C) {
03653         double nc=pc*NC;
03654         if (NC>128) (nc)*=1.20; else
03655         if (NC> 32) (nc)*=1.50; else
03656         if (NC>  4) (nc)*=2;   else nc=NC;
03657         (*nnz_C)=MIN(SPIDX_T(nc),NC);
03658     }
03659
03660 }
03661
03662 if (!flag) return 0;
03663
03664 #ifdef WB_CLK_SPARSE
03665     Wb::Clock clk("sparse:ctr-full",0);
03666 #endif
03667
03668     wbararray<TD> Af,Bf,ABf; this->toFull(Af); B.toFull(Bf);
03669
03670     Af.contract(F_L,ica,Bf,icb,ABf,pfinal,afac);
03671
03672     if (cfac!=0 && C.D.len) { C*=cfac;
03673         wbararray<TD> Cf; C.toFull(Cf);
03674         ABf+=Cf;
03675     }
03676
03677     C.init(F_L,ABf,eps); return 1;
03678 };
03679
03680 template <class TD>
03681 wbsparray<TD>& wbsparray<TD>::contract(
03682     const char* F, int L, const ctrIdx &ica,
03683     const wbsparray<TD> &B, const ctrIdx &icb, wbsparray<TD> &C,
03684     const wbpem &pfinal, const TD& afac, const TD& cfac,
03685     char sflag,
03686     TD eps
03687 ) const {
03688
03689     static int nvlog=0;
03690
03691     #ifdef WB_CLK_SPARSE
03692         Wb::Clock clk("sparse:ctr:all",0);
03693     #endif
03694
03695     if (D.len<=1 && B.D.len<=1) {
03696         if (contract_scalar(F_L,ica,B,icb,C,pfinal,afac,cfac)) {
03697             return C;
03698         }
03699
03700         if (!sflag && typeid(TD)!=typeid(double)) sflag='s';
03701
03702         if (this==&C || &B==&C) { wbsparray<TD> X(C);
03703             contract(F,L,ica,B,icb,X,pfinal,afac,cfac,sflag,eps);
03704             return X.save2(C);
03705         }
03706
03707         char iA=isDiag(FL), iB=B.isDiag(FL);
03708         if (iA || iB) {
03709             if (iA && iB) {
03710                 if ((ica.len && ica.len<=2) && (icb.len && icb.len<=2))
03711                     return contract_diag_diag(F_L,ica,B,icb,C,pfinal,afac,cfac);
03712                 else {
03713                     if (ica.len || icb.len) wblog(FL,
03714                         "ERR %s() got %d/%d !?",FCT,ica.len,icb.len);
03715                     return tensorProdX(FL,B,C,ica.conj,afac,icb.conj,cfac,&pfinal);
03716                 }
03717             }
03718             else if (iA) {
03719                 wbsparray<TD> A2; diag2reg(FL,A2);
03720                 return A2.contract(F,L,ica,B,icb,C,pfinal,afac,cfac,sflag,eps);
03721             }
03722             else if (iB) {
03723                 wbsparray<TD> B2; B.diag2reg(FL,B2);
03724                 return contract(F,L,ica,B2,icb,C,pfinal,afac,cfac,sflag,eps);

```

```

03725     }
03726 }
03727
03728 wvector<SPIDX_T> sC; if (cfac) sC=C.SIZE; int e=0;
03729
03730 if ((e=checkContract(F_L,SIZE,ica,B.SIZE,icb, &pfinal, &sC))) {
03731     if (e<0) {
03732         if (sC.len) {
03733             C.init(sC);
03734             return C;
03735         }
03736     }
03737     else wblog(FL,"ERR %s()",FCT);
03738 }
03739
03740 if (!D.len || !B.D.len) {
03741     if (cfac) { C.D*=cfac; } else { C.init(sC); }
03742     if (e<0) {
03743         if (C.SIZE.len || C.IDX.dim2) wblog(FL,"ERR %s() !?",FCT);
03744         if (!C.D.len) { C.D.init(1); C.D[0]=0; }
03745     }
03746     return C;
03747 }
03748
03749 if (contract_check_2full(F_L,
03750     ica,B,icb,C,pfinal,afac,cfac,sflag,eps)) { return C; }
03751
03752 wbsparray<TD> C0;
03753
03754 if (cfac!=0) { C.D*=cfac; }
03755 if (afac==0) {
03756     if (!cfac) C.init(sC.Permute(pfinal));
03757     return C;
03758 }
03759 if (cfac!=0) C.save2(C0);
03760
03761 if (B.IDX.dim2==1) {
03762     if (B.SIZE.len!=1 || ica.len!=1) wblog(FL,
03763         "ERR %s() got S=[%s] !?",FCT,SSTR(B));
03764     wvector<TD> b; B.toFull(b);
03765     contract(FL,ica[0],b,C);
03766
03767     C.Permute(pfinal); if (afac!=1) C*=afac;
03768     if (!C0.isEmpty()) { C+=C0; }
03769
03770     return C;
03771 }
03772
03773 #ifdef WB_CLK_SPARSE
03774 Wb::Clock cl2("sparse:ctr:2",0);
03775 #endif
03776
03777 sparseIndex2D<SPIDX_T> a2, b2;
03778
03779 toIndex2D(ica,a2,2,'r');
03780 B.toIndex2D(icb,b2,1);
03781
03782 if (!a2.IJ.dim1 || a2.IJ.dim2!=2 || !a2.cIdx.len ||
03783     !b2.IJ.dim1 || b2.IJ.dim2!=2 || !b2.cIdx.len)
03784     wblog(FL,"ERR %s() unexpected IJ data (%s [%ld]; %s [%ld])",FCT,
03785         SSTR(a2.IJ), a2.cIdx.len,
03786         SSTR(b2.IJ), b2.cIdx.len
03787     );
03788
03789 if (b2.rmaj) wblog(FL,
03790     "ERR %s() got unexpected sparse dimensions\n B: %s @ [%s]",
03791     FCT,SSTR(B), STR(icb)
03792 );
03793
03794 int np=1; if (sp_num_threads>1) {
03795     np=(IDX.dim1+B.IDX.dim1)/1024;
03796     if (np>sp_num_threads) { np=sp_num_threads; }
03797     if (unsigned(np)>b2.S[1]) { np=b2.S[1]; }
03798     if (np<1) { np=1; }
03799 }
03800
03801 #ifdef LD_CLEBSCH_QS
03802 if (np<sp_num_threads && CG_VERBOSE>5) {
03803     static time_t tlast=0;
03804     time_t tnow=time(NULL);
03805     if (tnow>tlast+60) { size_t sl=(1<<26);
03806         if (numel()>sl || B.numel()>sl || sC.prod(0)>sl) {
03807             wblog(FL,"CTR %12s * %12s @ %d/%d threads",
03808                 SSTR(a2), SSTR(b2), np, sp_num_threads
03809             );
03810             tlast=tnow;
03811         }
03812     }

```

```

03812     }
03813 }
03814 #endif
03815
03816 if (a2.rmaj) {
03817     size_t s1=a2.S[0], s2=b2.S[1], la=a2.IJ.dim1-1;
03818     const size_t nk=b2.cIdx.len-1;
03819     const SPIDX_T *pa=a2.P.data, *pb=b2.P.data;
03820
03821     if (s1*s2>(1<<20)) wblog(FL,
03822         "ERR %s() got a.rmaj=%d having %dx%d * %dx%d !?!",FCT,
03823         a2.S[0], a2.S[1], b2.S[0], b2.S[1]
03824     );
03825
03826     wbararray<TD> Cx(s1,s2);
03827     Wb::LogException ex;
03828
03829 #pragma omp parallel for num_threads(np)
03830     for (unsigned k=0; k<nk; ++k) { if (!ex) { try {
03831         size_t ib=b2.cidx(k), lb=b2.cidx(k+1);
03832
03833         if (lb>ib) { --lb;
03834             size_t ia=0, ib_=ib;
03835             SPIDX_T *Ia=a2.IJ.data, *Ib=b2.IJ.data+2*ib;
03836             char c;
03837
03838             while (ib<=lb) {
03839                 c=NUMCMP (Ia[1],Ib[0]);
03840
03841                 if (c==0) {
03842                     if (ia<la) {
03843                         if (Ia[0]!=Ia[2]) c=-2;
03844                     }
03845                     else {
03846                         c+=3;
03847                     }
03848
03849                     if (ib<lb) {
03850                         if (!c && Ib[1]!=Ib[3]) c+=2;
03851                     }
03852
03853                     if (Ia[0]>=s1 || Ib[1]>=s2) wblog(FL,
03854                         "ERR %s() index out of bounds (%ld/%ld; %ld/%ld)",
03855                         FCT, Ia[0], s1, Ib[1], s2);
03856
03857                     Cx(Ia[0],Ib[1]) += D[pa[ia]] * B.D[pb[ib]];
03858                 }
03859
03860                 if (ia<la && (c<0 || ib==lb)) {
03861                     if (Ia[0]==Ia[2]) { if (Ia[1]>=Ia[3]) wblog(FL,
03862                         "ERR %s() a.IJ not sorted or unique !?",FCT); }
03863                     else {
03864                         if (Ia[0]>Ia[2]) wblog(FL,
03865                             "ERR %s() a.IJ not sorted !?",FCT);
03866                         Ib=-2*(ib-ib_); ib=ib_;
03867                     }
03868                     ++ia; Ia+=2;
03869                 }
03870                 else if (c) {
03871                     if (ib<lb) {
03872                         if (Ib[1]==Ib[3]) { if (Ib[0]>=Ib[2]) wblog(FL,
03873                             "ERR %s() b.IJ not sorted or unique !?",FCT); }
03874                         else {
03875                             if (Ib[1]>Ib[3]) wblog(FL,
03876                                 "ERR %s() b.IJ not sorted !?",FCT);
03877                             ib_=ib+1; Ia=-2*ia; ia=0;
03878                         }
03879                     }
03880                     ++ib; Ib+=2;
03881                 }
03882                 else {
03883                     if (ia<la && Ia[1]>=Ia[3])
03884                         wblog(FL,"ERR %s() a.IJ not sorted !?",FCT);
03885                     if (ib<lb && Ib[0]>=Ib[2])
03886                         wblog(FL,"ERR %s() b.IJ not sorted !?",FCT);
03887
03888                     ++ia; Ia+=2;
03889                     ++ib; Ib+=2;
03890                 }
03891             }}}
03892             catch (Wb::LogException &e_) { ex+=e_; }
03893             catch (...) { ++ex; }
03894         }
03895     ex.report (FLF);
03896
03897     C.init (FL,Cx).Reshape (sC);
03898     if (!C.SIZE.len && !C.D.len) {

```



```

03899         C.D.init(1); C.D[0]=0;
03900     }
03901
03902     C.Permute(pfinal); if (afac!=1) C*=afac;
03903     if (!C0.isEmpty()) { C+=C0; }
03904
03905     return C;
03906 }
03907
03908 wbMatrix<SPIDX_T> MARK(np,a2.S[0]);
03909 groupIndex<widxt_t> kc; { kc.init(b2.S[1]); }
03910
03911 const SPIDX_T *pa=a2.P.data, *pb=b2.P.data;
03912 SPIDX_T k=0, ltot=0, nnzc=0;
03913
03914 #pragma omp parallel for num_threads(np) reduction(+:ltot)
03915 for (k=0; k<b2.S[1]; ++k) {
03916     int id=omp_get_thread_num();
03917     widxt_t *mark=MARK.ref(id,0);
03918     SPIDX_T i, i_,i1,i2, j, j_, j1=b2.cidx(k), j2=b2.cidx(k+1), nnzc=0;
03919     for (j_=j1; j_<j2; ++j_) { j=b2.ridx(j_);
03920         i1=a2.cidx(j); i2=a2.cidx(j+1);
03921         for (i_=i1; i_<i2; ++i_) { i=a2.ridx(i_);
03922             if (mark[i]<k+1) { mark[i]=k+1; ++nnzc; }
03923         }
03924     }
03925     kc.setD(k,nnzc);
03926     ltot+=nnzc;
03927 }
03928
03929 nnzc=kc.init_ic();
03930 if (nnzc!=ltot) wblog(FL,"ERR %s() "
03931     "nnz inconsistency (%ld/%ld) !?",FCT,ltot,nnzc);
03932
03933 #ifdef WB_CLK_SPARSE
03934     cl2.Switch("sparse:ctr:3");
03935 #endif
03936
03937 if (!nnzc) {
03938     if (C0.isEmpty()) C.init(sC); else C0.save2(C);
03939     return C;
03940 }
03941
03942 wbMatrix<TD> W(np,a2.S[0]);
03943 wbMatrix<SPIDX_T> IC_(np,a2.S[0]), IC(np,a2.S[0]);
03944 MARK.set(0);
03945
03946 ctrIdx ika, ikb;
03947 int largeD=0;
03948
03949 ica.invert( SIZE.len,ika);
03950 icb.invert( B.SIZE.len,ikb);
03951
03952 { size_t DX=(1<<26);
03953
03954     if (D.len>DX || B.D.len>DX || nnzc>DX) { largeD=2; nvlog=1; }
03955     else {
03956         DX=(1<<20);
03957         if (D.len>DX || B.D.len>DX || nnzc>DX) { largeD=1;
03958             if ((++nvlog)%24==1) largeD|=2;
03959         }
03960     }
03961
03962     if (largeD&2) {
03963         wblog(FL,"TST %s() large sparse arrays (%d/%d threads)\n"
03964             " A: %-20s (%9.3g) @ %s\n B: %-20s (%9.3g) @ %s\n"
03965             "> C: %-20s (%9.3g)",FCT, np, sp_num_threads,
03966             SSTR_(this), D.len/double( numel()), STR(ica),
03967             SSTR(B), B.D.len/double(B.numel()), STR(icb),
03968             STR(sC), nnzc/double(sC.prod(1)));
03969     }
03970     else if (largeD) {
03971         wblog(FL,"CTR %s_%s * %s_%s = %s (np=%d/%d)", SSTR_(this),
03972             STR(ica), SSTR(B), STR(icb), SSTR(sC), np, sp_num_threads);
03973     }
03974     if (largeD) { Wb::MemStat(FL); doflush(); }
03975 }
03976
03977 C.init(sC,nnzc);
03978 ltot=0;
03979
03980 #pragma omp parallel for num_threads(np) reduction(+:ltot)
03981 for (k=0; k<b2.S[1]; ++k) {
03982     int id=omp_get_thread_num();
03983     wbvector<SPIDX_T> ic(IC.dim2,IC.ref(id,0),'r');
03984     SPIDX_T *ic_=IC_.ref(id,0), is=0;
03985     widxt_t *mark=MARK.ref(id,0);

```

```

03986     TD *w=W.ref(id,0);
03987
03988     SPIDX_T i, i_, i1, i2, j, j_, j1=b2.cidx(k), j2=b2.cidx(k+1);
03989     bool sortflag=((j2-j1)<SPIDX_T(ceil(0.2*MARK.dim2)));
03990
03991     for (j_=j1; j_<j2; ++j_) { j=b2.ridx(j_);
03992         const TD &Bjk=B.D[pb[j_]];
03993         i1=a2.cidx(j); i2=a2.cidx(j+1);
03994         for (i_=i1; i_<i2; ++i_) { i=a2.ridx(i_);
03995             if (mark[i]<k+1) { mark[i]=k+1; ic_[i]=pa[i_];
03996                 if (sortflag) ic[is++]=i;
03997                 w[i] = D[pa[i_]] * Bjk;
03998             }
03999             else {
04000                 w[i]+=(D[pa[i_]] * Bjk);
04001             }
04002         }
04003     }
04004
04005     SPIDX_T l=kc.getGroup(k);
04006
04007     if (sortflag) {
04008         if (is) {
04009             ic.len=is; ic.Sort();
04010             for (i_=0; i_<ic.len; ++i_) { i=ic[i_];
04011                 C.IDX.recSetP(l, IDX.rec(ic_[i]), ika, B.IDX.rec(pb[j1]), ikb);
04012                 C.D[l++]=w[i];
04013             }
04014             ic.len=MARK.dim2;
04015         }
04016         ltot+=is;
04017     }
04018     else {
04019         for (i=0; i<MARK.dim2; ++i) {
04020             if (mark[i]==k+1) {
04021                 C.IDX.recSetP(l, IDX.rec(ic_[i]), ika, B.IDX.rec(pb[j1]), ikb);
04022                 C.D[l++]=w[i]; ++ltot;
04023             }
04024         }
04025     }
04026 }
04027
04028 if (ltot!=C.D.len) wblog(FL,
04029     "ERR %s() %d/%d/%d !?", FCT, ltot, C.D.len, nnzc);
04030
04031 if (largeD) {
04032     size_t s=C.memSize();
04033     if (s>(1<<30)) wblog(FL,
04034         "TST %s() size(C) = %s", FCT, Wb::size2Str(s).data);
04035     Wb::MemStat(FL);
04036 }
04037 #ifdef WB_CLK_SPARSE
04038     cl2.stop();
04039 #endif
04040
04041 C.Permute(pfinal); if (afac!=1) C*=afac;
04042 if (!C0.isEmpty()) { C+=C0; }
04043
04044 return C;
04045 };
04046
04047 template <class TD>
04048 wbspararray<TD>& wbspararray<TD>::contract(
04049     const char *F, int L,
04050     unsigned k, const wbvector<TD> &B, wbspararray<TD> &C
04051 ) const {
04052
04053     if (k>=SIZE.len) {
04054         if (B.len!=1) wblog(F_L, "ERR %s() "
04055             "size mismatch (%s ; %d @ %d)", FCT, SSTR_(this), B.len, k+1);
04056         C=*this; C*=B.data[0];
04057         if (Wb::abs(B.data[0])<TD(1E-8)) C.Compress();
04058         return C;
04059     }
04060
04061     if (SIZE[k]!=B.len) wblog(F_L,
04062         "ERR %s() size mismatch (%s ; %d @ %d)", FCT, SSTR_(this), B.len, k+1);
04063     if (IDX.dim1!=D.len || IDX.dim2!=SIZE.len) wblog(FL,
04064         "ERR %s() size inconsistency (%dx%d <> %dx%d) !?",
04065         FCT, IDX.dim1, IDX.dim2, D.len, SIZE.len);
04066
04067     if (IDX.dim2>1) {
04068         const SPIDX_T *idx=IDX.data+k;
04069         const TD *b=B.data; TD *c; wbindindex Ik;
04070
04071         Ik.Index_ex(IDX.dim2, k); C.init();
04072         IDX.getCols(Ik, C.IDX); C.D=D; c=C.D.data;

```

```

04073     SIZE.select(Ik,C.SIZE);
04074
04075     for (SPIDX_T n=IDX.dim2, i=0; i<D.len; ++i, idx+=n) {
04076         if ((*idx)>=B.len) wblog(F_L,
04077             "ERR %s() index out of bounds (%d/%d)",FCT,(*idx)+1,B.len);
04078         c[i]*=b[*idx];
04079     }
04080
04081     C.Compress();
04082 }
04083 else if (IDX.dim2==1) {
04084     if (int(k)>=0 && k) wblog(F_L,
04085         "ERR %s() ctrIdx out of bounds (%d/1)",FCT,k+1);
04086     C.initScalar();
04087
04088     const SPIDX_T *idx=IDX.data;
04089     const TD *a=D.data, *b=B.data;
04090     TD &c=C.D[0]; c=0;
04091
04092     for (SPIDX_T i=0; i<D.len; ++i) {
04093         if (idx[i]>=B.len) wblog(F_L,
04094             "ERR %s() index out of bounds (%d/%d)",FCT,idx[i]+1,B.len);
04095         c+=a[i]*b[idx[i]];
04096     }
04097 }
04098 else {
04099     info(F_L); wblog(F_L,"ERR %s()",FCT);
04100 }
04101
04102 return C;
04103 };
04104
04105 template <class TD>
04106 wbararray<TD>& wbsparray<TD>::contract(
04107     const char* F, int L, const ctrIdx &ica,
04108     const wbsparray<TD> &B, const ctrIdx &icb, wbararray<TD> &C,
04109     const wbpperm &pfinal, const TD& afac, const TD& cfac
04110 ) const {
04111
04112     wbararray<TD> Af,Bf,X;
04113
04114     if (cfac && !C.isEmpty()) { C.save2(X); X*=cfac; }
04115
04116     this->toFull(Af); B.toFull(Bf);
04117     Af.contract(F_L,ica,Bf,icb,C,pfinal,afac);
04118
04119     if (!X.isEmpty()) {
04120         if (!X.sameSize(C)) wblog(F_L,
04121             "ERR %s() severe size mismatch (%s <> %s; %g)",
04122             FCT, SSTR(C), SSTR(X), cfac);
04123         C+=X;
04124     }
04125     return C;
04126 };
04127
04128 template <class TD>
04129 wbararray<TD>& contract(const char* F, int L,
04130     const wbararray<TD> &A, const wbindex &ic1,
04131     const wbsparray<TD> &B, const wbindex &ic2, wbararray<TD> &C,
04132     const wbpperm &pfinal, const TD& afac, const TD& cfac
04133 ){
04134     unsigned m=ic1.len, r1=A.rank(), r2=B.rank(F_L); int e=0;
04135     wbararray<TD> X;
04136
04137     if ((void*)&A==(void*)&C) wblog(F,L,
04138         "ERR sparse::%s() must have distinct target space",FCT);
04139
04140     wvector<SPIDX_T> sC; if (cfac) sC=C.SIZE;
04141     if ((e=checkContract(F_L,A.SIZE,ic1,B.SIZE,ic2, &pfinal, &sC)) {
04142         if (e<0) { C.init(sC); if (sC.len) return C; }
04143         else wblog(F_L,"ERR %s()",FCT);
04144     }
04145
04146     if (afac==0 || !B.D.len) {
04147         if (cfac!=0) { C*=cfac; } else { C.init(sC.Permute(pfinal)); }
04148         return C;
04149     }
04150
04151     if (cfac!=0) { C*=cfac; C.save2(X); }
04152     C.init(sC);
04153
04154     wvector<SPIDX_T> s1,D2;
04155     wbMatrix<SPIDX_T> I2;
04156     wbpperm Pa,P2;
04157
04158     A.SIZE.getI(ic1,s1); Pa.init2End(ic1,r1);
04159     B.IDX.cols2End(ic2,I2).groupRecs(P2,D2,r2-m);

```

```

04160
04161     wbindex Ia(r1),Ic(sC.len);
04162     SPIDX_T i, ig,d,k,l, k1=r1-ic1.len, k2=r2-ic2.len;
04163     SPIDX_T *i2, *ia=Ia.data, *ic=Ic.data, *pa=Pa.data;
04164     wbindex K1(s1);
04165
04166     for (l=ig=0; ig<D2.len; ++ig) { d=D2[ig];
04167     for (k=0; k<d; ++k,++l) {
04168         TD b=B.D.data[P2[l]]; if (b==0) continue; if (afac!=1) b*=afac;
04169
04170         i2=I2.ref(l); if (k==0) {
04171             for (i=0; i<k2; ++i) ic[k1+i]=i2[i]; }
04172             for (i=0; i<m; ++i) ia[pa[k1+i]]=i2[k2+i];
04173
04174             for (K1.reset(); ++K1;) {
04175                 for (i=0; i<k1; ++i) { ia[pa[i]]=ic[i]=K1.data[i]; }
04176                 if (b== 1) { C(Ic)+=A(Ia); } else
04177                 if (b==-1) { C(Ic)-=A(Ia); } else { C(Ic)+=b*A(Ia); }
04178             }
04179         }}
04180
04181     C.Permute(pfinal); if (!X.isEmpty()) C+=X;
04182     return C;
04183 };
04184
04185 template <class TD>
04186 wbararray<TD>& contract(const char* F, int L,
04187     const wbspararray<TD> &A, const wbindex &ic1,
04188     const wbararray<TD> &B, const wbindex &ic2, wbararray<TD> &C,
04189     const wbperm &pfinal, const TD& afac, const TD& cfac
04190 ){
04191     unsigned m=ic1.len, r1=A.rank(FL), r2=B.rank(); int e=0;
04192     wbararray<TD> X;
04193
04194     if ((void*)&B==(void*)&C) wblog(F,L,
04195         "ERR sparse::%s() must have distinct target space",FCT);
04196
04197     wvector<SPIDX_T> sC; if (cfac) sC=C.SIZE;
04198     if ((e=checkContract(F_L,A.SIZE,ic1,B.SIZE,ic2, &pfinal, &sC))) {
04199         if (e<0) { C.init(sC); if (sC.len) return C; }
04200         else wblog(FL,"ERR %s()",FCT);
04201     }
04202
04203     if (afac==0 || !A.D.len) {
04204         if (cfac!=0) { C*=cfac; } else { C.init(sC.Permute(pfinal)); }
04205         return C;
04206     }
04207
04208     if (cfac!=0) { C*=cfac; C.save2(X); }
04209     C.init(sC);
04210
04211     wvector<SPIDX_T> s2,D1;
04212     wbMatrix<SPIDX_T> I1;
04213     wbperm Pb,P1;
04214
04215     A.IDX.cols2End(ic1,I1).groupRecs(P1,D1,r1-m);
04216     B.SIZE.getI(ic2,s2); Pb.init2End(ic2,r2);
04217
04218     wbindex Ib(r2),Ic(sC.len);
04219     SPIDX_T i, ig,d,k,l, k1=r1-ic1.len, k2=r2-ic2.len, *i1,
04220     *ib=Ib.data, *ic=Ic.data, *pb=Pb.data;
04221     wbindex K2(s2);
04222
04223     for (l=ig=0; ig<D1.len; ++ig) { d=D1[ig];
04224     for (k=0; k<d; ++k,++l) {
04225         TD a=A.D.data[P1[l]]; if (a==0) continue; if (afac!=1) a*=afac;
04226
04227         i1=I1.ref(l); if (k==0) {
04228             for (i=0; i<k1; ++i) ic[i]=i1[i]; }
04229             for (i=0; i<m; ++i) ib[pb[k2+i]]=i1[k1+i];
04230
04231             for (K2.reset(); ++K2;) {
04232                 for (i=0; i<k2; ++i) { ib[pb[i]]=ic[k1+i]=K2.data[i]; }
04233                 if (a== 1) { C(Ic)+=B(Ib); } else
04234                 if (a==-1) { C(Ic)-=B(Ib); } else { C(Ic)+=a*B(Ib); }
04235             }
04236         }}
04237
04238     C.Permute(pfinal); if (!X.isEmpty()) C+=X;
04239     return C;
04240 };
04241
04242 template <class TD>
04243 TD Wb::VMatVprod(
04244     const wbspararray<TD> &v1, const wbspararray<TD> &M, const wbspararray<TD> &v2
04245 ){
04246     wbspararray<TD> x,X; wbindex i1(1), i2(1);

```

```

04247     i1[0]=0; i2[0]=1;
04248
04249     M.contract (FL,i2,v2,i1,X);
04250     v1.contract (FL,i1,X,i1,x);
04251
04252     x.checkSize (FL);
04253     if (x.D.len && !x.isScalar()) {
04254         MXPut (FL,"q").add (M,"M").add (v1,"v1").add (v2,"v2")
04255             .add (X,"X").add (x,"x");
04256         wblog (FL,"ERR %s() got %s",FCT,SSTR(x));
04257     }
04258
04259     return (x.D.len ? x[0] : 0.);
04260 };
04261
04262 template <class TD>
04263 wbsparray<TD>& Wb::MatProd(
04264     const wbsparray<TD> &A, const wbsparray<TD> &B, wbsparray<TD> &C,
04265     char aflag0, char bflag0, TD afac, TD cfac
04266 ){
04267     if (&C==&A || &C==&B) {
04268         wbsparray<TD> X; if (cfac) X=C;
04269         Wb::MatProd (A,B,X,aflag0,bflag0,afac,cfac);
04270         return X.save2 (C);
04271     }
04272
04273     int ra=A.rank(), rb=B.rank(), e=0;
04274     if (ra!=2) { if (ra || !A.isEmpty()) e+=1; }
04275     if (rb!=2) { if (rb || !B.isEmpty()) e+=2; }
04276
04277     if (e) wblog (FL,
04278         "ERR %s() expects matrices (%s; %s)",FCT,SSTR(A),SSTR(B));
04279     if (!ra || !rb) {
04280         if (!C.isEmpty() && cfac!=1) C*=cfac;
04281         return C;
04282     }
04283
04284     opFlags<TD> aflag(aflag0), bflag(bflag0);
04285     const wbsparray<TD>
04286         &AX = aflag.applyConjOrRef (FL,A),
04287         &BX = bflag.applyConjOrRef (FL,B);
04288     ctrIdx ia(1),ib(1);
04289
04290     ia[0]=(aflag.trans() ? 0:1);
04291     ib[0]=(bflag.trans() ? 1:0);
04292
04293     AX.contract (FL,ia,BX,ib,C, wbpem(), afac, cfac);
04294
04295     if (&AX!=&A) { delete &AX; }
04296     if (&BX!=&B) { delete &BX; }; return C;
04297 };
04298
04299 template <class TD>
04300 wbsparray<TD>& wbsparray<TD>::comm(const char *F, int L,
04301     const wbsparray<TD> &B, wbsparray<TD> &C, char aflag, char bflag
04302 ) const {
04303
04304     if (&C==this || &C==&B) {
04305         wbsparray<TD> X; comm (F,L,B,X,aflag,bflag);
04306         return X.save2 (C);
04307     }
04308
04309     SPIDX_T d=-1;
04310     if (!isSMatrix (FL,&d) || !B.isSMatrix (FL,&d)) wblog (F_L,
04311         "ERR %s() invalid operators for [A,B] (%s; %s)", FCT,
04312         SSTR_(this), SSTR (B)
04313     );
04314
04315     Wb::MatProd (*this,B,C,aflag,bflag);
04316     Wb::MatProd (B,*this,C,bflag,aflag,TD(-1.),TD(1.));
04317
04318     return C;
04319 };
04320
04321 #endif
04322

```

5.58 wbsparray.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : wbsparray (sparse array of arbitrary dimension)
00004  *

```

```

00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_SPARRAY_HH__
00021 #define __WB_SPARRAY_HH__
00022
00023 // #define WB_CLK_SPARSE
00024
00025     int sp_num_threads = 0;
00026
00027 bool mxIsWbsparray(
00028     const char *F, int L, const mxArray *a, unsigned k=0);
00029
00030 bool mxIsWbsparray(
00031     const char *F, int L, const mxArray *a, unsigned k,
00032     const mxArray **as, const mxArray **ad, const mxArray **ai
00033 );
00034
00035 bool mxIsWbsparray(const mxArray *a) { return mxIsWbsparray(0,0,a); }
00036
00037 #define wbSparray    wbsparray<double>
00038 #define wbSparrayTD  wbsparray<TD>
00039
00040 template<class TD>
00041 class wbsparray {
00042
00043     public:
00044
00045     wbsparray() : isref(0) {};
00046
00047     wbsparray(SPIDX_T d1) : isref(0) { if (d1) init(d1); };
00048     wbsparray(SPIDX_T d1, SPIDX_T d2) : isref(0) { init(d1,d2); };
00049     wbsparray(SPIDX_T d1, SPIDX_T d2, SPIDX_T d3) : isref(0) { init(d1,d2,d3); };
00050     wbsparray(SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T d4)
00051         : isref(0) { init(d1,d2,d3,d4); };
00052
00053     wbsparray(const wbvector<SPIDX_T> &d, SPIDX_T l=0)
00054         : isref(0) { init(d,l); };
00055
00056     wbsparray(char *sidx, SPIDX_T l=0)
00057         : isref(0) { init(Str2Idx(sidx),l); };
00058
00059     wbsparray(const wbsparray &a)
00060         : isref(0) { SIZE=a.SIZE; IDX=a.IDX; D=a.D; };
00061
00062     wbsparray(const char *F, int L, const mxArray *a)
00063         : isref(0) { init(F_L,a); };
00064
00065     wbsparray(const mxArray *a) : isref(0) { init(FL,a); };
00066
00067     wbsparray& init(
00068         const char *F, int L, const wbarray<TD> &A, TD eps=0
00069     );
00070
00071     template<class TA>
00072     wbarray<TA>& toFull(wbarray<TA> &A) const;
00073
00074     template<class TA>
00075     wbvector<TA>& toFull(wbvector<TA> &A) const;
00076
00077     template<class DB>
00078     wbsparray& init(const wbsparray<DB> &a) {
00079         if ((void*)this!=(void*)&a) {
00080             SIZE=a.SIZE; IDX=a.IDX; D=a.D;
00081             isref=0;
00082         }
00083         return *this;
00084     };
00085
00086     SPIDX_T catRecs(SPIDX_T i0, const wbsparray &a, SPIDX_T id);
00087
00088     wbsparray& init(const wbvector<SPIDX_T> &S0, SPIDX_T l=0) {
00089         SIZE=S0; D.init(l);
00090         if (S0.len) IDX.init(l,S0.len);
00091         else {

```

```

00092         if (l>1) wblog(FL,
00093             "WRN %s() use initDiag() with S=[] and nz=%d",FCT,l);
00094         IDX.init();
00095     }
00096     return *this;
00097 };
00098
00099 template<class I2>
00100 wbsparray& init(const wbvector<I2> &S_, SPIDX_T l=0) {
00101     wbvector<SPIDX_T> S0(S_);
00102     return init(S0,l);
00103 };
00104
00105 wbsparray& init() {
00106     SIZE.init(); IDX.init(); D.init(); isref=0;
00107     return *this;
00108 };
00109
00110 wbsparray& init(SPIDX_T d1) {
00111     if (d1) SIZE.init(1,&d1); else SIZE.init();
00112     IDX.init(); D.init(); isref=0;
00113     return *this;
00114 };
00115
00116 wbsparray& init(SPIDX_T d1, SPIDX_T d2) {
00117     SPIDX_T s[] = {d1,d2}; SIZE.init(2,s); IDX.init(); D.init();
00118     return *this;
00119 };
00120
00121 wbsparray& init(SPIDX_T d1, SPIDX_T d2, SPIDX_T d3) {
00122     SPIDX_T s[] = {d1,d2,d3}; SIZE.init(3,s); IDX.init(); D.init();
00123     return *this;
00124 };
00125
00126 wbsparray& init(SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T d4) {
00127     SPIDX_T s[] = {d1,d2,d3,d4}; SIZE.init(4,s); IDX.init(); D.init();
00128     return *this;
00129 };
00130
00131 wbsparray& initz() {
00132     IDX.init(0,SIZE.len); D.init();
00133     return *this;
00134 };
00135
00136 wbsparray& initz(SPIDX_T d1, SPIDX_T n) {
00137     SIZE.init(1,&d1); IDX.init(n,SIZE.len); D.init(n);
00138     return *this;
00139 };
00140
00141 wbsparray& initz(SPIDX_T d1, SPIDX_T d2, SPIDX_T n) {
00142     SPIDX_T s[] = {d1,d2}; SIZE.init(2,s); IDX.init(n,SIZE.len); D.init(n);
00143     return *this;
00144 };
00145
00146 wbsparray& initz(SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T n) {
00147     SPIDX_T s[] = {d1,d2,d3}; SIZE.init(3,s); IDX.init(n,SIZE.len); D.init(n);
00148     return *this;
00149 };
00150
00151 wbsparray& init_rzd(SPIDX_T r, SPIDX_T n) {
00152     SIZE.init(r); IDX.init(n,r); D.init(n); isref=0;
00153     return *this;
00154 };
00155
00156 wbsparray& init_nnz(SPIDX_T n=0, char kflag=0) {
00157     if (isref) {
00158         if (SIZE.isref) wblog(FL,"ERR %s() got isref "
00159             "%d,%d,%d",FCT,isref,SIZE.isref,IDX.isref,D.isref);
00160         isref=0;
00161     }
00162     if (!n) {
00163         if (!SIZE.len) { IDX.init(); }
00164         else {
00165             if (kflag) { IDX.init(n,SIZE.len/2); }
00166             else { IDX.init(n,SIZE.len); }
00167         }
00168         D.init(); return *this;
00169     }
00170
00171     if (!SIZE.len) wblog(FL,
00172         "ERR sparse::%s() got empty size SIZE!",FCT);
00173     if (kflag) {
00174         if (SIZE.len%2) wblog(FL,
00175             "ERR %s() even rank expected (%d)",FCT,SIZE.len);
00176         IDX.init(n,SIZE.len/2);
00177     }
00178     else { IDX.init(n,SIZE.len); }

```

```

00179         D.init(n); return *this;
00180     };
00181
00182     wbsparray& setRand(double x=1, char sflag=0);
00183     wbsparray& setRand_data(double x=1, char sflag=0);
00184
00185     template<class DB>
00186     wbsparray& init(const char *F, int L,
00187         SPIDX_T d1, SPIDX_T d2, const wbsparray<DB> &B);
00188
00189     template<class DB>
00190     wbsparray& init(const char *F, int L,
00191         SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, const wbsparray<DB> &B);
00192
00193     template<class DB>
00194     wbsparray& init(const char *F, int L,
00195         SPIDX_T d1, SPIDX_T d2, SPIDX_T d3, SPIDX_T d4, const wbsparray<DB> &B);
00196
00197     wbsparray& initEmpty(const wbsparray &a) {
00198         if (this!=&a) SIZE=a.SIZE;
00199         IDX.init(0,SIZE.len); D.init();
00200         return *this;
00201     };
00202
00203     wbsparray& init(
00204         const char *F, int L, const mxArray *a, unsigned k=0);
00205
00206     wbsparray& init2ref(const wbsparray &a) {
00207         SIZE=a.SIZE; IDX.init2ref(a.IDX); D.init2ref(a.D);
00208         isref=1; return *this;
00209     };
00210
00211     wbsparray& unRef() {
00212         if (!isref) return *this;
00213         IDX.unRef(); D.unRef(); isref=0;
00214         return *this;
00215     };
00216
00217     wbsparray& initCAT(const char *F, int L,
00218         wbvector< wbsparray* > &X,
00219         wbvector<SPIDX_T> *I0=NULL, char del=0
00220     );
00221
00222     wbsparray& initCAT(const char *F, int L,
00223         wbvector< wbsparray > &X, const wbvector<SPIDX_T> *I0=NULL
00224     ){
00225         wbvector< wbsparray* > xp(X.len);
00226         for (unsigned i=0; i<X.len; ++i) xp[i]=(&X[i]);
00227         return initCAT(F,L,xp,I0);
00228     };
00229
00230     wbsparray& initCAT(
00231         const char *F, int L, const wbvector< const wbsparray* > &X,
00232         unsigned dim
00233     );
00234
00235     wbsparray& initCAT(
00236         const char *F, int L, const wbvector< wbsparray > &X,
00237         unsigned dim
00238     ){
00239         wbvector< const wbsparray* > xp(X.len);
00240         for (unsigned i=0; i<X.len; ++i) xp[i]=(&X[i]);
00241         return initCAT(F,L,xp,dim);
00242     };
00243
00244     wbsparray& Cat(const char *F, int L,
00245         const wbsparray &B, unsigned dim
00246     );
00247
00248     wbsparray& init_kron(
00249         const wbvector<SPIDX_T> &sa, const wbvector<SPIDX_T> &sb,
00250         SPIDX_T nab, char kflag
00251     ){
00252         setSIZE_kron(FL,sa,sb);
00253         return init_nnz(nab,kflag);
00254     };
00255
00256     void setSIZE_kron(const char *F, int L,
00257         const wbvector<SPIDX_T> &sa, const wbvector<SPIDX_T> &sb,
00258         char kflag=0);
00259     void setSIZE_kron(const char *F, int L, char kflag=0);
00260
00261     wbsparray& setRec(SPIDX_T i, SPIDX_T j) {
00262         if (i>=D.len || j>=D.len || IDX.dim1!=D.len) wblog(FL,"ERR %s() "
00263             "index out of bounds (%d,%d/%d,%d",FCT,i,j,IDX.dim1,D.len);
00264         if (i!=j) { D[i]=D[j]; IDX.recSet(i,j); }
00265     }

```



```

00266         return *this;
00267     };
00268
00269     template<class T2>
00270     wbsparray& setRec(SPIDX_T i, SPIDX_T i1, SPIDX_T i2, const T2& x) {
00271         if (IDX.dim2!=2) wblog(FL,"ERR %s() "
00272             "rank-2 tensor required (%d: %s)",FCT,SIZE.len,SSTR_(this));
00273         if (i>=D.len || IDX.dim1!=D.len) wblog(FL,"ERR %s() "
00274             "index out of bounds (%d/%d,%d)",FCT,i,IDX.dim1,D.len);
00275         SPIDX_T *I=IDX.rec(i);
00276         I[0]=i1; I[1]=i2; D[i]=x;
00277         return *this;
00278     };
00279
00280     wbsparray& setRecP(SPIDX_T i, const SPIDX_T *I, const TD &x) {
00281         if (i>=D.len && IDX.dim1!=D.len) wblog(FL,"ERR %s() "
00282             "index out of bounds (%d /%d,%d)",FCT,i,IDX.dim1,D.len);
00283         IDX.recSetP(i,I); D[i]=x;
00284         return *this;
00285     };
00286
00287     wbsparray& setRec_kron(
00288         SPIDX_T l, TD x, unsigned r, SPIDX_T *i1, SPIDX_T *i2,
00289         unsigned kflag=0
00290     );
00291
00292     wbsparray& Resize(SPIDX_T l) {
00293         IDX.Resize(l,IDX.dim2); D.Resize(l);
00294         return *this;
00295     };
00296
00297     wbsparray& REDSIZE_NNZ(SPIDX_T l) {
00298         if (l) {
00299             if (IDX.dim1) {
00300                 if (l>IDX.dim1) wblog(FL,"ERR %s() "
00301                     "range out of bounds (IDX: %ld/%ld)",FCT,l,IDX.dim1);
00302                 IDX.dim1=l;
00303             }
00304             if (D.len) {
00305                 if (l>D.len) wblog(FL,"ERR %s() "
00306                     "range out of bounds (D: %ld/%ld)",FCT,l,D.len);
00307                 D.len=l;
00308             }
00309         }
00310         else { IDX.init(0,SIZE.len); D.init(); }
00311         return *this;
00312     };
00313
00314     wbsparray& Resize(const wbvector<SPIDX_T> &S);
00315
00316     wbsparray& operator= (const wbsparray &a) {
00317         if ((void*)this!=(void*)&a) init(a);
00318         return *this;
00319     };
00320     template<class DB>
00321     wbsparray& operator= (const wbsparray<DB> &a) {
00322         if ((void*)this!=(void*)&a) init(a);
00323         return *this;
00324     };
00325
00326     wbsparray& save2(wbsparray &a, char ref=0) {
00327         if (this!=&a) {
00328             SIZE.save2(a.SIZE); IDX.save2(a.IDX,ref); D.save2(a.D);
00329             a.isref=isref; isref=0;
00330         }
00331         return a;
00332     };
00333
00334     SPIDX_T sub2ind(SPIDX_T k) const {
00335         return Wb::sub2ind(SIZE.data,IDX.rec(k),SIZE.len);
00336     };
00337
00338     void ind2sub(SPIDX_T k, SPIDX_T *I) const {
00339         Wb::ind2sub(k, SIZE.data, I, SIZE.len);
00340     };
00341
00342     void ind2sub(SPIDX_T k, wbindex &I) const {
00343         if (I.len!=SIZE.len) I.init(SIZE.len);
00344         Wb::ind2sub(k, SIZE.data, I.data, SIZE.len);
00345     };
00346
00347     wbsparray<TD>& make2D(const wbindex &ic, wbsparray<TD> &B,
00348         char pos=1
00349     ) const;
00350
00351     sparseIndex2D<SPIDX_T>& toIndex2D(
00352         const wbindex &ic, sparseIndex2D<SPIDX_T> &a,

```

```

00353     char pos,
00354     char use_rmaj=0
00355 ) const;
00356
00357 wbsparray& initDiag(SPIDX_T d) {
00358     D.init(d); SIZE.init(); IDX.init();
00359     return *this;
00360 };
00361
00362 wbsparray& initDiag(const wbvector<TD> &dd) {
00363     D.init(dd); SIZE.init(); IDX.init();
00364     return *this;
00365 };
00366
00367 wbsparray& initDiag2(SPIDX_T d, TD dval=1) {
00368     initz(d,d, d); D.set(dval); {
00369         SPIDX_T i=0, l=d-1, *idx=IDX.data;
00370         for (; i<d; ++i, idx+=2) { idx[0]=l-i; idx[1]=i; }
00371     }
00372     return *this;
00373 };
00374
00375 bool isDiagMatrix(TD eps=1E-14) const;
00376
00377 bool isDiag(const char *F=0, int L=0) const {
00378     if (SIZE.len) { checkSize(F_LF); }
00379     return (!IDX.data && D.len ? 1 : 0);
00380 };
00381
00382 wbvector<TD>& getDiag(const char *F, int L, wbvector<TD> &dd) const;
00383
00384 TD getDiag(const char *F, int L, SPIDX_T k) const;
00385
00386 wbsparray& getDiag(const char *F, int L, wbsparray &A) const {
00387     A.init(); getDiag(F,L,A.D);
00388     return A;
00389 };
00390
00391 wbsparray& diag2reg(const char *F=0, int L=0, unsigned r=-1);
00392 wbsparray& diag2reg(
00393     const char *F, int L, wbsparray &A, unsigned r=-1) const;
00394
00395 wbsparray& toDiag(const char *F=0, int L=0) {
00396     wbsparray X; getDiag(F_L,X);
00397     return X.save2(*this);
00398 };
00399
00400 int toDiagSafe(const char *F, int L, TD eps=1E-14) {
00401     if (!isDiagMatrix(eps)) { if (F) {
00402         wblog(F_L, "ERR %s() got non-diagonal matrix",FCT); }
00403         return 0;
00404     }
00405     wbsparray X; getDiag(F_L,X);
00406     X.save2(*this); return 1;
00407 };
00408
00409 wbsparray& initIdentity(SPIDX_T d, TD dval=1){
00410     initDiag(d); D.set(dval);
00411     return *this;
00412 };
00413
00414 wbsparray& initIdentity(SPIDX_T d1, SPIDX_T d2, sSPIDX_T m=0, TD dval=1){
00415     if (d1==d2 && !m) { return initIdentity(d1,dval); }
00416
00417     sSPIDX_T k1=0, k2=0, i1=0, i2=d1, j1=0, j2=d2;
00418     SPIDX_T l, s[] = {d1,d2};
00419     if (m>=0) { j1=-m; j2=-m; } else { i1=m; i2+=m; }
00420
00421     k2=MIN(i2,j2); l=(k2>k1 ? k2-k1 : 0);
00422     SIZE.init(2,s); IDX.init(l,2); D.init2val(l,dval);
00423
00424     if (l) { SPIDX_T *idx=IDX.data;
00425         for (SPIDX_T i=0; i<l; ++i, ++k1, idx+=2) {
00426             idx[0]=k1-i1;
00427             idx[1]=k1-j1;
00428         }
00429     }
00430
00431     return *this;
00432 };
00433
00434 wbsparray& initIdentity(
00435     const wbvector<SPIDX_T> &S0, int m=-1, sSPIDX_T k=0, char fflag='f');
00436 wbsparray& initIdentityB3(
00437     SPIDX_T d1, SPIDX_T d2, SPIDX_T D,
00438     sSPIDX_T i0=0, char fflag='f'
00439 );

```

```

00440
00441 unsigned rank(const char *F=NULL, int L=0) const {
00442     checkSize(F,LF);
00443     if (!SIZE.len) {
00444         return (isDiag() ? 2 : 0);
00445     }
00446     return SIZE.len;
00447 };
00448
00449 SPIDX_T dim() const {
00450     if (isDiag()) return D.len;
00451     if (SIZE.len!=2 || SIZE[0]!=SIZE[1]) wblog(FL,
00452         "ERR %s() got %s array",FCT,SSTR_(this));
00453     return SIZE[0];
00454 };
00455
00456 SPIDX_T dim(unsigned k) const {
00457     if (!k) wblog(FL,
00458         "ERR %s() input dim not 1-based (%d)",FCT,k);
00459     return dim0(--k);
00460 };
00461
00462 SPIDX_T dim0(unsigned k) const {
00463     if (k<2 && isDiag()) { return D.len; }
00464     if (k<SIZE.len) { return SIZE[k]; }
00465     else wblog(FL,"ERR %s() "
00466         "index out of bounds (%d having %s)",FCT,k+1,SSTR_(this));
00467     return 0;
00468 };
00469
00470 SPIDX_T matDim(unsigned k=1) {
00471     if (!k || k>2) wblog(FL,
00472         "ERR %s() index out of bounds (%d/%d)",FCT,k);
00473     if (isDiag(FL)) { return D.len; } else
00474     if (k==1) { return Wb::prodRange(SIZE.data,SIZE.len/2); }
00475     else {
00476         unsigned l=SIZE.len/2;
00477         return Wb::prodRange(SIZE.data+l,SIZE.len-l);
00478     }
00479 };
00480
00481 SPIDX_T maxDim(unsigned r=-1) const {
00482     SPIDX_T d=0;
00483
00484     if (int(r)<0) {
00485         if (SIZE.len ) { d=SIZE.max(); } else
00486         if (isDiag(FL)) { d=D.len; }
00487     }
00488     else if (SIZE.len) {
00489         if (!r || r>SIZE.len) wblog(FL,
00490             "ERR %s() invalid dim range (%d/%d)",FCT,r,SIZE.len);
00491         d=SIZE[0];
00492         for (unsigned i=1; i<r; ++i) { if (d<SIZE[i]) d=SIZE[i]; }
00493     }
00494     else if (isDiag(FL)) {
00495         if (!r || r>2) wblog(FL,
00496             "ERR %s() invalid dim range (%d/2)",FCT,r);
00497         d=D.len;
00498     }
00499     else if (r) {
00500         wblog(FL,"ERR %s() invalid dim range (%d/0)",FCT,r);
00501     }
00502
00503     return d;
00504 };
00505
00506 TD norm2(char tnorm=0) const {
00507     return Wb::overlap(D.data,D.data,D.len,1,tnorm);
00508 };
00509
00510 wbststring sizeStr() const {
00511     wbststring s;
00512     if (SIZE.len) { s=SIZE.toStrf("","x"); }
00513     else if (D.len) {
00514         s.init(16); s.pushf(FL,"%dx%d",D.len,D.len);
00515     }
00516     else { s="[]"; }
00517     return s;
00518 };
00519
00520 TD SkipTinyCols(TD eps=1E-14) {
00521     unsigned r=rank(); if (!r && isEmpty()) return 0;
00522     SPIDX_T l=0, i=0; TD e2=0;
00523     wbstvector<TD> a2; norm2vec(r-1,a2);
00524     WBINDEX P(a2.len);
00525
00526     for (; i<a2.len; ++i) {

```

```

00527         if (a2[i]>eps) { P[l++]=i; } else { e2+=a2[i]; }
00528     }
00529     if (l) P.len=l; else P.init();
00530     Select0(P,r-1);
00531
00532     return e2;
00533 };
00534
00535 TD SkipTiny(TD eps=1E-14) {
00536     TD e2=0; SPIDX_T l=0, i=0;
00537     if (eps) {
00538         Wb::scale_eps(eps,D.data,D.len);
00539         for (i=0; i<D.len; ++i) {
00540             if (Wb::abs(D[i])>eps) { if (l<i) { setRec(l,i); }; ++l; }
00541             else { e2+=Wb::norm2(D[i]); }
00542         }
00543     }
00544     else {
00545         for (i=0; i<D.len; ++i) {
00546             if (D[i]!=eps) { if (l<i) { setRec(l,i); }; ++l; }
00547         }
00548     }
00549
00550     if (l<i) { IDX.ResizeXRecs(l); D.Shorten2(l); }
00551     return e2;
00552 };
00553
00554 bool isVector(const char *F=0, int L=0) const {
00555     unsigned r=rank(F_L); if (r==1) return 1;
00556     return (SIZE.count(1)>=(r-1));
00557 };
00558
00559 bool isMatrix(const char *F=0, int L=0) const { return rank(F_L)==2; };
00560 bool isRank(unsigned r) const { return (rank(FL)==r); };
00561
00562 bool isSMatrix(const char *F=0, int L=0, SPIDX_T *d=NULL) const {
00563     unsigned r=rank(F_L);
00564     if (r!=2 || (SIZE.len && SIZE[0]!=SIZE[1])) return 0;
00565     if (d) {
00566         SPIDX_T d0=(SIZE.len ? SIZE[0] : D.len);
00567         if (sSPIDX_T(*d)>=0) { if ((*d)!=d0) return 0; }
00568         else (*d)=d0;
00569     }
00570     return 1;
00571 };
00572
00573 bool isSMatrix(const char *F, int L, SPIDX_T d) const {
00574     unsigned r=rank(F_L); SPIDX_T d0=(SIZE.len ? SIZE[0] : D.len);
00575     if (r!=2 || (SIZE.len && SIZE[0]!=SIZE[1]) || d!=d0) return 0;
00576     return 1;
00577 };
00578
00579 bool isEmpty() const {
00580     return (SIZE.len || IDX.dim1 || D.len ? 0 : 1);
00581 };
00582
00583 bool isEmpty_() const {
00584     if (SIZE.len || IDX.dim1 || D.len) {
00585         if (IDX.dim1!=D.len || IDX.dim2!=SIZE.len) wblog(FL,
00586             "ERR %s() size mismatch (%dx%d / %s)",
00587             FCT,D.len,SIZE.len,SSTR(IDX));
00588         return 0;
00589     }
00590     return 1;
00591 };
00592
00593 explicit operator bool() const { return !isEmpty_(); }
00594 bool operator! () const { return isEmpty_(); }
00595
00596 bool isZero(TD eps=0, char flag=0) const;
00597
00598 SPIDX_T nnz() const {
00599     if (isScalar(FL))
00600         return (D.len==1 && D.data[0]!=0 ? 1 : 0);
00601     else return D.len;
00602 };
00603
00604 SPIDX_T nnz(const TD eps1, const TD eps2=1E9) const {
00605     SPIDX_T i=0, n=0; TD a,e=0;
00606     for (i=0; i<D.len; ++i) {
00607         a=Wb::abs(D[i]); if (a>eps1) ++n;
00608         else if (a>eps2 && e<a) e=a;
00609     }
00610     if (e) wblog(FL,
00611         "WRN %s() got small value(s) %.3g (%g;%g)",FCT,
00612         double(e), double(eps1), double(eps2));
00613     return n;

```

```

00614     };
00615
00616     SPIDX_T nnz_diag() const {
00617         if (isDiag(FL)) return D.nnz();
00618         if (IDX.dim2!=2) wblog(FL,
00619             "ERR %s() requires rank-2 tensor (S=%s)",FCT,SSTR_(this));
00620         SPIDX_T i=0, n=0; const SPIDX_T *I=IDX.data;
00621         for (; i<IDX.dim1; ++i, I+=2) { if (I[0]==I[1] && D[i]!=0) ++n; }
00622         return n;
00623     };
00624
00625     SPIDX_T numel() const {
00626         return (isDiag(FL) ? D.len*D.len : SIZE.prod());
00627     };
00628
00629     TD norm(char tnorm=0) const {
00630         return Wb::sqrt(Wb::overlap(D.data,D.data,D.len,1,tnorm));
00631     };
00632
00633     template<class T2>
00634     wbvector<T2>& norm2vec(unsigned k,
00635         wbvector<T2> &A, char tnorm=0
00636     ) const;
00637
00638     void swap(wbsparray &B) {
00639         if (this!=&B) {
00640             if (isref || B.isref) wblog(FL,
00641                 "ERR %s() got arrays with isref=(%d,%d)",FCT,isref,B.isref);
00642             SIZE.swap(B.SIZE);
00643             IDX.swap(B.IDX);
00644             D.swap(B.D);
00645         }
00646     };
00647
00648     wbsparray& TimesEl(const wbsparray &B, char tnorm=0);
00649
00650     TD dotProd(
00651         const char *F, int L, const wbsparray &B, char tnorm=0) const;
00652
00653     TD froNorm2(const wbsparray &B) const { return dotProd(0,0,B); };
00654
00655     double normDiff2(const wbsparray &B) const {
00656         wbsparray X(*this); X-=B; return (double)X.norm2();
00657     };
00658     double normDiff (const wbsparray &B) const {
00659         wbsparray X(*this); X-=B; return (double)X.norm();
00660     };
00661
00662     TD Normalize(char tnorm=0, char qflag=0);
00663     TD NormalizeCol(SPIDX_T k, char tnorm=0, char qflag=0);
00664
00665     wbsparray& QRdecomp(const char *F, int L,
00666         wbsparray &Q,
00667         char posR=0,
00668         TD eps=1E-14
00669     );
00670
00671     wbsparray& OrthoNormalizeColsQR(
00672         const char *F=NULL, int L=0,
00673         TD eps=1E-14
00674     );
00675
00676     wbsparray& OrthoNormalizeColsGS(
00677         const char *F=NULL, int L=0, char tnorm=0,
00678         char qxflag=0, TD eps=1E-14, unsigned np=1
00679     );
00680
00681     wbsparray& setCol(
00682         const char *F, int L, SPIDX_T k, const wbsparray &v);
00683
00684     wbsparray& setCol(SPIDX_T k, const wbsparray &v) {
00685         return setCol(0,0,k,v);
00686     };
00687
00688     double sparsity() const {
00689         if (!D.len) return 0;
00690         if (isScalar(FL)) return (D.len==1 && D.data[0]!=0 ? 1 : 0);
00691         return D.len/double(numel());
00692     };
00693
00694     int checkTrailingSingletons(const char *F=0, int L=0, unsigned *r=NULL) const;
00695
00696     int SkipTrailingSingletons(const char *F=0, int L=0, unsigned r=-1);
00697     int skipTrailingSingletons(
00698         const char *F, int L, wbsparray &c, unsigned r=-1) const;
00699
00700     bool hasSingletons(const wbindex &I) const;

```

```

00701
00702     wbsparray& skipSingletons(
00703         const char *F, int L, const wbindex &I, wbsparray &A
00704     ) const;
00705
00706     wbsparray& AddTrailingSingletons(
00707         const char *F, int L, unsigned r);
00708
00709     wbsparray& initScalar(const TD x=0, unsigned r=-1) {
00710         if (int(r)<=0) {
00711             if (D.len!=1) { D.init(1); }; D[0]=x;
00712             if (SIZE.len) { SIZE.init(); }
00713             IDX.init();
00714         }
00715         else {
00716             SIZE.init(r).set(1);
00717             if (x) {
00718                 if (D.len!=1) { D.init(1); }; D[0]=x;
00719                 IDX.init(1,r);
00720             }
00721             else { IDX.init(0,r); D.init(); }
00722         }
00723         isref=0; return *this;
00724     };
00725
00726     char isScalar(const char *F=0, int L=0, char zflag=0) const {
00727         if (!SIZE.len) {
00728             if (IDX.dim1 || IDX.dim2) sperror_this(F_L);
00729             return (zflag ? D.len<=1 : D.len==1);
00730         }
00731         return SIZE.allEqual(1);
00732     };
00733
00734     wbsparray& toScalar(const char *F=0, int L=0, unsigned r=-1);
00735
00736     TD getScalar(const char *F=0, int L=0) const;
00737
00738     char checkSize(
00739         const char *F=0, int L=0, const char *fct=0, const char *istr=0
00740     ) const;
00741
00742     char checkSize2(const char *F, int L, const wbsparray &B) const;
00743
00744     void check_IDX_range(const char *F=0, int L=0) const;
00745
00746     bool isProptoId() const { TD q=0; return isProptoId(q); }
00747     bool isProptoId(TD &q,TD eps=1E-14) const;
00748
00749     bool isIdentityMatrix(TD eps=1E-14) const {
00750         TD q=1; return isProptoId(q,eps);
00751     };
00752
00753     char isIdentity(TD eps=1E-14) const;
00754     char isIdentity(const wbpem &P, TD *dval=NULL, TD eps=1E-14) const;
00755
00756     bool hasSize(SPIDX_T d1) const;
00757     bool hasSize(SPIDX_T d1, SPIDX_T d2) const;
00758     bool hasSize(SPIDX_T d1, SPIDX_T d2, SPIDX_T d3) const;
00759
00760     template<class T>
00761     wvector<T>& getSize(wvector<T> &S) const {
00762         if (SIZE.len) { return S.initT(SIZE); }
00763         if (D.len && isDiag()) { return S.init(2).set(D.len); }
00764         return S.init();
00765     };
00766
00767     char fitsSize(const wbsparray &, char strict=0) const;
00768
00769     char sameSize(const wbsparray &, char strict=0) const;
00770     char sameSizeUp2Singletons(
00771         const wvector<SPIDX_T> &S, char strict=0) const;
00772
00773     char matchWithSingletons(const char *F, int L,
00774         const wvector<SPIDX_T> &S, wvector<int> &M
00775     ) const;
00776
00777     char sameUptoFac(
00778         const wbsparray &B, TD *fac=NULL, char lex=-1,
00779         TD eps=1E-12) const;
00780
00781     bool operator== (const wbsparray &B) const {
00782         if (this==&B) return 1;
00783         if (SIZE!=B.SIZE) return 0;
00784         if (IDX==B.IDX && D==B.D) return 1;
00785         return (normDiff2(B)<1E-20);
00786     };
00787

```

```

00788     bool operator!= (const wbsparray &B) const {
00789         return !(*this==B);
00790     };
00791
00792     wbsparray& Plus(const char *F, int L,
00793         const wbsparray &B, TD bfac=1, TD afac=1, TD eps=0
00794     );
00795
00796     TD sum() const { return D.sum(); };
00797     TD aMin() const { return D.aMin(1); };
00798     TD aMax() const { return D.aMax(); };
00799
00800     SPIDX_T findRecSortedP(SPIDX_T *idx, SPIDX_T n,
00801         char lex=-1
00802     );
00803
00804     wbsparray& Eldiv(const char *F, int L, const wbvector<TD> &x_) {
00805         SPIDX_T i=0, j;
00806         const SPIDX_T *idx=IDX.data;
00807         const TD *x=x_.data;
00808
00809         if (!isVector()) wblog(F_L,
00810             "ERR %s() invalid vector (%s)",FCT,SSTR_(this));
00811         for (; i<SIZE.len; ++i) { if (SIZE[i]>1) { idx+=i; break; } }
00812
00813         for (i=0; i<IDX.dim1; ++i, idx+=IDX.dim2) { j=(*idx);
00814             if (j>=x_.len) wblog(F_L,
00815                 "ERR %s() index out of bounds (%d/%d)",FCT,j+1,x_.len);
00816             if (x[j]==0) wblog(F_L,
00817                 "ERR %s() div/0 !? (%g)",FCT,double(x[j]));
00818             D.data[i]/=x[j];
00819         }
00820         return *this;
00821     };
00822
00823     wbsparray& operator+=(const wbsparray &B) { return Plus(F_L,B,+1); }
00824     wbsparray& operator-=(const wbsparray &B) { return Plus(F_L,B,-1); }
00825
00826     wbsparray& Conj() { D.Conj(); return *this; }
00827
00828     wbsparray& operator*=(TD x) {
00829         if (isref) {
00830             D*=x;
00831         }
00832         else {
00833             if (x!=0) { if (x!=1) D*=x; }
00834             else init_nnz();
00835         }
00836         return *this;
00837     };
00838
00839     wbsparray& operator/=(TD x) {
00840         if (x==0) wblog(F_L,"ERR %s() div/0",FCT);
00841         if (x!=1) D/=x;
00842         return *this;
00843     };
00844
00845     const TD& operator[](SPIDX_T i) const {
00846         if (i>=D.len) {
00847             wblog(F_L,"ERR %s() index out of bounds (%d/%d; %s)",
00848                 FCT,i,D.len,SSTR_(this));
00849         }
00850         return D.data[i];
00851     };
00852
00853     TD& operator[](SPIDX_T i) {
00854         if (i>=D.len) {
00855             if (i==0 && IDX.dim1==0) {
00856                 if (SIZE.len && SIZE.allEqual(1)) {
00857                     IDX.init(1,SIZE.len);
00858                     D.init(1); return D.data[i];
00859                 }
00860                 else if (!SIZE.len && IDX.dim2==0) {
00861                     IDX.init();
00862                     D.init(1); return D.data[i];
00863                 }
00864             }
00865             wblog(F_L,"ERR %s() index out of bounds (%d/%d; %s)",
00866                 FCT,i,D.len,SSTR_(this));
00867         }
00868         return D.data[i];
00869     };
00870
00871     TD value(const SPIDX_T *idx, SPIDX_T n, char lex=-1
00872     ) const {
00873         SPIDX_T i=IDX.findRecSorted(idx,n,lex);
00874         return (sSPIDX_T(i)<0 ? TD(0.) : D.data[i]);

```

```

00875     };
00876
00877     wbststring info2Str(const char *istr=0) const {
00878         wbststring s(32);
00879         if (istr) {
00880             s.pushf(FL,"%s(%s; %dx%d; %d)",istr,
00881                 SSTR_(this), IDX.dim1,IDX.dim2,D.len);
00882         }
00883         else {
00884             s.pushf(FL,"%s; %dx%d; %d",
00885                 SSTR_(this), IDX.dim1,IDX.dim2,D.len);
00886         }
00887         return s;
00888     };
00889
00890     size_t memSize() const {
00891         size_t m=sizeof(TD);
00892 #ifdef QS_USING_MPFR
00893         if (typeid(TD)==typeid(Wb::quad)) {
00894             m+=16;
00895         }
00896 #endif
00897         return ( sizeof(isref)
00898             + sizeof(SIZE) + SIZE.len*sizeof(SPIDX_T)
00899             + sizeof(IDX ) + IDX.numel()*sizeof(SPIDX_T)
00900             + sizeof(D ) + D.len*m
00901         );
00902     };
00903
00904     void info(const char* istr="") const { return info(0,0,istr); };
00905     void info(const char* F, int L, const char* "=") const;
00906     void print(const char* istr="", const char *fmt="") const;
00907
00908     void print(
00909         const char *F, int L,
00910         const char *istr="", const char *fmt=""
00911     ) const;
00912
00913     void printdata(const char *istr="ans", const char *fmt="") const;
00914
00915     void sperror_this(
00916         const char *F, int L, const char *fct=0, const char *istr=0
00917     ) const {
00918         wblog(F_L,"ERR %s() %s S=%s (%dx%d; %d) %s",
00919             fct ? fct : "sparse", istr ? istr : "got",
00920             SSTR_(this), IDX.dim1,IDX.dim2,D.len,
00921             isref ? " **REF** ":"");
00922     };
00923 };
00924
00925     TD trace() const;
00926
00927     template<class T2>
00928     wbvector<T2>& trace(unsigned k, wbvector<T2> &A) const;
00929
00930     bool isSym_aux(
00931         const char *file, int line, const char *fct,
00932         const wbsparray &B, TD eps, TD* xref,
00933         const char symflag='s',
00934         const char lflag=1
00935     ) const;
00936
00937     bool isHConj(
00938         const wbsparray &B, TD eps=1E-12, TD* xref=NULL
00939     ) const { return isSym_aux(FLF, B, eps, xref, 's'); };
00940
00941     bool isHConj(
00942         TD eps=1E-12, TD* xref=NULL
00943     ) const { return isSym_aux(FLF,*this,eps,xref,'s'); };
00944
00945     bool isAHerm(
00946         const wbsparray &B, TD eps=1E-12, TD* xref=NULL
00947     ) const { return isSym_aux(FLF, B, eps, xref, 'a'); };
00948
00949     bool isAHerm(
00950         TD eps=1E-12, TD* xref=NULL
00951     ) const { return isSym_aux(FLF,*this,eps,xref,'a'); };
00952
00953     wbsparray& Sort(char lex=-1) {
00954         if (IDX.data) {
00955             if (IDX.dim1!=D.len) wblog(FL,
00956                 "ERR %s() %d/%d !?",FCT,IDX.dim1,D.len);
00957             wbperrm P; IDX.SortRecs(P,+1,lex); D.Select(P);
00958         }
00959         return *this;
00960     };
00961

```



```

00962     double Compress(
00963         const char *F=0, int L=0, TD eps=0,
00964         char lex=-1,
00965         const wbpemr *P=NULL
00966     );
00967
00968     bool isCompressed(
00969         char lex=-1, char dir=0
00970     ) const { return IDX.isUniqueSorted(dir,lex); };
00971
00972     int splitSparseCM(
00973         const char *F, int L, wvector< wbsparray > &X,
00974         TD eps=0, char ref=0, char tnorm=0);
00975
00976     wbsparray& reshape(
00977         const wvector<SPIDX_T> &S, wbsparray &A) const;
00978
00979     wbsparray& Reshape(const wvector<SPIDX_T> &S){
00980         wbsparray X; reshape(S,X); return X.save2(*this);
00981     };
00982
00983     wbsparray& Reshape(SPIDX_T s1, SPIDX_T s2){
00984         wbsparray X; wvector<SPIDX_T> S(2); S[0]=s1; S[1]=s2;
00985         reshape(S,X); return X.save2(*this);
00986     };
00987
00988     wbsparray& Reshape(SPIDX_T s1, SPIDX_T s2, SPIDX_T s3){
00989         wbsparray X; wvector<SPIDX_T> S(3); S[0]=s1; S[1]=s2; S[2]=s3;
00990         reshape(S,X); return X.save2(*this);
00991     };
00992
00993     wvector<TD>& getCol(SPIDX_T k, wvector<TD> &v) const;
00994     wvector<TD>& getRow(SPIDX_T k, wvector<TD> &v) const;
00995
00996     wbsparray& getBlock(const char *F, int L,
00997         SPIDX_T i1, SPIDX_T i2, SPIDX_T j1, SPIDX_T j2,
00998         wbsparray<TD> &B
00999     ) const;
01000
01001     wbsparray& getCol(SPIDX_T k, wbsparray<TD> &B) {
01002         if (k>=dim0(k)) wblog(FL,"ERR %s() "
01003             "col-index out of bounds (%ld / %s)",ECT,k,SSTR_(this));
01004         return getBlock(FL,0,-1,k,k,B);
01005     };
01006
01007     wbsparray& splitBlock(const char *F, int L,
01008         sSPIDX_T i1, sSPIDX_T i2, sSPIDX_T j1, sSPIDX_T j2,
01009         wbsparray<TD> &B
01010     );
01011
01012     wbsparray& SplitQR(sSPIDX_T k, sSPIDX_T k_, wbsparray<TD> &X) {
01013         save2(X); X.splitBlock(FL,k,-1,k_,-1,*this);
01014         return *this;
01015     };
01016
01017     TD getHouseholderVec(const char *F, int L,
01018         SPIDX_T k, SPIDX_T j, wbsparray<TD> &b,
01019         TD eps2=0
01020     ) const;
01021
01022     wbsparray& IDX_Shift(sSPIDX_T i1, sSPIDX_T i2);
01023
01024     wbsparray<TD>& LProject1(
01025         const wbsparray<TD> &b, wbsparray<TD> &X,
01026         TD xfac=1, char tnorm=0) const;
01027
01028     SPIDX_T LContractVec_omp(
01029         const wbsparray<TD> &b, wvector<TD> &x, char tnorm) const;
01030
01031     wbsparray& splitLast(SPIDX_T k, wbsparray &a) const;
01032
01033     wbsparray& select0(
01034         const WBINDEX &,
01035         unsigned dim,
01036         wbsparray&
01037     ) const;
01038
01039     wbsparray& Select0(const WBINDEX &I, unsigned dim) {
01040         wbsparray X; save2(X); X.select0(I,dim,*this);
01041         return *this;
01042     };
01043
01044     wbsparray& Permute(const char* s, char iflag=0);
01045     wbsparray& Permute(const wbpemr&, char iflag=0);
01046     wbsparray& permute(const char* s, wbsparray&, char iflag=0) const;
01047     wbsparray& permute(const wbpemr&, wbsparray&, char iflag=0) const;
01048

```

```

01049     wbsparray& transpose(const char *F, int L, wbsparray &A) const;
01050     wbsparray& Transpose(const char *F=0, int L=0) {
01051         wbsparray X; transpose(F,L,X); X.save2(*this);
01052         return *this;
01053     };
01054
01055     wbsparray& htranspose(const char *F, int L, wbsparray &A) const {
01056         transpose(F,L,A); A.D.Conj();
01057         return A;
01058     };
01059     wbsparray& hTranspose(const char *F=0, int L=0) {
01060         wbsparray X; htranspose(F,L,X); X.save2(*this);
01061         return *this;
01062     };
01063
01064     wbsparray& ColPermute(const wbpperm &P, char iflag=0);
01065     wbsparray& MatPermute(const wbpperm &P, char iflag=0);
01066
01067     wbsparray& mtimes(
01068         const char *F, int L, const wbsparray&B, wbsparray&C,
01069         char sflag=0, const TD eps=0
01070     ){
01071         const wbsparray &A=*this;
01072         if (A.rank()!=2 || B.rank()!=2) wblog(FL,
01073             "ERR %s() requires rank-2 wbsparrays (%s * %s)",
01074             FCT, SSTR(A), SSTR(B));
01075         wbindx ia(1), ib(1); ia[0]=1; ib[0]=0;
01076
01077         return contract(F,L,ia,B,ib,C,wbpperm(),1,0,sflag,eps);
01078     };
01079
01080     char contract_scalar(
01081         const char* F, int L, const ctrIdx &ica,
01082         const wbsparray &B, const ctrIdx &icb, wbsparray &C,
01083         const wbpperm &pfinal=wbpperm(), const TD& afac=1., const TD& cfac=0.
01084     ) const;
01085
01086     wbsparray& contract_diag_diag(
01087         const char* F, int L, const ctrIdx &ica,
01088         const wbsparray &B, const ctrIdx &icb, wbsparray &C,
01089         const wbpperm &pfinal, const TD& afac, const TD& cfac
01090     ) const;
01091
01092     char contract_check_2full(
01093         const char* F, int L, const ctrIdx &ica,
01094         const wbsparray &B, const ctrIdx &icb, wbsparray &C,
01095         const wbpperm &pfinal, const TD& afac, const TD& cfac,
01096         char flag, TD eps, SPIDX_T *nnz_C=NULL
01097     ) const;
01098
01099     wbsparray& contract(const char *F, int L, const ctrIdx &ica,
01100         const wbsparray &B, const ctrIdx &icb, wbsparray &C,
01101         const wbpperm &pfinal=wbpperm(), const TD& afac=1., const TD& cfac=0.,
01102         char sflag=0, const TD eps=0
01103     ) const;
01104
01105     wbsparray& contract(const char *F, int L, const char *ica,
01106         const wbsparray &B, const char *icb, wbsparray &C,
01107         const wbpperm &pfinal=wbpperm(), const TD& afac=1., const TD& cfac=0.,
01108         char sflag=0, const TD eps=0
01109     ) const {
01110
01111         return contract(F,L,
01112             ctrIdx(FL,ica), B, ctrIdx(FL,icb), C,
01113             pfinal,afac,cfac,sflag,eps
01114         );
01115     };
01116
01117     wbarray<TD>& contract(const char* F, int L, const ctrIdx &ica,
01118         const wbsparray &B, const ctrIdx &icb, wbarray<TD> &C,
01119         const wbpperm &pfinal, const TD& afac, const TD& cfac
01120     ) const;
01121
01122     wbsparray& contract(const char *F, int L,
01123         unsigned il, const wvector<TD> &B, wbsparray &C
01124     ) const;
01125
01126     wbsparray& comm(const char *F, int L,
01127         const wbsparray &B, wbsparray &C,
01128         char aflag='N', char bflag='N'
01129     ) const;
01130
01131     wbsparray& tensorProdX(
01132         const char *F, int L, const wbsparray& B, wbsparray& C,
01133         char conjA=0, const TD &afac=1, char conjB=0, const TD &cfac=0,
01134         const wbpperm *Pfinal=NULL
01135     ) const;

```

```

01136
01137 wbsparray& tensorProd(
01138     const char *F, int L, const wbsparray& B, wbsparray& C,
01139     char aflag='N', char bflag='N', char kflag=0
01140 ) const;
01141
01142 wbsparray& Kron(const wbsparray& B,
01143     char aflag='N', char bflag='N', char kflag='k'
01144 ){
01145     wbsparray X; tensorProd(FL,B,X,aflag,bflag,kflag);
01146     return X.save2(*this);
01147 };
01148
01149 wbsparray& kron(
01150     const char *F, int L, const wbsparray& B, wbsparray& C,
01151     char aflag='N', char bflag='N', char kflag='k'
01152 ) const {
01153     return tensorProd(F,L,B,C,aflag,bflag,kflag);
01154 };
01155
01156 mxArray* toMxSp() const;
01157
01158 mxArray* toMx(int m=-99) const {
01159     unsigned r=rank(FL);
01160
01161     if (m==-99 && r<=2 && Wb::isBaseType(typeid(TD))) {
01162         return toMxSp();
01163     }
01164
01165     mxArray *a=mxCreateStruct(1,1);
01166     add2MxStruct(a,0,m);
01167     return a;
01168 };
01169
01170 mxArray* IDtoMx() const;
01171
01172 mxArray* mxCreateStruct(unsigned m, unsigned n) const;
01173 void add2MxStruct(mxArray *a, unsigned i, int ma=-99) const;
01174
01175 mxArray* mxCreateCell(unsigned m, unsigned n) const;
01176 void add2MxCell(mxArray *a, unsigned i) const;
01177
01178 mxArray* toMx0(int m=-99) const {
01179     mxArray *a=mxCreateStructX(1,1,m);
01180     add2MxStructX(a,0,m);
01181     return a;
01182 };
01183 mxArray* mxCreateStructX(unsigned m, unsigned n, int ma=-99) const;
01184 void add2MxStructX(mxArray *a, unsigned i, int ma=-99) const;
01185
01186 mxArray* toMX() const {
01187     mxArray *a=mxCreateSTRUCT(1,1);
01188     return add2MxSTRUCT(a,0);
01189 };
01190 mxArray* mxCreateSTRUCT(unsigned m, unsigned n) const;
01191 mxArray* add2MxSTRUCT(mxArray *a, unsigned i) const;
01192
01193 void put(const char *vname, unsigned ma=-99, const char *ws="caller"
01194 ) const { put(0,0,vname,ma,ws); };
01195
01196 void put0(const char *F, int L,
01197     const char *vname, const char *ws="caller"
01198 ) const { put(F,L,vname,-99,ws,'X'); };
01199
01200 void put(const char *F, int L, const char *vname,
01201     unsigned ma=-99, const char *ws="caller", char flag=0
01202 ) const {
01203
01204     mxArray *a=0; {
01205         if (!flag) a=toMx(ma); else
01206             if (flag=='X') a=toMX(); else
01207                 if (flag=='0') a=toMx0(); else
01208                     wblog(F,L,"ERR %s() invalid flag=%s",FCT,Wb::char2Str(flag).data);
01209     }
01210     int i=mexPutVariable(ws,vname,a);
01211
01212     if (i) wblog(F,L,
01213         "ERR failed to write variable `%s' (%d)",vname,i);
01214     if (F) wblog(F,L,
01215         "I/O putting variable '%s' to %s",vname,ws);
01216     mxDestroyArray(a);
01217 };
01218
01219 wbvector<SPIDX_T> SIZE;
01220 wbMatrix<SPIDX_T> IDX;
01221
01222 wbvector<TD> D;

```

```

01223
01224     char isref;
01225
01226 protected:
01227
01228     unsigned get2DSize(
01229         const char *F, int L,
01230         SPIDX_T &d1, SPIDX_T &d2, int m=-1, char fflag=0) const;
01231
01232     unsigned get2DIndex(
01233         wbMatrix<SPIDX_T> &IJ, SPIDX_T &d1, SPIDX_T &d2,
01234         int m=-1) const;
01235
01236 private:
01237
01238     TD dotProd_full_diag(const wbsparray &B, char cflag) const;
01239
01240     wbsparray& timesEl_full_diag(
01241         const wbsparray &B, wbsparray &X, char cflag
01242     ) const;
01243
01244 };
01245
01246 template<class TD> inline
01247 wbsparray<TD> operator+(const wbsparray<TD> &A) { return A; };
01248
01249 template<class TD> inline
01250 wbsparray<TD> operator-(const wbsparray<TD> &A) {
01251     wbsparray<TD> X(A); TD *x=X.D.data;
01252     for (SPIDX_T n=X.D.len, i=0; i<n; ++i) { x[i]=-x[i]; }
01253     return X;
01254 };
01255
01256 template<class TD>
01257 class indexSparseRef {
01258 public:
01259
01260     indexSparseRef(const wbsparray<TD> &A_, unsigned M2=1)
01261         : M(0), A(NULL) { init(F_L,A_,M2); };
01262
01263     indexSparseRef(
01264         const char *F, int L, const wbsparray<TD> &A_, unsigned M2=1)
01265         : M(0), A(NULL) { init(F_L,A_,M2); };
01266
01267     indexSparseRef& init(const char *F, int L,
01268         const wbsparray<TD> &A_, unsigned M2=1);
01269
01270     SPIDX_T numCols() const;
01271     SPIDX_T numRows() const;
01272     SPIDX_T nnz(SPIDX_T k) const;
01273
01274     TD LContractVec(
01275         const wbsparray<TD> &b, SPIDX_T k, char tnorm=0);
01276
01277     mxArray* toMx() const;
01278
01279     unsigned M;
01280
01281     wbvector<SPIDX_T> JA;
01282
01283     wbvector<sSPIDX_T> JJ;
01284
01285     const wbsparray<TD> *A;
01286
01287 protected:
01288 private:
01289
01290 };
01291
01292 template <class TD>
01293 indexSparseRef<TD>& indexSparseRef<TD>::init(
01294     const char *F, int L, const wbsparray<TD> &A_, unsigned M2
01295 ) {
01296     unsigned i, r=A_.rank(F_L);
01297
01298     if (r<2 || A_.SIZE.len!=r || A_.IDX.dim2!=r) wblog(F_L,
01299         "ERR %s() got empty or diagonal wbsparray !? (%s)",FCT,SSTR(A_));
01300
01301     if (!M2 || M2>=r) wblog(F_L,
01302         "ERR %s() M2 out of bounds (%d/%d)",FCT,M2,r);
01303
01304     M=r-M2; A=(&A_);
01305
01306     const SPIDX_T *s=A_.SIZE.data, *idx=A_.IDX.data;
01307     SPIDX_T l, n=A_.IDX.dim1, D1=s[0], D2=s[M];

```

```

01310
01311     sSPIDX_T j,I,J, Ip=-1, Jp=-1;
01312
01313     for (i=1; i<M; ++i) D1*=s[i];
01314     for (++i; i<r; ++i) D2*=s[i];
01315
01316     #ifndef WB_SKIP_ASSERT
01317     { double d1=s[0], d2=s[M];
01318       for (i=1; i<M; ++i) d1*=s[i];
01319       for (++i; i<r; ++i) d2*=s[i];
01320       if ((1+ceil(log2(d1))) >= 8*sizeof(SPIDX_T) ||
01321           (1+ceil(log2(d2))) >= 8*sizeof(SPIDX_T)) wblog(FL,"ERR %s() "
01322         "got possible indexSparseRef overrun (%.3g, %.3g / %.3g)",
01323         FCT, log2(d1), log2(d2), 8*sizeof(SPIDX_T)
01324       );
01325     }
01326     #endif
01327
01328     if (!D1 || !D2) wblog(FL,"ERR %s() got empty wbsparray !?",FCT);
01329
01330     JA.init(D2+1);
01331     JJ.init2val(MIN(D1,D2),-1);
01332
01333     for (l=0; l<n; ++l, idx+=r) {
01334       I=Wb::sub2ind(s,idx,M); J=Wb::sub2ind(s+M,idx+M,M2);
01335
01336       if (J>Jp) { for (j=Jp+1; j<=J; ++j) JA[j]=1; }
01337       else {
01338         bool i=(J==Jp); if ((i && I<=Ip) || !i) wblog(FL,
01339           "ERR %s() wbsparray not sorted/col-major !?",FCT);
01340       }
01341
01342       if (I==J) JJ[J]=1;
01343
01344       Ip=I; Jp=J;
01345     }
01346
01347     J=D2; if (J>Jp) { for (j=Jp+1; j<=J; ++j) JA[j]=1; }
01348
01349     return *this;
01350 };
01351
01352 template <class TD> inline
01353 SPIDX_T indexSparseRef<TD>::numCols() const {
01354   if (!A || JA.len<1) wblog(FL,
01355     "ERR %s() not yet initialized !?",FCT);
01356   return (JA.len-1);
01357 };
01358
01359 template <class TD> inline
01360 SPIDX_T indexSparseRef<TD>::numRows() const {
01361   if (!A || !M || A->SIZE.len<=M) wblog(FL,
01362     "ERR %s() got uninitialized object ?!",FCT);
01363   const SPIDX_T *S=A->SIZE.data; SPIDX_T D1=S[0];
01364   for (unsigned i=1; i<M; ++i) D1*=S[i];
01365   return D1;
01366 };
01367
01368 template <class TD> inline
01369 SPIDX_T indexSparseRef<TD>::nnz(SPIDX_T k) const {
01370   if (k+1>JA.len) wblog(FL,
01371     "ERR %s() index out of bounds (%d/%d) !?",FCT,k,JA.len);
01372   return (JA[k+1]-JA[k]);
01373 };
01374
01375 template <class TD> inline
01376 TD indexSparseRef<TD>::LContractVec(
01377   const wbsparray<TD> &B, SPIDX_T k, char tnorm
01378 ){
01379   TD x=0;
01380
01381   if (!A) wblog(FL,"ERR %s() got empty A !?",FCT);
01382   if (A->SIZE.len!=2 || M!=1) wblog(FL,"ERR %s() "
01383     "only applies to rank-2 tensors (%s; %d)",FCT,SSTR_(A),M);
01384   if (k+1>JA.len) wblog(FL,
01385     "ERR %s() index out of bounds (%d/%d) !?",FCT,k,JA.len);
01386
01387   SPIDX_T d1=A->SIZE[0], d2=A->SIZE[1];
01388   SPIDX_T i=0, i2=B.IDX.dim1, j=JA[k], j2=JA[k+1];
01389
01390   if (k>=d2) wblog(FL,
01391     "ERR %s() index out of bounds (%d/%d) !?",FCT,k,d2);
01392   if (B.SIZE.len!=1 || B.SIZE[0]!=d1) wblog(FL,
01393     "ERR %s() size mismatch (%s <> %s)",FCT,SSTR_(A),SSTR_(B));
01394   if (j==j2) return x;
01395
01396   SPIDX_T r=2, jr=j*r;

```

```

01397     char cflag = ((tnorm || WbUtil<TD>::hasConj()) ? 1 : 0);
01398     const SPIDX_T *I=B.IDX.data, *J=A->IDX.data;
01399     const TD *a=A->D.data, *b=B.D.data;
01400
01401     while (i<i2 && j<j2) {
01402         if (I[i ]<J[jr]) { ++i; } else
01403         if (J[jr]<I[i ]) { ++j; jr+=r; } else {
01404             if (cflag)
01405                 { x+=Wb::CONJ(b[i])*a[j]; }
01406             else { x+=      b[i] *a[j]; }
01407
01408             ++i; ++j; jr+=r;
01409
01410             if (i<i2 && I[i]<=I[i-1]) wblog(FL,
01411                 "ERR %s() A: input sparse IDX not sorted !?",FCT);
01412             if (j<j2 && J[jr]<=J[jr-r]) wblog(FL,
01413                 "ERR %s() B: input sparse IDX not sorted !?",FCT);
01414         }
01415     }
01416     return x;
01417 };
01418
01419 template <class TD>
01420 mxArray* indexSparseRef<TD>::toMx() const {
01421
01422     if (A==NULL) wblog(FL,"ERR %s() A not yet initialized !?",FCT);
01423
01424     const char *fields[]={ "SIZE", "M", "JA", "JJ" };
01425     mxArray *a=mxCreateStructMatrix(1,1,4,fields);
01426
01427     mxSetFieldByNumber(a,0, 0, A->SIZE.toMx());
01428     mxSetFieldByNumber(a,0, 1, numtoMx(M));
01429     mxSetFieldByNumber(a,0, 2, JA.toMx());
01430     mxSetFieldByNumber(a,0, 3, JJ.toMx());
01431
01432     return a;
01433 };
01434
01435 template<class TD>
01436 class iterSparseCMref {
01437 public:
01438
01439     iterSparseCMref(const char *F, int L, wbsparray<TD> &A);
01440
01441     SPIDX_T next(wbsparray<TD> &a) {
01442         if ((++it)+1>=I0.len) { it=-1; return (it+1); }
01443         init(it,a,'r'); return (it+1);
01444     };
01445
01446     wbsparray<TD>& getIter(SPIDX_T i, wbsparray<TD> &a, char ref=0) const;
01447     TD norm2(SPIDX_T i, char tnorm=0) const;
01448
01449     SPIDX_T numIter() const {
01450         if (!IX.len) wblog(FL,
01451             "ERR %s() got non-initialized/empty object",FCT);
01452         return IX.len-1;
01453     };
01454
01455     bool gotLast() const {
01456         if (sSPIDX_T(it)<0 || it+1>=IX.len) wblog(FL,
01457             "ERR %s() not within loop (%d/%d)",FCT,it,IX.len-1);
01458         return (it+2==IX.len);
01459     };
01460
01461     SPIDX_T index() const {
01462         if (sSPIDX_T(it)<0 || it+1>=IX.len) wblog(FL,
01463             "ERR %s() not within loop (%d/%d)",FCT,it,IX.len-1);
01464         return IX[it];
01465     };
01466
01467     mxArray* toMx() const;
01468
01469     SPIDX_T it;
01470
01471     wvector<SPIDX_T> I0;
01472     wvector<SPIDX_T> IX;
01473     wbMatrix<SPIDX_T> IDX;
01474
01475 protected:
01476 private:
01477
01478     unsigned r;
01479     const SPIDX_T *S;

```

```

01484     const TD* data;
01485 };
01486
01487 template <class TD>
01488 mxArray* iterSparseCMref<TD>::toMx() const {
01489
01490     const char *fields[]={ "SIZE", "r", "it", "I0", "IX", "IDX", "data" };
01491     mxArray *a=mxCreateStructMatrix(1,1,7,fields);
01492
01493     mxSetFieldByNumber(a,0, 0, wvector<SPIDX_T>(r,S,'r').toMx());
01494     mxSetFieldByNumber(a,0, 1, numtoMx(r));
01495     mxSetFieldByNumber(a,0, 2, numtoMx(it));
01496     mxSetFieldByNumber(a,0, 3, I0.toMx());
01497     mxSetFieldByNumber(a,0, 4, IX.toMx());
01498     mxSetFieldByNumber(a,0, 5, IDX.toMx());
01499     mxSetFieldByNumber(a,0, 6, wvector<TD>(IDX.dim1,data,'r').toMx());
01500
01501     return a;
01502 };
01503
01504 template <class TD>
01505 iterSparseCMref<TD>::iterSparseCMref(
01506     const char *F, int L, wbsparray<TD> &A) : r(A.SIZE.len) {
01507
01508     unsigned l=r-1; SPIDX_T D;
01509     if (r<2 || !(D=A.SIZE.last())) wblog(F_L,
01510         "ERR %s() got %s (%d)",FCT,SSTR(A),D);
01511
01512     A.Compress(F_L);
01513     S=A.SIZE.data; data=A.D.data; it=-1;
01514
01515     if (A.IDX.dim1!=A.D.len || A.SIZE.len!=A.IDX.dim2)
01516         A.sperror_this(F_LF);
01517
01518     if (A.IDX.dim1==0) {
01519         I0.init(1);
01520         IX.init(1); IX[0]=D;
01521         IDX.init(0,r-1); return;
01522     }
01523     if (A.IDX.dim1==1) {
01524         I0.init(2); I0[1]=1;
01525         IX.init(2); IX[0]=A.IDX(0,1); IX[1]=D;
01526         IDX.init2ref(1,1,A.IDX.data);
01527         return;
01528     }
01529
01530     SPIDX_T i0=0, i=1, k=1, *I=A.IDX.ref(1,1);
01531     const int d2=A.IDX.dim2;
01532     wvector<SPIDX_T> Dg;
01533
01534     for (k=1; i<A.IDX.dim1; ++i, I+=d2) { if (I[0]!=I[-d2]) ++k; }
01535     Dg.init(k); IX.init(k+1);
01536
01537     k=0; I=A.IDX.ref(0,1);
01538     IX[0]=I[0]; I+=d2;
01539
01540     for (i=1; i<A.IDX.dim1; ++i, I+=d2) {
01541         if (I[0]!=I[-d2]) { Dg.data[k++]=i-i0; i0=i; IX[k]=I[0]; }
01542     }
01543     Dg.data[k++]=i-i0; IX[k]=D;
01544
01545     Dg.cumsum_(I0,'N');
01546     A.IDX.getCols(0,A.IDX.dim2-2,IDX);
01547 };
01548
01549 template <class TD>
01550 wbsparray<TD>& iterSparseCMref<TD>::getIter(
01551     SPIDX_T i, wbsparray<TD> &a, char ref) const {
01552
01553     if (i+1>=I0.len) wblog(F_L,
01554         "ERR %s() index out of bounds (%d/%d)",FCT,i,I0.len);
01555
01556     SPIDX_T i0=I0[i], n=I0[i+1]-i0;
01557     if (sSPIDX_T(n)<=0) wblog(F_L,"ERR %s() got I0(%d:%d) !?",FCT,i0,i0+n);
01558
01559     if (ref=='r' || ref=='R') {
01560         a.SIZE.init2ref(r-1,S);
01561         a.D.init2ref(n, data+i0);
01562         a.IDX.init2ref(n, IDX.dim2, IDX.rec(i0));
01563         a.isref=1;
01564     }
01565     else {
01566         a.SIZE.init(r-1,S);
01567         a.D.init(n, data+i0);
01568         a.IDX.init(n, IDX.dim2, IDX.rec(i0));
01569     }
01570

```

```

01571     return a;
01572 };
01573
01574 template <class TD>
01575 TD iterSparseCMref<TD>::norm2(SPIDX_T i, char tnorm) const {
01576
01577     if (i+1>=I0.len) wblog(FL,
01578         "ERR %s() index out of bounds (%d/%d)",FCT,i,I0.len);
01579
01580     SPIDX_T i0=I0[i], n=I0[i+1]-i0;
01581
01582     return Wb::overlap(data+i0,data+i0,n,1,tnorm);
01583 };
01584
01585 template <class T>
01586 inline int checkContract(const char* F, int L,
01587     const wbvector<T> &SA, const ctrIdx &ica,
01588     const wbvector<T> &SB, const ctrIdx &icb,
01589     const wbpem *pfinal=NULL, wbvector<T> *SC=NULL
01590 );
01591
01592 template <class TD>
01593 wbarray<TD>& contract(const char *F, int L,
01594     const wbsparray<TD>&, const wbindex&,
01595     const wbarray<TD>&, const wbindex&, wbarray<TD>&,
01596     const wbpem &pfinal=wbperm(), const TD& afac=1., const TD& cfac=0.
01597 );
01598
01599 template <class TD>
01600 wbarray<TD>& contract(const char *F, int L,
01601     const wbarray<TD>&, const wbindex&,
01602     const wbsparray<TD>&, const wbindex&, wbarray<TD>&,
01603     const wbpem &pfinal=wbperm(), const TD& afac=1., const TD& cfac=0.
01604 );
01605
01606 namespace Wb {
01607
01608 template <class TD>
01609 TD VMatVprod(
01610     const wbsparray<TD> &v1, const wbsparray<TD> &M,
01611     const wbsparray<TD> &v2
01612 );
01613
01614 template <class TD>
01615 wbsparray<TD>& MatProd(
01616     const wbsparray<TD> &A, const wbsparray<TD> &B, wbsparray<TD> &C,
01617     char aflag='N', char bflag='N', TD afac=TD(1.), TD cfac=TD(0.)
01618 );
01619
01620 };
01621
01622 template <class TD> inline
01623 wbarray<TD>& contract(const char *F, int L,
01624     const wbsparray<TD> &A, const char* s1,
01625     const wbsparray<TD> &B, const char* s2, wbsparray<TD> &C,
01626     const wbpem &pfinal=wbperm(), const TD& afac=1., const TD& cfac=0.
01627 ){
01628     return
01629     A.contract(F_L, wbindex(s1,1), B,wbindex(s2,1), C, pfinal,afac,cfac);
01630 };
01631
01632 template <class TD> inline
01633 wbarray<TD>& contract(const char *F, int L,
01634     const wbsparray<TD> &A, const char* s1,
01635     const wbarray<TD> &B, const char* s2, wbarray<TD> &C,
01636     const wbpem &pfinal=wbperm(), const TD& afac=1., const TD& cfac=0.
01637 ){
01638     return
01639     contract(F,L, A,wbindex(s1,1), B,wbindex(s2,1), C, pfinal,afac,cfac);
01640 };
01641
01642 template <class TD> inline
01643 wbarray<TD>& contract(const char *F, int L,
01644     const wbarray<TD> &A, const char* s1,
01645     const wbsparray<TD> &B, const char* s2, wbarray<TD> &C,
01646     const wbpem &pfinal=wbperm(), const TD& afac=1., const TD& cfac=0.
01647 ){
01648     return
01649     contract(F,L, A,wbindex(s1,1), B,wbindex(s2,1), C, pfinal,afac,cfac);
01650 };
01651
01652 template<class TD> inline
01653 void wbsparray<TD>::info(const char* F, int L, const char* istr) const {
01654
01655     SPIDX_T nz=D.len; checkSize(F_L);
01656
01657     if (!D.len) {

```



```

01658     if (SIZE.len) {
01659         wblog(F_L,"sparse %3s %12s (all zero)",
01660             istr && istr[0] ? istr:"", SSTR_(this));
01661     }
01662     else {
01663         wblog(F_L,"sparse %3s (empty)",
01664             istr && istr[0] ? istr:"", SSTR_(this));
01665     }
01666     return;
01667 }
01668
01669 if (isDiag(F_L)) {
01670     if (nz>4) {
01671         wblog(F_L,"sparse %3s %12s (diag)",
01672             istr && istr[0] ? istr:"", SSTR_(this));
01673     }
01674     else if (nz>1) {
01675         wblog(F_L,"sparse %3s %8sdiag( [%s] )",
01676             istr && istr[0] ? istr:"", SSTR_(this), STR(D));
01677     }
01678     else {
01679         wblog(F_L,"sparse %3s %5s(scalar) %-8.4g",
01680             istr && istr[0] ? istr:"", SSTR_(this), double(D[0]));
01681     }
01682     return;
01683 }
01684
01685 if (nz>4) {
01686     wblog(F_L,"sparse %3s %12s (nz=%d @ %-6.3g)",
01687         istr && istr[0] ? istr:"", SSTR_(this), nnz(), sparsity());
01688 }
01689 else {
01690     wblog(F_L,"sparse %3s %12s {%s}",
01691         istr && istr[0] ? istr:"", SSTR_(this), D.toStrf("%.4g", "", "").data);
01692 }
01693 };
01694
01695 template<class TD> inline
01696 void wbsparray<TD>::printdata(const char *istr, const char *fmt0) const {
01697
01698     SPIDX_T i; const char *fmt; wbstring FMT;
01699
01700     if (fmt0 && fmt0[0])
01701         { fmt=fmt0; }
01702     else { FMT.init2Fmt((TD)0); fmt=FMT.data; }
01703
01704     checkSize(FLF);
01705
01706     if (isScalar()) {
01707         printf("%s = %4s (scalar)\n",istr,D.len ? NSTRf(D[0],fmt) : 0);
01708     }
01709     else if (isDiag()) {
01710         printf("%s = diag([",istr);
01711         for (i=0; i<D.len; ++i) printf(" %4s",NSTRf(D[i],fmt));
01712         printf(" ] (D=%ld)\n",D.len);
01713     }
01714     else if (SIZE.len==1) {
01715         printf("%s = [",istr);
01716         for (i=0; i<IDX.dim1; ++i) {
01717             printf(" %d %.8g",IDX(i,1)+1,double(D[i]));
01718         }
01719         printf(" ] (vector, D=%ld)\n",SIZE[0]);
01720     }
01721     else if (SIZE.len==2) {
01722         printf("%s = [ %% ( %ld x %ld )\n",istr,SIZE[0],SIZE[1]);
01723         for (i=0; i<IDX.dim1; ++i) {
01724             printf(" %5d %4d %.8g\n",IDX(i,0)+1,IDX(i,1)+1,double(D[i]));
01725         }
01726         printf("];\n");
01727     }
01728     else { wblog(FL,
01729         "ERR %s() only applies to rank<=2 (%s)",FCT,SSTR_(this));
01730     }
01731 };
01732
01733 template<class TD> inline
01734 unsigned wbsparray<TD>::get2DSize(
01735     const char *F, int L,
01736     SPIDX_T &d1, SPIDX_T &d2, int m_,
01737     char fflag
01738 ) const {
01739
01740     const SPIDX_T* const s=SIZE.data;
01741     unsigned m=0, i=0, r=rank(F_L);
01742
01743     if (isDiag()) wblog(FL,"ERR %s() got DIAG_REP",FCT);
01744

```

```

01745     if (!r) { d1=d2=0; return 0; }
01746
01747     if (m_<0) {
01748         if (r%2 && fflag) wblog(F,L,"ERR sparse::%s() "
01749             "even rank or 2nd argument required (%s)",FCT,SSTR_(this));
01750         else {
01751             SPIDX_T dm=SIZE[0], d=std::sqrt(double(numel()));
01752             for (m=1; m<SIZE.len; ++m) {
01753                 if (dm>=d) break; else dm*=SIZE[m];
01754             }
01755         }
01756     }
01757 }
01758 else { m=m_;
01759     if (m>r) wblog(F,L,"ERR sparse::%s() "
01760         "invalid m=%d (%s)",FCT,m,SSTR_(this));
01761 }
01762
01763 if (m>0) { for (d1=s[i++]; i<m; ++i) { d1*=s[i]; }} else d1=1;
01764 if (m<r) { for (d2=s[i++]; i<r; ++i) { d2*=s[i]; }} else d2=1;
01765
01766 return m;
01767 };
01768
01769 template<class TD> inline
01770 unsigned wbsparray<TD>::get2DIndex(
01771     wbMatrix<SPIDX_T> &IJ, SPIDX_T &d1, SPIDX_T &d2, int m_
01772 ) const {
01773     if (isDiag(FL) || !D.len) {
01774         d1=d2=D.len; IJ.init(d1,2);
01775         for (SPIDX_T *I=IJ.data, i=0; i<d1; ++i, I+=2) { I[0]=I[1]=i; }
01776         return 1;
01777     }
01778     if (!IDX.dim2) sperror_this(FLF);
01779
01780     SPIDX_T i,j,k, n=IDX.dim1;
01781     unsigned r=rank(FL), l=r-1, m=get2DSize(FL,d1,d2,m_);
01782
01783     IJ.init(IDX.dim1,2);
01784
01785     if (!d1 || !d2 || !r) {
01786         if (IJ.data || D.data) wblog(FL,
01787             "ERR %s() got d1=%d, d2=%d, yet S=[%s] (%d) !?",
01788             FCT,d1,d2,STR(SIZE),D.len);
01789         return m;
01790     }
01791 }
01792
01793 SPIDX_T *ij=IJ.data; char e=0;
01794 const SPIDX_T *idx=IDX.data, *s=SIZE.data;
01795
01796 if (!m || m==r) {
01797     for (i=0; i<n; ++i, ij+=2, idx+=r) {
01798         for (j=1, k=idx[j--]; j<r; --j) { k=k*s[j]+idx[j]; };
01799         if (m==r)
01800             { ij[0]=k; if (k>=d1) { e=1; break; }; }
01801         else { ij[1]=k; if (k>=d2) { e=2; break; }; }
01802     }
01803 }
01804 else {
01805     for (i=0; i<n; ++i, ij+=2, idx+=r) {
01806         j=1;
01807         for (k=idx[j--]; j>=m; --j) { k=k*s[j]+idx[j]; };
01808         ij[1]=k;
01809         for (k=idx[j--]; j<r; --j) { k=k*s[j]+idx[j]; };
01810         ij[0]=k;
01811         if (ij[0]>=d1 || ij[1]>=d2) { e=3; break; }
01812     }
01813 }
01814
01815 if (e) wblog(FL,
01816     "ERR %s() index out of bounds (%d: (%d,%d) %dx%d; %s: %d; %d)",
01817     FCT, i+1, ij[0]+1, ij[1]+1, d1,d2,SSTR_(this),m, e
01818 );
01819
01820 return m;
01821 };
01822
01823 template<class TD> inline
01824 wbsparray<TD> & wbsparray<TD>::initIdentity(
01825     const wbvector<SPIDX_T> &S0,
01826     int m,
01827     sSPIDX_T id,
01828     char fflag
01829 ) {
01830     SPIDX_T i,j,k, n=IDX.dim1;
01831     unsigned r=rank(FL), l=r-1, m=get2DSize(FL,d1,d2,m_);
01832
01833     IJ.init(IDX.dim1,2);
01834
01835     if (!d1 || !d2 || !r) {
01836         if (IJ.data || D.data) wblog(FL,
01837             "ERR %s() got d1=%d, d2=%d, yet S=[%s] (%d) !?",
01838             FCT,d1,d2,STR(SIZE),D.len);
01839         return m;
01840     }
01841 }
01842
01843 template<class TD> inline
01844 wbsparray<TD> & wbsparray<TD>::initIdentity(
01845     const wbvector<SPIDX_T> &S0,
01846     int m,
01847     sSPIDX_T id,
01848     char fflag
01849 ) {
01850     SPIDX_T i,j,k, n=IDX.dim1;
01851     unsigned r=rank(FL), l=r-1, m=get2DSize(FL,d1,d2,m_);
01852
01853     IJ.init(IDX.dim1,2);
01854
01855     if (!d1 || !d2 || !r) {
01856         if (IJ.data || D.data) wblog(FL,
01857             "ERR %s() got d1=%d, d2=%d, yet S=[%s] (%d) !?",
01858             FCT,d1,d2,STR(SIZE),D.len);
01859         return m;
01860     }
01861 }

```

```

01832 ){
01833     SPIDX_T d1=1,d2=1;
01834     wbindex I(S0.len); SIZE=S0; init(S0);
01835
01836     get2DSize(FL,d1,d2,m);
01837
01838     if (d1 && d2) { SPIDX_T i=0, n=0;
01839         if (id==0) {
01840             n=MIN(d1,d2); init_nnz(n); D.set(1);
01841             for (; i<n; ++i) ind2sub(i+d1*i,IDX.rec(i));
01842         }
01843         else if (id>0) { SPIDX_T k=id; if (k<d2) {
01844             n=MIN(d1,d2-k); init_nnz(n); D.set(1);
01845             for (; i<n; ++i) ind2sub(i+d1*(i+k),IDX.rec(i));
01846         }}
01847         else if (id<0) { SPIDX_T k=-id; if (k<d1) {
01848             n=MIN(d1-k,d2); init_nnz(n); D.set(1);
01849             for (; i<n; ++i) ind2sub((i+k)+d1*i,IDX.rec(i));
01850         }}
01851     }
01852
01853     if (!D.len && fflag)
01854         wblog(FL,"ERR sparse::%s() to empty (k=%d given %s)",
01855             FCT, id, SSTR_(this)
01856         );
01857
01858     return *this;
01859 };
01860
01861 template<class TD> inline
01862 wbsparray<TD>& wbsparray<TD>::initIdentityB3(
01863     SPIDX_T d1, SPIDX_T d2, SPIDX_T D,
01864     sSPIDX_T i0,
01865     char fflag
01866 ){
01867     SPIDX_T s[3]={d1,d2,D}; SIZE.init(3,s);
01868     return initIdentity(SIZE,2,i0,fflag);
01869 };
01870
01871 template<class TD>
01872 class sparseCollector2D {
01873
01874 public:
01875
01876     sparseCollector2D(SPIDX_T m=0, SPIDX_T n=0)
01877         : M(m), N(n), nnz(0), NIJ(NULL), data(NULL) { SPIDX_T MN=M*N;
01878         if (MN) {
01879             WB_NEW(data,MN); MEM_SET<TD>(data,MN);
01880             WB_NEW(NIJ,MN); memset(NIJ,0,MN*sizeof(SPIDX_T));
01881         }
01882     };
01883
01884     ~sparseCollector2D() { clear(); }
01885
01886     void clear() {
01887         if ((data==NULL) ^ (NIJ==NULL)) wblog(FL,
01888             "ERR %s() got partial initialization (%lX, %lX) !?",FCT,data,NIJ);
01889         if (data) { WB_DELETE(data); }
01890         if (NIJ) { WB_DELETE(NIJ); }
01891         M=N=nnz=0;
01892     };
01893
01894     void init(SPIDX_T m=0, SPIDX_T n=0) { SPIDX_T MN=M*N;
01895         clear(); M=m; N=n; nnz=0;
01896         if (MN) {
01897             WB_NEW(data,MN); MEM_SET<TD>(data,MN);
01898             WB_NEW(NIJ,MN); memset(NIJ,0,MN*sizeof(SPIDX_T));
01899         }
01900     };
01901
01902     sparseCollector2D& add(
01903         const char *F, int L, SPIDX_T i, SPIDX_T j, const TD& a
01904     ){
01905         if (i>=M || j>=N || !data) wblog(F_L,
01906             "ERR %s() index out of bounds (%d,%d)/(%dx%d) [%lX]",
01907             FCT,i,j,M,N,data);
01908         SPIDX_T k=i*N+j;
01909         data[k]+=a; if ((++NIJ[k])==1) ++nnz;
01910         return *this;
01911     };
01912
01913     void getData(wbMatrix<SPIDX_T> &IJ, wbvector<TD> &X) {
01914         SPIDX_T i,j, k=0, l=0;
01915         IJ.init(nnz,2); X.init(nnz);
01916
01917         for (i=0; i<M; ++i)
01918             for (j=0; j<N; ++j, ++l) if (data[l]!=0) {

```

```

01919         if (k>=nnz || !NIJ[l]) wblog(FL,"ERR %s() "
01920             "possibly incorrect nnz (%g/%g; %d) !?",FCT,k,nnz,NIJ[l]);
01921         X.data[k]=data[l];
01922         IJ(k,0)=i; IJ(k,1)=j; ++k;
01923     }
01924     if (nnz && k<nnz) {
01925         if (k) { IJ.dim1=k; X.len=k; }
01926         else {
01927             wblog(FL,"WRN %s() got all-zero data",FCT);
01928             IJ.init(0,2); X.init();
01929         }
01930     }
01931 };
01932
01933 mxArray* toMx() const {
01934     const char *fields[]={"M","N","nnz","data","NIJ"};
01935     mxArray *S=mxCreateStructMatrix(1,1,5,fields);
01936
01937     mxSetFieldByNumber(S,0,0, numtoMx(M));
01938     mxSetFieldByNumber(S,0,1, numtoMx(N));
01939     mxSetFieldByNumber(S,0,2, numtoMx(nnz));
01940     mxSetFieldByNumber(S,0,3, wbMatrix<TD>(M,N,data,'r').toMx());
01941     mxSetFieldByNumber(S,0,4, wbMatrix<SPIDX_T>(M,N,NIJ,'r').toMx());
01942
01943     return S;
01944 };
01945
01946 protected:
01947 private:
01948
01949     SPIDX_T M,N,nnz,*NIJ;
01950     TD* data;
01951 };
01952
01953 #endif
01954

```

5.59 wbstring.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : wbstring
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the license for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_STRING_CC__
00021 #define __WB_STRING_CC__
00022
00023 // ----- //
00024 // string length l_ includes trailing 0 character
00025 // tags: filesize fsize kB MB GB
00026
00027 char* Wb::memsize2Str(double x, char *s, unsigned l_) {
00028     unsigned i=0, l=(int(l_)<0 ? 16 : l_);
00029     if (!s) wblog(FL,"ERR %s() got null string (%d)",FCT,l);
00030     if (l) {
00031         if (x<5E3) { i=snprintf(s,l,"%g bytes",x); } else
00032         if (x<1E6) { i=snprintf(s,l,"%3g kB", x/(1<10)); } else
00033         if (x<1E9) { i=snprintf(s,l,"%3gM", x/(1<20)); } else
00034             { i=snprintf(s,l,"%3gG", x/(1<30)); }
00035     }
00036     if (i>=l) wblog(FL,
00037         "WRN %s() string out of bounds ('%s', %d/%d)",FCT,s,strlen(s),l);
00038     return s;
00039 };
00040
00041 wbstring Wb::repHome(const wbstring &file) {
00042     return Wb::repHome(file.data);
00043 };
00044

```

```

00045 namespace Wb {
00046
00047 template<class T>
00048 wbstring num2Str(const T &x, const char *f, unsigned l) {
00049     wbstring s(int(l)<16 ? 16 : l), fmt;
00050     if (!f || !f[0]) {
00051         fmt.init2Fmt(x,-1,4); f=fmt.data;
00052     }
00053     l=snprintf(s.data,s.len,f,x);
00054     if (l>=s.len) wblog(FL,"ERR %s() string out of bounds "
00055         "%d/%d)\n%s '%s' -> '%s'",FCT,l,s.len,TSTR(T),f,s.data);
00056     return s;
00057 };
00058
00059 template<>
00060 wbstring num2Str(const wbcomplex &x, const char *f,
00061     unsigned l QS_UNUSED_VAR
00062 ){ return x.toString(f && f[0] ? f : "%.4g"); };
00063
00064 #ifdef QS_USING_MPFR
00065 template<>
00066 wbstring num2Str(const Wb::quad &x, const char *f, unsigned l
00067 ){ return num2Str(double(x),f,l); };
00068 #endif
00069
00070 template<>
00071 wbstring num2Str(const int &x, const char *f, unsigned l) {
00072     return int2Str(x,f,l);
00073 };
00074 template<>
00075 wbstring num2Str(const unsigned &x, const char *f, unsigned l) {
00076     return int2Str(x,f,l);
00077 };
00078 template<>
00079 wbstring num2Str(const long &x, const char *f, unsigned l) {
00080     return int2Str(x,f,l);
00081 };
00082 template<>
00083 wbstring num2Str(const unsigned long &x, const char *f, unsigned l) {
00084     return int2Str(x,f,l);
00085 };
00086
00087 template<>
00088 wbstring num2Str(const char &x,
00089     const char *f QS_UNUSED_VAR, unsigned l) {
00090     return char2Str(x,l);
00091 };
00092 template<>
00093 wbstring num2Str(const unsigned char &x,
00094     const char *f QS_UNUSED_VAR, unsigned l) {
00095     return char2Str(x,l);
00096 };
00097
00098 wbstring int2Str(long x, const char *f, unsigned l) {
00099     wbstring s_(l?1:32);
00100     unsigned i=0, j,k=0, m=0, n=0; char *s=s_.data;
00101
00102     l=snprintf(s,s_.len,f && f[0]? f:"%ld",x);
00103     for (; s[i]; ++i) {
00104         if (!isdigit(s[i])) { if (s[i]!='-') ++m; else ++k; }
00105         else {
00106             for (++n, ++i; s[i]; ++i) { if (isdigit(s[i])) ++n; else break; }
00107             break;
00108         }
00109     }
00110     if (s[i] || i!=1 || !n || k>1) wblog(FL,
00111         "ERR %s() invalid int `%s' !?",FCT,s);
00112
00113     i=(n-1)/3;
00114     j=1; if (i>m) { j+=(i-m); }
00115     if (j>=s_.len) wblog(FL,
00116         "ERR %s() string out of bounds (%ld / %d)",FCT,x,s_.len);
00117
00118     if (n>3) { s[j]=0; --n;
00119         for (i=1; i<=l; --n) { if (i && j) {
00120             s[--j]=s[--i];
00121             if (j>1 && n<1 && (l-i)%3==0) { s[--j]=','; }
00122             } else { break; } }
00123     if (j<i) wblog(FL,
00124         "ERR %s() invalid int %ld `%s' (%d/%d/%d) !?",FCT,x,s,i,j,n);
00125     }
00126     return s_;
00127 };
00128
00129 wbstring char2Str(int x, unsigned l) {
00130     wbstring s(l?1:16);
00131

```

```

00132     if (isalnum(x)) l=snprintf(s.data,s.len,"%c",x); else
00133     if (isprint(x)) l=snprintf(s.data,s.len,"%c",x);
00134     else l=snprintf(s.data,s.len,"%d",x);
00135
00136     if (l>=s.len) wblog(FL,
00137         "WRN %s() string out of bounds `%s' (%d/%d)",FCT,s.data,l,s.len);
00138     return s;
00139 };
00140
00141 };
00142
00143 int Wb::findRegex(const char *str, const char *pat, char icase) {
00144
00145     int q=0; regex_t r;
00146
00147     if (!str) wblog(FL,"ERR %s() got null input !?",FCT);
00148     if (!pat || !pat[0]) wblog(FL,
00149         "ERR %s() got invalid regexp (empty)",FCT);
00150
00151     if (strstr(pat,"\\d")) wblog(FL,
00152         "ERR %s() does not support \\d for digits (use [0-9], instead)",FCT);
00153
00154     q=(REG_EXTENDED | REG_NOSUB); if (icase) { q|=REG_ICASE; }
00155     if ((q=regcomp(&r,pat, q)) wblog(FL,
00156         "ERR %s() invalid regexp '%s' (e=%d)",FCT,pat,q);
00157     if (!regexec(&r,str,0,NULL,0)) { q=1; }
00158
00159     regfree(&r);
00160     return q;
00161 };
00162
00163 template<class T>
00164 wbstring& wbstring::operator<< (const T& x) {
00165     wblog(FL,"ERR %s() type `%s' not specialized",FCT,TSTR(x));
00166     return *this;
00167 };
00168
00169 template<class T>
00170 wbstring& wbstring::operator<< (const T* x) {
00171     wblog(FL,"ERR %s() type `%s' not specialized",FCT,TSTR(*x));
00172     return *this;
00173 };
00174
00175 template<>
00176 wbstring& wbstring::operator<< (const char* s) {
00177     return (*this)+=s;
00178 };
00179 template<>
00180 wbstring& wbstring::operator<< (const char& x) {
00181     return (*this)+=Wb::char2Str(x);
00182 };
00183
00184 template<>
00185 wbstring& wbstring::operator<< (const int& x) {
00186     return (*this)+=Wb::int2Str(x);
00187 };
00188 template<>
00189 wbstring& wbstring::operator<< (const unsigned& x) {
00190     return (*this)+=Wb::int2Str(x);
00191 };
00192 template<>
00193 wbstring& wbstring::operator<< (const long& x) {
00194     return (*this)+=Wb::int2Str(x);
00195 };
00196 template<>
00197 wbstring& wbstring::operator<< (const unsigned long& x) {
00198     return (*this)+=Wb::int2Str(x);
00199 };
00200
00201 template<>
00202 wbstring& wbstring::operator<< (const double& x) {
00203     return (*this)+=Wb::num2Str(x);
00204 };
00205 template<>
00206 wbstring& wbstring::operator<< (const float& x) {
00207     return (*this)+=Wb::num2Str(x);
00208 };
00209 template<>
00210 wbstring& wbstring::operator<< (const wcomplex& x) {
00211     return (*this)+=x.toStr();
00212 };
00213
00214 size_t wbstring::toHash(unsigned offset) const {
00215
00216     size_t h=0;
00217
00218     if (offset) {

```

```

00219         if (!data) wblog(F_L,
00220             "ERR %s() got offset=%d for null string",FCT,offset);
00221         if (offset>=len) { wblog(F_L,
00222             "WRN %s() offset out of bounds (%d/%d)",FCT,offset,len);
00223             return h;
00224         }
00225         for (unsigned i=0; i<offset; ++i) { if (!data[i]) wblog(F_L,
00226             "ERR %s() string terminating prior to offset "
00227             "(i=%d/%d, len=%d)\n%s",FCT,i+1,offset,len,data);
00228         }
00229     }
00230
00231     if (data) {
00232         h=Wb::string_hash(data+offset,len-offset); }
00233     return h;
00234 };
00235
00236 size_t Wb::string_hash(const char *s, unsigned n,
00237     size_t hid
00238 ) {
00239     if (s) { unsigned i=0;
00240         if (int(n)<0) {
00241             for (; s[i]; ++i) { hid = ((hid<5) + hid) ^ s[i]; }
00242         }
00243         else {
00244             for (; i<n && s[i]; ++i) { hid = ((hid<5) + hid) ^ s[i]; }
00245             if (s[i]) { wblog(F_L,
00246                 "WRN %s() string not zero-terminated (n=%d)",FCT,n);
00247             }
00248         }
00249     }
00250     else if (n) wblog(F_L,"ERR %s() got n=%d for null string",FCT,n);
00251     return hid;
00252 };
00253
00254 inline wbstring& wbstring::push(
00255     const char *F, int L, const char* s, char extend) {
00256
00257     if (!s || !s[0]) { return *this; }
00258
00259     unsigned n1=(data ? strlen(data) : 0), n2=strlen(s);
00260
00261     if (n1+n2>=len && extend) {
00262         unsigned l=n1+n2+1, m=(l%32); if (m) { l=l+(32-m); }
00263         if (WBLOG_RESIZE) wblog(F_L,
00264             "TST %s() resizing %d -> %d",FCT,len,l);
00265         Resize(l);
00266     }
00267
00268     if (n1+n2<len) { strcpy(data+n1,s); }
00269     else if (!len) wblog(F_L,
00270         "ERR %s() uninitialized string (%d; %p)",FCT,len,data);
00271     else {
00272         if (n1+1<len) { strncpy(data+n1,s,len-n1); data[len-1]=0; }
00273         wblog(F_L,
00274             "WRN %s() string out of bounds (%d+%d/%d)%N%N> %s%N",
00275             FCT,n1,n2,len,data);
00276     }
00277
00278     return *this;
00279 };
00280
00281 wbstring& wbstring::pushf(const char *F, int L, const char *fmt, ...) {
00282
00283     if (fmt && fmt[0]) {
00284         size_t l, n1=strlen(data), n2=len-n1;
00285         if (len<=1) wblog(F_L,"ERR %s() got empty string (%d)",FCT,len);
00286         if (n1>=len) wblog(F_L,
00287             "ERR %s() got string without null-terminator (%d)!",FCT,len);
00288
00289         for (int retry=1; retry<=2; ++retry) {
00290             #pragma omp critical (got_va_list)
00291             { va_list args;
00292                 va_start(args,fmt);
00293                 l=vsnprintf(data+n1,n2,fmt,args);
00294                 va_end(args);
00295             }
00296             if (l<n2) { break; }
00297             if (retry==1) {
00298                 l+=(n1+1); l+=(-1 % (l>128 ? 256 : 64));
00299                 if (WBLOG_RESIZE) wblog(F_L,
00300                     "TST %s() resizing %d -> %d",FCT,len,l);
00301                 Resize(n1+l+1); n2=len-n1;
00302             }
00303             else { data[len-1]=0; wblog(F_L,
00304                 "ERR %s() string out of bounds (%d->%d/%d)%N%N> %s%N",
00305                 FCT,n1,n1+1,len,data);

```

```

00306     }
00307     }
00308     }
00309     return *this;
00310 };
00311
00312 int Wb::String::pushf(const char *fmt, ...) {
00313
00314     if (!fmt || !fmt[0]) { return -1; }
00315     if (len<=1) wblog(FL,
00316         "ERR %s got empty string (%d)",fline? fline:FCT, len);
00317     if (ipos+strlen(fmt)>=len) wblog(FL,
00318         "ERR %s string out of bounds (%d+%d/%d)",fline? fline:FCT,
00319         ipos,strlen(fmt),len);
00320
00321     unsigned ip0=ipos;
00322
00323     #pragma omp critical (got_va_list)
00324     { va_list args;
00325         va_start(args,fmt);
00326         ipos+=vsprintf(data+ipos,len-ipos,fmt,args);
00327         va_end(args);
00328     }
00329
00330     if (ipos>=len) { data[len-1]=0; wblog(FL,
00331         "ERR %s() string out of bounds (%d->%d/%d)%N%N> %s%N",
00332         fline? fline:FCT, ip0,ipos,len,data);
00333     }
00334
00335     return (ipos-ip0);
00336 };
00337
00338 inline int wbstring::printf(const char *F, int L,
00339     const char *fmt, ...
00340 ){
00341     int l=-1;
00342     if (fmt && fmt[0]) {
00343         if (len<=1) wblog(F_L,"ERR %s() got empty string (%d)",FCT,len);
00344
00345         va_list args;
00346         va_start(args,fmt);
00347         l=vsprintf(data,len,fmt,args);
00348         va_end(args);
00349
00350         if (unsigned(l)>=len) { data[len-1]=0; wblog(F_L,
00351             "ERR %s() string out of bounds (%d/%d)%N%N> %s%N",FCT,l,len,data);
00352         }
00353     }
00354     return l;
00355 };
00356
00357 inline wbstring& wbstring::cpy(const char *F, int L,
00358     const char *s
00359 ){
00360     if (s) {
00361         size_t l=strlen(s);
00362         if (l+1>=len) wblog(F_L,
00363             "ERR %s() string out of bounds (%s; %d/%d)",FCT,s,l,len);
00364         if (s[0]) strcat(data,s);
00365         else data[0]=0;
00366     }
00367     else wblog(F_L,"ERR %s() got NULL string",FCT);
00368
00369     return *this;
00370 };
00371
00372 int wbstring::RegEx_replace(
00373     const char *F, int L, const char *pat, const char *rep,
00374     char  icase,
00375     char  gflag
00376 ) {
00377
00378     int rval=0;
00379     if (!data || !data[0]) { return rval; }
00380
00381     auto rflags = std::regex_constants::ECMAScript;
00382     if (icase) { rflags |= std::regex_constants::icase; }
00383
00384     try {
00385         string sout=regex_replace(data,regex(pat,rflags),rep,
00386             gflag ? std::regex_constants::match_default :
00387                 std::regex_constants::format_first_only);
00388         const char *s=sout.data();
00389         rval=strcmp(data,s);
00390         if (rval) { init(s); }
00391     }
00392     catch (...) {

```



```

00393         wblog(F_L, "ERR regex_replace() invalid regex_t = '%s'", pat);
00394     }
00395
00396     return rval;
00397 };
00398
00399 #endif
00400

```

5.60 wbststring.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : wbststring
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the license for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_STRING_HH__
00021 #define __WB_STRING_HH__
00022
00023 namespace Wb {
00024
00025     char* memsize2Str(double x, char *s, unsigned l);
00026
00027     wbststring repHome(const wbststring &file);
00028
00029     int findRegEx(const char *s, const char *pat, char ica=0);
00030
00031     size_t string_hash(const char *s, unsigned n=-1, size_t hid=0);
00032 };
00033
00034 class wbststring : public wbvector<char> {
00035
00036 public:
00037
00038     wbststring (unsigned n=0) : wbvector<char>(n) {};
00039
00040     wbststring (unsigned n, const char c)
00041         : wbvector<char>(n+1) { set(c); data[len-1]=0; };
00042
00043     wbststring (const char* s, unsigned n) { init(s,n); };
00044
00045     wbststring (const wbststring &s) : wbvector<char>() { init(s.data); };
00046     wbststring (const char* s1) : wbvector<char>() { init(s1); };
00047
00048     wbststring (
00049         const char* s1, const char* s2,
00050         const char* s3=NULL, const char* s4=NULL
00051     ) : wbvector<char>() {
00052         unsigned
00053             l1 = (s1? strlen(s1) : 0),
00054             l2 = (s2? strlen(s2) : 0),
00055             l3 = (s3? strlen(s3) : 0),
00056             l4 = (s4? strlen(s4) : 0);
00057
00058         RENEW(l1+l2+l3+l4+1);
00059         if (l1) { strcpy(data, s1); }
00060         if (l2) { strcpy(data+l1, s2); }
00061         if (l3) { strcpy(data+l1+l2, s3); }
00062         if (l4) { strcpy(data+l1+l2+l3, s4); }
00063         data[len-1]=0;
00064     };
00065
00066 #ifndef NOMEX
00067     wbststring(const mxArray *a) : wbvector<char>() { init(0,0,a); };
00068     wbststring& init(const mxArray *a) { return init(0,0,a); }
00069 #endif
00070
00071     wbststring (const char *F, int L, const mxArray *a)
00072         : wbvector<char>() { init(F,L,a); };

```

```

00073
00074 #if 0
00075 ~wbstring() {
00076     if (len) {
00077         if (data[len-1]) {
00078             unsigned i=0; for (; i<len && data[i]; ++i) { };
00079             if (i>=len) { wblog(FL,
00080                 "WRN %s() string out of bounds? (len=%d/%d)",FCT,i,len);
00081                 fprintf(stdout,"\n");
00082                 for (i=0; i<len && data[i]; ++i) fputc(data[i],stdout);
00083                 fprintf(stdout,"\n\n");
00084             }
00085         }
00086     }
00087 };
00088 #endif
00089
00090 wbstring& init(unsigned n=0) { RENEW(n); return *this; };
00091
00092 wbstring& init(const char *s) {
00093     if (s) {
00094         if (s[0])
00095             { RENEW(strlen(s)+1, s); data[len-1]=0; }
00096         else { RENEW(1); data[0]=0; }
00097     } else { RENEW(0); }
00098     return *this;
00099 };
00100
00101 wbstring& init(const char* s,unsigned n) {
00102     unsigned l=(s? strlen(s) : -1);
00103     RENEW(n+1,s,l); data[n]=0;
00104     return *this;
00105 };
00106
00107 wbstring& init(const char *F, int L, const mxArray *a) {
00108     int i,n;
00109     if (!a || !(n=mxGetNumberOfElements(a))) {
00110         RENEW(0); return *this;
00111     };
00112
00113     if (!mxIsChar(a)) wblog(F_L,
00114         "ERR initializing string with type '%s'",mxGetClassName(a));
00115
00116     RENEW(++n);
00117     i=mxGetString(a,data,n);
00118
00119     if (i) wblog(F_L,"ERR failed to read string >%s<",data);
00120     return *this;
00121 };
00122
00123 wbstring& initrep(const char* s, unsigned n) {
00124     if (s && s[0] && n) {
00125         unsigned l=strlen(s); init(l*n+1);
00126         if (l==1) { memset(data,s[0],n); data[len-1]=0; }
00127         else {
00128             unsigned i=0; char *d=data;
00129             for (; i<n; ++i, d+=l) { strcpy(d,s); }
00130         }
00131     }
00132     else { init(); }
00133
00134     return *this;
00135 };
00136
00137 wbstring& initrep(char c, unsigned n) {
00138     if (n) {
00139         if (!isprint(c)) wblog(FL,
00140             "WRN %s() got non-print char=%c<%d>",FCT,c,c);
00141         init(n+1); data[len-1]=0; memset(data,c,n);
00142     }
00143     else { init(); }
00144     return *this;
00145 };
00146
00147 void set(char c) {
00148     if (len) {
00149         memset(data, c, len-1); data[len-1]=0; }
00150 };
00151
00152 wbstring& operator= (const char* s) { return init(s); }
00153 wbstring& operator= (const wbstring &s) { return init(s.data); }
00154
00155 bool gotTerm0() {
00156     for (unsigned i=0; i<len; ++i) { if (data[i]==0) return 1; }
00157     return 0;
00158 };
00159

```

```

00160     bool strchr(const char *s) {
00161         if (data && data[0]) {
00162             if (!s || !s[0]) wblog(FL,"WRN %s() got empty char set !?",FCT);
00163             else { for (; *s; ++s) { if (::strchr(data,*s)) return 1; } }
00164         }
00165         return 0;
00166     };
00167
00168     wbststring& push(const char *F, int L, const char* s, char extend=1);
00169     wbststring& push(const char* s, char extend=1) {
00170         return push(0,0,s,extend); };
00171
00172     wbststring& pushf(const char *F, int L, const char *fmt, ...);
00173     wbststring& cpy(const char *F, int L, const char *s);
00174
00175     int RegEx_replace(const char *F, int L, const char *pat, const char *rep,
00176         char icase=0, char gflag=1);
00177
00178     int printf(const char *F, int L, const char *fmt, ...);
00179
00180     wbststring& time() { return time(::time(NULL)); };
00181     wbststring& time(const time_t &t) { RENEW(32);
00182         #pragma omp critical (got_CPTR_TIME)
00183         { ctime_r(&t,data); }
00184         return *this;
00185     };
00186
00187     wbststring& time_sys() { return time_sys(::time(NULL)); };
00188     wbststring& time_sys(const time_t &t) { RENEW(32);
00189         #pragma omp critical (got_CPTR_TIME)
00190         { strftime(data,31,"%a %b %d %H:%M:%S %Z %Y",localtime(&t)); }
00191         return *this;
00192     };
00193
00194     wbststring& Upper() {
00195         for (unsigned n=strlen(data), i=0; i<n; i++)
00196             data[i]=std::toupper(data[i]);
00197         return *this;
00198     };
00199
00200     wbststring& Lower() {
00201         for (unsigned n=strlen(data), i=0; i<n; i++)
00202             data[i]=std::tolower(data[i]);
00203         return *this;
00204     };
00205
00206     wbststring toupper() const {
00207         wbststring sout(*this); sout.Upper(); return sout; };
00208
00209     wbststring tolower() const {
00210         wbststring sout(*this); sout.Lower(); return sout; };
00211
00212     wbststring& Chomp() {
00213         if (data && data[0]) { int i=strlen(data);
00214             for (; i>=0; --i) {
00215                 if (data[i]>20 && data[i]!=' ') { data[i+1]=0; break; }
00216             }
00217             if (i<0) data[0]=0;
00218         }
00219         return *this;
00220     };
00221
00222     char getopt(char c) {
00223         for (unsigned i=0; data[i]; i++) {
00224             if (data[i]==c) {
00225                 data[i]=-1;
00226                 return 1;
00227             }
00228         }; return 0;
00229     };
00230
00231     char getopt() {
00232         for (unsigned i=0; data[i]; i++) {
00233             if (data[i]>0) { return data[i]; }
00234         }; return 0;
00235     }
00236
00237     const char* basename(const char c=='/') const {
00238         if (!data) return data;
00239         return Wb::basename(data,c);
00240     };
00241
00242     mxArray* toMx() const {
00243         return mxCreateString(data ? data : "(null)");
00244     };
00245
00246     size_t toHash(unsigned offset=0) const;

```

```

00247
00248 void add2MxStruct(mxArray *S, unsigned i) const {
00249     mxSetCell(S,i,toMx());
00250 };
00251
00252 template <class T>
00253 char* init2Fmt(const T& x QS_UNUSED_VAR, int n=-1, int p=-1){
00254     (*this)=wbstring(Wb::num2Fmt<T>(FL,n,p));
00255     return data;
00256 };
00257
00258 template <class T> wbstring& operator<< (const T* x);
00259 template <class T> wbstring& operator<< (const T& x);
00260
00261 wbstring operator+ (const wbstring &s) const { return (*this)+s.data; };
00262 wbstring operator+ (const char *s) const {
00263     wbstring sout;
00264     unsigned n1=data? strlen(data):0, n2=s && s[0]? strlen(s):0;
00265     sout.init(n1+n2);
00266     if (n1) { strcpy(sout.data, data); }
00267     if (n2) { strcpy(sout.data+n1, s); }
00268     return sout;
00269 };
00270
00271 wbstring& operator+=(const char *s) {
00272     if (data && s && s[0]) {
00273         unsigned n1=strlen(data), n2=strlen(s);
00274         if (n2) { Resize(n1+n2); strcpy(data+n1, s); }
00275     }
00276     else if (s) { (*this)=s; }
00277     return *this;
00278 };
00279
00280 wbstring& operator+=(const wbstring &s) {
00281     if (s.data) { (*this)+=s.data; }
00282     return *this;
00283 };
00284
00285 unsigned cat(unsigned &k, const char *s) {
00286     if (s) { unsigned i=0;
00287         for (; s[i]!=0 && k<len; ++i) { data[k++]=s[i]; }
00288         if (s[i]) wblog(FL,"ERR wbstring::cat out of bounds (%d/%d)",k,len);
00289     }
00290     data[k]=0; return k;
00291 };
00292
00293 wbstring toStr(char level=0) const {
00294     if (data) {
00295         if (isPrint()>level)
00296             { return *this; }
00297         else { return wbvector<char>::toStr(); }
00298     }
00299     else { return wbstring("(null)"); }
00300 };
00301
00302 char& operator[] (unsigned i) const { return data[i]; };
00303
00304 unsigned strlen_() const {
00305     return (data && data[0] ? strlen(data) : 0); };
00306
00307 bool isEmpty() const { return (data && data[0] ? 0 : 1); };
00308
00309 char isPrint() const {
00310     if (data) { unsigned i=0, n=0;
00311         for (; i<len && data[i]; ++i) {
00312             if (!isprint(data[i])) { return 0; }
00313             for (; i<len; ++i) { if (data[i]) {
00314                 if (isprint(data[i])) { ++n; } else { return 0; }
00315             } }
00316             return (n ? 2 : 1);
00317         }
00318         return 0;
00319     };
00320
00321 bool operator! () const { return (data && data[0] ? 0 : 1); }
00322 explicit operator bool() const { return (data && data[0] ? 1 : 0); };
00323
00324 unsigned isName(int L=-1) const {
00325     unsigned i, n=strlen(data);
00326     if (L>=0 && L<(int)n) return 0;
00327     for (i=0; i<n; i++) if (!isalnum(data[i])) return 0;
00328     return n;
00329 };
00330
00331 int operator==(const char *s) const {
00332     return (strcmp(data, s)==0);
00333 };

```

```

00334     int operator!= (const char *s) const {
00335         return (strcmp(data, s)!=0);
00336     };
00337
00338     int operator== (const wbsttring &s) const {
00339         return strcmp(data, s.data)==0;
00340     };
00341     int operator!= (const wbsttring &s) const {
00342         return strcmp(data, s.data)!=0;
00343     };
00344     int operator< (const wbsttring &s) const {
00345         return strcmp(data, s.data)<0;
00346     };
00347     int operator<= (const wbsttring &s) const {
00348         return strcmp(data, s.data)<=0;
00349     };
00350     int operator> (const wbsttring &s) const {
00351         return strcmp(data, s.data)>0;
00352     };
00353     int operator>= (const wbsttring &s) const {
00354         return strcmp(data, s.data)>=0;
00355     };
00356
00357     int firstLocation (const wbsttring &, unsigned k=0);
00358     int firstLocation (const char*, unsigned offset=0);
00359
00360     std::istream & getline (std::istream &istr, char c='\n');
00361
00362     protected:
00363     private:
00364 };
00365
00366 std::istream & wbsttring::getline (std::istream &istr, char c) {
00367     const unsigned maxlen=256;
00368     char tmp[maxlen];
00369     istr.getline(tmp, maxlen-1, c);
00370
00371     init(strlen(tmp)+1); strcpy (data, tmp);
00372
00373     return istr;
00374 }
00375
00376 int wbsttring::firstLocation (const wbsttring &s, unsigned k) {
00377     return firstLocation(s.data,k);
00378 }
00379
00380 int wbsttring::firstLocation (const char *s, unsigned offset) {
00381     int i, j, n=strlen(data), m=strlen(s);
00382
00383     for (j=0, i=offset; i<n && j<m; i++, j++) {
00384         if (data[i] != s[j]) {
00385             i-=j;
00386             j=-1;
00387         }
00388     }
00389
00390     if (j==m) return i-m;
00391     else return -1;
00392 }
00393
00394 namespace Wb {
00395     template<class T>
00396     wbsttring num2Str(const T &x, const char *fmt=0, unsigned l=0);
00397
00398     wbsttring int2Str(long x, const char *fmt=0, unsigned l=0);
00399     wbsttring char2Str(int x, unsigned l=0);
00400
00401     wbsttring repStr(const char* s_, unsigned n) {
00402         wbsttring s;
00403         return s.initrep(s_,n);
00404     };
00405
00406     class String : public wbvector<char> {
00407     public:
00408         String(unsigned n)
00409             : wbvector<char>(n), ipos(0), fline(NULL) { if (n) { data[n]=0; } };
00410
00411         String(const char *F, int L, unsigned n)
00412             : wbvector<char>(n), ipos(0), fline(NULL) { if (n) { data[n]=0; }
00413             if (F && F[0]) { fline=shortFL(F,L); }
00414             };
00415
00416         int pushf(const char *fmt, ...);
00417
00418     protected:
00419     private:

```

```

00421
00422     unsigned ipos;
00423     const char *fline;
00424 };
00425
00426 };
00427
00428 #include <queue>
00429
00430 class wblogbuf_struct {
00431
00432 public:
00433
00434     wblogbuf_struct() : file(NULL), line(0), text(NULL) {};
00435
00436     wblogbuf_struct(const char *s) : file(NULL), line(0), text(NULL) {
00437         if (s) {
00438             unsigned n=strlen(s);
00439             WB_NEW(text,n+1); strcpy(text,s);
00440         }
00441     };
00442
00443     wblogbuf_struct(const char *F, int L, const char *s)
00444         : file(NULL), line(L), text(NULL) { unsigned i=0,k=0,n;
00445         if (F) {
00446             for (; F[i]; i++) { if (F[i]=='/' || F[i]=='\\') k=i+1; }
00447             n=strlen(F+k); WB_NEW(file,n+1); strcpy(file,F+k);
00448         }
00449         if (s) { n=strlen(s); WB_NEW(text,n+1); strcpy(text,s); }
00450     };
00451
00452     wblogbuf_struct(const wblogbuf_struct &S)
00453         : file(S.file), line(S.line), text(S.text)
00454     ){
00455     };
00456
00457     ~wblogbuf_struct () {
00458         if (file) { WB_DELETE(file); }
00459         if (text) { WB_DELETE(text); }
00460         line=0;
00461     };
00462
00463     void unset() { file=NULL; line=0; text=NULL; };
00464
00465     char *file; int line; char *text;
00466
00467 protected:
00468 private:
00469 };
00470
00471 class wblogbuffer {
00472
00473 public:
00474
00475     void push(const char *F, int L, const char *s) {
00476         wblogbuf_struct S(F,L,s);
00477         buf.push(S); S.unset();
00478     };
00479
00480     void flush() {
00481         while (!buf.empty()) {
00482             wblogbuf_struct S=buf.front();
00483             wblog(
00484                 S.file ? S.file : __FILE__,
00485                 S.line ? S.line : __LINE__, S.text
00486             );
00487             S.unset();
00488             buf.pop();
00489         }
00490         doflush();
00491     };
00492
00493 protected:
00494 private:
00495
00496     queue<wblogbuf_struct> buf;
00497 };
00498
00499     wblogbuffer wblogBuf;
00500
00501 #endif
00502

```

5.61 wbutil.hh

```

00001  /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : QSpace utility routines
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WBUTIL_HCC__
00021 #define __WBUTIL_HCC__
00022
00023 // NB! util.hh cannot use wstring etc. as members of class
00024 // definitions here since wstring.hh etc. need to be fully
00025 // defined (if an object is used as a member of a class,
00026 // the compiler wants to know its full definition, so the
00027 // forward declarations in wlib.h do not suffice)
00028 // Wb, Sep03, 16
00029
00030 namespace Wb {
00031
00032 class Path {
00033 public:
00034
00035     Path() {} ;
00036     Path(const char *s) { init(FL,s); } ;
00037     Path(const char *F, int L, const char *s) { init(F,L,s); } ;
00038
00039     int init() { buf.init(); idx.init(); return 0; } ;
00040
00041     int init(
00042         const char *F, int L, const char *s, const char *delim=":",
00043         unsigned n_=-1
00044     );
00045
00046     int append(
00047         const char *F, int L, const char *s, const char *delim=":"
00048     );
00049
00050     char* operator[] (unsigned i) {
00051         if (i+1>=idx.len) wblog(FL,
00052             "ERR %s() index out of bounds (%d/%d) !?",FCT,i,idx.len-1);
00053         return buf.data+idx[i];
00054     };
00055     const char* operator[] (unsigned i) const {
00056         if (i+1>=idx.len) wblog(FL,
00057             "ERR %s() index out of bounds (%d/%d) !?",FCT,i,idx.len-1);
00058         return buf.data+idx[i];
00059     };
00060
00061     const char* last() const {
00062         if (idx.len<2) {
00063             if (!idx.len)
00064                 wblog(FL,"ERR %s() path got uninitialized path !?",FCT);
00065             else wblog(FL,"ERR %s() path got path with idx.len=%d !?",FCT,idx.len);
00066         }
00067         return (*this)[idx.len-2];
00068     };
00069
00070     unsigned numel() const {
00071         if (idx.len<2) {
00072             if (!idx.len) wblog(FL,
00073                 "ERR %s() got uninitialized Path (%d,%d)",FCT,idx.len,buf.len);
00074             else wblog(FL,"ERR %s() got Path with idx.len=%d",FCT,idx.len);
00075         }
00076         return (idx.len-1);
00077     };
00078
00079     Wb::iterator<Path,char*> begin() {
00080         return Wb::iterator<Path,char*>(this);
00081     };
00082     Wb::citerator<Path,char*> cbegin() const {
00083         return Wb::citerator<Path,char*>(this);
00084     };
00085

```

```

00086     void print(const char *istr="") const;
00087
00088     wstring buf;
00089     wvector<unsigned> idx;
00090
00091 protected:
00092 private:
00093
00094 };
00095
00096 void Path::print(const char *istr) const {
00097     unsigned i=0, n=numel();
00098
00099     if (istr && istr[0]) wblog(FL,"==> Contents of %s\n",istr);
00100     for (; i<n; ++i) { PRINTF(" %4d. %s\n",i+1,(*this)[i]); }
00101     PRINTF("\n");
00102 };
00103
00104 int Path::init(
00105     const char *F, int L, const char *s, const char *delim,
00106     unsigned n_
00107 ){
00108
00109     if (!s || !s[0]) {
00110         if (int(n_)>0) buf.init(n_); else buf.init();
00111         idx.init(); return 0;
00112     }
00113
00114     if (!delim || !delim[0]) wblog(F_L,
00115         "ERR %s() invalid delim '%s' !?",FCT,delim?delim:"(null)");
00116
00117     unsigned i,m0,m=0, n=strlen(s); char *b,*b0;
00118
00119     if (int(n_)<0) { buf=s; }
00120     else {
00121         if (n_<n) wblog(F_L,"ERR %s() "
00122             "input path length out of bounds %d/%d !?",FCT,n,n_);
00123         buf.init(n_); strcpy(buf.data,s);
00124     }
00125
00126     b0=b=buf.data;
00127
00128     while ((b0=strsep(&b,delim))) { ++m; }
00129
00130     idx.init(m+1); m0=m; b=buf.data;
00131
00132     for (m=i=0; i<=n; ++i) {
00133         idx[m++]=i;
00134
00135         if (!b[i]) continue;
00136
00137         do { ++i; } while (b[i] && i<=n);
00138     }
00139
00140     if (m!=m0) wblog(FL,"ERR %s() m=%d/%d !?",FCT,m,m0);
00141     idx[m]=n;
00142
00143     if (idx.last()>=buf.len) wblog(FL,
00144         "ERR %s() index out of bounds (%d/%d) !?",FCT,idx.last(),buf.len);
00145     return m;
00146 };
00147
00148 int Path::append(
00149     const char *F, int L, const char *s, const char *delim
00150 ){
00151     if (!idx.len) { return init(F,L,s,delim); }
00152
00153     if (!s || !s[0]) { buf.init(); idx.init(); return 0; }
00154     if (!delim || !delim[0]) wblog(FL,
00155         "ERR %s() invalid delim '%s' !?",FCT,delim?delim:"(null)");
00156
00157     unsigned i,l=idx.len-1,m0,m=0, n=strlen(s);
00158     unsigned i0=idx[l]+1;
00159     char *b=buf.data+i0, *b0=b;
00160
00161     if (i0+n>=buf.len) wblog(FL,
00162         "ERR %s() index out of bounds (%d+%d/%d) !?",FCT,i0,n,buf.len);
00163     strcpy(b0,s);
00164
00165     while ((b0=strsep(&b,delim))) { ++m; }
00166
00167     idx.Resize(idx.len+m);
00168     m0=m; b=buf.data+i0;
00169
00170     for (m=i=0; i<=n; ++i) {
00171         idx[l+(m++)]=i0+i;
00172     }

```



```

00173         if (!b[i]) continue;
00174
00175         do { ++i; } while (b[i] && i<=n);
00176     }
00177
00178     if (m!=m0) wblog(FL,"ERR %s() m=%d/%d !?",FCT,m,m0);
00179     idx[l+m]=n;
00180
00181     if (idx.last()>=buf.len) wblog(FL,
00182         "ERR %s() index out of bounds (%d/%d) !?",FCT,idx.last(),buf.len);
00183
00184     return m;
00185 };
00186
00187 double getTimeNow() {
00188
00189     struct timespec now;
00190     int e=clock_gettime(CLOCK_REALTIME,&now);
00191     if (e<0) wblog(FL,"ERR %s() gettimeofday returned error (e=%d)",FCT,e);
00192
00193     return (now.tv_sec + (now.tv_nsec/1e9));
00194 };
00195
00196 template<class T>
00197 void getPerms(wbMatrix<T> &PP,unsigned r, bool l2r=0) {
00198     if (r<=2) {
00199         if (!r) { PP.init(); } else
00200             if (r==1) { PP.init(1,1); }
00201         else {
00202             const T pp[4] = {0, 1, 1, 0};
00203             PP.init(2,2,pp);
00204         }; return;
00205     }
00206     else if (r>12) {
00207         double N=1, i=0; for (; i<r; ++i) { N*=(i+1); }
00208         wblog(FL,"ERR %s() n=%d exceeds bounds (=> %.4g perms)",FCT,r,N);
00209     }
00210
00211     unsigned i,j,l,n,p,k,k_,b=0; size_t N=1;
00212
00213     for (i=0; i<r; ++i) { N*=size_t(i+1); }
00214     PP.init(N,r);
00215
00216     if (l2r) {
00217         for (i=0; i<r; ++i) { PP(0,i)=PP(1,i)=i; };
00218         SWAP(PP(1,0),PP(1,1)); l=2;
00219
00220         for (p=2; p<r; ++p) { n=1;
00221             for (j=0; j<n; ++j, ++b) {
00222                 for (k=0; k<p; ++k, ++l) {
00223                     k_=( b%2 ? k: p-k-1 );
00224
00225                     if (l>=N) wblog(FL,"ERR %s() index out of bounds %d/%d",FCT,l,N);
00226                     for (i=0; i<k_; ++i) { PP(1,i)=PP(j,i); }; PP(1,i)=p;
00227                     for (++i; i<=p; ++i) { PP(1,i)=PP(j,i-1); }
00228                     for ( ; i<r; ++i) { PP(1,i)=i; }
00229                 }}
00230             }
00231         }
00232     else {
00233         for (i=0; i<r; ++i) { PP(0,i)=PP(1,i)=i; };
00234         SWAP(PP(1,i-2),PP(1,i-1));
00235         l=2;
00236
00237         for (p=r-3; p<r; --p) { n=1;
00238             for (j=0; j<n; ++j, ++b) {
00239                 for (k=p+1; k<r; ++k, ++l) {
00240                     k_=( b%2 ? p+r-k : k );
00241
00242                     if (l>=N) wblog(FL,"ERR %s() index out of bounds %d/%d",FCT,l,N);
00243                     for (i=0; i<p; ++i) { PP(1,i)=i; }
00244                     for ( ; i<k_; ++i) { PP(1,i)=PP(j,i+1); }; PP(1,i)=p;
00245                     for (++i; i<r; ++i) { PP(1,i)=PP(j,i); }
00246                 }}
00247             }
00248         }
00249     if (l!=N) wblog(FL,"WRN %s() index mismatch %d/%d",FCT,l,N);
00250 };
00251
00252 class counter {
00253 public:
00254     counter() : n(0) {};
00255
00256     counter(const char *F, int L, const char *fmt_) : n(0) {
00257         file=F; line=L; fmt=fmt_;
00258     };
00259

```

```

00260 ~counter() { if (n) {
00261     if (!fmt) wblog(FL,"ERR %s() got undefined fmt",FCT);
00262     unsigned n=128; char s[n];
00263     snprintf(s,n,fmt.data,n);
00264     if (file)
00265         { wblog(file.data, line,"--> %s",s); }
00266     else { wblog(FL,
00267         "--> %s",s); }
00267     };
00268
00269     counter& operator++() { ++n; return *this; }
00270
00271     long n;
00272     int line;
00273     wstring file, fmt;
00274
00275 protected:
00276 private:
00277 };
00278
00279 };
00280
00281 #endif

```

5.62 wbvector.cc

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : wbvector (template vector class)
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_SIMPLE_VECTOR_CC__
00021 #define __WB_SIMPLE_VECTOR_CC__
00022
00023 /* ----- */
00024 /* ----- */
00025
00026 template <class T>
00027 wbvector<T>& wbvector<T>::set(const wbvector<size_t> &IO, const T &x) {
00028     const size_t *I=IO.data;
00029     for (size_t i=0; i<IO.len; ++i) {
00030         if (I[i]>=len) wblog(FL,
00031             "ERR %s() index out of bounds (%d/%d)",FCT,I[i],len);
00032         data[I[i]]=x;
00033     }
00034     return *this;
00035 };
00036
00037 template <class T>
00038 wbvector<T>& wbvector<T>::add(const wbvector<size_t> &IO, const T &x) {
00039     const size_t *I=IO.data;
00040     for (size_t i=0; i<IO.len; ++i) {
00041         if (I[i]>=len) wblog(FL,
00042             "ERR %s() index out of bounds (%d/%d)",FCT,I[i],len);
00043         data[I[i]]+=x;
00044     }
00045     return *this;
00046 };
00047
00048 template <class T>
00049 bool wbvector<T>::anyUnequal(
00050     const wbvector<size_t> &IO, const T &x, size_t *k
00051 ) const {
00052
00053     const size_t *I=IO.data;
00054     for (size_t i=0; i<IO.len; ++i) {
00055         if (I[i]>=len) wblog(FL,
00056             "ERR %s() index out of bounds (%d/%d)",FCT,I[i],len);
00057         if (data[I[i]]!=x) { if (k) (*k)=I[i]; return 1; }
00058     }

```

```

00059     return 0;
00060 };
00061
00062 template <class T>
00063 bool wbvector<T>::anyEqual(
00064     const wbvector<size_t> &I0, const T &x, size_t *k
00065 ) const {
00066
00067     const size_t *I=I0.data;
00068     for (size_t i=0; i<I0.len; ++i) {
00069         if (I[i]>=len) wblog(FL,
00070             "ERR %s() index out of bounds (%d/%d)",FCT,I[i],len);
00071         if (data[I[i]]==x) { if (k) (*k)=I[i]; return 1; }
00072     }
00073     return 0;
00074 };
00075
00076 template <class T>
00077 wbvector<T>& wbvector<T>::Set(
00078     const wbvector<size_t> &I, const wbvector<T> &v
00079 ){
00080     if (!I.isUnique()) wblog(FL,"ERR index not unique");
00081     if (I.len!=v.len) wblog(FL,
00082         "ERR length mismatch (%d/%d)",I.len,v.len);
00083
00084     for (size_t j,i=0; i<I.len; i++) { j=I[i];
00085         if (j>=len) wblog(FL,
00086             "ERR index out of bounds (%d: %d/%d)",i,j,len);
00087
00088         data[j]=v[i];
00089     }
00090     return *this;
00091 };
00092
00093 template <class T>
00094 bool wbvector<T>::isUnique() const {
00095     wbvector v(*this); wbperr p; Wb::hpsort(v,p);
00096     for (size_t i=1; i<v.len; ++i) if (v[i]==v[i-1]) return 0;
00097     return 1;
00098 };
00099
00100 template <>
00101 bool wbvector<double>::isNormal() const {
00102     for (size_t i=0; i<len; ++i)
00103         if (data[i] && !isnormal(data[i])) return 0;
00104     return 1;
00105 };
00106
00107 template <>
00108 bool wbvector<float>::isNormal() const {
00109     for (size_t i=0; i<len; ++i)
00110         if (data[i] && !isnormal(data[i])) return 0;
00111     return 1;
00112 };
00113
00114 template <> bool wbvector<unsigned>::isNormal() const { return 1; };
00115 template <> bool wbvector<size_t> >::isNormal() const { return 1; };
00116 template <> bool wbvector<int> >::isNormal() const { return 1; };
00117 template <> bool wbvector<char> >::isNormal() const { return 1; };
00118 template <> bool wbvector<long> >::isNormal() const { return 1; };
00119
00120 template <class T>
00121 T wbvector<T>::normDiff2(const wbvector<T> &v, T fac) const {
00122     if (len!=v.len) wblog(FL,
00123         "ERR %s() vector length mismatch (%d/%d)",FCT,len,v.len);
00124     return Wb::rangeNormDiff2(data,v.data,len,fac);
00125 };
00126
00127 template <class T>
00128 double wbvector<T>::normDiff2_(const wbvector<T> &v, double fac) const {
00129     double n2=0; unsigned i=0;
00130
00131     if (len!=v.len) wblog(FL,
00132         "ERR %s() vector length mismatch (%d/%d)",FCT,len,v.len);
00133
00134     if (fac==1) {
00135         for (; i<len; ++i) { n2+=data[i].normDiff2(v.data[i]); }
00136     }
00137     else if (fac!=0) { T x;
00138         for (; i<len; ++i) {
00139             x=v.data[i]; x*=fac; n2+=data[i].normDiff2(x);
00140         }
00141     }
00142     return n2;
00143 };
00144
00145 template <class T>

```

```

00146 T wbvector<T>::cumsum_(wbvector<T> &S, char xflag) const {
00147     T Stot=0;
00148
00149     S.init( xflag? len+1 : len);
00150     if (len>1) {
00151         size_t i=0, l=S.len-1; T *x=S.data;
00152         for (; i<l; ++i) { x[i+1]=x[i]+data[i]; }
00153         Stot=(xflag ? x[l] : x[l]+data[l]);
00154     }
00155     else if (len) {
00156         if (xflag) { S[1]=data[0]; }
00157         Stot=data[0];
00158     }
00159     return Stot;
00160 };
00161
00162 template <class T>
00163 T wbvector<T>::Cumsum(char zflag) const {
00164     if (!len) return 0;
00165     if (zflag) {
00166         T q,x=data[0]; data[0]=0;
00167         for (size_t i=1; i<len; ++i) {
00168             q=data[i]; data[i]=data[i-1]+x; x=q;
00169         }
00170         return data[len-1]+x;
00171     }
00172     else {
00173         for (size_t i=1; i<len; ++i) { data[i]+=data[i-1]; }
00174         return data[len-1];
00175     }
00176 };
00177
00178 template <class T>
00179 template <class T2>
00180 T wbvector<T>::cumsum0prod(wbvector<T2> &b, wbvector<T> &cs, char xflag) const {
00181     size_t i=0, l=(xflag ? len+1 : len);
00182     if (b.len!=len) wblog(FL,
00183         "ERR %s() got length mismatch (%d/%d)",FCT,len,b.len);
00184     cs.init(l); if (!len) return 0;
00185
00186     for (--l; i<l; ++i) { cs[i+1]=cs[i]+data[i]*b.data[i]; }
00187     return (xflag ? cs[i] : cs[i]+data[i]*b.data[i]);
00188 };
00189
00190 template <class T>
00191 wbvector<T>& wbvector<T>::Eldiv(const wbvector<T> &v) const {
00192     if (len!=v.len) wblog(FL,
00193         "ERR %s() size mismatch (%d/%d)",FCT,len,v.len);
00194     for (size_t i=0; i<len; ++i) {
00195         data[i] /= v.data[i];
00196         if (v.data[i]==0) wblog(FL,
00197             "ERR %s() div/0 !? (%g)",FCT,double(data[i])
00198         );
00199     }
00200     return *this;
00201 };
00202
00203 template <class T>
00204 wbvector<T>& wbvector<T>::tensorProd(
00205     const wbvector<T> &v2, wbvector<T> &vv) const {
00206
00207     if (&vv==this || &vv==&v2) {
00208         wbvector<T> x;
00209         return this->tensorProd(v2,x).save2(vv);
00210     }
00211
00212     vv.init(len*v2.len);
00213     if (vv) {
00214         size_t i, j=0, l=0;
00215         const T *d2=v2.data; T *d=vv.data;
00216
00217         for (; j<v2.len; ++j) {
00218             for (i=0; i<len; ++i, ++l) { d[l]=data[i]*d2[j]; }
00219         }
00220         return vv;
00221     };
00222
00223 template <class T>
00224 wbvector<T>& wbvector<T>::RevertSigns() {
00225
00226     size_t i=0;
00227     for (; i<len; ++i) { if (data[i]) break; }
00228     if (i<len) {
00229         if (data[i]>0 && (-data[i])>0) wblog(FL,"ERR %s() "
00230             "got unsigned data type %s",FCT,TSTR(T));
00231         for (; i<len; ++i) data[i]=-data[i];
00232     }

```

```

00233     return *this;
00234 };
00235
00236 template <class T>
00237 void wbvector<T>::selectSU(const WBINDEX &I) {
00238     size_t i,n;
00239     WBINDEX mark(len);
00240     for (i=0; i<I.len; i++) {
00241         if (I[i]>=len)
00242             wblog(FL,"ERR index out of bounds (%d/%d).", i,len);
00243         mark[I[i]]++;
00244     }
00245     for (n=i=0; i<mark.len; i++)
00246         if (mark[i]) mark[n++]=i;
00247
00248     mark.Resize(n);
00249     Select(mark);
00250 };
00251
00252 template <class T>
00253 wbvector<T>& wbvector<T>::select(
00254     const WBINDEX &I, wbvector<T> &a) const {
00255
00256     a.init(I.len); if (!I.len) { return a; }
00257
00258     for (size_t i=0; i<a.len; ++i) {
00259         if (I[i]>=len) wblog(FL,
00260             "ERR %s() index out of bounds (%d/%d)",FCT,I[i],len);
00261         a.data[i]=data[I[i]];
00262     }
00263     return a;
00264 };
00265
00266 template <class T>
00267 void wbvector<T>::select(const WBINDEX &I, T* r) const {
00268     if (I.len==0) return;
00269
00270     for (size_t i=0; i<I.len; i++) {
00271         if (I[i]>=len) wblog(FL,
00272             "ERR Index out of bounds (%d/%d).", I[i],len);
00273
00274         r[i]=data[I[i]];
00275     }
00276 };
00277
00278 template <class T>
00279 wbvector<T>& wbvector<T>::Select(const WBINDEX &I) {
00280
00281     if (isref) wblog(FL,"ERR must not resize vector reference!");
00282     size_t i; T* d0=data;
00283
00284     if (I.len==0) { init(); return *this; }
00285     WB_NEW(data,I.len);
00286
00287     for (i=0; i<I.len; ++i) {
00288         if (I[i]>=len) wblog(FL,
00289             "ERR index out of bounds (%d/%d)", I[i],len);
00290         data[i]=d0[I[i]];
00291     }
00292
00293     len=I.len;
00294     WB_DELETE(d0); return *this;
00295 };
00296
00297 template <class T>
00298 wbvector<T>& wbvector<T>::BlockSelect(
00299     const WBINDEX &I, const WBINDEX &D
00300 ){
00301     WBINDEX dc;
00302     size_t i,j,l,n; T *d0=data, *d;
00303
00304     if (isref) wblog(FL,"ERR shall not resize vector reference!");
00305     if (I.len==0) { init(); return *this; }
00306
00307     for (n=i=0; i<I.len; i++) {
00308         j=I[i]; if (j>=D.len) wblog(FL,
00309             "ERR index out of bounds (%d/%d).",j,D.len);
00310         n+=D[j];
00311     }
00312
00313     i=D.cumsum_(dc);
00314     if (i!=len) wblog(FL,"ERR severe size mismatch (%d/%d; %d)",i,len,n);
00315
00316     WB_NEW(data,n); len=n;
00317
00318     for (n=l=i=0; i<I.len; ++i, l+=n) { j=I[i];
00319         d=d0+dc[j]; n=D[j];

```

```

00320         for (j=0; j<n; j++) data[l+j]=d[j];
00321     }
00322
00323     WB_DELETE(d0); return *this;
00324 };
00325
00326 template <class T>
00327 template <class T1, class T2, class T3, class T4>
00328 wbvector<T>& wbvector<T>::Cat(
00329     const wbvector<T1> *v1, const wbvector<T2> *v2,
00330     const wbvector<T3> *v3, const wbvector<T3> *v4
00331 ){
00332     return Cat(
00333         v1 ? v1->data : (T1*)0, v1 ? v1->len : 0,
00334         v2 ? v2->data : (T2*)0, v2 ? v2->len : 0,
00335         v3 ? v3->data : (T3*)0, v3 ? v3->len : 0,
00336         v4 ? v4->data : (T4*)0, v4 ? v4->len : 0
00337     );
00338 };
00339
00340 template <class T>
00341 template <class T1, class T2, class T3, class T4>
00342 wbvector<T>& wbvector<T>::Cat(
00343     const T1* v1, size_t l1, const T2* v2, size_t l2,
00344     const T3* v3, size_t l3, const T4* v4, size_t l4
00345 ){
00346     wbvector<T> X(l1+l2+l3+l4);
00347     size_t i; T *x=X.data;
00348
00349     for (i=0; i<l1; ++i) { x[i]=T(v1[i]); }; x+=l1;
00350     for (i=0; i<l2; ++i) { x[i]=T(v2[i]); }; x+=l2;
00351     for (i=0; i<l3; ++i) { x[i]=T(v3[i]); }; x+=l3;
00352     for (i=0; i<l4; ++i) { x[i]=T(v4[i]); }
00353
00354     return X.save2(*this);
00355 };
00356
00357 template <class T>
00358 wbvector<T>& wbvector<T>::Cat(const wbvector<wbvector<T> > &vv) {
00359     size_t i,n=0; T *d;
00360
00361     for (i=0; i<vv.len; i++) n+=vv[i].len;
00362     init(n); d=data;
00363
00364     for (i=0; i<vv.len; i++) {
00365         memcpy(d,vv[i].data,sizeof(T)*vv[i].len); d+=vv[i].len;
00366     }
00367     return *this;
00368 };
00369
00370 template <class T>
00371 wbvector<T>& wbvector<T>::Cat(const wbvector<wbvector<T>* > &vv) {
00372     size_t i,n=0; T *d;
00373
00374     for (i=0; i<vv.len; i++) n+=(vv[i] ? vv[i]->len : 0);
00375     init(n); d=data;
00376
00377     for (i=0; i<vv.len; i++) { if (vv[i]) {
00378         const wbvector<T> &v=*vv[i];
00379         memcpy(d,v.data,sizeof(T)*v.len); d+=v.len;
00380     } }
00381     return *this;
00382 };
00383
00384 template <class T>
00385 void wbvector<T>::info(const char *istr) const {
00386
00387     size_t l=0, n=32; char sstr[n];
00388     size_t b=len*sizeof(T);
00389     wbsttring tstr;
00390
00391     if (typeid(T)==typeid(double)) tstr="double";
00392     else tstr.init(TSTR(T));
00393
00394     if (b<(1<10)) l=snprintf(sstr,n,"%ld ", b); else
00395     if (b<(1<20)) l=snprintf(sstr,n,"%3g kB",b/double(1<10)); else
00396     l=snprintf(sstr,n,"%3g MB",b/double(1<20));
00397     if (l>=n) wblog(FL,"ERR %s() string out of bounds (%d/%d)",FCT,l,n);
00398
00399     printf(" %-12s %10ld %12s @ %p %s vector%s\n",
00400         istr, len, sstr, (void*)data, tstr.data, isref ? " ***ISREF***" : "");
00401 }
00402
00403 template <class T>
00404 void wbvector<T>::print(const char *istr, char mflag) const {
00405
00406     mxArray *a;

```

```

00407
00408     if (mflag==1) {
00409         char fmt[8];
00410
00411         if (typeid(T)==typeid(double))
00412             strcpy(fmt," %8g");
00413         else strcpy(fmt," %4d");
00414
00415         if (istr[0]) printf("%s:",istr);
00416
00417         for (size_t i=0; i<len; i++) printf(fmt,data[i]);
00418         printf("%s\n", isref ? " ***ISREF***" : "");
00419
00420         return;
00421     }
00422
00423     a=mxCreateDoubleMatrix(1,len,mxREAL);
00424
00425     double *d=mxGetDoubles(a);
00426     for (size_t i=0; i<len; i++) { d[i]=(double)data[i]; }
00427
00428     if (!mflag) {
00429         if (istr[0])
00430             { wb_printf("\n%s = [1x%d double]\n\n", istr, len); }
00431         else { wb_printf("\n"); }
00432     }
00433     else {
00434         wb_printf("\n%s = [%s\n",
00435             istr[0]? istr:"ans", isref? " *isref*:"");
00436     }
00437
00438     Wb::CallMatlab(0,NULL,1,&a,"disp");
00439
00440     if (mflag>1) wb_printf("];\n");
00441
00442     mxDestroyArray(a);
00443 };
00444
00445 template <class T> inline
00446 wbindex& wbvector<T>::find(const T &x, wbindex &I, char iflag) const {
00447
00448     if (!len) { I.init(); }
00449     else {
00450         size_t i=0, l=0;
00451         for (; i<len; ++i) { if (data[i]==x) ++l; }
00452
00453         if (!iflag) { I.init(1); if (I.len) { l=i=0;
00454             for (; i<len; ++i) { if (data[i]==x) I[l++]=i; } }
00455         }
00456         else { I.init(len-1); if (I.len) { l=i=0;
00457             for (; i<len; ++i) { if (data[i]!=x) I[l++]=i; } }
00458         }
00459     }
00460     return I;
00461 };
00462
00463 template <class T> inline
00464 wbindex& wbvector<T>::find(const T &x, wbindex &I, wbindex &Ix) const {
00465
00466     if (!len) { I.init(); Ix.init(); }
00467     else {
00468         size_t i=0, l=0, k=0;
00469         for (; i<len; ++i) { if (data[i]==x) ++l; }
00470         I.init(1); Ix.init(len-1); l=i=0;
00471         for (; i<len; ++i) { if (data[i]==x) I[l++]=i; else Ix[k++]=i; }
00472     }
00473     return I;
00474 };
00475
00476 template <class T> inline
00477 wbindex& wbvector<T>::findGT(const T &x, wbindex &I, char iflag) const {
00478
00479     if (!len) { I.init(); }
00480     else {
00481         size_t i=0, l=0;
00482         for (; i<len; ++i) { if (data[i]>x) ++l; }
00483
00484         if (!iflag) { I.init(1); if (I.len) { l=i=0;
00485             for (; i<len; ++i) { if (data[i]>x) I[l++]=i; } }
00486         }
00487         else { I.init(len-1); if (I.len) { l=i=0;
00488             for (; i<len; ++i) { if (!(data[i]>x)) I[l++]=i; } }
00489         }
00490     }
00491     return I;
00492 };
00493

```

```

00494 template <class T> inline
00495 wbindex& wbvector<T>::findGT(const T &x, wbindex &I, wbindex &Ix) const {
00496
00497     if (!len) { I.init(); Ix.init(); }
00498     else {
00499         size_t i=0, l=0, k=0;
00500         for (; i<len; ++i) { if (data[i]>x) ++l; };
00501         I.init(l); Ix.init(len-l); l=i=0;
00502         for (; i<len; ++i) { if (data[i]>x) I[l++]=i; else Ix[k++]=i; }
00503     }
00504     return I;
00505 };
00506
00507 template <class T> inline
00508 size_t wbvector<T>::numZeros(T eps) const {
00509     size_t n=0, i=0;
00510     if (!eps) {
00511         for (; i<len; ++i) { if (!data[i]) { ++n; } }
00512     }
00513     else if (eps>0) {
00514         for (; i<len; ++i) { if (Wb::abs(data[i])<eps) { ++n; } }
00515     }
00516     else {
00517         wblog(FL,"WRN %s() got eps=%3g (-> 0) !?",FCT,double(eps));
00518         for (; i<len; ++i) { if (!data[i]) { ++n; } }
00519     }
00520     return n;
00521 };
00522
00523 template <class T> inline
00524 size_t wbvector<T>::numEQ(const T &x) const {
00525     size_t n=0, i=0;
00526     for (; i<len; ++i) { if (data[i]==x) ++n; }
00527     return n;
00528 };
00529
00530 template <class T> inline
00531 size_t wbvector<T>::numGT(const T &x) const {
00532     size_t n=0, i=0;
00533     for (; i<len; ++i) { if (data[i]>x) ++n; }
00534     return n;
00535 };
00536
00537 template <class T> inline
00538 size_t wbvector<T>::numLT(const T &x) const {
00539     size_t n=0, i=0;
00540     for (; i<len; ++i) { if (data[i]<x) ++n; }
00541     return n;
00542 };
00543
00544 template <class T> inline
00545 wbindex& wbvector<T>::findRange(
00546     const T &x1, const T &x2, wbindex &I, const char *w
00547 ) const {
00548
00549     if (len==0) { I.init(); return I; }
00550
00551     bool i1=1,i2=1; size_t k=0;
00552     if (w) {
00553         if ((w[0]!='[' && w[0]!='']') || (w[1]!='[' && w[1]!='']') || w[2])
00554             wblog(FL,"ERR invalid interval specification '%s'",w);
00555         i1=(w[0]=='[');
00556         i2=(w[1]==']');
00557     }
00558
00559     I.init(len);
00560     for (size_t i=0; i<len; i++) { const T &x=data[i];
00561         if (x<x1 || x>x2 || (x==x1 && !i1) || (x==x2 && !i2)) continue;
00562         I[k++]=i;
00563     }
00564     if (k) I.len=k; else I.init();
00565
00566     return I;
00567 };
00568
00569 template <class T>
00570 size_t wbvector<T>::findValues(const wbvector<T> &B0, wbindex &Ia) const {
00571
00572     size_t l=0,i=0,j=0, m=len, n=B0.len;
00573     char c;
00574
00575     wbvector<T> A(*this), B(B0);
00576     wbperm P1,P2; wbindex I;
00577     T *a, *b;
00578
00579     if (A.isEmpty() || B.isEmpty()) { Ia.init(); return 0; }
00580

```



```

00581     A.Sort(P1);
00582     B.Sort(P2); I.init(m); a=A.data; b=B.data;
00583
00584     while (i<m && j<n) { c=NUMCMP(a[i],b[j]);
00585         if (c<0) i++; else
00586             if (c>0) j++;
00587         else {
00588             do { I[l++]=i++; } while (i<m && a[i-1]==a[i]);
00589             do { j++; } while (j<n && b[j-1]==b[j]);
00590         }
00591     }
00592
00593     if (l) I.len=l; else I.init();
00594
00595     P1.get(I,Ia);
00596
00597     return Ia.len;
00598 }
00599
00600 template <class T> inline
00601 wbvector<T>& wbvector<T>::getI(size_t k, wbvector<T> &v) const {
00602
00603     if (k>=len) wblog(FL,
00604         "ERR Index out of bounds (%d/%d) !?",k,len);
00605
00606     v.init(len-l);
00607     for (size_t l=0, i=0; i<len; i++) if (i!=k) v.data[l++]=data[i];
00608     return v;
00609 };
00610
00611 template <class T> inline
00612 wbvector<T>& wbvector<T>::getI(
00613     const wbindex &I, wbvector<T> &v, char uflag
00614 ) const {
00615
00616     wbindex J; I.invert(len,J,uflag);
00617     return select(J,v);
00618 };
00619
00620 template <class T> inline
00621 wbvector<T>& wbvector<T>::initI(
00622     const wbvector<T> &a, const wbindex &ia,
00623     const wbvector<T> &b, const wbindex &ib, char uflag
00624 ){
00625     if (!a.len && !b.len) {
00626         if (ia.len || ib.len) wblog(FL,
00627             "ERR %s() index out of bounds (%d+%d)",FCT,ia.len,ib.len);
00628         return init();
00629     }
00630
00631     size_t i=0, j=0, na=a.len, nb=b.len, n=na+nb;
00632     char ma[n], *mb=ma+na; memset(ma,0,n*sizeof(char));
00633
00634     if (uflag) {
00635         for (; i<ia.len; ++i) { j=ia[i];
00636             if (j>=na) wblog(FL,
00637                 "ERR %s() index out of bounds (%d/%d)",FCT,j+1,a.len);
00638             if ((++ma[j])>1) wblog(FL,
00639                 "ERR %s() index not unique (%d/%d)",FCT,j+1,a.len);
00640         };
00641     }
00642     for (i=0; i<ib.len; ++i) { j=ib[i];
00643         if (j>=nb) wblog(FL,
00644             "ERR %s() index out of bounds (%d/%d)",FCT,j+1,b.len);
00645         if ((++mb[j])>1) wblog(FL,
00646             "ERR %s() index not unique (%d/%d)",FCT,j+1,b.len);
00647     };
00648     na-=ia.len; nb-=ib.len;
00649 }
00650
00651 else {
00652     for (; i<ia.len; ++i) { j=ia[i];
00653         if (j>=a.len) wblog(FL,
00654             "ERR %s() index out of bounds (%d/%d)",FCT,j+1,a.len);
00655         if ((++ma[j])==1) --na; else ma[j]=1;
00656     }
00657     for (i=0; i<ib.len; ++i) { j=ib[i];
00658         if (j>=nb) wblog(FL,
00659             "ERR %s() index out of bounds (%d/%d)",FCT,j+1,b.len);
00660         if ((++mb[j])==1) --nb; else mb[j]=1;
00661     }
00662 }
00663
00664     init(na+nb);
00665
00666     if (len) { j=0;
00667         if (na) {

```

```

00668         for (i=0; i<a.len; ++i) { if (!ma[i]) { data[j]=a[i]; ++j; } }
00669     }
00670     if (nb) {
00671         for (i=0; i<b.len; ++i) { if (!mb[i]) { data[j]=b[i]; ++j; } }
00672     }
00673     if (j!=len) wblog(FL,"ERR %s() %d/%d !?",FCT,j,len);
00674 }
00675
00676 return *this;
00677 };
00678
00679 template <class T>
00680 template <class T2>
00681 wbvector<T>& wbvector<T>::initT(
00682     const char *F, int L, const wbvector<T2> &v) {
00683
00684     RENEW(v.len);
00685     for (size_t i=0; i<len; ++i) {
00686         data[i]=T(v.data[i]);
00687         if (T2(data[i])!=v.data[i]) wblog(F_L,
00688             "ERR %s() got rounded value: %g (%s) -> %g (%s)",
00689             FCT, double(data[i]), TSTR(T2),
00690             double(v.data[i]), TSTR(T)
00691         );
00692     }
00693     return *this;
00694 };
00695
00696 template <class T>
00697 template <class TD> inline
00698 wbvector<T>& wbvector<T>::initT(const wbspparray<TD> &S) {
00699
00700     if (!S.isVector(FL)) wblog(FL,
00701         "ERR %s() invalid sparse vector (%s)",FCT,S.sizeStr().data);
00702
00703     size_t i=0, j=0, n=S.IDX.dim1, m=S.IDX.dim2;
00704     const SPIDX_T *idx=S.IDX.data;
00705
00706     for (; i<S.SIZE.len; ++i) { if (S.SIZE[i]>1) { idx+=i; break; } }
00707
00708     RENEW(S.numel());
00709
00710     for (i=0; i<n; ++i, idx+=m) { j=(*idx);
00711         if (j>=len) wblog(FL,
00712             "ERR %s() index out of bounds (%d/%d)",FCT,j+1,len);
00713         data[j]=T(S.D.data[i]);
00714     }
00715
00716     return *this;
00717 };
00718
00719 bool mxIsWbvector(
00720     const char *F, int L, const mxArray *a,
00721     size_t *n_, const char *istr, char dflag
00722 ){
00723     size_t n=0;
00724
00725     if (a && (n=mxGetNumberOfElements(a))) { int i=0;
00726         if (istr && istr[0])
00727             i=Mx::IsDblVector(0,0,a,0,dflag);
00728         else i=Mx::IsDblVector(F,L,a,0,dflag);
00729
00730         if (!i) {
00731             if (F) wblog(F,L,
00732                 "ERR %s() %s%snnumeric vector required\n%s (%s; %d)", FCT,
00733                 istr?istr:"", istr?" ":"", str, mxGetClassName(a), n);
00734             return 0;
00735         }
00736     }
00737
00738     if (n_) { (*n_)=n; }
00739     return n;
00740 };
00741
00742 template <class T>
00743 int wbvector<T>::init_base(
00744     const char *F, int L,
00745     const mxArray *a, const char *istr, char tcheck, char ref
00746 ){
00747     size_t n=0;
00748
00749     if (!a || !mxIsWbvector(F_L,a,&n,istr,'*')) {
00750         init(); return 1;
00751     }
00752
00753     Mx::Array<T> A(a);
00754     if (ref && A.data)

```

```

00755         { init2ref(n,A.data); }
00756     else { init(n); A.copyTo(data,tcheck); }
00757
00758     if (data && !Wb::is_finite(data,1)) wblog(PFL,
00759         "WRN %s() encountered nan or inf in mex input data",FCT);
00760
00761     return 0;
00762 };
00763
00764 #ifdef QS_USING_MPFR
00765
00766 template<>
00767 int wbvector<Wb::quad>::init_mpfr(
00768     const char *F, int L, const mxArray *a, char base,
00769     const char *istr, char tcheck
00770 ){
00771     if (!a) { init();
00772         if (F) wblog(FL,"ERR %s() got null mxArray",FCT);
00773         return -1;
00774     }
00775     if (!mxIsNumChar(a)) { init();
00776         if (F) wblog(FL,
00777             "ERR %s() invalid type `%s'",FCT,mxGetClassName(a));
00778         return -2;
00779     }
00780     if (mxIsEmpty(a)) { init(); return 0; }
00781
00782     if (base<=0 && mxIsWbvector(0,0,a) && mxIsDouble(a)) {
00783         wbvector<double> X; X.init(F_L,a,istr,tcheck);
00784         this->initT(X); return 0;
00785     }
00786
00787     if (base<=0) base=30; else
00788     if (base==1 || base>62) wblog(FL,
00789         "ERR %s() invalid base=%d<%c>\n"
00790         "(interpreted as base for Wb::quad)",FCT,base,base);
00791
00792     unsigned n=0, np=1; int e=0;
00793
00794     unsigned r=mxGetNumberOfDimensions(a);
00795     const size_t *sz=mxGetDimensions(a);
00796
00797     wbvector<char> ss; char *sd=NULL;
00798
00799     if (r!=2) {
00800         if (F) wblog(FL,"ERR invalid rank-%d array",r);
00801         init(); return -3;
00802     }
00803     if (!sz[0] || sz[0]>62 || (sz[0]==1 && (!sz[1] || sz[1]>62))) {
00804         if (F) wblog(FL,"ERR invalid %ldx%ld array",sz[0],sz[1]);
00805         init(); return -4;
00806     }
00807
00808     if (mxIsInt8(a)) {
00809         Mx::Array<char> A(FL,a);
00810         sd=A.data;
00811
00812         if (sz[0]>1)
00813             { n=sz[0]; init(sz[1]); }
00814         else { n=sz[1]; init(sz[0]); }
00815     }
00816     else if (mxIsChar(a) && (sz[0]==1 || sz[1]==1)) {
00817         Mx::Array<mxChar> A(FL,a);
00818
00819         if (sz[0]>1)
00820             { n=sz[0]; init(sz[1]); }
00821         else { n=sz[1]; init(sz[0]); }
00822
00823         ss.initT(n,A.data);
00824         sd=ss.data;
00825     }
00826     else {
00827         MXPput(FL,"Idbg")
00828             .addP(mxDuplicateArray(a),"a").add(wbstring(istr),"istr");
00829         wblog(FL,"ERR %s() invalid input (%s: %d x %d)",
00830             FCT,mxGetClassName(a),sz[0],sz[1]);
00831     }
00832
00833     if (len>=64) {
00834         np=MAX( QSP_NUM_THREADS, OMP_NUM_THREADS );
00835     }
00836
00837     #pragma omp parallel for num_threads(np)
00838     for (size_t k=0; k<len; ++k) { if (!e) {
00839
00840         const char *sk=sd+k*n; char cx=0;
00841         unsigned i=0, i0=0;

```

```

00842
00843     try {
00844         while (i<n && sk[i]!=' ') { ++i; }; i0=i;
00845         while (i<n && sk[i]!=' ' && sk[i]) { ++i; }
00846
00847         if (i0<i && i<n) {
00848             cx=sk[i]; ((char*)sk)[i]=0;
00849             data[k].init_s(F_L,sk+i0,base); ((char*)sk)[i]=cx;
00850         }
00851         else { char s[n+1]; ++e; strncpy(s,sk,n); s[n]=0;
00852             wblog(FL,"... error i=%4ld/%ld: '%s' (%d..%d/%d)",
00853                 k,len,s,i0,i,n);
00854         }
00855     }
00856     catch (...) {
00857         ++e; if (cx) { ((char*)sk)[i]=cx; }
00858     }
00859 }
00860 if (e) wblog(FL,"ERR %s() e=%d",FCT,e);
00861
00862 return 0;
00863 };
00864
00865 template<>
00866 int wbvector<Wb::quad>::init(
00867     const char *F, int L,
00868     const mxArray *a, const char *istr, char tcheck,
00869     char ref QS_UNUSED_VAR
00870 ){
00871     unsigned base=(tcheck<2 ? 30 : tcheck);
00872
00873     if (base>30) wblog(FL,
00874         "WRN %s() using tcheck=%d as mpfr base !?",FCT,tcheck);
00875
00876     return init_mpfr(F,L,a,base,istr,tcheck);
00877 };
00878
00879 #endif
00880
00881 template <class T>
00882 int wbvector<T>::init_Struct(
00883     const char *F, int L, const mxArray *a, const char *istr, char ref
00884 ){
00885     if (!a || !mxIsStruct(a)) { if (F) wblog(F,L,
00886         "ERR %s() invalid input %s",FCT,istr?istr:"");
00887         init(); return 1;
00888     }
00889
00890     unsigned i=0, n=mxGetNumberOfElements(a);
00891     init(n);
00892
00893     for (; i<n; ++i) { data[i].init(F_L,a,ref,i); }
00894
00895     return 0;
00896 };
00897
00898 template <class T> inline
00899 void wbvector<T>::Sort(char dir) {
00900     wbpperm p; Wb::hpsort(*this,p,dir);
00901 };
00902
00903 template <class T> inline
00904 void wbvector<T>::sort(wbvector &v2) const {
00905     wbpperm p; v2=(*this); Wb::hpsort(v2,p);
00906 };
00907
00908 template <class T> inline
00909 wbpperm& wbvector<T>::pSort(wbpperm &p) const {
00910     if (!isSorted()) { wbvector x(*this); Wb::hpsort(x,p); }
00911     else p.init();
00912     return p;
00913 };
00914
00915 template <class T>
00916 inline wbvector<T>& wbvector<T>::Permute(const wbpperm &P, char iflag) {
00917     const wperm_t *p=P.data; char q=0;
00918
00919     if (!P.len || (P.len<=len && (q=P.isValidPerm())>1)) { return *this; }
00920
00921     if (P.len!=len) {
00922         wbpperm P_; P_.init(P,iflag,len);
00923         return Permute(P_);
00924     }
00925
00926     if (q) {
00927         size_t i=0; wbvector<T> v(*this);
00928         if (!iflag)

```

```

00929         { for (; i<len; ++i) data[i]=v.data[p[i]]; }
00930     else { for (; i<len; ++i) data[p[i]]=v.data[i]; }
00931     }
00932     else if (P.len<12) wblog(FL,
00933         "ERR invalid permutation [%s] (len=%d/%d)", STR(P), P.len, len);
00934     else wblog(FL, "ERR invalid permutation (len=%d/%d)", P.len, len);
00935
00936     return *this;
00937 };
00938
00939 template <class T>
00940 inline wbvector<T>& wbvector<T>::permute(
00941     wbvector<T> &v, const wbperm &P, char iflag
00942 ) const {
00943
00944     if (P.isEmpty()) { v=*this; return v; }
00945     const wperm_t *p=P.data;
00946
00947     if (len!=P.len || !validPerm(P)) {
00948         if (P.len<12) wblog(FL,
00949             "ERR invalid permutation [%s] (%d/%d)", STR(P), len, P.len);
00950         else wblog(FL, "ERR invalid permutation (%d/%d)", len, P.len);
00951     }
00952
00953     if (!P.isIdentityPerm()) { v.init(len);
00954         if (iflag==0) {
00955             for (size_t i=0; i<len; i++) v.data[i]=data[p[i]]; }
00956         else { for (size_t i=0; i<len; i++) v.data[p[i]]=data[i]; }
00957     }
00958     else v=(*this);
00959
00960     return v;
00961 };
00962
00963 template <class T> inline
00964 wbvector<T>& wbvector<T>::blockPermute(
00965     const wbperm &P, wbvector<T> &v, char iflag
00966 ) const {
00967
00968     if (isEmpty() || P.isEmpty() || P.isIdentityPerm()) {
00969         if (P.len && len%P.len) wblog(FL,
00970             "ERR %s() data mismatch (%d mod %d !?)", FCT, len, P.len);
00971         v=*this; return v;
00972     }
00973     if (&v==this) {
00974         wbvector<T> x(*this);
00975         return x.blockPermute(P, v, iflag);
00976     }
00977
00978     if (len%P.len || !validPerm(P)) wblog(FL,
00979         "ERR invalid permutation (%d/%d)", len, P.len);
00980     v.init(len);
00981
00982     size_t i, j, m=len/P.len;
00983     const wperm_t *p=P.data; const T *d0=data; T *d=v.data;
00984
00985     if (iflag==0) {
00986         for (i=0; i<P.len; ++i, d+=m) { d0=data+m*p[i];
00987             for (j=0; j<m; ++j) d[j]=d0[j]; }
00988     }
00989     else {
00990         for (i=0; i<P.len; ++i, d0+=m) { d=v.data+m*p[i];
00991             for (j=0; j<m; ++j) d[j]=d0[j]; }
00992     }
00993
00994     return v;
00995 };
00996
00997 template <class T> inline
00998 int wbvector<T>::blockCompare(
00999     const wbperm &Pa, const wbvector &B, const wbperm &Pb
01000 ) const {
01001
01002     if (len!=B.len ||
01003         (Pa.len && len%Pa.len) || (Pb.len && B.len%Pb.len))
01004         wblog(FL, "ERR %s() got length mismatch (%d@%d, %d@%d)",
01005             FCT, len, Pa.len, B.len, Pb.len);
01006     };
01007
01008     if ((!Pa.len && !Pb.len) || Pa.len==len || Pb.len==B.len) {
01009         if (Pa.len) {
01010             if (Pb.len) {
01011                 if (Pa.len!=Pb.len) wblog(FL,
01012                     "ERR %s() got length mismatch (%d@%d, %d@%d)",
01013                     FCT, len, Pa.len, B.len, Pb.len);
01014             }
01015         }

```

```

01016         const wperm_t*pa=Pa.data, *pb=Pb.data;
01017         for (size_t i=0; i<len; ++i) {
01018             if (data[pa[i]]!=B.data[pb[i]]) {
01019                 return (data[pa[i]]<B.data[pb[i]] ? -1 : +1);
01020             }
01021         }
01022     }
01023     else {
01024         const wperm_t *pa=Pa.data;
01025         for (size_t i=0; i<len; ++i) {
01026             if (data[pa[i]]!=B.data[i]) {
01027                 return (data[pa[i]]<B.data[i] ? -1 : +1);
01028             }
01029         }
01030     }
01031 }
01032 else {
01033     if (Pb.len) {
01034         const wperm_t *pb=Pb.data;
01035         for (size_t i=0; i<len; ++i) {
01036             if (data[i]!=B.data[pb[i]]) {
01037                 return (data[i]<B.data[pb[i]] ? -1 : +1);
01038             }
01039         }
01040     }
01041     else {
01042         for (size_t i=0; i<len; ++i) {
01043             if (data[i]!=B.data[i]) {
01044                 return (data[i]<B.data[i] ? -1 : +1);
01045             }
01046         }
01047     }
01048 }
01049 }
01050 else {
01051     if (Pa.len) {
01052         if (Pb.len) {
01053             const wperm_t *pa=Pa.data, *pb=Pb.data;
01054             size_t j, i=0, m=len/Pa.len;
01055             const T *a, *b;
01056
01057             if (Pa.len!=Pb.len) wblog(FL,
01058                 "ERR %s() got invalid permutations (%d@d, %d@d)",
01059                 FCT,len,Pa.len,B.len,Pb.len
01060             );
01061
01062             for (; i<Pa.len; ++i) { a=data+m*pa[i]; b=B.data+m*pb[i];
01063                 for (j=0; j<m; ++j) {
01064                     if (a[j]!=b[j]) return (a[j]<b[j] ? -1 : +1);
01065                 }
01066             }
01067         }
01068         else {
01069             const wperm_t *pa=Pa.data;
01070             size_t j, i=0, m=len/Pa.len;
01071             const T *a, *b=B.data;
01072
01073             for (; i<Pa.len; ++i, b+=m) { a=data+m*pa[i];
01074                 for (j=0; j<m; ++j) {
01075                     if (a[j]!=b[j]) { return (a[j]<b[j] ? -1 : +1); }
01076                 }
01077             }
01078         }
01079     }
01080     else {
01081         if (Pb.len) {
01082             const wperm_t *pb=Pb.data;
01083             size_t j, i=0, m=B.len/Pb.len;
01084             const T *a=data, *b;
01085
01086             for (; i<Pb.len; ++i, a+=m) { b=B.data+m*pb[i];
01087                 for (j=0; j<m; ++j) {
01088                     if (a[j]!=b[j]) { return (a[j]<b[j] ? -1 : +1); }
01089                 }
01090             }
01091         }
01092     }
01093 }
01094
01095 return 0;
01096 };
01097
01098 template <class T>
01099 inline int wbvector<T>::set2Group(
01100     const wperm &P, const WBINDEX &D,
01101     const T* S0,
01102     char iflag

```

```

01103 ){
01104     size_t i,j,l,d,n=D.len; int e=0;
01105     wbvector<T> X;
01106
01107     if (!S0 && len!=P.len) wblog(FL,
01108         "ERR %s() severe size mismatch (%d,%d)", FCT, len, P.len);
01109
01110     if (n==0) {
01111         if (P.len) wblog(FL,
01112             "ERR empty object inconsistency (%d,%d)", P.len, D.len);
01113         return 0;
01114     }
01115
01116     if (S0==NULL || S0==data) { save2(X); S0=X.data; }
01117
01118     init(n);
01119
01120     for (l=i=0; i<n; i++, l+=d) { d=D[i];
01121         data[i]=S0[P[l]];
01122
01123         for (j=1; j<d; j++) if (S0[P[l+j]]!=data[i]) {
01124             e++; if (iflag)
01125                 sprintf_str("%g/%g", (double)data[i], (double)S0[P[l+j]]);
01126         }
01127     }
01128
01129     return e;
01130 }
01131
01132 template<class T>
01133 mxArray* wbvector<T>::mxCreateStruct(unsigned m, unsigned n) const {
01134     return mxCreateCellMatrix(m,n);
01135 }
01136
01137 template<class T>
01138 void wbvector<T>::add2MxStruct(mxArray *S, unsigned i, char tflag) const {
01139     mxSetCell(S,i,toMx(tflag));
01140 }
01141
01142 template<class T>
01143 mxArray* wbvector<T>::mxCreateCell(unsigned m, unsigned n) const {
01144     return mxCreateCellMatrix(m,n);
01145 }
01146
01147 template<class T>
01148 void wbvector<T>::add2MxCell(mxArray *S, unsigned i, char tflag) const {
01149     mxSetCell(S,i,toMx(tflag));
01150 }
01151
01152 template<class T> inline
01153 void wbvector<T>::add2MxStruct(
01154     mxArray *S, const char *vname, char tflag) const {
01155
01156     int fid; mxArray *a;
01157
01158     fid = mxAddField2Scalar(FL,S,vname);
01159
01160     if (!tflag)
01161         a=mxCreateDoubleMatrix(1,len,mxREAL);
01162     else a=mxCreateDoubleMatrix(len,1,mxREAL);
01163
01164     double *d=mxGetDoubles(a);
01165     for (size_t i=0; i<len; ++i) d[i]=(double)data[i];
01166
01167     mxSetFieldByNumber(S,0,fid, a);
01168 };
01169
01170 template<class T> inline
01171 wbsting wbvector<T>::sizeStr() const {
01172     wbsting s;
01173     if (len) { s=toStrf("","x"); } else { s="[]"; }
01174     return s;
01175 };
01176
01177 template<class T> inline
01178 wbsting wbvector<T>::toStr() const { return toStr(-1, " "); }
01179
01180 template<class T> inline
01181 wbsting wbvector<T>::toStr(int n, const char *sep) const {
01182
01183     wbsting s(MAX(size_t(32), len*MAX(16,int(n)+6))), fmt;
01184     fmt.init2Fmt((T)0,n);
01185
01186     for (size_t i=0; i<len; ++i) {
01187         if (i>0) { s.push(FL,sep); }
01188         s.pushf(FL,fmt.data,data[i]);
01189     }

```

```

01190     return s;
01191 };
01192
01193 template<> inline
01194 wbsting wvector<wbcomplex>::toStr(int n, const char *sep) const {
01195
01196     wbsting s(MAX((size_t)32, len*MAX(16,abs(n)+6))), fmz, fmd;
01197     fmz.init2Fmt(wbcomplex(0),n); fmd.init2Fmt(double(0),n);
01198
01199     for (size_t i=0; i<len; ++i) { if (i) { s.push(FL,sep); }
01200         if (data[i].r && data[i].i)
01201             s.pushf(FL,fmz.data,data[i].r,data[i].i);
01202         else if (data[i].r)
01203             { s.pushf(FL,fmd.data,data[i].r); }
01204         else { s.pushf(FL,fmd.data,data[i].i).push("i"); }
01205     }
01206     return s;
01207 };
01208
01209 #ifdef QS_USING_MPFR
01210 template<> inline
01211 wbsting wvector<Wb::quad>::toStr(int n, const char *sep) const {
01212
01213     wbsting s(MAX((size_t)32, len*MAX(16,int(n)+6))), fmt;
01214     fmt.init2Fmt((double)0,n);
01215
01216     for (size_t i=0; i<len; ++i) {
01217         if (i>0) { s.push(FL,sep); }
01218         s.pushf(FL,fmt.data,double(data[i]));
01219     }
01220
01221     return s;
01222 };
01223 #endif
01224
01225 template<class T> inline
01226 wbsting wvector<T>::toStrf (
01227     const char *fmt_, const char *sep, unsigned stride, const char *sep2
01228 ) const {
01229
01230     wbsting s(MAX(32U, unsigned(len)*16)), fmt;
01231     unsigned i_, isep=0;
01232
01233     if (fmt_ && fmt_[0])
01234         { fmt=fmt_; }
01235     else { fmt.init2Fmt(T(0)); }
01236
01237     for (unsigned i=0; i<len; ++i) { if (++isep>1) {
01238         s.push (FL,sep); }
01239         s.pushf(FL,fmt.data,data[i]); if (stride) { i_=i+1;
01240         if (!(i_%stride) && i_<len) { s.push(FL,sep2); isep=0; }}
01241     }
01242
01243     return s;
01244 };
01245
01246 template<> inline
01247 wbsting wvector<wbcomplex>::toStrf(
01248     const char *fmt_, const char *sep, unsigned stride, const char *sep2
01249 ) const {
01250
01251     wbsting s;
01252     unsigned w,i=0; char fmt[16], s1[32], flag=1;
01253
01254     snprintf(fmt,16,"%s",fmt_ && fmt_[0] ? fmt_ : "%.4g");
01255     w=2 + sprintf_str(fmt, sqrt(2.))
01256         + sprintf_str(fmt,-sqrt(2.)*1e-20);
01257
01258     s.init(32+(4+i)*len);
01259
01260     for (size_t i=0; i<len; ++i) {
01261         if (i>0) { if (flag) { s.push(FL,sep); } else { flag=1; }}
01262
01263         snprintf(s1,32,"%s",data[i].toStr(fmt).data);
01264         s.pushf(FL,"%*s",w,s1);
01265         if (stride && ((i+1)%stride)==0 && i+1<len) {
01266             s.push(FL,sep2); flag=0;
01267         }
01268     }
01269
01270     return s;
01271 };
01272
01273 #ifdef QS_USING_MPFR
01274
01275 template<> inline
01276 wbsting wvector<Wb::quad>::toStrf(

```



```

01277     const char *fmt_, const char *sep, unsigned stride, const char *sep2
01278 ) const {
01279
01280     wbvector<double> x; x.init(*this);
01281     return x.toStrf(fmt_, sep, stride, sep2);
01282 };
01283
01284 #endif
01285
01286 #ifdef __WB_MPFR_HH__
01287
01288 template <> inline
01289 mxArray* wbvector< Wb::quad >::toMx(char base) const {
01290
01291     if (base<=0) base=30; else
01292     if (base==1 || base>62) wblog(FL,
01293         "ERR %s() invalid base=%d<%c>\n"
01294         "(interpreted as base for Wb::quad)", FCT, base, base);
01295
01296     unsigned nmax=0, n=0; size_t S[2];
01297     mxArray *a;
01298
01299     wbsting s_; char *s=NULL;
01300     if (len) { data[0].toStr(s_, base); n=s_.len; s=s_.data; }
01301
01302     if (len!=1)
01303         { S[0]=n; S[1]=len; }
01304     else { S[0]=len; S[1]=n; }
01305
01306     if (len==1 && data[0]==Wb::quad(int(double(data[0]))) {
01307         unsigned i=0, l=0;
01308
01309         while (i<n && s[i] && s[i]!='.') { ++i; }
01310         if (i<n && s[i]) { l=i;
01311             for (++i; i<n && s[i]; ++i) { if (s[i]!='0')
01312                 wblog(FL, "ERR %s() expecting integer (%s) !?", FCT, s);
01313             }
01314             s[l]=0; S[1]=l+1;
01315
01316             return a=Mx::Array<mxChar>(2, S).copyFromTR(s);
01317         }
01318     }
01319
01320     char *cdat=NULL;
01321     a=Mx::Array<char>(2, S).Return(cdat);
01322
01323     if (!len) { return a; }
01324
01325     unsigned np=1; int e=0;
01326     if (len>=64) {
01327         np=MAX( QSP_NUM_THREADS, OMP_NUM_THREADS );
01328     }
01329
01330     #pragma omp parallel for num_threads(np)
01331     for (size_t k=0; k<len; ++k) { if (!e) {
01332         unsigned i=0, i0=0;
01333         char *sk=cdat+k*n;
01334
01335         try { data[k].toStr(sk, n, base, -1, 'a'); }
01336         catch (...) { ++e; }
01337
01338         while (i<n && s[i]==' ') { ++i; }; i0=i;
01339         while (i<n && s[i]!=' ' && s[i]) { ++i; }
01340
01341         if (i<2 || i0>=i || s[i] || (++i)>=n) { ++e;
01342             char s[n+1]; strncpy(s, sk, n); s[n]=0;
01343             wblog(FL, "... error i=%4ld/%ld: '%s' (%d..%d/%d)",
01344                 k, len, s, i0, i, n);
01345         }
01346
01347         #pragma omp critical (mpfr_collecting_nmax)
01348         { if (nmax<i) nmax=i; }
01349     } }
01350
01351     if (e) wblog(FL, "ERR %s() e=%d", FCT, e);
01352
01353     if (nmax==n) wblog(FL,
01354         "WRN %s<Wb::quad> got n=%d/%d (len=%ld)", FCT, nmax, n, len);
01355     if (nmax>n) wblog(FL,
01356         "ERR %s<Wb::quad> got n=%d/%d (len=%ld)", FCT, nmax, n, len);
01357
01358     return a;
01359 };
01360
01361 #endif
01362
01363 template <class T>

```

```

01364 inline wbvector<T>& wbvector<T>::Move2_FE(
01365     const WBINDEX &I,
01366     char iflag
01367 ){
01368     size_t i,k;
01369     wbvector<char> mark(len);
01370     wbvector<T> x;
01371
01372     if (I.isEmpty()) return *this;
01373     if (!isUniqueIdxSet(I,len)) {
01374         wblog(FL, "ERR Move2_FE Index out of bounds [%s; %d]",
01375             I.toString().data, len); return *this;
01376     }
01377
01378     for (i=0; i<I.len; i++) mark[I[i]]++;
01379
01380     select(I,x);
01381
01382     if (iflag=='E') {
01383         for (k=i=0; i<len; i++)
01384             if (mark[i]==0) {
01385                 if (k!=i) data[k]=data[i];
01386                 k++;
01387             }
01388         memcpy(data+k, x.data, x.len*sizeof(T));
01389     }
01390     else
01391     if (iflag=='F') {
01392         for (k=i=len-1; int(i)>=0; i--)
01393             if (mark[i]==0) {
01394                 if (k!=i) data[k]=data[i];
01395                 k--;
01396             }
01397         memcpy(data, x.data, x.len*sizeof(T));
01398     }
01399     else wblog(FL,"ERR Move2_FE - Invalid flag %c<%d>", iflag, iflag);
01400
01401     return *this;
01402 }
01403
01404 template <class T>
01405 inline wbvector<T>& wbvector<T>::Skip(size_t il) {
01406
01407     if (il>=len) {
01408         wblog(FL, "ERR Skip - index out of range (%d,%d)\n[%s; %d]",
01409             il, len, toString().data, il); return *this;
01410     }
01411
01412     for (size_t i=il+1; i<len; i++) data[i-1]=data[i];
01413     Resize(len-1);
01414
01415     return *this;
01416 }
01417
01418 template <class T>
01419 inline wbvector<T>& wbvector<T>::Skip(
01420     const WBINDEX &I,
01421     char rflag
01422 ){
01423     size_t i,k;
01424     wbvector<char> mark(len); char *m=mark.data;
01425
01426     for (k=0; k<I.len; k++) {
01427         if (I[k]<len) m[I[k]]++;
01428     }
01429     else {
01430         wblog(FL, "ERR Skip - index out of range (%d,%d)\n[%s; %d]",
01431             I[k]+1, len, toString().data, k+1); return *this;
01432     }
01433
01434     for (k=i=0; i<len; ++i) {
01435         if (m[i]==0) { if (k!=i) { data[k]=data[i]; }
01436             ++k;
01437         }
01438     }
01439
01440     if (!rflag) Resize(k);
01441     else {
01442         if (k) len=k; else init();
01443     }
01444
01445     return *this;
01446 };
01447
01448 #endif
01449

```

5.63 wbvector.hh

```

00001 /* -----
00002  * Project : QSpace tensor library (v4.0)
00003  * Class   : wbvector (template vector class)
00004  *
00005  * Copyright 2024 Andreas Weichselbaum
00006  *
00007  * Licensed under the Apache License, Version 2.0 (the "License");
00008  * you may not use this file except in compliance with the License.
00009  * You may obtain a copy of the License at
00010  *
00011  *     http://www.apache.org/licenses/LICENSE-2.0
00012  *
00013  * Unless required by applicable law or agreed to in writing, software
00014  * distributed under the License is distributed on an "AS IS" BASIS,
00015  * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
00016  * See the License for the specific language governing permissions and
00017  * limitations under the License.
00018  * ----- */
00019
00020 #ifndef __WB_SIMPLE_VECTOR_HCC__
00021 #define __WB_SIMPLE_VECTOR_HCC__
00022
00023 template <class T> class wbvector;
00024
00025 WBINDEX Index(widx_t i1, widx_t i2);
00026 char isUniqueIdxSet(const WBINDEX &I, widx_t imax);
00027 char validPerm(const WBPPerm &P, wperm_t d=-1);
00028 void getIPerm (const WBPPerm &P, WBPPerm &iP);
00029
00030 bool mxIsWbvector(
00031     const char *F, int L, const mxArray *a,
00032     size_t *n=NULL, const char *istr=NULL, char dflag='d');
00033
00034 bool mxIsWbvector(const mxArray *a) {
00035     return mxIsWbvector(0,1,a);
00036 };
00037
00038 template <class T>
00039 class wbvector {
00040
00041 public:
00042
00043     wbvector() : data(NULL), len(0), isref(0) {};
00044
00045     wbvector(size_t l, T x) : data(NULL), len(0), isref(0) {
00046         RENEW(l,NULL,0,0);
00047         set(x);
00048     };
00049
00050     wbvector(size_t l, const T* d0=NULL, char ref=0)
00051         : data(NULL), len(0), isref(0) {
00052         if (ref && d0 && l)
00053             { init2ref(l,d0); }
00054         else { RENEW(l,d0); }
00055     };
00056
00057     wbvector(const wbvector &v)
00058         : data(NULL), len(0), isref(0) { RENEW(v.len, v.data); };
00059
00060     template <class T2>
00061     wbvector(const wbvector<T2> &v)
00062         : data(NULL), len(0), isref(0) { initT(v.len,v.data); }
00063
00064     wbvector(const wbvector &v1, const wbvector &v2)
00065         : data(NULL), len(0), isref(0) {
00066
00067         size_t i, k=0;
00068         RENEW(v1.len+v2.len);
00069
00070         for (i=0; i<v1.len; ++i, ++k) data[k]=v1[i];
00071         for (i=0; i<v2.len; ++i, ++k) data[k]=v2[i];
00072     };
00073
00074     wbvector(const char *F, int L, const mxArray *a, char check_type=1, char ref=0)
00075         : data(NULL), len(0), isref(ref) {
00076         init(F,L,a,NULL,check_type,ref);
00077     };
00078
00079     wbvector(const mxArray *a, char check_type=1, char ref=0)
00080         : data(NULL), len(0), isref(ref) {
00081         init(F,L,a,NULL,check_type,ref);
00082     };
00083
00084     virtual ~wbvector() {
00085         if (!isref && data) { WB_DELETE(data); }

```

```

00086     };
00087
00088     wbvector& init(size_t l=0) { return RENEW(l); };
00089     wbvector& init(size_t l, const T* d0) { return RENEW(l,d0); };
00090
00091     wbvector& init(size_t l, const T* d0, char ref) {
00092         if (ref && d0 && l)
00093             { return init2ref(l,d0); }
00094         else { return RENEW(l,d0); }
00095     };
00096
00097     wbvector& initp(size_t l, const T* d0, size_t n,
00098         char init=1
00099     ){ return RENEW(l,d0,n,init); };
00100
00101     wbvector& init2val(size_t l, const T& x) {
00102         return RENEW_VAL(l,x); };
00103
00104     wbvector& initStride(size_t l, const T* d0, size_t stride) {
00105         RENEW(l,NULL,0,0);
00106         for (size_t i=0; i<len; ++i, d0+=stride) { data[i]=*d0; }
00107         return *this;
00108     };
00109
00110     wbvector& init_stride(
00111         const T* d0, size_t n, size_t m, size_t D0
00112     ){
00113         RENEW(n*m);
00114         Wb::cpyStride(data, d0, n, m, -1, D0);
00115         return *this;
00116     };
00117
00118     wbvector& init(const wbvector &v) {
00119         RENEW(v.len);
00120         for (size_t i=0; i<len; ++i) data[i]=v.data[i];
00121         return *this;
00122     };
00123
00124     wbvector& init(size_t n, const wbvector &v) {
00125         RENEW(n); if (n>v.len) n=v.len;
00126         for (size_t i=0; i<n; ++i) data[i]=v.data[i];
00127         return *this;
00128     };
00129
00130     wbvector& init_data(const T *d0) {
00131         for (size_t i=0; i<len; ++i) { data[i]=d0[i]; }
00132         return *this;
00133     };
00134
00135     template<class T0>
00136     wbvector& init_data(const T0 *d0) {
00137         for (size_t i=0; i<len; ++i) { data[i]=T(d0[i]); }
00138         return *this;
00139     };
00140
00141     template<class T0>
00142     wbvector& init_data(const char *F, int L, const T0 *d0) {
00143         for (size_t i=0; i<len; ++i) { data[i]=T(d0[i]);
00144             if (T0(data[i])!=d0[i]) wblog(F,L,
00145                 "ERR type conversion changes value (%g->%g) !?",
00146                 double(d0[i]), double(data[i])
00147             );
00148         }
00149         return *this;
00150     };
00151
00152     wbvector<T>& init(const wbsparray<T> &S) { return initT(S); };
00153
00154     template <class T2>
00155     wbvector<T>& initT(const wbsparray<T2> &S);
00156
00157     template <class T2>
00158     wbvector<T>& initT(size_t l, const T2* v) {
00159         RENEW(l);
00160         for (size_t i=0; i<len; ++i) data[i]=T(v[i]);
00161         return *this;
00162     };
00163
00164     template <class T2>
00165     wbvector<T>& initT(const wbvector<T2> &v) {
00166         RENEW(v.len);
00167         for (size_t i=0; i<len; ++i) data[i]=T(v.data[i]);
00168         return *this;
00169     };
00170
00171     template <class T2>
00172     wbvector<T>& initT_(const wbvector<T2> &v) {

```

```

00173         RENEW(v.len);
00174         for (size_t i=0; i<len; ++i) data[i]=T(v.data[i], '!');
00175         return *this;
00176     };
00177
00178     template <class T2>
00179     wbvector<T>& initT(const char *F, int L, const wbvector<T2> &v);
00180
00181     wbvector& init2ref(const wbvector &v) {
00182         return init2ref(v.len, v.data);
00183     };
00184
00185     wbvector& init2ref(size_t l, const T* d0) {
00186         if (l && !d0) wblog(FL, "ERR %s() got null ref (n=%d)", FCT, l);
00187         if (data && !isref) { RENEW(0); }
00188         isref=1; len=l; data=(len ? (T*)d0 : NULL);
00189         return *this;
00190     };
00191
00192     wbvector& unRef();
00193
00194     wbvector& init(size_t l1, const T* d1, size_t l2, const T* d2) {
00195         RENEW(l1+l2); MEM_COPY<T>(data, len, l1, d1, d2);
00196         return *this;
00197     };
00198
00199     int init(
00200         const char *F, int L, const mxArray *a, const char *istr=NULL,
00201         char check_type QS_UNUSED_VAR =1,
00202         char ref=0
00203     ) { return init_Struct(F, L, a, istr, ref); };
00204
00205     int init_Struct(
00206         const char *F, int L, const mxArray *a,
00207         const char *istr=NULL, char ref=0
00208     );
00209
00210     int init_base(
00211         const char *F, int L, const mxArray *a,
00212         const char *istr=NULL, char check_type=1, char ref=0
00213     );
00214
00215     int init(
00216         const mxArray *a, const char *istr=NULL, char check_type=1
00217     ){ return init(0, 0, a, istr, check_type); };
00218
00219     int init_mpfir(
00220         const char *F, int L, const mxArray *a, char base=-1,
00221         const char *istr="", char tcheck=1
00222     );
00223
00224     void initDef(size_t n) {
00225         if (data) { WB_DELETE(data); }
00226         len=n; if (n) { WB_NEW(data, n); }
00227     };
00228
00229     void initDef(const wbvector &v) {
00230         if (len!=v.len) initDef(v.len);
00231         for (size_t i=0; i<v.len; ++i) data[i]=v[i];
00232     };
00233
00234     wbvector& operator= (const wbvector &v) {
00235         if (this!=&v) { RENEW(v.len, v.data); }
00236         return *this;
00237     };
00238
00239     template<class TB>
00240     wbvector<T>& operator= (const wbvector<TB> &v) {
00241         init(v.len);
00242         for (size_t i=0; i<v.len; ++i) { data[i]=T(v.data[i]); }
00243         return *this;
00244     };
00245
00246     wbvector& set(const T &x) {
00247         for (size_t i=0; i<len; ++i) { data[i]=x; }
00248         return *this;
00249     };
00250
00251     wbvector& setMin(const T &x) {
00252         for (size_t i=0; i<len; ++i) { if (data[i]<x) data[i]=x; }
00253         return *this;
00254     };
00255     wbvector& setMax(const T &x) {
00256         for (size_t i=0; i<len; ++i) { if (data[i]>x) data[i]=x; }
00257         return *this;
00258     };
00259

```

```

00260     wbvector& set(const wbvector<size_t> &I, const T &x);
00261     wbvector& add(const wbvector<size_t> &I, const T &x);
00262
00263     wbvector& Set(const wbvector<size_t> &I, const wbvector &v);
00264
00265     void Sort(wbperm &p, char dir=+1) { Wb::hpsort(*this,p,dir); };
00266     void sort(wbvector &v2, wbperm &p) const { v2=(*this); Wb::hpsort(v2,p); };
00267
00268     void Sort(char dir=+1);
00269     void sort(wbvector &v2) const;
00270
00271     wbperm& pSort(wbperm &p) const;
00272
00273     bool isSorted() const {
00274         for (size_t i=1; i<len; ++i) if (data[i-1]> data[i]) return 0;
00275         return 1;
00276     };
00277     bool isSortedU() const {
00278         for (size_t i=1; i<len; ++i) if (data[i-1]>=data[i]) return 0;
00279         return 1;
00280     };
00281
00282     bool isUnique() const;
00283
00284     int contains(const T& x) const {
00285         for (size_t i=0; i<len; ++i) { if (data[i]==x) return 1; }
00286         return 0;
00287     };
00288
00289     bool isNormal() const;
00290
00291     bool isFinite() const {
00292         return Wb::is_finite(data,len);
00293     };
00294
00295     size_t findl(const T& x) const {
00296         for (size_t i=0; i<len; ++i) { if (data[i]==x) return i; }
00297         return -1;
00298     };
00299     size_t findl() const {
00300         for (size_t i=0; i<len; ++i) { if (data[i]) return i; }
00301         return -1;
00302     };
00303
00304     size_t count(const T& x) const {
00305         size_t i=0, n=0;
00306         for (; i<len; ++i) if (data[i]==x) ++n;
00307         return n;
00308     };
00309
00310     size_t icount(const T& x) const {
00311         size_t i=0,n=0;
00312         for (; i<len; ++i) if (data[i]!=x) ++n;
00313         return n;
00314     };
00315
00316     size_t nnz() const {
00317         size_t i=0, n=0;
00318         for (; i<len; ++i) if (data[i]) ++n;
00319         return n;
00320     };
00321     size_t nnz(T eps) const {
00322         if (eps==0) { return nnz(); }
00323         else { return len-numZeros(eps); }
00324     };
00325
00326     size_t numZeros(T eps=T(1E-14)) const;
00327     size_t numEQ(const T&) const;
00328     size_t numGT(const T&) const;
00329     size_t numLT(const T&) const;
00330
00331     wbindex& find(const T &x, wbindex &I, char iflag=0) const;
00332     wbindex& find(const T &x, wbindex &I, wbindex &Ix) const;
00333     wbindex& findGT(const T &x, wbindex &I, char iflag=0) const;
00334     wbindex& findGT(const T &x, wbindex &I, wbindex &Ix) const;
00335
00336     wbindex& findzeros(wbindex &I) const { return find(0,I); }
00337     wbindex& find(wbindex &I) const { return find(0,I,'i'); }
00338
00339     wbindex& find(wbindex &I, wbindex &Ix) const {
00340         find(0,Ix,I); return I; }
00341
00342     wbvector<size_t> find() const {
00343         wbvector<size_t> I;
00344         return find(I);
00345     };
00346

```

```

00347     wbindex& findRange(
00348         const T &x1, const T &x2, wbindex &I, const char *w=NULL
00349     ) const;
00350
00351     int findClosestSorted(T r) const;
00352     int findClosest(T r) const;
00353
00354     size_t findValues(const wbvector &v, wbindex &I) const;
00355
00356     wbvector& setp(const T *x) {
00357         for (size_t i=0; i<len; ++i) { data[i]=x[i]; }
00358         return *this;
00359     };
00360
00361     void set(size_t offset, const T *x, size_t n) {
00362         if (offset+n>=len) wblog(FL,
00363             "ERR index out of bounds (%d+%d; %d)",offset,n,len);
00364         MEM_COPY<T>(data+offset,n,x);
00365     };
00366
00367     void reset() { if (len) MEM_SET<T>(data,len); };
00368
00369     void cpy(const T* v) {
00370         if (len) MEM_COPY<T>(data,len,v);
00371     };
00372
00373     void cpy(const T *x, size_t m) {
00374         if (m>len) { wblog(FL,"ERR index out of bounds (%d/%d)",m,len); m=len; }
00375         if (m) MEM_COPY<T>(data,m,x);
00376     };
00377
00378     wbvector& save2(wbvector &v) {
00379         swap(v); if (data!=v.data) { init(); }
00380         return v;
00381     };
00382
00383     void swap(wbvector &v) {
00384         if (this!=&v) {
00385             SWAP(len, v.len );
00386             SWAP(data, v.data );
00387             SWAP(isref, v.isref);
00388         }
00389     };
00390
00391     wbvector& swap(size_t i, size_t j) {
00392         if (i<len && j<len)
00393             { T x=data[i]; data[i]=data[j]; data[j]=x; }
00394         else { wblog(FL,
00395             "ERR %s() index out of bounds (%d,%d/%d)",FCT,i+1,j+1,len); }
00396         return *this;
00397     };
00398
00399     wbvector& Collect(T& A1, T& A2) {
00400         init(2); A1.save2(data[0]); A2.save2(data[1]);
00401         return *this;
00402     };
00403
00404     wbvector& Collect(T& A1, T& A2, T& A3) {
00405         init(3); A1.save2(data[0]); A2.save2(data[1]); A3.save2(data[2]);
00406         return *this;
00407     };
00408
00409     wbvector& Skip(size_t);
00410     wbvector& Skip(const WBINDEX &, char rflag=0);
00411
00412     wbvector& Move2_FE (const WBINDEX &I, char FE_flag);
00413     wbvector& Move2front(const WBINDEX &I) { return Move2_FE(I,'F'); }
00414     wbvector& Move2end (const WBINDEX &I) { return Move2_FE(I,'E'); }
00415
00416     wbvector& Move2front (size_t k) {
00417         if (k>=len) wblog(FL,"ERR index out of bounds (%d,%d)", k, len);
00418         T xk=data[k];
00419         for (size_t i=k; i>0; i--) data[i]=data[i-1];
00420         data[0]=xk; return *this;
00421     }
00422
00423     wbvector& Move2end (size_t k) {
00424         if (k>=len) wblog(FL,"ERR index out of bounds (%d,%d)", k, len);
00425         T xk=data[k];
00426         for (size_t i=k+1; i<len; i--) data[i-1]=data[i];
00427         data[len-1]=xk; return *this;
00428     }
00429
00430     wbvector& mvLastTo (size_t k) {
00431         if (k>=len) wblog(FL,"ERR index out of bounds (%d,%d)", k, len);
00432         T xk=data[len-1];
00433         for (size_t i=len-1; i>k; i--) data[i]=data[i-1];

```

```

00434     data[k]=xk; return *this;
00435 }
00436
00437 wbvector& mvFirstTo (size_t k) {
00438     if (k>=len) wblog(FL,"ERR index out of bounds (%d,%d)", k, len);
00439     T xk=data[0];
00440     for (size_t i=0; i<k; i++) data[i]=data[i+1];
00441     data[k]=xk; return *this;
00442 }
00443
00444 bool checkSameLength(
00445     const wbvector &a,
00446     const char *F=NULL, int L=0, const char *istr=""
00447 ) const;
00448
00449 bool checkSameLength(
00450     const wbvector &a,
00451     const wbvector &b,
00452     const char *F=NULL, int L=0, const char *istr=""
00453 ) const;
00454
00455 template <class T_>
00456 void operator+= (const T_ &a) {
00457     if (a==T(0)) return;
00458     for (size_t i=0; i<len; ++i) data[i]+=a;
00459 };
00460
00461 template <class T_>
00462 void operator-= (const T_ &a) {
00463     if (a==T(0)) return;
00464     for (size_t i=0; i<len; ++i) data[i]-=a;
00465 };
00466
00467 template <class T_>
00468 wbvector& operator*= (const T_ &a) {
00469     if (len && a!=T_(1)) {
00470         if (isref) wblog(FL,"ERR %s() reference is considered const",FCT);
00471         if (!a) { MEM_SET<T>(data,len); }
00472         else { size_t i=0;
00473             if (a==T_(-1))
00474                 for (; i<len; ++i) { data[i]=-data[i]; }
00475             else for (; i<len; ++i) { data[i]*=a; }
00476         }
00477     }
00478     return *this;
00479 };
00480
00481 template <class T_>
00482 wbvector& operator/= (const T_ &a) {
00483     if (len && a!=T_(1)) { size_t i=0;
00484         if (isref) wblog(FL,"ERR %s() reference is considered const",FCT);
00485         if (!a) wblog(FL,"ERR got div/0");
00486
00487         if (a==T_(-1)) { for (; i<len; ++i) data[i]=-data[i]; } else
00488         if (len<4 || WbUtil<T>::isInt()) { for (; i<len; ++i) data[i]/=a; }
00489         else {
00490             T x=1/T(a);
00491             if (fabs(double(x*a-1))>1E-12)
00492                 { for (; i<len; ++i) { data[i]/=a; }}
00493             else { for (; i<len; ++i) { data[i]*=x; }}
00494         }
00495     }
00496     return *this;
00497 };
00498
00499 template <class T_>
00500 wbvector operator* (const T_ &a) const {
00501     wbvector u(*this); if (a!=1) u*=a;
00502     return u;
00503 };
00504
00505 template <class T_>
00506 wbvector operator+ (const T_ &a) const {
00507     wbvector u(*this);
00508     if (a!=0) for (size_t i=0; i<u.len; ++i) { u.data[i]+=a; }
00509     return u;
00510 };
00511
00512 template <class T_>
00513 wbvector operator- (const T_ &a) const {
00514     wbvector u(*this);
00515     if (a!=0) for (size_t i=0; i<u.len; ++i) { u.data[i]-=a; }
00516     return u;
00517 };
00518
00519 wbvector operator+ (const wbvector &b) const {
00520     wbvector u(*this); checkSameLength(b,FLF);

```



```

00521         for (size_t i=0; i<u.len; i++) u.data[i]+=b.data[i];
00522         return u;
00523     };
00524
00525     wbvector operator- (const wbvector &b) const {
00526         wbvector u(*this); checkSameLength(b,FLF);
00527         for (size_t i=0; i<u.len; i++) u.data[i]-=b.data[i];
00528         return u;
00529     };
00530
00531     void operator+= (const wbvector &b) {
00532         checkSameLength(b,FLF);
00533         for (size_t i=0; i<len; ++i) data[i]+=b.data[i];
00534     };
00535
00536     void operator-= (const wbvector &b) {
00537         checkSameLength(b,FLF);
00538         for (size_t i=0; i<len; ++i) data[i]-=b.data[i];
00539     };
00540
00541     wbvector& Plus (const T* b) {
00542         for (size_t i=0; i<len; ++i) data[i]+=b[i];
00543         return *this;
00544     };
00545
00546     wbvector& Plus (const T* b, T bfac) {
00547         size_t i=0;
00548         if (bfac==+1) for (; i<len; ++i) data[i]+=b[i]; else
00549         if (bfac==-1) for (; i<len; ++i) data[i]-=b[i]; else
00550         if (bfac!= 0) for (; i<len; ++i) data[i]+=(bfac*b[i]);
00551         return *this;
00552     };
00553
00554     wbvector& Plus (const wbvector &b, T bfac=1) {
00555         checkSameLength(b,FLF);
00556         return Plus(b.data,bfac);
00557     };
00558
00559     void plus (
00560         const wbvector &b, wbvector &c, T bfac=1, T cfac=0
00561     ) const {
00562         if (cfac) {
00563             checkSameLength(b,c,FLF); c*=cfac; c.Plus(*this); }
00564         else { checkSameLength(b, FLF); c=*this; }
00565
00566         c.Plus(b,bfac);
00567     };
00568
00569     wbvector& times(T fac, wbvector &x) const {
00570         x=*this; x*=fac; return x;
00571     };
00572
00573     wbvector& RevertSigns();
00574
00575     int isum() const {
00576         int s=0; if (len) {
00577             size_t i=1; s=data[0]; for (; i<len; ++i) { s+=data[i]; }
00578         }; return s;
00579     };
00580
00581     T sum() const {
00582         T s=T(); if (len) {
00583             size_t i=1; s=data[0]; for (; i<len; ++i) { s+=data[i]; }
00584         }; return s;
00585     };
00586
00587     T sum(size_t il, const char *estr) const {
00588         if (!estr || strcmp(estr,"end")) wblog(FL,
00589             "ERR invalid usage (got '%s')", estr);
00590         return sum(il,len-1);
00591     };
00592
00593     T sum(size_t il, size_t i2) const {
00594         T s=T();
00595
00596         if (!len) { if (!il && !i2) return s; }
00597         if (i1>=len || i2>=len) wblog(FL,
00598             "ERR index out of bounds (%d,%d; %d)",i1,i2,len);
00599         if (i1<=i2) {
00600             size_t i=i1+1; s=data[i1]; for (; i<=i2; ++i) { s+=data[i]; }
00601         }
00602         return s;
00603     };
00604
00605     T cumsum_(wbvector &S, char xflag=0) const;
00606     T Cumsum(char zflag=0) const;
00607

```

```

00608     template <class T2>
00609     T cumsum0prod(wbvector<T2> &b, wbvector<T> &cs, char xflag) const;
00610
00611     wbvector& Eldiv( const wbvector &v) const;
00612
00613     wbvector& tensorProd(const wbvector &v2, wbvector &vv) const;
00614
00615     T scalarProd(const wbvector &v) const {
00616         if (len!=v.len) wblog(FL,
00617             "ERR scalarProd - length mismatch (%d/%d)",len,v.len);
00618
00619         if (len) return scalarProd(v.data);
00620         else return 0;
00621     };
00622
00623     T scalarProd(const T *v) const {
00624         T s=T(); for (size_t i=0; i<len; ++i) s+=(data[i]*Wb::CONJ(v[i]));
00625         return s;
00626     };
00627
00628     T norm() const { return Wb::sqrt(norm2()); }
00629
00630     T norm2() const {
00631         T s=0; if (len==0) return s;
00632         s=Wb::norm2(data[0]);
00633
00634         for (size_t i=1; i<len; i++) s+=Wb::norm2(data[i]);
00635         return s;
00636     };
00637
00638     double normDiff(const wbvector &v, T fac=1) const {
00639         return std::sqrt(double(normDiff2(v,fac)));
00640     };
00641
00642     T normDiff2( const wbvector &v, T fac=1) const;
00643     double normDiff2_(const wbvector &v, double fac=1) const;
00644
00645     T GSProject(const T *v, char isnorm=0, char tflag=0) const {
00646         return gs_project_range(data,v,len,1,isnorm,tflag);
00647     };
00648
00649     wbvector& Normalize(const char *F=NULL, int L=0) {
00650         double n=std::sqrt(norm2());
00651         if (n==0) {
00652             if (F) wblog(F,L,"WRN normalize zero vector!? - randomize");
00653             setRand(1,-0.5); return Normalize(F,L);
00654         }
00655         (*this)*=(1./n); return *this;
00656     };
00657
00658     wbvector& add(const T* d) {
00659         for (size_t i=0; i<len; ++i) data[i]+=d[i];
00660         return *this;
00661     };
00662
00663     wbvector& subtract(const T* d) {
00664         for (size_t i=0; i<len; ++i) data[i]-=d[i];
00665         return *this;
00666     };
00667
00668     void getReal(wbvector<double> &R) const;
00669     void getImag(wbvector<double> &I) const;
00670     void set(const wbvector<double> &R, const wbvector<double> &I);
00671     wbvector& Conj() { return *this; };
00672
00673     wbvector& applyFlag(char &flag, const T &fac, wbvector &a);
00674
00675     bool allEqual() const {
00676         for (size_t i=1; i<len; ++i) if (data[i]!=data[0]) return 0;
00677         return 1;
00678     }
00679
00680     bool allZero() const { return allEqual(T(0)); };
00681     bool allOnes() const { return allEqual(T(1)); };
00682
00683     bool allEqual(const T &a) const {
00684         bool q=1; size_t i=0;
00685         if (!len) wblog(FL,"WRN %s() got len=%d",FCT,len);
00686         for (; i<len; ++i) { if (data[i]!=a) { q=0; break; } }
00687         return q;
00688     }
00689
00690     bool allEqual2(const T &a, T eps=1E-15) const {
00691         if (!eps) { return allEqual(a); }
00692         else {
00693             T x, x2=0; if (a!=0) eps*=a;
00694             if (!len) wblog(FL,"WRN %s() got len=%d",FCT,len);

```

```

00695         for (size_t i=0; i<len; ++i) { x=data[i]-a; x2+=(x*x); }
00696         return (Wb::abs(x2)<eps);
00697     }
00698 };
00699
00700 bool allUnequal(const T &a) const {
00701     bool q=1; size_t i=0;
00702     if (!len) wblog(FL,"WRN %s() got len=%d",FCT,len);
00703     for (; i<len; ++i) { if (data[i]==a) { q=0; break; }}
00704     return q;
00705 }
00706
00707 bool any() const { bool q=0; size_t i=0;
00708     for (; i<len; ++i) { if (data[i]) { q=1; break; }}
00709     return q;
00710 }
00711
00712 bool anyUnequal(const T &a) const {
00713     bool q=0; size_t i=0;
00714     for (; i<len; ++i) { if (data[i]!=a) { q=1; break; }}
00715     return q;
00716 }
00717
00718 bool anyEqual(const T &a, size_t *i1=NULL) const {
00719     bool q=0; size_t i=0;
00720     for (; i<len; ++i) { if (data[i]==a) {
00721         if (i1) { (*i1)=i; }
00722         q=1; break;
00723     }}
00724     return q;
00725 };
00726
00727 bool anyGT(const T &a) const {
00728     bool q=0; size_t i=0;
00729     for (; i<len; ++i) { if (data[i]> a) { q=1; break; }}
00730     return q;
00731 };
00732
00733 bool anyGE(const T &a) const {
00734     bool q=0; size_t i=0;
00735     for (; i<len; ++i) { if (data[i]>=a) { q=1; break; }}
00736     return q;
00737 };
00738
00739 bool anyLT(const T &a) const {
00740     bool q=0; size_t i=0;
00741     for (; i<len; ++i) { if (data[i]< a) { q=1; break; }}
00742     return q;
00743 };
00744
00745 bool anyLE(const T &a) const {
00746     bool q=0; size_t i=0;
00747     for (; i<len; ++i) { if (data[i]<=a) { q=1; break; }}
00748     return q;
00749 };
00750
00751 bool allGT(const T &a) const { return !anyLE(a); }
00752 bool allGE(const T &a) const { return !anyLT(a); }
00753 bool allLE(const T &a) const { return !anyGT(a); }
00754 bool allLT(const T &a) const { return !anyGE(a); }
00755
00756 bool allIn(const T &a1, const T &a2) const {
00757     bool q=1; size_t i=0;
00758     if (!len) wblog(FL,"WRN %s() got len=%d",FCT,len);
00759     for (; i<len; ++i) { if (data[i]<a1 || data[i]>a2) { q=0; break; }}
00760     return q;
00761 };
00762
00763 bool anySmallVals(const T eps=1E-15) const {
00764     size_t i=0; if (eps>0)
00765         { for (; i<len; ++i) { if (Wb::abs(data[i])<eps) return 1; }}
00766     else { for (; i<len; ++i) { if (data[i]) return 1; }}
00767     return 0;
00768 };
00769
00770 bool isEqual(const T* v) const {
00771     return (!memcmp(data, v, len*sizeof(T)));
00772 }
00773
00774 bool anyEqual(const wbvector<size_t> &I, const T &x,
00775     size_t *k=NULL) const;
00776 bool anyUnequal(const wbvector<size_t> &I, const T &x,
00777     size_t *k=NULL) const;
00778
00779 bool isEmpty() const {
00780     if ((!len) ^ (!data)) wblog(FL,
00781         "ERR severe data inconsistency (%ld,%p)",len,data);
00782     return (data ? 0 : 1);
00783 };

```

```

00782
00783     explicit operator bool() const { return isEmpty(); }
00784     bool operator! () const { return isEmpty(); }
00785
00786     bool operator==(const wbvector &v) const { return isEqual(v); };
00787     bool operator!=(const wbvector &v) const { return !isEqual(v); };
00788
00789     bool isEqual(const wbvector &v) const {
00790         bool q=1;
00791         if (this!=&v) {
00792             if (len!=v.len) { q=0; } else
00793                 if (data!=v.data) {
00794                     for (size_t i=0; i<len; ++i) {
00795                         if (data[i]!=v.data[i]) { q=0; break; }
00796                     }
00797                 }
00798             }
00799         return q;
00800     };
00801
00802     bool isEqualR(const wbvector &v) const {
00803         if (len!=v.len) return 0;
00804         for (size_t l=len-1, i=0; i<len; ++i) {
00805             if (data[i]!=v.data[l-i]) return 0;
00806         }
00807         return 1;
00808     };
00809
00810     bool operator^(const wbvector &v) const {
00811         if (this==&v) { return (len ? 0 : 1); }
00812         if (len!=v.len) return 0;
00813
00814         for (size_t i=0; i<len; ++i) {
00815             if (!(data[i]==0) ^ (v.data[i]==0)) return 0; }
00816
00817         return 1;
00818     };
00819
00820     bool deepEqualP(const wbvector &B) const;
00821
00822     char cmp(const wbvector &v) const;
00823
00824     bool operator< (const wbvector &v) const { return cmp(v)< 0; };
00825     bool operator<= (const wbvector &v) const { return cmp(v)<=0; };
00826     bool operator> (const wbvector &v) const { return cmp(v)> 0; };
00827     bool operator>= (const wbvector &v) const { return cmp(v)>=0; };
00828
00829     int operator< (const T &r) const {
00830         for (size_t i=0; i<len; ++i) if (data[i]>=r) return 0;
00831         return (len ? 1 : 0);
00832     };
00833
00834     T& el(size_t i) const {
00835         if (i>=len) wblog(FL,"ERR element index out of bounds (%d/%d)",i+1,len);
00836         return data[i];
00837     };
00838
00839     T elx(size_t i, const T &x) const {
00840         return (i<len ? data[i] : x);
00841     };
00842
00843     const T& operator[] (size_t i) const { return data[i]; };
00844     T& operator[] (size_t i) { return data[i]; };
00845
00846     T& last() const {
00847         if (!len) wblog(FL,"ERR %s() got empty %s vector",FCT,TSTR(T));
00848         return data[len-1];
00849     };
00850
00851     T& last(size_t &i) const {
00852         if (!len) wblog(FL,"ERR %s() got empty %s vector",FCT,TSTR(T));
00853         return data[i=len-1];
00854     };
00855
00856     template<class TI>
00857     T& last(TI &i_) const { size_t i=len-1; i_=i;
00858         if (!len) wblog(FL,"ERR %s() got empty %s vector",FCT,TSTR(T));
00859         return data[i];
00860     };
00861
00862     T& p2last() const {
00863         if (!len) wblog(FL,"ERR %s() got empty %s vector",FCT,TSTR(T));
00864         return data+(len-1);
00865     };
00866
00867     T* ref(size_t i) const { return (data+i); };
00868

```

```

00869     wbvector& setRand(double fac=1., double shift=0.);
00870     void setIdx (double shift=0., double fac=1.);
00871
00872     int set2Group(const wbperm &P,
00873         const WBINDEX &D, const T* S0=NULL, char iflag=0
00874     );
00875
00876     wbvector& Resize_(size_t n) {
00877         if (isref) wblog(FL,"ERR %s() got reference (%d)",FCT,isref);
00878
00879         if (n) {
00880             if (n<=len && (len<32 || (4*n)>len))
00881                 { len=n; }
00882             else { Resize(n); }
00883         }
00884         else init();
00885
00886         return *this;
00887     };
00888
00889     wbvector& Shorten2(size_t n) {
00890         if (!n) {
00891             if (isref) wblog(FL,"ERR %s() got reference (%d)",FCT,isref);
00892             init();
00893         }
00894         else if (n<len && (len<32 || (4*n)>len)) { len=n; }
00895         else if (n!=len) {
00896             if (n>len) wblog(FL,
00897                 "WRN %s() got increasing size %ld -> %ld",FCT,len,n);
00898             Resize(n);
00899         }
00900         return *this;
00901     };
00902
00903     wbvector& Resize(size_t n, const T* d=NULL) {
00904         if (isref) wblog(FL,"ERR %s() got reference (%d)",FCT,isref);
00905
00906         if (n==len) { return *this; }
00907         if (!len) { return RENEW(n,d); };
00908         if (!n) { len=0; if (data) WB_DELETE(data); return *this; }
00909
00910         T *d0=data;
00911         WB_NEW(data,n);
00912         MEM_COPY<T>(data, n, (n<len ? n : len), d0, d);
00913         len=n;
00914         WB_DELETE(d0); return *this;
00915     };
00916
00917     wbvector& resize(size_t n, wbvector& X) const {
00918         if (!n) { X.init(); } else
00919         if (n==len) { X=(*this) ; }
00920         else {
00921             WB_NEW(X.data,n); X.len=n;
00922             if (len) { MEM_COPY<T>(X.data, MIN(n,len), data); }
00923             if (n>len) { MEM_SET<T>(X.data+len,n-len); }
00924         }
00925         return X;
00926     };
00927
00928     wbvector& cat(const wbvector &v2) {
00929         Resize(len+v2.len, v2.data);
00930         return *this;
00931     };
00932     wbvector& Cat(const wbvector &v1, const wbvector &v2) {
00933         RENEW(v1.len+v2.len);
00934         MEM_COPY<T>(data,len,v1.len,v1.data,v2.data);
00935         return *this;
00936     };
00937     wbvector& Cat(const wbvector<wbvector > &vv);
00938     wbvector& Cat(const wbvector<wbvector* > &vv);
00939
00940     template <class T1, class T2, class T3, class T4>
00941     wbvector& Cat(
00942         const T1* v1, size_t l1, const T2* v2, size_t l2,
00943         const T3* v3=0, size_t l3=0,
00944         const T4* v4=0, size_t l4=0
00945     );
00946
00947     template <class T1, class T2, class T3, class T4>
00948     wbvector& Cat(
00949         const wbvector<T1> *v1,const wbvector<T2> *v2,
00950         const wbvector<T3> *v3=0,
00951         const wbvector<T3> *v4=0
00952     );
00953
00954     wbvector& Append(const wbvector &v2) {
00955         Resize(len+v2.len, v2.data);

```

```

00956         return *this;
00957     };
00958     wbvector& Append(const T &x) {
00959         Resize(len+1, &x);
00960         return *this;
00961     };
00962     wbvector& Append(size_t d2, const T *x) {
00963         Resize(len+d2, x);
00964         return *this;
00965     };
00966
00967     wbvector append(const wbvector &v) const {
00968         wbvector v2; append(v.data,v.len,v2);
00969         return v2;
00970     };
00971
00972     wbvector& append(const wbvector &v2, wbvector &vout) const {
00973         append(v2.data, v2.len, vout);
00974         return vout;
00975     };
00976     void append(const T *x, size_t d2, wbvector &vout) const {
00977         vout.RENEW(len+d2);
00978         MEM_COPY<T>(vout.data,vout.len,len,data,x);
00979     };
00980
00981     int recLess (const T* v) const;
00982     int recLessE(const T* v) const;
00983     int recCompare(const T* v) const;
00984
00985     T max() const;
00986     T max(size_t *k) const;
00987     T max(unsigned &k) const { T x;
00988         size_t l=0; x=max(&l); Wb::safeConvert (FL,l,k);
00989         return x;
00990     };
00991
00992     T max_(T x) const;
00993
00994     T min() const;
00995     T min(size_t *k) const;
00996     T min(unsigned &k) const { T x;
00997         size_t l=0; x=min(&l); Wb::safeConvert (FL,l,k);
00998         return x;
00999     };
01000     T min_(T x) const;
01001
01002     T aMax(size_t *k=NULL) const;
01003
01004     T aMax(unsigned &k) const { T x;
01005         size_t k_; x=aMax(&k_); Wb::safeConvert (FL,k_,k);
01006         return x;
01007     };
01008
01009     T aMin(char zflag=0, size_t *k=NULL) const;
01010
01011     T maxneg(size_t *k=NULL) const;
01012     T minpos(size_t *k=NULL) const;
01013
01014     T avg() const;
01015     T std() const;
01016
01017     void range(T &xmin, T&xmax) const;
01018     T range() const;
01019
01020     wbvector& Permute(const wbperm &P, char iflag=0);
01021     wbvector& permute(wbvector &, const wbperm &P, char iflag=0) const;
01022
01023     wbvector permute(const wbperm &P, char iflag=0) const {
01024         wbvector<T> X; permute(X,P,iflag);
01025         return X;
01026     };
01027
01028     wbvector& blockPermute(
01029         const wbperm &P, wbvector &, char iflag=0) const;
01030
01031     wbvector& BlockPermute(const wbperm &P, char iflag=0) {
01032         wbvector<T> x(*this);
01033         return x.blockPermute(P,*this,iflag);
01034     };
01035
01036     int blockCompare(
01037         const wbperm &pa, const wbvector &b, const wbperm &pb) const;
01038
01039     void get(const WBINDEX &I0, wbvector &v) const {
01040         const widx_t *I=I0.data;
01041         size_t i=0, n=I0.len; v.init(n);
01042

```

```

01043         for (; i<n; i++) {
01044             if (I[i]>=len) wblog(FL,
01045                 "ERR index out of bounds (%d/%d) !?", I[i], len);
01046             v.data[i]=data[I[i]];
01047         }
01048     };
01049
01050     void get(const wbvector<size_t> &I, T* v) const {
01051         for (size_t i=0; i<I.len; i++) {
01052             if (I[i]>=len)
01053                 wblog(FL, "ERR index out of bounds (%d/%d) !?", I[i], len);
01054             v[i]=data[I[i]];
01055         }
01056         return;
01057     };
01058
01059     void get(size_t i1, size_t i2, wbvector &v) const {
01060         if (i1>=len || i2>=len) wbdie(FL, "index out of range.");
01061         if (i2<i1)
01062             v.init();
01063         else v.init(i2-i1, data+i1);
01064     };
01065
01066     void selectSU(const WBINDEX &I);
01067     void select(const WBINDEX &I, T* r) const;
01068     wbvector& select(const WBINDEX &I, wbvector &a) const;
01069     wbvector& Select(const WBINDEX &I);
01070     wbvector& BlockSelect(const WBINDEX &I, const WBINDEX &D);
01071
01072     wbvector select(const WBINDEX &I) const {
01073         wbvector dI; select(I, dI);
01074         return dI;
01075     };
01076
01077     wbvector& getI(size_t k, wbvector &v) const;
01078     wbvector& getI(const wbindex &k, wbvector &v, char uflag=1) const;
01079
01080     wbvector& getI(const WBINDEX &k, wbvector &v, char uflag=1) const {
01081         return getI((const wbindex&)k, v, uflag);
01082     };
01083
01084     wbvector& initI(const wbvector &a, const wbindex &ia){
01085         return a.getI(ia, *this); };
01086     wbvector& initI(
01087         const wbvector &a, const wbindex &ia,
01088         const wbvector &b, const wbindex &ib,
01089         char uflag=1
01090     );
01091
01092     wbvector& Flip() {
01093         size_t i, m2=len/2, l=len-1;
01094         for (i=0; i<m2; i++) SWAP(data[i], data[l-i]);
01095         return *this;
01096     };
01097
01098     void flip(wbvector &a) const {
01099         if (&a==this) { a.Flip(); return; }
01100         a.init(len);
01101         for (size_t l=len-1, i=0; i<len; ++i) a.data[i]=data[l-i];
01102     };
01103
01104     wbvector flip() const {
01105         wbvector a; flip(a);
01106         return a;
01107     };
01108
01109     wbvector operator[] (const wbvector<size_t>& I) const {
01110         wbvector v(I.len);
01111         size_t i=0, *const k=I.data;
01112
01113         for (; i<I.len; ++i) {
01114             if (k[i]>=len) wblog(FL,
01115                 "ERR index out of bounds (%d: %d/%d)", i, I.max(), len);
01116             v[i]=data[k[i]];
01117         }
01118         return v;
01119     };
01120
01121     T prod() const {
01122         if (!len) wblog(FL, "WRN %s() empty data set (returning 0)", FCT);
01123         return prod(0);
01124     };
01125
01126     T prod(T x) const {

```

```

01130         if (len) {
01131             size_t i=1; x=data[0]; for (; i<len; ++i) { x*=data[i]; }
01132         }; return x;
01133     };
01134
01135     T prod(size_t i, size_t i2) const {
01136         T x=0;
01137         if (i<len && i<=i2) {
01138             if ((++i2)>len) { i2=len; }
01139             for (x=data[i++]; i<i2; ++i) { x*=data[i]; }
01140         }
01141         else wblog(FL,
01142             "ERR empty index set [%d,%d] / %d (returning 0)",i,i2,len);
01143         return x;
01144     };
01145
01146     T prod(size_t i, size_t i2, T x) const {
01147         if (long(i)<0) { i=len+i; }
01148         if (long(i2)<0) { i2=len+i2; }
01149         if (i<len && i<=i2) {
01150             if ((++i2)>len) { i2=len; }
01151             for (x=data[i++]; i<i2; ++i) { x*=data[i]; }
01152         }
01153         return x;
01154     };
01155
01156     template <class TI>
01157     T prod(const TI *I, size_t n) const {
01158         T x=0; if (n) { size_t i=1;
01159             x=el(I[0]); for (; i<n; ++i) { x*=el(I[i]); }
01160         }
01161         return x;
01162     };
01163
01164     template <class T1, class T2>
01165     T2 prod(const T1 *I, size_t n, T2 &s2) const {
01166         size_t i,k;
01167         char m[len]; for (i=0; i<len; ++i) m[i]=0;
01168         for (i=0; i<n; ++i) { k=I[i];
01169             if (k>len) wblog(FL,
01170                 "ERR %s() index out of bounds (%d/%d)",FCT,k,len);
01171             if ((++m[k])>1) wblog(FL,"ERR %s() index not unqie",FCT);
01172         }
01173         if (!len) { s2=0; return 0; }
01174         else {
01175             T2 s1=1; s2=1;
01176             for (i=0; i<len; ++i) { (m[i] ? s1 : s2) *= data[i]; }
01177             return s1;
01178         }
01179     };
01180
01181     void prod2(T &p1, T &p2, const T &p0=0) const {
01182         size_t l=len/2; if (len%2) wblog(FL,
01183             "ERR cannot split vector into two equal parts (%d)",len);
01184
01185         if (len==0) { p1=p0; p2=p0; return; }
01186         p1=data[0]; p2=data[1];
01187
01188         for (size_t i=1; i<l; i++) { p1*=data[i]; p2*=data[i+1]; }
01189     }
01190
01191     void info(const char *istr) const;
01192     void print(const char *istr, char mflag=0) const;
01193     void printdata(const char *istr=0) const {
01194         if (istr && istr[0]) printf("%s = ",istr);
01195         printf("[ %s ]\n",STR_(this));
01196     };
01197
01198     void put(
01199         const char *F, int L,
01200         const char *vname="ans", char tflag=0, const char *ws="base"
01201     ) const;
01202
01203     void put(const char *vname="ans", char tflag=0, const char *ws="base"
01204     ) const { put(0,0,vname,tflag,ws); }
01205
01206     void dispWithMsg(const char* file, int line,
01207         const char *msg, const char* vname="") const {
01208         print(vname);
01209         wblog(file,line,msg);
01210     }
01211
01212     void mat2mx(mxArray* &a, char dim=2) const;
01213     void mat2mxs(mxArray* a, char field_nr, char dim=2) const;
01214
01215     mxArray* toMx_Struct(const char tflag=0) const;
01216     mxArray* toMxP_S(const char tflag=0) const;

```



```

01217
01218     mxArray* toMx (const char tflag=0) const { return toMx_Struct (tflag); };
01219     mxArray* toMxP(const char tflag=0) const { return toMx_P(tflag); };
01220
01221     mxArray* toMx_base(const char tflag=0) const {
01222         size_t m=1, n=len; if (tflag) { m=n; n=1; }
01223         return Mx::Array<double>(m,n).copyFromTR(data);
01224     };
01225
01226     mxArray* toMx_base_d(const char tflag=0) const {
01227         size_t m=1, n=len; if (tflag) { m=n; n=1; }
01228         return Mx::Array<double>(m,n).copyFromTR(data);
01229     };
01230
01231     mxArray* toMx_offset(const T& offset, const char tflag=0) const;
01232
01233     mxArray* mxCreateStruct(unsigned m, unsigned n) const;
01234     void add2MxStruct(mxArray *S, unsigned i, char tflag=0) const;
01235     void add2MxStruct(mxArray *S, const char *vname, char tflag=0) const;
01236
01237     mxArray* mxCreateCell(unsigned m, unsigned n) const;
01238     void add2MxCell(mxArray *S, unsigned i, char tflag=0) const;
01239
01240     void putg(const char *vname="ans", char tflag=0) const {
01241         put(vname,tflag,"global");
01242         return;
01243     };
01244
01245     void putx(const char *vname="ans", char tflag=0) const {
01246         put(vname,tflag,"base");
01247         return;
01248     };
01249
01250     wbsting toStr(int n, const char *sep) const;
01251     wbsting toStr() const;
01252
01253     wbsting sizeStr() const;
01254
01255     wbsting toStrf(
01256         const char *fmt0="",
01257         const char *sep=" ",
01258         unsigned stride=0,
01259         const char *sep2=""
01260     ) const;
01261
01262     T *data;
01263
01264     size_t len;
01265
01266     char isref;
01267
01268     protected:
01269
01270     wbvector& RENEW(size_t n, const T* d=NULL, size_t l=-1, char init=1) {
01271         if (data) {
01272             if (isref ) { data=NULL; } else
01273             if (n!=len) { WB_DELETE(data); }
01274         }
01275         isref=0; len=n; if (!n) { return *this; }
01276
01277         if (!data) { WB_NEW(data,len); }
01278
01279         if (long(l)<0) { l=len; }
01280
01281         if (d && l) {
01282             MEM_COPY<T>(data,l<len ? l:len,d); if (l<len && init) {
01283                 MEM_COPY<T>(data+l,len-l,NULL); }
01284         }
01285         else if (init) {
01286             MEM_COPY<T>(data, len, NULL);
01287         }
01288
01289         return *this;
01290     };
01291
01292     wbvector& RENEW_VAL(size_t n, const T& x) {
01293         if (isref) { data=NULL; isref=0; } else
01294         if (n!=len && data) { WB_DELETE(data); }
01295
01296         len=n;
01297         if (len) {
01298             if (!data) { WB_NEW(data,len); }
01299             for (size_t i=0; i<len; ++i) { data[i]=x; }
01300         }
01301         return *this;
01302     };
01303

```

```

01304 private:
01305
01306 };
01307
01308 template <class T>
01309 wbvector<T>& wbvector<T>::unRef() {
01310
01311     if (!isref) return *this;
01312     if (!len) {
01313         if (data) wblog(FL, "ERR data=%lX (%d)", data, len);
01314         isref=0; return *this;
01315     }
01316
01317     T const* const d0=data;
01318
01319     WB_NEW(data, len); MEM_COPY<T>(data, len, d0);
01320     isref=0;
01321
01322     return *this;
01323 };
01324
01325 template <class T>
01326 bool wbvector<T>::deepEqualP(const wbvector<T> &B) const {
01327     if (len!=B.len) return 0;
01328
01329     if (data!=B.data) {
01330         for (size_t i=0; i<len; i++) { if (data[i]==B.data[i]) continue;
01331             if ((!data[i] || !B.data[i])) return 0; else
01332             if (!(*data[i]==(*B.data[i]))) return 0;
01333         }
01334     }
01335
01336     return 1;
01337 };
01338
01339 template <class T> inline
01340 char wbvector<T>::cmp(const wbvector<T> &v) const {
01341
01342     size_t i=0, n=MIN(len, v.len); const T *b=v.data;
01343     for (; i<n; ++i) {
01344         if (data[i]<b[i]) return -1;
01345         if (data[i]>b[i]) return +1;
01346     }
01347
01348     if (len<v.len) return -1;
01349     if (len>v.len) return +1; else return 0;
01350 };
01351
01352 template <class T> inline
01353 bool wbvector<T>::checkSameLength(
01354     const wbvector<T> &a,
01355     const char *F, int L, const char *istr
01356 ) const {
01357
01358     bool i=(len==a.len);
01359     if (!i && F) {
01360         if (istr && istr[0]) sprintf_str("%s() - ", istr); else str[0]=0;
01361         wblog(F, L, "ERR %svector length mismatch (%d/%d)", str, len, a.len);
01362     }
01363     return i;
01364 };
01365
01366 template <class T> inline
01367 bool wbvector<T>::checkSameLength(
01368     const wbvector<T> &a,
01369     const wbvector<T> &b,
01370     const char *F, int L, const char *istr
01371 ) const {
01372
01373     bool i=(len==a.len && len==b.len);
01374     if (!i && F) {
01375         if (istr && istr[0]) sprintf_str("%s() - ", istr); else str[0]=0;
01376         wblog(F, L, "ERR %svector length mismatch (%d,%d/%d)", str, a.len, b.len, len);
01377     }
01378     return i;
01379 };
01380
01381 template <class T> inline
01382 void wbvector<T>::getReal(wbvector<double> &R) const {
01383     R.init(len);
01384     for (size_t i=0; i<len; ++i) { R.data[i]=double(data[i]); }
01385 };
01386
01387 template <> inline
01388 void wbvector<wbcomplex>::getReal(wbvector<double> &R) const {
01389     R.init(len);
01390     for (size_t i=0; i<len; ++i) { R.data[i]=data[i].r; }

```

```

01391 };
01392
01393 template <class T> inline
01394 void wbvector<T>::getImag(wbvector<double> &I) const {
01395     I.init(len);
01396 };
01397
01398 template <> inline
01399 void wbvector<wbcomplex>::getImag(wbvector<double> &I) const {
01400     I.init(len);
01401     for (size_t i=0; i<len; ++i) { I.data[i]=data[i].i; }
01402 };
01403
01404 template <> inline
01405 void wbvector<wbcomplex>::set(
01406     const wbvector<double> &R, const wbvector<double> &I) {
01407     size_t i=0; const double *re=R.data, *im=I.data;
01408
01409     init(R.len ? R.len : I.len);
01410
01411     if (R.len) {
01412         if (I.len) {
01413             if (R.len!=I.len) wblog(FL,
01414                 "ERR %s() got length inconsistency (%d/%d)",FCT,R.len,I.len);
01415             for (; i<len; ++i) { data[i].init(re[i],im[i]); }
01416         }
01417         else { for (; i<len; ++i) { data[i].init(re[i],0); } }
01418     }
01419     else if (I.len) { for (; i<len; ++i) { data[i].init(0,im[i]); } }
01420 };
01421
01422 template <> inline
01423 wbvector<wbcomplex>& wbvector<wbcomplex>::Conj() {
01424     for (size_t i=0; i<len; i++) data[i].Conj();
01425     return *this;
01426 };
01427
01428 template <class T> inline
01429 wbvector<T>& wbvector<T>::applyFlag(
01430     char &flag, const T &fac, wbvector<T> &a
01431 ) {
01432     if (!strchr("NCTc",flag))
01433         wblog(FL,"ERR %s() invalid flag %c<%d>",__FUNCTION__,flag,flag);
01434
01435     if (flag=='c') flag='C';
01436     if (flag=='C' && typeid(T)!=typeid(wbcomplex)) flag='N';
01437
01438     if (fac!=1 || flag=='C') { a=(*this);
01439         if (flag=='C') a.Conj();
01440         if (fac!=1) a*=fac;
01441     }
01442     else a<<*this;
01443 };
01444
01445 template <class T> inline
01446 T wbvector<T>::min() const {
01447     if (len) { size_t i=1; T x=data[0];
01448         for (; i<len; ++i) { if (x>data[i]) { x=data[i]; } }
01449         return x;
01450     }
01451     wblog(FL,"ERR %s() got empty vector",FCT);
01452     return 0;
01453 };
01454
01455 template <class T> inline
01456 T wbvector<T>::min_(T x) const {
01457     if (len) { size_t i=1; x=data[0];
01458         for (; i<len; ++i) { if (x>data[i]) { x=data[i]; } }
01459         return x;
01460     }
01461 };
01462
01463 template <class T> inline
01464 T wbvector<T>::min(size_t *k) const {
01465     if (!k) { return min(); }
01466     if (len) { size_t i=1; T x=data[0]; *k=0;
01467         for (; i<len; ++i) { if (x>data[i]) { x=data[i]; *k=i; } }
01468         return x;
01469     }
01470     wblog(FL,"ERR %s() got empty vector",FCT);
01471     return 0;
01472 };
01473
01474 template <class T> inline
01475 T wbvector<T>::max() const {

```

```

01478
01479     if (len) { unsigned i=1; T x=data[0];
01480         for (; i<len; ++i) { if (x<data[i]) { x=data[i]; } }
01481         return x;
01482     }
01483     wblog(FL,"ERR %s() got empty vector",FCT);
01484     return 0;
01485 };
01486
01487 template <class T> inline
01488 T wbvector<T>::max_(T x) const {
01489     if (len) { size_t i=1; x=data[0];
01490         for (; i<len; ++i) { if (x<data[i]) { x=data[i]; } }
01491     }
01492     return x;
01493 };
01494
01495 template <class T> inline
01496 T wbvector<T>::max(size_t *k) const {
01497     if (!k) { return max(); }
01498     if (len) { size_t i=1; T x=data[0]; *k=0;
01499         for (; i<len; ++i) { if (x<data[i]) { x=data[i]; *k=i; } }
01500     }
01501     wblog(FL,"ERR %s() got empty vector",FCT);
01502     return 0;
01503 };
01504
01505 template <class T> inline
01506 T wbvector<T>::minpos(size_t *k) const {
01507     if (len) {
01508         T x=0; size_t i=0; if (k) { (*k)=-1; }
01509         for (; i<len; ++i) { if (data[i]>0) {
01510             x=data[i]; if (k) { (*k)=i; }; ++i; break;
01511         } }
01512         for (; i<len; ++i) { if (data[i]>0 && x>data[i]) {
01513             x=data[i]; if (k) { (*k)=i; }
01514         } }
01515         return x;
01516     }
01517     wblog(FL,"WRN %s() of empty vector",FCT);
01518     return 0;
01519 };
01520
01521 template <class T> inline
01522 T wbvector<T>::maxneg(size_t *k) const {
01523     if (len) {
01524         T x=0; size_t i=0; if (k) { (*k)=-1; }
01525         for (; i<len; ++i) { if (data[i]<0) {
01526             x=data[i]; if (k) { (*k)=i; }; ++i; break;
01527         } }
01528         for (; i<len; ++i) { if (data[i]<0 && x<data[i]) {
01529             x=data[i]; if (k) { (*k)=i; }
01530         } }
01531         return x;
01532     }
01533     wblog(FL,"WRN %s() of empty vector",FCT);
01534     return 0;
01535 };
01536
01537 template <class T> inline
01538 T wbvector<T>::aMax(size_t *k_) const {
01539     if (len) {
01540         size_t k=-1, i=0; T a,x=0;
01541         for (; i<len; ++i) {
01542             a=Wb::abs(data[i]); if (x<a) { x=a; k=i; }
01543         }
01544         if (k_) (*k_)=k;
01545         return x;
01546     }
01547     else {
01548         if (k_) { (*k_)=-1; wblog(FL,"WRN %s() got null vector",FCT); }
01549         return 0;
01550     }
01551 };
01552
01553 template <class T> inline
01554 T wbvector<T>::aMin(char zflag, size_t *k_) const {
01555     if (!len) {
01556         if (k_) { (*k_)=-1; wblog(FL,"WRN %s() got null vector",FCT); }
01557         return 0;
01558     }
01559 }
01560
01561
01562
01563
01564

```

```

01565     size_t i=0, k=-1; double a,x=0;
01566
01567     if (zflag==0) {
01568         x=Wb::abs(data[i++]); k=0; if (x!=0) {
01569             for (; i<len; ++i) { a=Wb::abs(data[i]); if (x>a) {
01570                 x=a; k=i; if (x==0) break;
01571             }
01572         }
01573     }
01574     else {
01575         for (; i<len; ++i) {
01576             a=Wb::abs(data[i]); if (a) { x=a; k=i; break; }
01577         }
01578         for (; i<len; ++i) { a=Wb::abs(data[i]);
01579             if (a && x>a) { x=a; k=i; }
01580         }
01581     }
01582     if (k_) (*k_)=k;
01583     return x;
01584 };
01585
01586 template <class T> inline
01587 void wbvector<T>::range(T &xmin, T&xmax) const {
01588     if (len==0) {
01589         xmin=xmax=0;
01590         wblog(FL,"WRN %s() of null vector",FCT);
01591     }
01592     else {
01593         xmin=xmax=data[0];
01594         for (size_t i=1; i<len; i++) {
01595             if (xmin>data[i]) xmin=data[i];
01596             if (xmax<data[i]) xmax=data[i];
01597         }
01598     }
01599 };
01600
01601 template <class T> inline
01602 T wbvector<T>::range() const {
01603     T xmin, xmax; range(xmin,xmax);
01604     return (xmax-xmin);
01605 };
01606
01607 template <class T> inline
01608 T wbvector<T>::avg() const {
01609
01610     if (!len) {
01611         wblog(FL,"WRN %s() of null vector !?",FCT);
01612         return 0;
01613     }
01614     else {
01615         T x=data[0];
01616         for (size_t i=1; i<len; ++i) { x+=data[i]; }
01617         return (x/T(len));
01618     }
01619 };
01620
01621 template <class T> inline
01622 T wbvector<T>::std() const {
01623
01624     if (len<2) {
01625         wblog(FL,"WRN %s() of vector of length %d !?",FCT,len);
01626         return 0;
01627     }
01628     else {
01629         T x=data[0], x2=data[0]*data[0];
01630         for (size_t i=1; i<len; ++i) { x+=data[i]; x2+=data[i]*data[i]; }
01631         return Wb::sqrt( ( x2 - (x*x)/T(len) ) / T(len-1) );
01632     }
01633 };
01634
01635 template <class T> inline
01636 int wbvector<T>::findClosest(T r) const {
01637     if (len==0) return -1;
01638
01639     T dbl, eps=Wb::abs(data[0]-r); size_t i,k=0;
01640
01641     for (i=1; i<len; i++) {
01642         dbl = Wb::abs(data[i]-r);
01643         if (dbl<eps) { k=i; eps=dbl; if (eps==0) break; }
01644     }
01645
01646     return k;
01647 }
01648
01649 template <class T>
01650 int wbvector<T>::findClosestSorted(T r) const {
01651     if (len==0) return -1;

```

```

01652
01653     size_t i, k, n1=0, n2=len, e=0;
01654     char c, cref;
01655
01656     if (len==0) return -1;
01657     if (len==1) return 0;
01658
01659     cref=NUMCMP(data[0], data[len-1]);
01660
01661     if (cref) {
01662         if (len>6) {
01663             for (i=1; i<4; i++) if (NUMCMP(data[i-1],data[i])==-cref) e++;
01664             for (i=len-3; i<len; i++)
01665                 if (NUMCMP(data[i-1],data[i])==-cref) e++;
01666         }
01667         else
01668             for (i=1; i<len; i++) if (NUMCMP(data[i-1],data[i])==-cref) e++;
01669     }
01670     else for (i=1; i<len; i++) if (NUMCMP(data[i-1],data[i])) e++;
01671
01672     if (e) wblog(FL, "ERR Data not sorted! (%d)", e);
01673     if (cref==0) { wblog(FL,
01674         "WRN find closest: all data constant!?\n(%g; %g %dx)",
01675         (double)r, (double)data[0], len); return 0;
01676     }
01677
01678     c=NUMCMP(data[len-1],r); if (c!=-cref) return len-1;
01679     c=NUMCMP(data[0],r); if (c!= cref) return 0;
01680
01681     for (k=0;;) {
01682         if (c==cref) {
01683             n1=n2/2; n2=-n1; if (n1) k+=n1; else {
01684                 if (k+1<len && (Wb::abs(r-data[k]) > Wb::abs(r-data[k+1]))) k++;
01685                 break;
01686             }
01687         }
01688         else if (c) {
01689             n2=n1/2; n1=-n2; if (n2) k-=n2; else {
01690                 if (k) if (Wb::abs(r-data[k]) > Wb::abs(r-data[k-1])) k--;
01691                 break;
01692             }
01693         }
01694         else break;
01695
01696         c=NUMCMP(data[k],r);
01697     }
01698
01699     return k;
01700 }
01701
01702 template <class T>
01703 wbvector<T>& wbvector<T>::setRand(double fac, double shift) {
01704     size_t i;
01705     static char first_call=1;
01706
01707     if (first_call) { wb_srand(); first_call=0; }
01708     fac/=(double)RAND_MAX;
01709
01710     if (shift==0.)
01711         for (i=0; i<len; ++i) data[i]=(T)(fac*rand());
01712     else for (i=0; i<len; ++i) data[i]=(T)(fac*rand()+shift);
01713
01714     return *this;
01715 };
01716
01717 template<>
01718 wbvector<wbcomplex>& wbvector<wbcomplex>::setRand(double fac, double shift) {
01719     size_t i;
01720     static char first_call=1;
01721
01722     if (first_call) { wb_srand(); first_call=0; }
01723     fac/=(double)RAND_MAX;
01724
01725     if (shift==0.) {
01726         for (i=0; i<len; ++i)
01727             data[i].set(fac*rand(), fac*rand());
01728     }
01729     else {
01730         for (i=0; i<len; ++i)
01731             data[i].set(fac*rand()+shift, fac*rand()+shift);
01732     }
01733
01734     return *this;
01735 };
01736
01737 template <class T> inline
01738 void wbvector<T>::setIdx(double shift, double fac) {

```

```

01739     size_t i;
01740
01741     if (shift==0.) {
01742         if (fac==1.)
01743             for (i=0; i<len; i++) data[i]=(T)(i);
01744         else for (i=0; i<len; i++) data[i]=(T)(fac*i);
01745     }
01746     else {
01747         if (fac==1.)
01748             for (i=0; i<len; i++) data[i]=(T)(i+shift);
01749         else for (i=0; i<len; i++) data[i]=(T)(fac*i+shift);
01750     }
01751 };
01752
01753 template <class T>
01754 void wbvector<T>::mat2mx(mxArray* &a, char dim) const {
01755
01756     if (dim==1) a=mxCreateDoubleMatrix(len,1,mxREAL); else
01757     if (dim==2) a=mxCreateDoubleMatrix(1,len,mxREAL);
01758     else wblog(FL,"ERR %s() invalid dim=%d",FCT,dim);
01759
01760     double *d=mxGetDoubles(a);
01761     for (size_t i=0; i<len; ++i) d[i]=(double)data[i];
01762 };
01763
01764 template <class T>
01765 void wbvector<T>::mat2mxs(mxArray* a, char fid, char dim) const {
01766
01767     size_t i=-1, m=1, n=len;
01768
01769     if (!mxIsStruct(a) || (i=mxGetNumberOfElements(a))!=1) wblog(FL,
01770 "ERR %s() invalid input structure (n=%d)",FCT,i);
01771     if (fid>=int(i=mxGetNumberOfFields(a))) wblog(FL,
01772 "ERR %s() field index out of bounds (%d/%d)",FCT,fid,i);
01773
01774     if (dim==1) { m=len; n=1; } else
01775     if (dim==2) { m=1; n=len; } else
01776         wblog(FL,"ERR %s() invalid dim=%d",FCT,dim);
01777
01778     mxArray *b=mxCreateDoubleMatrix(m,n,mxREAL);
01779     double *d=mxGetDoubles(b);
01780
01781     for (i=0; i<len; ++i) { d[i]=(double)data[i]; }
01782
01783     mxSetFieldByNumber(a,0,fid,b);
01784 };
01785
01786 template <class T> inline
01787 int wbvector<T>::recLess(const T* v) const {
01788
01789     for (size_t i=0; i<len; i++)
01790         if (data[i]<v[i]) return 1; else
01791         if (data[i]>v[i]) return 0;
01792
01793     return 0;
01794 }
01795
01796 template <class T> inline
01797 int wbvector<T>::recLessE(const T* v) const {
01798
01799     for (size_t i=0; i<len; i++)
01800         if (data[i]<v[i]) return 1; else
01801         if (data[i]>v[i]) return 0;
01802
01803     return 1;
01804 }
01805
01806 template <class T> inline
01807 int wbvector<T>::recCompare(const T* v) const {
01808
01809     for (size_t i=0; i<len; i++)
01810         if (data[i]<v[i]) return -1; else
01811         if (data[i]>v[i]) return +1;
01812
01813     return 0;
01814 }
01815
01816 template<>
01817 mxArray* wbvector<long>::toMx_base(const char tflag) const {
01818     size_t m=1, n=len; if (tflag) { m=n; n=1; }
01819     return Mx::Array<long>(m,n).copyFromTR(data);
01820 };
01821
01822 template<>
01823 mxArray* wbvector<size_t>::toMx_base(const char tflag) const {
01824     size_t m=1, n=len; if (tflag) { m=n; n=1; }
01825     return Mx::Array<size_t>(m,n).copyFromTR(data);

```

```

01826 };
01827
01828 template<>
01829 mxArray* wbvector<wbcomplex>::toMx_base(const char tflag) const {
01830     size_t m=1, n=len; if (tflag) { m=n; n=1; }
01831     return Mx::Array<wbcomplex>(m,n).copyFromTR(data);
01832 };
01833
01834 template<> int wbvector<double>::init(
01835     const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
01836 { return init_base(F,L,a,istr,tcheck,ref); }
01837
01838 template<> int wbvector<wbcomplex>::init(
01839     const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
01840 { return init_base(F,L,a,istr,tcheck,ref); }
01841
01842 template<> int wbvector<char>::init(
01843     const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
01844 { return init_base(F,L,a,istr,tcheck,ref); }
01845
01846 template<> int wbvector<int>::init(
01847     const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
01848 { return init_base(F,L,a,istr,tcheck,ref); }
01849
01850 template<> int wbvector<unsigned>::init(
01851     const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
01852 { return init_base(F,L,a,istr,tcheck,ref); }
01853
01854 template<> int wbvector<size_t>::init(
01855     const char *F, int L, const mxArray *a, const char *istr, char tcheck, char ref)
01856 { return init_base(F,L,a,istr,tcheck,ref); }
01857
01858 template<> mxArray* wbvector<double>::toMx(const char tflag) const {
01859     return toMx_base(tflag); }
01860 template<> mxArray* wbvector<wbcomplex>::toMx(const char tflag) const {
01861     return toMx_base(tflag); }
01862
01863 template<> mxArray* wbvector<unsigned>::toMx(const char tflag) const {
01864     return toMx_base(tflag); };
01865 template<> mxArray* wbvector<int>::toMx(const char tflag) const {
01866     return toMx_base(tflag); }
01867 template<> mxArray* wbvector<char>::toMx(const char tflag) const {
01868     return toMx_base(tflag); }
01869 template<> mxArray* wbvector<uint8_T>::toMx(const char tflag) const {
01870     return toMx_base(tflag); }
01871 template<> mxArray* wbvector<short>::toMx(const char tflag) const {
01872     return toMx_base(tflag); };
01873 template<> mxArray* wbvector<uint16_T>::toMx(const char tflag) const {
01874     return toMx_base(tflag); };
01875 template<> mxArray* wbvector<long>::toMx(const char tflag) const {
01876     return toMx_base(tflag); };
01877 template<> mxArray* wbvector<size_t>::toMx(const char tflag) const {
01878     return toMx_base(tflag); };
01879
01880 template<class T>
01881 mxArray* wbvector<T>::toMx_Struct(const char tflag) const {
01882
01883     mxArray *S;
01884
01885     if (tflag)
01886         S=data->mxCreateStruct(len,1);
01887     else S=data->mxCreateStruct(1,len);
01888
01889     for (size_t i=0; i<len; ++i) { data[i].add2MxStruct(S,i); }
01890     return S;
01891 }
01892
01893 template<class T>
01894 mxArray* wbvector<T>::toMxP_S(const char tflag) const {
01895
01896     mxArray *S;
01897
01898     if (tflag)
01899         S=(*data)->mxCreateStruct(len,1);
01900     else S=(*data)->mxCreateStruct(1,len);
01901
01902     for (size_t i=0; i<len; ++i) { data[i]->add2MxStruct(S,i); }
01903     return S;
01904 };
01905
01906 template<class T>
01907 mxArray* wbvector<T>::toMx_offset(const T& offset, const char tflag)
01908 const {
01909     if (!len) { return mxCreateDoubleMatrix(0,0,mxREAL); }
01910
01911     mxArray *a=(tflag ?
01912         mxCreateDoubleMatrix(len,1,mxREAL) :

```



```

01913         mxCreateDoubleMatrix(1,len,mxREAL)
01914     );
01915
01916     double *d=mxGetDoubles(a);
01917     for (size_t i=0; i<len; ++i) { d[i]=double(data[i]+offset); }
01918
01919     return a;
01920 };
01921
01922 template <class T>
01923 void wbvector<T>::put(
01924     const char *file, int line, const char *vname, char tflag,
01925     const char* ws
01926 ) const {
01927
01928     size_t i;
01929     mxArray *a=toMx(tflag);
01930
01931     if (file) wblog(file,line,
01932         "I/O putting vector '%s' to %s%s",vname,ws,tflag ? " (t)" : ".");
01933
01934     i=mexPutVariable(ws, vname, a);
01935
01936     if (i==1) {
01937         info("this"); wblog(__FILE__,
01938             "ERR failed to put variable %s into workspace %s.",vname,ws);
01939     }
01940
01941     mxDestroyArray(a);
01942     return;
01943 }
01944
01945 inline WBINDEX Index(size_t i1, size_t i2) {
01946     WBINDEX ii;
01947
01948     if (i1>i2) return ii;
01949
01950     ii.init(i2-i1+1);
01951     for (size_t i=0; i<ii.len; i++) ii[i]=i+i1;
01952
01953     return ii;
01954 };
01955
01956 inline char isUniqueIdxSet(const WBINDEX &I0, widx_t imax) {
01957
01958     size_t i, n=I0.len, e=0;
01959     const widx_t *I=I0.data;
01960     wbvector<char> mark(imax); char *m=mark.data;
01961
01962     for (i=0; i<n; i++) if (I[i]<imax) m[I[i]]++; else e++;
01963     for (i=0; i<imax; i++) if (m[i]>1) e++;
01964
01965     return (e==0);
01966 }
01967
01968 char validPerm(const WBPERM &P, wperm_t d) {
01969     char rval=0, id=0; size_t i=0, n=P.len;
01970     const wperm_t *p=P.data;
01971     wbvector<char> mark(n);
01972
01973     if (swperm_t(d)>=0) { if (P.len!=d) return rval; }
01974     for (i=0; i<n; ++i) {
01975         if (p[i]==i) { ++id; } else
01976             if (p[i]>=n || mark[p[i]]++) { return rval; }
01977     }
01978
01979     return (rval=(unsigned(id)==n? 2:1));
01980 };
01981
01982 inline void getIPerm(const WBPERM &P, WBPERM &iP) {
01983
01984     size_t i,n=P.len;
01985
01986     #ifdef __WBDEBUG__
01987         if (!validPerm(P))
01988             wbdie(FLINE,"invalid permutation.");
01989     #endif
01990
01991     iP.init(n);
01992     for (i=0; i<n; i++) iP[P[i]]=i;
01993
01994     return;
01995 }
01996
01997 }
01998
01999 char isIdentityPerm(const WBPERM &P) {

```

```

02000
02001     for (size_t i=0; i<P.len; i++) if (P[i]!=i) return 0;
02002     return 1;
02003 }
02004
02005 #endif
02006

```

5.64 xmap.hh

```

00001 #ifndef __WB_XMAP_HCC__
00002 #define __WB_XMAP_HCC__
00003
00004 /* ----- *
00005  * class XMAP - map to store contraction pattern
00006  * AW (C) Jun 2006
00007  * ----- */
00008
00009 // ----- //
00010 // ----- //
00011 // central element for contraction map
00012
00013 class XMAP_Node {
00014 public:
00015     XMAP_Node ()
00016     : Z(NULL), R(NULL), I(NULL),
00017       tflag(0), ic(0), il(0), i2(0), isref(0), z(1.) {};
00018
00019 ~XMAP_Node () {};
00020
00021 free(){
00022     if (isref) { Z=R=NULL; }
00023     else {
00024         if (Z) { delete Z; Z=NULL; }
00025         if (R) { delete R; R=NULL; }
00026     }
00027 };
00028
00029 void init(const mxArray *a);
00030
00031 wbvector<unsigned> size() const {
00032     if (Z) return (*Z).SIZE; else
00033     if (R) return (*R).SIZE;
00034     else return wbvector<unsigned>();
00035 };
00036
00037 unsigned rank() const {
00038     if (Z) return (*Z).rank(); else
00039     if (R) return (*R).rank();
00040     else return 0;
00041 };
00042
00043 bool sameSize(const XMAP_Node &B) const {
00044     if ((!R ^ !Z) || (!B.R ^ !B.Z)) wblog(FL,
00045         "ERR Severe XMAP inconsistency.");
00046     return (
00047         (R ? (*R).SIZE : (*Z).SIZE) !=
00048         (B.R ? (*B.R).SIZE : (*B.Z).SIZE)
00049     );
00050 };
00051
00052 template<class T>
00053 void set(
00054     const wbArray<T> &A,
00055     unsigned ic, unsigned il, unsigned i2, wbcomplex z, char ref=1
00056 );
00057
00058 void set(
00059     unsigned ic, unsigned il, unsigned i2, wbcomplex z, char ref=1
00060 );
00061
00062 wbArray<wbcomplex> *Z;
00063 wbArray<double> *R;
00064
00065 char tflag;
00066 wbperm P;
00067
00068 unsigned ic;
00069 unsigned il;
00070 unsigned i2;

```

```

00074
00075     unsigned isref;
00076
00077     wbcomplex z;
00078
00079 protected:
00080 private:
00081
00082 };
00083
00084 class XMAP : public wbvector<XMAP_Node> {
00085 public:
00086     XMAP(unsigned n) : isref(1) { initDef(n); };
00087
00088     ~XMAP() {};
00089
00090     free() {
00091         for (unsigned i=0; i<len; i++)
00092             data[i].free();
00093     };
00094
00095     void init(unsigned n) { free(); initDef(n); };
00096
00097     template<class T>
00098     void set(
00099         unsigned k, const wbArray<T> &A,
00100         unsigned ic, unsigned il, unsigned i2, wbcomplex z
00101     ) { data[k].set(A,ic,il,i2,z, isref); };
00102
00103     void set(
00104         unsigned k,
00105         unsigned ic, unsigned il, unsigned i2, wbcomplex z
00106     ) { data[k].set(ic,il,i2,z, isref); };
00107
00108     char isref;
00109
00110 protected:
00111 private:
00112 };
00113
00114 void XMAP_Node::init(const mxArray *a) {
00115
00116     if (!a) wblog(FL,
00117         "ERR Input mxArray is NULL.");
00118
00119     if (!mxIsDouble(a) || mxIsSparse(a)) wblog(FL,
00120         "ERR Need full double array upon input.");
00121
00122     free();
00123     (*( mxIsComplex(a) ? Z : R )).init(a);
00124 }
00125
00126 template<class T>
00127 void XMAP_Node::set(
00128     unsigned k, const wbArray<T> &A,
00129     unsigned ic_, unsigned il_, unsigned i2_, wbcomplex z_, char ref
00130 ){
00131     wblog(FL,"ERR XMAP::set() not available for data type %s",
00132         TSTR(T));
00133 };
00134
00135 void XMAP_Node::set(
00136     unsigned k, const wbArray<wbcomplex> &A,
00137     unsigned ic_, unsigned il_, unsigned i2_, wbcomplex z_, char ref
00138 ){
00139     ic=ic_; il=il_; i2=i2_; z=z_;
00140
00141     if (A.rank()!=2) wblog(FL,
00142         "ERR expecting rank-2 tensors to contract.");
00143
00144     if (ref) {
00145         Z=&A; isref=1;
00146     }
00147     else {
00148         if (!Z) Z = new wbArray<wbcomplex>;
00149         (*Z)=A; (*Z)*=z;
00150         z=1.;
00151     }
00152 };
00153
00154 void XMAP_Node::set(
00155     const wbArray<double> &A,
00156     unsigned ic_, unsigned il_, unsigned i2_, wbcomplex z_, char ref
00157 ){
00158     ic=ic_; il=il_; i2=i2_; z=z_;
00159
00160     if (A.rank()!=2) wblog(FL,

```

```

00161     "ERR expecting rank-2 tensors to contract.");
00162
00163     if (ref) {
00164         R=&A; isref=1;
00165     }
00166     else {
00167         if (z.i==0.) {
00168             if (!R) R = new wbArray<double>;
00169             (*R)=A; (*R)*=z.r;
00170         }
00171         else {
00172             if (!Z) Z = new wbArray<wbcomplex>;
00173             (*Z).set(A, wbArray<double>());
00174             (*Z)*=z;
00175         }
00176         z=1.;
00177     }
00178 };
00179
00180 void XMAP_Node::set(
00181     unsigned ic_, unsigned i1_, unsigned i2_, wbcomplex z_
00182 ){
00183     ic=ic_; i1=i1_; i2=i2_; z=z_;
00184
00185     if (Z && R) wblog(FL,
00186         "ERR Both Z and R are set (0x%lX, 0x%lX) ???", Z, R);
00187
00188     if (Z) (*Z)*=z; else
00189     if (R) {
00190         if (z.i==0) (*R)*=z.r;
00191         else {
00192             Z = new wbArray<wbcomplex>;
00193             Z.set(R, wbArray<double>()); (*Z)*=z;
00194             delete R; R=NULL;
00195         }
00196     };
00197
00198     z=1.;
00199 };
00200
00201 void XMAP::init(const mxArray *a) {
00202     unsigned i,N;
00203
00204     if (mxIsCell(a)) {
00205         N=mxGetNumberOfElements(a);
00206         init(N);
00207
00208         for (i=0; i<N; i++)
00209             data[i].init(mxGetCell(a,i));
00210     }
00211     else
00212     if (mxIsDouble(a)) {
00213         N=1; init(N); data[0].init(a);
00214     }
00215
00216     for (i=0; i<N; i++) {
00217         if (data[i].rank()!=2) wblog(FL,
00218             "ERR Expecting rank-2 tensors (%d)", data[i].rank());
00219     }
00220 }
00221
00222 #endif
00223

```

Index

[__wb_msgbuf](#), [11](#)

[blockSpaceQS< TQ, TD >](#), [20](#)

[cdata< TD >](#), [25](#)
[CData< TQ, TD >](#), [21](#)
[CG::FileLock](#), [56](#)
[cgc_contract_id< TM >](#), [31](#)
[cgdStatus](#), [38](#)
[cgrType](#), [39](#)
[CPAT< TQ, TD >](#), [43](#)
[CPAT_Set< TQ, TD >](#), [43](#)
[cpu_time](#), [44](#)
[CRef< TQ >](#), [45](#)
[CStore< TQ >](#), [47](#)
[ctrlIdx](#), [49](#)

[DY::Symmetry](#), [134](#)
[DY::weight_info< TQ >](#), [210](#)
[DY::Weights< TQ >](#), [211](#)

[genRG_base< TQ, TD >](#), [56](#)
[genRG_struct< TQ, TD >](#), [57](#)
[groupIndex< T >](#), [58](#)

[icFlags](#), [59](#)
[indexSparseRef< TD >](#), [59](#)
[is_pointer_< T >](#), [60](#)
[is_pointer_< T * >](#), [60](#)
[is_pointer_< T ** >](#), [61](#)
[is_pointer_< T *** >](#), [61](#)
[itag_](#), [61](#)
[iTags](#), [62](#)
[IterOM_DATA< TD >](#), [71](#)
[iterSparseCMref< TD >](#), [71](#)

[MEM_SET< T >](#), [74](#)
[MEM_SET< T * >](#), [74](#)
[MHash< TM >](#), [74](#)
[Mx::Array< T >](#), [12](#)
[mxArray_buf](#), [78](#)
[MXPut](#), [78](#)
[mxStruct](#), [79](#)

[NRGData< TQ, TD >](#), [80](#)
[NRGIndex< TQ, TD >](#), [82](#)

[opFlags< T >](#), [83](#)
[OPTS](#), [84](#)

[pairPatternCG< T >](#), [84](#)

[QBlock< TQ, TD, TE >](#), [86](#)
[QDir](#), [87](#)
[QHash< TQ >](#), [94](#)
[QHash_](#), [94](#)
[QMap< TQ >](#), [94](#)
[QSet< TQ >](#), [95](#)
[qset< TQ >](#), [97](#)
[QSpace< TQ, TD >](#), [104](#)
[QType](#), [111](#)
[QVec](#), [112](#)

[RCStore](#), [120](#)
[RIXS_para](#), [121](#)
[RSpecData< TS >](#), [122](#)
[RStore< TQ, TD >](#), [122](#)

[sparseCollector2D< TD >](#), [124](#)
[sparseIndex2D< T >](#), [124](#)
[Spectral< TS >](#), [125](#)
[STAT_DGEMM](#), [126](#)
[SVD_Data< TQ, TD >](#), [134](#)

[TDSpectral< TS >](#), [135](#)
[tensor_ref_](#), [135](#)
[tensorRef_](#), [136](#)
[tensorRefs](#), [137](#)

[Wb::ARGV](#), [11](#)
[Wb::Bitset](#), [13](#)
[Wb::bitset< T >](#), [20](#)
[Wb::cc_quad](#), [20](#)
[Wb::citerator< T, T2 >](#), [40](#)
[Wb::citerator< T, T2 * >](#), [40](#)
[Wb::CleanUp](#), [41](#)
[Wb::Clock](#), [41](#)
[Wb::ClockSet](#), [42](#)
[Wb::counter](#), [42](#)
[Wb::gpara_](#), [58](#)
[Wb::IOstat](#), [60](#)
[Wb::iterator< T, T2 >](#), [70](#)
[Wb::iterator< T, T2 * >](#), [70](#)
[Wb::line_stat_](#), [72](#)
[Wb::LogException](#), [72](#)
[Wb::matFile](#), [73](#)
[Wb::mpfr_< P >](#), [75](#)
[Wb::MREC](#), [77](#)
[Wb::myname_](#), [80](#)
[Wb::num2Fmt< T >](#), [83](#)
[Wb::Path](#), [85](#)
[Wb::Profile](#), [85](#)

Wb::rand, [120](#)
Wb::SBUF, [123](#)
Wb::SigHandler, [123](#)
Wb::sptr< T >, [126](#)
Wb::String, [127](#)
Wb::termcolor, [144](#)
Wb::tmpFile, [144](#)
Wb::VersionInfo, [145](#)
Wb::WbListMTRACK, [170](#)
wbarry< T >, [145](#)
wbarrRef< T >, [153](#)
wbcomplex, [154](#)
wbdebug_dummy, [155](#)
wbhist, [155](#)
wbIndex, [156](#)
wbindex, [163](#)
wblog::stdio_buf, [127](#)
wblogbuf_struct, [170](#)
wblogbuffer, [171](#)
wbMatrix< T >, [171](#)
wbMsg, [179](#)
wbperm, [179](#)
wbperm_helper< T >, [187](#)
wbRecs< T >, [188](#)
wbREF< TX >, [188](#)
wbsparray< TD >, [188](#)
wbstring, [194](#)
wbtop, [202](#)
WbUtil< T >, [203](#)
wbvector< T >, [203](#)

X3Map< TQ, TD >, [211](#)
x3map< TQ, TD >, [212](#)
XMAP, [212](#)
XMAP_CMat, [219](#)
XMAP_Node, [220](#)